

Mission Autonomy Interface Control Document

For

A-GRA

6.0a

31 JULY 2026

Revision History

Version	Description	Release Date
5.0a	Initial Public ASK 5.0a Release	21 APR 2026
6.0a	ASK 6.0a Release	31 JUL 2026

Table of Contents

1	Introduction	40
1.1	Purpose	40
1.2	Scope	40
1.3	ICD Syntax	40
1.3.1	Requirements Syntax	40
1.3.2	Message Syntax	40
2	Reference Standards	41
3	Mission Autonomy A-GRA Overview	42
3.1	A-GRA Introduction	42
3.2	System Description	42
3.3	MA L1 External System Interface	44
3.3.1	C2 Interface	45
3.3.2	Peer Interface	45
3.3.3	Mission Systems Interface	45
3.3.4	Mission Debrief Interface	45
3.3.5	Mission Plan Interface	46
3.3.6	Vehicle Interface	46
3.4	Autonomy Schema	46
3.5	Network Stack Interoperability	46
4	System Interface Compliance	48
4.1	System Interface Compliance Introduction	48
4.1.1	Required Fields for A-GRA Compliance	49
4.1.2	Message Set Tables	49
5	C2 Interface	52
5.1	Interface Description	53
5.2	Interface Interactions	53
5.2.1	C2 Acknowledgment Configurations Package	55
5.2.2	C2 CAP Behaviors Package	62
5.2.3	C2 Communications Package	63
5.2.4	C2 Contingency Behaviors Package	70
5.2.5	C2 COP Behaviors Package	110
5.2.6	C2 Engagement Criteria Behaviors Package	115

5.2.7	C2 Generic Behaviors Package.....	119
5.2.8	C2 Mission Plan Behaviors Package.....	159
5.2.9	C2 RBAC Behaviors Package	297
5.2.10	C2 Sensor/Vehicle Tasking Behaviors Package	316
5.2.11	C2 Tactic Behaviors Package	320
5.2.12	C2 Task Prioritization and Mission Context Behaviors Package	341
5.2.13	C2 Teaming Behaviors Package.....	360
5.2.14	C2 Threat Assessment Package.....	369
5.2.15	C2 Weapon Employment Behaviors Package.....	370
5.3	Interface Compliance.....	390
6	Mission Debrief Interface.....	392
6.1	Mission Debrief Interface Description.....	392
6.2	Mission Debrief Interface Interactions	392
6.3	Mission Debrief Interface Compliance	392
7	Mission Plan Interface.....	393
7.1	Interface Description	393
7.1.1	Mission Data Bundle	394
7.1.2	Mission Plan Message	395
7.2	Interface Interactions.....	396
7.2.1	Capabilities Package.....	396
7.2.2	Mission Plan Package.....	426
7.2.3	Operational Data Package.....	440
7.2.4	Policy Package.....	461
7.2.5	PriorityList Package.....	467
7.2.6	Requirements Package.....	472
7.3	Interface Compliance.....	475
8	Mission Systems Interface	476
8.1	Interface Description	476
8.2	Interface Interactions.....	478
8.2.1	Communications Service Package.....	479
8.2.2	Contingencies Package	502
8.2.3	COP Package.....	508
8.2.4	Data Fusion Package.....	516

8.2.5	Decentralized Messaging Service Package	519
8.2.6	Mission Plan Behaviors Package	534
8.2.7	PNT Service Package	538
8.2.8	Sensor/Vehicle Tasking Package	540
8.2.9	Status Service Package	559
8.2.10	Weapon Employment Package	571
8.3	Interface Compliance.....	594
9	Peer Interface	595
9.1	Interface Description	595
9.2	Interface Interactions.....	595
9.2.1	Peer Acknowledgment Configurations Package	597
9.2.2	Peer CAP Package	600
9.2.3	Peer Communications Package	601
9.2.4	Peer Contingencies Package.....	609
9.2.5	Peer COP and Coordination Package.....	638
9.2.6	Peer COP Vehicle State Package	649
9.2.7	Peer Engagement Criteria Behaviors Package.....	667
9.2.8	Peer Generic Package	669
9.2.9	Peer Geozones Package	670
9.2.10	Peer Mission Plan Management Package	673
9.2.11	Peer Package Management and Leader Election Package.....	740
9.2.12	Peer Task Prioritization and Mission Context Package.....	766
9.2.13	Peer Tasking Package.....	775
9.2.14	Peer Threat Assessment Package.....	813
9.2.15	Peer Weapon Employment Package	814
9.3	Interface Compliance.....	834
10	Vehicle Interface	836
10.1	Interface Description	836
10.2	Interface Interactions.....	837
10.2.1	Contingencies Package	837
10.2.2	Control and Tasking Package	844
10.2.3	COP Package	865
10.2.4	Data Validation Package	867

10.2.5	Route Plan Behaviors Package.....	873
10.2.6	Status Package.....	885
10.2.7	Weapon Employment Package	909
10.3	Interface Compliance.....	911
11	Component Interface Compliance.....	911

Table of Figures

Figure 3-1: MA L1 Logical Interfaces	43
Figure 3-2: MA L1 Logical Connections.....	44
Figure 4-1: Message Set Table Flow Chart.....	51
Figure 4-2: Fields Decision Path: Optional vs. Required	52
Figure 5-1: C2 Interactions.....	54
Figure 5-2: C2 Message Acknowledgment Configuration Status	55
Figure 5-3: C2 Updates Message Acknowledgment Configuration	57
Figure 5-4: Message Acknowledgment Configuration Status	59
Figure 5-5: Update Message Acknowledgment Configuration	61
Figure 5-6: Share CAP Execution Status	63
Figure 5-7: C2 Overrides Data Dissemination Plan	65
Figure 5-8: C2 Updates Data Dissemination Plan.....	67
Figure 5-9: MA Updates Comms Gateway Status	69
Figure 5-10: MA Updates Data Dissemination Plan Activation Status	70
Figure 5-11: C2 Activate a Contingency	72
Figure 5-12: C2 Incorrect Messaging Sequence Error Handling	78
Figure 5-13: C2 Lost Communication	83
Figure 5-14: C2 Malformed Message/Precondition Not Met Handling	86
Figure 5-15: C2 Receive Contingency Status	87
Figure 5-16: Define Autonomous Contingency Re-Plan	89
Figure 5-17: Define Contingency Alert.....	91
Figure 5-18: Define Contingency Plan.....	93
Figure 5-19: Triggered Autonomous Contingency Re-Plan	96
Figure 5-20: Triggered Contingency Alert.....	98
Figure 5-21: Triggered Contingency Plan.....	100
Figure 5-22: C2 Lost Comms with Team Leader	102
Figure 5-23: Define Team Lost Comms Contingency	104
Figure 5-24: Team Leader Lost Comms with Multiple Team Members.....	107
Figure 5-25: Team Member Total Comms Lost with Team.....	109
Figure 5-26: C2 Requests the COP Configuration Settings.....	111
Figure 5-27: C2 Updates the COP Configuration Settings	113
Figure 5-28: Synchronize Global COP to C2.....	115

Figure 5-29: Activate Engagement Criteria.....	118
Figure 5-30: C2 Commanded Geozone.....	120
Figure 5-31: C2 Plan Approval Status	122
Figure 5-32: C2 Publish Control Status	123
Figure 5-33: C2 Receives Geozone Status	124
Figure 5-34: C2 Receives Message Acknowledgment	125
Figure 5-35: C2 Sends Message Acknowledgment	126
Figure 5-36: Control Mode Authorization.....	127
Figure 5-37: Direct HSA/CSA Command Sequence	129
Figure 5-38: Direct Waypoint Command	131
Figure 5-39: Planned HSA/CSA Command Sequence	132
Figure 5-40: Planned Waypoint Command.....	134
Figure 5-41: Receive Execution Status	136
Figure 5-42: Report Departure Checkpoints to C2	140
Figure 5-43: Request Generation of Route Metrics.....	143
Figure 5-44: Respond to Approval Request	145
Figure 5-45: Send Flight Phase Status	146
Figure 5-46: Send Heartbeat.....	147
Figure 5-47: Send Subsystem Status to C2	148
Figure 5-48: Share Vehicle State Data	149
Figure 5-49: Station-Keeping	153
Figure 5-50: Systems Needed Request.....	156
Figure 5-51: Update Barometric Pressure	158
Figure 5-52: Update MA-FA Control Designations	159
Figure 5-53: Activate or Deactivate Mission Plan.....	161
Figure 5-54: C2 Commanded Abort.....	163
Figure 5-55: C2 Detect Mission Plan Route Discontinuity Response.....	164
Figure 5-56: C2 Receive Abort Status	165
Figure 5-57: C2 Receive Plan Modification Notification	166
Figure 5-58: C2 Request Plan Patch	168
Figure 5-59: Command Mission Replan	169
Figure 5-60: MA Notifies C2 of Replanning	171
Figure 5-61: MA Provides Replan Status to C2.....	172

Figure 5-62: QB Commanded RTB.....	174
Figure 5-63: Query C2 for Missing Data.....	176
Figure 5-64: Query Deconfliction Data	178
Figure 5-65: Receive Autonomy Settings	181
Figure 5-66: Receive MA Plan Status.....	183
Figure 5-67: Respond to Airfield Report Query	185
Figure 5-68: Send Response Plan.....	187
Figure 5-69: Send Team Level Command.....	190
Figure 5-70: Synchronize Contingency Plan	195
Figure 5-71: Update C2 with Package Mission Plan Synchronization Status	196
Figure 5-72: Zone Alerts.....	197
Figure 5-73: Query Activity Plan	199
Figure 5-74: Query Activity Plan Execution Status.....	201
Figure 5-75: Query Airfield Report.....	203
Figure 5-76: Query Control Status	205
Figure 5-77: Query Mission Plan.....	207
Figure 5-78: Query Mission Plan Activation Status	209
Figure 5-79: Query Mission Plan Execution Status.....	211
Figure 5-80: Query OpPoint	213
Figure 5-81: Query OpZone	215
Figure 5-82: Query Package	217
Figure 5-83: Query Planning Function.....	219
Figure 5-84: Query Planning Function Status	221
Figure 5-85: Query Response Plan	223
Figure 5-86: Query Response Plan Execution Status.....	225
Figure 5-87: Query Route Activity Plan	227
Figure 5-88: Query Route Activity Plan Execution Status.....	229
Figure 5-89: Query Route Metrics.....	231
Figure 5-90: Query Route Plan	233
Figure 5-91: Query Route Plan Execution Status	235
Figure 5-92: Query System Status.....	237
Figure 5-93: Query Task Plan	239
Figure 5-94: Query Task Plan Execution Status	241

Figure 5-95: Query Weather	243
Figure 5-96: Send Action - C2 to MA	244
Figure 5-97: Send Action Plan - C2 to MA	245
Figure 5-98: Send Activity Plan - C2 to MA	246
Figure 5-99: Send Airfield Report - C2 to MA	247
Figure 5-100: Send Approval Authority - C2 to MA	248
Figure 5-101: Send Approval Policy - C2 to MA	249
Figure 5-102: Send Capability Set - C2 to MA	250
Figure 5-103: Send Constraint Priority List - C2 to MA	251
Figure 5-104: Send Constraint Priority List Activation Data - C2 to MA	252
Figure 5-105: Send Engagement Criteria - C2 to MA	253
Figure 5-106: Send Entity - C2 to MA	254
Figure 5-107: Send Execution Priority List - C2 to MA	255
Figure 5-108: Send Execution Priority List Activation Data - C2 to MA	256
Figure 5-109: Send Interoperability Policy - C2 to MA	257
Figure 5-110: Send Leadership Metrics - C2 to MA	258
Figure 5-111: Send Mission Plan - C2 to MA	259
Figure 5-112: Send Operator - C2 to MA	260
Figure 5-113: Send Operator Role - C2 to MA	261
Figure 5-114: Send OpLine - C2 to MA	262
Figure 5-115: Send OpPlan - C2 to MA	263
Figure 5-116: Send OpPoint - C2 to MA	264
Figure 5-117: Send OpRouting - C2 to MA	265
Figure 5-118: Send OpVolume - C2 to MA	266
Figure 5-119: Send OpZone - C2 to MA	267
Figure 5-120: Send Package - C2 to MA	268
Figure 5-121: Send Response - C2 to MA	269
Figure 5-122: Send Response Plan - C2 to MA	270
Figure 5-123: Send Route Activity Plan - C2 to MA	271
Figure 5-124: Send Route Metrics - C2 to MA	272
Figure 5-125: Send Route Plan - C2 to MA	273
Figure 5-126: Send Task - C2 to MA	274
Figure 5-127: Send Task Plan - C2 to MA	275

Figure 5-128: Send Weather Dataset - C2 to MA	276
Figure 5-129: Send Action - MA to C2	277
Figure 5-130: Send Action Plan - MA to C2	278
Figure 5-131: Send Activity Plan - MA to C2	279
Figure 5-132: Send Approval Authority - MA to C2	280
Figure 5-133: Send Approval Policy - MA to C2	281
Figure 5-134: Send Capability Set - MA to C2	282
Figure 5-135: Send Entity - MA to C2	283
Figure 5-136: Send Mission Plan - MA to C2	284
Figure 5-137: Send Op Line - MA to C2	285
Figure 5-138: Send Op Routing - MA to C2	286
Figure 5-139: Send Op Volume - MA to C2	287
Figure 5-140: Send Op Zone - MA to C2	288
Figure 5-141: Send Operator - MA to C2	289
Figure 5-142: Send Operator Notification - MA to C2	290
Figure 5-143: Send Operator Role - MA to C2	291
Figure 5-144: Send Response - MA to C2	292
Figure 5-145: Send Response Plan - MA to C2	293
Figure 5-146: Send Route Activity Plan - MA to C2	294
Figure 5-147: Send Route Plan - MA to C2	295
Figure 5-148: Send Task - MA to C2	296
Figure 5-149: Send Task Plan - MA to C2	297
Figure 5-150: Admin Create Role	299
Figure 5-151: Admin Update MA Role Request Permissions	301
Figure 5-152: Admin Update Role Permissions	303
Figure 5-153: Authorize	305
Figure 5-154: C2 Unauthorized Response	306
Figure 5-155: Controller Update Broadcast	307
Figure 5-156: Query Operator Roles	308
Figure 5-157: Query Role Permissions	310
Figure 5-158: Request Approval for Task Allocation Plan	317
Figure 5-159: Synchronize Assessment	319
Figure 5-160: Advertise Tactic Capabilities	321

Figure 5-161: Cancel Tactic	322
Figure 5-162: Cold	324
Figure 5-163: Command Tactic.....	326
Figure 5-164: Drop	329
Figure 5-165: Holster.....	331
Figure 5-166: Receive Package Tactic Status	333
Figure 5-167: Set Tactic Shot Doctrine and Intercept Maneuver Parameters.....	335
Figure 5-168: Shoot.....	337
Figure 5-169: Shoot Now	339
Figure 5-170: Target.....	341
Figure 5-171: C2 Define Global Override	343
Figure 5-172: Provide Risk Assessment Results	344
Figure 5-173: Request Route Threat Assessment	345
Figure 5-174: Update Acceptable Level of Risk	348
Figure 5-175: Update Package Survivability Risk Definition	349
Figure 5-176: Update Survivability Risk Definition	350
Figure 5-177: Activate Constraint Priority List	353
Figure 5-178: Update Constraint Priority List Activation Status	354
Figure 5-179: Activate Execution Priority List.....	357
Figure 5-180: Update Execution Priority List Activation Status	358
Figure 5-181: Activate Ignore Zone Constraints.....	360
Figure 5-182: Changing a Package Leader.....	362
Figure 5-183: Command Team/Platform via Task	364
Figure 5-184: Creating or Changing a Package.....	366
Figure 5-185: Distribute C2 Track Data to Facilitate Global Fusion and Update COP	367
Figure 5-186: Receive Package Status	368
Figure 5-187: Uploading a Task.....	369
Figure 5-188: C2 Threat Assessment.....	370
Figure 5-189: DLZ Compound Convergence	371
Figure 5-190: Exchange Shot Queue Prioritization	372
Figure 5-191: Mission Store Status Exchange	373
Figure 5-192: Query Entity Track History	375
Figure 5-193: Query Shot Queue Prioritization	377

Figure 5-194: Release Consent	379
Figure 5-195: Report Intercept Pairings.....	380
Figure 5-196: Report Shot Allocations	381
Figure 5-197: Examples of Station ID	382
Figure 5-198: Store Inventory and Descriptions Exchange	383
Figure 5-199: Target Designation	384
Figure 5-200: Target Designation Request	386
Figure 5-201: Threat Prioritization	388
Figure 5-202: Update Shot Queue Prioritization	389
Figure 5-203: WEZ Compound Convergence	390
Figure 7-1: Mission Data Bundle.....	395
Figure 7-2: Mission Plan Interface Interactions	396
Figure 7-3: MA Receives Action Capability from MP	397
Figure 7-4: MA Receives Air Sample Capability from MP.....	398
Figure 7-5: MA Receives AMTI Capability from MP	399
Figure 7-6: MA Receives AO Capability from MP	400
Figure 7-7: MA Receives Cargo Delivery Capability from MP	401
Figure 7-8: MA Receives COMINT Capability from MP	402
Figure 7-9: MA Receives Communications Relay Capability from MP.....	403
Figure 7-10: MA Receives Communications Terminal Capability from MP	404
Figure 7-11: MA Receives Counterspace Capability from MP.....	405
Figure 7-12: MA Receives EA Capability from MP	406
Figure 7-13: MA Receives Effect Capability from MP	407
Figure 7-14: MA Receives ESM Capability from MP.....	408
Figure 7-15: MA Receives Flight Capability from MP	409
Figure 7-16: MA Receives Gateway Capability from MP	410
Figure 7-17: MA Receives IFF Capability from MP	411
Figure 7-18: MA Receives Opaque Capability from MP	412
Figure 7-19: MA Receives Orbit Change Capability from MP.....	413
Figure 7-20: MA Receives Orbital Surveillance Capability from MP.....	414
Figure 7-21: MA Receives Orbital Surveillance Sensor Capability from MP	415
Figure 7-22: MA Receives PO Capability from MP	416
Figure 7-23: MA Receives Radar Altimeter Capability from MP	417

Figure 7-24: MA Receives Reference Capability from MP	418
Figure 7-25: MA Receives Refuel Capability from MP	419
Figure 7-26: MA Receives Response Capability from MP	420
Figure 7-27: MA Receives SAR Capability from MP	421
Figure 7-28: MA Receives SMTI Capability from MP	422
Figure 7-29: MA Receives Strike Capability from MP	423
Figure 7-30: MA Receives System Deployment Capability from MP	424
Figure 7-31: MA Receives Tactical Order Capability from MP	425
Figure 7-32: MA Receives Weather Radar Capability from MP	426
Figure 7-33: MA Receives Action from MP	427
Figure 7-34: MA Receives Action Plan from MP	428
Figure 7-35: MA Receives Activity Metrics from MP	429
Figure 7-36: MA Receives Activity Plan from MP	430
Figure 7-37: MA Receives Analysis Route from MP	431
Figure 7-38: MA Receives Mission Plan from MP	432
Figure 7-39: MA Receives OpPlan from MP	433
Figure 7-40: MA Receives Response from MP	434
Figure 7-41: MA Receives Response Plan from MP	435
Figure 7-42: MA Receives Route Activity Plan from MP	436
Figure 7-43: MA Receives Route Metrics from MP	437
Figure 7-44: MA Receives Route Plan from MP	438
Figure 7-45: MA Receives Task from MP	439
Figure 7-46: MA Receives Task Plan from MP	440
Figure 7-47: MA Receives Access Assessment from MP	441
Figure 7-48: MA Receives Airfield Report from MP	442
Figure 7-49: MA Receives Assessment from MP	443
Figure 7-50: MA Receives Designation from MP	444
Figure 7-51: MA Receives DLZ from MP	445
Figure 7-52: MA Receives Entity from MP	446
Figure 7-53: MA Receives Foreign Key Pair from MP	447
Figure 7-54: MA Receives Leadership Metrics from MP	448
Figure 7-55: MA Receives OpLine from MP	449
Figure 7-56: MA Receives OpPoint from MP	450

Figure 7-57: MA Receives OpRouting from MP	451
Figure 7-58: MA Receives OpVolume from MP	452
Figure 7-59: MA Receives OpZone from MP	453
Figure 7-60: MA Receives Package from MP	454
Figure 7-61: MA Receives Planning Function from MP	455
Figure 7-62: MA Receives Store Loadout Configuration from MP.....	456
Figure 7-63: MA Receives Survivability Risk Level from MP	457
Figure 7-64: MA Receives System Readiness from MP	458
Figure 7-65: MA Receives Weather Dataset from MP	459
Figure 7-66: MA Receives Weather from MP	460
Figure 7-67: MA Receives WEZ from MP	461
Figure 7-68: MA Receives Approval Authority from MP	462
Figure 7-69: MA Receives Approval Policy from MP	463
Figure 7-70: MA Receives Authorization from MP	464
Figure 7-71: MA Receives Engagement Criteria from MP	465
Figure 7-72: MA Receives Interoperability Policy from MP	466
Figure 7-73: MA Receives Operator Role from MP	467
Figure 7-74: MA Receives Constraint Priority List Activation Data from MP	468
Figure 7-75: MA Receives Constraint Priority List from MP	469
Figure 7-76: MA Receives Execution Priority List Activation Data from MP	470
Figure 7-77: MA Receives Execution Priority List from MP	471
Figure 7-78: MA Receives Prioritization List from MP	472
Figure 7-79: MA Receives Capability Set from MP	473
Figure 7-80: MA Receives Requirement Metrics from MP	474
Figure 7-81: MA Receives Requirement Set from MP	475
Figure 8-1: Mission Systems Interactions.....	479
Figure 8-2: Communications Lost	481
Figure 8-3: Configure Comm Terminal	482
Figure 8-4: Configure Gateway	484
Figure 8-5: Establish Comm Terminal	486
Figure 8-6: Maintain Comm Terminal	487
Figure 8-7: Override Data Dissemination Plan	489
Figure 8-8: Publish Comm Team Report.....	491

Figure 8-9: Publish Network Endpoint Availability	492
Figure 8-10: Receive Communication Error Report	493
Figure 8-11: Request Communication Asset Status	494
Figure 8-12: Update Comm Terminal Plan	495
Figure 8-13: Update Comms Gateway Status	497
Figure 8-14: Update Data Dissemination Plan	500
Figure 8-15: Update Data Dissemination Plan Activation Status	502
Figure 8-16: Intra-Vehicle Comms Failure_MS	503
Figure 8-17: Logging Failure	504
Figure 8-18: MS Malformed Message Response	505
Figure 8-19: Payload Management Failure	506
Figure 8-20: Sensor Failure	507
Figure 8-21: MS Comms Status Updates to COP	508
Figure 8-22: MS Sensor AMTI Status Updates to COP	510
Figure 8-23: MS Sensor PO Status Updates to COP	512
Figure 8-24: MS Weapon Status Updates to COP	515
Figure 8-25: Distribute Single Sensor Track Data to Perform Local Fusion	517
Figure 8-26: Receive Processed Track	519
Figure 8-27: Receive Message Subscription Status	522
Figure 8-28: Receive UCI Wrapped Message	523
Figure 8-29: State Machine for Send UCI Wrapped Message UCI Diagram	524
Figure 8-30: Example 1	527
Figure 8-31: Example 2	527
Figure 8-32: Send UCI Wrapped Message	528
Figure 8-33: Subscribe to Message Topic	530
Figure 8-34: Receive Plans	535
Figure 8-35: Verify Plan Patch Is Possible - MS	537
Figure 8-36: Request PNT Navigation Data	539
Figure 8-37: Advertise Sensor AMTI Capability	541
Figure 8-38: Advertise Sensor ESM Capability	542
Figure 8-39: Advertise Sensor PO Capability	543
Figure 8-40: Command Sensor AMTI Capability	545
Figure 8-41: Command Sensor ESM Capability	548

Figure 8-42: Command Sensor PO Capability	551
Figure 8-43: Publish External Weather Reports	553
Figure 8-44: Report Sensor AMTI Capability State	555
Figure 8-45: Report Sensor ESM Capability State	556
Figure 8-46: Report Sensor PO Capability State	558
Figure 8-47: Report Weather Radar Capability Observations	559
Figure 8-48: Mission Systems Capability Heartbeat	561
Figure 8-49: MS Responds to Query for Capabilities	564
Figure 8-50: Publish Periodic Passive Optical Sensor Configuration	566
Figure 8-51: Receive Periodic Health, Status, & BIT Information	568
Figure 8-52: Request Mission System Health, Status & BIT Information	570
Figure 8-53: Abort Release Sequence	572
Figure 8-54: Confirm Threat ID	574
Figure 8-55: Exchange DLZ	575
Figure 8-56: Exchange WEZ	577
Figure 8-57: Fire Command Exchange	578
Figure 8-58: Jettison Release Sequence	580
Figure 8-59: Mission Store Release Initialization Sequence	582
Figure 8-60: Mission Store Status Exchange	584
Figure 8-61: Receive Changes to Stores Primary Mission Plan Data	586
Figure 8-62: Receive Weapon Flyout State	588
Figure 8-63: Release Consent Sequence	590
Figure 8-64: Send Weapon Entity Data	592
Figure 8-65: Store Inventory and Descriptions Exchange	593
Figure 9-1: Peer to Peer Interactions	596
Figure 9-2: P2P Message Acknowledgment Configuration Status	597
Figure 9-3: P2P Update Message Acknowledgment Configurations	599
Figure 9-4: Receive CAP Execution Status	601
Figure 9-5: Peer Overrides Data Dissemination Plan	603
Figure 9-6: Peer Updates Comms Gateway Status	605
Figure 9-7: Peer Updates Data Dissemination Plan	607
Figure 9-8: Peer Updates Data Dissemination Plan Activation Status	609
Figure 9-9: Define Autonomous Contingency Re-Plan	611

Figure 9-10: Define Team Lost Comms Contingency	613
Figure 9-11: Multiple Peers Lost Comms with Team Leader	616
Figure 9-12: P2P Incorrect Messaging Sequence Error Handling	624
Figure 9-13: P2P Malformed Message/Precondition Not Met Handling	627
Figure 9-14: P2P Path Deconfliction Proposal	629
Figure 9-15: Peer Activate a Contingency	630
Figure 9-16: Peer Receive Contingency Status	631
Figure 9-17: Peer Total Comms Lost with Team	632
Figure 9-18: Team Leader Lost Comms with C2	635
Figure 9-19: Triggered Contingency Alert to Peer	638
Figure 9-20: Configure COP and Request COP Configuration Settings Status (P2P)	640
Figure 9-21: Coordinate Simultaneous Arrival Time	642
Figure 9-22: COP Follower Configuration Update	644
Figure 9-23: Distribute Sensor Track Data to Facilitate Package Fusion and Update COP	647
Figure 9-24: Synchronize Global COP to Peer	649
Figure 9-25: P2P Send Platform Override Status	650
Figure 9-26: P2P State Data	651
Figure 9-27: Peer Heartbeat	653
Figure 9-28: Peer Publish Control Status	654
Figure 9-29: Send Abort to Peer	655
Figure 9-30: Send AMTI Capability State to Peer	657
Figure 9-31: Send PO Capability State to Peer	659
Figure 9-32: Send Strike Capability State to Peer	661
Figure 9-33: Station-Keeping Capabilities	665
Figure 9-34: Activate Peer Engagement Criteria	669
Figure 9-35: Peer Sends Message Acknowledgment	670
Figure 9-36: Peer Command Geozone	672
Figure 9-37: P2P Contingency Alert	673
Figure 9-38: P2P Detect Mission Plan Route Discontinuity Response	674
Figure 9-39: P2P Request Plan Patch	675
Figure 9-40: Peer Receive Plan Modification Notification	677
Figure 9-41: Query Deconfliction Data	679
Figure 9-42: Synchronize Contingency Plan	681

Figure 9-43: Update Peer Airfield Data.....	682
Figure 9-44: P2P Lead Updates Member with Constraint Priority List	683
Figure 9-45: P2P Lead Updates Member with Constraint Priority List Activation Data	684
Figure 9-46: P2P Lead Updates Member with Engagement Criteria	684
Figure 9-47: P2P Lead Updates Member with Execution Priority List.....	685
Figure 9-48: P2P Lead Updates Member with Execution Priority List Activation Data	686
Figure 9-49: P2P Member to Lead Update with Constraint Priority List Activation Status	687
Figure 9-50: P2P Member to Lead Update with Execution Priority List Activation Status	687
Figure 9-51: P2P Member to Lead Update with System Notification	688
Figure 9-52: P2P Query Activity Plan	690
Figure 9-53: P2P Query Airfield Report.....	692
Figure 9-54: P2P Query Mission Plan.....	694
Figure 9-55: P2P Query OpZone	696
Figure 9-56: P2P Query Response Plan	698
Figure 9-57: P2P Query Route Activity Plan	700
Figure 9-58: P2P Query Route Plan	702
Figure 9-59: P2P Query Task Plan	704
Figure 9-60: P2P Query Weather	706
Figure 9-61: Send Action Plan to Peer	708
Figure 9-62: Send Action to Peer.....	709
Figure 9-63: Send Activity Plan to Peer.....	710
Figure 9-64: Send Approval Authority to Peer.....	711
Figure 9-65: Send Approval Policy to Peer.....	712
Figure 9-66: Send Capability Set to Peer	713
Figure 9-67: Send Component Status to Peer	714
Figure 9-68: Send Entity to Peer.....	715
Figure 9-69: Send Flight Activity to Peer	716
Figure 9-70: Send Flight Capability Status to Peer	717
Figure 9-71: Send Flight Capability to Peer.....	718
Figure 9-72: Send Mission Plan Execution Status to Peer	719
Figure 9-73: Send Mission Plan to Peer.....	720
Figure 9-74: Send Navigation Report to Peer	721
Figure 9-75: Send Operator Role to Peer.....	722

Figure 9-76: Send Operator to Peer	723
Figure 9-77: Send OpLine to Peer	724
Figure 9-78: Send OpPlan to Peer.....	725
Figure 9-79: Send OpRouting to Peer	726
Figure 9-80: Send OpVolume to Peer.....	727
Figure 9-81: Send OpZone to Peer.....	728
Figure 9-82: Send Position Report to Peer.....	729
Figure 9-83: Send Response Plan to Peer.....	730
Figure 9-84: Send Response to Peer	731
Figure 9-85: Send Route Activity Plan to Peer.....	732
Figure 9-86: Send Route Deconfliction Alert to Peer	733
Figure 9-87: Send Route Plan Execution Status to Peer	734
Figure 9-88: Send Route Plan to Peer.....	735
Figure 9-89: Send Subsystem Status to Peer	736
Figure 9-90: Send System Status to Peer	737
Figure 9-91: Send Task Plan to Peer.....	738
Figure 9-92: Send Task Status to Peer	739
Figure 9-93: Send Task to Peer	740
Figure 9-94: Bully Leader Election.....	743
Figure 9-95: Create a Full Package Amongst Peers.....	745
Figure 9-96: Current Package Lead Assigns New Package Lead.....	746
Figure 9-97: Initializing a Package Amongst Peers.....	748
Figure 9-98: Leader Commanded Package Addition	750
Figure 9-99: Maximum Consensus Leader Election	752
Figure 9-100: Notify Peers of Package Formation Command from Unauthorized C2	754
Figure 9-101: Off Nominal Leader Election	756
Figure 9-102: Peer Declares Package Leadership to Peers.....	758
Figure 9-103: Peer Joining a Package	759
Figure 9-104: Raft Leader Election	761
Figure 9-105: Share DLFS with Peers.....	762
Figure 9-106: Static Fitness Score Leader Election	765
Figure 9-107: Peer Provides Route Risk Assessment Results.....	767
Figure 9-108: Update Peer Acceptable Level of Risk	769

Figure 9-109: Update Peer Survivability Risk Definition	770
Figure 9-110: Activate Peer Constraint Priority List	771
Figure 9-111: Activate Peer Execution Priority List	773
Figure 9-112: Activate Peer Ignore Zone Constraints	774
Figure 9-113: Allocate an Altitude Stacked Marshall	776
Figure 9-114: Allocating an Escort Task.....	778
Figure 9-115: Command Peer Capability	781
Figure 9-116: Command Peer via Action.....	785
Figure 9-117: Command Peer(s) via Task.....	787
Figure 9-118: Control Mode Authorization.....	789
Figure 9-119: Package Plan Approval Status.....	791
Figure 9-120: Peer Activate or Deactivate Mission Plan.....	793
Figure 9-121: Peer Responds to Approval Request.....	795
Figure 9-122: Peer to Peer HSA/CSA Command	797
Figure 9-123: Peer to Peer Waypoint Follow.....	799
Figure 9-124: Peer Validate Action Plan.....	801
Figure 9-125: Provide and Receive Formation Status	803
Figure 9-126: Report Task Status to Leader	804
Figure 9-127: Respond to Command from Team Lead	809
Figure 9-128: P2P Threat Assessment.....	813
Figure 9-129: Centralized Computation of Target Assignment.....	815
Figure 9-130: Distributed Computation of Target Assignment.....	816
Figure 9-131: DLZ Compound Convergence	817
Figure 9-132: Peer Assign Target Pairings.....	818
Figure 9-133: Peer Report Tactic Status	820
Figure 9-134: Peer Send Tactic Parameters	821
Figure 9-135: Peer Target Designation Request.....	822
Figure 9-136: Query Entity History.....	824
Figure 9-137: Query Entity Track History	827
Figure 9-138: Query Shot Queue Prioritization	829
Figure 9-139: Send Shot Queue Prioritization.....	831
Figure 9-140: Threat Prioritization	832
Figure 9-141: Update Shot Queue Prioritization	833

Figure 9-142: WEZ Compound Convergence	834
Figure 10-1: Vehicle Interface Interactions	837
Figure 10-2: Collision Avoidance	838
Figure 10-3: Intra-Vehicle Comms Failure	839
Figure 10-4: MA Failsafe	840
Figure 10-5: Mechanical Damage Reporting	841
Figure 10-6: Sensor Failure	842
Figure 10-7: VI Malformed Message Response	844
Figure 10-8: Control by Curve Following	847
Figure 10-9: Control by HSA/CSA Command	850
Figure 10-10: Control by Waypoint Following	853
Figure 10-11: Control Mode Authorization	855
Figure 10-12: MA-VI Command Task	857
Figure 10-13: Modify Capabilities	859
Figure 10-14: Receive Control Request	861
Figure 10-15: Unpair Control Assignment	863
Figure 10-16: Update C2 Control Designations	864
Figure 10-17: VI Updates to COP	866
Figure 10-18: Checksum Validation	868
Figure 10-19: Query for Missing Data	870
Figure 10-20: Route Plan Data Validation	872
Figure 10-21: Activate Route	874
Figure 10-22: Convert and Upload Route	878
Figure 10-23: Prepare for Route Activation	879
Figure 10-24: Receive Deactivate Route	881
Figure 10-25: Validate Route Plan	882
Figure 10-26: Verify Plan Patch Is Possible - VI	883
Figure 10-27: VI Deactivate Route	884
Figure 10-28: Exchange Heartbeat - Subsystem Status Reports	886
Figure 10-29: Publish Control Status	888
Figure 10-30: Query Airfield Update	890
Figure 10-31: Query Route Plan	893
Figure 10-32: Receive Barometric Pressure	895

Figure 10-33: Receive Execution Status	897
Figure 10-34: Receive Vehicle Performance Values.....	899
Figure 10-35: Receive Vehicle State Data.....	901
Figure 10-36: Request Terrain Data	904
Figure 10-37: Vehicle Status Reporting.....	906
Figure 10-38: VI Responds to Query for Flight Capabilities.....	908
Figure 10-39: Validate Release Envelope	910

Table of Tables

Table 1-1: Requirements Syntax.....	40
Table 1-2: Message Syntax.....	41
Table 2-1: Referenced Standards.....	41
Table 3-1: Network Stack Layers.....	47
Table 4-1: MA L1 Interface Compliance References	49
Table 5-1: C2 Message Acknowledgment Configuration Status	55
Table 5-2: C2 Updates Message Acknowledgment Configuration	57
Table 5-3: Message Acknowledgment Configuration Status	59
Table 5-4: Update Message Acknowledgment Configuration.....	61
Table 5-5: Share CAP Execution Status.....	63
Table 5-6: C2 Overrides Data Dissemination Plan.....	65
Table 5-7: C2 Updates Data Dissemination Plan.....	68
Table 5-8: MA Updates Comms Gateway Status.....	69
Table 5-9: MA Updates Data Dissemination Plan Activation Status	70
Table 5-10: C2 Activate a Contingency	72
Table 5-11: C2 Incorrect Messaging Sequence Error Handling	79
Table 5-12: C2 Lost Communication	84
Table 5-13: C2 Malformed Message/Precondition Not Met Handling	86
Table 5-14: C2 Receive Contingency Status.....	87
Table 5-15: Define Autonomous Contingency Re-Plan	89
Table 5-16: Define Contingency Alert.....	91
Table 5-17: Define Contingency Plan	94
Table 5-18: Triggered Autonomous Contingency Re-Plan	96
Table 5-19: Triggered Contingency Alert.....	98
Table 5-20: Triggered Contingency Plan.....	100
Table 5-21: C2 Lost Comms with Team Leader.....	102
Table 5-22: Define Team Lost Comms Contingency	104
Table 5-23: Team Leader Lost Comms with Multiple Team Members.....	107
Table 5-24: Team Member Total Comms Lost with Team.....	109
Table 5-25: C2 Requests the COP Configuration Settings.....	111
Table 5-26: C2 Updates the COP Configuration Settings.....	113
Table 5-27: Synchronize Global COP to C2.....	115

Table 5-28: Activate Engagement Criteria	118
Table 5-29: C2 Commanded Geozone	120
Table 5-30: C2 Plan Approval Status.....	122
Table 5-31: C2 Publish Control Status	123
Table 5-32: C2 Receives Geozone Status	124
Table 5-33: C2 Receives Message Acknowledgment.....	125
Table 5-34: C2 Sends Message Acknowledgment	126
Table 5-35: Control Mode Authorization	128
Table 5-36: Direct HSA/CSA Command Sequence	130
Table 5-37: Direct Waypoint Command.....	131
Table 5-38: Planned HSA/CSA Command Sequence	132
Table 5-39: Planned Waypoint Command.....	134
Table 5-40: Receive Execution Status.....	137
Table 5-41: Report Departure Checkpoints to C2	141
Table 5-42: Request Generation of Route Metrics.....	144
Table 5-43: Respond to Approval Request	145
Table 5-44: Send Flight Phase Status	146
Table 5-45: Send Heartbeat.....	147
Table 5-46: Send Subsystem Status to C2.....	148
Table 5-47: Share Vehicle State Data	150
Table 5-48: Station-Keeping	154
Table 5-49: Systems Needed Request.....	156
Table 5-50: Update Barometric Pressure	158
Table 5-51: Update MA-FA Control Designations	159
Table 5-52: Activate or Deactivate Mission Plan	161
Table 5-53: C2 Commanded Abort	163
Table 5-54: C2 Detect Mission Plan Route Discontinuity Response	164
Table 5-55: C2 Receive Abort Status	165
Table 5-56: C2 Receive Plan Modification Notification	166
Table 5-57: C2 Request Plan Patch	168
Table 5-58: Command Mission Replan	169
Table 5-59: MA Notifies C2 of Replanning	172
Table 5-60: MA Provides Replan Status to C2.....	172

Table 5-61: QB Commanded RTB	174
Table 5-62: Query C2 for Missing Data	176
Table 5-63: Query Deconfliction Data.....	179
Table 5-64: Receive Autonomy Settings	181
Table 5-65: Receive MA Plan Status	183
Table 5-66: Respond to Airfield Report Query	185
Table 5-67: Send Response Plan	187
Table 5-68: Send Team Level Command.....	191
Table 5-69: Synchronize Contingency Plan	195
Table 5-70: Update C2 with Package Mission Plan Synchronization Status	196
Table 5-71: Zone Alerts	197
Table 5-72: Query Activity Plan.....	200
Table 5-73: Query Activity Plan Execution Status	202
Table 5-74: Query Airfield Report	204
Table 5-75: Query Control Status	206
Table 5-76: Query Mission Plan.....	208
Table 5-77: Query Mission Plan Activation Status	210
Table 5-78: Query Mission Plan Execution Status	212
Table 5-79: Query OpPoint	214
Table 5-80: Query OpZone	216
Table 5-81: Query Package	218
Table 5-82: Query Planning Function	220
Table 5-83: Query Planning Function Status.....	222
Table 5-84: Query Response Plan.....	224
Table 5-85: Query Response Plan Execution Status.....	226
Table 5-86: Query Route Activity Plan.....	228
Table 5-87: Query Route Activity Plan Execution Status	230
Table 5-88: Query Route Metrics	232
Table 5-89: Query Route Plan	234
Table 5-90: Query Route Plan Execution Status.....	236
Table 5-91: Query System Status	238
Table 5-92: Query Task Plan	240
Table 5-93: Query Task Plan Execution Status.....	242

Table 5-94: Query Weather	244
Table 5-95: Send Action - C2 to MA	244
Table 5-96: Send Action Plan - C2 to MA	245
Table 5-97: Send Activity Plan - C2 to MA	246
Table 5-98: Send Airfield Report - C2 to MA	247
Table 5-99: Send Approval Authority - C2 to MA	248
Table 5-100: Send Approval Policy - C2 to MA	249
Table 5-101: Send Capability Set - C2 to MA	250
Table 5-102: Send Constraint Priority List - C2 to MA	251
Table 5-103: Send Constraint Priority List Activation Data - C2 to MA	252
Table 5-104: Send Engagement Criteria - C2 to MA	253
Table 5-105: Send Entity - C2 to MA	254
Table 5-106: Send Execution Priority List - C2 to MA	255
Table 5-107: Send Execution Priority List Activation Data - C2 to MA	256
Table 5-108: Send Interoperability Policy - C2 to MA	257
Table 5-109: Send Leadership Metrics - C2 to MA	258
Table 5-110: Send Mission Plan - C2 to MA	259
Table 5-111: Send Operator - C2 to MA	260
Table 5-112: Send Operator Role - C2 to MA	261
Table 5-113: Send OpLine - C2 to MA	262
Table 5-114: Send OpPlan - C2 to MA	263
Table 5-115: Send OpPoint - C2 to MA	264
Table 5-116: Send OpRouting - C2 to MA	265
Table 5-117: Send OpVolume - C2 to MA	266
Table 5-118: Send OpZone - C2 to MA	267
Table 5-119: Send Package - C2 to MA	268
Table 5-120: Send Response - C2 to MA	269
Table 5-121: Send Response Plan - C2 to MA	270
Table 5-122: Send Route Activity Plan - C2 to MA	271
Table 5-123: Send Route Metrics - C2 to MA	272
Table 5-124: Send Route Plan - C2 to MA	273
Table 5-125: Send Task - C2 to MA	274
Table 5-126: Send Task Plan - C2 to MA	275

Table 5-127: Send Weather Dataset - C2 to MA.....	276
Table 5-128: Send Action - MA to C2	277
Table 5-129: Send Action Plan - MA to C2.....	278
Table 5-130: Send Activity Plan - MA to C2	279
Table 5-131: Send Approval Authority - MA to C2	280
Table 5-132: Send Approval Policy - MA to C2.....	281
Table 5-133: Send Capability Set - MA to C2.....	282
Table 5-134: Send Entity - MA to C2	283
Table 5-135: Send Mission Plan - MA to C2	284
Table 5-136: Send Op Line - MA to C2	285
Table 5-137: Send Op Routing - MA to C2.....	286
Table 5-138: Send Op Volume - MA to C2.....	287
Table 5-139: Send Op Zone - MA to C2	288
Table 5-140: Send Operator - MA to C2.....	289
Table 5-141: Send Operator Notification - MA to C2	290
Table 5-142: Send Operator Role - MA to C2	291
Table 5-143: Send Response - MA to C2.....	292
Table 5-144: Send Response Plan - MA to C2	293
Table 5-145: Send Route Activity Plan - MA to C2	294
Table 5-146: Send Route Plan - MA to C2	295
Table 5-147: Send Task - MA to C2	296
Table 5-148: Send Task Plan - MA to C2.....	297
Table 5-149: Admin Create Role.....	299
Table 5-150: Admin Update MA Role Request Permissions	302
Table 5-151: Admin Update Role Permissions.....	303
Table 5-152: Authorize	305
Table 5-153: C2 Unauthorized Response	306
Table 5-154: Controller Update Broadcast.....	307
Table 5-155: Query Operator Roles.....	308
Table 5-156: Query Role Permissions.....	311
Table 5-157: RBAC Roles	312
Table 5-158: Synchronize Assessment	320
Table 5-159: Advertise Tactic Capabilities	321

Table 5-160: Cancel Tactic	322
Table 5-161: Cold	324
Table 5-162: Command Tactic.....	327
Table 5-163: Drop.....	329
Table 5-164: Holster	331
Table 5-165: Receive Package Tactic Status.....	333
Table 5-166: Set Tactic Shot Doctrine and Intercept Maneuver Parameters.....	335
Table 5-167: Shoot.....	337
Table 5-168: Shoot Now.....	339
Table 5-169: Target	341
Table 5-170: C2 Define Global Override	343
Table 5-171: Provide Risk Assessment Results.....	344
Table 5-172: Request Route Threat Assessment	345
Table 5-173: Update Acceptable Level of Risk	348
Table 5-174: Update Package Survivability Risk Definition	349
Table 5-175: Update Survivability Risk Definition	350
Table 5-176: Activate Constraint Priority List	353
Table 5-177: Update Constraint Priority List Activation Status	354
Table 5-178: Activate Execution Priority List.....	357
Table 5-179: Update Execution Priority List Activation Status.....	358
Table 5-180: Activate Ignore Zone Constraints.....	360
Table 5-181: Changing a Package Leader.....	362
Table 5-182: Command Team/Platform via Task.....	364
Table 5-183: Creating or Changing a Package	366
Table 5-184: Distribute C2 Track Data to Facilitate Global Fusion and Update COP	367
Table 5-185: Receive Package Status	368
Table 5-186: Uploading a Task.....	369
Table 5-187: C2 Threat Assessment.....	370
Table 5-188: DLZ Compound Convergence.....	371
Table 5-189: Exchange Shot Queue Prioritization	372
Table 5-190: Mission Store Status Exchange	373
Table 5-191: Query Entity Track History.....	375
Table 5-192: Query Shot Queue Prioritization.....	377

Table 5-193: Release Consent	379
Table 5-194: Report Intercept Pairings.....	380
Table 5-195: Report Shot Allocations	381
Table 5-196: Store Inventory and Descriptions Exchange.....	383
Table 5-197: Target Designation.....	384
Table 5-198: Target Designation Request.....	386
Table 5-199: Threat Prioritization.....	388
Table 5-200: Update Shot Queue Prioritization.....	389
Table 5-201: WEZ Compound Convergence	390
Table 5-202: C2 Interface Compliance Requirements	391
Table 6-1: Mission Debrief Interface Compliance Requirements	392
Table 7-1: MA Receives Action Capability from MP.....	397
Table 7-2: MA Receives Air Sample Capability from MP	398
Table 7-3: MA Receives AMTI Capability from MP	399
Table 7-4: MA Receives AO Capability from MP	400
Table 7-5: MA Receives Cargo Delivery Capability from MP	401
Table 7-6: MA Receives COMINT Capability from MP	402
Table 7-7: MA Receives Communications Relay Capability from MP	403
Table 7-8: MA Receives Communications Terminal Capability from MP	404
Table 7-9: MA Receives Counterspace Capability from MP	405
Table 7-10: MA Receives EA Capability from MP	406
Table 7-11: MA Receives Effect Capability from MP	407
Table 7-12: MA Receives ESM Capability from MP	408
Table 7-13: MA Receives Flight Capability from MP	409
Table 7-14: MA Receives Gateway Capability from MP	410
Table 7-15: MA Receives IFF Capability from MP	411
Table 7-16: MA Receives Opaque Capability from MP.....	412
Table 7-17: MA Receives Orbit Change Capability from MP	413
Table 7-18: MA Receives Orbital Surveillance Capability from MP.....	414
Table 7-19: MA Receives Orbital Surveillance Sensor Capability from MP	415
Table 7-20: MA Receives PO Capability from MP	416
Table 7-21: MA Receives Radar Altimeter Capability from MP	417
Table 7-22: MA Receives Reference Capability from MP.....	418

Table 7-23: MA Receives Refuel Capability from MP	419
Table 7-24: MA Receives Response Capability from MP	420
Table 7-25: MA Receives SAR Capability from MP	421
Table 7-26: MA Receives SMTI Capability from MP	422
Table 7-27: MA Receives Strike Capability from MP	423
Table 7-28: MA Receives System Deployment Capability from MP	424
Table 7-29: MA Receives Tactical Order Capability from MP	425
Table 7-30: MA Receives Weather Radar Capability from MP	426
Table 7-31: MA Receives Action from MP	427
Table 7-32: MA Receives Action Plan from MP	428
Table 7-33: MA Receives Activity Metrics from MP	429
Table 7-34: MA Receives Activity Plan from MP	430
Table 7-35: MA Receives Analysis Route from MP	431
Table 7-36: MA Receives Mission Plan from MP	432
Table 7-37: MA Receives OpPlan from MP	433
Table 7-38: MA Receives Response from MP	434
Table 7-39: MA Receives Response Plan from MP	435
Table 7-40: MA Receives Route Activity Plan from MP	436
Table 7-41: MA Receives Route Metrics from MP	437
Table 7-42: MA Receives Route Plan from MP	438
Table 7-43: MA Receives Task from MP	439
Table 7-44: MA Receives Task Plan from MP	440
Table 7-45: MA Receives Access Assessment from MP	441
Table 7-46: MA Receives Airfield Report from MP	442
Table 7-47: MA Receives Assessment from MP	443
Table 7-48: MA Receives Designation from MP	444
Table 7-49: MA Receives DLZ from MP	445
Table 7-50: MA Receives Entity from MP	446
Table 7-51: MA Receives Foreign Key Pair from MP	447
Table 7-52: MA Receives Leadership Metrics from MP	448
Table 7-53: MA Receives OpLine from MP	449
Table 7-54: MA Receives OpPoint from MP	450
Table 7-55: MA Receives OpRouting from MP	451

Table 7-56: MA Receives OpVolume from MP	452
Table 7-57: MA Receives OpZone from MP	453
Table 7-58: MA Receives Package from MP	454
Table 7-59: MA Receives Planning Function from MP	455
Table 7-60: MA Receives Store Loadout Configuration from MP	456
Table 7-61: MA Receives Survivability Risk Level from MP	457
Table 7-62: MA Receives System Readiness from MP	458
Table 7-63: MA Receives Weather Dataset from MP	459
Table 7-64: MA Receives Weather from MP	460
Table 7-65: MA Receives WEZ from MP	461
Table 7-66: MA Receives Approval Authority from MP	462
Table 7-67: MA Receives Approval Policy from MP	463
Table 7-68: MA Receives Authorization from MP	464
Table 7-69: MA Receives Engagement Criteria from MP	465
Table 7-70: MA Receives Interoperability Policy from MP	466
Table 7-71: MA Receives Operator Role from MP	467
Table 7-72: MA Receives Constraint Priority List Activation Data from MP	468
Table 7-73: MA Receives Constraint Priority List from MP	469
Table 7-74: MA Receives Execution Priority List Activation Data from MP	470
Table 7-75: MA Receives Execution Priority List from MP	471
Table 7-76: MA Receives Prioritization List from MP	472
Table 7-77: MA Receives Capability Set from MP	473
Table 7-78: MA Receives Requirement Metrics from MP	474
Table 7-79: MA Receives Requirement Set from MP	475
Table 7-80: Mission Plan Interface Compliance Requirements	475
Table 8-1: Communications Lost	481
Table 8-2: Configure Comm Terminal.....	482
Table 8-3: Configure Gateway	484
Table 8-4: Establish Comm Terminal.....	486
Table 8-5: Maintain Comm Terminal.....	487
Table 8-6: Override Data Dissemination Plan	489
Table 8-7: Publish Comm Team Report	491
Table 8-8: Publish Network Endpoint Availability	492

Table 8-9: Receive Communication Error Report	493
Table 8-10: Request Communication Asset Status.....	494
Table 8-11: Update Comm Terminal Plan	495
Table 8-12: Update Comms Gateway Status	497
Table 8-13: Update Data Dissemination Plan	501
Table 8-14: Update Data Dissemination Plan Activation Status	502
Table 8-15: Intra-Vehicle Comms Failure_MS	503
Table 8-16: Logging Failure	504
Table 8-17: MS Malformed Message Response	505
Table 8-18: Payload Management Failure.....	506
Table 8-19: Sensor Failure.....	507
Table 8-20: MS Comms Status Updates to COP	508
Table 8-21: MS Sensor AMTI Status Updates to COP	510
Table 8-22: MS Sensor PO Status Updates to COP	512
Table 8-23: MS Weapon Status Updates to COP	516
Table 8-24: Distribute Single Sensor Track Data to Perform Local Fusion.....	517
Table 8-25: Receive Processed Track.....	519
Table 8-26: Receive Message Subscription Status.....	522
Table 8-27: Receive UCI Wrapped Message	523
Table 8-28: Message Send Destination Logic.....	525
Table 8-29: Send UCI Wrapped Message.....	528
Table 8-30: Subscribe to Message Topic	530
Table 8-31: EXI Settings	532
Table 8-32: Receive Plans	535
Table 8-33: Verify Plan Patch Is Possible - MS.....	538
Table 8-34: Request PNT Navigation Data	539
Table 8-35: Advertise Sensor AMTI Capability.....	541
Table 8-36: Advertise Sensor ESM Capability	542
Table 8-37: Advertise Sensor PO Capability	543
Table 8-38: Command Sensor AMTI Capability.....	545
Table 8-39: Command Sensor ESM Capability.....	549
Table 8-40: Command Sensor PO Capability	551
Table 8-41: Publish External Weather Reports	553

Table 8-42: Report Sensor AMTI Capability State	555
Table 8-43: Report Sensor ESM Capability State	556
Table 8-44: Report Sensor PO Capability State.....	558
Table 8-45: Report Weather Radar Capability Observations.....	559
Table 8-46: Mission Systems Capability Heartbeat.....	561
Table 8-47: MS Responds to Query for Capabilities	564
Table 8-48: Publish Periodic Passive Optical Sensor Configuration	566
Table 8-49: Receive Periodic Health, Status, & BIT Information	568
Table 8-50: Request Mission System Health, Status & BIT Information	571
Table 8-51: Abort Release Sequence.....	572
Table 8-52: Confirm Threat ID	574
Table 8-53: Exchange DLZ	575
Table 8-54: Exchange WEZ	577
Table 8-55: Fire Command Exchange.....	579
Table 8-56: Jettison Release Sequence.....	581
Table 8-57: Mission Store Release Initialization Sequence.....	582
Table 8-58: Mission Store Status Exchange	584
Table 8-59: Receive Changes to Stores Primary Mission Plan Data	587
Table 8-60: Receive Weapon Flyout State	588
Table 8-61: Release Consent Sequence.....	590
Table 8-62: Send Weapon Entity Data	592
Table 8-63: Store Inventory and Descriptions Exchange.....	593
Table 8-64: Mission Systems Interface Compliance Requirements	594
Table 9-1: P2P Message Acknowledgment Configuration Status	597
Table 9-2: P2P Update Message Acknowledgment Configurations.....	599
Table 9-3: Receive CAP Execution Status	601
Table 9-4: Peer Overrides Data Dissemination Plan.....	603
Table 9-5: Peer Updates Comms Gateway Status	605
Table 9-6: Peer Updates Data Dissemination Plan.....	607
Table 9-7: Peer Updates Data Dissemination Plan Activation Status	609
Table 9-8: Define Autonomous Contingency Re-Plan	611
Table 9-9: Define Team Lost Comms Contingency	614
Table 9-10: Multiple Peers Lost Comms with Team Leader	617

Table 9-11: P2P Incorrect Messaging Sequence Error Handling	624
Table 9-12: P2P Malformed Message/Precondition Not Met Handling	627
Table 9-13: P2P Path Deconfliction Proposal	629
Table 9-14: Peer Activate a Contingency	630
Table 9-15: Peer Receive Contingency Status.....	631
Table 9-16: Peer Total Comms Lost with Team	633
Table 9-17: Team Leader Lost Comms with C2.....	636
Table 9-18: Triggered Contingency Alert to Peer	638
Table 9-19: Configure COP and Request COP Configuration Settings Status (P2P)	641
Table 9-20: Coordinate Simultaneous Arrival Time.....	642
Table 9-21: COP Follower Configuration Update	644
Table 9-22: Distribute Sensor Track Data to Facilitate Package Fusion and Update COP	647
Table 9-23: Synchronize Global COP to Peer	649
Table 9-24: P2P Send Platform Override Status.....	650
Table 9-25: P2P State Data	652
Table 9-26: Peer Heartbeat.....	653
Table 9-27: Peer Publish Control Status	654
Table 9-28: Send Abort to Peer	655
Table 9-29: Send AMTI Capability State to Peer	657
Table 9-30: Send PO Capability State to Peer	659
Table 9-31: Send Strike Capability State to Peer	662
Table 9-32: Station-Keeping Capabilities	665
Table 9-33: Activate Peer Engagement Criteria	669
Table 9-34: Peer Sends Message Acknowledgment	670
Table 9-35: Peer Command Geozone	672
Table 9-36: P2P Contingency Alert.....	673
Table 9-37: P2P Detect Mission Plan Route Discontinuity Response.....	674
Table 9-38: P2P Request Plan Patch	675
Table 9-39: Peer Receive Plan Modification Notification	677
Table 9-40: Query Deconfliction Data.....	680
Table 9-41: Synchronize Contingency Plan	681
Table 9-42: Update Peer Airfield Data	682
Table 9-43: P2P Lead Updates Member with Constraint Priority List	683

Table 9-44: P2P Lead Updates Member with Constraint Priority List Activation Data.....	684
Table 9-45: P2P Lead Updates Member with Engagement Criteria.....	684
Table 9-46: P2P Lead Updates Member with Execution Priority List	685
Table 9-47: P2P Lead Updates Member with Execution Priority List Activation Data	686
Table 9-48: P2P Member to Lead Update with Constraint Priority List Activation Status.....	687
Table 9-49: P2P Member to Lead Update with Execution Priority List Activation Status	687
Table 9-50: P2P Member to Lead Update with System Notification.....	688
Table 9-51: P2P Query Activity Plan.....	691
Table 9-52: P2P Query Airfield Report	693
Table 9-53: P2P Query Mission Plan.....	695
Table 9-54: P2P Query OpZone	697
Table 9-55: P2P Query Response Plan.....	699
Table 9-56: P2P Query Route Activity Plan.....	701
Table 9-57: P2P Query Route Plan	703
Table 9-58: P2P Query Task Plan	705
Table 9-59: P2P Query Weather.....	707
Table 9-60: Send Action Plan to Peer.....	708
Table 9-61: Send Action to Peer.....	709
Table 9-62: Send Activity Plan to Peer	710
Table 9-63: Send Approval Authority to Peer	711
Table 9-64: Send Approval Policy to Peer.....	712
Table 9-65: Send Capability Set to Peer	713
Table 9-66: Send Component Status to Peer.....	714
Table 9-67: Send Entity to Peer	715
Table 9-68: Send Flight Activity to Peer	716
Table 9-69: Send Flight Capability Status to Peer	717
Table 9-70: Send Flight Capability to Peer	718
Table 9-71: Send Mission Plan Execution Status to Peer.....	719
Table 9-72: Send Mission Plan to Peer	720
Table 9-73: Send Navigation Report to Peer.....	721
Table 9-74: Send Operator Role to Peer.....	722
Table 9-75: Send Operator to Peer.....	723
Table 9-76: Send OpLine to Peer	724

Table 9-77: Send OpPlan to Peer.....	725
Table 9-78: Send OpRouting to Peer.....	726
Table 9-79: Send OpVolume to Peer.....	727
Table 9-80: Send OpZone to Peer.....	728
Table 9-81: Send Position Report to Peer.....	729
Table 9-82: Send Response Plan to Peer.....	730
Table 9-83: Send Response to Peer.....	731
Table 9-84: Send Route Activity Plan to Peer.....	732
Table 9-85: Send Route Deconfliction Alert to Peer.....	733
Table 9-86: Send Route Plan Execution Status to Peer.....	734
Table 9-87: Send Route Plan to Peer.....	735
Table 9-88: Send Subsystem Status to Peer.....	736
Table 9-89: Send System Status to Peer.....	737
Table 9-90: Send Task Plan to Peer.....	738
Table 9-91: Send Task Status to Peer.....	739
Table 9-92: Send Task to Peer.....	740
Table 9-93: Leader Election Method Static/Dynamic Support.....	741
Table 9-94: Bully Leader Election.....	744
Table 9-95: Create a Full Package Amongst Peers.....	746
Table 9-96: Current Package Lead Assigns New Package Lead.....	746
Table 9-97: Initializing a Package Amongst Peers.....	749
Table 9-98: Leader Commanded Package Addition.....	750
Table 9-99: Maximum Consensus Leader Election.....	753
Table 9-100: Notify Peers of Package Formation Command from Unauthorized C2.....	754
Table 9-101: Off Nominal Leader Election.....	757
Table 9-102: Peer Declares Package Leadership to Peers.....	758
Table 9-103: Peer Joining a Package.....	759
Table 9-104: Raft Leader Election.....	761
Table 9-105: Share DLFS with Peers.....	762
Table 9-106: Static Fitness Score Leader Election.....	766
Table 9-107: Peer Provides Route Risk Assessment Results.....	767
Table 9-108: Update Peer Acceptable Level of Risk.....	769
Table 9-109: Update Peer Survivability Risk Definition.....	770

Table 9-110: Activate Peer Constraint Priority List.....	771
Table 9-111: Activate Peer Execution Priority List	773
Table 9-112: Activate Peer Ignore Zone Constraints	774
Table 9-113: Allocate an Altitude Stacked Marshall.....	777
Table 9-114: Allocating an Escort Task	778
Table 9-115: Command Peer Capability.....	781
Table 9-116: Command Peer via Action	786
Table 9-117: Command Peer(s) via Task.....	787
Table 9-118: Control Mode Authorization	790
Table 9-119: Package Plan Approval Status	791
Table 9-120: Peer Activate or Deactivate Mission Plan	794
Table 9-121: Peer Responds to Approval Request.....	795
Table 9-122: Peer to Peer HSA/CSA Command.....	798
Table 9-123: Peer to Peer Waypoint Follow	800
Table 9-124: Peer Validate Action Plan.....	802
Table 9-125: Provide and Receive Formation Status	803
Table 9-126: Report Task Status to Leader	804
Table 9-127: Respond to Command from Team Lead.....	809
Table 9-128: P2P Threat Assessment.....	813
Table 9-129: Centralized Computation of Target Assignment.....	815
Table 9-130: Distributed Computation of Target Assignment	816
Table 9-131: DLZ Compound Convergence.....	817
Table 9-132: Peer Assign Target Pairings.....	818
Table 9-133: Peer Report Tactic Status	820
Table 9-134: Peer Send Tactic Parameters	821
Table 9-135: Peer Target Designation Request.....	822
Table 9-136: Query Entity History	824
Table 9-137: Query Entity Track History.....	827
Table 9-138: Query Shot Queue Prioritization.....	829
Table 9-139: Send Shot Queue Prioritization	831
Table 9-140: Threat Prioritization.....	832
Table 9-141: Update Shot Queue Prioritization.....	833
Table 9-142: WEZ Compound Convergence	834

Table 9-143: Peer Interface Compliance Requirements	835
Table 10-1: Collision Avoidance.....	838
Table 10-2: Intra-Vehicle Comms Failure	839
Table 10-3: MA Failsafe	840
Table 10-4: Mechanical Damage Reporting	841
Table 10-5: Sensor Failure.....	842
Table 10-6: VI Malformed Message Response	844
Table 10-7: Control by Curve Following	848
Table 10-8: Control by HSA/CSA Command.....	851
Table 10-9: Control by Waypoint Following	853
Table 10-10: Control Mode Authorization	856
Table 10-11: MA-VI Command Task	858
Table 10-12: Modify Capabilities.....	859
Table 10-13: Receive Control Request.....	862
Table 10-14: Unpair Control Assignment.....	863
Table 10-15: Update C2 Control Designations.....	865
Table 10-16: VI Updates to COP	867
Table 10-17: Checksum Validation	869
Table 10-18: Query for Missing Data	871
Table 10-19: Activate Route.....	874
Table 10-20: Convert and Upload Route	878
Table 10-21: Prepare for Route Activation	879
Table 10-22: Receive Deactivate Route	881
Table 10-23: Validate Route Plan	882
Table 10-24: Verify Plan Patch Is Possible - VI.....	883
Table 10-25: VI Deactivate Route.....	884
Table 10-26: Exchange Heartbeat - Subsystem Status Reports	887
Table 10-27: Publish Control Status	888
Table 10-28: Query Airfield Update	891
Table 10-29: Query Route Plan	894
Table 10-30: Receive Barometric Pressure.....	895
Table 10-31: Receive Execution Status.....	898
Table 10-32: Receive Vehicle Performance Values.....	899

Table 10-33: Receive Vehicle State Data.....	901
Table 10-34: Request Terrain Data	904
Table 10-35: Vehicle Status Reporting	906
Table 10-36: VI Responds to Query for Flight Capabilities	908
Table 10-37: Validate Release Envelope	910
Table 10-38: Vehicle Interface Compliance Requirements.....	911
Table 11-1: Component Interface Compliance.....	912

1 INTRODUCTION

1.1 Purpose

This Interface Control Document (ICD) is an export of the Mission Autonomy (MA) system and its interface architecture, as defined in Cameo Enterprise Architecture using the Systems Modeling Language (SysML). The interfaces described in this document represent the Autonomy - Government Reference Architecture (A-GRA): the reference autonomy architecture to support complete day-in-the-life autonomy capabilities that future MA instantiations will be based on.

The MA ICD provides key documentation for the MA system and interactions between MA software and external systems. For additional context in terms of the application of these interactions within the A-GRA and their tracing to other elements within the architecture, please refer to the SysML model or the Vignette Document for A-GRA. There are two architecture levels to MA – system level and component level. Level 1 (L1), the system level, defines the MA system and systems that interface with MA. Level 2 (L2), the component level, will be defined in a future version of the A-GRA and will encompass the component archetypes that the MA system is composed of. The A-GRA defines the architecture, compliance, and interfaces at each MA architecture level. Adherence to this ICD is mandatory for compliance and critical for future integration of MA systems. With the information in this ICD, stakeholders will have a detailed understanding of the requirements to develop an MA system and a system that interfaces with MA, which will allow for the development of advanced autonomous systems.

1.2 Scope

This document contains the following:

- A-GRA system level definition
- A-GRA component level definition
- A-GRA interface definition
- A-GRA compliance requirements

1.3 ICD Syntax

This section captures the syntax used throughout this ICD.

1.3.1 Requirements Syntax

The Requirements Syntax table details the syntax used for requirements in this document.

Table 1-1: Requirements Syntax

Term	Usage
shall	is required for compliance

1.3.2 Message Syntax

Throughout this document, messages from Universal Command and Control (C2) Interface (UCI) standards are referenced. The Message Syntax table outlines the syntax used for referring to these messages.

Table 1-2: Message Syntax

Message Type	Syntax
UCI Message	<PascalCase>

2 REFERENCE STANDARDS

The documents listed in the Referenced Standards table are cited throughout this document.

Table 2-1: Referenced Standards

Version Number	Title	Release Date
Version 2.3	Open Mission Systems (OMS) Standard	13 December 2023
Version 2.3	Universal Command and Control (C2) Interface (UCI) Standard	13 December 2023
Version 2.3	UCI Schema Style and Design Specification	13 December 2023
Version 1.0	Efficient XML Interchange (EXI) Format 1.0 (Second Edition)	11 February 2014
Version 6.0	Vignette Document	31 July 2026

3 MISSION AUTONOMY A-GRA OVERVIEW

3.1 A-GRA Introduction

The A-GRA is a general-purpose autonomy architecture design consisting of documentation, a Model Based Systems Engineering (MBSE) model, and data models that capture a common approach to an autonomy architecture that is compatible with MA L1 interfaces.

The A-GRA enables autonomy capabilities to evolve and grow. It is platform, mission, and implementation agnostic, but enables rapid composition and integration. It is expected that the A-GRA will support multiple variations and evolutions of PSIs of MA solutions. The ASK release includes the Autonomy-Government Reference Implementation (A-GRI) to exemplify and guide compliant MA solution implementations.

3.2 System Description

The primary advantage of the A-GRA is the versatility in its implementation. Each instantiation of MA is responsible for taking the various inputs from sensors, operators, and external entities, and performing mission-specific processing to yield a sequence of concrete tasks for the Autonomous Collaborative Platforms (ACPs) to execute. A-GRA compliant components are modular, adaptable, scalable, and portable to enable creation of varying capabilities that are themselves adaptable, scalable, and upgradable. The A-GRA defines six logical L1 interfaces: Vehicle Interface (VI), Peer (P2P), Command and Control (C2), Mission Systems (MS), Mission Plan (MP), and Mission Debrief (MD), shown in the MA L1 Logical Interfaces figure.

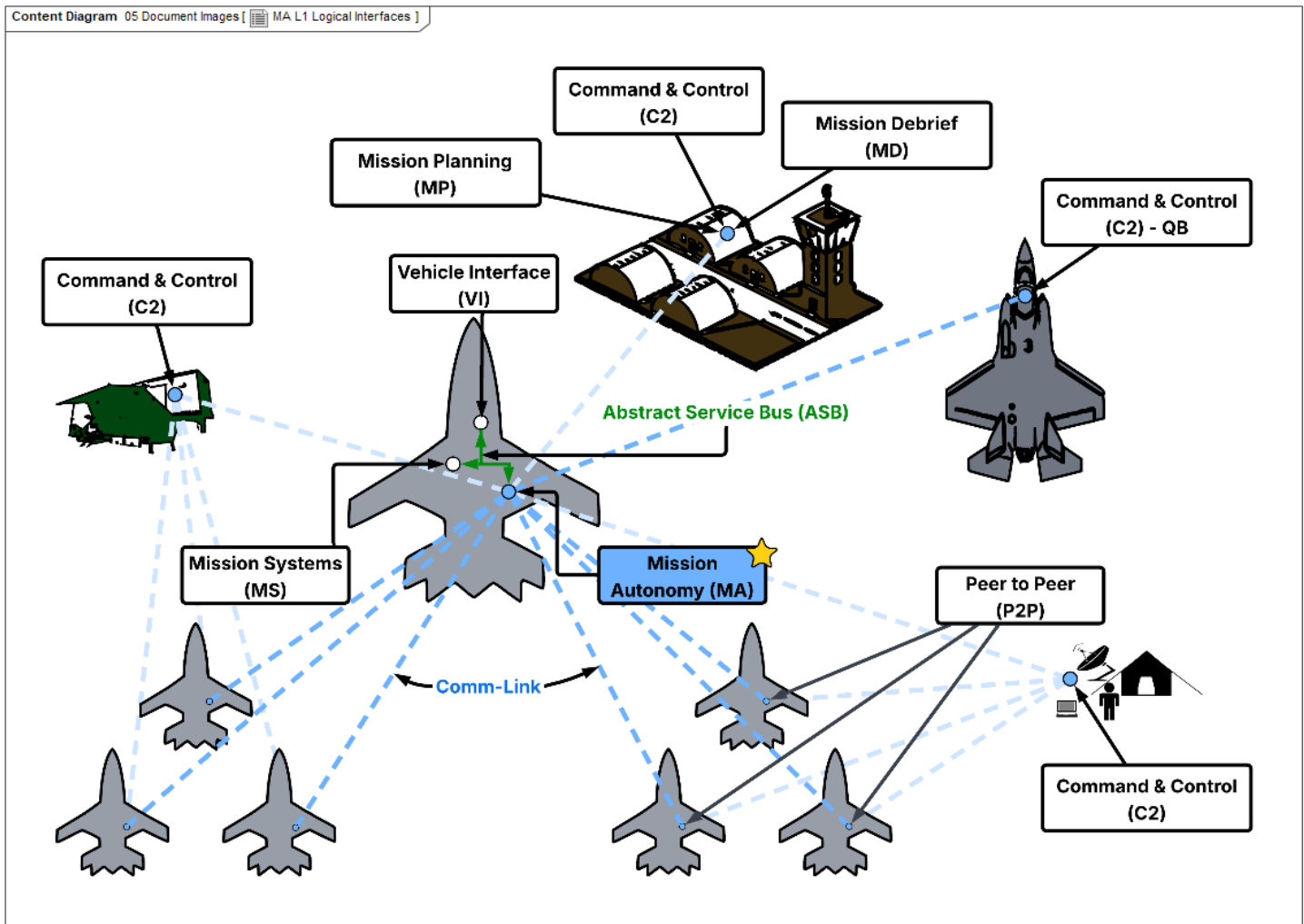


Figure 3-1: MA L1 Logical Interfaces

MA directly communicates with onboard Mission Systems and the VI, while also having a logical connection to external platforms. Mission Systems provide MA with access to network management, effects and payload management, sensor management, and inter-platform communication. The VI provides MA with vehicle control and flight status. Between these two L1 interfaces, MA can autonomously control the Platform.

Off-board the platform, MA can engage in machine-to-machine communication with other MA instances via the Peer interface, enabling dynamic behaviors with teams of ACPs. Correspondingly, operators can interact with MA instances via the C2, Mission Plan, and Mission Debrief interfaces respectively. These interfaces define how MA can be commanded from one or more operators, ingest Mission Plans, and support post-mission analysis. The L1 MA interfaces allow MA to have both intra- and inter-platform autonomous management, enabling the full scale of autonomy required for the Air Force. The MA L1 Logical Connections figure is a logical depiction of the MA boundary and where MA lies within the ACP.

This ICD defines the MA system and its external interfaces. It does not stipulate any requirements for the composition of external systems, therefore the composition of the external systems that interact with MA in the MA L1 Logical Connections figure are notional. For the mission planner interface, this ICD does not specify how mission data files will be loaded onto the vehicle, but instead focuses on

the data structures MA will expect to interact with to access mission plan information. This is defined in the Mission Plan Interface section of this ICD. In addition, the Mission Debrief interface currently describes the format for logging data that will be made available post-mission to external mission debrief systems.

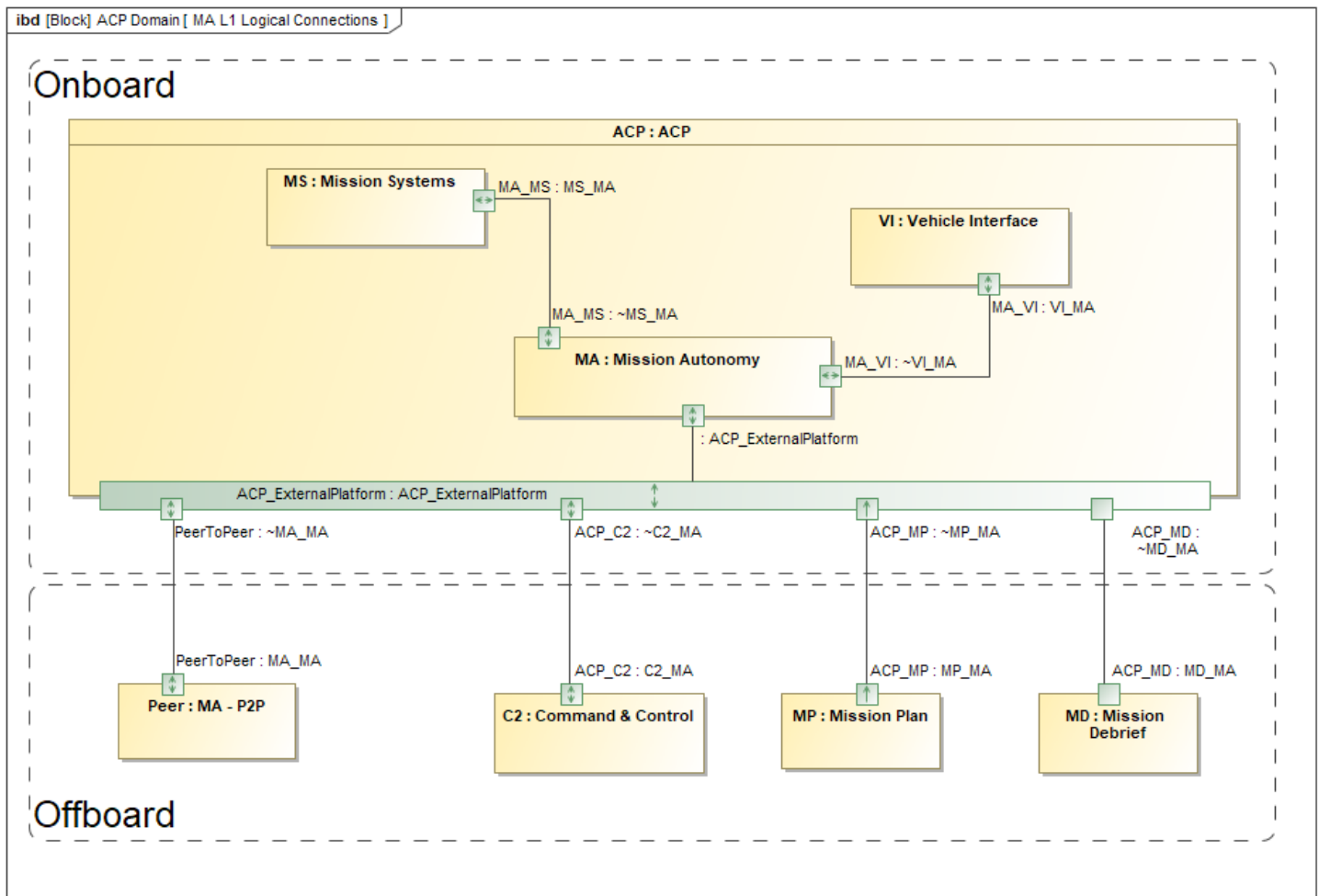


Figure 3-2: MA L1 Logical Connections

3.3 MA L1 External System Interface

The A-GRA defines interface and networking semantics sufficient to ensure interoperability between MA and its L1 interfaces. The onboard interfaces, VI and MS, shall use the Open Mission Systems (OMS) standard, including the Critical Abstraction Layer (CAL) API. A specific CAL implementation, serialization format, and Abstract Service Bus (ASB) implementation is not defined. These parameters are intended to be chosen during integration of specific MA implementation to a specific platform. The offboard interfaces, C2 and Peer, will use a defined wire format. This ensures that integration does not need to be performed for every pair of offboard interface implementation and platform implementation. For both types of interfaces, the UCI schema combined with A-GRA extensions define the specific message schema to be used, and the message sequences defined in the A-GRA ICD represents the Minimum Message Set (MMS) for each interface as well as the minimum set of message flows.

The VI and MS interfaces are onboard. The OMS services defining the boundaries of the onboard A-

GRA Level 1 Interfaces will be recompiled to the chosen platform-specific CAL implementation. The interface to MS will be the responsibility of the Mission Systems Integrator (MSI) to adapt to the relevant A-GRA L1 interfaces. For example, this may be accomplished either through building a MS adapter between the MA ASB and MS ASB or working together with the MA provider to build MA and MS services against a common CAL/ASB implement and version. Similarly, it will be the responsibility of the MSI or platform provider to adapt Vehicle Interface (VI) to the A-GRA L1 VI. For example, this may be accomplished with an OMS Isolator connecting the platform-specific VI to the MA ASB (in the event of separate MA and MS ASB) or connecting VI to the shared MS/MA ASB.

The C2 and Peer interfaces are offboard. For these interfaces, the A-GRA defines the message encoding as Efficient XML Interchange (EXI) 1.0, an efficient encoding of XML data. Details about this format can be found in the W3C recommendation at <https://www.w3.org/TR/exi/> and in the EXI primer located at <https://www.w3.org/TR/exi-primer/>. MA places messages destined for the offboard interfaces on the platform ASB along with metadata defining the destination of the message and priority. The MS comms manager skill will adapt the message if necessary and transmit it on the appropriate link to reach its destination. Inbound messages are processed in reverse and once decoded placed on the platform ASB for consumption by MA.

3.3.1 C2 Interface

The role of a C2 system in the operation of MA-enabled ACPs is to exercise authority and direction, through Commander's Intent, from a commander with RBAC of one or more ACPs during mission execution. The C2 interface covers command of an ACP in relation to: teaming, formation, RBAC, contingencies, interactions to vehicle management via C2, and heartbeat status. On the C2 interface, a formation is commanded via the Task path: *MA_TaskMT* to mission-plan/define a formation and *MA_TaskCommandMT* to activate it; *MA_FlightCommandMT* is not used to set a formation on the C2 interface. The C2 interface sends and receives off-board OTA network messages via the DMS (Decentralized Messaging Service). The C2 interface also includes the interactions for mid-execution updates to Mission Plan elements.

3.3.2 Peer Interface

The role of the Peer interface is to provide communication between multiple MA-enabled ACPs, enabling data to be shared to facilitate collaborative perception and behaviors among one or more teams of ACPs. The Peer interface covers sharing information relating to teaming and formation, RBAC, contingency, and discovery via UCI messaging. ACPs send and receive off-board OTA network messages via the DMS (Decentralized Messaging Service).

3.3.3 Mission Systems Interface

The Mission Systems interface is the connection between MA and onboard mission systems such as capabilities (e.g. Skills), sensor subsystems, payloads, Core Services (deployment and compute infrastructure abstractions), and Mission Data processing services. The Mission Systems interface relies on OMS, the standard for Mission Systems compliance, and other MOSA Open Transports for all interactions. All data exchanges across this interface can use the OMS specified Abstract Service Bus (ASB).

3.3.4 Mission Debrief Interface

The Mission Debrief (MD) interface supports the transfer of data related to activity and decisions that occur in MA or are captured from other onboard L1 Subsystems. This interface enables MA to

externally transfer captured data, which can be used for mission analysis/debrief and after-action review system features (e.g., playback, causal analysis, reports, simulation).

3.3.5 Mission Plan Interface

The L1 Mission Plan (MP) interface defines how the Mission Autonomy (MA) system accesses available mission data to facilitate mission execution. This data includes traditional mission planning inputs, such as weather, airspace boundaries, and airfield data; as well as data specific to MA, including route and task plans. The L1 MP interface is intended primarily for before-action use, but it can also be utilized during missions. Other L1 subsystems such as C2 and P2P may also utilize this interface, which is defined and owned by MP.

3.3.6 Vehicle Interface

The VI defines how MA commands vehicle motion to the platform. From MA's perspective, the VI is a system that includes guidance, navigation, high- and low-level flight control algorithms, and any flight safety-related functionality, some of which may be beyond the scope of a traditional VMS. MA focuses purely on mission execution by providing commands to the VI, but MA does not handle safety-critical functions; the VI must accept or reject commands as necessary to ensure safe operation.

3.4 Autonomy Schema

The Autonomy Schema is a custom UCI schema based off UCI 2.3, listed in the Referenced Standards table, created specifically for autonomy purposes. MA utilizes the Autonomy Schema to accomplish autonomy tasking and execute autonomy behaviors. This ICD is expected to be used alongside the Autonomy Schema. It is highly recommended to open the Autonomy Schema with an XML viewer tool.

In a future release of the A-GRA, the Autonomy Schema will be formally submitted as a UCI extension schema and have its own Change Package process. The purpose of this is to stay in line with the UCI schema, while also expediting the Change Package process. Autonomy Schema changes will be submitted to the OACWG for inclusion.

3.5 Network Stack Interoperability

Background

Interoperability between interconnected A-GRA-compliant systems is required to enable ACPs, C2 systems, and other DoD programs to communicate. In order to ensure interoperability between A-GRA-compliant systems, there exists a layered model of interconnection and the complete selection of specific protocols, tools, and configurations for each layer of the interconnection stack. It is understood that deviations may be required for legacy systems or special circumstances.

The layering indicated below is not a strict mapping to the [OSI model](#). For the purposes of the A-GRA, the Distributed Message Service is considered the Application, layers 1-5 are defined in the table below. Layers 6 and 7 are included for context, but are already defined elsewhere. Identification of specific protocols for each layer does not imply that other standards and applications aren't also being used simultaneously for other purposes.

Table 3-1: Network Stack Layers

#	Layer	Proposed Standard
7	Autonomy Applications	A-GRA (e.g. MP, MA, C2, MD, etc.) message interactions
6	Presentation	UCI (serialized in Efficient XML Interchange (EXI) format, compressed, etc.)
5	Application	DMS (Data Distribution Service (DDS), Common Data Representation (CDR))
4	Transport Security	TLS/mTLS/DTLS
3	Transport	TCP/UDP
2	Network / Internet	IP, IPsec
1	Physical + Data Link	Defined by host system

Layer Context:

Link layer: Host systems provide the physical link layer, and as such, it's not strictly under the control of A-GRA to dictate what capabilities the host system links provide. It is assumed that links provide connectivity between all other nodes of the same link type and configuration (e.g. a network) and ability to transport arbitrary data. Different links exist and without a bridge, translator, or similar capability, multiple separate link-types are not automatically connected. Links are subject to a variety of real-world limitations, and network design and link-terminal configuration is outside the scope of A-GRA. Notably, platforms can provide multiple links and as a result systems can be multi-homed, having network addresses on each of the available links.

Network layer: It is recommended that host platforms support at least IP-based communication over physical links. It is acknowledged that legacy systems which do not support this recommendation, and link-specific adaptation will be used for these links. Network management, provisioning, routing, etc. are out of scope of the A-GRA, but it is assumed that networks are OSI layer 3 routable.

Transport layer: The use of TCP and UDP is recommended for transport layer communication. It is understood that TCP has increased session overhead and assumed that underlying network layers are stable enough for TCP connections. We will evaluate QUIC for future use. We recommend using Transport Layer Security, specifically Mutual TLS (mTLS) and Datagram TLS (DTLS) to provide authentication and confidentiality for data in transit.

Application layer: The Distributed Message Service is the application for the purposes of the A-GRA. It is responsible for performing content routing across multiple physical links, brokering messages in the pub/sub and point-to-point messaging semantics, and functioning as the gateway between A-GRA applications and other nodes in the network. The DMS is also responsible for routing to legacy links, selecting the correct link to utilize for given message traffic. DMS provides security and message brokering using DDS and DDS-Security and operates in a mesh configuration to the

maximum extent possible. DMS wraps, unwraps, and transforms data as necessary between onboard A-GRA messages and the format required for offboard links, Common Data Representation (CDR) for IP links or a link-specific format. The DMS specification will detail necessary DDS settings, topic mappings, the Interface Definition Language (IDL) schema for A-GRA messages, and options for Quality of Service (QoS) settings. A single DMS is assumed to exist within each system on the network. It is assumed that an RTPS relay or node-list-based discovery may be needed if network topology does not allow for RTPS/SPDP-based discovery.

4 SYSTEM INTERFACE COMPLIANCE

4.1 System Interface Compliance Introduction

The A-GRA prescribes compliance requirements for MA to meet the intended behavior as defined by the reference architecture. L1 interface compliance is defined at the system level, where the MA system is viewed as a black box interacting with external systems.

Mission Use Cases (MUCs) identify specific implementation needs for different ACP solutions. These include a Core set of behaviors that all A-GRA-compliant systems must meet, as well as select use cases such as Carrier-based or Electronic Support Measure-capable. MUCs are considered to be L0 as they describe the use of an ACP, not necessarily a single L1 system. For L1 interface compliance, all implementations must meet the interactions delegated to the Core MUC and all interactions delegated to other MUCs relevant to their implementation. Compliance to interactions not delegated to MUCs do not count towards A-GRA compliance.

To claim L1 interface compliance, an MA L1 system must implement all compliance requirements for each implemented L1 interface as governed by the MA L1 Compliance Requirements, detailed in the MA L1 Interface Compliance References table. This implementation must be verified for compliance using the A-GRI Software Compliance Tool. For platforms/systems supporting MUCs in addition to Core, that functionality shall be implemented in a way that complies with the specified interfaces and interactions delegated to the MUC.

The A-GRA imposes L1 interface compliance requirements on C2, Peer, Mission Plan, Mission Systems, VI, and Mission Debrief interfaces. The C2, Peer, and VI L1 interfaces require implementation of an MMS and corresponding message interactions, defined by the sequence diagrams in this ICD. The Mission Systems L1 interface strictly follows the OMS standard, requiring implementation of the Mission Systems' OMS Service Contracts. The Mission Plan L1 interface is defined in this ICD as the minimum required Mission Data elements required per Mission Data Load. The MA L1 Interface Compliance References table summarizes the interface compliance information and points to document sections which define and discuss these requirements.

Table 4-1: MA L1 Interface Compliance References

Interface	Interface Definition Section	Interface MMS Section
C2	Section 4.1	Section 6.1.2.1
Peer	Section 4.2	Section 6.1.2.2
Mission Plan	Section 4.3	Section 6.1.2.3
Mission Debrief	Section 4.4	None
Mission Systems	Section 4.5	Section 6.1.2.4
Vehicle Interface	Section 4.6	Section 6.1.2.5

4.1.1 Required Fields for A-GRA Compliance

All fields that are required in Autonomy Schema are required to be A-GRA compliant. In addition, all the required fields specified in the Message Set Tables are required for A-GRA compliance. Some fields within Autonomy Schema are ChoiceTypes. These fields require that a choice must be made if that field is used. However, which option to choose is up to the implementation. This ICD contains some sequences that require a specific option in a ChoiceType field must be selected for that sequence to be A-GRA compliant. If a specific ChoiceType option is required for a specific sequence, that option will be listed in the required fields column for that sequence in the Message Set Table. Likewise, some sequences in this ICD may specify that a specific field marked optional in UCI must be used in the sequence. To comply with this ICD, all fields marked as required by the sequences must be populated. Therefore, even if a field is marked optional by UCI but is required to perform a sequence defined in this ICD, the field shall be considered required for A-GRA compliance.

4.1.2 Message Set Tables

The message set tables beneath the sequence diagrams refine the message usage requirements for the specified use case. The message set tables are meant to supplement the Autonomy Schema and are not meant to be a replacement. In the message set tables, the Required Fields column only notes fields that are required for the specified interaction and are not required in Autonomy Schema. This includes fields that may be listed as optional in the schema, but are required for the specific interaction or if a specific option in a ChoiceType field is required to be used for the specific interaction. For message fields that are required for the interaction and are ChoiceTypes, the A-GRA will specify which (if any) of the ChoiceType options are required to be supported. For example, *MA_FlightCommandMT* includes the *FlightControlMode* field that is a ChoiceType with 3 options. In the "Control by HSA/CSA Command" interaction, it is required that the choice *HSA_CSA* be selected and used. This is specified in the required fields for that interaction. If a ChoiceType is optional in Autonomy Schema and required for an interaction, but any option is acceptable, only the ChoiceType field will be specified in the Required Fields column (not a specific ChoiceType option). There may be special cases where only a subset of applicable ChoiceTypes are available for a specific interaction, (e.g. if the schema allows for 5 options but the interaction only allows for 3), these cases will be specified in either a note on the sequence diagram or within the documentation of the interaction. ChoiceTypes that are already required in Autonomy Schema are required for all interactions and will not be listed in the required fields. If a ChoiceType option is not called out in any interaction, then that ChoiceType option should be considered optional and not required for A-GRA compliance. Fields that are already noted as required in the Autonomy Schema will not be included in the column as they will be required in all use cases. The message set tables do not contain an exhaustive list of required fields and the Autonomy Schema should be referenced for the remainder of the required and optional

fields for a message.

In summary, an implementation shall consider the following as required for A-GRA compliance for any specific interaction:

- Fields
 - Message set tables will not contain a complete list of required fields.
 - Fields and ChoiceTypes already required by the Autonomy Schema will not be listed in the required fields but will remain required for compliance.
 - Fields marked as required in a interaction's message set table may only be required for that specific interaction.
 - Fields marked as optional in the schema but marked as required in an interaction's message set table shall be considered as required to comply with the interaction.
- ChoiceType
 - ChoiceType options designated by interactions as required shall be considered the only options available to comply with the interaction.
 - If only a ChoiceType field is specified as a required field in a interaction's message set table, then any ChoiceType option is acceptable.
 - ChoiceType options not called out in any interaction shall be considered optional.
 - Diagram notes or documentation may restrict a list of ChoiceType options to only a subset of options.

The flow chart below depicts exactly which fields/ChoiceTypes will be specified in the Required Fields column in the Message Set Table.

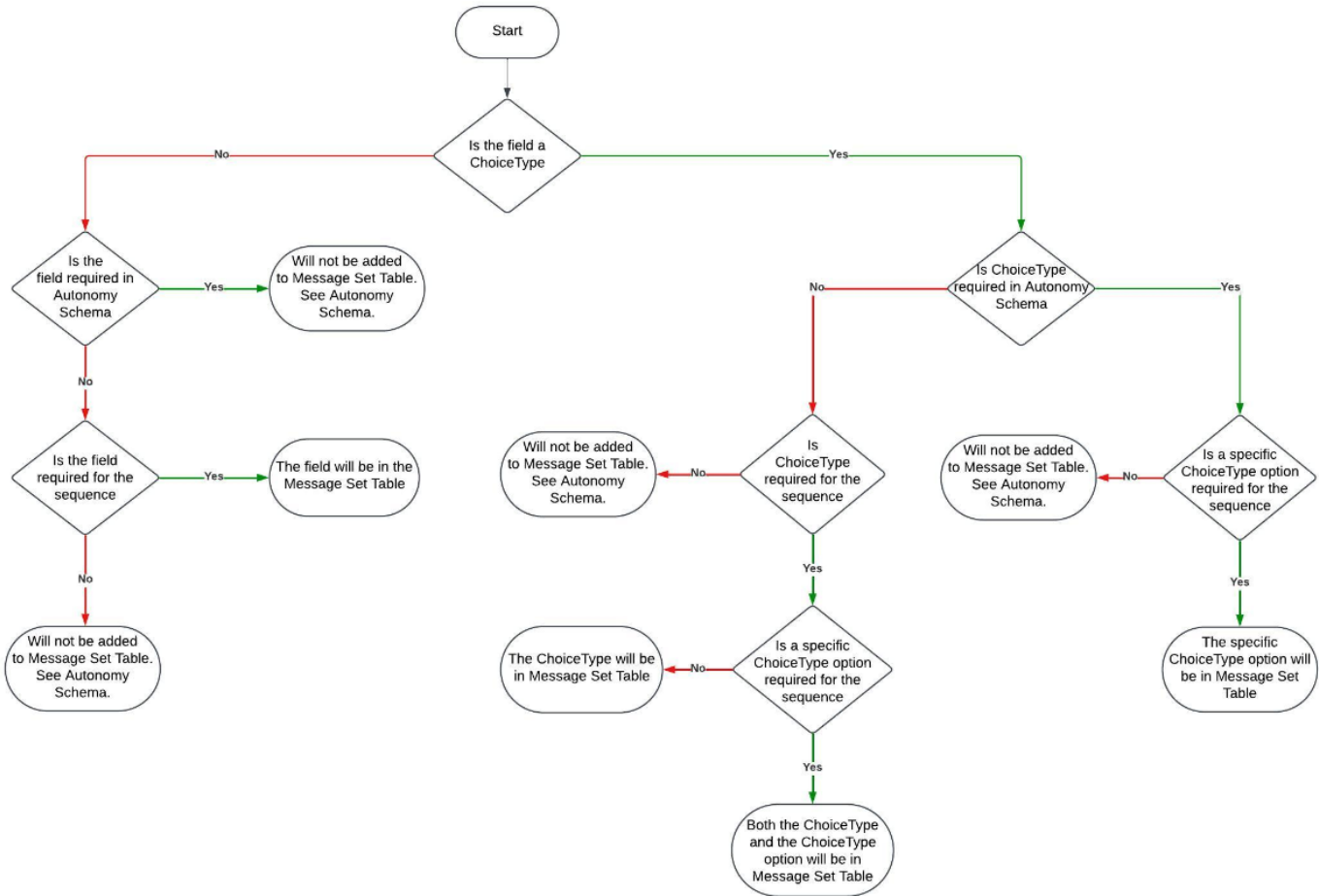


Figure 4-1: Message Set Table Flow Chart

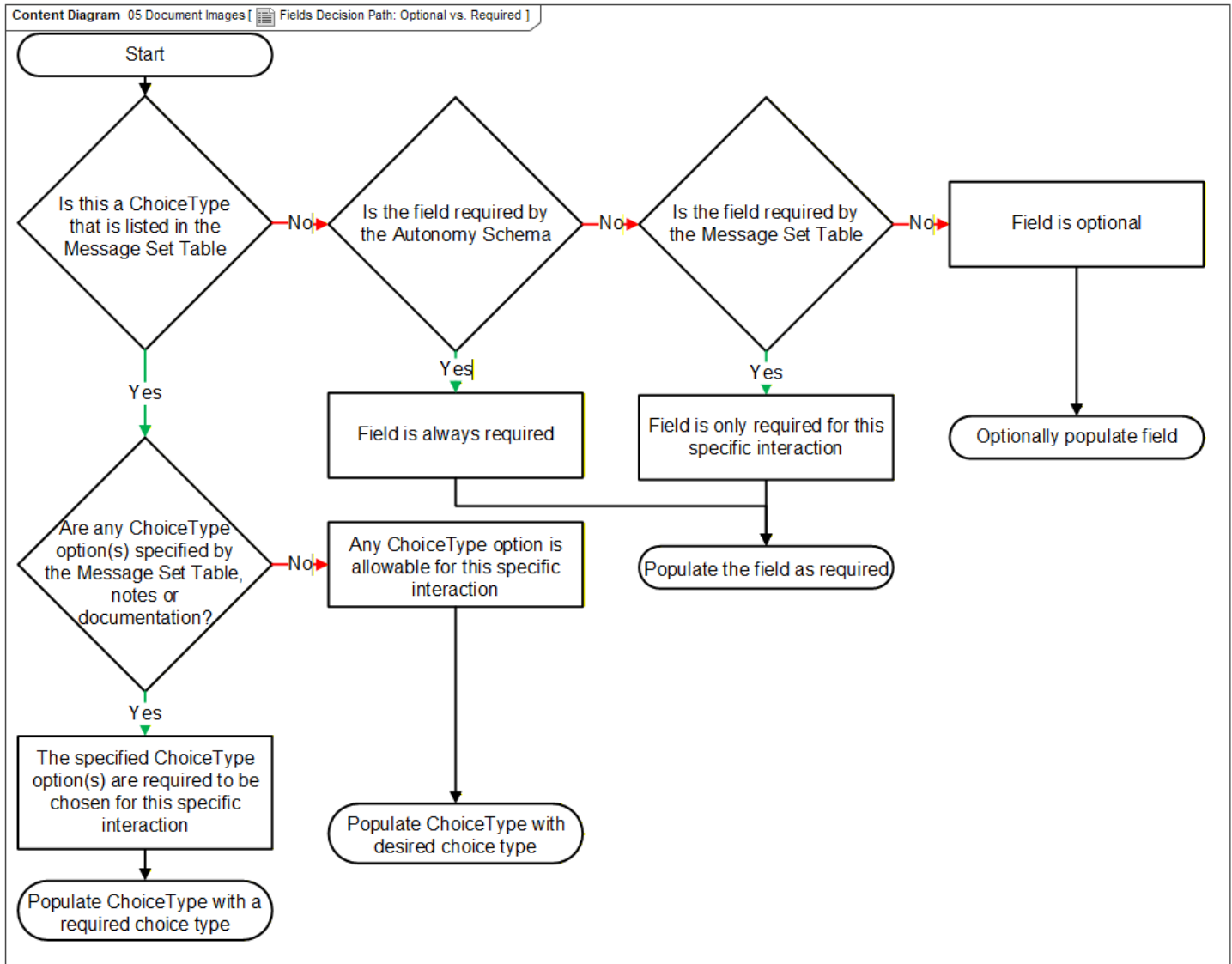


Figure 4-2: Fields Decision Path: Optional vs. Required

5 C2 INTERFACE

C2 for "Autonomous" Collaborative Platforms may seem like an oxymoron—the platforms for Mission "Autonomy" are not RPAs—however, the A-GRA is currently intended for semi-autonomous capabilities and platforms. The semi-autonomous designation keeps humans in the loop as much as is necessary for mission execution. The A-GRA C2 interface documentation should be approached with that context in mind.

The C2 L1 interface includes all interactions between a C2 "entity," e.g., a user, and MA. Compliant MA solutions must appropriately process and respond to all required C2 L1 interactions. C2 is typically accomplished utilizing a C2 system (C2 application + C2 node) with a Human Machine Interface (HMI), however the A-GRA is method agnostic. Compliant C2 applications must only support all interactions required by their users. Compliance for C2 applications means implementing interactions according to the A-GRA definitions. C2 applications do not need to support every A-GRA defined C2 L1 interaction to meet compliance.

5.1 Interface Description

C2 is accomplished via three distinct paradigms, each designed for different operational situations and intents: Direct, Planned (Indirect), and Responsive.

Direct C2

This paradigm involves the issuance of live commands directly to executing MA(s) to address immediate needs. It allows C2 entities to dynamically manage assets and supersede pre-planned actions without manually creating or updating plans.

Planned C2

This paradigm refers to C2 performed by interacting with MA via the on-board mission plan(s) and data. Thus, it can be thought of as indirect C2. C2 entities can activate, deactivate, update, or send new mission plans and/or data to affect actions and responses throughout a mission as needed to achieve desired mission outcomes.

Responsive C2

This paradigm relies upon a trigger-and-response model where automated services monitor the environment for predefined events. When a trigger condition is met, the service automatically initiates a predetermined response.

These paradigms should be kept in mind when consuming C2 ICD content. Future versions of the ASK will present a clearer organization of the content.

5.2 Interface Interactions

The C2 - MA interface assumes "off-board" or "Over-The-Air" (OTA) communications are necessary for C2 entities to reach MA and does not include the interaction with MS in diagrams. In a platform implementation, the Decentralized Messaging Service (DMS), defined in the Mission Systems section, is the current approach to accomplish back and forth MA and C2 communication OTA. Both C2 and MA must utilize compliant DMS software and aligned configurations. The DMS utilizes the Data Distribution Service (DDS) middleware for communication which is also specified by the A-GRA. The core method for messages to transfer between C2 and MA utilizes a UCI message wrapper (*MA_TxDataPayloadCommandMT*) to send messages to the DMS and another UCI message wrapper (*MA_RxDataPayloadMT*) the DMS publishes when messages are received. These wrapper messages are specified in more detail in the Mission Systems section. The "Additional Comms Capability" MUC specifies a more holistic approach to managing the OTA communications with a Data Dissemination Plan. The Data Dissemination Plan specifies how often and whether ingress/egress messages will be subscribed/published directly on the ASB or if the UCI wrappers can be used. MA is responsible for authentication of the received messages before processing.

The DMS interaction must be implemented for both C2 and MA, either directly or via platform or enclave capabilities, for a compliant end-to-end system. See the appropriate Mission Systems content for details.



Figure 5-1: C2 Interactions

5.2.1 C2 Acknowledgment Configurations Package

5.2.1.1 C2 Message Acknowledgment Configuration Status

This interaction provides the current Message Acknowledgment Configuration defining what messages C2 will respond to with an acknowledgment upon receipt. C2 will report the current Message Acknowledgment Configuration on change by sending a *MA_MessageAcknowledgmentConfigurationMT* message.

The *DefaultGroupMessageFreq* field defines the frequency at which the Grouped Acknowledgment message is set. If the field is not set, there is no grouping. The *DefaultMessagesToAck* field sets the list of messages that an Acknowledgment message will be sent for when received. If the field is not set, all messages listed in *MA_AckMessageEnum* will receive a response. The *ReceiverSpecificSettings* field specifies how MA will respond to individual C2s and/or Peers if they do not fall under the default settings. If MA does not support Message Acknowledgment Configuration, the *ConfigurationSupported* field will be set to "FALSE" and it will respond with an ungrouped acknowledgment for all possible messages listed in *MA_AckMessageEnum*.

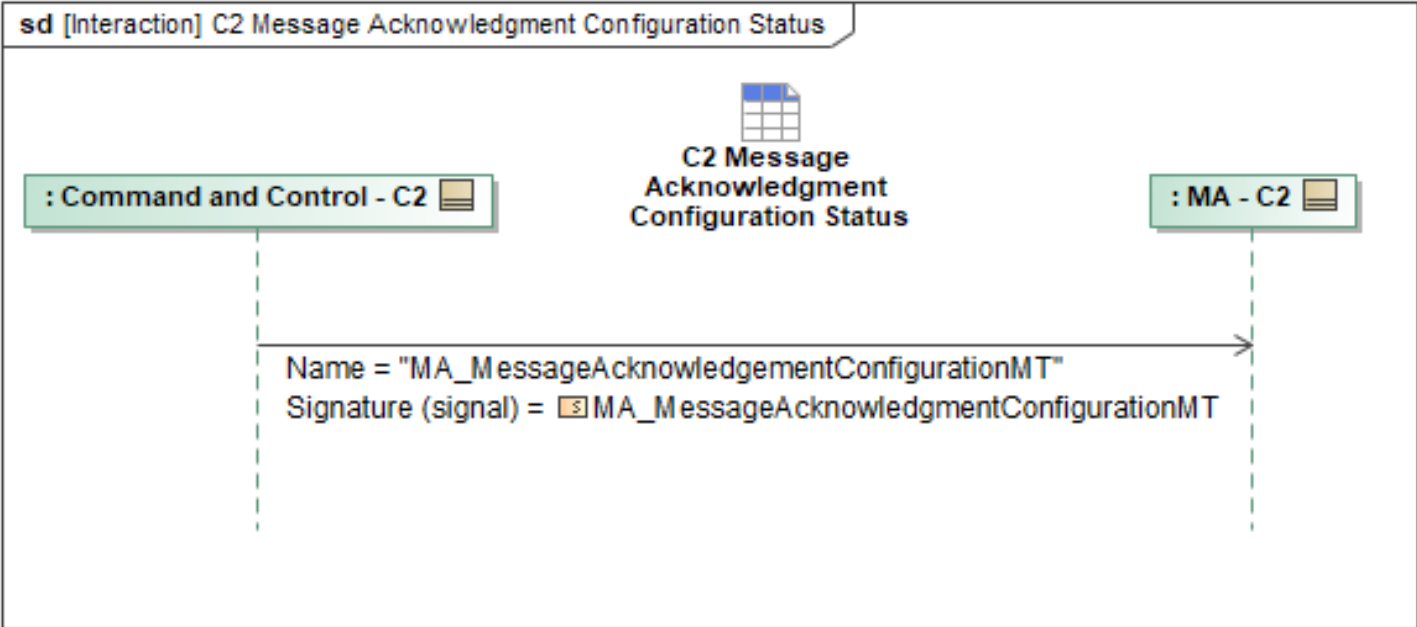


Figure 5-2: C2 Message Acknowledgment Configuration Status

Table 5-1: C2 Message Acknowledgment Configuration Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessageAcknowledgementConfigurationMT: MA_MessageAcknowledgmentConfigurationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.1.2 C2 Updates Message Acknowledgment Configuration

This interaction specifies when C2 wants to set or update the C2 Message Acknowledgment Configuration. C2 can update default responses by filling out the *DefaultGroupMessageFreq* and

DefaultMessagesToAck. C2 can also use the *RequestedReceiverSpecificSetting* field to define the messages that C2 expects MA to respond with an acknowledgment for and if the messages will be grouped. C2 should only request *RequestedReceiverSpecificSetting* for itself and not other systems.

C2 sends the *MA_MessageAcknowledgmentConfigurationRequestMT* message to MA. MA may accept or reject the configuration request. MA accepts the request by sending a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_1* with the *RequestProcessingState* set to "COMPLETED", followed by a *MA_MessageAcknowledgmentConfigurationMT* that reflects the updated configuration.

MA may reject the configuration request for the following reasons:

- It does not support configuration. MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "SYSTEM_UNAVAILABLE."
- There is limited bandwidth. MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message, sets the *RequestProcessingStateReason* to "INSUFFICIENT_RESOURCES", and fills out the description field.
- The system receives an unsupported input. MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "INVALID_INPUT_PARAMETER."
- MA requests *RequestedReceiverSpecificSetting* for another system. MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "INELIGIBLE_CONTROL_SOURCE."

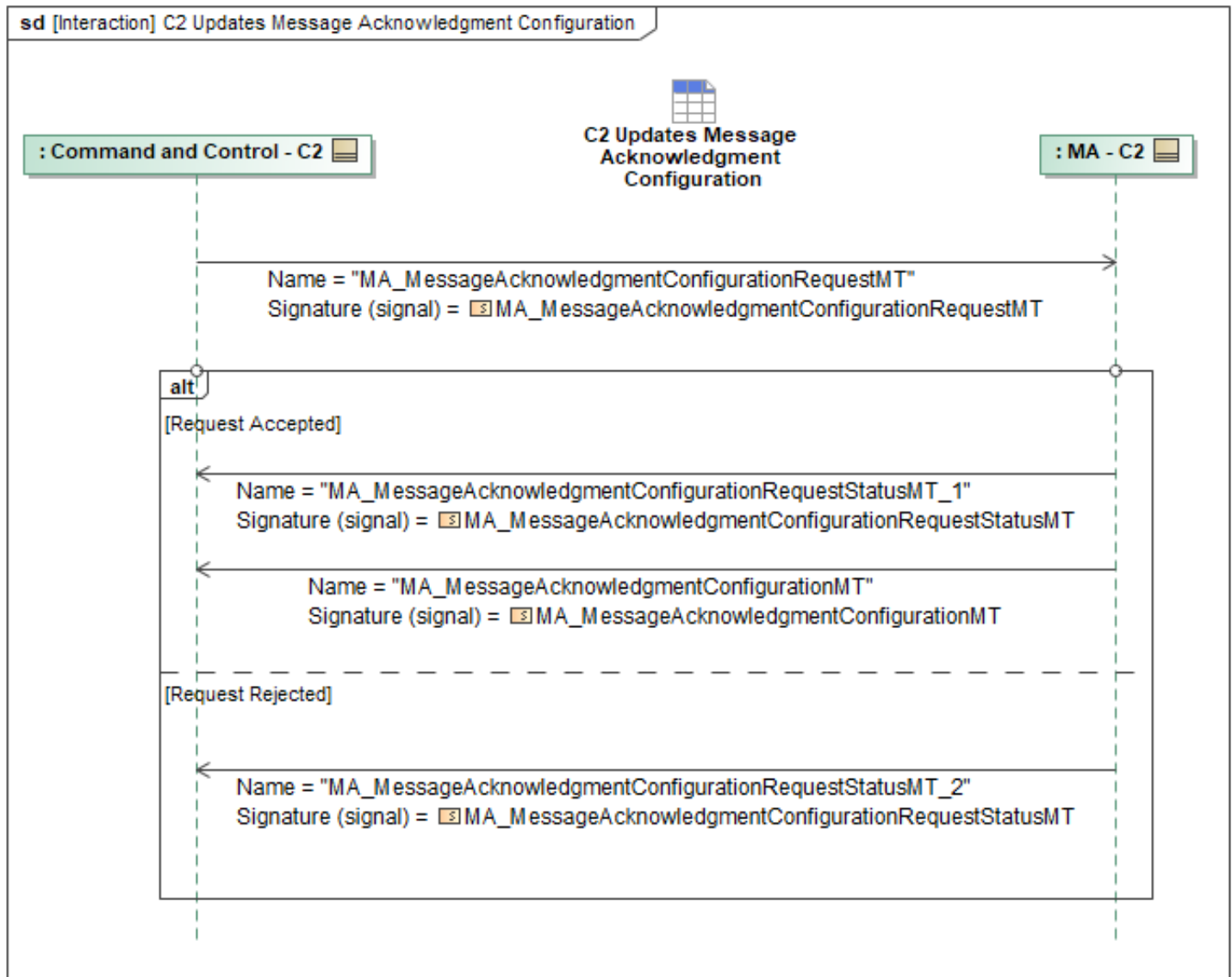


Figure 5-3: C2 Updates Message Acknowledgment Configuration

Table 5-2: C2 Updates Message Acknowledgment Configuration

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessageAcknowledgmentConfigurationMT: MA_MessageAcknowledgmentConfigurationMT	None
MA_MessageAcknowledgmentConfigurationReq uestMT: MA_MessageAcknowledgmentConfigurationReq uestMT	None
MA_MessageAcknowledgmentConfigurationReq uestStatusMT_1: MA_MessageAcknowledgmentConfigurationReq uestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessageAcknowledgmentConfigurationRequestStatusMT_2: MA_MessageAcknowledgmentConfigurationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.1.3 Message Acknowledgment Configuration Status

This interaction provides the current Message Acknowledgment Configuration defining what messages MA will respond to with an acknowledgment upon receipt. MA will report the current Message Acknowledgment Configuration on change by sending a *MA_MessageAcknowledgmentConfigurationMT* message.

The *DefaultGroupMessageFreq* field defines the frequency at which the Grouped Acknowledgment message is set. If the field is not set, there is no grouping. The *DefaultMessagesToAck* field sets the list of messages that an Acknowledgment message will be sent for when received. If the field is not set, all messages listed in *MA_AckMessageEnum* will receive a response. The *ReceiverSpecificSettings* field specifies how MA will respond to individual C2s and/or Peers if they do not fall under the default settings. If MA does not support Message Acknowledgment Configuration, the *ConfigurationSupported* field will be set to "FALSE" and it will respond with an ungrouped acknowledgment for all possible messages listed in *MA_AckMessageEnum*.

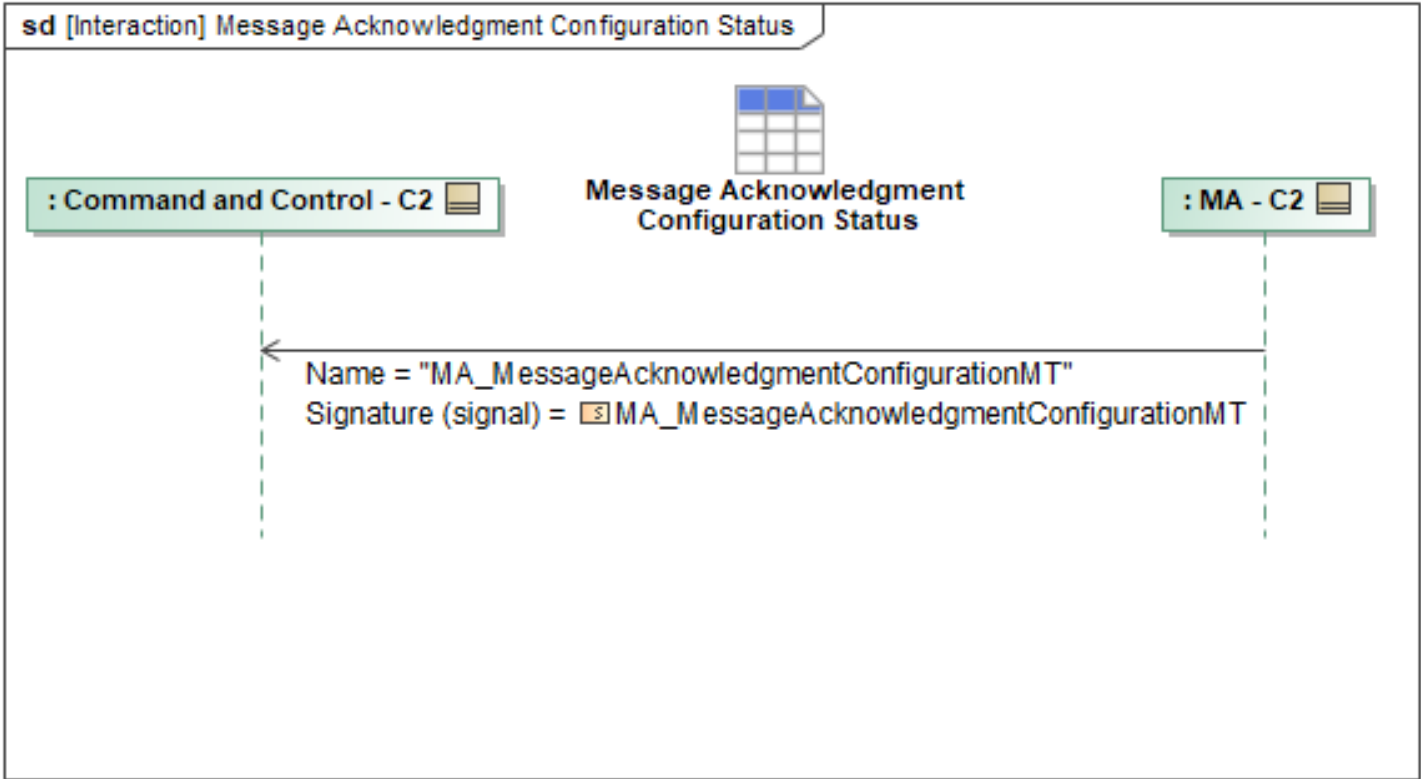


Figure 5-4: Message Acknowledgment Configuration Status

Table 5-3: Message Acknowledgment Configuration Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessageAcknowledgmentConfigurationMT: MA_MessageAcknowledgmentConfigurationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.1.4 Update Message Acknowledgment Configuration

This interaction specifies when MA wants to set or update the C2 Message Acknowledgment Configuration. MA can update default responses by filling out the *DefaultGroupMessageFreq* and *DefaultMessagesToAck*. MA can also use the *RequestedReceiverSpecificSetting* field to define the messages that MA expects C2 to respond with an acknowledgment for and if the messages will be grouped. MA should only request *RequestedReceiverSpecificSetting* for itself and not other systems.

MA sends the *MA_MessageAcknowledgmentConfigurationRequestMT* message to C2. C2 may accept or reject the configuration request. C2 accepts the request by sending a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_1* with the *RequestProcessingState* set to "COMPLETED", followed by a *MA_MessageAcknowledgmentConfigurationMT* that reflects the updated configuration.

C2 may reject the configuration request for the following reasons:

- It does not support configuration. C2 sends a

MA_MessageAcknowledgmentConfigurationRequestStatusMT_2 message and sets the *RequestProcessingStateReason* to "SYSTEM_UNAVAILABLE."

- There is limited bandwidth. C2 sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message, sets the *RequestProcessingStateReason* to "INSUFFICIENT_RESOURCES", and fills out the description field.
- The system receives an unsupported input. C2 sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "INVALID_INPUT_PARAMETER."
- MA requests *RequestedReceiverSpecificSetting* for another system. C2 sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "INELIGIBLE_CONTROL_SOURCE."

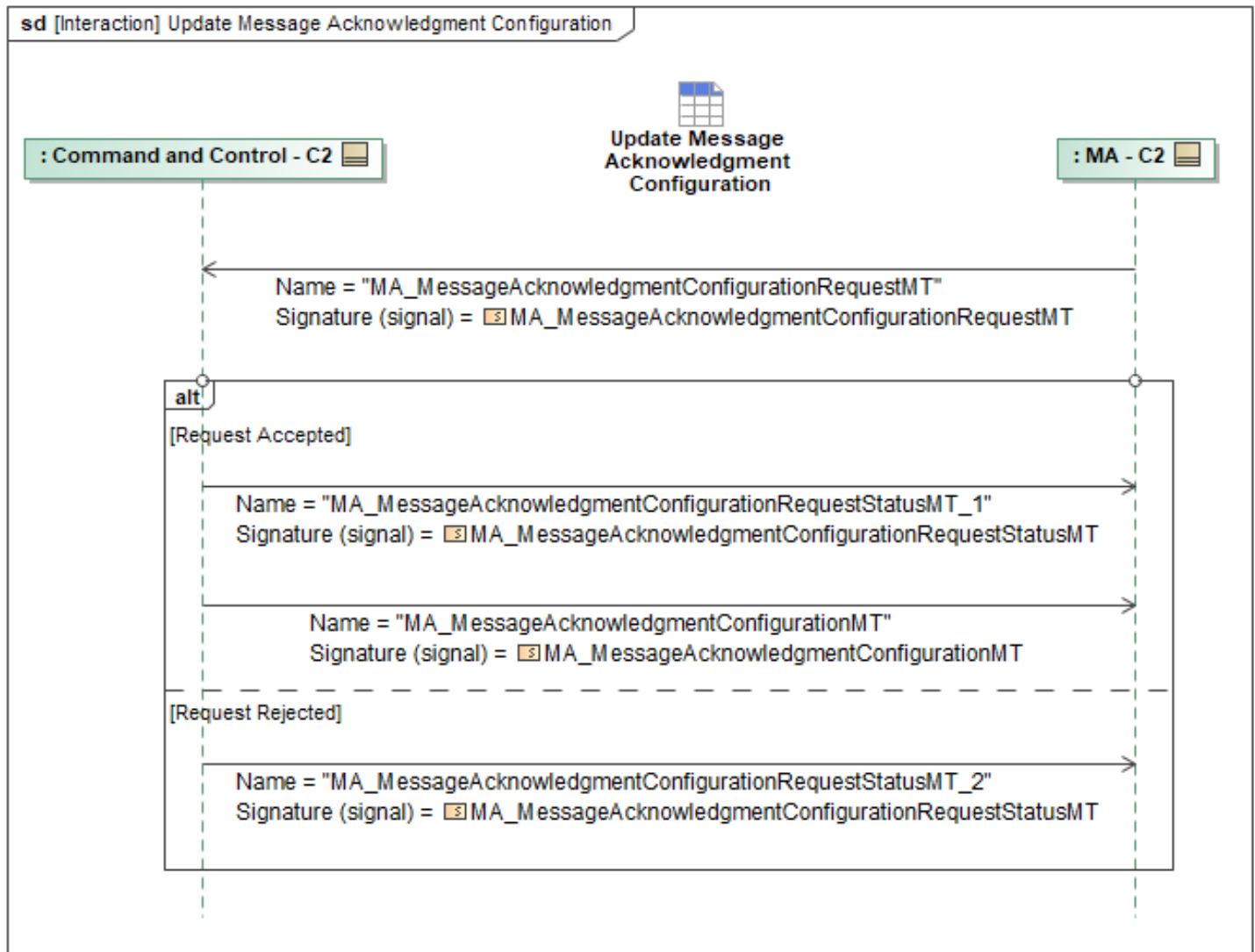


Figure 5-5: Update Message Acknowledgment Configuration

Table 5-4: Update Message Acknowledgment Configuration

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessageAcknowledgmentConfigurationMT: MA_MessageAcknowledgmentConfigurationMT	None
MA_MessageAcknowledgmentConfigurationRequestMT: MA_MessageAcknowledgmentConfigurationRequestMT	None
MA_MessageAcknowledgmentConfigurationRequestStatusMT_1: MA_MessageAcknowledgmentConfigurationRequestStatusMT	None
MA_MessageAcknowledgmentConfigurationRequestStatusMT_2: MA_MessageAcknowledgmentConfigurationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.2 C2 CAP Behaviors Package

5.2.2.1 Share CAP Execution Status

This interaction specifies the CAP Execution Status message which is sent from MA to C2. This message is expected to be sent periodically to C2 when one or multiple ACPs are executing a CAP (commanded by either an Action or a Task). When multiple ACPs are executing a CAP, the ACPs are assumed to be within a Package. A single ACP performing CAP may be within a Package but is not required to be within a Package for a single-ship mission.

If the ACP is the Package Leader, then the Leader MA sends *MA_CAP_ExecutionStatusReportMT* periodically to C2 after receiving an individual execution status report from all Followers (See "Receive CAP Execution Status"). *MA_CAP_ExecutionStatusReportMT* defines fields to capture both package-level and individual ACP data. At the package-level, the *PackageID* shall be specified, and the *PackageRequirementID* field shall include the unique ID of the package-level CAP Task (if the Package is commanded via a Task). If the CAP was commanded as a result of an Action, the *PackageRequirementID* field shall include the unique ID of the package-level CAP Action. Additionally, the Leader MA should include the *AOI_Coverage* and *PlanMetrics* fields when reporting CAP Execution Status to C2. The leader will compute these fields prior to this interaction. The *PlanMetrics* are computed by the leader when computing the CAP routes, and the *AOI_Coverage* is computed by the leader upon receiving threat zone information and Field of Regard reports from package members. The *ExecutionStatus* field shall contain the status of each individual ACP executing CAP within the package. The *SystemIDs* shall be set to the IDs of the ACPs, *RequirementIDs* shall be set to the IDs of the vehicle-level CAP Tasks, and the *CAP_Status* object shall be defined. Included in the *CAP_Status* object is the required *CAP_State* field and optional status fields.

If the ACP is executing a CAP for a single-ship mission (ACP is not part of a Package), MA will send *MA_CAP_ExecutionStatusReportMT* periodically to C2 at a constant rate in addition to anytime its *CAP_State* is updated. At the package-level, no fields are required. At the vehicle level, the *ExecutionStatus* shall be defined for just the single ACP where the *SystemID*, *RequirementID*, and *CAP_Status* object are required.

MA will send a *TaskStatusMT* message to C2 as part of the COP and therefore is not specified as part of this interaction. If the ACP is the package leader, it will send a *TaskStatusMT* message for each vehicle performing a CAP. Within the *TaskStatusMT* message, the *TimeWindow* field may be defined to specify the remaining time-on-station for performing the CAP Task.

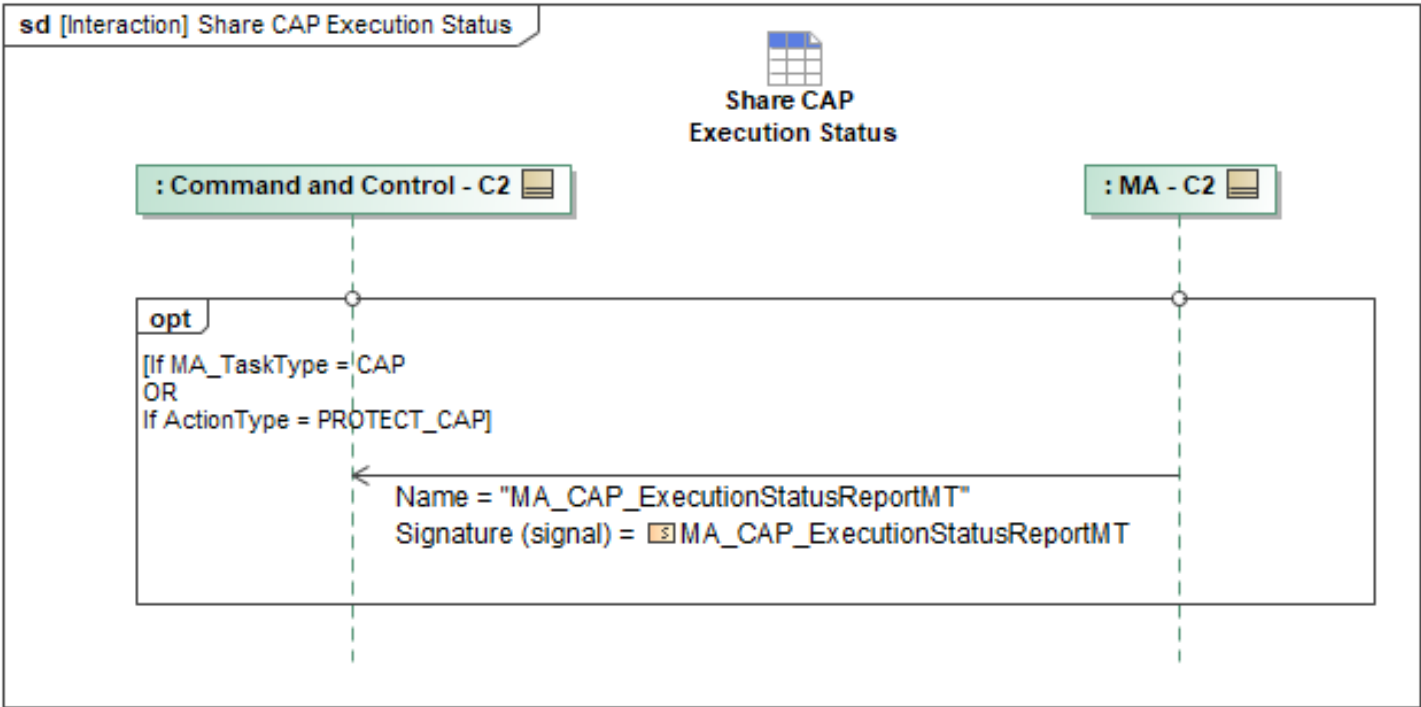


Figure 5-6: Share CAP Execution Status

Table 5-5: Share CAP Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CAP_ExecutionStatusReportMT: MA_CAP_ExecutionStatusReportMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.3 C2 Communications Package

5.2.3.1 C2 Overrides Data Dissemination Plan

Mission Use Case: Additional Comms Capabilities

In some situations a modification to the Data Dissemination Plan is required that are short term in nature and a complete update to the *MA_DataDisseminationPlanMT* message is not required. The *MA_DataDisseminationPlanOverrideRequestMT* can be used for these sorts of modifications. C2 can send the *MA_DataDisseminationPlanOverrideRequestMT* message to MA which forwards it to MS. MS can then accept or reject the request(s). If the request(s) are accepted, MS sends the *MA_DataDisseminationPlanOverrideRequestStatusMT_1* message to MA who forwards it to C2 with the Request(s) having the *Request.RequestProcessingState* set to "QUEUED", "PROCESSING", "CANCELED", or "COMPLETED" as needed. Depending on the original request, the *EffectivityActivationStatus* or *OperatorModificationStatus* fields will be filled out as needed. The interaction "MA Updates Data Dissemination Plan Activation Status" is initiated which will provide an update to the active status and include the *OverrideModificationState* associated with the override request status. For each applicable Gateway Activity, MS sends the *MA_GatewayActivityMT* message(s) to MA who forwards it to C2 which will include any modifications that were made. If the

request is NOT accepted, MS sends the *MA_DataDisseminationPlanOverrideRequestStatusMT_2* message to MA who forwards it to C2 with the *RequestProcessingState* set to "REJECTED" or "FAILED", and the *RequestProcessingStateReason* filled out as needed. The *EffectivityActivationStatus* or *OperatorModificationStatus* fields will be filled out outlining the reason for the rejections/failures.

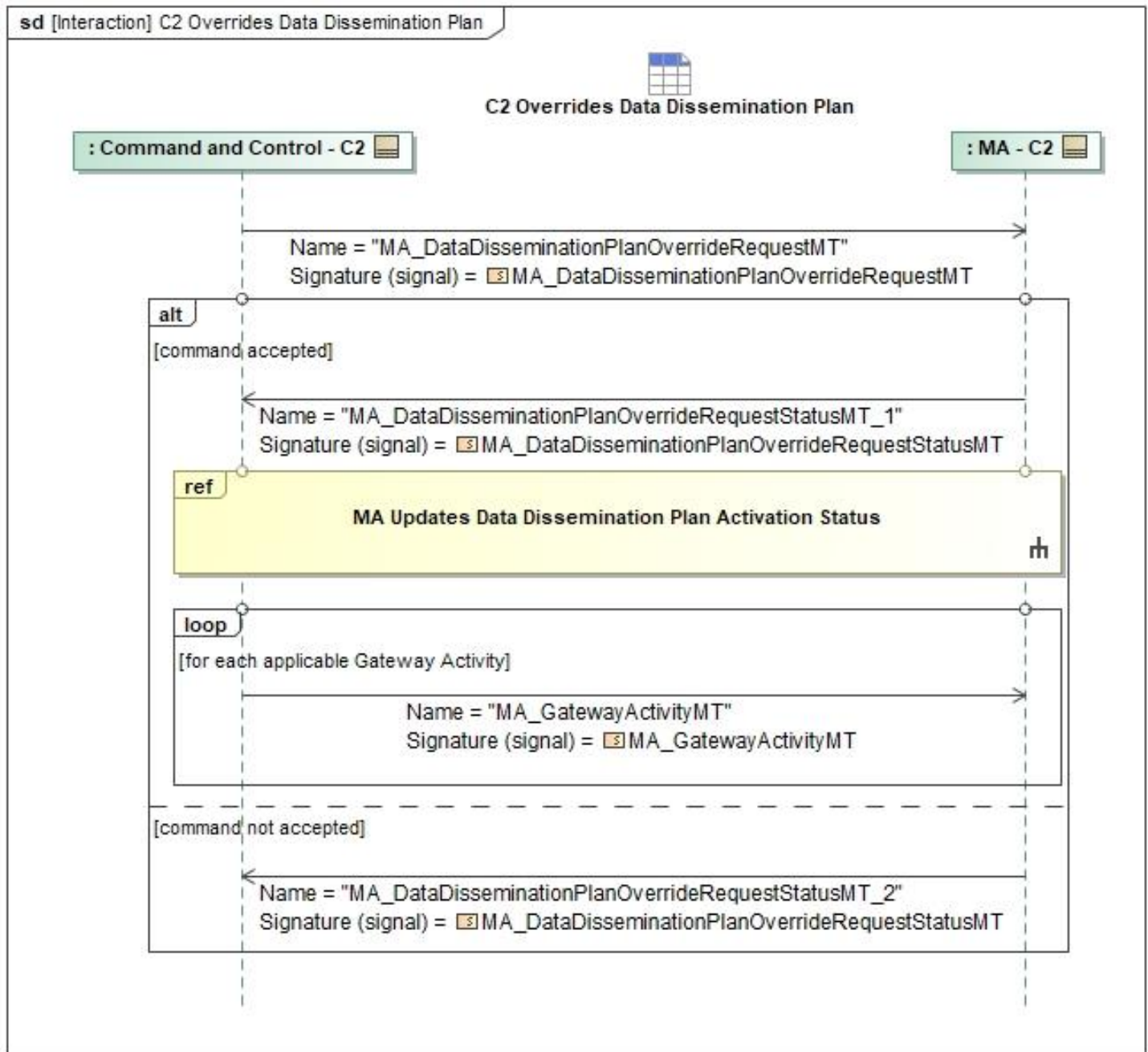


Figure 5-7: C2 Overrides Data Dissemination Plan

Table 5-6: C2 Overrides Data Dissemination Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanOverrideRequestMT: MA_DataDisseminationPlanOverrideRequestMT	AllocatedToSystemIDs: MA_PackageSystemType
MA_DataDisseminationPlanOverrideRequestStatusMT_1: MA_DataDisseminationPlanOverrideRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanOverrideRequestStatusMT_2: MA_DataDisseminationPlanOverrideRequestStatusMT	RequestProcessingStateReason: CannotComplyType
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.3.2 C2 Updates Data Dissemination Plan

Mission Use Case: Additional Comms Capabilities

The *MA_DataDisseminationPlanMT* is used to configure how messages ingress and egress over the air. MS manages/implements the Data Dissemination Plan and Mission Autonomy acts as a pass through if C2 needs to update and/or activate the plan. The MS section "Update Data Dissemination Plan" details the contents of the *MA_DataDisseminationPlanMT* message and how it can be configured.

If the Data Dissemination Plan needs update or is a new plan, C2 sends the *MA_DataDisseminationPlanMT* message and the *MA_DataDisseminationPlanActivationCommandMT* message to MA who forwards them to MS. If the command is accepted by MS, MS sends the *MA_DataDisseminationPlanActivationCommandStatusMT_1* message to MA who forwards it to C2 with the *CommandProcessingStateEnum* set to "ACCEPTED", the interaction "MA Updates Data Dissemination Plan Activation Status" is initiated, and for each applicable Gateway Activity MS sends the *MA_GatewayActivityMT* message(s) to MA who forwards them to C2. If the command is NOT accepted, MS send the *MA_DataDisseminationPlanActivationCommandStatusMT_2* message to MA who forwards it to C2 with the *CommandProcessingStateEnum* set to "REJECTED" and the *PlanActivationCommandStatus* is set according to the reason it was rejected.

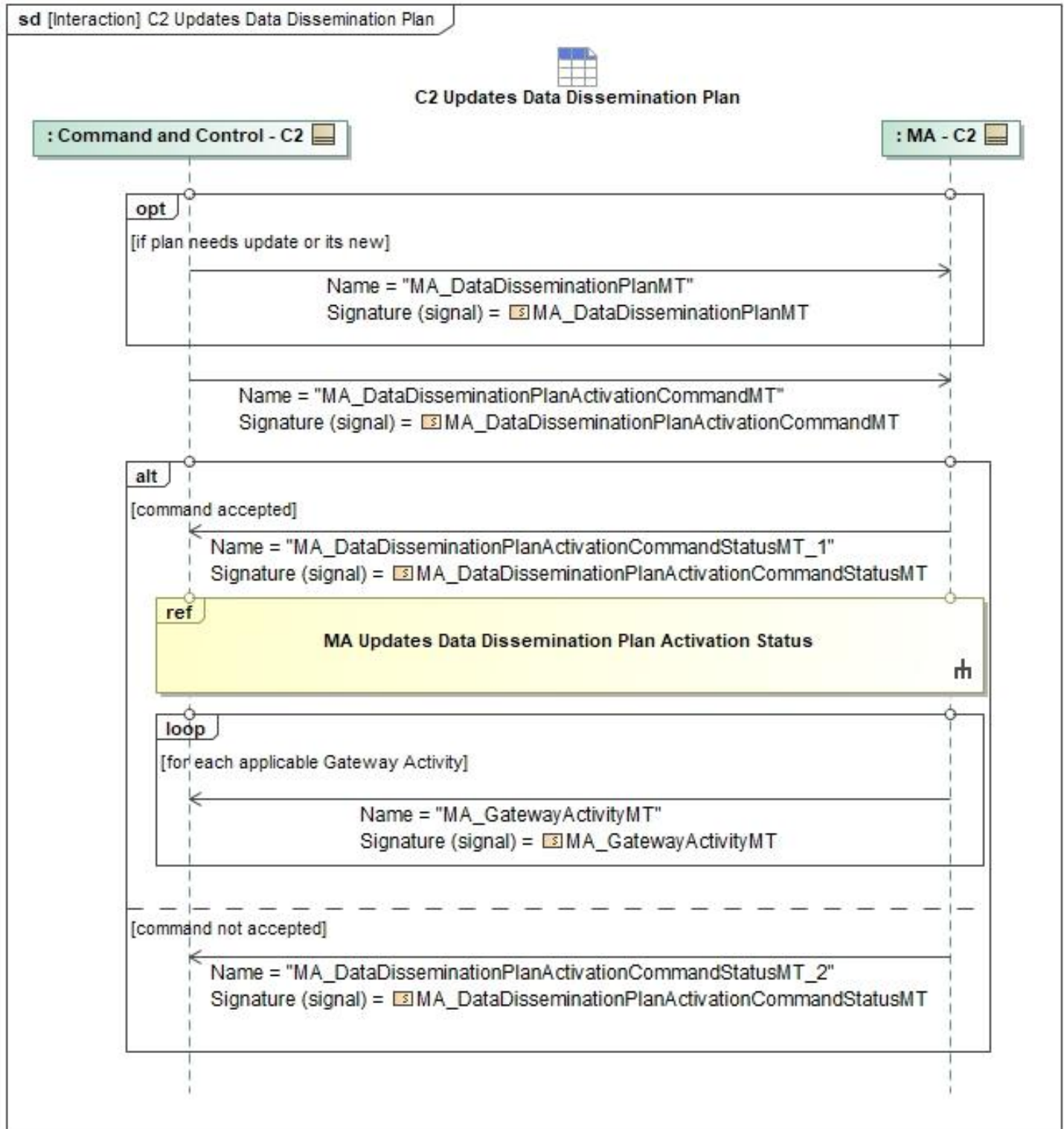


Figure 5-8: C2 Updates Data Dissemination Plan

Table 5-7: C2 Updates Data Dissemination Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanActivationCommandMT: MA_DataDisseminationPlanActivationCommandMT	Applicability: MA_PackageSystemType
MA_DataDisseminationPlanActivationCommandStatusMT_1: MA_DataDisseminationPlanActivationCommandStatusMT	None
MA_DataDisseminationPlanActivationCommandStatusMT_2: MA_DataDisseminationPlanActivationCommandStatusMT	RequestProcessingStateReason: CannotComplyType
MA_DataDisseminationPlanMT: MA_DataDisseminationPlanMT	None
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.3.3 MA Updates Comms Gateway Status

Mission Use Case: Additional Comms Capabilities

The Gateway Capability in MS describes how messages egress or ingress over the air. To provide information about each of the active gateways available for use, MA receives and forwards the *GatewayCapabilityMT* and *GatewayCapabilityStatusMT* messages from MS to C2. Then, for each active Gateway Activity, MA also forwards the *MA_GatewayActivityMT* message(s) to C2. These messages provide the information needed to understand the state of the over the air messaging, they will provide context for how things are currently configured, and provide information for how to configure over the air messaging. MS will typically configure the gateways internally, but the information can be useful when creating the plans that MS uses.

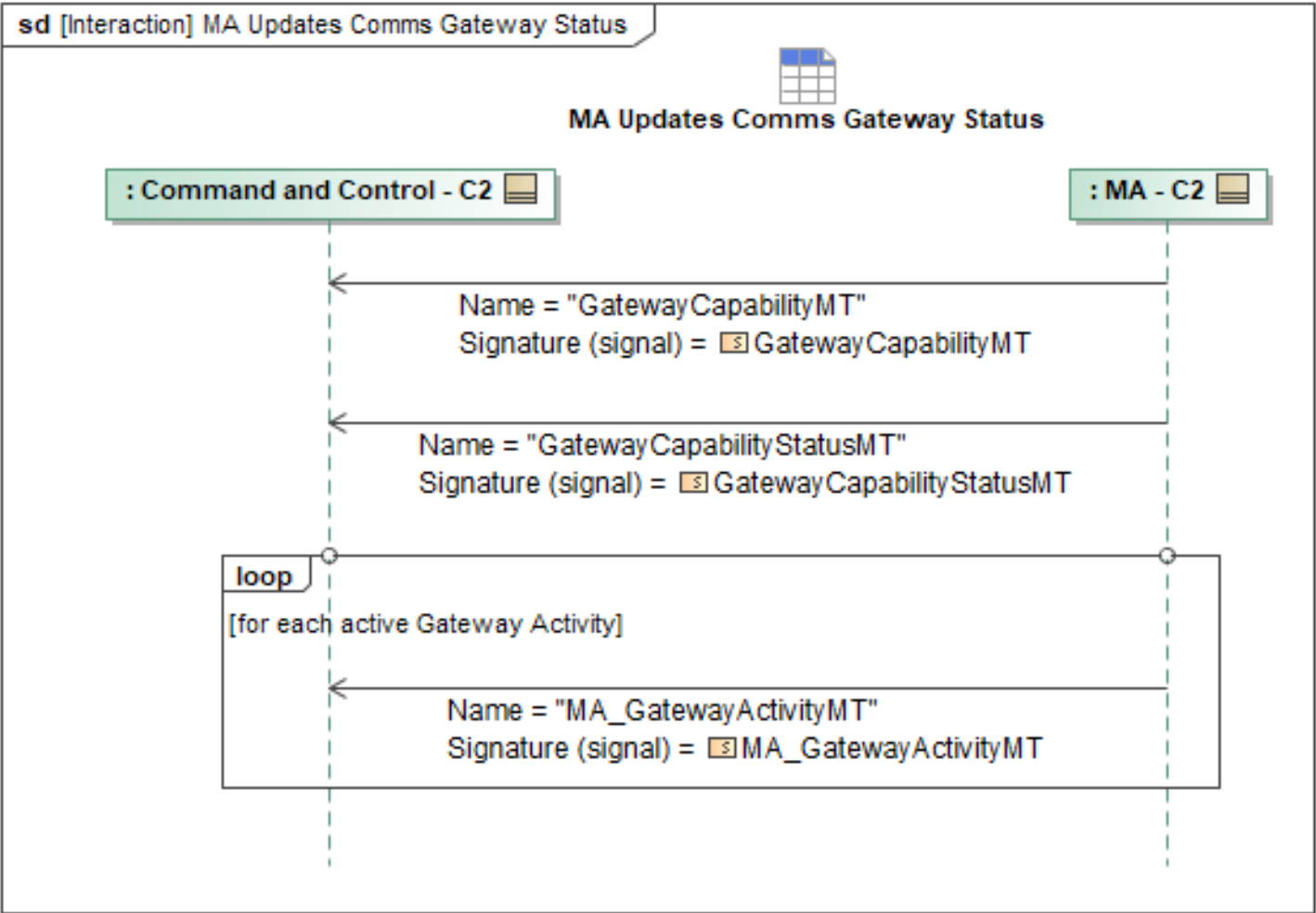


Figure 5-9: MA Updates Comms Gateway Status

Table 5-8: MA Updates Comms Gateway Status

Message Name (Name:Type)	Required Fields (Name:Type)
GatewayCapabilityMT: GatewayCapabilityMT	None
GatewayCapabilityStatusMT: GatewayCapabilityStatusMT	None
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.3.4 MA Updates Data Dissemination Plan Activation Status

Mission Use Case: Additional Comms Capabilities

The MS will send the *MA_DataDisseminationPlanActivationStatusMT* message to MA who forwards it to C2 that will reflect the active state of Data Dissemination Plan. This can be sent periodically, but will always be sent upon change to the active state. The message will reflect the active Effectivities

and Override Modifications of the Data Dissemination Plan.

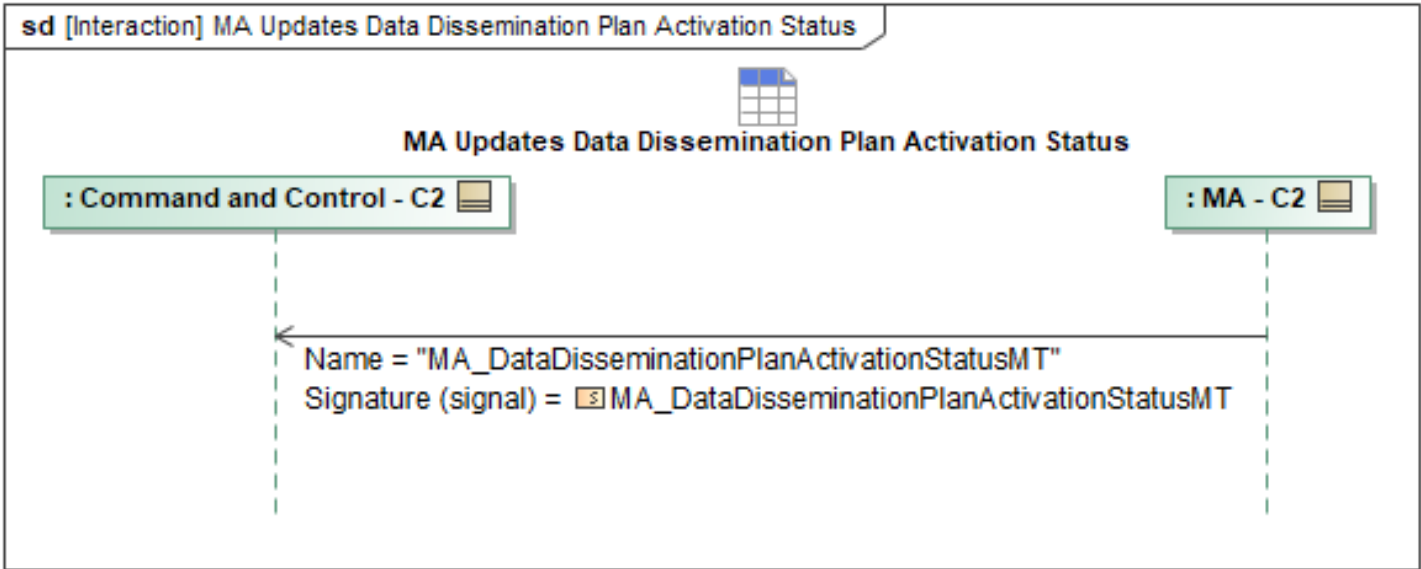


Figure 5-10: MA Updates Data Dissemination Plan Activation Status

Table 5-9: MA Updates Data Dissemination Plan Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanActivationStatusMT: MA_DataDisseminationPlanActivationStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4 C2 Contingency Behaviors Package

C2 systems have the ability to send contingency plans to MA in the form of a Response. Responses are sets of “Trigger - Response” pairs generated by C2 and sent to MA. The Trigger can be any sort of message generated internally by MA or another onboard system, or it can be sent from C2 to command the desired Response. The Response can generate new requirements/tasks, activate a previously developed Mission Plan or part of a Mission Plan, generate a contingency alert, or do nothing at all.

In the event of a contingency, MA can be configured to react in one of three different ways: alert generation only (all will generate an alert), activate a specific mission plan, or activate autonomous replanning. A C2 system will use the *PlanningFunction** family of UCI messages along with Response Requirements to achieve these three contingency reactions. The sections below break down these use cases further. The first defines sequences for setting up the three autonomous reactions, the second defines the sequences for the message flow when a trigger is tripped, and the third defines how to command these responses directly.

In order to set up a Trigger-Response pair, a C2 system will send a *PlanningFunctionSettingsCommandMT* message to define a trigger for MA to monitor. This message also specifies what sort of action MA should take once its trigger is tripped. If a C2 System wishes to specify an existing plan to execute as a reaction to a contingency, it will also need to construct a

Response Requirement containing the desired plan. The specifics of these messages and fields are discussed in the following sections.

MA alerts the C2 systems when a contingency is triggered and notifies it of the status of the contingency response. The corresponding sequence to create a contingency trigger and optionally define a response action must have been completed by either C2 or during Mission Planning before these triggered contingency sequences can occur.

For lost communication contingencies, MA assumes that the loss is reported by the Mission Systems Communications Manager or is determined by the absence of a heartbeat message past a configurable timeout interval. During planned communication outages, MA depends on MS for lost comms notification.

5.2.4.1 C2 Activate a Contingency

C2 sends a *MA_ResponseCommandMT* to activate or deactivate a previously uploaded *MA_ResponseMT*. The *MA_ResponseCommandMT* will set the *ConstraintOverride.Allocation.RequiredAllocation.Allocation* field to indicate the target *SystemIDs* or *PackageID* to activate the *MA_ResponseMT*. Optionally, the *AssociatedPlan* field can be used to link back to a plan which contains the *MA_ResponseMT*. MA responds with a *MA_ResponseCommandStatusMT* to indicate if the command was successful or not.

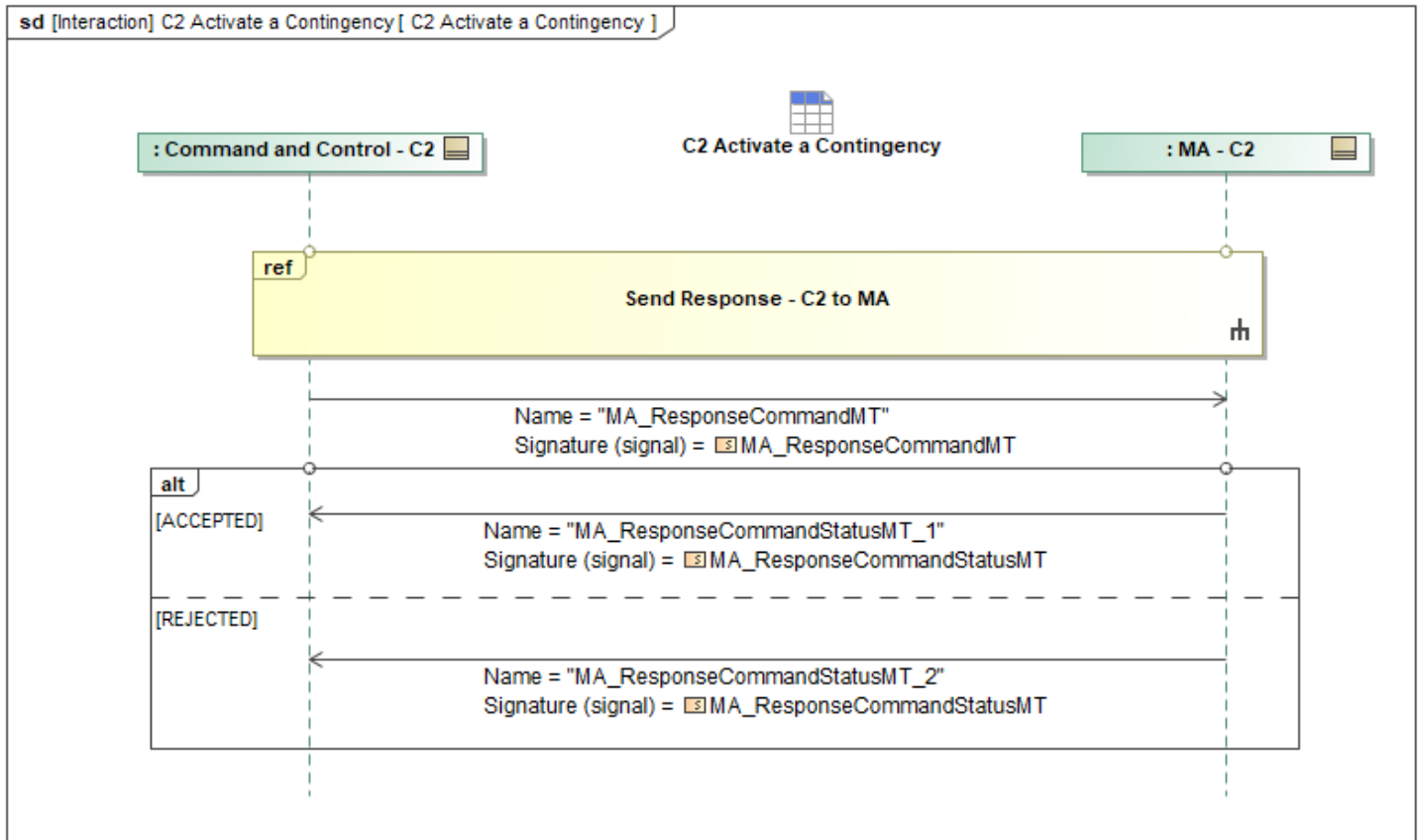


Figure 5-11: C2 Activate a Contingency

Table 5-10: C2 Activate a Contingency

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseCommandMT: MA_ResponseCommandMT	None
MA_ResponseCommandStatusMT_1: MA_ResponseCommandStatusMT	None
MA_ResponseCommandStatusMT_2: MA_ResponseCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.2 C2 Incorrect Messaging Sequence Error Handling

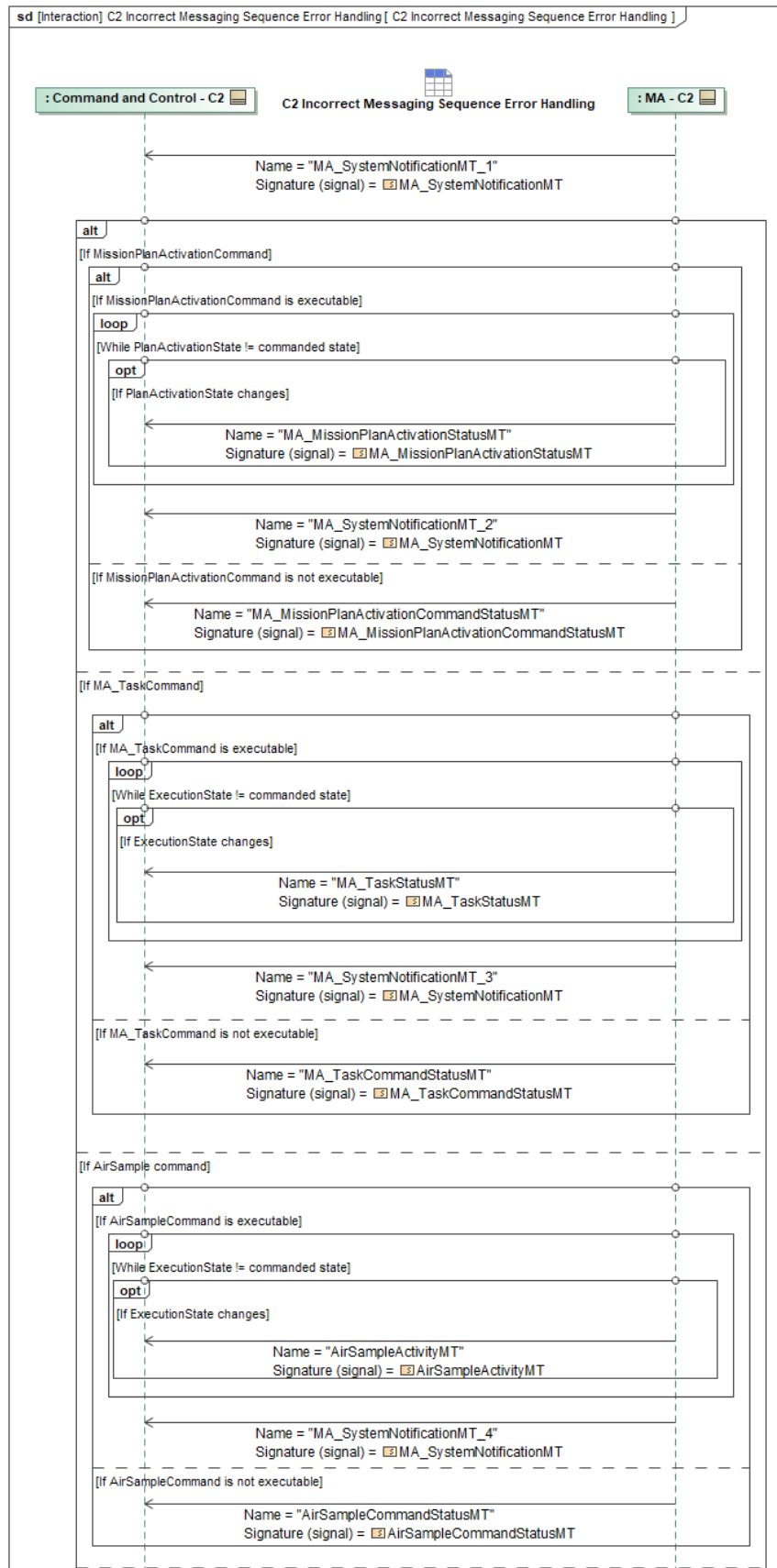
When MA receives a command, it is possible that the commanded state of an object (e.g., *MissionPlan*, *Task*, *EA_Capability*) is more than one state away from the object's current state. If this is the case, an Incorrect Messaging Sequence error is triggered. Upon identification of an Incorrect Messaging Sequence error, MA will send *MA_SystemNotificationMT* to C2 to indicate that the commanded state is out of sequence and that corrective state transitions will be performed. Within the message, MA sets *NotificationState* to *ACTIVE* and sets *NotificationDetail.Reason* to *STATE_OR_SETTINGS*. *NotificationState* communicates that the error exists and *NotificationDetail.Reason* indicates that the error exists due to the object's current state.

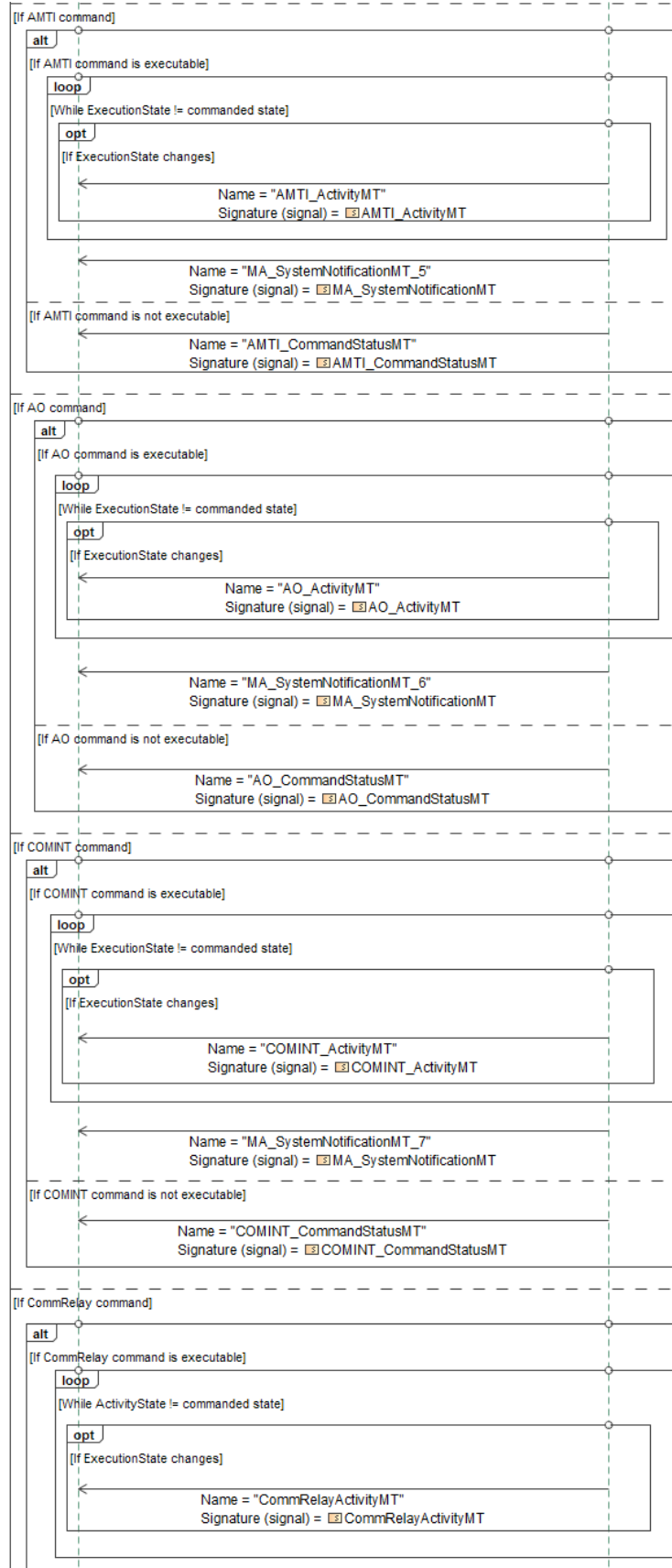
AssociatedMessage.MessageType identifies the command associated with the error, and *AssociatedMessage.AssociatedID* will reference the ID of the associated command.

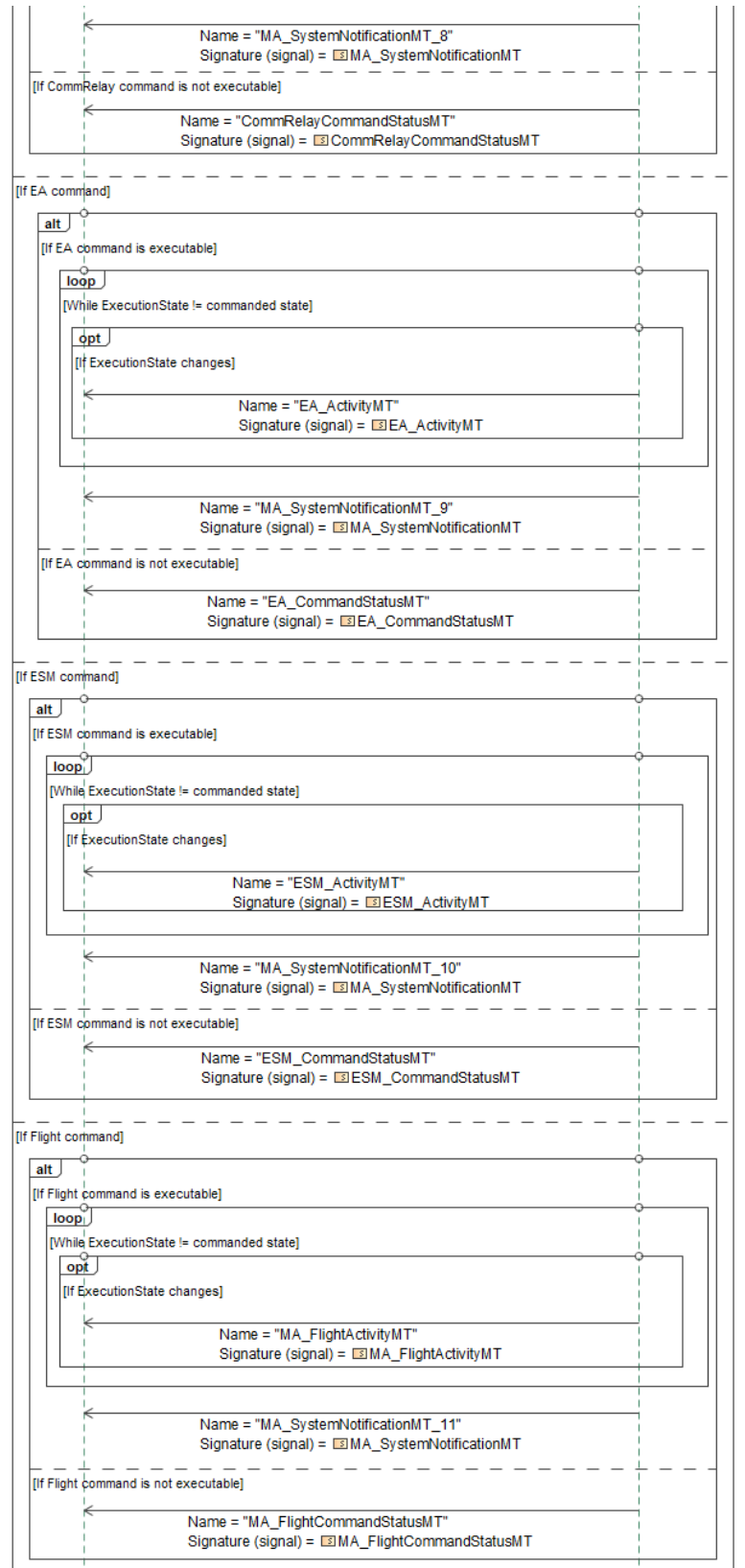
If MA determines that the command cannot be carried out for any reason, MA will publish the corresponding **CommandStatus* message with *CommandProcessingState* set to *REJECTED*. If MA determines that the command can be carried out, MA will publish the corresponding **CommandStatus* message with *CommandProcessingState* set to *ACCEPTED*, and will progress the object's status towards the commanded state.

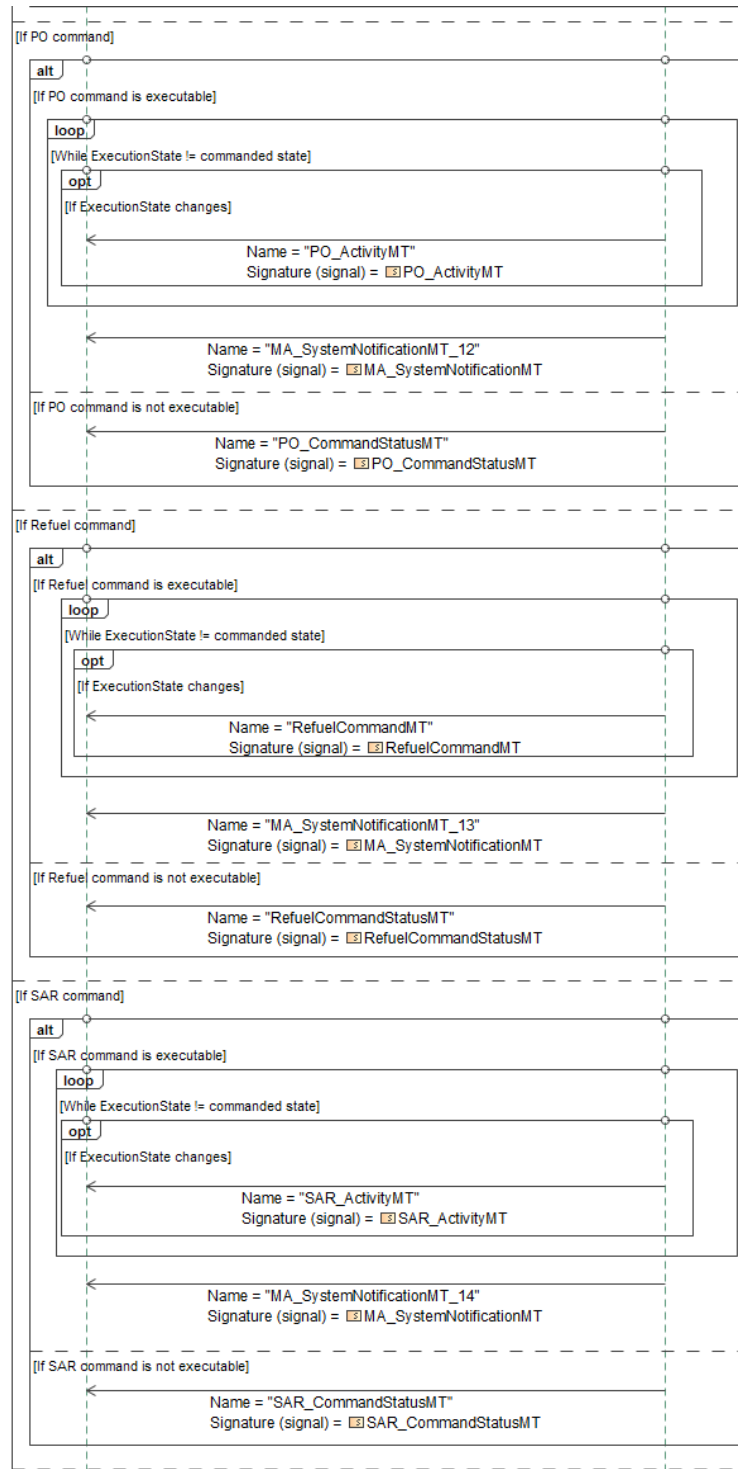
MA communicates the object's status when the object's status changes during corrective actions. Status updates are not intended to be periodic; they are sent when MA transitions the object to a different state while progressing toward the commanded state. Object status reflects the object's actual state at the time the status is reported and provides visibility into the intermediate transitions performed by MA during resolution of the Incorrect Messaging Sequence error. The applicable status message depends on the affected object. For mission plan activation commands, MA uses *MA_MissionPlanActivationStatus* to communicate the current *PlanActivationState* (e.g., *READY_FOR_ACTIVATION*, *ACTIVATING*, *ACTIVATED*, *ACTIVE_DEGRADED*, *DEACTIVATING*, and *DEACTIVATED*). For a TaskCommand, MA uses *MA_TaskStatusMT* to communicate the current *ExecutionState* (e.g. *AWAITING_PLAN_ACTIVATION*, *EXECUTING*, *COMPLETED*, *DROPPED*, *FAILED*, *CANCELED*). For UCI capability commands, MA uses the capability's corresponding **Activity* message (e.g., *EA_Activity*, *AMTI_Activity*, *AO_Activity*) to communicate the current *ActivityState* (e.g., *ENABLED*, *ACTIVE_PARTIALLY_CONSTRAINED*, *ACTIVE_UNCONSTRAINED*, *COMPLETED*, *FAILED*).

The commanded state represents the state requested by the originating command and serves as the target state against which MA evaluates progress during executing corrective actions. When the current object state matches the commanded state, MA considers the Incorrect Messaging Sequence Error to be resolved and will send updated *MA_SystemNotificationMT* with *NotificationState* set to *CLOSED*.









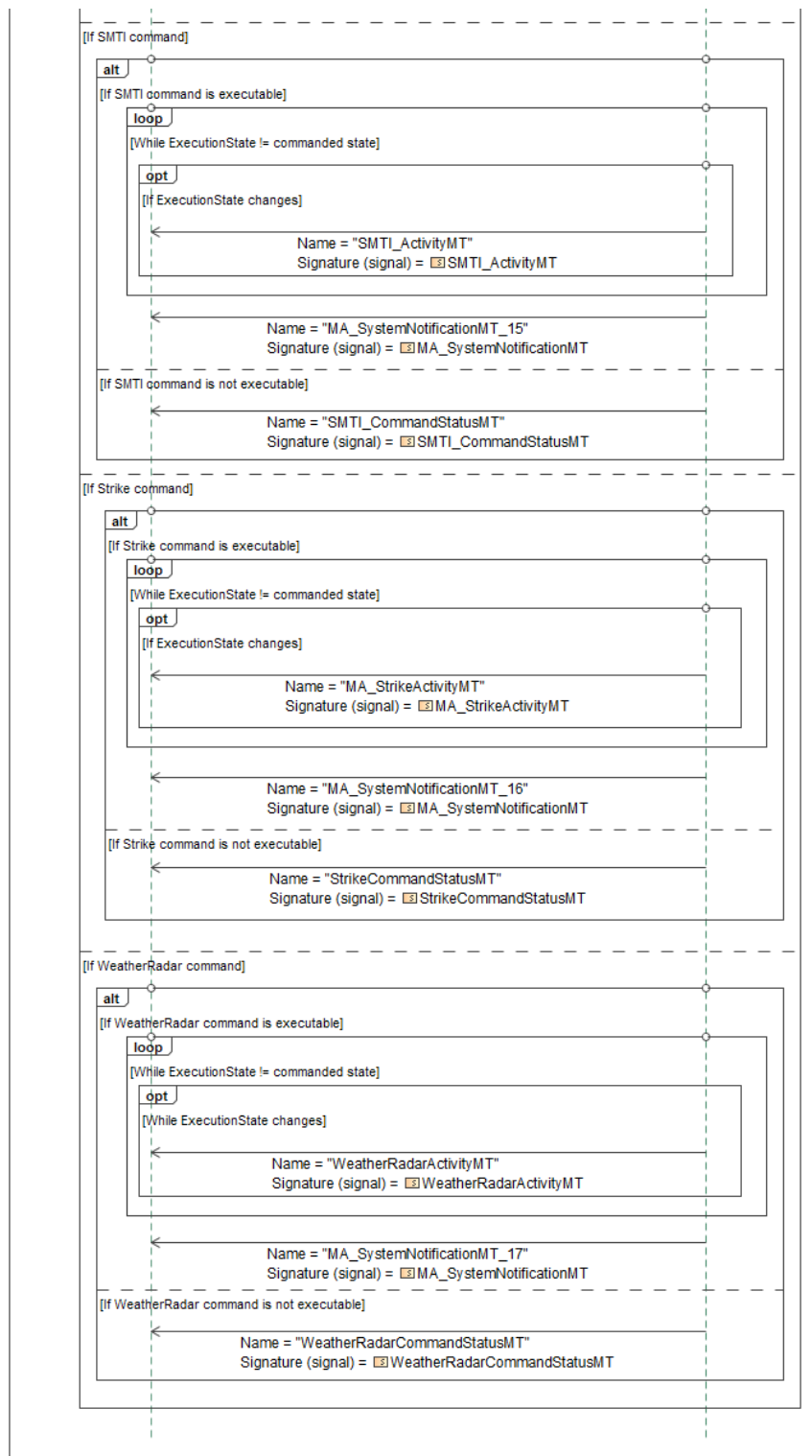


Figure 5-12: C2 Incorrect Messaging Sequence Error Handling

Table 5-11: C2 Incorrect Messaging Sequence Error Handling

Message Name (Name:Type)	Required Fields (Name:Type)
AirSampleActivityMT: AirSampleActivityMT	None
AirSampleCommandStatusMT: AirSampleCommandStatusMT	None
AMTI_ActivityMT: AMTI_ActivityMT	None
AMTI_CommandStatusMT: AMTI_CommandStatusMT	None
AO_ActivityMT: AO_ActivityMT	None
AO_CommandStatusMT: AO_CommandStatusMT	None
COMINT_ActivityMT: COMINT_ActivityMT	None
COMINT_CommandStatusMT: COMINT_CommandStatusMT	None
CommRelayActivityMT: CommRelayActivityMT	None
CommRelayCommandStatusMT: CommRelayCommandStatusMT	None
EA_ActivityMT: EA_ActivityMT	None
EA_CommandStatusMT: EA_CommandStatusMT	None
ESM_ActivityMT: ESM_ActivityMT	None
ESM_CommandStatusMT: ESM_CommandStatusMT	None
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_MissionPlanActivationCommandStatusMT: MA_MissionPlanActivationCommandStatusMT	None
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	None
MA_StrikeActivityMT: MA_StrikeActivityMT	None
MA_SystemNotificationMT_1: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_2: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_3: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_4: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_5: MA_SystemNotificationMT	AssociatedID: ID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT_6: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_7: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_8: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_9: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_10: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_11: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_12: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_13: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_14: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_15: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_16: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_17: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None
PO_ActivityMT: PO_ActivityMT	None
PO_CommandStatusMT: PO_CommandStatusMT	None
RefuelCommandMT: RefuelCommandMT	None
RefuelCommandStatusMT: RefuelCommandStatusMT	None
SAR_ActivityMT: SAR_ActivityMT	None
SAR_CommandStatusMT: SAR_CommandStatusMT	None
SMTI_ActivityMT: SMTI_ActivityMT	None
SMTI_CommandStatusMT: SMTI_CommandStatusMT	None
StrikeCommandStatusMT: StrikeCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
WeatherRadarActivityMT: WeatherRadarActivityMT	None
WeatherRadarCommandStatusMT: WeatherRadarCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.3 C2 Lost Communication

A C2 system can configure MA to generate a *MissionContingencyAlertMT* and activate an existing plan when losing communications with C2. This sequence will follow the sequence laid out in section *Define Contingency Plan*, with some specifics. To define the contingency settings, C2 System will send a *MA_PlanningFunctionSettingsCommandMT* to MA with the *SystemID* field set to the unique ID of the system whose planning functions settings are being commanded or changed, the *MissionPlanningAutonomy.Trigger.CommsLost* set with the *MissionPlanningAutonomy.Trigger.CommsLost.Threshold* set to the amount of time in seconds to wait with no communications before triggering the lost communications contingency, and the *MissionPlanningAutonomy.AutonomousPlanningResponseChoice.AlertOnly* field set to "FALSE" indicating that MA should not initiate autonomous replanning. When MA receives a *MA_PlanningFunctionSettingsCommandMT*, if C2 is authorized it will reply with a *MA_PlanningFunctionSettingsCommandStatusMT* with the *CommandProcessingState* set to either "ACCEPTED" if the new autonomy settings were accepted or "REJECTED" if the autonomy settings were not able to be applied. The C2 system will send the MA a *MA_ResponseMT* with the *ResponseType* field set to "PLAN_ACTIVATION", the *Option.Trigger.AnyMessage.MessageType* field set to "MISSION_CONTINGENCY_ALERT", and the *Option.Response.ActivatePlan.MissionPlanID* field set to the desired *MissionPlanID* in order to configure the plan that should be activated in the event of lost communication with C2. C2 Systems can further specify what should be activated by using *Option.Response.ActivatePlan.ActivationDetails.ByMissionPlan*, *Option.Response.ActivatePlan.ActivationDetails.BySubPlan*, and *Option.Response.ActivatePlan.ActivationDetails.ByExecutionPlanSet*. *ByMissionPlan* indicates that the response should activate all sub-*Plans included in the mission plan. The *BySubPlan* field offers the ability to select specific sub-*Plan to activate, which is recommended for this specific contingency scenario. The *ByExecutionPlanSet* field offers the ability to select a specific *ExecutionPlanSet* to activate, which is recommended when using a mission plan operating with an *ExecutionSequence*.

Following the *MA_ResponseMT* messages, C2 systems will send a *ResponsePlanCommandMT* message with the *CommandState* set to "NEW" or "UPDATE" to tell Mission Autonomy to generate or edit a *ResponsePlanMT* based on the *MA_ResponseMTs* that were sent. A *CommandState* of "NEW" indicates that the Response is new and doesn't currently exist. *CommandState* of "UPDATE" indicates that C2 systems wish to perform an update to an existing *ResponsePlanMT*. Once a Plan has been successfully generated, Mission Autonomy will send out a second *ResponsePlanCommandStatusMT* with *PlanningStatus.CommandProcessingState* to "ACCEPTED", *PlanningStatus.CommandStatus* to "COMPLETED" and the *ResultingPlanID* field filled out with ID of the new Response Plan. Mission Autonomy will also send a *ResponsePlanMT* containing the newly generated Response Plan to C2 systems with corresponding *PlanCommandID*, specifying the

originating *ResponsePlanCommandID*, and *Plan:AllocatedResponse* to indicate that a response should be assigned to the system that the plan is allocated to.

Mission Autonomy will then request the C2 systems to approve the Response Plan via a *MA_ApprovalRequestMT* message with the *RequirementID* of the Response Plan set in *ApprovalReferences.ApprovalItem.PlanApproval*. The C2 system will send back an *MA_ApprovalRequestStatusMT* message with *OperatorID* populated with the approving operator's UUID, and *ApprovalRequestProcessingState* set as "APPROVED" if the Response Plan is deemed acceptable or "REJECTED" if the Response Plan is not acceptable. If the Response Plan is rejected, Mission Autonomy will initiate replanning and propose a new Response Plan to C2. If the Response Plan is accepted, the plan is ready to be executed. MA sends *ResponsePlanApprovalStatusMT* to verify that the Response Plan was successfully approved. To begin execution of the new Response Plan, C2 systems will send a *MA_MissionPlanActivationCommandMT* message with *Command.MissionPlanID* set to the ID of the approved Response Plan and the *ActivationDetails.BySubPlan:ResponsePlan* populated with the applicable Response Plan. MA will respond with a *MA_MissionPlanActivationCommandStatusMT* notifying C2 of the command's status. Once activated, response monitoring will begin and send *ResponseActivityMT_1* message to the C2 system. An *ActivityStatus* of "ENABLED" means the trigger is currently being monitored. An *ActivityStatus* of "ACTIVE" indicates the response trigger was tripped and the response is currently executing the desired *Plan specified in *Option.Response.ActivatePlan.ActivationDetails*. An *ActivityStatus* of "COMPLETED" means that the desired *Plan has finished execution.

In the event that there are lost communications, which is defined by no *SystemStatusMT* received for the duration of the *MissionPlanningAutonomy.Trigger.CommsLost.Threshold*, MA will activate the contingency plan and execute it. Though the messages will not deliver to C2 due to the comms loss, MA generates *MissionContingencyAlertMT* in response to the contingency being activated. Additionally, MA sends *MA_ResponseActivityMT_2*, *_3*, and *_4* with information on the *ActivatedPlans* of the contingency response plan and the *AlertID* to specify the *MissionContingencyAlert* for the Response.

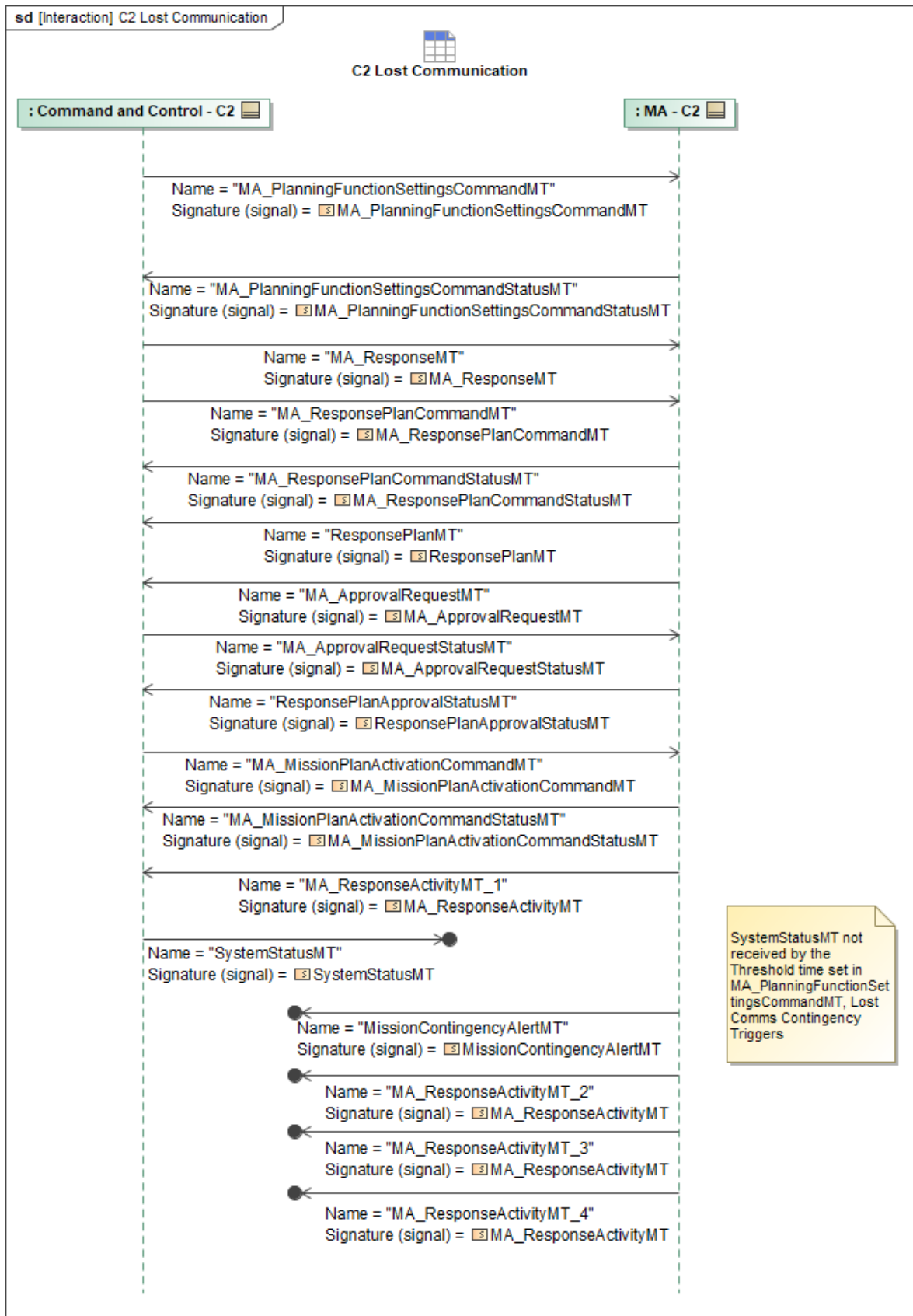


Figure 5-13: C2 Lost Communication

Table 5-12: C2 Lost Communication

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalRequestMT: MA_ApprovalRequestMT	None
MA_ApprovalRequestStatusMT: MA_ApprovalRequestStatusMT	OperatorID: OperatorID_Type
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	ResponsePlan: ResponsePlanActivationType
MA_MissionPlanActivationCommandStatusMT: MA_MissionPlanActivationCommandStatusMT	None
MA_PlanningFunctionSettingsCommandMT: MA_PlanningFunctionSettingsCommandMT	Threshold: DurationType AlertOnly: EmptyType MissionPlanningAutonomy: MA_MissionPlanningAutonomySettingType CommsLost: CommsLostTriggerDataType CommsLost: CommsLostTriggerDataType AlertOnly: EmptyType
MA_PlanningFunctionSettingsCommandStatus MT: MA_PlanningFunctionSettingsCommandStatus MT	None
MA_ResponseActivityMT_1: MA_ResponseActivityMT	MissionPlanID: MissionPlanID_Type
MA_ResponseActivityMT_2: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: PlansReferenceType
MA_ResponseActivityMT_3: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: PlansReferenceType
MA_ResponseActivityMT_4: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MA_ResponseMT: MA_ResponseMT	AnyMessage: QueryMessageType MessageType: MessageTypeEnum ActivatePlan: MA_MissionPlanActivationCommandType
MA_ResponsePlanCommandMT: MA_ResponsePlanCommandMT	None
MA_ResponsePlanCommandStatusMT: MA_ResponsePlanCommandStatusMT	ResultingPlanID: ResponsePlanID_Type
MissionContingencyAlertMT: MissionContingencyAlertMT	None
ResponsePlanApprovalStatusMT: ResponsePlanApprovalStatusMT	None
ResponsePlanMT: ResponsePlanMT	PlanCommandID: ResponsePlanCommandID_ChoiceType

Message Name (Name:Type)	Required Fields (Name:Type)
	AllocatedResponse: ResponseAllocationType
SystemStatusMT: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.4 C2 Malformed Message/Precondition Not Met Handling

Upon receipt of a malformed message, MA will send *MA_SystemNotificationMT* to C2, indicating that the message could not be successfully processed. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with *INVALID_INPUT_PARAMETER*. *NotificationDetail.Description* will contain a human-readable string stating which fields are malformed. *AssociatedMessage* will reference the malformed message. The *MessageType* will be set to the same type as the malformed message. *AssociatedID* will be populated with a reference ID from the malformed message if applicable, the *UUID* of that message. If additional data is required to resolve the condition and the fault remains active, MA will issue *QueryDataRequestMT* with the *Query* field containing the ID of the specific message. C2 will respond with *QueryDataRequestStatusMT*, specifically the *Message* field within the *Result* field will contain the corrected message containing the originally missing data. If the fault has been addressed, then MA will only send a new *MA_SystemNotificationMT* with the same fields as above and *NotificationState* set to CLOSED.

Upon receipt of a command from C2 for which the required inputs are not present on the platform/data store, MA will send *MA_SystemNotificationMT* to C2, indicating that the command could not be successfully executed. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with *UNKNOWN_ID*. *NotificationDetail.Description* will contain a human-readable string stating which data is not present. *AssociatedMessage* will reference the attempted command. The *MessageType* will be set to the same type as the command. *AssociatedID* will be populated with a reference ID from the command if applicable, the *UUID* of that command. If additional data is required to resolve the condition and the fault remains active, MA can issue *QueryDataRequestMT* with the *Query* field containing the ID of the specific command. C2 would then respond with *QueryDataRequestStatusMT*, specifically the *Message* field within the *Result* field will contain the corrected command containing the originally missing data. If the fault has been addressed, then MA will only send a new *MA_SystemNotificationMT* with the same fields as above and *NotificationState* set to CLOSED.

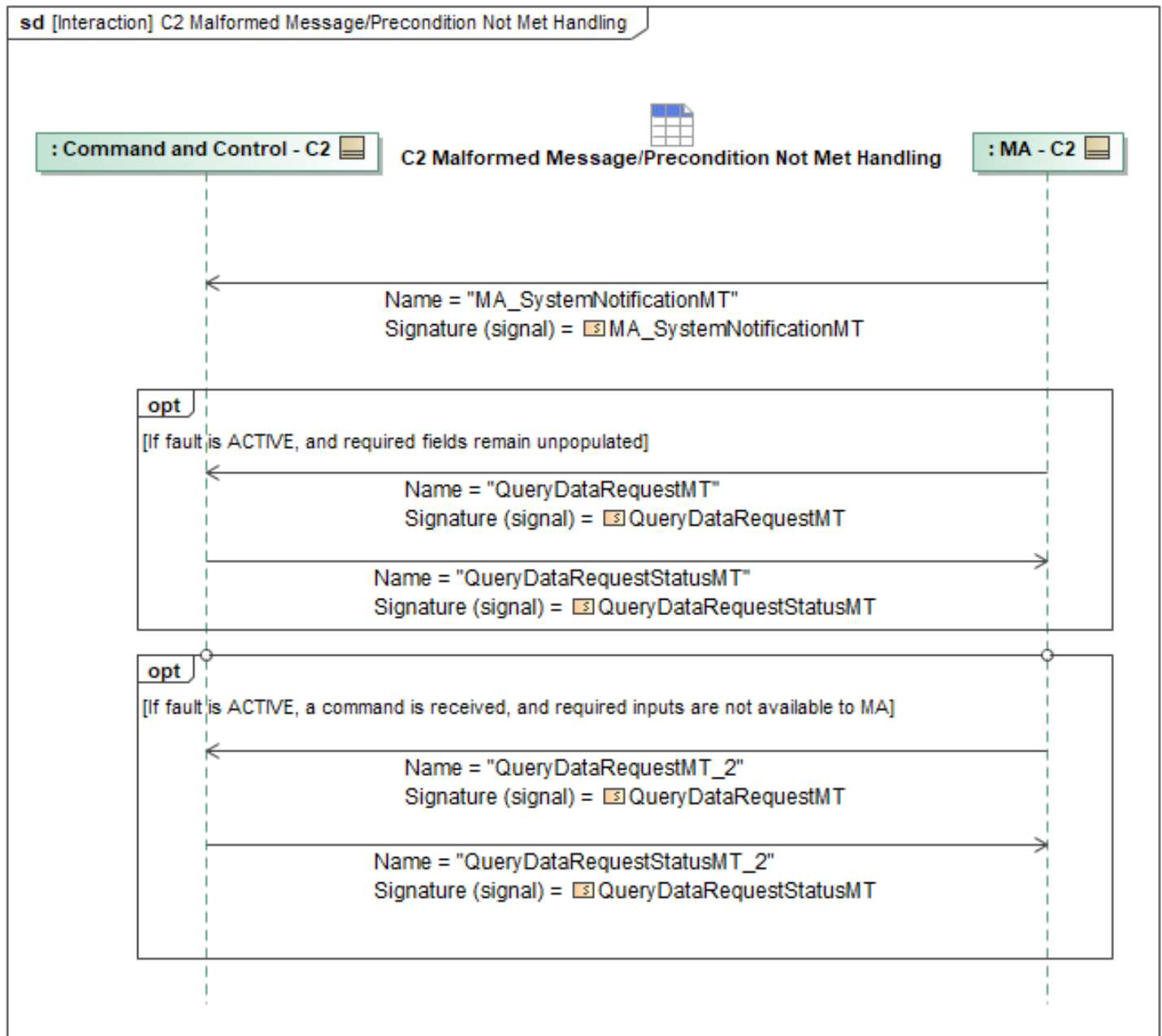


Figure 5-14: C2 Malformed Message/Precondition Not Met Handling

Table 5-13: C2 Malformed Message/Precondition Not Met Handling

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	Description: VisibleString1024Type AssociatedID: ID_Type
QueryDataRequestMT: QueryDataRequestMT	Query: QueryPET
QueryDataRequestMT_2: QueryDataRequestMT	None
QueryDataRequestStatusMT: QueryDataRequestStatusMT	Message: MessageType

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.5 C2 Receive Contingency Status

MA will periodically publish *MA_ResponseActivityMT* to convey monitoring and triggered status for all active contingencies. An *ActivityState* of *ENABLED* indicates MA is actively monitoring the trigger condition for the *Response* while *DISABLED* means monitoring has stopped. When the trigger condition has been satisfied, the *ActivityState* will change to *COMPLETED* with the *RequirementInstantiation* or *ActivatedPlans* fields filled out if the *Response* was configured to perform a response action, otherwise only *AlertID* will be set.

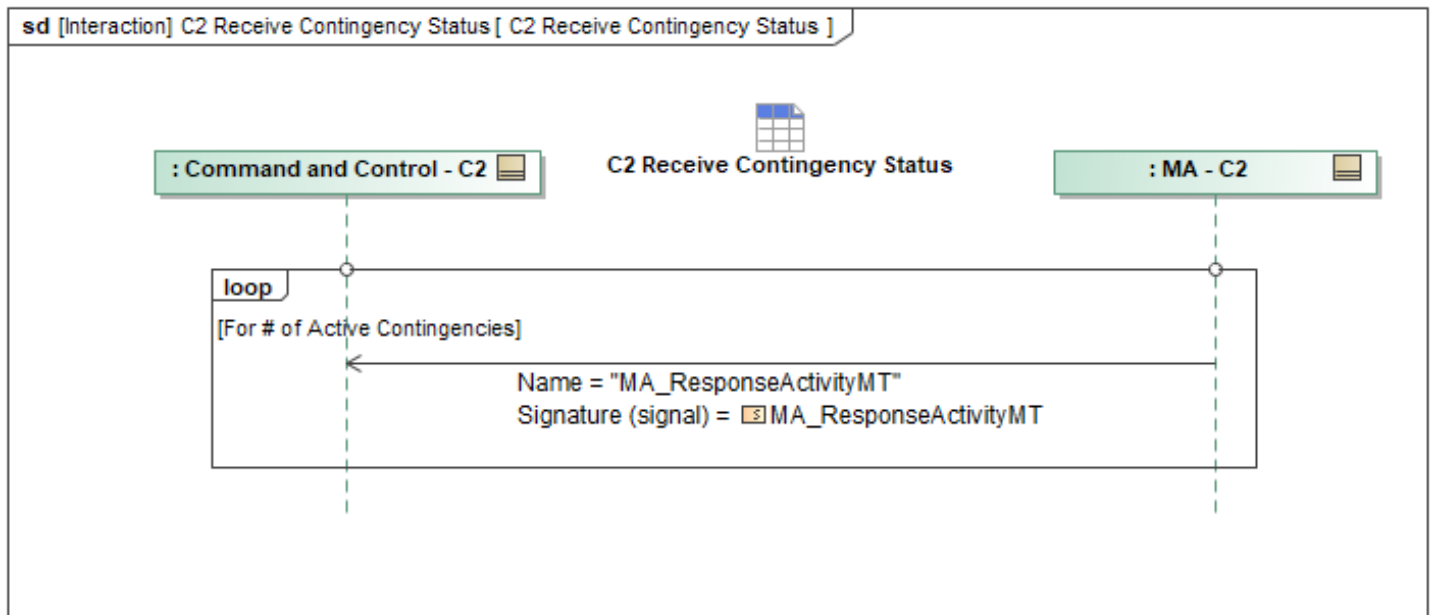


Figure 5-15: C2 Receive Contingency Status

Table 5-14: C2 Receive Contingency Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseActivityMT: MA_ResponseActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.6 Define Autonomous Contingency Re-Plan

C2 systems can configure MA to initiate autonomous replanning in the event of a contingency. C2 systems will send a *MA_PlanningFunctionSettingsCommandMT* to set the contingency trigger and indicate the desired replanning that should occur if the contingency condition is met.

MissionPlanningAutonomy.AutonomousPlanningResponseChoice field should use *AutonomousPlanningAction* or *AlertOnly* field should be "FALSE".

MissionPlanningAutonomy.AutonomousPlanningResponseChoice.AutonomousPlanningAction is used to configure what type of replanning MA should initiate.

MissionPlanningAutonomy.AutonomousPlanningResponseChoice.AutonomousPlanningAction should include a *AutonomousPlanningActionID* to be used to reference this planning action, as well as *AllowedType* field which will configure the specifics of the planning process.

AllowedType.PlanningProcessID should be set to the desired ID for this planning process and will be used to reference this process in future messages. *AllowedType.ForPlanningUseOnly* should be set to "FALSE" as this plan must be executed to resolve the contingency.

When MA receives *MA_PlanningFunctionSettingsCommandMT*, and if C2 is authorized, it replies with *MA_PlanningFunctionSettingsCommandStatusMT_2*, with *CommandedProcessingState* set to "ACCEPTED" if the system is able to apply the autonomy settings or "REJECTED" if the system is not able to apply these settings.

In order for MA to execute a generated plan, the plan must first be approved per the *ApprovalPolicy*. Mission Planning will set the criteria for what plans can be approved with the use of *MA_ApprovalPolicyMT* by an operator vs. which plans can be automatically activated by MA.

In the event C2 wishes to modify an existing approval policy, C2 sends an *MA_ApprovalPolicyMT* message. For contingencies, the approval policy will have an *ApprovalPolicyID* set to the desired ID of the *ApprovalPolicy*. The Plans field should have *Plans.ApplicablePlanningActivity* set to *_PLAN, where the asterisk indicates the appropriate requirement for this planning service. Additionally, the approval policy should have *ByTriggerPolicy.TriggerSource* set to "AUTONOMOUS_TRIGGER" to indicate this policy applies to replanning from autonomous triggers and not operator-commanded replanning. *ByTriggerPolicy.Trigger* can be used to specify that this policy is only for a specific type of trigger. The *MA_ApprovalPolicyMT* will also have *PlanActivation.Policy.ApprovalRequirement* set to either "AUTO_APPROVE" if C2 wishes the contingency replan to be executed immediately, or "APPROVAL_REQUIRED" which will ask a C2 operator for approval of the proposed plan immediately before execution. *MA_ApprovalPolicyMT* with the desired settings, specifically *Plans.ByTriggerPolicy.Policy* can be set to "APPROVAL_REQUIRED" to require MA to wait for operator approval before activating an autonomous replan in response to the defined contingency trigger.

For more information on execution sequences with respect to autonomous replanning, see section Triggered Autonomous Contingency Replan.

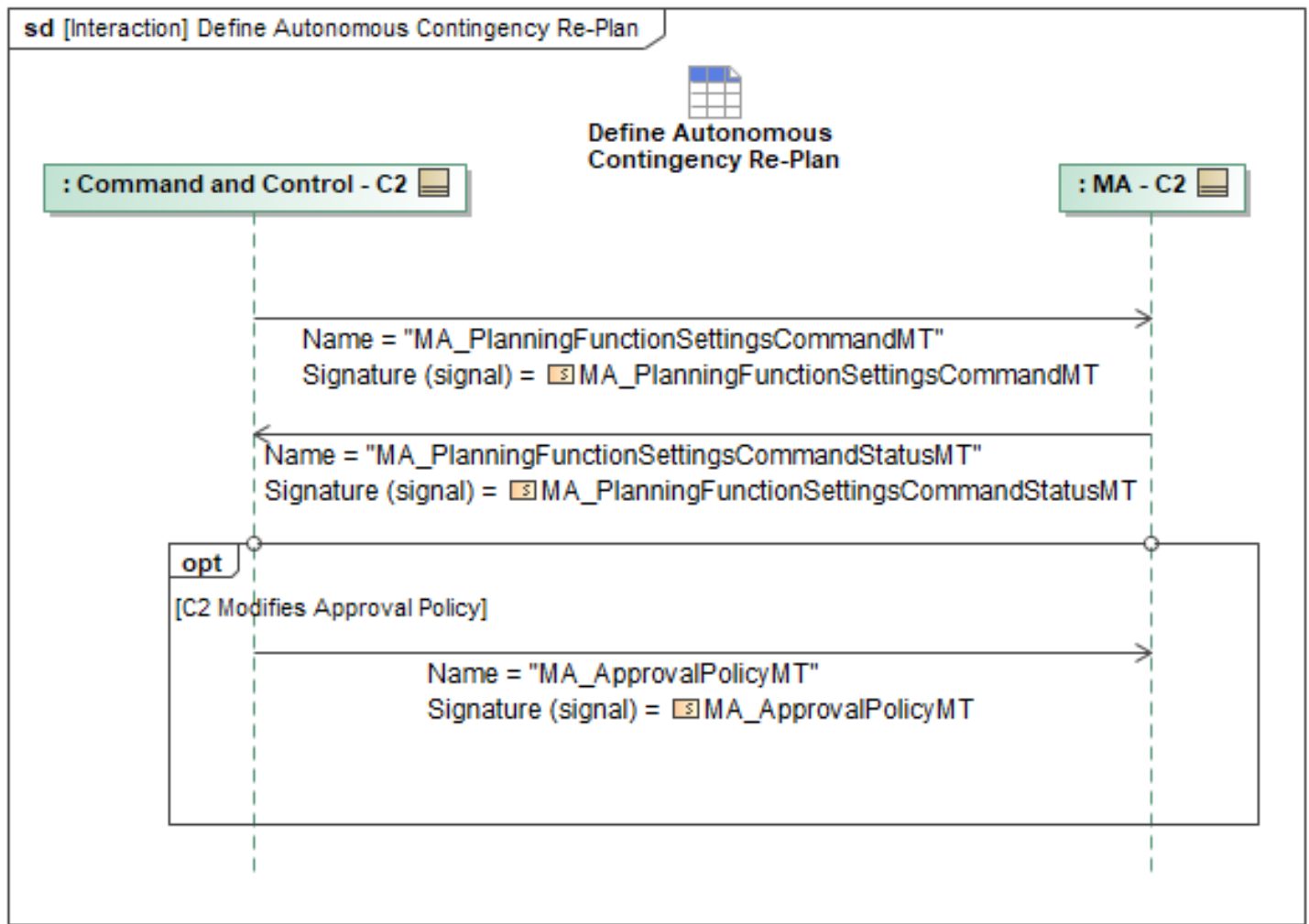


Figure 5-16: Define Autonomous Contingency Re-Plan

Table 5-15: Define Autonomous Contingency Re-Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	PlanActivation: PlanActivationPolicyType Plans: PlanPolicyType ByTriggerPolicy: ByTriggerPolicyType DataRecordInstanceID: DataRecordInstanceID_Type
MA_PlanningFunctionSettingsCommandMT: MA_PlanningFunctionSettingsCommandMT	MissionPlanningAutonomy: MissionPlanningAutonomySettingType
MA_PlanningFunctionSettingsCommandStatus MT: MA_PlanningFunctionSettingsCommandStatus MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.7 Define Contingency Alert

C2 systems can configure MA to generate a *MissionContingencyAlertMT* when a contingency is triggered. C2 Systems will send a *MA_PlanningFunctionSettingsCommandMT* message to MA. *MissionPlanningAutonomy.Trigger* field is used to configure what type of contingency to monitor for. The *PlanningByCaseTriggerType* offers a wide range of possible contingency conditions that can be set. It offers typical contingency conditions like "CommsLost", "EnduranceLow", "OffRoute" etc, as well as other triggers more specific to UCI like "CapabilityAdded" or "RequirementFailed". The *PlanningByCaseTriggerType* also includes types for predicting future contingencies; for example, the "RouteVulnerability" type can be used to set vulnerability thresholds that can predict the likelihood of a future contingency. Depending on the type of trigger chosen, more information may need to be provided. As an example, A "Lost Comms" trigger requires a duration value to measure how long communications have been interrupted for (see section C2 Lost Communications where the lost communications example is walked through in more detail).

MissionPlanningAutonomy.AutonomousPlanningResponseChoice field must be set to "Alert Only" to indicate to MA that an alert should be generated for the chosen Trigger when its conditions are met. This field also indicates that MA should not initiate autonomous contingency replanning. When MA receives *MA_PlanningFunctionSettingsCommandMT* and C2 is authorized, it will reply with a *MA_PlanningFunctionSettingsCommandStatusMT* with the *CommandProcessingState* field set to either "ACCEPTED" if the new autonomy settings were accepted or "REJECTED" if the autonomy settings were rejected. The autonomy settings can be rejected if the system is already constrained by a different set of autonomy settings or has been tasked with other requirements that would interfere with the system's ability to effectively perform the autonomous processes described in the message. If MA rejects the autonomy settings, it will set the *CommandProcessingStateReason* field in *MA_PlanningFunctionSettingsCommandStatusMT* to indicate why the autonomy settings were rejected. C2 systems can use *CommandProcessingStateReason* field in the returned status message to reconfigure the *MissionPlanningAutonomy* field in *MA_PlanningFunctionSettingsCommandMT* and resend to MA.

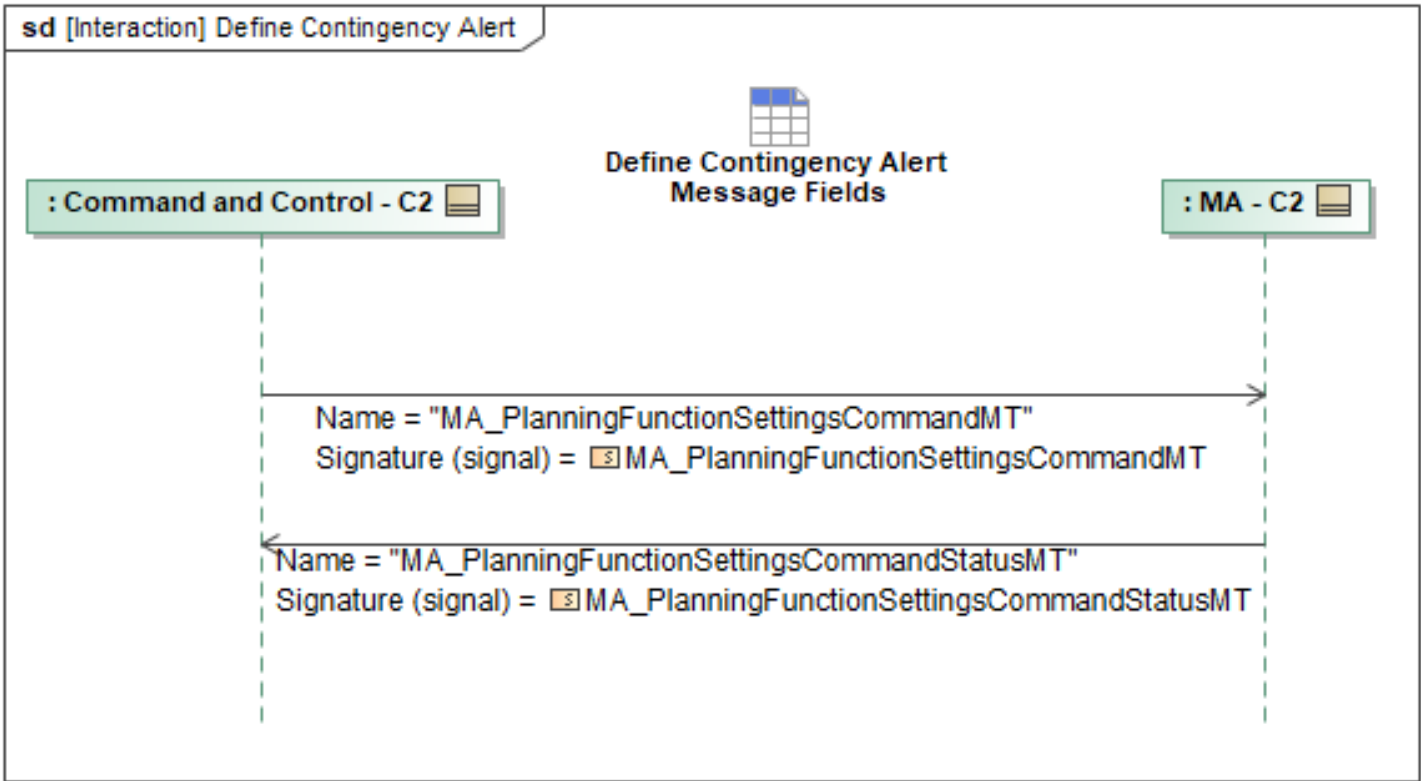


Figure 5-17: Define Contingency Alert

Table 5-16: Define Contingency Alert

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanningFunctionSettingsCommandMT: MA_PlanningFunctionSettingsCommandMT	None
MA_PlanningFunctionSettingsCommandStatus MT: MA_PlanningFunctionSettingsCommandStatus MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.8 Define Contingency Plan

C2 systems will send an *MA_ApprovalPolicyMT* message for each unique contingency plan and will be sent alongside a corresponding *MA_ResponseMT*. Information such as timeout policies and approval durations will be addressed within *ApprovalPolicy.Plans.DefaultPolicy*. Following approval, C2 Systems will use *MA_ResponseMT* in order to activate a specific plan based on a contingency trigger. Refer to section *Define Contingency Alert* for details on setting up an alert with *PlanningFunctionSettingsCommandMT*. C2 systems next step will be to configure a *MA_ResponseMT* to activate a plan when this alert is generated.

The *MA_ResponseMT* will specify the mission plan to be activated when the specified trigger is tripped. The *ResponseType* field should be set to "PLAN_ACTIVATION". The *MA_ResponseMT Option.Trigger* should contain a "QueryMessageType" that will be filtering for

MissionContingencyAlertMTs from the system configured with the previous *PlanningFunctionSettingsCommandMT*. The *QueryMessage* should match *SourceSystemID*, *ContingencyCondition.Trigger*, and *ContingencyCondition.AutonomousActionStatus.AlertOnly* fields in the *MissionContingencyAlertMT*. C2 systems will set *Option.Response.ActivatePlan.MissionPlanID* field to the desired *MissionPlanID* in order to configure the plan that should be activated in the event of the contingency. C2 Systems can further specify what should be activated by using *Option.Response.ActivatePlan.ActivationDetails.ByMissionPlan*, *Option.Response.ActivatePlan.ActivationDetails.BySubPlan*, and *Option.Response.ActivatePlan.ActivationDetails.ByExecutionPlanSet* fields. *ByMissionPlan* indicates that the response should activate all sub-*Plans included in the mission plan. The *BySubPlan* field offers the ability to select specific sub-*Plans to activate, like for example a RTB route plan in response to a "Endurance Low" contingency alert. The *ByExecutionPlanSet* field offers the ability to select a specific *ExecutionPlanSet* to activate, such as one corresponding to a 'Transit' portion of the mission. C2 systems can send one or multiple *ResponseMTs* depending on the number of desired plans that need to be activated to complete the mission.

Following the *MA_ResponseMT* messages, C2 systems will send a *ResponsePlanCommandMT* message with the *CommandState* set to "NEW" or "UPDATE" to tell Mission Autonomy to generate or edit a *ResponsePlanMT* based on the *MA_ResponseMTs* that were sent. A *CommandState* of "NEW" indicates that the Response is new and doesn't currently exist. *CommandState* of "UPDATE" indicates that C2 systems wish to perform an update to an existing *ResponsePlanMT*. Once a Plan has been successfully generated, Mission Autonomy will send out a second *ResponsePlanCommandStatusMT* with *PlanningStatus.CommandProcessingState* to "ACCEPTED", *PlanningStatus.CommandStatus* to "COMPLETED" and the *ResultingPlanID* field filled out with ID of the new Response Plan. Mission Autonomy will also send a *ResponsePlanMT* containing the newly generated Response Plan with corresponding *PlanCommandID*, specifying the originating *ResponsePlanCommandID*, and *Plan:AllocatedResponse* to indicate that a response should be assigned to the system that the plan is allocated to.

Mission Autonomy will then request the C2 systems to approve the Response Plan via a *MA_ApprovalRequestMT* message with the *RequirementID* of the Response Plan set in *ApprovalReferences.ApprovalItem.PlanApproval*. The C2 system will send back an *MA_ApprovalRequestStatusMT* message with *ApprovalRequestProcessingState* set as "APPROVED" if the Response Plan is deemed acceptable or "REJECTED" if the Response Plan is not acceptable. If the Response Plan is rejected, Mission Autonomy will initiate replanning and propose a new Response Plan to C2. If the Response Plan is accepted, the plan is ready to be executed. MA sends *ResponsePlanApprovalStatusMT* to verify that the Response Plan was successfully approved. To begin execution of the new Response Plan, C2 systems will send a *MA_MissionPlanActivationCommandMT* message with *Command.MissionPlanID* set to the ID of the approved Response Plan. MA will respond with a *MA_MissionPlanActivationCommandStatusMT* notifying C2 of the command's status. Once activated, response monitoring will begin and send *ResponseActivityMT* messages to the C2 system. An *ActivityStatus* of "ENABLED" means the trigger is currently being monitored. An *ActivityStatus* of "ACTIVE" indicates the response trigger was tripped and the response is currently executing the desired *Plan specified in *Option.Response.ActivatePlan.ActivationDetails*. An *ActivityStatus* of "COMPLETED" means that the desired *Plan has finished execution.

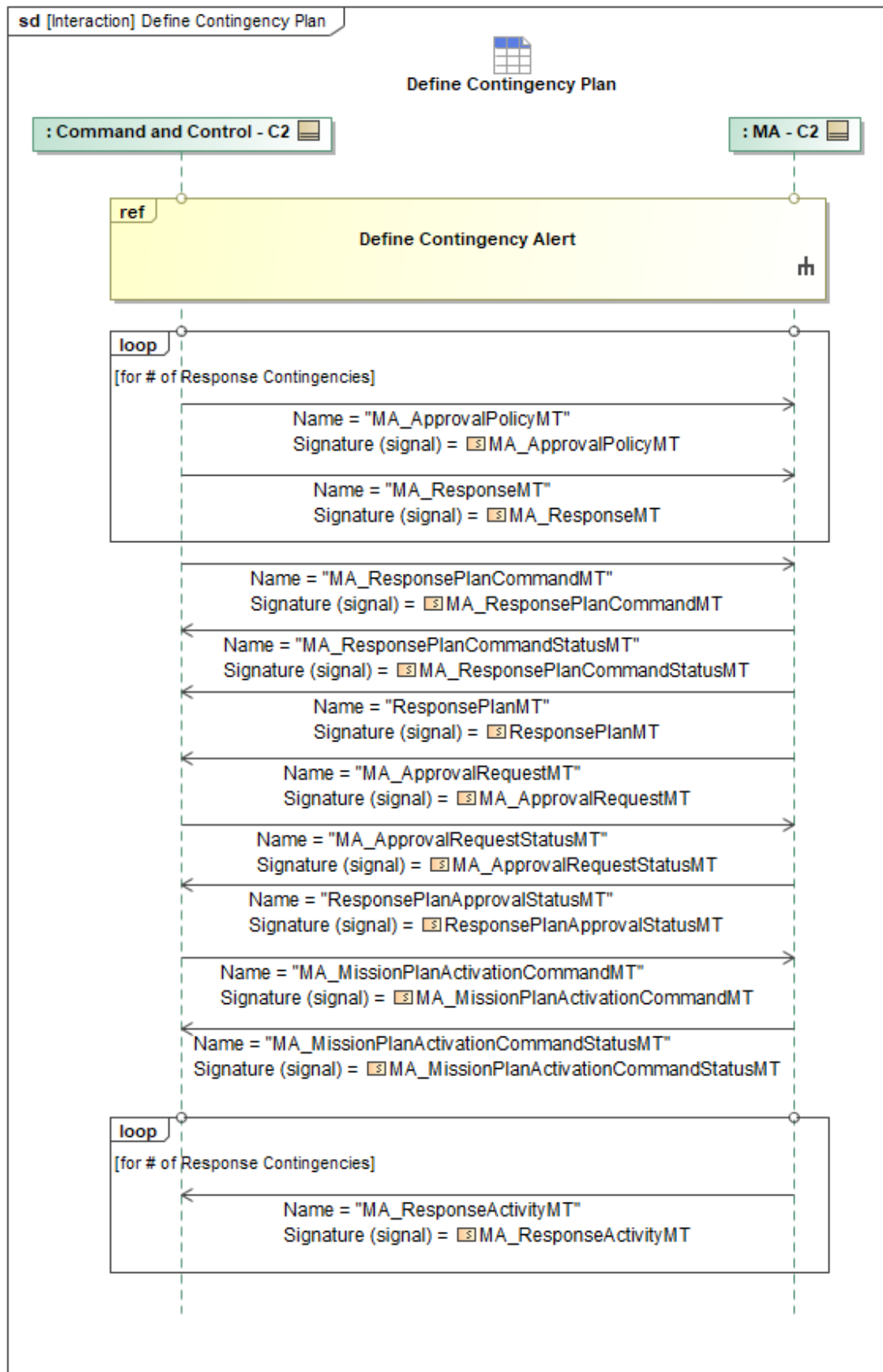


Figure 5-18: Define Contingency Plan

Table 5-17: Define Contingency Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	DefaultPolicy: ApprovalPolicyBaseType Plans: PlanPolicyType
MA_ApprovalRequestMT: MA_ApprovalRequestMT	None
MA_ApprovalRequestStatusMT: MA_ApprovalRequestStatusMT	OperatorID: OperatorID_Type
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	None
MA_MissionPlanActivationCommandStatusMT: MA_MissionPlanActivationCommandStatusMT	None
MA_ResponseActivityMT: MA_ResponseActivityMT	MissionPlanID: MissionPlanID_Type
MA_ResponseMT: MA_ResponseMT	None
MA_ResponsePlanCommandMT: MA_ResponsePlanCommandMT	MissionPlanID: MissionPlanID_Type
MA_ResponsePlanCommandStatusMT: MA_ResponsePlanCommandStatusMT	ResultingPlanID: ResponsePlanID_Type
ResponsePlanApprovalStatusMT: ResponsePlanApprovalStatusMT	None
ResponsePlanMT: ResponsePlanMT	PlanCommandID: ResponsePlanCommandID_ChoiceType AllocatedResponse: ResponseAllocationType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.9 Triggered Autonomous Contingency Re-Plan

When a trigger for a contingency is tripped, the MA will send the C2 systems a *MissionContingencyAlertMT_1* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detects the contingency and is generating the alert, the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field set to "FALSE" to indicate that an action will be taken, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that has the contingency condition, the *ContingencyCondition.ConflictState* field set to "CONFLICTED" to indicate that the conflict that caused the trigger has not been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique ID for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions, the *ContingencyCondition.AutonomousActionStatus.AutonomousPlanningActionStatus.AutonomousPlanningActionID* field set to the ID of the *AutonomousPlanningAction* and the *ContingencyCondition.AutonomousActionStatus.AutonomousPlanningActionStatus.Status* field set to "IN_PROGRESS".

Next, MA must check the approval required to execute the appropriate plan. In a previous sequence (Update Contingency Plan), the contingency plans were loaded and *ApprovalPolicyMT* sent. *ApprovalPolicyMT* contains an *ApprovalPolicyID*, *PlanID*, the plan itself, and approval requirements under the *ApprovalPolicyID*, *Plans.ApplicablePlanningActivity*, and *ApprovalPolicy.Plans.ByTriggerPolicy.Policy* fields, respectively. If *ApprovalPolicy.Plans.ByTriggerPolicy.Policy = Approval_Required*, MA will send the plan to be executed to the C2 systems. The plan could be any type of plan (i.e *MA_TaskPlanMT*, *MA_RoutePlanMT*, *ActionPlanMT*, etc.) depending on the constraints that were given to Mission Autonomy when the trigger was defined. The sequence diagram is using an *MA_RoutePlanMT* as an example.

If the *ApprovalPolicy* was previously set to "operator" when the contingency was setup, then after the *MA_RoutePlanMT_1* is sent, the MA will send the C2 systems an *MA_ApprovalRequestMT* with the *RequestID* field set to the unique ID of the request message, the *Approver.NonOperatorIdentifier.SystemID* set to the unique ID of the system, the *Approver.NonOperatorIdentifier.ServiceID* set to the unique ID of the service, the *ApprovalReferences.ApprovalPolicyID* set to the unique ID of the *ApprovalPolicy* that resulted in the *MA_ApprovalRequestMT*, the *ApprovalReferences.ApprovalItem.PlanApproval* set to the the unique ID of the Plan that is under review for approval, and the *RespondByField* set to the date and time that the request must be approved by. The C2 systems will respond with an *MA_ApprovalRequestStatusMT* with the *RequestID* set to the unique ID of the request message corresponding to this status message, the *RequestProcessingState* set to "COMPLETED", the *OperatorID* field filled out so MA has awareness of who approved the request, and the *ApprovalRequestProcessingState* set to "APPROVED". If it is not approved, then MA will send another *MA_RoutePlanMT_1* message until the operator approves it. If the *ApprovalPolicy* was not set to "operator", the MA will use the plan it generated without any external approval via *MA_RoutePlanMT_2*.

The execution status of the activated *Plan will be sent via **PlanExecutionStatusMT*. The example in the sequence diagram is for a *RoutePlanExecutionStatusMT* to be sent periodically. When the contingency has been cleared, the MA will send the C2 systems a *MissionContingencyAlertMT_2* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detected the contingency and generated the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that had the contingency condition, the *ContingencyCondition.ConflictState* field set to "CLEARED" to indicate that the conflict that caused the trigger has been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique id for this conflict, and the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field set to "FALSE" and the *ContingencyCondition.AutonomousActionStatus.AutonomousPlanningActionStatus.Status* field set to "SUCCESSFUL" to indicate successful action to clear the contingency.

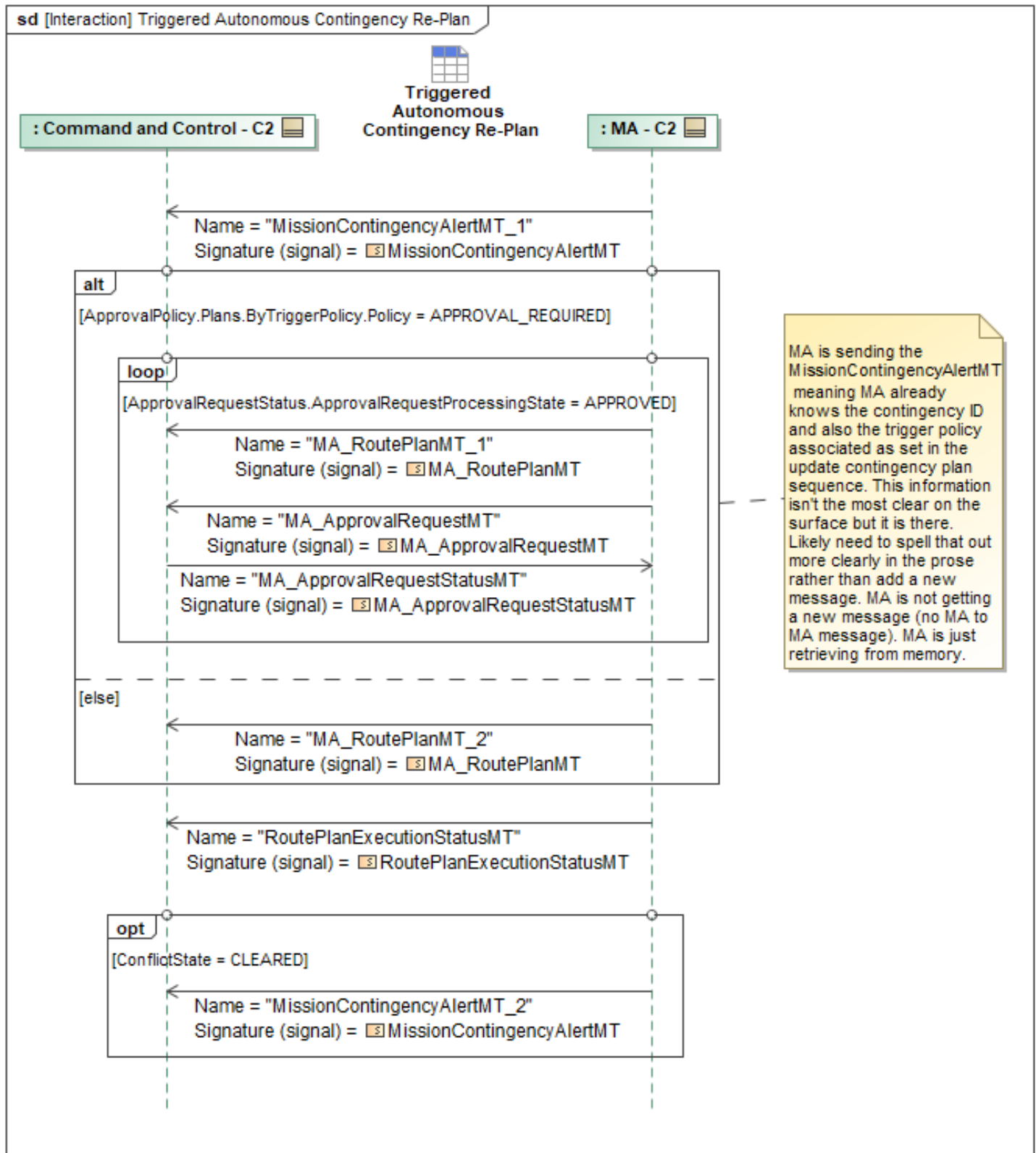


Figure 5-19: Triggered Autonomous Contingency Re-Plan

Table 5-18: Triggered Autonomous Contingency Re-Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalRequestMT: MA_ApprovalRequestMT	None
MA_ApprovalRequestStatusMT: MA_ApprovalRequestStatusMT	OperatorID: OperatorID_Type
MA_RoutePlanMT_1: MA_RoutePlanMT	None
MA_RoutePlanMT_2: MA_RoutePlanMT	None
MissionContingencyAlertMT_1: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType
MissionContingencyAlertMT_2: MissionContingencyAlertMT	None
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	PlanExecutionStatus: RoutePlanExecutionStateType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.10 Triggered Contingency Alert

When a trigger for a contingency is tripped, the MA will send the C2 systems a *MissionContingencyAlertMT_1* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detects the contingency and is generating the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that has the contingency condition, the *ContingencyCondition.ConflictState* field set to "CONFLICTED" to indicate that the conflict that caused the trigger has not been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique ID for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions, and the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field set to "TRUE" to indicate that there will be no action taken.

When the contingency has been cleared, the MA will send the C2 systems a *MissionContingencyAlertMT_2* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detected the contingency and generated the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that had the contingency condition, the *ContingencyCondition.ConflictState* field set to "CLEARED" to indicate that the conflict that caused the trigger has been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique id for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions, and the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field set to "TRUE" to indicate that no was action was taken.

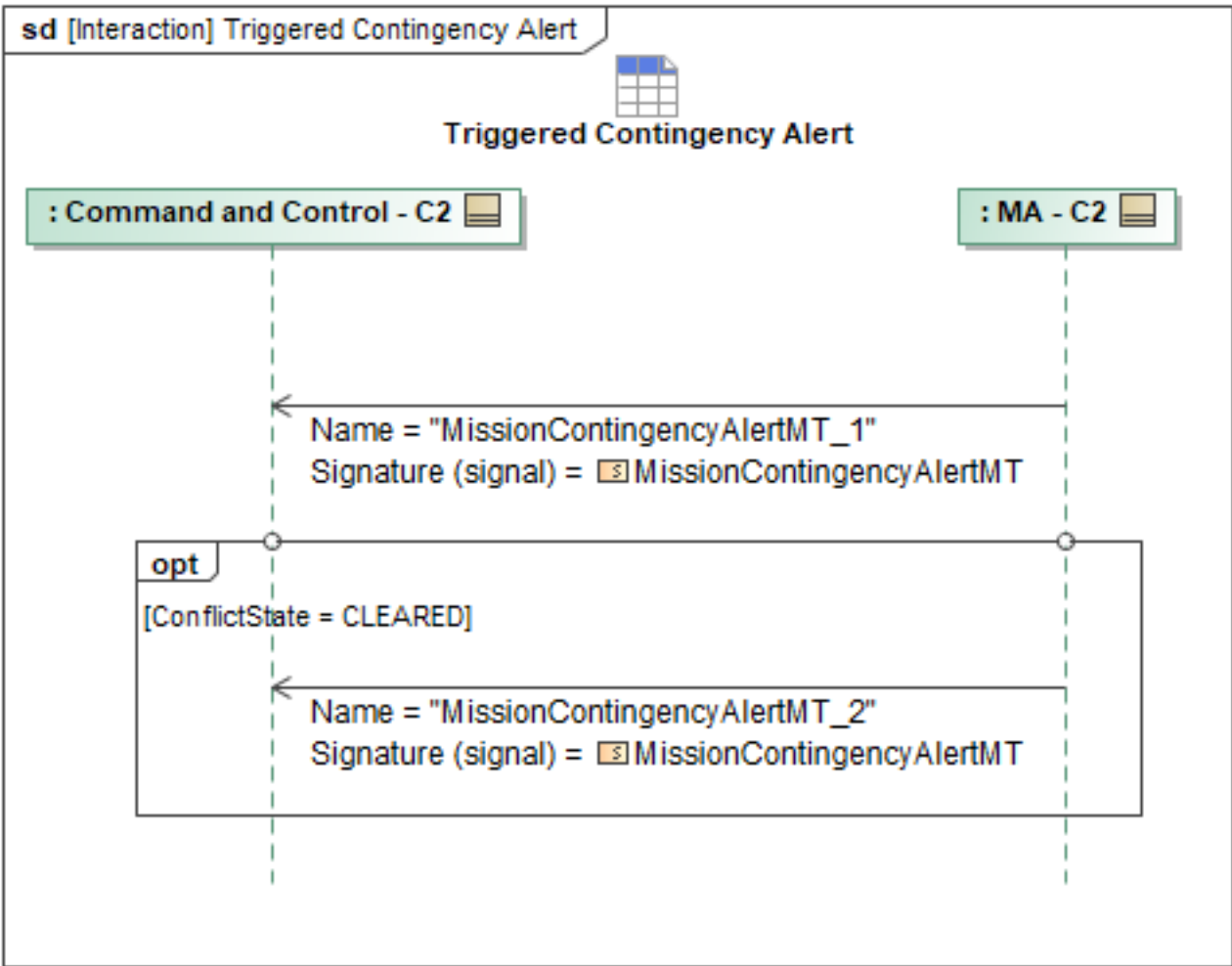


Figure 5-20: Triggered Contingency Alert

Table 5-19: Triggered Contingency Alert

Message Name (Name:Type)	Required Fields (Name:Type)
MissionContingencyAlertMT_1: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType
MissionContingencyAlertMT_2: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.11 Triggered Contingency Plan

When a trigger for a contingency is tripped, the MA will send the C2 systems a *MissionContingencyAlertMT_1* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detects the contingency and is generating the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that has the contingency condition, the *ContingencyCondition.ConflictState* field set to "CONFLICTED" to indicate that the conflict that caused the trigger has not been resolved, the

ContingencyCondition.Conflict.ConflictID set to the unique ID for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions, and the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field set to "TRUE" to indicate that the autonomous planning will not be happening because this contingency was linked to the activation of a specific plan in the ResponsePlan (see Section Define Contingency Plan).

MA sends a *MA_ResponseActivityMT_1* with the *Activity.ActivityState* field set to "ACTIVE_*" (depending on if the Activity is able to be performed based on resource limitations), the *Activity.ActivatedPlans.MissionPlanID* field set to the ID of the MissionPlan if a MissionPlan is being activated, and the *Activity.ActivatedPlans.*PlanID(s)* field(s) set to indicate activation of desired sub-*Plans into the same activation state. A separate *Activity* must be used to set plans to a different *ActivityState*. Once the response activity has been completed, MA sends a *MA_ResponseActivityMT_2* with the *ActivityState* field set to "COMPLETED" and the *Activity.ActivatedPlans.MissionPlanID* and/or *Activity.ActivatedPlans.*PlanID* field set to the ID of the *Plan(s) that was/were executed. MA sends another *MA_ResponseActivityMT_3* with the *ActivityState* field set to "ENABLED" to indicate that the trigger is enabled, the *Activity.ActivatedPlans.MissionPlanID* field set to the ID of the MissionPlan that was activated when the contingency was tripped, and the *Activity.AlertID* field set to the *MissionContingencyAlertID* of the *MissionContingencyAlertMT*.

When the contingency has been cleared, the MA will send the C2 systems a *MissionContingencyAlertMT_2* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detected the contingency and generated the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that had the contingency condition, the *ContingencyCondition.ConflictState* field set to "CLEARED" to indicate that the conflict that caused the trigger has been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique id for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions, and the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field set to "TRUE" to indicate that no was action taken.

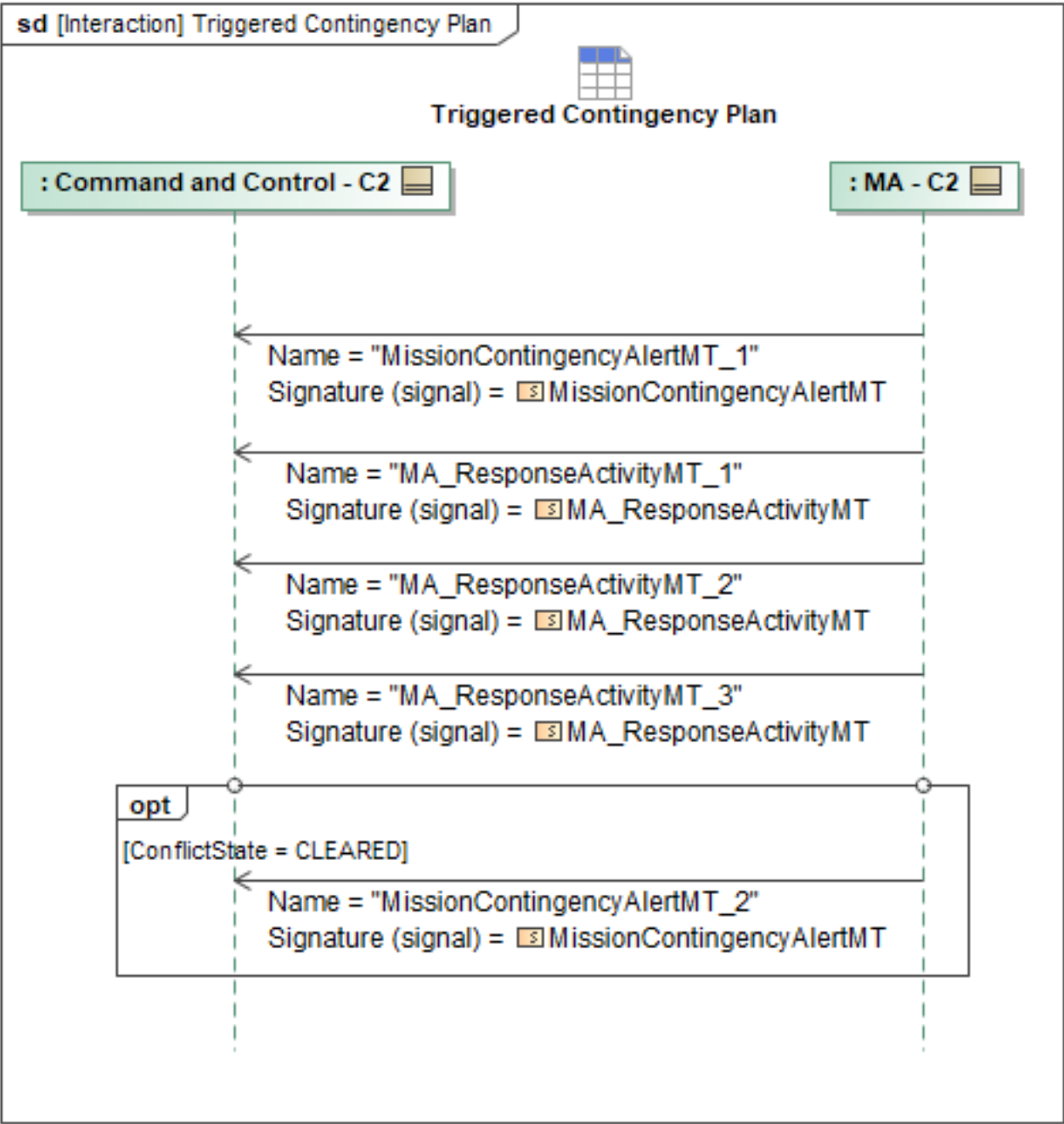


Figure 5-21: Triggered Contingency Plan

Table 5-20: Triggered Contingency Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseActivityMT_1: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: PlansReferenceType
MA_ResponseActivityMT_2: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: PlansReferenceType

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseActivityMT_3: MA_ResponseActivityMT	MissionPlanID: MissionPlanID_Type
MissionContingencyAlertMT_1: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType
MissionContingencyAlertMT_2: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.12 Team Contingencies

5.2.4.12.1 C2 Lost Comms with Team Leader

In the event that a Team leader loses connection with C2, the lost comms contingency response behavior is activated (see *Define Team Lost Comms Contingency*). In normal operations, C2 sends SystemStatusMT_1 to MA periodically, and MA sends SystemStatusMT_2 to C2 periodically. The lost comms condition occurs when SystemStatusMT_1 and SystemStatusMT_2 fail to reach their intended recipients. If the Team Leader regains communication with C2 via receipt of a *SystemStatusMT_3 message* (returning to normal periodic behavior), the Team Leader will send C2 a fresh *PackageStatusMT* updating it of any changes that may have occurred while the team could not communicate with C2. See section *Team Leader Lost Comms with C2* in the peer section for more information.

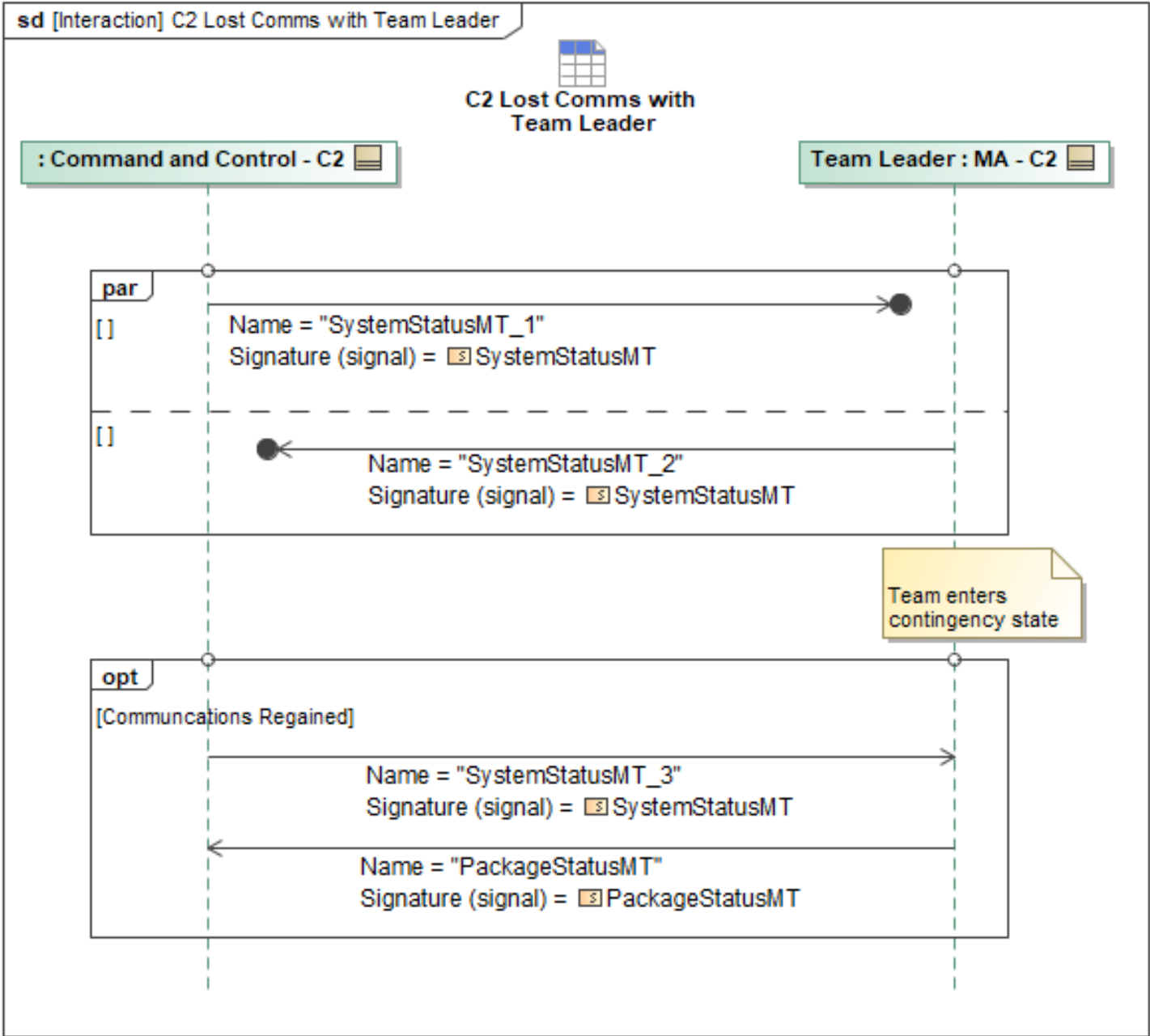


Figure 5-22: C2 Lost Comms with Team Leader

Table 5-21: C2 Lost Comms with Team Leader

Message Name (Name:Type)	Required Fields (Name:Type)
PackageStatusMT: PackageStatusMT	None
SystemStatusMT_1: SystemStatusMT	None
SystemStatusMT_2: SystemStatusMT	None
SystemStatusMT_3: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

5.2.4.12.2 Define Team Lost Comms Contingency

For C2 to define a contingency for a team, a C2 system will send a *MA_SystemManagementRequestMT* to the Team Leader. The *ReferenceSystemID* field will be set to the system ID of the Team Leader. The *RequestType* will be *VehicleSettings*, with *VehicleSettings.LostCommTimeout* set to the maximum time allowed between *SystemStatusMT*s before triggering a lost comms contingency. The Team Leader will then communicate the contingency information to its team, see section *Define Teams Lost Comms Contingency* in the Peer Section for more information. Once the Team Leader has gotten confirmation that all team members have been configured, it will send a *MA_SystemManagementRequestStatusMT* back to C2. The *MA_SystemManagementRequestStatusMT* will have the *RequestProcessingState* field set to "COMPLETED". If the *MA_SystemManagementRequestMT* was not able to be applied to the entire team, then the Team Leader will send C2 systems a *MA_SystemManagementRequestStatusMT* with the *RequestProcessingState* set to "FAILED" and the *RequestProcessingStateReason* set to the reason the *MA_SystemManagementRequestMT* couldn't be applied. Additionally, a contingency could also be triggered by an alert generated by the Comms Manager when it detects a communication outage. See section *Mission Systems* for more information on the Comms Manager.

C2 will then continue to communicate with the Team Leader, following the contingency set up as described in section *Define Contingency Plan*. The Team Leader will route *ResponsePlanMT*s and other contingency messages as needed. See section *Define Team Lost Comms Contingency* in the Peer section for more information on how a Team Leader defines a contingency for its team members.

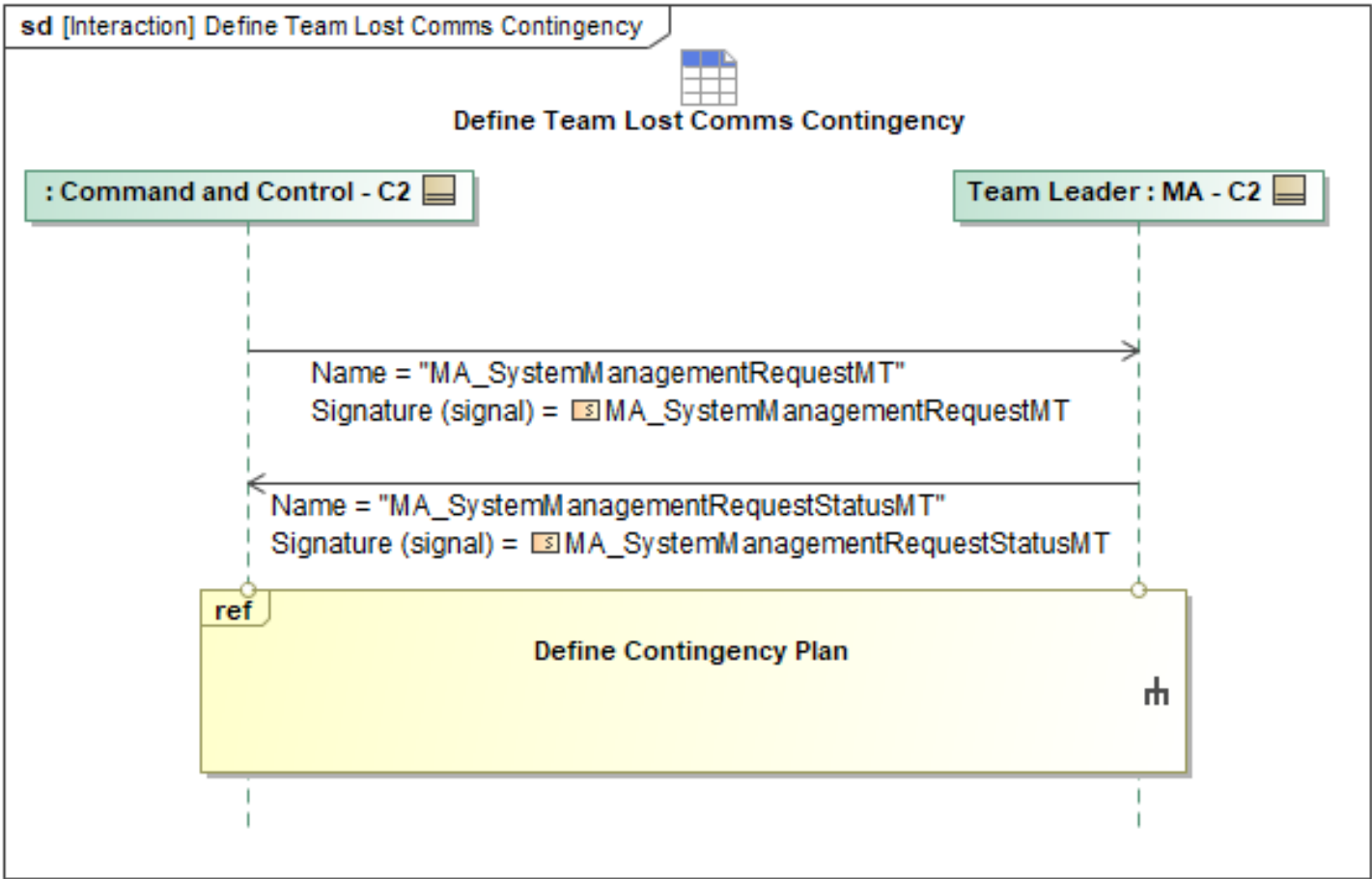


Figure 5-23: Define Team Lost Comms Contingency

Table 5-22: Define Team Lost Comms Contingency

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemManagementRequestMT: MA_SystemManagementRequestMT	LostCommTimeout: DurationType
MA_SystemManagementRequestStatusMT: MA_SystemManagementRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.12.3 Team Leader Lost Comms with Multiple Team Members

In the event that multiple peers are disconnected from the Team Leader, the team will split into two separate teams. It is assumed that if both halves of the segmented team can communicate with the same C2 system, they will communicate via that C2 system so there will be no need to split the team. If a segment of a team can no longer communicate with its team leader or its original C2 system, but can connect to a new C2 system, then it is expected that the disconnected segment will begin communicating with that C2 system as described below. An example of such a situation is if a team experiences a geographical split, where one side loses communication with the rest of the team and C2, but there is another C2 node currently in range. If the disconnected segment cannot connect to

any other C2 systems, it should enter a contingency state as described in section *C2 Lost Comms with Team Leader*.

If the side of the split team that has the Team Leader can contact a C2 system, it will notify C2 with a *MissionContingencyAlertMT* followed by sending each *PackageStatusMT* periodically which will give C2 an update on the team as a whole. In the event that Team Leader cannot communicate with C2, see section *C2 Lost Comms With Team Leader*. The *MissionContingencyAlertMT* will have a *MissionContingencyAlertID* field set to the alert ID generated for this contingency instance. The *SourceSystemID* will be set to the system ID of the Team Leader. The *MissionContingencyAlertMT* will have a *ContingencyCondition* element for each team member it can no longer communicate with. The field *ContingencyCondition.ConflictedSystemID* will correspond to the system whose lost communication is being reported. The field *ContingencyCondition.ConflictState* will be set to "CONFLICTED" to indicate that the contingency is still ongoing. The field *ContingencyCondition.Conflict.ConflictID* will hold an ID for this particular contingency. The *ContingencyCondition.Trigger.Source* should be set to "AUTONOMOUS_TRIGGER" to indicate this alert was generated from MA. The *ContingencyCondition.Trigger.ByCaseTrigger* should be set to "CommsLost". The *ContingencyCondition.AutonomousActionStatus* should be set to *AlertOnly*. The *PackageStatusMT* will have a *PackageID* field to specify which package this status update pertains to. It will also set the *PackageState* field to "DEGRADED" indicating there has been an issue within the package, but it is still operating with some functionality. The *PackageSource* field will be set to "ACTUAL", indicating that there is a known issue with the package. There will also be *PackagePartnerStatus* element for each peer that is part of the team. Each *PackagePartnerStatus* element will have the *SystemID* field set to the system ID of the team member that they are describing. For fully operational peers, the *PackagePartnerStatus.PartnerState* will be set to "OPERATIONAL" (indicating the system is healthy) and the *PackagePartnerStatus.PartnerSource* will be set to "ACTUAL", meaning the Team Leader has up to date information that this system is healthy. For team members in the package experiencing lost communications, the *PackagePartnerStatus.PartnerState* field will be set to "FAILED", indicating that the lost team members are not expected to rejoin this team if they are able to regain communication. The *PackagePartnerStatus.PartnerSource* field will be set to "ESTIMATED" as the Team Leader does not have up to date information about the state of the peer experiencing the lost communications.

After notifying C2 of the systems that lost communication, the Team Leader will send a *PackageMT_1* to indicate to C2 that the team's makeup has been altered and that this message reflects the currently operational team members. The *PackageID* will be set to the original team's package ID, as the side of the split with the Team Leader will retain the original team's package ID. The *LeadSystemID* field will be set to the system ID of the current Team Leader. The *PackageMT_1* will only contain *PackagePartner* elements for the team members that the Team Leader can still communicate with. The *PackagePartner.SystemID* should be set to the system ID of each member in this team and can list as many *PackagePartner.CapabilityID* elements as needed for the corresponding system ID. Right after notifying C2 of this change to the team configuration, the Team Leader will send a *PackageStatusMT_1* periodically with the fields filled out to showcase a fully operational team for this package ID.

The side of the split that doesn't have a Team Leader will immediately go about the process of electing a new Leader (see section *Create a Full Package Amongst Peers*). If the new Team Leader can connect to a C2 system, it will send its own *PackageMT_2* to C2. This *PackageMT_2* will indicate the new makeup for this side of the split team. The *PackageMT_2* will have a new package ID set in the *PackageID* field, the newly elected Team Leader's system ID will be set in the *LeadSystemID*

field, and there will be a *PackagePartner* element for each team member on this side of the split. The *PackagePartner.SystemID* should be set to the system ID of each member in this team, and can list as many *PackagePartner.CapabilityID* elements as needed for the corresponding system ID. After the new Team Leader has let C2 know that its team exists, it will give a status update by sending a *PackageStatusMT_2* to C2, indicating that all team members are operational. If the new Team Leader cannot establish communication with C2, it should behave as described in the section *C2 Lost comms with Team Leader*.

In the event that a split team is able to reconnect, team leads need to communicate status between each other and their respective C2 nodes. First, each team Leader sends the other Team Leader a system status notification using *SystemStatusMT_1* and *SystemStatusMT_2*, periodically and respectively. Then the two teams will each send an *OperatorNotificationMT* to let C2 know that they have regained communication with each other. The original team leader sends *OperatorNotificationMT_1* to the original C2, and the new team leader sends *OperatorNotificationMT_2* to the other C2. The *OperatorNotificationMT* will have an *OperatorNotificationID* set to the ID for this notification. The *Title* field will be set to a string that reads "*Split Team Regained Communication*", the *Category* field will be set to "SYSTEM", the *Severity* field will be set to "INFORMATIONAL" and the *Source* field will be set to each Team Leader's system ID. An operator can then decide if it wants to keep them as two separate teams or merge them together after being notified. (See *Creating or Changing a Package* section for more information on managing teams).

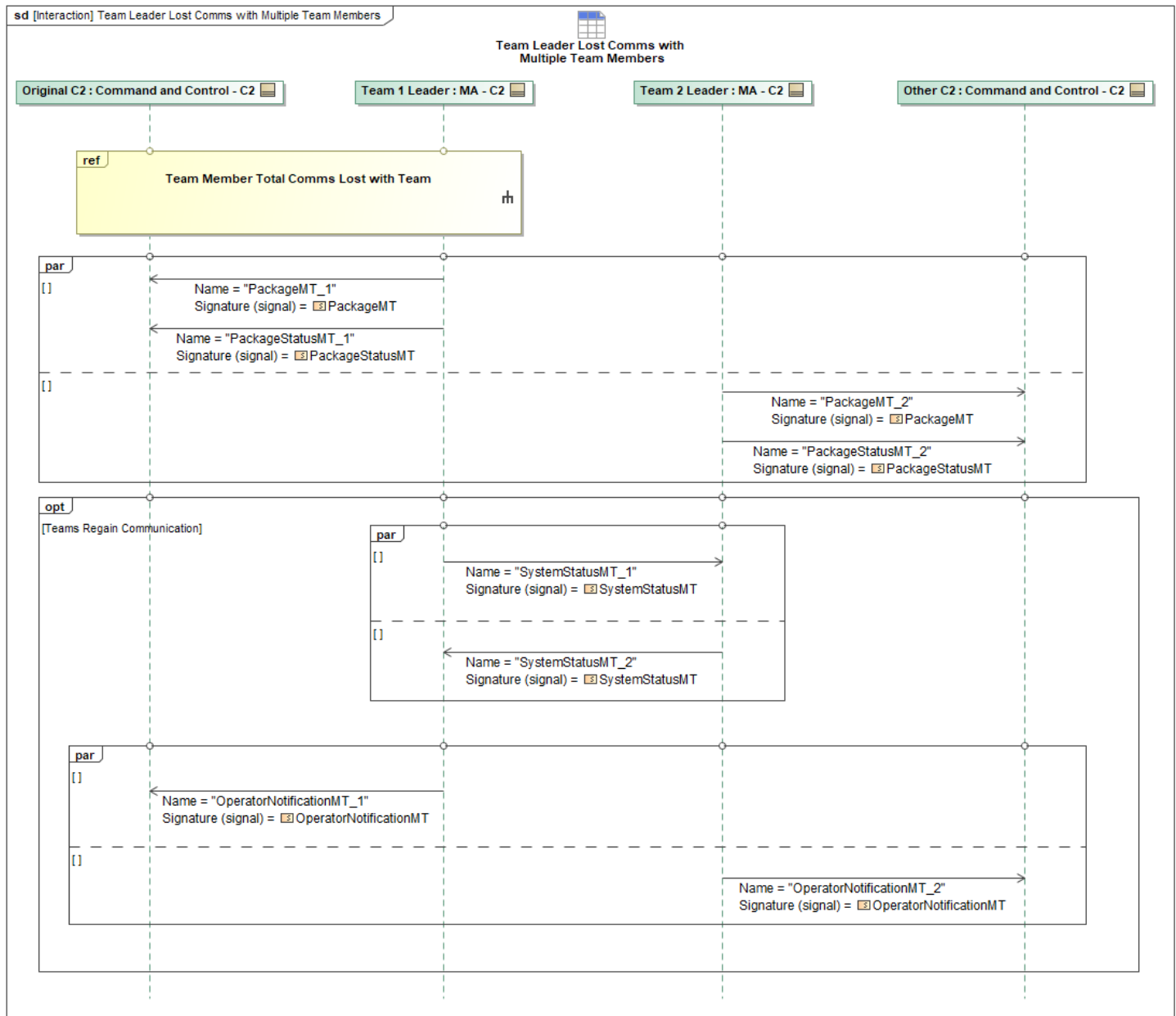


Figure 5-24: Team Leader Lost Comms with Multiple Team Members

Table 5-23: Team Leader Lost Comms with Multiple Team Members

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorNotificationMT_1: OperatorNotificationMT	Source: NotificationSourceType
OperatorNotificationMT_2: OperatorNotificationMT	Source: NotificationSourceType
PackageMT_1: PackageMT	LeadSystemID: SystemID_Type
PackageMT_2: PackageMT	LeadSystemID: SystemID_Type
PackageStatusMT_1: PackageStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
PackageStatusMT_2: PackageStatusMT	None
SystemStatusMT_1: SystemStatusMT	None
SystemStatusMT_2: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.4.12.4 Team Member Total Comms Lost with Team

When the Team Leader loses communication with one of its team members, it will alert the C2 by sending a *MissionContingencyAlertMT_1* with the *MissionContingencyAlertID* field set to the alert ID generated for this contingency, the *Source.SystemID* field set to the system that detected the communications outage, the *ContingencyCondition.ConflictedSystemID* field set to the system ID of the system that has lost its communications, the *ContingencyCondition.ConflictState* field set to "CONFLICTED" to indicate that the contingency has not been resolved, the *ContingencyCondition.Conflict.ConflictID* field set to the ID for this contingency, the *ConflictState.Trigger.Source* field set to "AUTONOMOUS_TRIGGER" to indicate that this alert was generated from MA, the *ContingencyCondition.Trigger.ByCaseTrigger* field set to "CommsLost", the *ContingencyCondition.AutonomousActionStatus.AlertOnly*. It will also update the C2 on the package by sending a *PackageStatusMT_1* with the *PackageID* field set to the package that this status update pertains to, the *PackageState* field set to "DEGRADED" indicating there has been an issue within the package, but it is still operating with some functionality, the *PackageSource* field will be set to "ACTUAL" indicating that there is a known issue with the package. There will also be *PackagePartnerStatus* element for each peer that is part of the team. Each *PackagePartnerStatus* element will have the *SystemID* field set to the system ID of the team member that they are describing. For fully operational peers, the *PackagePartnerStatus.PartnerState* will be set to "OPERATIONAL" (indicating the system is healthy) and the *PackagePartnerState.PartnerSource* will be set to "ACTUAL", meaning the Team Leader has up to date information that this system is healthy. For team members in the package experiencing lost communications, the *PackagePartnerStatus.PartnerState* field will be set to either "DEGRADED" if the Team Leader thinks it will regain communications or "FAILED" if it thinks the communication loss is permanent. In either case, the *PackagePartnerStatus.PartnerSource* field will be set for "ESTIMATED" as the Team Leader does not have up to date information about the state of the peer experiencing the lost communications.

If the team member is able to communicate again, the Team Leader will send the C2 a *MissionContingencyAlertMT_2* with all the same field values except the *ContingencyCondition.ConflictState* field set to "CLEARED" to indicate that the contingency has been resolved. The Team Leader will also send the C2 an updated *PackageStatusMT_2* with the *PackagePartnerStatus.PartnerState* field that was previously set to "DEGRADED" or "FAILED" for this team member set to "OPERATIONAL".

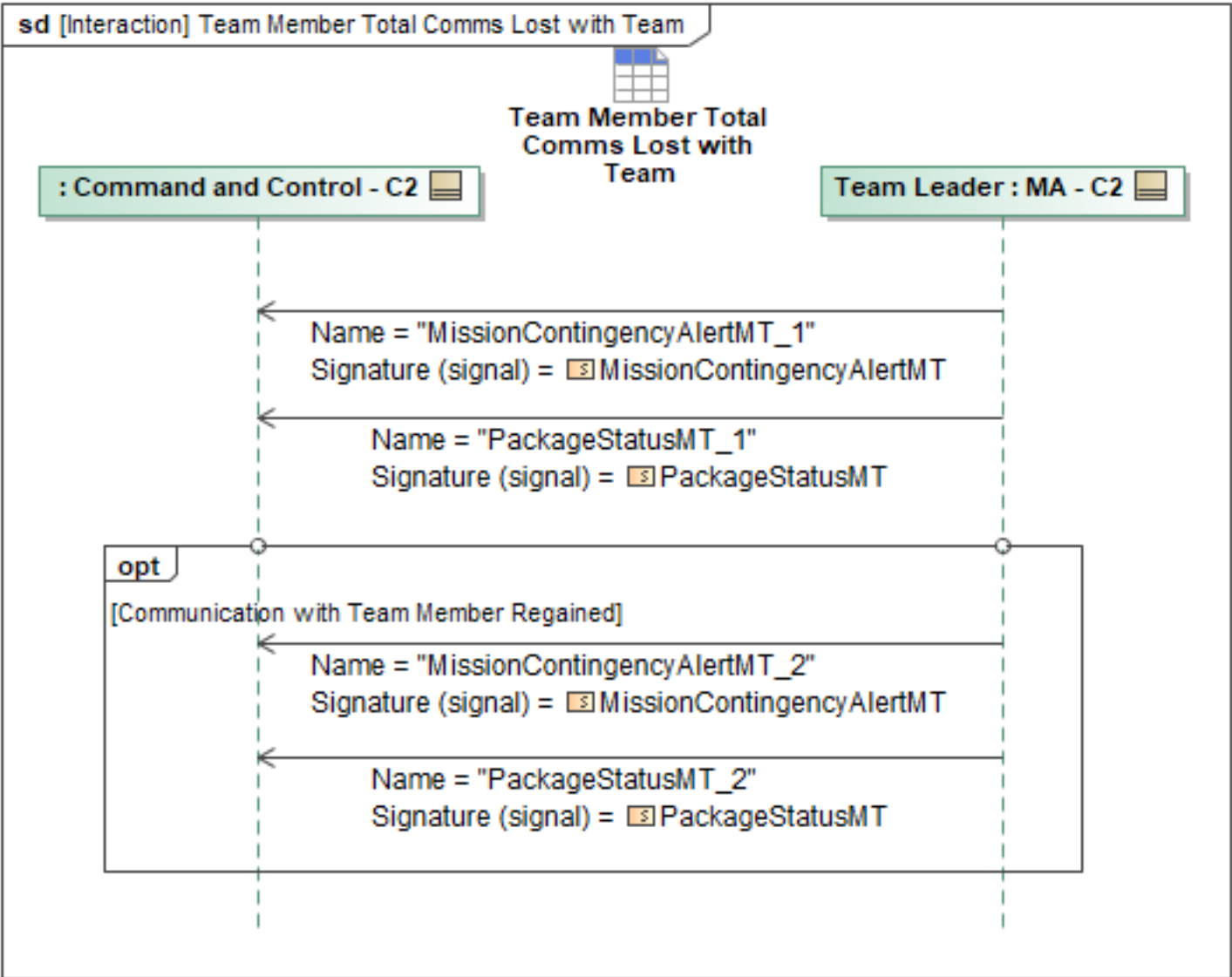


Figure 5-25: Team Member Total Comms Lost with Team

Table 5-24: Team Member Total Comms Lost with Team

Message Name (Name:Type)	Required Fields (Name:Type)
MissionContingencyAlertMT_1: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType
MissionContingencyAlertMT_2: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType
PackageStatusMT_1: PackageStatusMT	None
PackageStatusMT_2: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.5 C2 COP Behaviors Package

5.2.5.1 C2 Requests the COP Configuration Settings

To request an update of the current configuration settings, C2 may send *MA_COP_ConfigurationSettingsRequestMT*, requesting a status update on the COP Configuration Settings from one or more packages. While processing the request, the COP Lead may optionally send a *MA_COP_ConfigurationSettingsRequestStatusMT_1* with *RequestProcessingState* set to *PROCESSING*. Once the request is completed, the COP Lead will send *MA_COP_ConfigurationSettingsRequestStatusMT_2* with the *RequestProcessingState* set to *COMPLETED* and the current settings in the *ConfigurationSettings*. If unable to complete the command, the COP Lead will send *MA_COP_ConfigurationSettingsRequestStatusMT_3* with the *RequestProcessingState* set to *FAILED*.

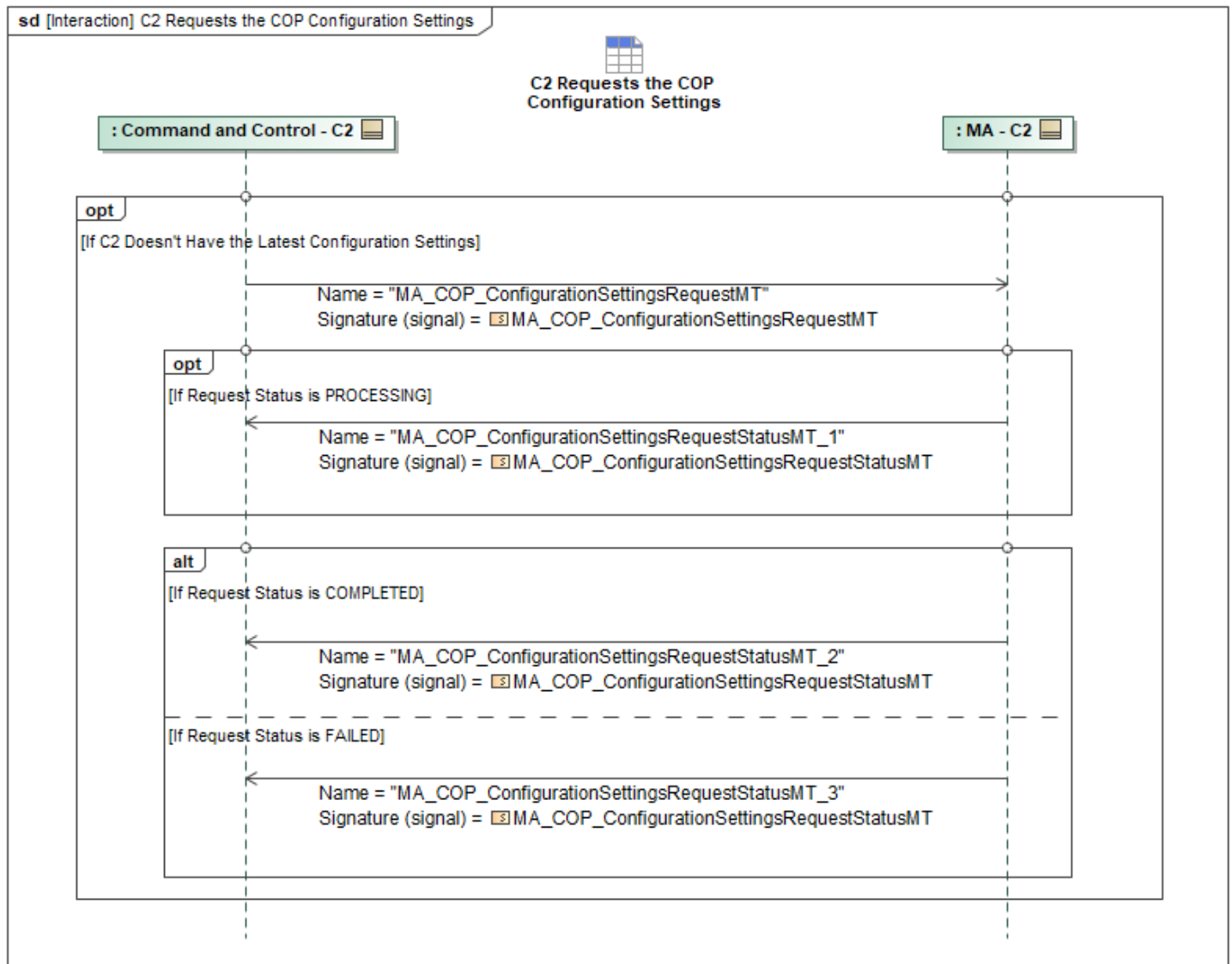


Figure 5-26: C2 Requests the COP Configuration Settings

Table 5-25: C2 Requests the COP Configuration Settings

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsRequestMT: MA_COP_ConfigurationSettingsRequestMT	None
MA_COP_ConfigurationSettingsRequestStatus MT_1: MA_COP_ConfigurationSettingsRequestStatus MT	None
MA_COP_ConfigurationSettingsRequestStatus MT_2: MA_COP_ConfigurationSettingsRequestStatus MT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsRequestStatus MT_3: MA_COP_ConfigurationSettingsRequestStatus MT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.5.2 C2 Updates the COP Configuration Settings

This interaction describes the process of C2 configuring the Common Operating Picture (COP). If C2 configures or updates the COP configuration settings, the *MA_COP_ConfigurationSettingsCommandMT* will be sent to MA on the Recipient ACPs, with the *PackageID*, *COP_LeaderID*, and *RequestAuthority* field set to *ORDER*, to update their COP configuration settings accordingly. To update a Configuration item, set *ConfigurationID* to the UUID of the item to be updated. To add a Configuration item, the *ConfigurationID* is optional. To delete a Configuration item, set the *ConfigurationID* to the UUID of the item to be removed and omit the remaining fields in the Configuration. Once received, MA on the recipient ACP can optionally respond with *MA_COP_ConfigurationSettingsCommandStatusMT* with *CommandProcessingState* set to *RECEIVED*. If both C2 and the Lead ACP send *MA_COP_ConfigurationSettingsCommandMT* to Recipient ACPs, the C2 command will be prioritized over the Lead ACP command. If the command is completed, the COP Lead will send *MA_COP_ConfigurationSettingsCommandStatusMT* with *CommandProcessingState* set to *ACCEPTED* and *MA_COP_ConfigurationSettingsMT* with the updated settings. If unable to execute the command, the COP Lead will send *MA_COP_ConfigurationSettingsCommandStatusMT* with *CommandProcessingState* set to *REJECTED*.

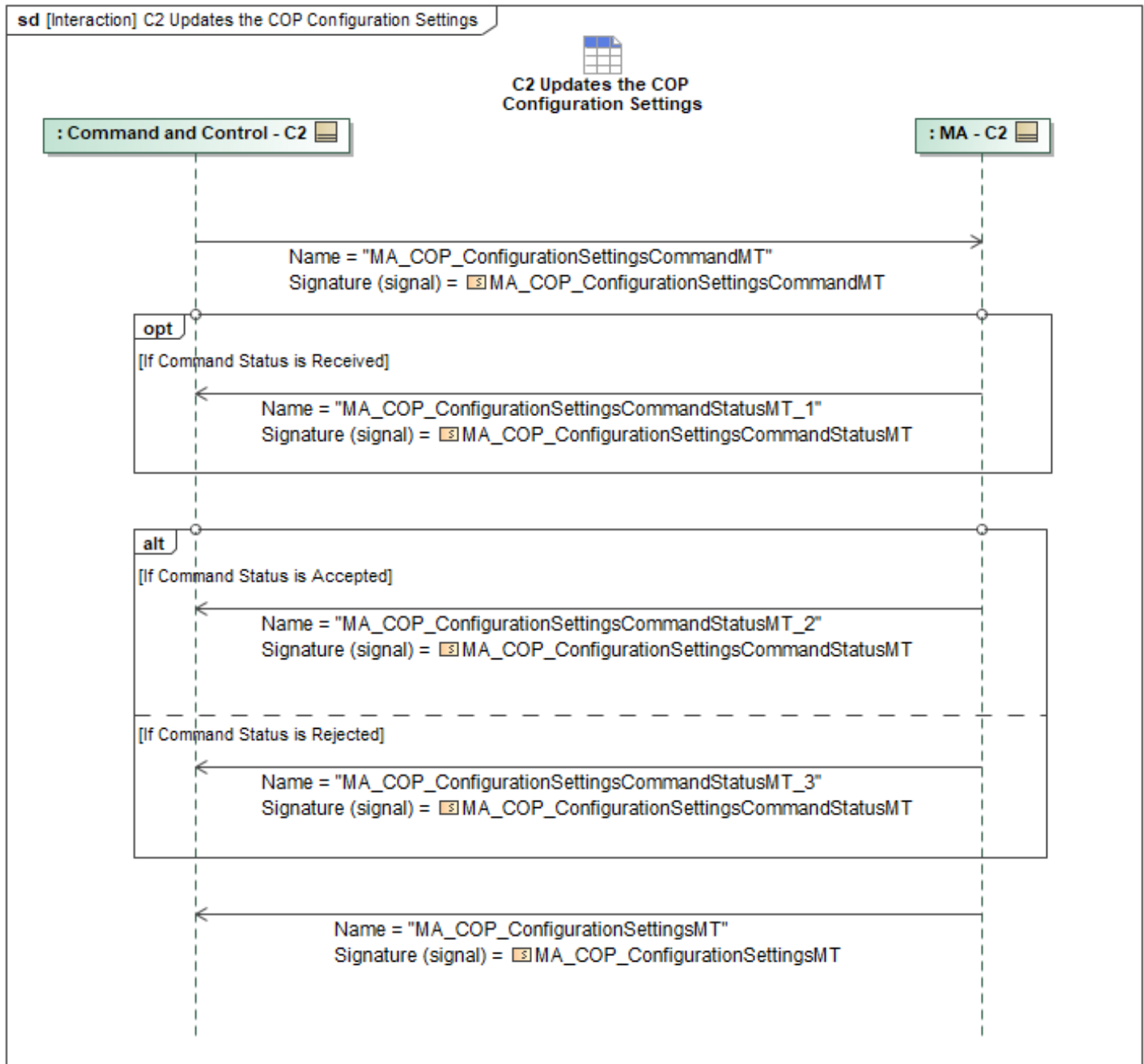


Figure 5-27: C2 Updates the COP Configuration Settings

Table 5-26: C2 Updates the COP Configuration Settings

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsCommandMT: MA_COP_ConfigurationSettingsCommandMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_1: MA_COP_ConfigurationSettingsCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsCommandStatusMT_2: MA_COP_ConfigurationSettingsCommandStatusMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_3: MA_COP_ConfigurationSettingsCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_COP_ConfigurationSettingsMT: MA_COP_ConfigurationSettingsMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.5.3 Synchronize Global COP to C2

The Synchronize Global COP to C2 interaction supports the use case in which the COP Leader has generated a new or updated Global COP and needs to distribute the data to C2. The output of the Global COP includes updated Entity data and updated Team States. This information needs to be synchronized to C2 each time the Global COP is updated. The COP Leader sends *EntityMT* and *PackageStatusMT* to C2. *EntityMT* includes the *MeasurementID* and the *SourceTypeIdIdentifier* fields to provide context for the updated measurements and originating sources, respectively. MA sends the *PackageStatusMT* periodically which communicates the package status, as well as Team States of each member in the package using *PackagePartnerStatus*.

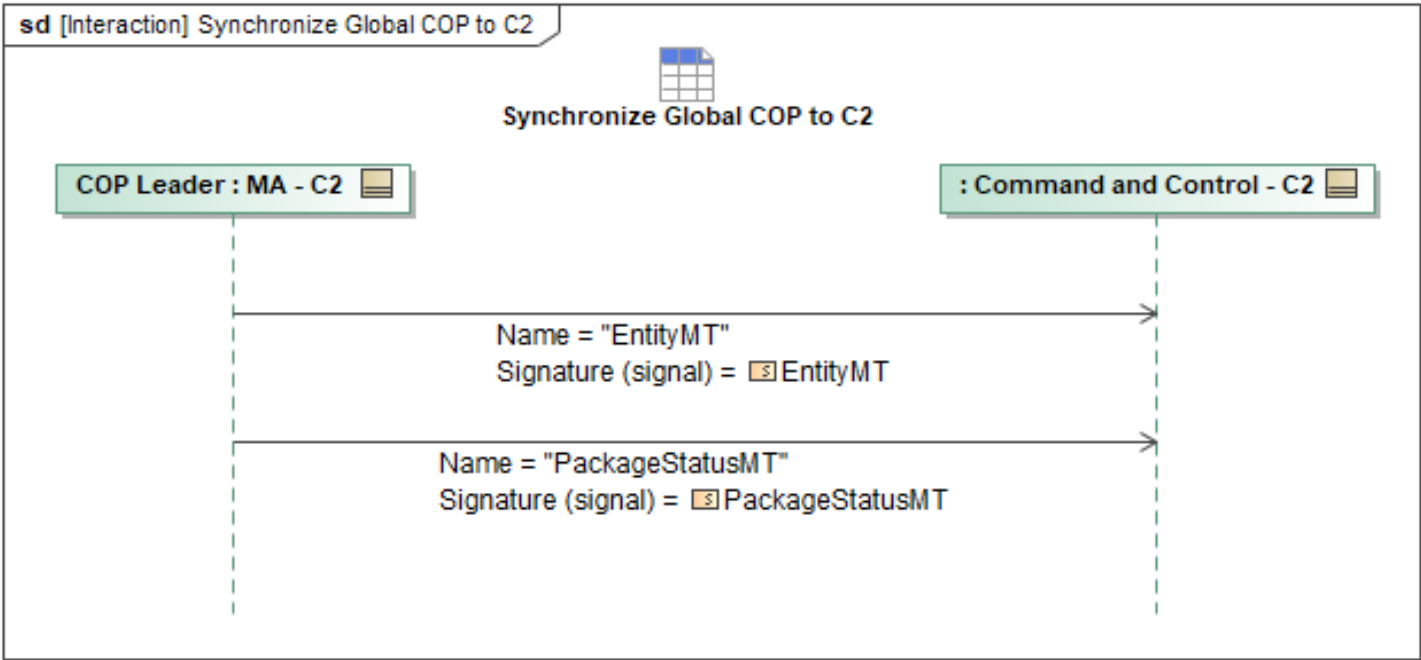


Figure 5-28: Synchronize Global COP to C2

Table 5-27: Synchronize Global COP to C2

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	MeasurementID: MeasurementID_Type SourceTypeIdentifier: EntitySourceIdentifierType
PackageStatusMT: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.6 C2 Engagement Criteria Behaviors Package

The Engagement Criteria schema allows an authorized operator to set specific criteria that an ACP must use while engaging a Target. This includes defining where an ACP can engage a Target, how it can identify *EntityMTs*, what is considered a Target, what a Targeting Act is, who can authorize a Targeting Act against a Target, and how it can keep Custody of that Target. There can only be a single engagement criteria active at any given time. An engagement criteria message (*MA_EngagementCriteriaMDT*) can be activated by using the command message *MA_EngagementCriteriaCommandMDT*. If the *MA_EngagementCriteriaCommandMDT.ApplicableRegion* field is left empty, the engagement criteria specified is active everywhere. When the field is not empty, it restricts the engagement criteria to only be active in specific geozone(s). The Target and the actual weapons release must occur within that geozone(s). The ACP may start engagement activities while not inside the geozone(s), but cannot perform that actual weapons release outside of that area. For example, while the ACP is outside of the geozone(s), C2 can designate a Target and the Requirements can start to execute, but the Strike Activity must not fire on the Target unless both the ACP and Target are inside the geozone(s).

Identity Matrix:

The Engagement Criteria Identity Matrix (*MA_EngagementCriteriaMDT.IdMatrixCriteria*) can be used to classify the identity of *EntityMT* (*EntityMDT.Identity.Standard.StandardIdentity* field set to *FRIEND*, *NEUTRAL*, *SUSPECT*, *HOSTILE*, or *UNKNOWN* if unable to classify). If this field is not populated, then ACPs will use the Entity classified identity or populated by Fusion and not attempt to classify the identity (or confirm identity of entities). If populated, Fusion will apply the identity criteria combined with onboard/shared sensor data to identify/classify entities and possibly alter their identity as appropriate. Any identity updates will be shared via existing COP/fusion processes. An authorized C2 can classify the identity of *EntityMTs* which would take precedence.

Targeting:**Definitions:**

- An "Enemy" is a track that lacks positive Friend markers and has been positively identified as an enemy by Enemy Electromagnetic Indication (EEI), by Visual Identification (VID), or other approved indications (*EntityMDT.Identity.Standard.StandardIdentity* is set to *SUSPECT*).
- A "Threat" is an Enemy that has shown hostile intent or demonstrated a hostile act, or other approved indications (*EntityMDT.Identity.Standard.StandardIdentity* is set to *HOSTILE*).
- A "Targeting Act" is any Requirement (e.g. *MA_TaskMT*, *ActionMT*, etc.) which requires approval under the *ApprovalPolicyMT* referenced by the *ApprovalAuthorityMT* referenced by *MA_EngagementCriteriaMDT.TargetAuthorityCriteria.ApprovalAuthorityID* or within the *ApprovalPolicyMT* referenced by *MA_EngagementCriteriaMDT.TargetAuthorityCriteria.ApprovalPolicyID*. The Requirements listed in this approval policy will typically include *EffectMTs* of type *ATTRIT* or *DEFEAT**, *ActionMTs* of type *ATTACK** or *TARGET*, *MA_TaskMTs* of type *STRIKE* or *TACTICAL_ORDER* and possibly other Requirements that may result in a hostile act against an enemy.
- A "Target" is a *TargetType* entity that has been designated for engagement via kinetic or non-kinetic means by a Target Authority and is the subject of a Targeting Act.
- A "Target Authority" is an operator/system/service listed in the *ApprovalAuthorityMT* message referenced by *MA_EngagementCriteriaMDT.TargetAuthorityCriteria.ApprovalAuthorityID* or *MA_EngagementCriteriaMDT.TargetAuthorityCriteria.AuthorizedOperatorRoles*.
- A "Self Promoted Target" is when MA uses the Engagement Criteria Target Designation Criteria (*MA_EngagementCriteriaMDT.TargetDesignationCriteria*) to trigger a *MA_DesignationRequestMT* to be sent to a Target Authority.

An ACP may execute a Targeting Act against an entity (thereby making that entity a Target) if and only if at least one of the following is true:

1. A Target Authority has sent a *MA_DesignationMT* message that designates the entity as a *TARGET*. MA may request such a designation message via a *MA_DesignationRequestMT*.
2. A Target Authority has commanded (i.e. via *MissionPlanActivationCommandMT*, *TaskCommandMT*, *ActionCommandMT*, etc.) a Targeting Act via direct C2.
3. A Target Authority approves a Targeting Act via a standard UCI *ApprovalRequestMT* / *ApprovalRequestStatusMT* interaction.

If a Target Authority has commanded a Targeting Act or has sent a *MA_DesignationMT* that triggers a *ResponseMT* that starts executing a Targeting Act, an *ApprovalRequestMT* / *ApprovalRequestStatusMT* interaction is not required even if the Targeting Authority linked *ApprovalPolicy* specifies it is required. This would only be needed if a C2 that is not a Target Authority commanded a Targeting Act or has sent a *MA_DesignationMT* that triggers a *ResponseMT* that starts executing a Targeting Act.

Target Custody:

The Engagement Criteria Target Custody Criteria (*MA_EngagementCriteriaMDT.TargetCustodyCriteria*) details how the ACP will ensure that it has custody of the target during the weapons engagement process. If this criteria/field is populated, the Strike Activity can use this to determine if it does or does not have custody of the Target. If custody is lost this would pause or cancel the Strike Activity on that Target. If the criteria specifies that the *EntityMT* must be *HOSTILE* and the Engagement Criteria Identity Matrix is populated, the Identity Matrix Criteria can be used to continually confirm that the *EntityMT* is *HOSTILE* given the provided criteria and not just the set *EntityMT* identity setting.

5.2.6.1 Activate Engagement Criteria

The Activate Engagement Criteria sequence diagram specifies the interaction of C2 commanding MA to activate one of its stored engagement criteria activation schema. First, C2 sends a *MA_EngagementCriteriaCommandMT* message to MA. The *AppliesToID* field can have multiple entries of *PackageID* or *SystemID*. This allows a C2 to send a Lead MA the Package ID (or multiple sub Package IDs) or a set of System IDs. The *MA_EngagementCriteriaCommandMT* message shall specify the *EngagementCriteriaID* field containing the ID of the Engagement Criteria message that C2 wishes to be active. Any new *MA_EngagementCriteriaCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_EngagementCriteriaCommandMT* is sent with the *CommandedState* set to "CANCEL", this will cancel the active command and active engagement criteria. In response to receiving the command, MA shall send *MA_EngagementCriteriaCommandStatusMT_1* to C2. The *CommandProcessingState* will be set to "RECEIVED" to indicate receipt of the command. For this status message and all other status messages (*MA_EngagementCriteriaCommandStatus* & *MA_EngagementCriteriaStatus*), the *AppliesToID* field shall be set to the Package ID if MA is the leader and has received a status message from all members of the Package ID, otherwise it shall be set to System ID(s) and/or sub Package ID(s) that the status message applies to. The MA shall then use the *EngagementCriteriaID* to query its onboard MP to retrieve the *MA_EngagementCriteriaMT*. If the MA is a non-package ACP, it shall send *MA_EngagementCriteriaCommandStatusMT_2* to C2 with the *CommandProcessingState* set to "ACCEPTED". The MA will send *MA_EngagementCriteriaStatusMT_1* to C2 containing the *EngagementCriteriaID* and other configured data as needed. Optionally, if MA is the lead ACP, then MA shall message its follower ACPs with the *MA_EngagementCriteriaCommandMT* (See "Activate Peer Engagement Criteria"). After receiving the respective responses from the follower ACPs, MA (the lead ACP) would send C2 the *MA_EngagementCriteriaCommandStatusMT_3* with the *CommandProcessingState* set to "ACCEPTED" and the *MA_EngagementCriteriaStatusMT_1* both reflecting the *AppliesToID* as needed and other configured data.

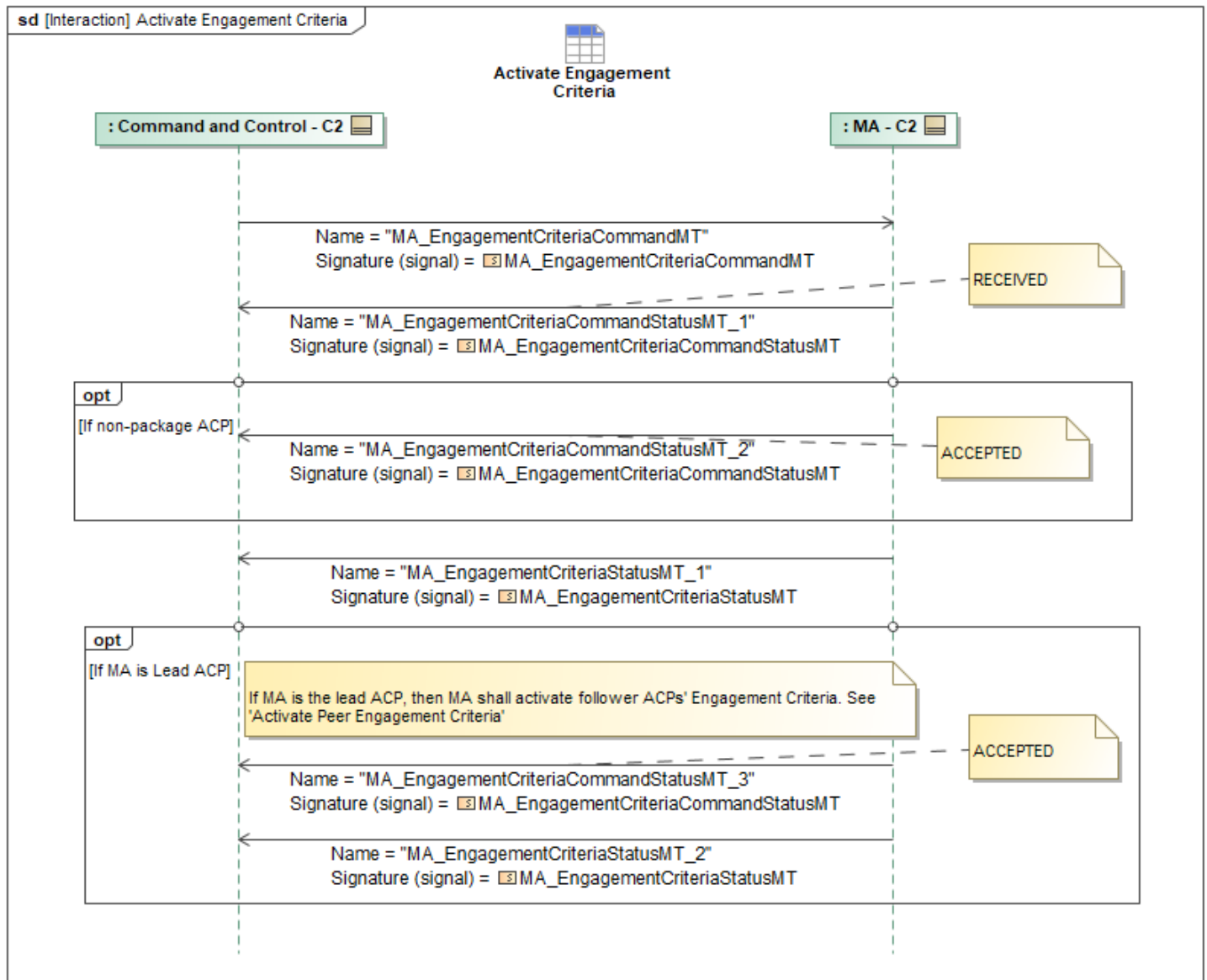


Figure 5-29: Activate Engagement Criteria

Table 5-28: Activate Engagement Criteria

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EngagementCriteriaCommandMT: MA_EngagementCriteriaCommandMT	None
MA_EngagementCriteriaCommandStatusMT_1: MA_EngagementCriteriaCommandStatusMT	None
MA_EngagementCriteriaCommandStatusMT_2: MA_EngagementCriteriaCommandStatusMT	None
MA_EngagementCriteriaCommandStatusMT_3: MA_EngagementCriteriaCommandStatusMT	None
MA_EngagementCriteriaStatusMT_1: MA_EngagementCriteriaStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EngagementCriteriaStatusMT_2: MA_EngagementCriteriaStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7 C2 Generic Behaviors Package

5.2.7.1 C2 Commanded Geozone

This diagram contains the collection of data messages used when C2 commands the state of any Op* message (e.g., *MA_OpZoneMT*, *MA_OpVolumeMT*, etc.) currently stored on MA, independent of mission plan. When the state of a given Op* message is "ACTIVE", the constraints imposed by said message must be considered by MA.

Upon receipt of an *MA_OpCommandMT*, MA shall store the command locally. The *MA_OpCommandMT* includes a reference to an existing Op* message (*OpID*), an indication of whether the command is to activate or de-activate (*CommandedState*), and any relevant conditions that must be met in order for the command to execute (*ConstraintsOverride*). Additionally, the *Ranking* field allows MA to resolve conflicting command messages through comparative ranking. MA acknowledges a received command by sending *MA_OpCommandStatusMT_1* with *CommandProcessingState* of "RECEIVED". MA then evaluates the conditions and ranking of the received *MA_OpCommandMT* and sends an *MA_OpCommandStatusMT_2* message with *CommandProcessingState* of "ACCEPTED" if successful. Otherwise, *MA_OpCommandStatusMT_3* is sent with the *CommandProcessingState* of "REJECTED", and the accompanying *CommandProcessingStateReason* populated with the reason for the rejection (e.g., "RANKING").

MA continuously monitors the conditions specified in *ConstraintsOverride* for all stored *MA_OpCommandMT* messages. If there are no conditions specified, or all conditions are met, MA changes the state of the referenced Op* message and sends an *MA_OpStatusMT* notifying C2 of the updated status (*CurrentState*) of the Op* message and the command that triggered the change (*CommandID*).

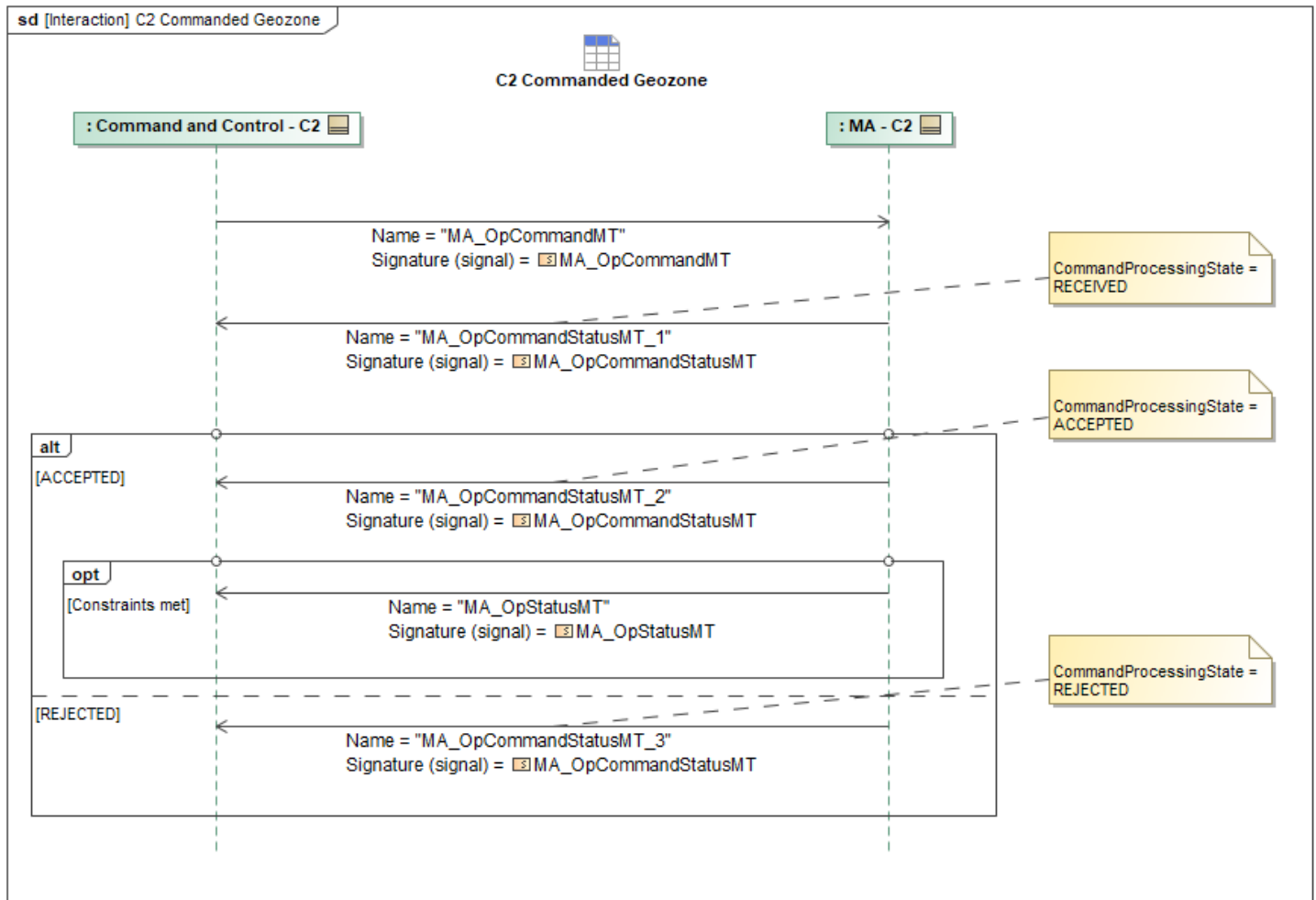


Figure 5-30: C2 Commanded Geozone

Table 5-29: C2 Commanded Geozone

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpCommandMT: MA_OpCommandMT	OpID: OpID_ChoiceType
MA_OpCommandStatusMT_1: MA_OpCommandStatusMT	None
MA_OpCommandStatusMT_2: MA_OpCommandStatusMT	None
MA_OpCommandStatusMT_3: MA_OpCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_OpStatusMT: MA_OpStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.2 C2 Plan Approval Status

The C2 Plan Approval Status sequence diagram specifies the interaction of an ACP and the C2 authority where the ACP has received approval from C2 already for the plans indicated, and the ACP

is responding with the confirmation that the plans have now been designated as approved. While the Approval Status specifically statuses the response of the Approval Request, the plan approval status messages are a status of the approval state of the plan itself, and they are utilized by the system to confirm individual plans approval or rejection status. As each executable plan is approved, the ACP sends the plan approval status to C2 via the *TaskPlanApprovalStatusMT*, *MissionPlanApprovalStatusMT*, *RoutePlanApprovalStatusMT*, and *ActivityPlanApprovalStatusMT* messages. For each of the messages sent, the *PlanID* and *PlanApprovalState* is populated with the applicable plan UUID and approval state, respectively.

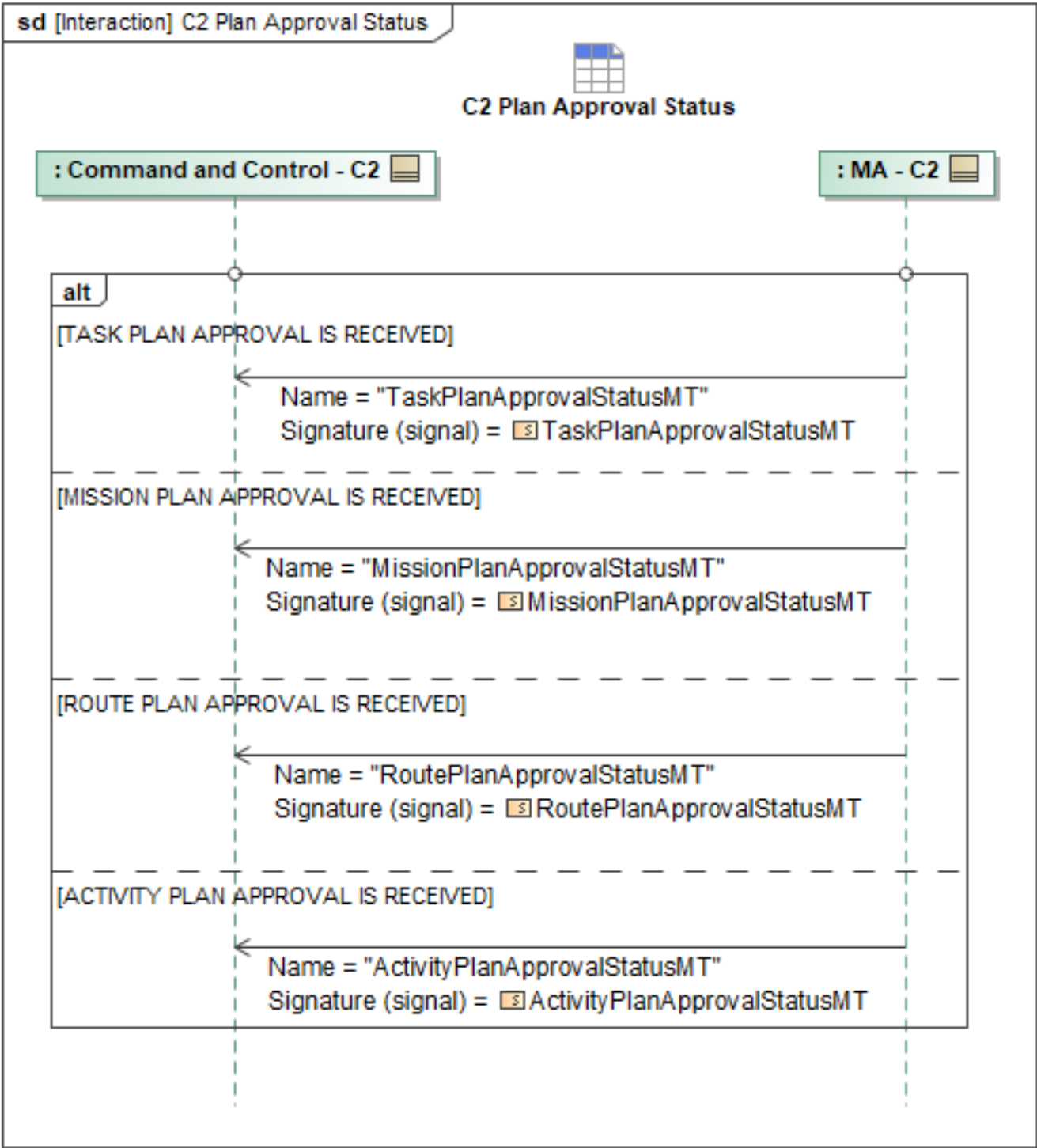


Figure 5-31: C2 Plan Approval Status

Table 5-30: C2 Plan Approval Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityPlanApprovalStatusMT: ActivityPlanApprovalStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MissionPlanApprovalStatusMT: MissionPlanApprovalStatusMT	None
RoutePlanApprovalStatusMT: RoutePlanApprovalStatusMT	None
TaskPlanApprovalStatusMT: TaskPlanApprovalStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.3 C2 Publish Control Status

Flight Autonomy (FA), through the Vehicle Interface (VI), will periodically publish the *ControlStatusMT* message to Mission Autonomy (MA), indicating the current controlling system and service. MA sends *ControlStatusMT* periodically to relay the *PRIMARY_CONTROLLER* and *SECONDARY_CONTROLLER* designations to C2 for monitoring. This is depicted through the C2 Publish Control Status interaction, where the status is indicated by specifying through *ControlStatusMT/MessageData/ControlType/CapabilityControl/PrimaryController* and *ControlStatusMT/MessageData/ControlType/CapabilityControl/SecondaryController*.

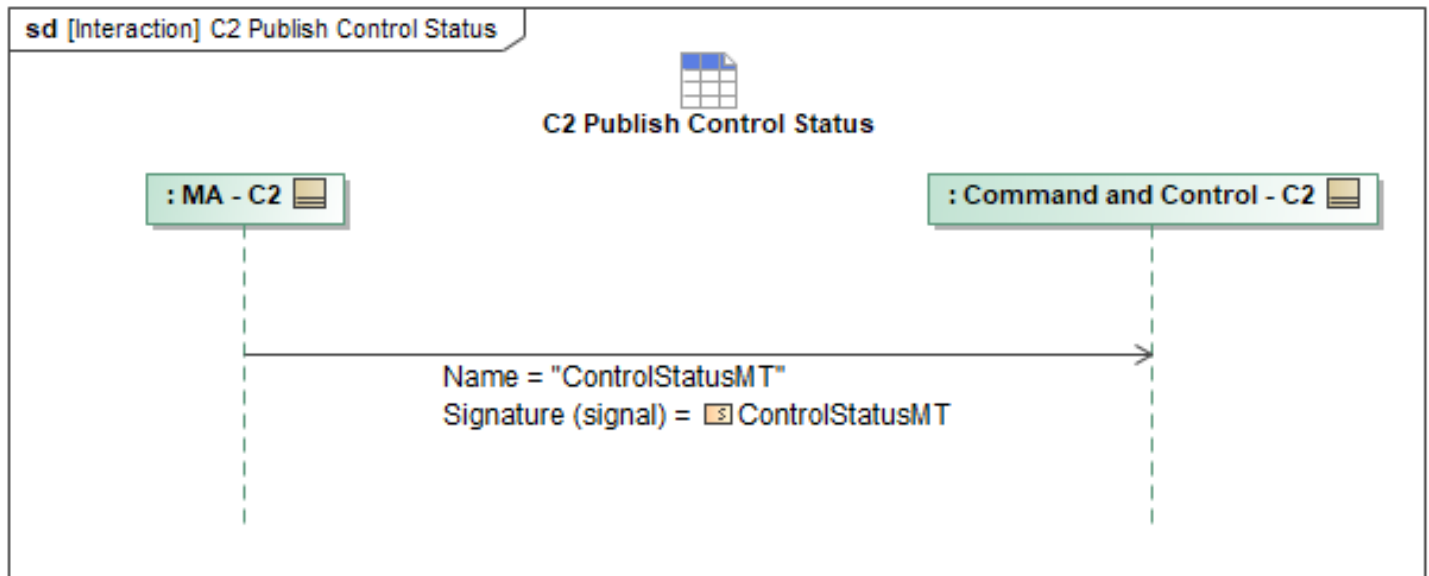


Figure 5-32: C2 Publish Control Status

Table 5-31: C2 Publish Control Status

Message Name (Name:Type)	Required Fields (Name:Type)
ControlStatusMT: ControlStatusMT	PrimaryController: PrimaryControllerType SecondaryController: SecondaryControllerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.4 C2 Receives Geozone Status

This diagram contains the data message used when the state of any Op* message (e.g., *MA_OpZoneMT*, *MA_OpVolumeMT*, etc.) is changed. This may occur as a result of an *MA_OpCommandMT* or *MA_OpPlanMT* once the constraints on the message are met.

The *MA_OpStatusMT* shall include a reference to the Op* message that changed state (*OpID*), the updated state of the Op* message (*CurrentState*), and a reference to the command or plan that caused the state change (*CommandID*).

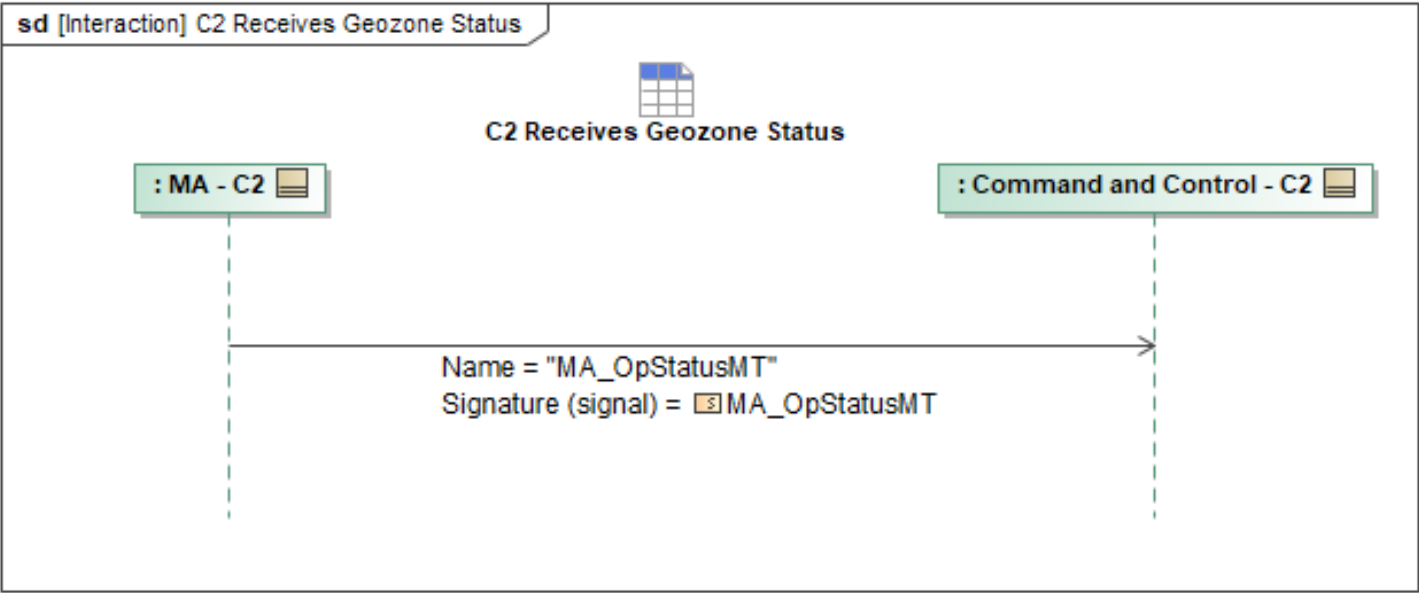


Figure 5-33: C2 Receives Geozone Status

Table 5-32: C2 Receives Geozone Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpStatusMT: MA_OpStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.5 C2 Receives Message Acknowledgment

C2 Acknowledgment Configurations sequences defines which messages C2 would expect MA to respond with an acknowledgment of receipt. If MA receives a configured message, MA will send a *MA_SystemNotificationMT* with the *NotificationState* set to *CONFIRMED*, *Source* set to its own *SystemID*, *Severity* set to *INFORMATIONAL*, *SystemSubjectIDs* set to its own *SystemID*, *SystemPerspective* set to *SOURCE*, and *AssociatedMessage* set to the *MessageType* of the received message and the *AssociatedID* of the applicable ID in the received message. The *MA_SystemNotificationMT* can also be sent at limited periodic rates for acknowledgments based on the Message Acknowledgment Configuration where it would group together all received *AssociatedMessages* over the periodic time that was configured. This is done by adding multiple entries to the *AssociatedMessage* field for each message. The *MA_SystemNotificationMT* would only be sent at that periodic rate if there was a received message during that time.

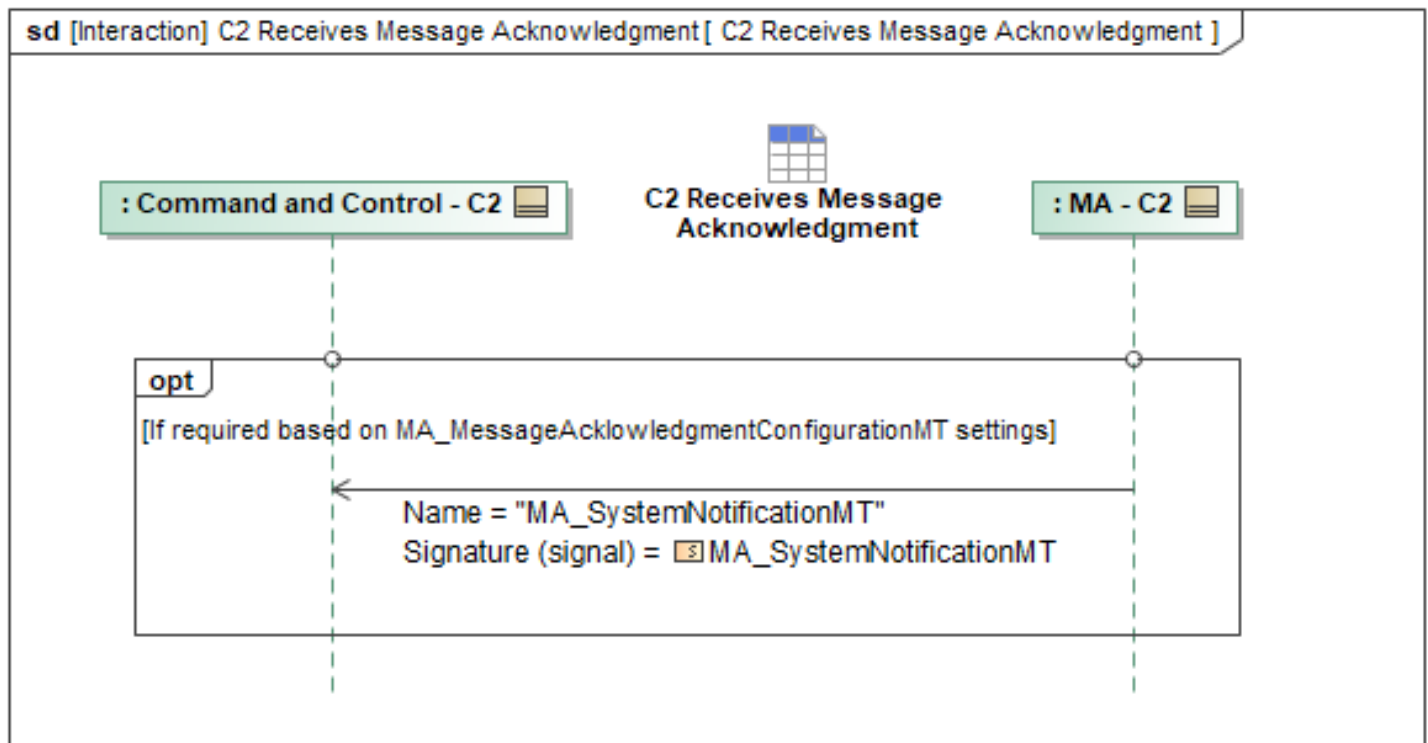


Figure 5-34: C2 Receives Message Acknowledgment

Table 5-33: C2 Receives Message Acknowledgment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.6 C2 Sends Message Acknowledgment

C2 Acknowledgment Configurations sequences defines which messages MA would expect C2 to respond with an acknowledgment of receipt. If C2 receives a message which it is configured to acknowledge receipt of, C2 will send a *MA_SystemNotificationMT* with the *NotificationState* set to *CONFIRMED*, *Source* set to its own *SystemID*, *Severity* set to *INFORMATIONAL*, *SystemSubjectIDs* set to its own *SystemID*, *SystemPerspective* set to *SOURCE*, and *AssociatedMessage* set to the *MessageType* of the received message and the *AssociatedID* of the applicable ID in the received message. The *MA_SystemNotificationMT* can also be sent at limited periodic rates for acknowledgments based on the Message Acknowledgment Configuration where it would group together all received *AssociatedMessages* over the periodic time that was configured. This is done by adding multiple entries to the *AssociatedMessage* field for each message. The *MA_SystemNotificationMT* would only be sent at that periodic rate if there was a received message during that time.

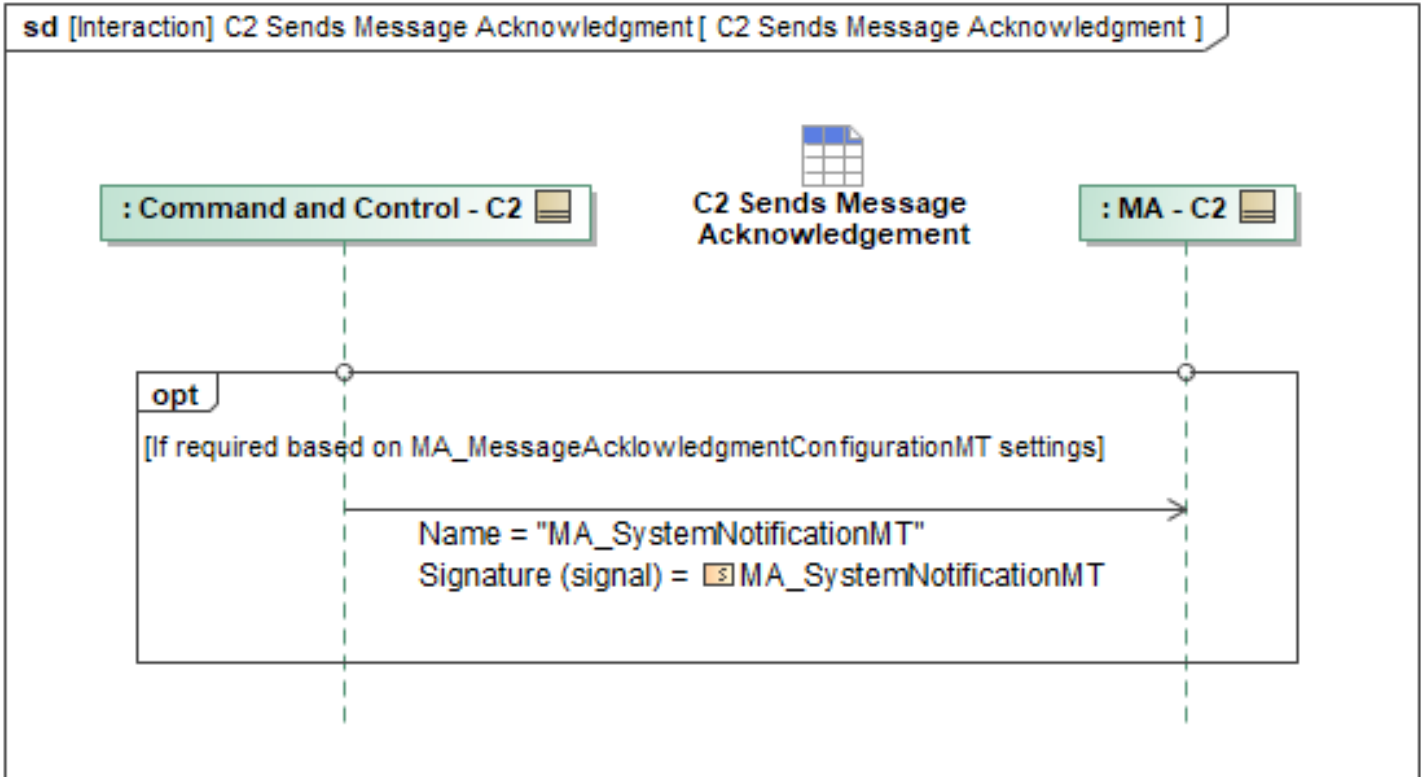


Figure 5-35: C2 Sends Message Acknowledgment

Table 5-34: C2 Sends Message Acknowledgment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.7 Control Mode Authorization

C2 should be made aware of the capabilities that MA has been granted by Flight Autonomy. For information on the VI-MA interaction, see Control Authorization in the Vehicle Interface for more details. MA communicates with C2 how MA can control the vehicle as MA sends *MA_FlightCapabilityMT* periodically. The *MA_FlightCapabilityMT* message requires the *FlightCapabilityPerformanceProfile* field be populated with the appropriate subtypes of the capabilities advertised and any parameters need by MA for commanding each type present. MA sends the *MA_FlightCapabilityStatusMT* message periodically to communicate the availability of control modes. Both of these messages should be passed to C2 nodes when connecting to MA and *MA_FlightCapabilityStatusMT* should be forwarded along to C2 nodes whenever there is a change in flight capabilities.

Once the required capabilities are available, C2 can issue commands to the MA via the Direct HSA/CSA Command use case or the Direct Waypoint Command interaction. The *MA_FlightCommandMT* message within these use cases carries control mode-specific data, as

described in later sections. ***MA_FlightCommandMT*** is not used to set or command a formation on the C2–MA interface; a formation is planned and commanded via the Task path (***MA_TaskMT*** and ***MA_TaskCommandMT***).

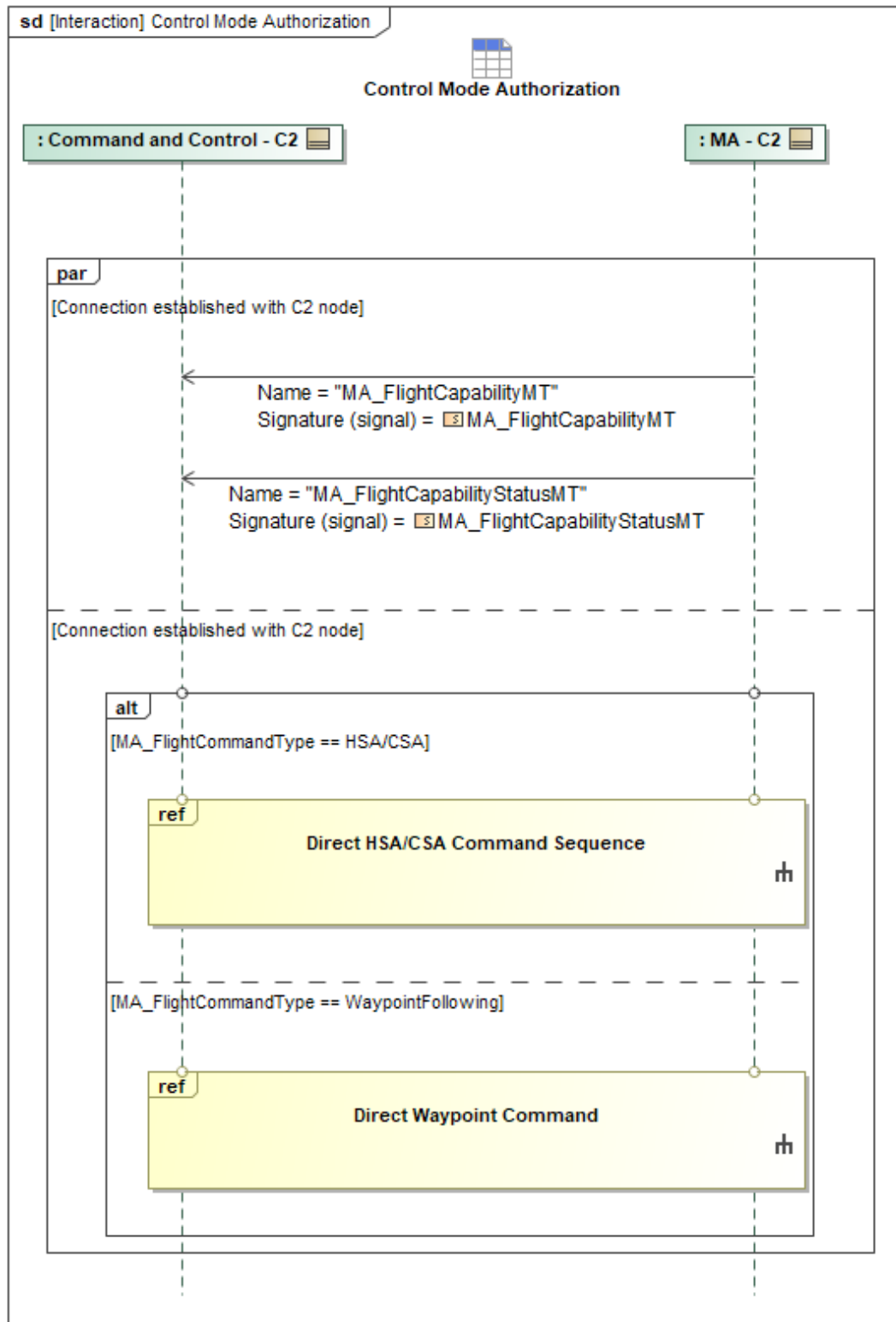


Figure 5-36: Control Mode Authorization

Table 5-35: Control Mode Authorization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.8 Direct HSA/CSA Command Sequence

The C2 system can send heading/speed/altitude or course/speed/altitude (HSA/CSA) commands to MA via the *MA_FlightCommandMT* message. The C2 system sends *MA_FlightCommandMT* to command the system and invokes a previously advertised capability. Note: *MA_FlightCommandMT* is not used to set or command a formation on the C2–MA interface; a formation is planned and commanded via the Task path (*MA_TaskMT* and *MA_TaskCommandMT*). MA responds with *MA_FlightCommandStatusMT*, either approving or rejecting the command. If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* will specify the *CannotComplyDetails* field to provide further details. VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and Flight field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command. The break may also occur if the *Activity State* is *COMPLETED* or *FAILED*.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until either control is taken away from MA or MA relinquishes control.

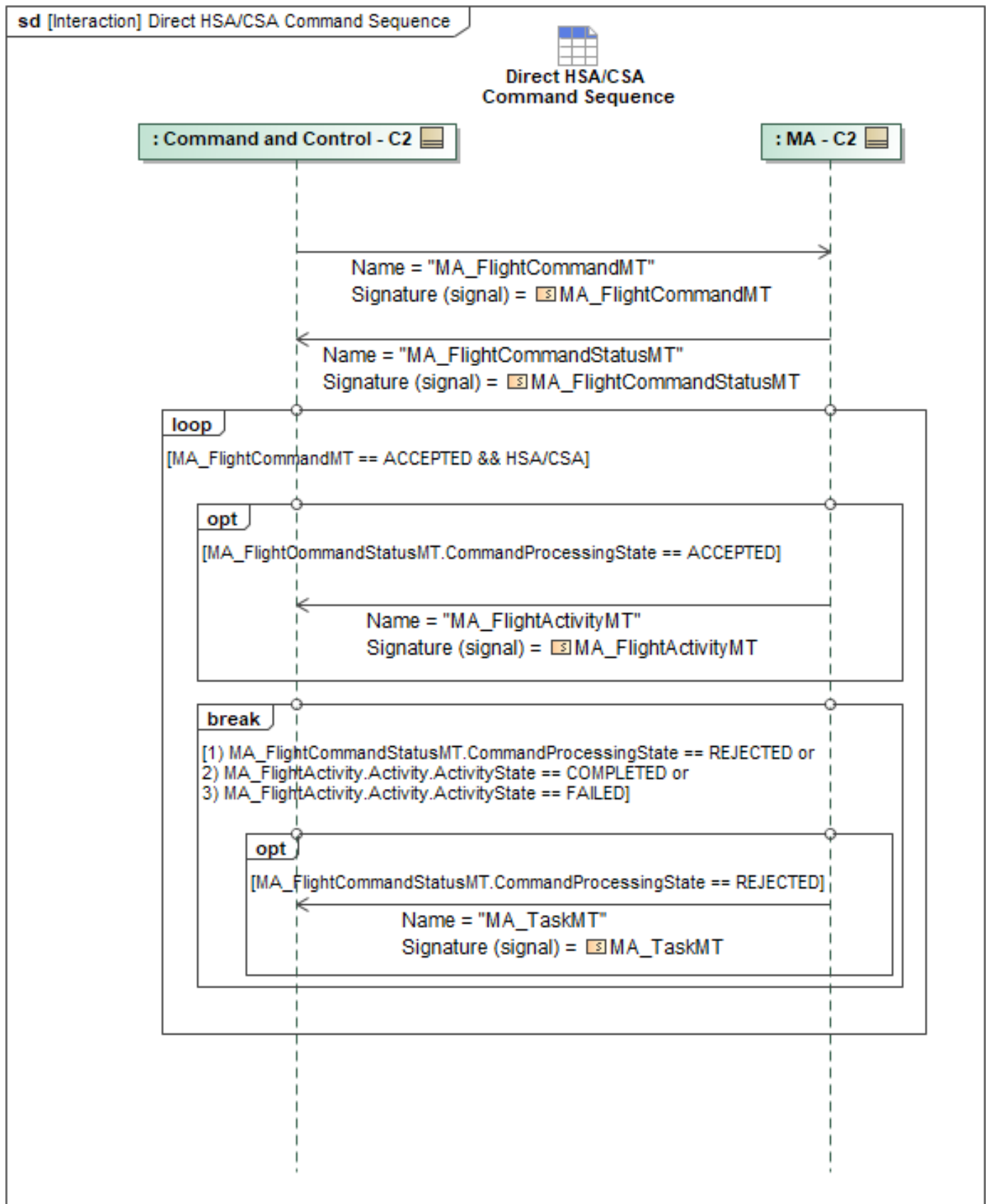


Figure 5-37: Direct HSA/CSA Command Sequence

Table 5-36: Direct HSA/CSA Command Sequence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	HSA_CSA: MA_HSA_CSA_Type
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.9 Direct Waypoint Command

For Direct C2, before the communication of new waypoints begins, MA needs to transmit to C2 its capabilities through the Capability Publication sequence so that MA can let C2 know what it can control. These capabilities can then be invoked via the *MA_FlightCommandMT* message to execute a waypoint command. Note: *MA_FlightCommandMT* is not used to set or command a formation on the C2–MA interface; a formation is planned and commanded via the Task path (*MA_TaskMT* and *MA_TaskCommandMT*). MA then responds with an *MA_FlightCommandStatusMT* accepting or rejecting the command. If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* message will specify the *CannotComplyDetails* field to provide further details. VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command. The break may also occur if the *Activity State* is *COMPLETED* or *FAILED*.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until either control is taken away from MA, or MA relinquishes control.

sd [Interaction] Direct Waypoint Command

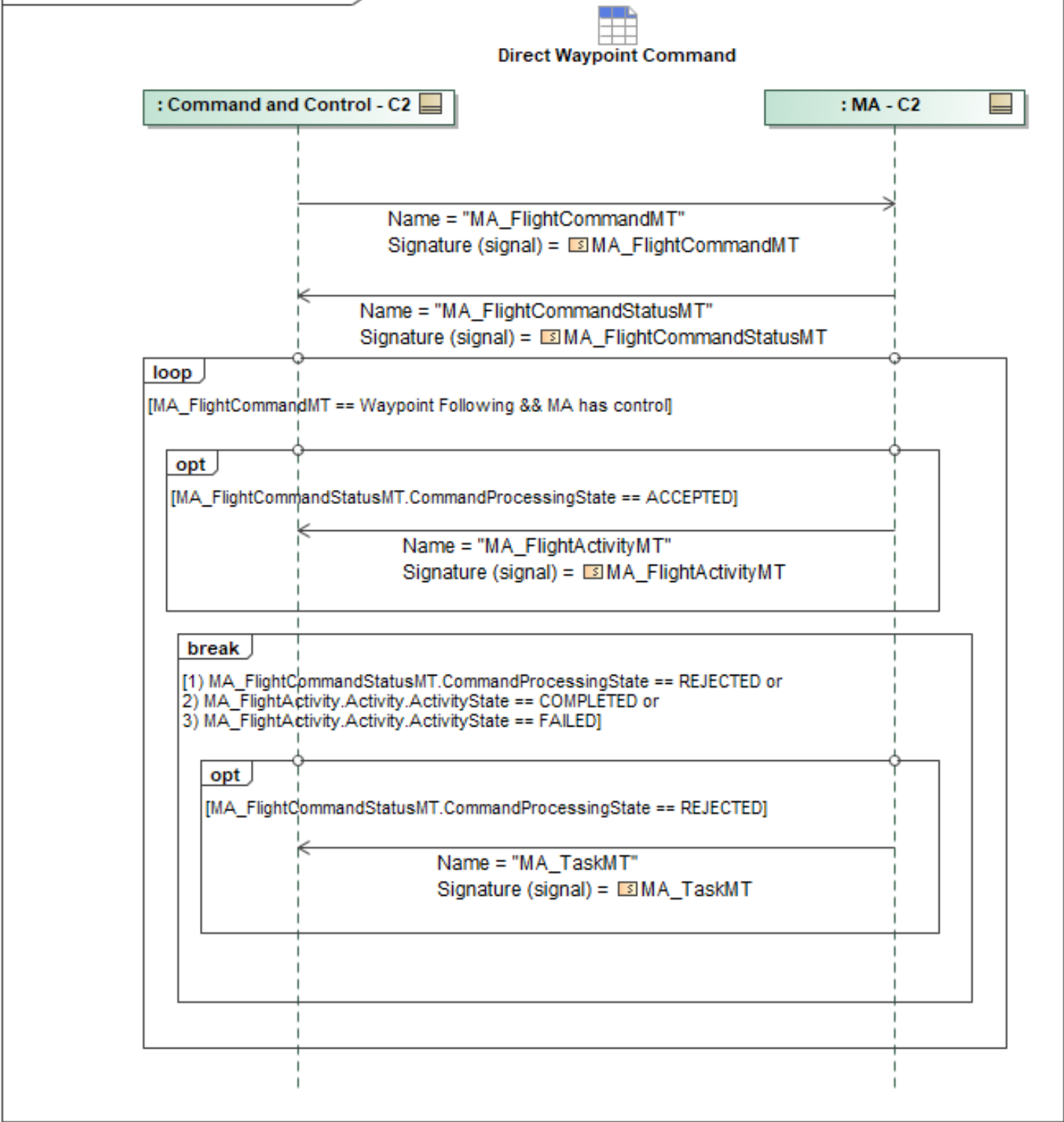


Figure 5-38: Direct Waypoint Command

Table 5-37: Direct Waypoint Command

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	WaypointFollowing: MA_WaypointFollowingType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.10 Planned HSA/CSA Command Sequence

To direct a vehicle to follow an HSA/CSA flight control mode via Planned C2, *MA_TaskMT* should be used with *FlightTaskType* populated to define the *HSA_CSA* flight control mode. C2 first sends out a *MA_MissionPlanMT*, followed by an *MA_TaskPlanMT* and *MA_TaskMT*.

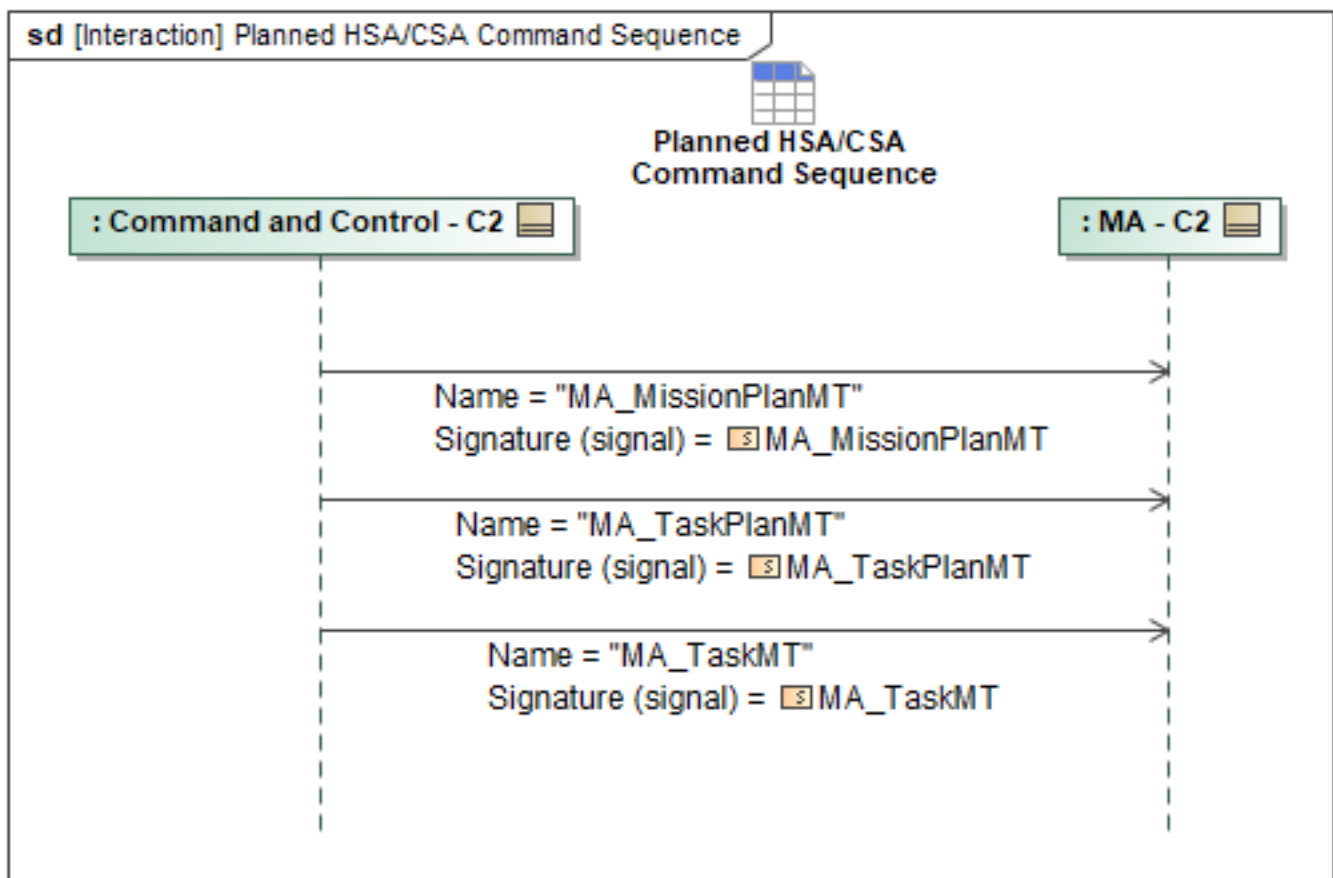


Figure 5-39: Planned HSA/CSA Command Sequence

Table 5-38: Planned HSA/CSA Command Sequence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	HSA_CSA: MA_HSA_CSA_Type
MA_TaskPlanMT: MA_TaskPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.11 Planned Waypoint Command

Waypoint following, alternatively referred to as route following, is a control mode where a C2 node provides MA a series of waypoints describing a desired flight path. Waypoint behavior may include simple commands such as fly-by or fly-through; complex commands including payload action and loiter/hold; or bounded by constraints such as vehicle attitude or timing.

For Planned C2, system can upload multiple routes to MA before selecting one to execute. C2 starts by sending the *MA_MissionPlanMT* with the route referenced in the associated *SubPlans*. A well-conditioned *MA_RoutePlanMT* message will simply include an *MA_RoutePlanType* for MA to execute.

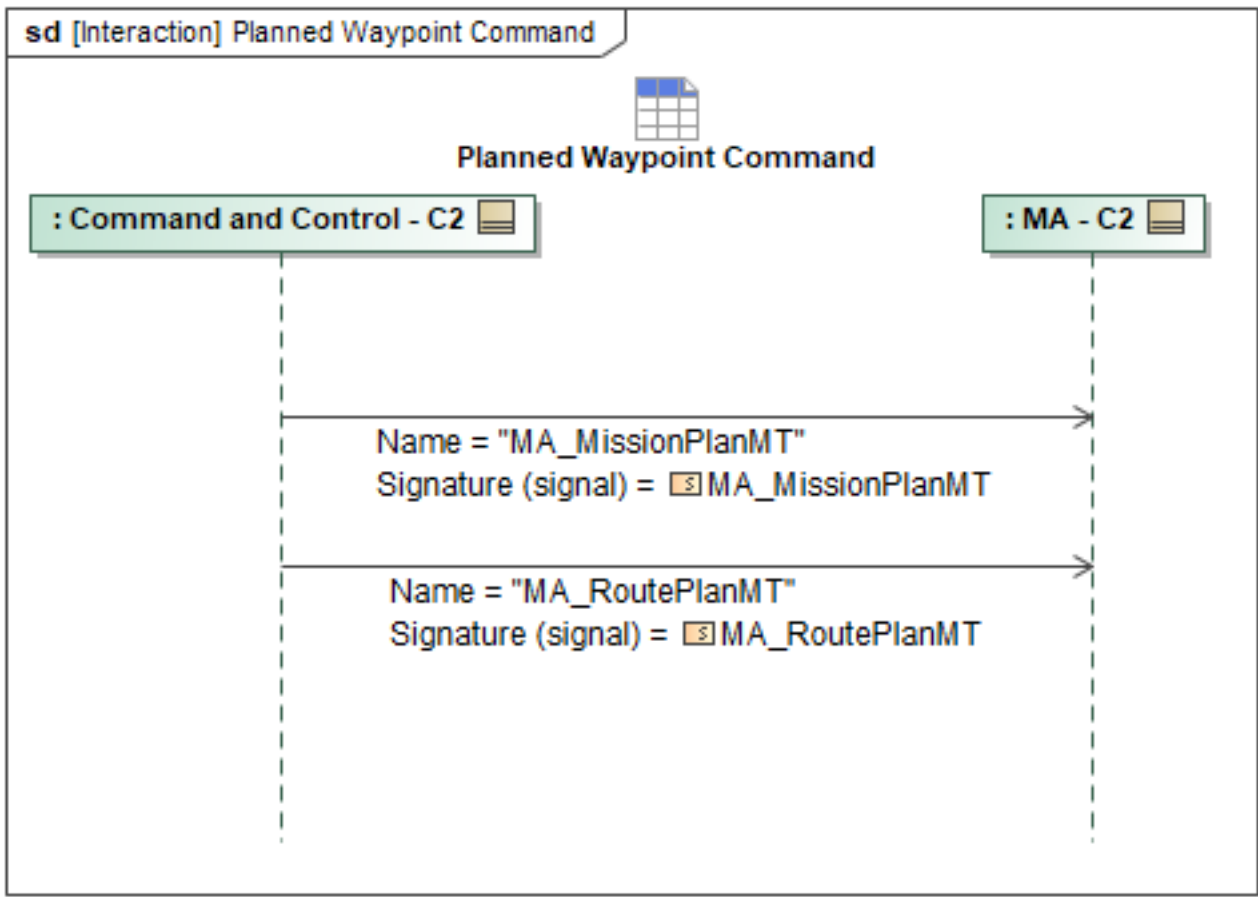


Figure 5-40: Planned Waypoint Command

Table 5-39: Planned Waypoint Command

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	None
MA_RoutePlanMT: MA_RoutePlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.12 Receive Execution Status

MA will periodically provide updates to C2 on the status of actively executing or queued actions, tasks, plans, routes, and missions.

MA will send *MA_ActionStatusMT* and *MA_TaskStatusMT* periodically to convey the status of the ongoing Action or Task. Both *MA_ActionStatusMT* and *MA_TaskStatusMT* contain the required *ExecutionState* field denoting the executing state. When a task or action is allocated to an entire Package, the Lead ACP shall aggregate status reporting by populating the *SystemID* or *ExecutingSystemID* field within *MA_TaskStatusMT* and *MA_ActionStatusMT* messages with the *PackageID* UUID. This represents the collective state of the team. In an off-nominal condition where 1 or more Package members have an off-nominal *Status, the Lead ACP shall revert to sending individual *StatusMT messages strictly for the subset of platforms that failed.

If C2 allocates a requirement to a specific subset of platforms (via discrete *SystemID* allocations rather than *PackageID*), the Lead ACP shall bypass package-level aggregation and return individual status messages for those tasked members.

If MA is executing a route, it will send *RoutePlanExecutionStatusMT* and *RouteActivityPlanExecutionStatusMT* periodically containing the required field *PlanExecutionStatus.PlanExecutionState* and *PlanExecutionStatus.RouteActivityPlanExecutionStateType*, respectively, denoting the executing state.

If MA is executing a mission, MA will send *MissionPlanExecutionStatusMT* and *MA_MissionPlanActivationStatusMT* periodically containing the required field *PlanExecutionStatus.MissionPlanExecutionStateType* and *PlanActivationState.PlanActivationStateEnum*, respectively, denoting the executing state.

If MA is executing an activity plan or task plan, it will send *ActivityPlanExecutionStatusMT* and *TaskPlanExecutionStatusMT* periodically containing the required field *PlanExecutionStatus.ActivityPlanExecutionStateType* and *PlanExecutionStatus.TaskPlanExecutionStateType*, respectively, denoting the executing state.

If MA is executing a response, it will send *ResponsePlanExecutionStatusMT* periodically containing the required field *PlanExecutionStatus.ResponsePlanExecutionStateType* denoting the executing state of the response plan.

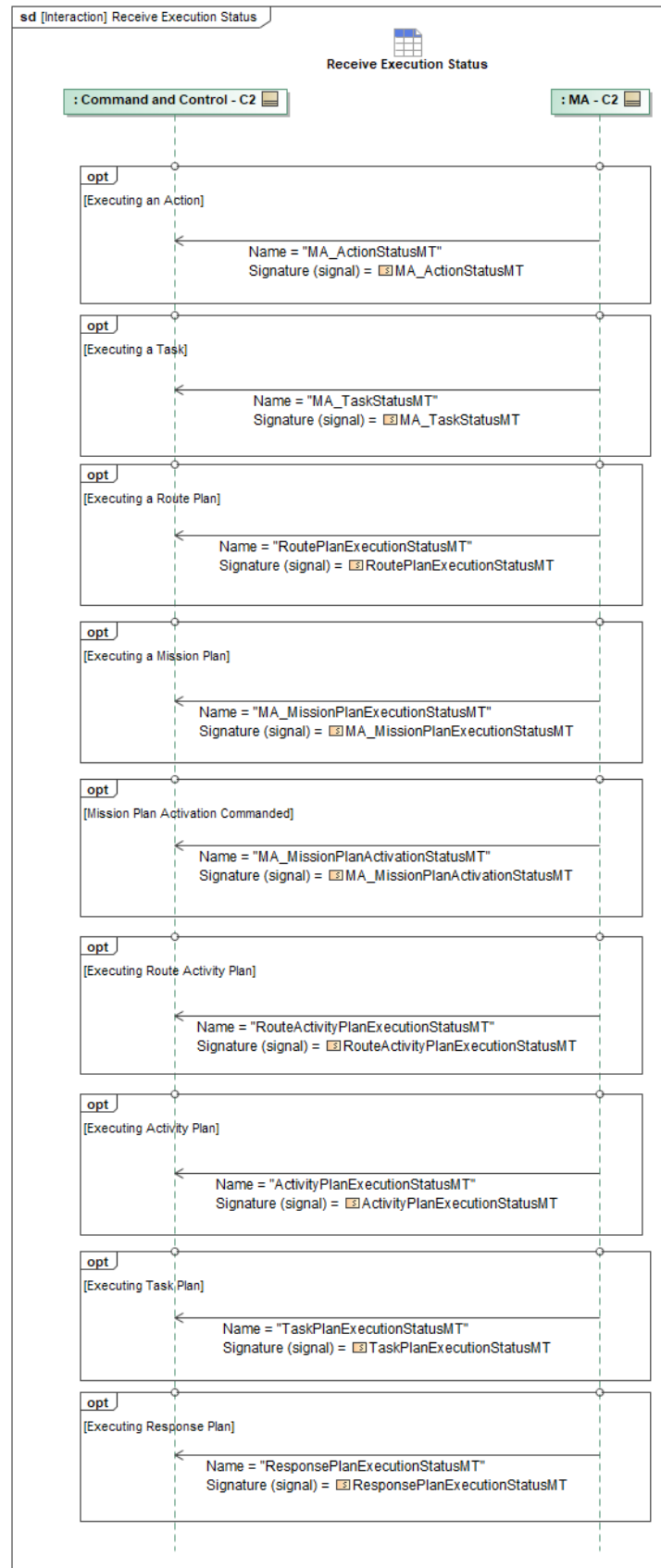


Figure 5-41: Receive Execution Status

Table 5-40: Receive Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityPlanExecutionStatusMT: ActivityPlanExecutionStatusMT	None
MA_ActionStatusMT: MA_ActionStatusMT	None
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	None
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None
ResponsePlanExecutionStatusMT: ResponsePlanExecutionStatusMT	None
RouteActivityPlanExecutionStatusMT: RouteActivityPlanExecutionStatusMT	None
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	None
TaskPlanExecutionStatusMT: TaskPlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.13 Report Departure Checkpoints to C2

MA will report departure checkpoints to C2 using an *OperatorNotificationMT* message. The diagram below shows all the *OperatorNotificationMT* signals associated with reporting departure checkpoints and the notes will indicate the details of the Title and any associations in the *OperatorNotificationMT* signals:

Runway Advancement

As an ACP releases the brakes and starts down the runway MA will report advancing down the runway to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *MA_LaunchTask* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *TASK* and the *AssociatedID* to the *MA_TaskMT:TaskID*.

Airborne

When an ACP has met the criteria for being airborne MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *MA_LaunchTask* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *TASK* and the *AssociatedID* to the *MA_TaskMT:TaskID*.

FA-MA Handoff

When an ACP has completed the FA-MA handoff MA will report that to C2 using an

OperatorNotificationMT message.

MA will associate *OperatorNotificationMT* with the *ControlStatusMT* message that indicates the handoff completed by setting *AssociatedMessageType* to *CONTROL_STATUS* and *AssociatedID* to the *ControlType:MissionControl:ControllerSystemID* which is the C2 that currently has mission control.

Departure Route Complete on Route Path

When an ACP has completed the departure route MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *MA_RoutePlanMT* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *ROUTE_PLAN* and the *AssociatedID* to the *MA_RoutePlan:RoutePlanID*.

Note: this sequence diagram could be used to report the completion of other route paths that are not departure routes

Departure Route Complete on Path Segment

When an ACP has completed the departure route segments, MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *MA_RoutePlanMT* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *ROUTE_PLAN* and the *AssociatedID* to the *MA_RoutePlan:PathSegmentID*.

Note: this sequence diagram could be used to report the completion of other route path segments that are not departure routes

Departure Orbit Join

When an ACP has started the departure orbit MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *Orbit MA_TaskMT* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *TASK* and the *AssociatedID* to the *MA_TaskMT:TaskID*.

Departure Orbit Completion

When an ACP has completed the departure orbit MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *Orbit MA_TaskMT* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *TASK* and the *AssociatedID* to the *MA_TaskMT:TaskID*.

Transition to Transit Completion on Route

When an ACP has completed the departure route MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *MA_RoutePlanMT* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *ROUTE_PLAN* and the *AssociatedID* to the *MA_RoutePlanMT:RoutePlanID*

Transition to Transit Completion on Route Path Segment

When an ACP has completed the route segment for Departure MA will report that to C2 using an *OperatorNotificationMT* message.

MA will associate this with the *MA_RoutePlanMT* by setting the *OperatorNotificationMT:AssociatedMessage:MessageType* to *ROUTE_PLAN* and the *AssociatedID* to the *MA_RoutePlanMT:RoutePlanID*.

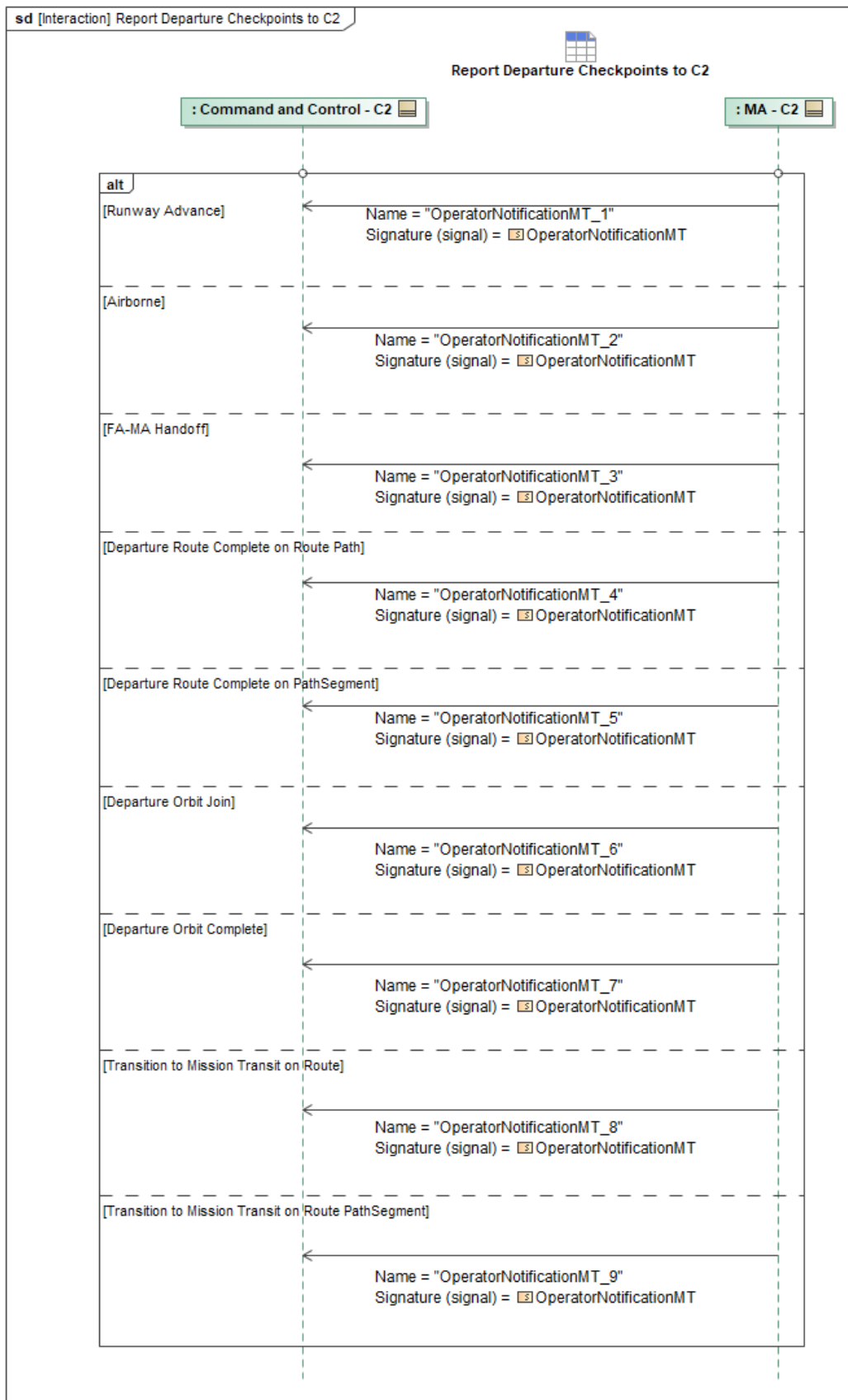


Figure 5-42: Report Departure Checkpoints to C2

Table 5-41: Report Departure Checkpoints to C2

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorNotificationMT_1: OperatorNotificationMT	TASK: AssociatedMessage: AssociatedMessageType AssociatedID: ID_Type
OperatorNotificationMT_2: OperatorNotificationMT	AssociatedID: ID_Type AssociatedMessage: AssociatedMessageType TASK:
OperatorNotificationMT_3: OperatorNotificationMT	AssociatedMessage: AssociatedMessageType CONTROL_STATUS: ControllerSystemID: SystemID_Type
OperatorNotificationMT_4: OperatorNotificationMT	ROUTE_PLAN: AssociatedMessage: AssociatedMessageType AssociatedID: ID_Type
OperatorNotificationMT_5: OperatorNotificationMT	AssociatedID: ID_Type AssociatedMessage: AssociatedMessageType ROUTE_PLAN:
OperatorNotificationMT_6: OperatorNotificationMT	TASK: AssociatedMessage: AssociatedMessageType AssociatedID: ID_Type
OperatorNotificationMT_7: OperatorNotificationMT	AssociatedID: ID_Type TASK: AssociatedMessage: AssociatedMessageType
OperatorNotificationMT_8: OperatorNotificationMT	AssociatedID: ID_Type AssociatedMessage: AssociatedMessageType ROUTE_PLAN:
OperatorNotificationMT_9: OperatorNotificationMT	AssociatedID: ID_Type AssociatedMessage: AssociatedMessageType ROUTE_PLAN:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.14 Request Generation of Route Metrics

RouteMetricsMT contains Route Metric Data and is stored or cached in data manager services which may have a database backend. This request to generate data is sent via the *RouteMetricsRequestMT* and responded to with *RouteMetricsRequestStatusMT* messages. Data management services respond with intermediate *RouteMetricsRequestStatusMT* messages with the *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and

"PROCESSING". When complete, the *RequestProcessingStateEnum* is set to "COMPLETED" and contains the requested data. When the original *RouteMetricsRequestStatusMT*'s optional field *ResultsInNative* is populated with the matching message type, the underlying data management services instead publishes the requested data in its original or raw format as a separate message and indicates *RouteDataRequestStatusMT* message's *RequestProcessingStateEnum* to "COMPLETED".

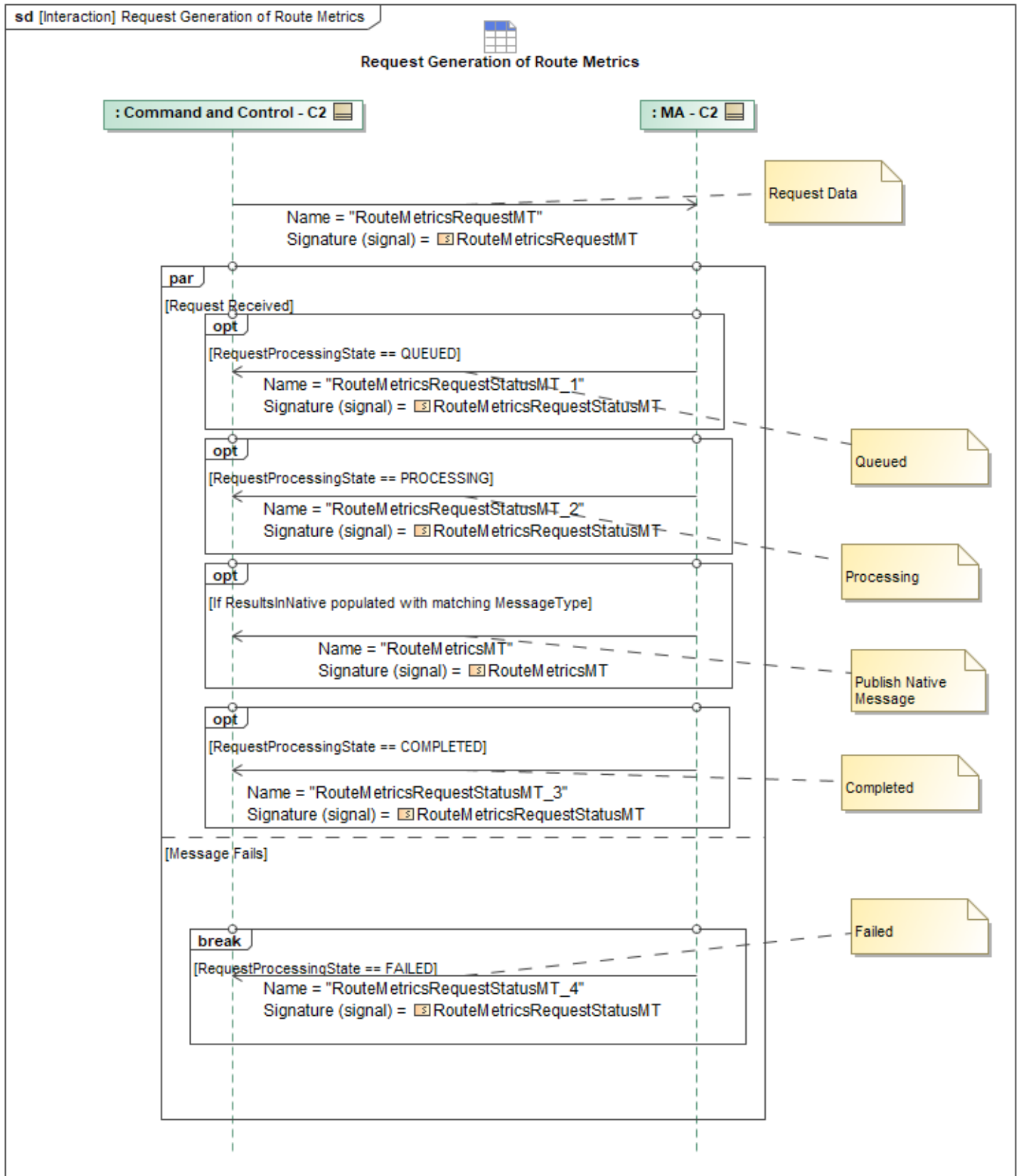


Figure 5-43: Request Generation of Route Metrics

Table 5-42: Request Generation of Route Metrics

Message Name (Name:Type)	Required Fields (Name:Type)
RouteMetricsMT: RouteMetricsMT	None
RouteMetricsRequestMT: RouteMetricsRequestMT	None
RouteMetricsRequestStatusMT_1: RouteMetricsRequestStatusMT	None
RouteMetricsRequestStatusMT_2: RouteMetricsRequestStatusMT	None
RouteMetricsRequestStatusMT_3: RouteMetricsRequestStatusMT	None
RouteMetricsRequestStatusMT_4: RouteMetricsRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.15 Respond to Approval Request

In the event that MA requires external approval, C2 can respond to the approval request sent by MA. MA publishes an *MA_ApprovalRequestMT*, and on receipt of this message, C2 will make an approval determination and respond to MA with *MA_ApprovalRequestStatusMT*.

MA does not require external approval in all situations. One example of a situation where external approval is required is the case where an *ApprovalPolicyMT* contains a *Policy* field that is set to *APPROVAL_REQUIRED*.

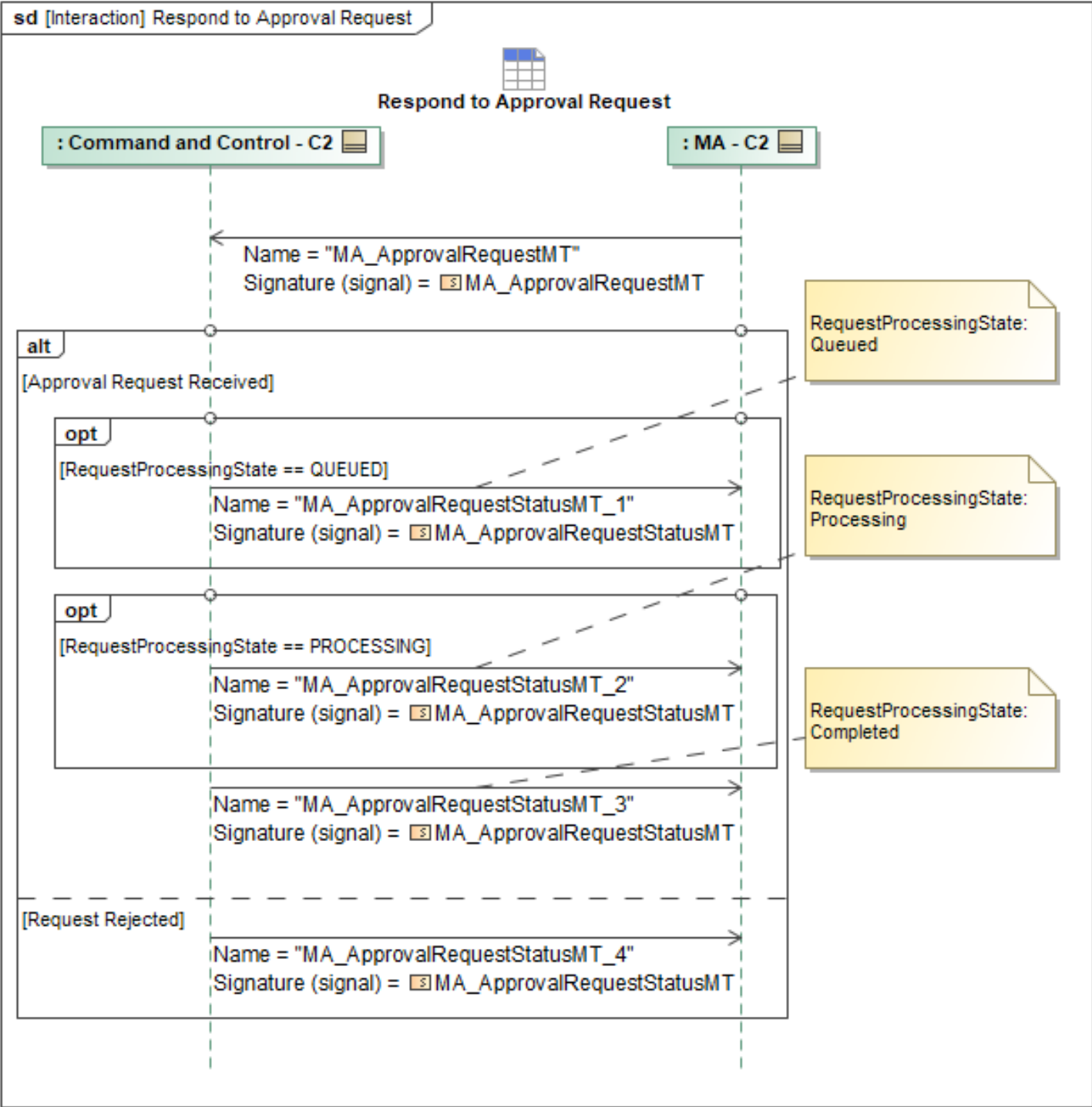


Figure 5-44: Respond to Approval Request

Table 5-43: Respond to Approval Request

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalRequestMT: MA_ApprovalRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalRequestStatusMT_1: MA_ApprovalRequestStatusMT	None
MA_ApprovalRequestStatusMT_2: MA_ApprovalRequestStatusMT	None
MA_ApprovalRequestStatusMT_3: MA_ApprovalRequestStatusMT	None
MA_ApprovalRequestStatusMT_4: MA_ApprovalRequestStatusMT	ApprovalRequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.16 Send Flight Phase Status

FA must establish a clear mechanism to publish the operational phase of flight to MA. This will also inform downstream systems, such as C2 and Peer, for monitoring. MA sends *MA_FlightActivityMT* periodically as it serves as the primary mechanism for FA to assert the current phase to MA, and then from MA to C2. The *FlightPhase* Boolean indicates the current phase of flight identified by Flight Autonomy, specifically its safety criticality. True indicates that the current flight phase is safety critical; False indicates a non-safety critical phase.

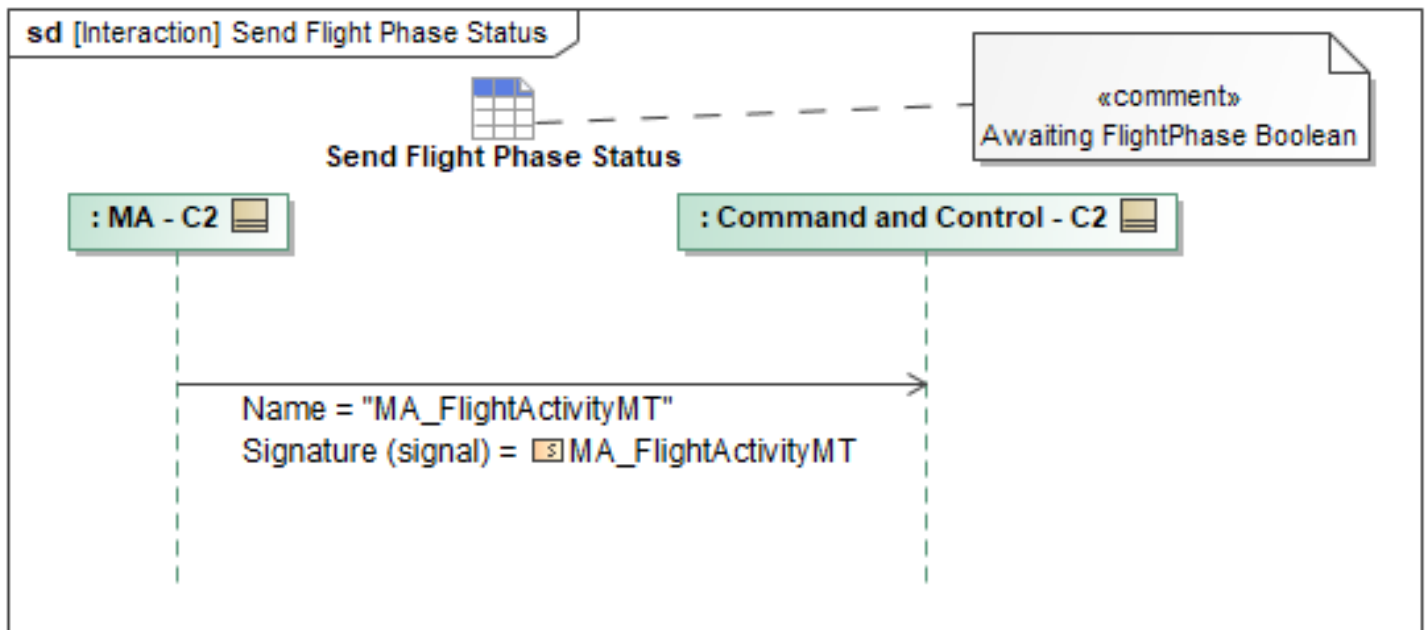


Figure 5-45: Send Flight Phase Status

Table 5-44: Send Flight Phase Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

5.2.7.17 Send Heartbeat

Both MA and C2 exchange SystemStatusMT periodically to convey a regular heartbeat. The C2 system should identify itself as such by setting *SystemStatusMT.PlatformStatus.CommandAndControlIndicator* to *True*. No response is required for heartbeat messages issued.

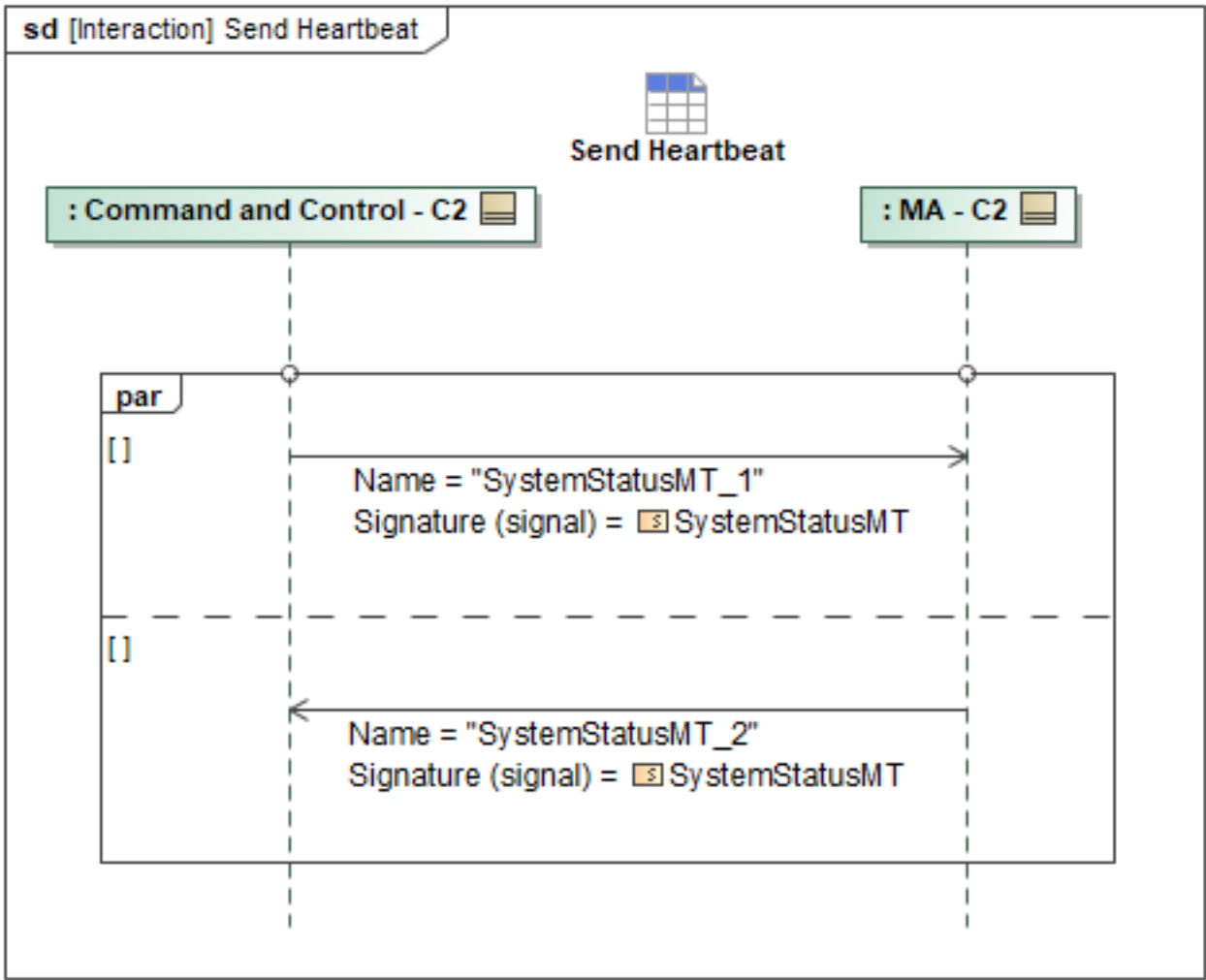


Figure 5-46: Send Heartbeat

Table 5-45: Send Heartbeat

Message Name (Name:Type)	Required Fields (Name:Type)
SystemStatusMT_1: SystemStatusMT	PlatformStatus: PlatformStatusType OperatorID: OperatorID_Type
SystemStatusMT_2: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.18 Send Subsystem Status to C2

MA can send *SubsystemStatusMT* periodically to inform C2 of the status of any subsystem on the platform.

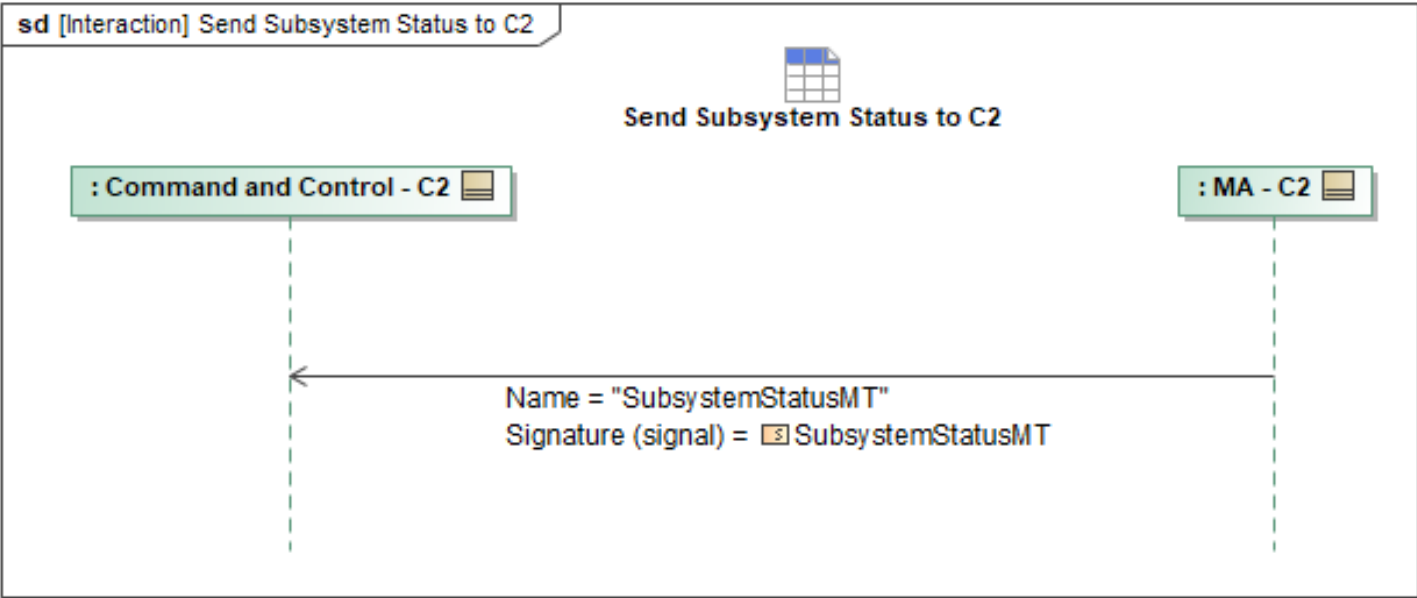


Figure 5-47: Send Subsystem Status to C2

Table 5-46: Send Subsystem Status to C2

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.19 Share Vehicle State Data

MA reports vehicle state data to C2 by sending *MA_FlightActivityMT*, *ComponentStatusMT*, *MA_PositionReportMT*, and *MA_NavigationReportMT* periodically. *MA_PositionReportMT* includes *InertialState*, defining *Position*, *DomainVelocity*, *GroundVelocity*, *CalibratedAirspeed*, and *Mach*. *DomainVelocity* represents the True Airspeed of the vehicle. *ComponentStatusMT* is optionally included if additional status data is required by C2. The transmission of these four messages being sent from MA to C2 will allow for the C2 instance(s) to gain a full picture of the state and status of the ACP.

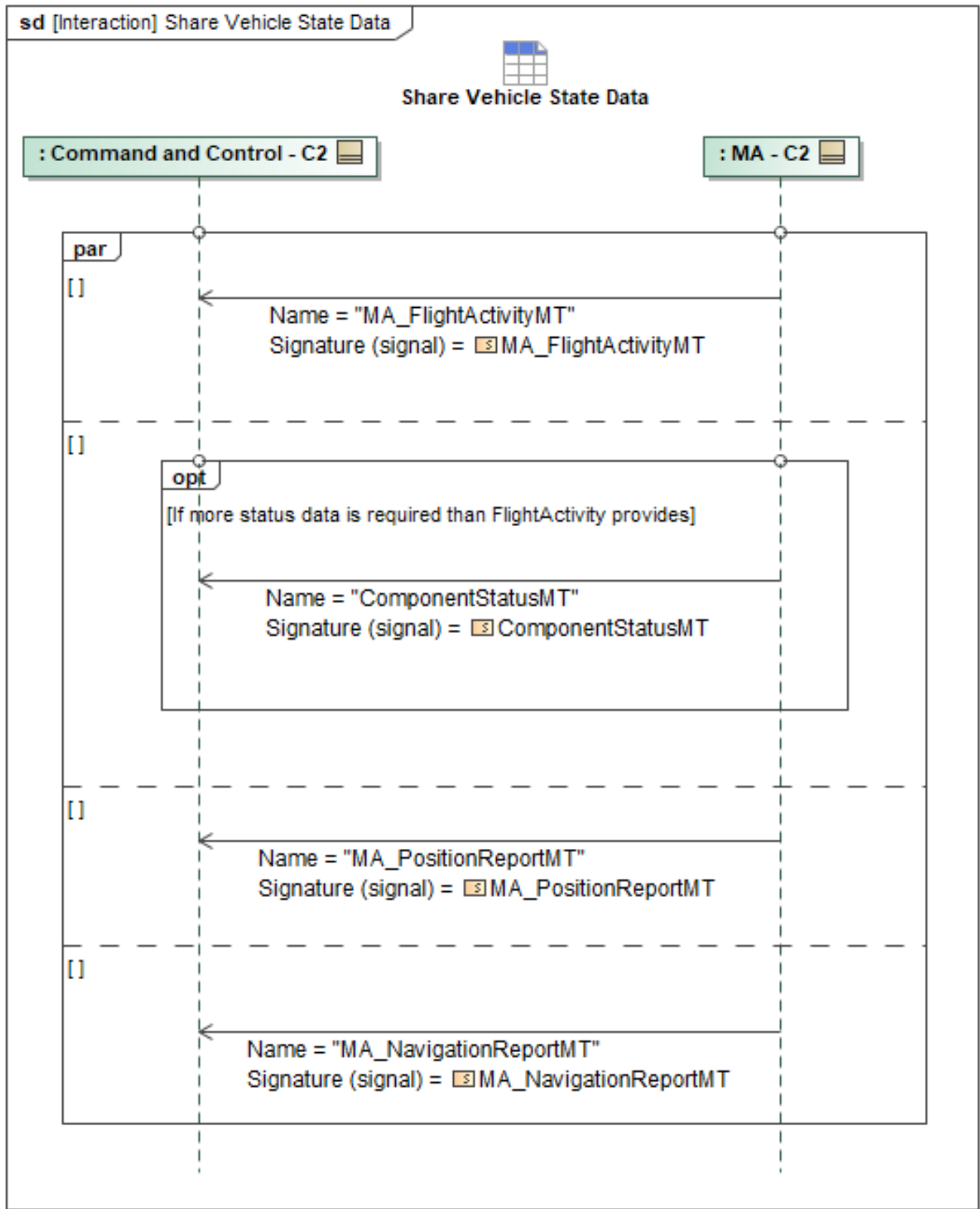


Figure 5-48: Share Vehicle State Data

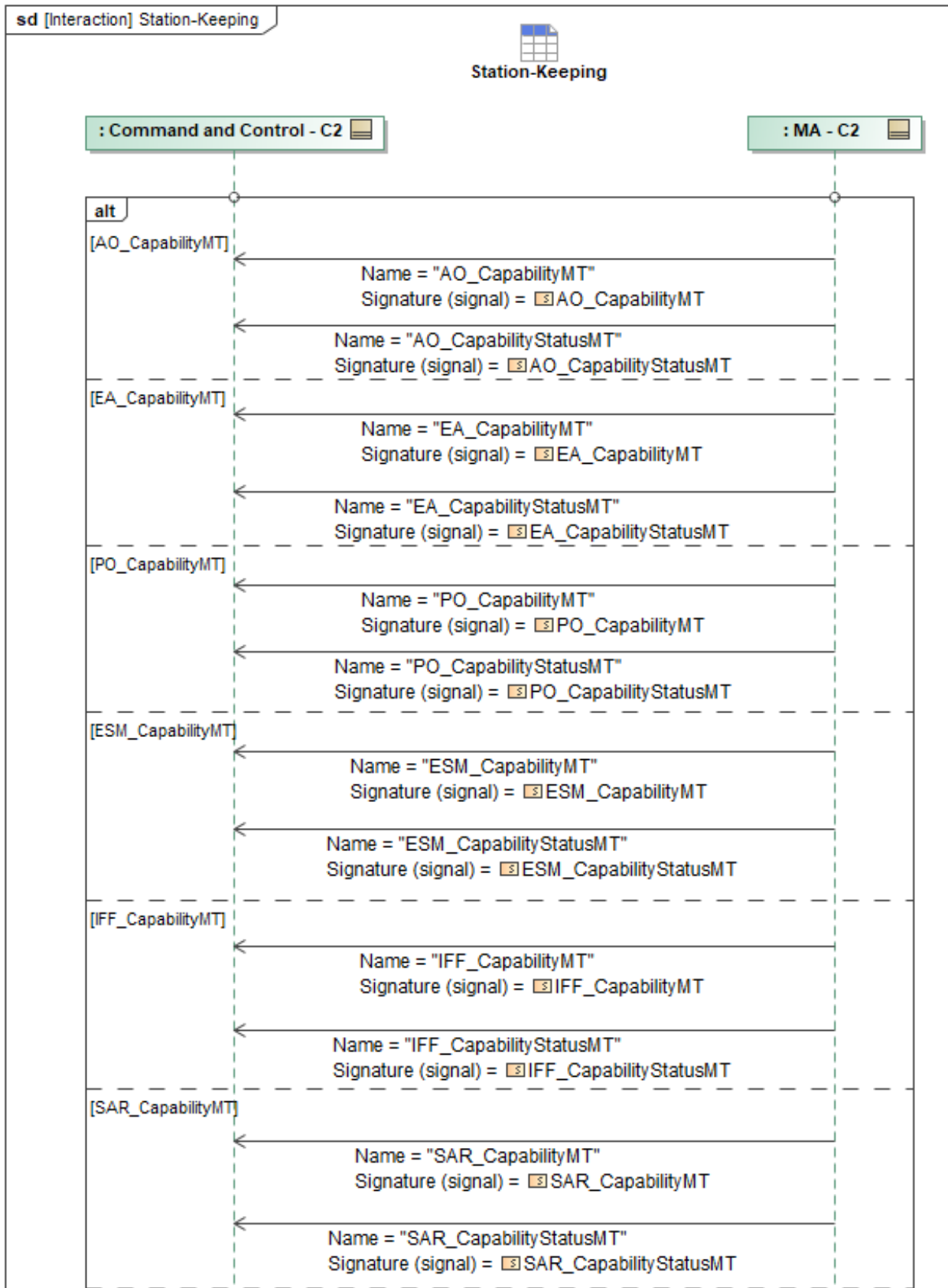
Table 5-47: Share Vehicle State Data

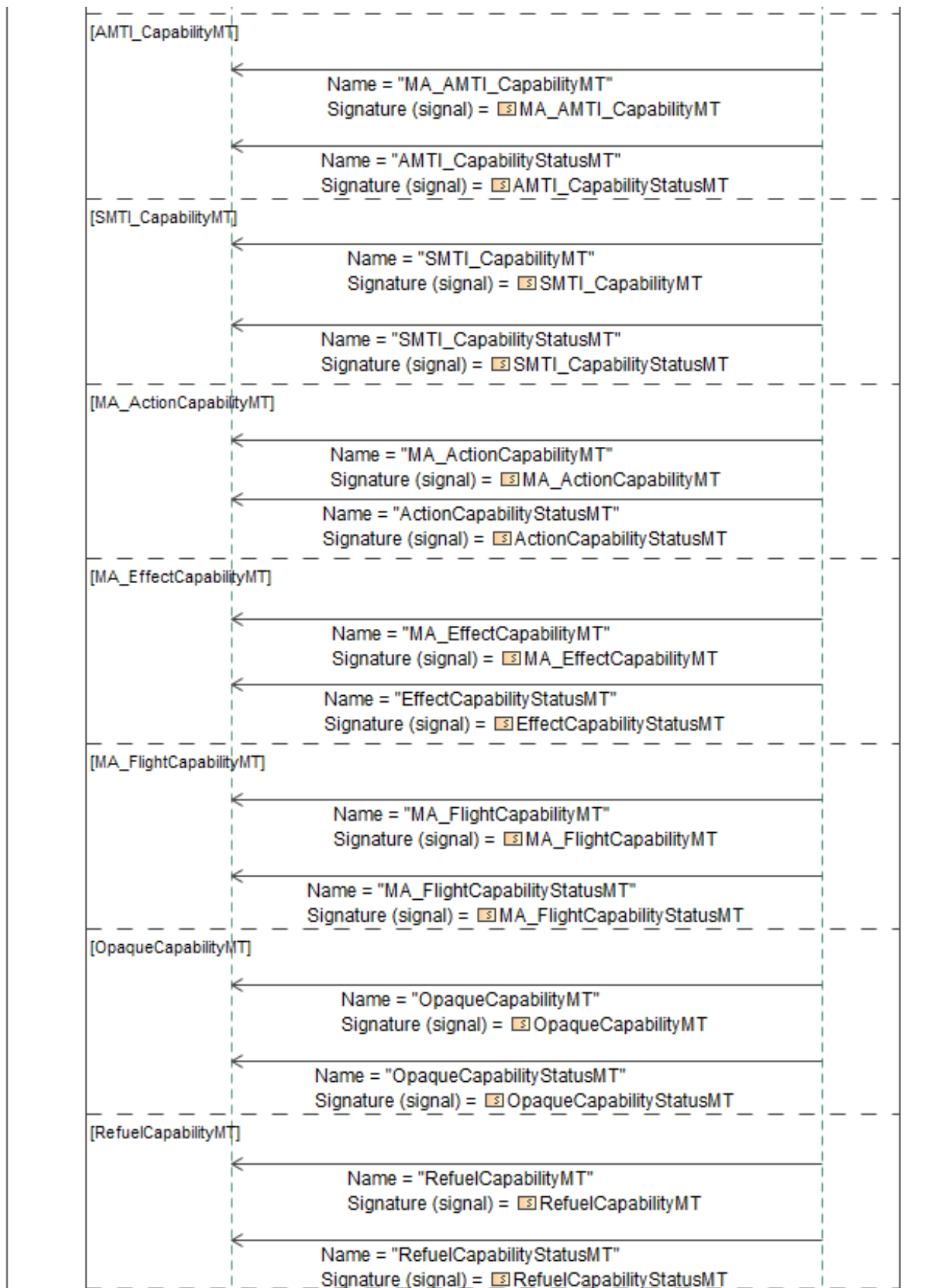
Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_NavigationReportMT: MA_NavigationReportMT	None
MA_PositionReportMT: MA_PositionReportMT	DomainVelocity: Velocity3D_Type GroundVelocity: Velocity2D_Type CalibratedAirspeed: SpeedType Mach: MachType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.20 Station-Keeping

The known set of station-keeping capabilities is presented in the following sequence diagram. A platform may advertise multiple of these capabilities (*[Capability]Activity* and *[Capability]Statuses*) based on platform configuration and each is sent periodically as a separate instance of this sequence.





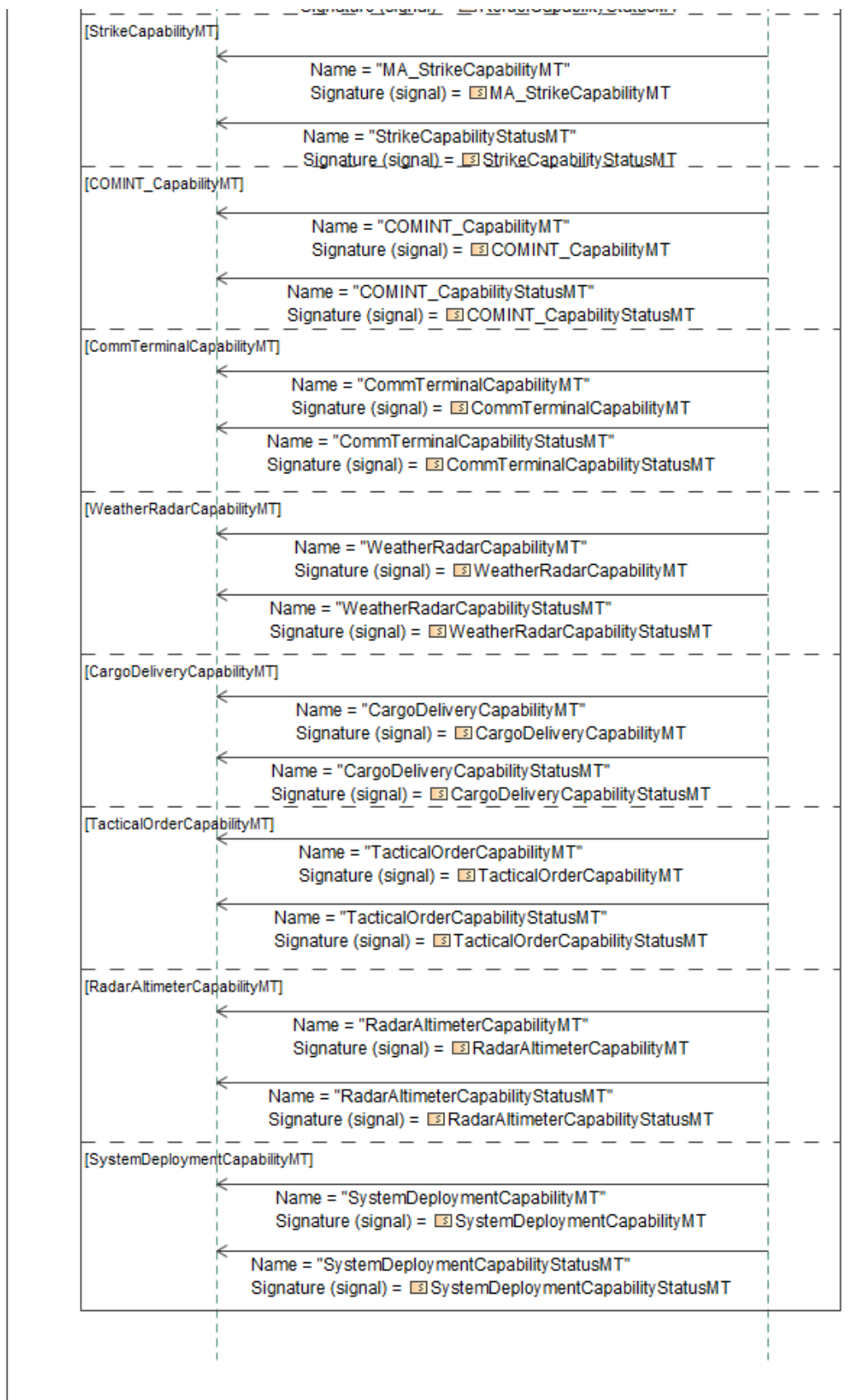


Figure 5-49: Station-Keeping

Table 5-48: Station-Keeping

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCapabilityStatusMT: ActionCapabilityStatusMT	None
AMTI_CapabilityStatusMT: AMTI_CapabilityStatusMT	None
AO_CapabilityMT: AO_CapabilityMT	None
AO_CapabilityStatusMT: AO_CapabilityStatusMT	None
CargoDeliveryCapabilityMT: CargoDeliveryCapabilityMT	None
CargoDeliveryCapabilityStatusMT: CargoDeliveryCapabilityStatusMT	None
COMINT_CapabilityMT: COMINT_CapabilityMT	None
COMINT_CapabilityStatusMT: COMINT_CapabilityStatusMT	None
CommTerminalCapabilityMT: CommTerminalCapabilityMT	None
CommTerminalCapabilityStatusMT: CommTerminalCapabilityStatusMT	None
EA_CapabilityMT: EA_CapabilityMT	None
EA_CapabilityStatusMT: EA_CapabilityStatusMT	None
EffectCapabilityStatusMT: EffectCapabilityStatusMT	None
ESM_CapabilityMT: ESM_CapabilityMT	None
ESM_CapabilityStatusMT: ESM_CapabilityStatusMT	None
IFF_CapabilityMT: IFF_CapabilityMT	None
IFF_CapabilityStatusMT: IFF_CapabilityStatusMT	None
MA_ActionCapabilityMT: MA_ActionCapabilityMT	None
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	None
MA_EffectCapabilityMT: MA_EffectCapabilityMT	None
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
OpaqueCapabilityMT: OpaqueCapabilityMT	None
OpaqueCapabilityStatusMT: OpaqueCapabilityStatusMT	None
PO_CapabilityMT: PO_CapabilityMT	None
PO_CapabilityStatusMT: PO_CapabilityStatusMT	None
RadarAltimeterCapabilityMT: RadarAltimeterCapabilityMT	None
RadarAltimeterCapabilityStatusMT: RadarAltimeterCapabilityStatusMT	None
RefuelCapabilityMT: RefuelCapabilityMT	None
RefuelCapabilityStatusMT: RefuelCapabilityStatusMT	None
SAR_CapabilityMT: SAR_CapabilityMT	None
SAR_CapabilityStatusMT: SAR_CapabilityStatusMT	None
SMTI_CapabilityMT: SMTI_CapabilityMT	None
SMTI_CapabilityStatusMT: SMTI_CapabilityStatusMT	None
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	None
SystemDeploymentCapabilityMT: SystemDeploymentCapabilityMT	None
SystemDeploymentCapabilityStatusMT: SystemDeploymentCapabilityStatusMT	None
TacticalOrderCapabilityMT: TacticalOrderCapabilityMT	None
TacticalOrderCapabilityStatusMT: TacticalOrderCapabilityStatusMT	None
WeatherRadarCapabilityMT: WeatherRadarCapabilityMT	None
WeatherRadarCapabilityStatusMT: WeatherRadarCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.21 Systems Needed Request

The Systems Needed Request sequence diagram specifies the interaction of a Lead ACP and the Command and Control (C2) authority where the Lead ACP has identified a need for an additional package asset and is requesting support from C2 to identify which asset to add. This supports scenarios in which the Lead ACP identified that it needs an additional asset in order to fulfill its

current package level action or tasking. The Lead ACP doesn't necessarily have insight into the assets currently available, or what other assets' capabilities are, therefore the Lead ACP needs support from C2 in order to determine the best asset to add to the package. This request can be utilized in tandem with the Creating or Changing a package interaction to complete the formal join command.

Once the Lead ACP has determined the need for a new asset and a need for C2 support in selecting and allocating that asset, the Lead ACP sends the *MA_SystemsNeededRequestMT* message to C2. The *MA_SystemsNeededRequestMT* fields are used to communicate the needed capabilities or tasking which the new ACP will fulfill. *RequestState* is utilized to communicate if the request is *NEW* or *UPDATED*, depending on if the request is being sent for the first time or not. *Candidate* is used as a *ChoiceType* to populate either a list of candidate systems via *SystemCandidate*, or an enumeration of *SystemCapabilityDescription* that the new asset needs to have. *ProposedRequirements* are used to communicate a specific Task, Effect, Action, or Response via an existing ID for that data object.

After C2 has processed the request, it responds to the Lead ACP with the *MA_SystemsNeededRequestStatusMT* message. The *RequestProcessingState* reports the progress towards processing the request. Once a *RequestProcessingState* of *COMPLETED* is reported, the *Option* field with *SystemsNeeded* and *SystemID* is populated to specify the asset selected to fulfill the request.

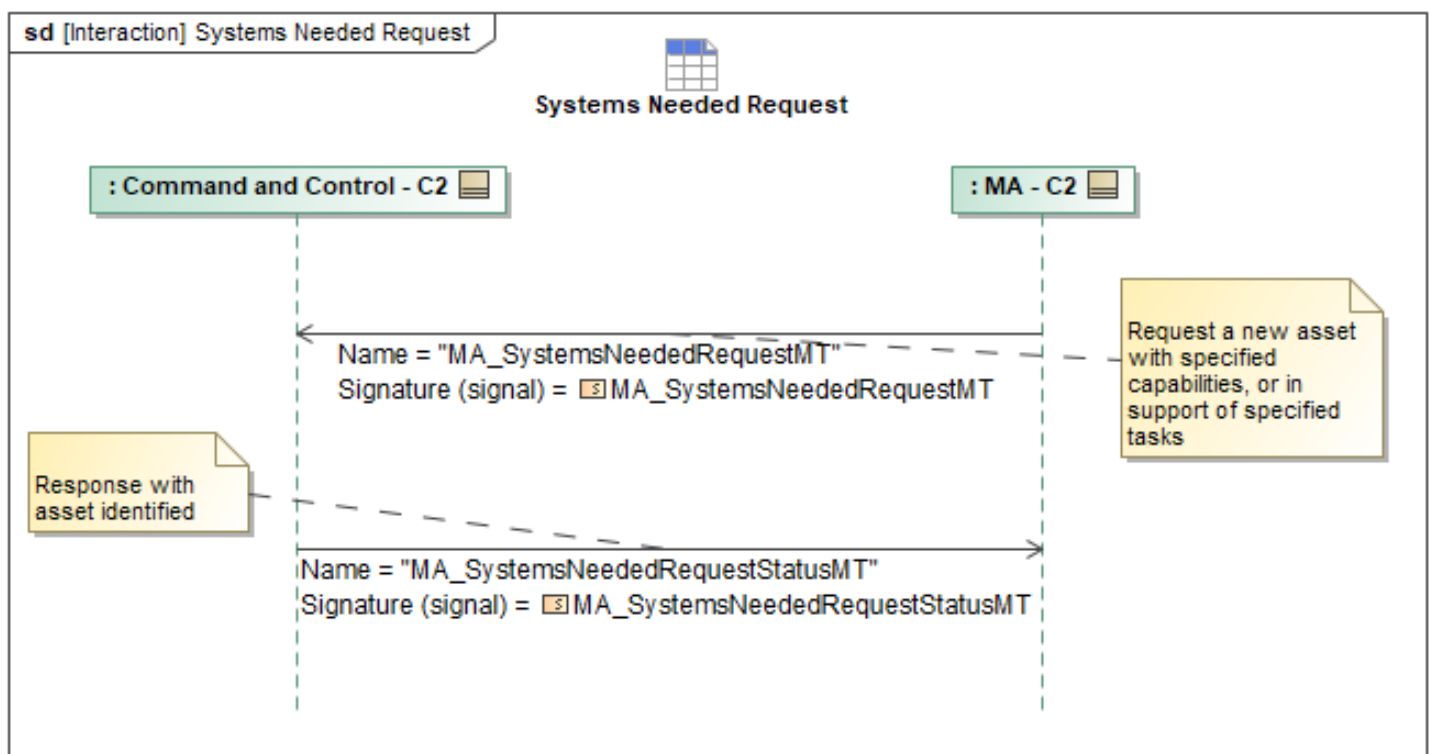


Figure 5-50: Systems Needed Request

Table 5-49: Systems Needed Request

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemsNeededRequestMT: MA_SystemsNeededRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemsNeededRequestStatusMT: MA_SystemsNeededRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.22 Update Barometric Pressure

The C2 system can send the barometric pressure reference command to MA via *MA_SystemManagementRequestMT*. This value is the QNH pressure and this interaction allows the value to be changed in order to accurately calculate the vehicle's altitude above mean sea level. The *RequestType* will be *VehicleSettings*, with *VehicleSettings.QNH_Setting* set to the barometric pressure in kilopascals (kPa).

In response, MA sends C2 the *MA_SystemManagementRequestStatusMT* message, accepting or rejecting the command. The *MA_SystemManagementRequestStatusMT* will have the *RequestProcessingState* field set to "COMPLETED". If the *MA_SystemManagementRequestMT* was not able to be applied, then the *RequestProcessingState* will be set to "FAILED" and the *RequestProcessingStateReason* set to the reason the *MA_SystemManagementRequestMT* couldn't be applied.

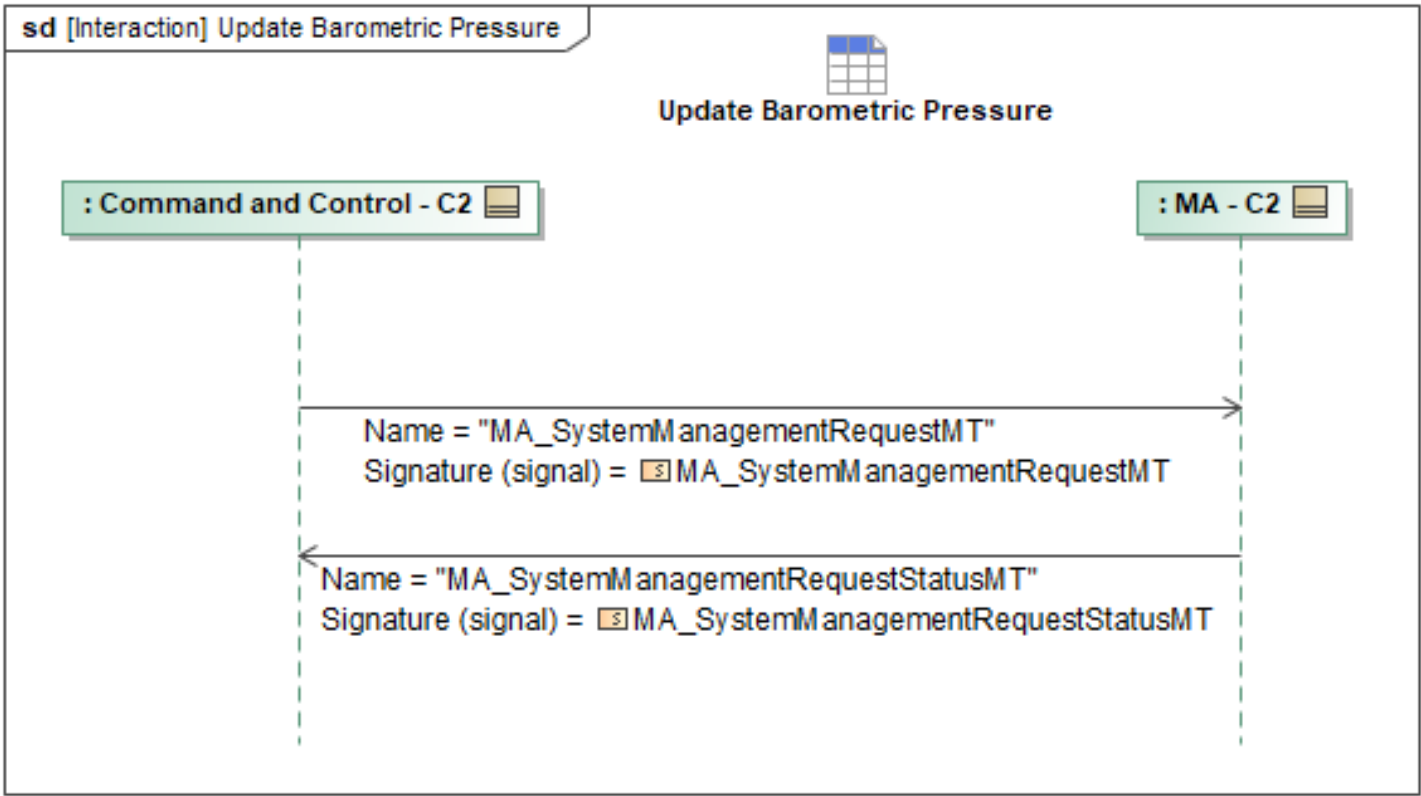


Figure 5-51: Update Barometric Pressure

Table 5-50: Update Barometric Pressure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemManagementRequestMT: MA_SystemManagementRequestMT	None
MA_SystemManagementRequestStatusMT: MA_SystemManagementRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.7.23 Update MA-FA Control Designations

C2 may command the aircraft control paradigm including revoking or authorizing MA to command the flight of the aircraft. C2 will revoke or authorize MA to command specific flight capabilities of the aircraft. To communicate the authorized flight capabilities, C2 will send MA a *MA_FlightCapabilityMT* populating the *CapabilityType.MA_FlightCapabilityEnum* field with the allowable control modes that MA can command. To revoke flight capabilities, C2 will not include them in the *MA_FlightCapabilityEnum* and to authorize flight capabilities C2 will select them as a part of the *MA_FlightCapabilityEnum*.

To enact the control paradigm, C2 will then send a *ControlRequestMT* populating the *Controller* as the MA System ID and the *Controllee* as the FA system ID as MA will be controlling FA through flight commands. C2 will populate the *ControlType* for MA as *CAPABILITY_SECONDARY*, as the *CAPABILITY_PRIMARY* control type is reserved for FA as the safety critical system on the aircraft.

Lastly, the *CapabilityID* will be set as the *CapabilityID* of the *MA_FlightCapabilityMT* that proceeded the *ControlRequestMT*.

Both the *MA_FlightCapabilityMT* and *ControlRequestMT* will be forwarded from MA to FA to inform FA of the C2 desired control paradigm. This sequence is outlined in the VI. MA will then respond with the *ControlRequestStatusMT* to acknowledge the commanded control paradigm.

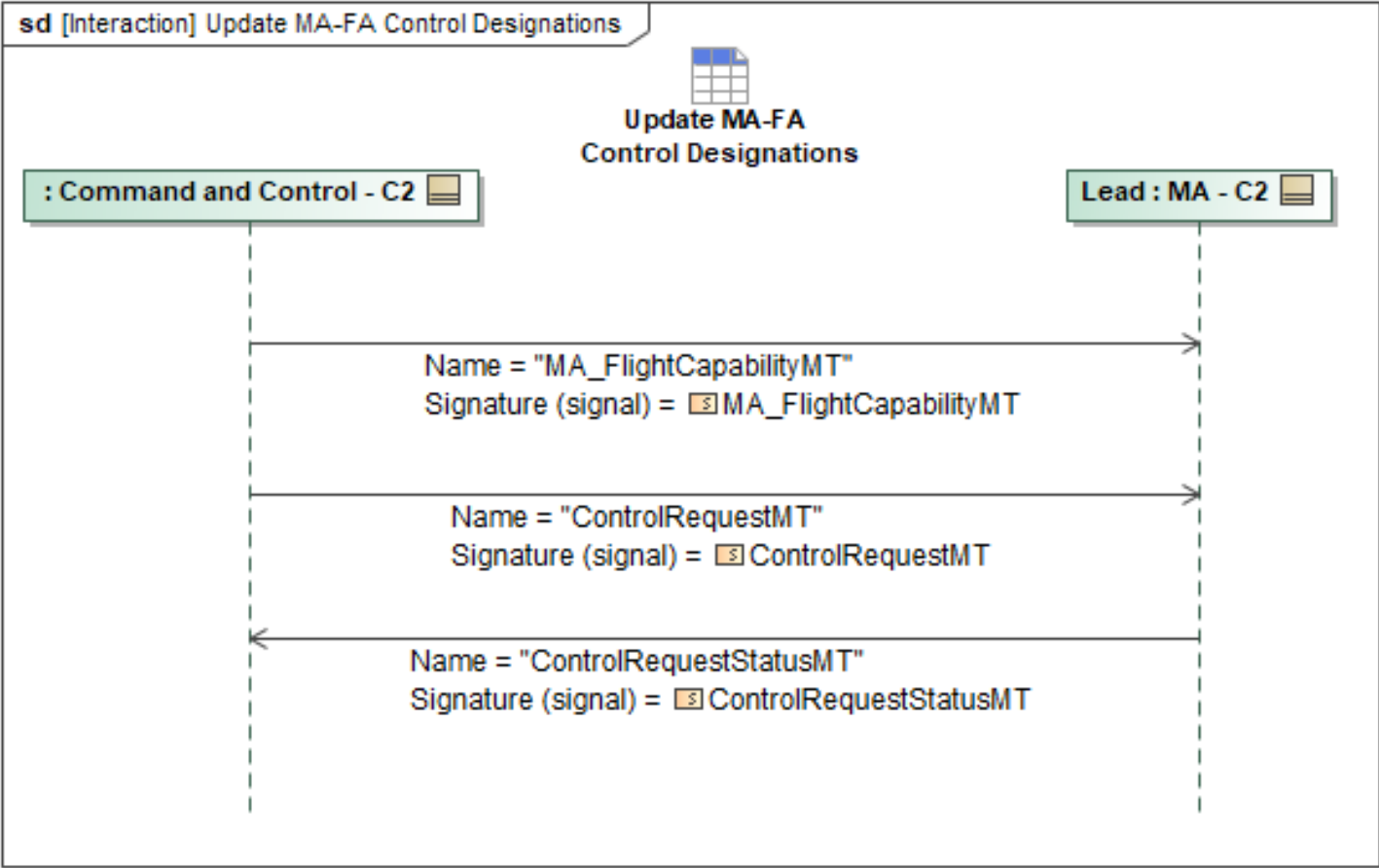


Figure 5-52: Update MA-FA Control Designations

Table 5-51: Update MA-FA Control Designations

Message Name (Name:Type)	Required Fields (Name:Type)
ControlRequestMT: ControlRequestMT	CapabilityID: CapabilityID_Type
ControlRequestStatusMT: ControlRequestStatusMT	None
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8 C2 Mission Plan Behaviors Package

5.2.8.1 Activate or Deactivate Mission Plan

A C2 node can activate or deactivate a mission plan by sending a

MA_MissionPlanActivationCommandMT message. The *MA_MissionPlanActivationCommandMT's CommandState* should be *NEW*, the *Command* field should specify which mission plan to activate or deactivate, and *CommandID* should be a unique identifier for the command. To activate or deactivate a mission plan and all its sub-plans, send *Command.ActivationDetails.ByMissionPlan*. To activate or deactivate only a subset of subplans belonging to a mission plan, send *Command.ActivationDetails.BySubPlan* detailing which subplans should be activated or deactivated. To activate or deactivate only a specific *ExecutionPlanSet* belonging to a mission plan, send *Command.ActivationDetails.ByExecutionPlanSet*.

MA will send *MA_MissionPlanActivationStatusMT* and *MA_MissionPlanExecutionStatusMT* messages periodically.

MA will send at least one *MA_MissionPlanActivationCommandStatusMT* message. While the command is being initiated by MA, it can optionally send status updates to C2 via *MA_MissionPlanActivationCommandStatusMT_1* with *CommandStatus* set to *QUEUED* or *PROCESSING*. Once the command status has reached terminal status, *MA_MissionPlanActivationCommandStatusMT_2* is sent with *CommandProcessingState* *ACCEPTED* or *REJECTED* and *CommandStatus* *COMPLETED* or *FAILED*.

If a mission plan or subplan was activated or deactivated successfully, MA will send C2 a *MA_MissionPlanActivationStatusMT* detailing the current state of the mission plan. If the mission plan has subplans, *MA_MissionPlanActivationStatusMT_1* is sent and it must include *SubPlanActivationState*. If there are no SubPlans, then *MA_MissionPlanActivationStatusMT_2* is sent. If plan execution status has changed, MA will send C2 a *MA_MissionPlanExecutionStatusMT* with *Source* set to *ACTUAL* and *MA_PlanExecutionStatus* detailing the change in execution status. For a mission plan operating with an *ExecutionSequence*, *PlanExecutionStatus.ExecutionPlanSetStatus* can be included to provide statuses on the *ExecutionPlanSets* within the mission plan.

See the behaviors "Update RoutePlan" and "Update Mission Plan" on how the different plans interact in sequence

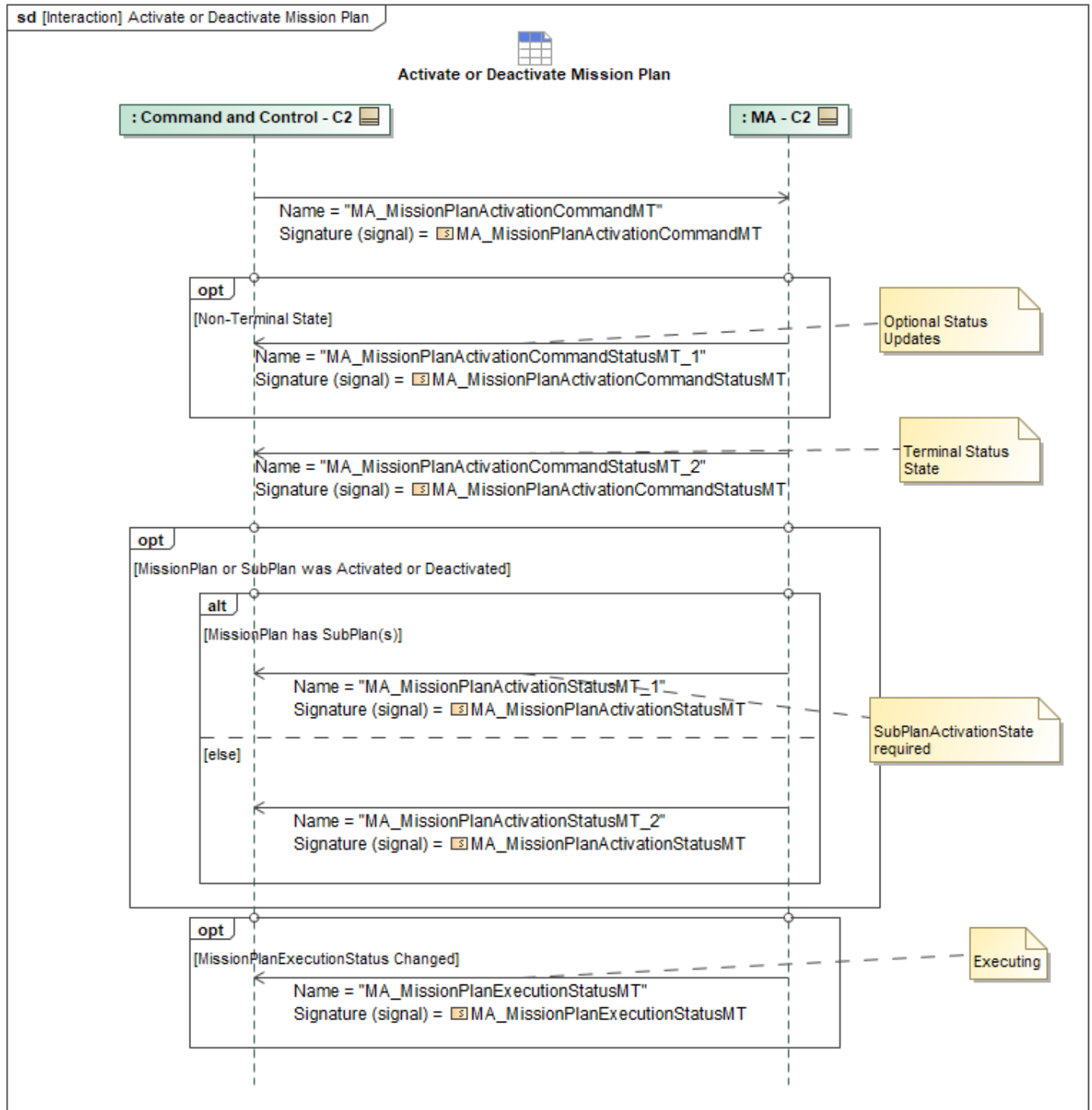


Figure 5-53: Activate or Deactivate Mission Plan

Table 5-52: Activate or Deactivate Mission Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	None
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	None
MA_MissionPlanActivationStatusMT_1: MA_MissionPlanActivationStatusMT	SubPlanActivationState: PlanActivationStateType
MA_MissionPlanActivationStatusMT_2: MA_MissionPlanActivationStatusMT	None
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	PlanExecutionStatus: MissionPlanExecutionStateType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.2 C2 Commanded Abort

A C2 system may also wish to command MA to abort its current behavior. This sequence begins with the *MA_MissionPlanActivationCommandMT* sent from the C2 system to MA, which includes the following set fields: *MissionPlanActivationCommandID* specifying the *UUID* of the abort command, the *CommandState* field denoting the abort command by the enum value "CANCEL", and the *Command* field containing the MissionPlanID and optional SubPlan(s) or ExecutionPlanSet(s) to cancel executing. Within the plan to abort, the *CommandType* field is set to the *PlanActivationCommandEnum* value of "DEACTIVATE."

MA shall first respond with a *MA_MissionPlanActivationCommandStatusMT* reflecting the acceptance and active processing of the abort command. Specifically, this message confirms the command through a crossed referenced *CommandID* matching the *UUID* of the abort command, *ActivationStatus.CommandProcessingState* enumerated value specifying "ACCEPTED", and *ActivationStatus.CommandStatus* set to "COMPLETED".

ActivationCommandByState.PlanActivationCommandState will be set to the *PlanActivationStateEnum* value of "DEACTIVATED", and *ActivationCommandByState.Plans* will be set to the specific plan to be deactivated (can be any type), as was specified in the *MA_MissionPlanActivationCommandMT*. Should MA encounter any issues during the processing phase for this command, then MA shall inform the C2 system through an alternate *ActivationStatus.CommandStatus* of "FAILED" and *ActivationStatus.CommandProcessingState* of "REJECTED".

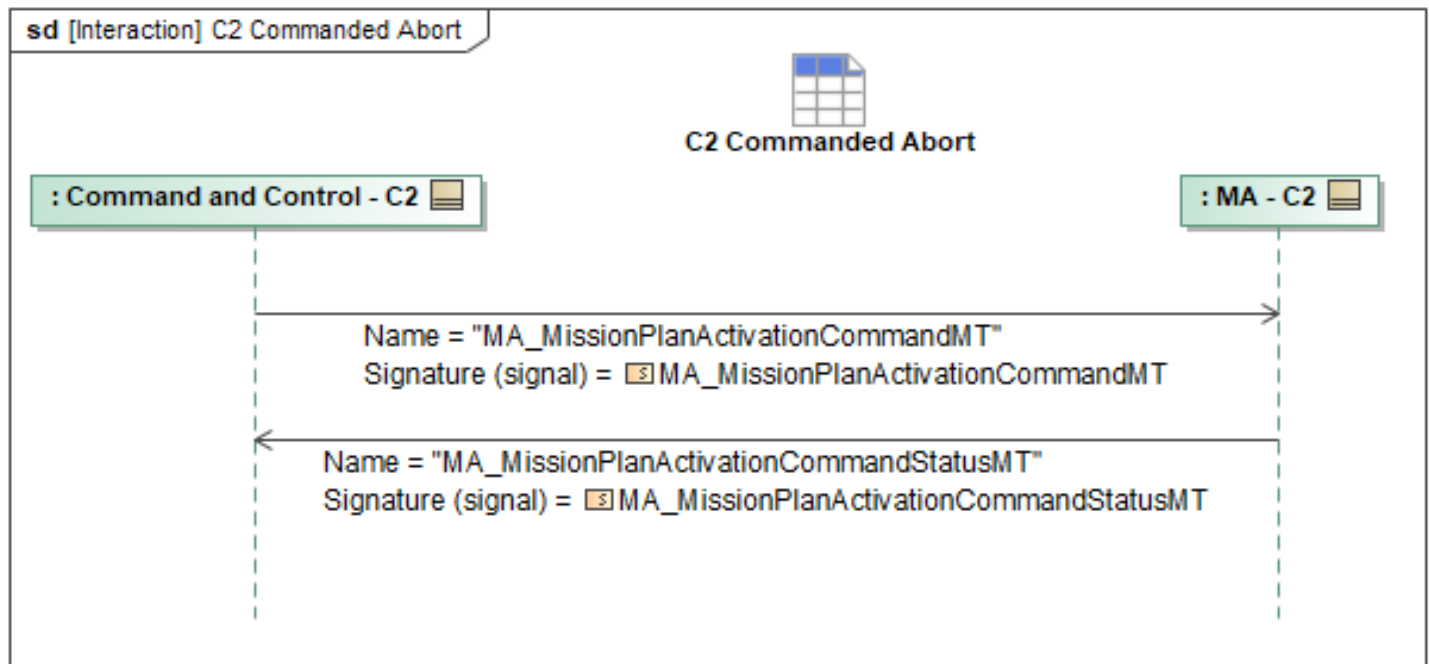


Figure 5-54: C2 Commanded Abort

Table 5-53: C2 Commanded Abort

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	BySubPlan: MA_MissionPlanSubplanActivationType
MA_MissionPlanActivationCommandStatusMT: MA_MissionPlanActivationCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.3 C2 Detect Mission Plan Route Discontinuity Response

Upon identification of a Mission Plan Route Discontinuity Error, MA will send *MA_SystemNotificationMT* to C2 and P2P, to indicate that a Mission Plan Route Discontinuity condition exists. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with INFEASIBLE_ROUTE. This indicates that the ACP can no longer continue along the nominal Mission Plan path from its current or projected position. *AssociatedMessage.MessageType* will have the ROUTE_PLAN enumeration to indicate to C2 that the route is no longer feasible. *AssociatedMessage.AssociatedID* may reference the affected *PathSegmentID* that precedes the discontinuity, or alternatively, the *FlightCommandID* can be used if a *PathSegmentID* does not exist or cannot be referenced.

At the time of the discontinuity, MA also sends *MA_PositionReportDetailed* so that C2 has access to the current and projected position data relevant to the error condition. The *Kinematics* field communicates the ACP's current position and velocity associated with the discontinuity. Resolution actions may include MA executing a plan modification or receiving a resolution from C2 or P2P. Upon resolution of the error, MA will send *MA_SystemNotificationMT* to C2 and P2P with *NotificationState* set to CLOSED and the same other fields mentioned in the initial *MA_SystemNotificationMT*,

including *NotificationDetail.Reason* and *AssociatedMessage* details, to provide a consistent update on the resolved condition.

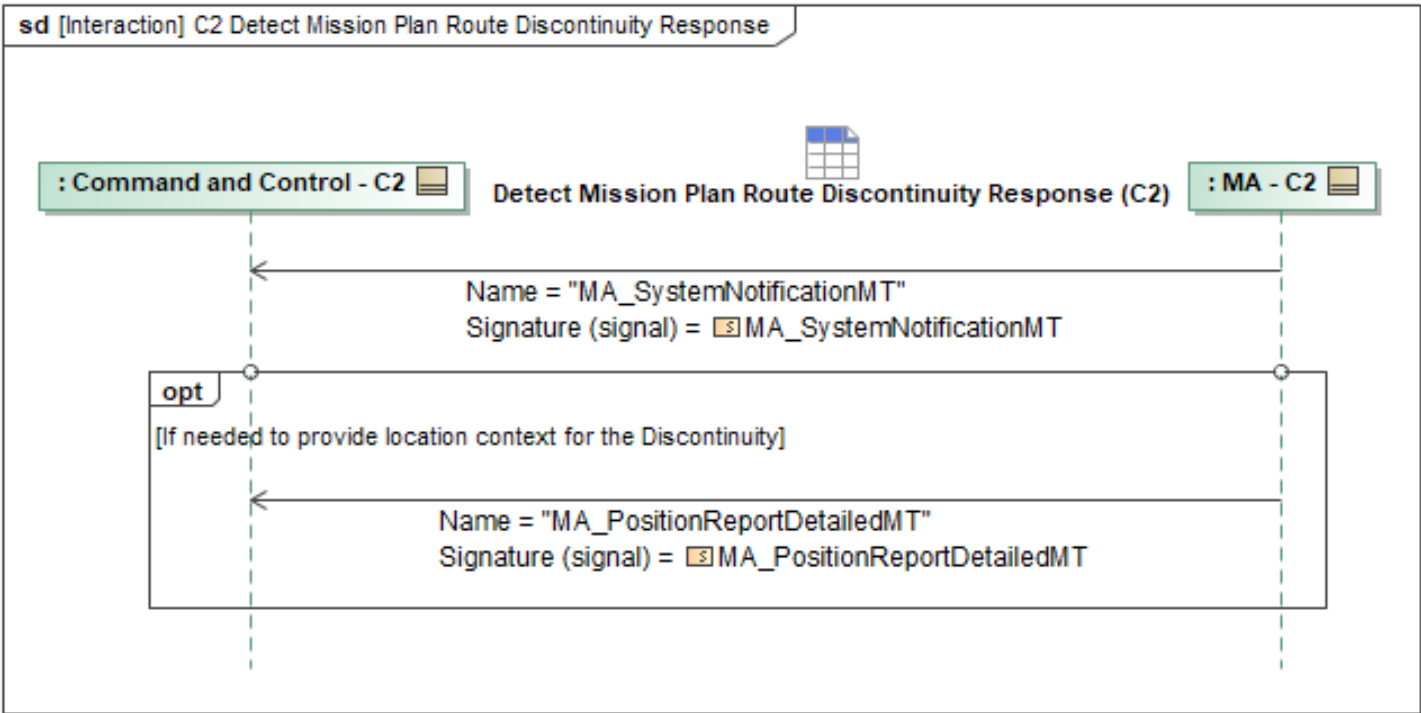


Figure 5-55: C2 Detect Mission Plan Route Discontinuity Response

Table 5-54: C2 Detect Mission Plan Route Discontinuity Response

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	None
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedID: ID_Type Description: VisibleString1024Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.4 C2 Receive Abort Status

This interaction between MA and C2 periodically forwards the *MA_MissionPlanActivationStatus* and *RoutePlanExecutionStatus* from MA to C2. *MA_MissionPlanActivationStatusMT* is sent with information populated for *RoutePlanID* of the route plan that has been aborted and *MissionPlanActivationStatus:SubPlanActivationState:ActivationState* is set to *DEACTIVATED*. Then, *RoutePlanExecutionStatusMT* is sent with information populated for *PlanID* of the route plan that has been aborted and a *PlanExecutionState* set to *Canceled*.

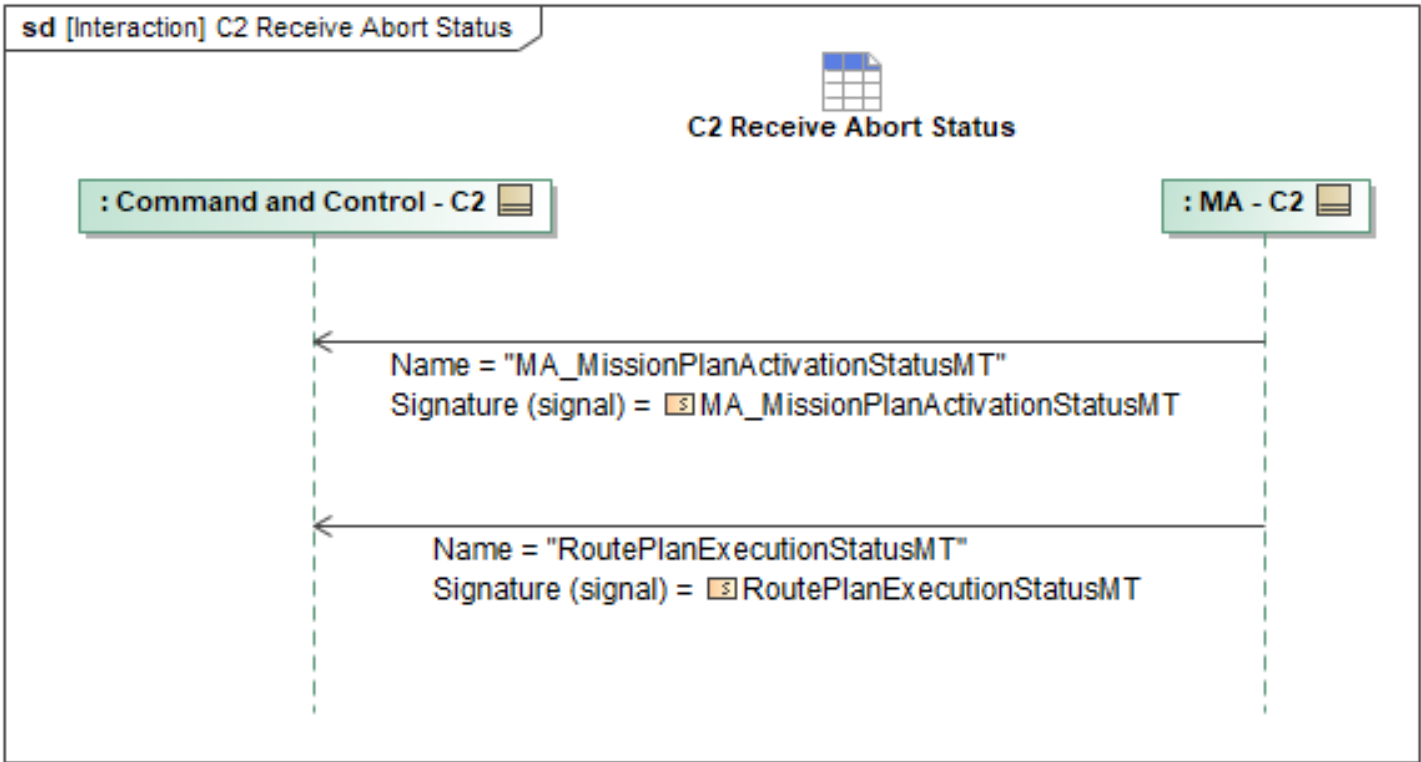


Figure 5-56: C2 Receive Abort Status

Table 5-55: C2 Receive Abort Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	RoutePlanID: RoutePlanID_Type DEACTIVATED: ActivationState: PlanActivationStateEnum
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	PlanExecutionState: PlanExecutionStateEnum PlanID: RoutePlanID_Type CANCELED:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.5 C2 Receive Plan Modification Notification

When MA updates a Plan, there may be C2 elements that need to be informed for situational awareness or synchronization reasons. This sequence describes the messaging that MA can use to broadcast the fact that MA has changed a Plan. If MA has modified a *MissionPlan* or a *SubPlan*, MA will send a *MA_PlanModificationRequestMT* to C2, requesting that C2 updates the Plan in question. Likewise, if MA has only modified a *RoutePlan*, MA will send a *RouteModificationRequestMT* to the same effect. Contained in the *MessageData* fields of these messages are fields that describe the Plan modifications which are populated as required to mirror the change that MA has already made to the Plan. Upon receipt of request, C2 will respond to MA with a *MA_PlanModificationRequestStatusMT* or *RouteModificationStatusMT* describing the status of the

Plan update.

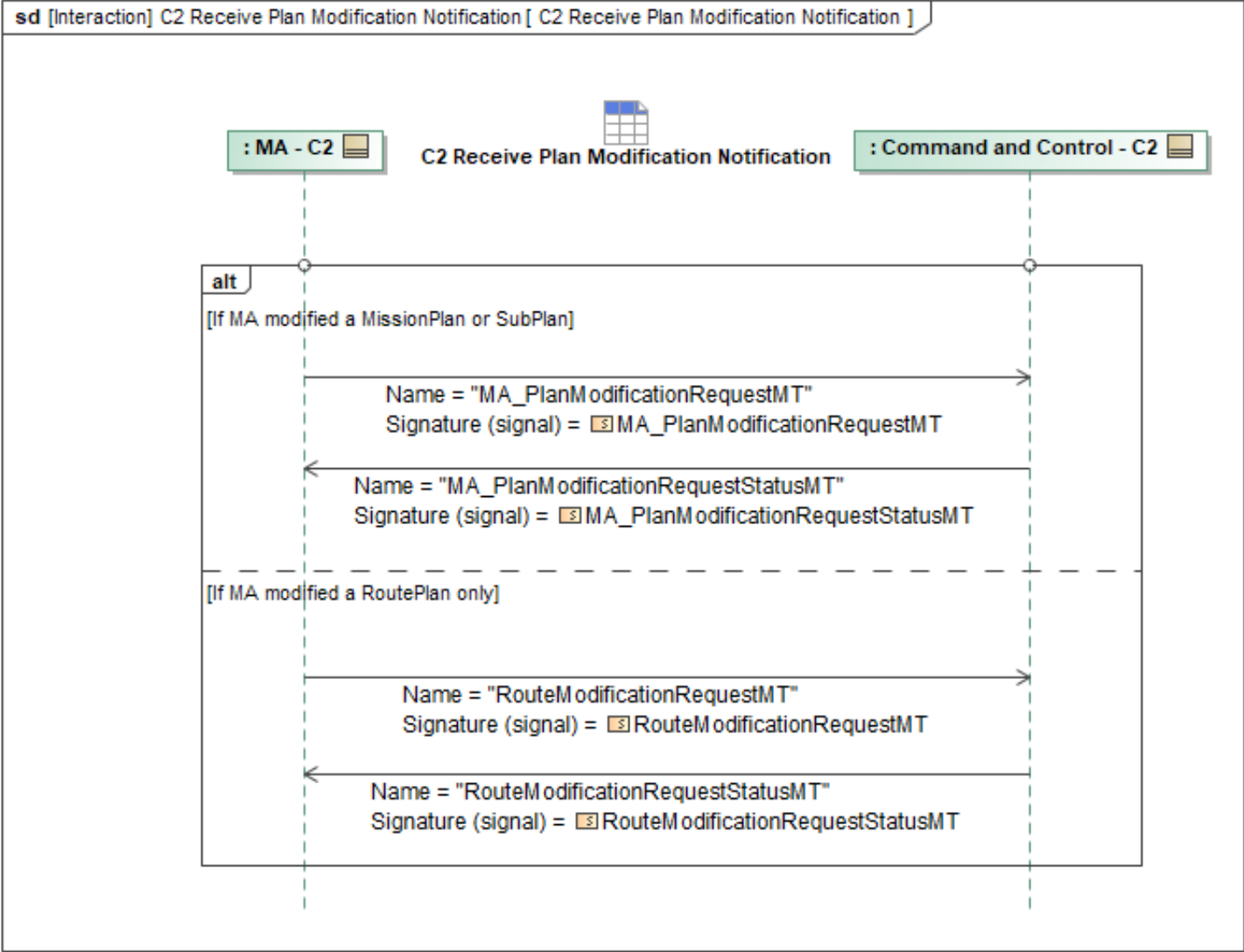


Figure 5-57: C2 Receive Plan Modification Notification

Table 5-56: C2 Receive Plan Modification Notification

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanModificationRequestMT: MA_PlanModificationRequestMT	None
MA_PlanModificationRequestStatusMT: MA_PlanModificationRequestStatusMT	None
RouteModificationRequestMT: RouteModificationRequestMT	None
RouteModificationRequestStatusMT: RouteModificationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.6 C2 Request Plan Patch

In the event that C2 determines that a Plan Patch is necessary, C2 can request that MA perform a Plan Patch. If the desired Plan Patch is applicable to a Route Plan, C2 sends MA a *RouteModificationRequestMT*. The *RouteModificationRequestMT* contains a *Path* field and a *PathSegment* field that communicate one of three possible outcomes: the addition of a path, removal of a path, or update a path, through the *Add*, *RemoveID*, or *Update* fields, respectively. Contained in the *PathSegment* field, the *PathID* field identifies the Path which the segment changes apply to.

If the desired Plan Patch is applicable to any other type of Plan, C2 sends MA a *MA_PlanModificationRequestMT*. The *MA_PlanModificationRequest* contains a *PlanModificationDetails* field which allows for a number of potential plan modification methods. For Plan Patching, the *SubPlansModification* field will be used to either add a *SubPlan* (*SubPlanAdditions* field) or retract a *SubPlan* (*SubPlanRetractions* field). MA then responds with the corresponding **StatusMT* message (*RouteModificationRequestStatusMT* or *MA_PlanModificationRequestStatusMT*), which contains the *RequestID* and the *RequestProcessingState* which indicates the status of the request.

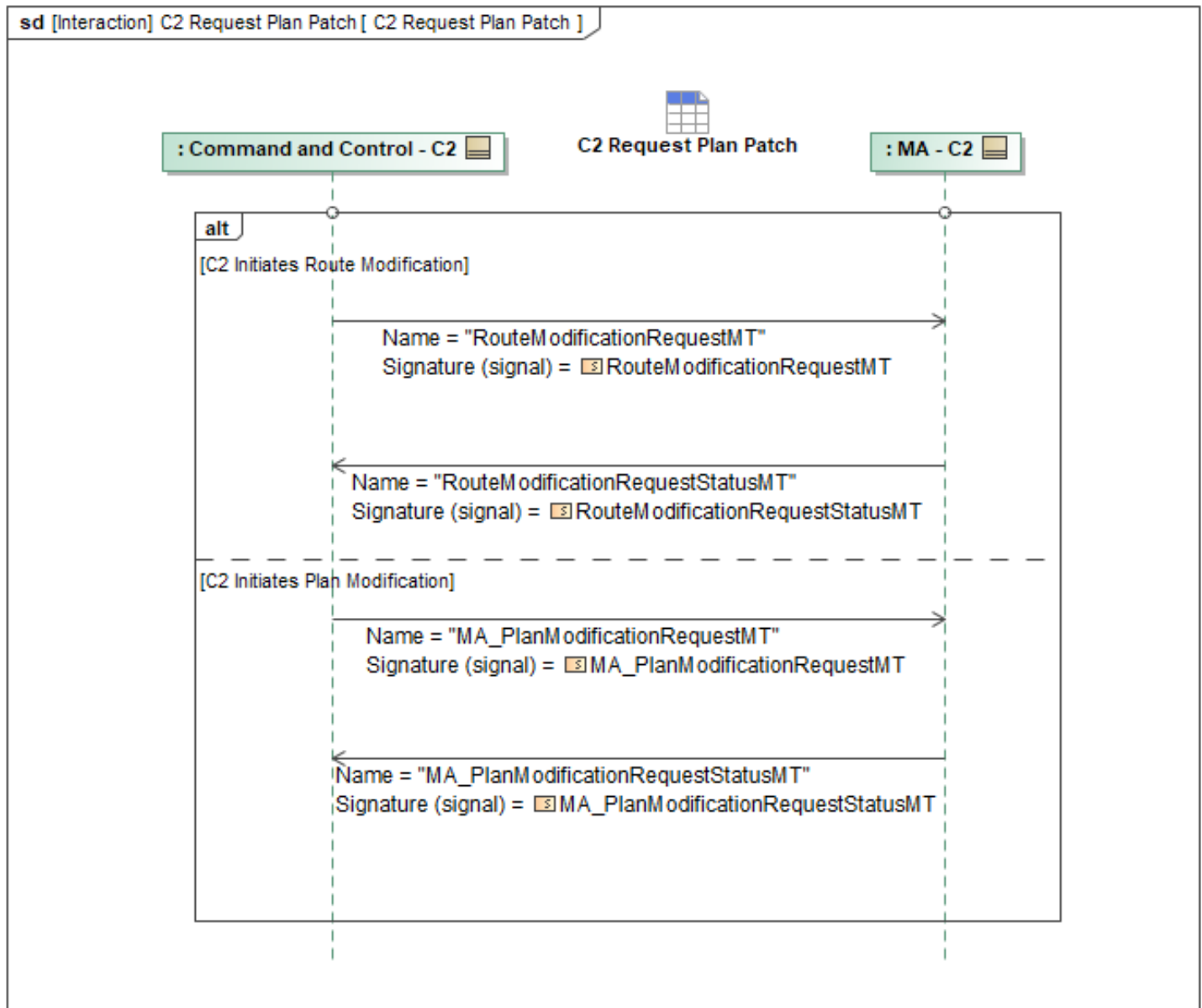


Figure 5-58: C2 Request Plan Patch

Table 5-57: C2 Request Plan Patch

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanModificationRequestMT: MA_PlanModificationRequestMT	None
MA_PlanModificationRequestStatusMT: MA_PlanModificationRequestStatusMT	None
RouteModificationRequestMT: RouteModificationRequestMT	None
RouteModificationRequestStatusMT: RouteModificationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.7 Command Mission Replan

MA can be commanded by C2 to generate a new mission plan. C2 sends a *MA_MissionPlanCommandMT* which may contain inputs to indicate data and references to other messages and their data to be used to create the *MA_MissionPlanMT*. Upon receipt of the *MA_MissionPlanCommandMT*, if C2 is authorized, MA begins generating a new mission plan, responding with *MA_MissionPlanCommandStatusMT* as required by the command. When MA is done generating the mission plan, it will publish a *MA_MissionPlanMT* message for approval. *MA_MissionPlanMT* will define the *ObjectState*, *ExecutionSequence*, *MissionPlanCommandID*, and optionally *SubPlans* if relevant. MA will include the *ResultingPlanIdentifier* field in the *MA_MissionPlanCommandStatusMT_2* to allow C2 to identify which *MA_MissionPlanMT* message has been generated as a result of its original *MA_MissionPlanCommandMT*.

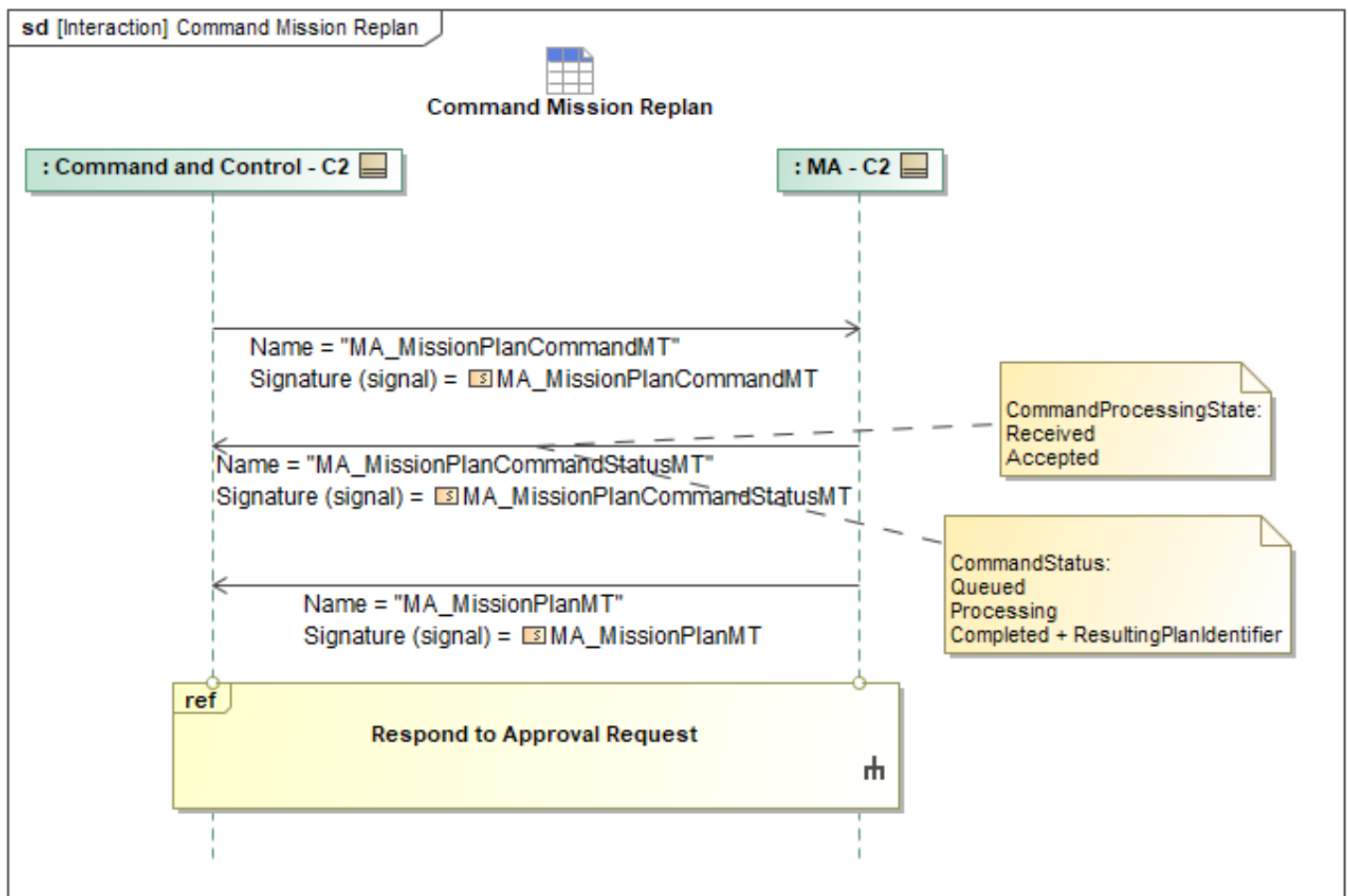


Figure 5-59: Command Mission Replan

Table 5-58: Command Mission Replan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanCommandMT: MA_MissionPlanCommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanCommandStatusMT: MA_MissionPlanCommandStatusMT	ResultingPlanIdentifier: PlansReferenceType
MA_MissionPlanMT: MA_MissionPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.8 MA Notifies C2 of Replanning

MA sends C2 an *MissionPlanStatusMT*, *RoutePlanStatusMT*, *TaskPlanStatusMT*, *RouteActivityPlanStatusMT*, *ActivityPlanStatusMT*, or *ActionPlanStatusMT* to indicate replanning is required and has begun. The message used is based on the type of replanning that needs to be done (MissionPlan, RoutePlan, TaskPlan, RouteActivityPlan, ActivityPlan, or ActionPlan respectively). Within the message, *PlanningState* is set to *PLANNING* and *ValidationState* is set to *INVALID* to indicate replanning has begun due to an issue with the current plan.

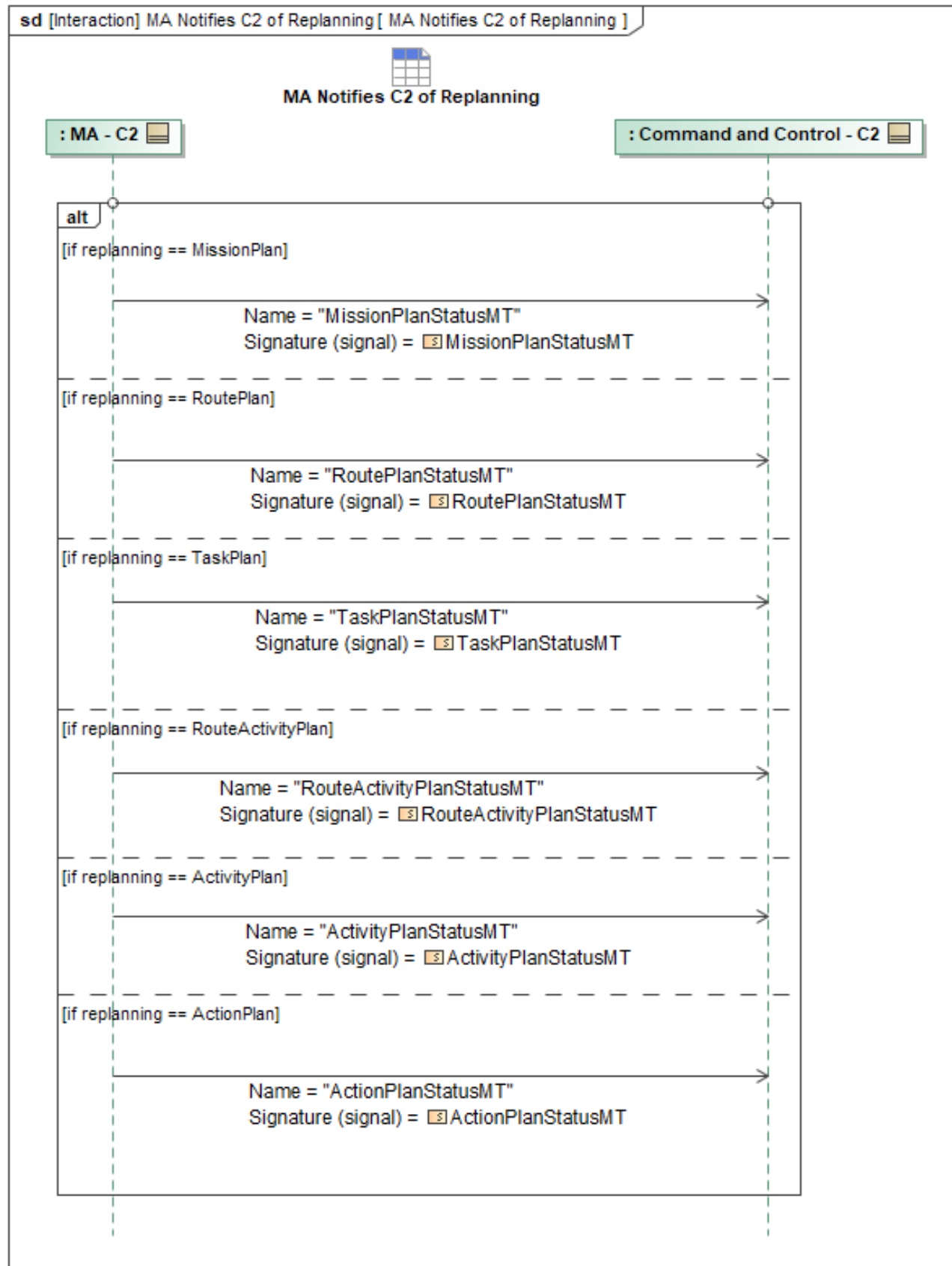


Figure 5-60: MA Notifies C2 of Replanning

Table 5-59: MA Notifies C2 of Replanning

Message Name (Name:Type)	Required Fields (Name:Type)
ActionPlanStatusMT: ActionPlanStatusMT	None
ActivityPlanStatusMT: ActivityPlanStatusMT	None
MissionPlanStatusMT: MissionPlanStatusMT	None
RouteActivityPlanStatusMT: RouteActivityPlanStatusMT	None
RoutePlanStatusMT: RoutePlanStatusMT	None
TaskPlanStatusMT: TaskPlanStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.9 MA Provides Replan Status to C2

This interaction enables MA to send additional *MA_MissionPlanCommandStatusMT* messages to C2 to report replanning status. This could be triggered by a contingency or other event that changed the planning or tasking status.

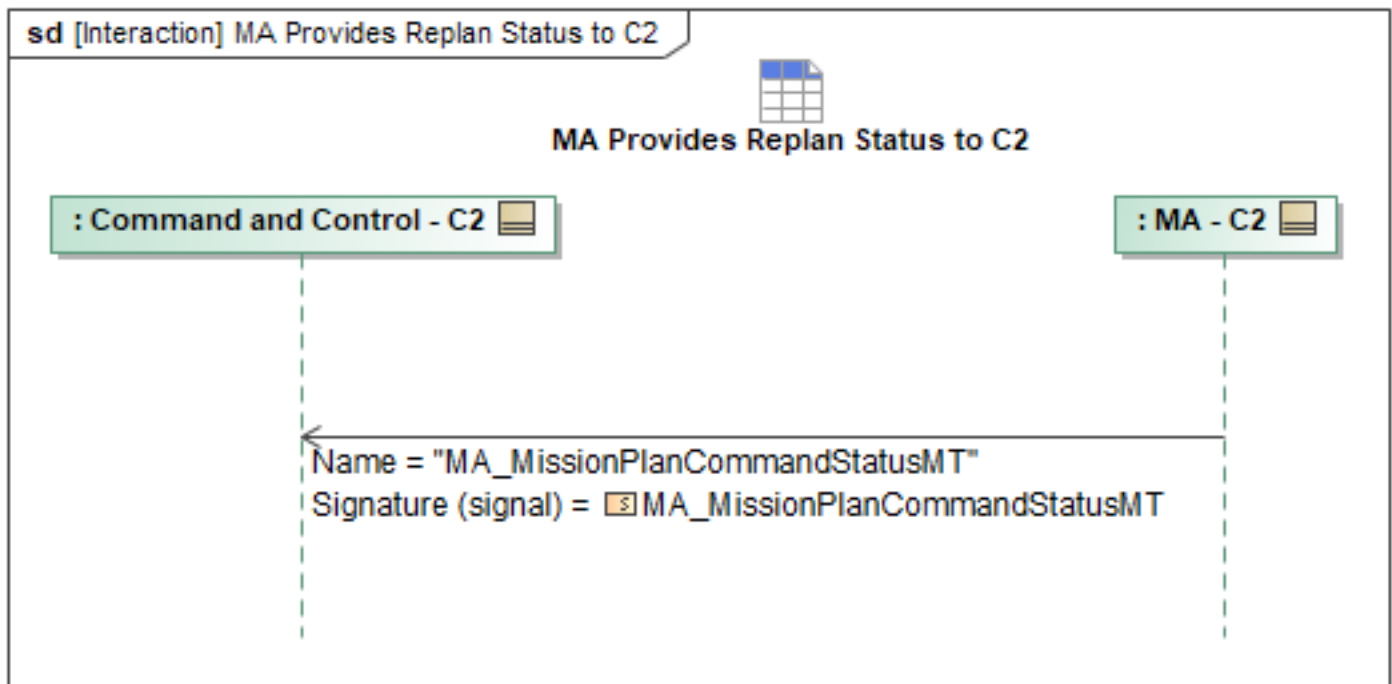


Figure 5-61: MA Provides Replan Status to C2

Table 5-60: MA Provides Replan Status to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanCommandStatusMT: MA_MissionPlanCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

5.2.8.10 QB Commanded RTB

At any point during an active connection with MA, a C2 system shall be capable of issuing commanded control over MA Contingencies, assuming the C2 system is authorized under the appropriate RBAC level. The appropriate RBAC level shall indicate the ability for C2 to specify or command a contingency response at or when a qualifying triggering event occurs. In the case for a commanded "*Return to Base*" (RTB) contingency, an authorized C2 system can either activate an existing RTB Action by sending *MA_ActionCommandMT* with the associated *CapabilityID*, or it can send *MA_ActionMT* with *ActionType* set to *RETURN_TO_BASE* which is then activated by *MA_ActionCommandMT*. MA shall acknowledge this message sequence with a *MA_MissionPlanActivationCommandStatusMT*, with *MissionPlanActivationCommandID* confirming the command's identity, *ActivationStatus.CommandProcessingState* as "*ACCEPTED*", and *ActivationStatus.CommandStatus* set to "*COMPLETED*", *ActivationCommandByState.PlanActivationCommandState* set to "*ACTIVATED*", and *ActivationCommandByState.Plans* will be set to the RTB *Plan that was specified in the *MA_MissionPlanActivationCommandMT* to be activated.

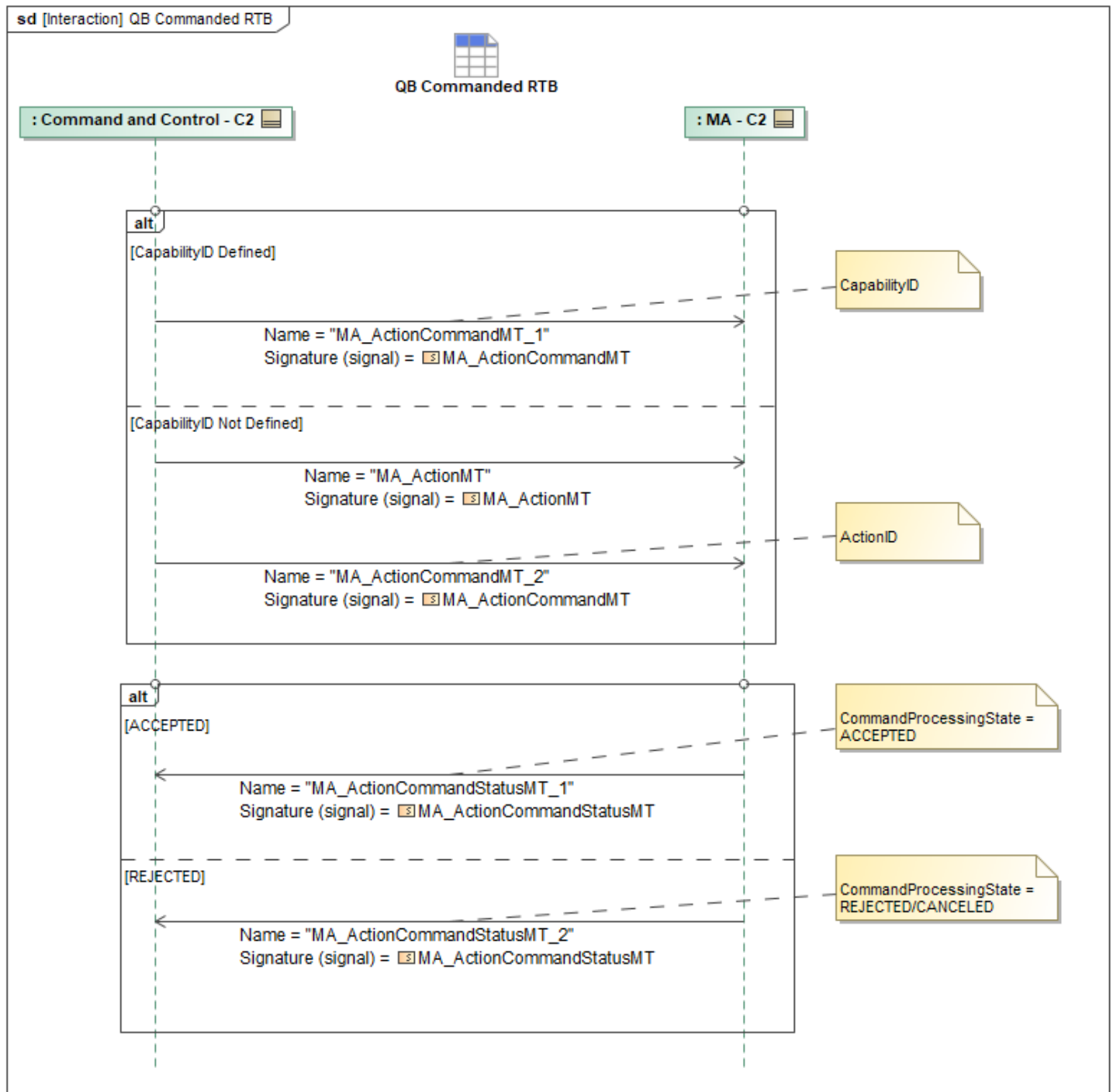


Figure 5-62: QB Commanded RTB

Table 5-61: QB Commanded RTB

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT_1: MA_ActionCommandMT	None
MA_ActionCommandMT_2: MA_ActionCommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandStatusMT_1: MA_ActionCommandStatusMT	None
MA_ActionCommandStatusMT_2: MA_ActionCommandStatusMT	None
MA_ActionMT: MA_ActionMT	ActionType: MA_ActionTypeEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.11 Query C2 for Missing Data

If MA identifies that there is Mission Plan data that is missing or inaccessible, most likely as part of pre-flight startup checks, it can request that C2 send the data.

This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *QueryDataRequestStatusMT_2* and *QueryDataRequestStatusMT_3* messages with *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and "PROCESSING", respectively.

When complete, the *QueryDataRequestStatusMT_3* message's *RequestProcessingStateEnum* is set to "COMPLETED" and contains the requested data.

When the original *QueryDataRequestMT*'s optional field *ResultsInNative* is populated with the matching message type, the underlying data management services instead publishes the requested data in its original or raw format as a separate message and indicates *QueryDataRequestStatusMT_3* message's *RequestProcessingStateEnum* to "COMPLETED".

If C2 is unable to complete the query it will ignore the *QueryDataRequestMT* command and reply with *QueryDataRequestStatusMT_4* where *RequestProcessingState* is set to "REJECTED". If the query is rejected because C2 does not have the requested information, the *Reason* field within *RequestProcessingStateReason* will be set to "UNKNOWN_ID". If the query is rejected because the missing requested information cannot be transmitted via UCI, the *Reason* field will be set instead to "INVALID_INPUT_PARAMETER".

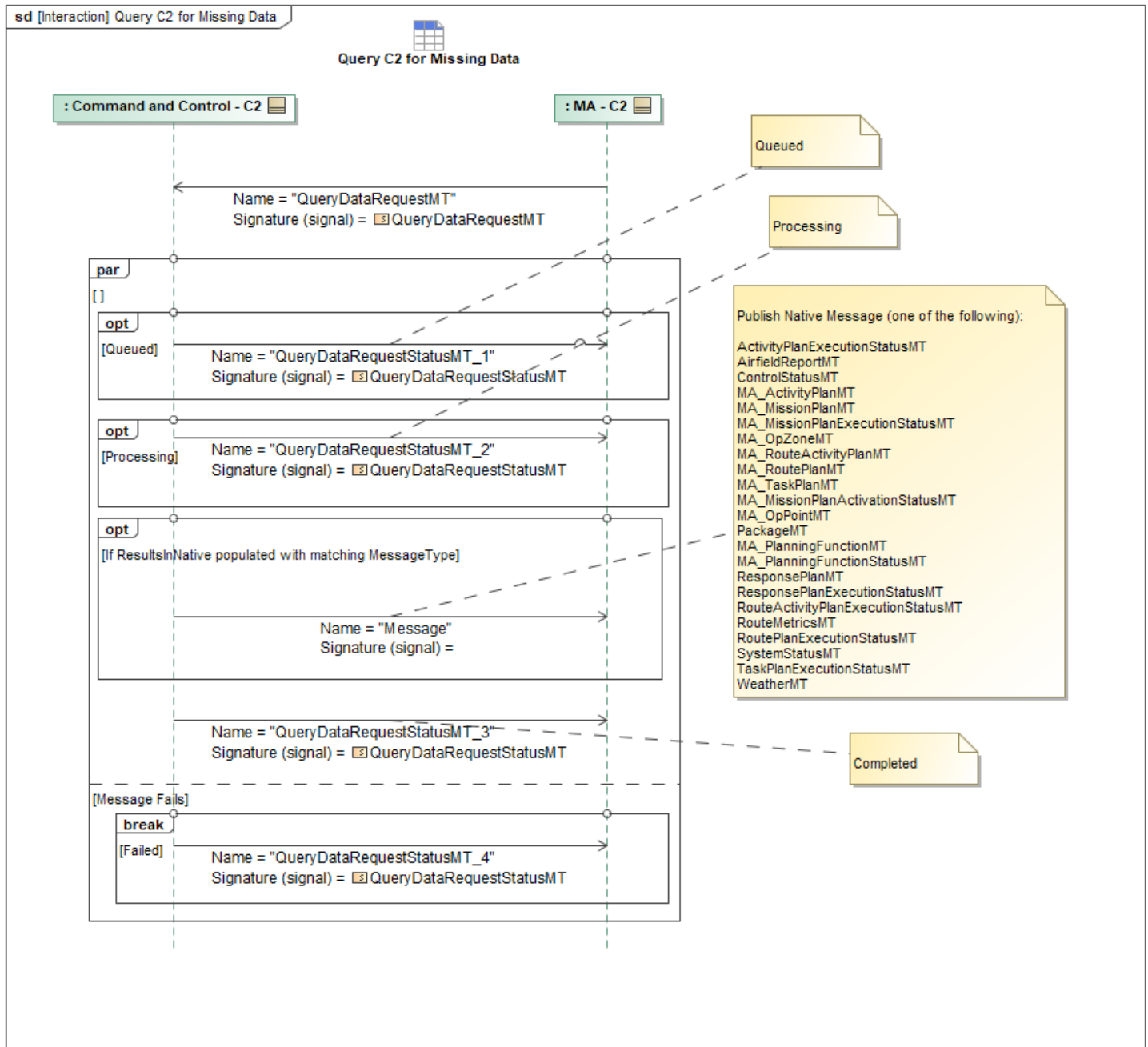


Figure 5-63: Query C2 for Missing Data

Table 5-62: Query C2 for Missing Data

Message Name (Name:Type)	Required Fields (Name:Type)
Message:	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.12 Query Deconfliction Data

Asset Deconfliction data, used to prevent the ACP from colliding with another platform, is defined by *MA_OpLineMT*, *MA_OpZoneMT*, and *MA_OpRoutingMT*.

MA_OpLineMT contains data used to convey a Corridor: Line with width along the designated Route, describing the passageway designated for Mission planning deconfliction for each ACP. Using the *Category* field set to *CORRIDOR*. The *LeftWidth* and *RightWidth* fields are required to describe the overall width of the corridor.

The defined data is to be used when deconflicting entities routes or generating routes to ensure proper separation. These parameters are applied at the vehicle level.

MA_OpZoneMT contains data used to convey Altitude. Data in the *Zone* field captures the shape of the region, along with the Max and Min altitudes. Describing the region designated for mission planning deconfliction for each ACP. The defined data is to be used when deconflicting entities routes or generating routes to ensure proper Altitude block compliance. These parameters are applied at the vehicle level.

MA_OpRoutingMT contains data used to convey a Bubble: The *SpecificBlueSeparation* and *SpecificRedSeparation* define parameters to use when deconflicting entity routes or generating routes to ensure compliance of the designated keep in and keep out zones. These parameters are applied at the vehicle type level.

Data is stored or cached in data manager services which may have a database backend. C2 can query this data via *QueryDataRequestMT* by including three instances of *MessageType* with their enumerations set to *OP_LINE*, *OP_ROUTING*, and *OP_ZONE* respectively. This will be responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *QueryDataRequestStatusMT_1* and *QueryDataRequestStatusMT_2* messages with *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and "PROCESSING", respectively. When complete, the *QueryDataRequestStatusMT_3* message's *RequestProcessingState* field is set to "COMPLETED" and contains the requested data. When the original *QueryDataRequestMT*'s optional field *ResultsInNative* is populated, the underlying data management services publishes the requested data in its original or raw format as a separate message from the *QueryDataRequestStatusMT_3* message at completion.

If C2 is unable to complete the query it will ignore the *QueryDataRequestMT* command and reply with *QueryDataRequestStatusMT_4* where *RequestProcessingState* is set to "REJECTED". If the query is rejected because C2 does not have the requested information, the *Reason* field within *RequestProcessingStateReason* will be set to "UNKNOWN_ID". If the query is rejected because the missing requested information cannot be transmitted via UCI, the *Reason* field will be set instead to "INVALID_INPUT_PARAMETER".

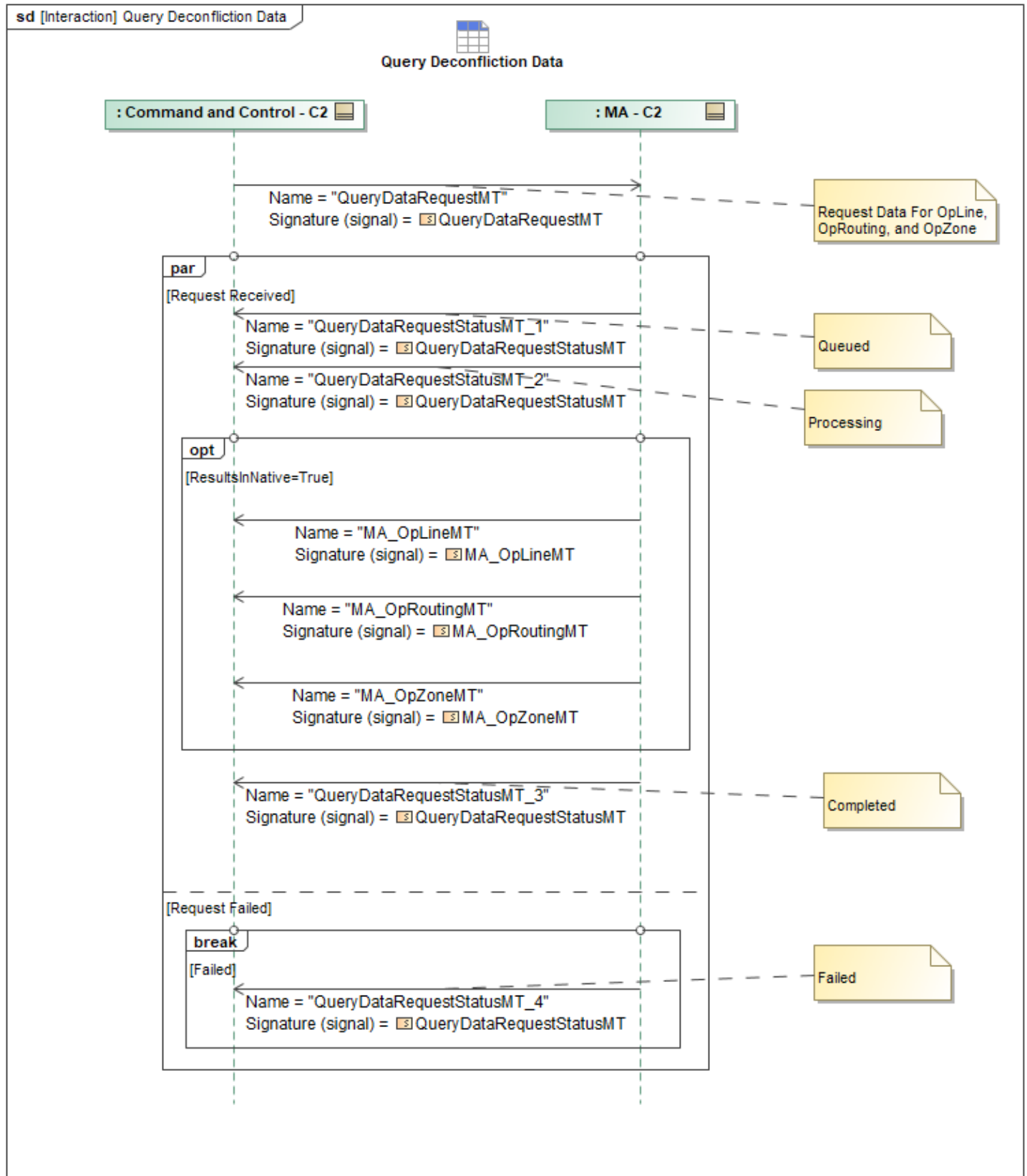


Figure 5-64: Query Deconfliction Data

Table 5-63: Query Deconfliction Data

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpLineMT: MA_OpLineMT	LeftWidth: DistanceType RightWidth: DistanceType
MA_OpRoutingMT: MA_OpRoutingMT	SpecificRedSeparation: SpecificRedSeparationType SpecificBlueSeparation: SpecificBlueSeparationType
MA_OpZoneMT: MA_OpZoneMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.13 Receive Autonomy Settings

MA sends *MA_PlanningFunctionMT* and *MA_PlanningFunctionStatusMT* periodically to allow C2 to programmatically and dynamically discover how autonomous a system is and how its associated planning services are configured.

MA_PlanningFunctionStatusMT allows a system to advertise up to 10 *PlanningInterfaces*, one per UCI *PlanType*. Furthermore, the system can advertise its ability to create *Plans (*PlanInterface* set to "PLAN_COMMAND"), validate *Plans (*PlanInterface* set to "PLAN_VALIDATION"), automated *Plan activation (*PlanInterface* set to "AUTOMATED_PLANNING"), planning of [Requirements]Plans (*PlanningInterface* set to "REQUIREMENT_MANAGEMENT"), scoring plans (*PlanningInterface* set to "PLAN_SCORES"), computing metrics for plans (*PlanningInterface* set to "PLAN_METRICS"), and assessing plans (*PlanInterface* set to "PLAN_ASSESSMENT"). The *PlanActivationAutonomy* is similar to the Command field of *MissionPlanActivationCommandMT* and can be thought of as a template for any automated plan activations as to how far to advance the plan state machine upon activation. These abilities, when onboard, are critical for automated planning and plan activation. Beyond these settings, *MA_PlanningFunctionStatusMT* provides subscribers with high level details on how plans will be generated. For example, *ResultsInMissionPlan* specifies that the generated *Plans will be composed in a generated *MissionPlanMT*. Plan generation can be constrained by parts with *OutputPlanParts* or by other plans via *ConstrainingPlans* or *OtherSystemConstrainingPlans*.

MA_PlanningFunctionMT is an even more detailed representation into a system's autonomous planning functions. Its main difference is it exposes individual planning functions by providing their *PlanningProcessID*. This is useful if the desire is to issue a **PlanCommandMT* directly to an individual planning process. *PlanningFunctionSettingsCommandMT* can be paired with information discovered in *MA_PlanningFunctionMT* to reconfigure an individual planning function.

MA_PlanningFunctionMT's fields are similar to *MA_PlanningFunctionStatusMT* but with more detail by virtue of exposing settings of individual planners rather than an aggregation of a platform's planning abilities.

The interaction starts with each planning service publishing *MA_PlanningFunctionMT* with the (required) supported *PlanType* (*InterfacePlanTypeEnum*) and (required) *PlanInterface* (*PlanningInterfaceEnum*). It is optional for C2 to consume these initial *MA_PlanningFunctionMT* messages as MA will support subsequent querying for this content. MA can also optionally publish *MA_PlanningFunctionStatusMT* to indicate a high level summary of all *PlanType* (*InterfacePlanTypeEnum*) and *PlanInterface* (*PlanningInterfaceEnum*). These messages should be published shortly after the corresponding planning services are ready to consume **PlanCommandMTs*, **PlanValidationCommandMT*, *AssessmentRequest*, *PlanScoresRequest*, **MetricsRequest*, etc depending on the interface supported by that particular service.

For best information on a given planner's availability, C2 should consider the last published *MA_PlanningFunctionMT* by that planner's unique *PlanningFunctionID* or consider using the C2 - Query Autonomy Settings message exchange.

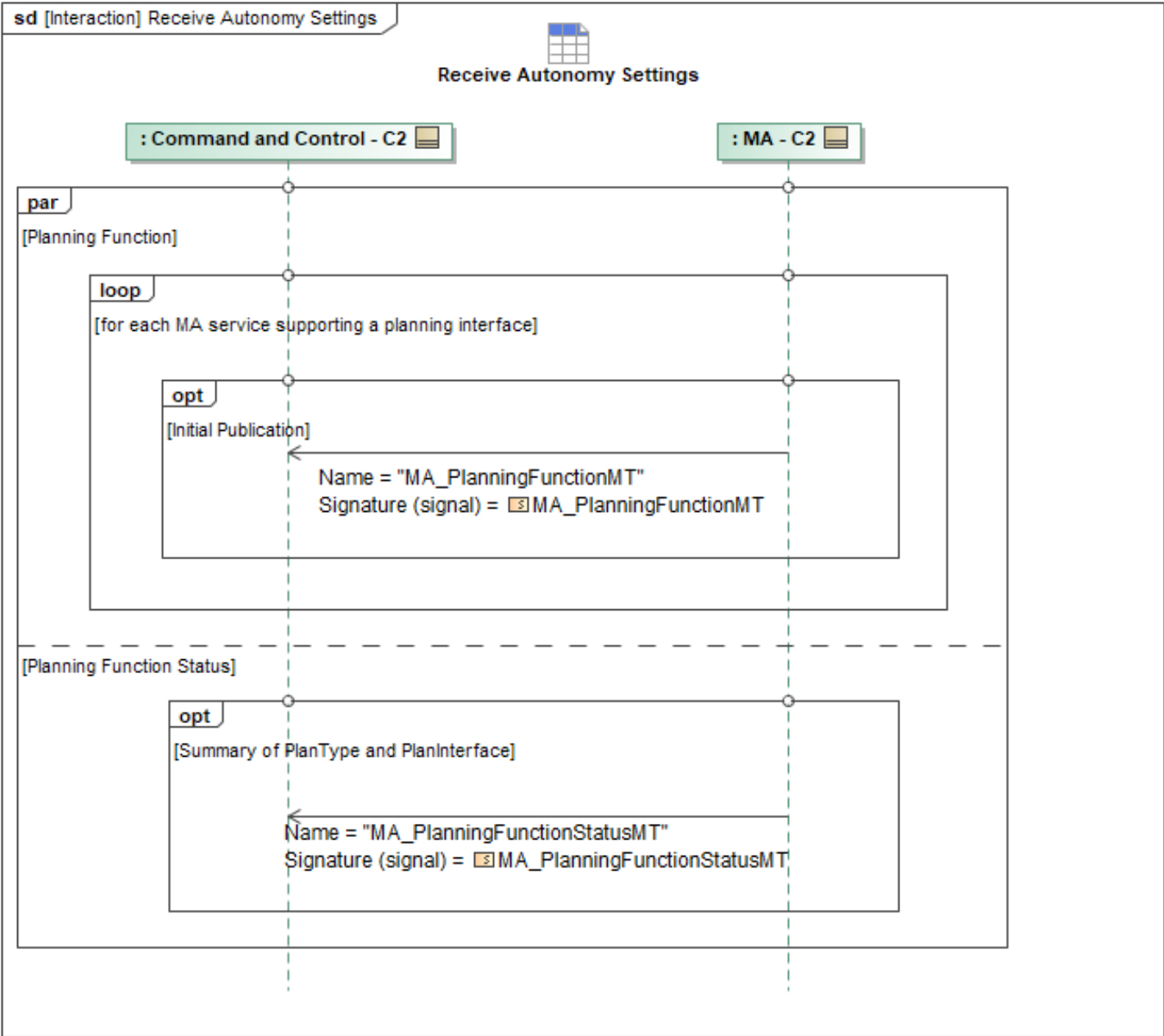


Figure 5-65: Receive Autonomy Settings

Table 5-64: Receive Autonomy Settings

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanningFunctionMT: MA_PlanningFunctionMT	None
MA_PlanningFunctionStatusMT: MA_PlanningFunctionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.14 Receive MA Plan Status

This interaction describes the process of MA providing status on known plans back to C2, enabling C2 to monitor the completeness and validity of all stored plan data. Unlike periodic status updates, this interaction is strictly event-driven and is only initiated when a change in the local plan store results in a status change, aiding in bookkeeping, error detection, and synchronization between MA and C2. For example, a MissionPlan links to a specific RoutePlan, and if that RoutePlan is modified or inadvertently removed by C2, MA automatically runs a consistency check, invalidates the parent MissionPlan, and issues a status message indicating the plan is no longer valid. After an update is completed, MA sends C2 a corresponding status message based on the *Plan that was updated via *MissionPlanStatusMT*, *TaskPlanStatusMT*, *ActivityPlanStatusMT*, *RouteActivityPlanStatusMT*, *RoutePlanStatusMT* or *ActionPlanStatusMT*. These messages convey the current completeness of the plan using the *PlanningState* field and the plan's integrity using the *ValidationState* field.

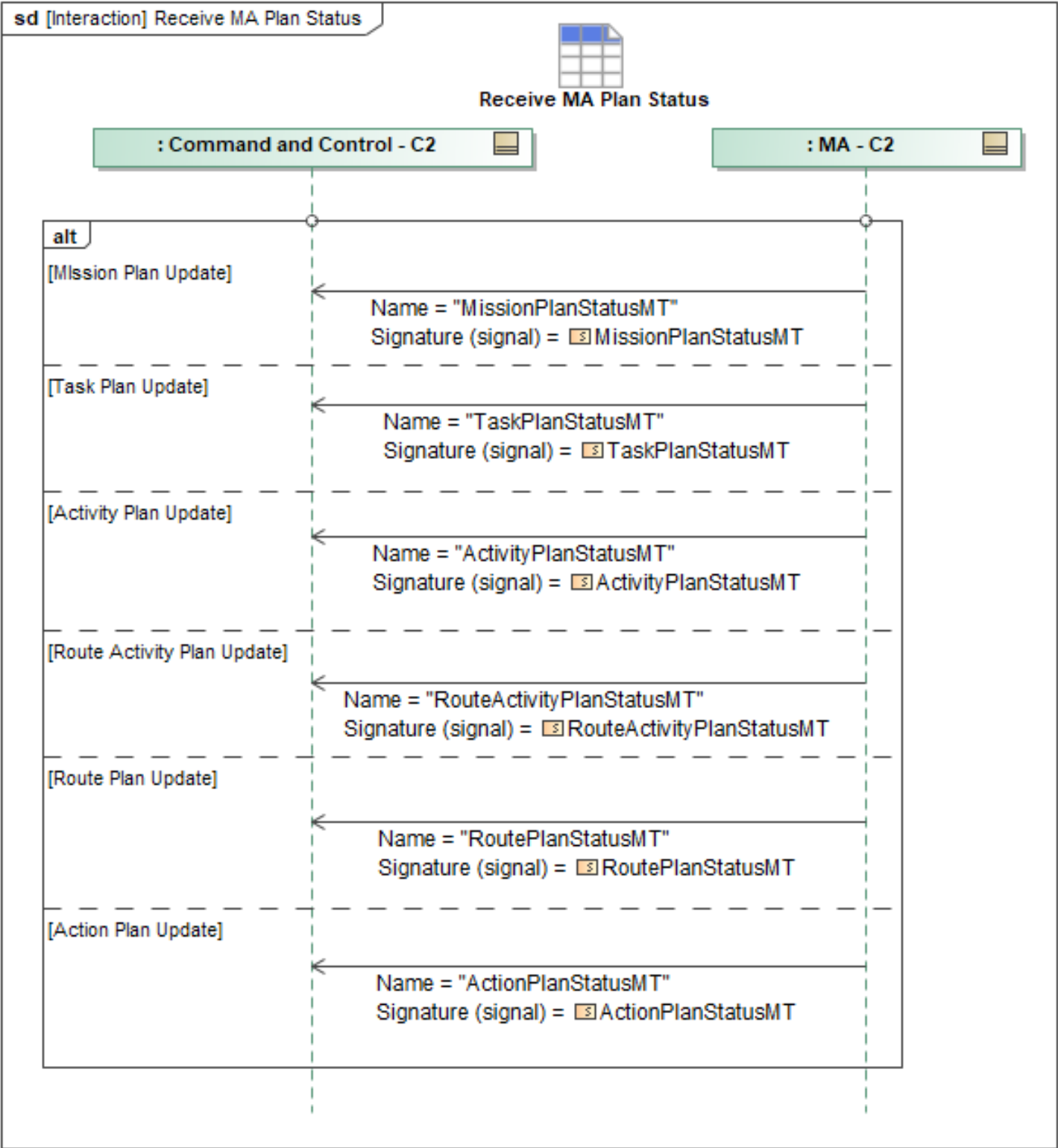


Figure 5-66: Receive MA Plan Status

Table 5-65: Receive MA Plan Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActionPlanStatusMT: ActionPlanStatusMT	None
ActivityPlanStatusMT: ActivityPlanStatusMT	None
MissionPlanStatusMT: MissionPlanStatusMT	None
RouteActivityPlanStatusMT: RouteActivityPlanStatusMT	None
RoutePlanStatusMT: RoutePlanStatusMT	None
TaskPlanStatusMT: TaskPlanStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.15 Respond to Airfield Report Query

AirfieldReportMT contains Airfield Data and is stored or cached in data manager services which may have a database backend. This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *QueryDataRequestStatusMT_1* and *QueryDataRequestStatusMT_2* messages with *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and "PROCESSING", respectively.

When complete, the *QueryDataRequestStatusMT_3* message's *RequestProcessingStateEnum* is set to "COMPLETED" and contains the requested data in the *Result* field.

When the original *QueryDataRequestMT*'s optional field *ResultsInNative* is populated with the matching message type, the underlying data management services instead publishes the requested data in its original or raw format as a separate message and indicates *QueryDataRequestStatusMT_3* message's *RequestProcessingStateEnum* to "COMPLETED".

If C2 is unable to complete the query it will ignore the *QueryDataRequestMT* command and reply with *QueryDataRequestStatusMT_4* where *RequestProcessingState* is set to "REJECTED". If the query is rejected because C2 does not have the requested information, the *Reason* field within *RequestProcessingStateReason* will be set to "UNKNOWN_ID". If the query is rejected because the missing requested information cannot be transmitted via UCI, the *Reason* field will be set instead to "INVALID_INPUT_PARAMETER".

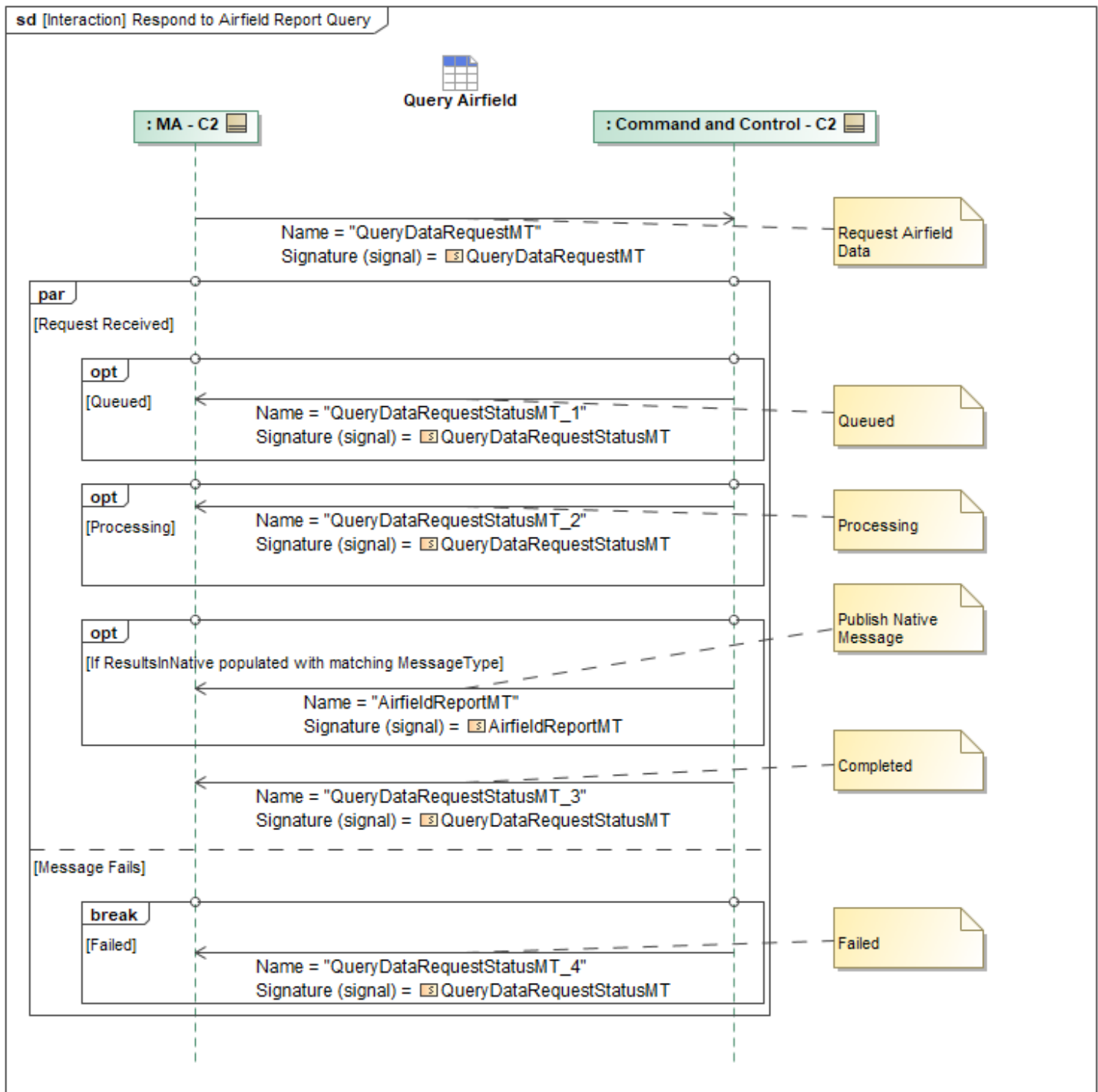


Figure 5-67: Respond to Airfield Report Query

Table 5-66: Respond to Airfield Report Query

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	None
QueryDataRequestMT: QueryDataRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.16 Send Response Plan

The diagram contains the collection of unique plan messages sent to Mission Autonomy following the logic described below for a contingency plan represented in the diagram. C2 systems will send an *MA_ApprovalPolicyMT* message for each unique contingency plan. The *DataRecordInstanceID* is included in the message to include tracking ID for the approval policy.

Response Plan Data is sent the via a *ResponsePlanMT* message for the plan that is being shared with Mission Autonomy. If the message being sent represents the first instance of the plan, the *ObjectState* field is set to "NEW". When sending an update to revise an existing plan, *ObjectState* is set to "UPDATED". The response plan message is populated with the definition to detail the plan being conveyed. MA will determine whether C2 is authorized to command any plans.

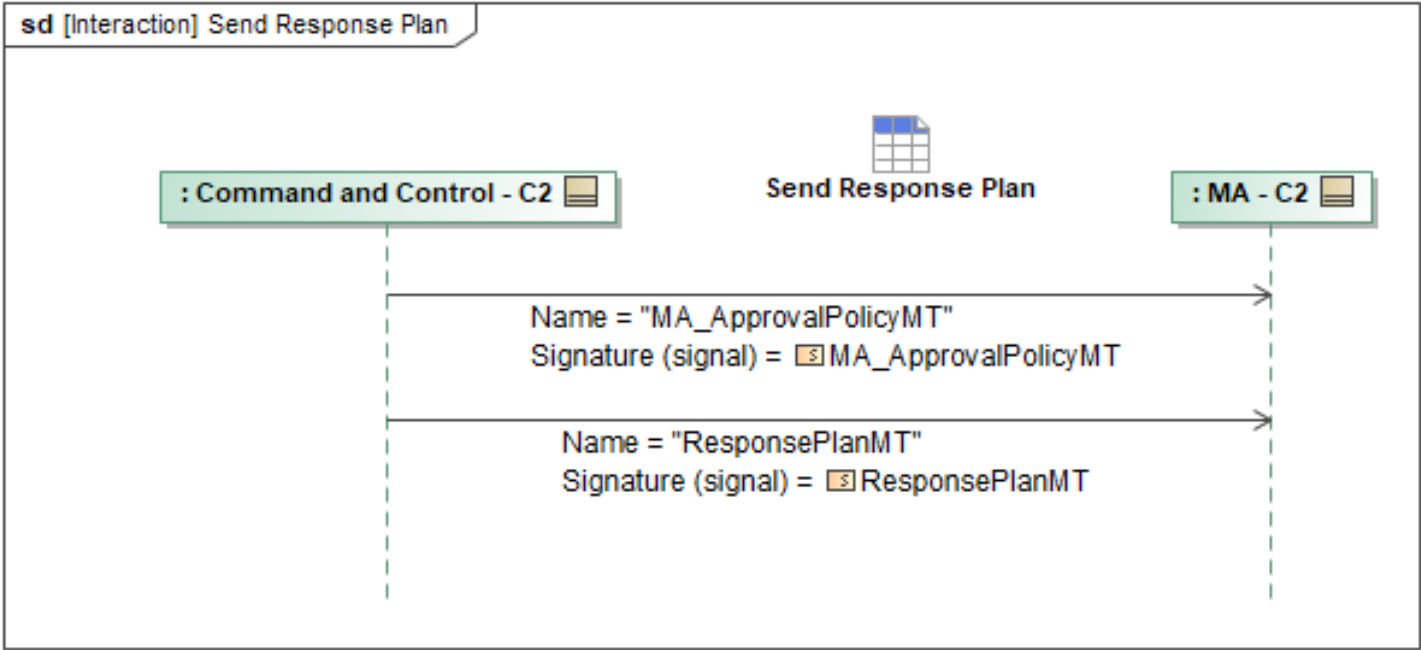


Figure 5-68: Send Response Plan

Table 5-67: Send Response Plan

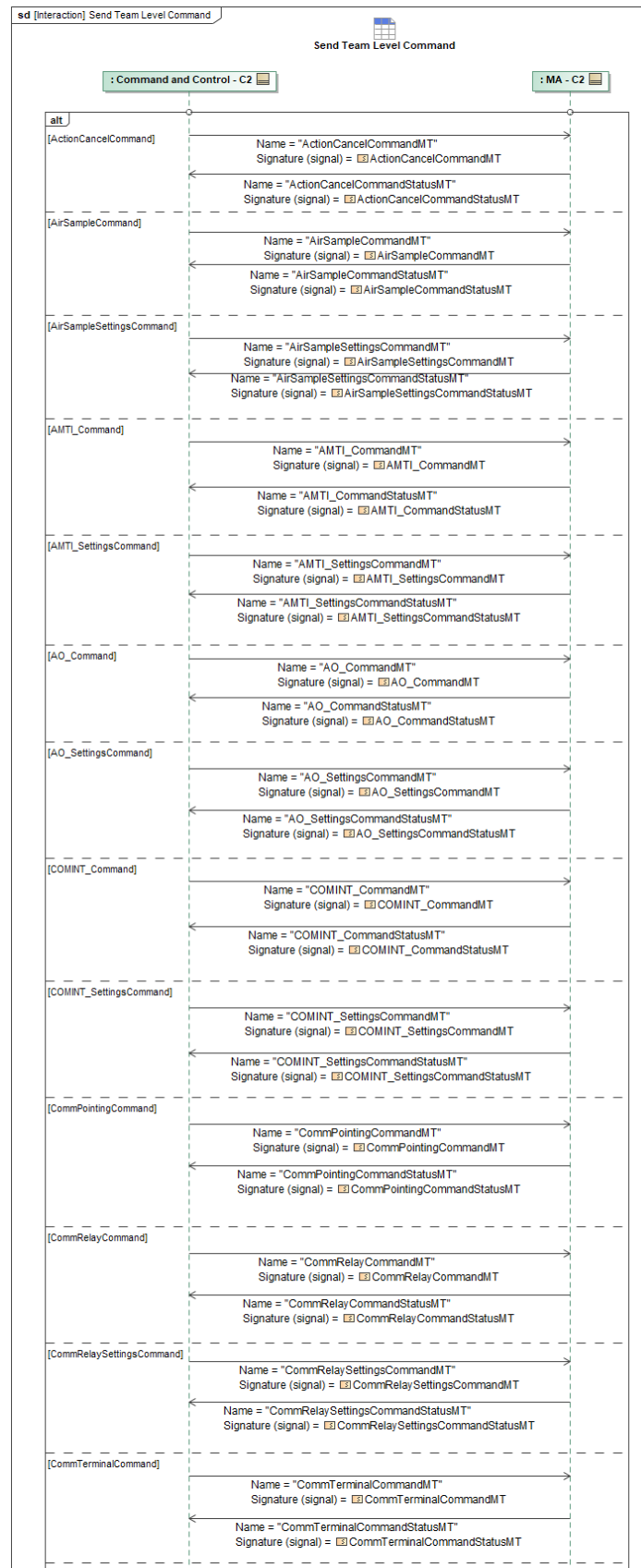
Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	DataRecordInstanceID: DataRecordInstanceID_Type
ResponsePlanMT: ResponsePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.17 Send Team Level Command

Package level tasking can be commanded to a team by C2. C2 sends a package level command (any UCI **CommandMT*) to the team lead MA, where the command can be decomposed or forwarded to the appropriate team members. Team lead MA responds to the command with a **CommandStatusMT* message that corresponds to the original command.

MA will respond to the original **CommandMT* message with a **CommandStatusMT* message response.



[CommTerminalPlanActivationCommand]	Name = "CommTerminalPlanActivationCommandMT" Signature (signal) = [] CommTerminalPlanActivationCommandMT
	Name = "CommTerminalPlanActivationCommandStatusMT" Signature (signal) = [] CommTerminalPlanActivationCommandStatusMT
[CommTerminalSettingsCommand]	Name = "CommTerminalSettingsCommandMT" Signature (signal) = [] CommTerminalSettingsCommandMT
	Name = "CommTerminalSettingsCommandStatusMT" Signature (signal) = [] CommTerminalSettingsCommandStatusMT
[DMPL_CancelCommand]	Name = "DMPL_CancelCommandMT" Signature (signal) = [] DMPL_CancelCommandMT
	Name = "DMPL_CancelCommandStatusMT" Signature (signal) = [] DMPL_CancelCommandStatusMT
[DoorCommand]	Name = "DoorCommandMT" Signature (signal) = [] DoorCommandMT
	Name = "DoorCommandStatusMT" Signature (signal) = [] DoorCommandStatusMT
[EA_Command]	Name = "EA_CommandMT" Signature (signal) = [] EA_CommandMT
	Name = "EA_CommandStatusMT" Signature (signal) = [] EA_CommandStatusMT
[ESM_Command]	Name = "ESM_CommandMT" Signature (signal) = [] ESM_CommandMT
	Name = "ESM_CommandStatusMT" Signature (signal) = [] ESM_CommandStatusMT
[ESM_SettingsCommand]	Name = "ESM_SettingsCommandMT" Signature (signal) = [] ESM_SettingsCommandMT
	Name = "ESM_SettingsCommandStatusMT" Signature (signal) = [] ESM_SettingsCommandStatusMT
[IFF_Command]	Name = "IFF_CommandMT" Signature (signal) = [] IFF_CommandMT
	Name = "IFF_CommandStatusMT" Signature (signal) = [] IFF_CommandStatusMT
[IFF_SettingsCommand]	Name = "IFF_SettingsCommandMT" Signature (signal) = [] IFF_SettingsCommandMT
	Name = "IFF_SettingsCommandStatusMT" Signature (signal) = [] IFF_SettingsCommandStatusMT
[PO_Command]	Name = "PO_CommandMT" Signature (signal) = [] PO_CommandMT
	Name = "PO_CommandStatusMT" Signature (signal) = [] PO_CommandStatusMT
[PO_SettingsCommand]	Name = "PO_SettingsCommandMT" Signature (signal) = [] PO_SettingsCommandMT
	Name = "PO_SettingsCommandStatusMT" Signature (signal) = [] PO_SettingsCommandStatusMT
[RadarAltimeterCommand]	Name = "RadarAltimeterCommandMT" Signature (signal) = [] RadarAltimeterCommandMT
	Name = "RadarAltimeterCommandStatusMT" Signature (signal) = [] RadarAltimeterCommandStatusMT
[RadarAltimeterSettingsCommand]	Name = "RadarAltimeterSettingsCommandMT" Signature (signal) = [] RadarAltimeterSettingsCommandMT
	Name = "RadarAltimeterSettingsCommandStatusMT" Signature (signal) = [] RadarAltimeterSettingsCommandStatusMT
[RefuelCommand]	Name = "RefuelCommandMT" Signature (signal) = [] RefuelCommandMT
	Name = "RefuelCommandStatusMT" Signature (signal) = [] RefuelCommandStatusMT

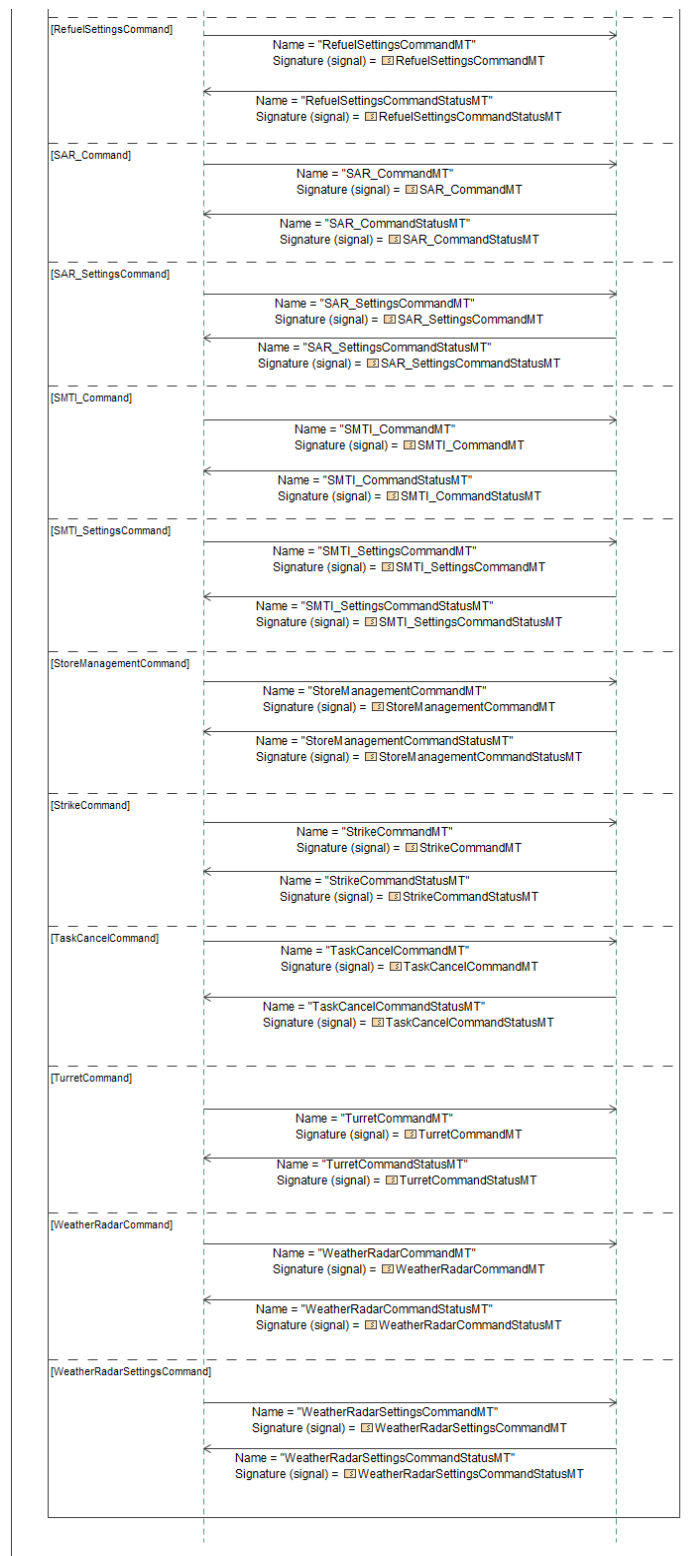


Figure 5-69: Send Team Level Command

Table 5-68: Send Team Level Command

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCancelCommandMT: ActionCancelCommandMT	None
ActionCancelCommandStatusMT: ActionCancelCommandStatusMT	None
AirSampleCommandMT: AirSampleCommandMT	None
AirSampleCommandStatusMT: AirSampleCommandStatusMT	None
AirSampleSettingsCommandMT: AirSampleSettingsCommandMT	None
AirSampleSettingsCommandStatusMT: AirSampleSettingsCommandStatusMT	None
AMTI_CommandMT: AMTI_CommandMT	None
AMTI_CommandStatusMT: AMTI_CommandStatusMT	None
AMTI_SettingsCommandMT: AMTI_SettingsCommandMT	None
AMTI_SettingsCommandStatusMT: AMTI_SettingsCommandStatusMT	None
AO_CommandMT: AO_CommandMT	None
AO_CommandStatusMT: AO_CommandStatusMT	None
AO_SettingsCommandMT: AO_SettingsCommandMT	None
AO_SettingsCommandStatusMT: AO_SettingsCommandStatusMT	None
COMINT_CommandMT: COMINT_CommandMT	None
COMINT_CommandStatusMT: COMINT_CommandStatusMT	None
COMINT_SettingsCommandMT: COMINT_SettingsCommandMT	None
COMINT_SettingsCommandStatusMT: COMINT_SettingsCommandStatusMT	None
CommPointingCommandMT: CommPointingCommandMT	None
CommPointingCommandStatusMT: CommPointingCommandStatusMT	None
CommRelayCommandMT: CommRelayCommandMT	None
CommRelayCommandStatusMT: CommRelayCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
CommRelaySettingsCommandMT: CommRelaySettingsCommandMT	None
CommRelaySettingsCommandStatusMT: CommRelaySettingsCommandStatusMT	None
CommTerminalCommandMT: CommTerminalCommandMT	None
CommTerminalCommandStatusMT: CommTerminalCommandStatusMT	None
CommTerminalPlanActivationCommandMT: CommTerminalPlanActivationCommandMT	None
CommTerminalPlanActivationCommandStatusMT: CommTerminalPlanActivationCommandStatusMT	None
CommTerminalSettingsCommandMT: CommTerminalSettingsCommandMT	None
CommTerminalSettingsCommandStatusMT: CommTerminalSettingsCommandStatusMT	None
DMPI_CancelCommandMT: DMPI_CancelCommandMT	None
DMPI_CancelCommandStatusMT: DMPI_CancelCommandStatusMT	None
DoorCommandMT: DoorCommandMT	None
DoorCommandStatusMT: DoorCommandStatusMT	None
EA_CommandMT: EA_CommandMT	None
EA_CommandStatusMT: EA_CommandStatusMT	None
ESM_CommandMT: ESM_CommandMT	None
ESM_CommandStatusMT: ESM_CommandStatusMT	None
ESM_SettingsCommandMT: ESM_SettingsCommandMT	None
ESM_SettingsCommandStatusMT: ESM_SettingsCommandStatusMT	None
IFF_CommandMT: IFF_CommandMT	None
IFF_CommandStatusMT: IFF_CommandStatusMT	None
IFF_SettingsCommandMT: IFF_SettingsCommandMT	None
IFF_SettingsCommandStatusMT: IFF_SettingsCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
PO_CommandMT: PO_CommandMT	None
PO_CommandStatusMT: PO_CommandStatusMT	None
PO_SettingsCommandMT: PO_SettingsCommandMT	None
PO_SettingsCommandStatusMT: PO_SettingsCommandStatusMT	None
RadarAltimeterCommandMT: RadarAltimeterCommandMT	None
RadarAltimeterCommandStatusMT: RadarAltimeterCommandStatusMT	None
RadarAltimeterSettingsCommandMT: RadarAltimeterSettingsCommandMT	None
RadarAltimeterSettingsCommandStatusMT: RadarAltimeterSettingsCommandStatusMT	None
RefuelCommandMT: RefuelCommandMT	None
RefuelCommandStatusMT: RefuelCommandStatusMT	None
RefuelSettingsCommandMT: RefuelSettingsCommandMT	None
RefuelSettingsCommandStatusMT: RefuelSettingsCommandStatusMT	None
SAR_CommandMT: SAR_CommandMT	None
SAR_CommandStatusMT: SAR_CommandStatusMT	None
SAR_SettingsCommandMT: SAR_SettingsCommandMT	None
SAR_SettingsCommandStatusMT: SAR_SettingsCommandStatusMT	None
SMTI_CommandMT: SMTI_CommandMT	None
SMTI_CommandStatusMT: SMTI_CommandStatusMT	None
SMTI_SettingsCommandMT: SMTI_SettingsCommandMT	None
SMTI_SettingsCommandStatusMT: SMTI_SettingsCommandStatusMT	None
StoreManagementCommandMT: StoreManagementCommandMT	None
StoreManagementCommandStatusMT: StoreManagementCommandStatusMT	None
StrikeCommandMT: StrikeCommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
StrikeCommandStatusMT: StrikeCommandStatusMT	None
TaskCancelCommandMT: TaskCancelCommandMT	None
TaskCancelCommandStatusMT: TaskCancelCommandStatusMT	None
TurretCommandMT: TurretCommandMT	None
TurretCommandStatusMT: TurretCommandStatusMT	None
WeatherRadarCommandMT: WeatherRadarCommandMT	None
WeatherRadarCommandStatusMT: WeatherRadarCommandStatusMT	None
WeatherRadarSettingsCommandMT: WeatherRadarSettingsCommandMT	None
WeatherRadarSettingsCommandStatusMT: WeatherRadarSettingsCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.18 Synchronize Contingency Plan

This diagram contains the messages sent between MA and C2 to synchronize an updated contingency plan. The *MA_ApprovalPolicyMT* and *ResponsePlanMT* will be sent with an *ObjectState* enumeration set to *UPDATE*.

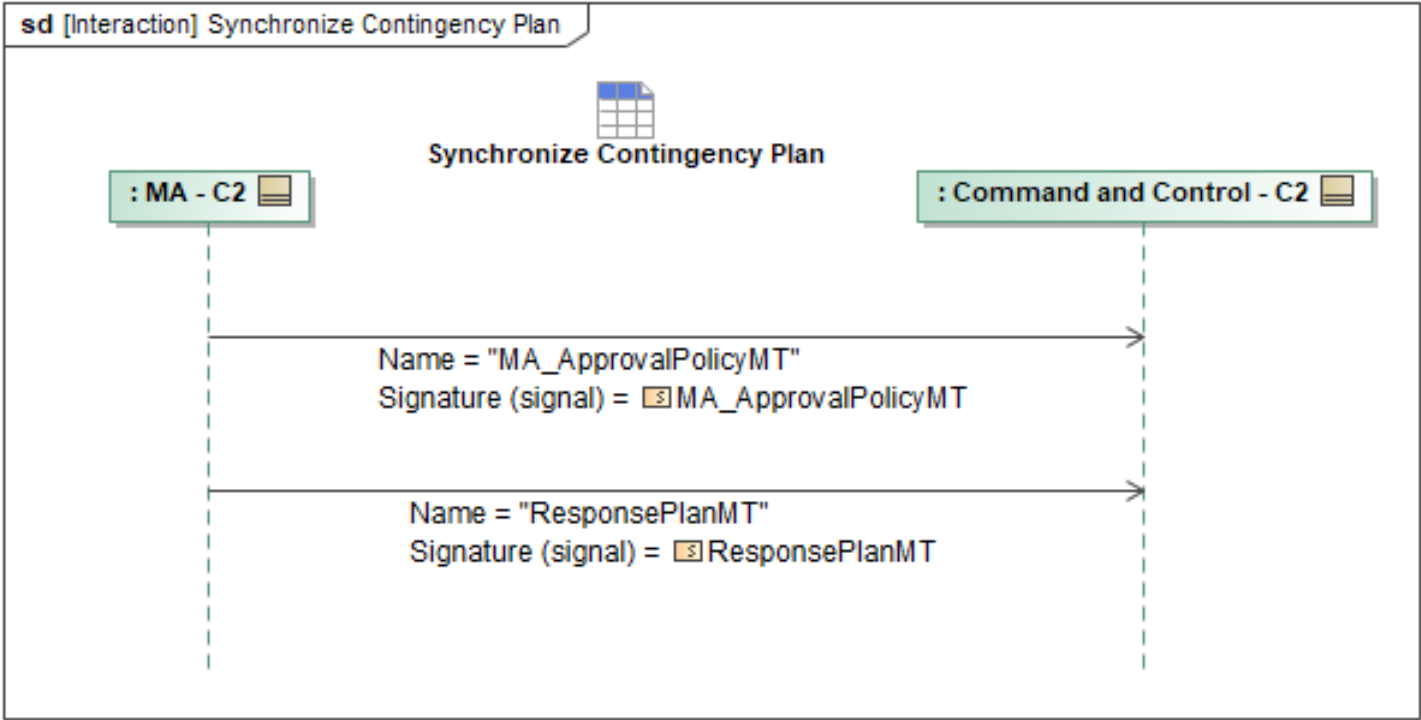


Figure 5-70: Synchronize Contingency Plan

Table 5-69: Synchronize Contingency Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	DataRecordInstanceID: DataRecordInstanceID_Type
ResponsePlanMT: ResponsePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.19 Update C2 with Package Mission Plan Synchronization Status

This interaction describes the process of the MA sending the C2 an update on the package mission plan synchronization status. This is done through a *MA_SystemNotificationMT* message with the *ApplicableToID* field containing either the relevant package or system ID. This allows a reference to the mission plan’s ID in order for the recipient to understand which plan was synchronized.

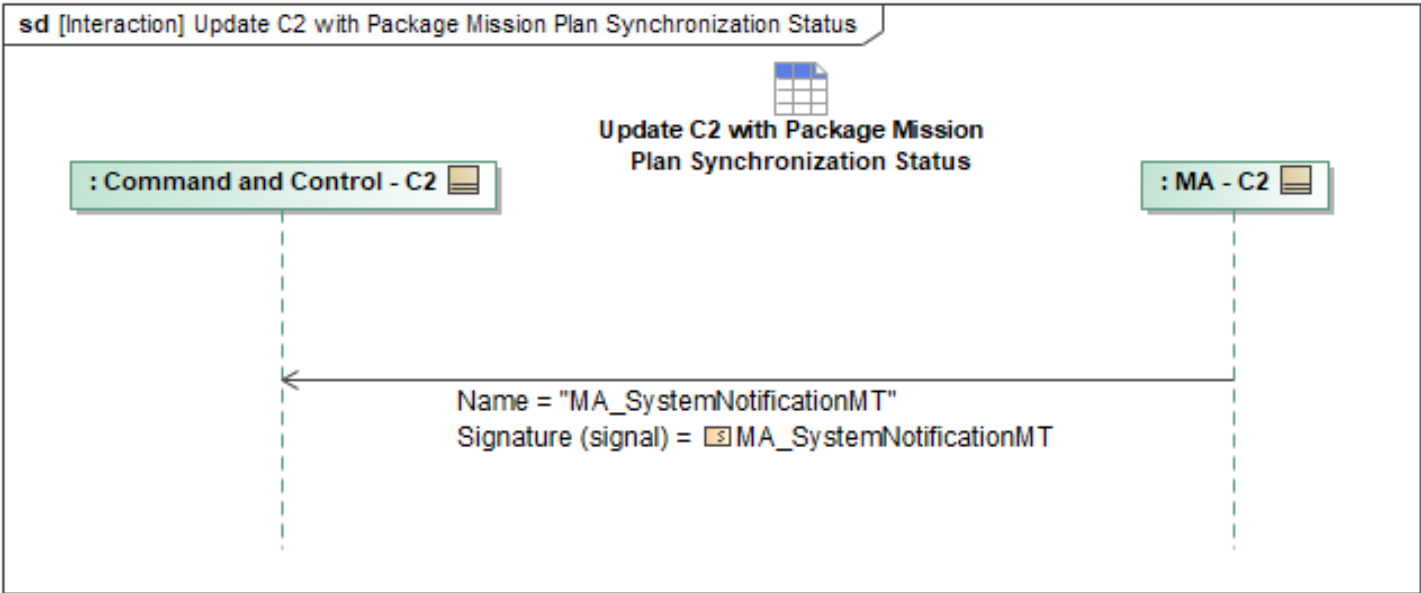


Figure 5-71: Update C2 with Package Mission Plan Synchronization Status

Table 5-70: Update C2 with Package Mission Plan Synchronization Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	AppliesTo: SubjectType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.20 Zone Alerts

When MA detects a conflict between a geozone and a plan/requirement (for example, a RoutePlan into a KOZ). MA sends an *OpNotificationMT* to alert C2 of the conflict. The geozone with the conflict is set within the *OpZoneID* field inside the *OpSubject* field. *AssociatedMessage* fields contain each conflicted Route, Plan, Requirement, etc., with the respective *AssociatedID* filled. *NotificationNarrative* contains a human-readable version of the conflict.

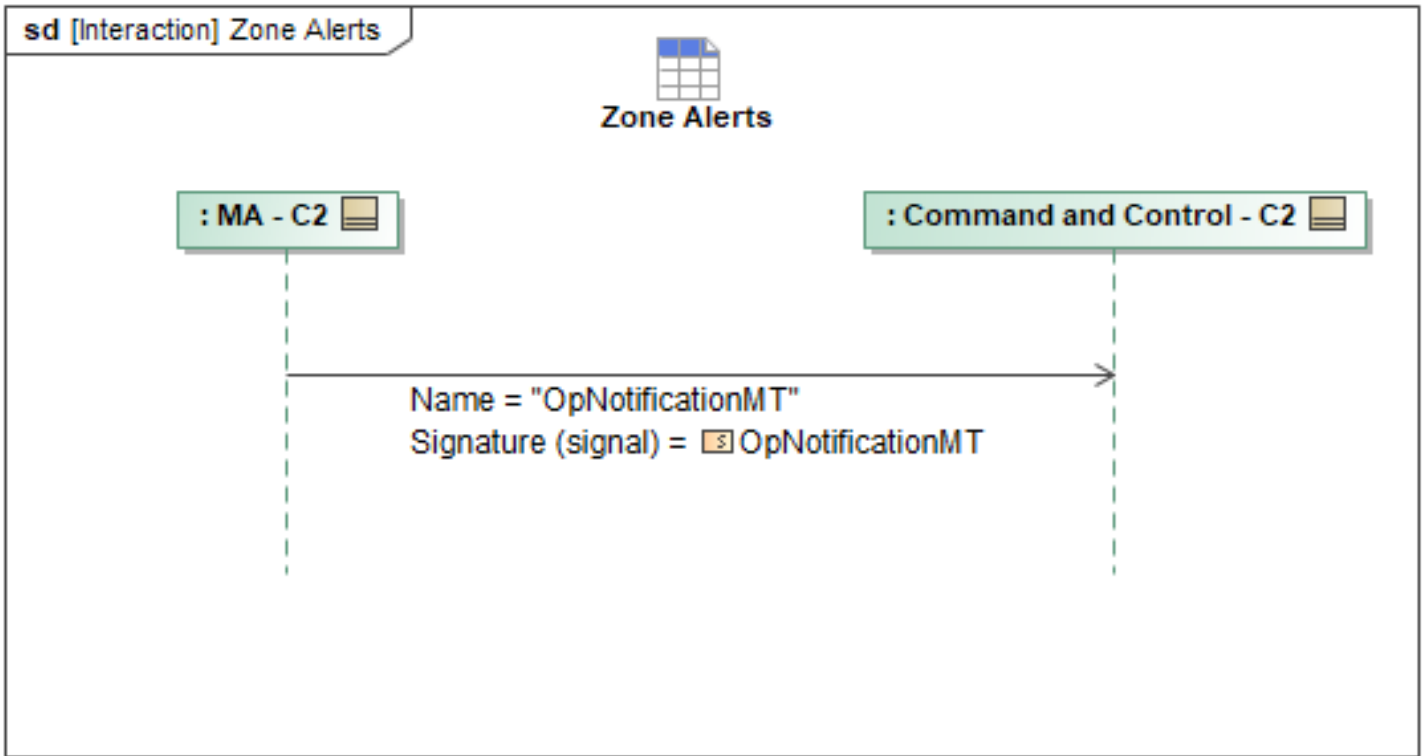


Figure 5-72: Zone Alerts

Table 5-71: Zone Alerts

Message Name (Name:Type)	Required Fields (Name:Type)
OpNotificationMT: OpNotificationMT	AssociatedID: ID_Type NotificationNarrative: VisibleString1024Type AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21 Query Data

5.2.8.21.1 Query Activity Plan

MA_ActivityPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_ActivityPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the

matching *MA_ActivityPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_ActivityPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_ActivityPlanMT*.

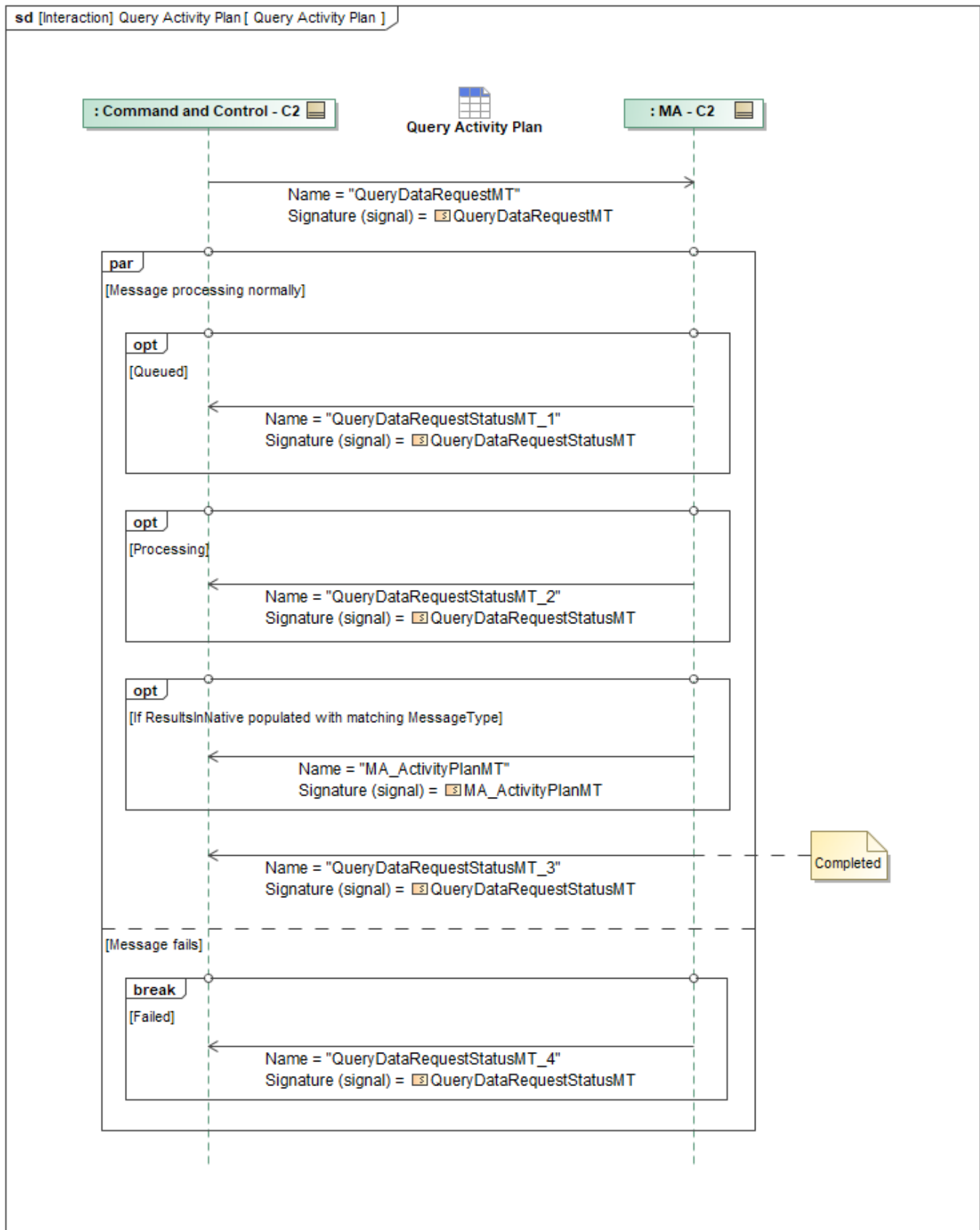


Figure 5-73: Query Activity Plan

Table 5-72: Query Activity Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.2 Query Activity Plan Execution Status

ActivityPlanExecutionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *ActivityPlanExecutionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *ActivityPlanExecutionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *ActivityPlanExecutionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *ActivityPlanExecutionStatusMT*.

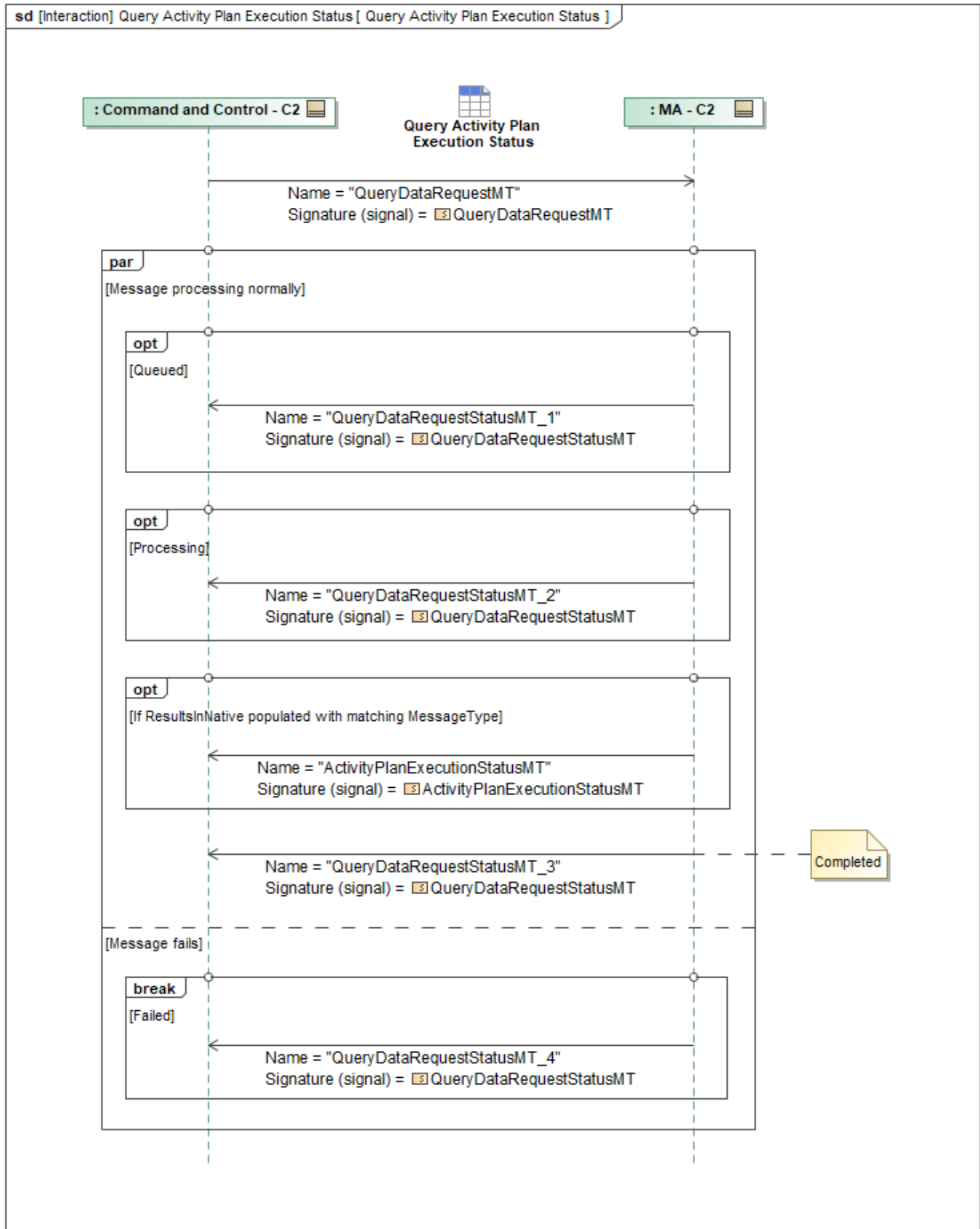


Figure 5-74: Query Activity Plan Execution Status

Table 5-73: Query Activity Plan Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityPlanExecutionStatusMT: ActivityPlanExecutionStatusMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.3 Query Airfield Report

AirfieldReportMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *AirfieldReportMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *AirfieldReportMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *AirfieldReportMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *AirfieldReportMT*.

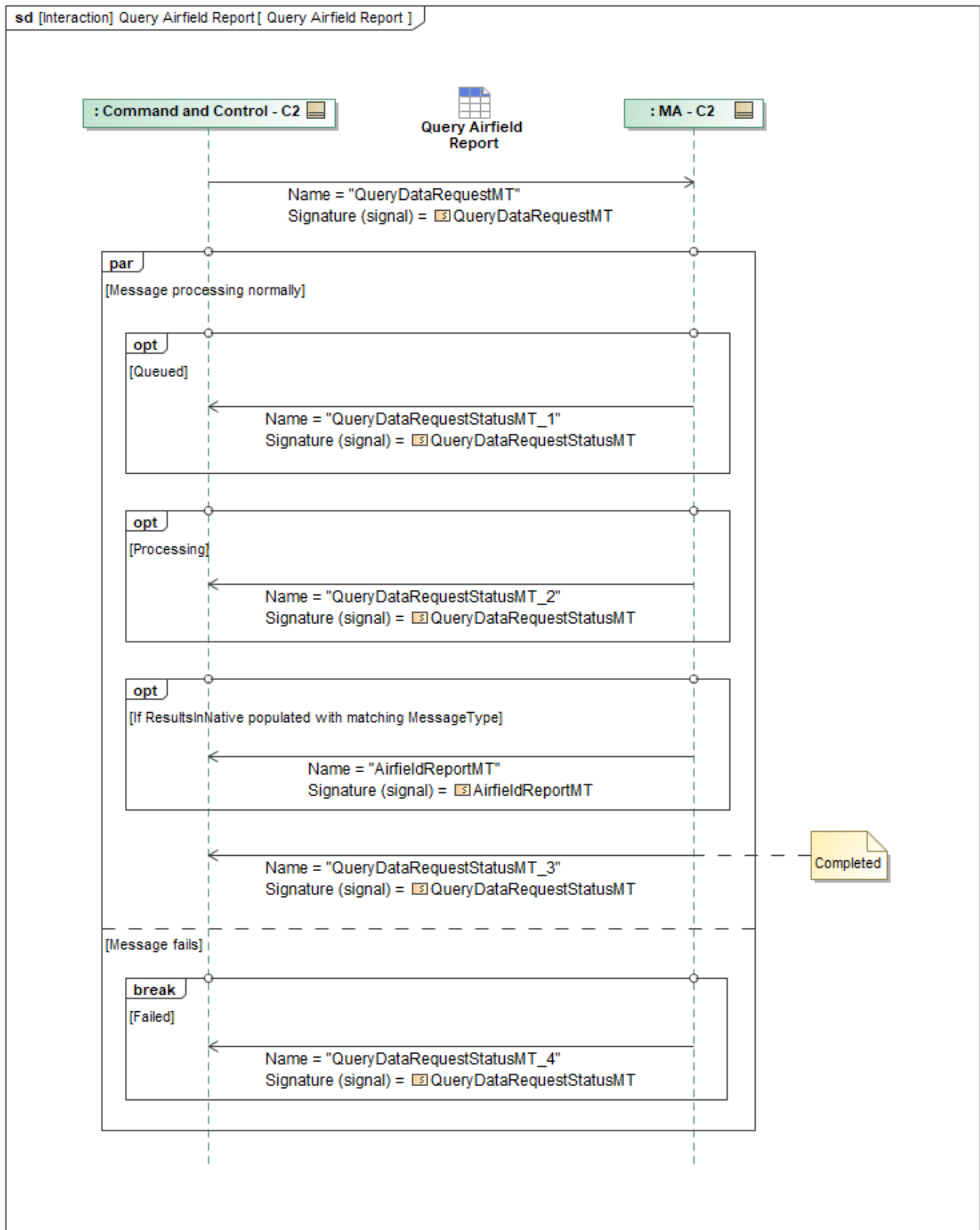


Figure 5-75: Query Airfield Report

Table 5-74: Query Airfield Report

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.4 Query Control Status

ControlStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *ControlStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *ControlStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *ControlStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *ControlStatusMT*.

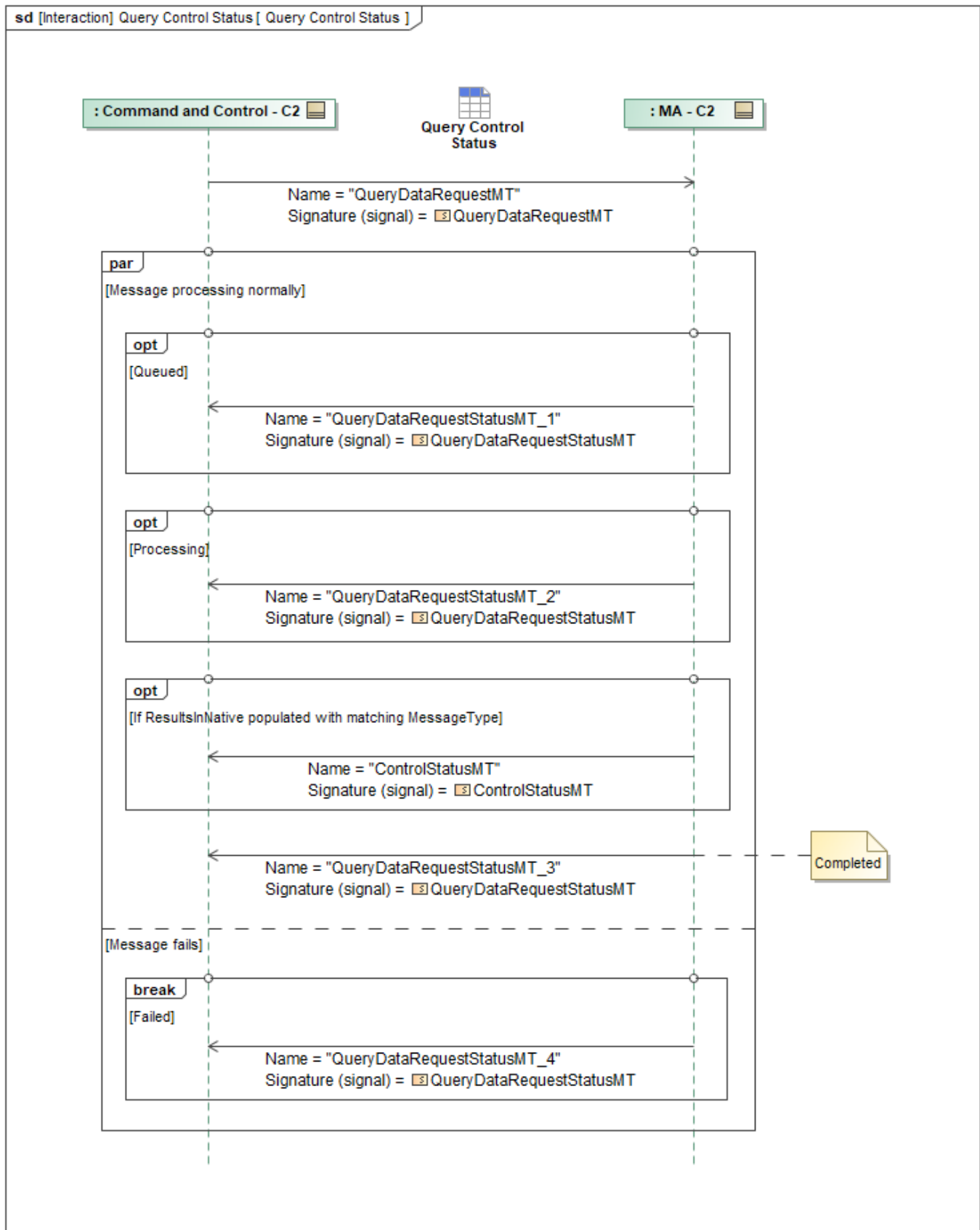


Figure 5-76: Query Control Status

Table 5-75: Query Control Status

Message Name (Name:Type)	Required Fields (Name:Type)
ControlStatusMT: ControlStatusMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.5 Query Mission Plan

MA_MissionPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_MissionPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_MissionPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_MissionPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_MissionPlanMT*.

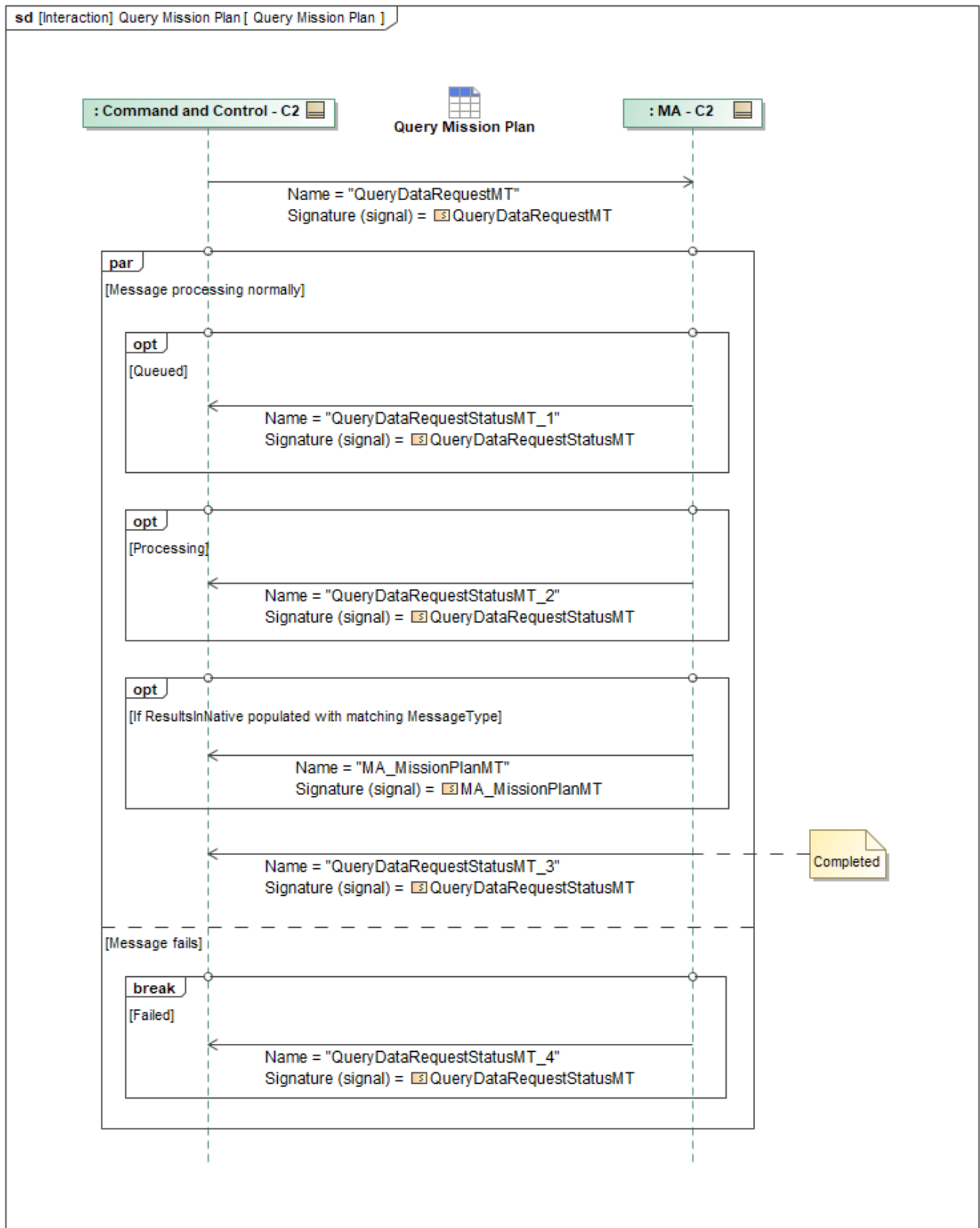


Figure 5-77: Query Mission Plan

Table 5-76: Query Mission Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	SubPlans: MA_PlansReferenceBaseType
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.6 Query Mission Plan Activation Status

MA_MissionPlanActivationStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_MissionPlanActivationStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_MissionPlanActivationStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_MissionPlanActivationStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_MissionPlanActivationStatusMT*.

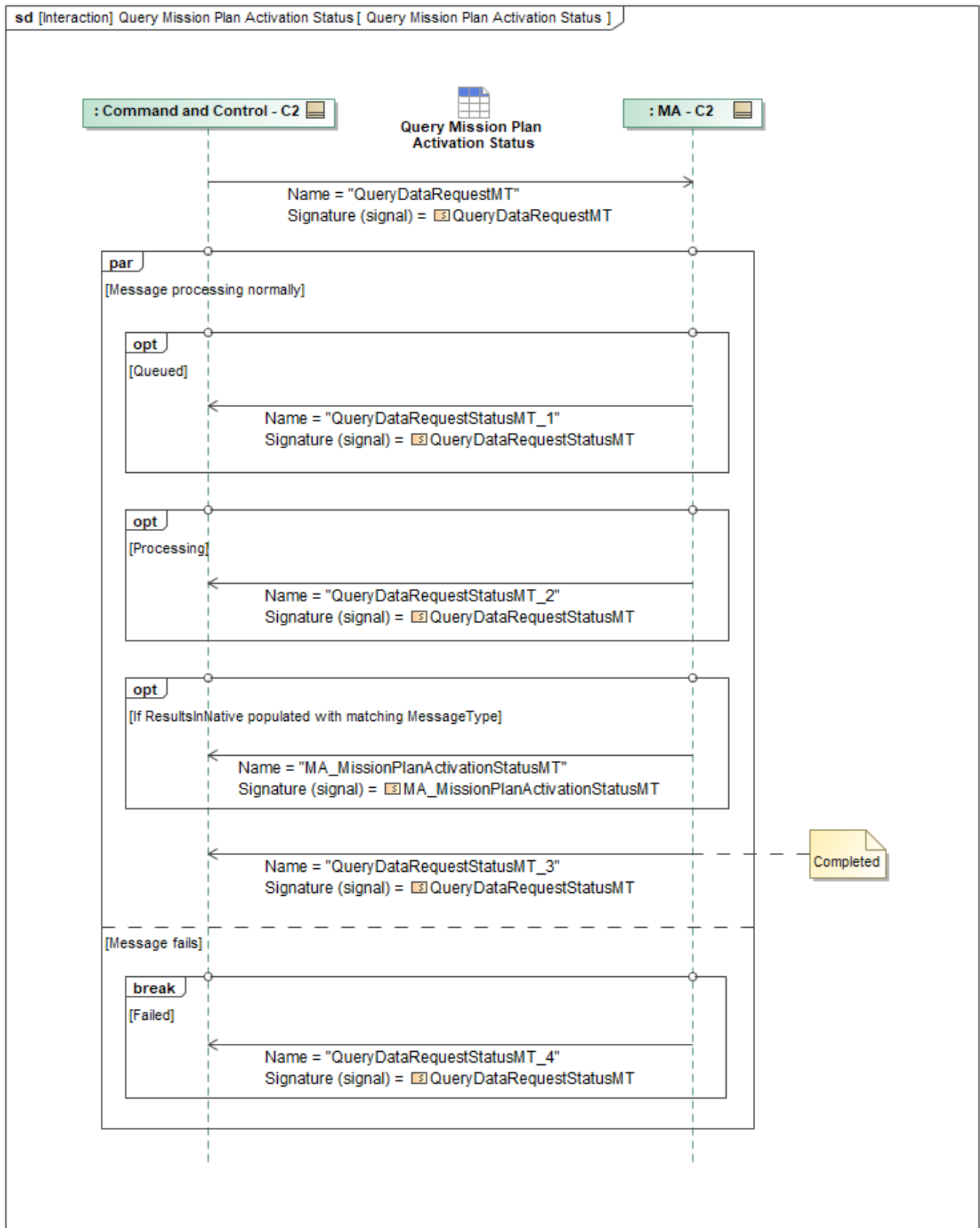


Figure 5-78: Query Mission Plan Activation Status

Table 5-77: Query Mission Plan Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.7 Query Mission Plan Execution Status

MA_MissionPlanExecutionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_MissionPlanExecutionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_MissionPlanExecutionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_MissionPlanExecutionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_MissionPlanExecutionStatusMT*.

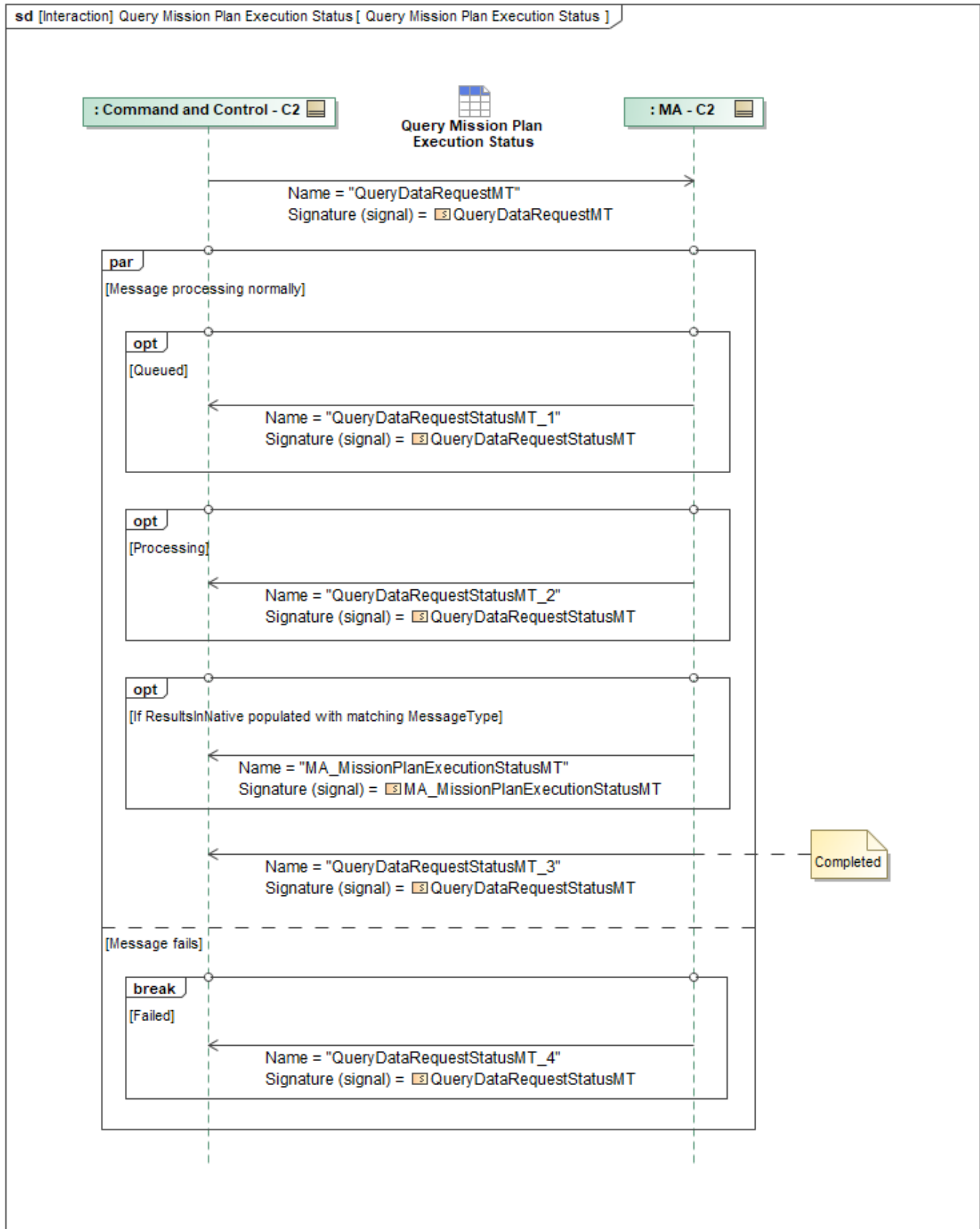


Figure 5-79: Query Mission Plan Execution Status

Table 5-78: Query Mission Plan Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.8 Query OpPoint

MA_OpPointMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_OpPointMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_OpPointMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_OpPointMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_OpPointMT*.

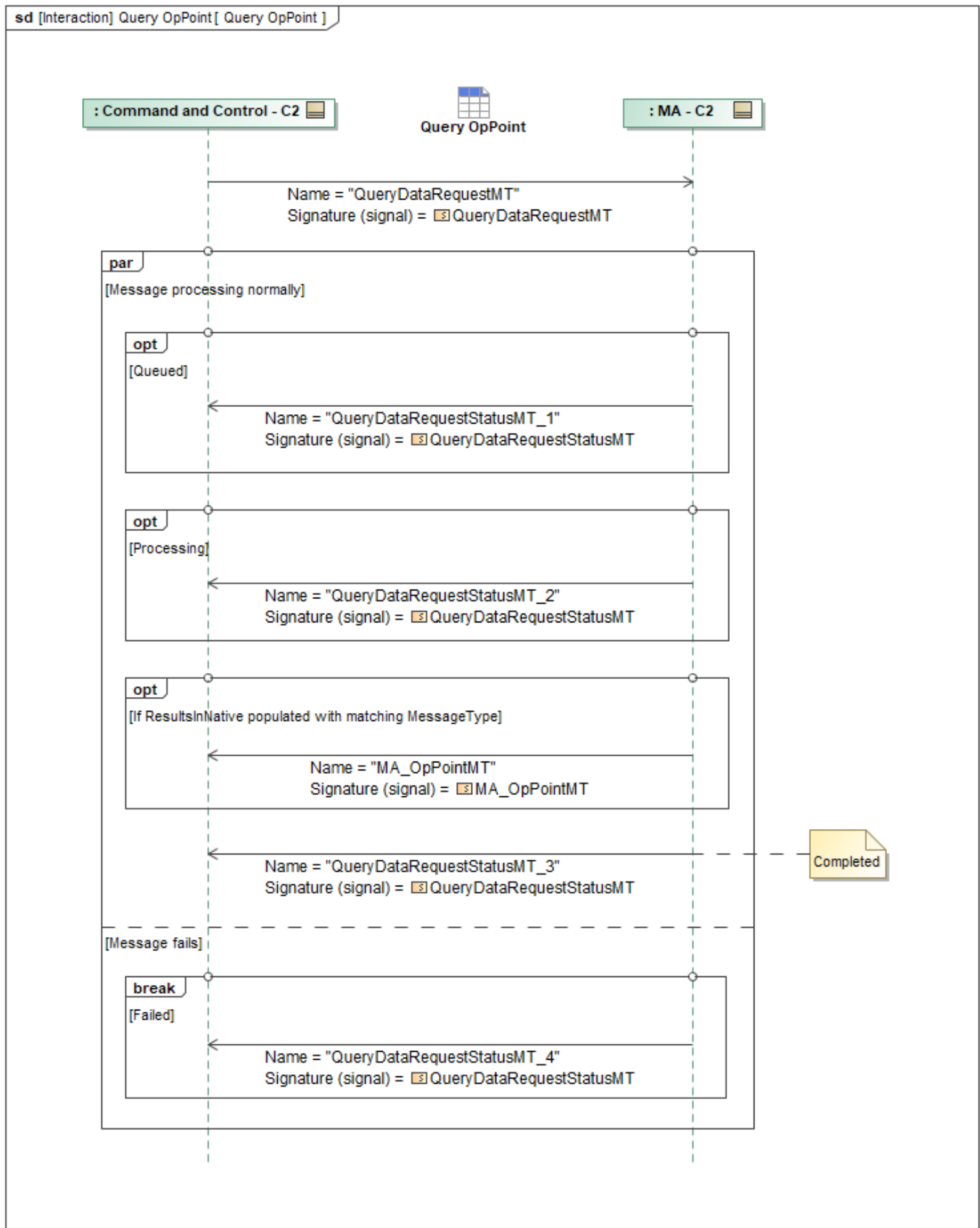


Figure 5-80: Query OpPoint

Table 5-79: Query OpPoint

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPointMT: MA_OpPointMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.9 Query OpZone

MA_OpZoneMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_OpZoneMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_OpZoneMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_OpZoneMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_OpZoneMT*.

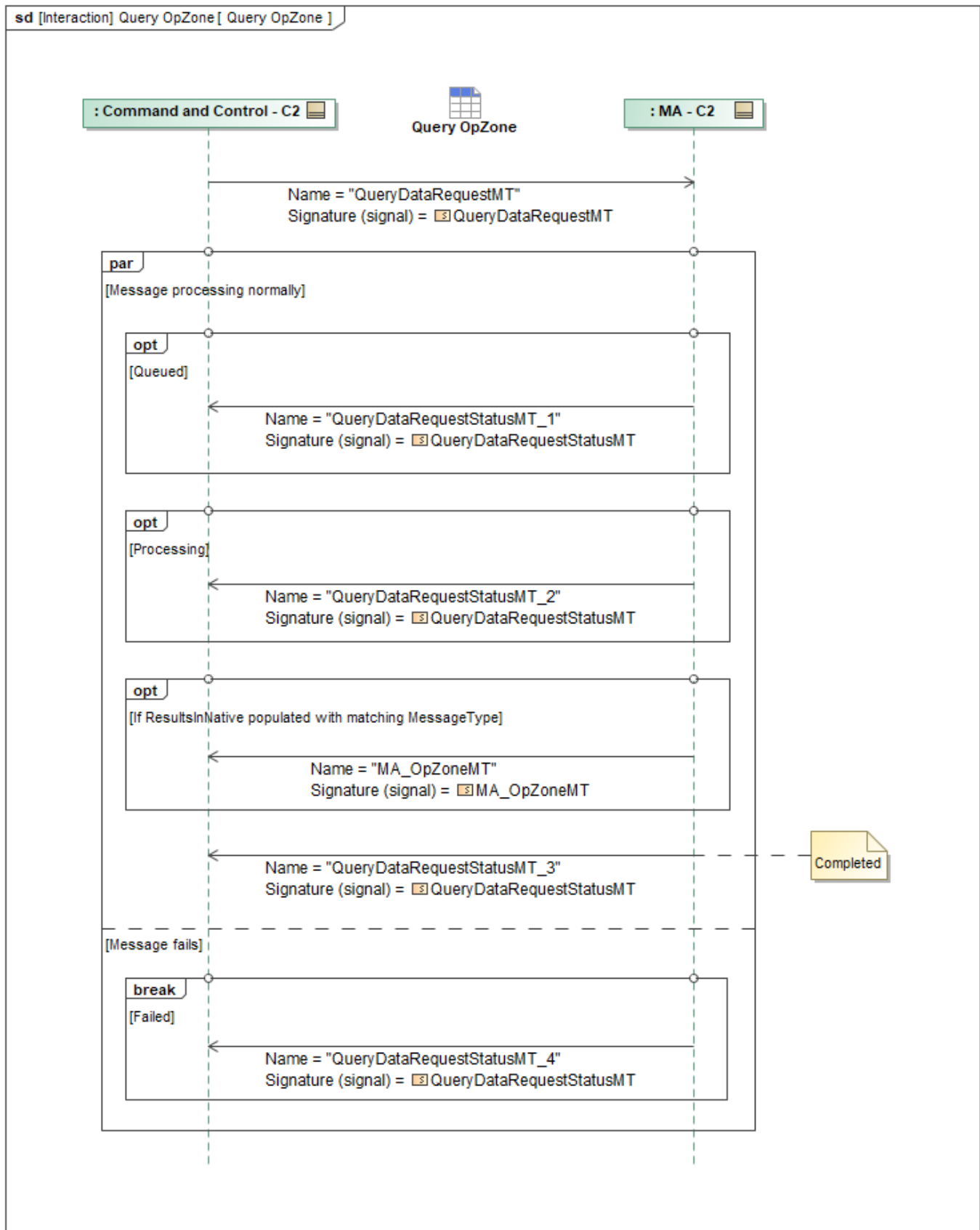


Figure 5-81: Query OpZone

Table 5-80: Query OpZone

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpZoneMT: MA_OpZoneMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.10 Query Package

PackageMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *PackageMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *PackageMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *PackageMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *PackageMT*.

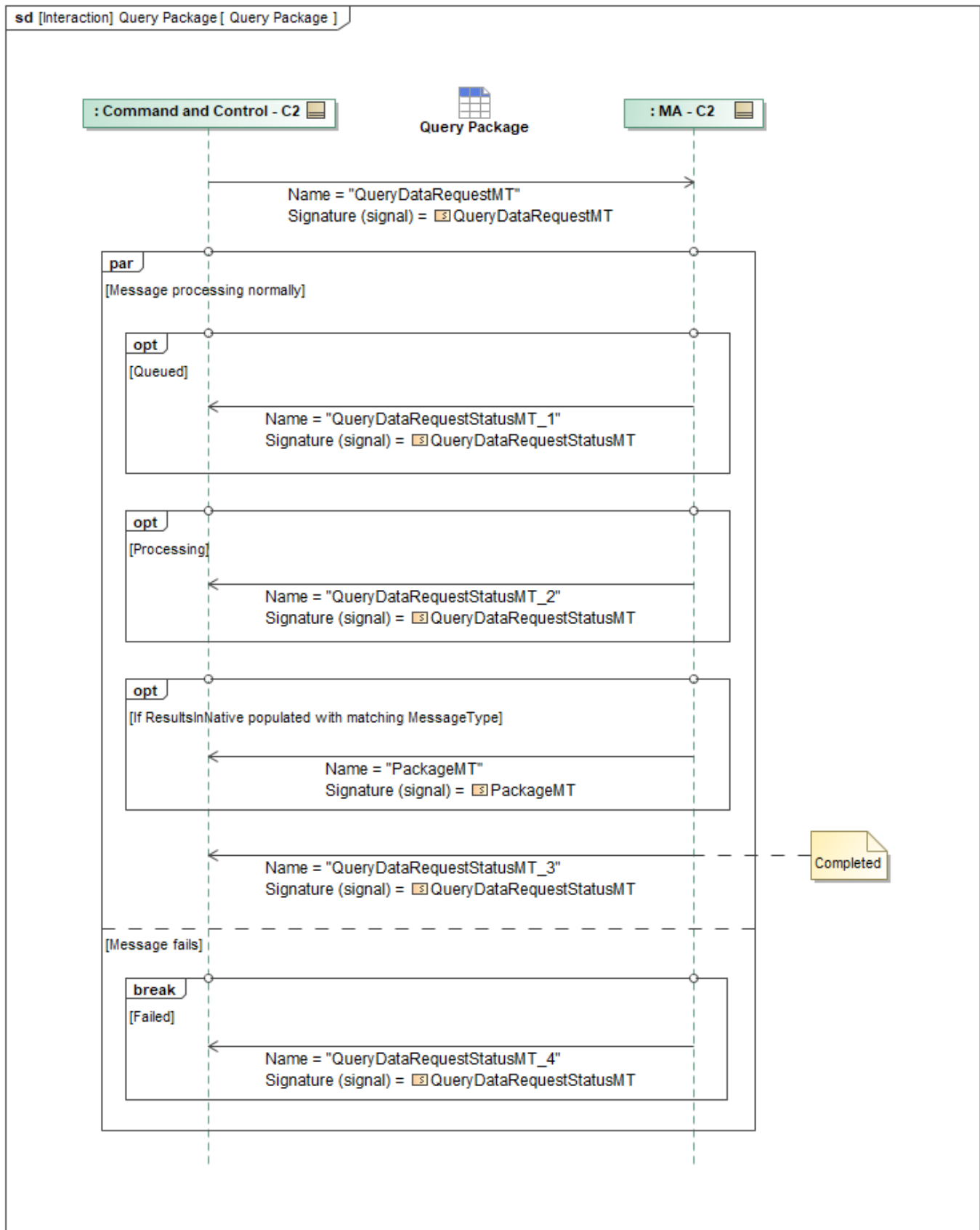


Figure 5-82: Query Package

Table 5-81: Query Package

Message Name (Name:Type)	Required Fields (Name:Type)
PackageMT: PackageMT	ObjectState: ObjectStateEnum
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.11 Query Planning Function

MA_PlanningFunctionMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_PlanningFunctionMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_PlanningFunctionMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_PlanningFunctionMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_PlanningFunctionMT*.

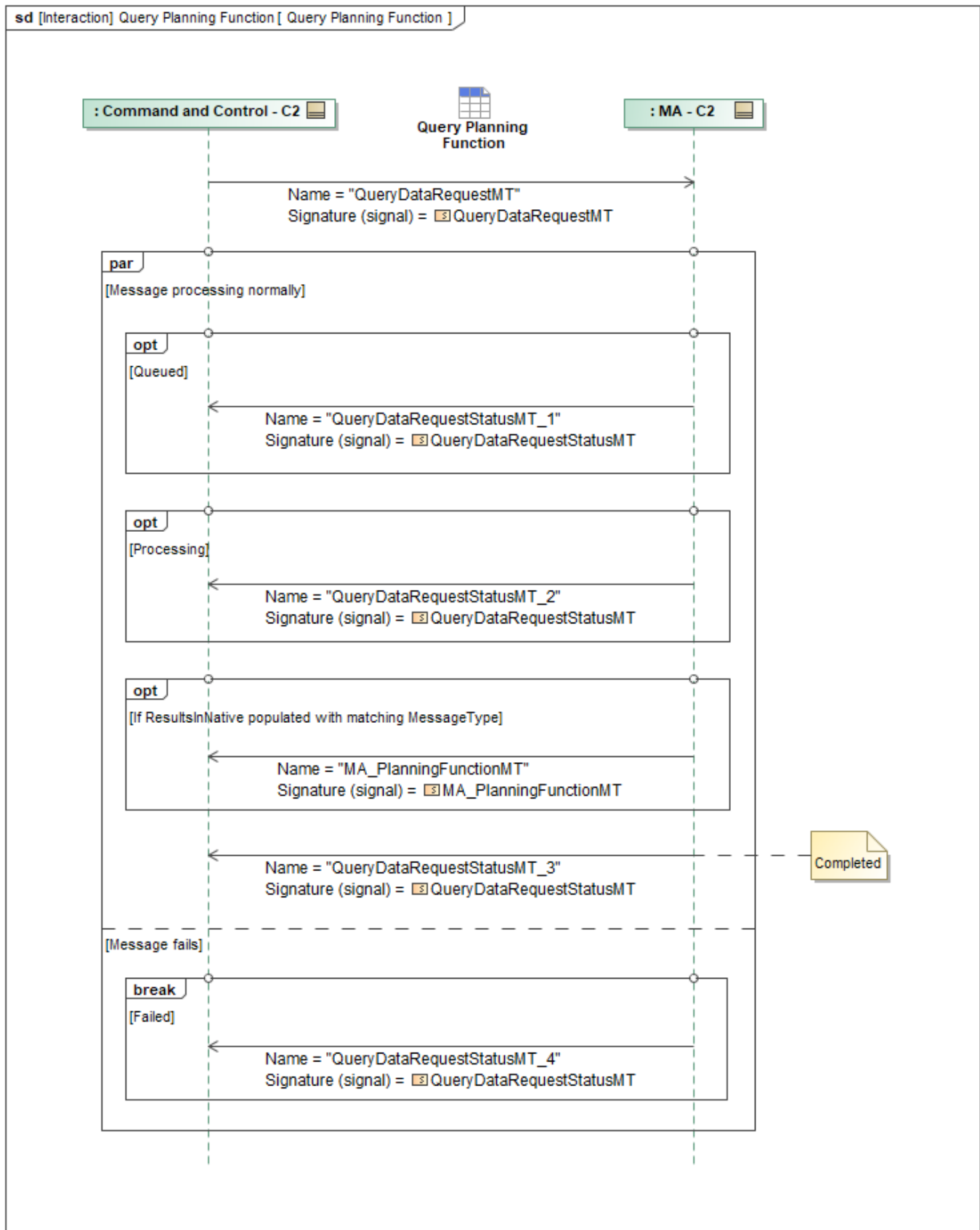


Figure 5-83: Query Planning Function

Table 5-82: Query Planning Function

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanningFunctionMT: MA_PlanningFunctionMT	PlanningInterfaces: PlanningInterfacesType PlanningInterfaces: PlanningInterfacesType
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.12 Query Planning Function Status

MA_PlanningFunctionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_PlanningFunctionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_PlanningFunctionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_PlanningFunctionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_PlanningFunctionStatusMT*.

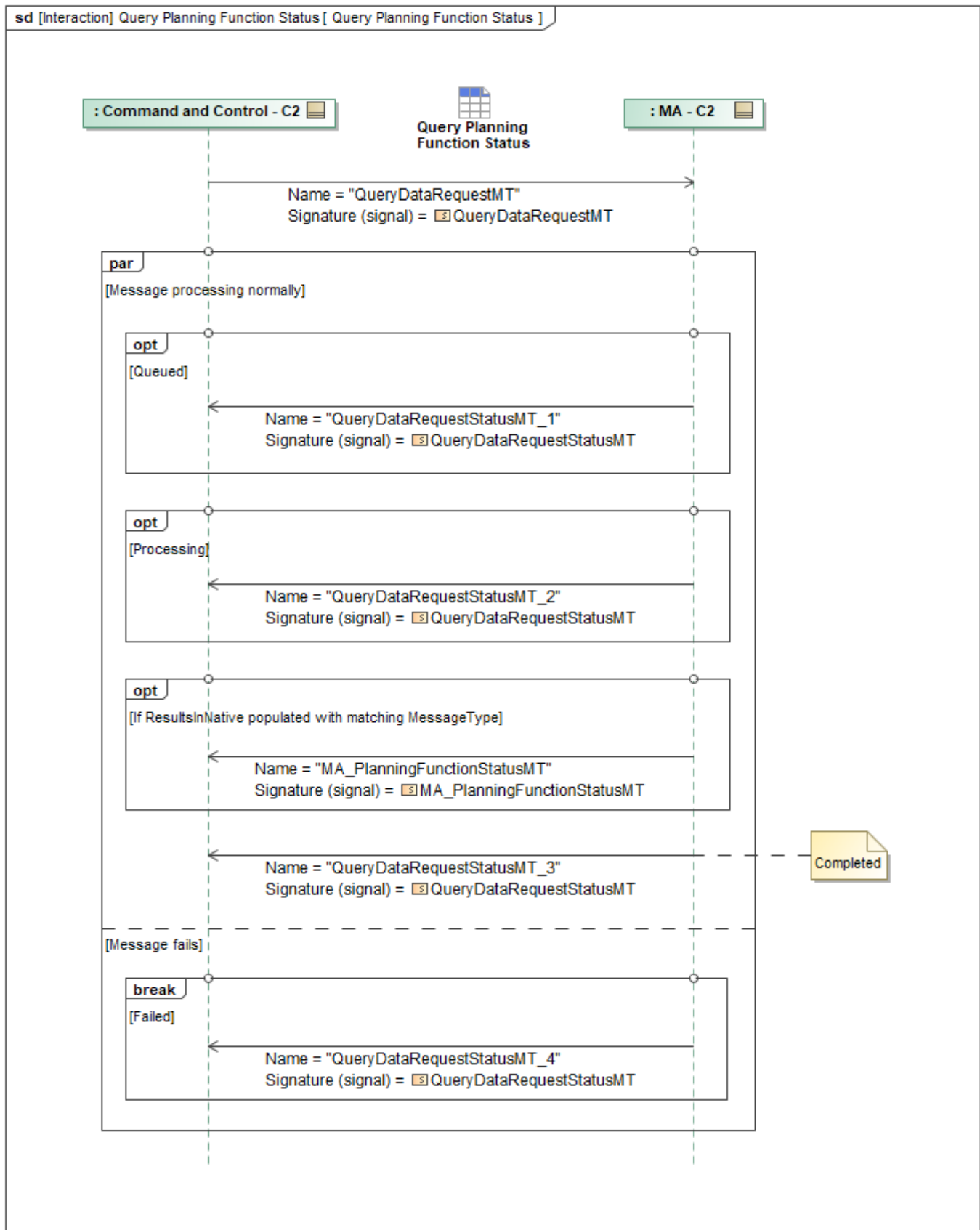


Figure 5-84: Query Planning Function Status

Table 5-83: Query Planning Function Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanningFunctionStatusMT: MA_PlanningFunctionStatusMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.13 Query Response Plan

ResponsePlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *ResponsePlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *ResponsePlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *ResponsePlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *ResponsePlanMT*.

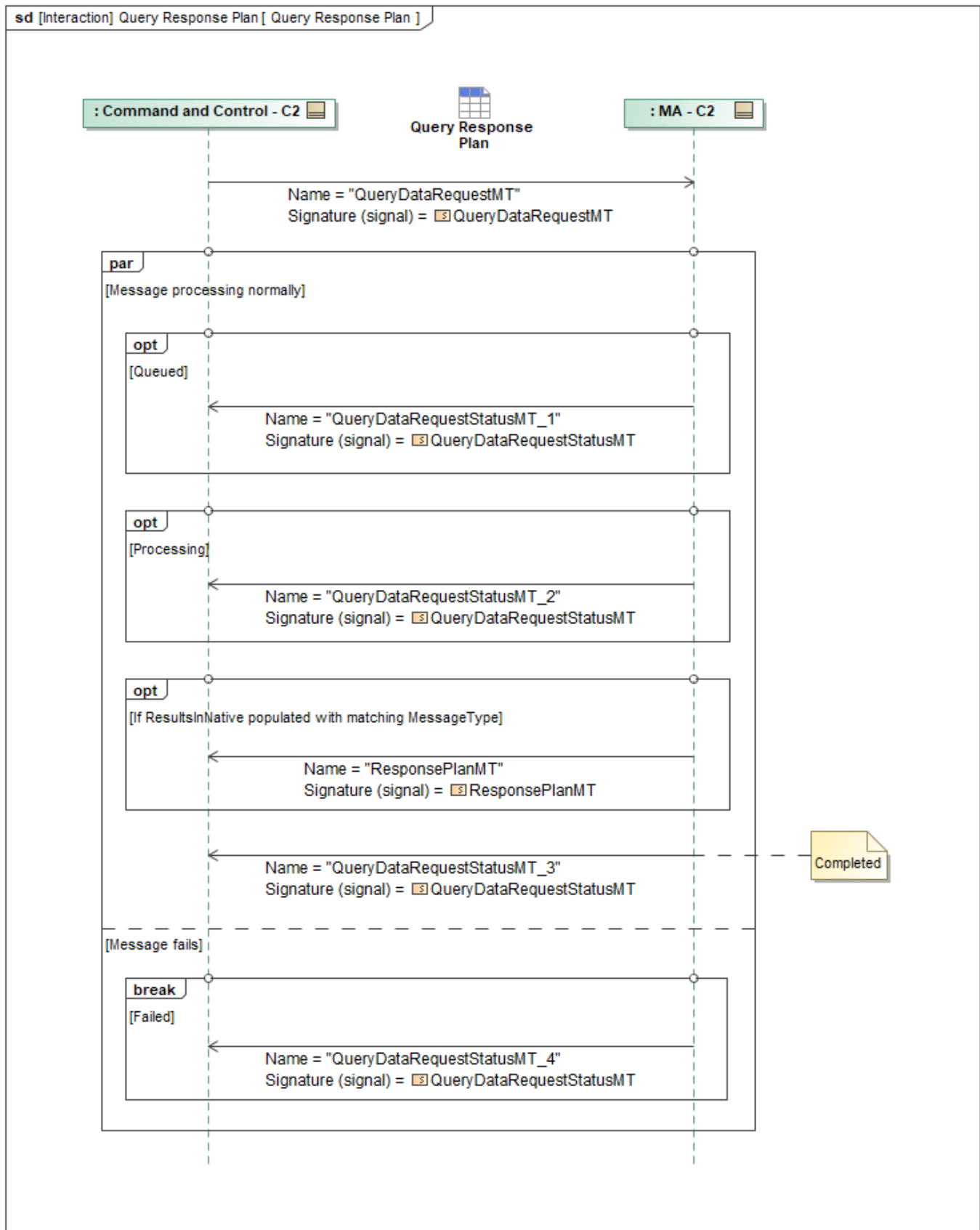


Figure 5-85: Query Response Plan

Table 5-84: Query Response Plan

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
ResponsePlanMT: ResponsePlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.14 Query Response Plan Execution Status

ResponsePlanExecutionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *ResponsePlanExecutionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *ResponsePlanExecutionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *ResponsePlanExecutionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *ResponsePlanExecutionStatusMT*.

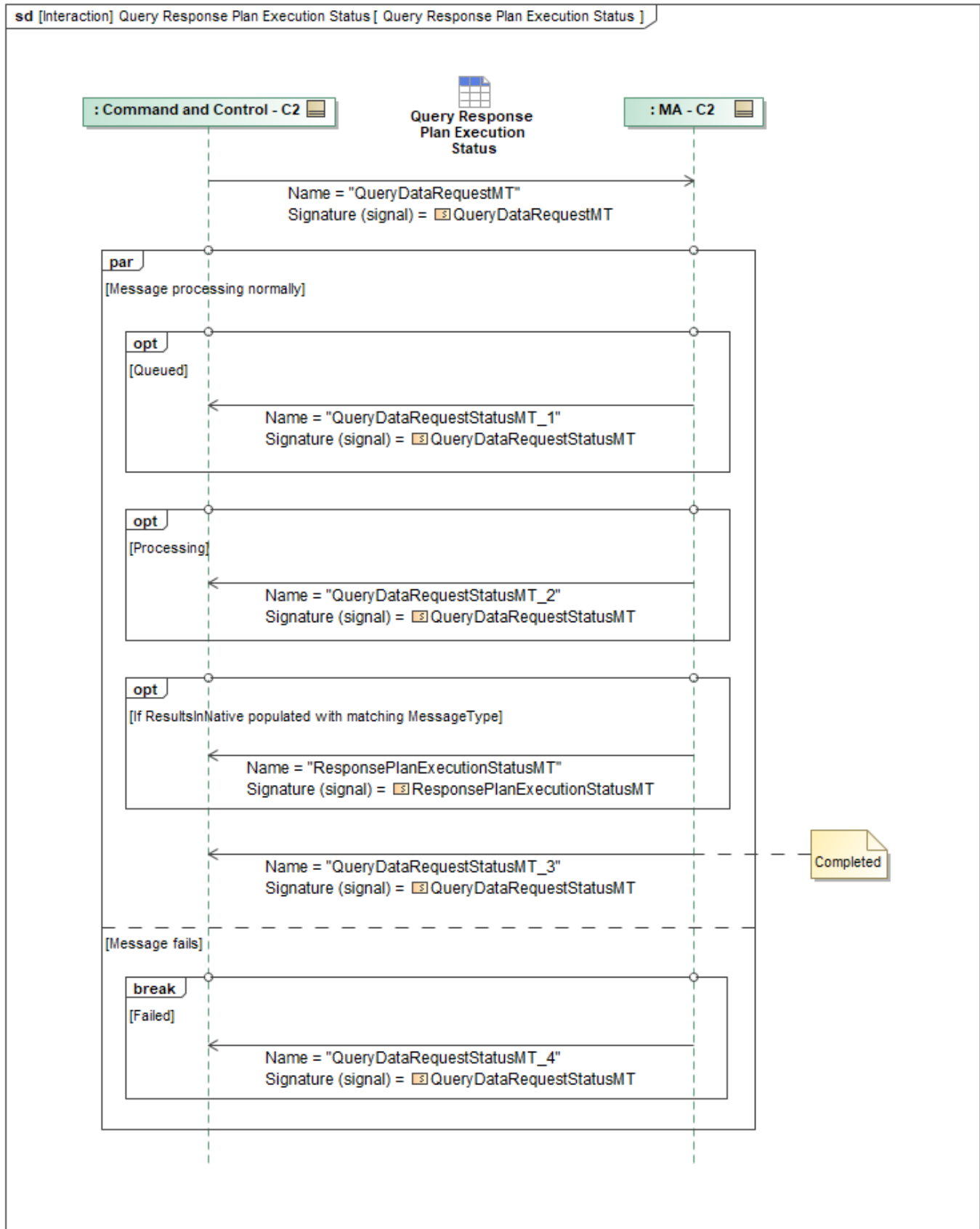


Figure 5-86: Query Response Plan Execution Status

Table 5-85: Query Response Plan Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
ResponsePlanExecutionStatusMT: ResponsePlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.15 Query Route Activity Plan

MA_RouteActivityPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_RouteActivityPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_RouteActivityPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_RouteActivityPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_RouteActivityPlanMT*.

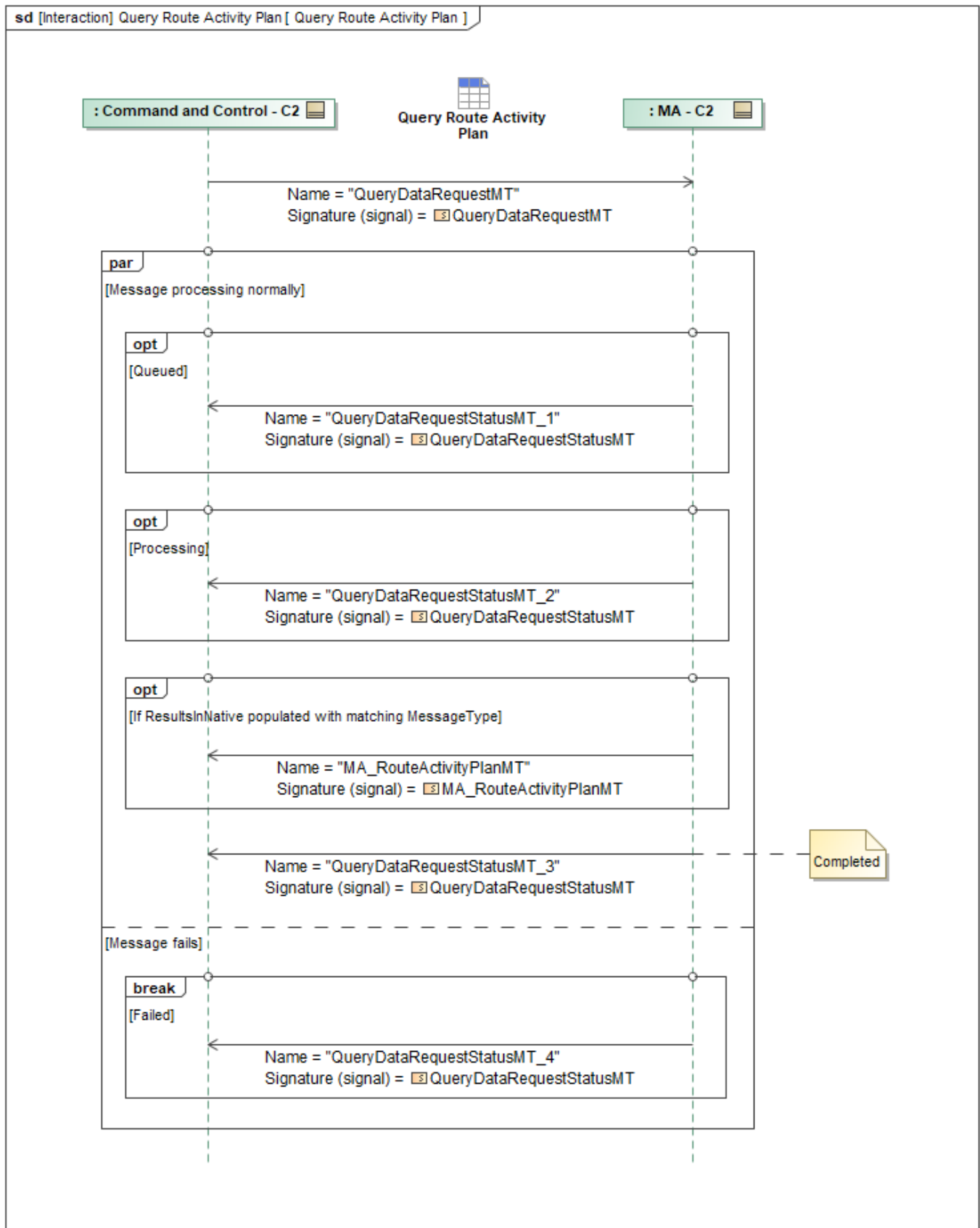


Figure 5-87: Query Route Activity Plan

Table 5-86: Query Route Activity Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteActivityPlanMT: MA_RouteActivityPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.16 Query Route Activity Plan Execution Status

RouteActivityPlanExecutionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *RouteActivityPlanExecutionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *RouteActivityPlanExecutionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *RouteActivityPlanExecutionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *RouteActivityPlanExecutionStatusMT*.

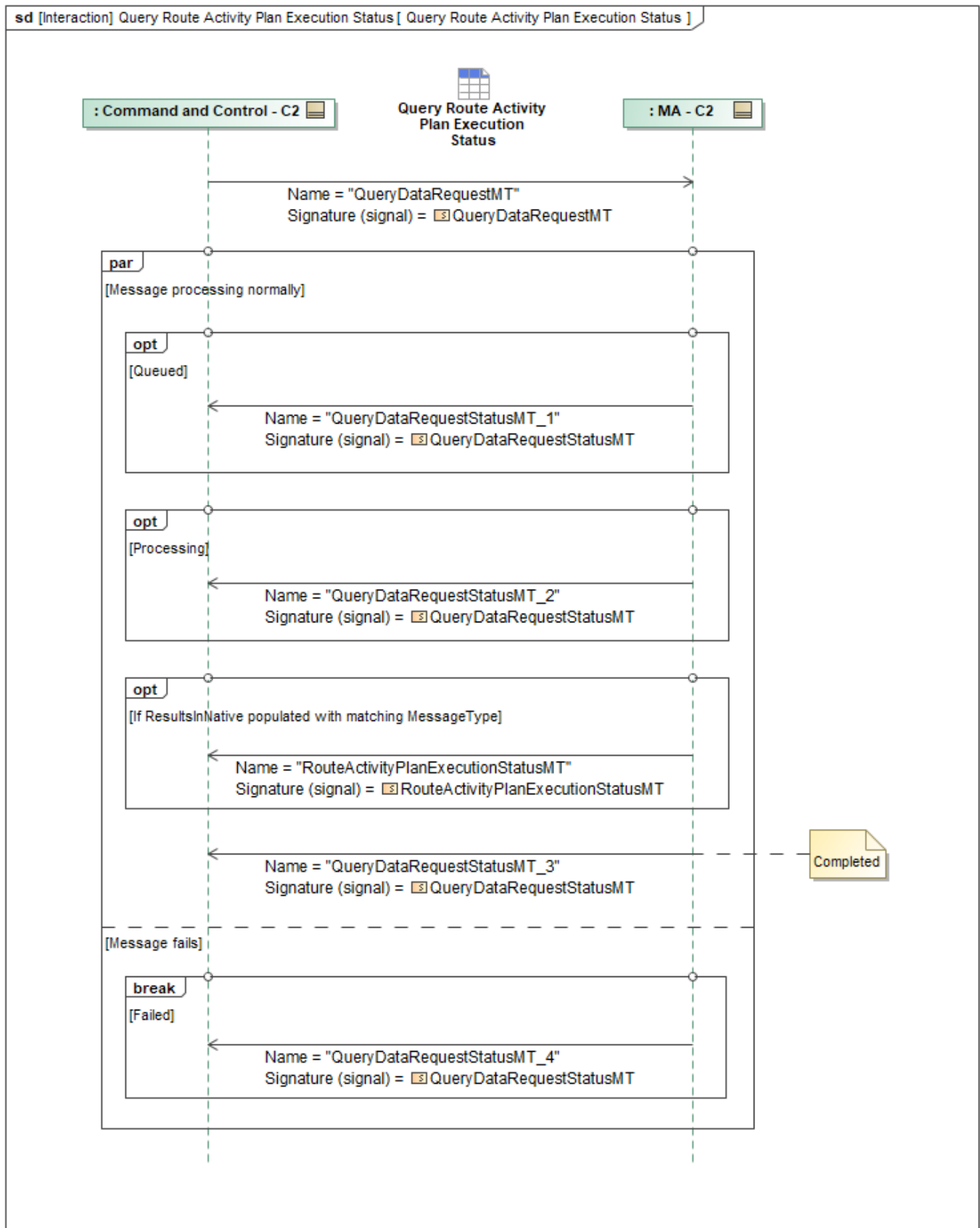


Figure 5-88: Query Route Activity Plan Execution Status

Table 5-87: Query Route Activity Plan Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
RouteActivityPlanExecutionStatusMT: RouteActivityPlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.17 Query Route Metrics

RouteMetricsMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *RouteMetricsMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *RouteMetricsMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *RouteMetricsMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *RouteMetricsMT*.

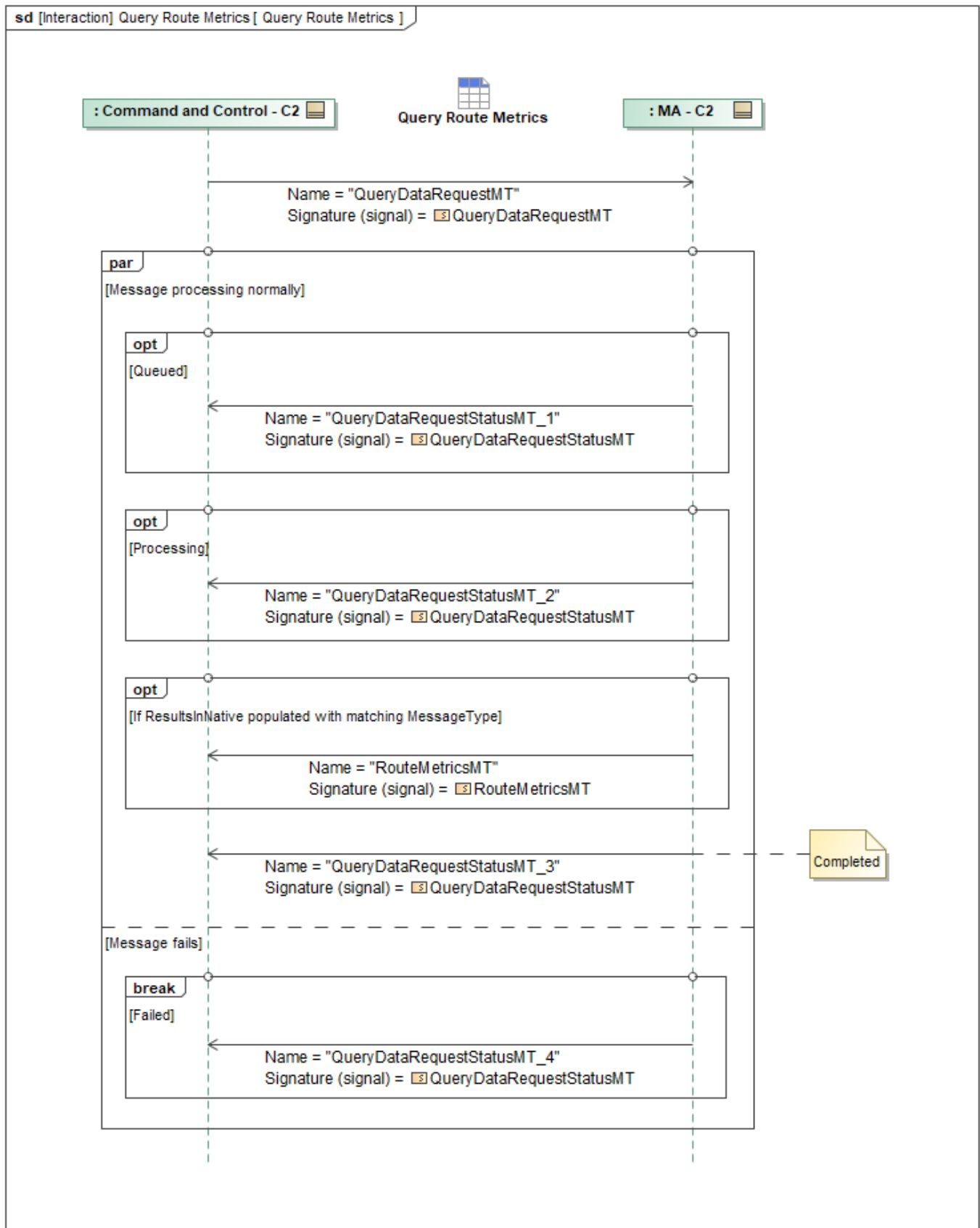


Figure 5-89: Query Route Metrics

Table 5-88: Query Route Metrics

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
RouteMetricsMT: RouteMetricsMT	EnduranceUsage: EnduranceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.18 Query Route Plan

MA_RoutePlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_RoutePlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_RoutePlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_RoutePlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_RoutePlanMT*.

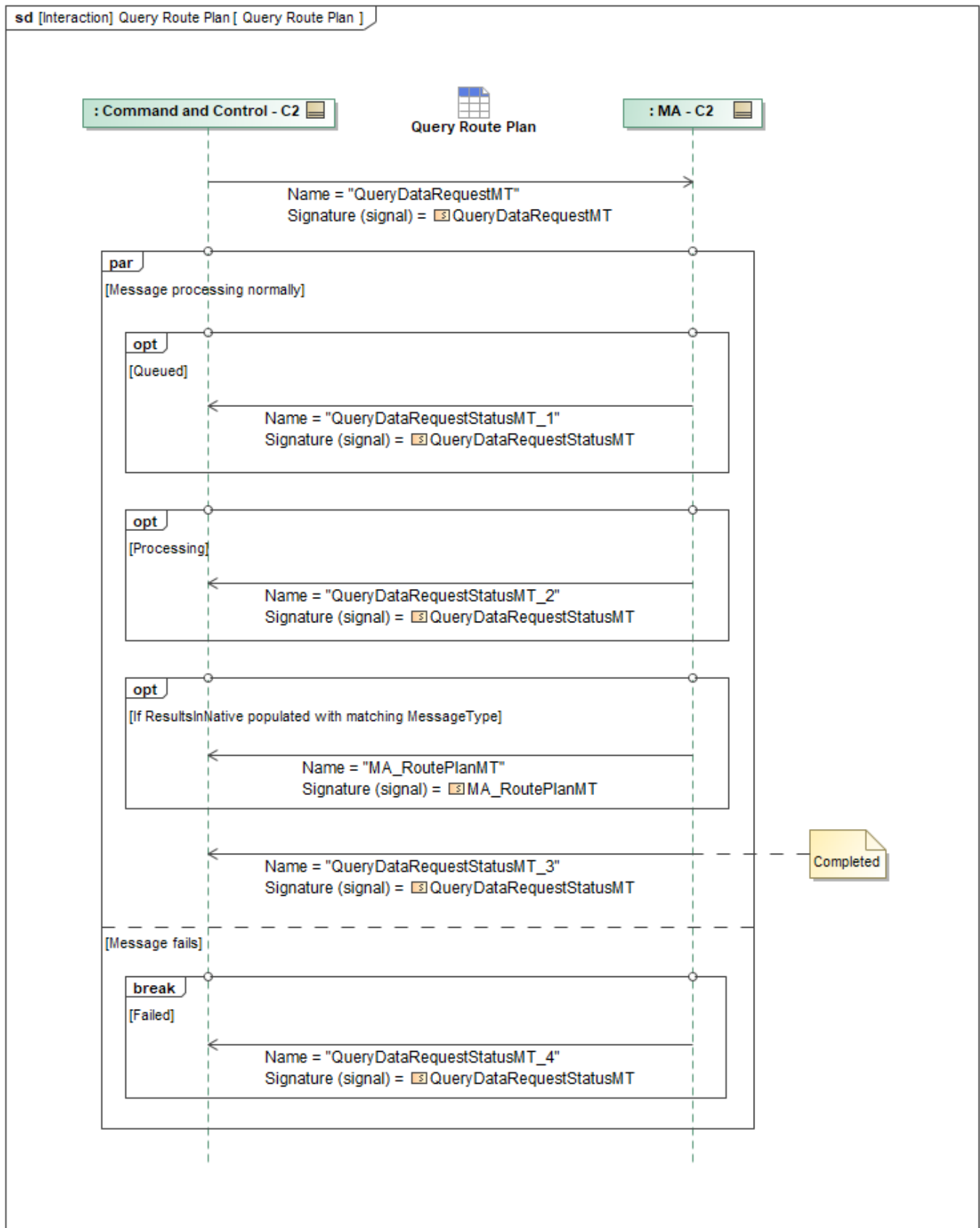


Figure 5-90: Query Route Plan

Table 5-89: Query Route Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.19 Query Route Plan Execution Status

RoutePlanExecutionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *RoutePlanExecutionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *RoutePlanExecutionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *RoutePlanExecutionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *RoutePlanExecutionStatusMT*.

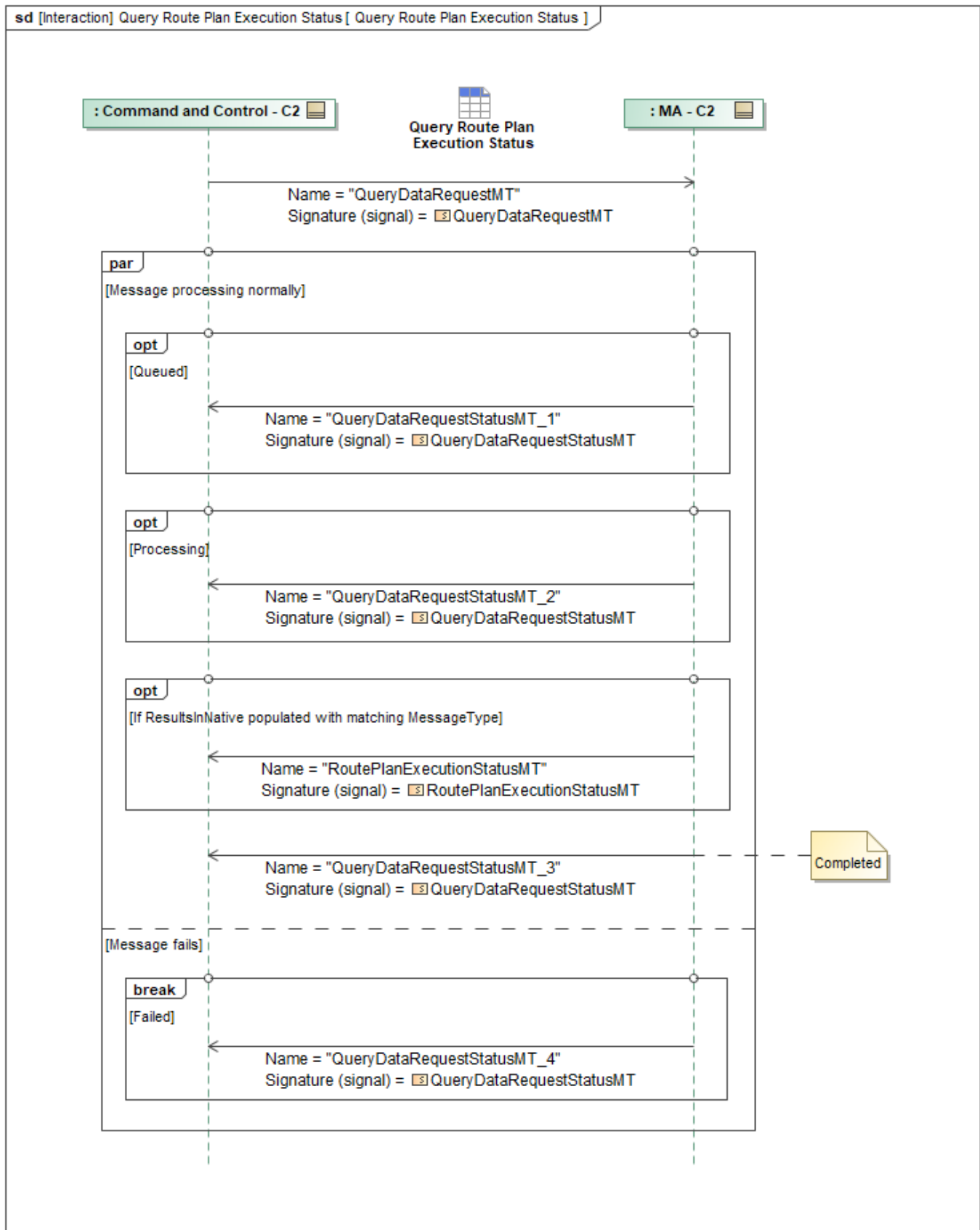


Figure 5-91: Query Route Plan Execution Status

Table 5-90: Query Route Plan Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.20 Query System Status

SystemStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *SystemStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *SystemStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *SystemStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *SystemStatusMT*.

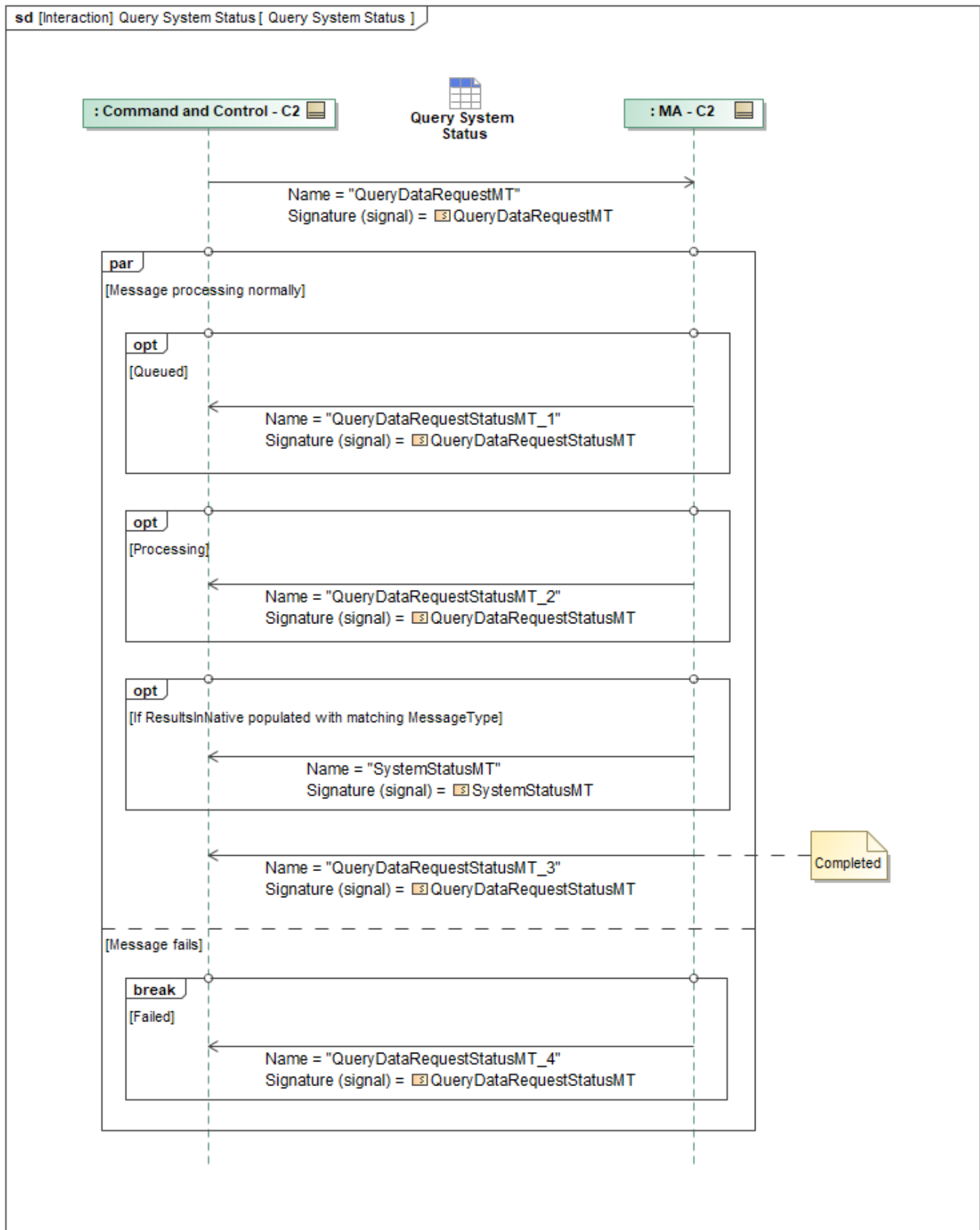


Figure 5-92: Query System Status

Table 5-91: Query System Status

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
SystemStatusMT: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.21 Query Task Plan

MA_TaskPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_TaskPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_TaskPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_TaskPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_TaskPlanMT*.

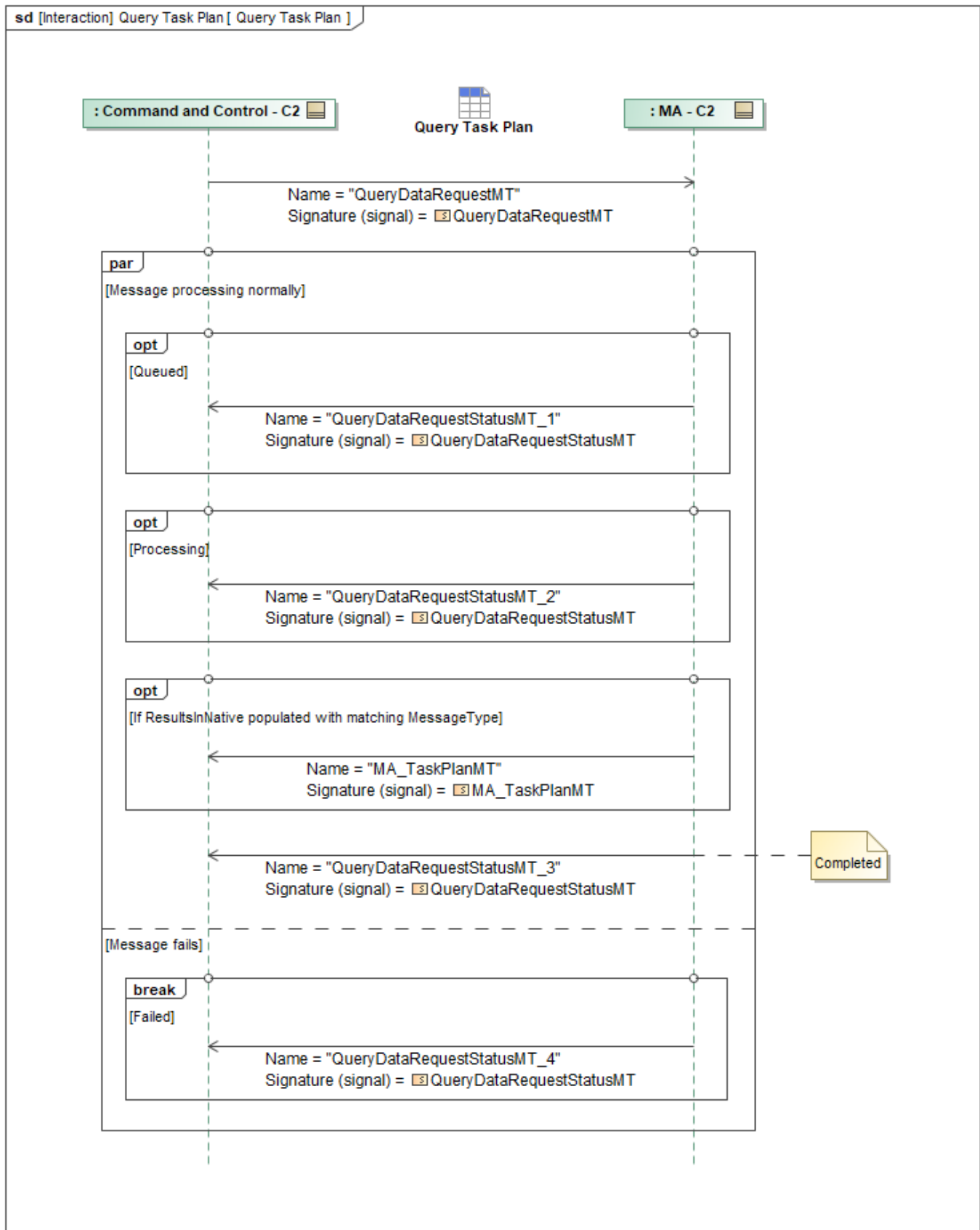


Figure 5-93: Query Task Plan

Table 5-92: Query Task Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskPlanMT: MA_TaskPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.22 Query Task Plan Execution Status

TaskPlanExecutionStatusMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *TaskPlanExecutionStatusMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *TaskPlanExecutionStatusMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *TaskPlanExecutionStatusMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *TaskPlanExecutionStatusMT*.

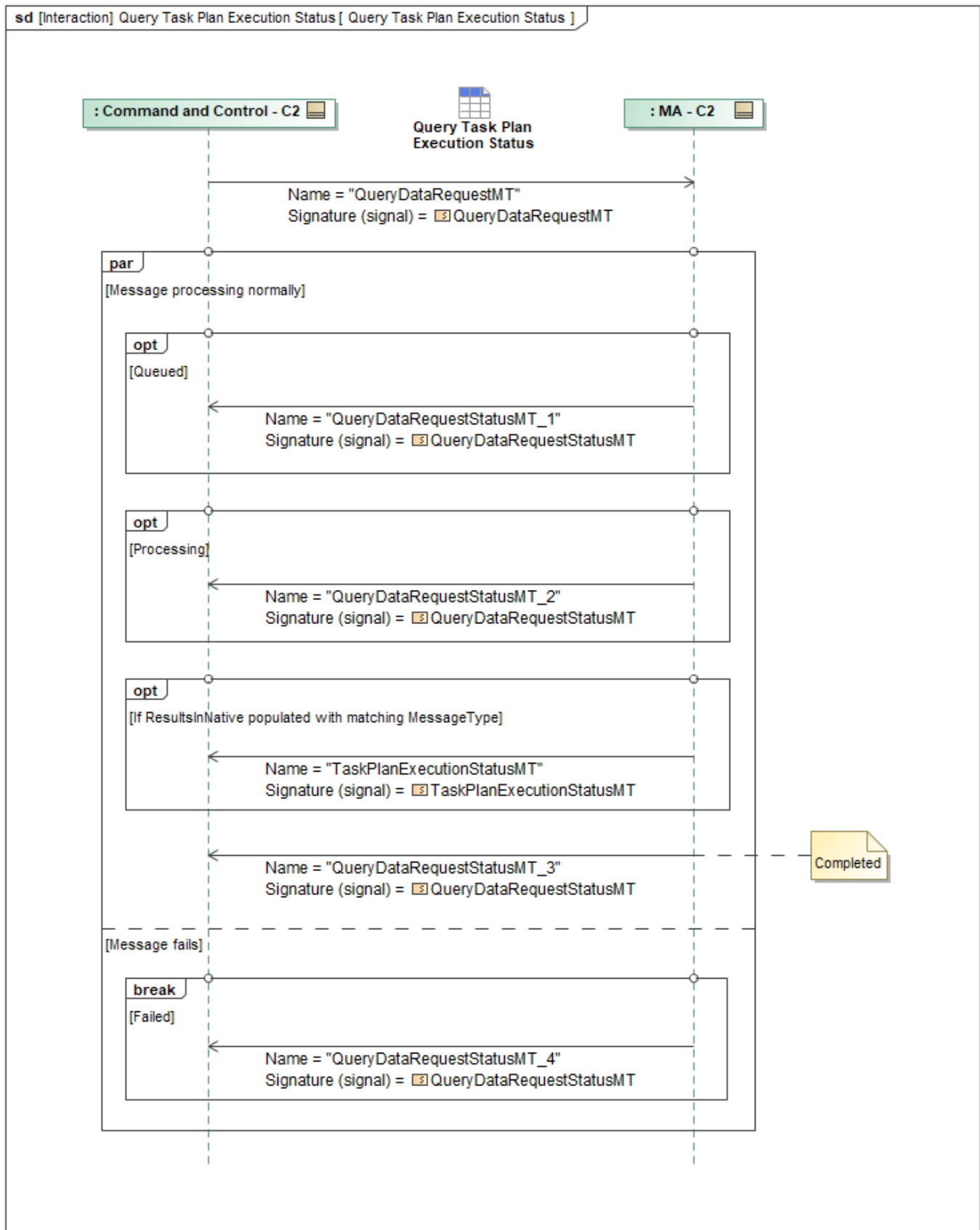


Figure 5-94: Query Task Plan Execution Status

Table 5-93: Query Task Plan Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
TaskPlanExecutionStatusMT: TaskPlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.21.23 Query Weather

WeatherMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *WeatherMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *WeatherMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *WeatherMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *WeatherMT*.

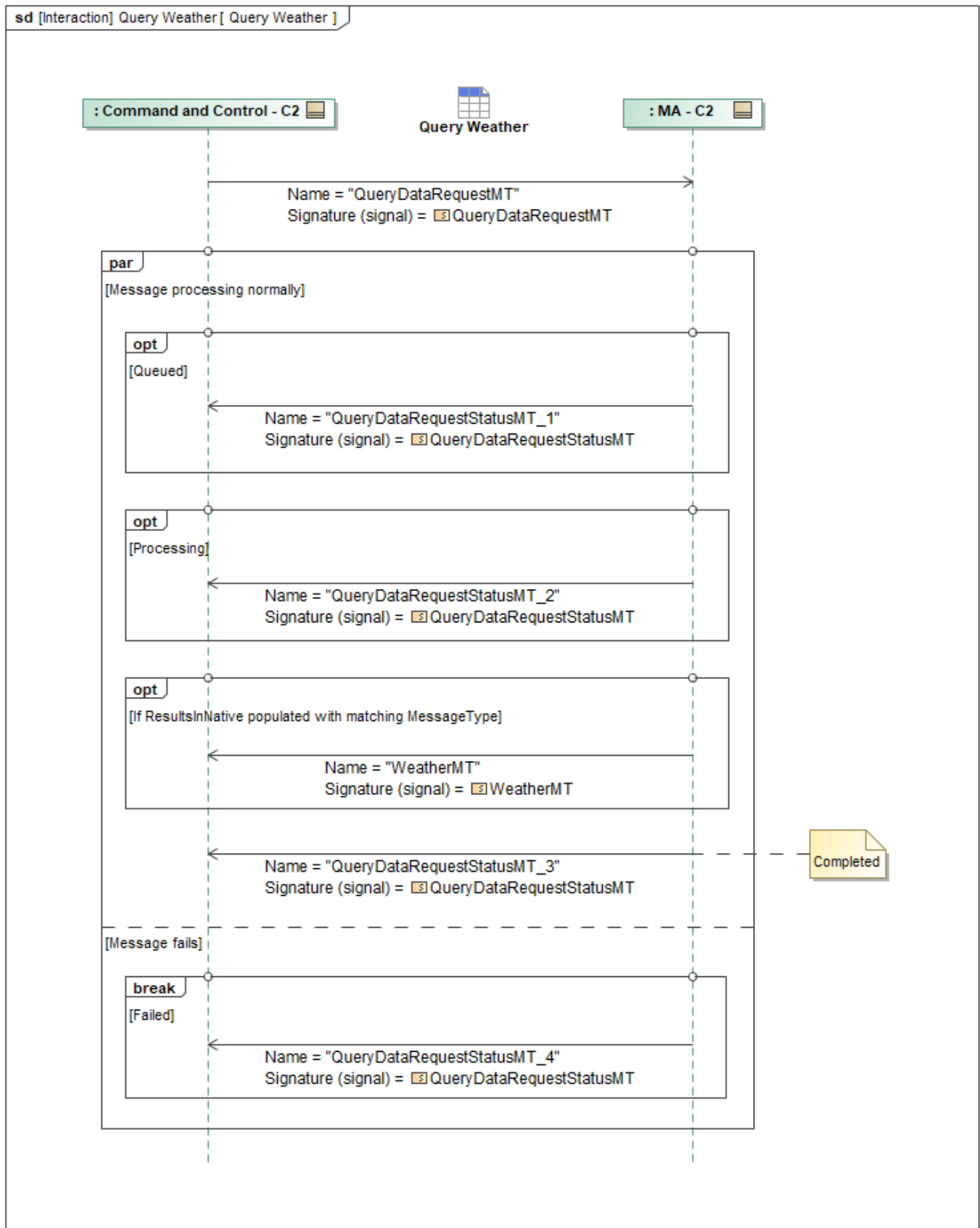


Figure 5-95: Query Weather

Table 5-94: Query Weather

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
WeatherMT: WeatherMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22 Send Data - C2 to MA

5.2.8.22.1 Send Action - C2 to MA

This interaction has C2 send *MA_ActionMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

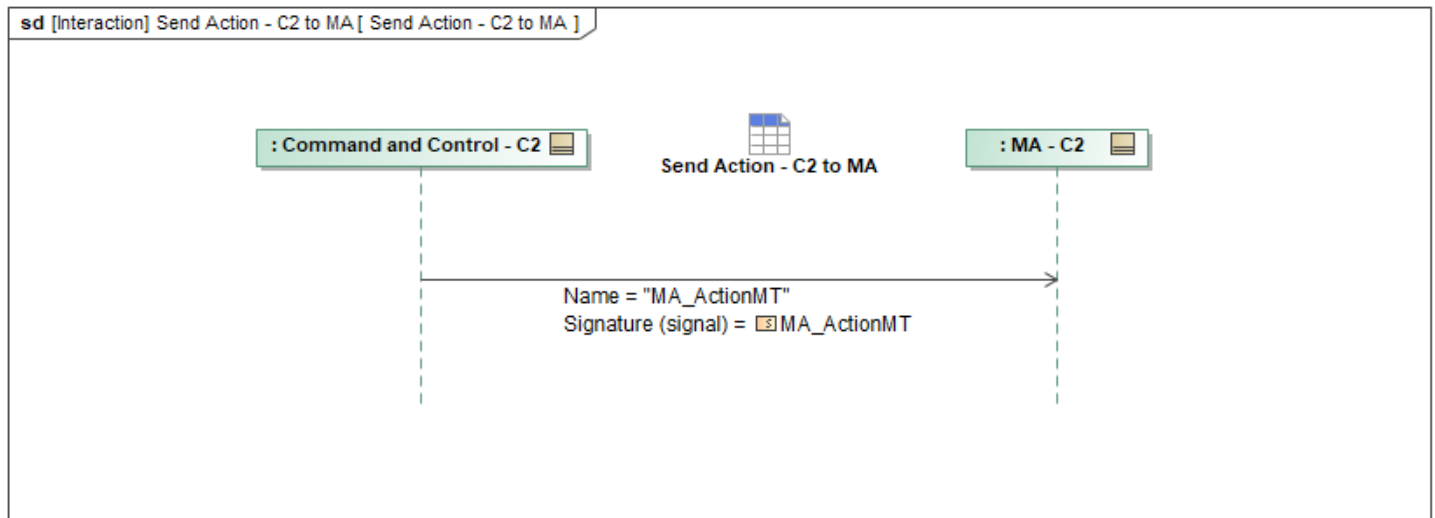


Figure 5-96: Send Action - C2 to MA

Table 5-95: Send Action - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionMT: MA_ActionMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

5.2.8.22.2 Send Action Plan - C2 to MA

This interaction has C2 send *MA_ActionPlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

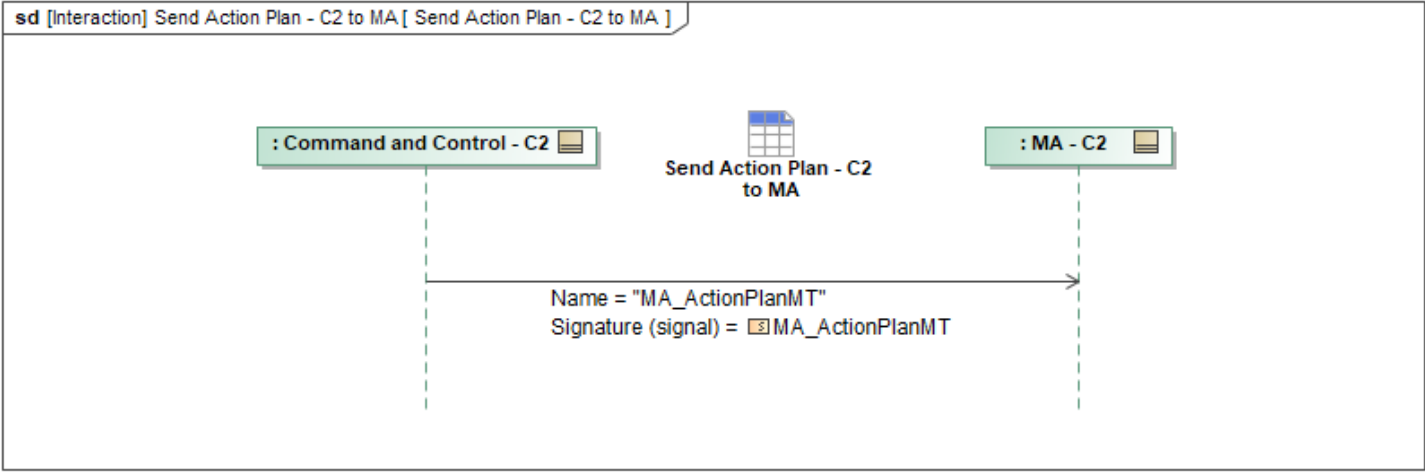


Figure 5-97: Send Action Plan - C2 to MA

Table 5-96: Send Action Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionPlanMT: MA_ActionPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.3 Send Activity Plan - C2 to MA

This interaction has C2 send *MA_ActivityPlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

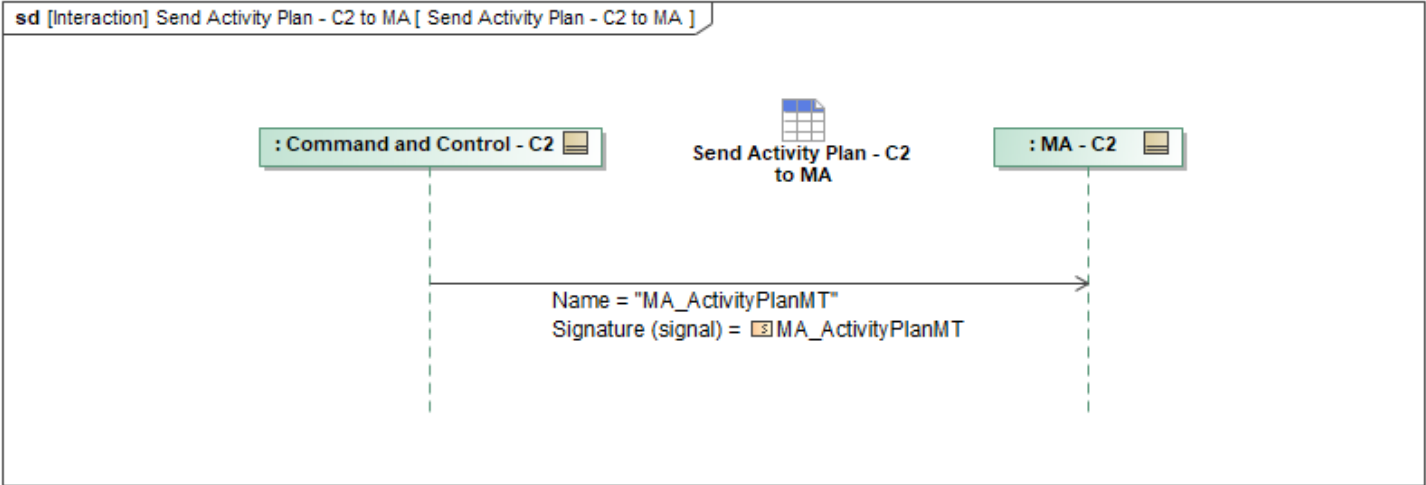


Figure 5-98: Send Activity Plan - C2 to MA

Table 5-97: Send Activity Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.4 Send Airfield Report - C2 to MA

This interaction has C2 send *AirfieldReportMT* to MA .

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

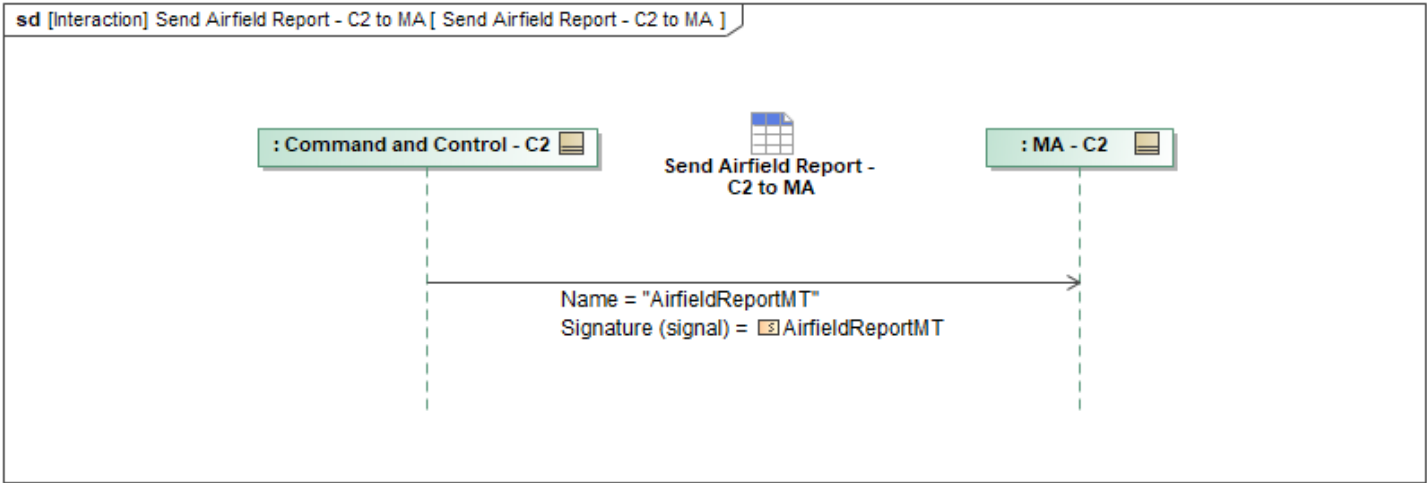


Figure 5-99: Send Airfield Report - C2 to MA

Table 5-98: Send Airfield Report - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.5 Send Approval Authority - C2 to MA

This interaction has C2 send *MA_ApprovalAuthorityMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

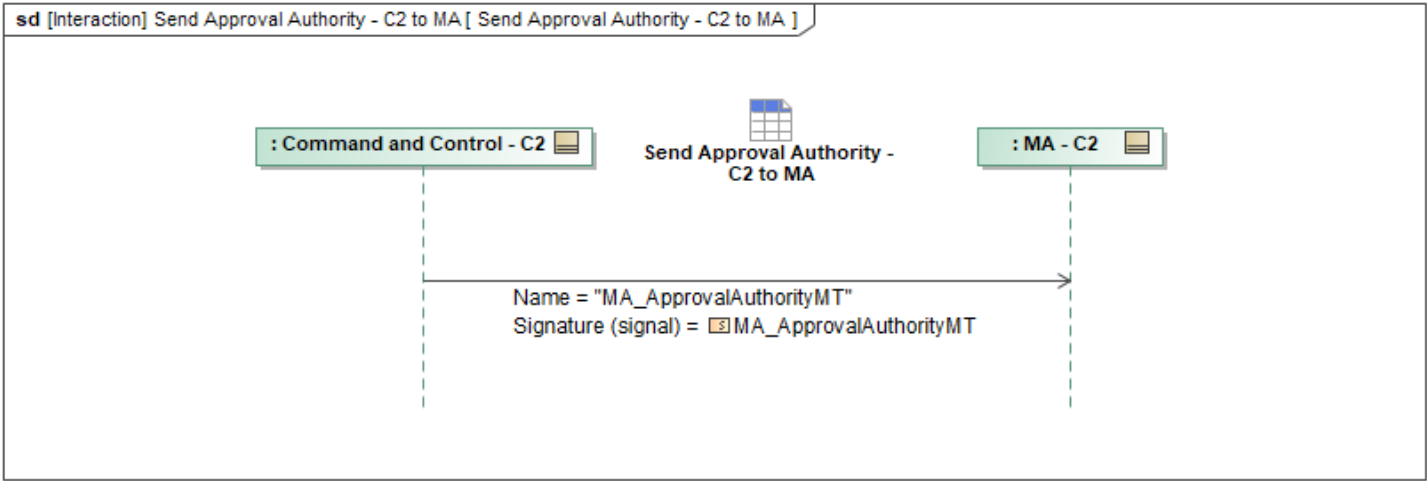


Figure 5-100: Send Approval Authority - C2 to MA

Table 5-99: Send Approval Authority - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalAuthorityMT: MA_ApprovalAuthorityMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.6 Send Approval Policy - C2 to MA

This interaction has C2 send *MA_ApprovalPolicyMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

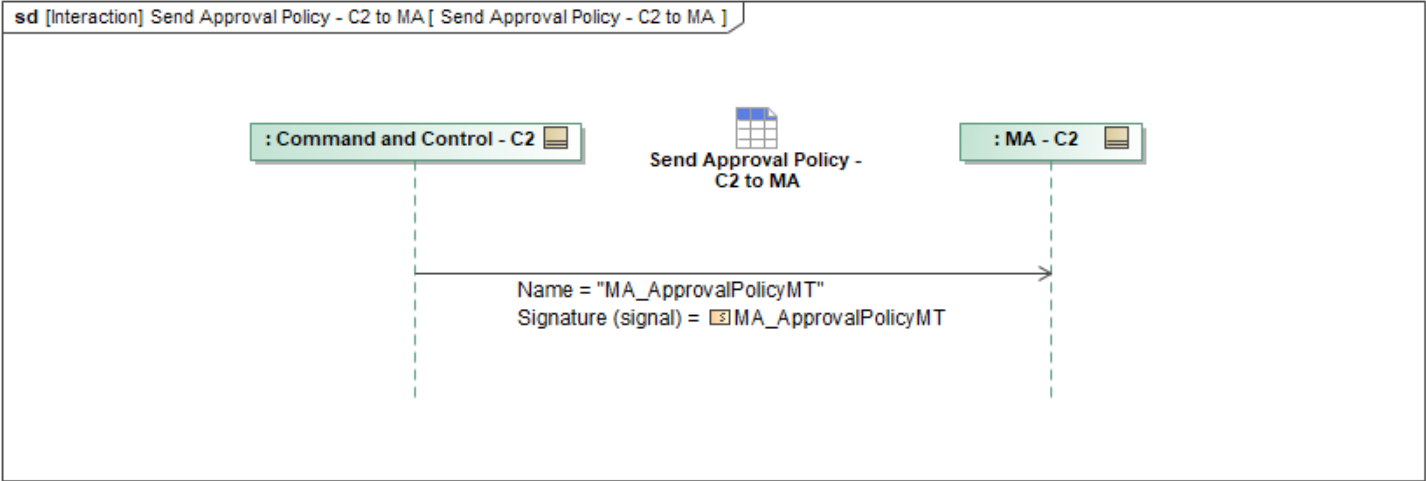


Figure 5-101: Send Approval Policy - C2 to MA

Table 5-100: Send Approval Policy - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.7 Send Capability Set - C2 to MA

This interaction has C2 send *MA_CapabilitySetMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

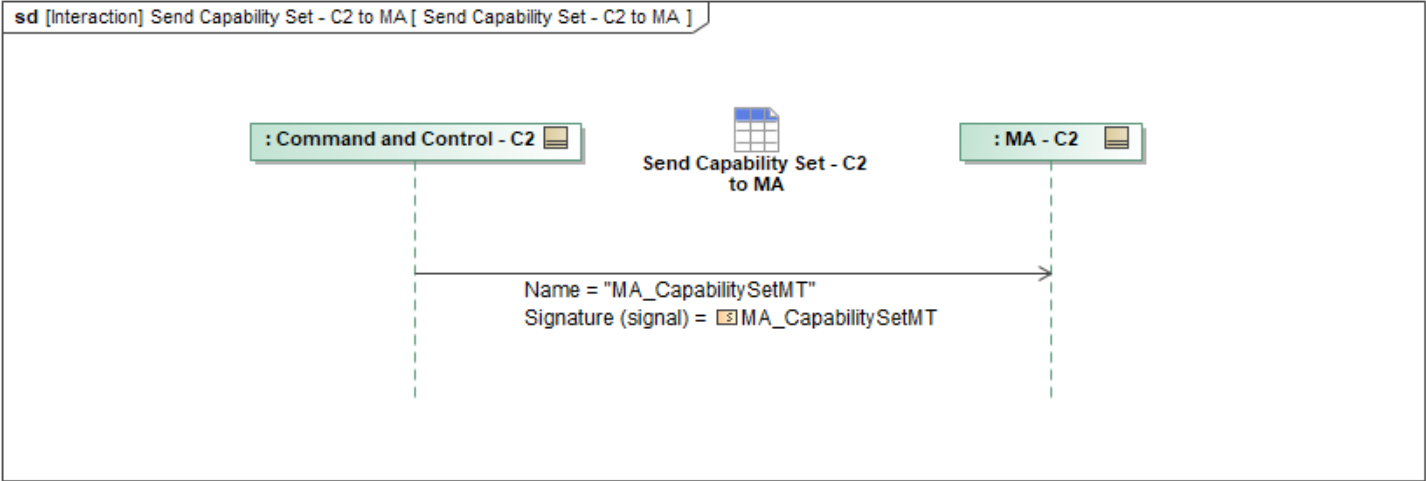


Figure 5-102: Send Capability Set - C2 to MA

Table 5-101: Send Capability Set - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CapabilitySetMT: MA_CapabilitySetMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.8 Send Constraint Priority List - C2 to MA

This interaction has C2 send *MA_ConstraintPriorityListMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

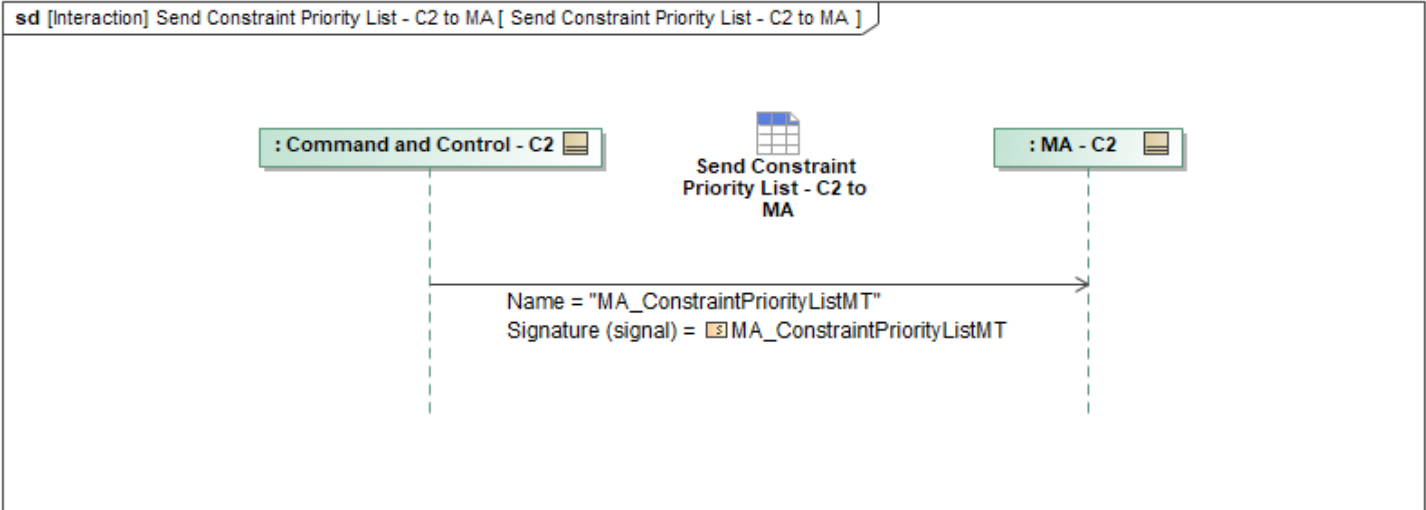


Figure 5-103: Send Constraint Priority List - C2 to MA

Table 5-102: Send Constraint Priority List - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListMT: MA_ConstraintPriorityListMT	ObjectState: ObjectStateEnum ListItem: MA_ConstraintPriorityListItemType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.9 Send Constraint Priority List Activation Data - C2 to MA

This interaction has C2 send *MA_ConstraintPriorityListActivationDataMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

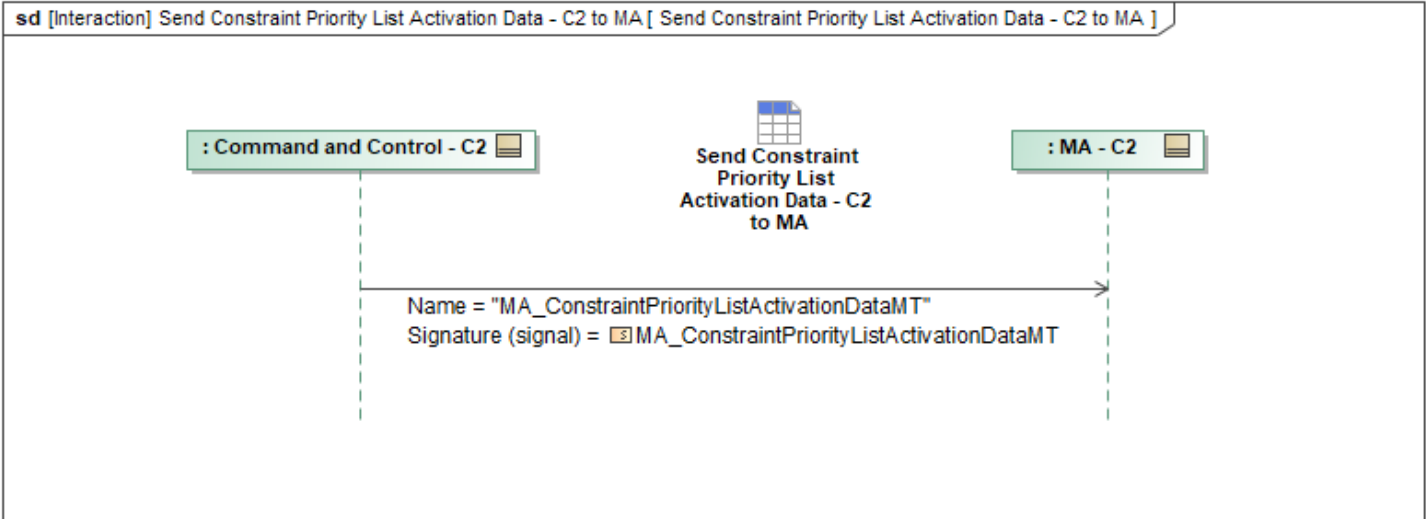


Figure 5-104: Send Constraint Priority List Activation Data - C2 to MA

Table 5-103: Send Constraint Priority List Activation Data - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationDataMT: MA_ConstraintPriorityListActivationDataMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.10 Send Engagement Criteria - C2 to MA

This interaction has C2 send *MA_EngagementCriteriaMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

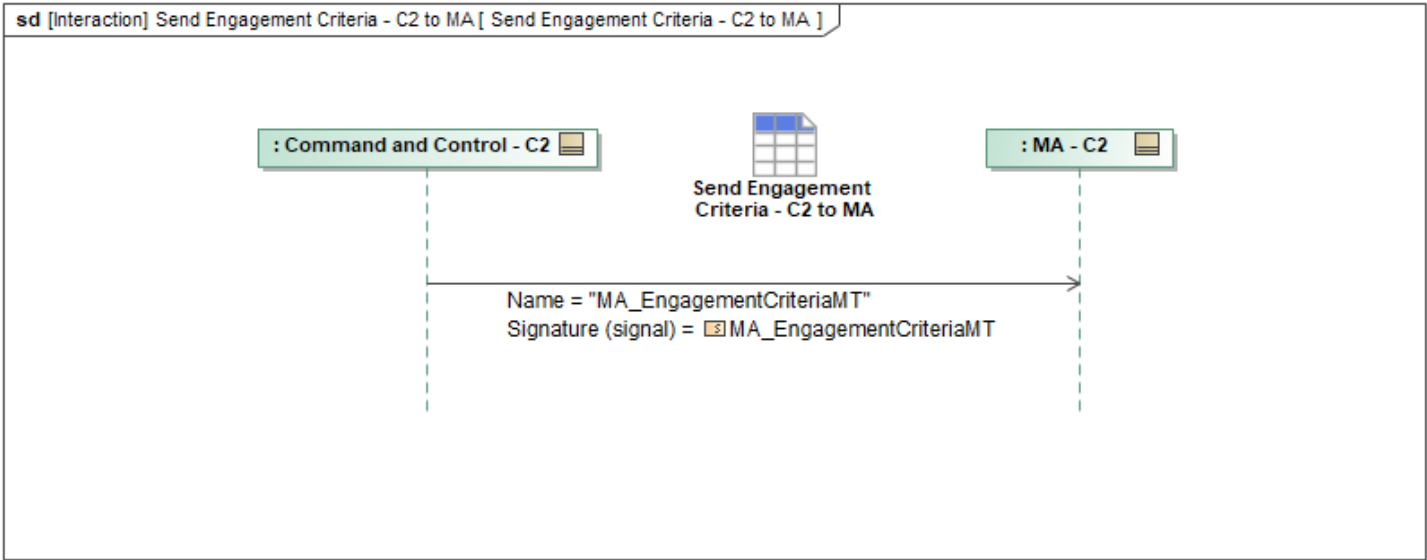


Figure 5-105: Send Engagement Criteria - C2 to MA

Table 5-104: Send Engagement Criteria - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EngagementCriteriaMT: MA_EngagementCriteriaMT	TargetAuthorityCriteria: MA_AuthorityCriteriaType TargetDesignationCriteria: MA_TargetDesignationCriteriaType TargetCustodyCriteria: MA_TargetCustodyCriteriaType IdMatrixCriteria: MA_IdMatrixCriteriaType TargetDesignationCriteria: MA_TargetDesignationCriteriaType TargetCustodyCriteria: MA_TargetCustodyCriteriaType IdMatrixCriteria: MA_IdMatrixCriteriaType TargetAuthorityCriteria: MA_AuthorityCriteriaType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.11 Send Entity - C2 to MA

This interaction has C2 send *EntityMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

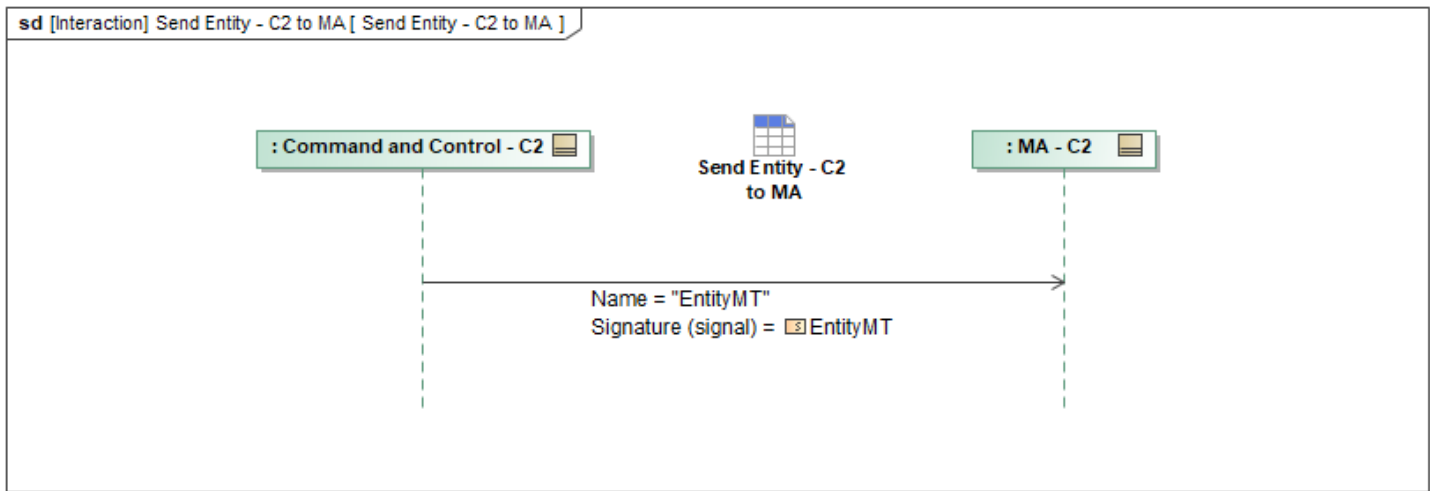


Figure 5-106: Send Entity - C2 to MA

Table 5-105: Send Entity - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.12 Send Execution Priority List - C2 to MA

This interaction has C2 send *MA_ExecutionPriorityListMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

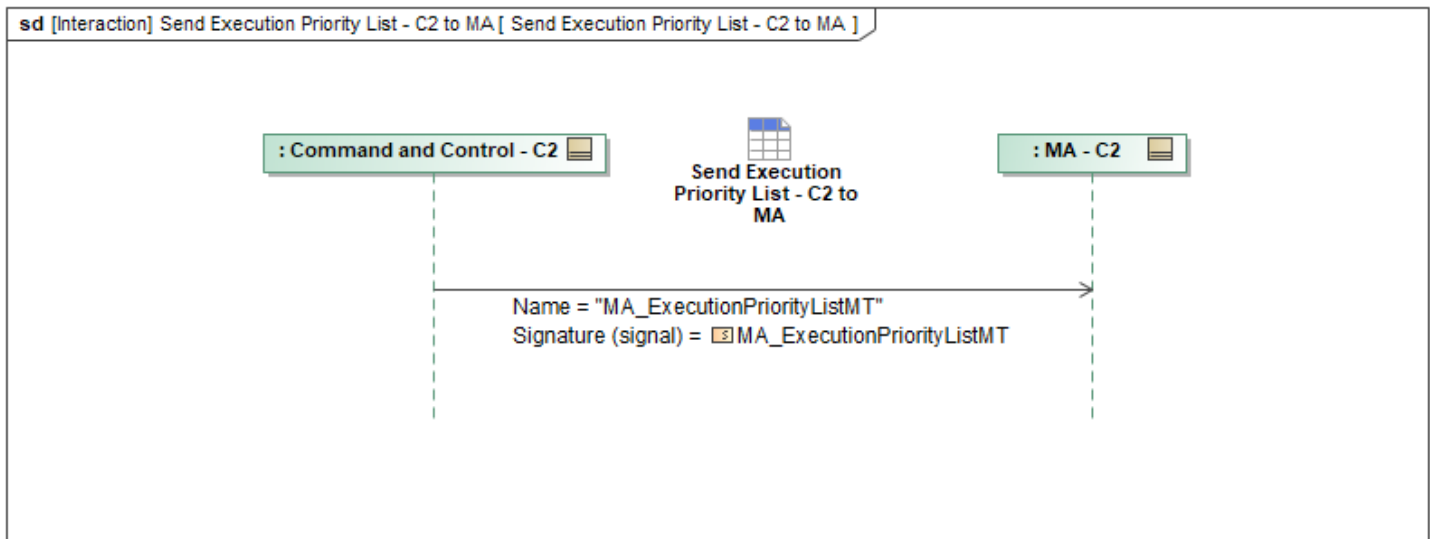


Figure 5-107: Send Execution Priority List - C2 to MA

Table 5-106: Send Execution Priority List - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListMT: MA_ExecutionPriorityListMT	ObjectState: ObjectStateEnum ListItem: MA_ExecutionPriorityListItemType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.13 Send Execution Priority List Activation Data - C2 to MA

This interaction has C2 send *MA_ExecutionPriorityListActivationDataMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

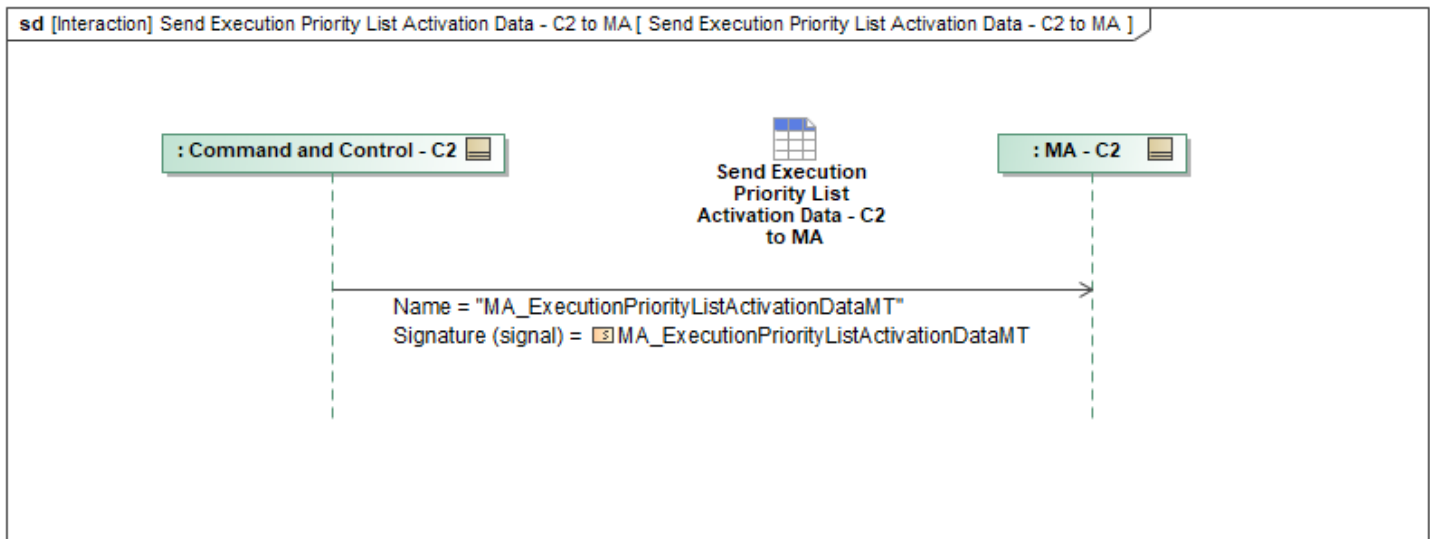


Figure 5-108: Send Execution Priority List Activation Data - C2 to MA

Table 5-107: Send Execution Priority List Activation Data - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationDataMT: MA_ExecutionPriorityListActivationDataMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.14 Send Interoperability Policy - C2 to MA

This interaction has C2 send *MA_InteroperabilityPolicyMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

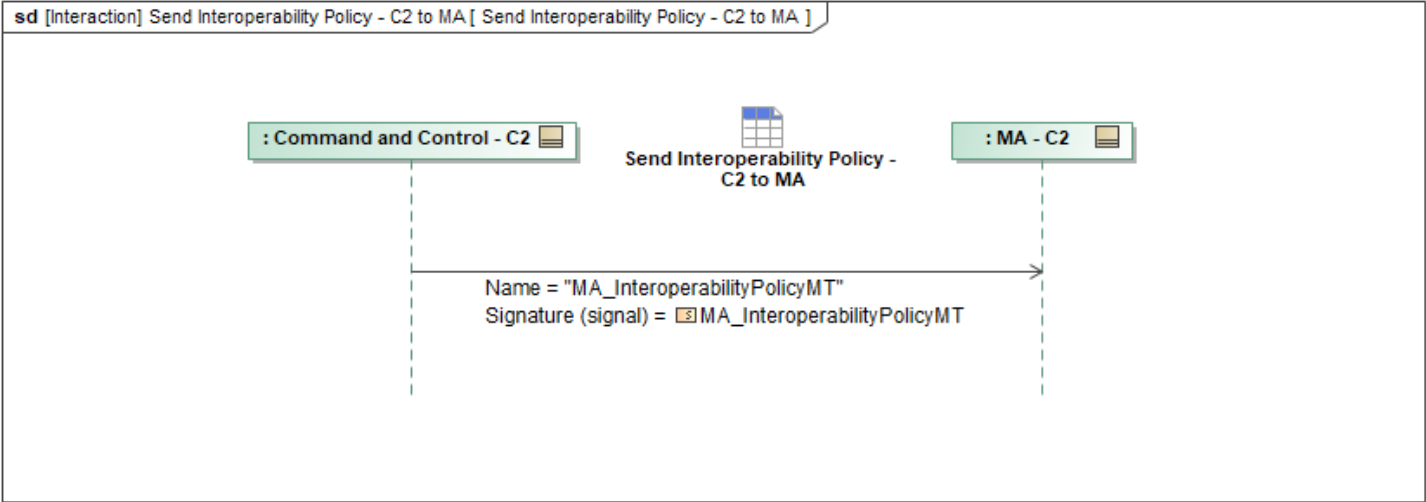


Figure 5-109: Send Interoperability Policy - C2 to MA

Table 5-108: Send Interoperability Policy - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_InteroperabilityPolicyMT: MA_InteroperabilityPolicyMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.15 Send Leadership Metrics - C2 to MA

This interaction has C2 send *MA_LeadershipMetricsMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

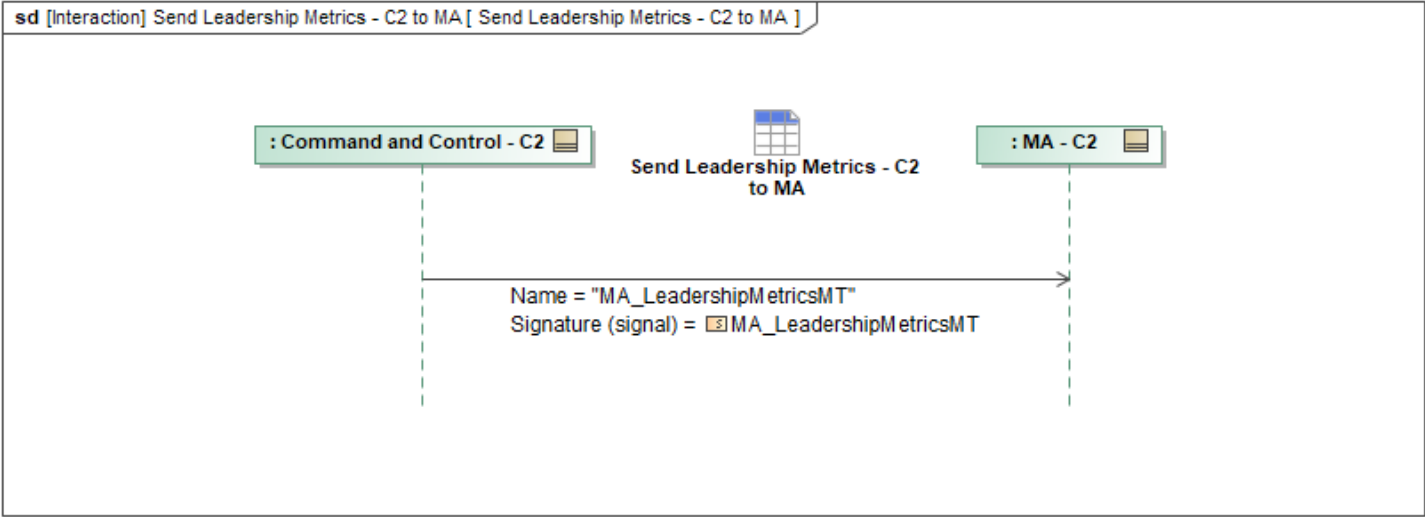


Figure 5-110: Send Leadership Metrics - C2 to MA

Table 5-109: Send Leadership Metrics - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeadershipMetricsMT: MA_LeadershipMetricsMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.16 Send Mission Plan - C2 to MA

This interaction has C2 send *MA_MissionPlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

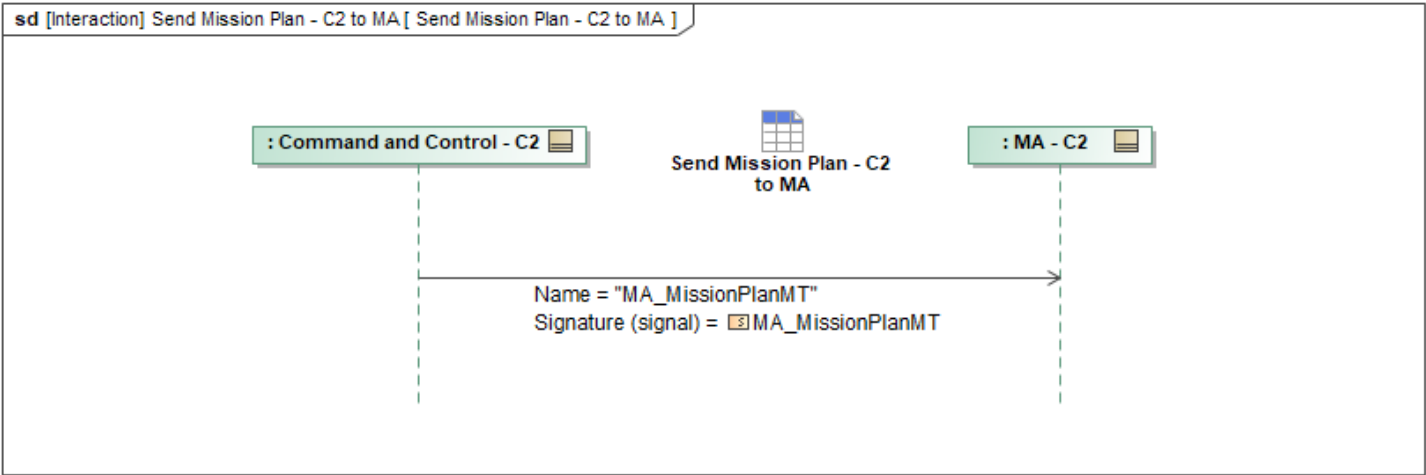


Figure 5-111: Send Mission Plan - C2 to MA

Table 5-110: Send Mission Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.17 Send Operator - C2 to MA

This interaction has C2 send *OperatorMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

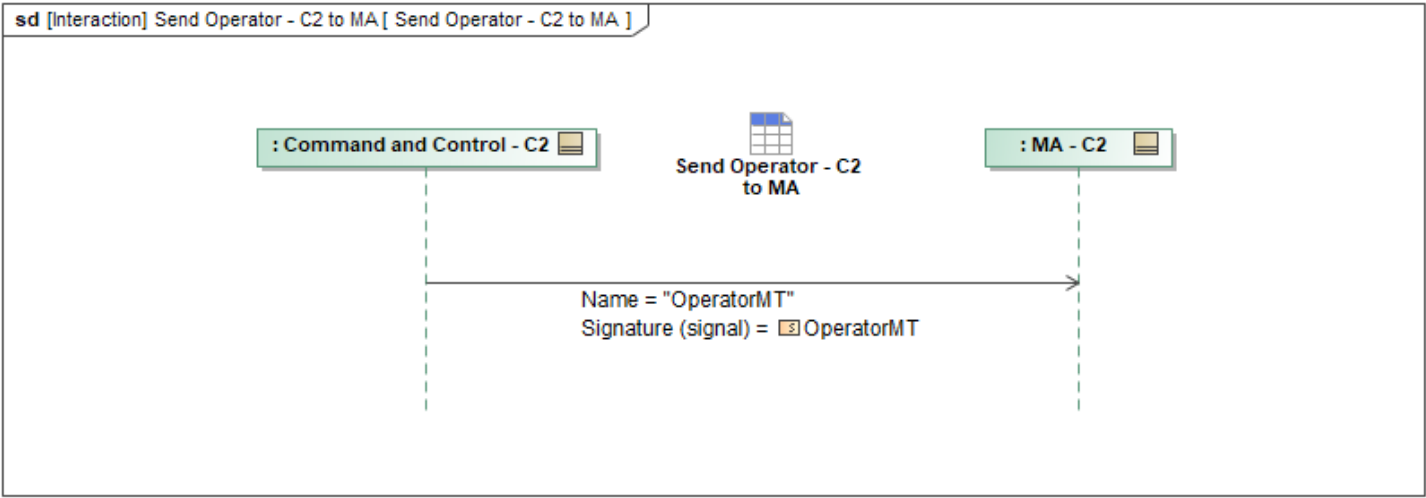


Figure 5-112: Send Operator - C2 to MA

Table 5-111: Send Operator - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorMT: OperatorMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.18 Send Operator Role - C2 to MA

This interaction has C2 send *MA_OperatorRoleMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

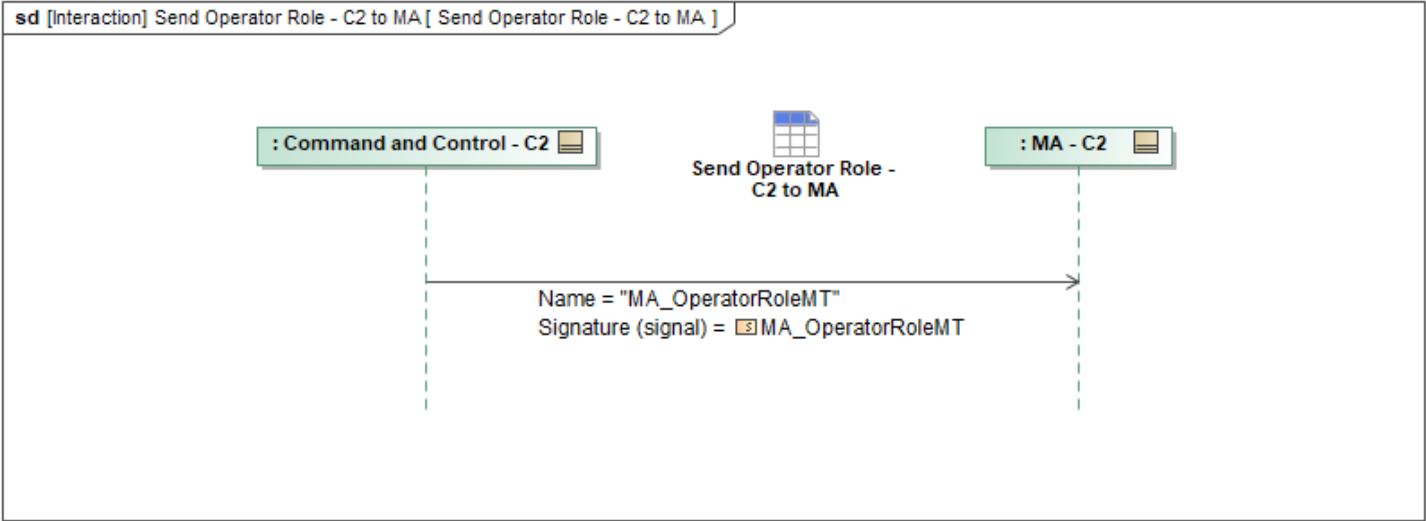


Figure 5-113: Send Operator Role - C2 to MA

Table 5-112: Send Operator Role - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.19 Send OpLine - C2 to MA

This interaction has C2 send *MA_OpLineMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

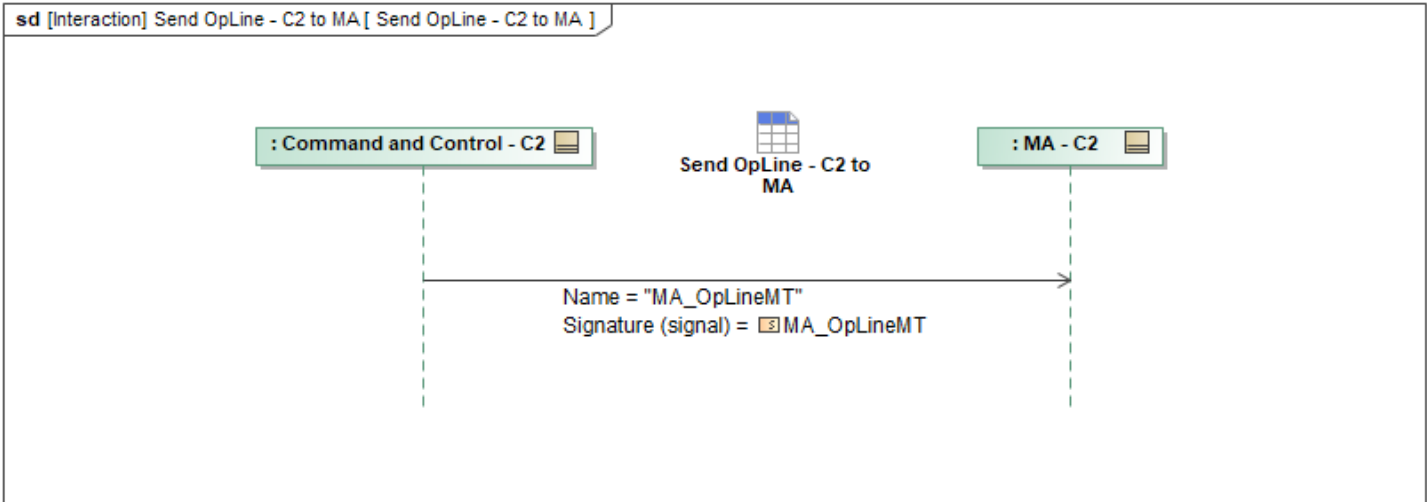


Figure 5-114: Send OpLine - C2 to MA

Table 5-113: Send OpLine - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpLineMT: MA_OpLineMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.20 Send OpPlan - C2 to MA

This interaction has C2 send *MA_OpPlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

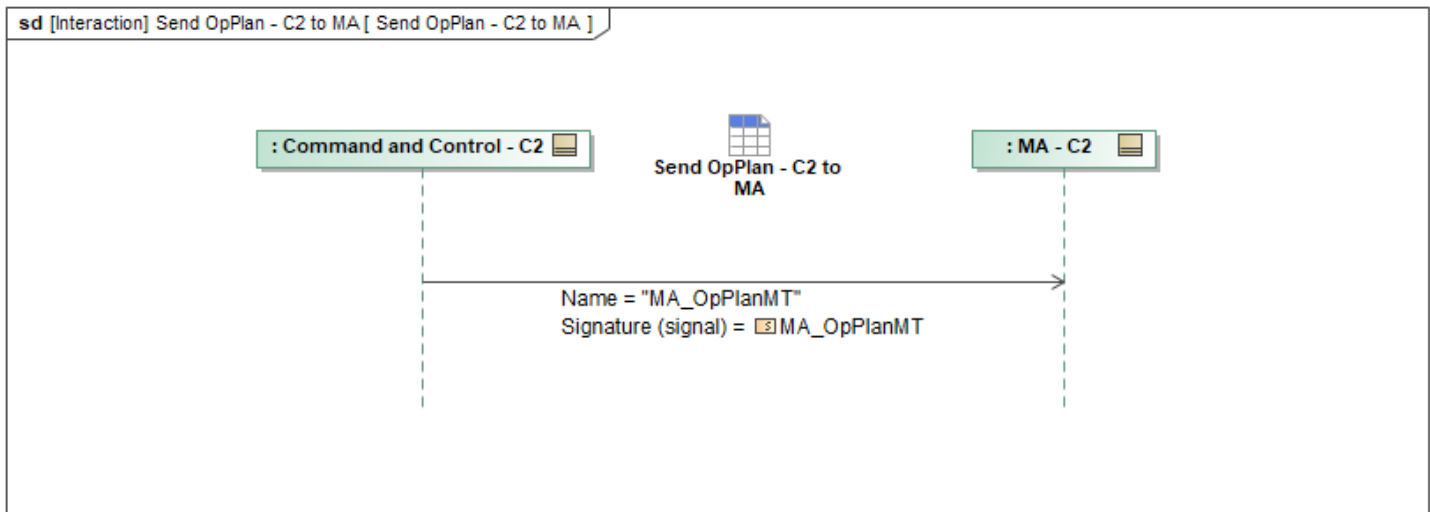


Figure 5-115: Send OpPlan - C2 to MA

Table 5-114: Send OpPlan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPlanMT: MA_OpPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.21 Send OpPoint - C2 to MA

This interaction has C2 send *MA_OpPointMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

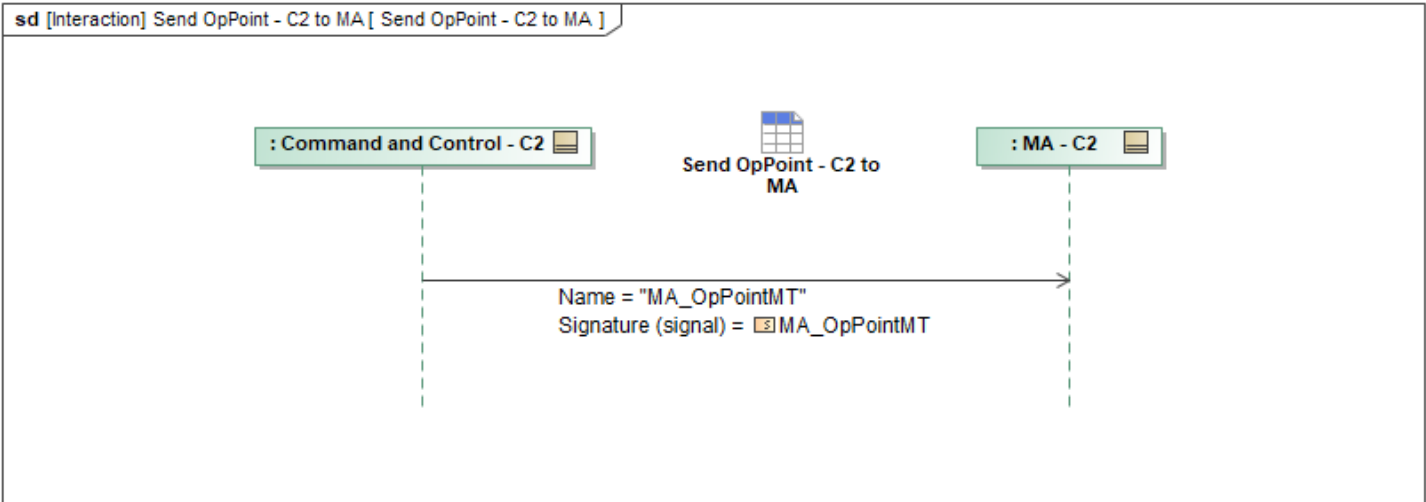


Figure 5-116: Send OpPoint - C2 to MA

Table 5-115: Send OpPoint - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPointMT: MA_OpPointMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.22 Send OpRouting - C2 to MA

This interaction has C2 send *MA_OpRoutingMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

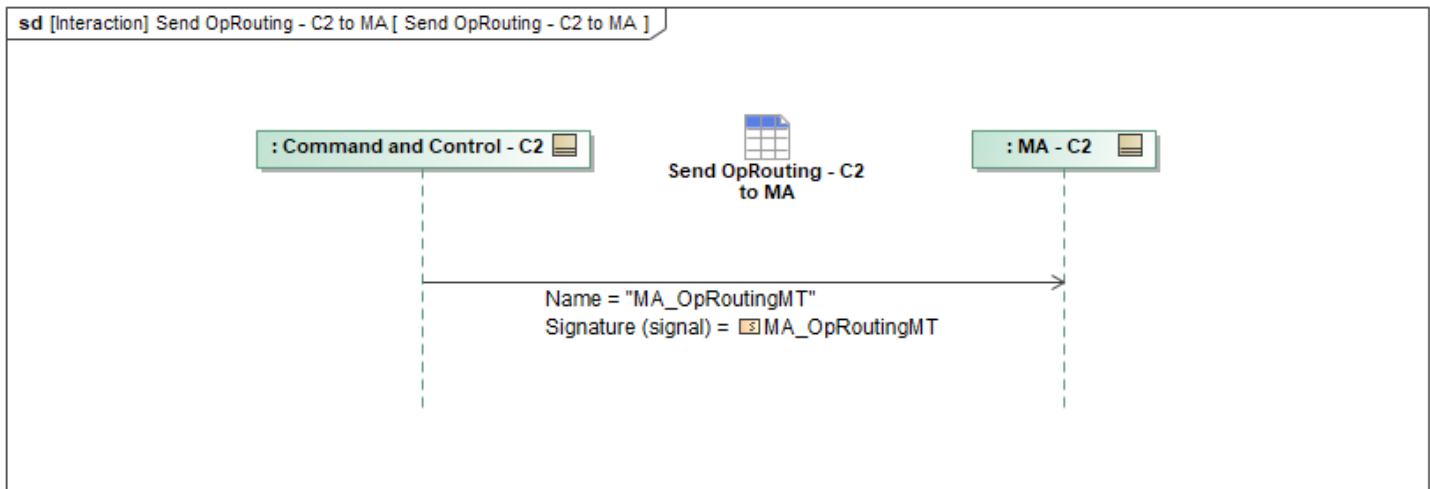


Figure 5-117: Send OpRouting - C2 to MA

Table 5-116: Send OpRouting - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpRoutingMT: MA_OpRoutingMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.23 Send OpVolume - C2 to MA

This interaction has C2 send *MA_OpVolumeMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

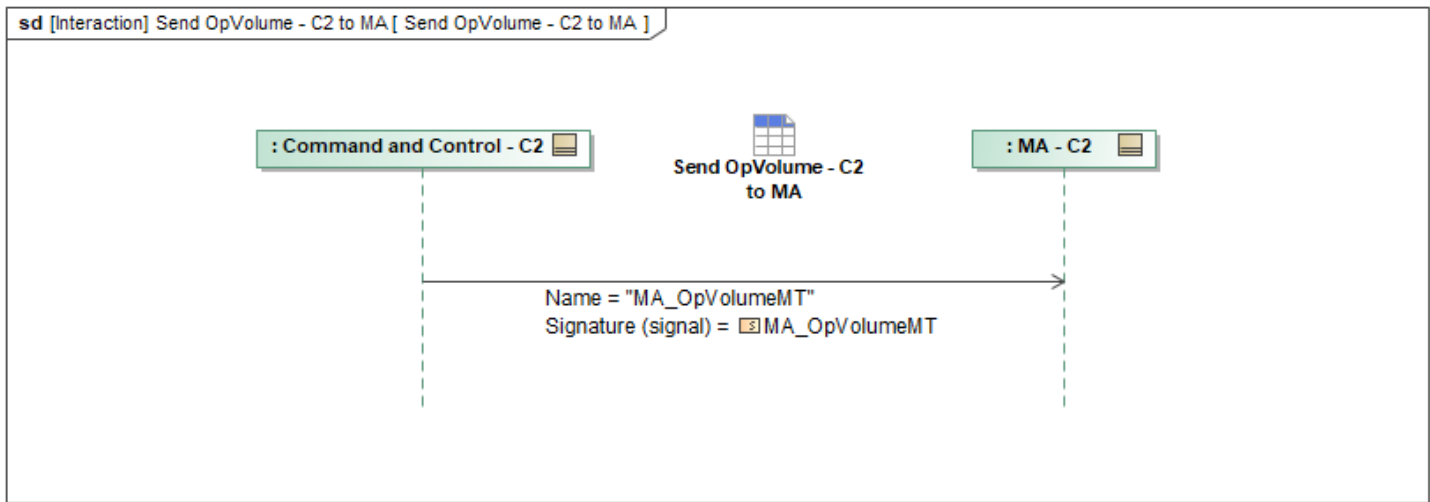


Figure 5-118: Send OpVolume - C2 to MA

Table 5-117: Send OpVolume - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpVolumeMT: MA_OpVolumeMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.24 Send OpZone - C2 to MA

This interaction has C2 send *MA_OpZoneMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

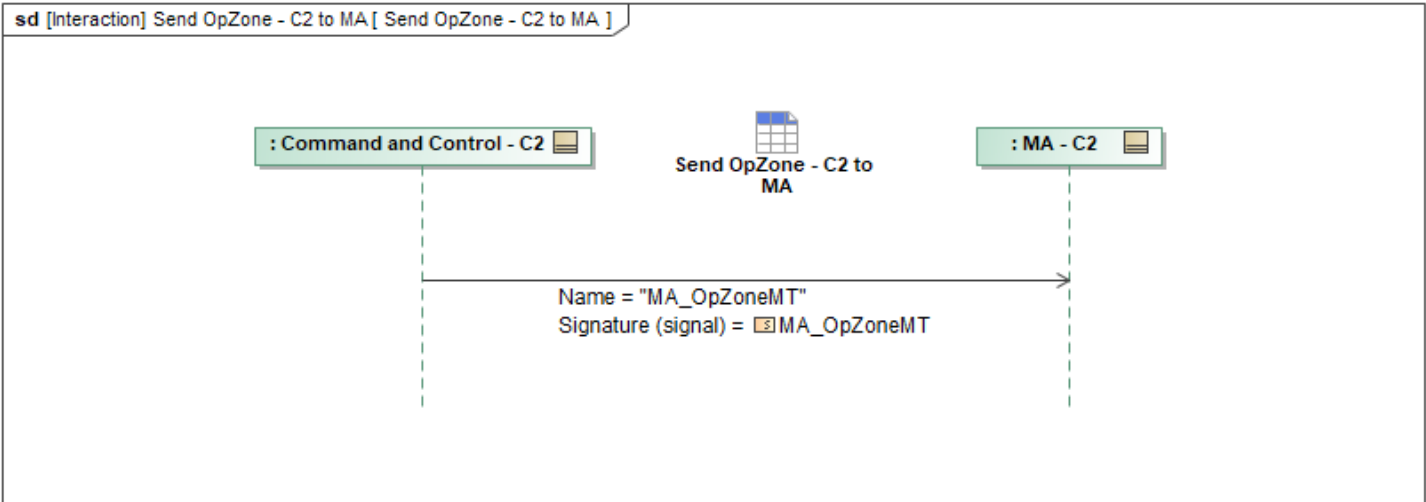


Figure 5-119: Send OpZone - C2 to MA

Table 5-118: Send OpZone - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpZoneMT: MA_OpZoneMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.25 Send Package - C2 to MA

This interaction has C2 send *PackageMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

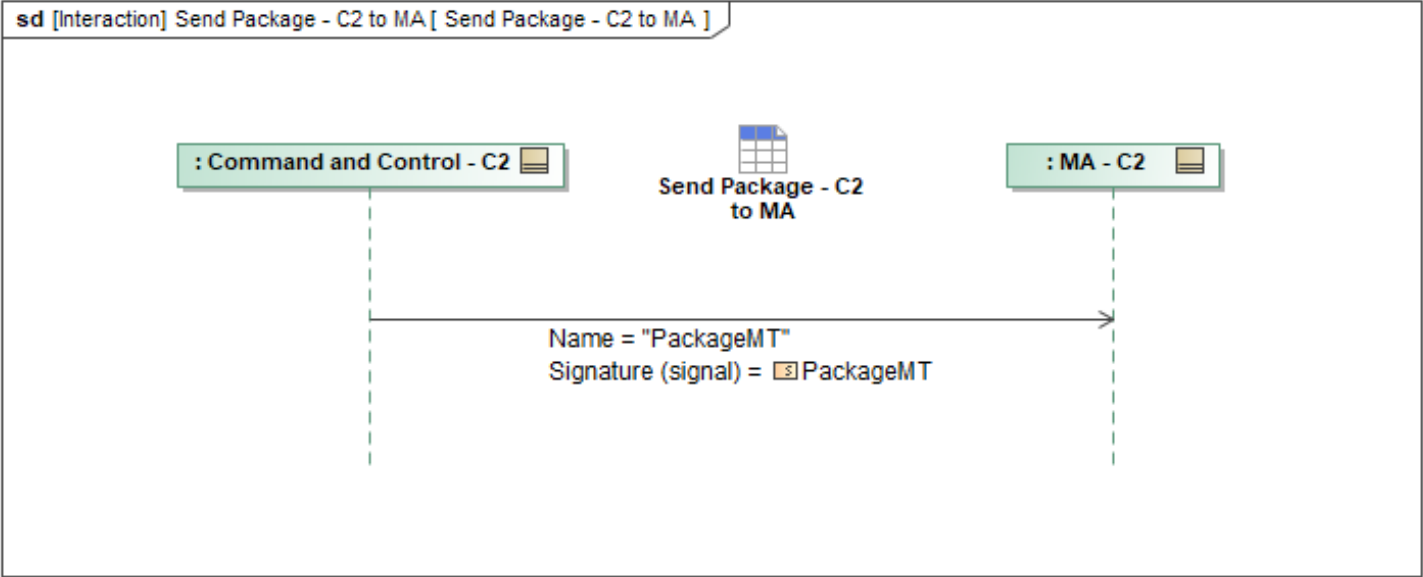


Figure 5-120: Send Package - C2 to MA

Table 5-119: Send Package - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
PackageMT: PackageMT	DataRecordInstanceID: DataRecordInstanceID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.26 Send Response - C2 to MA

This interaction has C2 send *MA_ResponseMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

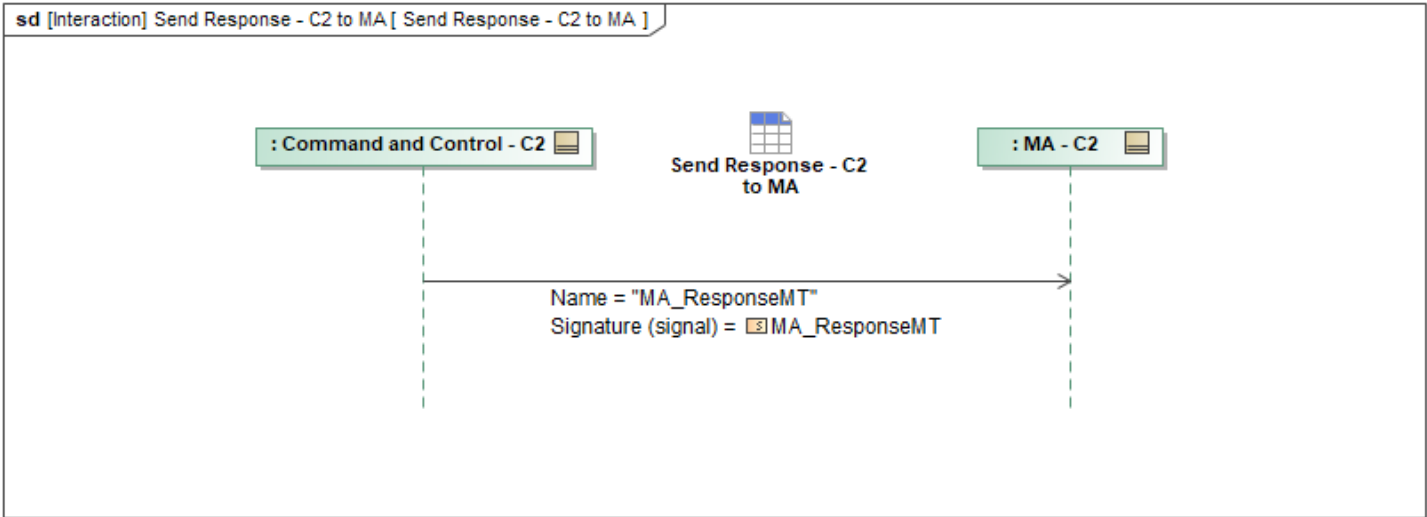


Figure 5-121: Send Response - C2 to MA

Table 5-120: Send Response - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseMT: MA_ResponseMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.27 Send Response Plan - C2 to MA

This interaction has C2 send *ResponsePlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

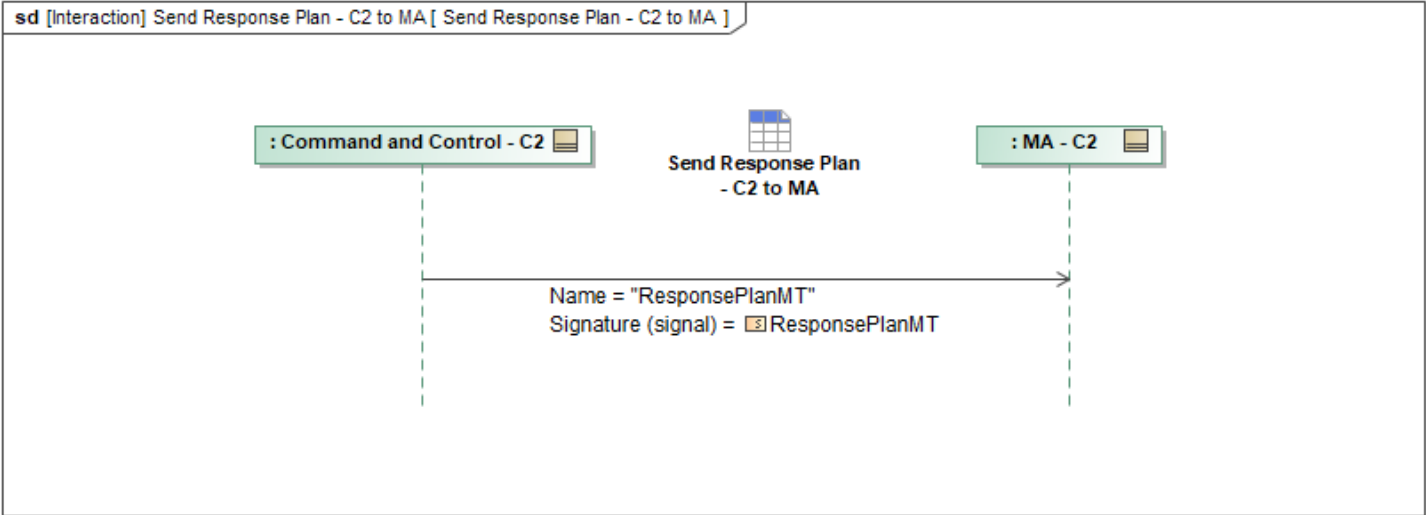


Figure 5-122: Send Response Plan - C2 to MA

Table 5-121: Send Response Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
ResponsePlanMT: ResponsePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.28 Send Route Activity Plan - C2 to MA

This interaction has C2 send *MA_RouteActivityPlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

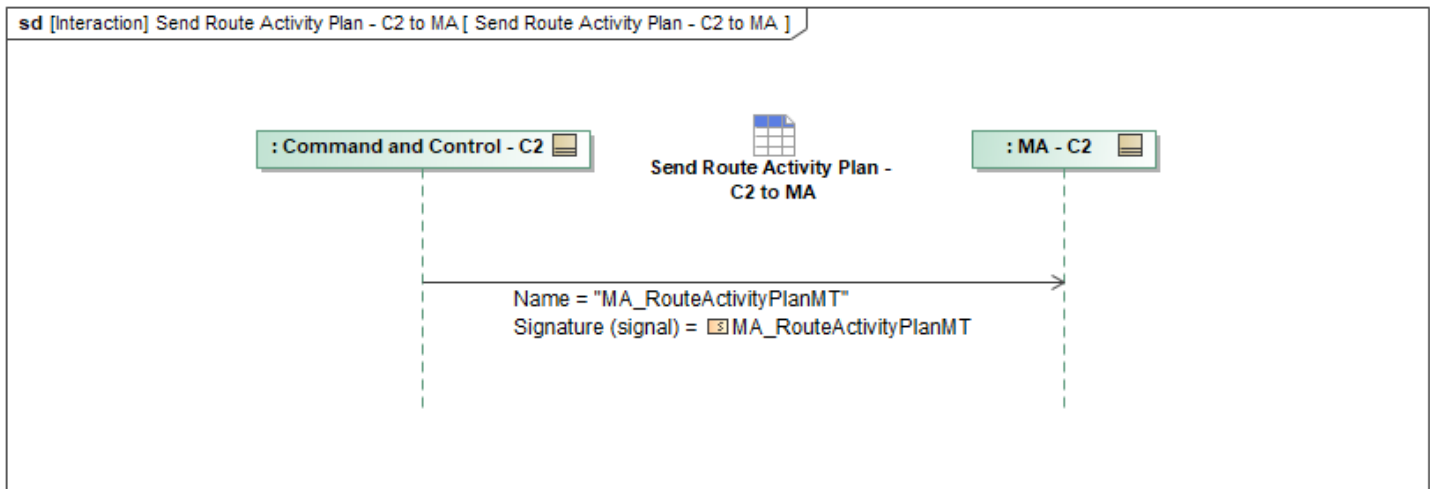


Figure 5-123: Send Route Activity Plan - C2 to MA

Table 5-122: Send Route Activity Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteActivityPlanMT: MA_RouteActivityPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.29 Send Route Metrics - C2 to MA

This interaction has C2 send *RouteMetricsMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

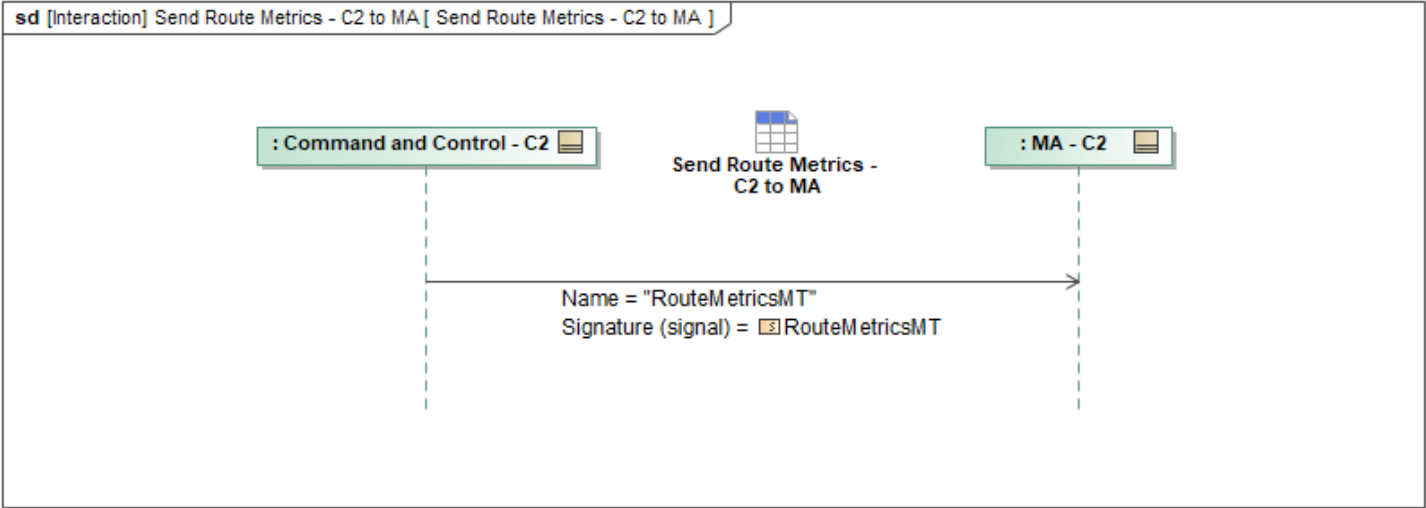


Figure 5-124: Send Route Metrics - C2 to MA

Table 5-123: Send Route Metrics - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
RouteMetricsMT: RouteMetricsMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.30 Send Route Plan - C2 to MA

This interaction has C2 send *MA_RoutePlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

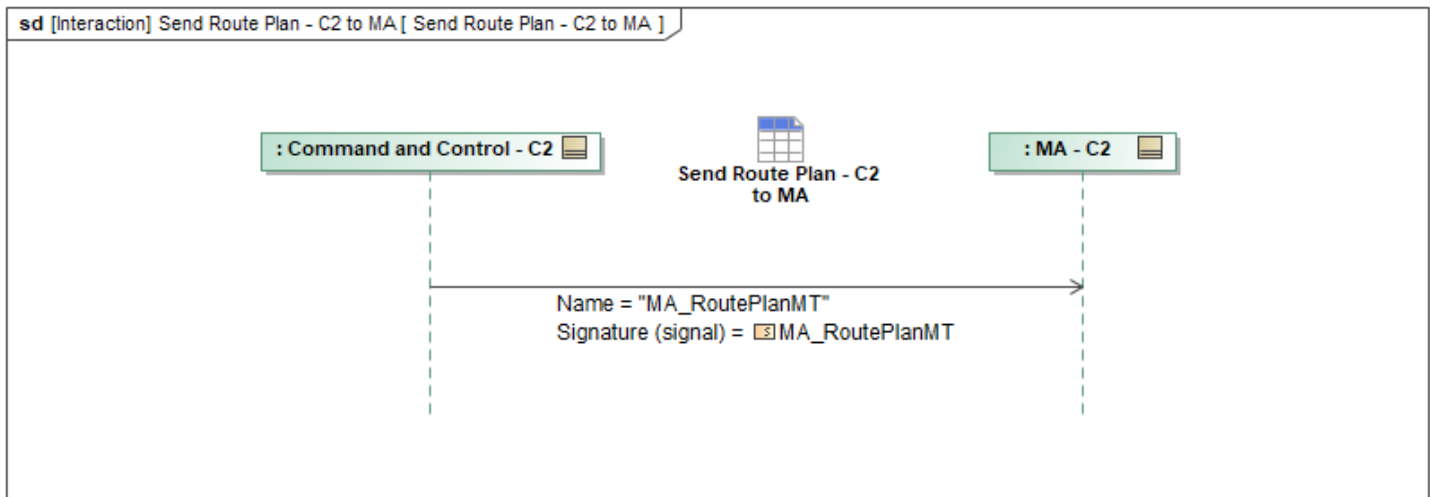


Figure 5-125: Send Route Plan - C2 to MA

Table 5-124: Send Route Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.31 Send Task - C2 to MA

This interaction has C2 send *MA_TaskMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

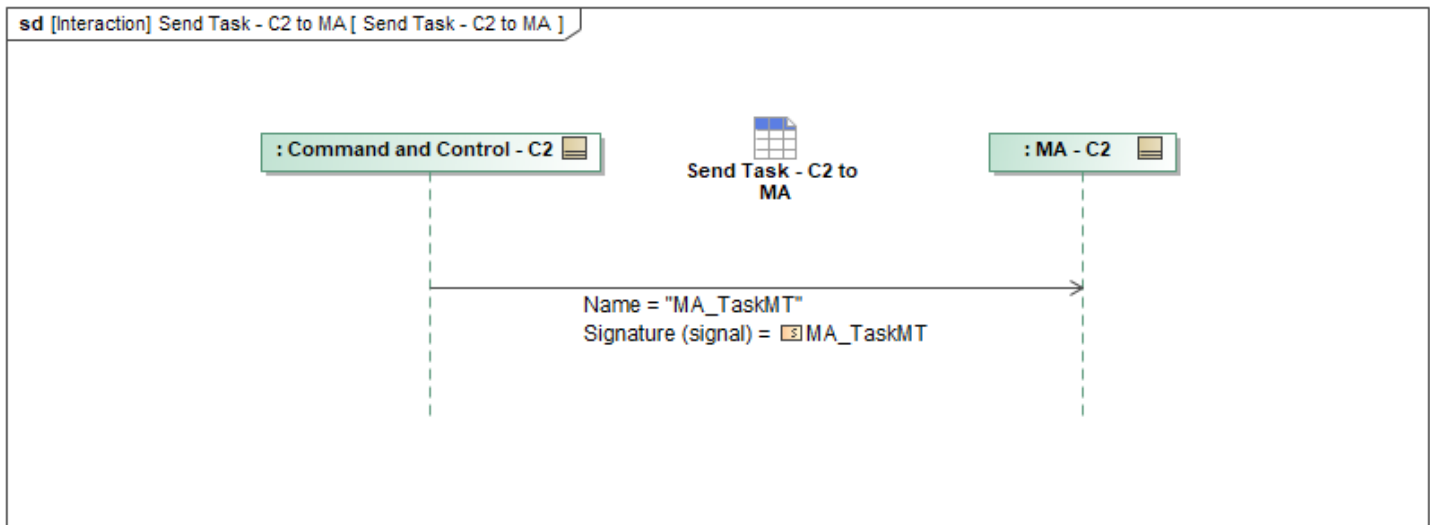


Figure 5-126: Send Task - C2 to MA

Table 5-125: Send Task - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.32 Send Task Plan - C2 to MA

This interaction has C2 send *MA_TaskPlanMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

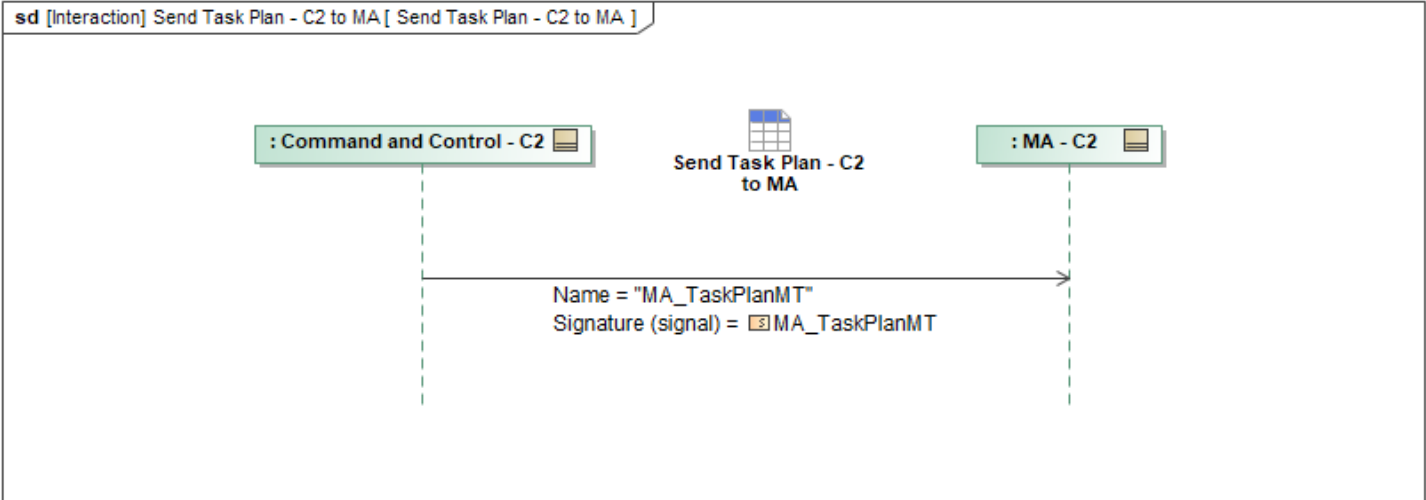


Figure 5-127: Send Task Plan - C2 to MA

Table 5-126: Send Task Plan - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskPlanMT: MA_TaskPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.22.33 Send Weather Dataset - C2 to MA

This interaction has C2 send *WeatherDatasetMT* to MA.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

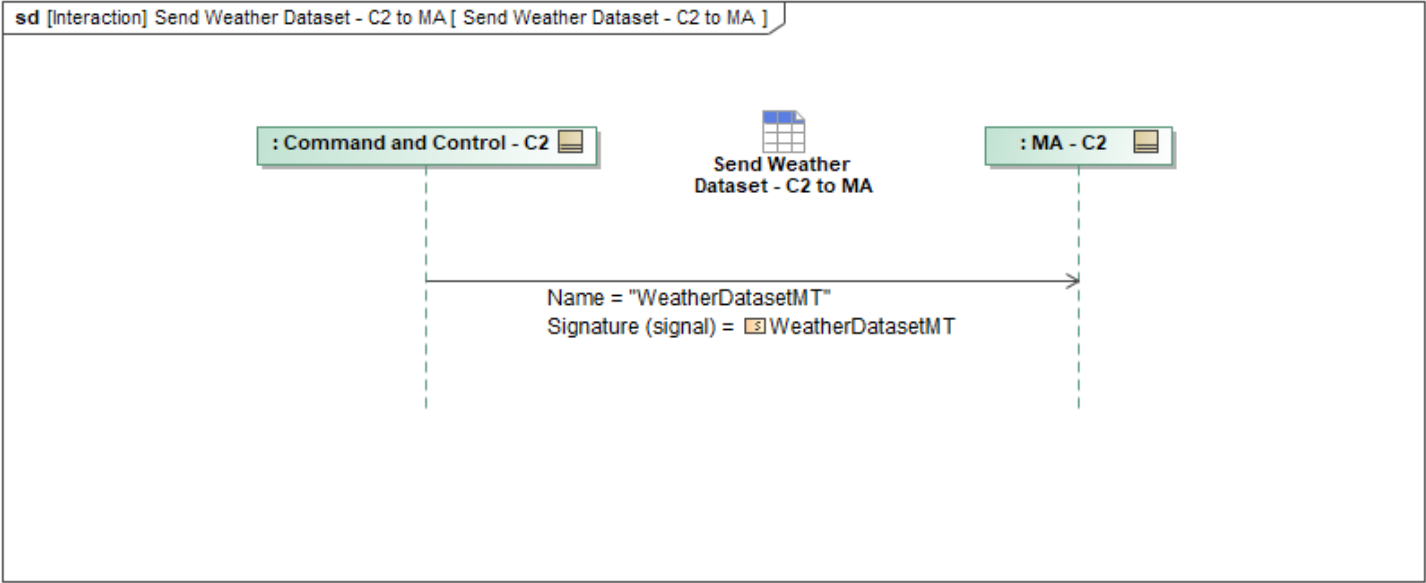


Figure 5-128: Send Weather Dataset - C2 to MA

Table 5-127: Send Weather Dataset - C2 to MA

Message Name (Name:Type)	Required Fields (Name:Type)
WeatherDatasetMT: WeatherDatasetMT	DataRecordInstanceID: DataRecordInstanceID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23 Send Data - MA to C2

5.2.8.23.1 Send Action - MA to C2

This interaction has MA send *MA_ActionMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

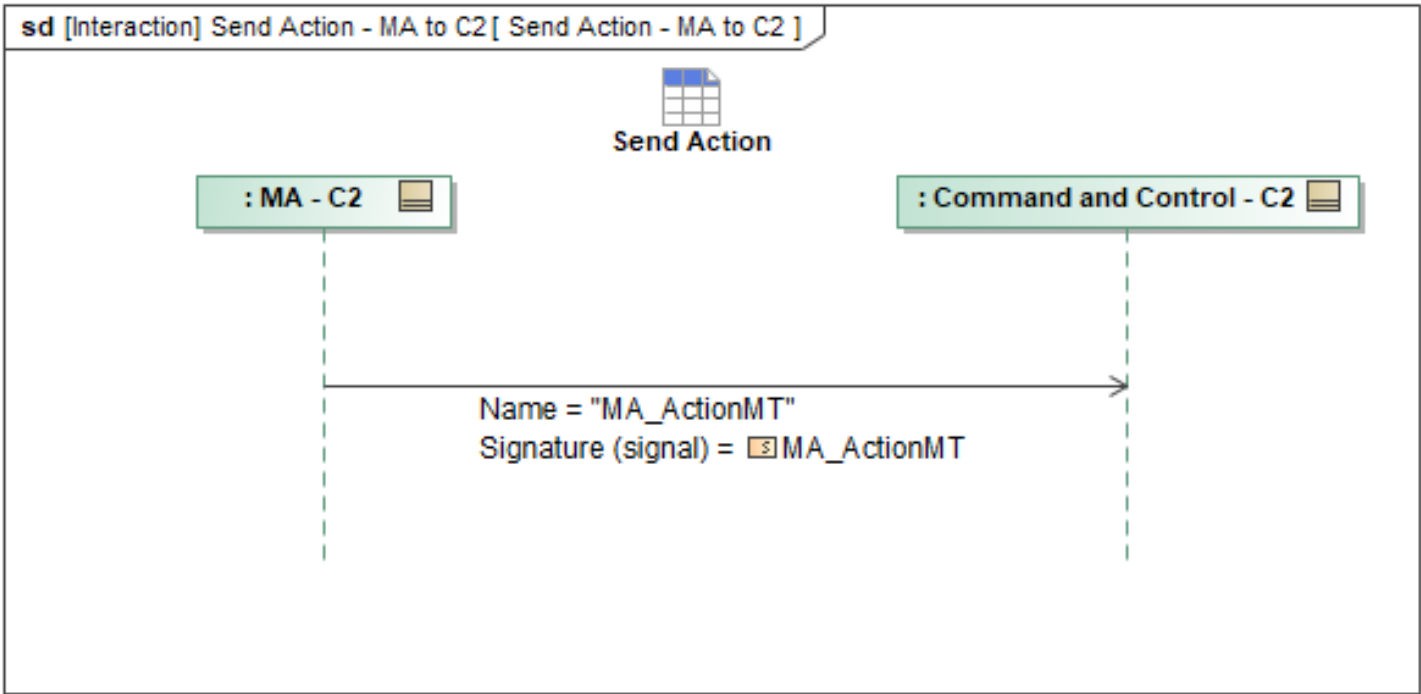


Figure 5-129: Send Action - MA to C2

Table 5-128: Send Action - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionMT: MA_ActionMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.2 Send Action Plan - MA to C2

This interaction has MA send *MA_ActionPlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

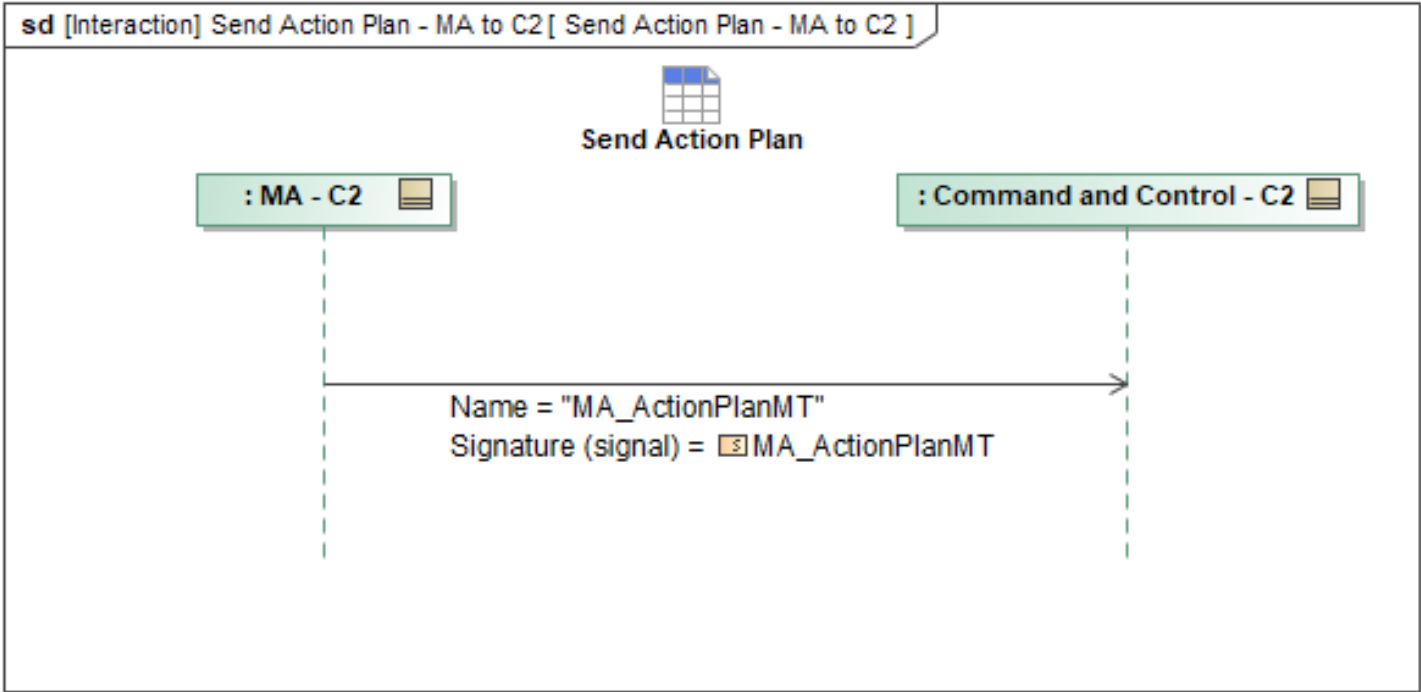


Figure 5-130: Send Action Plan - MA to C2

Table 5-129: Send Action Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionPlanMT: MA_ActionPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.3 Send Activity Plan - MA to C2

This interaction has MA send *MA_ActivityPlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

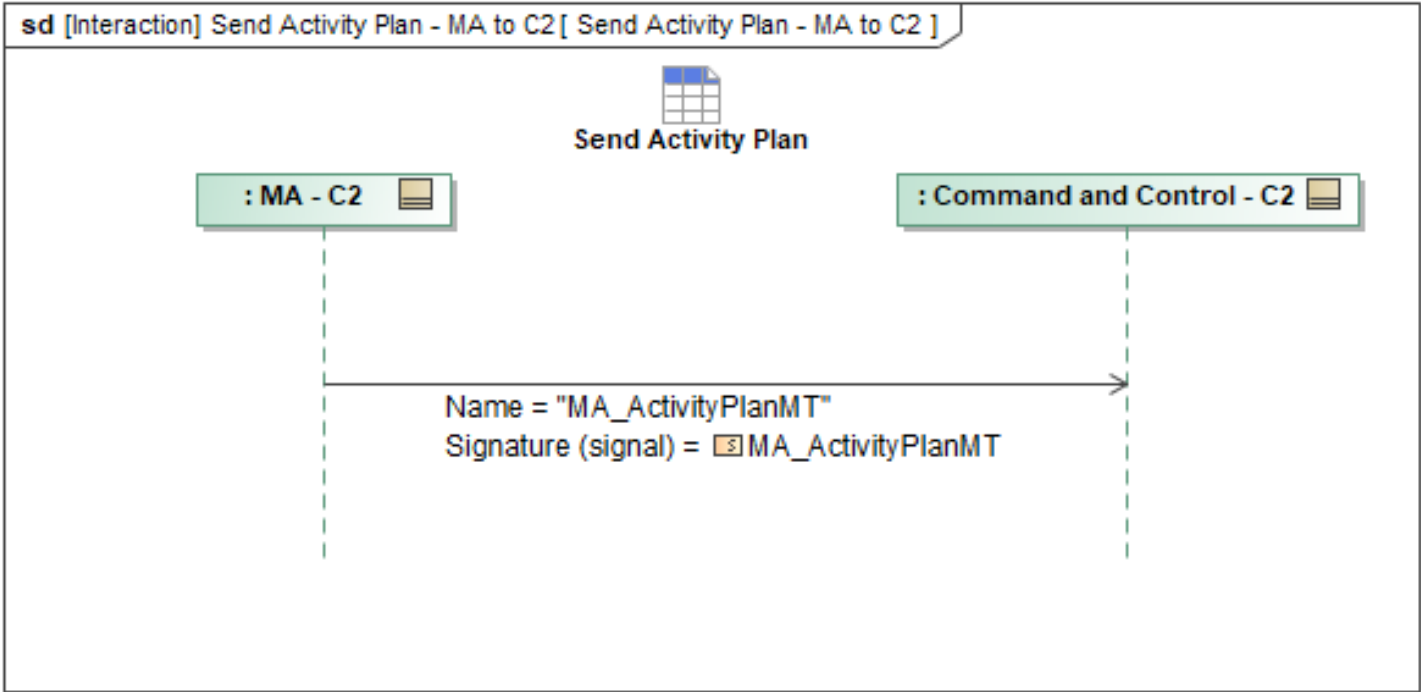


Figure 5-131: Send Activity Plan - MA to C2

Table 5-130: Send Activity Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.4 Send Approval Authority - MA to C2

This interaction has MA send *MA_ApprovalAuthorityMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

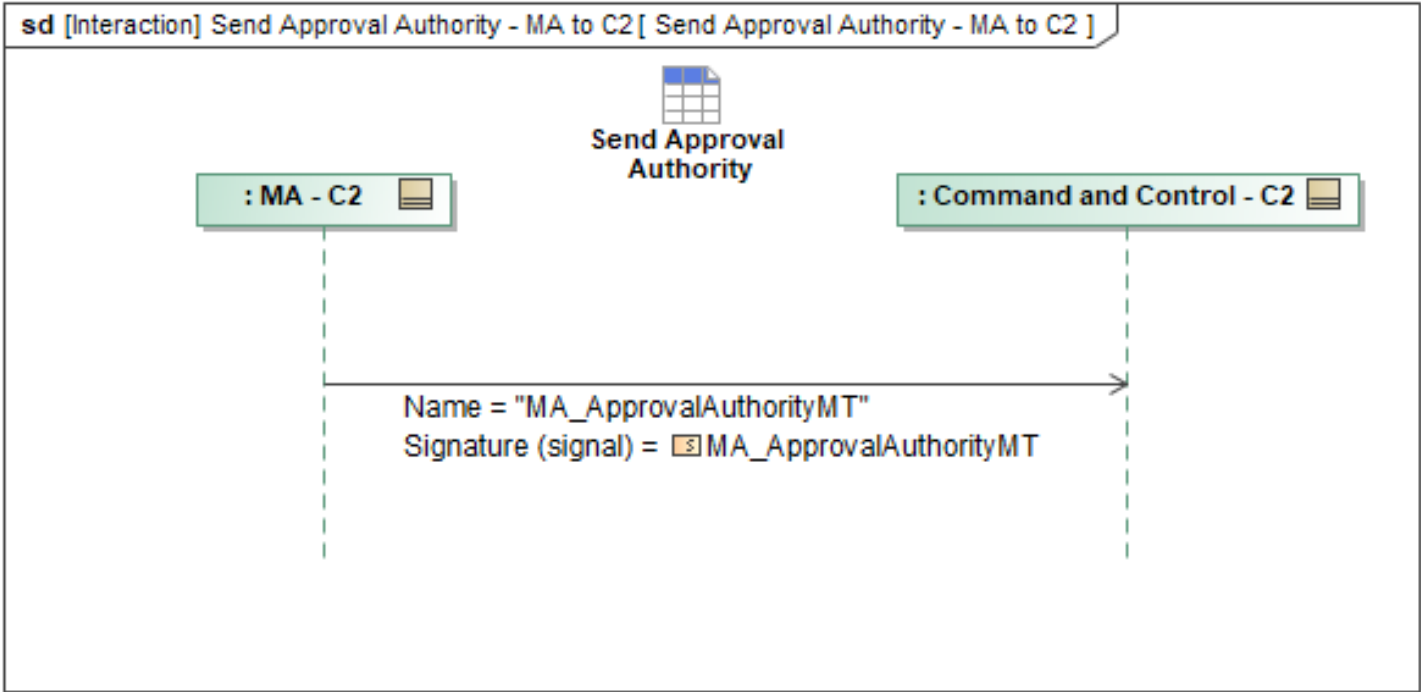


Figure 5-132: Send Approval Authority - MA to C2

Table 5-131: Send Approval Authority - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalAuthorityMT: MA_ApprovalAuthorityMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.5 Send Approval Policy - MA to C2

This interaction has MA send *MA_ApprovalPolicyMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

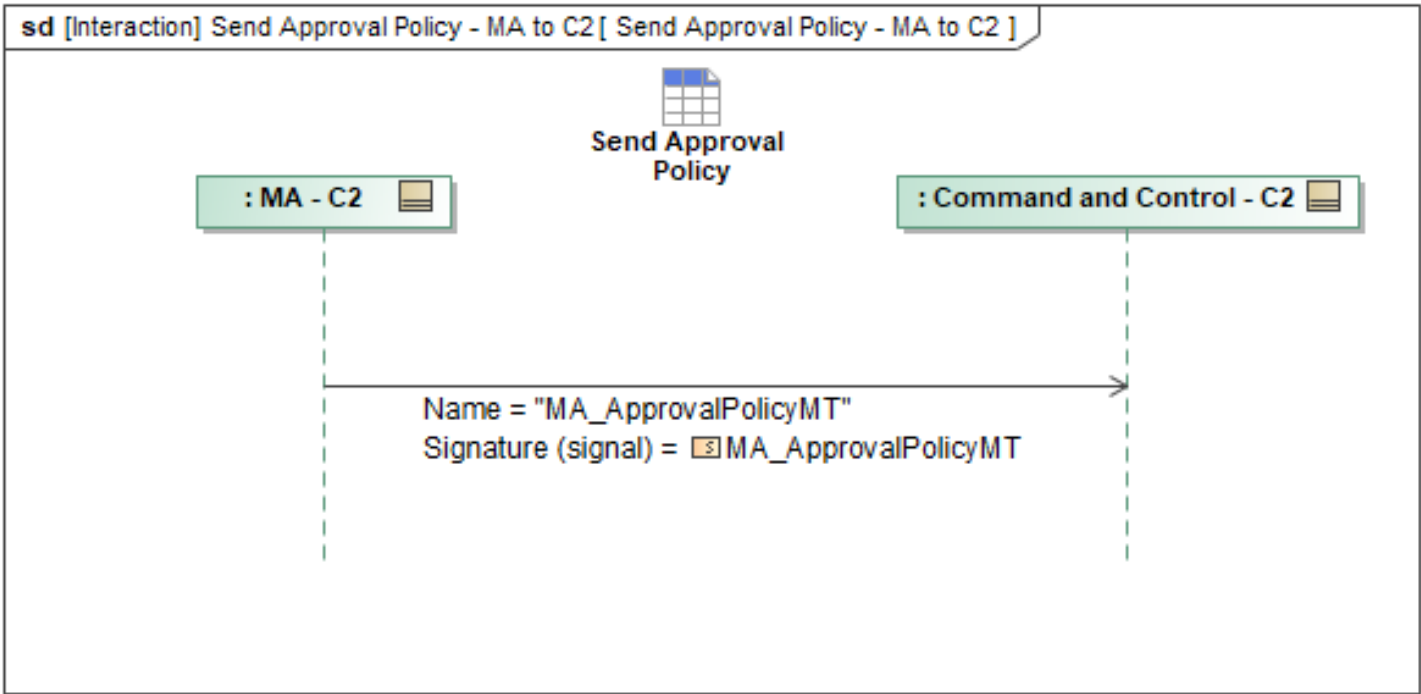


Figure 5-133: Send Approval Policy - MA to C2

Table 5-132: Send Approval Policy - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.6 Send Capability Set - MA to C2

This interaction has MA send *MA_CapabilitySetMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

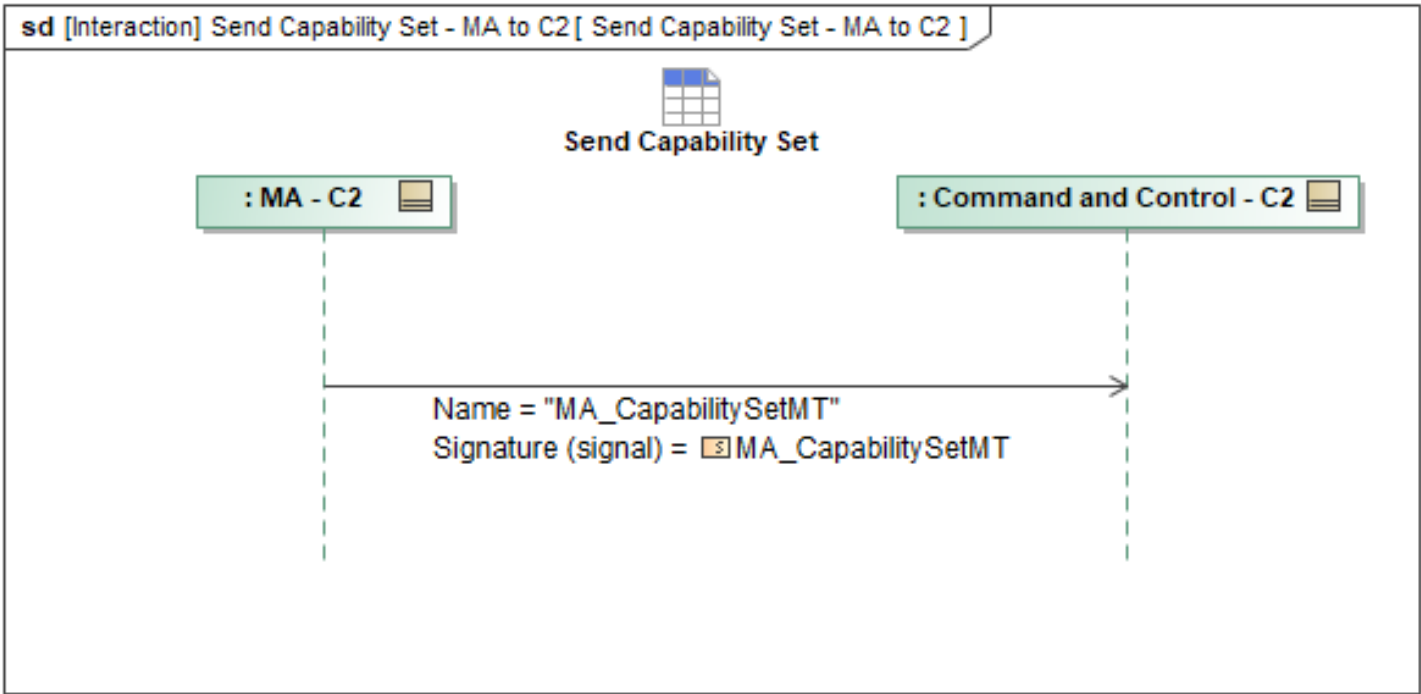


Figure 5-134: Send Capability Set - MA to C2

Table 5-133: Send Capability Set - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CapabilitySetMT: MA_CapabilitySetMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.7 Send Entity - MA to C2

This interaction has MA send *EntityMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

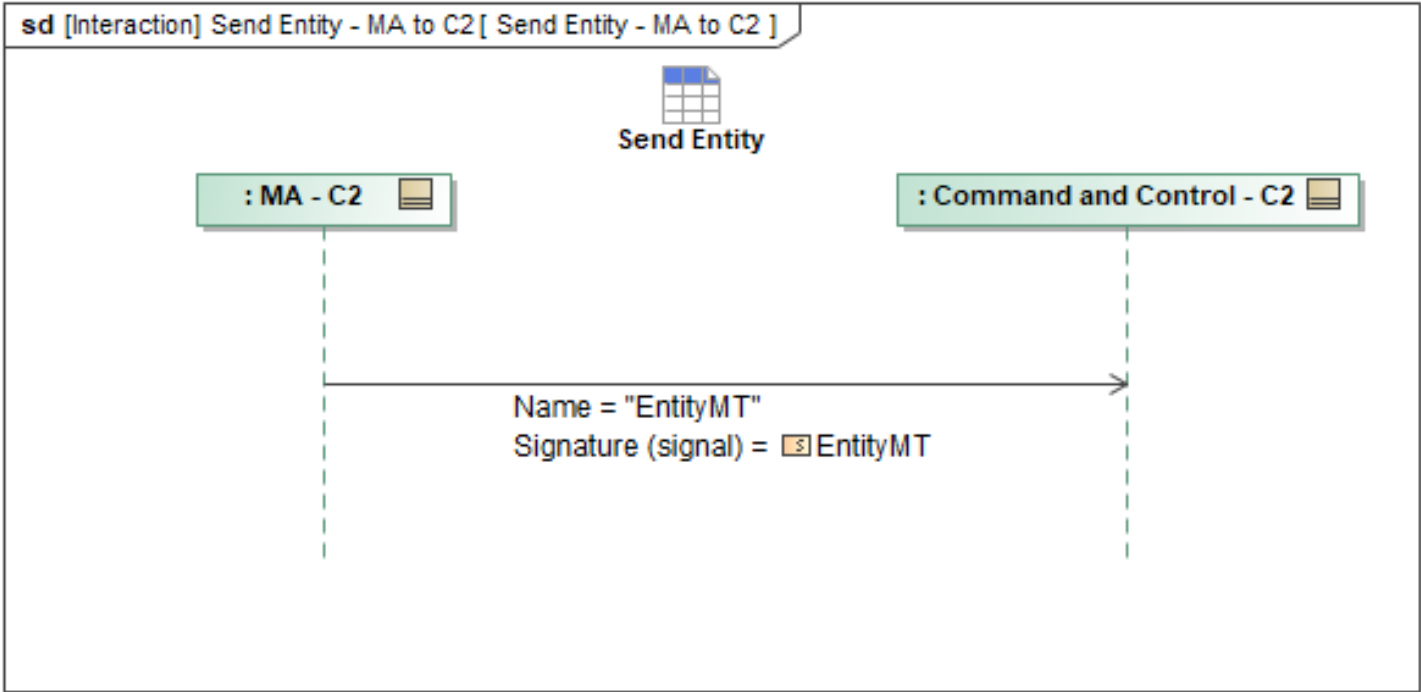


Figure 5-135: Send Entity - MA to C2

Table 5-134: Send Entity - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.8 Send Mission Plan - MA to C2

This interaction has MA send *MA_MissionPlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

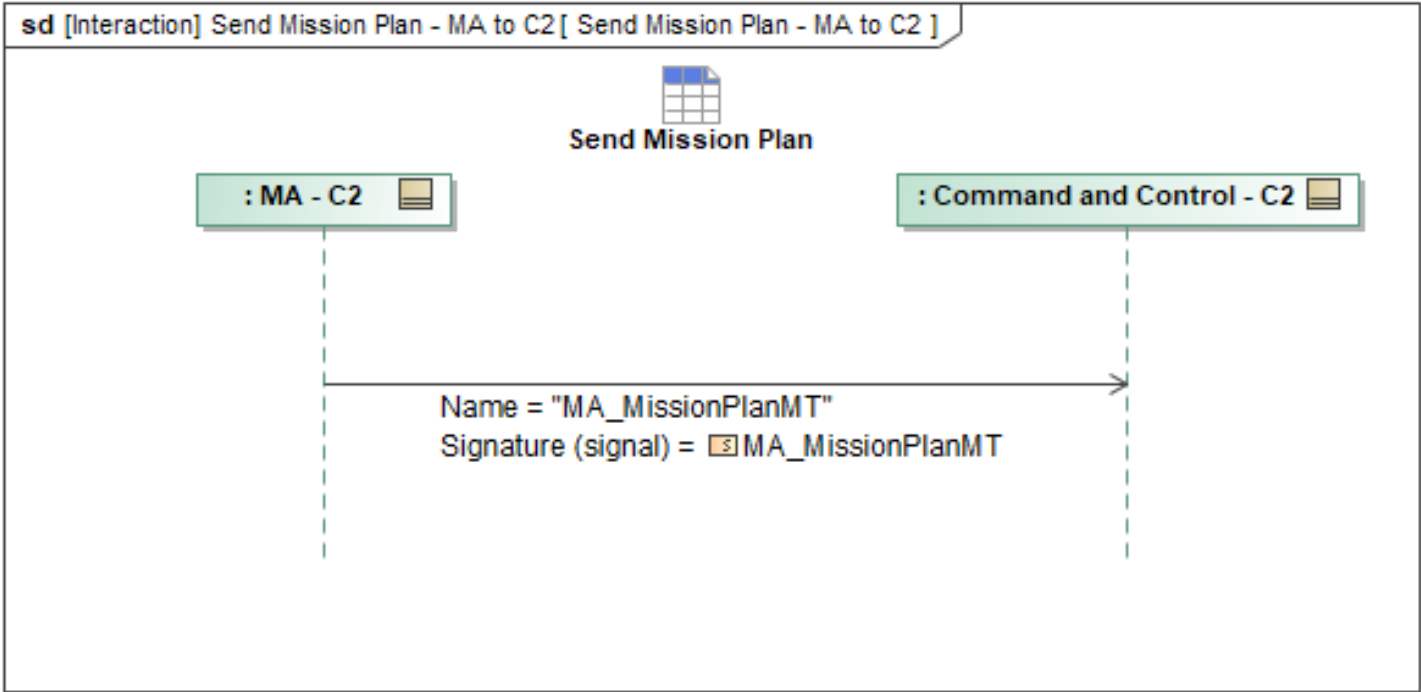


Figure 5-136: Send Mission Plan - MA to C2

Table 5-135: Send Mission Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.9 Send Op Line - MA to C2

This interaction has MA send *MA_OpLineMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

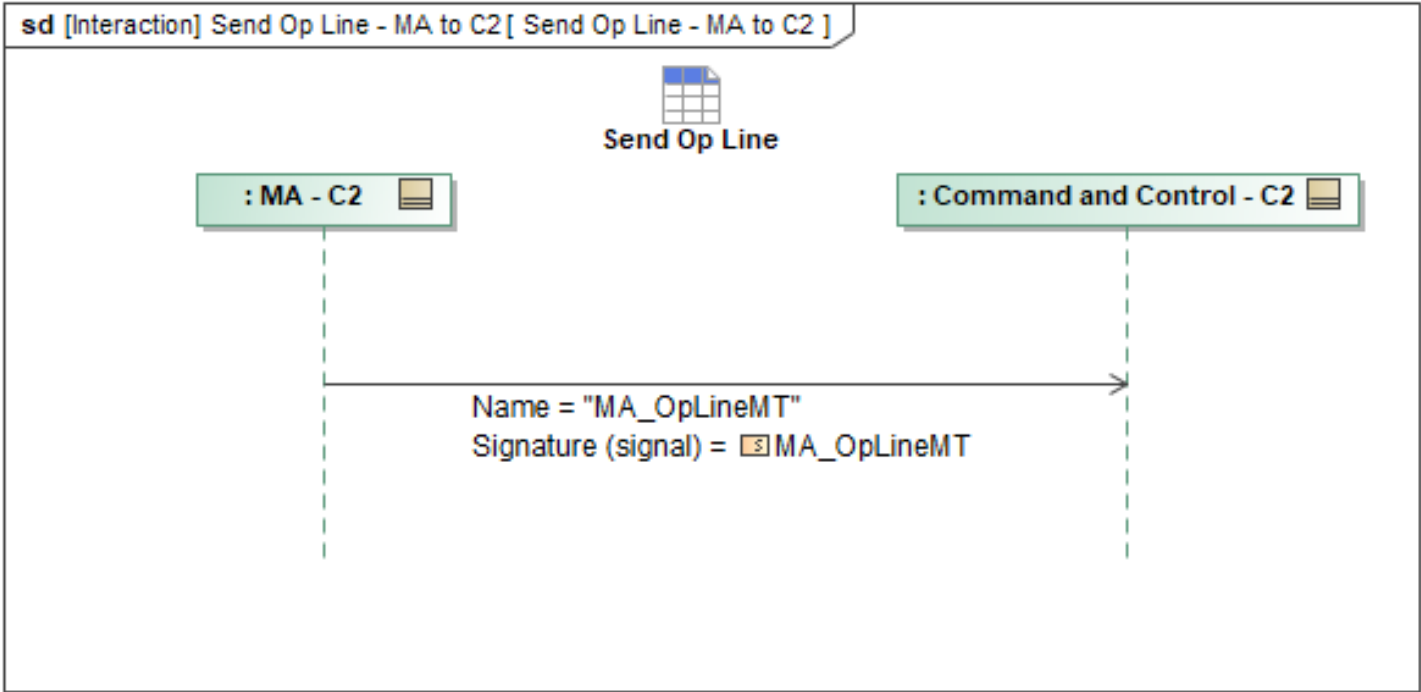


Figure 5-137: Send Op Line - MA to C2

Table 5-136: Send Op Line - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpLineMT: MA_OpLineMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.10 Send Op Routing - MA to C2

This interaction has MA send *MA_OpRoutingMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

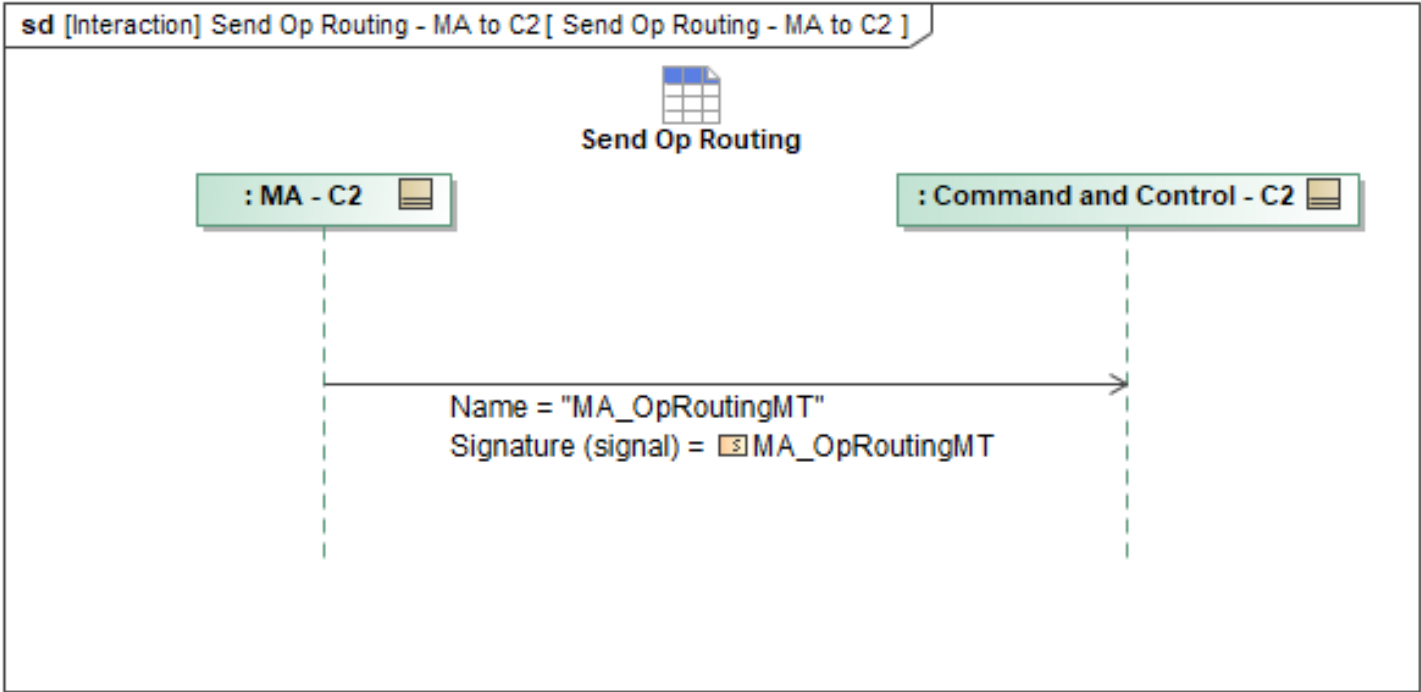


Figure 5-138: Send Op Routing - MA to C2

Table 5-137: Send Op Routing - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpRoutingMT: MA_OpRoutingMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.11 Send Op Volume - MA to C2

This interaction has MA send *MA_OpVolumeMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

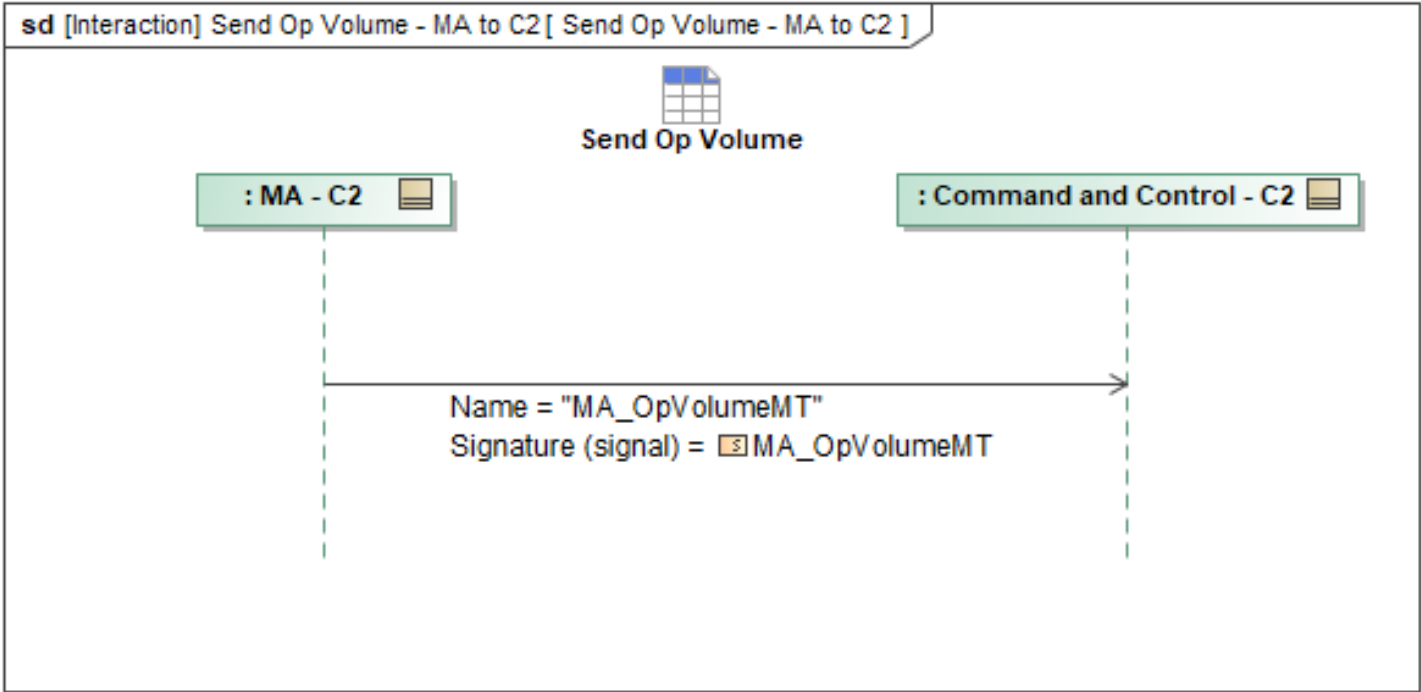


Figure 5-139: Send Op Volume - MA to C2

Table 5-138: Send Op Volume - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpVolumeMT: MA_OpVolumeMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.12 Send Op Zone - MA to C2

This interaction has MA send *MA_OpZoneMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

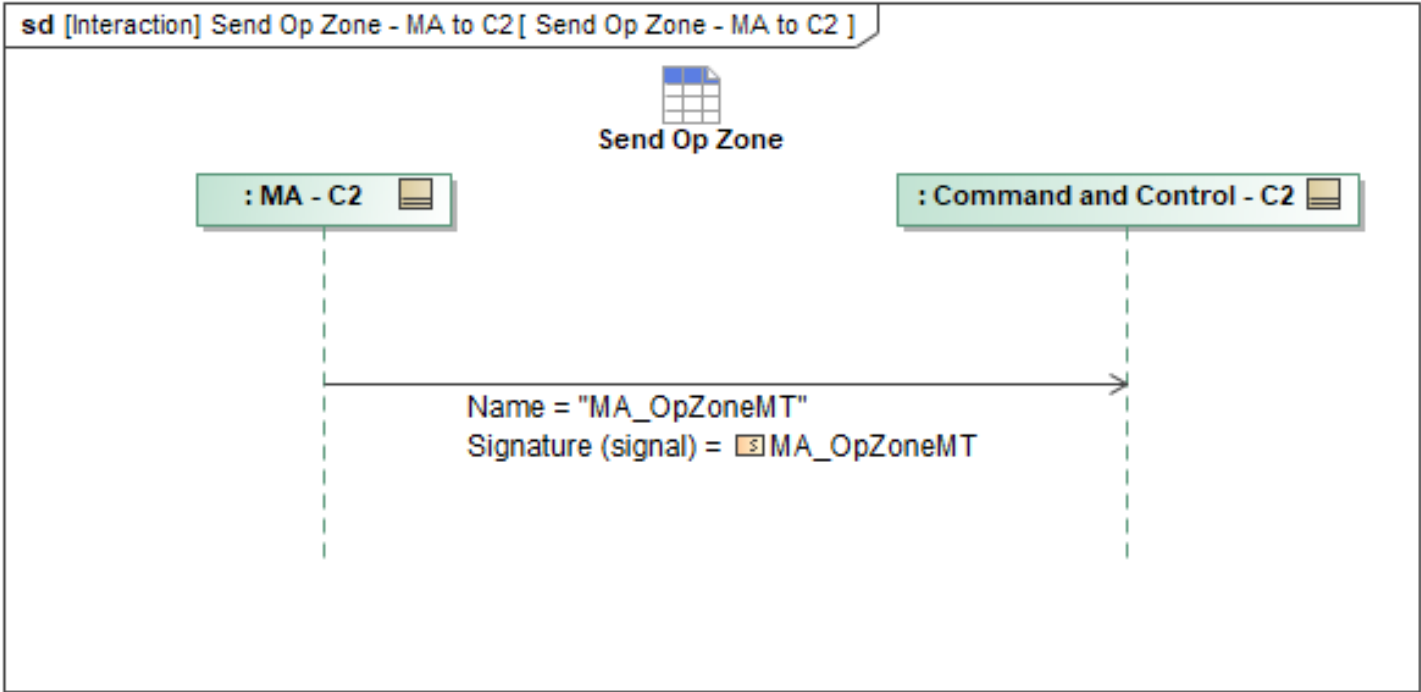


Figure 5-140: Send Op Zone - MA to C2

Table 5-139: Send Op Zone - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpZoneMT: MA_OpZoneMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.13 Send Operator - MA to C2

This interaction has MA send *OperatorMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

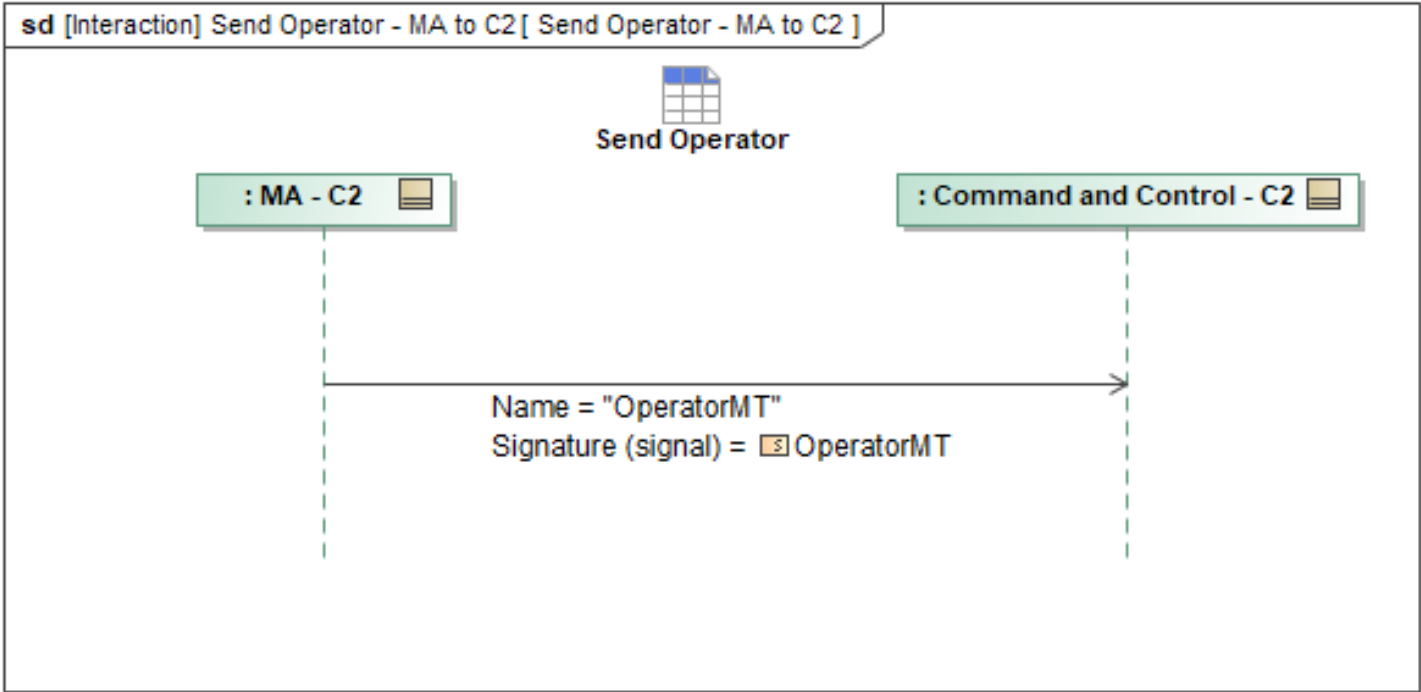


Figure 5-141: Send Operator - MA to C2

Table 5-140: Send Operator - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorMT: OperatorMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.14 Send Operator Notification - MA to C2

This interaction has MA send *OperatorNotificationMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

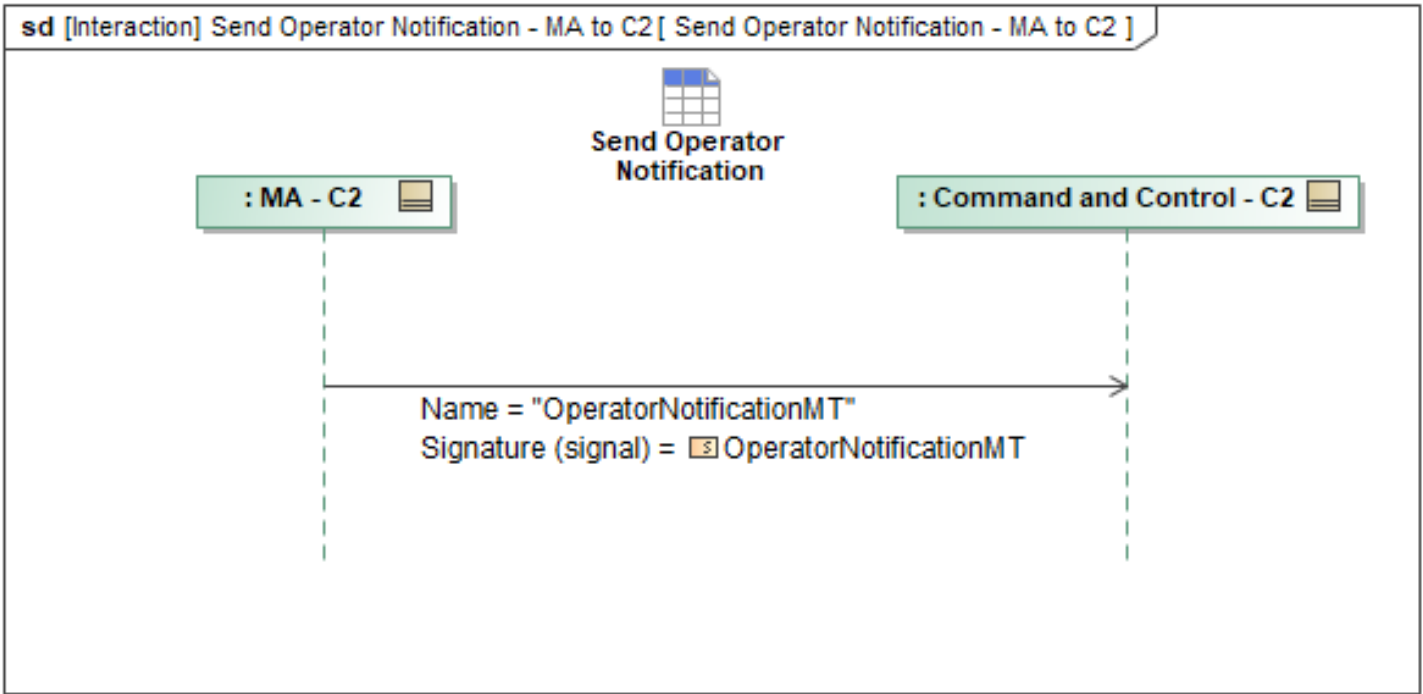


Figure 5-142: Send Operator Notification - MA to C2

Table 5-141: Send Operator Notification - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorNotificationMT: OperatorNotificationMT	Destinations: OperatorRoleType ObjectState: ObjectStateEnum Source: NotificationSourceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.15 Send Operator Role - MA to C2

This interaction has MA send *MA_OperatorRoleMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

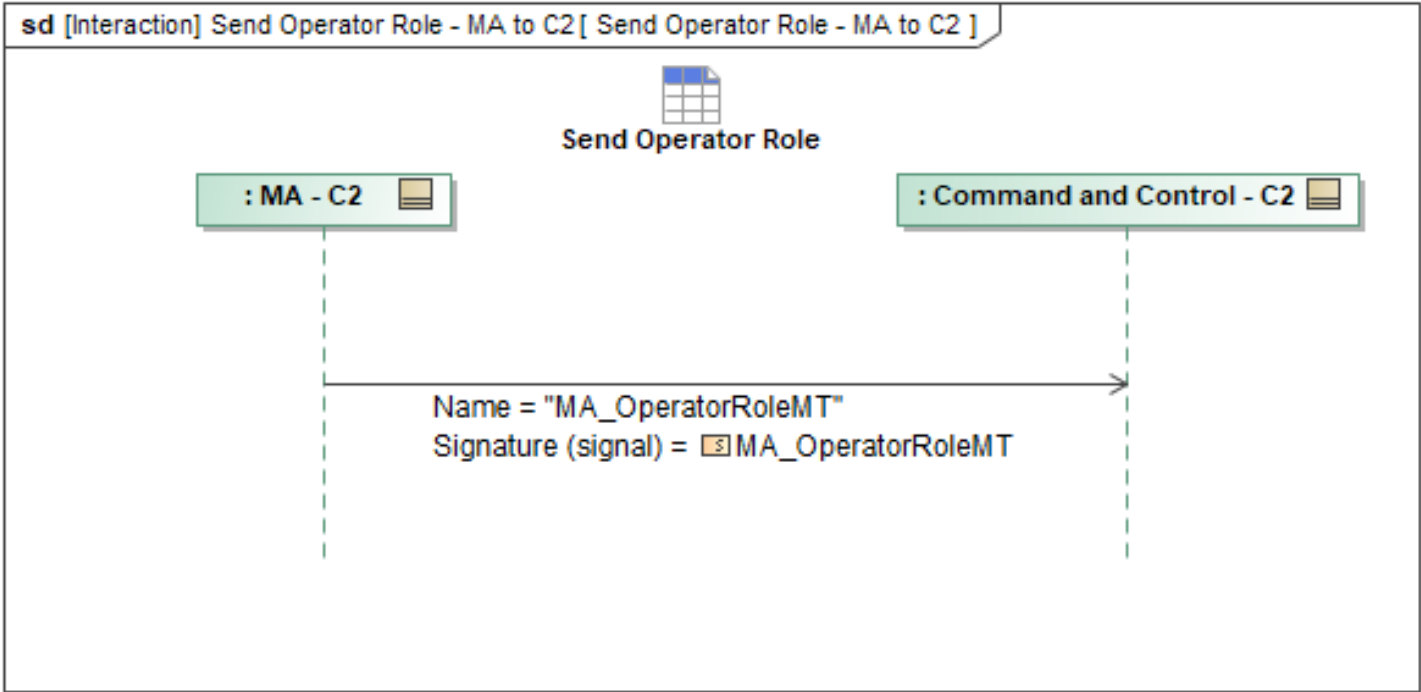


Figure 5-143: Send Operator Role - MA to C2

Table 5-142: Send Operator Role - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.16 Send Response - MA to C2

This interaction has MA send *MA_ResponseMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

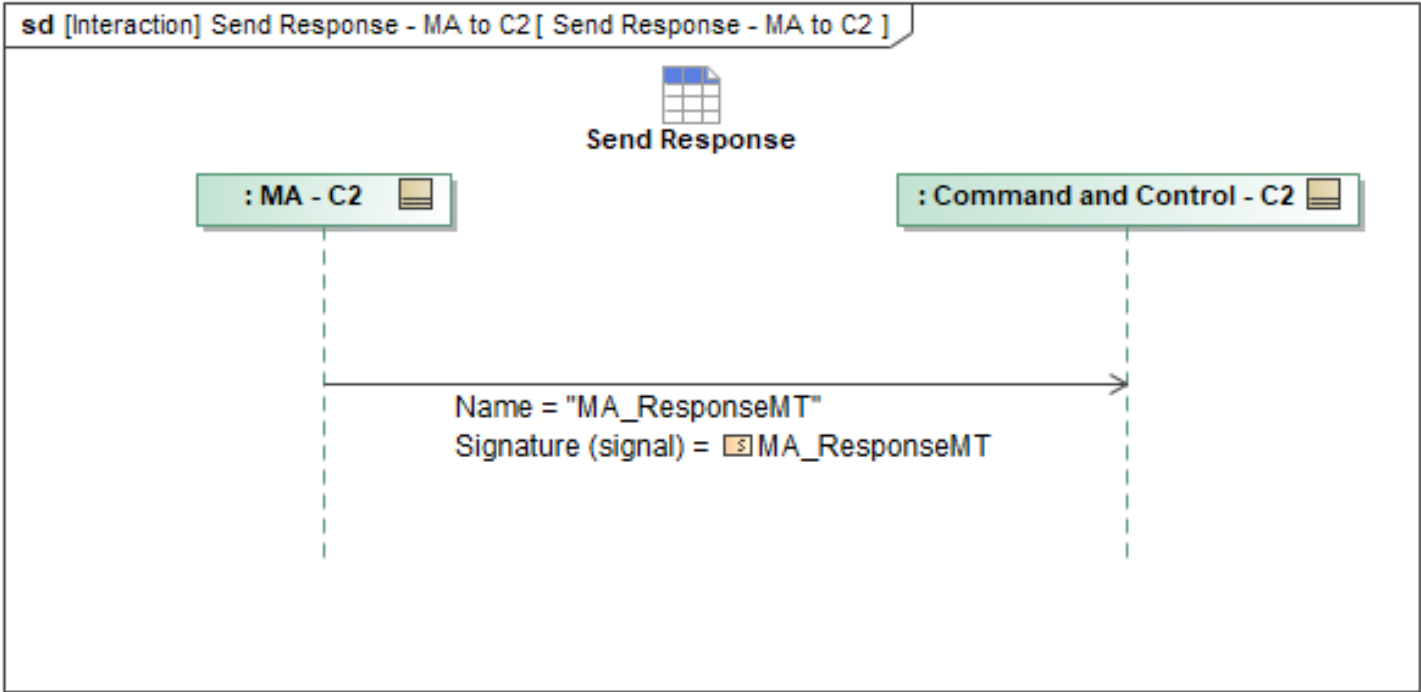


Figure 5-144: Send Response - MA to C2

Table 5-143: Send Response - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseMT: MA_ResponseMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.17 Send Response Plan - MA to C2

This interaction has MA send *ResponsePlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

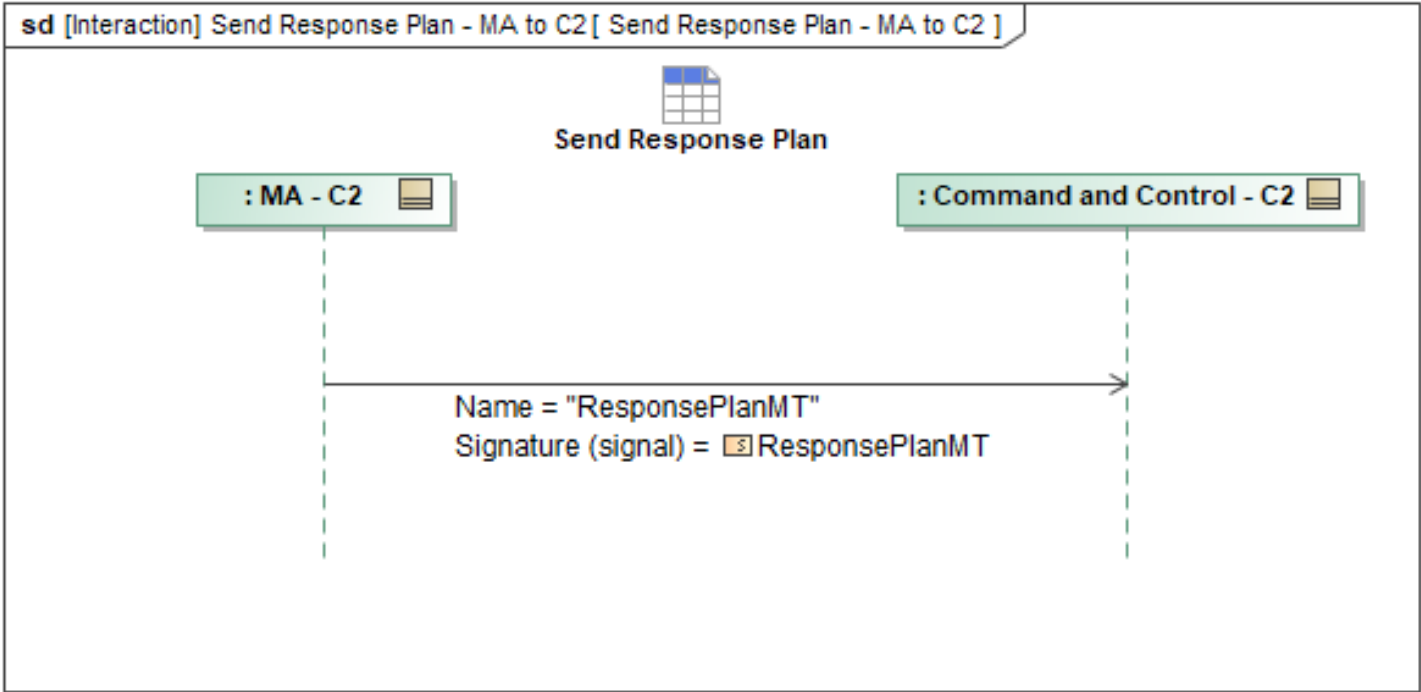


Figure 5-145: Send Response Plan - MA to C2

Table 5-144: Send Response Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
ResponsePlanMT: ResponsePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.18 Send Route Activity Plan - MA to C2

This interaction has MA send *MA_RouteActivityPlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

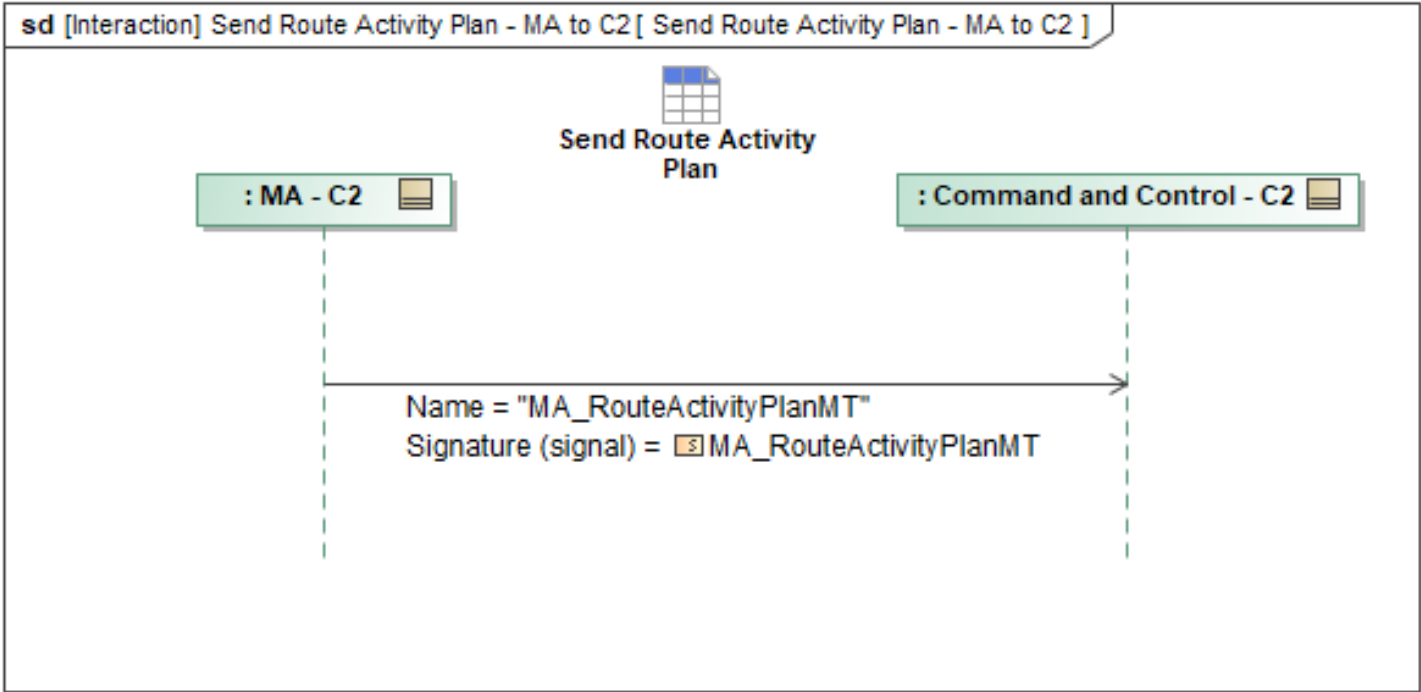


Figure 5-146: Send Route Activity Plan - MA to C2

Table 5-145: Send Route Activity Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteActivityPlanMT: MA_RouteActivityPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.19 Send Route Plan - MA to C2

This interaction has MA send *MA_RoutePlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

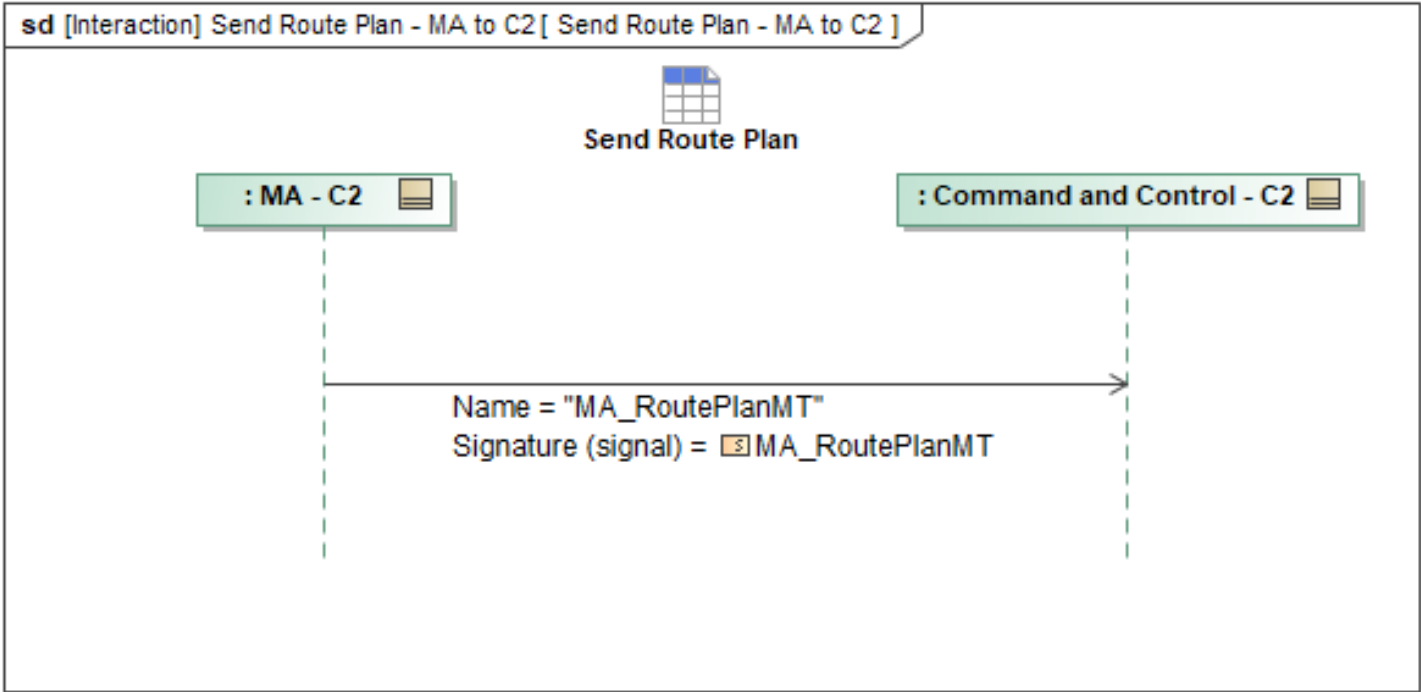


Figure 5-147: Send Route Plan - MA to C2

Table 5-146: Send Route Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.20 Send Task - MA to C2

This interaction has MA send *MA_TaskMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

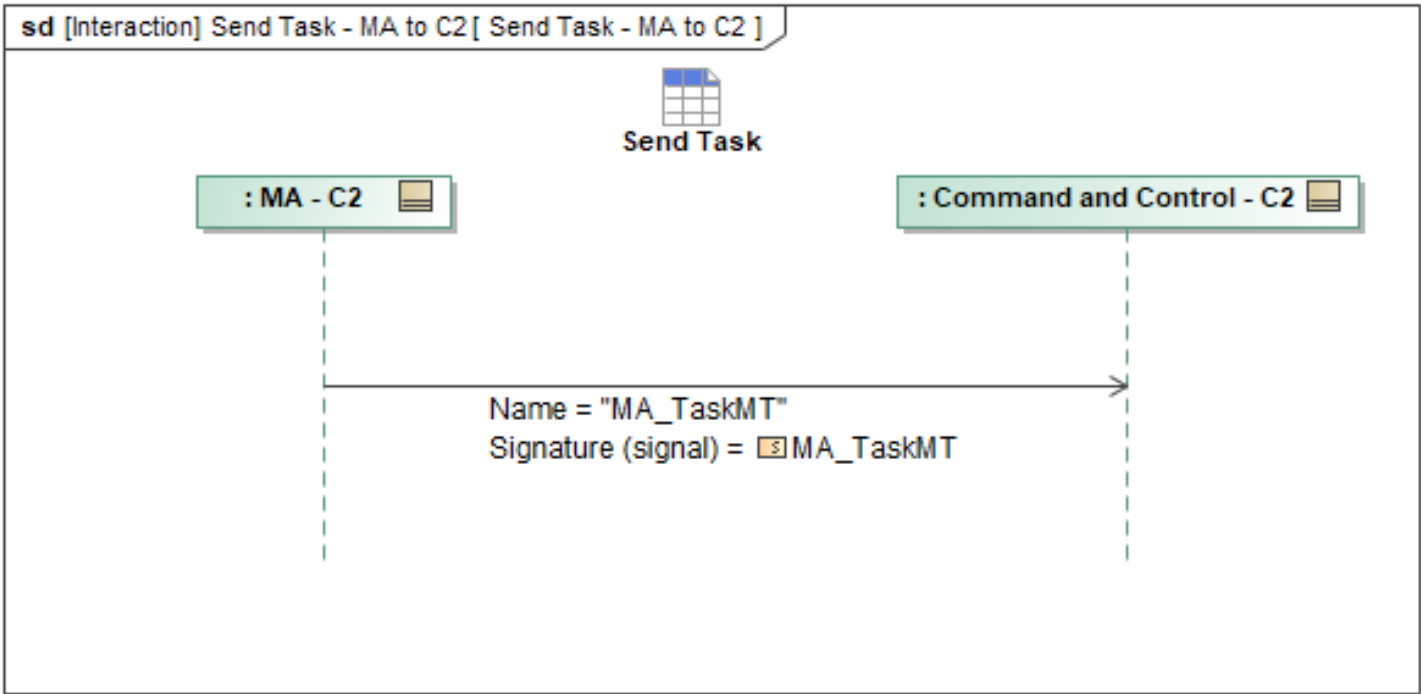


Figure 5-148: Send Task - MA to C2

Table 5-147: Send Task - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.8.23.21 Send Task Plan - MA to C2

This interaction has MA send *MA_TaskPlanMT* to C2.

If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*. When sending an update to revise existing data, *ObjectState* is set to *UPDATED*. Each message is populated with the definition to detail the data being conveyed.

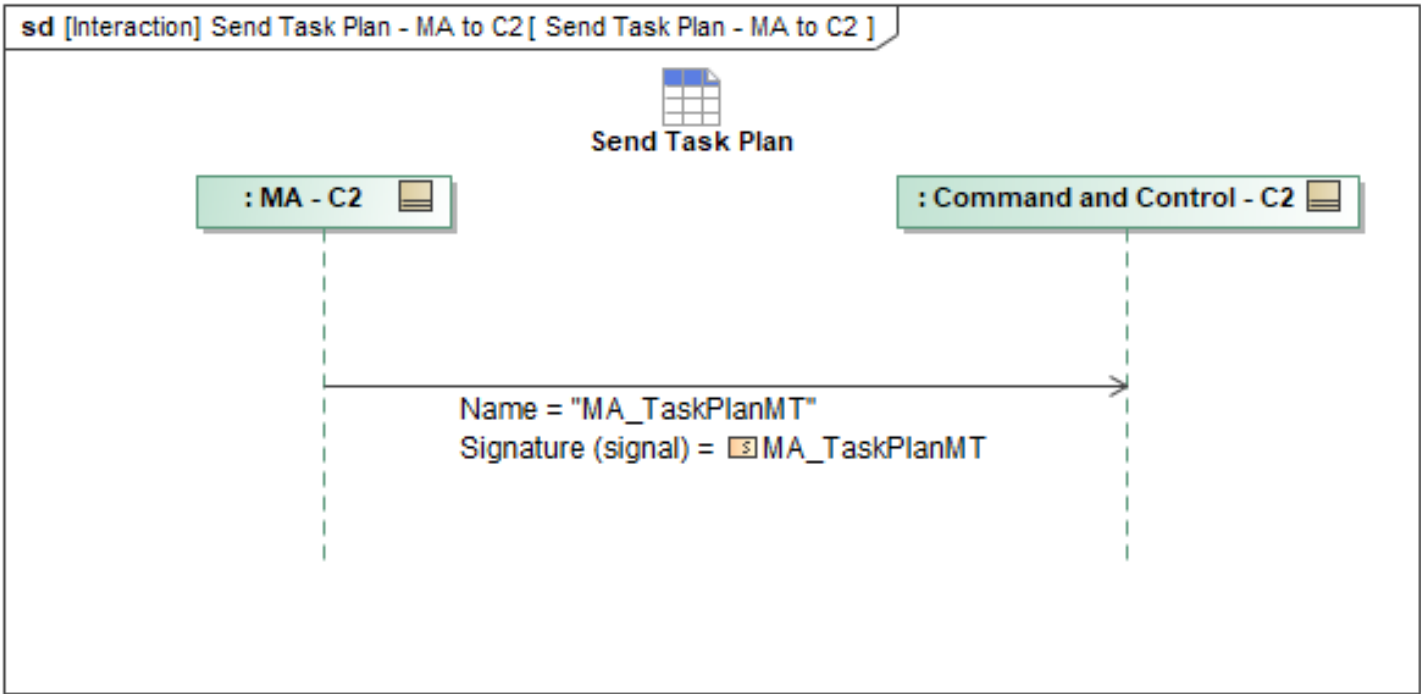


Figure 5-149: Send Task Plan - MA to C2

Table 5-148: Send Task Plan - MA to C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskPlanMT: MA_TaskPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9 C2 RBAC Behaviors Package

Overview

The A-GRA defines a configurable role-based access control (RBAC) paradigm. RBAC defines C2 user roles which will have a configurable set of permissions allocated to them. These permissions will govern what messages a C2 user with a role is authorized to send. These roles are defined during mission planning using *MA_OperatorRoleMT*. If needed, RBAC allows for user roles to be generated during Mission Execution by authorized C2 users. It is imperative that any new roles are broadcast on the network so all players are aware of new user roles and their permissions. This message will be the source of truth for user roles, and messages that use an operator role ID will use the operator role IDs present in the *MA_OperatorRoleMTs* defined for the mission. To define the set of permissions a C2 user role has over Mission Autonomy, the *MA_AuthorizationMT* message is used. In order to route approval requests to C2 users with specific roles, Mission Planners and authorized C2 users will use *MA_ApprovalPolicyMT* to configure what types of autonomous actions require an approval, and *MA_ApprovalAuthorityMT* to configure which C2 role is authorized to accept or reject the requested action.

The A-GRA defines a set of five initial roles, with example permissions that can be further customized. The "Admin" RBAC role is characterized by full authorization to send messages to

mission autonomy. In addition to having full permissions, the **"Admin"** role is the only role that can update the permissions of other roles during a mission. It is recommended that the **Admin** have full control over Mission Autonomy meaning that nothing is unauthorized. The **"QB"** role also has full authorization to command mission autonomy. The key difference between the **"Admin"** and **"QB"** roles is the ability to update the permissions and configuration of other roles. The **"Autonomous Vehicle Controller"** role (**AVC**) is authorized to perform flight tasks but has restrictions on specific weapon capabilities by default. The **AVC** role is designed to be highly configurable to meet mission needs. While all A-GRA roles are configurable, the **AVC** role serves as a generalized C2 role from which more specific roles will be derived. The Role Configuration section provides further details on configuring existing roles or generating new ones. The **"Launch and Recovery"** (**LRE**) role specifies permissions for takeoff and landing procedures. The **"Observer"** role is characterized by being unauthorized to send messages to mission autonomy; it is a read-only role.

When a C2 user starts a C2 application, they log in to the C2 system using their credentials. This authenticates the user and allows the system to retrieve the role data associated with that user. The C2 system (HMI/PVI) then configures itself to present only the options available for that RBAC role. Upon initial connection to the network, Mission Systems authenticates the C2 system and establishes secure communication. Once this process is complete, the C2 user is authorized to send messages to Mission Autonomy according to the permissions allocated to their role. If a C2 user needs to change their role during a mission, they will need to resend the **Authorize** sequence of messages declaring the new role. Alternatively, the **Admin** C2 user can change the permissions of the user's current C2 role, or create a new role to meet the specific needs of the mission.

It should also be noted that RBAC is not intended to implement a hierarchy of C2 systems to deconflict commanded tasks. Instead, it is expected that Mission Autonomy will always execute the last received command for prioritizing commands from C2 systems.

5.2.9.1 Admin Create Role

An authorized C2 that has declared as an admin may create an entirely new role not contained within the pre-mission planned RBAC configuration. It is assumed that the C2 admin will have the most up-to-date *MA_OperatorRoleMT*, *MA_AuthorizationMT*, *MA_ApprovalPolicyMT*, and *MA_ApprovalAuthorityMT* RBAC configuration in order to create a new role and configure it. If the C2 admin does not have the current configuration, they may query for them via the *Query Operator Roles* and *Query Role Permissions* interactions. To create a new role, the C2 admin sends *MA_OperatorRoleMT* with a unique *OperatorRoleID* representing the new role. The C2 admin is then required to configure the new role with an existing set of permissions or with new permissions via the *Admin Update MA Role Request Permissions* and *Admin Update Role Permissions* interactions after sharing the new role to MA. To complete the update, MA shares the updated list of roles to all connected authorized C2 via the *Controller Update Broadcast* interaction. MA will retain the updated *MA_OperatorRoleMT* along with *MA_ApprovalAuthorityMT*, *MA_ApprovalPolicyMT*, and *MA_AuthorizationMT* to share to any newly connected C2.

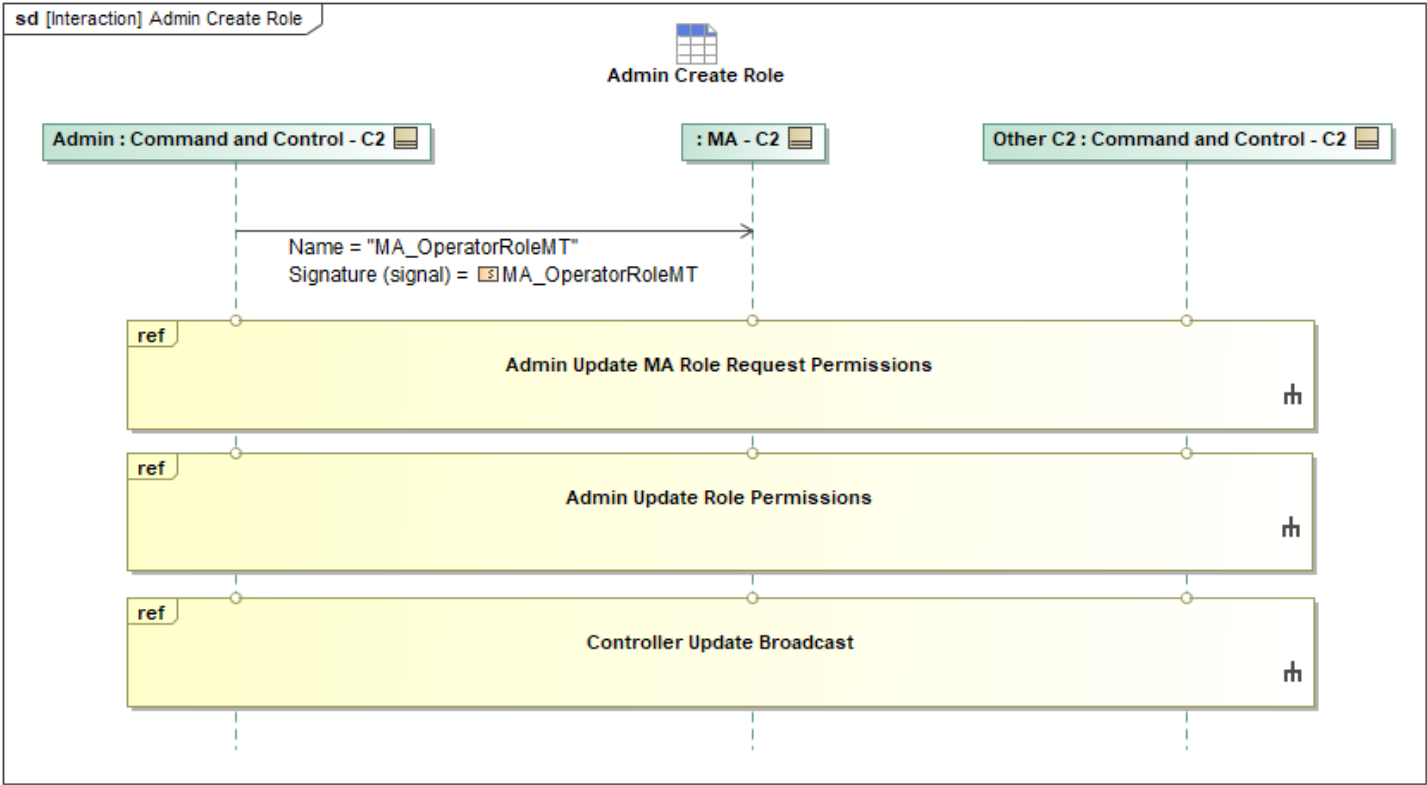


Figure 5-150: Admin Create Role

Table 5-149: Admin Create Role

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.2 Admin Update MA Role Request Permissions

An authorized C2 that has declared as an admin may update the individual pre-planned approval request permissions, the blanket approval request response permissions, and/or a group of approval request permissions for a given role during Mission Execution. These approval request permissions define the set of message types MA will request approval for in addition to which roles are permitted to respond to the request. It is assumed that the C2 admin will have the most up-to-date *MA_OperatorRoleMT*, *MA_AuthorizationMT*, *MA_ApprovalPolicyMT*, and *MA_ApprovalAuthorityMT* RBAC configuration in order to update them. If the C2 admin does not have the current configuration, they may query for them via the *Query Operator Roles* and *Query Role Permissions* interactions. The C2 admin will update the specific approval request permissions for a C2 role by sending an updated *MA_ApprovalPolicyMT* containing the complete set of configured approval request permissions for a role to MA. The C2 admin may also update the set of approval request permissions a role has by sending an updated *MA_ApprovalAuthorityMT*.

To update approval request permissions for a given role, the C2 admin would update the previously configured *MA_ApprovalPolicyMT* linked to that role's *OperatorRoleID* via *MA_ApprovalAuthorityMT*.

To configure specific approval request permissions, C2 admin would add a new requirement or update an existing requirement specified within the *RequirementExecution.ByRequirementPolicy.Requirement.RequirementTypes* field and update *RequirementExecution.ByRequirementPolicy.Policy*. The C2 admin would then send the updated *MA_ApprovalPolicyMT* to MA. To update blanket policies for a given role, the C2 admin may similarly configure *RequirementExecution.DefaultReponse* before sending the updated *MA_ApprovalPolicyMT* to MA. Setting a default response to approval request permissions for a role will automatically approve or reject any request sent to that role if it was not already defined in *RequirementExecution.ByRequirementPolicy.Requirement.RequirementTypes*.

To add an existing set of approval request permissions to a role, the C2 admin would send an updated *MA_ApprovalAuthorityMT* linking a role's *OperatorRoleID* to the existing *ApprovalPolicyID*. Similarly, to remove an existing set of approval request permissions that a role already has, the C2 admin would send an updated *MA_ApprovalAuthorityMT* removing the role's *OperatorID* from the existing *ApprovalPolicyID*.

If a set of approval request permissions does not exist, the C2 admin may create a new *MA_ApprovalPolicyMT* with a unique *ApprovalPolicyID*. The C2 admin would then update *MA_ApprovalAuthorityMT* with the new *ApprovalPolicyID* and the *OperatorRoleID* of the role receiving the new set of request permissions. The C2 admin would send both the new *MA_ApprovalPolicyMT* and updated *MA_ApprovalAuthorityMT* to finalize the permission update.

To complete the update MA will share the updated *MA_ApprovalPolicyMT* and/or *MA_ApprovalAuthorityMT* to all other C2 connected. MA will retain the updated *MA_ApprovalPolicyMT* and/or *MA_ApprovalAuthorityMT* to share to any newly connected C2.

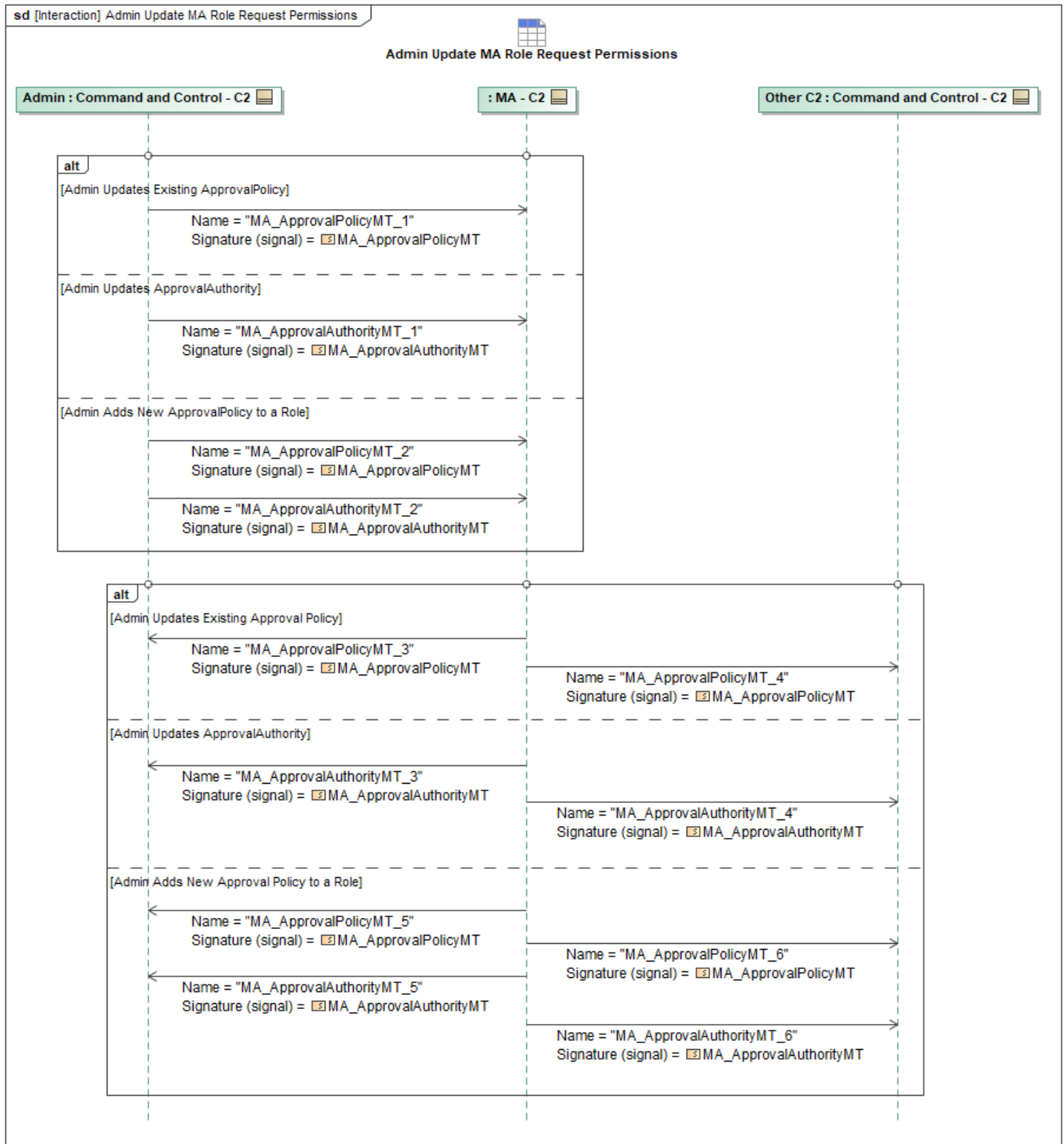


Figure 5-151: Admin Update MA Role Request Permissions

Table 5-150: Admin Update MA Role Request Permissions

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalAuthorityMT_1: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalAuthorityMT_2: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalAuthorityMT_3: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalAuthorityMT_4: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalAuthorityMT_5: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalAuthorityMT_6: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalPolicyMT_1: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_ApprovalPolicyMT_2: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_ApprovalPolicyMT_3: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_ApprovalPolicyMT_4: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_ApprovalPolicyMT_5: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_ApprovalPolicyMT_6: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.3 Admin Update Role Permissions

This interaction describes the updating of permissions for roles and the set of messages a role is permitted or not permitted to send. For updating the permissions for specific MA-generated messages that require approval and from which roles, see the Admin Update MA Role Request Permissions interaction. An authorized C2 that has declared as an admin may update the individual pre-planned permissions, the blanket permissions, and/or a group of permissions for a given role during Mission Execution via the *MA_AuthorizationMT* message. These role permissions define the set of message types that a role is either authorized or unauthorized to send via the *AuthorizedExceptFor* and *UnauthorizedExceptFor* fields provided under each C2 message type field. Exceptions to the blanket authorized/unauthorized messages may be specified under each message type field. For more information about *MA_AuthorizationMT*, refer to the C2 RBAC Behaviors Package description.

It is assumed that the C2 admin will have the most up-to-date *MA_OperatorRoleMT*, *MA_AuthorizationMT*, *MA_ApprovalPolicyMT*, and *MA_ApprovalAuthorityMT* RBAC configuration in order to update them. If the C2 admin does not have the current configuration, they may query for

them via the *Query Operator Roles* and *Query Role Permissions* interactions. The C2 admin will update specific permissions for a C2 role by sending an updated *MA_AuthorizationMT* containing the complete set of configured permissions for a role to MA. The C2 admin is required to define the default permissions for each message type field within *MA_AuthorizationMT* with the *AuthorizedExceptFor* or *UnauthorizedExceptFor* permissions type. Specific exceptions may be optionally specified underneath the chosen permissions type for each message type. If a set of permissions does not exist for a role (for example, if a new role was created that needs to be configured), the C2 admin may create a new *MA_AuthorizationMT*. To associate the permissions to a new role, the C2 admin will populate *OperatorRole* field under *UserIdentifier* with same *UUID* as that of the new role.

To complete the update MA will share the updated *MA_AuthorizationMT* to all other C2 connected. MA will retain the updated *MA_AuthorizationMT* to share to any newly connected C2.

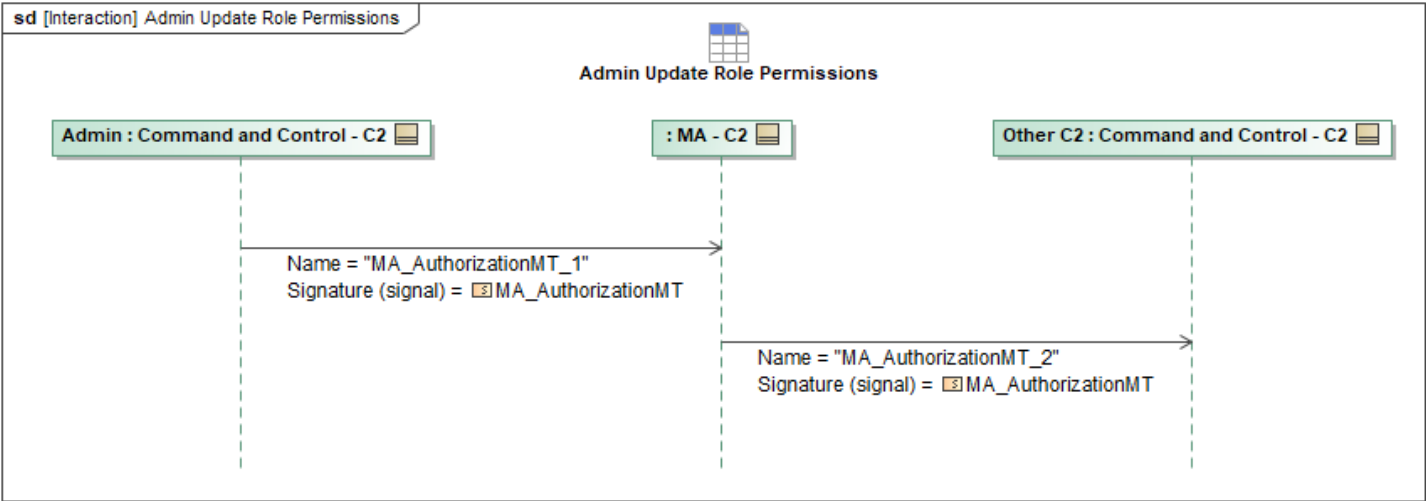


Figure 5-152: Admin Update Role Permissions

Table 5-151: Admin Update Role Permissions

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AuthorizationMT_1: MA_AuthorizationMT	ByRequirementExecution: MA_RequirementAuthorizationType PackageAuthorization: boolean ByPlatformSettingsModification: MA_SettingsModificationAuthorizationType ByPlanActivation: MA_PlanAuthorizationType ByDataActivation: MA_DataActivationAuthorizationType ByDataModification: MA_DataModificationAuthorizationType ByOperatorRequest: MA_OperatorRequestAuthorizationType OperatorRoleID: OperatorRoleID_Type OperatorRoleID: OperatorRoleID_Type PackageAuthorization: boolean
MA_AuthorizationMT_2: MA_AuthorizationMT	ByRequirementExecution: MA_RequirementAuthorizationType ByPlanActivation: MA_PlanAuthorizationType

Message Name (Name:Type)	Required Fields (Name:Type)
	ByPlatformSettingsModification: MA_SettingsModificationAuthorizationType ByDataActivation: MA_DataActivationAuthorizationType PackageAuthorization: boolean OperatorRoleID: OperatorRoleID_Type ByOperatorRequest: MA_OperatorRequestAuthorizationType ByDataModification: MA_DataModificationAuthorizationType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.4 Authorize

After authentication, C2 systems will declare their role to Mission Autonomy using *OperatorMT*. The field *OperatorID* will be filled out with the C2 System's Operator *UUID* and the *DescriptiveLabel* with the operator's callsign. The *OperatorStatus* will be set to *AVAILABLE* to indicate that this C2 system can receive requests from Mission Autonomy. The *OperatorRoleID* will be populated with an *OperatorRole UUID* that is associated with a pre-planned operator role (discussed in section C2 RBAC Behaviors Package). This *OperatorMT* message is used by mission autonomy to keep a mapping of *SystemID*'s or *OperatorID*'s to Operator Roles.

Once Mission Autonomy receives an *OperatorID*, it will advertise which C2 systems have authorized themselves as described in the Controller Update Broadcast sequence by sending an *MA_OperatorRoleMT* with the newly declared C2 system appended to the list in the *SystemID* field. The *MA_OperatorRoleMT* will also include the *UUID* of the updated role in the *OperatorRoleID* field.

If a C2 system wishes to change its declared role during a mission, it can send another *OperatorMT* message to repeat this sequence and update its role as described in the Update Operator Roles section.

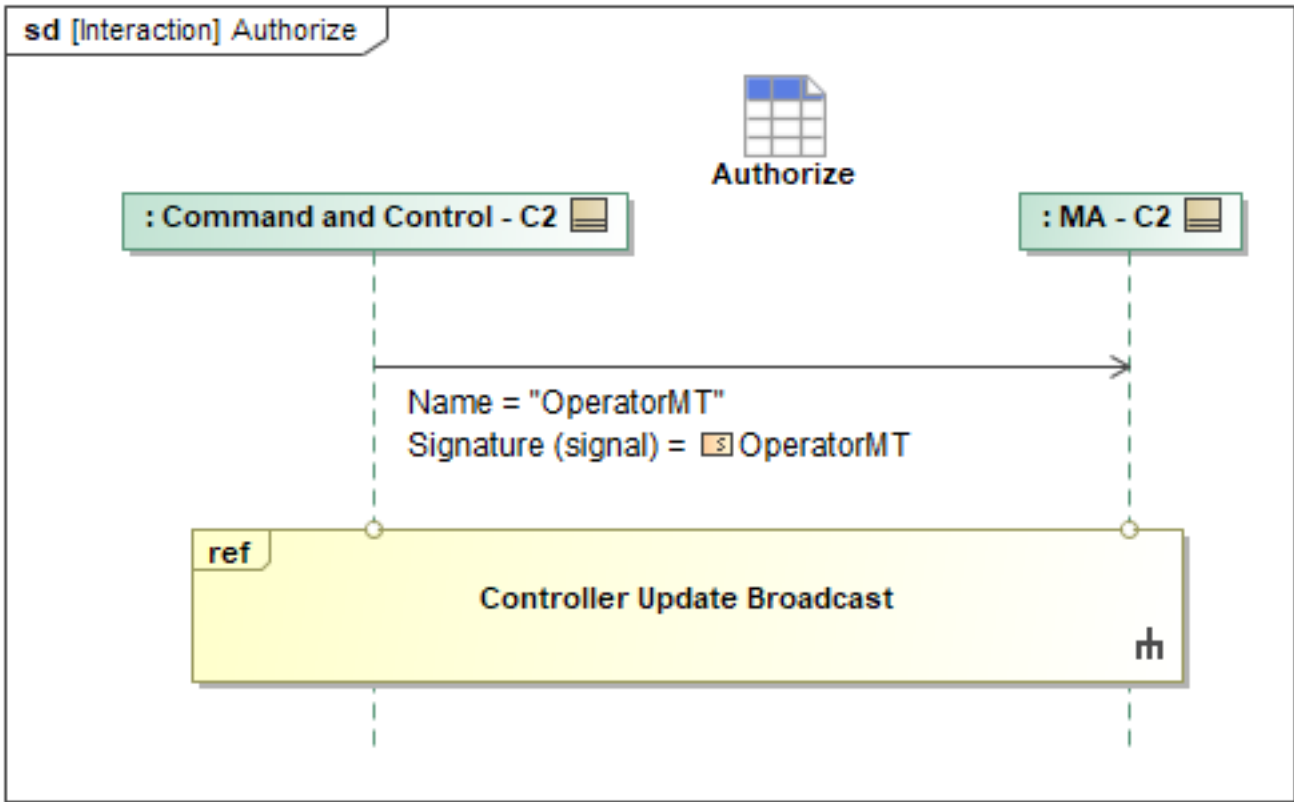


Figure 5-153: Authorize

Table 5-152: Authorize

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorMT: OperatorMT	OperatorRoleID: OperatorRoleID_Type DescriptiveLabel: VisibleString256Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.5 C2 Unauthorized Response

After a message is determined to be unauthorized, MA will send a response message to C2 based on UCI Primitive Type:

- For Data-1 and DataRecord-1 types, a *MA_SystemNotificationMT* message will be sent.
- For the Command-2 types, a *Command-2 Status Message* will be sent.
- For the ActionRequest-2 type, *ActionRequest-2 Status Message* will be sent.
- For the DataRequest-2 type, *DataRequest-2 Status Message* will be sent.

The Command-2, ActionRequest-2 and DataRequest-2 messages are each further specified by a corresponding table (Unauthorized Command-2 Message Pairs , Unauthorized ActionRequest-2 Message Pairs and Unauthorized DataRequest-2 Message Pairs, respectively) in the Model. Each table contains a triggering message, the corresponding unauthorization response message, and the

applicable interactions.

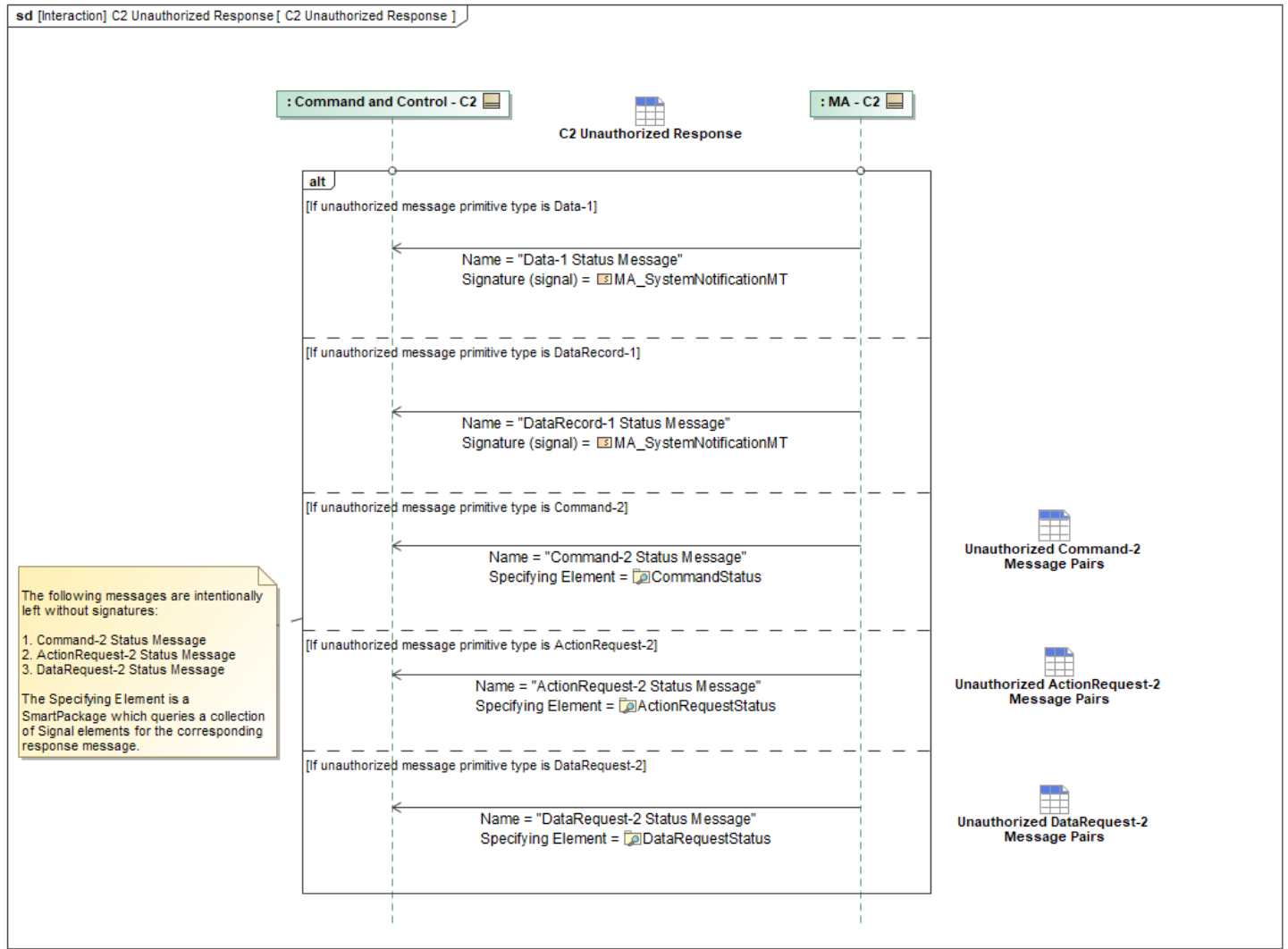


Figure 5-154: C2 Unauthorized Response

Table 5-153: C2 Unauthorized Response

Message Name (Name:Type)	Required Fields (Name:Type)
ActionRequest-2 Status Message:	RequestProcessingStateReason: CannotComplyType
Command-2 Status Message:	CommandProcessingStateReason: CannotComplyType
Data-1 Status Message: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType
DataRecord-1 Status Message: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType
DataRequest-2 Status Message:	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.6 Controller Update Broadcast

Whenever Mission Autonomy makes updates to the field *SystemID* in *MA_OperatorRoleMT*, it will broadcast its updated *MA_OperatorRoleMT* to other C2 systems. In addition to the *SystemID*, MA will also broadcast operators' callsigns in the *DescriptiveLabel* field for each *SystemID*. See Section C2 RBAC Behaviors Package for more information on what control is permitted per role.

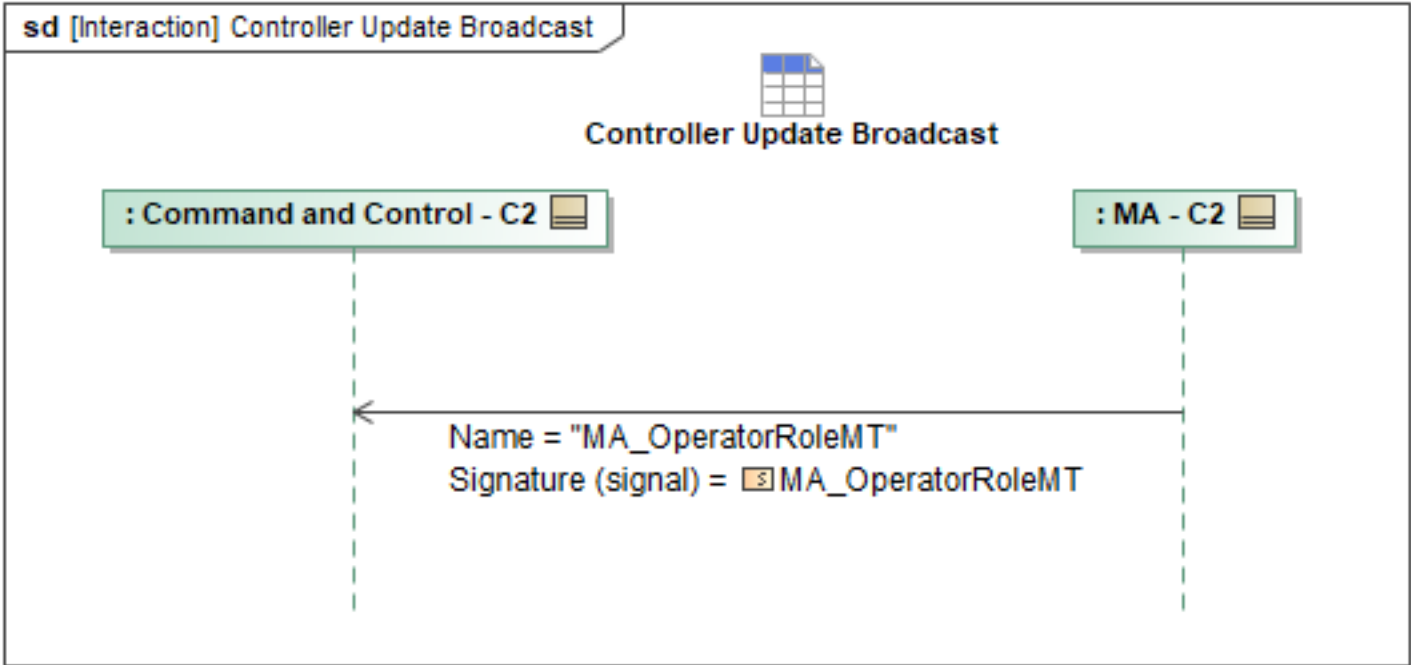


Figure 5-155: Controller Update Broadcast

Table 5-154: Controller Update Broadcast

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	SystemID: SystemID_Type DescriptiveLabel: VisibleString256Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.7 Query Operator Roles

A C2 system can query the authorized operators and their roles from MA via *QueryDataRequestMT*. *MA_OperatorRoleMT* contains the list of authorized operators, their roles, and their callsigns and are stored or cached in data manager services which may have a database backend. This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate *RequestProcessingState* such as “QUEUED” and “PROCESSING” to indicate additional in-process states. The *QueryDataRequestStatusMT* message with a *RequestProcessingState* set to “COMPLETED” and *ResultsInNative* field set to “FALSE” contains the requested data. When *ResultsInNative* is set to “TRUE”, this causes the underlying data management services to publish the requested data in its original or raw form.

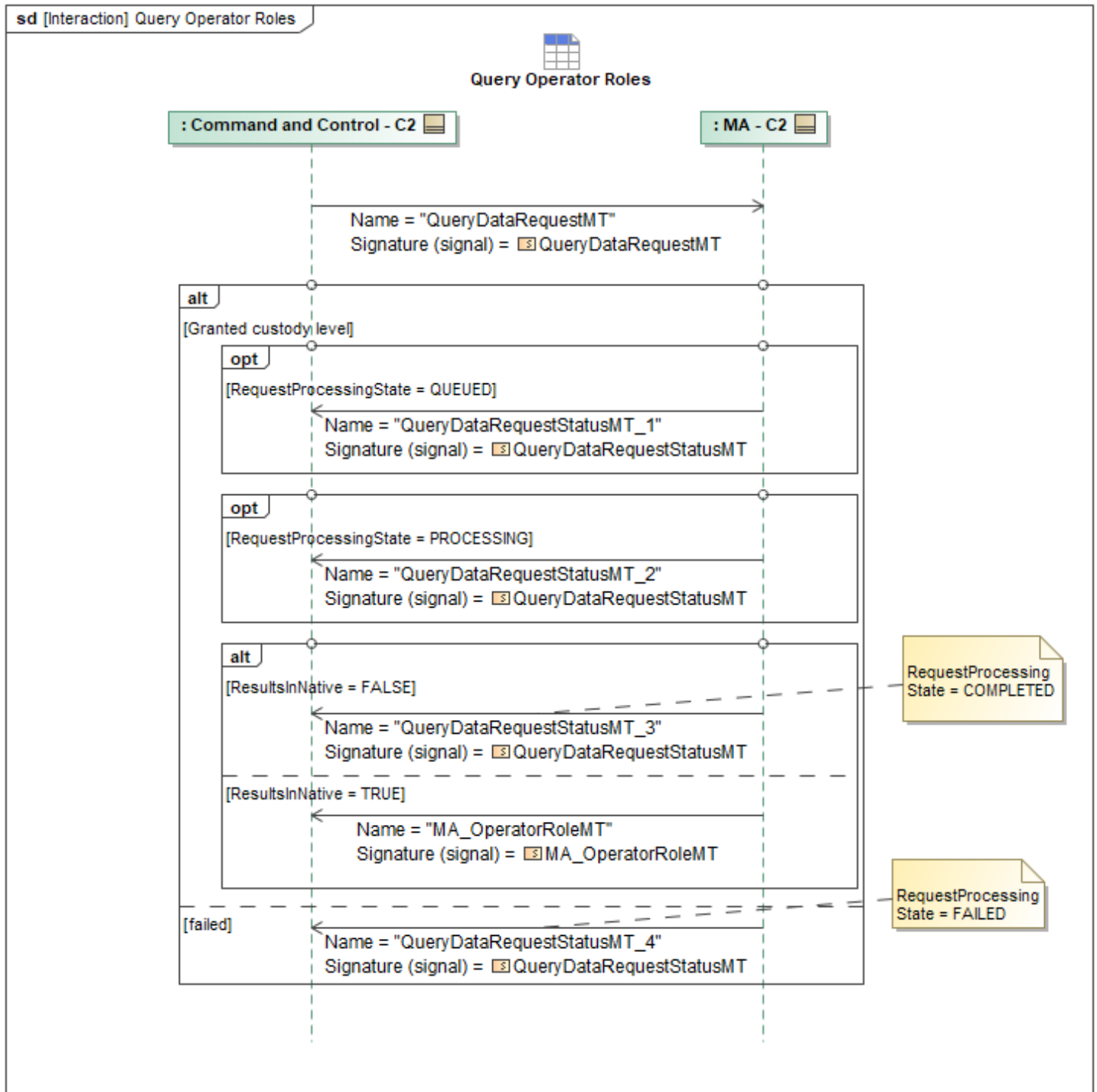


Figure 5-156: Query Operator Roles

Table 5-155: Query Operator Roles

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	DescriptiveLabel: VisibleString256Type SystemID: SystemID_Type
QueryDataRequestMT: QueryDataRequestMT	ResultsInNative: QueryTopicPairType

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	RequestProcessingState: RequestProcessingStateEnum
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	RequestProcessingState: RequestProcessingStateEnum
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	RequestProcessingState: RequestProcessingStateEnum
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingState: RequestProcessingStateEnum RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.8 Query Role Permissions

A C2 admin may query for the permissions configuration in order update them for a given role. Other authorized roles may query for them for situational awareness. A C2 system may query for the complete RBAC role permissions configuration as well as the approval request permissions configuration via *QueryDataRequestMT*. The C2 has the option to query for specific permissions messages for all RBAC roles, or for the permissions messages for a specific role by using that role's *OperatorRoleID UUID*. *MA_AuthorizationMT* contains the set of permissions for each role. *MA_ApprovalPolicyMT* contains the set of permissions that require approval when generated by MA and is used in conjunction with *MA_ApprovalAuthorityMT* to define which roles MA should route the approval request to. C2 may query for any combination of these messages from MA. These RBAC permission messages are stored or cached in data manager services which may have a database backend. This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate *RequestProcessingState* such as “*QUEUED*” and “*PROCESSING*” to indicate additional in-process states. The *QueryDataRequestStatusMT* message with a *RequestProcessingState* set to “*COMPLETED*” and *ResultsInNative* field set to “*FALSE*” contains the requested data. When *ResultsInNative* is set to “*TRUE*”, this causes the underlying data management services to publish the requested data in its original or raw form. If *QueryDataRequestMT* does not specify an *OperatorRoleID UUID* via *QueryPET*, MA will share the requested permissions messages for all roles defined within *MA_OperatorRoleMT*. If an *OperatorRoleID UUID* is specified in *QueryPET*, MA will respond with the requested permissions messages for only that role.

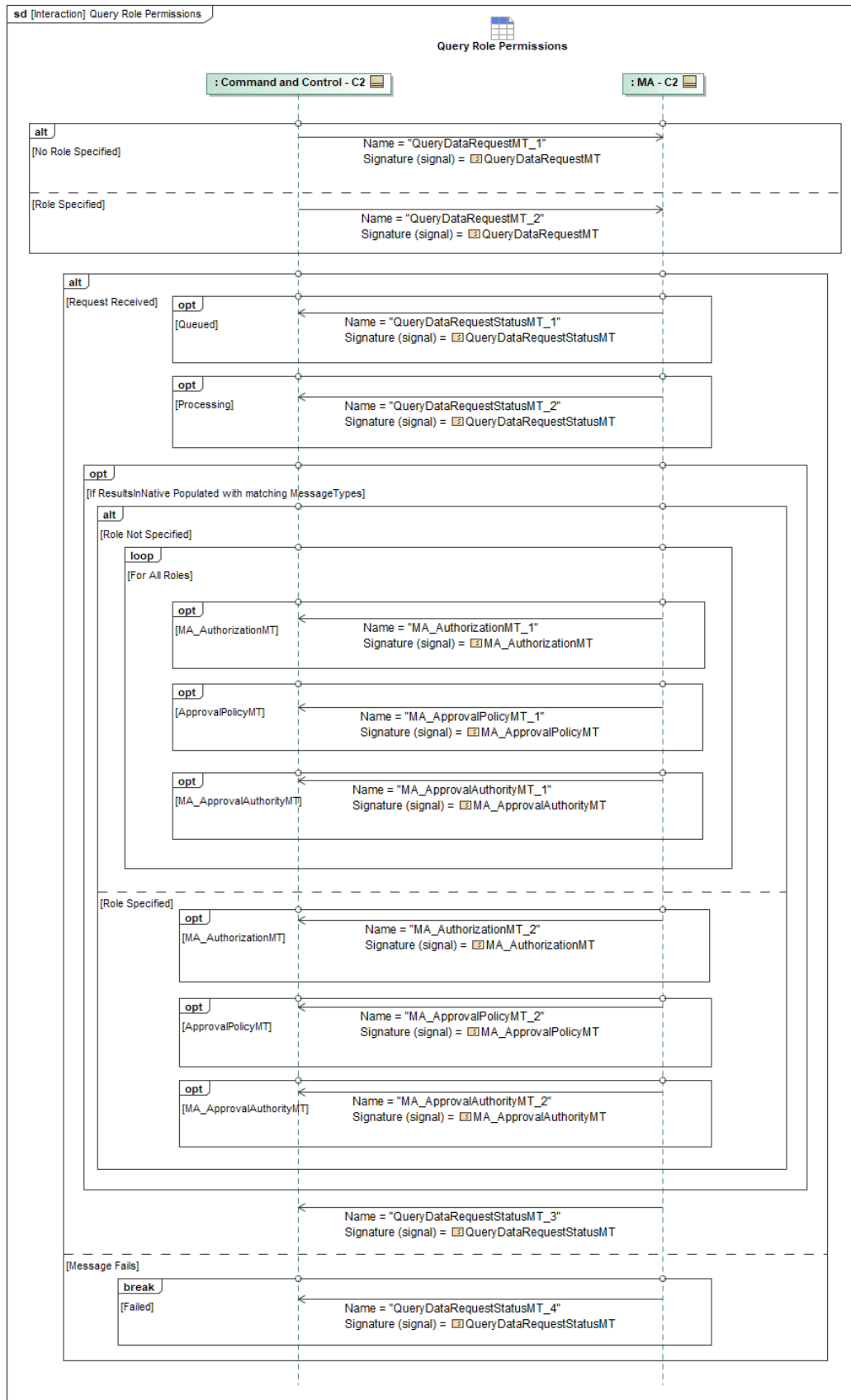


Figure 5-157: Query Role Permissions

Table 5-156: Query Role Permissions

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalAuthorityMT_1: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalAuthorityMT_2: MA_ApprovalAuthorityMT	SystemID: SystemID_Type
MA_ApprovalPolicyMT_1: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_ApprovalPolicyMT_2: MA_ApprovalPolicyMT	RequirementExecution: RequirementExecutionPolicyType
MA_AuthorizationMT_1: MA_AuthorizationMT	ByDataActivation: MA_DataActivationAuthorizationType ByPlanActivation: MA_PlanAuthorizationType OperatorRoleID: OperatorRoleID_Type ByDataModification: MA_DataModificationAuthorizationType PackageAuthorization: boolean ByPlatformSettingsModification: MA_SettingsModificationAuthorizationType ByRequirementExecution: MA_RequirementAuthorizationType ByOperatorRequest: MA_OperatorRequestAuthorizationType PackageAuthorization: boolean OperatorRoleID: OperatorRoleID_Type
MA_AuthorizationMT_2: MA_AuthorizationMT	ByPlanActivation: MA_PlanAuthorizationType OperatorRoleID: OperatorRoleID_Type ByDataModification: MA_DataModificationAuthorizationType ByDataActivation: MA_DataActivationAuthorizationType PackageAuthorization: boolean ByRequirementExecution: MA_RequirementAuthorizationType ByOperatorRequest: MA_OperatorRequestAuthorizationType ByPlatformSettingsModification: MA_SettingsModificationAuthorizationType PackageAuthorization: boolean OperatorRoleID: OperatorRoleID_Type
QueryDataRequestMT_1: QueryDataRequestMT	None
QueryDataRequestMT_2: QueryDataRequestMT	Query: QueryPET
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.9.9 RBAC Construct

Table 5-157: RBAC Roles

Roles	Controls
Admin	A C2 system with the Admin role will have full permissions to control mission autonomy. An Admin C2 system will have additional permissions to edit the existing mission plan to change role configuration during mission execution. This means that an Admin C2 system can update an existing role that needs new permissions, or create a totally new role. In the case that a new role is created, any user can switch to the new role via <i>MA_OperatorRoleMT</i> . This allows RBAC to remain flexible as a mission evolves. These updated roles will be sent out across the network to update all instances of C2 and Mission Autonomy.
QB	The QB Role is meant for C2 Users that have the most tactical awareness. These users have full authorization to command Mission Autonomy, with the exception of role configuration data.
AVC	This role is meant to be the most general of the C2 roles. It allows full flight and sensor permissions but limits weapon capability commands by default. It can be configured or used to create other roles as needed per mission. This is the "template" role. If desired, weapon capabilities can be configured for this role even if they are not authorized by default.
LRE	This role is intended to serve launch and recovery functions.
Observer	A C2 system with the Observer role will have no permissions to control mission autonomy. It is purely a listener to Mission Autonomy's data. This Observer C2 is not authorized to send any messages to mission autonomy, and mission autonomy will reject any messages that were sent from an observer C2.

*note for capability permissions, it is assumed that if a *CapabilityTypeEnum* is applicable for a permission, it applies to all enum values that are included in that capability data type. For example, the **AVC** role is authorized to command "*FLIGHT*" capabilities. This means that all *FlightCapabilityEnum* values are authorized. If permissions need to be subdivided further within a specific capability, the *CapabilityTaxonomyType* field can be used.

Role Configuration

In addition to the default roles, the A-GRA defines a process for how to create and configure roles based on a set of UCI configuration messages. In this way, mission planning and Admin C2 systems can ensure that RBAC will always fit the needs of the mission.

Roles are defined and configured through the use of four UCI messages:

MA_OperatorRoleMT: The *MA_OperatorRoleMT* is what defines a role. This is the message that other configuration messages will reference to define role permissions and request routing.

MA_ApprovalPolicyMT: The *MA_ApprovalPolicyMT* message defines what actions Mission Autonomy can take autonomously, and which require approval from C2 to execute the triggered autonomous action.

MA_ApprovalAuthorityMT: the *MA_ApprovalAuthorityMT* is what links *MA_ApprovalPolicyMT* to an approving authority. For RBAC, this message configures what roles Mission Autonomy will send its Approval Requests to.

MA_AuthorizationMT: The *MA_AuthorizationMT* message configures which roles are authorized to

task Mission Autonomy with certain message types.

MA_OperatorRoleMT

The *MA_OperatorRoleMT* message defines the user role. This message contains an *OperatorRoleID*, which serves as a unique identifier for the role throughout the mission. Agreement on this ID across all network nodes is required. The *Role* field is intended to reference an external database; however, with Role-Based Access Control (RBAC), the role's permissions are integrated directly into UCI messages. Consequently, the *Role* field's subfields, *Key* and *SystemName*, will be set to empty strings, as they are required by the schema but not utilized by Mission Autonomy. During mission planning, the *SystemID* field will remain empty, as no Command and Control (C2) system has yet declared itself as fulfilling this role. During mission execution, the *SystemID* field will be populated with an active list of C2 nodes that have declared themselves with this role. This list is advertised to other network nodes whenever it is updated.

MA_ApprovalPolicyMT

The *MA_ApprovalPolicyMT* is used to define which autonomous actions require approval. The *RequirementExecution* and *PlanActivation* fields enable the mission plan or an **Admin** C2 system to associate specific UCI requirements or plan types with a UCI Policy. This *Policy* will be set to "AUTO_APPROVE" if the UCI requirement or plan is authorized to be executed autonomously and "AUTO_REJECT" if the UCI requirement is not authorized to be executed autonomously. If executing this requirement or plan is authorized, but requires C2 approval to execute, then *Policy* will be set to "REQUIRES_APPROVAL". The *RequirementExecution* field includes the subfield "DefaultResponse," which allows the *MA_ApprovalPolicy* to specify a default enum (either approve or reject). Exceptions to this default policy can then be supplied through the *ByRequirementPolicy* field. This approach allows for a comprehensive role definition without explicitly associating every UCI requirement enumeration with a Policy.

MA_ApprovalAuthorityMT

The *MA_ApprovalAuthorityMT* message establishes the link between the *OperatorRoleID* and the *ApprovalPolicyID*, letting Mission Autonomy route approval messages generated from a policy to a specific role. The *OperatorRoleID* is specified in the *SystemApprovalPolicy.Approver.ApproverRole.OperatorRoleID* field, while the *ApprovalPolicyID* is specified in the *SystemApprovalPolicy.ApprovalPolicyID* field. The A-GRA expects each role to have a single *MA_ApprovalAuthorityMT* message defining its Approval Policies. This design facilitates minimal updates if a permission needs to be modified for a role during mission execution.

MA_AuthorizationMT

The *MA_AuthorizationMT* message is used to configure the set of permissions allowed to each C2 Operator Role. The *AuthorizationID* will be set with a *UUID*. The *UserIdentifier* choice type will be filled with the *OperatorRoleID* of the role being configured. The *SystemId* field will be set to the *SystemId* of Mission Autonomy instance that this configuration file is for. The *Precedence* field will be set to zero as it is unused by RBAC, but is required by the schema. The fields *ByPlanActivation*, *ByRequirementExecution*, *ByDataModification*, *ByDataActivation*, *ByOperatorRequest*, *ByPackageAuthorization* and *ByPlatformSettingsModification*, and will be configured to set permissions for each role. The following fields are required fields for RBAC.

The *ByPlanActivation* field authorizes the activation of mission plans with the *ByMissionPlan* field, and authorization to activate specific plan types by their enum with the *ByPlanParts* field. Note that

this field does not authorize updating or modifying a mission plan or plan parts, only their activation. For example, if a role was authorized to activate a *"RETURN_TO_BASE"* plan, it would have permission to command the activation of a Route Plan with the type *"RETURN_TO_BASE"*. However, this field does not authorize a C2 role to modify the route plan it is activating. That is a separate permission handled by the *ByDataModification* field.

The *ByRequirementExecution* field authorizes the C2 Operator Role to execute *Effects*, *Actions*, *Tasks*, *Capabilities* and *Responses* by their Enum type. This field similarly only gives permissions to command the execution of a UCI requirement, and does not include its modification. For example, if a role was configured to have permission to execute *"KINETIC_ATTACK"* type actions, then they could command any existing *MA_ActionMTs* with the type of *"KINETIC_ATTACK"* with *MA_ActionCommandMT*. This permission does not give the C2 user role authorization to modify the *MA_ActionMT* that it is activating. That is a separate permission handled by the *ByDataModification* field.

The *ByDataModification* field is used to specify what types of data this C2 user role is allowed to Create, Modify, and Delete. Note that this field does not include activating or deactivating data type messages like *MA_OpZone* or *MA_EngagementCriteriaEngagement*, that is handled by the *ByDataActivation* field.

The *ByDataActivation* field controls which types of data messages can be activated by a C2 user role, broken out by message type. An example of this is authorization to command *MA_EngagementCriteriaCommand*. Note that this field does not imply permission to modify these messages, just activate them.

The *ByOperatorRequest* field is used to indicate which types of Requests the C2 role is authorized to task Mission Autonomy with. An example of this is the *AssessmentRequestMT* message.

The *PackageAuthorization* field configures if this role is authorized to create new teams, modify existing teams and dissolve teams.

The *BySettingsCommandAuthorization* field gives the C2 user role permissions to modify the existing settings on Mission Autonomy. This allows for changing default settings on the different capabilities and services that Mission Autonomy has access to.

Role Definition

In order to define a role, Mission Autonomy will have an *MA_OperatorRoleMT* message with a *OperatorRoleID* that corresponds to the role. The *UUID* field will be set to the unique ID for that role. The *DescriptiveLabel* field may also be used to include a human readable text string to indicate the role associated with this *UUID*, but this is not required.

MA_ApprovalAuthorityMT will have a unique *ApprovalAuthorityID*. The *ApplicableSystemPackageID* choice type field will select *"SystemID"* and set the *UUID* to this instance of Mission Autonomy. The *Approver.ApproverRole.OperatorRoleID* field should be set to the *UUID* of the role defined in the corresponding *MA_OperatorRoleMT* message. The *RequiredApproval* field should be set to false if C2 nodes with this role should not receive requests from Mission Autonomy (like the observer role). The *ApprovalPolicyID* will be set to the ID of the applicable policy for the role being configured.

MA_ApprovalPolicyMT will have a unique *ApprovalPolicyID*. For the fields *Plans*, *PlanActivation*, and *RequirementExecution* the *Policy* field should be set to *"REQUIRES_APPROVAL"* if it is required that a certain autonomous action (specified in the sibling fields of the policy), needs approval before it can be executed. Further details on how to configure Approval Policies are given in the section

Respond to Approval Request.

The next section will cover how to configure default *MA_AuthorizationMT*'s for each of the default A-
GRA role permission sets.

Observer: The *AuthorizationID* will be set to a *UUID*. The *UserIdentifier* choice type will be set to *OperatorRole*, with the ID set to the *UUID* defined in the *MA_OperatorRoleMT* for the **Observer** role. This tells Mission Autonomy that this is the set of permissions authorized for the **Observer** Role. The *Precedence* field will be set to "0" as it is not utilized by RBAC, but is required by the schema. Each of the permission fields will chose "*UnauthorizedExceptFor*" from the choice type, and leave the exceptions list empty. The RBAC Permissions Table above outlines how the **Observer** role should be configured.

LRE: The *AuthorizationID* will be set to a *UUID*. The *UserIdentifier* choice type will be set to *OperatorRole*, with the ID set to the *UUID* defined in the *MA_OperatorRoleMT* for the **LRE** role. This tells Mission Autonomy that this is the set of permissions authorized for the **LRE** Role. The *Precedence* field will be set to "0" as it is not utilized by RBAC, but is required by the schema. The rest of the message will be configured in accordance with the RBAC permission table above.

AVC: The *AuthorizationID* will be set to a *UUID*. The *UserIdentifier* choice type will be set to *OperatorRole*, with the ID set to the *UUID* defined in the *MA_OperatorRoleMT* for the **AVC** role. This tells Mission Autonomy that this is the set of permissions authorized for the **AVC** Role. The *Precedence* field will be set to "0" as it is not utilized by RBAC, but is required by the schema. The rest of the message will be configured in accordance with the RBAC permission table above. If new roles need to be created, the new roles should be extended from the **AVC** role.

QB: The *AuthorizationID* will be set to a *UUID*. The *UserIdentifier* choice type will be set to *OperatorRole*, with the ID set to the *UUID* defined in the *MA_OperatorRoleMT* for the **QB** role. This tells Mission Autonomy that this is the set of permissions authorized for the **QB** Role. The *Precedence* field will be set to "0" as it is not utilized by RBAC, but is required by the schema. The rest of the message will be configured in accordance with the RBAC permission table above. The **QB** role will also be the approver for all strike requests generated by Mission Autonomy. To configure this, there will be an Approval Policy with the field *RequirementExecution.ByRequirementPolicy.Policy* set to "*REQUIRES_APPROVAL*" and the sibling *Requirement* field should have *Requirement.Requirement.Task* set to "*STRIKE*".

Admin: The *AuthorizationID* will be set to a *UUID*. The *UserIdentifier* choice type will be set to *OperatorRole*, with the ID set to the *UUID* defined in the *MA_OperatorRoleMT* for the **Admin** role. This tells Mission Autonomy that this is the set of permissions authorized for the **Admin** Role. The *Precedence* field will be set to "0" as it is not utilized by RBAC, but is required by the schema. The rest of the message will be configured in accordance with the RBAC permission table above.

If Mission Autonomy generates a plan, or triggers a response that causes it to activate an existing plan, The *ApprovalRequirementEnum* value of "*REQUEST_APPROVAL*" can be used to route an approval request to the correct role. For example, if mission autonomy reaches bingo fuel, it might try to activate an "*RETURN_TO_BASE*" Route Plan. If it was desired that **QB's** needed to approve activating an RTB plan, then the **QB's** *MA_ApprovalPolicyMT* would have the *PlanActivation.ByPlanParts.RoutePlan* set to the "*RETURN_TO_BASE*" enumeration. This would indicate that anytime Mission Autonomy tried to activate a "*RETURN_TO_BASE*" route plan that wasn't directly commanded, it should get an approval from a C2 system with the **QB** role. More details on sending and receiving approval requests can be found in the section **Respond to**

Approval Request. It is important to note that in the context of RBAC, the "*APPROVAL_REQUIRED*" *ApprovalRequirementEnum* is only used for plan activation commands that originate from inside Mission Autonomy and were not directly commanded by C2. If Mission Autonomy receives a command from C2, it should either execute the command or reject it - it should not ask C2 for permissions to execute.

5.2.10 C2 Sensor/Vehicle Tasking Behaviors Package

5.2.10.1 Request Approval for Task Allocation Plan

This interaction allows a lead to receive approval for an Action Plan if the *ApprovalPolicy* calls for C2 to approve *ActionPlans* of this type. MA sends the plans using the "Send Action Plan - MA to C2" sequence. Optionally, MA can send additional information associated with the *Plans* using the "Synchronize Assessment" sequence. Then the "Respond to Approval Request" sequence will be started by sending an *ApprovalRequestMT* with *ApprovalItem.PlanApproval* set to the ID of the Plan being approved. If approved, the MA may then execute the Plan if activation has been commanded or otherwise triggered.

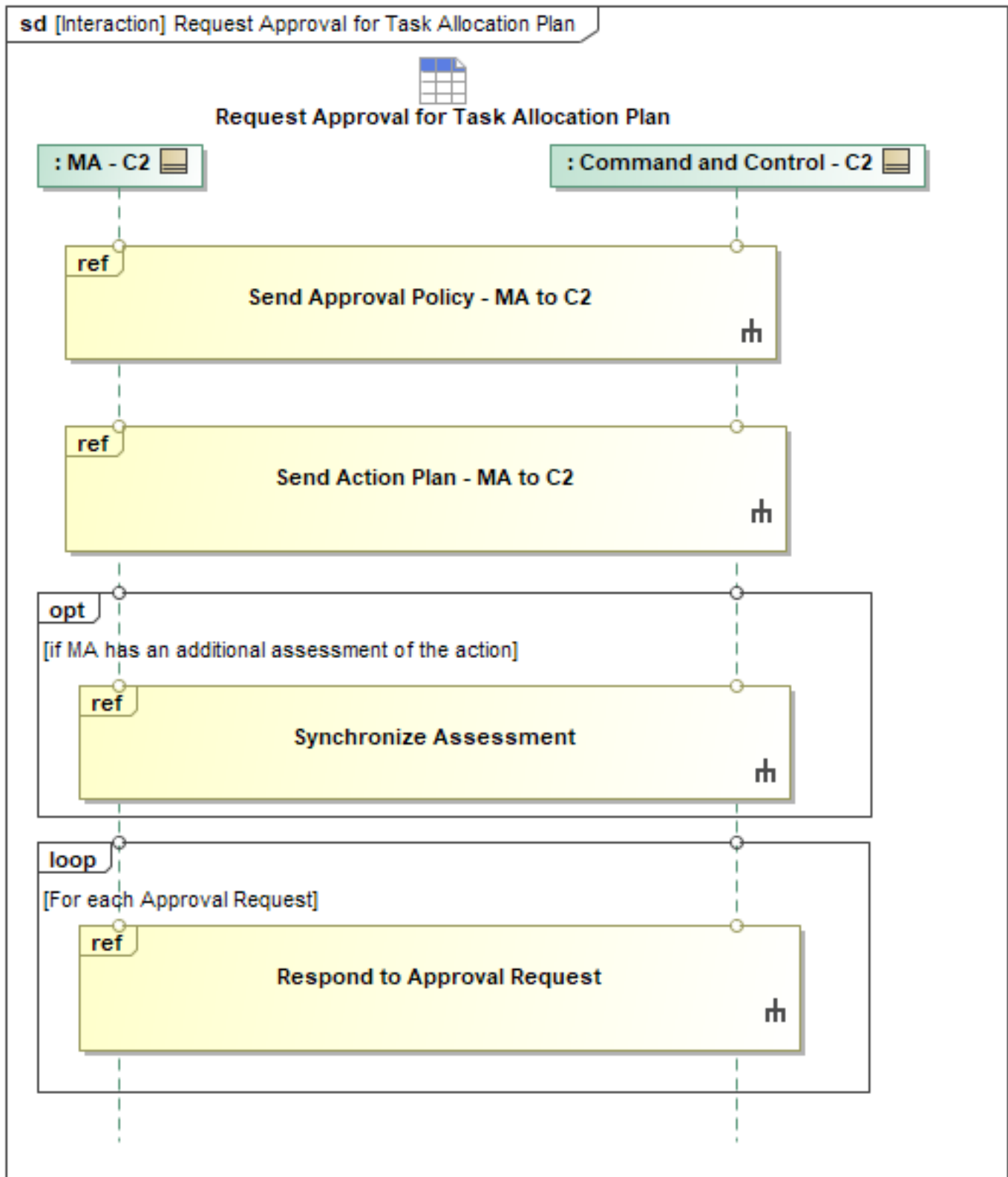


Figure 5-158: Request Approval for Task Allocation Plan

5.2.10.2 Synchronize Assessment

This interaction allows an ACP to send an Assessment to C2. The Assessment can be either initiated by MA or by C2.

To initiate the Assessment from C2, C2 sends a *MA_AssessmentRequestMT*. The required parameters depend on the type of assessment being requested. Optionally MA can send a *MA_AssessmentRequestStatusMT* when the request is *QUEUED* or *PROCESSING*. If the *MA_AssessmentRequestMT* can't be processed, MA responds with a *MA_AssessmentRequestStatusMT* with *RequestProcessingState* == *FAILED*. If the *AssessmentRequest* processing is completed, MA sends a *MA_AssessmentRequestStatusMT* with *RequestProcessingState* == *COMPLETED*. If the *ResultsInAssessmentMessage* field in the *MA_AssessmentRequestMT* is set to *FALSE*, then MA will include the Assessment in the *MA_AssessmentRequestStatusMT*. Otherwise MA will send a separate *MA_AssessmentMT*. To initiate the Assessment from MA, send *MA_AssessmentMT* to C2. The parameters in the Assessment should match the parameters that the ACP used to create the Assessment.

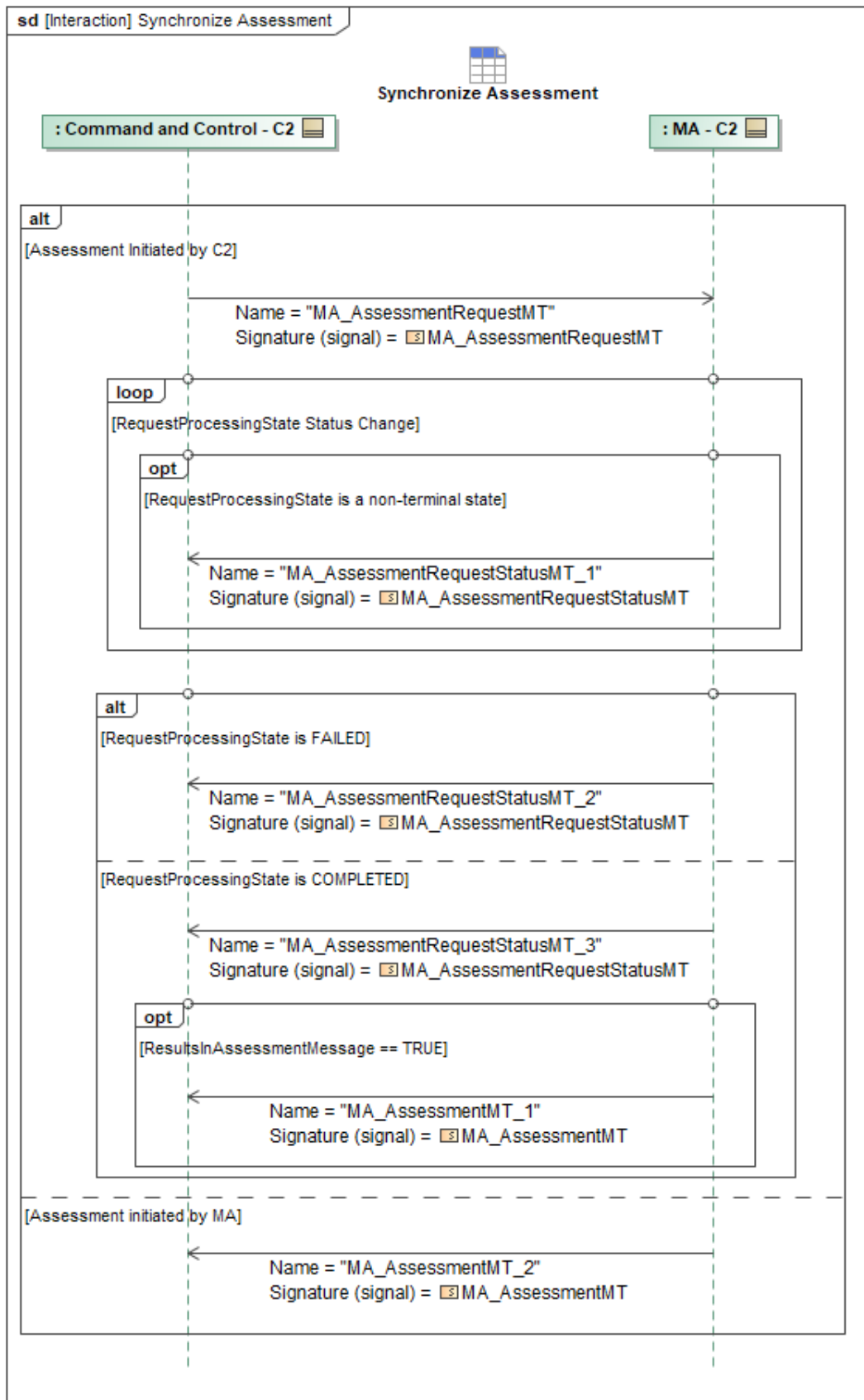


Figure 5-159: Synchronize Assessment

Table 5-158: Synchronize Assessment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentMT_1: MA_AssessmentMT	None
MA_AssessmentMT_2: MA_AssessmentMT	None
MA_AssessmentRequestMT: MA_AssessmentRequestMT	None
MA_AssessmentRequestStatusMT_1: MA_AssessmentRequestStatusMT	None
MA_AssessmentRequestStatusMT_2: MA_AssessmentRequestStatusMT	None
MA_AssessmentRequestStatusMT_3: MA_AssessmentRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11 C2 Tactic Behaviors Package

5.2.11.1 Advertise Tactic Capabilities

MA sends *MA_ActionCapabilityMT* periodically to advertise the Tactic capabilities of the platform, as well as the Tactics the platform is currently capable of performing based on system/subsystem status and available stores when MA sends *ActionCapabilityStatusMT* periodically.

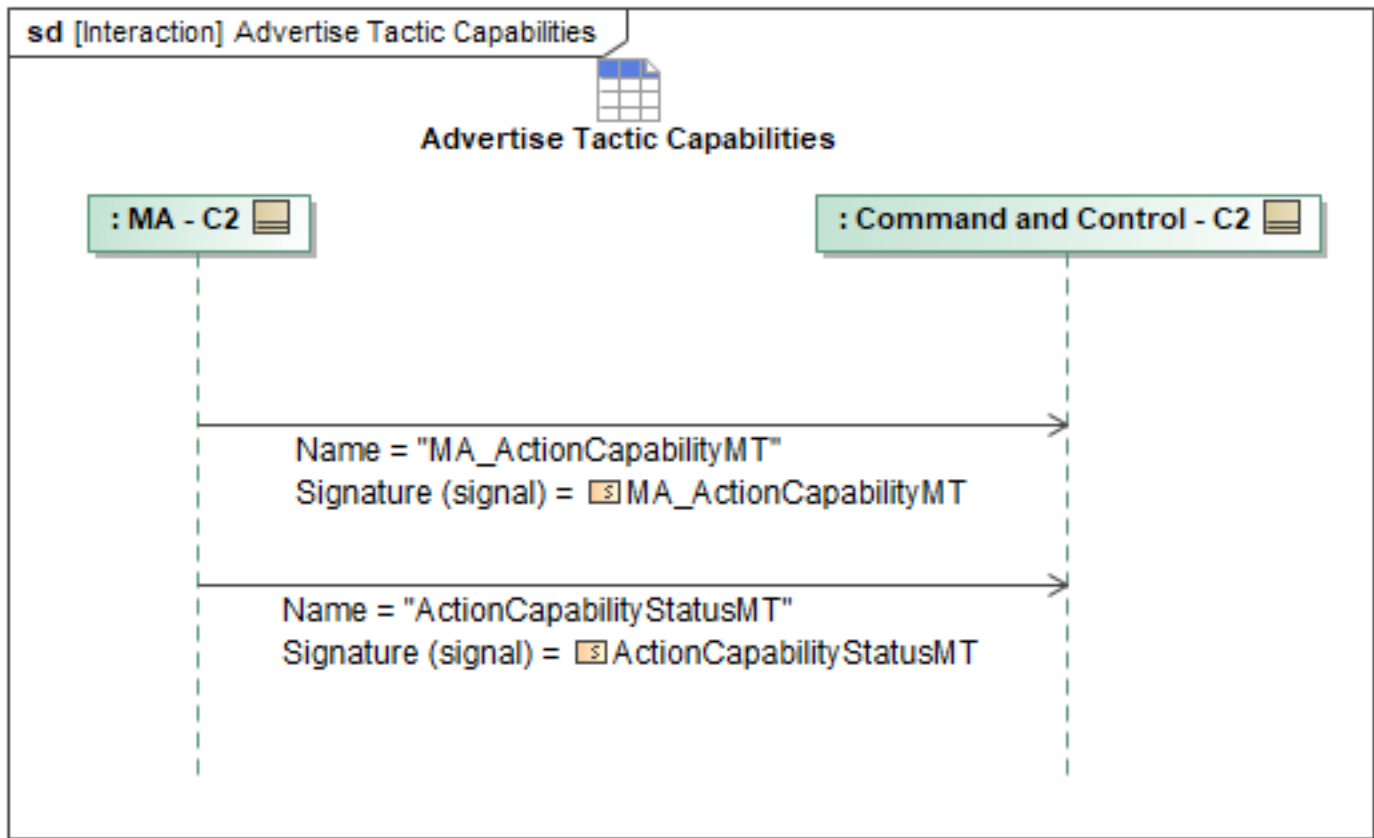


Figure 5-160: Advertise Tactical Capabilities

Table 5-159: Advertise Tactical Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCapabilityStatusMT: ActionCapabilityStatusMT	None
MA_ActionCapabilityMT: MA_ActionCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.2 Cancel Tactic

C2 can command MA to cancel a pending Tactic (Action), or to cancel an already executing Tactic when possible with *ActionCancelCommandMT*, to which MA will reply with a corresponding *ActionCancelCommandStatusMT_1*.

Canceling a Tactic should cause the platform to revert to a pre-defined fallback state (ex: return-to-base or returning to the original PlannedC2 mission plan) unless a new set of instructions is commanded.

In the case that the *ActionCancelCommandMT* is rejected, the returned *ActionCancelCommandStatusMT_2* message with *CommandProcessingState* = *REJECTED* must also include the *CommandProcessingStateReason* field to include an explanation why the command

was rejected.

As the Tactic is canceled, MA will continue to update C2 on the Tactic's status (see: Receive Tactic Status behavior) until the Tactic and it's sub-Tasks have all been successfully canceled/aborted.

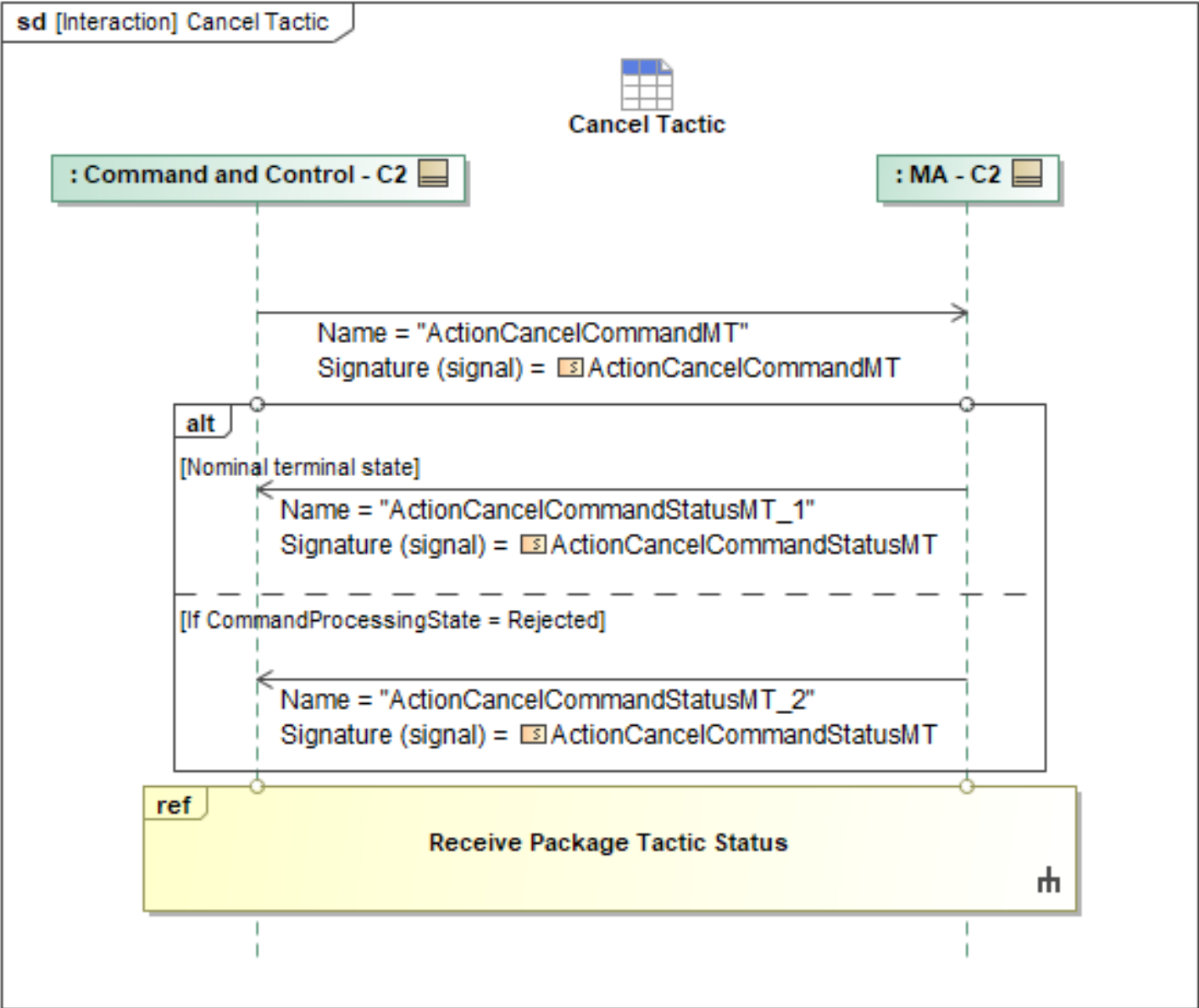


Figure 5-161: Cancel Tactic

Table 5-160: Cancel Tactic

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCancelCommandMT: ActionCancelCommandMT	None
ActionCancelCommandStatusMT_1: ActionCancelCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCancelCommandStatusMT_2: ActionCancelCommandStatusMT	CommandProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.3 Cold

The "Cold" command is a tactic that commands a single ship or package to cancel any active engagements and turn away from the fight. This is specified in the *MA_ActionMT* message under the *ActionType* field, using the *WITHDRAW MA_ActionTypeEnum* value. If updates to Cold Tactic Parameters are needed, the referenced sequence can be used to do so.

C2 will activate the "Cold" command by sending an *MA_ActionCommandMT* message to MA. The *MA_ActionCommandMT* message to start the tactic will only be accepted if MA is intercepting a target. MA will check if there is an ongoing tactic with a designated target. This would happen internally in MA, where MA would check if it were actively performing flight maneuvers towards a target. If MA is engaging a target, then the "Cold" command can be accepted.

MA will respond to C2 with a *MA_ActionCommandStatusMT* message. If the tactic is valid and MA is intercepting a track, the *MA_ActionCommandStatusMT_1* message will be used to confirm receipt of the command. If the message was rejected because MA was not intercepting a track, *MA_ActionCommandStatusMT_2* message will have the *CommandProcessingStateReason* field populated with the *INIT_CRITERIA_NOT_MET* value.

Once the command is accepted, MA will start sending the tactic status to C2. Any existing engagements will be canceled upon receipt of the *WITHDRAW* action command which will also be appropriately reported back to C2.

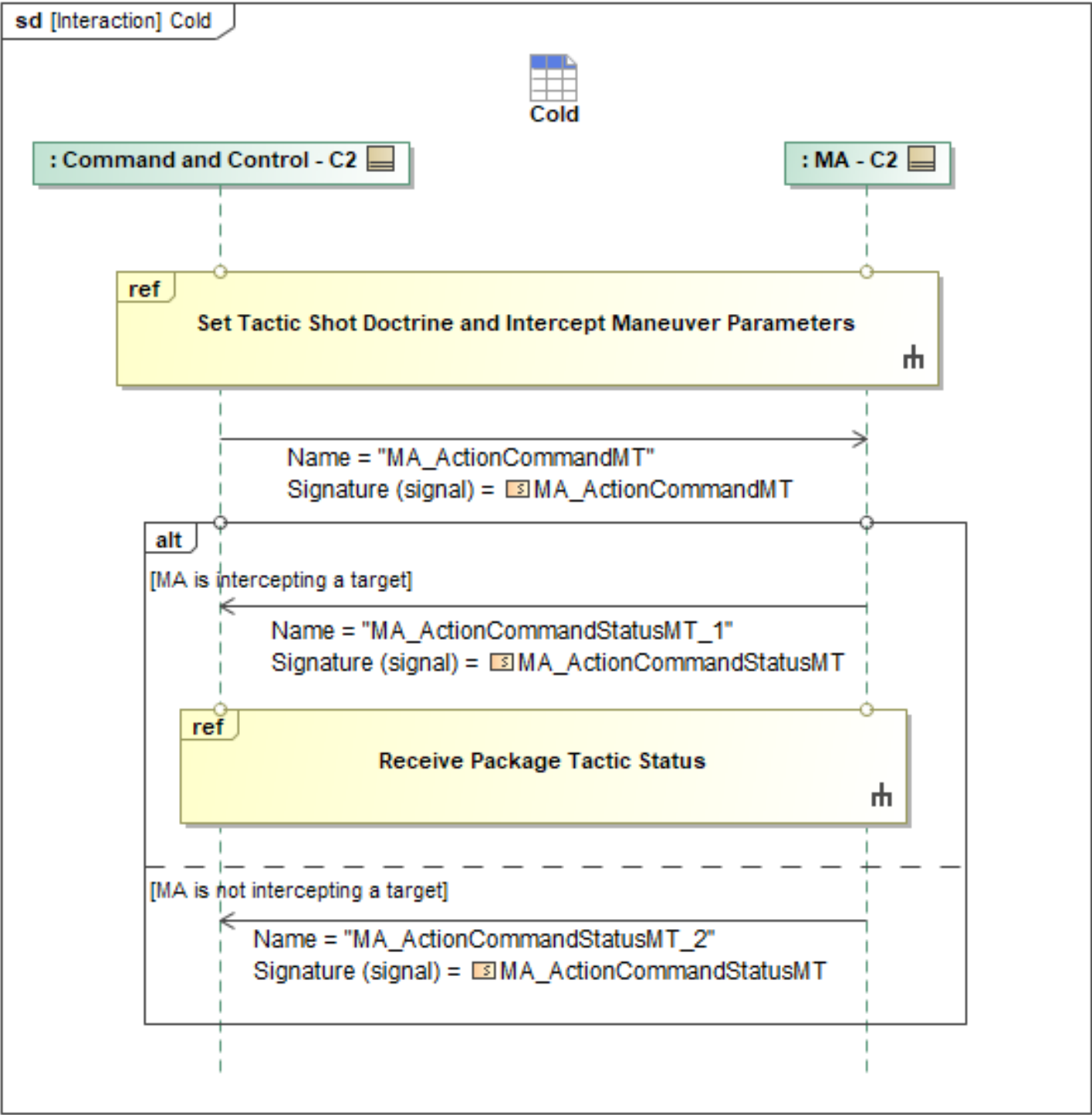


Figure 5-162: Cold

Table 5-161: Cold

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT: MA_ActionCommandMT	Capability: MA_ActionCapabilityCommandType ActionID: ActionID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandStatusMT_1: MA_ActionCommandStatusMT	None
MA_ActionCommandStatusMT_2: MA_ActionCommandStatusMT	CommandProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.4 Command Tactic

C2 will command all tactics, both to individual platforms, and packages, at the action level. MA will then be responsible for breaking down the action into tasks and/or activities. C2 will define the action type and tactical parameters by issuing *MA_ActionMT*. Then, C2 can command the vehicle to begin populating a Tactic Template and begin executing a Tactic by sending an *MA_ActionCommandMT* message to MA, which will then return *MA_ActionCommandStatusMT_1* messages to acknowledge receipt and confirm acceptance of the *MA_ActionCommandMT*.

Additionally, if the *MA_ActionCommand* was addressed to a *PackageID*, the associated *MA_ActionCommandStatus* should reflect the *Package ID*. If partial rejections occur during Lead-to-Follower delegation the Lead ACP must revert to individual messages to report multiple rejecting followers.

Once MA has accepted the command to begin the Tactic, MA will send *MA_ActionCommandStatusMT_2* as a response to C2 and it will then begin providing periodic status updates to C2 on the status of the Tactic via the "Receive Package Tactic Status behavior". The interaction is the same for individual systems providing a tactic status to C2. If C2 is authorized, but MA cannot accept the command, *MA_ActionCommandStatusMT_3* is the response.

If C2 allocates a requirement to a specific subset of platforms (via discrete *SystemID* allocations rather than *PackageID*), the Lead ACP shall bypass package-level aggregation and return individual status messages for those tasked members.

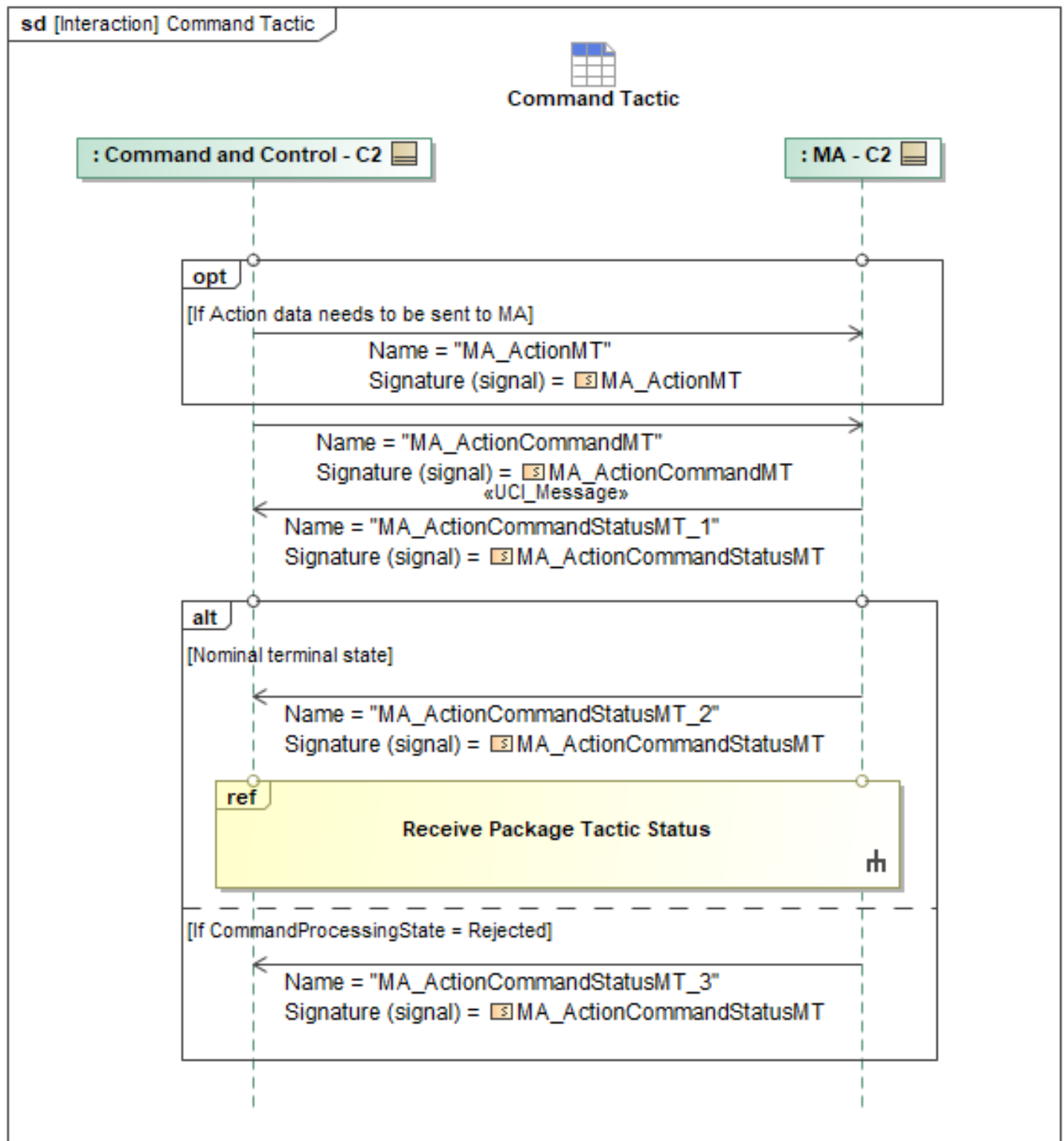


Figure 5-163: Command Tactic

Table 5-162: Command Tactic

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT: MA_ActionCommandMT	Capability: MA_ActionCapabilityCommandType
MA_ActionCommandStatusMT_1: MA_ActionCommandStatusMT	None
MA_ActionCommandStatusMT_2: MA_ActionCommandStatusMT	None
MA_ActionCommandStatusMT_3: MA_ActionCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_ActionMT: MA_ActionMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.5 Drop

The "Drop" command described in this C2 L1 interaction cancels any existing target pairings and shot allocations between ACPs and tracks. C2 can send the drop command and specify who it applies to via the *AppliesToIDs*. Within this field there are options to specify a *PackageID*, *SystemID*, or an *EmptyType* of "ALL" or "NON." The Drop of the existing "TARGET" Action will apply to all ACPs specified using that field. The Drop command is sent using the *MA_DesignationMT* UCI message. The *DesignationID* field is required to contain a UUID and represents this new designation of a dropped Entity. The *TypeOfDesignation* field should be set to "DESIGNATED_NO_TARGET" via the *DesignationEnum*. The *DesignationObjective* field should call out an *MA_ActionTypeEnum* of "TARGET". The *Target* field is where the *TargetType* is specified. This field can refer to an *EntityID*, or any other field contained within *TargetType* that should be dropped from ongoing or planned targeting actions. ACPs will acknowledge the "Drop" designation with an *MA_SystemNotificationMT*. If MA is positively acknowledging the *MA_DesignationMT* to drop track(s), and any ACPs were previously targeting and/or had shots allocated to the now-dropped tracks, ACPs will then send the *MA_ActionStatusMT* to report any ongoing "Target" activities as canceled. The UUID of the previously applicable *ActionID* corresponding to the "TARGET" *MA_ActionTypeEnum* within the "Target" command should be set to "CANCELED" so ACPs know to stop executing any target actions and designate selected tracks as non-targets.

When tracks are designated as non-targets, mission autonomy will trigger an action to cancel any existing target pairings and shot allocations. This update of targets uses the *RelationshipDesignationMT* message where the *Relationship* field contains an *EngagementStatus* field that contains an *ExternalCommandExecutionStateEnum*. This enum should be set to "CANCELED". The *RelationshipDesignationID* field contains a UUID indicating each individual unique relationship between a *Source* and a *Destination*. In this case of updating C2 with target pairings, or dropped tracks, the sources are ACPs represented by *SystemIDs* and the destinations are Entities represented by *EntityIDs*.

It is assumed that each ACP will then update any applicable *MA_StrikeActivityMTs* to remove each allocated shot, preventing it from firing, and therefore will report this activity by sending *MA_StrikeActivityMT* periodically. Messaging details are explained in the use cases "Receive Package Tactic Status" and "Report Tactic Status - Peer", which cover C2 and Peer to Peer

behaviors respectively. These use cases allow Peers to report the status of tactics back to a leader for the leader to then report the entire package's tactic status back to C2.

The *MA_SystemNotificationMT* message is the initial response that provides C2 an acknowledgment that the *MA_DesignationMT* message was received by MA successfully or not.

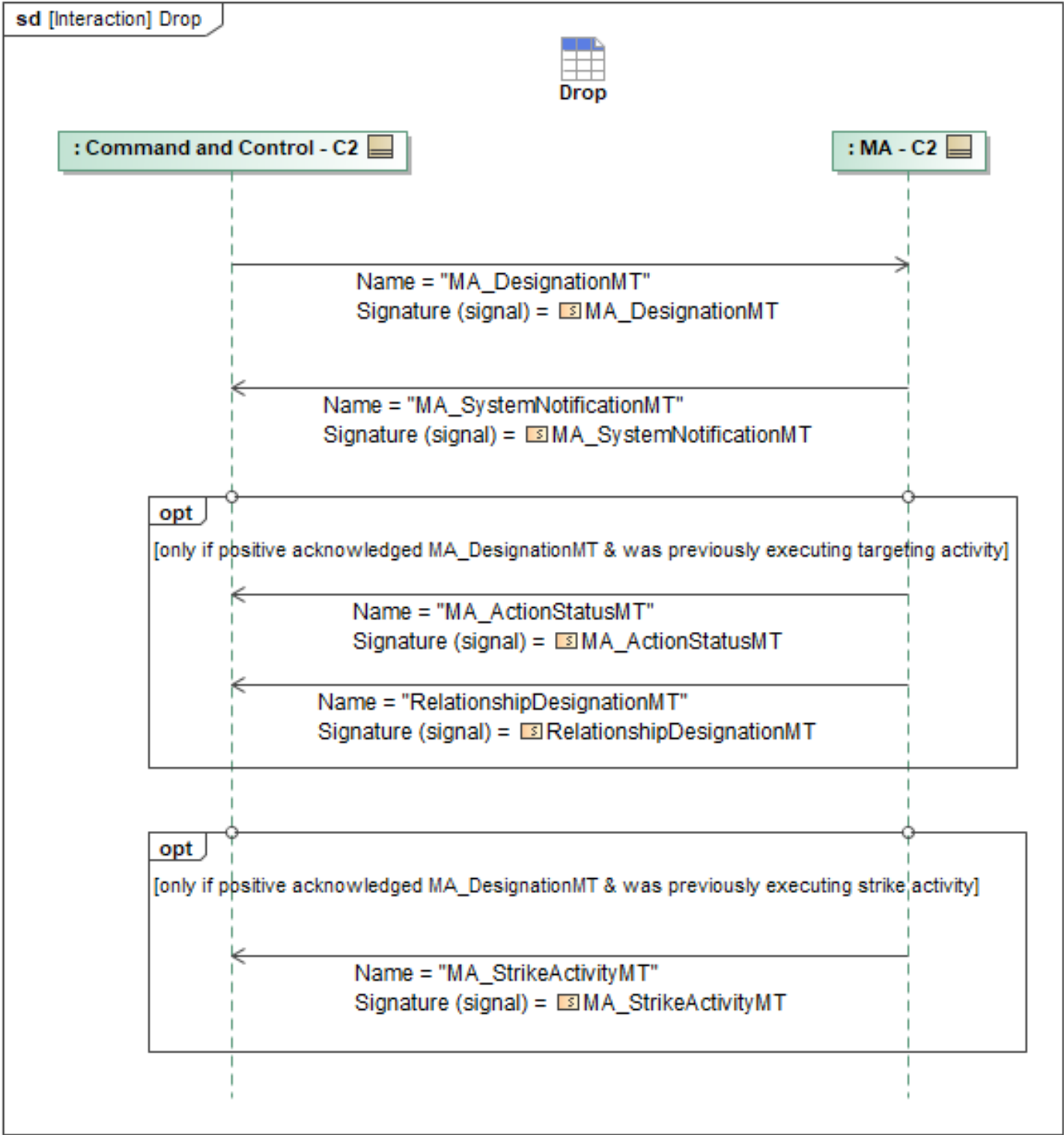


Figure 5-164: Drop

Table 5-163: Drop

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionStatusMT: MA_ActionStatusMT	SystemID: SystemID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationMT: MA_DesignationMT	DesignationType: DesignationEnum DESIGNATED_NO_TARGET: AppliesToIDs: MA_PackageSystemType DesignationObjective: MA_RequirementTaxonomyType DesignationType: DesignationEnum
MA_StrikeActivityMT: MA_StrikeActivityMT	None
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType
RelationshipDesignationMT: RelationshipDesignationMT	Relationship: RelationshipType Destination: GeoLocatedObjectType EngagementStatus: ExternalCommandExecutionStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.6 Holster

The “Holster” Tactic Command is meant to cancel any existing shot allocations between ACP(s) and tracks. If the ACP(s) are currently targeting and/or intercepting tracks, this command will keep the ACP(s) on the intercept course but remove their allocated shot. This command is sent using the *MA_DesignationMT* with a *DesignationData* field for each track that needs to be holstered. Each *DesignationData* must set the *TypeOfDesignation* field to “*DESIGNATED_NO_TARGET*” and the *Target.EntityID* set to the *EntityID* of the track to be holstered. The *DesignationObjective* field should have an *MA_ActionTypeEnum* of “*TARGET*” informing MA it can still continue its intercept behavior, but should un-allocate shots for the specified *EntityID*. Additionally, the *DesignationID* field must be set with a valid UUID and the *AppliesToIDs* will be set to the targeting System or Package's UUID.

ACP(s) acknowledge the “Holster” designation with the *MA_SystemNotificationMT* message. The *AssociatedMessageType* field will be set to “*MA_DESIGNATION*”. If positively acknowledged (meaning *NotificationDetail* is not included) and any ACP(s) previously had shots allocated, ACP(s) will update any applicable *StrikeActivityMT* to remove each allocated shot. MS would publish *StrikeActivityMT(s)* communicating any updates of current employment of weapons to MA, which MA can forward to C2. The removed shot allocation would be represented by the field *AssignedTarget being* no longer included by MS to MA, and then sent from MA to C2. This prevents the ACP's MS from firing. Updates to a strike activity will be reported via the *StrikeActivityMT* message.

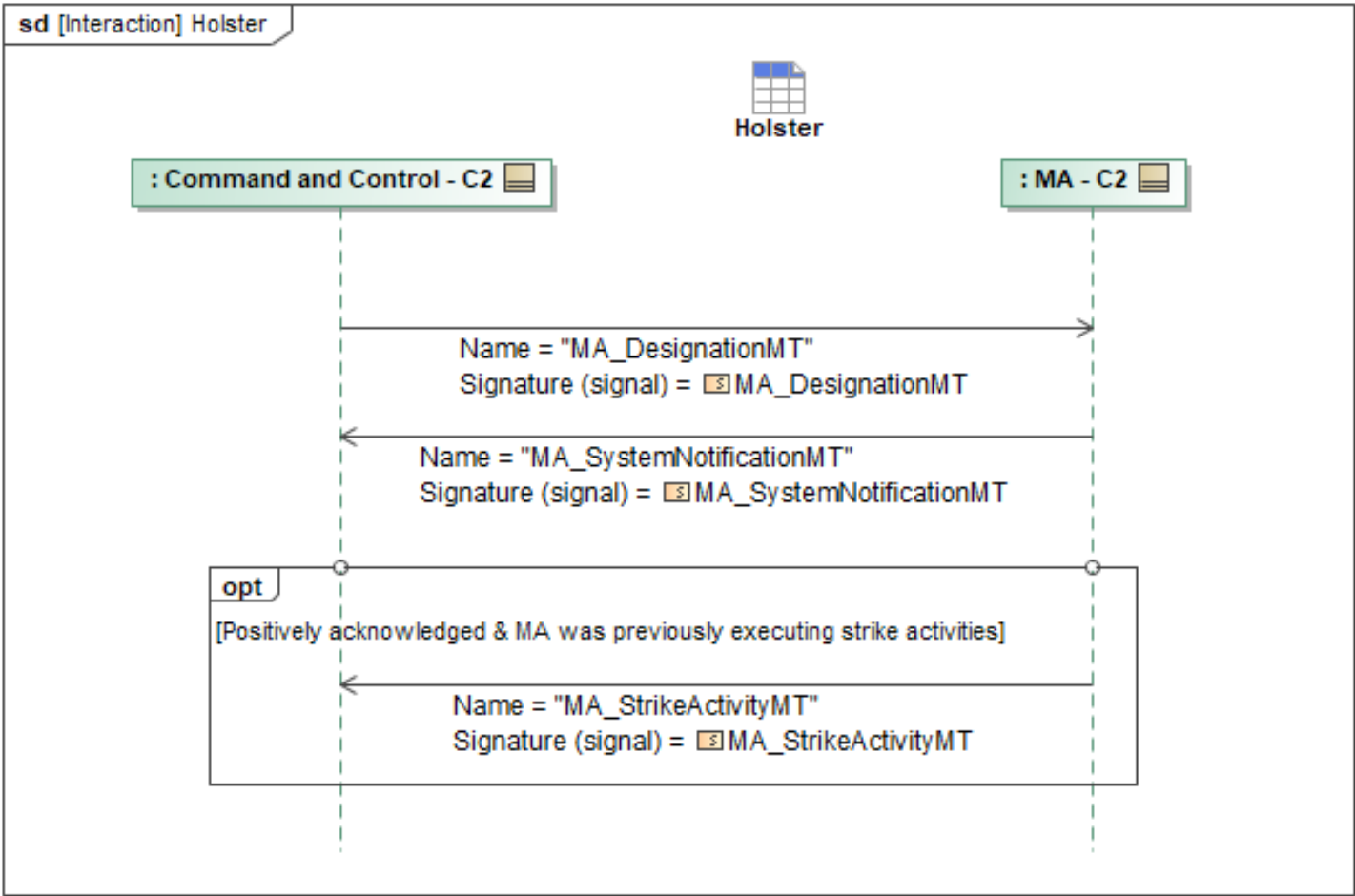


Figure 5-165: Holster

Table 5-164: Holster

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationMT: MA_DesignationMT	TypeOfDesignation: DesignationEnum DESIGNATED_NO_TARGET: AppliesToIDs: MA_PackageSystemType DesignationObjective: MA_RequirementTaxonomyType
MA_StrikeActivityMT: MA_StrikeActivityMT	ConsentState: ConsentEnum SelectedForRelease: boolean
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.7 Receive Package Tactic Status

MA will periodically provide updates to C2 on the status of a Tactic while the Tactic is being executed or queued to begin executing. MA sends *ActionActivityMT* periodically to convey periodic updates

regarding the ongoing activity executing the tactic. MA sends *MA_ActionStatusMT* periodically to convey the status of the Tactic as a whole, while MA sends *MA_TaskStatusMT* periodically so it can provide more detailed statuses on the respective Tasks that make up the Tactic. This can apply to an entire package, where the lead would communicate all of it's Peers statuses, or just a single ACP. The "Report Tactic Status - Peer" sequence describes how peers will communicate ongoing tactics to the lead, where the lead is responsible for reporting the package tactic status to C2. In the case of a single ship, the single ship would be the only ACP in the package, and the sequence would only be executed one time.

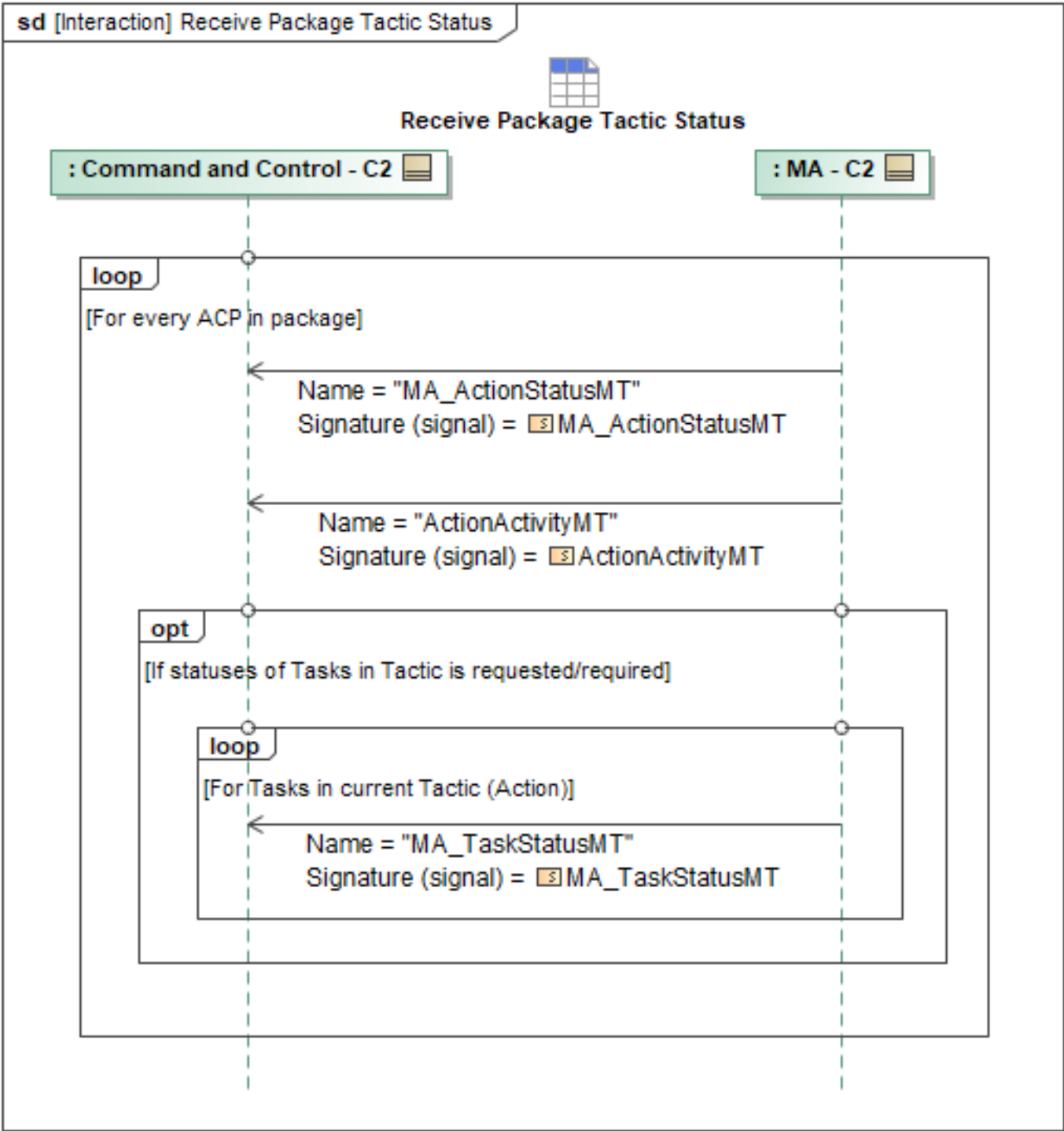


Figure 5-166: Receive Package Tactic Status

Table 5-165: Receive Package Tactic Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActionActivityMT: ActionActivityMT	None
MA_ActionStatusMT: MA_ActionStatusMT	PackageID: PackageID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
	PackageID: PackageID_Type
MA_TaskStatusMT: MA_TaskStatusMT	TaskID: TaskID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.8 Set Tactic Shot Doctrine and Intercept Maneuver Parameters

Updating parameters that may inform a shot doctrine and/or intercept maneuver tactics for Mission Autonomy to ingest is done via an *MA_ActionMT* message. A C2 operator would send an *MA_ActionMT* message to provide this update. This *MA_ActionMT* can also be setup during the mission planning process using the fields discussed below. Tactics will then be employed from C2, at the Action level, using an *MA_ActionCommandMT*. In the *MA_ActionMT*, targeting tactics will utilize the *MA_ActionTypeEnum* of *TARGET* while strikes will utilize *ATTACK_KINETIC*. Other *MA_ActionTypeEnum*'s can be used depending on the tactic desired, which are discussed in other C2 behaviors. Other Values related to a shot doctrine live within the *ActionConstraints* field. Specifically, within *ActionConstraints*, the field *EngagementParameters* contains fields to specify *InterceptTactic*, *ShotParameters*, and *EnableAutomaticRecalculation*, the later which specifies whether to enable automatic recalculation during engagement. The *InterceptTactic* is a choice type for C2 to set either a *SkateType* or *BanzaiType*. Within *SkateType*, a *ManeuverOutOfRange* can be set via a *DistanceType*. Within *BanzaiType* a *TerminationType* can be set which uses the *MA_InterceptTacticTerminationTypeEnum*. The enumeration values available are *BLOW_THROUGH*, *VELCRO*, or *COLLIDE*. *ShotParameters* contains *MissilesPerContact* and *ShootChoice*. The *MissilesPerContact* field denotes how many missiles should be used per entity contact, specified as an integer. The *ShootChoice* allows a choice between toggling *UseRangeProbabilityOfIntercept* or setting *ShotParameterDetails*. *ShotParameterDetails* contains *UseRangeTurnAndRunOrDMC* for C2 to specify an ACP to abide by a *RangeTurnAndRun* or follow a *DigitalManeuveringCueAngle*. The *Digital Maneuver Cue* value can either be specified using the *RangeTurnAndRun* value or the *DigitalManeuveringCueAngle*. If *RangeTurnAndRun* is desired, the *RangeTurnAndRun* value will specified within *MA_WEZ_MT* (see "Exchange WEZ" Interaction) or *DLZ_MT* (see "Exchange DLZ" Interaction) and the *UseRangeTurnAndRun* field should be populated. If the *DigitalManeuveringCueAngle* field is used, it should be a value in radians between 0.17452393 and 3.141592653589793238462, set in increments of 0.17452393 radians (which corresponds to 10 to 180 degrees set in increments of 10 degrees). *ShotParameterDetails* also specify the *ShootNLT* and *ShootNET* fields which signify shoot no later than and shoot no earlier than values allow C2 to specify a shot window. These fields are *DistanceTypes*.

When commanded via an *MA_ActionCommandMT*, an ACP will follow these tactic parameters when engaging a target.

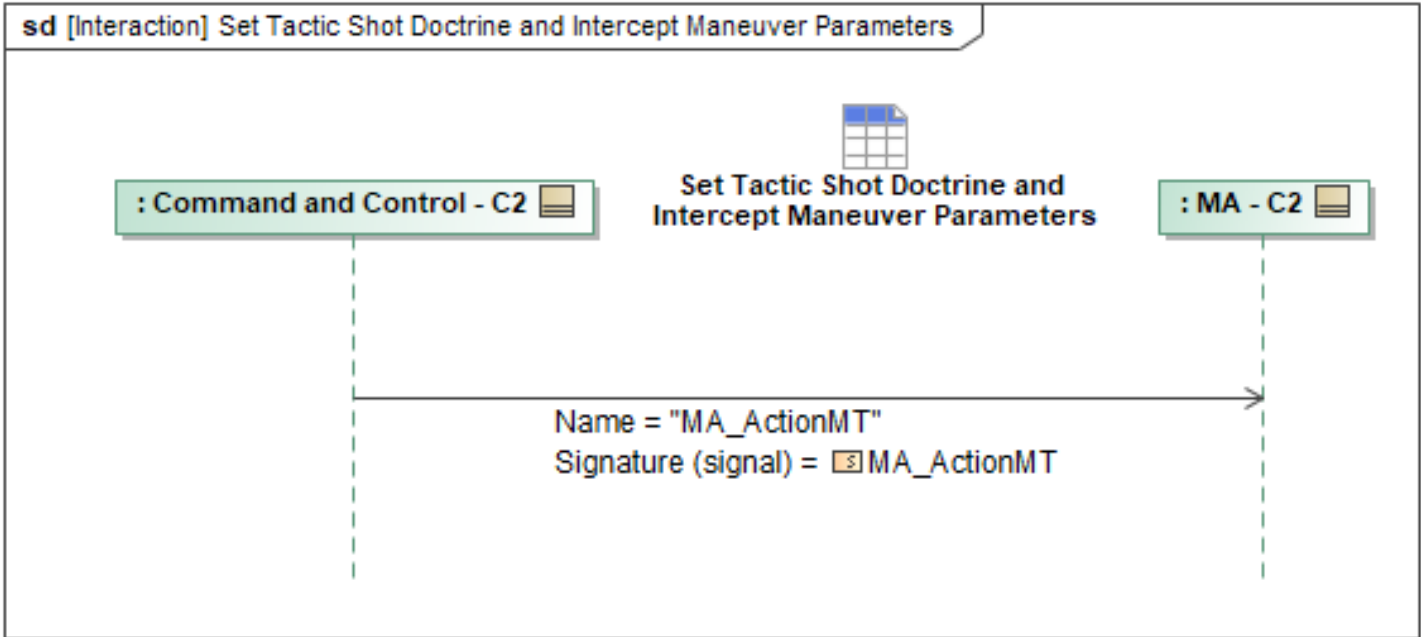


Figure 5-167: SetTacticShotDoctrineAndInterceptManeuverParameters

Table 5-166: SetTacticShotDoctrineAndInterceptManeuverParameters

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionMT: MA_ActionMT	ActionID: ActionID_Type ActionConstraints: MA_RequirementConstraintsType EngagementParameters: MA_EngagementParametersType ActionType: MA_ActionTypeEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.9 Shoot

A “Shoot” tactic commanded from an operator tells one or more ACPs to allocate shots to applicable tracks and to shoot them according to a pre-determined shot doctrine. The “Shoot” tactic can be executed using either a Capability-based or an Activity-based command path. The tactic begins executing when C2 sends an *MA_ActionCommandMT*. In the Capability-based path, C2 references a pre-planned *MA_ActionMT* via the *ActionID* field by *UUID* so MA references the desired shot doctrine parameters. In this case, the *ActionType* of the referenced *MA_ActionMT* needs to be *ATTACK_KINETIC*. The parameters of the tactic are configured during mission planning by setting the fields within *MA_ActionMT*.

If the operator uses the *Activity* path, autonomy advertises an ongoing *ActionActivity* and C2 sends an *MA_ActionCommandMT* whose *ActivityID* matches the advertised *ActivityID* for that ongoing play. In this Activity-based path, updates to the ongoing tactic’s shot doctrine and engagement parameters are conveyed via *ChangeShotDoctrine*, *ChangeActionConstraints*, and *ChangeAttackKineticParameters*.

If desired, the 5.2.9.8 “SetTacticShotDoctrineAndInterceptManeuverParameters” behavior may

still be used to change the underlying *MA_ActionMT* for an ongoing tactic.

If operators employing the *Shoot* tactic wish to use a different parameter value for anything within the shot doctrine, the *ConstraintOverride* field remains available. Contained within this field is the *ShotDoctrineParameters* field. This only needs to contain values if the commanded tactic should follow updated parameters. Fields populated in *ShotDoctrineParameters* override the default values.

For the *Activity Audible* path, omission versus explicit values follows - fields that are omitted from *ChangeShotDoctrine*, *ChangeActionConstraints*, or *ChangeAttackKineticParameters* remain at their current effective value for the ongoing play; Audibles do not implicitly reset unspecified parameters. Fields that are present with concrete values replace the current effective value for that parameter for the duration of the ongoing *ActionActivity*, until further update or tactic completion. Where supported, an explicit “use default” indicator may be used to revert a parameter to the value defined in the referenced *MA_ActionMT* for this play.

These overrides apply to the ongoing *ActionActivity* for the duration of that play. Subsequent Audibles referencing the same *ActivityID* may further update these parameters; parameters not mentioned in a given Audible retain their current effective values for that ongoing play. The *MA_ActionCommandMT* (whether using the *ActionID*-initiated baseline or the *ActivityID*-based Audible path) will need to contain information about the entities the Shoot command should be acted on. This information is contained within the *ActionConstraintOverride* field within *Capability* – specifically the *TargetObject*, where *EntityIDs* can be added *ByInstance*.

Additional updates to per-target missile allocation for an ongoing play are expressed via *PerTargetShotDoctrine* structure within *ChangeActionConstraints*, whose cardinality matches the *TargetObject* when used with *ChangeActionConstraints*. When *ChangeActionConstraints.TargetObject* and *MissilesPerContact* are provided for an ongoing *ActionActivity*, they are interpreted as an add/modify set for that play: targets newly introduced are added, and existing targets may have their associated missile counts updated. Targets that are part of the ongoing tactic and are not mentioned in *ChangeActionConstraints* remain unaffected by that Audible; removal of targets continues to be expressed via existing “Drop” behaviors rather than by omission from *TargetObject*. Similarly to the previous “Target” tactic, MA (lead) is responsible for allocating and assigning shots within a package of ACPs for n number of targets.

The allocated shot pairings are shared with C2 by a lead MA using the *RelationshipDesignationMT* message, which is covered by the “Report Shot Allocations” sequence. Furthermore, the ongoing statuses of all commanded shots are shared via the use of two behaviors on the Peer and C2 interfaces respectively: “Peer Report Tactic Status” and “Receive Package Tactic Status”.

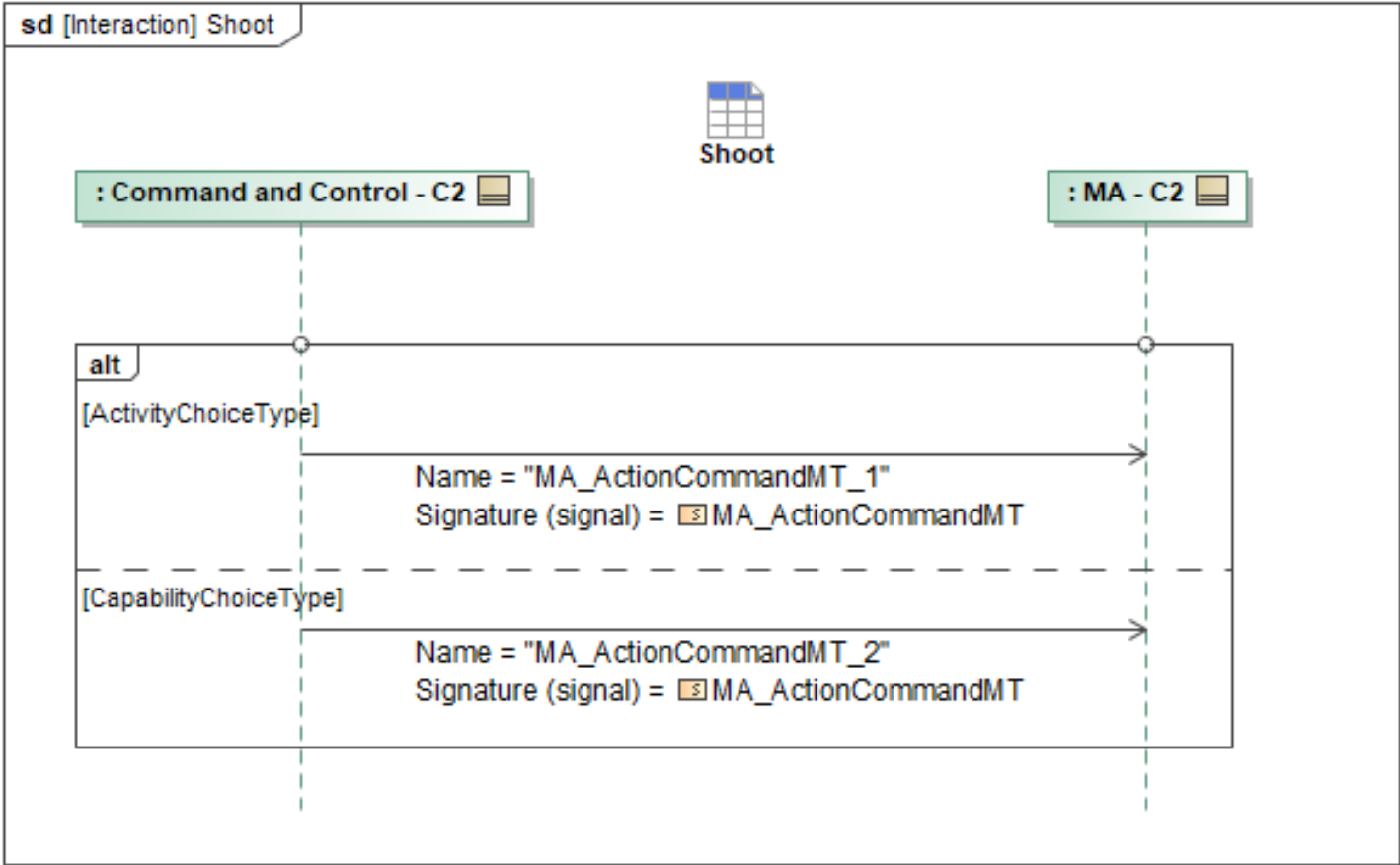


Figure 5-168: Shoot

Table 5-167: Shoot

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT_1: MA_ActionCommandMT	TargetObject: IdentityKindInstanceType MissilesPerContact: MA_ObjectShotCountChoiceType Activity: MA_ActivityCommandBaseType ChangeActionConstraints: MA_ActionConstraintType
MA_ActionCommandMT_2: MA_ActionCommandMT	ActionConstraintOverride: MA_ActionConstraintType Capability: MA_ActionCapabilityCommandType TargetObject: IdentityKindInstanceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.10 Shoot Now

A “Shoot Now” tactic command is very similar to the “Shoot” tactic, but rather than ensuring ACPs abide by specific shot-doctrine shot windows (Shoot NET and Shoot NLT), Shoot Now will not follow a shot window and relaxes release criteria to meet the Range Probable Intercept (RPI) value. The

“Shoot Now” tactic is expected to be set up pre-mission as a separate *MA_ActionMT*, with the same *ActionType = ATTACK_KINETIC* as the “Shoot” tactic. Operators can specify whatever parameters are desired to align with their operational intent for a “Shoot Now” tactic. However, it would be expected that the *ShootNLT*, *ShootNET*, and *DigitalManeuveringCueAngle* fields are excluded from the *ShotDoctrineParameters* within *ActionConstraints* in this pre-defined *MA_ActionMT*. The “Shoot Now” tactic can be executed using either a Capability-based or an Activity-based command path.

In the Capability-based path, the pre-defined *MA_ActionMT* is referenced by *UUID* via *ActionID* in *MA_ActionCommandMT*, and the pre-planned parameters are applied, including any pre-mission *ShotDoctrineParameters* that define how “Shoot Now” should behave by default.

When “Shoot Now” is used as an audible on an ongoing tactic, autonomy advertises an ongoing *ActionActivity* and C2 sends an *MA_ActionCommandMT* whose *ActivityID* matches the advertised *ActivityID* for that ongoing play. In this Activity-based path, updates to the ongoing tactic’s shot doctrine and engagement parameters are conveyed via *ChangeShotDoctrine* where *UseRangeProbabilityOfIntercept* is used to explicitly specify a shot constraint. *ShootNLT*, *ShootNET* and *DigitalManeuveringCueAngle* are still expected to be excluded. *ChangeActionConstraints*, *ChangeAttackKineticParameters*, and *Capability* are not used for this audible. If an operator wishes to override a certain field or value within a pre-defined shot doctrine, this can be done in the same way as for other tactics that utilize an *MA_ActionCommandMT* message. For example, if a “Shoot Now” tactic default includes *MissilesPerContact* = 1, but release of 2 missiles is needed within RPI, the *MissilesPerContact* field within *ConstraintOverride* should specify “2”. This will command this tactic only with the overriding constraints that are filled out.

For the Activity-based Audible path, omission versus explicit values follow these rules: fields that are omitted from *ChangeShotDoctrine*, *ChangeActionConstraints*, or *ChangeAttackKineticParameters* remain at their current effective value for the ongoing play; Audibles do not implicitly reset unspecified parameters. Fields that are present with concrete values replace the current effective value for that parameter for the duration of the ongoing *ActionActivity*, until further update or tactic completion. Where supported, an explicit “use default” indicator may be used to revert a parameter to the value defined in the referenced *MA_ActionMT* for this play. “Default shot doctrine” refers to the *ShotDoctrine* parameters as defined on the referenced *MA_ActionMT*, not to the state prior to any given Audible. When using the *ActivityID* path, these overrides apply to the ongoing *ActionActivity* for the duration of that play. Subsequent Audibles referencing the same *ActivityID* may further update these parameters; parameters not mentioned in a given Audible retain their current effective values for that ongoing play. The *MA_ActionCommandMT* message needs to contain information about the applicable targets the “Shoot Now” command applies to as well. This information is contained within the *ActionConstraintOverride* field within Capability – specifically the *TargetObject* field, where *EntityIDs* can be added *ByInstance*.

Additional updates to per-target missile allocation for an ongoing play are expressed via the *MissilePerTarget* within *PerTargetShotDoctrine*, whose cardinality matches the *TargetObject* when used with *ChangeActionConstraints*. When *ChangeActionConstraints.TargetObject* and *MissilesPerTarget* are provided for an ongoing *ActionActivity*, they are interpreted as an add/modify set for that play: targets newly introduced are added, and existing targets may have their associated missile counts updated. Targets that are part of the ongoing tactic and are not mentioned in *ChangeActionConstraints* remain unaffected by that Audible; removal of targets continues to be expressed via existing “Drop” behaviors rather than by omission from *TargetObject*.

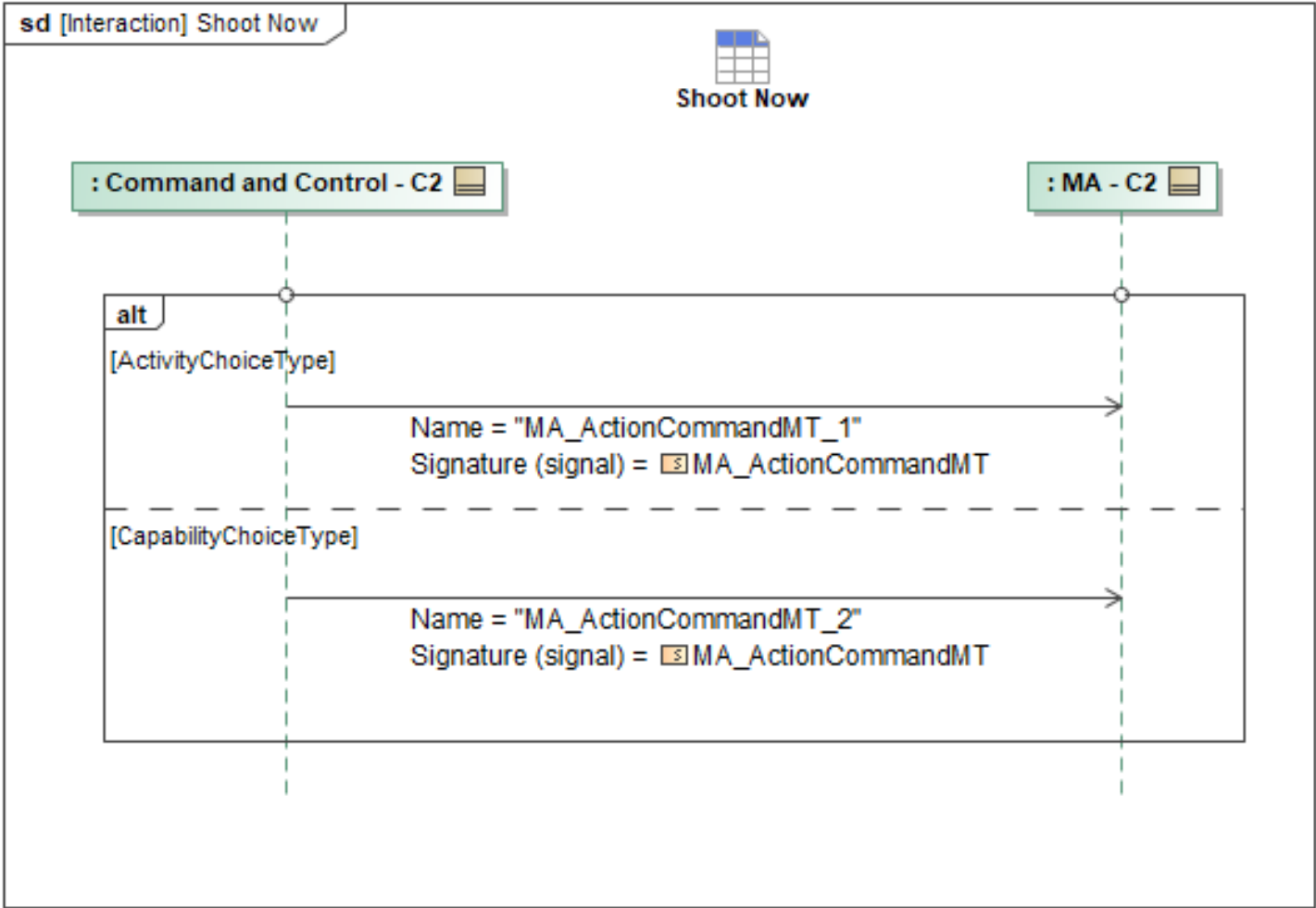


Figure 5-169: Shoot Now

Table 5-168: Shoot Now

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT_1: MA_ActionCommandMT	TargetObject: IdentityKindInstanceType MissilesPerContact: MA_ObjectShotCountChoiceType Activity: ActivityCommandBaseType ChangeActionConstraints: MA_ActionConstraintType
MA_ActionCommandMT_2: MA_ActionCommandMT	Capability: MA_ActionCapabilityCommandType ActionConstraintOverride: MA_ActionConstraintType TargetObject: IdentityKindInstanceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.11.11 Target

The Target tactic enables a C2 operator to command ACP platforms individually or as a package to target 1 to many entities. MA will abide by shot doctrine and intercept maneuver parameters that are

configured during mission planning via setting the fields within *MA_ActionMT* or can be modified during mission execution. For more details on updating these parameters see the "Set Tactic Shot Doctrine and Intercept Maneuver Parameters" C2 behavior. Targeting tactics commanded to an entire package result in the specific ACP-target pairings being determined by a lead MA to assign. Upon receipt of a target tactic command, a lead MA determines optimal pairings and makes the target assignment to its peers. This behavior is described in more detail in the Peer Interaction "Assign Target Pairings - Peer". The pre-planned target tactic needs to include an *MA_ActionMT* that specifies an *ActionType* of "TARGET". This targeting action will then be broken down into tasks and/or activities and shared amongst the peers to execute the desired intent. For C2 to command a tactic of "Target", the sequence is similar to the commanding of a generic tactic.

As mentioned, the "Target" tactic should be pre-planned in the form of a *MA_ActionMT* message that contains the desired intercept maneuver and shot doctrine parameters as well as specifying the *MA_ActionTypeEnum* to "TARGET". To command the tactic, C2 will just send the *MA_ActionCommandMT* message. Specifying the *TargetObjects* is needed and should be done within the *TargetObject* field nested within *ActionConstraintOverride*. One to many *EntityIDs* should be set here using *EntityID* under *ByInstance*. The specific UUIDs of all entities to be targeted is used. If an operator wishes to send a "Target" tactic with one or more of the default shot doctrine parameters overridden for the this tactic, they should be populated within the applicable *EngagementParameters* field. This field is nested within the *ConstraintOverride* field in the *MA_ActionCommandMT* message. By sending a target command with *EngagementParameters* within *ConstraintOverride*, that field will override the planned default parameters and MA will fall back to the default values for everything left out of *ConstraintOverride*. The commanded values will only be used for this command, and the defaults will still be active on MA.

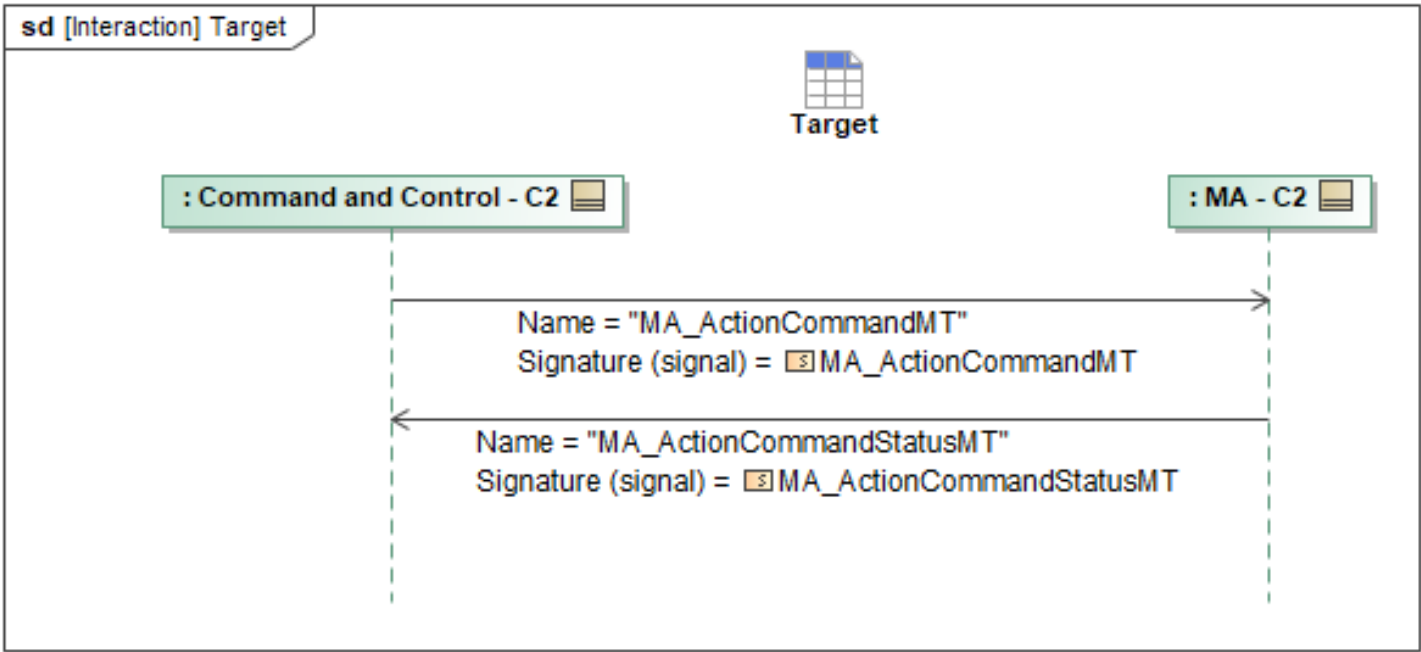


Figure 5-170: Target

Table 5-169: Target

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT: MA_ActionCommandMT	TargetObject: IdentityKindInstanceType ByInstance: TargetType Capability: MA_ActionCapabilityCommandType ActionConstraintOverride: MA_ActionConstraintType ActionID: ActionID_Type
MA_ActionCommandStatusMT: MA_ActionCommandStatusMT	CommandProcessingState: CommandProcessingStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12 C2 Task Prioritization and Mission Context Behaviors Package

5.2.12.1 C2 Define Global Override

The Command and Control (C2) system can send the global override settings to Mission Autonomy (MA) via *MA_PlatformOverrideCommandMT*. This global settings value will be a speed, altitude, formation, or Identification Friend or Foe (IFF) setting command to be sent to an individual Autonomy Capable Platform (ACP) or to a team lead, and this interaction allows an overriding value for one of those parameters to be set or cleared. If setting a new global override, then the *CommandState* should be set to *NEW*. The specific request (speed, altitude, formation, or IFF override) should be set via the *PlatformOverrideCommand* field. If sending a speed override, the *SpeedOverrideCommand* field should be populated. Likewise, if setting an altitude override, the *AltitudeOverrideCommand* should be used. If setting a global formation command, then the *FormationOverrideCommand* field

should be used. Finally, if setting IFF settings, the *IFF_OverrideCommand* field should be used. To cancel an existing global override, the *CommandState* should be set to *CANCEL* and the same *PlatformOverrideCommand* type as was used when sending the override should be populated. The exact value is a design detail left to implementation; the only expectation is that the corresponding field is populated as a mechanism for identifying which override has been cleared. The *Applicability* field indicates the unique identifier of the System or Package intended to have effects applied. The *ExpirationTime* field may be optionally populated when sending a new override command to define the time at which the override will expire; if unpopulated, the command is indefinite until canceled. Likewise, the *ApplicableZoneID* field may be optionally populated to define one or more zones within which the override is applicable; if unpopulated, the command is valid throughout the operating area.

MA sends C2 the *MA_PlatformOverrideCommandStatusMT* message, accepting or rejecting the command. If the command is rejected, the *MA_PlatformOverrideCommandStatusMT* message will specify the *CommandProcessingStateReason* field to provide further details. If the command is accepted, MA will reply with *MA_PlatformOverrideCommandStatusMT* where *CommandProcessingState* is set to *ACCEPTED* and will report the active platform override(s) by setting the *PlatformOverrideStatus* field with the value of each active override setting within *MA_PlatformOverrideStatusMT*.

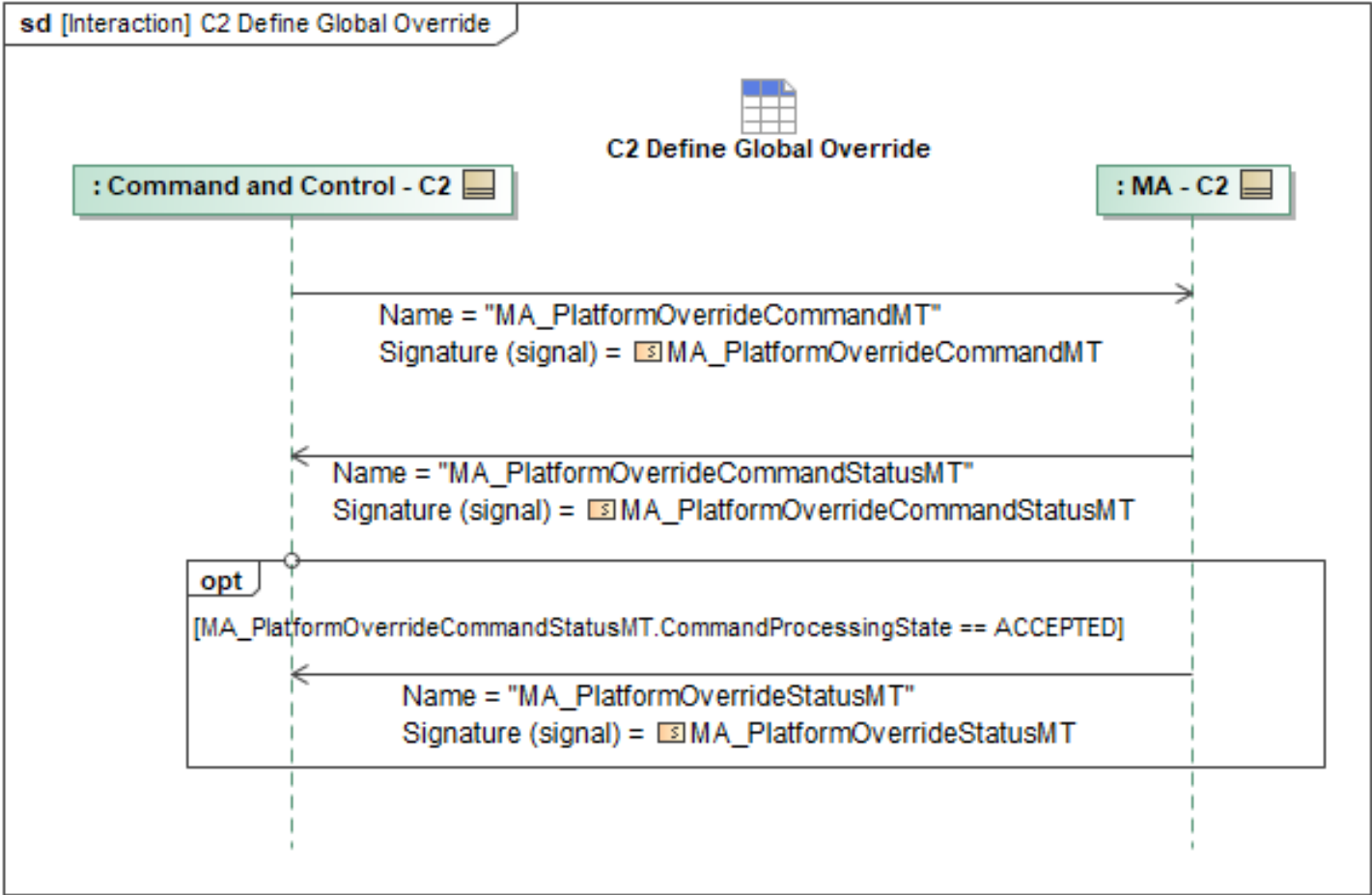


Figure 5-171: C2 Define Global Override

Table 5-170: C2 Define Global Override

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlatformOverrideCommandMT: MA_PlatformOverrideCommandMT	None
MA_PlatformOverrideCommandStatusMT: MA_PlatformOverrideCommandStatusMT	None
MA_PlatformOverrideStatusMT: MA_PlatformOverrideStatusMT	PlatformOverrideStatus: MA_PlatformDefaultAndOverrideDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.2 C2 Acceptable Levels of Risk Behaviors

5.2.12.2.1 Provide Risk Assessment Results

Mission Use Case: ALR

The optional Provide Risk Assessment Results sequence diagram specifies the interaction where an ACP provides C2 with risk assessment results if an updated threat triggers an assessment of the current Route Plan. Upon determining that the current Route Plan exceeds the specified Acceptable

Level of Risk (ALR) setting, the ACP may attempt to generate a new Route Plan through the threat environment which complies with the ALR setting. (If the risk assessment for the current route plan does not exceed the ALR setting, the risk assessment results will not be sent to C2, and this interaction would not apply). After a new Route Plan is generated, the ACP sends *MA_AssessmentMT* to C2 to inform of the risk assessment results, populating the *Data* of the *RouteThreatAssessment* field within the *Assessment* field of this message. The message will have a populated *AnalysisRouteID* to identify the Route Plan for which the assessment applies. If a Route Plan which complies with the ALR setting is not generated or able to be achieved, the *MA_AssessmentMT* message will convey this to C2 by populating the *Achievability* field within *AchievabilityAssessment* as *NOT_ACHIEVABLE*.

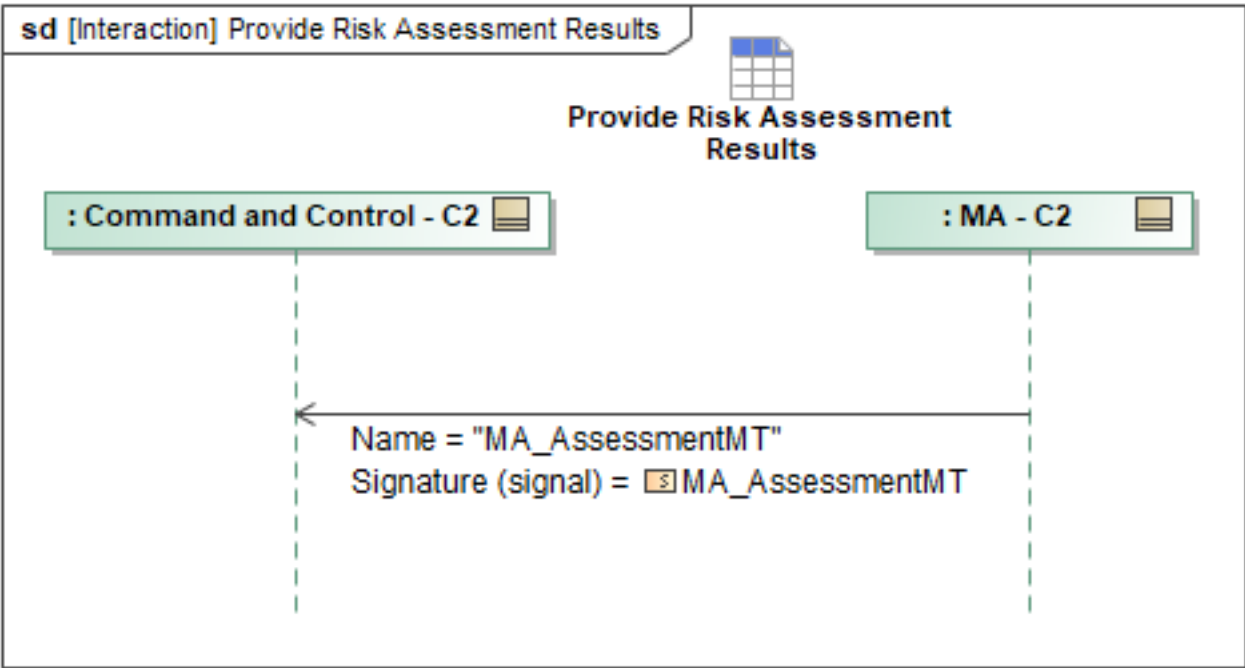


Figure 5-172: Provide Risk Assessment Results

Table 5-171: Provide Risk Assessment Results

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentMT: MA_AssessmentMT	AnalysisRouteID: AnalysisRouteID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.2.2 Request Route Threat Assessment

Mission Use Case: ALR

The optional Request Route Threat Assessment sequence diagram specifies the interaction where C2 commands an ACP to perform a risk assessment for a Route Plan. This interaction assumes that the Route Plan to be assessed has been previously uploaded to the ACP or that the “Update Route Plan” interaction has occurred where C2 uploads a new or updated Route Plan. C2 will request a risk assessment for a Route Plan, sending *MA_AssessmentRequestMT* with a specified *AnalysisRouteID* and the *Parameters* of the *RouteThreatAssessment* field within the *Category* field populated. MA may

optionally report status on the request to C2 using *MA_AssessmentRequestStatusMT_1* to convey the processing state and/or estimated completion information. MA will deliver the risk assessment results with the *MA_AssessmentRequestStatusMT_2* message, populating the *Data* of the *RouteThreatAssessment* field within the *Assessment* field.

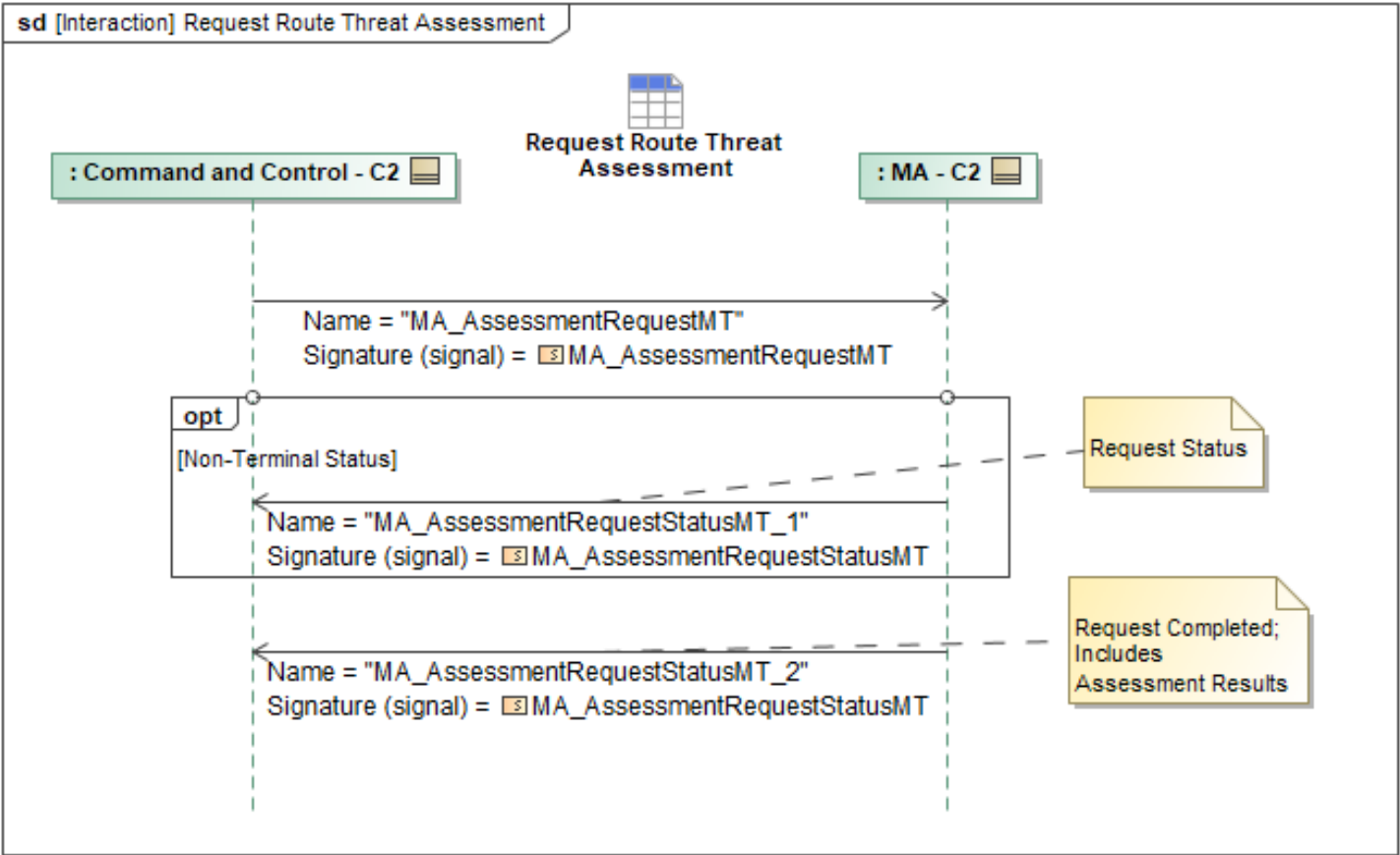


Figure 5-173: Request Route Threat Assessment

Table 5-172: Request Route Threat Assessment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentRequestMT: MA_AssessmentRequestMT	AnalysisRouteID: AnalysisRouteID_Type
MA_AssessmentRequestStatusMT_1: MA_AssessmentRequestStatusMT	None
MA_AssessmentRequestStatusMT_2: MA_AssessmentRequestStatusMT	Assessment: AssessmentType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.2.3 Update Acceptable Level of Risk

Mission Use Case: ALR

Overview

The Acceptable Level of Risk (ALR) command message (*MA_AcceptableLevelOfRiskCommandMT*) allows an operator to set a specific level of risk the MA is allowed to incur during the planning or execution of a Mission Plan. The *AcceptableRiskSetting* field allows the value to include *HIGH*, *MEDIUM*, *LOW*, or *NONE* at the package or vehicle level. This ALR setting applies to all tasks and actions for the vehicle or package. The *SurvivabilityRiskLevelID* field allows an operator to select a specific Survivability Risk Definition for MA, which has previously been defined in the Mission Plan or by C2 using the *SurvivabilityRiskLevelMT* message. The difference between ALR and Survivability Risk Level is that ALR defines the maximum allowed risk the planner is allowed to plan for and the *SurvivabilityRiskLevelMT* describes what *HIGH*, *MEDIUM*, and *LOW* mean with respect to the *MaximumContinuousExposure* for a given Vulnerability Type. Where *MaximumContinuousExposure* refers to the Composite and Cumulative Exposure Time allowed and the Vulnerability Type includes Acquisition (detection by acquisition radars), Track (exposure to track radars), Intercept (probably of intercept (impact) by a threat), Launch (probably of launch by a threat), and Kill (overall survivability prediction). *SurvivabilityRiskLevelMT* can be set in the mission plan or updated by C2 to define the various levels of risk. If the MA does not have an active ALR, this is the same as having an ALR of *NONE*. The MA (leader or individual ACP) will provide status on the commanded *MA_AcceptableLevelOfRiskCommandMT* messages using the *MA_AcceptableLevelOfRiskCommandStatusMT* message and will report its currently activated ALR setting and Survivability Risk Level to C2 using *MA_AcceptableLevelOfRiskStatusMT*. The C2 will send messages to a package lead or non-package ACPs. Package leads will distribute the messages to the rest of the follower ACPs.

Mission Execution

The ALR and *SurvivabilityRiskLevelMT* are used to inform tactical decisions and planning during mission execution. Any planner that supports the use of ALR may choose to alter tactics (i.e. produce a different plan) based on the planner's best assessment of the risk involved with executing the plan in the operational environment. Risk assessment is a complicated modeling process and is not currently governed by the A-GRA. During mission execution, if a new *MA_AcceptableLevelOfRiskCommandMT* is received/updated during execution, this should trigger a replan if necessary to avoid the current activity exceeding the new ALR setting based on the *SurvivabilityRiskLevelMT* definitions and the operational environment

Sequence Description

The Update Acceptable Level of Risk sequence diagram specifies the interaction where C2 commands updates to the ALR setting for a non-package ACP, an entire package, or specific ACP(s) within a package. First, C2 sends *MA_AcceptableLevelOfRiskCommandMT* to MA, where the *AppliesToID* field can have multiple entries of *PackageID* or *SystemID*. This allows C2 to send a Lead MA the Package ID (or multiple sub-Package IDs) or a set of System IDs. The *MA_AcceptableLevelOfRiskCommandMT* message shall define an *AcceptableRiskSetting*, which allows for the ALR to be set as *HIGH*, *MEDIUM*, *LOW*, or *NONE* at the package or vehicle level. It also defines a *SurvivabilityRiskLevelID* which allows for a *RiskSetting* to be populated to describe what *HIGH*, *MEDIUM*, and *LOW* mean with respect to the *MaximumContinuousExposure* for a given Vulnerability Type. Any new *MA_AcceptableLevelOfRiskCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_AcceptableLevelOfRiskCommandMT* is sent with the *CommandState* set to *CANCEL*, this will cancel the active command

In response to receiving the command, MA shall send

MA_AcceptableLevelOfRiskCommandStatusMT_1 to C2, with *CommandProcessingState* set to *RECEIVED* to indicate receipt of the command. For this status message, the *AppliesToID* field shall be set to the Package ID if MA is the leader and has received a status message from all members of the Package ID. Otherwise, it shall be set to the System ID(s) and/or sub-Package ID(s) for which the status message applies.

If the MA is a non-package ACP, it shall send *MA_AcceptableLevelOfRiskCommandStatusMT_2* to C2 with the *CommandProcessingState* set to *ACCEPTED* to indicate that its ALR setting has been updated. An ACP will then send *MA_AcceptableLevelOfRiskStatusMT_1* to C2 to provide status on its currently activated ALR setting and Survivability Risk Level, using the *AcceptableRiskSetting* and *SurvivabilityRiskLevelID* fields, and populating its System ID in the *AppliesToID* field. If MA is the lead ACP, then MA shall send its follower ACPs *MA_AcceptableLevelOfRiskCommandMT* (See "Update Peer Acceptable Level of Risk"). After receiving *ACCEPTED* status from all applicable follower ACPs, the MA lead sends C2 *MA_AcceptableLevelOfRiskCommandStatusMT_3* with the *CommandProcessingState* set to *ACCEPTED*, reflecting the *AppliesToID* as needed. The Lead ACP will then send *MA_AcceptableLevelOfRiskStatusMT_2* to C2 to provide status on the package's currently activated ALR setting and Survivability Risk Level, using the *AcceptableRiskSetting* and *SurvivabilityRiskLevelID* fields, and populating the Package ID in the *AppliesToID* field.

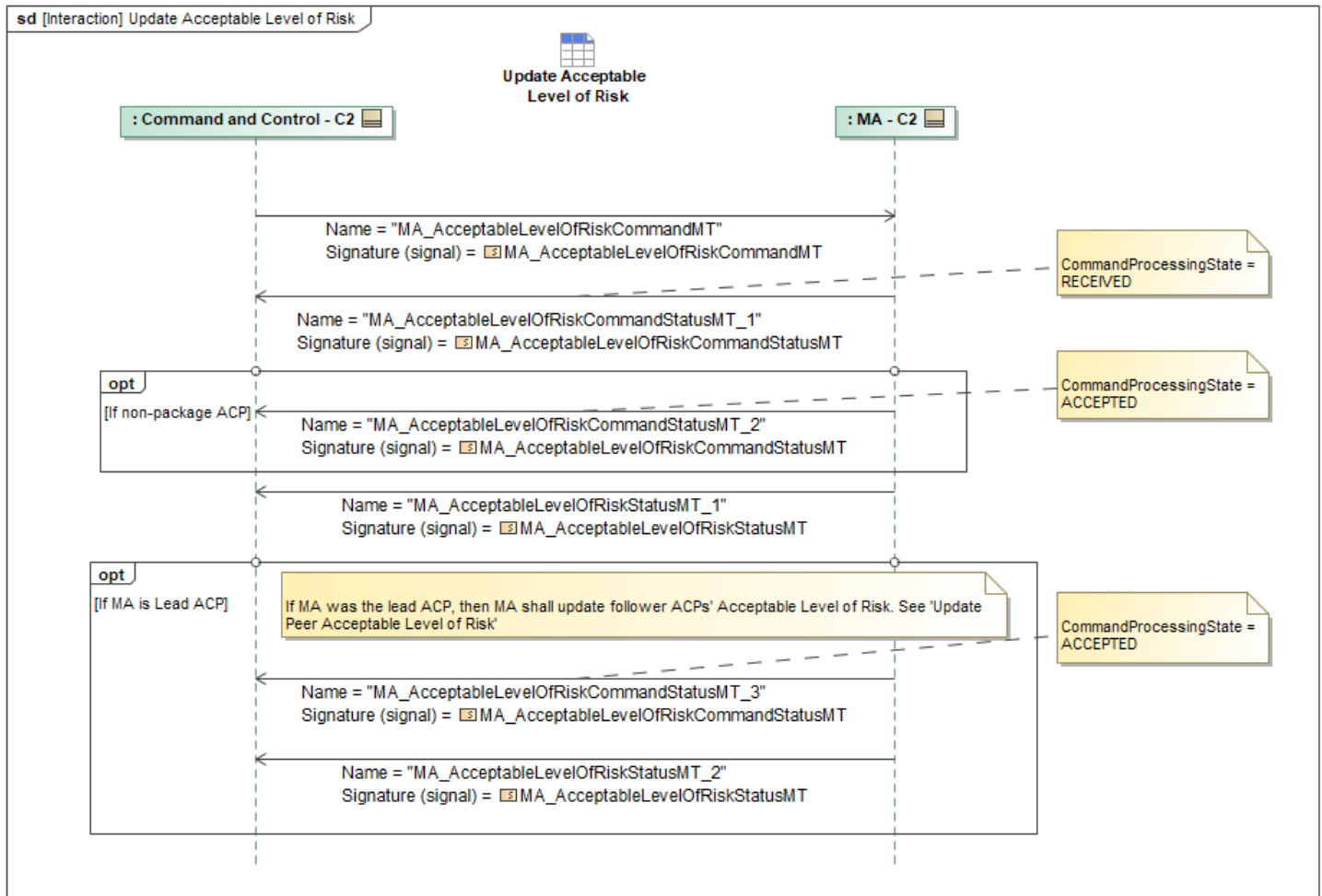


Figure 5-174: Update Acceptable Level of Risk

Table 5-173: Update Acceptable Level of Risk

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AcceptableLevelOfRiskCommandMT: MA_AcceptableLevelOfRiskCommandMT	None
MA_AcceptableLevelOfRiskCommandStatusMT_1: MA_AcceptableLevelOfRiskCommandStatusMT	None
MA_AcceptableLevelOfRiskCommandStatusMT_2: MA_AcceptableLevelOfRiskCommandStatusMT	None
MA_AcceptableLevelOfRiskCommandStatusMT_3: MA_AcceptableLevelOfRiskCommandStatusMT	None
MA_AcceptableLevelOfRiskStatusMT_1: MA_AcceptableLevelOfRiskStatusMT	None
MA_AcceptableLevelOfRiskStatusMT_2: MA_AcceptableLevelOfRiskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.2.4 Update Package Survivability Risk Definition

Mission Use Case: ALR

The optional Update Package Survivability Risk Definition sequence diagram specifies the interaction where C2 commands the Lead ACP to update the package members' Survivability Risk Definition data. First, C2 sends the Lead ACP the new Survivability Risk Definition data, and the Lead ACP confirms it has received the updated data, as shown in the referenced "Update Survivability Risk Definition" sequence diagram. Then, the Lead ACP updates the applicable follower ACPs with the data, which may be the entire package or a subset of followers. (See "Update Peer Survivability Risk Definition"). Once the Lead ACP has received a response from all followers, the Lead ACP will send *MA_SystemNotificationMT* to C2 to inform that the entire package has updated the Survivability Risk Definition. *MA_SystemNotificationMT* contains *SystemSubjectID*, *SystemPerspective*, and *AssociatedMessage* fields. The *SystemSubjectID* shall be set to the Package ID, the *SystemPerspective* enumeration shall be set to *RESPONSE_SUBJECT*, and the *AssociatedMessage* shall define the received message's *MessageTypeEnum* as *MA_SURVIVABILITY_RISK_LEVEL* with *AssociatedID* set to the *SurvivabilityRiskLevelID*.

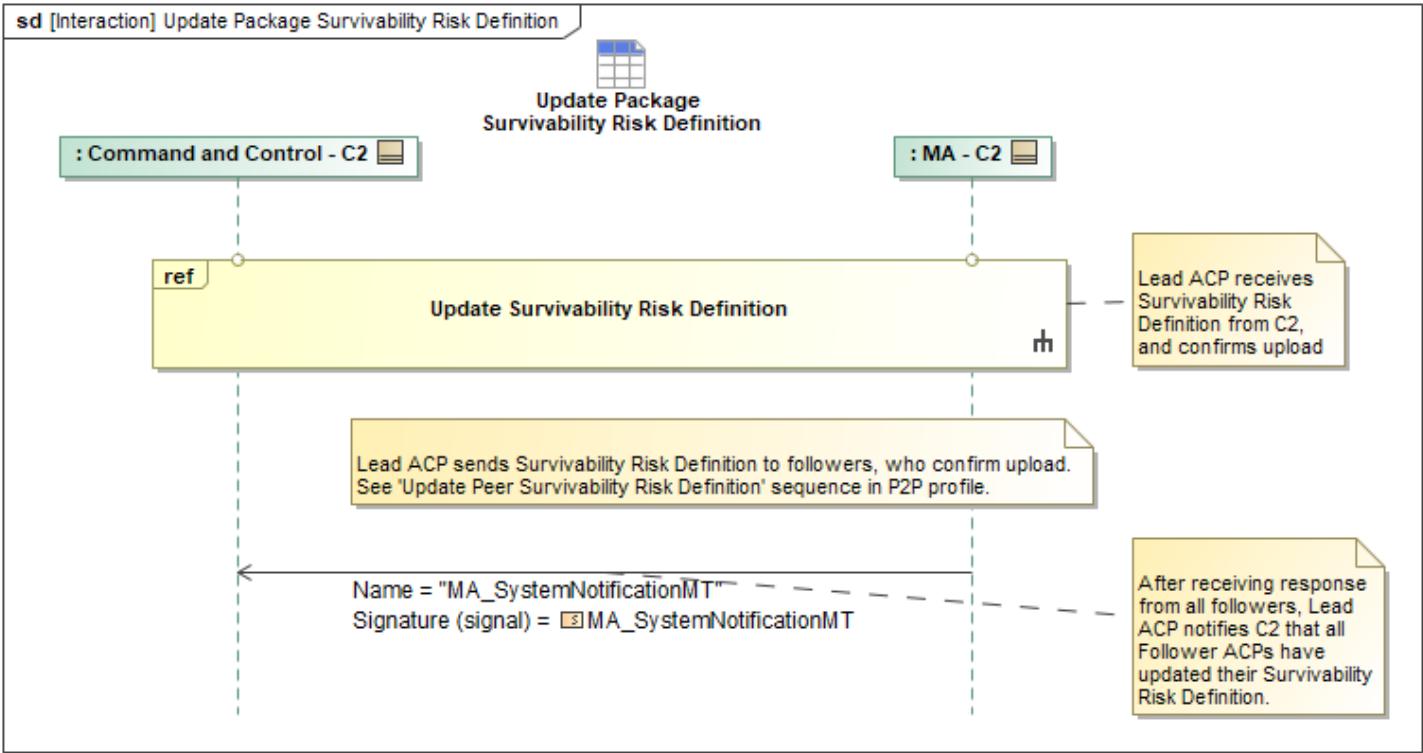


Figure 5-175: Update Package Survivability Risk Definition

Table 5-174: Update Package Survivability Risk Definition

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT:	AssociatedMessage: AssociatedMessageType
MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.2.5 Update Survivability Risk Definition

Mission Use Case: ALR

The optional Update Survivability Risk Definition sequence diagram specifies the interaction where C2 sends an updated Survivability Risk Definition to a Lead ACP or non-package ACP. C2 sends *SurvivabilityRiskLevelMT* with the *ObjectState* field set to *UPDATED*. The message has a *SurvivabilityRiskLevelID* field to uniquely identify the message and a *RiskSetting* field, which describes what the different risk settings (*HIGH*, *MEDIUM*, *LOW*, *NONE*) mean with respect to the *MaximumContinuousExposure* for a given Vulnerability Type. The *AppliesToSystemID* can be left blank, since the *MA_AcceptableLevelOfRiskCommandMT* will need to be used to activate a given Survivability Risk Definition and will specify the applicable System or Package IDs.

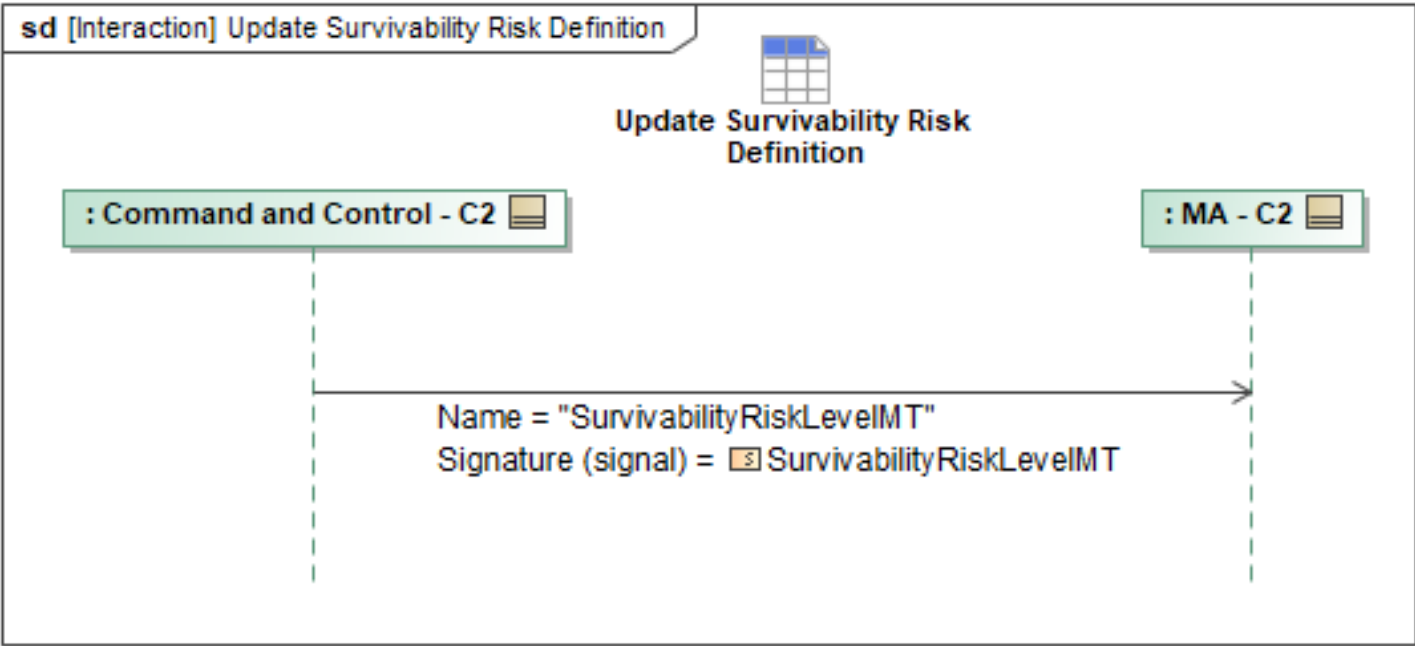


Figure 5-176: Update Survivability Risk Definition

Table 5-175: Update Survivability Risk Definition

Message Name (Name:Type)	Required Fields (Name:Type)
SurvivabilityRiskLevelMT: SurvivabilityRiskLevelMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.3 C2 Constraint Priority List Behaviors

5.2.12.3.1 Activate Constraint Priority List

Overview

The Constraint Priority List (CPL), *MA_ConstraintPriorityListMT*, allows an operator to set default

values for the *Priority* field in *OpZoneMT*, *OpVolumeMT*, and *OpLineMT* (further referred to as geozones) messages based on type. If the *Priority* field in those pre-planned messages are empty and the type is in the CPL, the *Priority* value from the CPL shall be used. A *Priority* value of zero (0) is the highest priority value. If the geozone message *Priority* field is empty and the geozone type is not in the CPL, the *Priority* field shall be set to the *DefaultPriority* from the CPL. If the *DefaultPriority* field is empty, the value would be blank or the highest priority ("above" 0). The *DefaultPriority* and *ListItem* fields are optional and if they are left blank, this indicates that only directly set *Priority* values in the geozones will be utilized and there is no default priority value. The C2 will send messages related to the CPL to a package lead or non-package ACPs. Package leads will distribute the messages to the rest of the follower ACPs. The *MA_ConstraintPriorityListCommandMT* message will identify which *MA_ConstraintPriorityListActivationDataMT* should be active. The *MA_ConstraintPriorityListActivationDataMT* defines a default CPL and CPLs associated with *OpZones* or *OpVolumes*. Only a single CPL shall be active at any given time. When the ACP is in an *OpZones* or *OpVolumes* associated with engagement criteria, it should be activated; otherwise, the default CPL would be activated.

Mission Execution

Prior to planning or execution, the *ActivationData*, CPLs, and associated geozones will be available for use. During planning, if there is an active non-empty *MA_ConstraintPriorityListMT*, the planner will assign the *Priority* values to the geozones as the types that are listed in the CPL, if they were not previously set. The planner will use the *Priority* values in the geozones to make decisions on routes and other items as needed. During mission execution, if there is a change to the active *MA_ConstraintPriorityListMT* message, due to a new geozone being entered or C2 sending a new/updated *MA_ConstraintPriorityListCommandMT* message, MA should trigger a replan. A replan is not necessary if it is determined that the updated *MA_ConstraintPriorityListMT* will not affect the current plan by no active geozones changing priority value. A *MA_ConstraintPriorityListActivationStatusMT* message will be sent upon the transition to a different *MA_ConstraintPriorityListMT*.

Sequence Description

The Activate Constraint Priority List sequence diagram below specifies the interaction of C2 commanding MA to activate one of its stored CPL activation schema. First C2 sends a *MA_ConstraintPriorityListCommandMT* message to MA. The *AppliesToID* field can have multiple entries of *PackageID* or *SystemID*. This allows a C2 to send a Lead MA the Package ID (or multiple sub Package IDs) or a set of System IDs. The *MA_ConstraintPriorityListCommandMT* message shall specify the *ConstraintPriorityListActivationDataID* field containing the ID of the Constraint Priority List Activation Data message (or schema) that C2 wishes to be active. Any new *MA_ConstraintPriorityListCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_ConstraintPriorityListCommandMT* is sent with the *CommandState* set to *CANCEL*, this will cancel the active command and CPL activation schema.

In response to receiving the command, MA shall send *MA_ConstraintPriorityListCommandStatusMT_1* to C2. The *CommandProcessingState* will be set to *RECEIVED* to indicate receipt of the command. For this status message and all other status messages (*MA_ConstraintPriorityListCommandStatus* & *MA_ConstraintPriorityListActivationStatus*), the *AppliesToID* field shall be set to the Package ID if MA is the leader and has received a status message from all members of the Package ID, otherwise it shall be set to System ID(s) and/or sub Package ID(s) that the status message applies to.

The MA shall then use the *ConstraintPriorityListActivationDataID* to query its onboard MP to retrieve the *MA_ConstraintPriorityListActivationDataMT* (see "Query Constraint Priority List Activation Data"). Then it will use the *MA_ConstraintPriorityListActivationDataMT* to query for all the CPL messages (see "Query Constraint Priority List"). If the MA is a non-package ACP, it shall send *MA_ConstraintPriorityListCommandStatusMT_2* to C2 with the *CommandProcessingState* set to *ACCEPTED*.

The MA shall use the ACP Location and the *MA_ConstraintPriorityListActivationDataMT* to determine which CPL ID should be activated (default or geozone based). Once doing so, MA will send *MA_ConstraintPriorityListActivationStatusMT_1* to C2 containing the *ConstraintPriorityListActivationDataID* and the *ActiveConstraintPriorityListID*. Optionally, if MA is the lead ACP, then MA shall message its follower ACPs with the *MA_ConstraintPriorityListCommandMT* (See "Activate Peer Constraint Priority List"). After receiving the respective responses from the follower ACPs, MA (the lead ACP) would send C2 the *MA_ConstraintPriorityListCommandStatusMT_3* with the *CommandProcessingState* set to *ACCEPTED* and the *MA_ConstraintPriorityListActivationStatusMT_2* both reflecting the *AppliesToID* as needed.

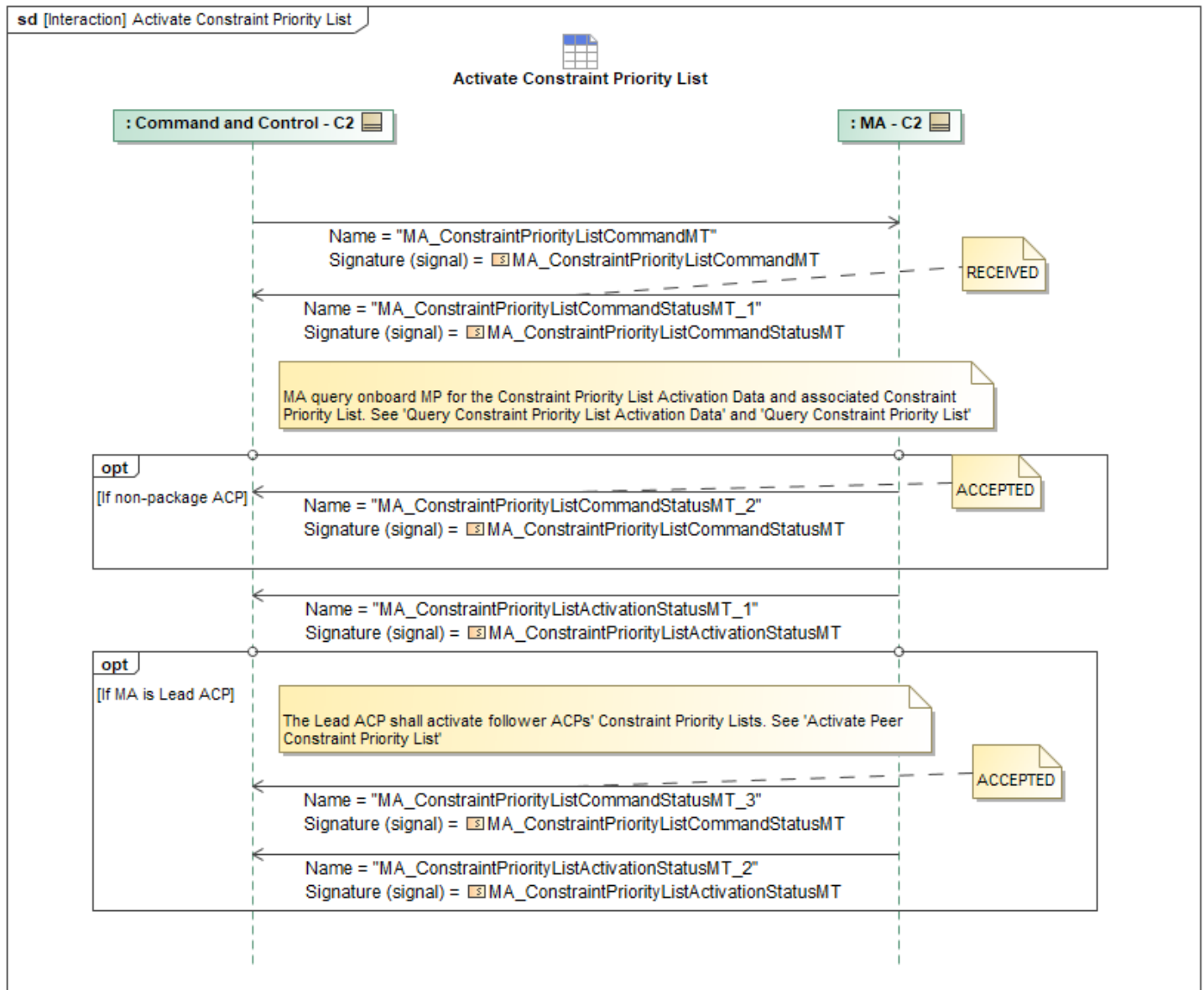


Figure 5-177: Activate Constraint Priority List

Table 5-176: Activate Constraint Priority List

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationStatusMT_1: MA_ConstraintPriorityListActivationStatusMT	None
MA_ConstraintPriorityListActivationStatusMT_2: MA_ConstraintPriorityListActivationStatusMT	None
MA_ConstraintPriorityListCommandMT: MA_ConstraintPriorityListCommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListCommandStatusMT_1: MA_ConstraintPriorityListCommandStatusMT	None
MA_ConstraintPriorityListCommandStatusMT_2: MA_ConstraintPriorityListCommandStatusMT	None
MA_ConstraintPriorityListCommandStatusMT_3: MA_ConstraintPriorityListCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.3.2 Update Constraint Priority List Activation Status

The Update Constraint Priority List Activation Status sequence diagram specifies the MA notifying C2 of a new active CPL after entering a new geozone. Prior to this interaction if this is the Lead ACP, all follower ACPs shall message the lead ACP their activation statuses. If this is the Lead ACP, it should wait for all package members to respond. MA then sends C2

MA_ConstraintPriorityListActivationStatusMT containing the *ConstraintPriorityListActivationDataID* and the *ActiveConstraintPriorityListID*. The *AppliesToID* shall be set to the Package ID if it is the lead and if all follower status have been received, or it shall be set to System ID(s) and/or sub Package ID(s) that the status message applies to.

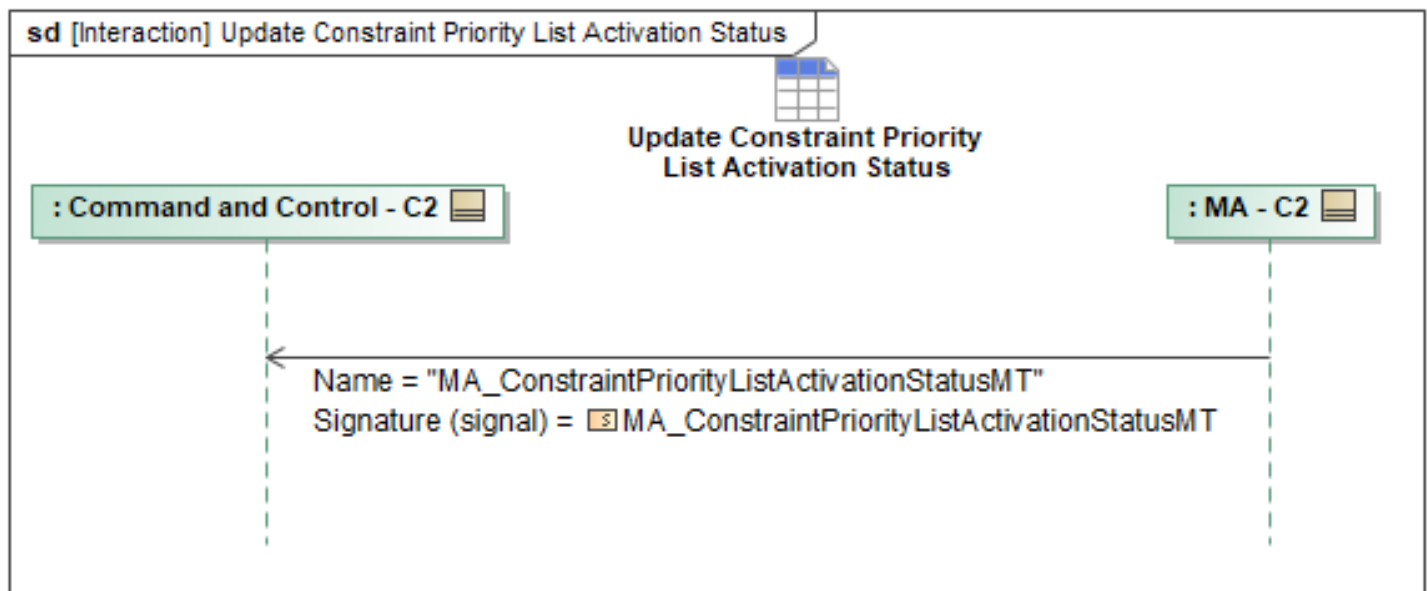


Figure 5-178: Update Constraint Priority List Activation Status

Table 5-177: Update Constraint Priority List Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationStatusMT: MA_ConstraintPriorityListActivationStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.4 C2 Execution Priority List Behaviors

5.2.12.4.1 Activate Execution Priority List

Overview

The Execution Priority List (EPL), *MA_ExecutionPriorityListMT*, allows an operator to set default values for the *Rank* field in the *TaskMT* and *ActionMT* (further referred to as task) messages based on the type. If the *Rank* field in those messages is empty and the type is in the EPL, the *Rank* from the EPL will be used. A *Rank* → *Priority* value of zero (0) is the highest priority value, and an empty field is the lowest. If the planned task message *Rank* field is empty and the task type is not in the EPL, the *Rank* field shall be left empty and have the lowest priority. The C2 will send messages to a package lead or non-package ACPs. Package leads will distribute the messages to the rest of the follower ACPs. The *MA_ExecutionPriorityListCommandMT* message will identify which *MA_ExecutionPriorityListActivationDataMT* should be active. The *MA_ExecutionPriorityListActivationDataMT* defines a default EPL and EPLs associated with *OpZones* or *OpVolumes*. Only a single EPL shall be active at any given time. When the ACP is in an *OpZones* or *OpVolumes* associated with an EPL, it should be activated otherwise the default EPL would be activated.

Mission Execution

Prior to planning or execution, the ActivationData, EPLs, and associated geozones will be available for use. During planning, if there is an active non-empty *MA_ExecutionPriorityListMT*, the planner will assign the *Rank* values to the tasks as the types that are listed in the EPL, if they were not previously set. Child tasks shall inherit the *Rank* value associated with a parent task. The *Rank* value does not explicitly specify the execution order of the tasks. During dynamic re-planning, if a team or an ACP is assigned a collection of tasks to execute, the resulting dynamic planning process should prioritize those tasks and actions based on their priority values. During mission execution, if there is a change to the active *MA_ExecutionPriorityListMT* message, due to a new geozone being entered or C2 sending a new/updated *MA_ExecutionPriorityListCommandMT* message, MA should trigger a replan. A replan is not necessary if it is determined that the updated *MA_ExecutionPriorityListMT* will not affect the active plan by no active tasks changing rank value or the changes do not affect the task sequence order. A *MA_ExecutionPriorityListCommandStatusMT* message will be sent upon the transition to a different *MA_ExecutionPriorityListMT*.

Sequence Description

The Activate Execution Priority List (EPL) sequence diagram below specifies the interaction of C2 commanding MA to activate one of its stored EPL activation schema. First C2 sends a *MA_ExecutionPriorityListCommandMT* message to MA. The *AppliesToID* field can have multiple entries of *PackageID* or *SystemID*. This allows a C2 to send a Lead MA the Package ID (or multiple sub Package IDs) or a set of System IDs. The *MA_ExecutionPriorityListCommandMT* message shall specify the *ExecutionPriorityListActivationDataID* field containing the ID of the Execution Priority List Activation Data message (or schema) that C2 wishes to be active. Any new *MA_ExecutionPriorityListCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_ExecutionPriorityListCommandMT* is sent with the *CommandState* set to *CANCEL*, this will cancel the active command and EPL activation schema.

In response to receiving the command, MA shall send *MA_ExecutionPriorityListCommandStatusMT_1* to C2. The *CommandProcessingState* will be set to *RECEIVED* to indicate receipt of the command. For this status message and all other status messages. (*MA_ExecutionPriorityListCommandStatus* & *MA_ExecutionPriorityListActivationStatus*), the *AppliesToID* field shall be set to the Package ID if MA is the leader and has received a status message from all members of the Package ID, otherwise it shall be set to System ID(s) and/or sub Package ID(s) that the status message applies to

The MA shall then use the *ExecutionPriorityListActivationDataID* to query its onboard MP to retrieve the *MA_ExecutionPriorityListActivationDataMT* (see "Query Execution Priority List Activation Data"). Then it will use the *MA_ExecutionPriorityListActivationDataMT* to query for all the EPL messages (see "Query Execution Priority List"). If the MA is a non-package ACP, it shall send *MA_ExecutionPriorityListCommandStatusMT_2* to C2 with the *CommandProcessingState* set to *ACCEPTED*.

The MA shall use the ACP Location and the *MA_ExecutionPriorityListActivationDataMT* to determine which EPL ID should be activated (default or geozone based). Once doing so, MA will send *MA_ExecutionPriorityListActivationStatusMT_1* to C2 containing the *ExecutionPriorityListActivationDataID* and the *ActiveExecutionPriorityListID*. Optionally, if MA is the lead ACP, then MA shall message its follower ACPs with the *MA_ExecutionPriorityListCommandMT* (See "Activate Peer Execution Priority List"). After receiving the respective responses from the follower ACPs, MA (the lead ACP) would send C2 the *MA_ExecutionPriorityListCommandStatusMT_3* with the *CommandProcessingState* set to *ACCEPTED* and the *MA_ExecutionPriorityListActivationStatusMT_2* both reflecting the *AppliesToID* as needed.

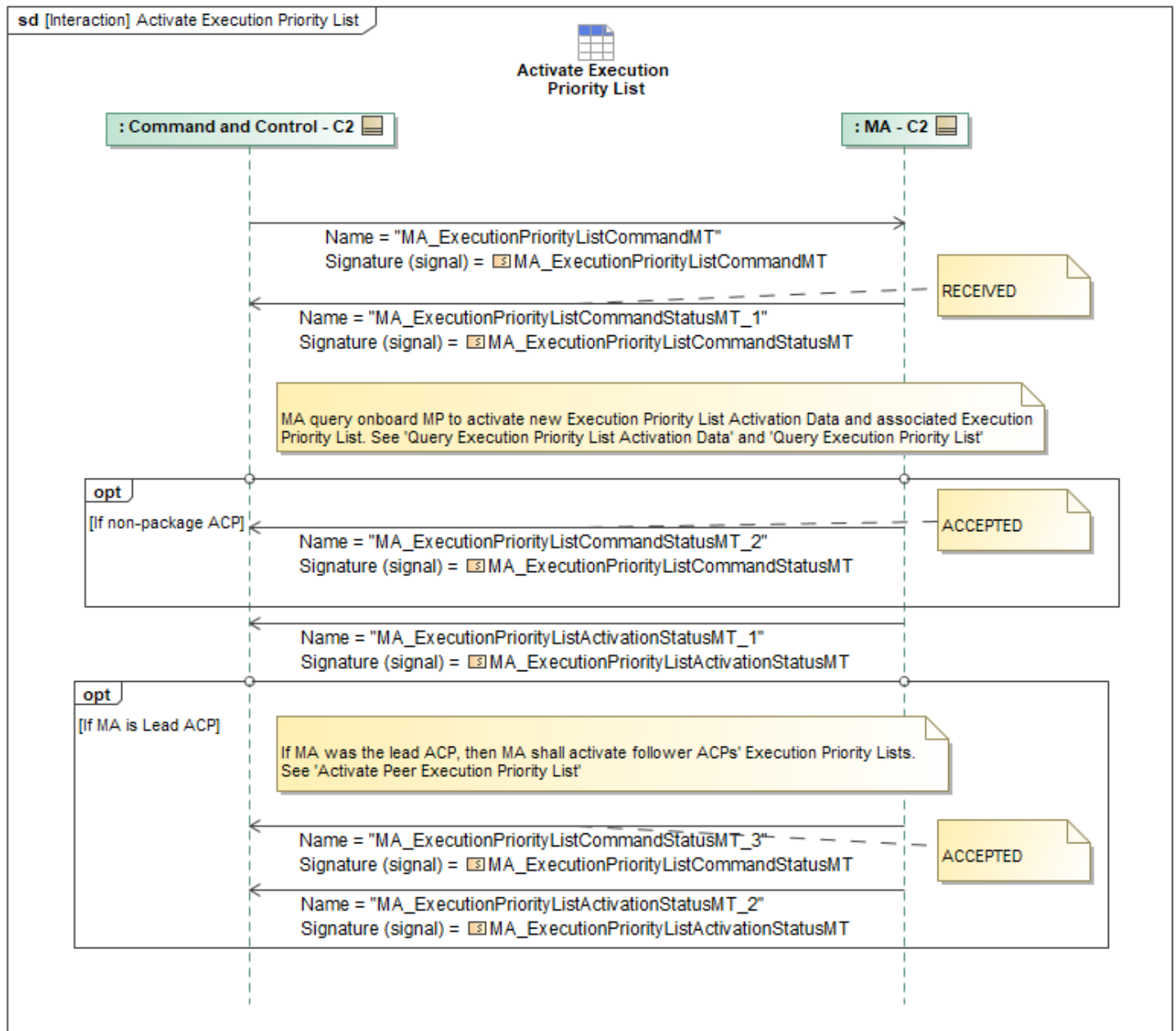


Figure 5-179: Activate Execution Priority List

Table 5-178: Activate Execution Priority List

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationStatusMT_1 : MA_ExecutionPriorityListActivationStatusMT	None
MA_ExecutionPriorityListActivationStatusMT_2 : MA_ExecutionPriorityListActivationStatusMT	None
MA_ExecutionPriorityListCommandMT: MA_ExecutionPriorityListCommandMT	None
MA_ExecutionPriorityListCommandStatusMT_1 : MA_ExecutionPriorityListCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListCommandStatusMT_2 : MA_ExecutionPriorityListCommandStatusMT	None
MA_ExecutionPriorityListCommandStatusMT_3 : MA_ExecutionPriorityListCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.4.2 Update Execution Priority List Activation Status

The Update Execution Priority List Activation Status sequence diagram specifies the MA notifying C2 of a new active EPL after entering a new geozone. Prior to this interaction if this is the Lead ACP, all follower ACPs shall message the lead ACP their activation statuses. If this is the Lead ACP, it should wait for all package members to respond. MA then sends C2

MA_ExecutionPriorityListActivationStatusMT containing the *ExecutionPriorityListActivationDataID* and the *ActiveExecutionPriorityListID*. The *AppliesToID* shall be set to the Package ID if it is the lead and if all follower status have been received, or it shall be set to System ID(s) and/or sub Package ID(s) that the status message applies to.

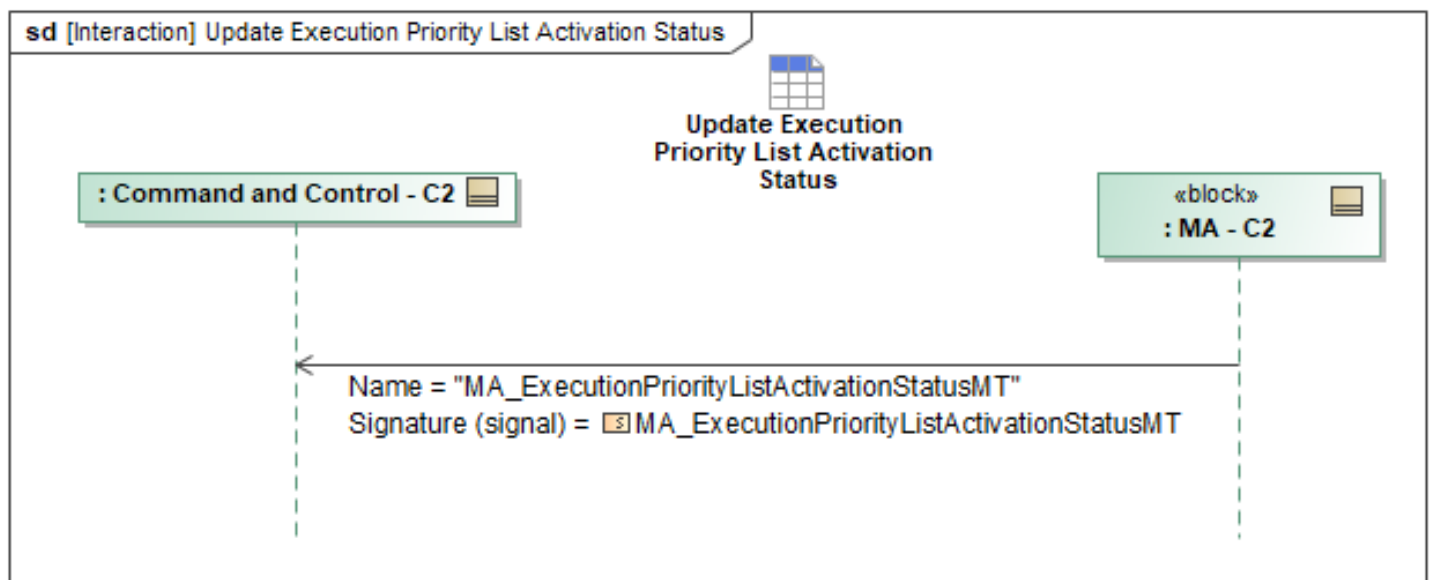


Figure 5-180: Update Execution Priority List Activation Status

Table 5-179: Update Execution Priority List Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationStatusMT: MA_ExecutionPriorityListActivationStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.12.5 C2 Ignore Zone Constraints Behaviors

5.2.12.5.1 Activate Ignore Zone Constraints

Overview

The Ignore Zone Constraints command message (*MA_IgnoreConstraintCommandMT*) allows an operator to specify individual tasks to ignore specific geozones. The task(s) are specified in the *ReferenceType* field and will be one of the following: Effect, Action, Task, Response, or Capability Command. The *ListOfConstraintsToIgnore* field allows setting one or more OpZone or OpVolume ID to be ignored. The *IgnoreAllConstraints* field allows the operator to override any and all constraints as an attempt to accomplish a task. Multiple command messages can be active at the same time and each can be updated or canceled as needed. The *MA_IgnoreConstraintCommandStatusMT* message will be used to provide status of the active commands. The C2 will send messages to a package lead or non-package ACPs. Package leads will distribute the messages to the rest of the follower ACPs.

Mission Execution

During planning, if there is one or more active *MA_IgnoreConstraintCommandMT* message, the planner will ignore the OpZone and/or OpVolumes when planning for the tasks specified in the *ReferenceType* field. During mission execution, if a new or updated *MA_IgnoreConstraintCommandMT* message is received with an active task specified in the *ReferenceType* field, this shall trigger a replan to occur.

Sequence Description

The Activate Ignore Zone Constraints sequence diagram below specifies the interaction of C2 commanding MA to ignore a zone constraint for specified task(s). Note that the intent of this sequence only covers OpZone and OpVolume constraints. First, C2 sends a *MA_IgnoreConstraintCommandMT* message to MA. The *AppliesToID* field can have multiple entries of *PackageID* or *SystemID*. This allows a C2 to send a Lead MA the Package ID (or multiple sub Package IDs) or a set of System IDs. The *MA_IgnoreConstraintCommandMT* message shall specify either a list of zone constraints to ignore (*ListOfConstraintsToIgnore*), or an indicator that all constraints should be ignored (*IgnoreAllConstraints*). It also contains the ID(s) of the task(s) the command applies to (*ReferenceType*). Multiple commands can be active on the MA and each should have a unique *CommandID*. For new commands, the *CommandState* shall be set to *NEW*. For C2 updates to active commands, the *CommandState* shall be set to *UPDATE* and the *CommandID* shall match that of the active message being updated. To cancel an active command, the *CommandState* shall be set to *CANCEL* with *CommandID* set to that of the message being cancelled.

In response to receiving the command, MA shall send *MA_IgnoreConstraintCommandStatusMT_1* to C2. The *CommandProcessingState* will be set to *RECEIVED* to indicate receipt of the command. For this status message the *AppliesToID* field shall be set to the Package ID if MA is the leader and has received a status message from all members of the Package ID. Otherwise, it shall be set to the System ID(s) and/or sub Package ID(s) for which the status message applies.

The MA shall perform the command within MA to ignore the specified constraints for the designated task(s). If the MA is a non-package ACP, it shall send the C2 *MA_IgnoreConstraintCommandStatusMT_2* with *CommandProcessingState* set to *ACCEPTED* to indicate it has completed the command. If MA is the lead ACP, then MA shall message its follower ACPs with the *MA_IgnoreConstraintCommandMT* (See "Activate Peer Ignore Zone Constraints"). After receiving the respective responses from the follower ACPs, MA (the lead ACP) sends C2 the

MA_IgnoreConstraintCommandStatusMT_3 with the *CommandProcessingState* set to *ACCEPTED*, reflecting the *AppliesToID* as needed.

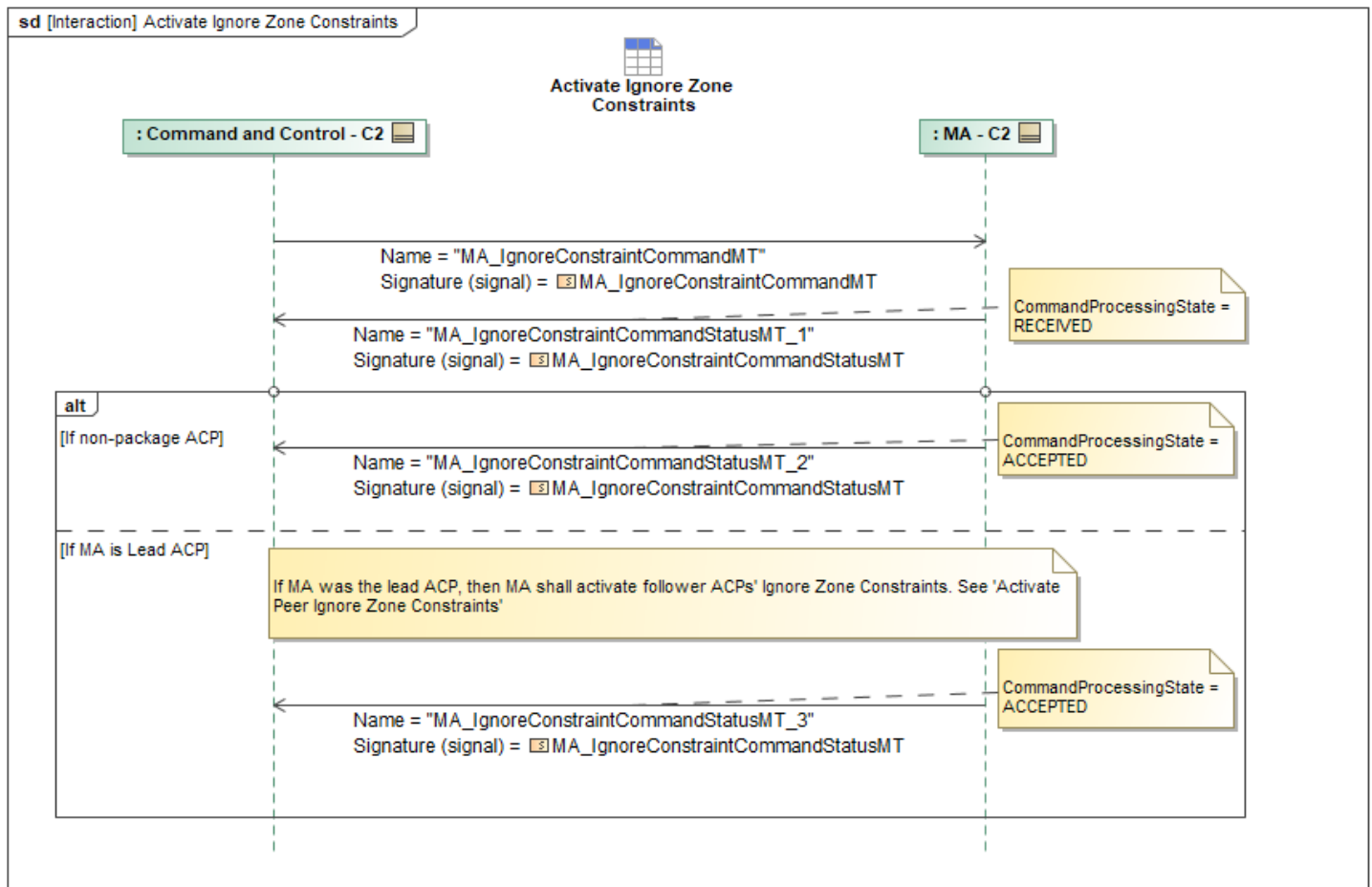


Figure 5-181: Activate Ignore Zone Constraints

Table 5-180: Activate Ignore Zone Constraints

Message Name (Name:Type)	Required Fields (Name:Type)
MA_IgnoreConstraintCommandMT: MA_IgnoreConstraintCommandMT	ListOfConstraintsToIgnore: MA_RegionChoiceType IgnoreAllConstraints: boolean
MA_IgnoreConstraintCommandStatusMT_1: MA_IgnoreConstraintCommandStatusMT	None
MA_IgnoreConstraintCommandStatusMT_2: MA_IgnoreConstraintCommandStatusMT	None
MA_IgnoreConstraintCommandStatusMT_3: MA_IgnoreConstraintCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.13 C2 Teaming Behaviors Package

A “Task” is a defined behavior that can be executed by multiple collaborating ACPs as a Joint System

Task (e.g. Altitude Stacked Marshall), or by a single ACP as a Single System Task (e.g. single vehicle tasks, and Escort assigned to a single formation). A formation is defined and commanded as a task, using *MA_TaskMT* and *MA_TaskCommandMT*. A task may either be directly commanded to invoke immediate execution or be used within a task plan for future execution when the plan is activated. The C2 system invokes a task in two stages. If the task has not yet been uploaded, then it is first uploaded as shown in the Uploading a Task sequence diagram. Then, the task can either be “assigned”, where C2 directly assigns systems, or it can be “delegated”, where the autonomy determines which systems are assigned at execution. Therefore, a task can be assigned using a plan, assigned directly, delegated using a plan, or delegated directly. If multiple C2 systems task the same package or its package members, the C2 RBAC levels and hierarchy dictate the higher priority tasking to execute. For package dissemination, C2 can send tasks, updates, and data to any ACP within a package. If the receiving ACP is not the package leader, the package partner will forward the appropriate messages to the package leader.

5.2.13.1 Changing a Package Leader

The Changing a Package Leader sequence diagram specifies the interaction enabling C2 to reassign the package leader or request a package leader election. If C2 is reassigning the leader, C2 shall send *MA_PackageManagementCommandMT_1* to the Initial Package Leader with the *ChangeType* field set to *UPDATE_PACKAGE_LEADER*. C2 shall specify the new leader with the *PackageLeaderID* field. Otherwise, if C2 is requesting a package leader election, C2 shall send *MA_PackageManagementCommandMT_2* to the Initial Package Leader with the *ChangeType* field set to *REQUEST_LEADER_ELECTION*. C2 shall specify the leader election method with the *PackageLeaderElectionMethod* field (If method is unspecified, the Bully Leader Election will be used). In response to receiving a *MA_PackageManagementCommandMT* message, the Initial Package Leader shall send C2 *MA_PackageManagementCommandStatusMT* confirming or rejecting the package change command. Once the new lead has been established, the New Package Leader shall send package status to C2 according to the "Receive Package Status" sequence. Lastly, the New Package Leader shall send *MA_PackagePartnerStatusReportMT_1*, and the Initial Package Leader (the now previous Package Leader) shall send *MA_PackagePartnerStatusReportMT_2*. These messages are sent to reflect any updates made during the package change.

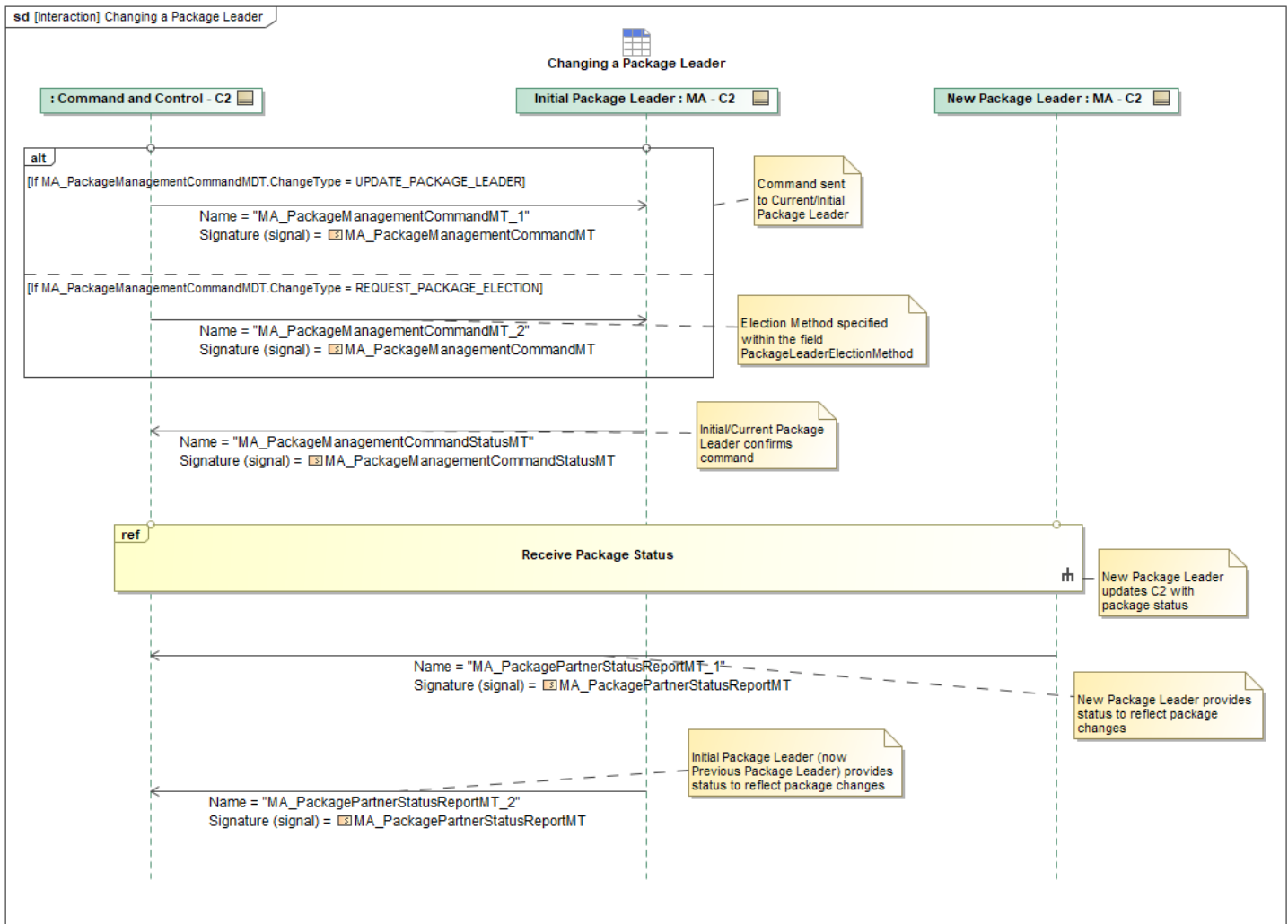


Figure 5-182: Changing a Package Leader

Table 5-181: Changing a Package Leader

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackageManagementCommandMT_1: MA_PackageManagementCommandMT	PackageLeaderID: SystemID_Type
MA_PackageManagementCommandMT_2: MA_PackageManagementCommandMT	None
MA_PackageManagementCommandStatusMT: MA_PackageManagementCommandStatusMT	None
MA_PackagePartnerStatusReportMT_1: MA_PackagePartnerStatusReportMT	PackageLeaderID: SystemID_Type
MA_PackagePartnerStatusReportMT_2: MA_PackagePartnerStatusReportMT	PackageLeaderID: SystemID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.13.2 Command Team/Platform via Task

MA_TaskMT contains different tasks that a system can perform against a specific entity or geographical location. *MA_TaskPlanMT* is the collection of these different Tasks for a team of ACPs and can be optionally sent to the ACP package to assist with commanding coordinated tasking. To command a team via tasking, C2 provides *MA_TaskMT* (and optionally *MA_TaskPlanMT*) messages with the specific *TaskType* field identified to dictate the task that should be performed. If sending the *MA_TaskPlanMT* message, the *PlanCommandID* field can be used to specify the planning command ID used to generate the Task Plan. C2 then sends an *MA_TaskCommandMT* to trigger immediate execution of the task, specifying the *TaskID* and *RequiredAllocation*. If C2 doesn't specify assigned assets and delegates to the ACP Lead to select package members, then *RequiredAllocation* should specify *PackageID*, *PlatformType*, and *PlatformTypeCategory*. If C2 desires to specify which package members are assigned the tasks, then *RequiredAllocation* should include *SystemID* of the ACPs requested. If the receiving MA system is not the Package lead, the Package partner will forward *MA_TaskMT* and *MA_TaskCommandMT* to the Package lead.

MA responds with a *MA_TaskCommandStatusMT* that tells C2 whether the command has been *ACCEPTED* or *REJECTED*. MA will then provide *MA_TaskStatusMT* (and *TaskPlanStatusMT* when a Task Plan was used) that describe the status of the *Task* (and *TaskPlan* when applicable), respectively.

Additionally, if the *MA_TaskCommand* was addressed to a *PackageID*, the associated *MA_TaskCommandStatus* should reflect the *Package ID*. If partial rejections occur during Lead-to-Follower delegation the Lead ACP must revert to individual messages to report multiple rejecting followers.

If C2 allocates a requirement to a specific subset of platforms (via discrete *SystemID* allocations rather than *PackageID*), the Lead ACP shall bypass package-level aggregation and return individual status messages for those tasked members.

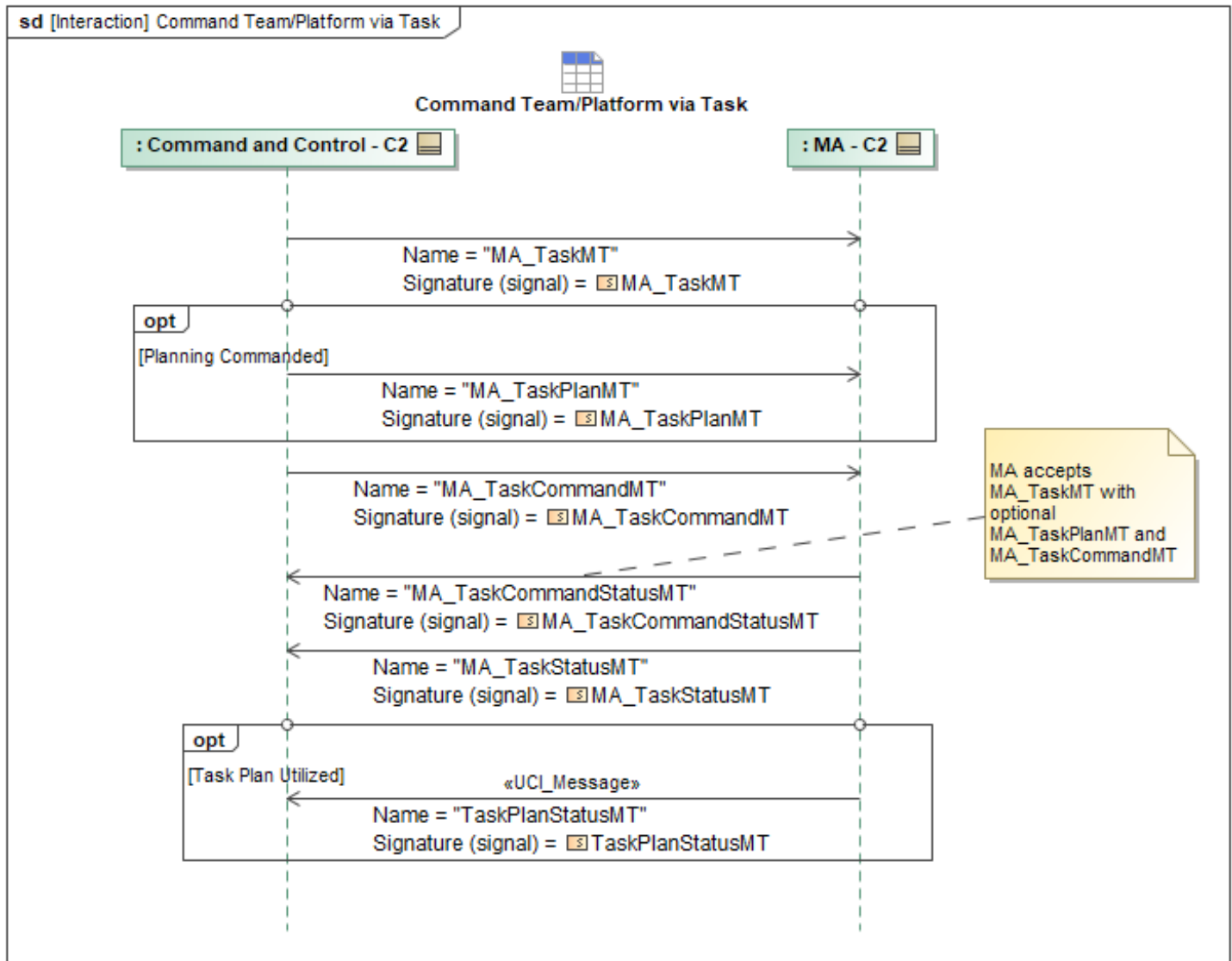


Figure 5-183: Command Team/Platform via Task

Table 5-182: Command Team/Platform via Task

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskCommandMT: MA_TaskCommandMT	RequiredAllocation: MA_RequirementAllocationConstraintType
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None
MA_TaskPlanMT: MA_TaskPlanMT	None
MA_TaskStatusMT: MA_TaskStatusMT	Metrics: MetricsType
TaskPlanStatusMT: TaskPlanStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.13.3 Creating or Changing a Package

The Creating or Changing a Package sequence diagram specifies the interaction enabling C2 to command ACP(s) to form a new package, join an existing package, or leave an existing package. Message *MA_PackageManagementCommandMT* is sent from the C2 system to each Commanded MA ACP(s) (1 or n ACPs, where n is the total number of assets). This message allows one or more Commanded MA ACP(s) to do one of the following based on specifying the *ChangeType* field:

- Form a package (*FORM_NEW_PACKAGE*)
- Join a package (*JOIN_PACKAGE*)
- Leave a package (*LEAVE_PACKAGE*)

The *MA_PackageManagementCommandMT* message may optionally include a *PackageLeaderID* in order to designate the leader when forming a package. The message may also include a list of *ConstraintIDs* to apply to the package. In response to receiving *MA_PackageManagementCommandMT*, each Commanded MA that received a message sends C2 a response *MA_PackageManagementCommandStatusMT* confirming or rejecting the *MA_PackageManagementCommandMT*. If the leader was not designated by C2, all associated ACPs would execute the specified *PackageLeaderElectionMethod* to determine which ACP will be the leader. Once the lead has been established, the Package Leader shall send package status to C2 according to the "Receive Package Status" sequence. The Package Leader will send *MA_PackagePartnerStatusReportMT* as a single message containing the status for all ACPs using the *SystemStatus* field to combine individual members.

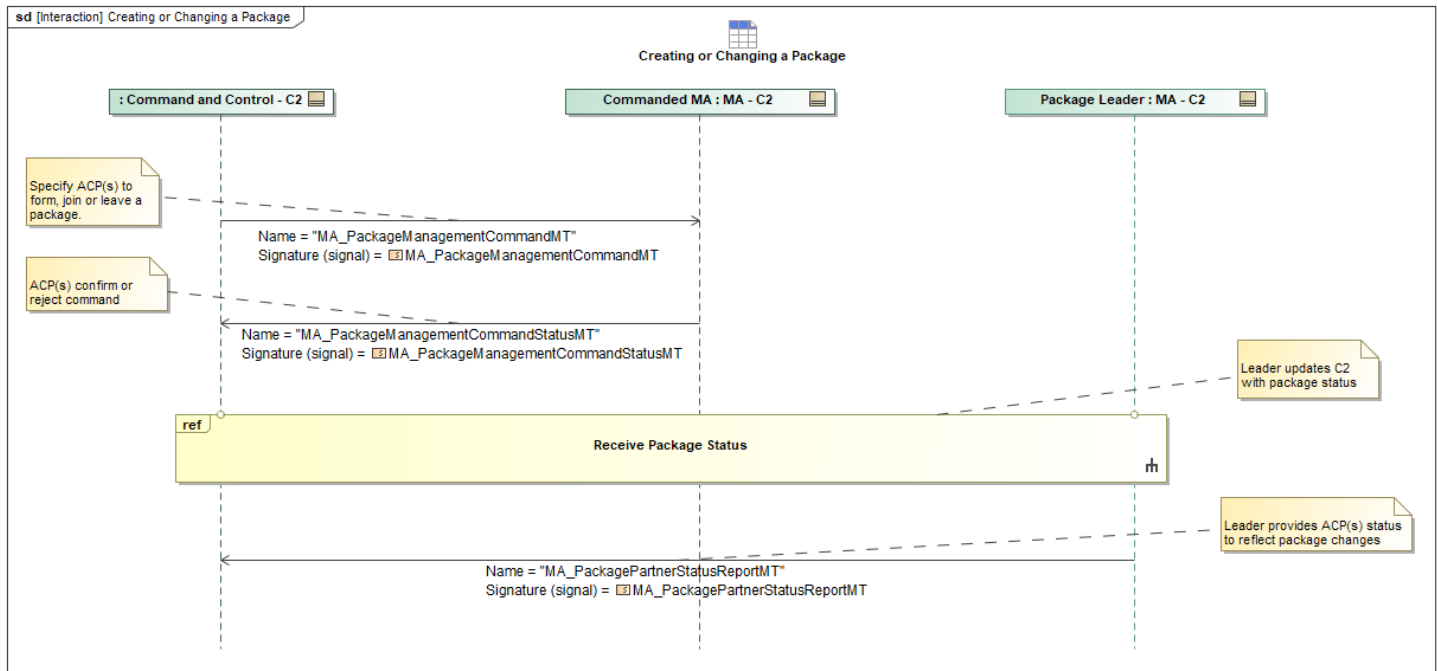


Figure 5-184: Creating or Changing a Package

Table 5-183: Creating or Changing a Package

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackageManagementCommandMT: MA_PackageManagementCommandMT	None
MA_PackageManagementCommandStatusMT: MA_PackageManagementCommandStatusMT	None
MA_PackagePartnerStatusReportMT: MA_PackagePartnerStatusReportMT	PackageLeaderID: SystemID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.13.4 Distribute C2 Track Data to Facilitate Global Fusion and Update COP

This interaction describes how C2 will distribute tracks via *EntityMT* data to MA in order to support global fusion and contribute to the development and maintenance of the common operating picture (COP). C2 will provide any C2 Source tracks and any External source tracks to MA via *EntityMT*, which can then be utilized by Fusion and COP services. *ObservationMeasurementReportMT* will be sent alongside *EntityMT* from MA to C2 to support any Fusion and COP functionality C2 wants to perform.

Within the *ObservationMeasurementReportMT* message, there are two optional paths to different sets of *SubsystemIDs* and *SystemIDs*:

1) *ObservationMeasurementReportMDT* → *ObservationMeasurement* → *MeasurementSource* → *ElementDetails* → *SourceElementIdentifier* → *SystemID* + *SubsystemID*

- These messages provide the system/subsystem that's the source of the OMR's kinematics

2) *ObservationMeasurementReportMDT* → *ObservationMeasurement* → *MeasurementSource* → *SourceIdentity* → *CapabilityID* + *SystemID* + *SubsystemID*

- These messages provide the system/subsystem and capability for the source of identity in the OMR (it's expected this will usually be the same system/subsystem as the kinematics) A client would want to know the capability type of the provided entity ID, and may have to subscribe to additional messages to understand the type (e.g. [Capability]StatusMT messages).

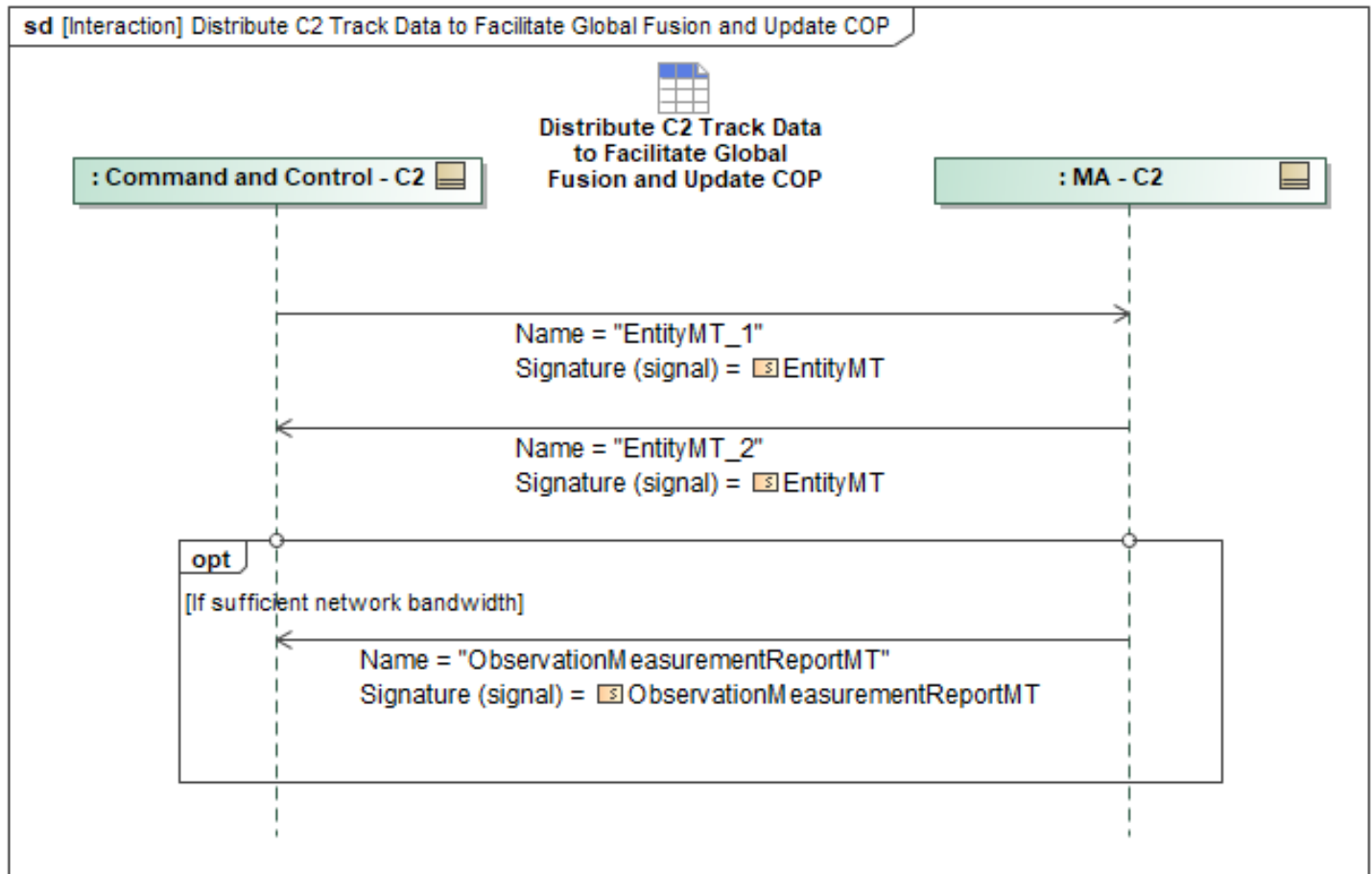


Figure 5-185: Distribute C2 Track Data to Facilitate Global Fusion and Update COP

Table 5-184: Distribute C2 Track Data to Facilitate Global Fusion and Update COP

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT_1: EntityMT	None
EntityMT_2: EntityMT	SourceTypeIdentifier: EntitySourceIdentifierType MeasurementID: MeasurementID_Type
ObservationMeasurementReportMT: ObservationMeasurementReportMT	MeasurementSource: ObservationMeasurementSourceType ElementDetails: ElementDetailsType SourceElementIdentifier: SourceKinematicsElementIdentityType SystemID: SystemID_Type SubsystemID: SubsystemID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
	SourceIdentity: MeasurementSourceIdentityType CapabilityID: CapabilityID_Type SystemID: SystemID_Type SubsystemID: SubsystemID_Type MeasurementKinematics: MeasurementKinematicsChoiceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.13.5 Receive Package Status

This interaction specifies the periodic package status messages which are sent from the Package Leader MA to C2 systems. MA sends *PackageMT* periodically and advertises the fully formed package, with all package member System IDs included in the *PackagePartner* field. MA sends *PackageStatusMT* periodically and contains status information for the package and all package members, including their state in the *PartnerState* field and capabilities in the *PartnerCapabilityStatus* field.

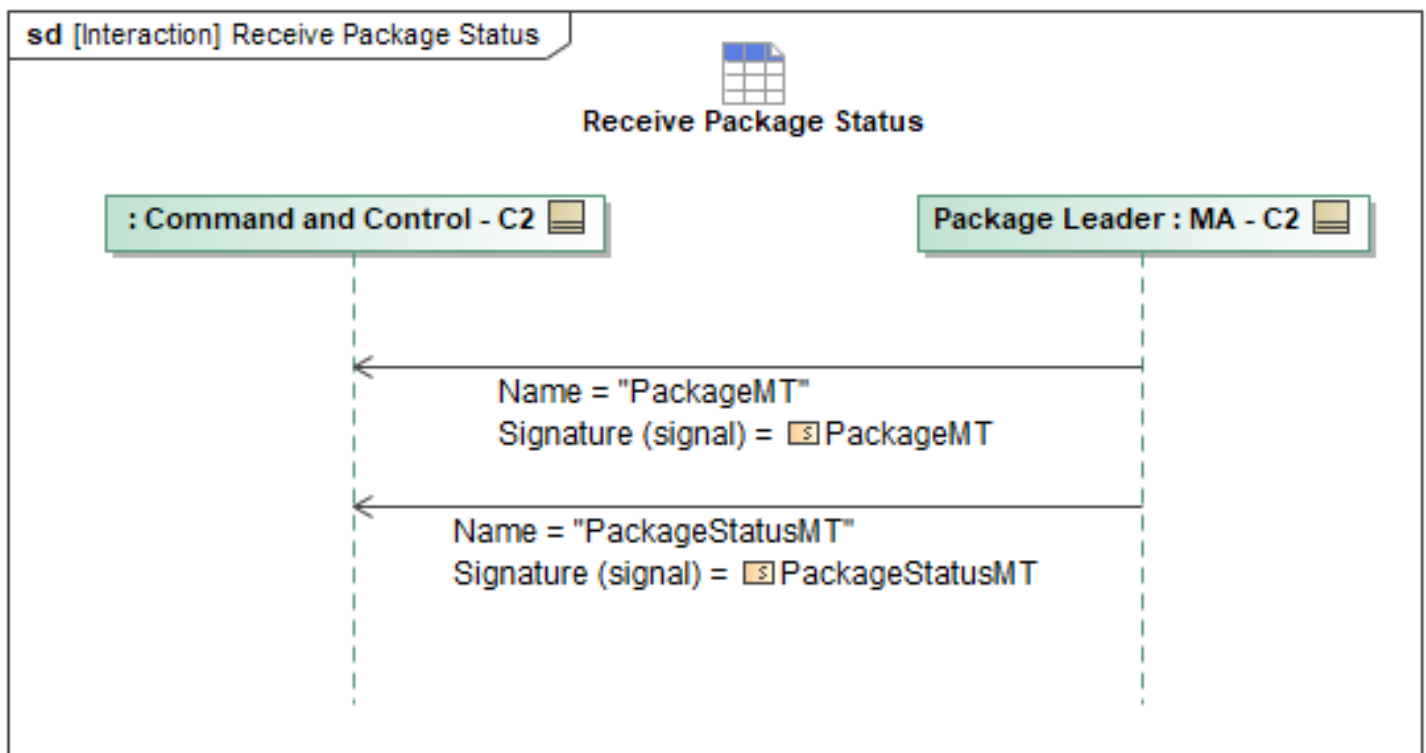


Figure 5-186: Receive Package Status

Table 5-185: Receive Package Status

Message Name (Name:Type)	Required Fields (Name:Type)
PackageMT: PackageMT	LeadSystemID: SystemID_Type
PackageStatusMT: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.13.6 Uploading a Task

To upload any task, the message *MA_TaskMT* is sent containing task parameters and a *TaskID*. No ACK is required for task messages issued. Optional constraint message(s) may be sent linking constraint(s) to the task. The example in the Uploading a Task sequence diagram works for any task type.

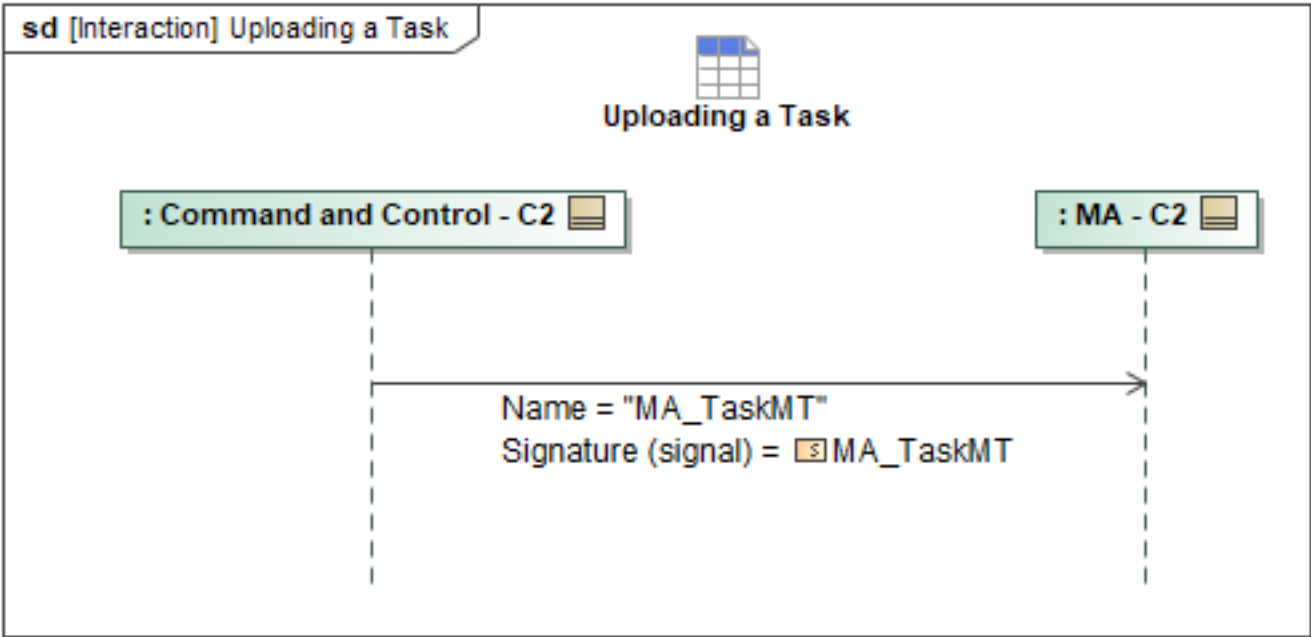


Figure 5-187: Uploading a Task

Table 5-186: Uploading a Task

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.14 C2 Threat Assessment Package

5.2.14.1 C2 Threat Assessment

The Threat Assessment sequence diagram specifies the interactions where an ACP provides C2 with vehicle threat assessment results. C2 may request an ACP to perform vehicle threat assessment(s) for specific Entity ID's or all known red Entity ID's by sending *MA_AssessmentRequest* with respective *VehicleThreatAssessmentRequestType* message. C2 can request the assessment be performed once or periodically at a defined rate by setting the *RequestFrequencyType* to PERIODIC and providing a value in *RequestFrequencyPeriod*. C2 may also request what capability MA will use to perform the vehicle threat assessment (i.e. onboard sensors), however if C2 wants MA to use

historical tracks provided by track management function then the *ThreatCapability* enumeration will be *Other_Unspecified_Detection*. The ACP will respond, once or periodically, with a *MA_AssessmentRequestStatusMT* and, if the request specified *ResultsInAssessmentMessage* to be FALSE, a *MA_AssessmentMT*. If periodic assessments were requested, C2 will send *MA_AssessmentRequestMT* with *RequestFrequencyType* set to STOP. MA will respond with the corresponding *MA_AssessmentRequestStatusMT* to indicate request complete.

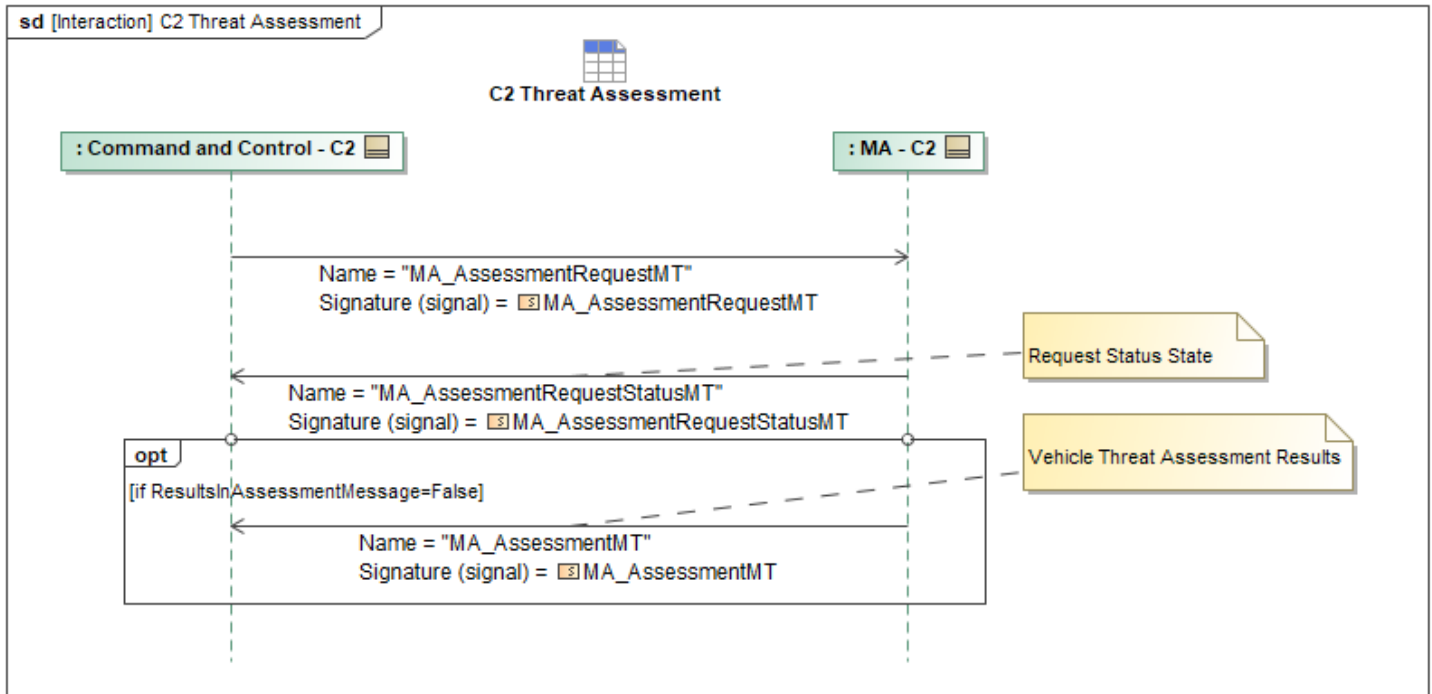


Figure 5-188: C2 Threat Assessment

Table 5-187: C2 Threat Assessment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentMT: MA_AssessmentMT	VehicleThreatAssessment: VehicleThreatAssessmentType ObjectState: ObjectStateEnum AssessmentRequestID: RequestID_Type
MA_AssessmentRequestMT: MA_AssessmentRequestMT	VehicleThreatAssessment: VehicleThreatAssessmentRequestType
MA_AssessmentRequestStatusMT: MA_AssessmentRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15 C2 Weapon Employment Behaviors Package

5.2.15.1 DLZ Compound Convergence

DLZ Compound Convergence is a fusion of multiple DLZ from team members within a flight group. The fusion of DLZ enables C2 to generate a composite view of flight group's weapon engagement

capabilities to assess tasking for the best shot. Once ACPs are producing a DLZ in support of strike task, MA sends DLZ_MT messages to C2 periodically.

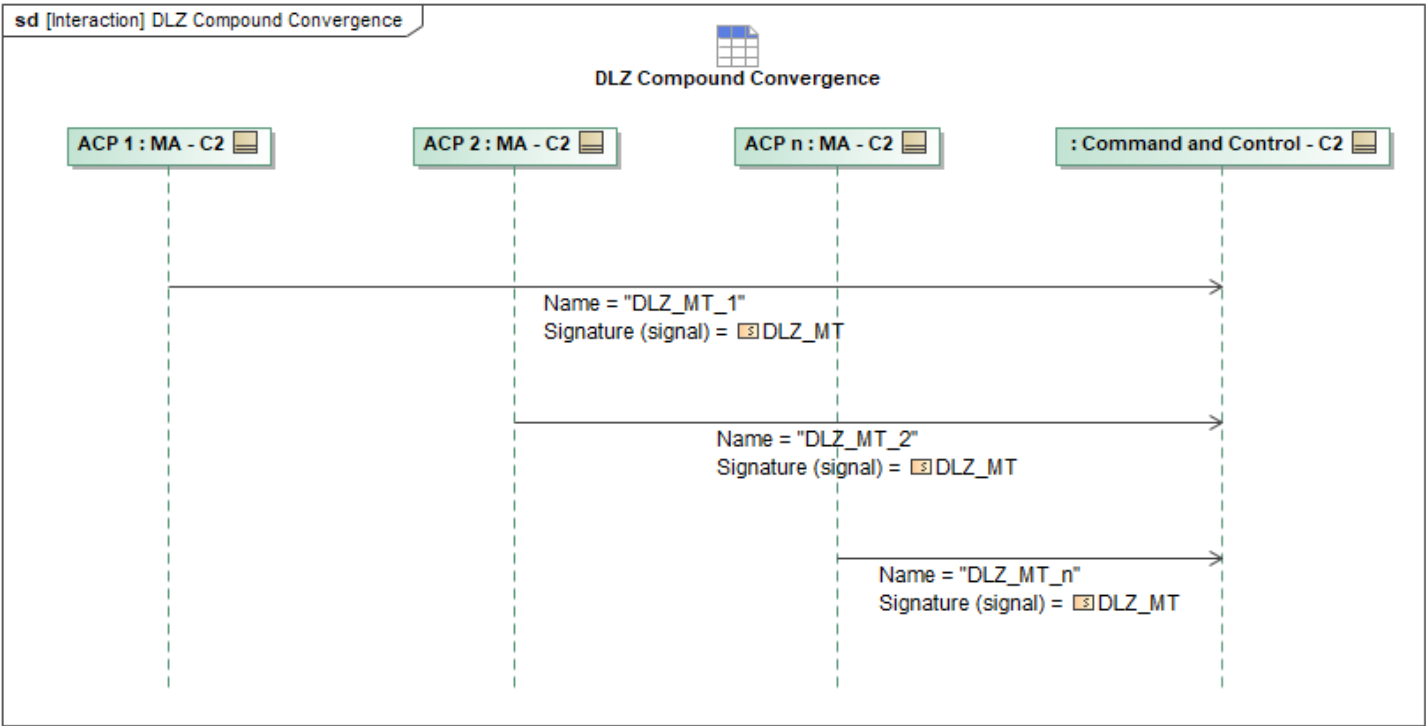


Figure 5-189: DLZ Compound Convergence

Table 5-188: DLZ Compound Convergence

Message Name (Name:Type)	Required Fields (Name:Type)
DLZ_MT_1: DLZ_MT	None
DLZ_MT_2: DLZ_MT	None
DLZ_MT_n: DLZ_MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.2 Exchange Shot Queue Prioritization

Shot Queue Prioritization List is a list of mission targets ranked by assessing multiple factors to determine which target entities should be engaged first. The shot queue will utilize *MA_PrioritizationList* with a *ListType* of *Mission_Target_List (enum)*. The list requires subjects (*EntityID*) to be assigned a priority, where the lowest assigned value is the highest priority level. The message also includes an optional field, *ProbabilityOfKill*, that may be included with each assessed subject entity. MA (Single Ship or Multi-Ship Lead) may send an updated shot queue prioritization list to C2 based on the active engagement criteria.

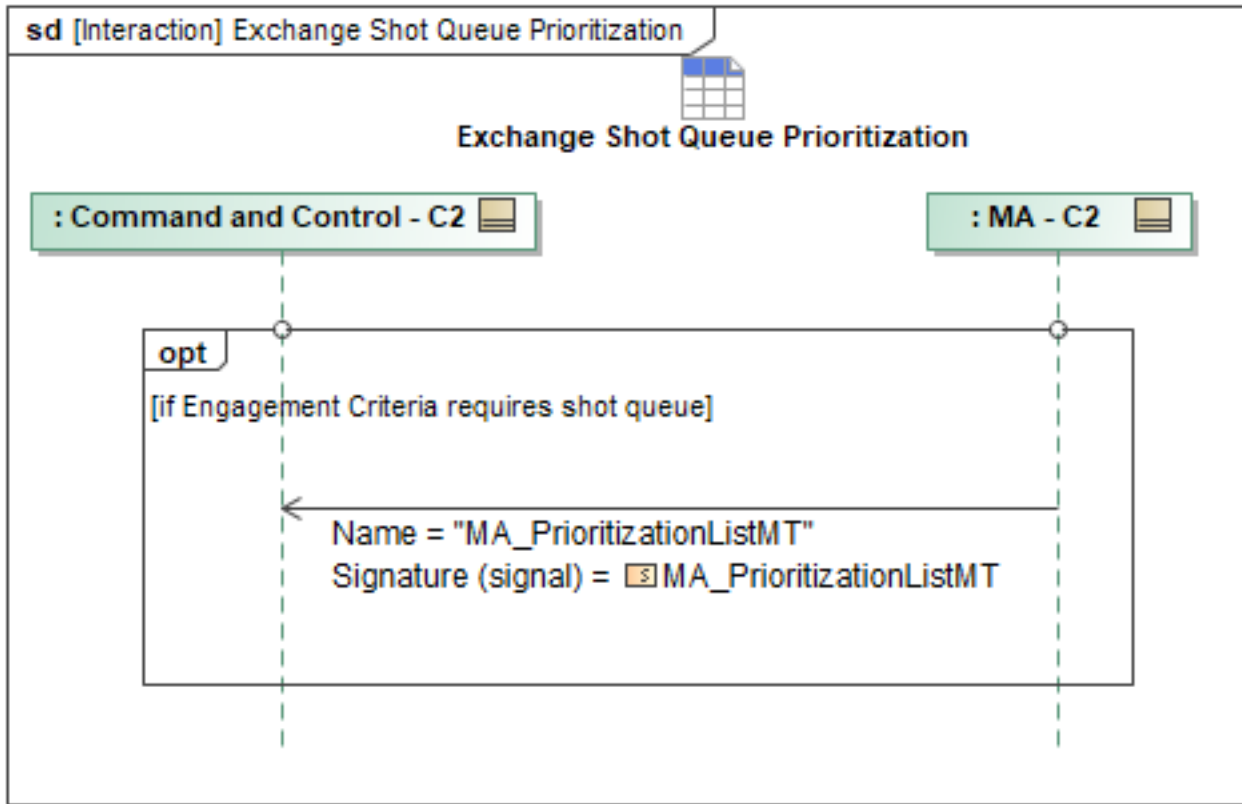


Figure 5-190: Exchange Shot Queue Prioritization

Table 5-189: Exchange Shot Queue Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	MISSION_TARGET_LIST: ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.3 Mission Store Status Exchange

Once mission store status is received through the MS Interface, MA sends *StrikeCapabilityStatusMT* periodically to C2 for each store capability ID regardless of the store presence. MA will also send *MA_StrikeCapabilityMT* periodically. C2 will utilize *StrikeCapabilityStatusMT* and *MA_StrikeCapabilityMT* to maintain persistent awareness of the store state, loaded mission data, and mission store parameters in accordance with the selected Mission Data File (MDF). In the event that MA has an unexpected or undesirable state reported, MA is then required to update *StrikeCapabilityStatus.MessageData.CapabilityStatus.AvailabilityInfo.AvailabilityReason*.

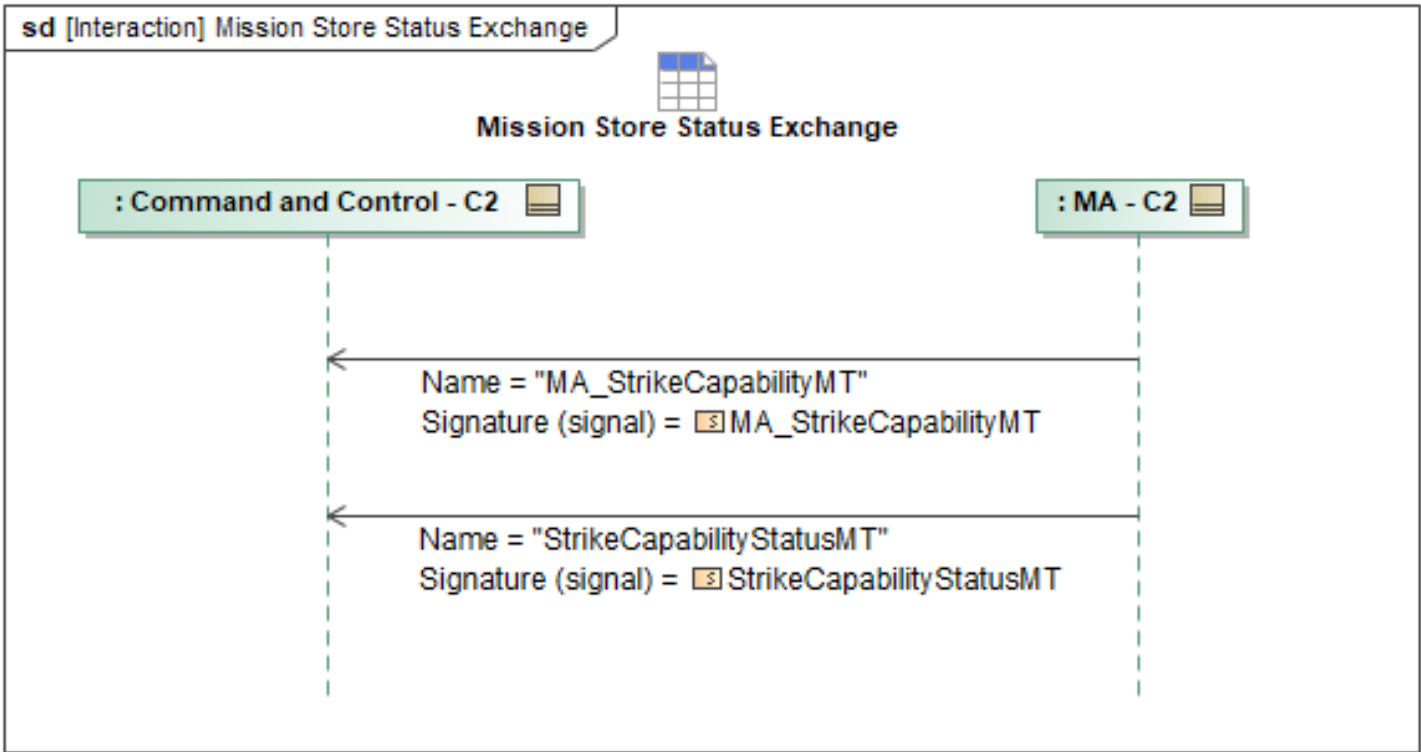


Figure 5-191: Mission Store Status Exchange

Table 5-190: Mission Store Status Exchange

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	StoreStatus: StoreStatusType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.4 Query Entity Track History

The Track Management functionality for multi-ship/single-ship weapon employment is to maintain target tracks historical data along with entity predictions data communicated from MA to C2 when requested using a data query request message. C2 queries the MA node to get the historical and prediction track data. The track data may be used by other functions such as sending external track reports, stimulating alerts/notifications if rule sets are triggered, and updating the track display, history, and attribute data. MA will be getting Entity data from the COP, so the Track Management Data Fusion engine will take the COP data and its sensor data to build track history. Track propagation service takes the track state updates and predicts the extrapolated position (Kinematics) of an entity in the battlespace and maintains these as a database within the MA.

The query for entity and propagation data from C2 to MA will be using the existing UCI message : *QueryDataRequest*.

- *ResultsInNative* field needs *MessageType* & *ReceiverAddress* for querying *EntityPropagation*

historical data from C2 to MA. This would not serve the purpose since it doesn't specify *EntityId(s)* that need to be queried.

- *QueryMessage* field *MessageType* & *Query* PET (Polymorphic Extension Type) for querying *Entity* and *EntityPropagation* historical data from C2 to MA.

To create a query, *QueryDataRequest* UCI message uses *QueryMessage* field. For example, *QueryDataRequestMT* (starting from the query element) shows querying for *EntityID* from the historical Entity messages stored by MA's Entity Management Service. The query can be combined for querying both *Entity* & *EntityPropagation* messages (historical) in the *QueryType* for *entityid*.

C2 receives *QueryDataRequestStatusMT_1* response from the MA node with intermediate response of *RequestProcessingState* field set to *QUEUED*, followed by *QueryDataRequestStatusMT_2* response with *RequestProcessingState* set to *PROCESSING*. Upon successful query for the entity track management or propagation data, MA would receive *QueryDataRequestStatusMT_3* message with the entity database along with the *RequestProcessingState* set to *COMPLETED*. A failure response can result in a *QueryDataRequestStatusMT_4* with *RequestProcessingState* set to *FAILED* or *REJECTED* or *CANCELED* from MA node to C2. For any non-compliance related issues, *QueryDataRequestStatusMT_4* response is sent with *RequestProcessingStateReason* set to appropriate non-compliance code.

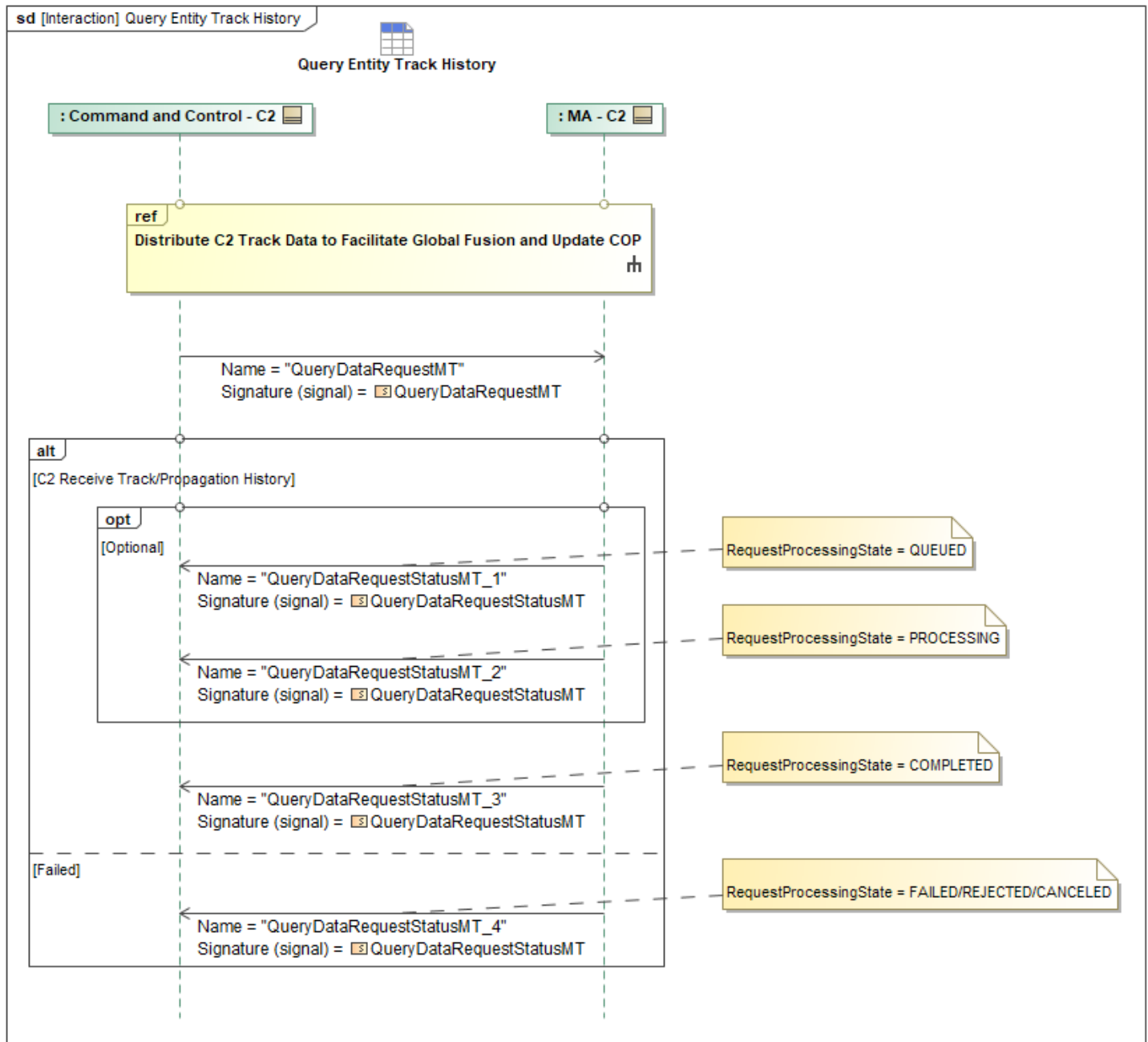


Figure 5-192: Query Entity Track History

Table 5-191: Query Entity Track History

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	QueryMessage: QueryMessageType
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.5 Query Shot Queue Prioritization

Shot Queue Prioritization List is a list of mission targets ranked by assessing multiple factors to determine which target entities should be engaged first. The shot queue will utilize *MA_PrioritizationListMT* with a *ListType* of *MISSION_TARGET_LIST enumeration*. The list requires subjects (EntityID) to be assigned a priority, where the lowest assigned value is the highest priority level. The message also includes an optional field, *ProbabilityOfKill*, that may be included with each assessed subject entity. C2 may query (*QueryDataRequestMT*) an ACP (Single Ship, Multi-Ship Lead, or Multi-Ship Peer) to request *MA_PrioritizationListMT* in its native form. MA will respond with *QueryDataRequestStatusMT* to provide state changes (Queued "*QueryDataRequestStatusMT_1*", Processing "*QueryDataRequestStatusMT_2*", and Completed "*QueryDataRequestStatusMT_3*") in the processing of the request as depicted in the sequence diagram. MA will send *MA_PrioritizationListMT* result in native form to the requesting *SystemID*.

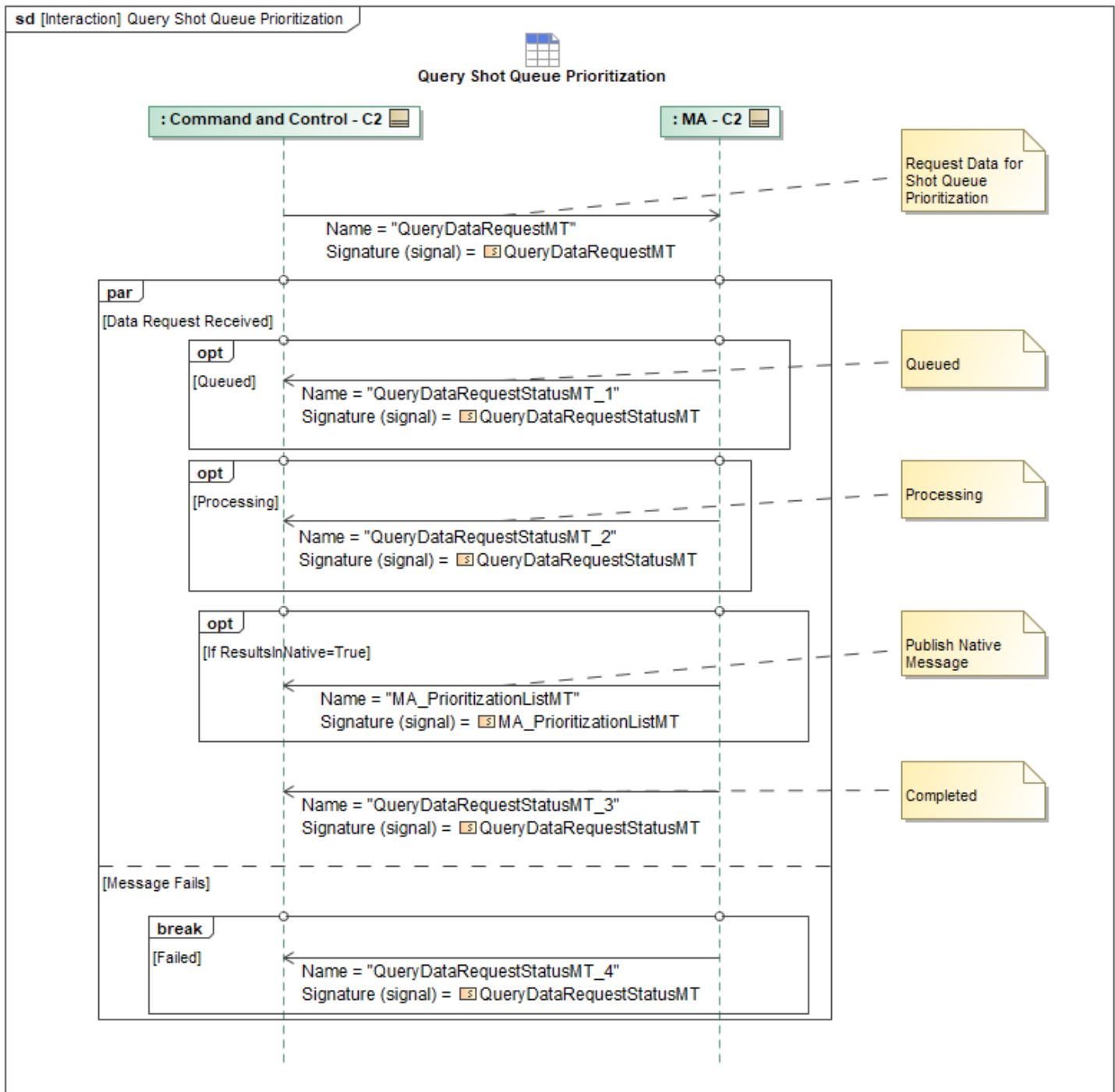


Figure 5-193: Query Shot Queue Prioritization

Table 5-192: Query Shot Queue Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	ObjectState: ObjectStateEnum MISSION_TARGET_LIST:
QueryDataRequestMT: QueryDataRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.6 Release Consent

In the following sequence, MA enforces the Weapons Authorization Policy by checking its Authorization list, compiled from receiving Weapons Authorization Engagement Criteria messages previously from C2 or MP.

- Once the appropriate "authorizers" have been determined, MA sends a *StrikeConsentRequestMT* to C2 to obtain final authorization to fire
- C2 responds with *StrikeConsentRequestStatusMT*

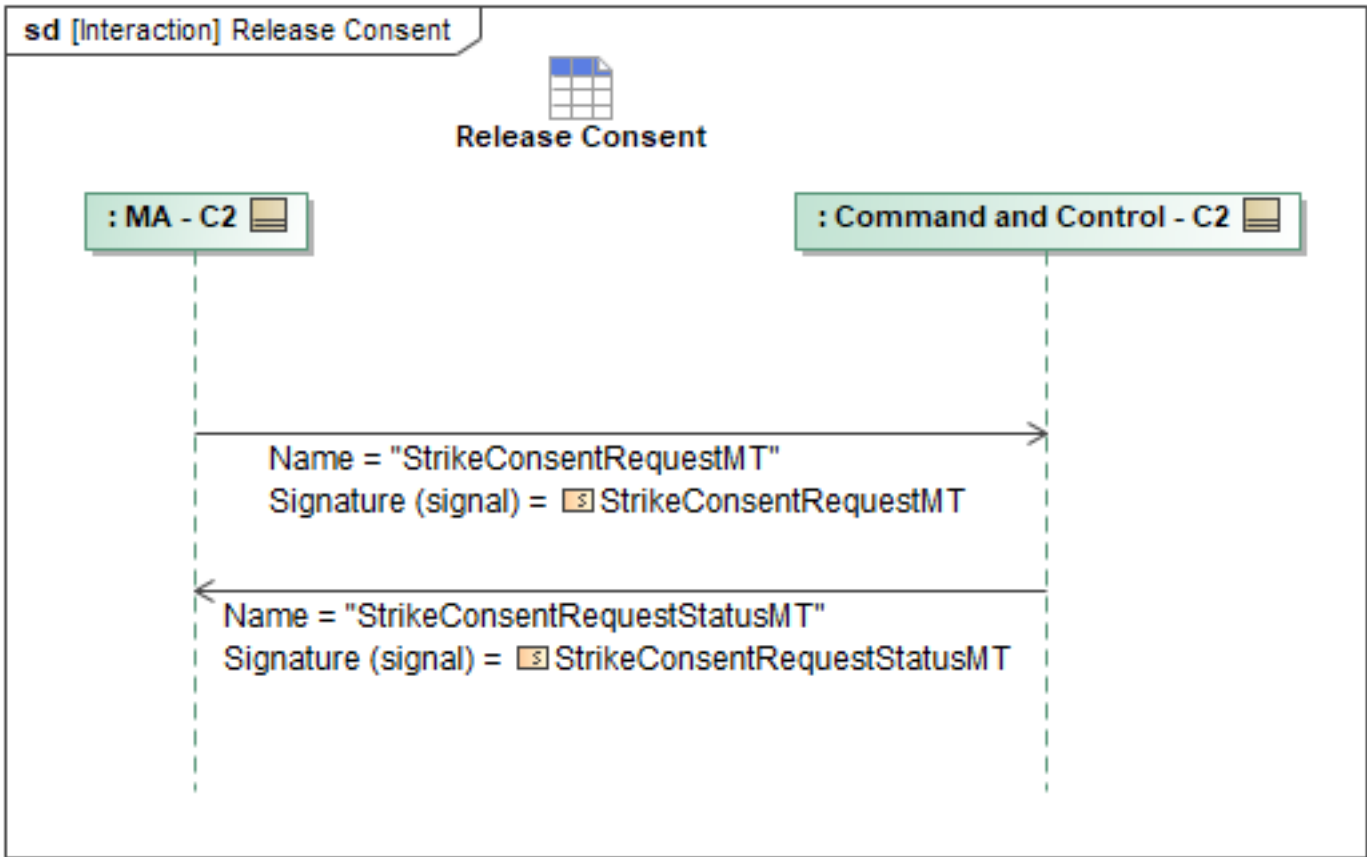


Figure 5-194: Release Consent

Table 5-193: Release Consent

Message Name (Name:Type)	Required Fields (Name:Type)
StrikeConsentRequestMT: StrikeConsentRequestMT	None
StrikeConsentRequestStatusMT: StrikeConsentRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.7 Report Intercept Pairings

The intercept pairings of an engagement describe the individual targets that each ACP will do tactical maneuvers towards. The assigned targets may be a single or group of enemy tracks. Whenever the package lead MA assigns target pairings it will determine the tactical maneuvers employed by each ACP. This sequence describes the messages from MA to C2 to report back that tactical assignment. This would occur after a Lead ACP has assigned the intercept pairings amongst a package a received a positive acknowledgment back from the "Peer Assign Target Pairings" interaction.

The selected intercept pairings should follow the intercept maneuver parameters included in the engagement criteria by C2 prior to an engagement described in more detail in section 4.2.2.8.5 Update Intercept Parameters. If C2 commands a targeting action to MA and intercept pairings have

been calculated, MA will return the *RelationshipDesignationMT* message back to inform C2 of the assigned intercept pairings for each ACP to target. In *RelationshipDesignationMT*, required fields for this sequence are the *Source* field set to the *SystemID* of the ACP, the *Destination* field set to the *EntityID* of the track being targeted, and the *Relationship.EngagementStatus* field set to *ENGAGING* to denote that the target is being intercepted.

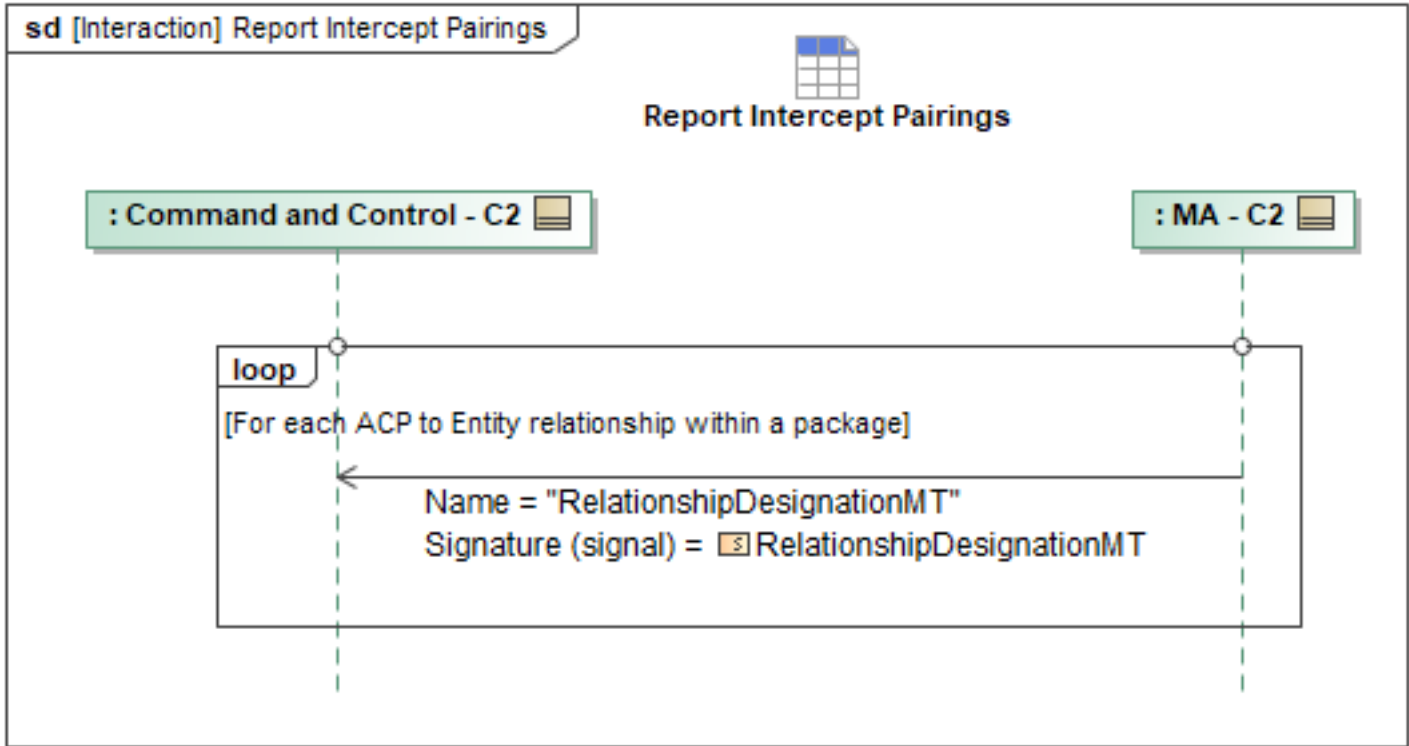


Figure 5-195: Report Intercept Pairings

Table 5-194: Report Intercept Pairings

Message Name (Name:Type)	Required Fields (Name:Type)
RelationshipDesignationMT: RelationshipDesignationMT	Relationship: RelationshipType Destination: GeoLocatedObjectType EngagementStatus: ExternalCommandExecutionStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.8 Report Shot Allocations

An ACP's shot allocation(s) describe targeting or shooting tactics commanded by C2 that have resulted in MA allocating one or more shots to a hostile target. The shots may be allocated to an individual target, or across a group of tracks. This sequence describes how a single or Lead MA reports allocated shots of an individual ACP, or multiple ACPs as part of a package via the team lead. These Shot Allocations will follow a shot doctrine which can be set pre-mission via fields within the *MA_ActionMT* message or updated by C2 using the "Set Tactic Shot Doctrine and Intercept Maneuver Parameters". Additionally, if C2 commands a shoot tactic, new shot doctrine parameters can be sent along with this tactic. See the "Shoot" tactic L1 behavior. If C2 commands 1 or more

ACPs to target or shoot any number of tracks, the Lead MA will calculate optimal target pairings along with allocating the shots across an entire package. Upon positive acknowledgement by Peers to a Lead MA is when this sequence of Reporting Shot Allocations to C2 is used by the Lead.

The *RelationshipDesignationMT* contains the needed information to convey the information to C2. For each allocated shot, MA should specify a *Source* and a *Destination*, which are both *GeoLocatedObjectTypes*. For the case of reporting shot allocations to C2, the *Source* field should contain a *SystemID*, representing the ACP with a shot allocated to a target. The *Destination* field should contain an *EntityID* which would represent the target being fired at. If the same ACP has allocated shots to multiple targets, multiple *RelationshipDesignationMT* messages should be sent to C2, denoting the same ACP via *SystemID* within the *Source* field, and different *EntityIDs* within the *Destination* field. Each individual shot allocation will also garner their own unique UUID sent in the *RelationshipDesignationID* field. This relationship denotes an ACP has a *WEAPON_ASSIGNED* for the *Destination* field. The *EngagementStatus* field is where the state of the relationship is conveyed to C2. Using the *ExternalCommandExecutionStateEnum*, MA should use the enumeration of *WEAPON_ASSIGNED* to denote the shots have been allocated towards a target. If an ACP has already reached target lock, an enum value of *LOCKED_ON* can be used to better describe the allocated shots status.

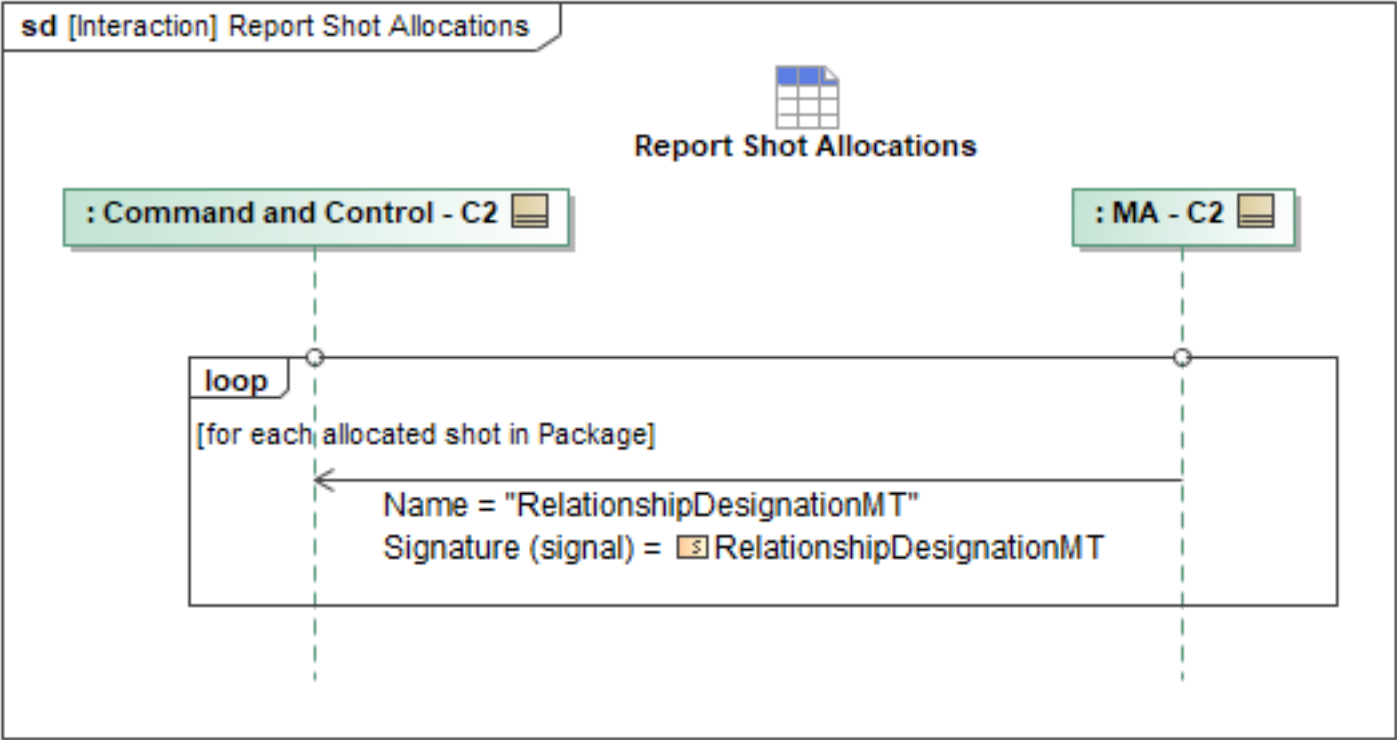


Figure 5-196: Report Shot Allocations

Table 5-195: Report Shot Allocations

Message Name (Name:Type)	Required Fields (Name:Type)
RelationshipDesignationMT: RelationshipDesignationMT	EngagementStatus: ExternalCommandExecutionStateEnum Destination: GeoLocatedObjectType WEAPON_ASSIGNED:

Message Name (Name:Type)	Required Fields (Name:Type)
	Relationship: RelationshipType
	EntityID: EntityID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.9 Store Inventory and Descriptions Exchange

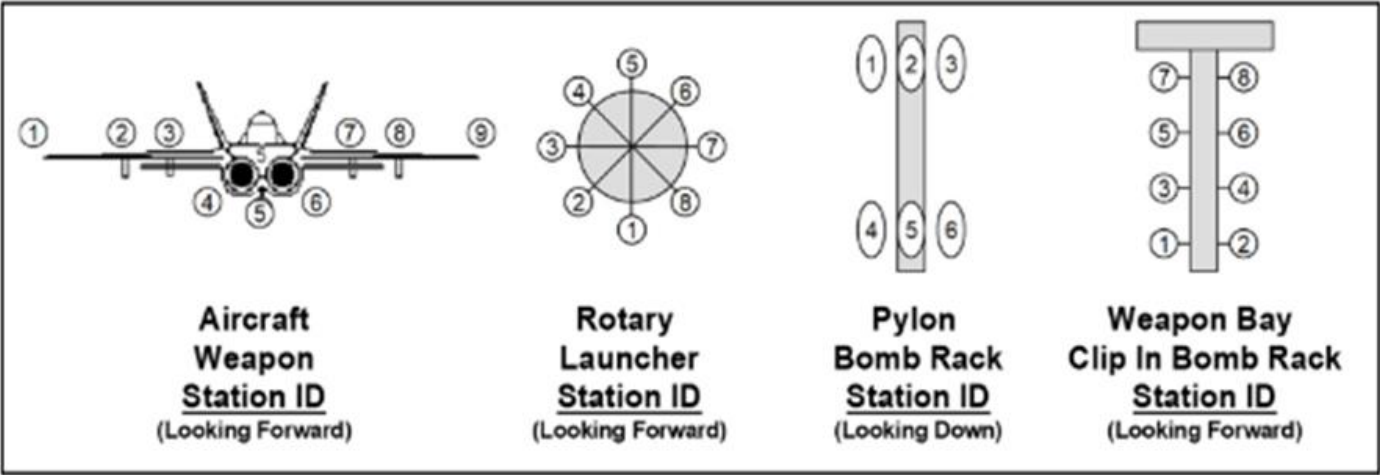


Figure 6.01R.13.00-1 - Examples of Station ID [01R/13/00...07]

Figure 5-197: Examples of Station ID

The supporting weapon subsystems (e.g. Stores Management System, Weapons Resource Manager) within MS are required to publish store inventory data (e.g. installed mission stores and carriages) to MA using *MA_StrikeCapabilityMT*. MA will monitor and forward the periodic *MA_StrikeCapabilityMT* message to determine the inventory of the installed store types, station locations and installed capabilities. MS will assign a capability ID for each installed mission store. The supporting capability is related to the carriage or rack that the store is attached to. MA can request the weapon subsystem via MS to verify the reported inventory is compatible with the platform using *StoreManagementCommandMT*. C2 can also initiate this request by sending a *StoreManagementCommandMT*, which MA will forward to MS. MA receives a response of *StoreManagementCommandStatusMT* from MS and should forward this to C2. Store compatibility infers the platform has completed weapon system integration testing to include safe separation testing.

This message is used for inventory and mission capability. This message will be updated as required when a tactic is selected, and a Mission Data File (MDF) is loaded. MA can directly forward this information to C2. C2 will remain aware of the mission store inventory and descriptions for overall mission effectiveness by receiving *StoreManagementCommandStatusMT*'s from MA, originating from MS.

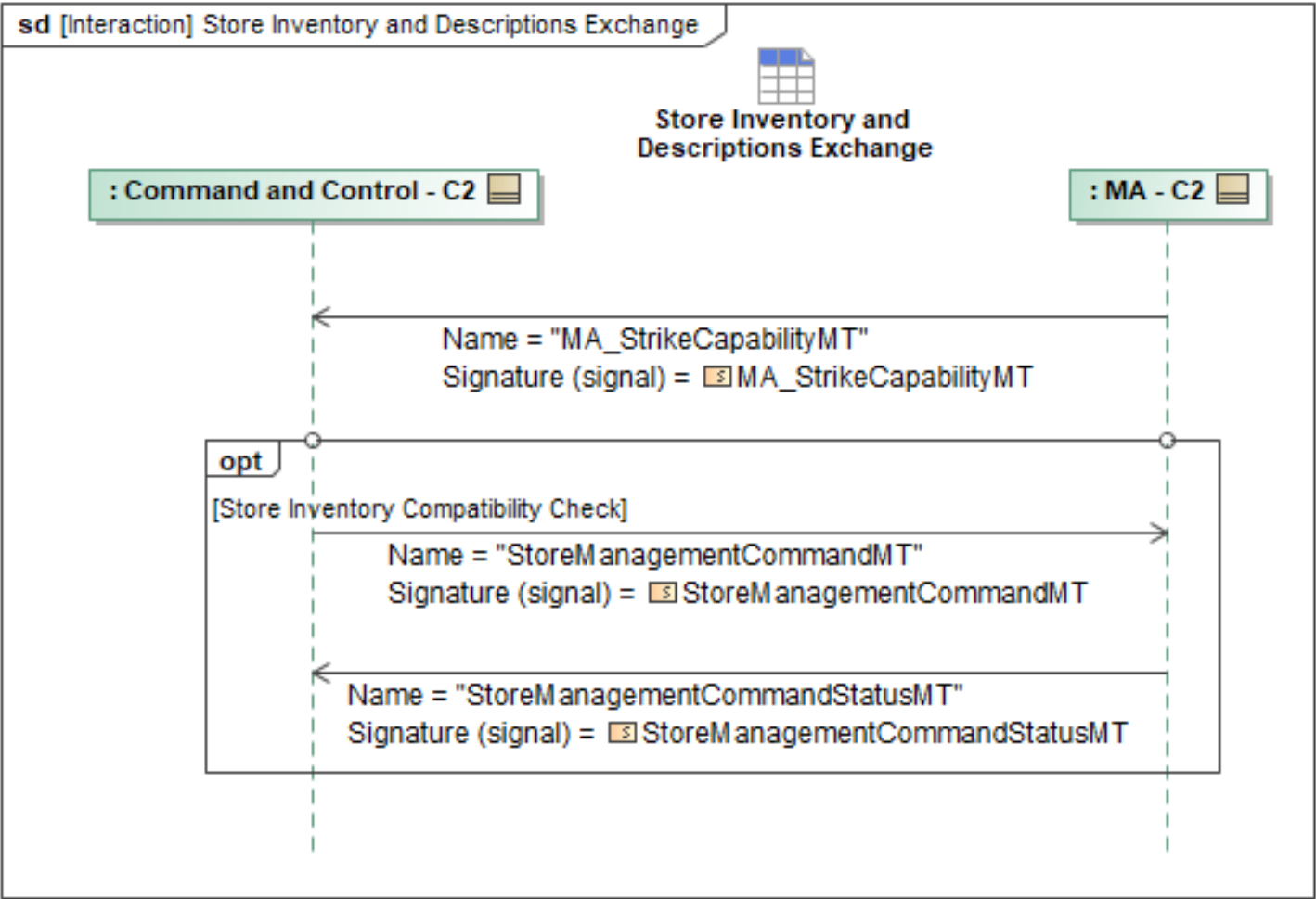


Figure 5-198: Store Inventory and Descriptions Exchange

Table 5-196: Store Inventory and Descriptions Exchange

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	Mnemonic: VisibleString256Type StoreTypeVariant: ForeignKeyType Powerable: boolean LoadoutMnemonic: VisibleString256Type Carriage: StoreCarriageCapabilityType
StoreManagementCommandMT: StoreManagementCommandMT	None
StoreManagementCommandStatusMT: StoreManagementCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.10 Target Designation

The Target Designation sequence diagram specifies the interaction where C2 designates one or more specific entities as a target. C2 sends *MA_DesignationMT* to MA, populating one or more *DesignationData* for the systems to receive a target designation. If an entity is to be designated as a target, the *DesignationEnum* within *TypeOfDesignation* will be set to *DESIGNATED_TARGET*. The *AppliesToID* field will specify the System ID or Package ID which shall be assigned to the target. The use of *PackageID* allows a Lead ACP to delegate the target assignment, whereas *SystemID* allows for direct assignment to a specific ACP. The optional *DesignationObjective* field allows for specification of a Requirement that the ACP should use against the target. To acknowledge receipt of the message, MA responds with the *MA_SystemNotificationMT* message, referencing the *SystemSubjectID*, *SystemPerspective*, and *AssociatedMessage* fields. The *SystemSubjectID* shall be set to the MA's System ID, the *SystemPerspective* enumeration shall be set to *RESPONSE_SUBJECT*, and *AssociatedMessage* shall define the received message's *MessageTypeEnum* as *DESIGNATION* and the *AssociatedID* shall be set to the *DesignationID*.

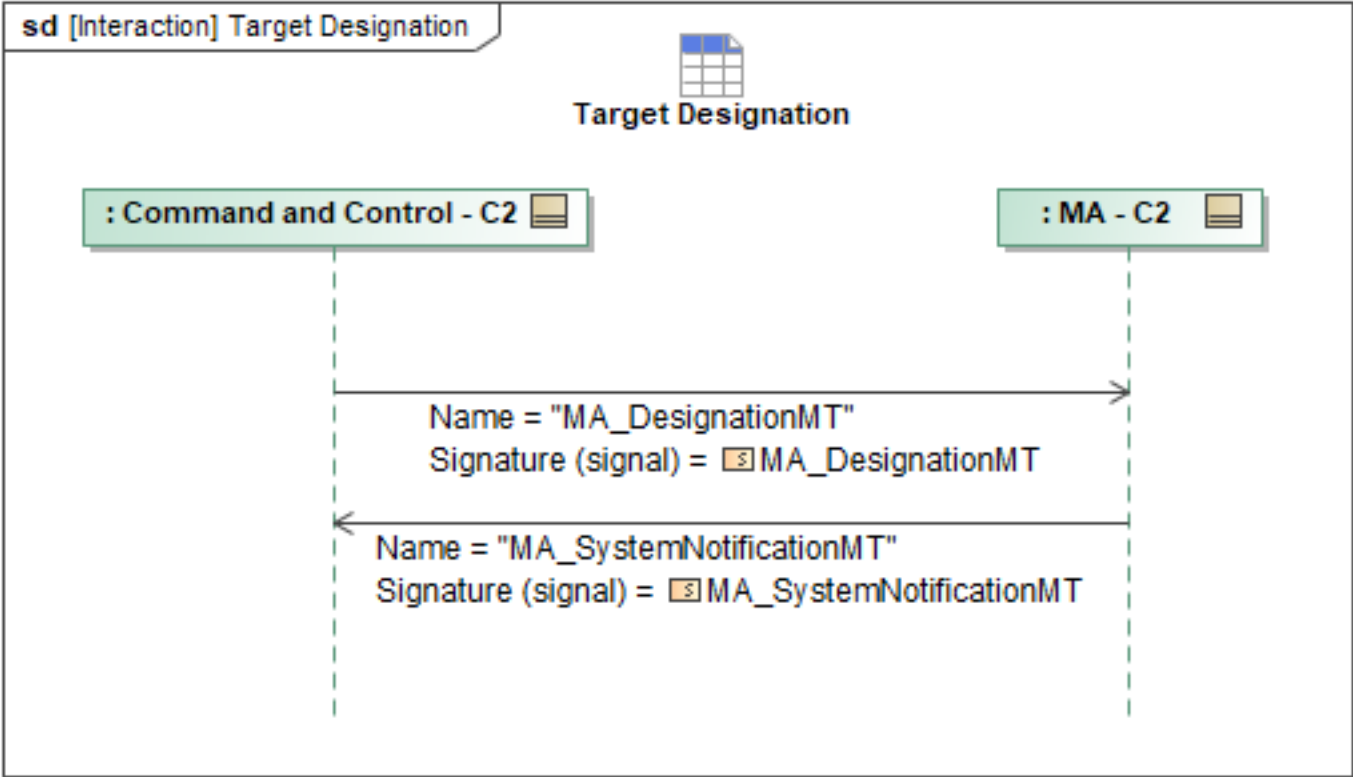


Figure 5-199: Target Designation

Table 5-197: Target Designation

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationMT: MA_DesignationMT	None
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.11 Target Designation Request

The Target Designation Request sequence diagram specifies the interaction that happens after MA identifies one or more targets and sends a *MA_DesignationRequestMT* to C2 to be Designated. The Target Designation Criteria can be used to identify Targets for engagement, and MA would send *MA_DesignationRequestMT* to an authorized C2 that is determined using the Target Authority Criteria. The *MA_DesignationRequestMT* contains one or more *DesignationData* that contains the target designation information. The *DesignationID* field is a specific identifier for that specific target. The Target field can be set to many items, but for an engagement, the Target Designation Criteria is only applicable to *EntityMTs*. The *TypeOfDesignation* field would likely be set to *DESIGNATED_TARGET* but could also be set to *DESIGNATED_NO_TARGET* if the service determines the target should no longer be a target. The *AppliesToID* field will specify the System ID or Package ID which shall be assigned to the target. The use of *PackageID* allows a Lead ACP to delegate the target assignment, whereas *SystemID* allows for direct assignment to a specific ACP. The optional *DesignationObjective* field allows for specification of a Requirement that the ACP should use against the target. For an engagement, these Requirement types would match what is in the *ApprovalPolicyMT* that is linked to the Engagement Criteria Target Authority. C2 can then use the information from the *DesignationData* field to populate a *MA_DesignationMT* message and send it back to the MA. C2 can update the information as needed. C2 will then send a *DesignationRequestStatusMT* populating the *RequestID* field with the same ID as was in the request. The *RequestProcessingState* field will be filled out as needed with the main options being *COMPLETED* (if a *MA_DesignationMT* was sent) or *REJECTED* (if C2 did not send a *MA_DesignationMT* message). The *DesignationDataStatus* field will be populated with the relevant target data from the request message. Including the associated *DesignationID* and the *RequestProcessingState* with similar options as stated above. There is an optional *RequestProcessingStateReason* field that can be filled out if desired.

If C2 is unable to complete the request it will ignore the *MA_DesignationRequestMT* command and reply with *MA_DesignationRequestStatusMT_4* where *RequestProcessingState* is set to "REJECTED". If the request is rejected because C2 does not have the request information, the *Reason* field within *RequestProcessingStateReason* will be set to "UNKNOWN_ID". If the request is rejected because the missing requested information cannot be transmitted via UIC, the *Reason* field will be set instead to "INVALID_INPUT_PARAMETER".

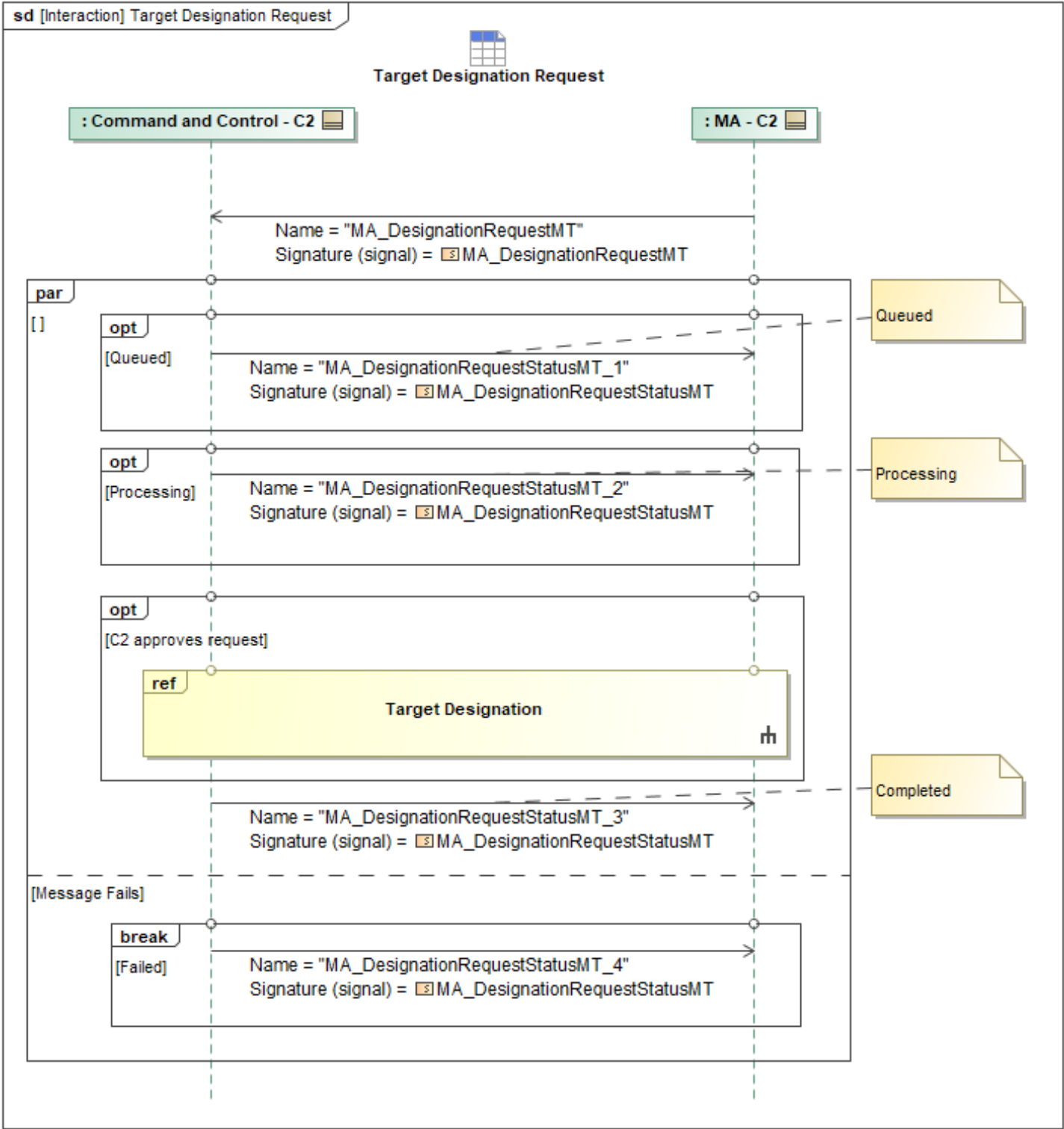


Figure 5-200: Target Designation Request

Table 5-198: Target Designation Request

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationRequestMT: MA_DesignationRequestMT	None
MA_DesignationRequestStatusMT_1: MA_DesignationRequestStatusMT	None
MA_DesignationRequestStatusMT_2: MA_DesignationRequestStatusMT	None
MA_DesignationRequestStatusMT_3: MA_DesignationRequestStatusMT	None
MA_DesignationRequestStatusMT_4: MA_DesignationRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.12 Threat Prioritization

Mission Use Case: ALR

The Threat Prioritization List ranks entities by assessing the movements and behaviors to determine threat levels to a respective position, whether it is the ACP ownership or another platform. Threat Prioritization will utilize *MA_PrioritizationListMT* with a *ListType* of *Threat_Target_Prioritization_List* enumeration. The list requires subjects (*EntityID*) to be assigned a priority, where the lowest assigned value is the highest threat level.

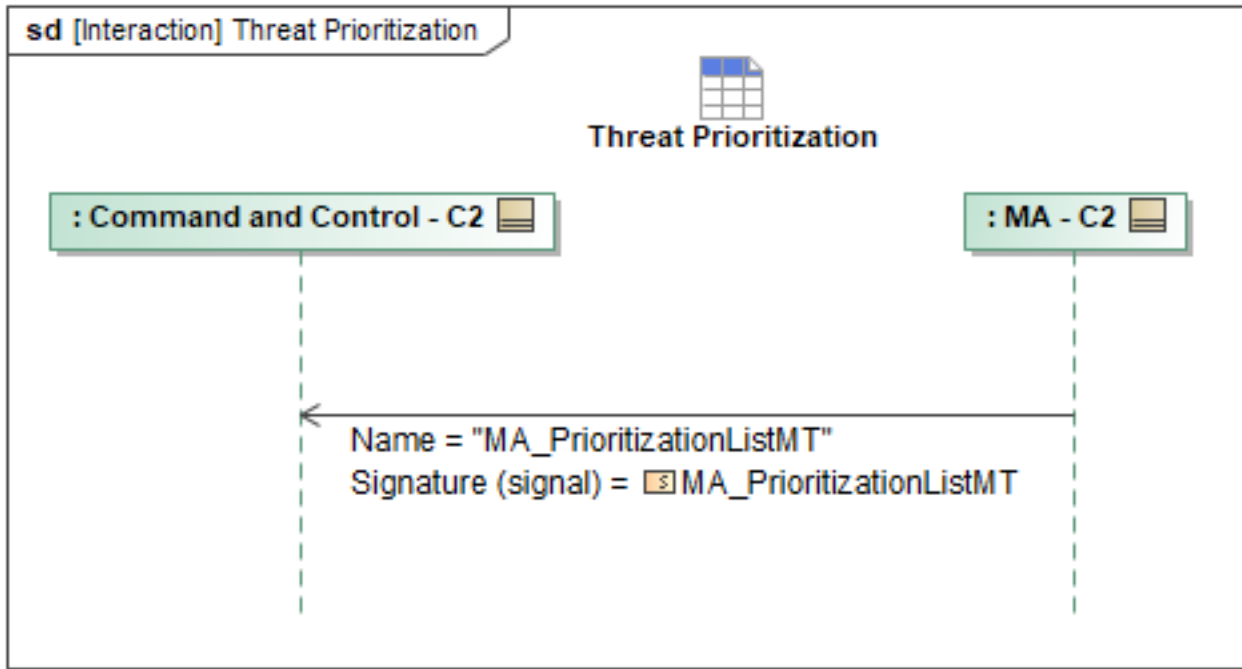


Figure 5-201: Threat Prioritization

Table 5-199: Threat Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	EntityID: EntityID_Type THREAT_TARGET_PRIORITIZATION_LIST:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.13 Update Shot Queue Prioritization

Shot Queue Prioritization List is a list of mission targets ranked by assessing multiple factors to determine which target entities should be engaged first. The shot queue will utilize *MA_PrioritizationListMT* with a ListType of *Mission_Target_List (enum)*. The list requires subjects (EntityID) to be assigned a priority, where the lowest assigned value is the highest priority level. The message also includes an optional field, *ProbabilityOfKill*, that may be included with each assessed subject entity. C2 may send an updated shot queue prioritization list to an ACP (Single Ship, Multi-Ship Lead, or Multi-Ship Peer). Once an updated *MA_PrioritizationListMT* is received, MA will respond to the originating System with an *MA_SystemNotificationMT*.

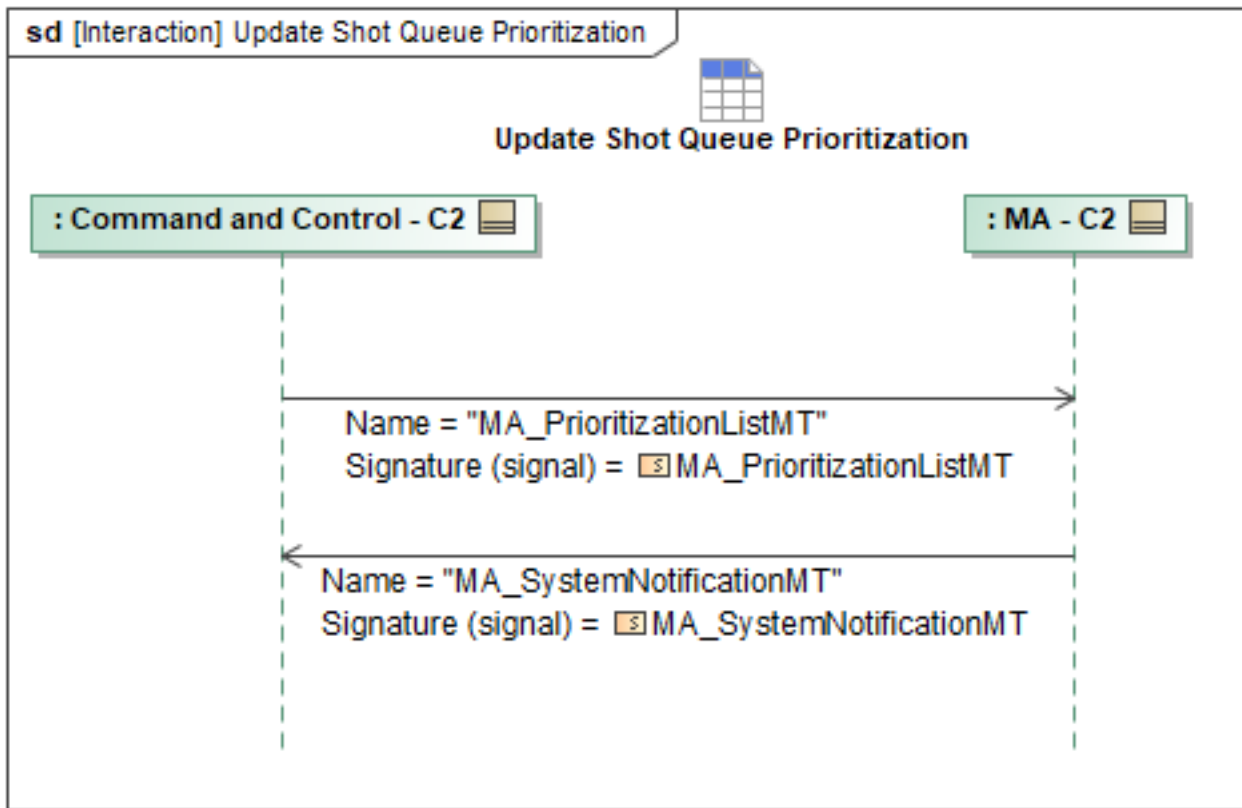


Figure 5-202: Update Shot Queue Prioritization

Table 5-200: Update Shot Queue Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	MISSION_TARGET_LIST: ObjectState: ObjectStateEnum
MA_SystemNotificationMT: MA_SystemNotificationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.2.15.14 WEZ Compound Convergence

WEZ Compound Convergence is a fusion of multiple WEZ from team members within a flight group. The fusion of WEZ enables C2 to generate a composite view of flight group’s weapon engagement capabilities to assess tasking for the best shot. Once ACPs are producing a WEZ in support of strike task, MA will send *MA_WEZ_MT* periodically to C2.

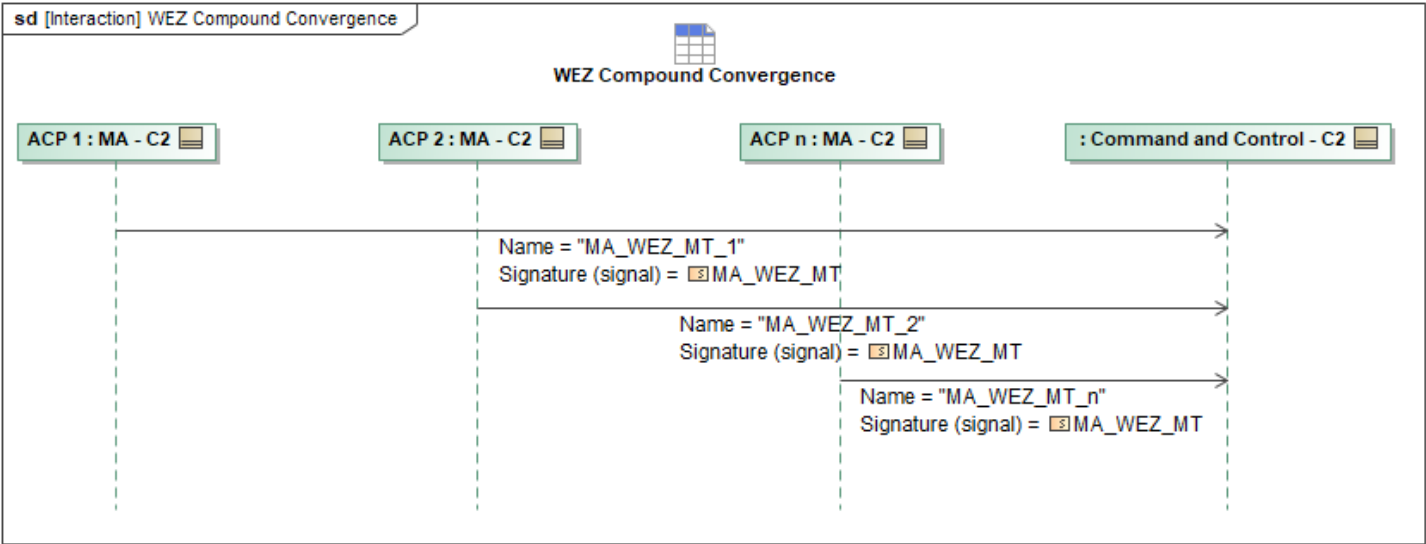


Figure 5-203: WEZ Compound Convergence

Table 5-201: WEZ Compound Convergence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_WEZ_MT_1: MA_WEZ_MT	None
MA_WEZ_MT_2: MA_WEZ_MT	None
MA_WEZ_MT_n: MA_WEZ_MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

5.3 Interface Compliance

The Interface Compliance table contains the compliance requirements for the L1 C2 interface.

Table 5-202: C2 Interface Compliance Requirements

ID	Requirement
MA-L1-001	A Level 1 C2 interface shall support the C2 Interface Minimum Message Set defined in the C2 MMS table in the C2 feature profile (A-GRA location: 1 Problem Domain > 3 Exchange Items > Minimum Message Set (MMS)) for all messages in Core Mission Use Case sequences and all non-Core Mission Use Case sequences required for platform specific implementations.
MA-L1-002	A Level 1 C2 interface shall implement all required messaging sequences in the C2 feature profile (A-GRA location: 1 Problem Domain > 2 White Box > 3 Functional Analysis > L1 Functional Analysis > L1 Interface Behaviors > C2 Interface Behavior), marked Core and all non-Core Mission Use Case sequences that apply to the platform specific implementation.
MA-L1-017	Upon receipt of a Malformed Message (a message missing expected fields (i.e., required by the behavior but not the UCI schema) or with invalid context-dependent information within the scope of that individual message) from the C2 interface, MA shall respond to the error by notifying C2 of the malformed message and requesting the corrected message according to the Malformed Message Response (C2) sequence.
MA-L1-021	Upon receipt of any message from the C2 interface that is unauthorized according to C2's role and the RBAC configuration, MA shall respond in accordance with the C2 Unauthorization Response sequence, selecting the response message that corresponds to C2's unauthorized message the three tables (Unauthorized ActionRequest-2 Message Pairs, Unauthorized Command-2 Message Pairs, Unauthorized DataRequest-2 Message Pairs). The tables include primitive type, triggering message, and the specific unauthorization response message.
MA-L1-023	A Level 1 C2 interface shall follow state transitions for CommandProcessingState and RequestProcessingState in accordance with OAC-SPC-001 Rev C, Section 5.1.1.1 and 5.1.2.1. Representations of non-terminal statuses are optional and should align with the definitions and usage prescribed by the UCI Schema Style and Design Specification, ensuring consistent message state handling.

6 MISSION DEBRIEF INTERFACE

The Mission Debrief (MD) interface supports the transfer of data related to activity and decisions that occur in MA or are captured from other onboard L1 Subsystems. This interface enables MA to externally transfer captured data, which can be used for mission analysis/debrief and after-action review system features (e.g., playback, causal analysis, reports, simulation).

6.1 Mission Debrief Interface Description

Data captured by MA enables Mission Debrief (MD) systems to play back missions, analyze missions, and provide a deeper understanding of MA's decisions. Other L1 subsystems may also use the interactions defined by the MD interface on their own interfaces in the same way MD does, such as C2 and P2P.

6.2 Mission Debrief Interface Interactions

The MD interface is designed primarily for after-action use, but it can also be utilized during missions. While a mission is underway, C2 and P2P may also utilize this interface, which is defined and owned by MD. All communication through the L1 MD interface is conducted via the Decentralized Messaging System (DMS).

6.3 Mission Debrief Interface Compliance

The Interface Compliance table contains the compliance requirements for the L1 MD interface.

Table 6-1: Mission Debrief Interface Compliance Requirements

ID	Requirement
MA-L1-015	A Level 1 Mission Debrief Interface shall support the external transfer of all debrief-relevant data sent on the ASB, with any necessary metadata.

7 MISSION PLAN INTERFACE

The L1 Mission Plan (MP) interface defines the Mission Data Bundle, which Mission Autonomy (MA) must ingest and internally manage to facilitate mission execution. This data includes traditional mission planning inputs, such as weather, airspace boundaries, and airfield data; as well as data specific to MA, including route and task plans. The Mission Data Bundle should be filled out thoroughly pre-mission. References to Mission Data Bundle elements are found throughout the A-GRA, which create dependencies within other interface interactions.

7.1 Interface Description

Mission Data refers to data elements that MA uses to determine what mission it must accomplish, including all constraints, restraints, priorities, and other parameters that may directly or indirectly impact behavior. It is possible for MA to accomplish the vast majority of an end-to-end mission with mission planned data alone, requiring only minimal human-in-the-loop interactions. Mission Data is dynamic, allowing C2, Peer, Mission Systems, and contingency driven updates during mission execution.

The supported types of data included in the Mission Data Bundle (see figure below) include weather, airspace restrictions, MA configurations, tasking, and mission context, etc. Some data is access-restricted, based on approval policy data and Authorization to access and modify elements to prevent unauthorized changes during the mission.

Mission Plans are the subset of the Mission Data that provide commands and tasking for MA. They encapsulate mission context and constraints to provide MA with the parameters necessary to execute a mission. Not all Mission Data is considered part of the Mission Plan. Some elements such as weather and vehicle characteristics are intended to be independent of any individual Mission Plan and are meant to provide information to MA for the entirety of the mission.

The Mission Plan schema used by MA is based on UCI 2.3, with currently no extensions or custom messages. Existing Mission Plan and related structures within UCI provide a suitable and standardized basis for MA applications. They include an assortment on needed *Plan formats that help to cover many of the facets expected in execution of a Mission. These sub-plans can include TaskPlan, RoutePlan, and other plans as defined in UCI 2.3. Subordinate plans provide a mechanism to create a hierarchy of Mission Plans with traceability to the originating plan. One or more Mission Plans may be included in the Mission Data Load with Mission Data elements potentially shared between Mission Plans. The Mission Data Load is the collection of all relevant data collected and produced during the pre-mission planning phase. It consists of various routes, plans, and datasets needed to effectively execute the mission. A Mission Plan is assumed to be inactive unless the Mission Data specifically declares its activation. When a Mission Plan is inactive, it is stored for future use and may be used for computational purposes such as state estimation and planning, but it is not considered as a direct source of behavioral guidance. When a Mission Plan is activated, MA is required to attempt to execute its parts to completion in order, which requires tracking progress, which is also used when responding to ad-hoc Direct C2 or contingencies, and resuming the mission plan.

Aspects of the Mission Data are platform and/or implementation specific and not specified in the A-GRA. What is highlighted here is that Vehicles and Mission Systems will both need to interact with the Mission Data so there's a need for majority of the data in a central, accessible location. A-GRA is agnostic to both the process to produce the required data and the storage method. It is assumed a

combination of tools will be integrated to produce the entire Mission Data Bundle.

The format of Mission Data entering MA will conform to the format and requirements outlined in this section. Once deserialized and loaded, the Mission Data is stored and acted upon by MA. Some Mission Data elements are not immediately actionable; they may be schedule-dependent or location-dependent or they may require explicit activation.

MA may receive Mission Data elements from each of the external interfaces. The initial Mission Data defined in the Mission Plan Interface section of the A-GRA provides the baseline Mission Data (including Mission Plans) for MA to execute. Portions of the Mission Data can be modified, updated, and augmented dynamically during mission execution from sources both internal and external to MA via entry points such as the C2 (see Section 4.2), Peer interfaces (see Section 4.3), Mission Systems (see Section 4.6), and Vehicle Interface (see Section 4.7). Mission Data can also be queried and shared between connected entities via the C2 and Peer interfaces. Details of these interactions are captured in their respective sections in the A-GRA. It is important to note that not all elements of the Mission Plan are exposed to each interface, with most operations occurring on a per element basis.

7.1.1 Mission Data Bundle

The Mission Data Bundle is a collection of data that contains UCI and non-UCI information that the ACP will use to complete the mission. The Mission Data Bundle will be loaded onto the ACP before the start of the mission, and is required to be present for the mission to begin. The Mission Data Bundle diagram defines the collection of messages that MA will receive during initialization. The Mission Data Bundle is considered a static bundle of data which is provided to MA at the start of the mission. The original Mission Data Bundle itself remains unchanged, serving as a persistent record of the Mission Data that was provided to MA. The diagram describes example data categories, such as operational data, Mission Plan, and requirements, and others to aid readability and traceability. Note the multiplicities of the messages displayed on the diagram indicate that some of the messages are optional.

FA will have its own data bundle that contains the information that FA will use to operate. It is assumed that any information in FA's data bundle that MA will need to access will be contained within the Mission Data Bundle.

Another component of the Mission Data Bundle is the *MA_CapabilitySet* message, which acts as the Minimum Essential Capabilities List (MECL). The purpose of the MECL is to explicitly identify all required capabilities for the vehicle to successfully complete a given mission.

In the *MA_CapabilitySet's MessageData* field, Capability IDs are tied to their respective Applicable System, mission and/or Mission Plan. The intent of this message is to convey the minimum essential capabilities required to complete a mission, which can be evaluated against to detect Mission Capability Fault contingencies. To evaluate MECL, MA sweeps through all listed capabilities in the *MA_CapabilitySet* and compares them to relevant *CapabilityStatus* messages to ensure the full criteria set is met.

Note: The linked Capability messages will contain actual details about capability implementation.

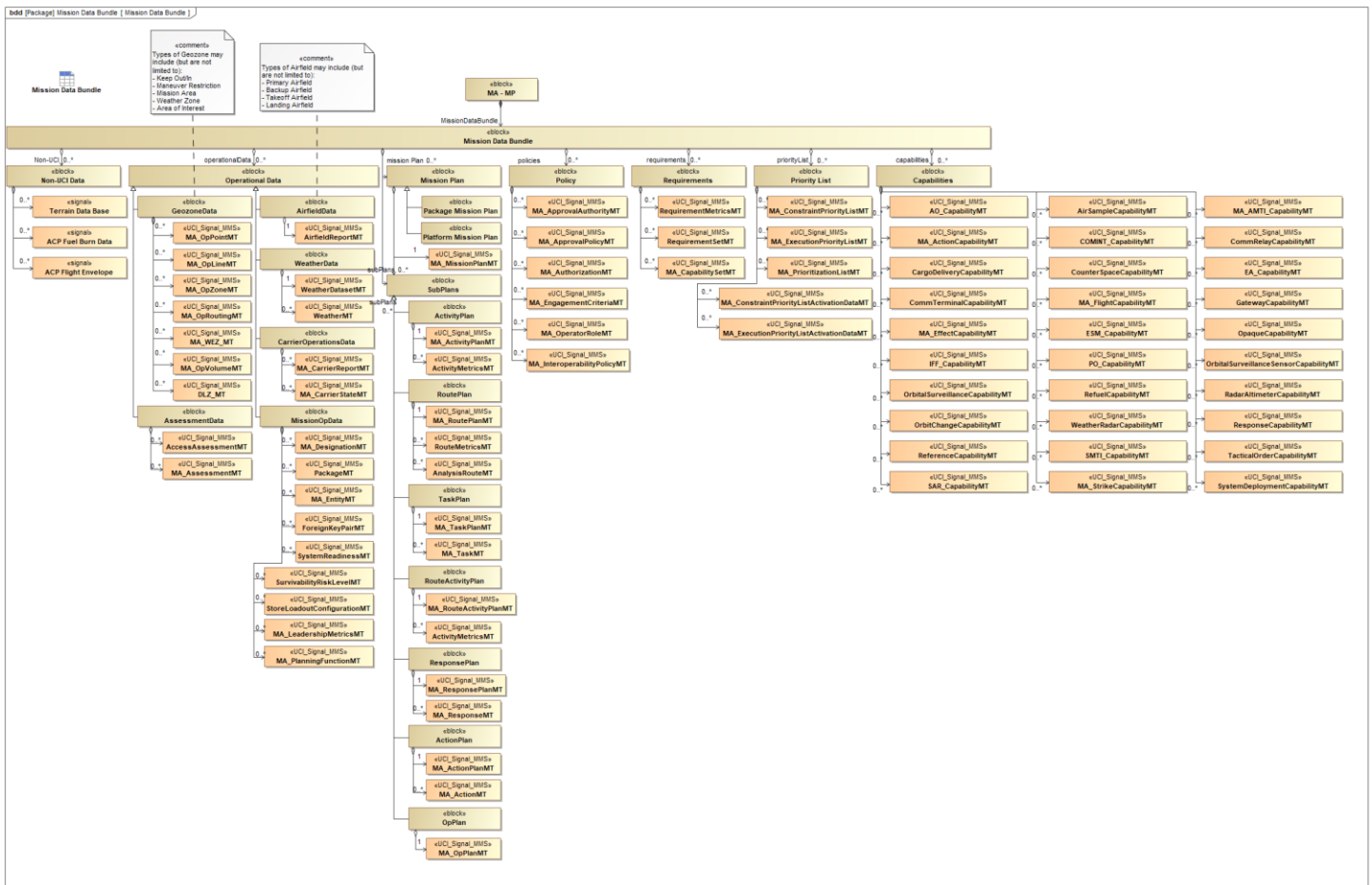


Figure 7-1: Mission Data Bundle

7.1.2 Mission Plan Message

The Mission Plan schema used by MA is based on UCI 2.3. Existing Mission Plan and related structures within UCI provide a suitable and standardized basis for MA applications. The Mission Plan schema includes various *Plan formats that help to cover many of the facets expected in execution of a Mission. These can include Task Plan, Route Plan, and other plans as defined in UCI 2.3. Subordinate plans provide a mechanism to create a hierarchy of Mission Plans with traceability to the originating plan.

One or more Mission Plans may be included in the Mission Data Bundle with Mission Data elements potentially shared between Mission Plans. The Mission Data Bundle is the collection of all relevant data collected and produced during the pre-mission planning phase. It consists of various routes, plans, and datasets needed to effectively execute the mission. When a Mission Plan is inactive, it is stored for future use and may be used for computational purposes such as state estimation and planning, but it is not considered as a direct source of behavioral guidance. When a Mission Plan is activated, MA is required to attempt to execute its parts.

At the start of each mission or deployment, MA receives the Mission Data Bundle that will contain the Mission Data needed to execute its mission. The Mission Data Bundle is used to initialize the Mission Data. Aspects of the Mission Data Bundle are platform-dependent and implementation-specific and not specified in this ICD. What is highlighted here is that Vehicles and Mission Systems will both

need to interact with the Mission Data so there's a need for much of the data to be accessible to these systems. A-GRA assumes that a process exists to generate the Mission Data Bundle from an existing mission planning tool by augmenting the output from mission planning with additional data needed to support autonomy execution.

7.2 Interface Interactions

The L1 MP Interface represents the influence that an external entity, i.e. the Mission Planner, has on MA. It specifies minimum mission plan requirements (i.e. format, data types, fields) for what is expected from mission planners to support MA. These message exchanges are defined and owned by MP, but may be used by other L1 subsystems in their own interfaces with MA, such as C2 and P2P.

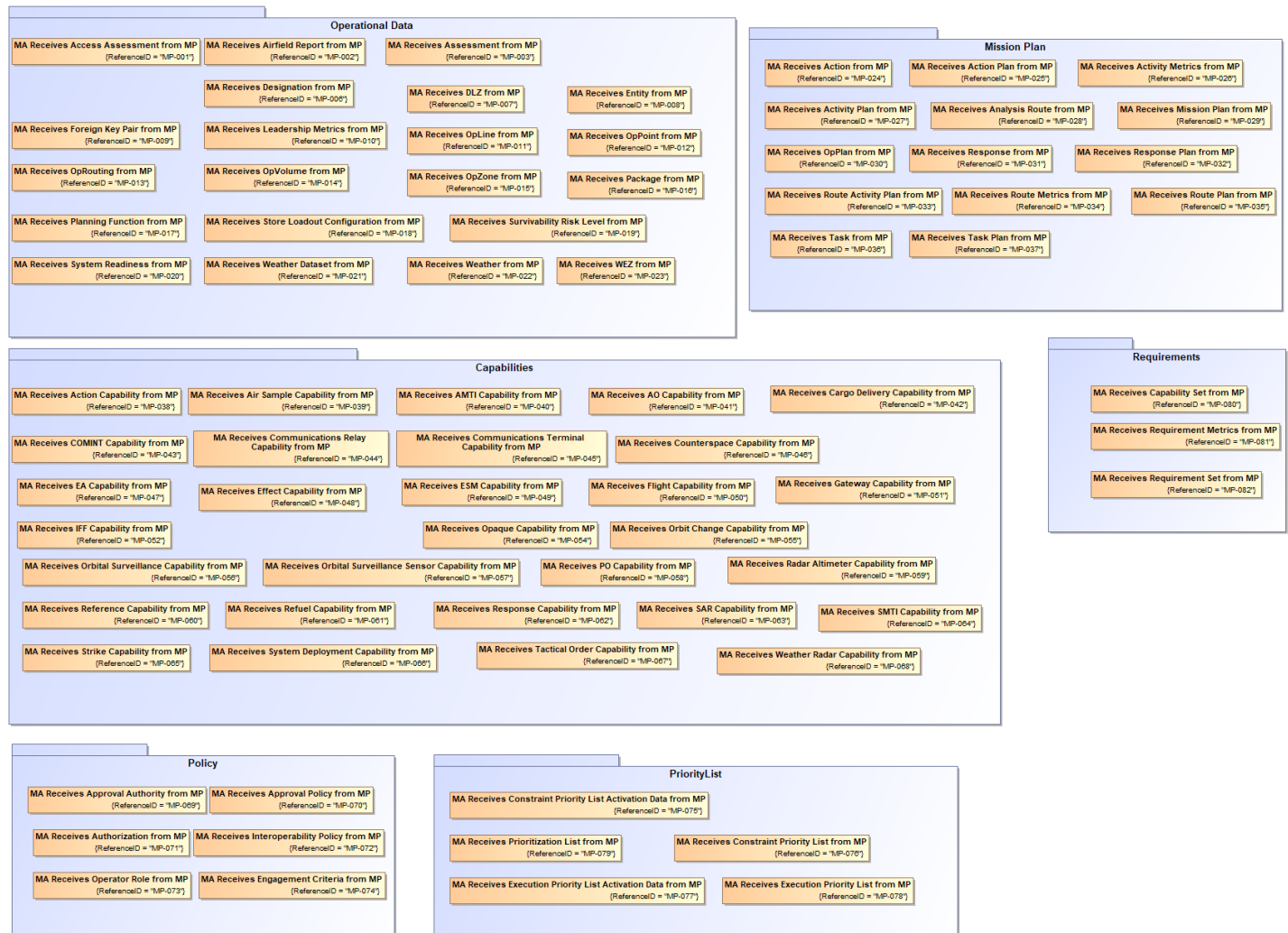


Figure 7-2: Mission Plan Interface Interactions

7.2.1 Capabilities Package

7.2.1.1 MA Receives Action Capability from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ActionCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data

from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

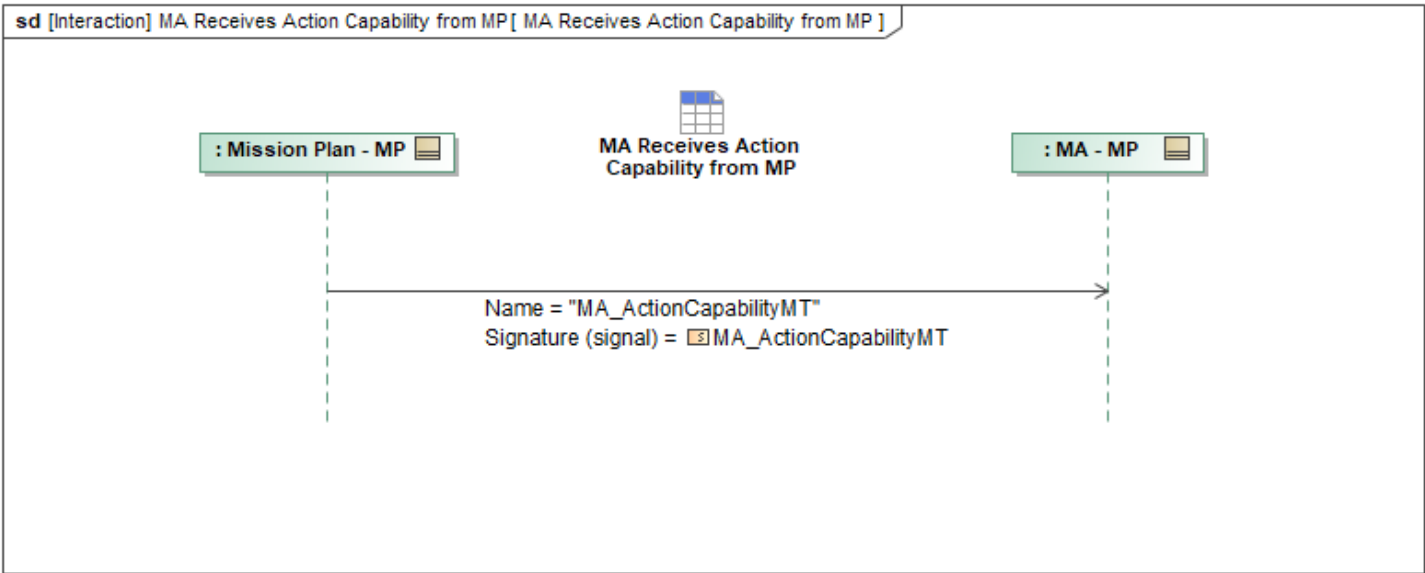


Figure 7-3: MA Receives Action Capability from MP

Table 7-1: MA Receives Action Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCapabilityMT: MA_ActionCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.2 MA Receives Air Sample Capability from MP

This interaction describes how Mission Autonomy (MA) receives *AirSampleCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

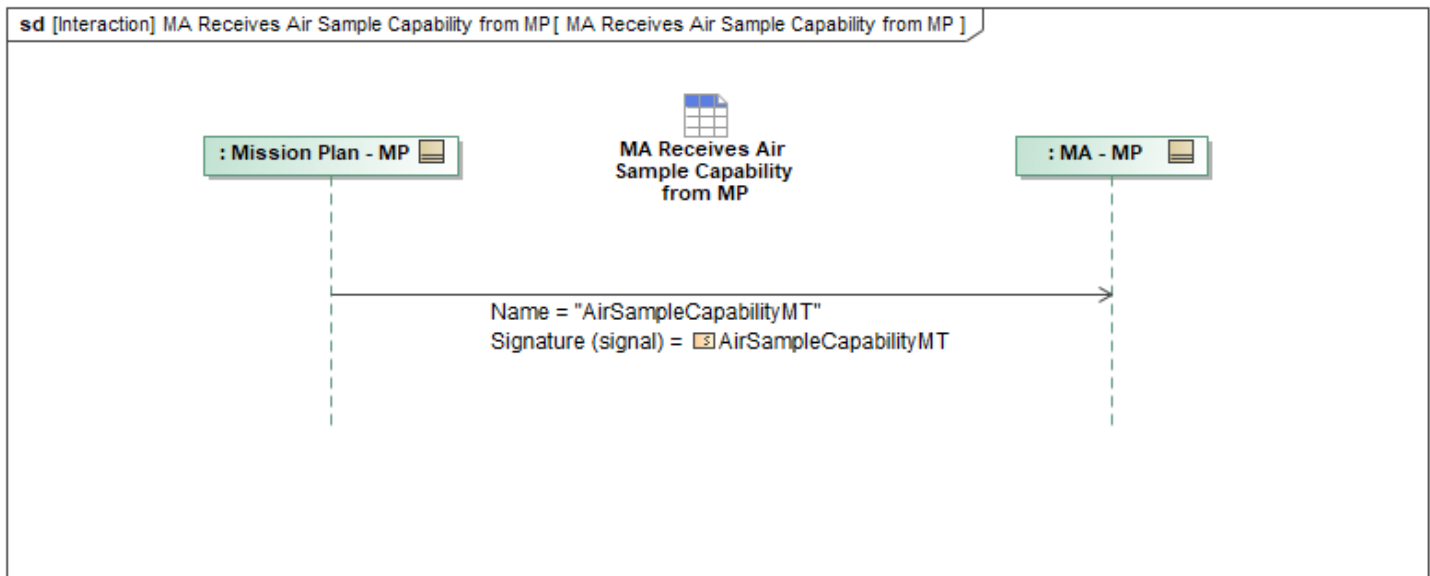


Figure 7-4: MA Receives Air Sample Capability from MP

Table 7-2: MA Receives Air Sample Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
AirSampleCapabilityMT: AirSampleCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.3 MA Receives AMTI Capability from MP

Mission Use Case: AMTI

This interaction describes how Mission Autonomy (MA) receives *MA_AMTI_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

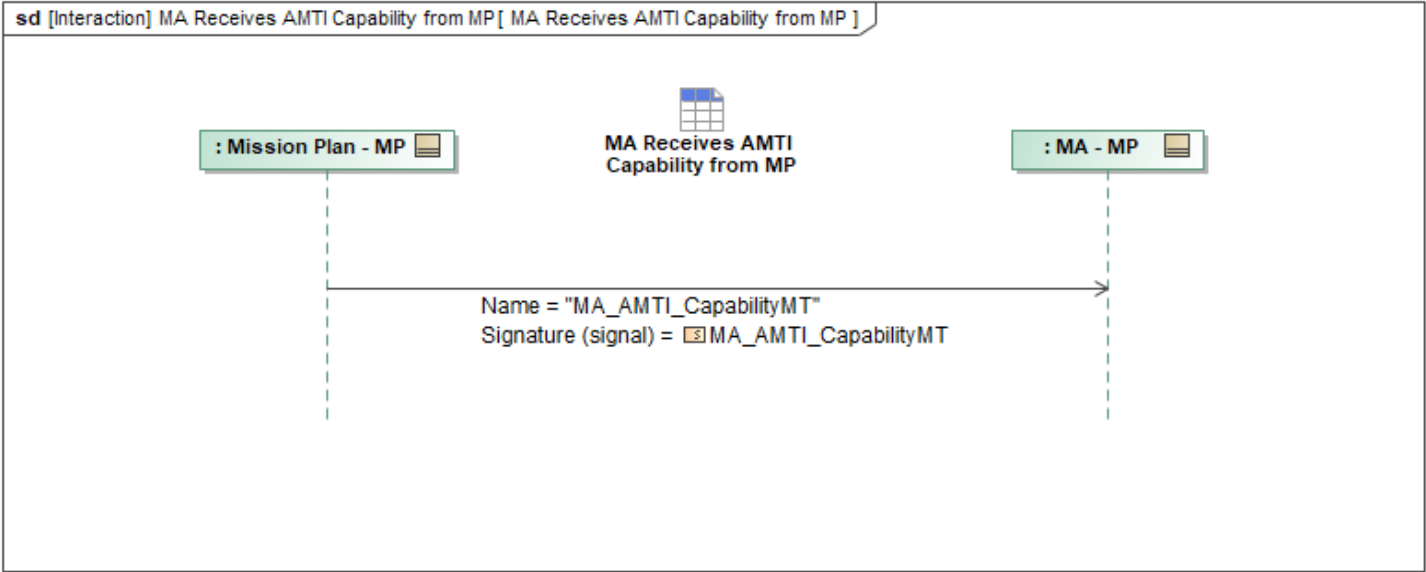


Figure 7-5: MA Receives AMTI Capability from MP

Table 7-3: MA Receives AMTI Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.4 MA Receives AO Capability from MP

This interaction describes how Mission Autonomy (MA) receives *AO_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

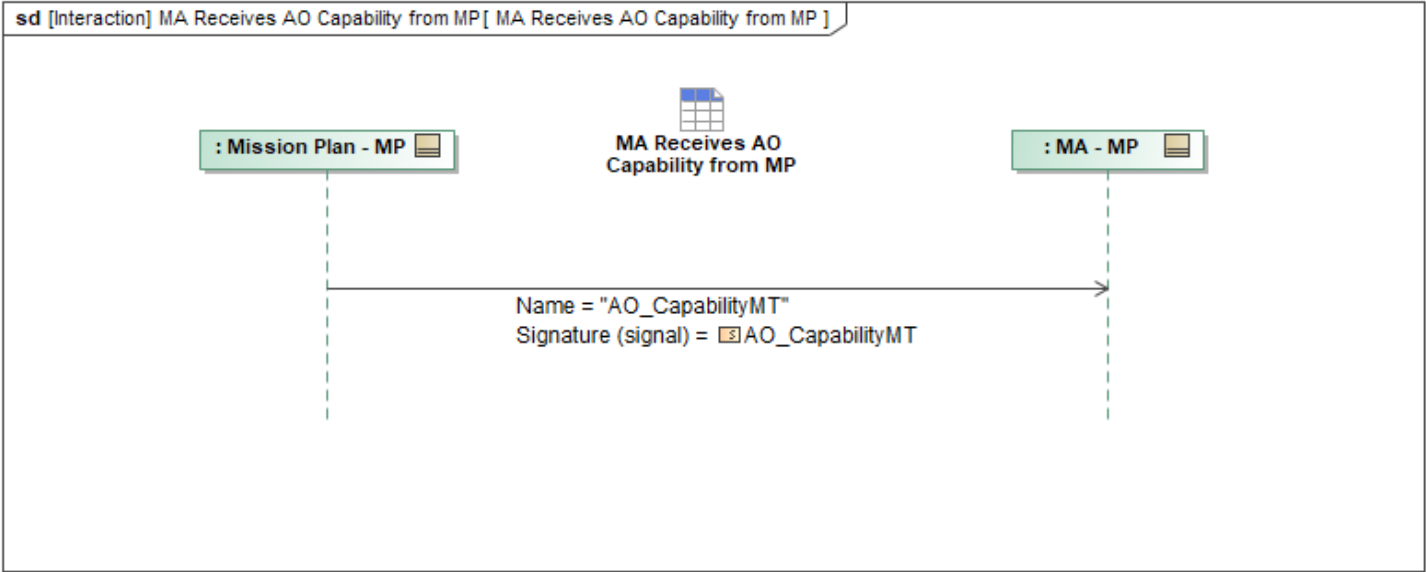


Figure 7-6: MA Receives AO Capability from MP

Table 7-4: MA Receives AO Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
AO_CapabilityMT: AO_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.5 MA Receives Cargo Delivery Capability from MP

This interaction describes how Mission Autonomy (MA) receives *CargoDeliveryCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

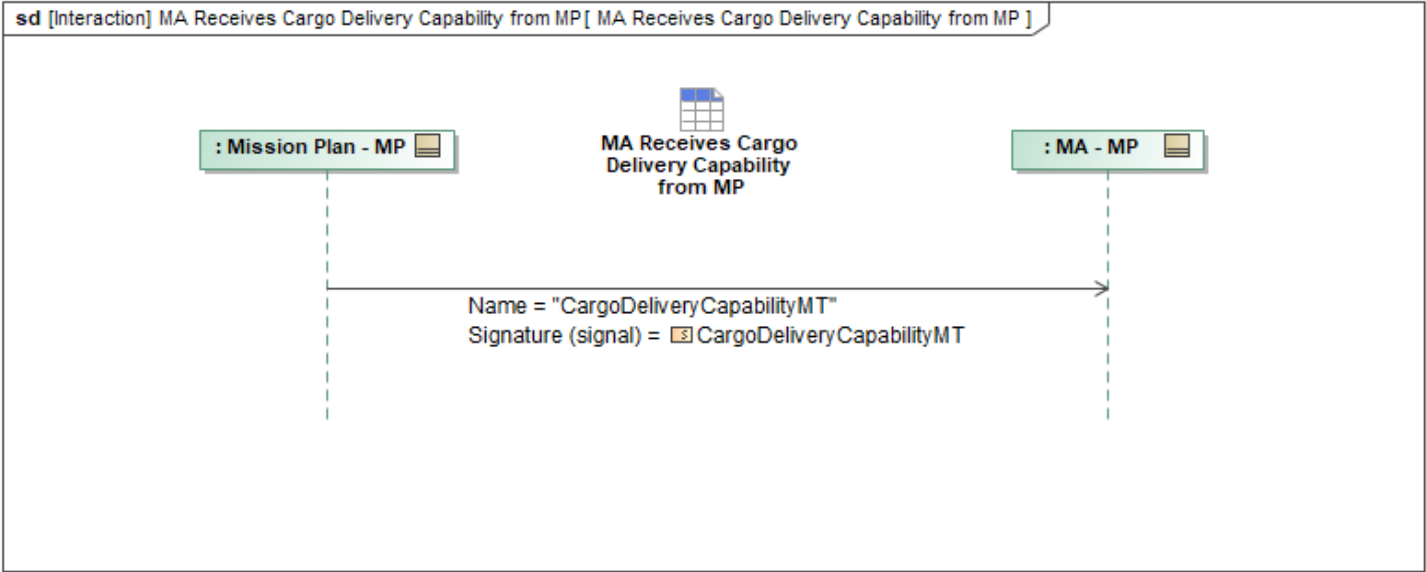


Figure 7-7: MA Receives Cargo Delivery Capability from MP

Table 7-5: MA Receives Cargo Delivery Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
CargoDeliveryCapabilityMT: CargoDeliveryCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.6 MA Receives COMINT Capability from MP

This interaction describes how Mission Autonomy (MA) receives *COMINT_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

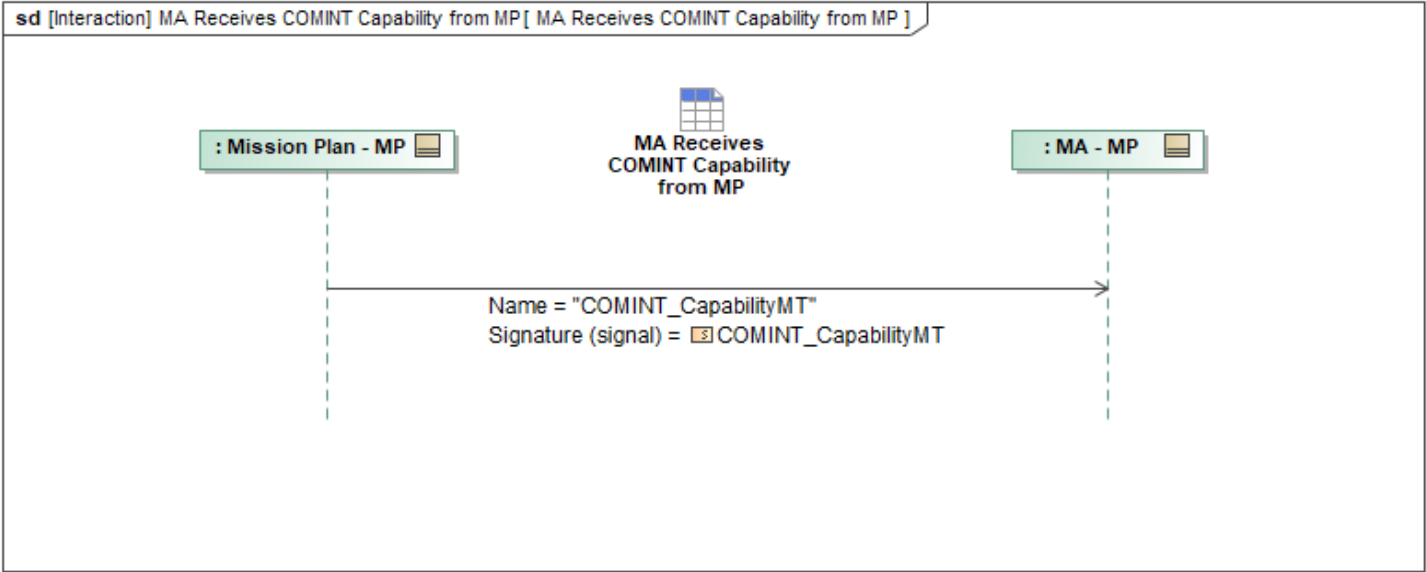


Figure 7-8: MA Receives COMINT Capability from MP

Table 7-6: MA Receives COMINT Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
COMINT_CapabilityMT: COMINT_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.7 MA Receives Communications Relay Capability from MP

Mission Use Case: Additional Comms Capabilities

This interaction describes how Mission Autonomy (MA) receives *CommRelayCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

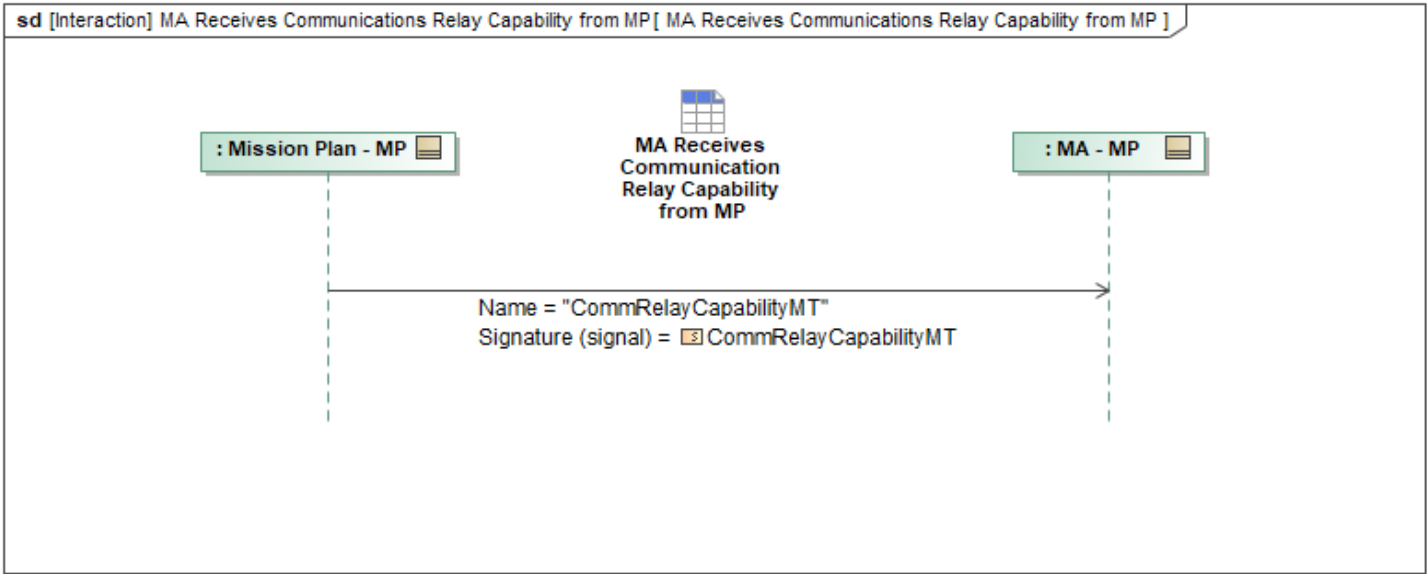


Figure 7-9: MA Receives Communications Relay Capability from MP

Table 7-7: MA Receives Communications Relay Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
CommRelayCapabilityMT: CommRelayCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.8 MA Receives Communications Terminal Capability from MP

Mission Use Case: Additional Comms Capabilities

This interaction describes how Mission Autonomy (MA) receives *CommTerminalCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

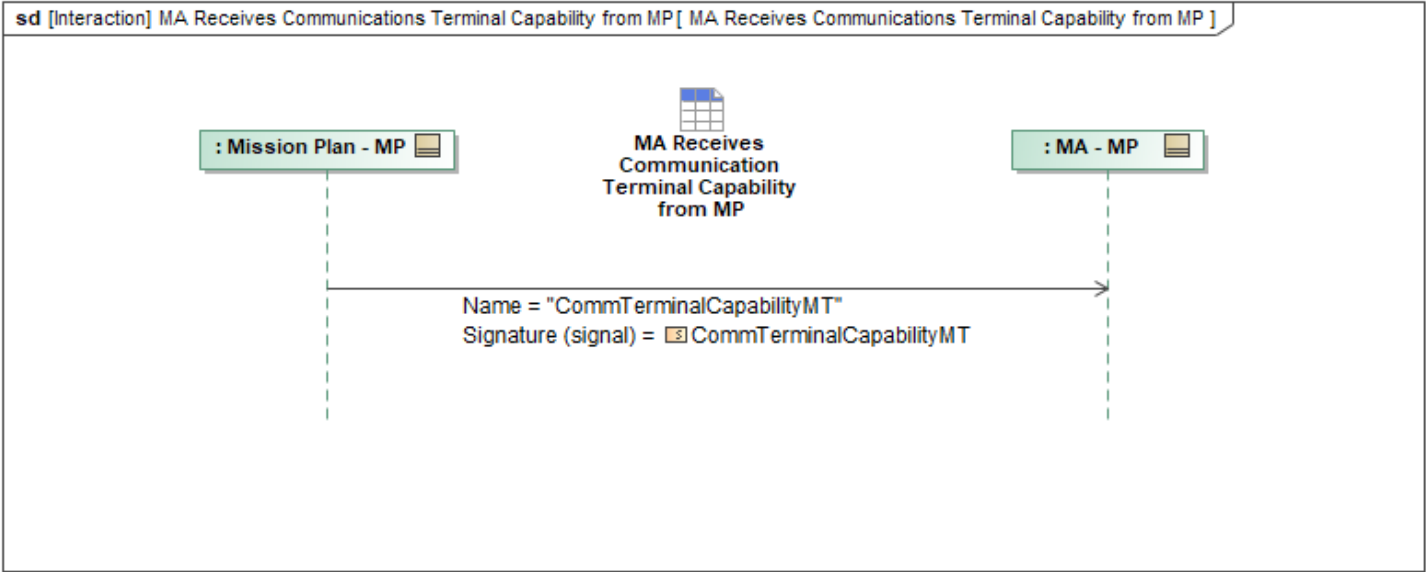


Figure 7-10: MA Receives Communications Terminal Capability from MP

Table 7-8: MA Receives Communications Terminal Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalCapabilityMT: CommTerminalCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.9 MA Receives Counterspace Capability from MP

This interaction describes how Mission Autonomy (MA) receives *CounterSpaceCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

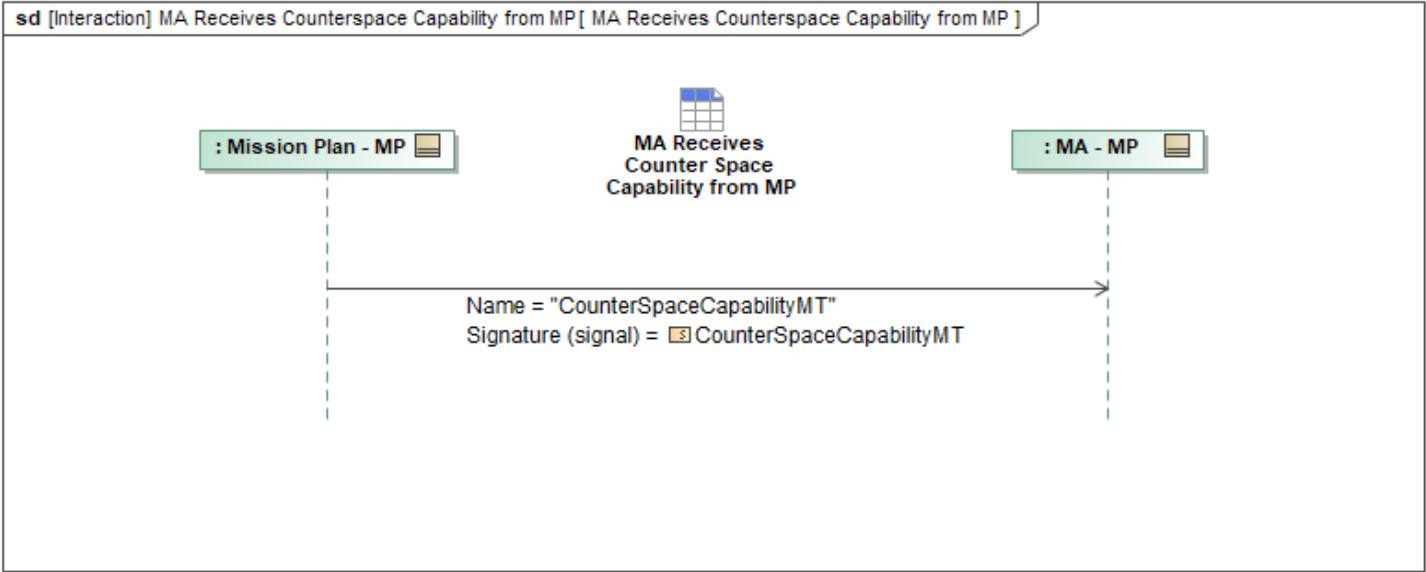


Figure 7-11: MA Receives Counterspace Capability from MP

Table 7-9: MA Receives Counterspace Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
CounterSpaceCapabilityMT: CounterSpaceCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.10 MA Receives EA Capability from MP

This interaction describes how Mission Autonomy (MA) receives *EA_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

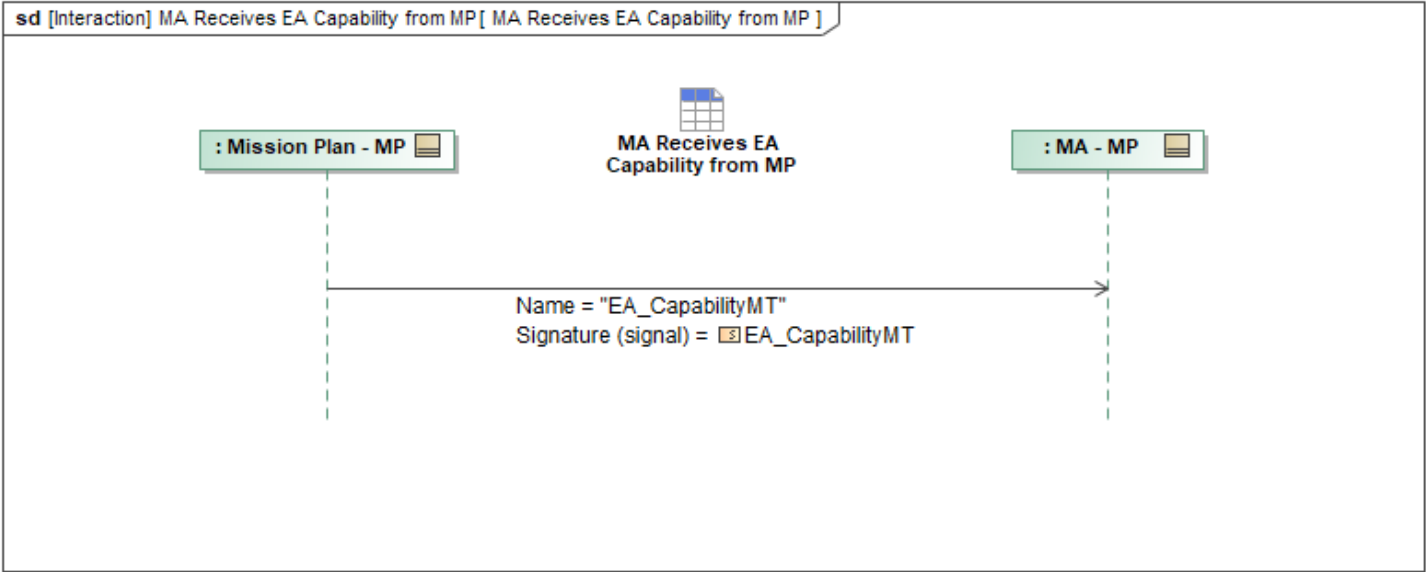


Figure 7-12: MA Receives EA Capability from MP

Table 7-10: MA Receives EA Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
EA_CapabilityMT: EA_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.11 MA Receives Effect Capability from MP

This interaction describes how Mission Autonomy (MA) receives *MA_EffectCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

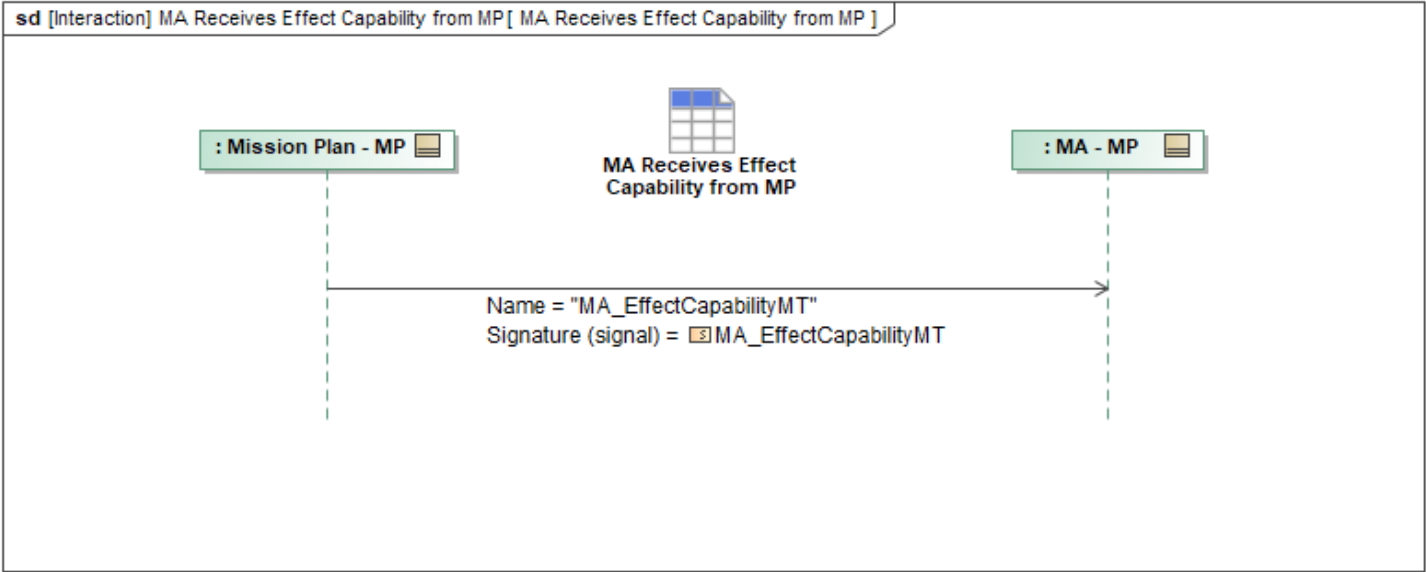


Figure 7-13: MA Receives Effect Capability from MP

Table 7-11: MA Receives Effect Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EffectCapabilityMT: MA_EffectCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.12 MA Receives ESM Capability from MP

Mission Use Case: ESM

This interaction describes how Mission Autonomy (MA) receives *ESM_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

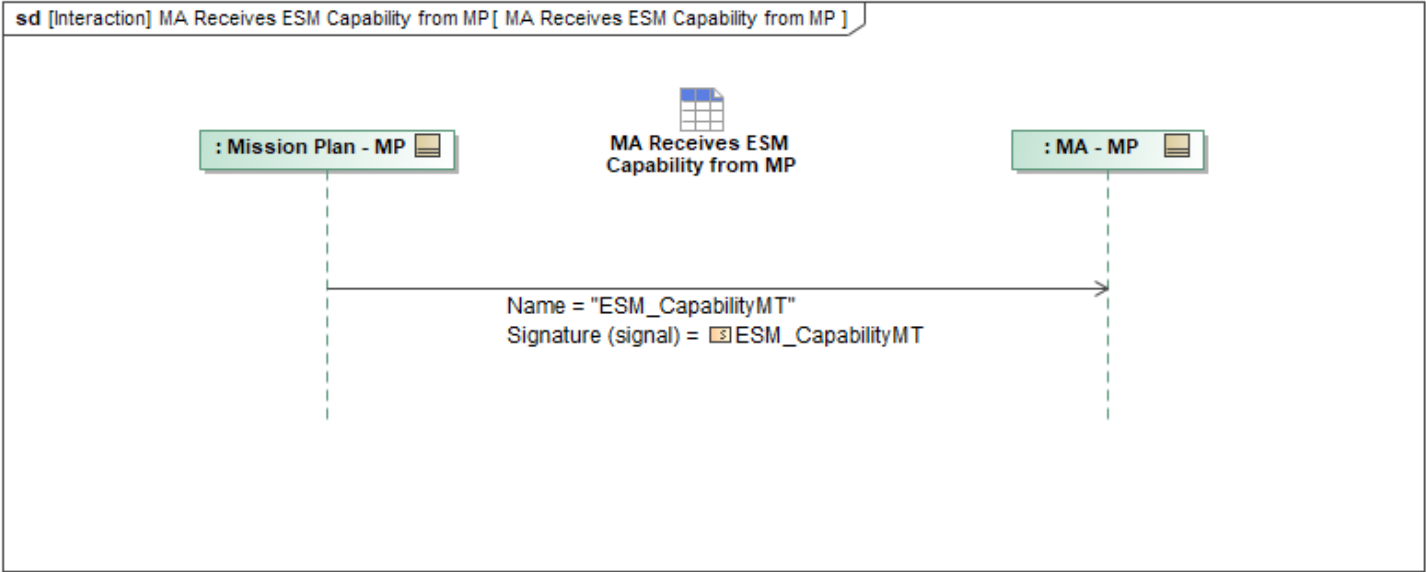


Figure 7-14: MA Receives ESM Capability from MP

Table 7-12: MA Receives ESM Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
ESM_CapabilityMT: ESM_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.13 MA Receives Flight Capability from MP

This interaction describes how Mission Autonomy (MA) receives *MA_FlightCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

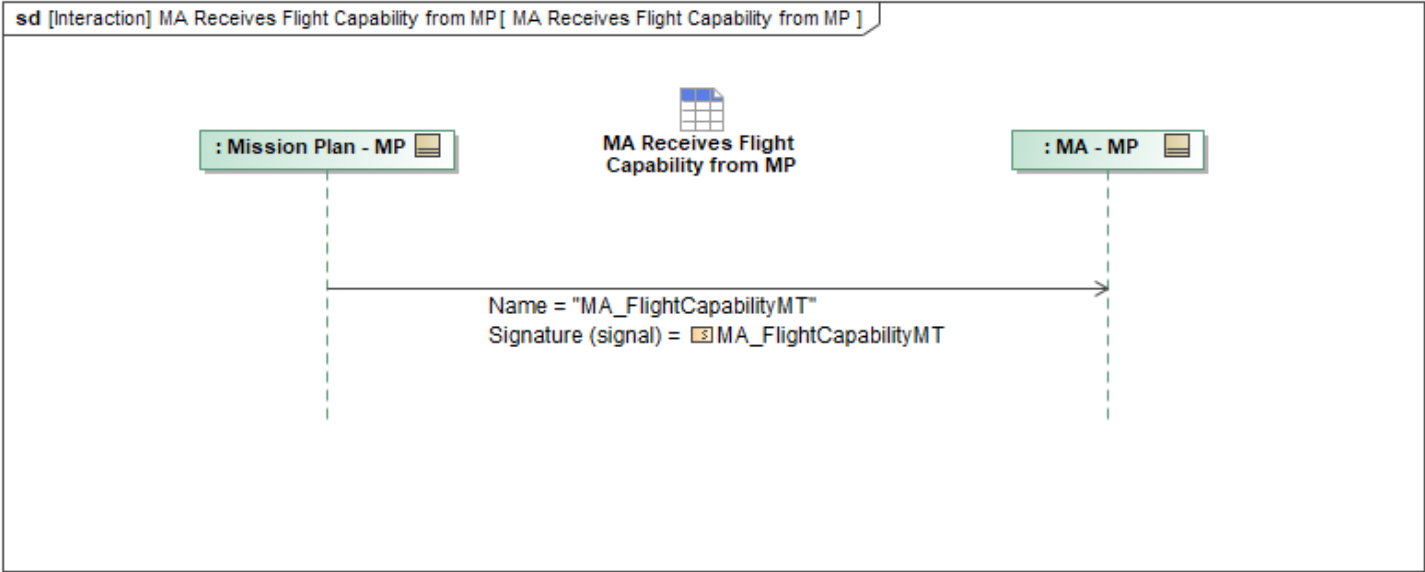


Figure 7-15: MA Receives Flight Capability from MP

Table 7-13: MA Receives Flight Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.14 MA Receives Gateway Capability from MP

This interaction describes how Mission Autonomy (MA) receives *GatewayCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

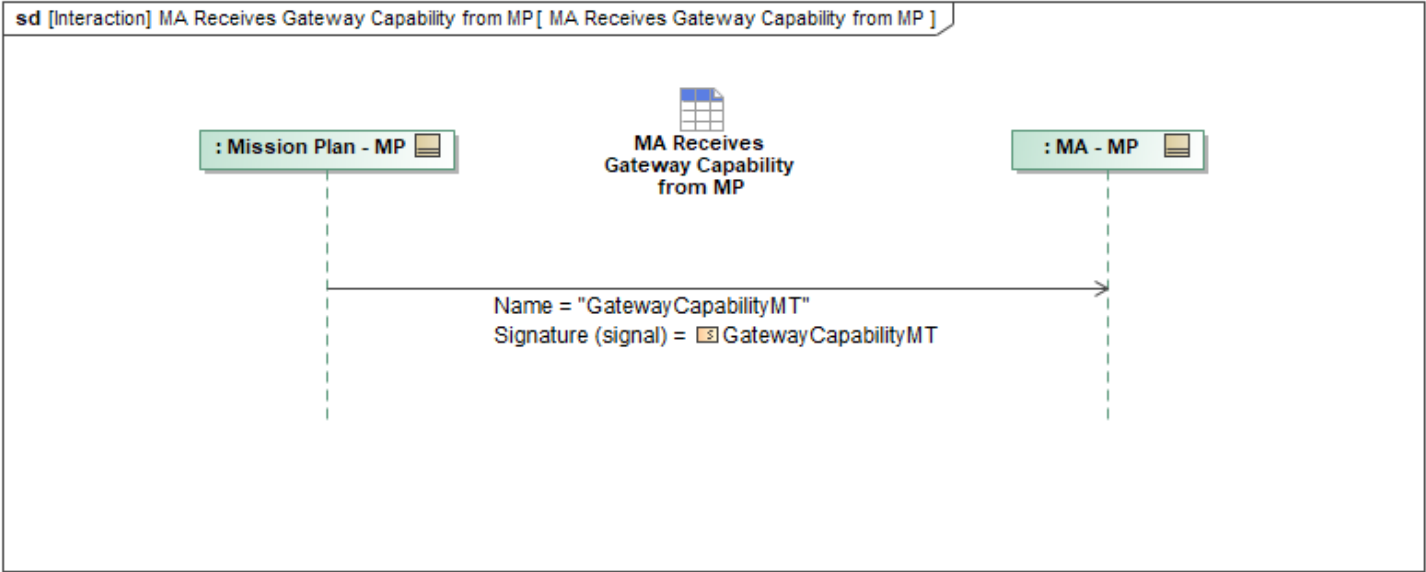


Figure 7-16: MA Receives Gateway Capability from MP

Table 7-14: MA Receives Gateway Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
GatewayCapabilityMT: GatewayCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.15 MA Receives IFF Capability from MP

This interaction describes how Mission Autonomy (MA) receives *IFF_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

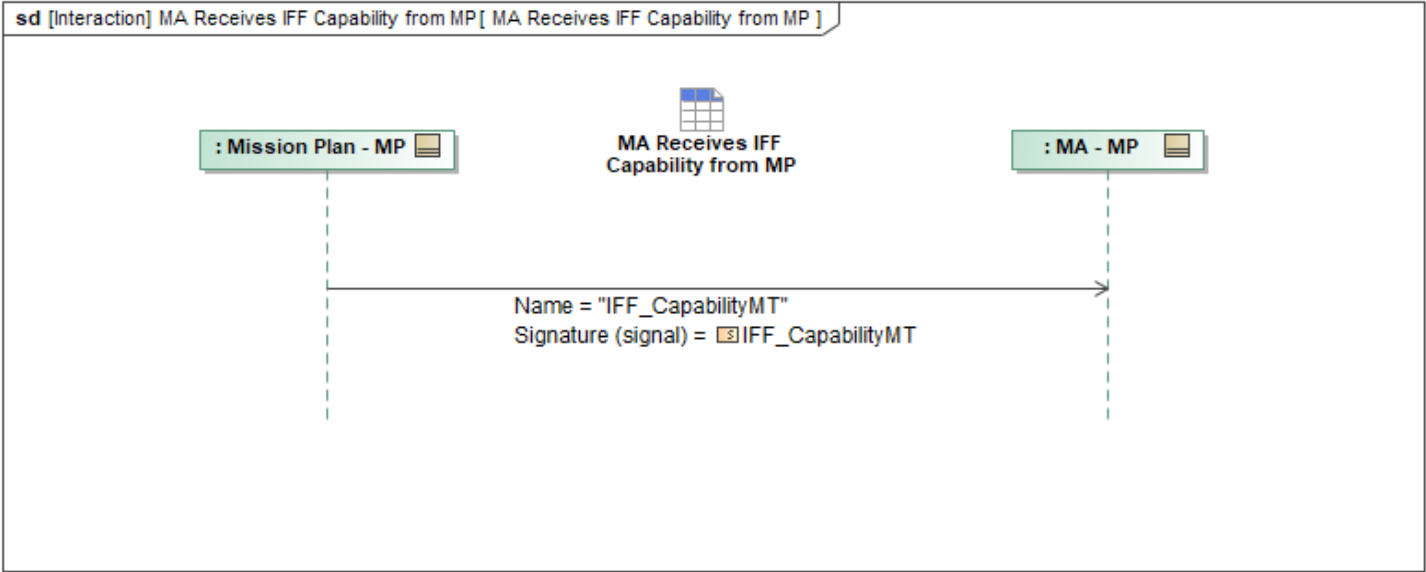


Figure 7-17: MA Receives IFF Capability from MP

Table 7-15: MA Receives IFF Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
IFF_CapabilityMT: IFF_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.16 MA Receives Opaque Capability from MP

This interaction describes how Mission Autonomy (MA) receives *OpaqueCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

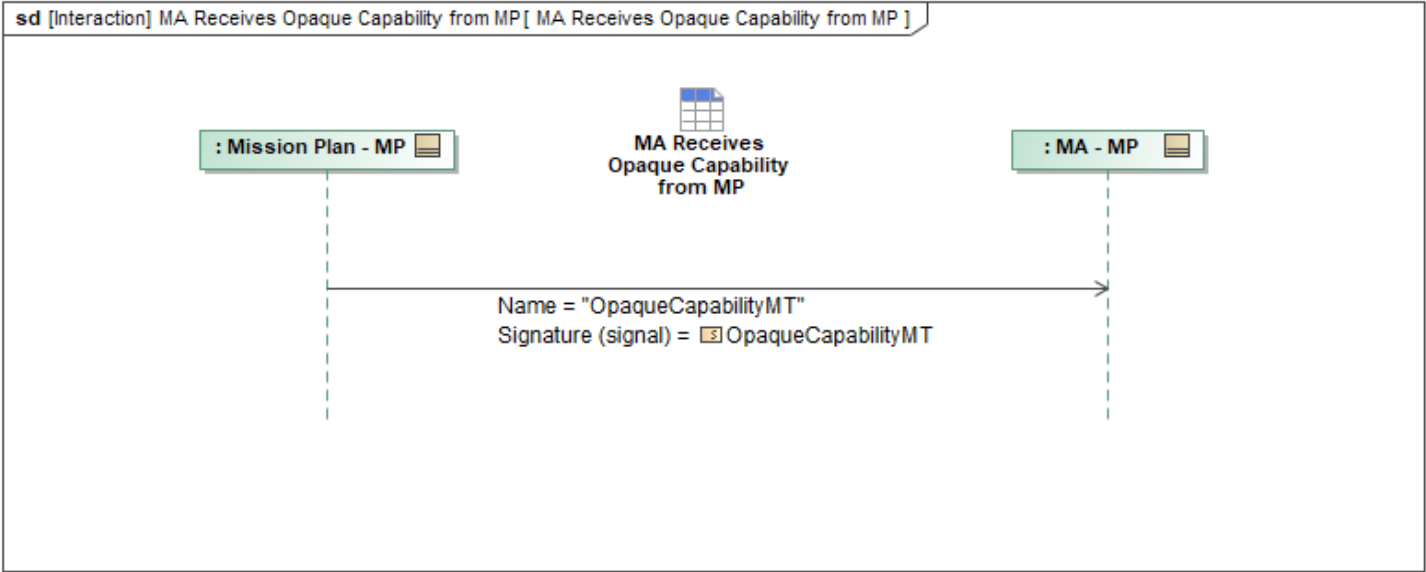


Figure 7-18: MA Receives Opaque Capability from MP

Table 7-16: MA Receives Opaque Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
OpaqueCapabilityMT: OpaqueCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.17 MA Receives Orbit Change Capability from MP

This interaction describes how Mission Autonomy (MA) receives *OrbitChangeCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

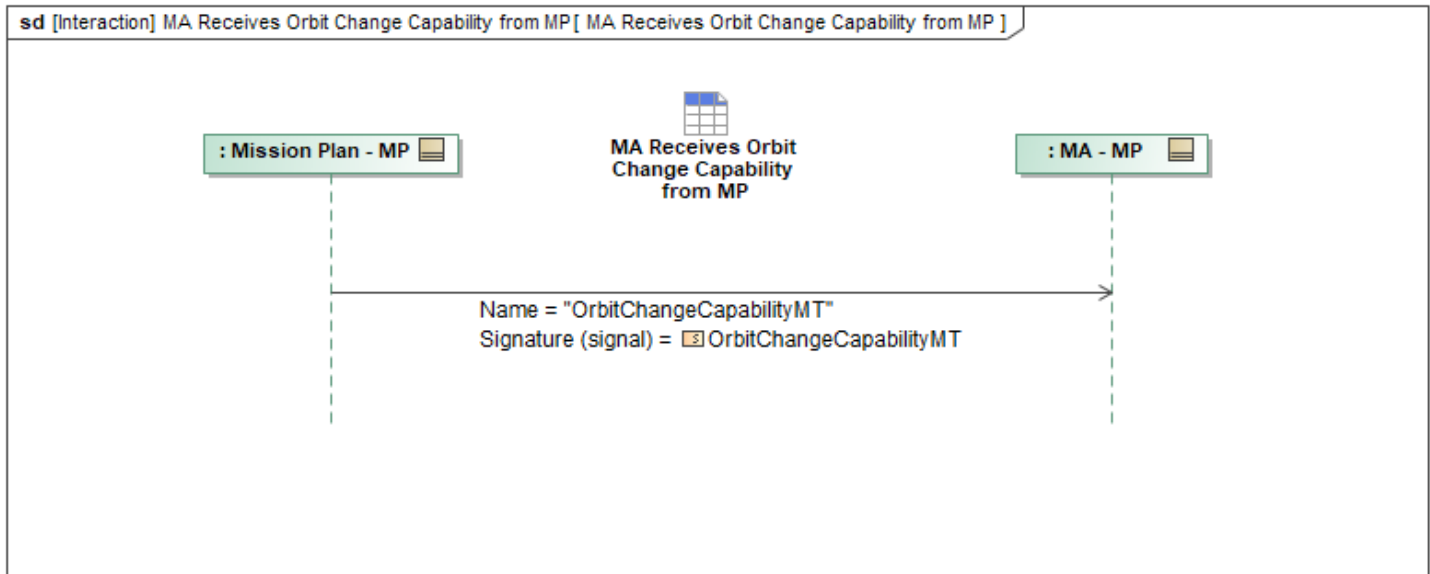


Figure 7-19: MA Receives Orbit Change Capability from MP

Table 7-17: MA Receives Orbit Change Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
OrbitChangeCapabilityMT: OrbitChangeCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.18 MA Receives Orbital Surveillance Capability from MP

This interaction describes how Mission Autonomy (MA) receives *OrbitalSurveillanceCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

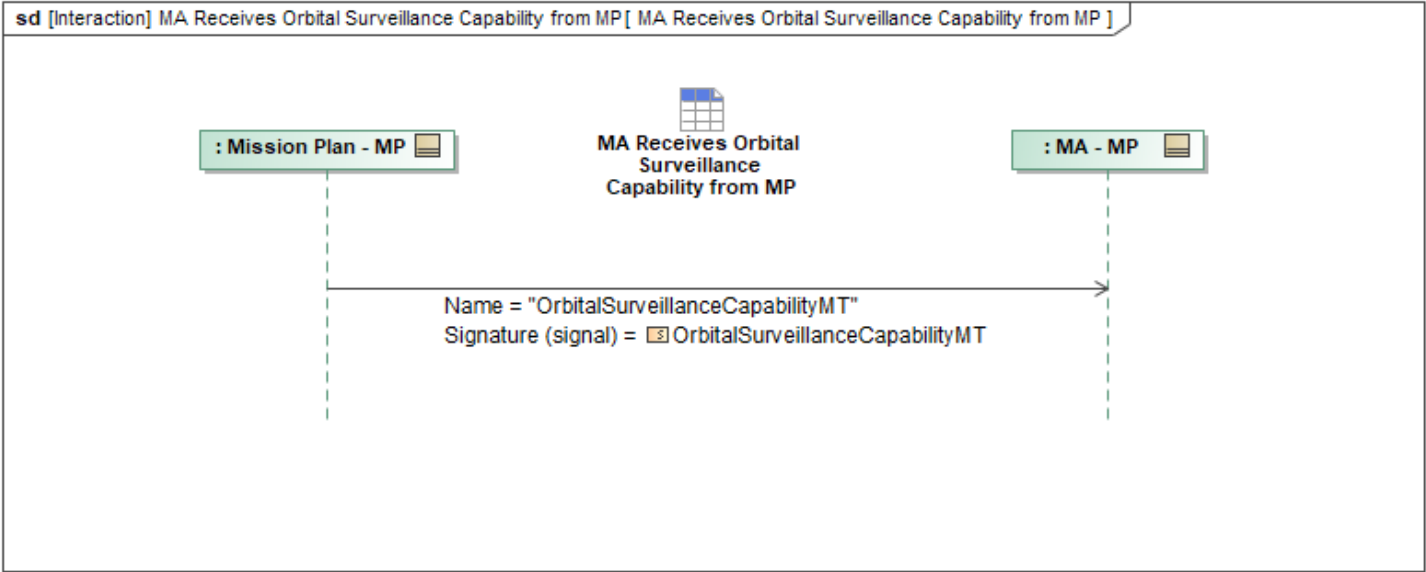


Figure 7-20: MA Receives Orbital Surveillance Capability from MP

Table 7-18: MA Receives Orbital Surveillance Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
OrbitalSurveillanceCapabilityMT: OrbitalSurveillanceCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.19 MA Receives Orbital Surveillance Sensor Capability from MP

This interaction describes how Mission Autonomy (MA) receives *OrbitalSurveillanceSensorCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

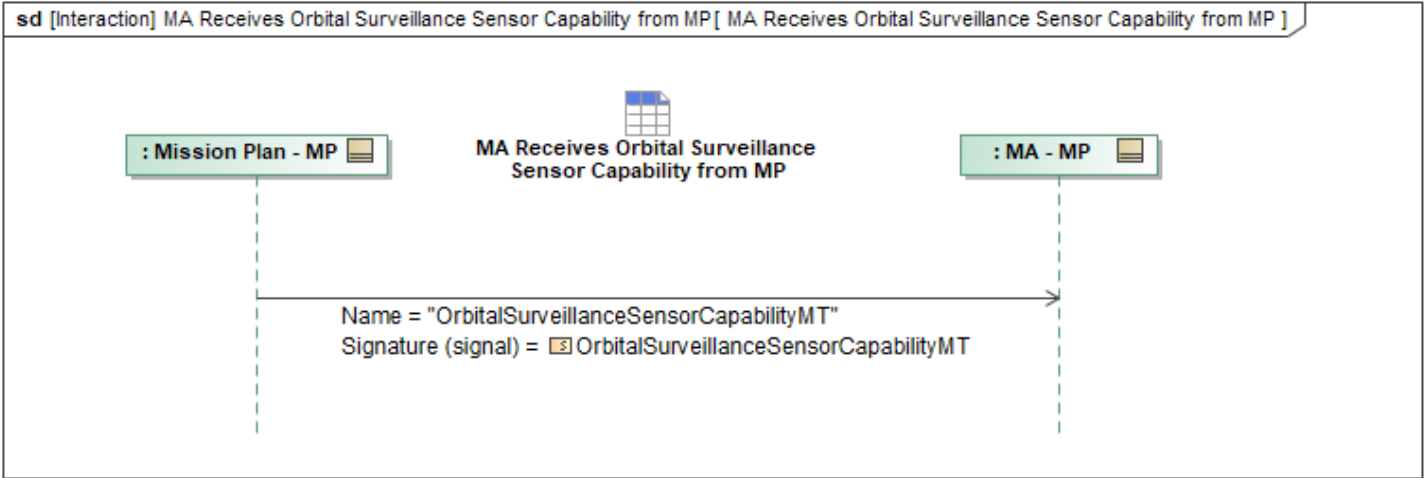


Figure 7-21: MA Receives Orbital Surveillance Sensor Capability from MP

Table 7-19: MA Receives Orbital Surveillance Sensor Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
OrbitalSurveillanceSensorCapabilityMT: OrbitalSurveillanceSensorCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.20 MA Receives PO Capability from MP

Mission Use Case: PO

This interaction describes how Mission Autonomy (MA) receives *PO_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

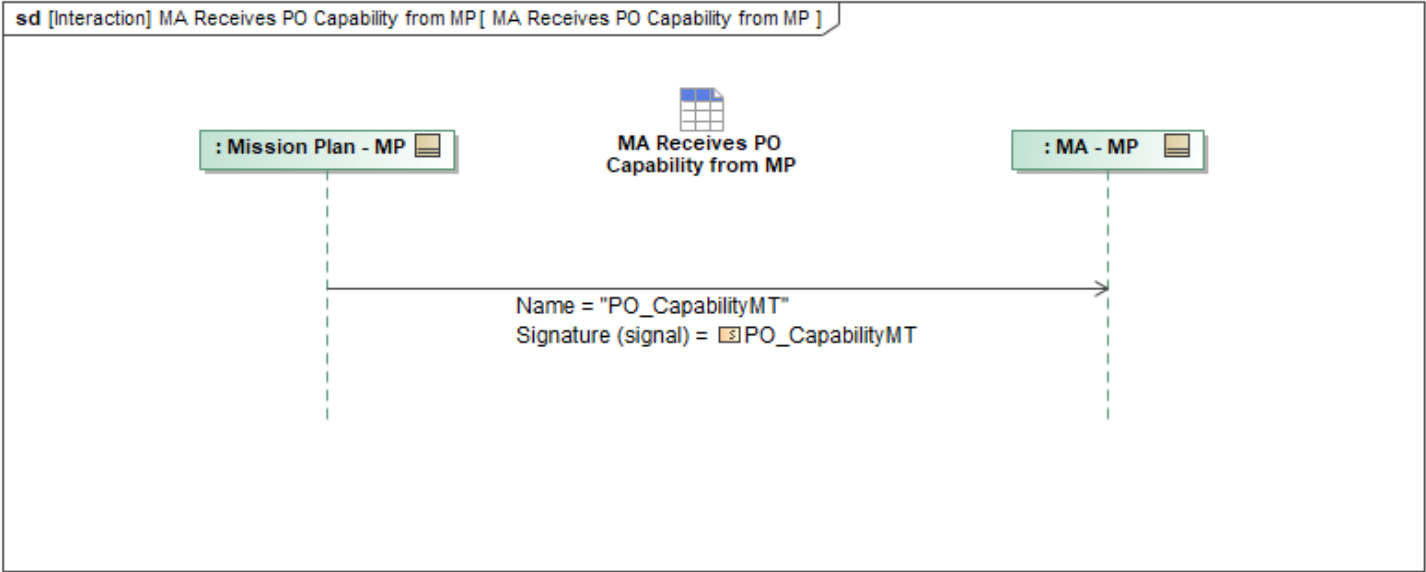


Figure 7-22: MA Receives PO Capability from MP

Table 7-20: MA Receives PO Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
PO_CapabilityMT: PO_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.21 MA Receives Radar Altimeter Capability from MP

This interaction describes how Mission Autonomy (MA) receives *RadarAltimeterCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

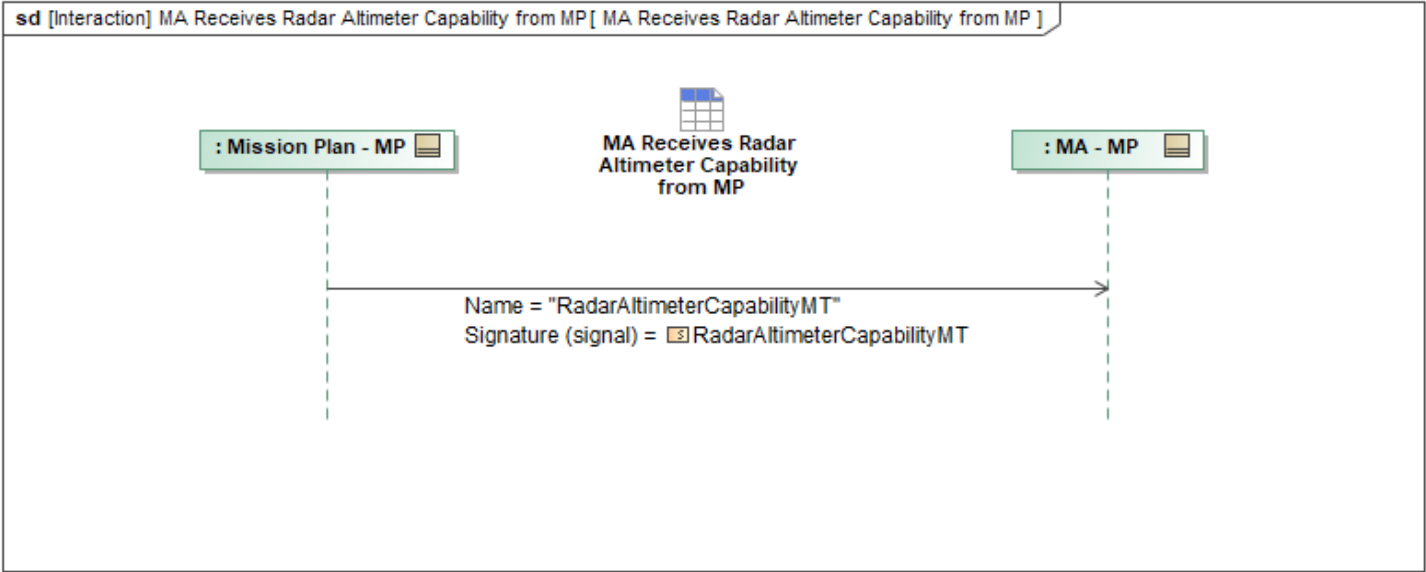


Figure 7-23: MA Receives Radar Altimeter Capability from MP

Table 7-21: MA Receives Radar Altimeter Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
RadarAltimeterCapabilityMT: RadarAltimeterCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.22 MA Receives Reference Capability from MP

This interaction describes how Mission Autonomy (MA) receives *ReferenceCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

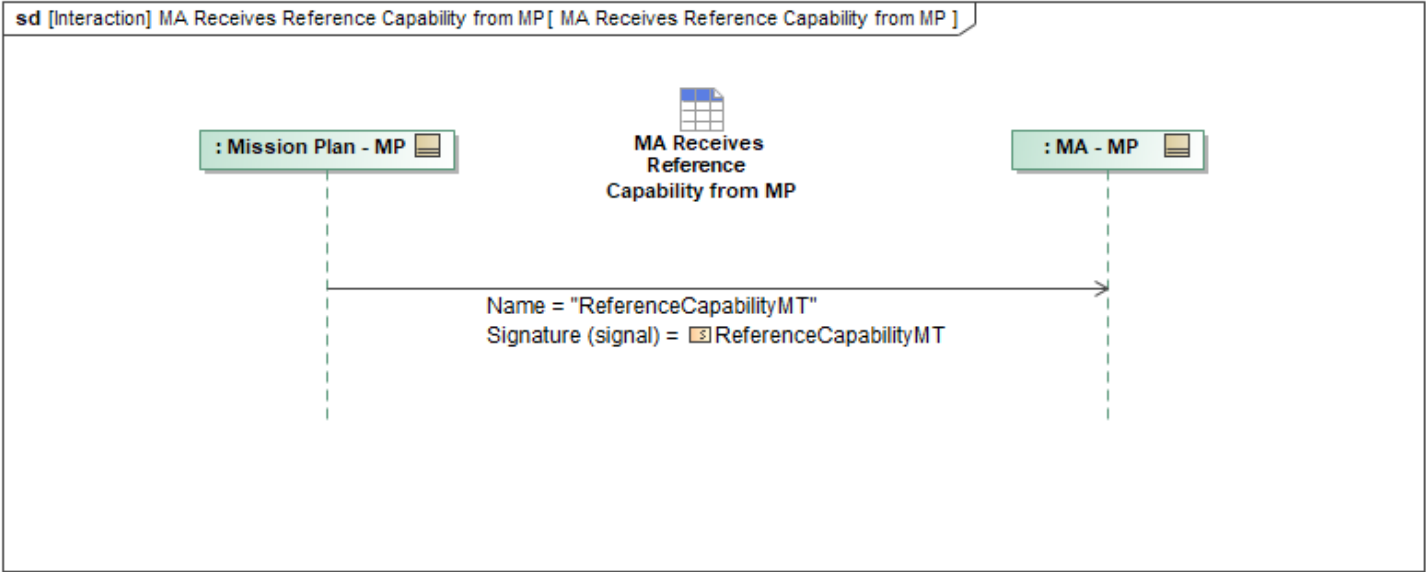


Figure 7-24: MA Receives Reference Capability from MP

Table 7-22: MA Receives Reference Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
ReferenceCapabilityMT: ReferenceCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.23 MA Receives Refuel Capability from MP

This interaction describes how Mission Autonomy (MA) receives *RefuelCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

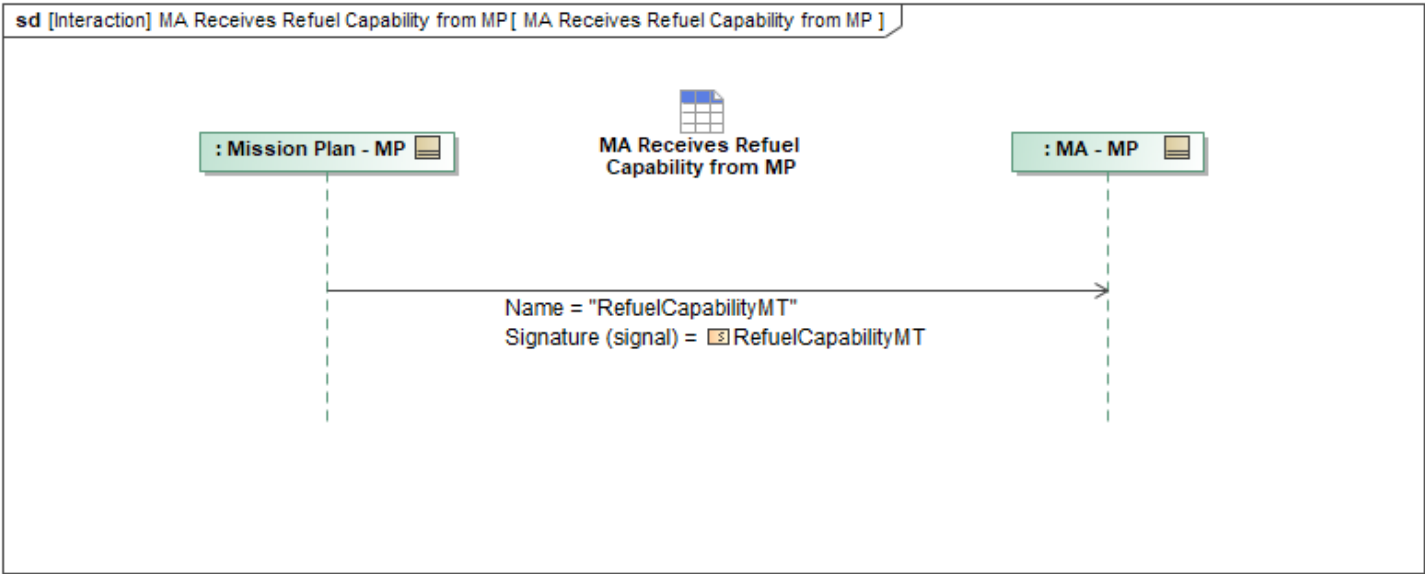


Figure 7-25: MA Receives Refuel Capability from MP

Table 7-23: MA Receives Refuel Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
RefuelCapabilityMT: RefuelCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.24 MA Receives Response Capability from MP

This interaction describes how Mission Autonomy (MA) receives *ResponseCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

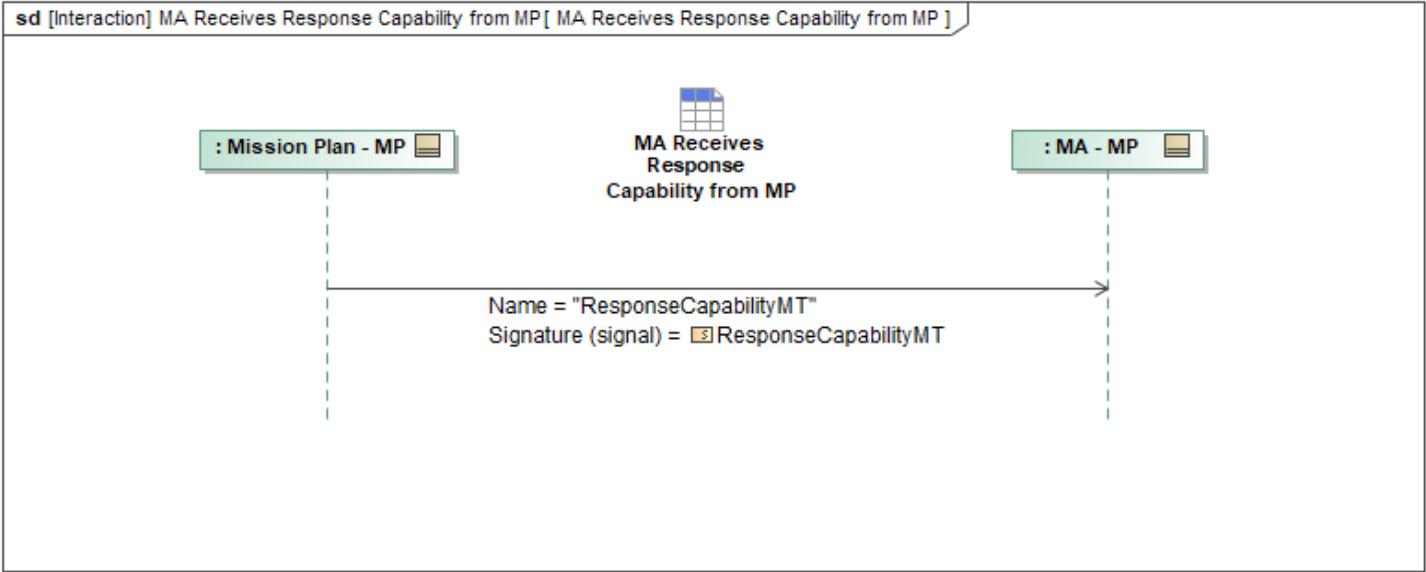


Figure 7-26: MA Receives Response Capability from MP

Table 7-24: MA Receives Response Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
ResponseCapabilityMT: ResponseCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.25 MA Receives SAR Capability from MP

This interaction describes how Mission Autonomy (MA) receives *SAR_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

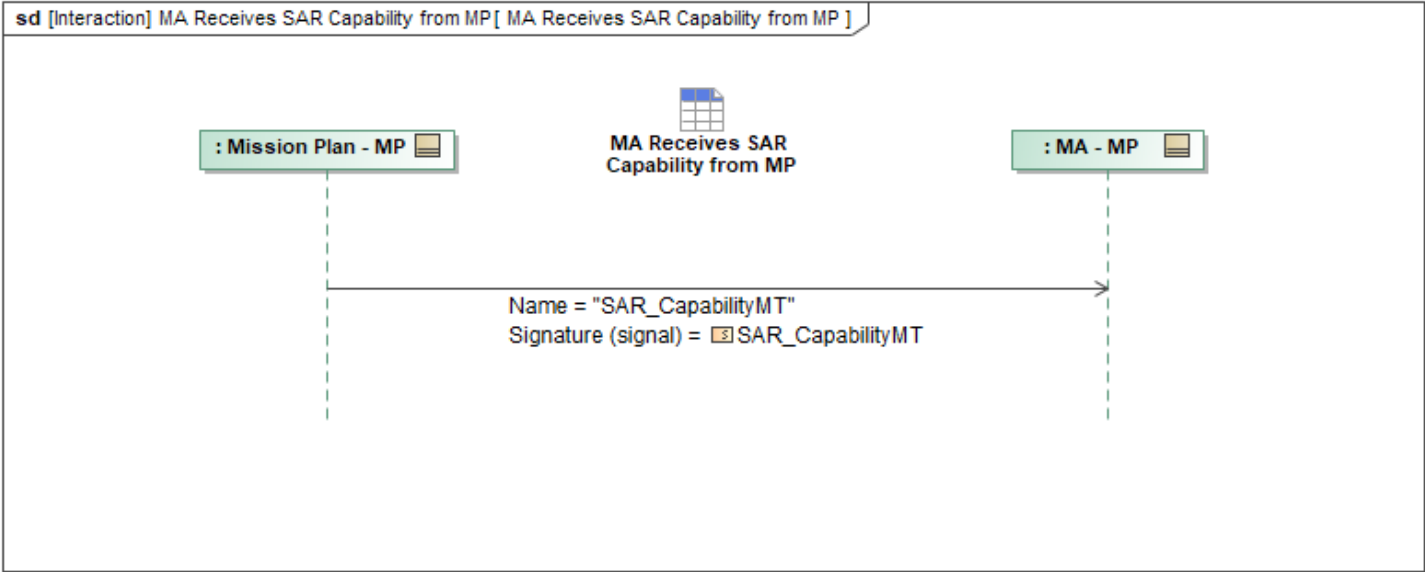


Figure 7-27: MA Receives SAR Capability from MP

Table 7-25: MA Receives SAR Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
SAR_CapabilityMT: SAR_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.26 MA Receives SMTI Capability from MP

This interaction describes how Mission Autonomy (MA) receives *SMTI_CapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

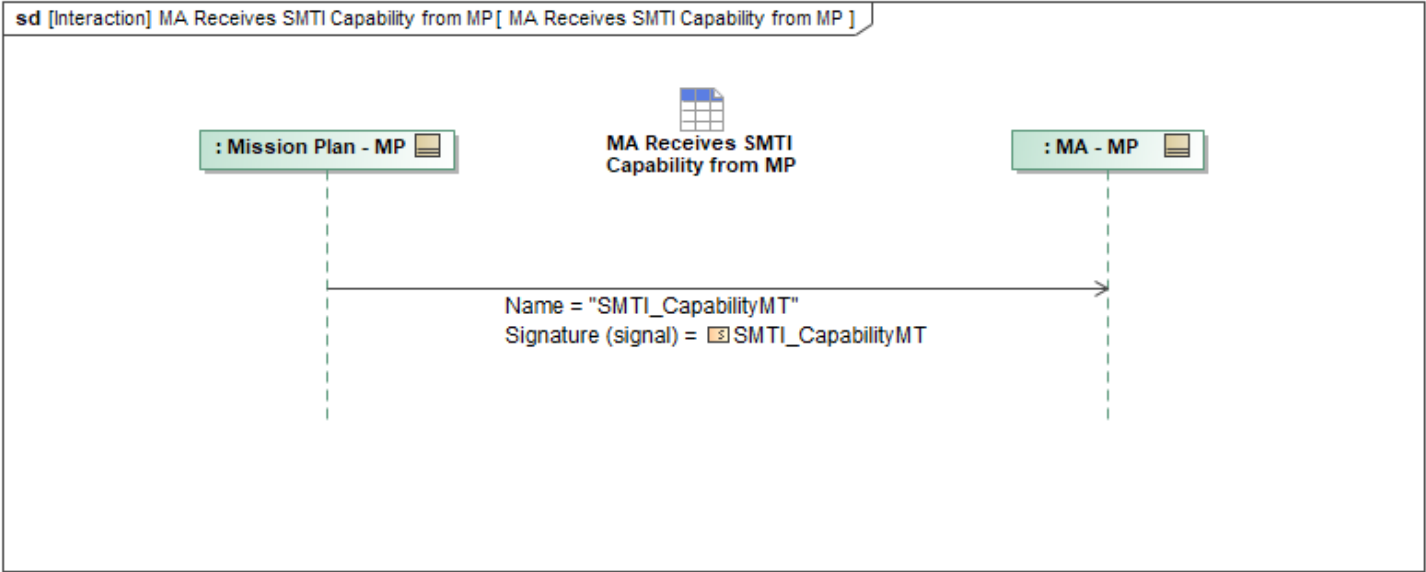


Figure 7-28: MA Receives SMTI Capability from MP

Table 7-26: MA Receives SMTI Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
SMTI_CapabilityMT: SMTI_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.27 MA Receives Strike Capability from MP

Mission Use Case: Store Management

This interaction describes how Mission Autonomy (MA) receives *MA_StrikeCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

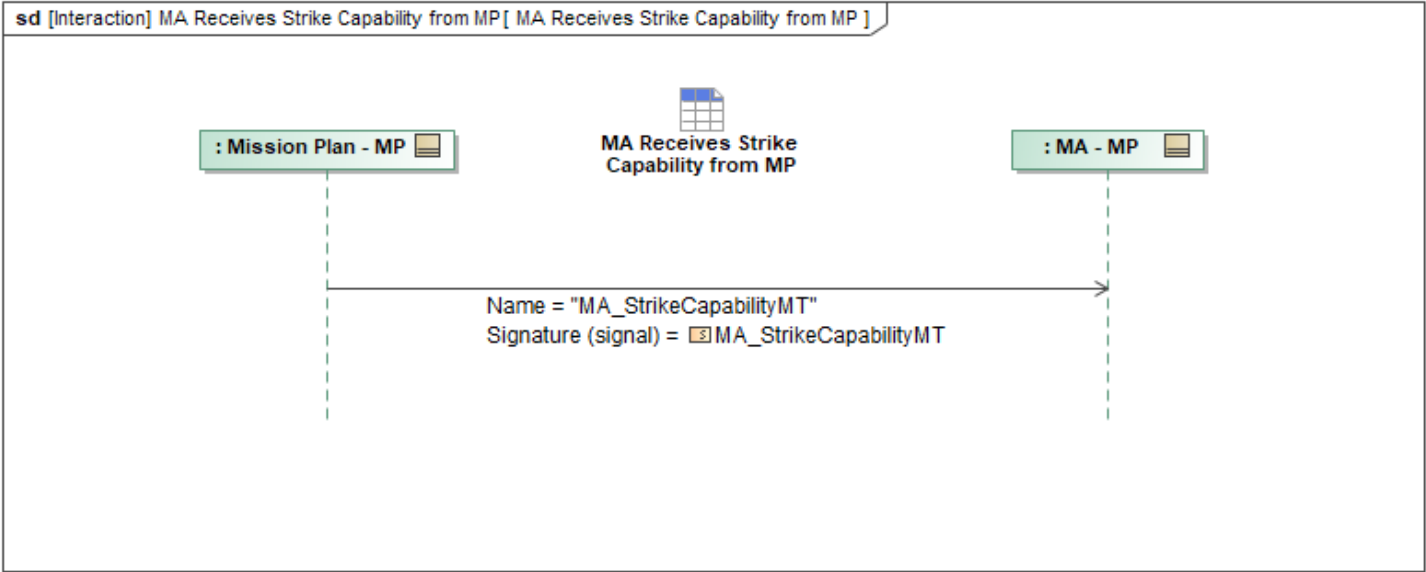


Figure 7-29: MA Receives Strike Capability from MP

Table 7-27: MA Receives Strike Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.28 MA Receives System Deployment Capability from MP

This interaction describes how Mission Autonomy (MA) receives *SystemDeploymentCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

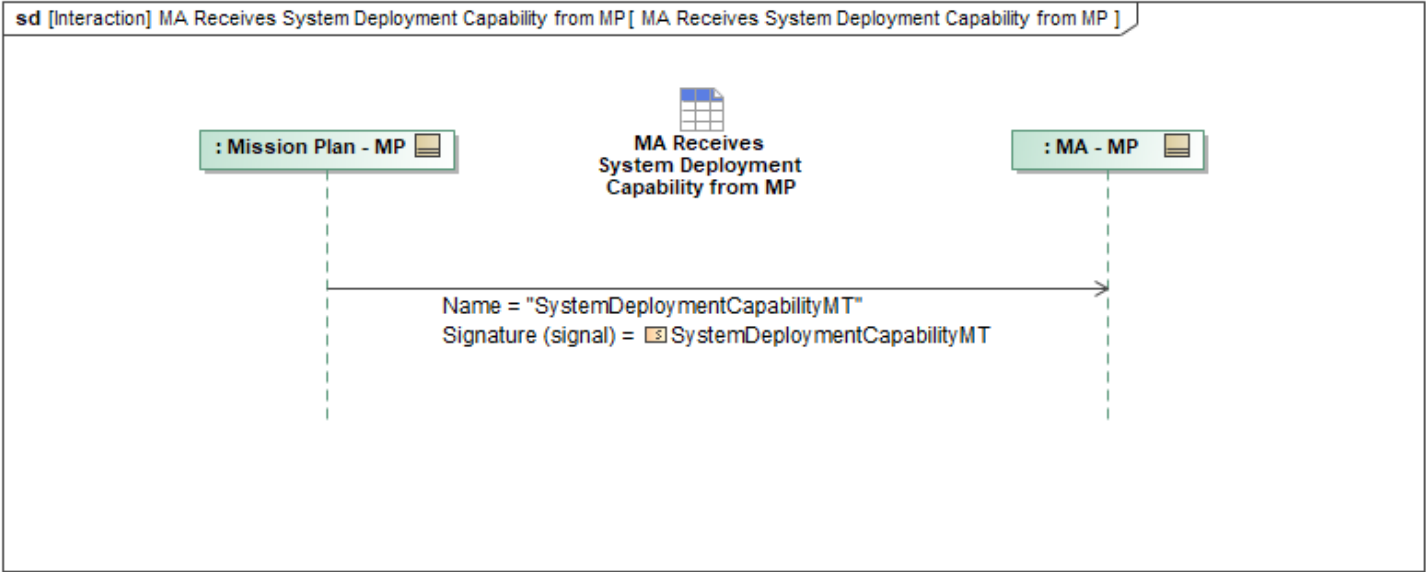


Figure 7-30: MA Receives System Deployment Capability from MP

Table 7-28: MA Receives System Deployment Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
SystemDeploymentCapabilityMT: SystemDeploymentCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.29 MA Receives Tactical Order Capability from MP

This interaction describes how Mission Autonomy (MA) receives *TacticalOrderCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

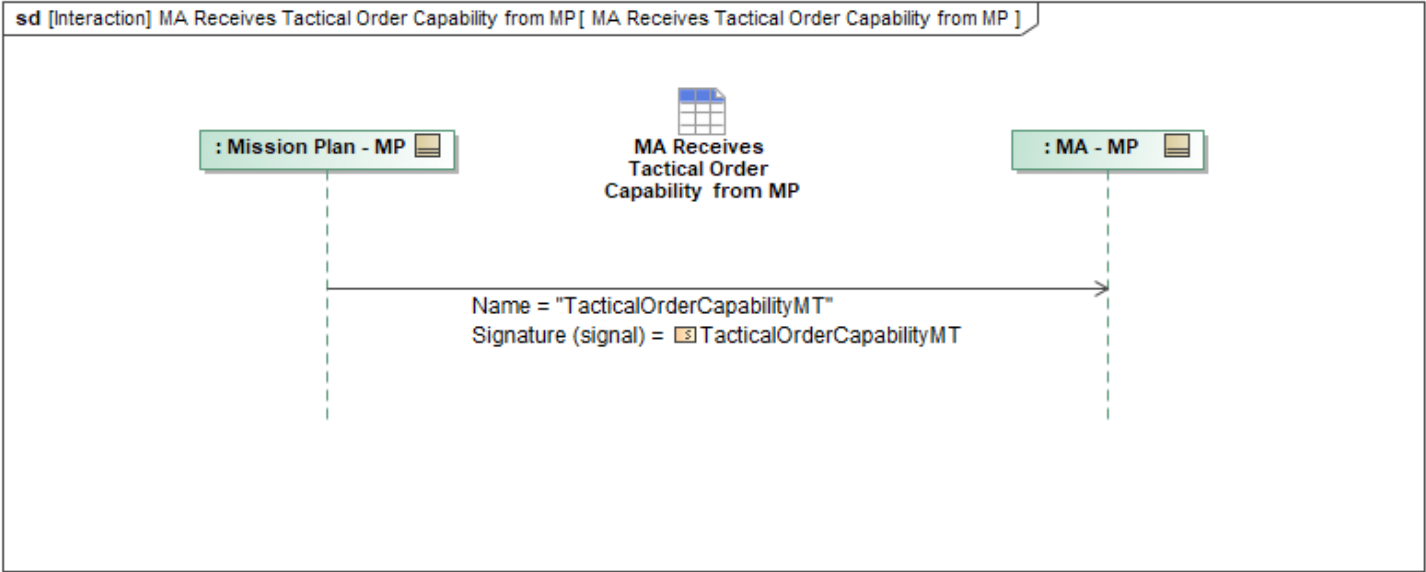


Figure 7-31: MA Receives Tactical Order Capability from MP

Table 7-29: MA Receives Tactical Order Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
TacticalOrderCapabilityMT: TacticalOrderCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.1.30 MA Receives Weather Radar Capability from MP

Mission Use Case: Weather Radar

This interaction describes how Mission Autonomy (MA) receives *WeatherRadarCapabilityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

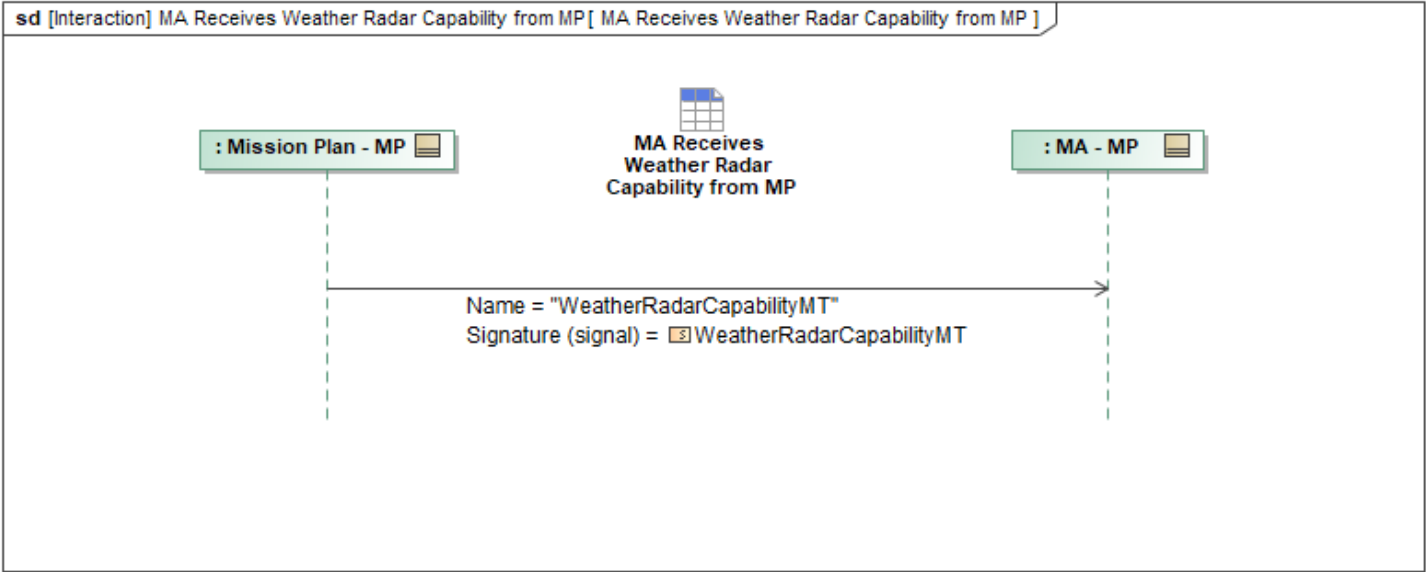


Figure 7-32: MA Receives Weather Radar Capability from MP

Table 7-30: MA Receives Weather Radar Capability from MP

Message Name (Name:Type)	Required Fields (Name:Type)
WeatherRadarCapabilityMT: WeatherRadarCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2 Mission Plan Package

7.2.2.1 MA Receives Action from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ActionMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

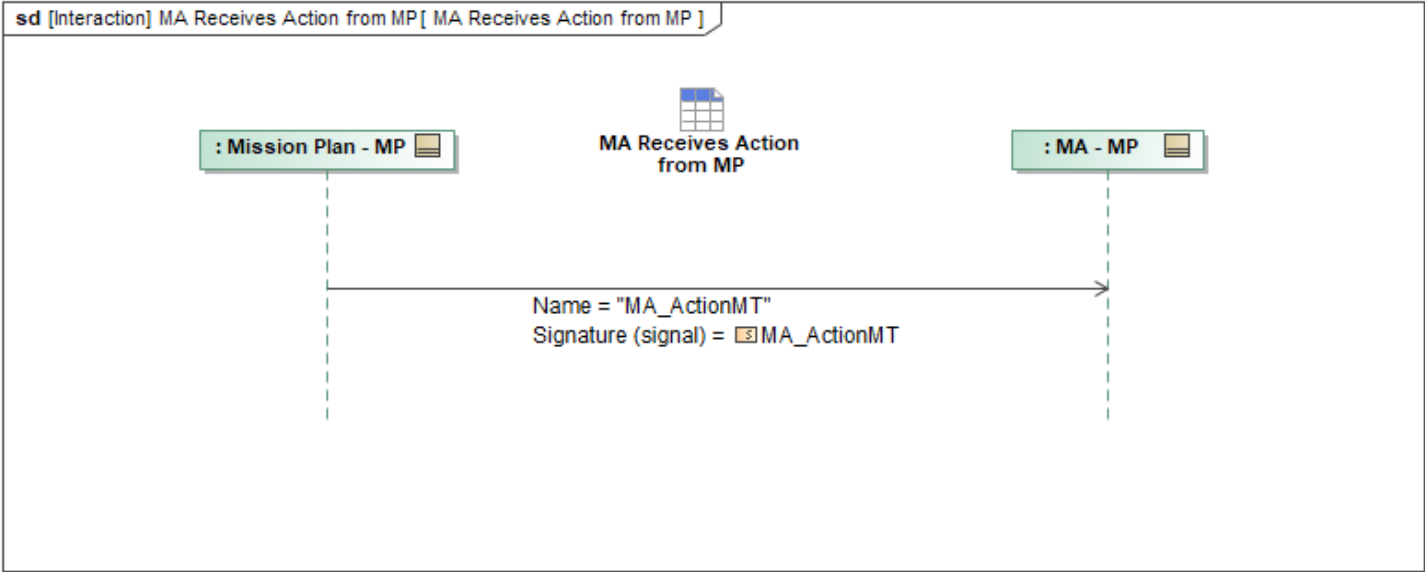


Figure 7-33: MA Receives Action from MP

Table 7-31: MA Receives Action from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionMT: MA_ActionMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.2 MA Receives Action Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ActionPlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

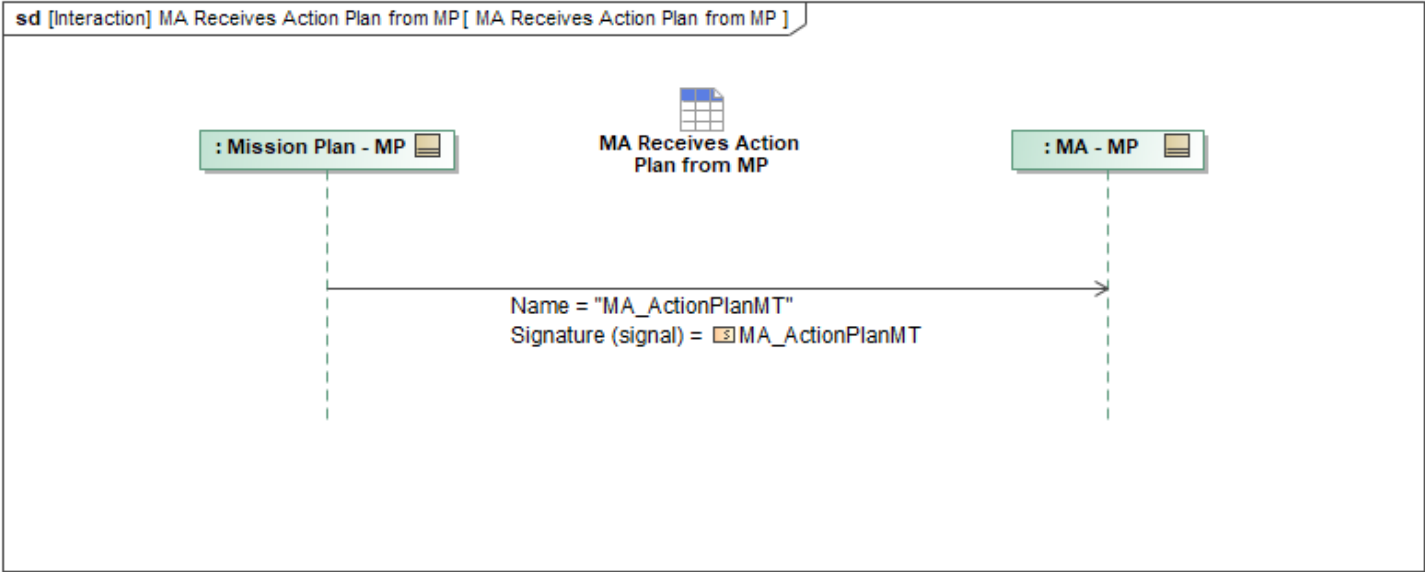


Figure 7-34: MA Receives Action Plan from MP

Table 7-32: MA Receives Action Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionPlanMT: MA_ActionPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.3 MA Receives Activity Metrics from MP

This interaction describes how Mission Autonomy (MA) receives *ActivityMetricsMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

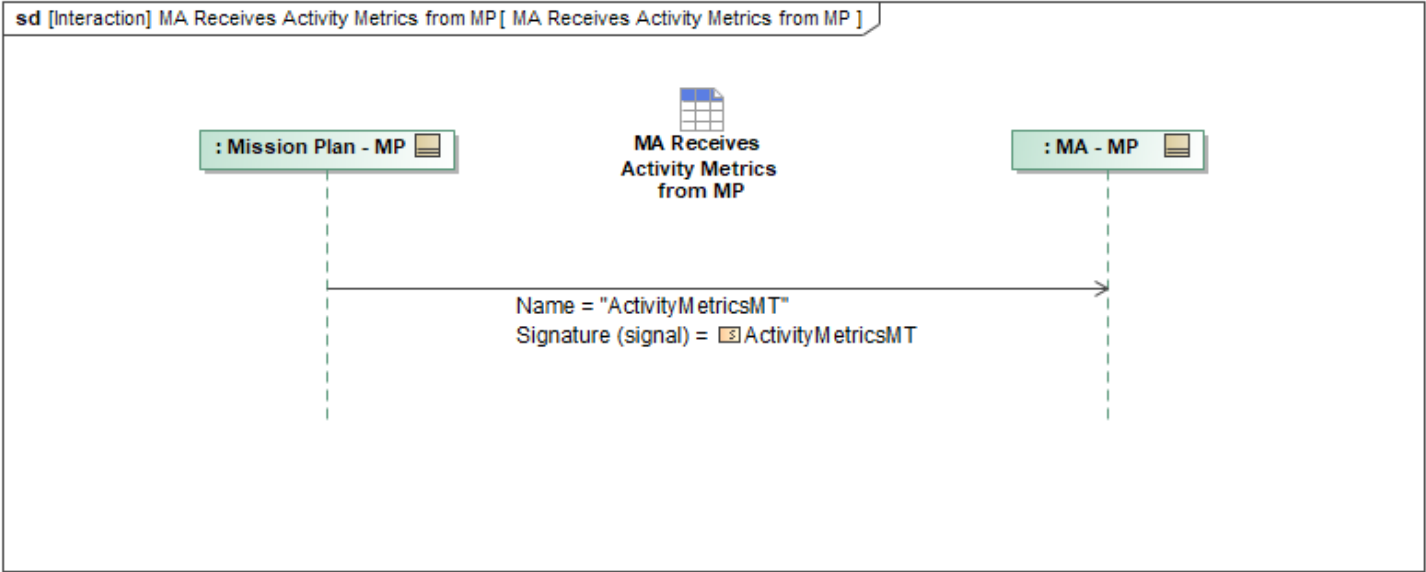


Figure 7-35: MA Receives Activity Metrics from MP

Table 7-33: MA Receives Activity Metrics from MP

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityMetricsMT: ActivityMetricsMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.4 MA Receives Activity Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ActivityPlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

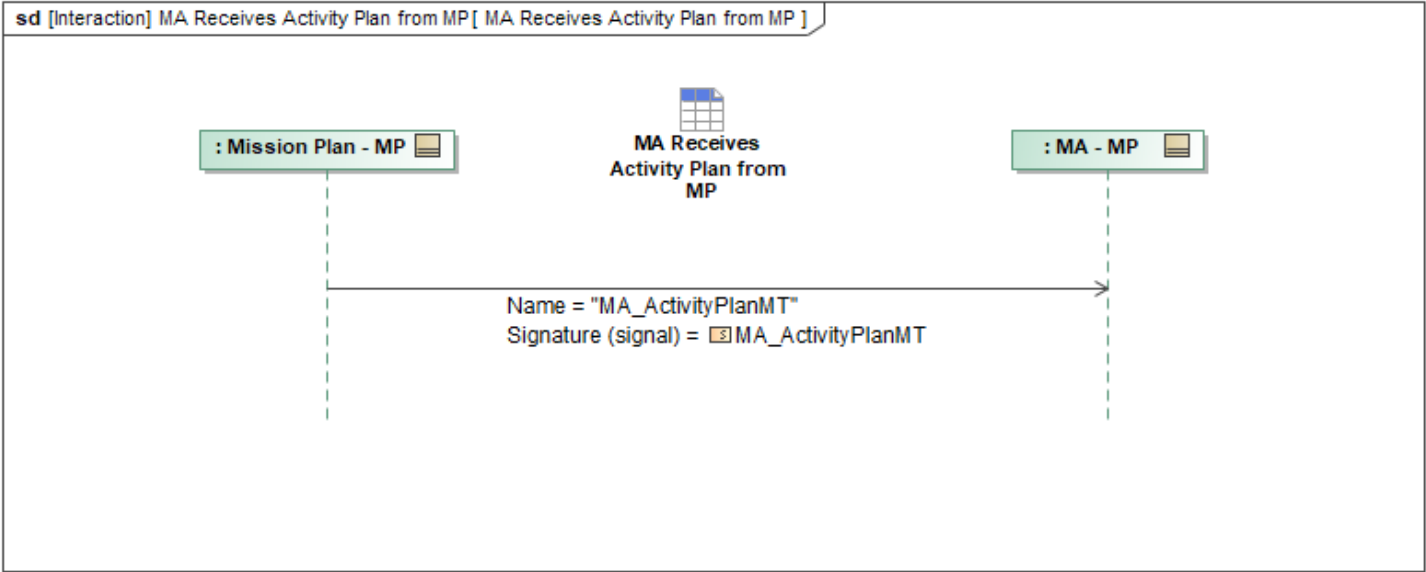


Figure 7-36: MA Receives Activity Plan from MP

Table 7-34: MA Receives Activity Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.5 MA Receives Analysis Route from MP

This interaction describes how Mission Autonomy (MA) receives *AnalysisRouteMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

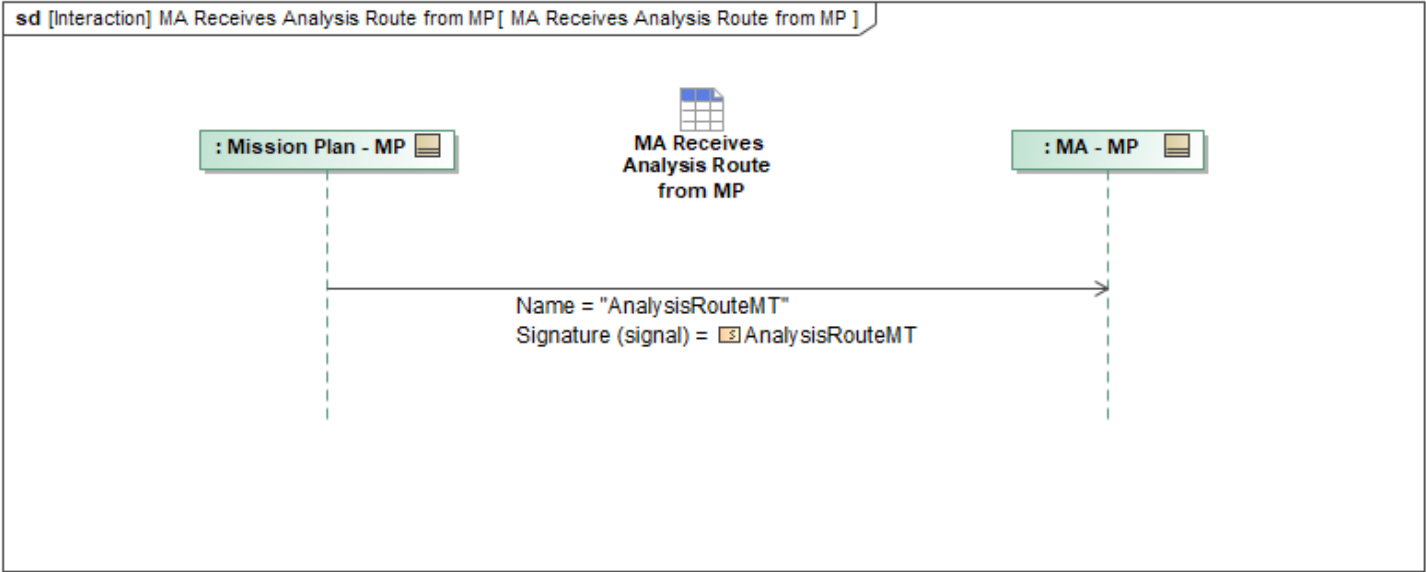


Figure 7-37: MA Receives Analysis Route from MP

Table 7-35: MA Receives Analysis Route from MP

Message Name (Name:Type)	Required Fields (Name:Type)
AnalysisRouteMT: AnalysisRouteMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.6 MA Receives Mission Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_MissionPlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

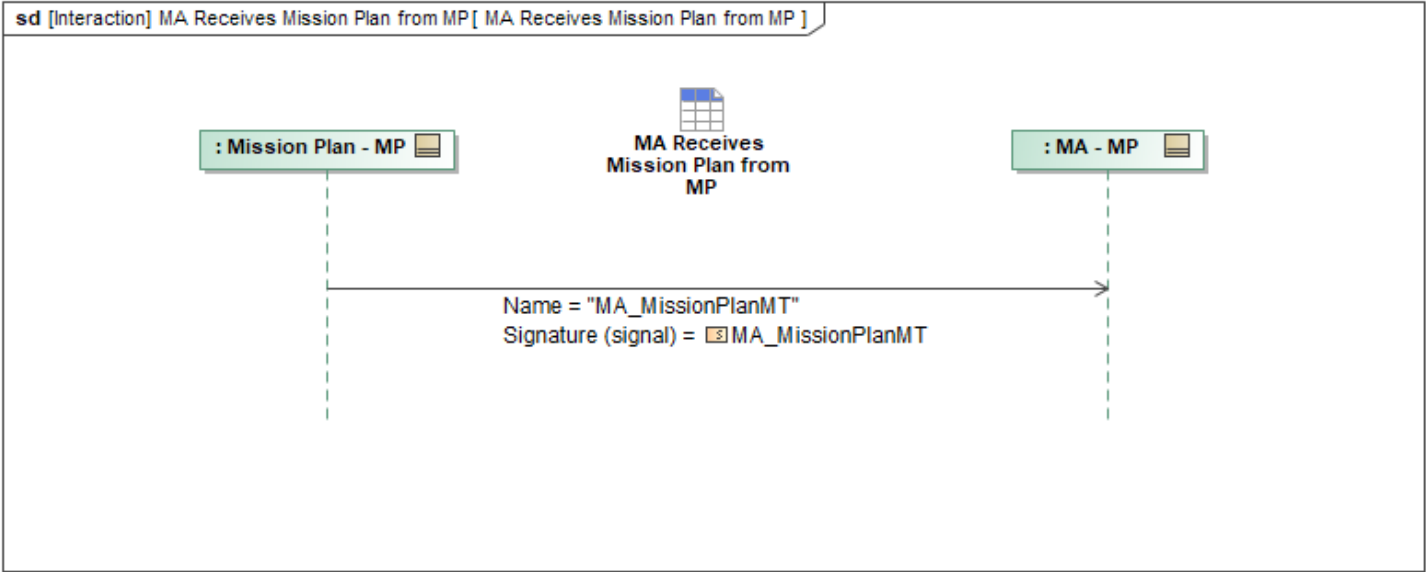


Figure 7-38: MA Receives Mission Plan from MP

Table 7-36: MA Receives Mission Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.7 MA Receives OpPlan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OpPlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

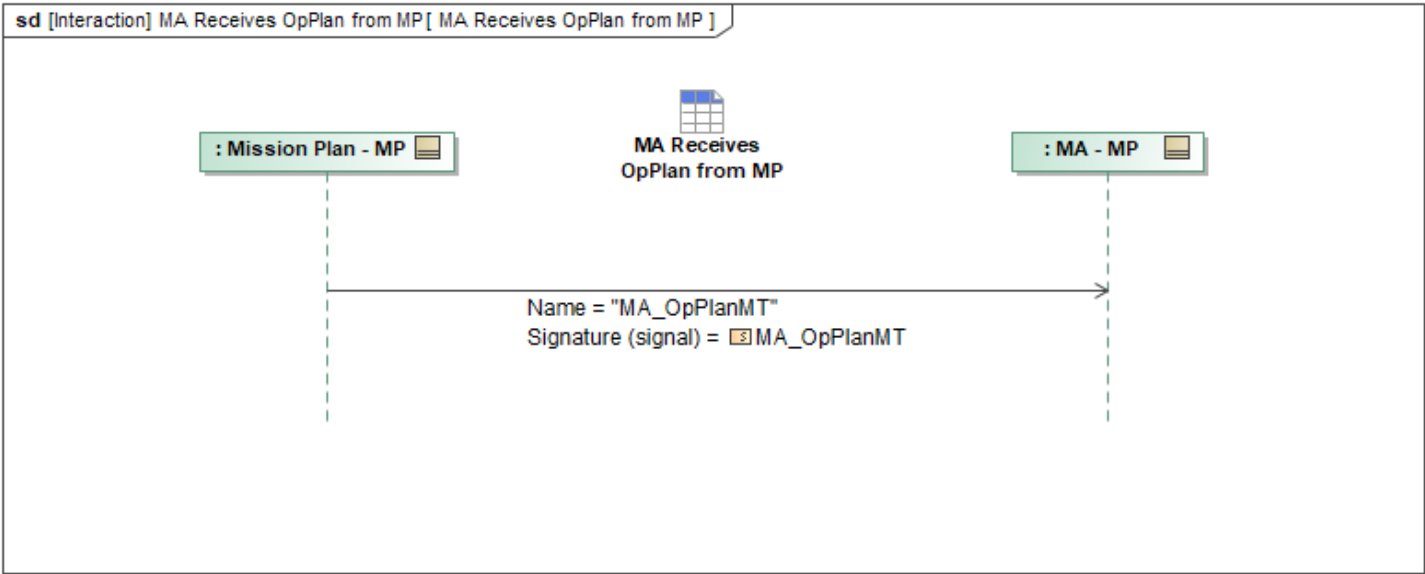


Figure 7-39: MA Receives OpPlan from MP

Table 7-37: MA Receives OpPlan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPlanMT: MA_OpPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.8 MA Receives Response from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ResponseMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

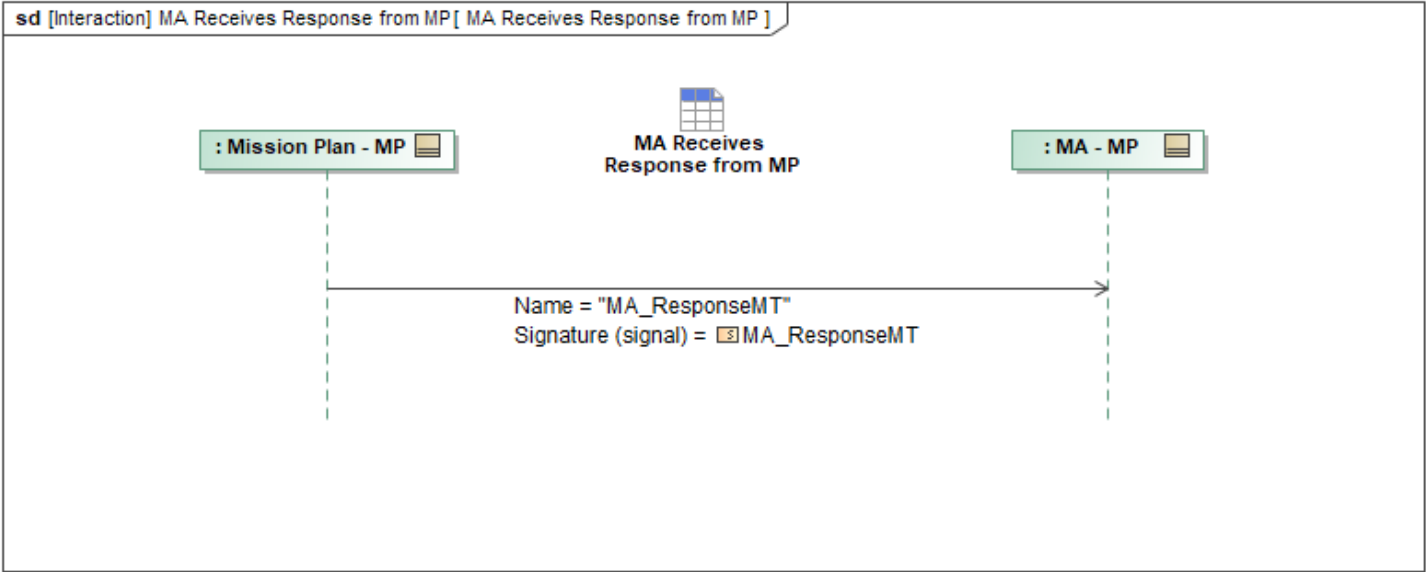


Figure 7-40: MA Receives Response from MP

Table 7-38: MA Receives Response from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseMT: MA_ResponseMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.9 MA Receives Response Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ResponsePlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

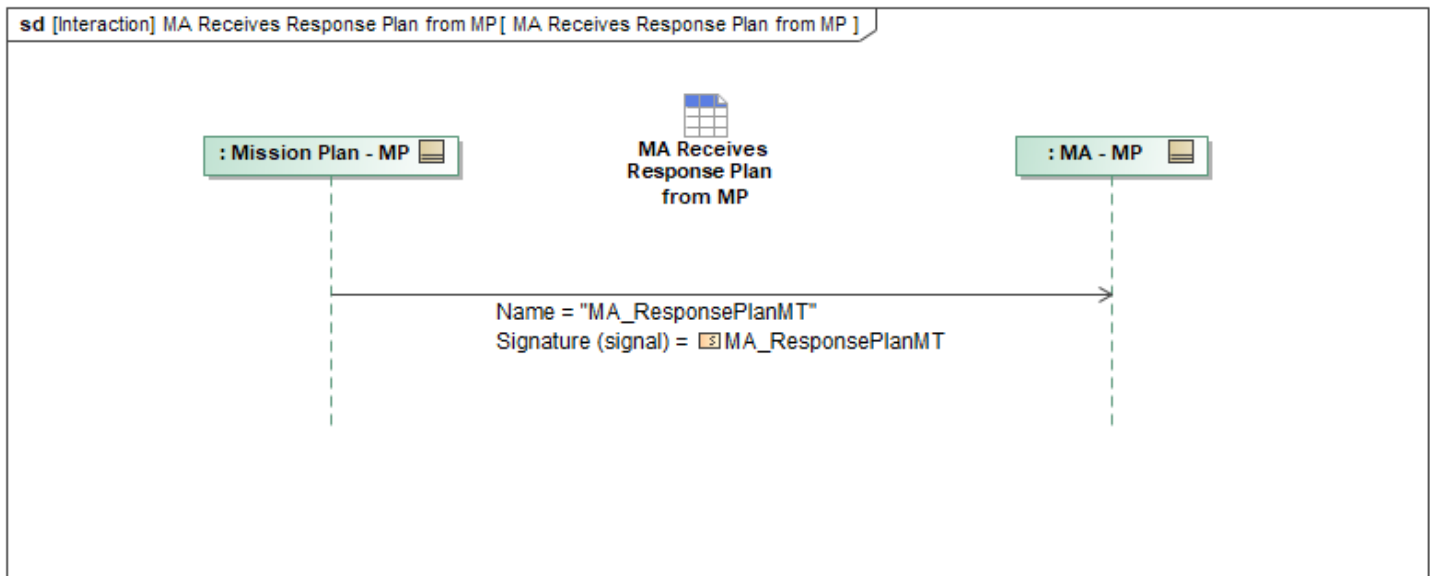


Figure 7-41: MA Receives Response Plan from MP

Table 7-39: MA Receives Response Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponsePlanMT: MA_ResponsePlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.10 MA Receives Route Activity Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_RouteActivityPlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

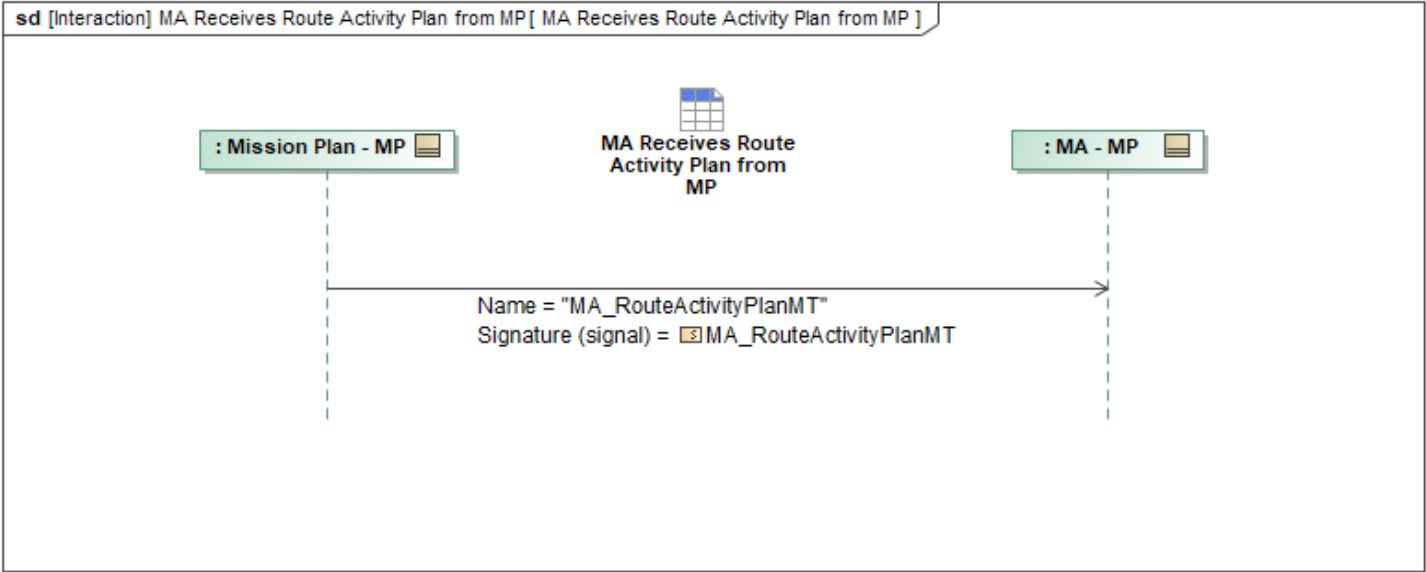


Figure 7-42: MA Receives Route Activity Plan from MP

Table 7-40: MA Receives Route Activity Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteActivityPlanMT: MA_RouteActivityPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.11 MA Receives Route Metrics from MP

This interaction describes how Mission Autonomy (MA) receives *RouteMetricsMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

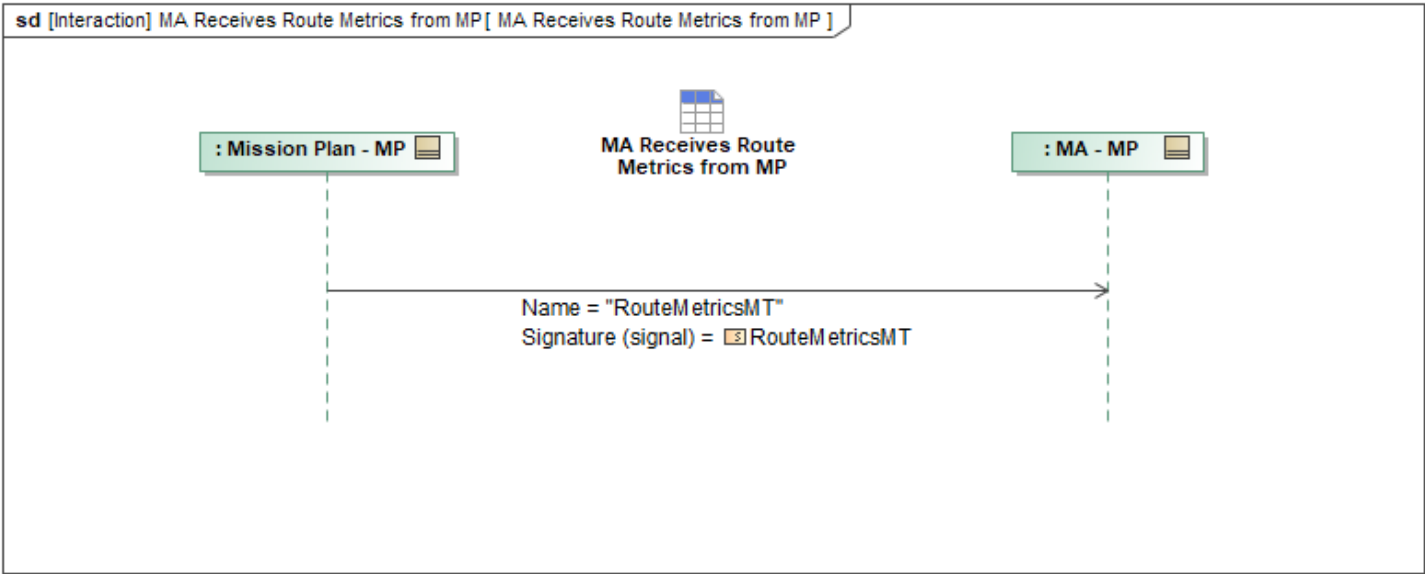


Figure 7-43: MA Receives Route Metrics from MP

Table 7-41: MA Receives Route Metrics from MP

Message Name (Name:Type)	Required Fields (Name:Type)
RouteMetricsMT: RouteMetricsMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.12 MA Receives Route Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_RoutePlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

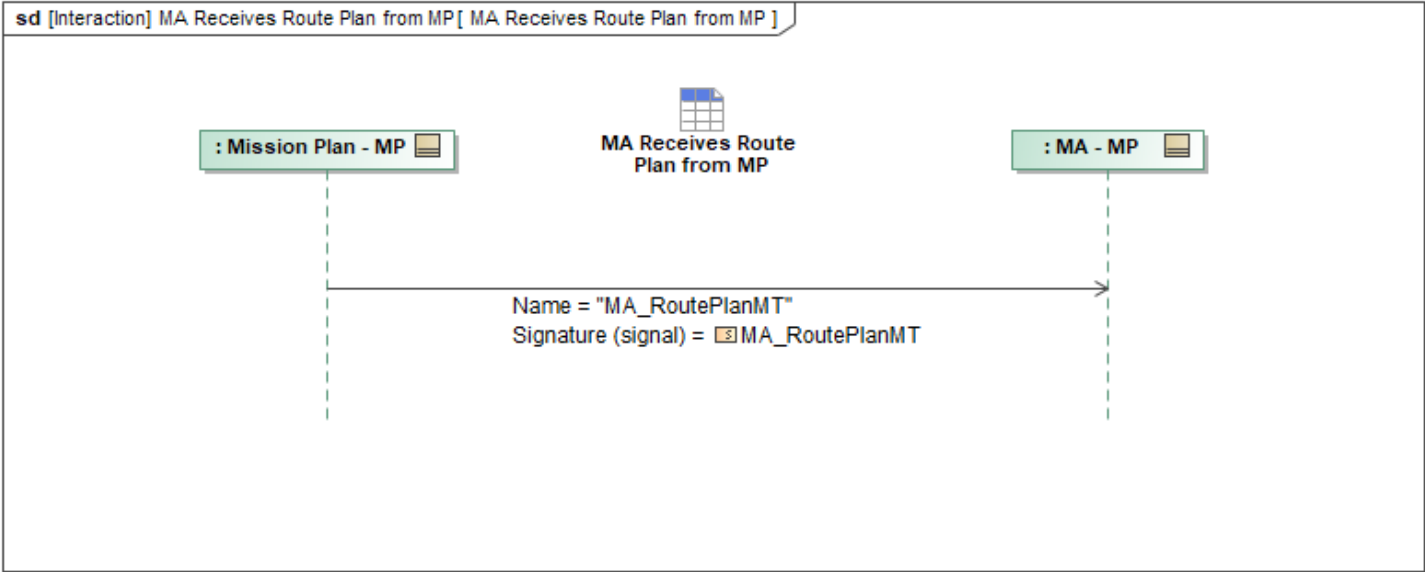


Figure 7-44: MA Receives Route Plan from MP

Table 7-42: MA Receives Route Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.13 MA Receives Task from MP

This interaction describes how Mission Autonomy (MA) receives *MA_TaskMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

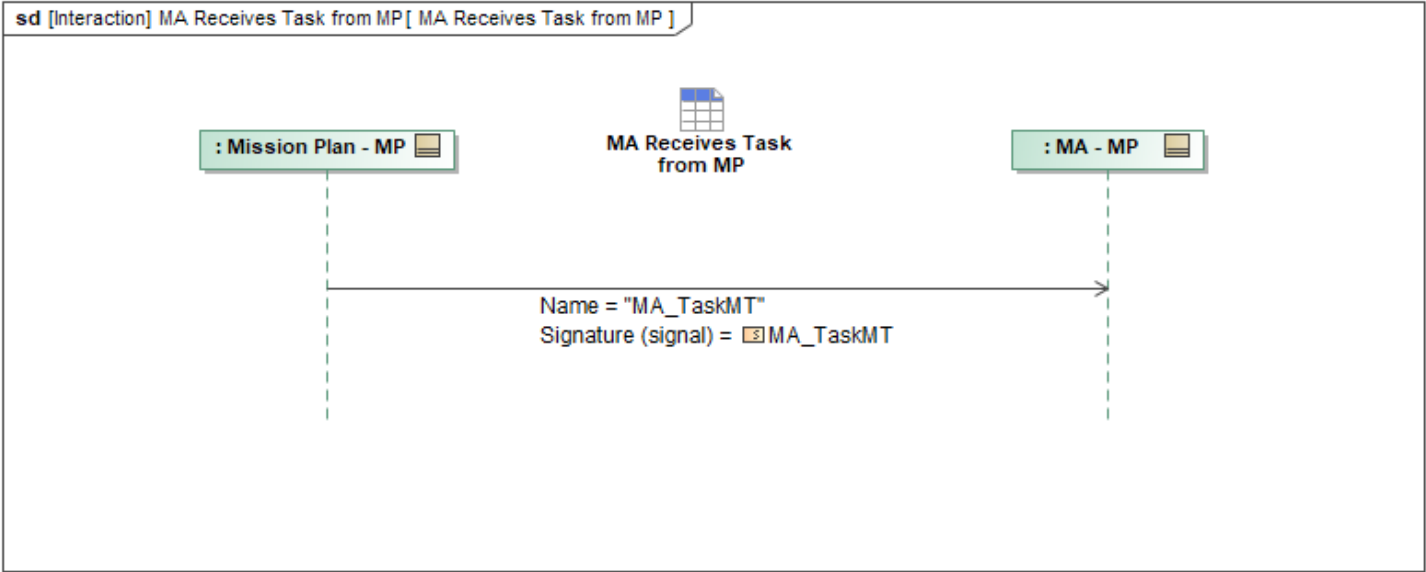


Figure 7-45: MA Receives Task from MP

Table 7-43: MA Receives Task from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.2.14 MA Receives Task Plan from MP

This interaction describes how Mission Autonomy (MA) receives *MA_TaskPlanMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

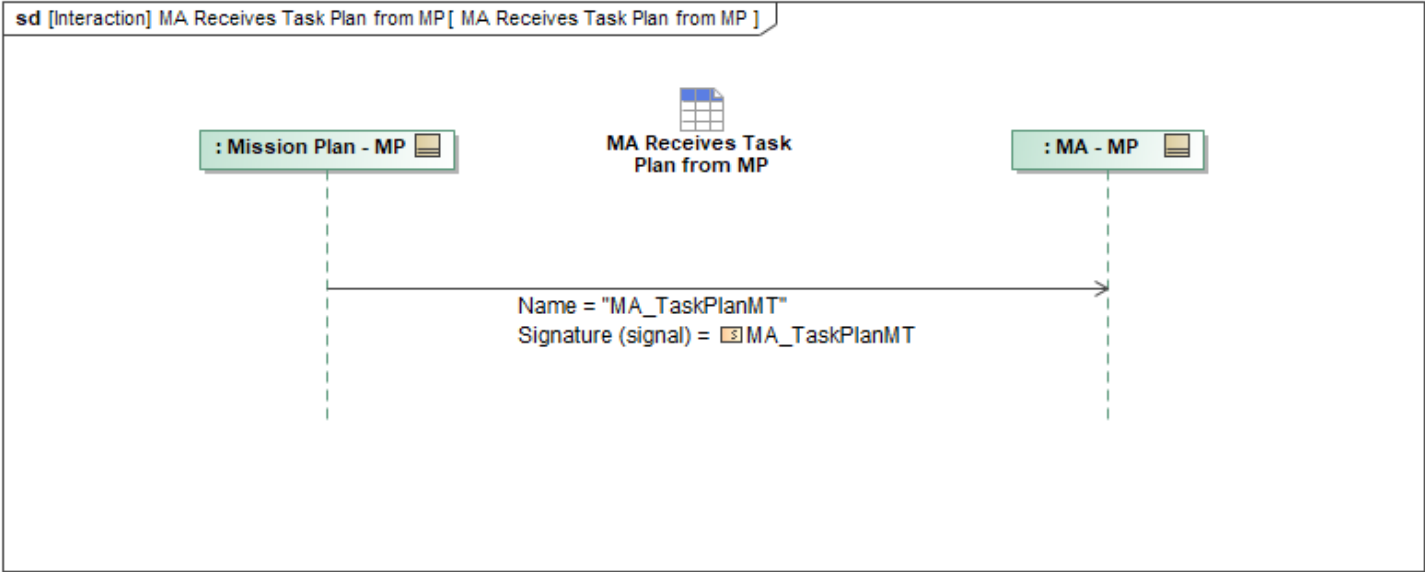


Figure 7-46: MA Receives Task Plan from MP

Table 7-44: MA Receives Task Plan from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskPlanMT: MA_TaskPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3 Operational Data Package

7.2.3.1 MA Receives Access Assessment from MP

This interaction describes how Mission Autonomy (MA) receives *AccessAssessmentMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

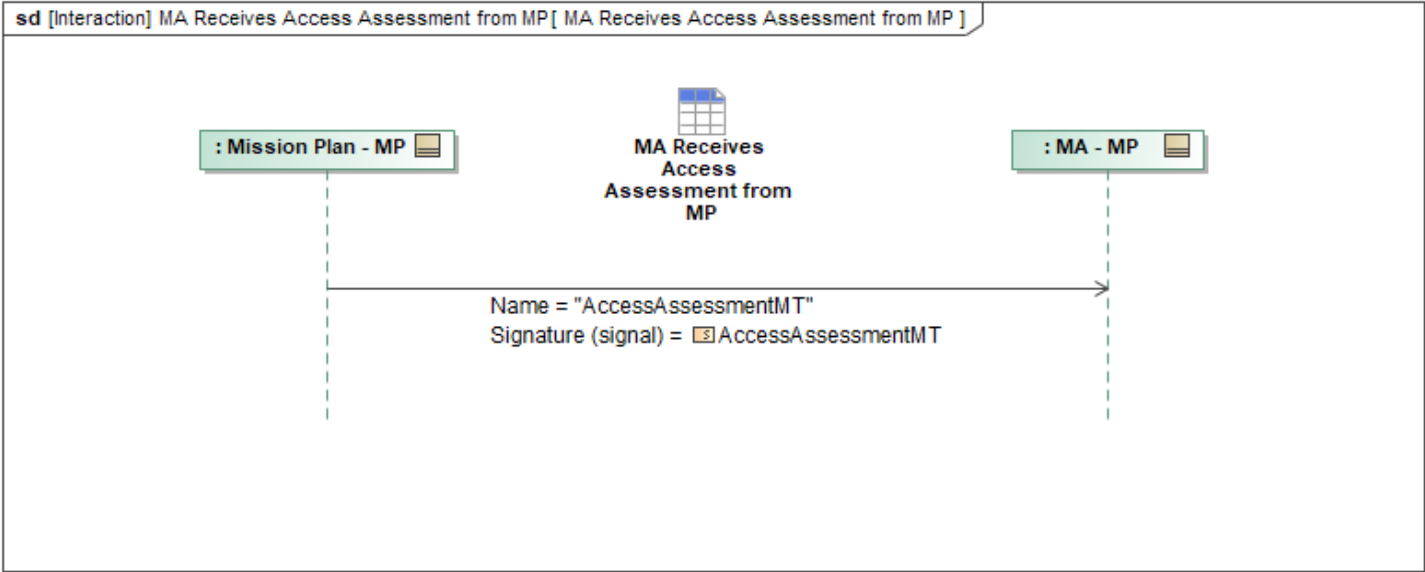


Figure 7-47: MA Receives Access Assessment from MP

Table 7-45: MA Receives Access Assessment from MP

Message Name (Name:Type)	Required Fields (Name:Type)
AccessAssessmentMT: AccessAssessmentMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.2 MA Receives Airfield Report from MP

This interaction describes how Mission Autonomy (MA) receives *AirfieldReportMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

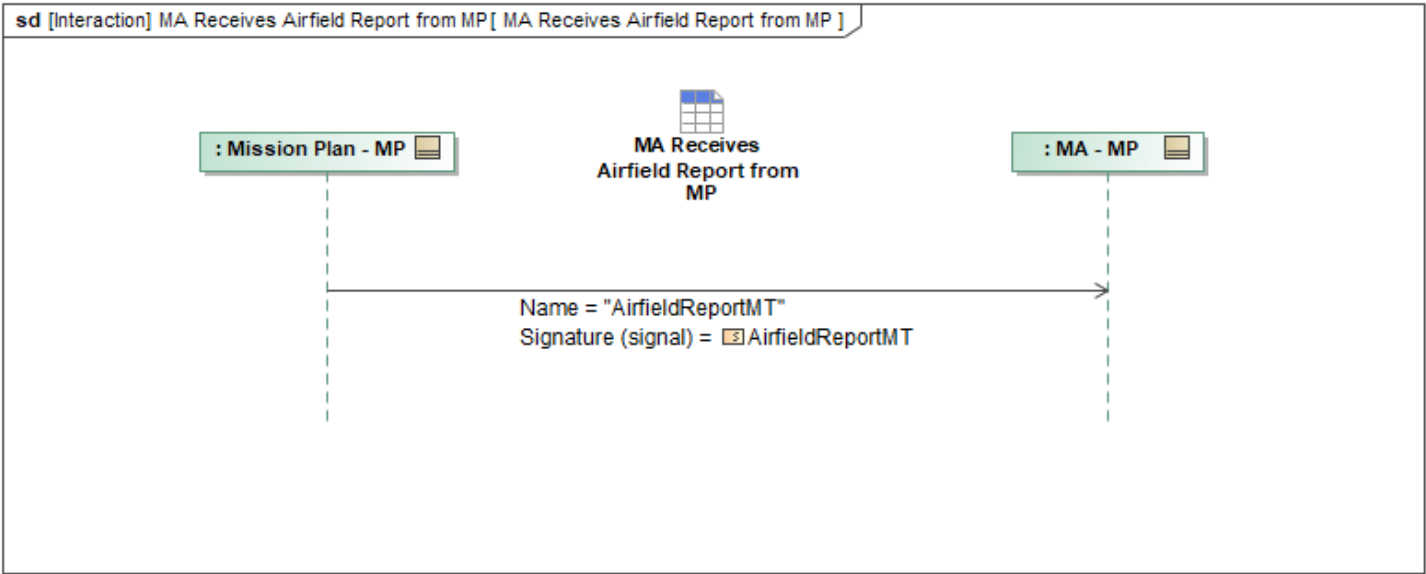


Figure 7-48: MA Receives Airfield Report from MP

Table 7-46: MA Receives Airfield Report from MP

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.3 MA Receives Assessment from MP

This interaction describes how Mission Autonomy (MA) receives *MA_AssessmentMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

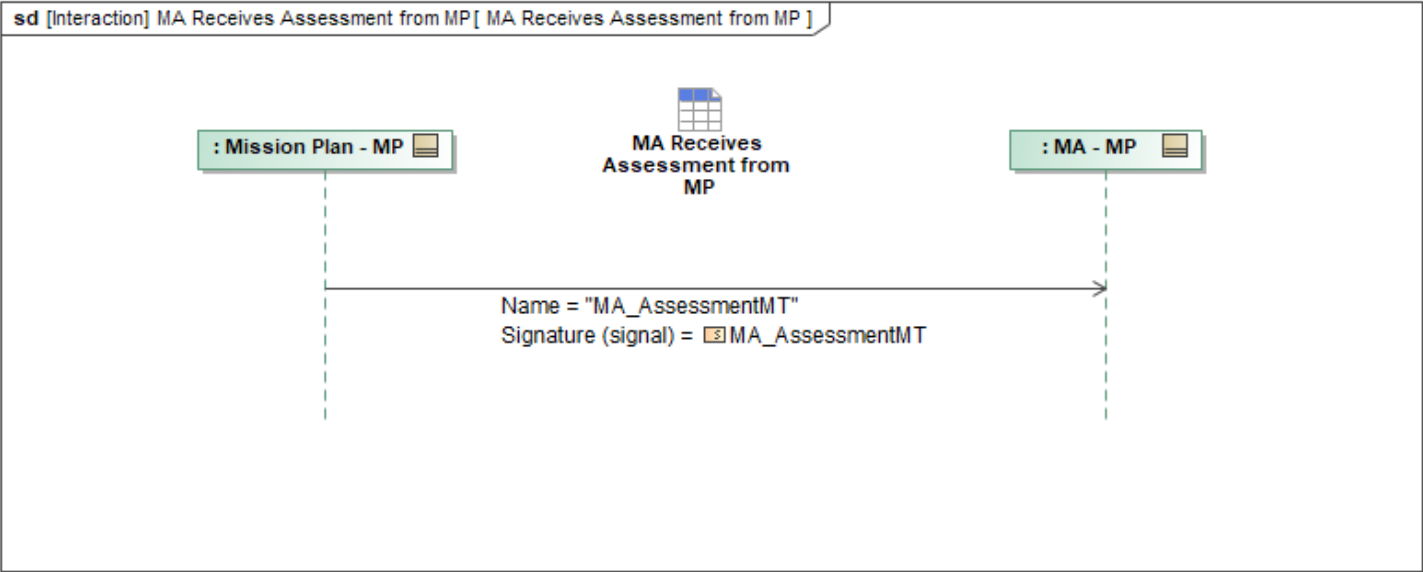


Figure 7-49: MA Receives Assessment from MP

Table 7-47: MA Receives Assessment from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentMT: MA_AssessmentMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.4 MA Receives Designation from MP

This interaction describes how Mission Autonomy (MA) receives *MA_DesignationMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

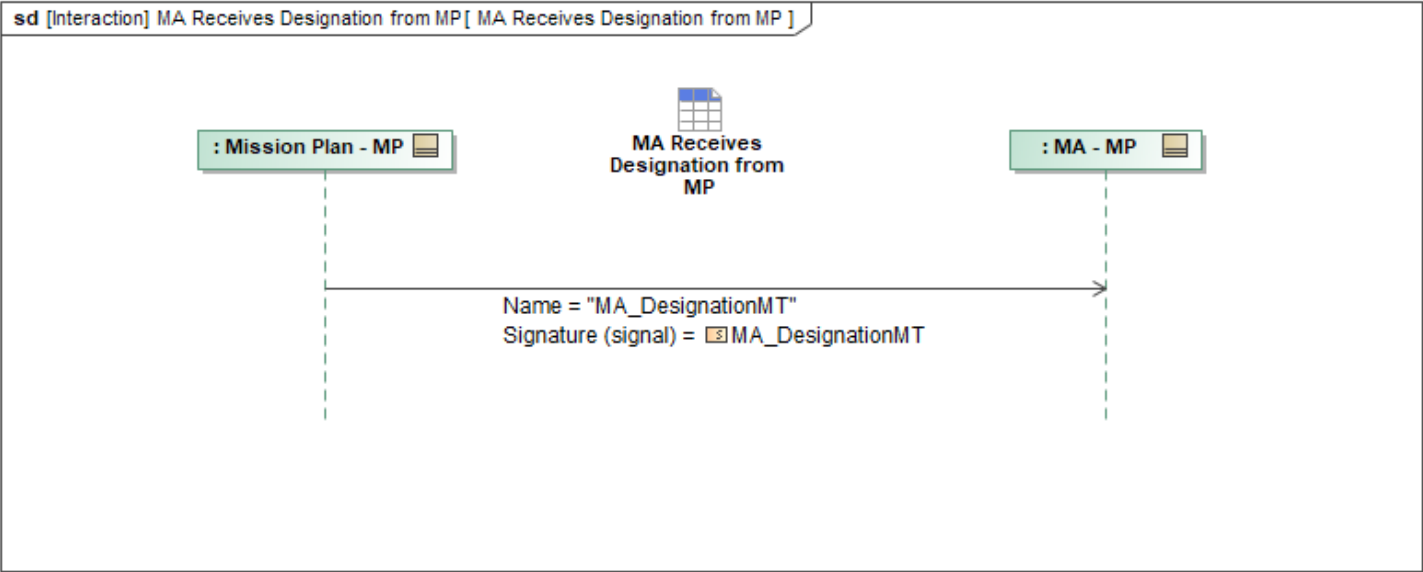


Figure 7-50: MA Receives Designation from MP

Table 7-48: MA Receives Designation from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationMT: MA_DesignationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.5 MA Receives DLZ from MP

This interaction describes how Mission Autonomy (MA) receives *DLZ_MT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

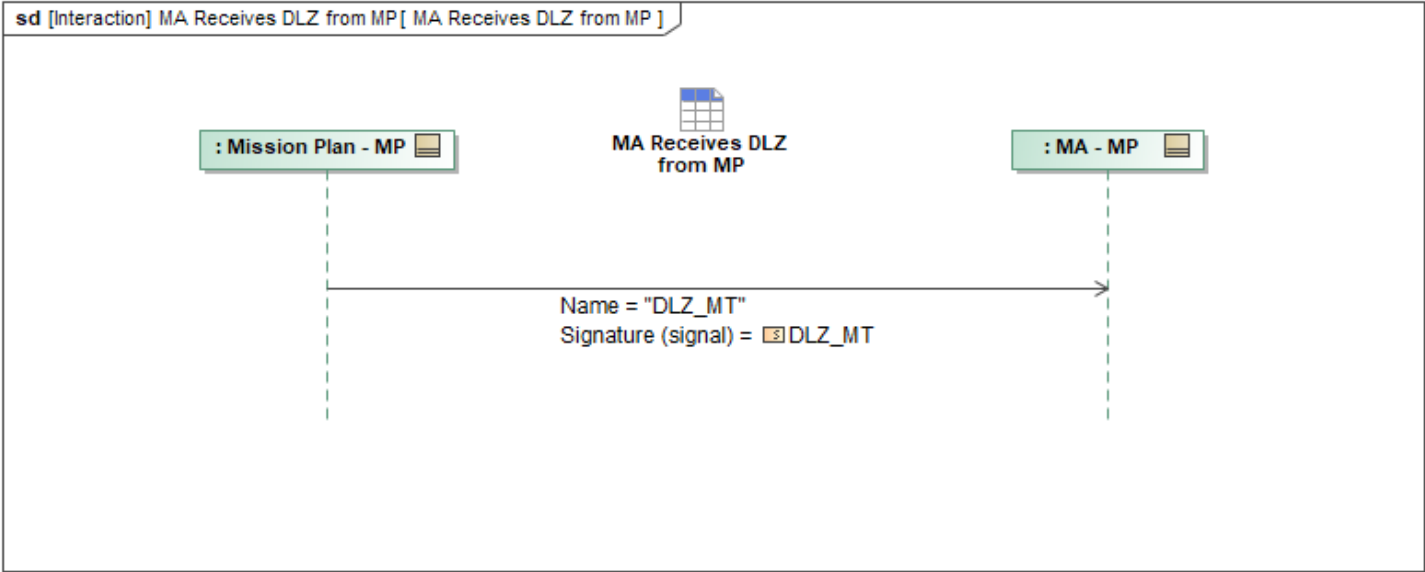


Figure 7-51: MA Receives DLZ from MP

Table 7-49: MA Receives DLZ from MP

Message Name (Name:Type)	Required Fields (Name:Type)
DLZ_MT: DLZ_MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.6 MA Receives Entity from MP

This interaction describes how Mission Autonomy (MA) receives *MA_EntityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

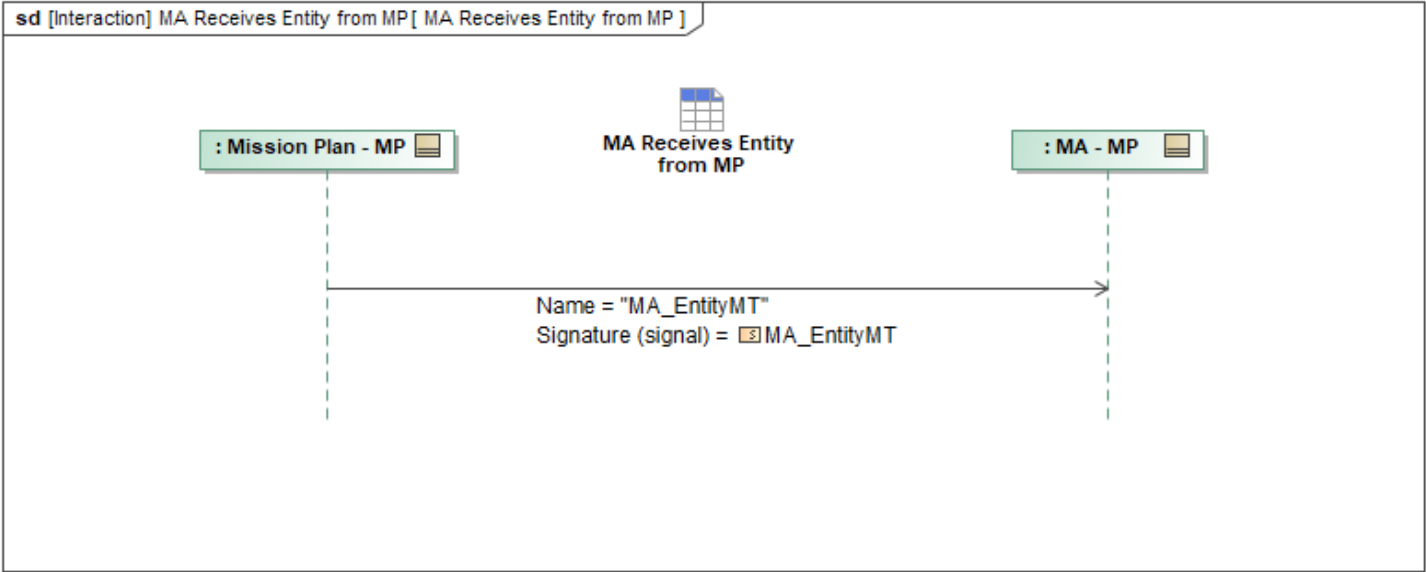


Figure 7-52: MA Receives Entity from MP

Table 7-50: MA Receives Entity from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EntityMT: MA_EntityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.7 MA Receives Foreign Key Pair from MP

This interaction describes how Mission Autonomy (MA) receives *ForeignKeyPairMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

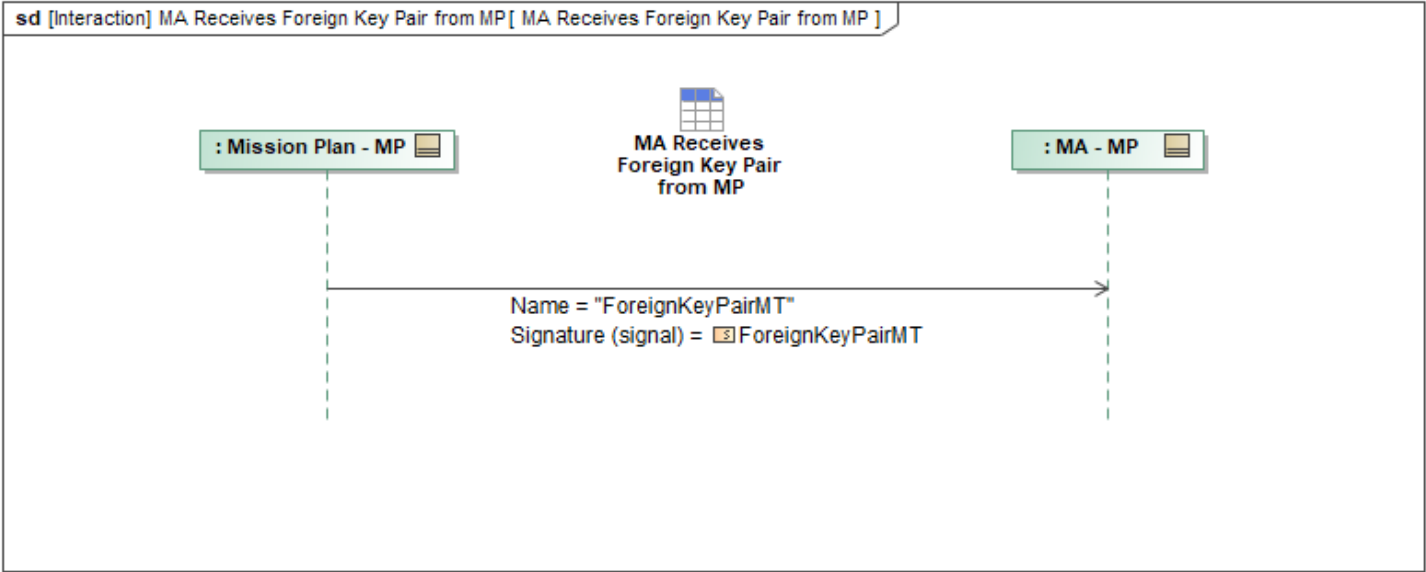


Figure 7-53: MA Receives Foreign Key Pair from MP

Table 7-51: MA Receives Foreign Key Pair from MP

Message Name (Name:Type)	Required Fields (Name:Type)
ForeignKeyPairMT: ForeignKeyPairMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.8 MA Receives Leadership Metrics from MP

Mission Use Case: Bully Leader Election, Max Consensus Leader Election, Static Fitness Score Leader Election

This interaction describes how Mission Autonomy (MA) receives *MA_LeadershipMetricsMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

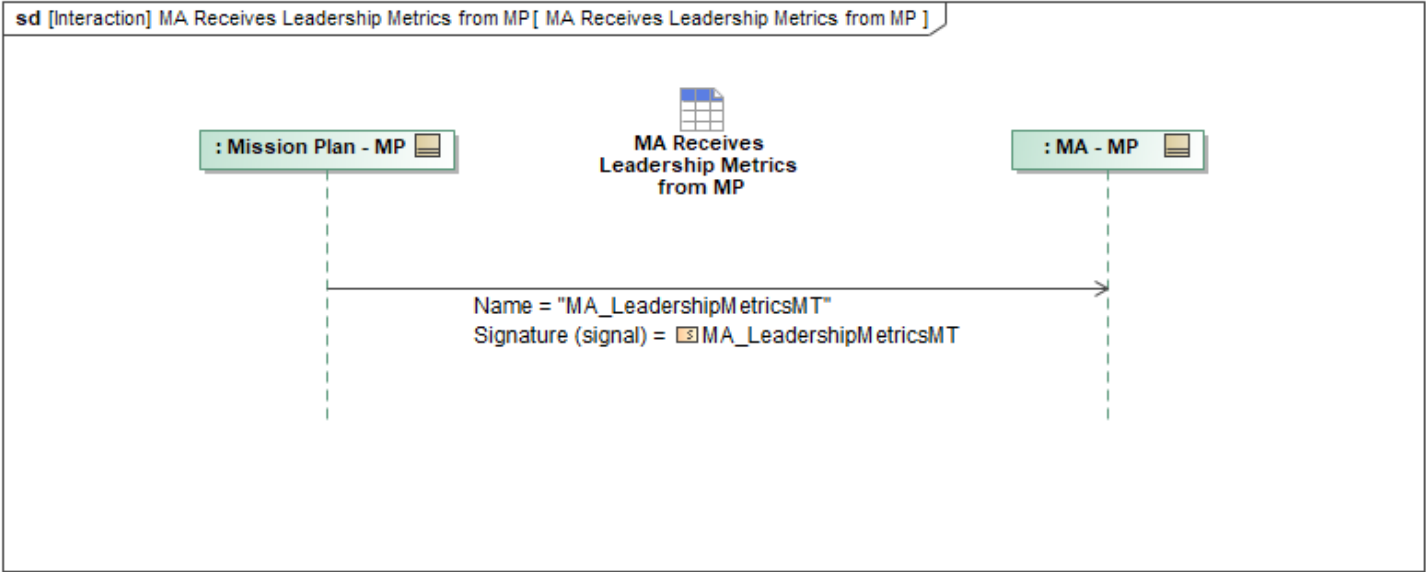


Figure 7-54: MA Receives Leadership Metrics from MP

Table 7-52: MA Receives Leadership Metrics from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeadershipMetricsMT: MA_LeadershipMetricsMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.9 MA Receives OpLine from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OpLineMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

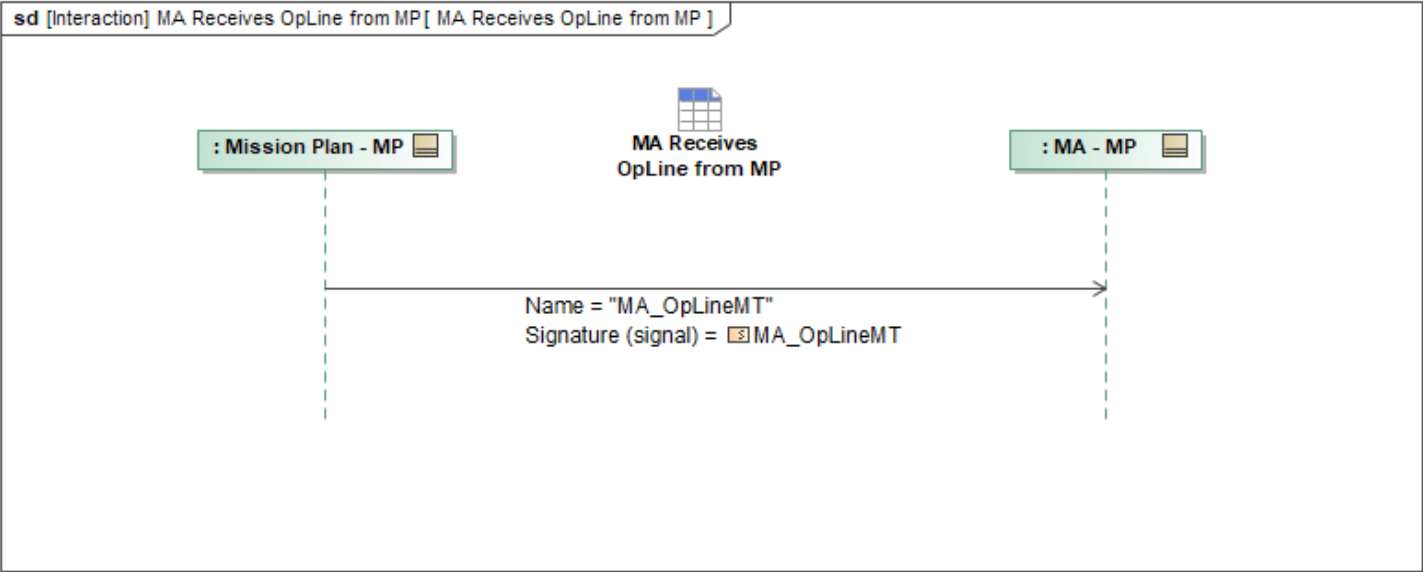


Figure 7-55: MA Receives OpLine from MP

Table 7-53: MA Receives OpLine from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpLineMT: MA_OpLineMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.10 MA Receives OpPoint from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OpPointMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

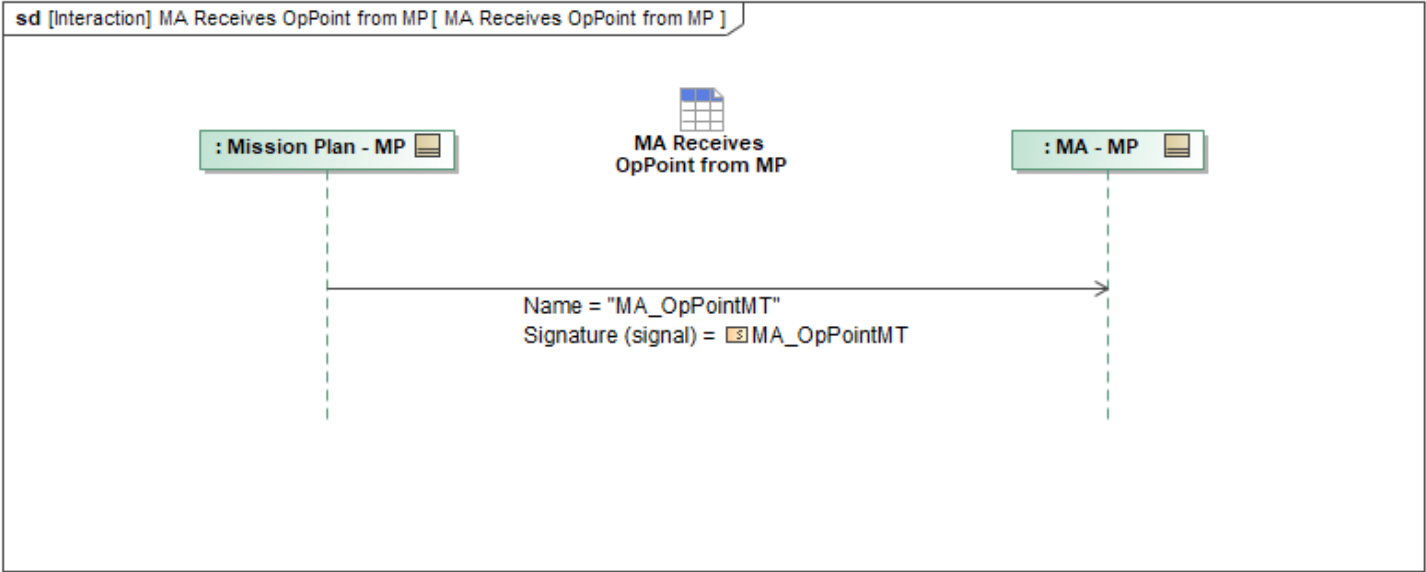


Figure 7-56: MA Receives OpPoint from MP

Table 7-54: MA Receives OpPoint from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPointMT: MA_OpPointMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.11 MA Receives OpRouting from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OpRoutingMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

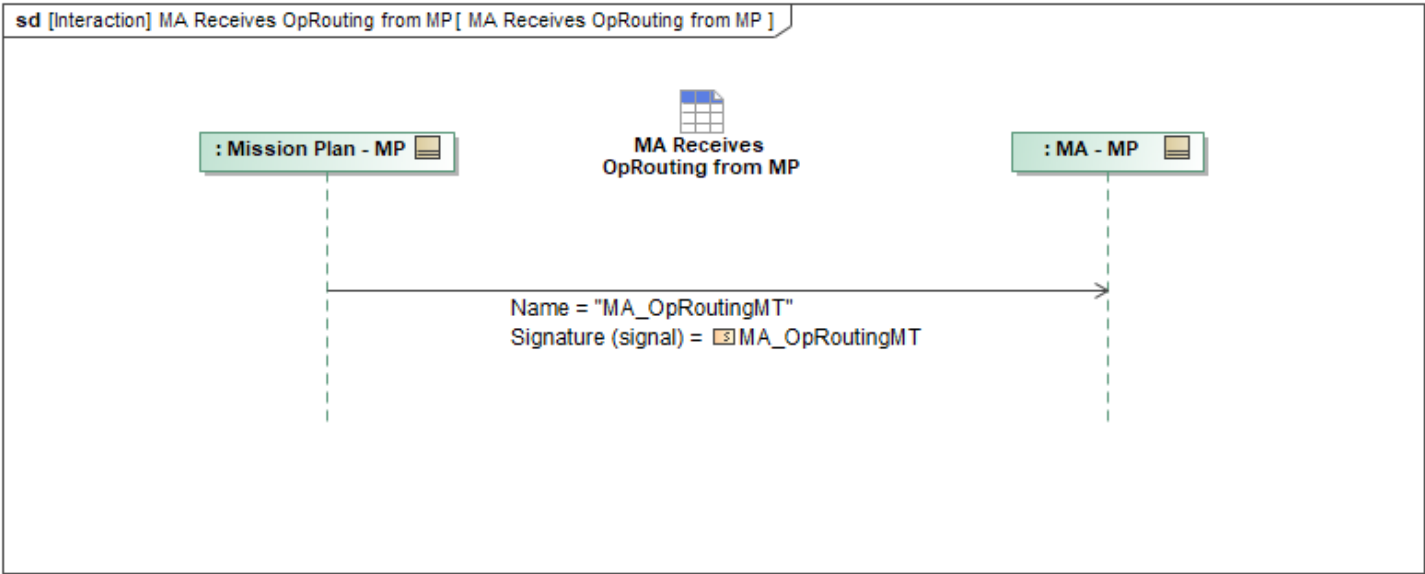


Figure 7-57: MA Receives OpRouting from MP

Table 7-55: MA Receives OpRouting from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpRoutingMT: MA_OpRoutingMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.12 MA Receives OpVolume from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OpVolumeMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

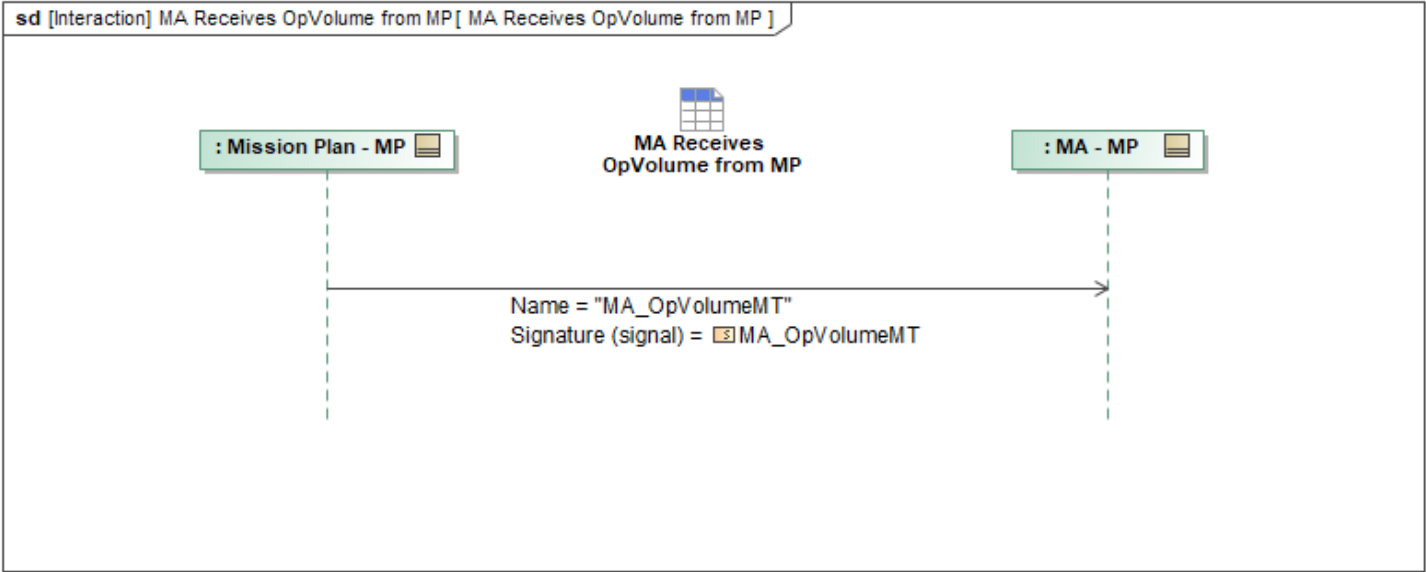


Figure 7-58: MA Receives OpVolume from MP

Table 7-56: MA Receives OpVolume from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpVolumeMT: MA_OpVolumeMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.13 MA Receives OpZone from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OpZoneMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

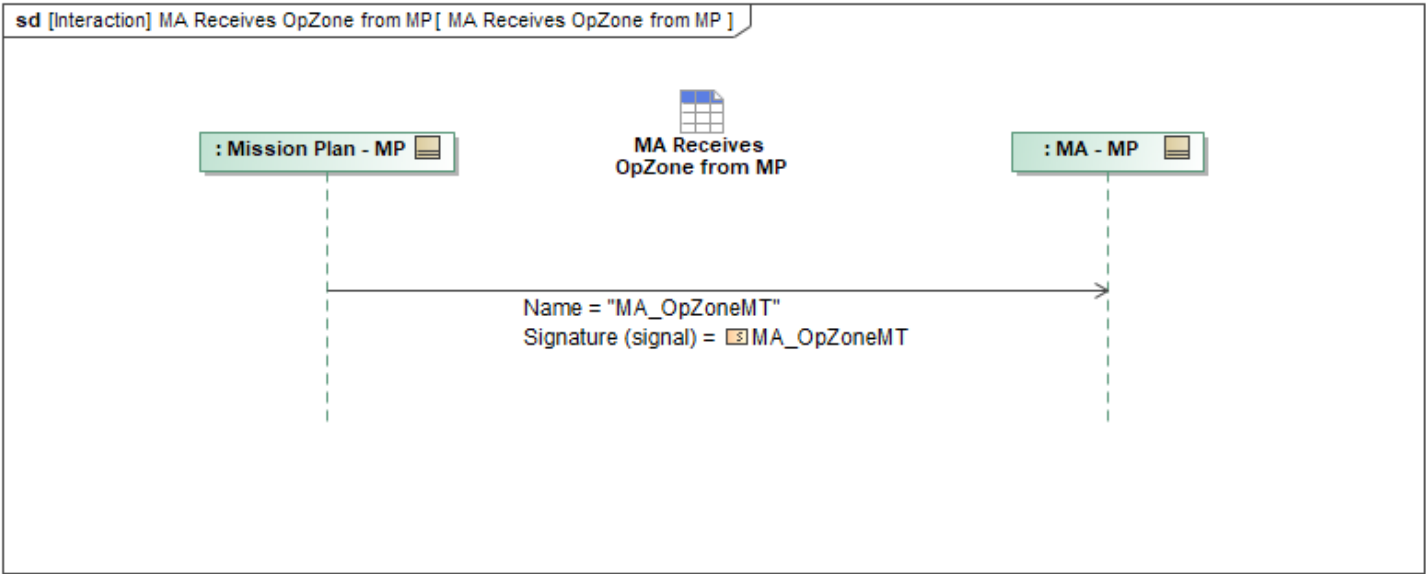


Figure 7-59: MA Receives OpZone from MP

Table 7-57: MA Receives OpZone from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpZoneMT: MA_OpZoneMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.14 MA Receives Package from MP

This interaction describes how Mission Autonomy (MA) receives *PackageMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

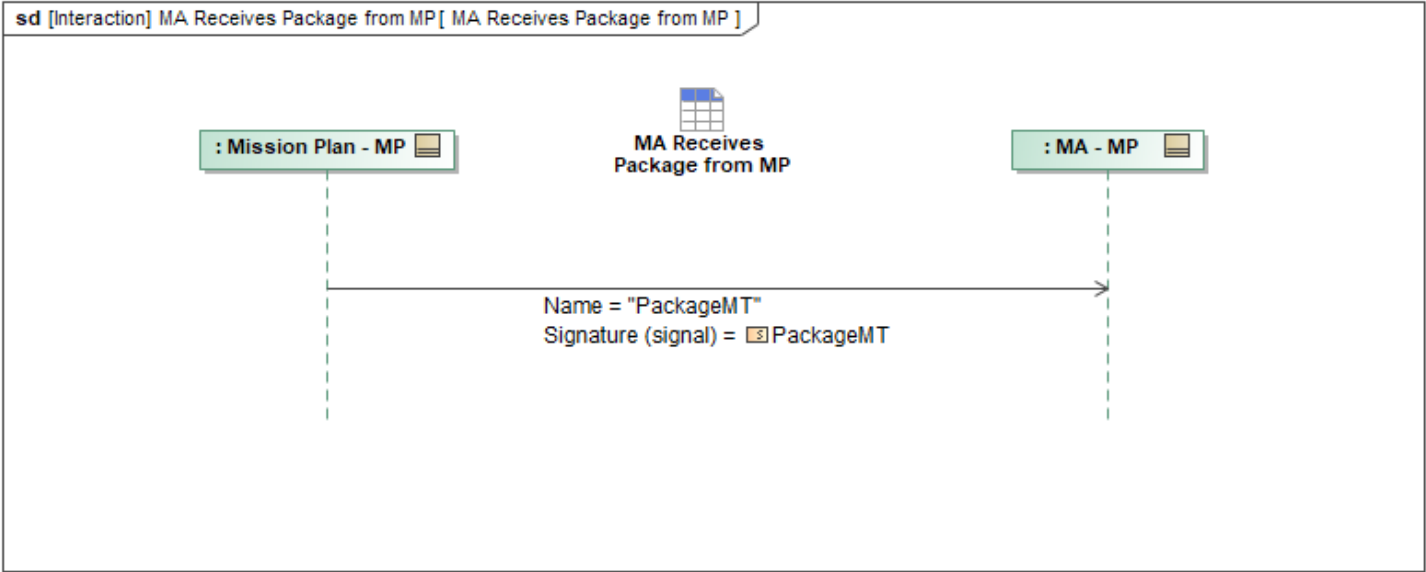


Figure 7-60: MA Receives Package from MP

Table 7-58: MA Receives Package from MP

Message Name (Name:Type)	Required Fields (Name:Type)
PackageMT: PackageMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.15 MA Receives Planning Function from MP

This interaction describes how Mission Autonomy (MA) receives *MA_PlanningFunctionMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

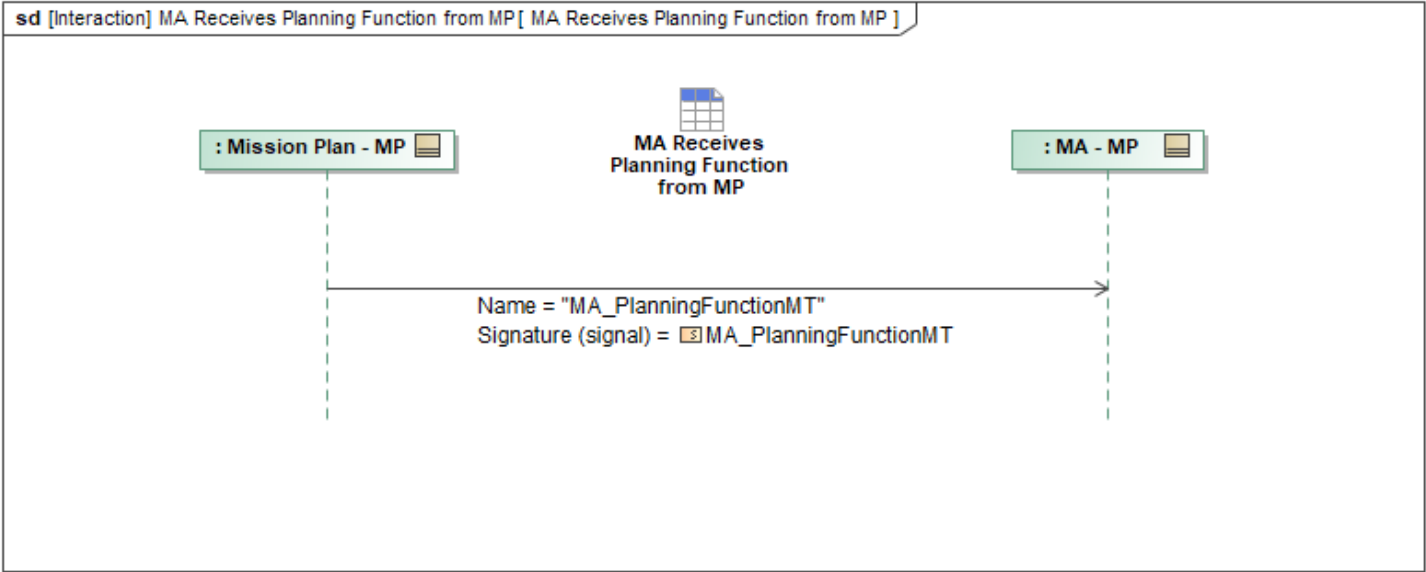


Figure 7-61: MA Receives Planning Function from MP

Table 7-59: MA Receives Planning Function from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanningFunctionMT: MA_PlanningFunctionMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.16 MA Receives Store Loadout Configuration from MP

Mission Use Case: Store Management

This interaction describes how Mission Autonomy (MA) receives *StoreLoadoutConfigurationMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

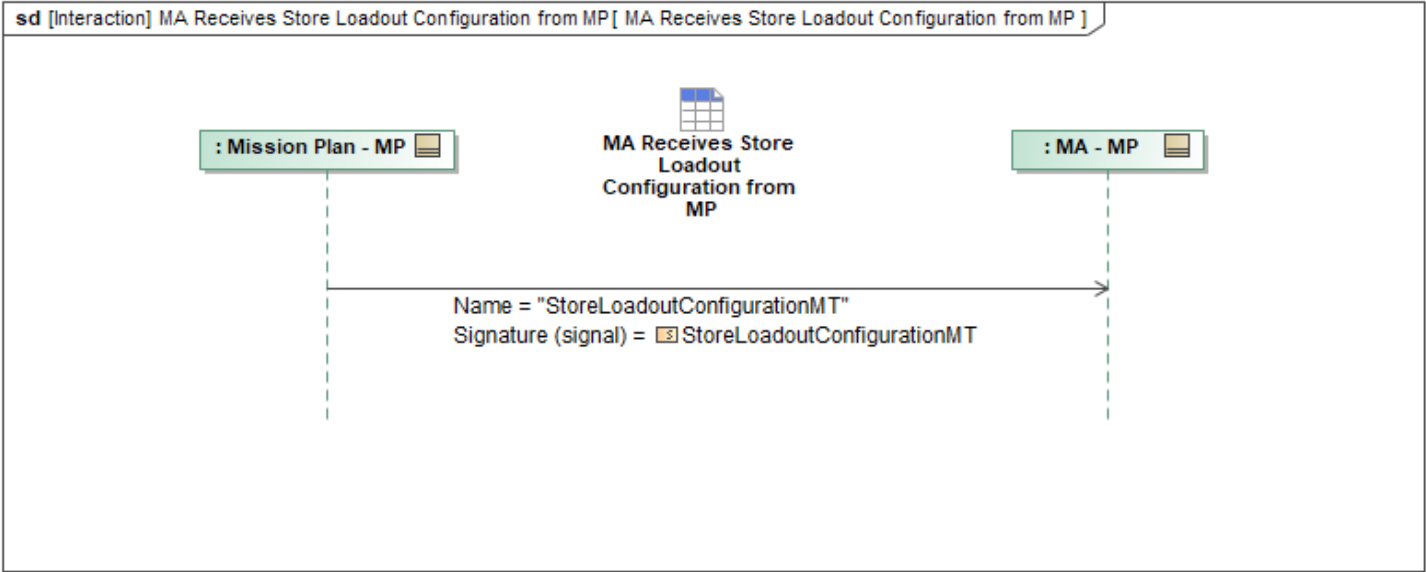


Figure 7-62: MA Receives Store Loadout Configuration from MP

Table 7-60: MA Receives Store Loadout Configuration from MP

Message Name (Name:Type)	Required Fields (Name:Type)
StoreLoadoutConfigurationMT: StoreLoadoutConfigurationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.17 MA Receives Survivability Risk Level from MP

Mission Use Case: ALR

This interaction describes how Mission Autonomy (MA) receives *SurvivabilityRiskLevelMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

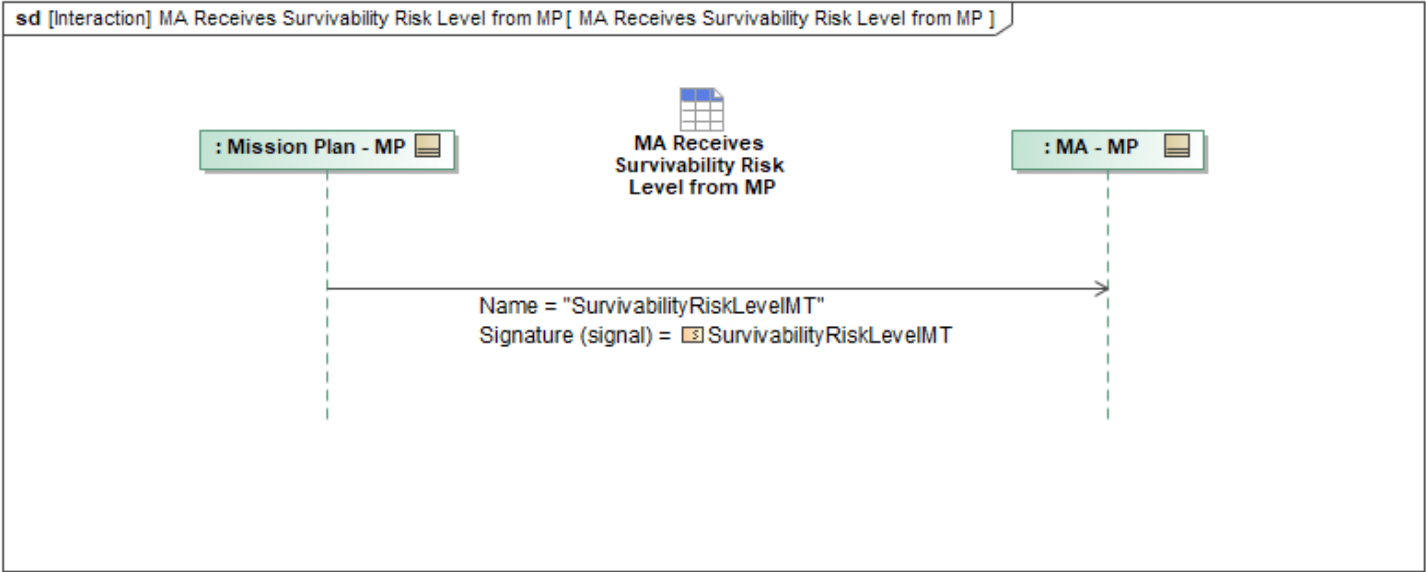


Figure 7-63: MA Receives Survivability Risk Level from MP

Table 7-61: MA Receives Survivability Risk Level from MP

Message Name (Name:Type)	Required Fields (Name:Type)
SurvivabilityRiskLevelMT: SurvivabilityRiskLevelMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.18 MA Receives System Readiness from MP

This interaction describes how Mission Autonomy (MA) receives *SystemReadinessMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

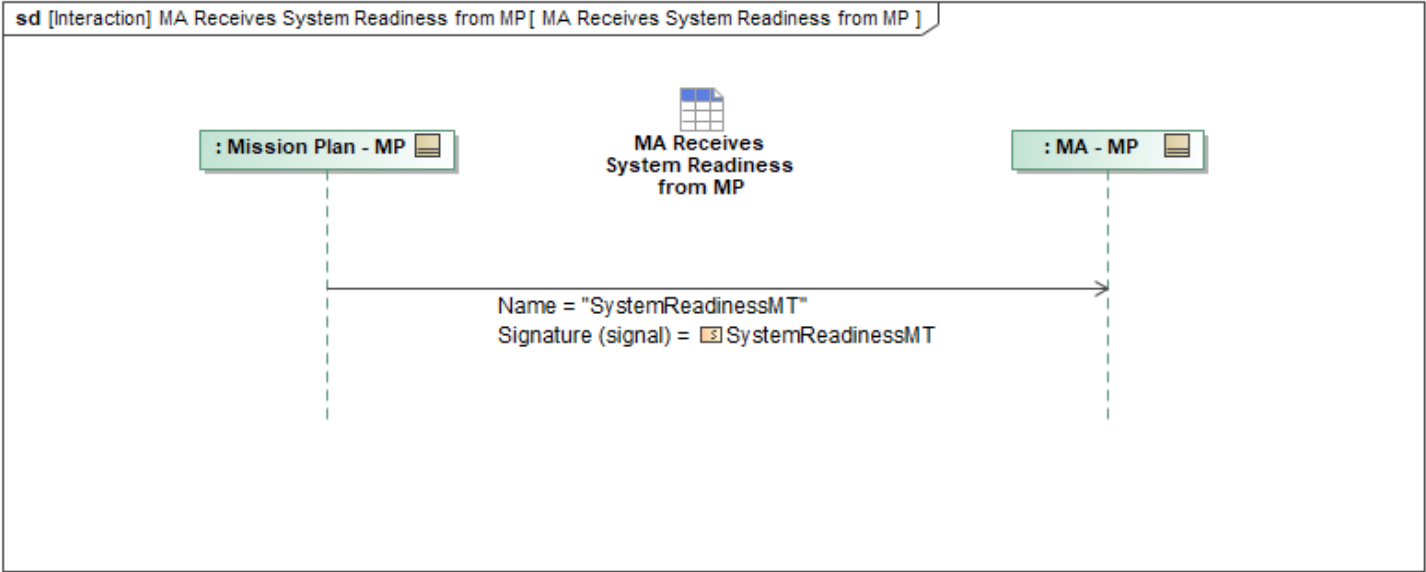


Figure 7-64: MA Receives System Readiness from MP

Table 7-62: MA Receives System Readiness from MP

Message Name (Name:Type)	Required Fields (Name:Type)
SystemReadinessMT: SystemReadinessMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.19 MA Receives Weather Dataset from MP

This interaction describes how Mission Autonomy (MA) receives *WeatherDatasetMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

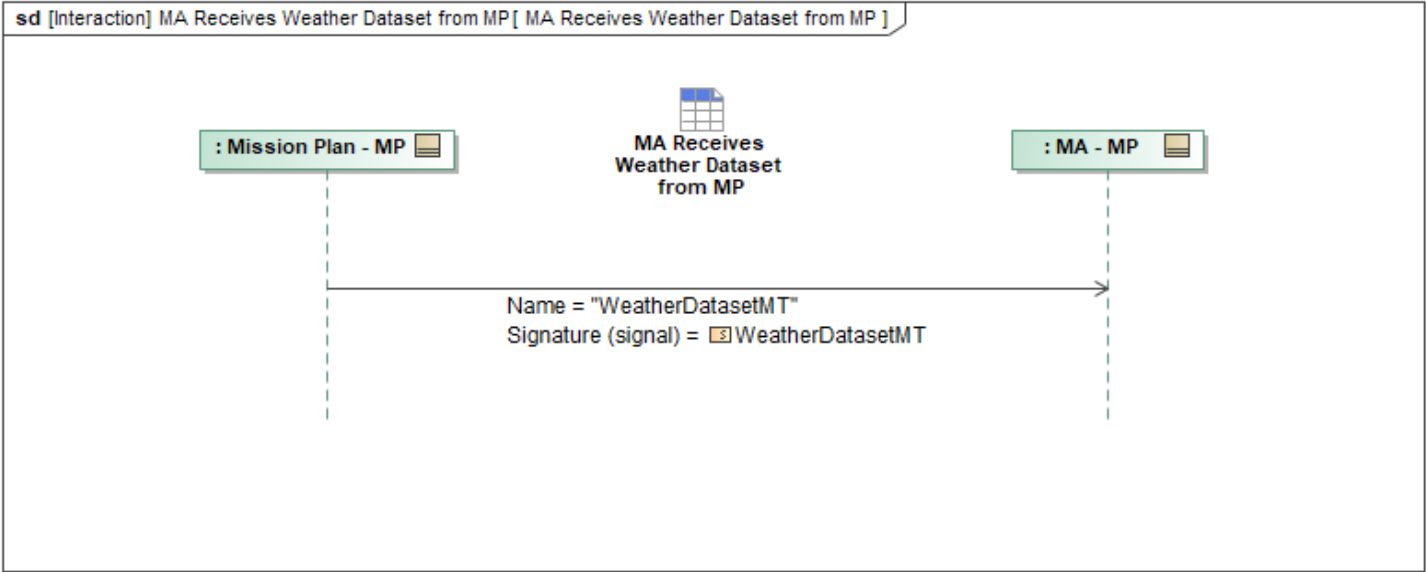


Figure 7-65: MA Receives Weather Dataset from MP

Table 7-63: MA Receives Weather Dataset from MP

Message Name (Name:Type)	Required Fields (Name:Type)
WeatherDatasetMT: WeatherDatasetMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.20 MA Receives Weather from MP

This interaction describes how Mission Autonomy (MA) receives *WeatherMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

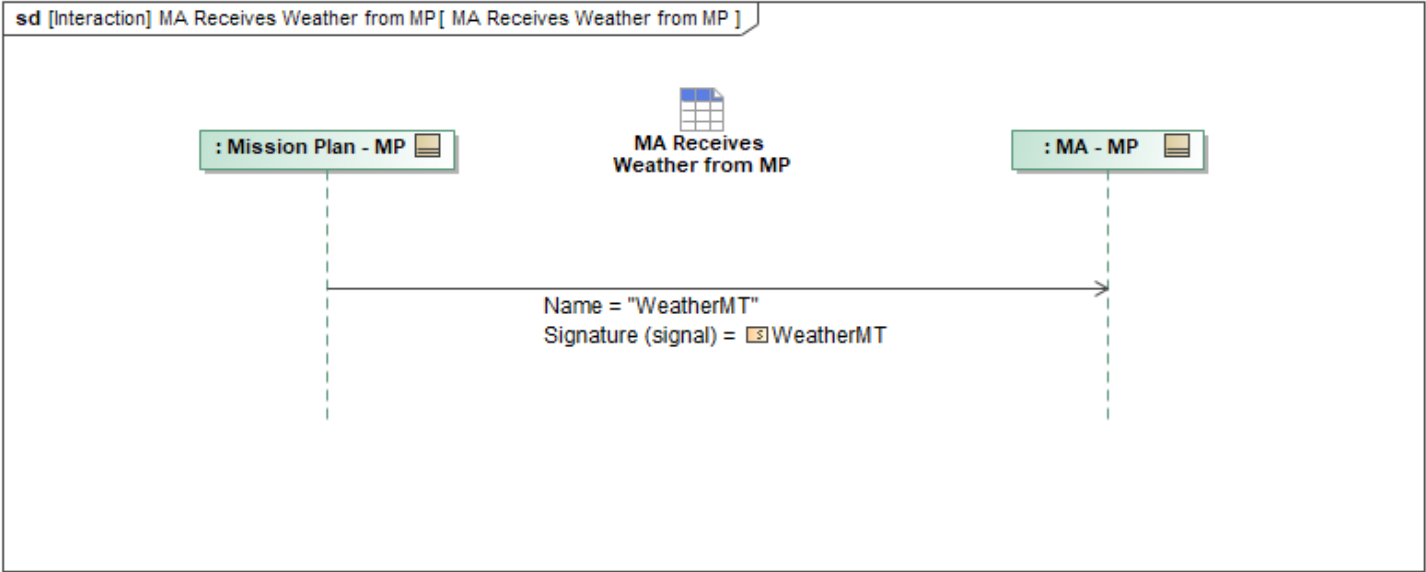


Figure 7-66: MA Receives Weather from MP

Table 7-64: MA Receives Weather from MP

Message Name (Name:Type)	Required Fields (Name:Type)
WeatherMT: WeatherMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.3.21 MA Receives WEZ from MP

This interaction describes how Mission Autonomy (MA) receives *MA_WEZ_MT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

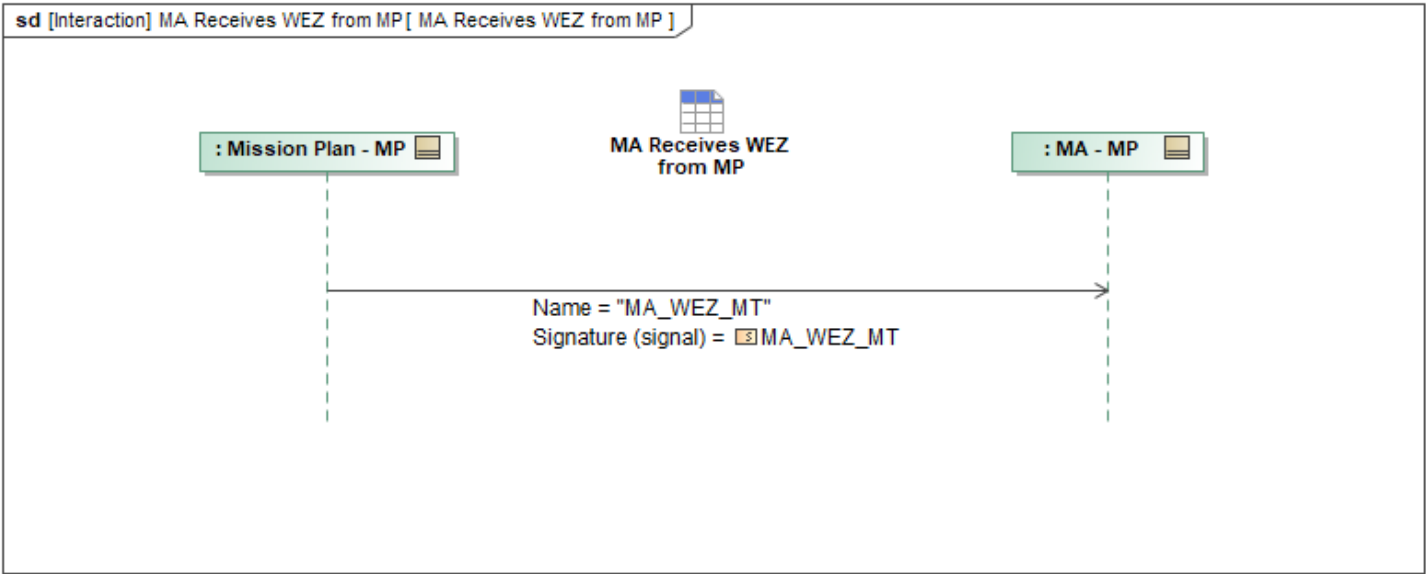


Figure 7-67: MA Receives WEZ from MP

Table 7-65: MA Receives WEZ from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_WEZ_MT: MA_WEZ_MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.4 Policy Package

7.2.4.1 MA Receives Approval Authority from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ApprovalAuthorityMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

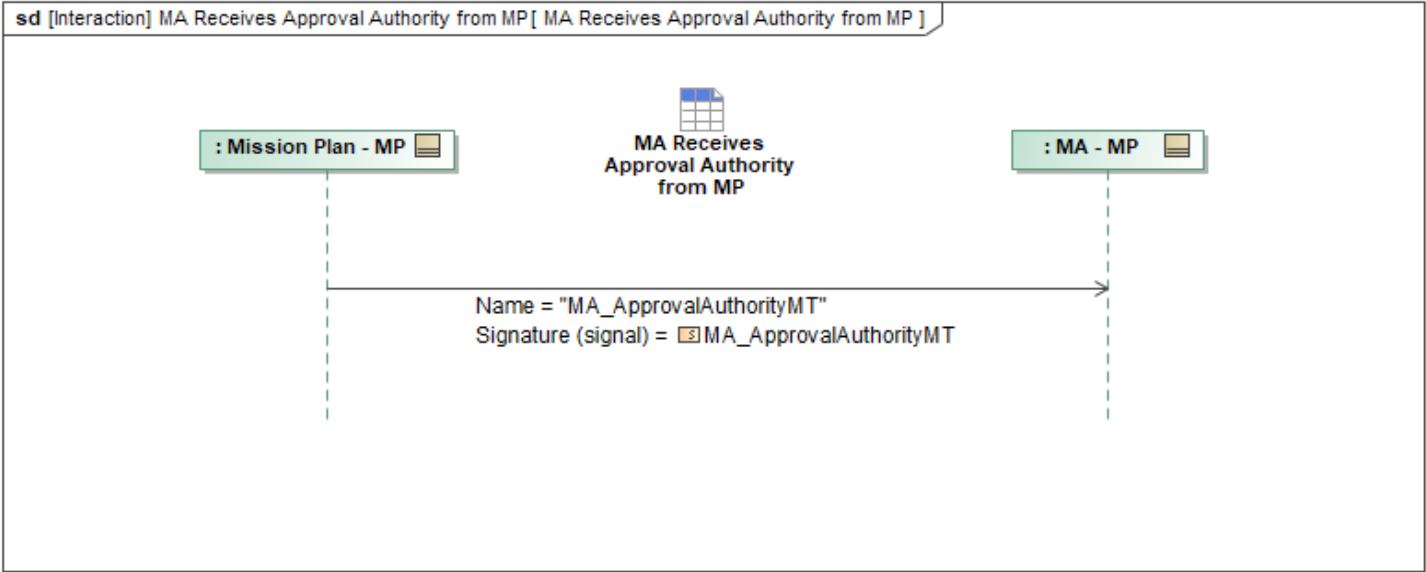


Figure 7-68: MA Receives Approval Authority from MP

Table 7-66: MA Receives Approval Authority from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalAuthorityMT: MA_ApprovalAuthorityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.4.2 MA Receives Approval Policy from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ApprovalPolicyMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

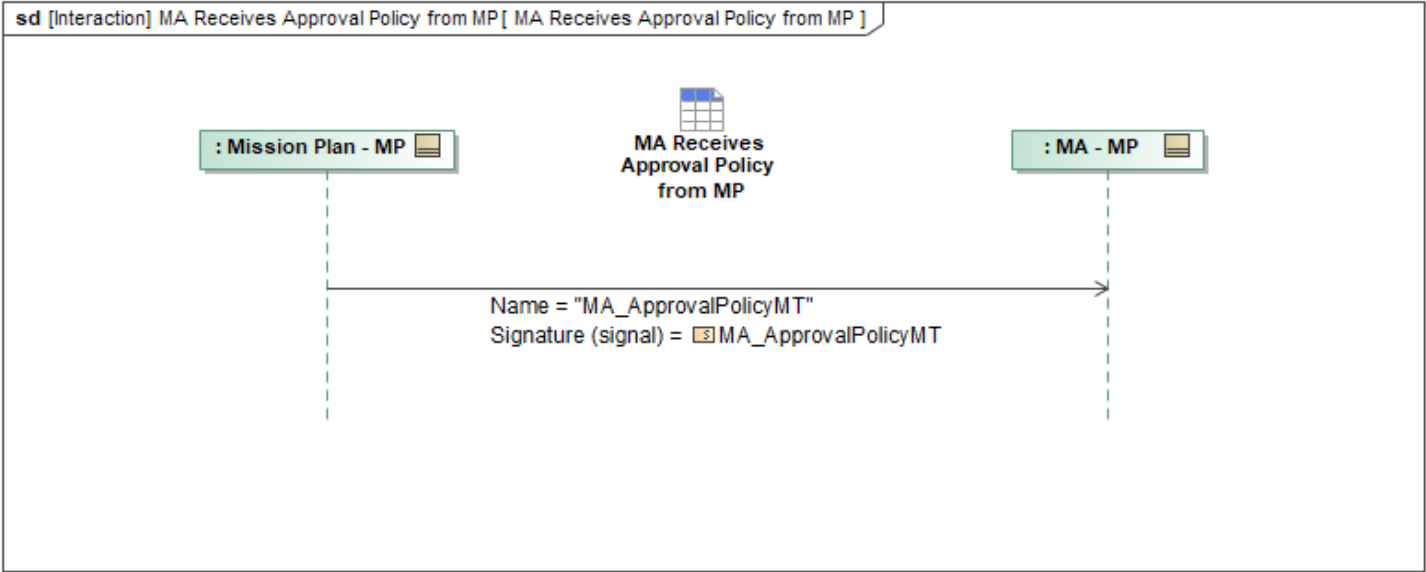


Figure 7-69: MA Receives Approval Policy from MP

Table 7-67: MA Receives Approval Policy from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.4.3 MA Receives Authorization from MP

This interaction describes how Mission Autonomy (MA) receives *MA_AuthorizationMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

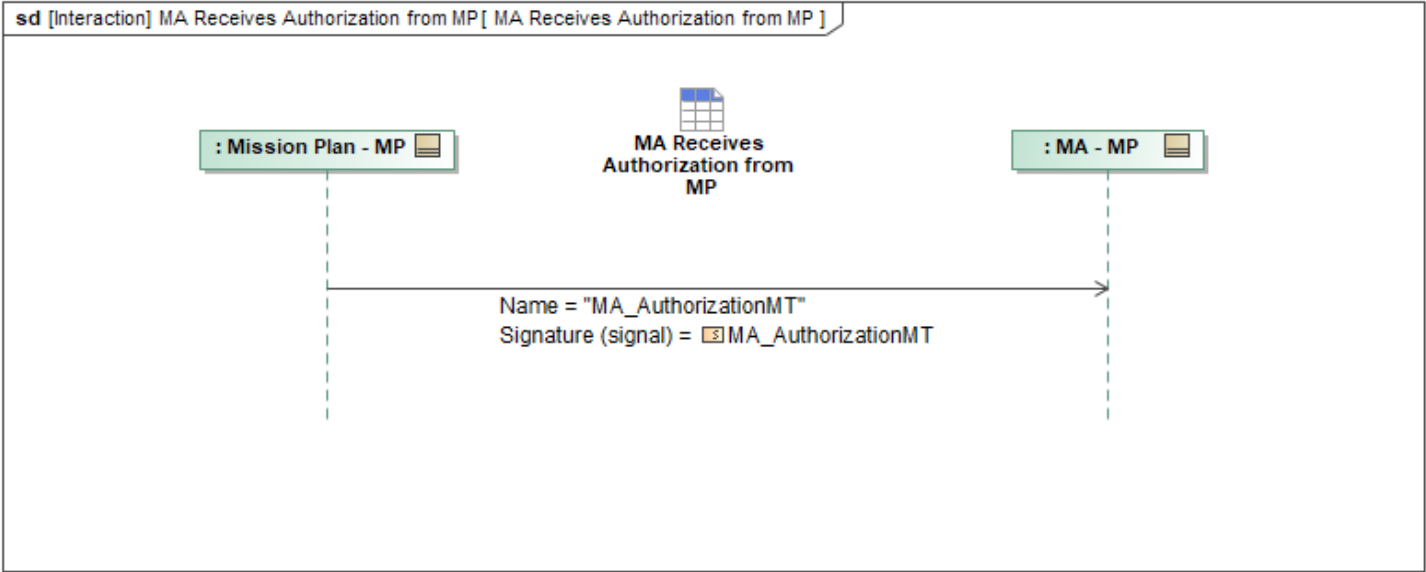


Figure 7-70: MA Receives Authorization from MP

Table 7-68: MA Receives Authorization from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AuthorizationMT: MA_AuthorizationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.4.4 MA Receives Engagement Criteria from MP

This interaction describes how Mission Autonomy (MA) receives *MA_EngagementCriteriaMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

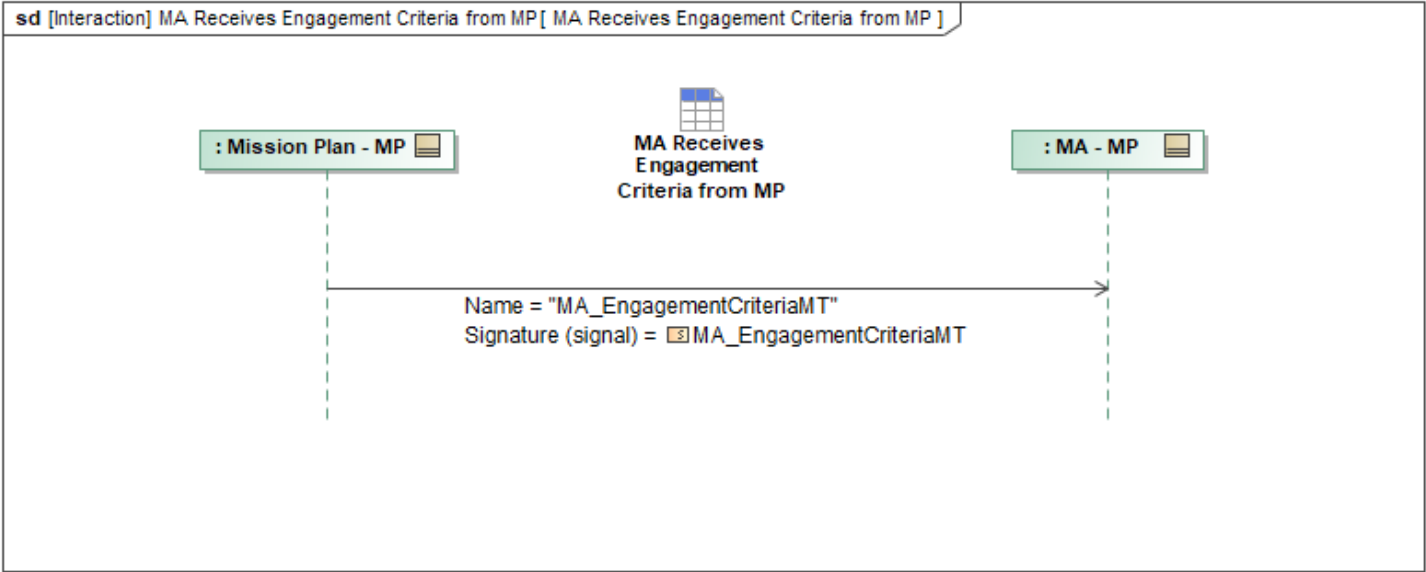


Figure 7-71: MA Receives Engagement Criteria from MP

Table 7-69: MA Receives Engagement Criteria from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EngagementCriteriaMT: MA_EngagementCriteriaMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.4.5 MA Receives Interoperability Policy from MP

This interaction describes how Mission Autonomy (MA) receives *MA_InteroperabilityPolicyMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

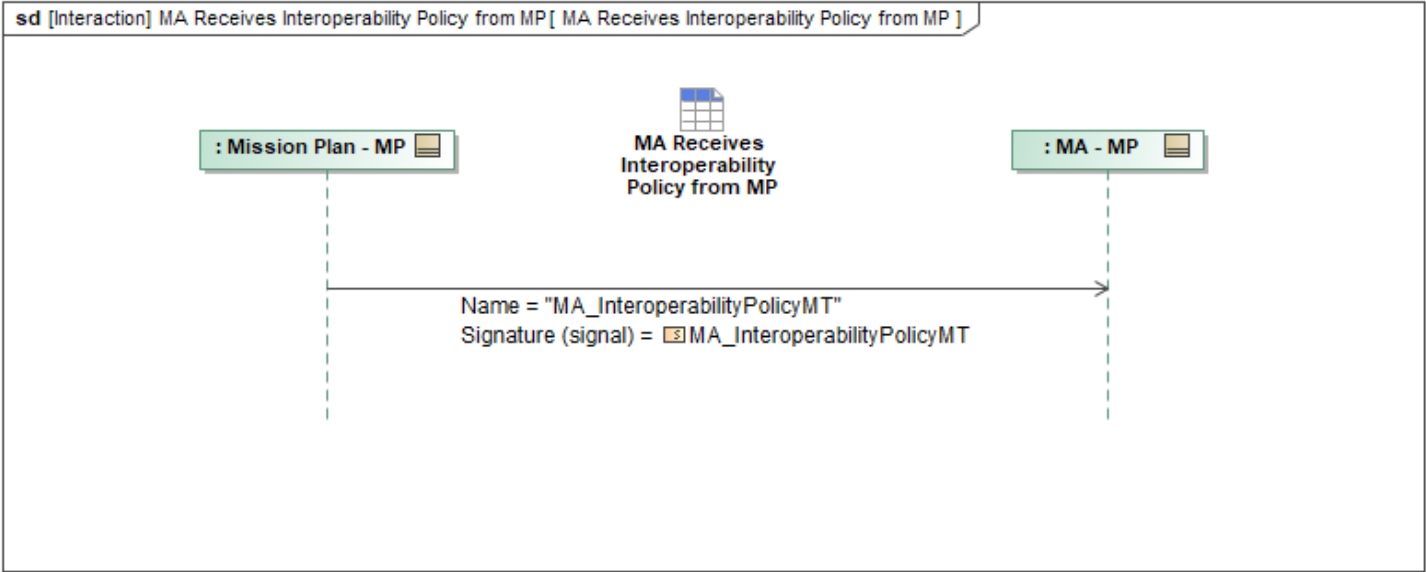


Figure 7-72: MA Receives Interoperability Policy from MP

Table 7-70: MA Receives Interoperability Policy from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_InteroperabilityPolicyMT: MA_InteroperabilityPolicyMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.4.6 MA Receives Operator Role from MP

This interaction describes how Mission Autonomy (MA) receives *MA_OperatorRoleMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

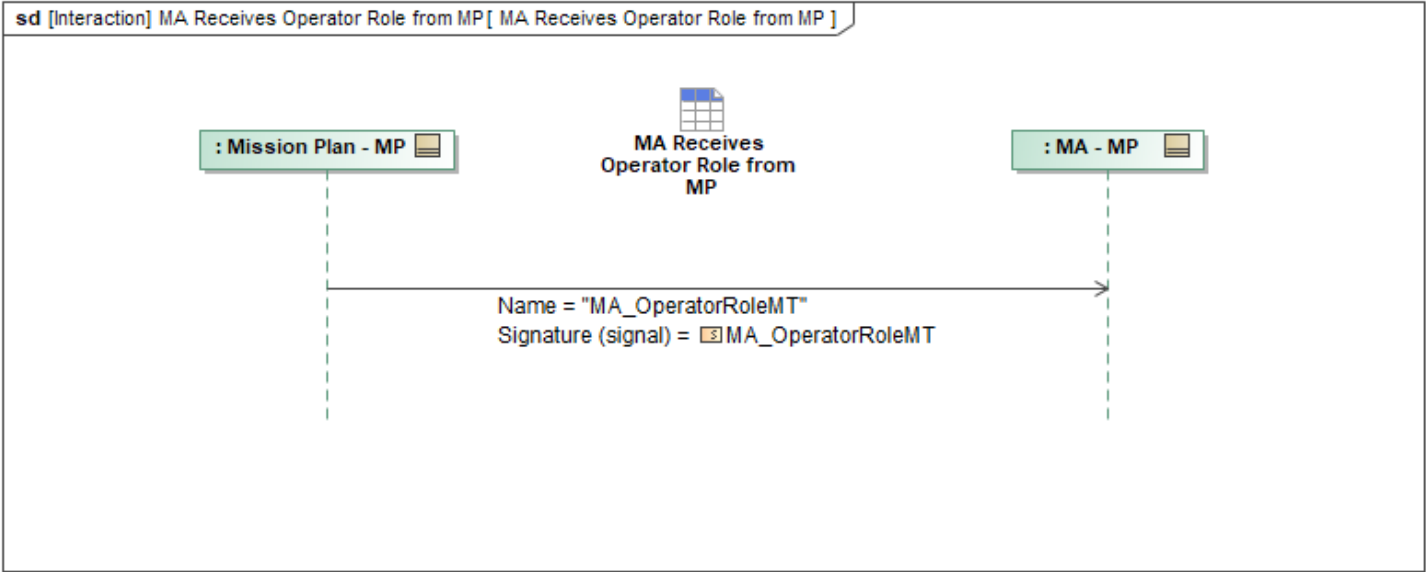


Figure 7-73: MA Receives Operator Role from MP

Table 7-71: MA Receives Operator Role from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.5 PriorityList Package

7.2.5.1 MA Receives Constraint Priority List Activation Data from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ConstraintPriorityListActivationDataMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

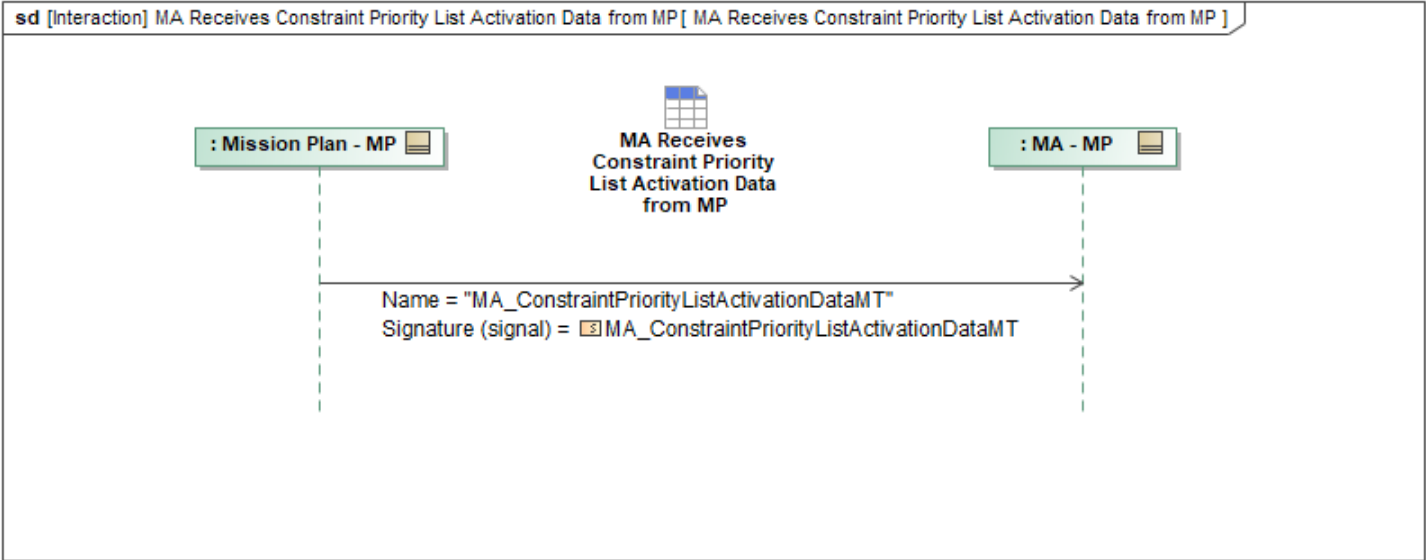


Figure 7-74: MA Receives Constraint Priority List Activation Data from MP

Table 7-72: MA Receives Constraint Priority List Activation Data from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationDataMT: MA_ConstraintPriorityListActivationDataMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.5.2 MA Receives Constraint Priority List from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ConstraintPriorityListMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

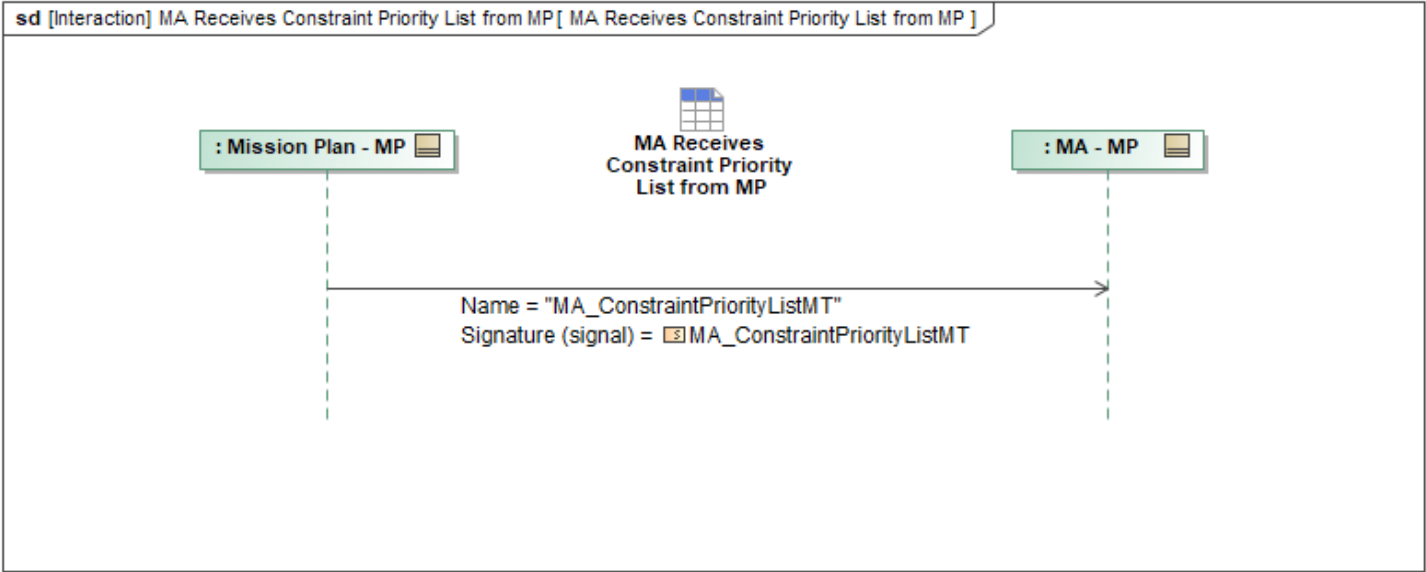


Figure 7-75: MA Receives Constraint Priority List from MP

Table 7-73: MA Receives Constraint Priority List from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListMT: MA_ConstraintPriorityListMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.5.3 MA Receives Execution Priority List Activation Data from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ExecutionPriorityListActivationDataMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

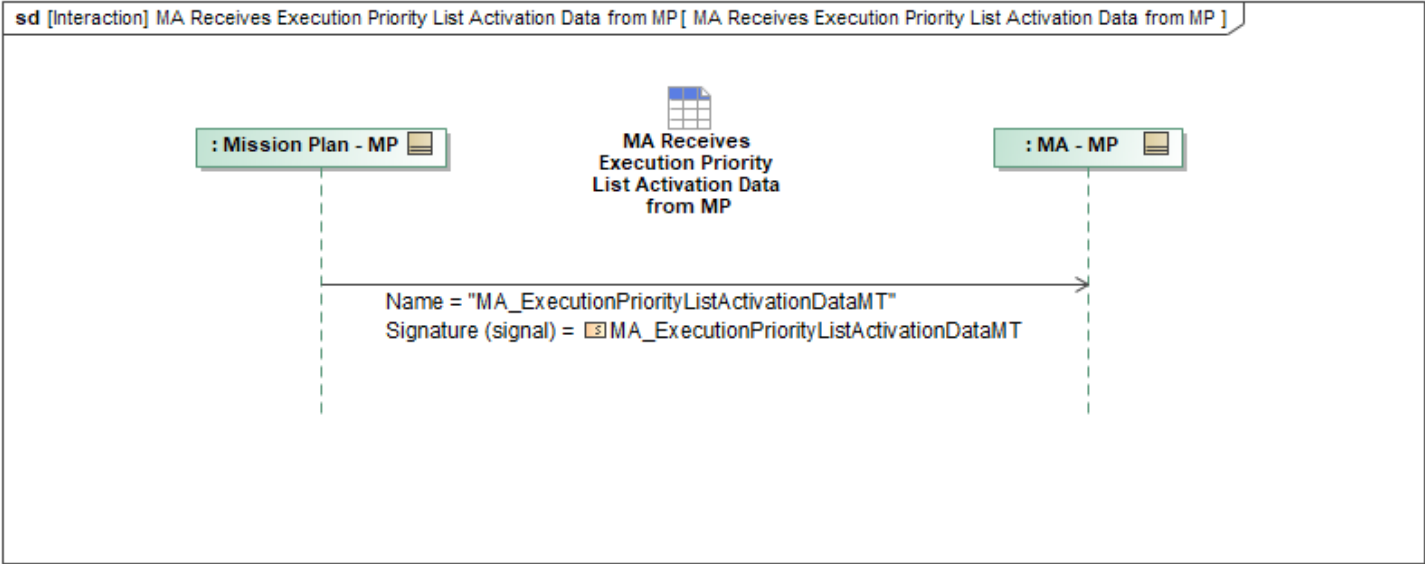


Figure 7-76: MA Receives Execution Priority List Activation Data from MP

Table 7-74: MA Receives Execution Priority List Activation Data from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationDataMT: MA_ExecutionPriorityListActivationDataMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.5.4 MA Receives Execution Priority List from MP

This interaction describes how Mission Autonomy (MA) receives *MA_ExecutionPriorityListMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

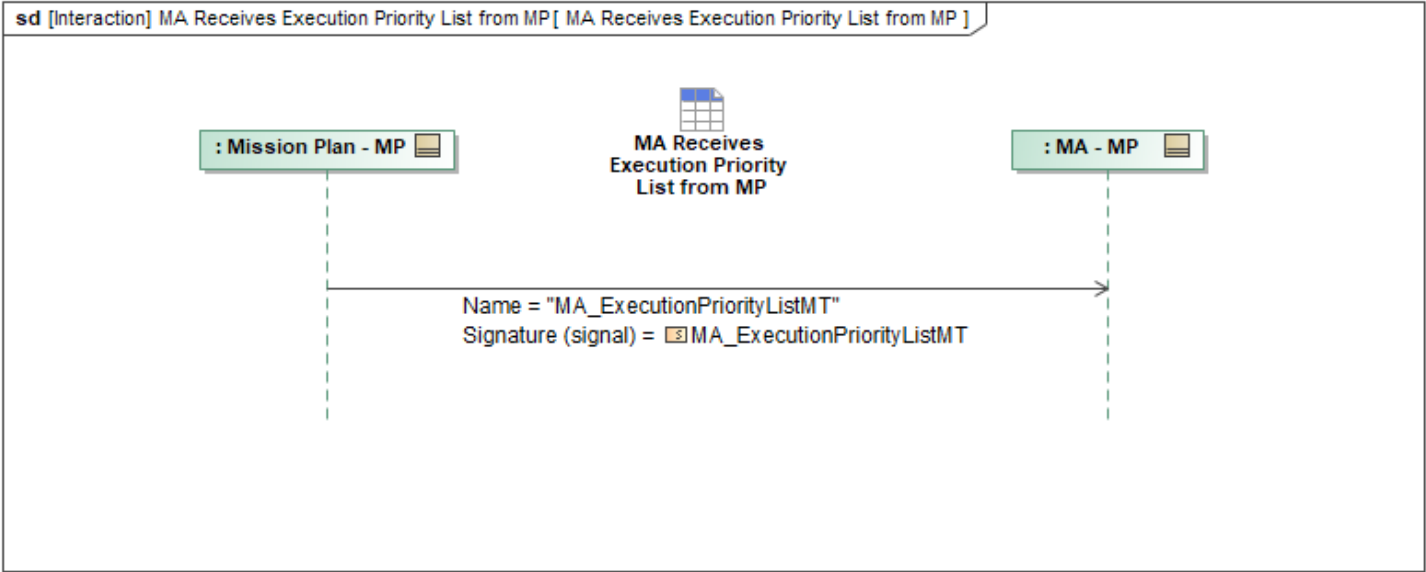


Figure 7-77: MA Receives Execution Priority List from MP

Table 7-75: MA Receives Execution Priority List from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListMT: MA_ExecutionPriorityListMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.5.5 MA Receives Prioritization List from MP

This interaction describes how Mission Autonomy (MA) receives *MA_PrioritizationListMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

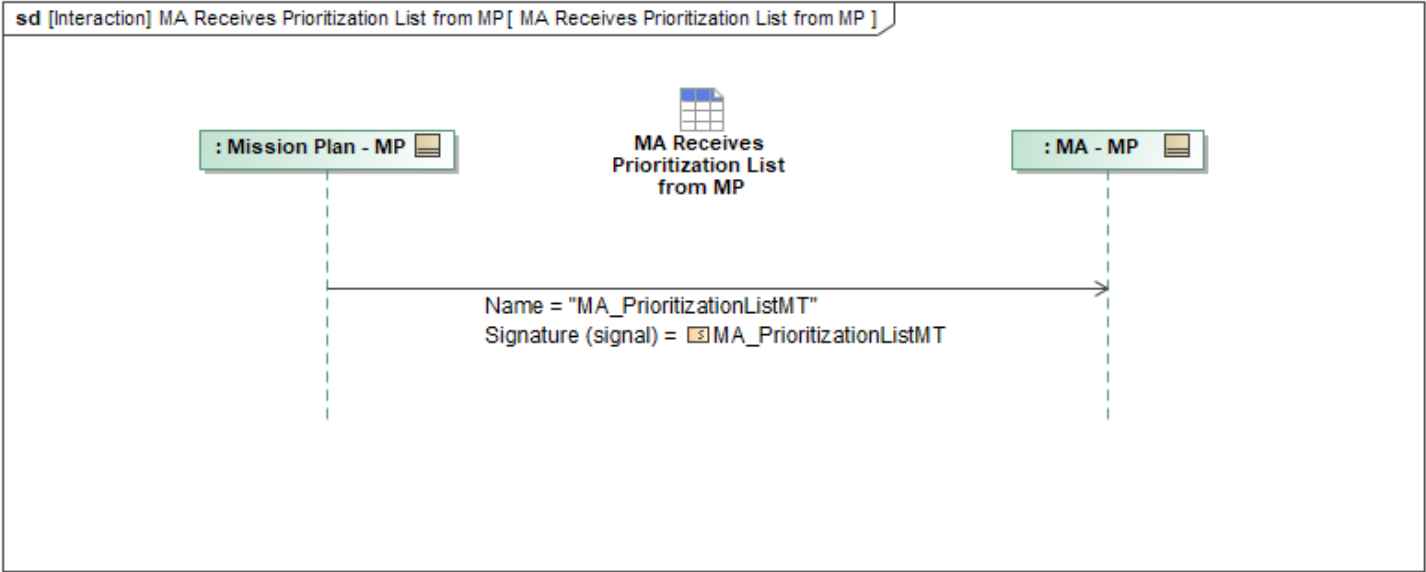


Figure 7-78: MA Receives Prioritization List from MP

Table 7-76: MA Receives Prioritization List from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.6 Requirements Package

7.2.6.1 MA Receives Capability Set from MP

This interaction describes how Mission Autonomy (MA) receives *MA_CapabilitySetMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

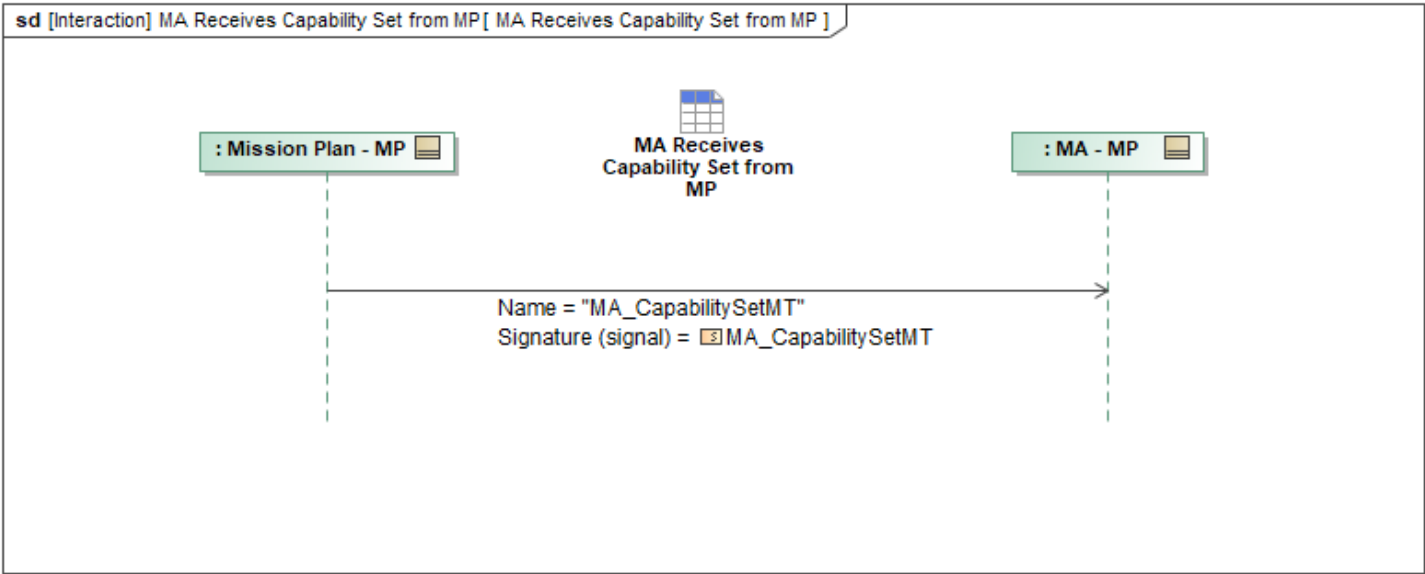


Figure 7-79: MA Receives Capability Set from MP

Table 7-77: MA Receives Capability Set from MP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CapabilitySetMT: MA_CapabilitySetMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.6.2 MA Receives Requirement Metrics from MP

This interaction describes how Mission Autonomy (MA) receives *RequirementMetricsMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

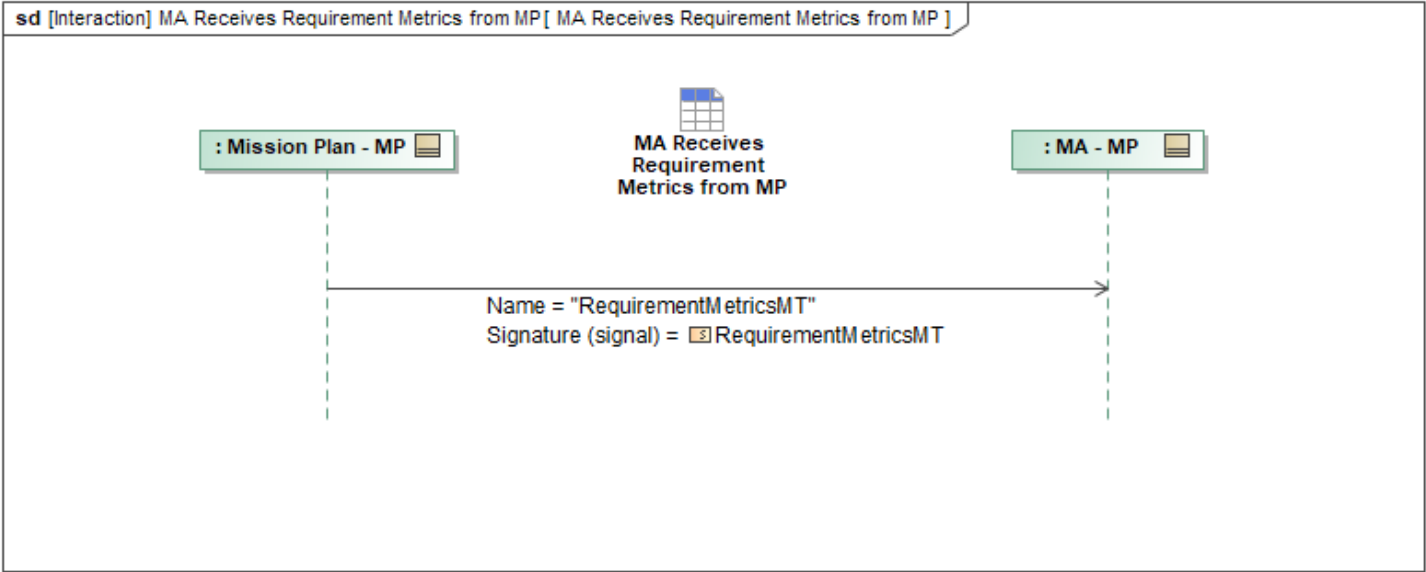


Figure 7-80: MA Receives Requirement Metrics from MP

Table 7-78: MA Receives Requirement Metrics from MP

Message Name (Name:Type)	Required Fields (Name:Type)
RequirementMetricsMT: RequirementMetricsMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.2.6.3 MA Receives Requirement Set from MP

This interaction describes how Mission Autonomy (MA) receives *RequirementSetMT* from the Mission Plan interface. The transmission of this message allows MA to access available mission data from MP during initialization. If the message being sent represents the first instance of the data, the *ObjectState* field is set to *NEW*.

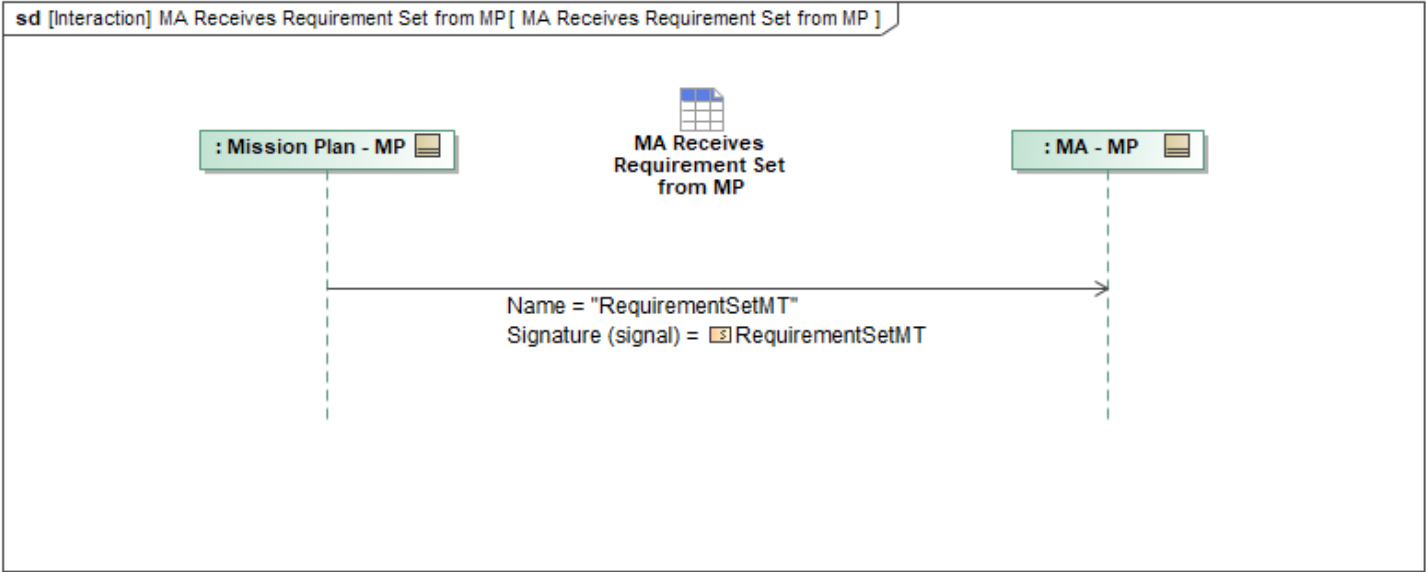


Figure 7-81: MA Receives Requirement Set from MP

Table 7-79: MA Receives Requirement Set from MP

Message Name (Name:Type)	Required Fields (Name:Type)
RequirementSetMT: RequirementSetMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

7.3 Interface Compliance

The Interface Compliance table contains the compliance requirements for the L1 MP interface.

Table 7-80: Mission Plan Interface Compliance Requirements

ID	Requirement
MA-L1-005	A Level 1 Mission Plan interface shall support the Mission Plan Interface Minimum Message Set defined in the MP MMS Table in the Mission Plan feature profile (A-GRA location: 1 Problem Domain > 3 Exchange Items > Minimum Message Set (MMS)) for all messages in Core Mission Use Case sequences and all non-Core Mission Use Case sequences required for platform specific implementations.
MA-L1-022	A Level 1 Mission Plan interface shall implement all required messaging sequences in the Mission Plan feature profile (A-GRA location: 1 Problem Domain > 2 White Box > 3 Functional Analysis > L1 Functional Analysis > L1 Interface Behaviors > Mission Plan Interface Behavior), marked Core and all non-Core Mission Use Case sequences that apply to the platform specific implementation.

8 MISSION SYSTEMS INTERFACE

The Mission Systems (MS) interface is the connection between MA and onboard MS such as sensor subsystems, payloads, and communications services. The major capabilities of the current L1 MS interface include the following:

- Data exchanges between A-GRA compliant, Mission Autonomy equipped platforms utilizing onboard Mission Systems
- Providing status data to MA such as position, navigation, and timing (PNT) data, and system status of the local and remote platforms
- Providing MS-generated operational data such as entities tracked by MS systems

The L1 MS Interface captures the UCI-based interactions that MA will use to control and interface with the both the local Mission Systems and other MA equipped ACP's. The design intent is to provide a succinct, service-based interface to capabilities provided by the MS that MA will interface with. The message set and content is intended to capture the minimal information exchange between MA and MS to support the capabilities required for the decomposed system use cases. Concerns that can be managed solely by MA, such as which ACP is the leader of an ACP team, are not exposed by MA in the L1 interface. Similarly, the L1 interface abstracts the detailed operation and implementation of MS subsystems such as network interfaces, and details such as the exact route messages take to reach the node which is the leader are not exposed by MS. The Mission Systems interface relies on OMS and A-GRA requires MA and MS to communicate via the platform hosted ASB. The A-GRA does not make any assumptions about the CAL used in MS subsystems except for the DMS.

8.1 Interface Description

Below we arrange the MS to MA interactions into a set of functional groups we refer to as services. These groupings are intended to help describe the capabilities the MS must provide and how they are accessed. They do not imply any requirements or restrictions on the internal architecture or implementation of MS. For each category, message definitions and interactions are detailed in the following section. We anticipate further refinement and extension will occur in future versions of the A-GRA.

Communications Service

The A-GRA L1 interface to MS seeks to enable communication services while isolating MA from the detailed management of the network, communication terminals and the like to the extent possible. The L1 interface defines message exchange with network endpoints currently visible to MS via publication/subscribe or a directed message. This is described below in Decentralized Message Service (DMS). To support that functionality, the Communication Service interactions provide insight into the network state. To connect to remote nodes on currently accessible network interfaces, MS may use a network discovery protocol, a pre-loaded configuration file, or other mechanism. Once connected, MS provides MA with a notification that a new node is accessible. MS provides a system identifier, analogous to a hostname in the case of an IP network, by which MA can direct messaging to the remote node. MS holds the detailed network and addressing information, making use of it when MA makes a request involving a particular system identifier. Once messaging is established between nodes, MA may exchange information with remote nodes about node type, capabilities, and other information as defined by the A-GRA MA L1 interfaces (C2, Peer). By abstracting addressing details

for MA, this enables Communications and DMS interactions to facilitate service across a set of heterogeneous networks. Methods that nodes use to exchange their system identifiers, or for consistent identifiers to be assigned for nodes on TDL or other networks that may not have the capability to provide a system identifier, has been considered and implemented. As that functionality is within the MS boundary, it is not currently included in the A-GRA interface. The comms terminal will automatically be configured based on a predetermined Comm Terminal Plan. The A-GRA does not currently require that an MA implementation make any use of the Comms Terminal Plan, though information such as any periods that lack comms and trigger specific configurations may be of use.

The "Additional Comms Capability" MUC specifies a more holistic approach to managing the OTA communications with a Comm Terminal Plan and other Comm Terminal Capability configuration/status messaging. The Comm Terminal Plan and Capability are similar to the Data Dissemination Plan described below which both add capability of updating the configurations at linked Mission Plans and other optional Triggers.

Decentralized Messaging Service

The L1 interface with MS defines interactions to send and receive messages between A-GRA compliant systems currently reachable by MS, either directly or via multiple network hops. Both publish/subscribe and directing a message to specific recipients are supported. The interface is designed to be consistent with the Gateway Capability message-passing paradigm of OMS while including additional information necessary to operate on over-the-air (OTA) networks. This includes message priority for bandwidth management and queuing as well as optional subscription filtering to eliminate unnecessary propagation of messages. The interface is designed to allow implementations to operate across heterogeneous network types (e.g. IP and non-IP) by utilizing generic asset IDs that can be mapped by the DMS onto multiple addressing schemes and by not defining interactions that require a connection-based network. The core method for messages to transfer between systems utilizes a UCI message wrapper (*MA_TxDataPayloadCommandMT*) to send messages to the DMS and another UCI message wrapper (*MA_RxDataPayloadMT*) the DMS publishes when messages are received. The prescribed UCI wrapper message require an *EncodedPayload* field for the message supplied by MA for transport. The datatype while crossing the L1 boundary between MA and MS is UTF-8 XML represented as hexBinary. The OTA wrapper message sent between the DMSs, *MA_DataPayloadWrapper* IDL, also requires an *EncodedPayload* field. The datatype while crossing between DMSs will be in EXI representation. Integration with assets that are not A-GRA compliant, and a linked send/respond message pair have been considered but are not included in the current ASK.

The "Additional Comms Capability" MUC specifies a more holistic approach to managing the OTA communications with a Data Dissemination Plan. The Data Dissemination Plan specifies how often and whether ingress/egress messages will be subscribed/published directly on the ASB or if the UCI wrappers can be used. The DMS, as a Gateway Capability, is managed by MS using the Data Dissemination Plan to configure the connection to other DMSs, the ingress/egress of messages, and how the ingress/egress occurs. The DMS can be configured to allow ingress/egress of UCI wrapped messages from/to topics and UCI wrapped messages that are intended to be sent only to specific destinations. Allowing direct message subscription from the local ASB is advantageous because it means MA does not need to manage how often and where messages need to be sent offboard, simplifying individual MA L2 service implementations.

Status Service

MS provides messages by which MA can request receipt of health, status, comms, or built-in test

(BIT) information for MS subsystems. MS will also report health, status, and BIT information of its subsystems to MA at a configurable periodic rate.

PNT Service

MS provides messages by which MA may request and receive platform position and current time. MA can also adjust the frequency PNT data is published.

Data Fusion Service

MS provides messages by which MA may receive processed tracks that are compiled from multiple sensor inputs.

8.2 Interface Interactions

The MS interactions with MA have been scoped as shown in the below figure. Within each interaction, there is a baseline set of functionalities that must be supported by MS and MA for a compliant system.

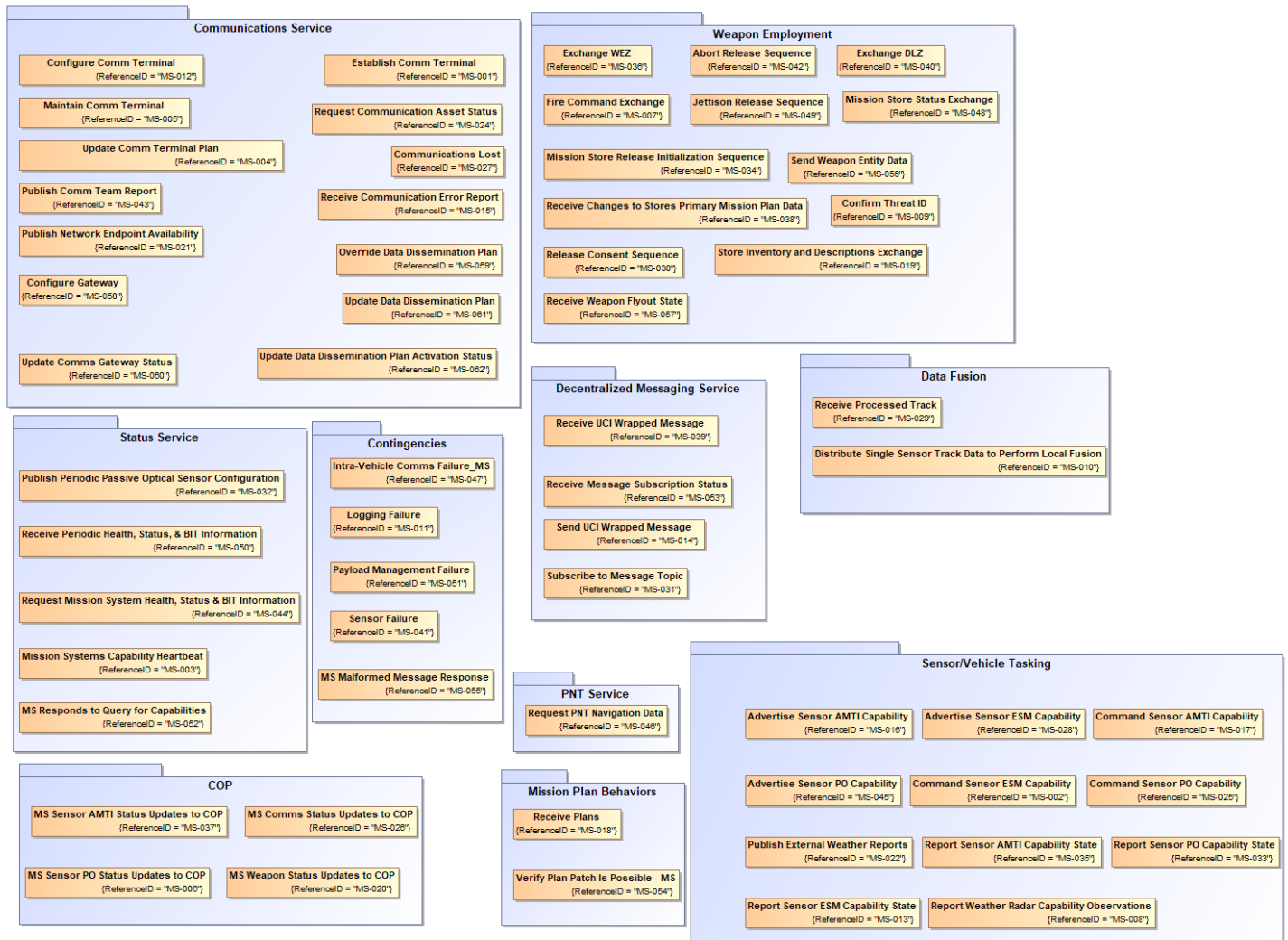


Figure 8-1: Mission Systems Interactions

8.2.1 Communications Service Package

MS is responsible for managing all parts of the communications systems that enable connecting to offboard systems, sending/receiving messages offboard, and other associated items. The parts of the communications system that will be covered in this section include the Comm Terminals, Gateways, and their plan management. The Comm Terminal Capability is used to manage the individual radio terminals that connect between systems. The Gateway Capability is used to manage what messages Egress/Ingress over those Comm Terminal systems. The core implementation includes MA monitoring specific aspects of the the comms system including loss of comms, state of the comms system, and available endpoints.

The "Additional Comms Capability" MUC includes the management of those capabilities where MA can use the associated Plan message set (*CommTerminalPlanMT* for Comm Terminals and *MA_DataDisseminationPlanMT* for Gateways). MS is responsible for using the Plans, when activated by their Activation Commands, to individually configure the Comm Terminal and Gateways. There is a method for MA to configure the Capabilities directly, but the Plans allow MS to manage and update the capabilities automatically. The Decentralized Messaging Service (DMS) is a specific type of

Gateway Capability with a method for sending/receiving UCI wrapped messages, which is the core functionality, and the "Additional Comms Capability" MUC adds subscribing/publishing messages directly to/from the local ASB. The DMS adds some clarity on how communications connections are to be established using DDS with IP based Comm Terminal links. The DMS Package section will go into further detail for how it supports UCI wrapped messaging. With the "Additional Comms Capability" MUC, the DMS will support the Gateway Capability configuration that is setup by the *MA_DataDisseminationPlanMT* message (see the "Update Data Dissemination Plan" sequence for more details).

8.2.1.1 Communications Lost

MS will monitor the communications link quality and link statistics of its capabilities onboard. If MS determines that a specific capability is degraded, MS will perform a failover activity to switch communications to an operational, redundant communications link if it's available. This degradation of a capability will be reported with *CommTerminalCapabilityStatusMT*, where *CapabilityStatus.AvailabilityInfo* [*Availability* and *AvailabilityReason*] fields provide details regarding the degraded capability. MS will also populate the updated *CapabilityPrecedence* as a result of the degraded capability. If a redundant link is not available and MS determines that each of its communications links are unavailable, MS will send a *SystemStatusMT* to MA with the *Communications.MissionCommunicationsState* of its *Communications* property set to *LOST* in order to trigger the "Communications Lost" contingency.

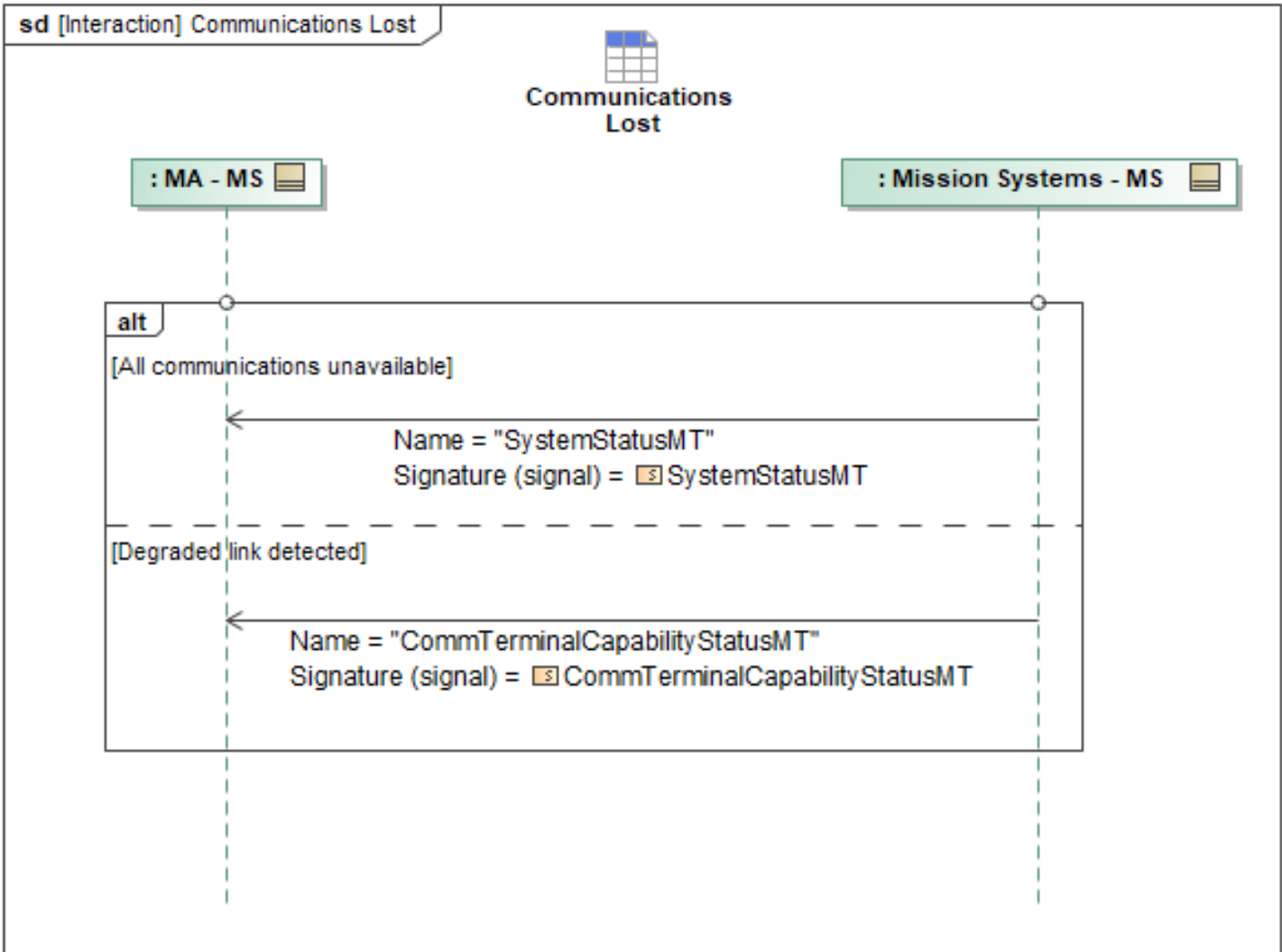


Figure 8-2: Communications Lost

Table 8-1: Communications Lost

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalCapabilityStatusMT: CommTerminalCapabilityStatusMT	CapabilityPrecedence: unsignedInt AvailabilityReason: CannotComplyType
SystemStatusMT: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.2 Configure Comm Terminal

Mission Use Case: Additional Comms Capabilities

Typically MS would handle the configuration of a Comm Terminal Capability using the Comm Terminal Plan, but if MA needs to make a direct modification MA can send a *CommTerminalCommandMT* directly to MS. If the command is accepted, MS responds with

CommTerminalCommandStatusMT_1 with the *Activity* that satisfies the associated *CommTerminalCommandMT*. MS will then provide status of the resulting *Activity* to MA through *CommTerminalActivityMT*. If the command is not accepted, MS responds with *CommTerminalCommandStatusMT_2* with *CommandProcessingStateReason* indicating why the command was not in an acceptable state.

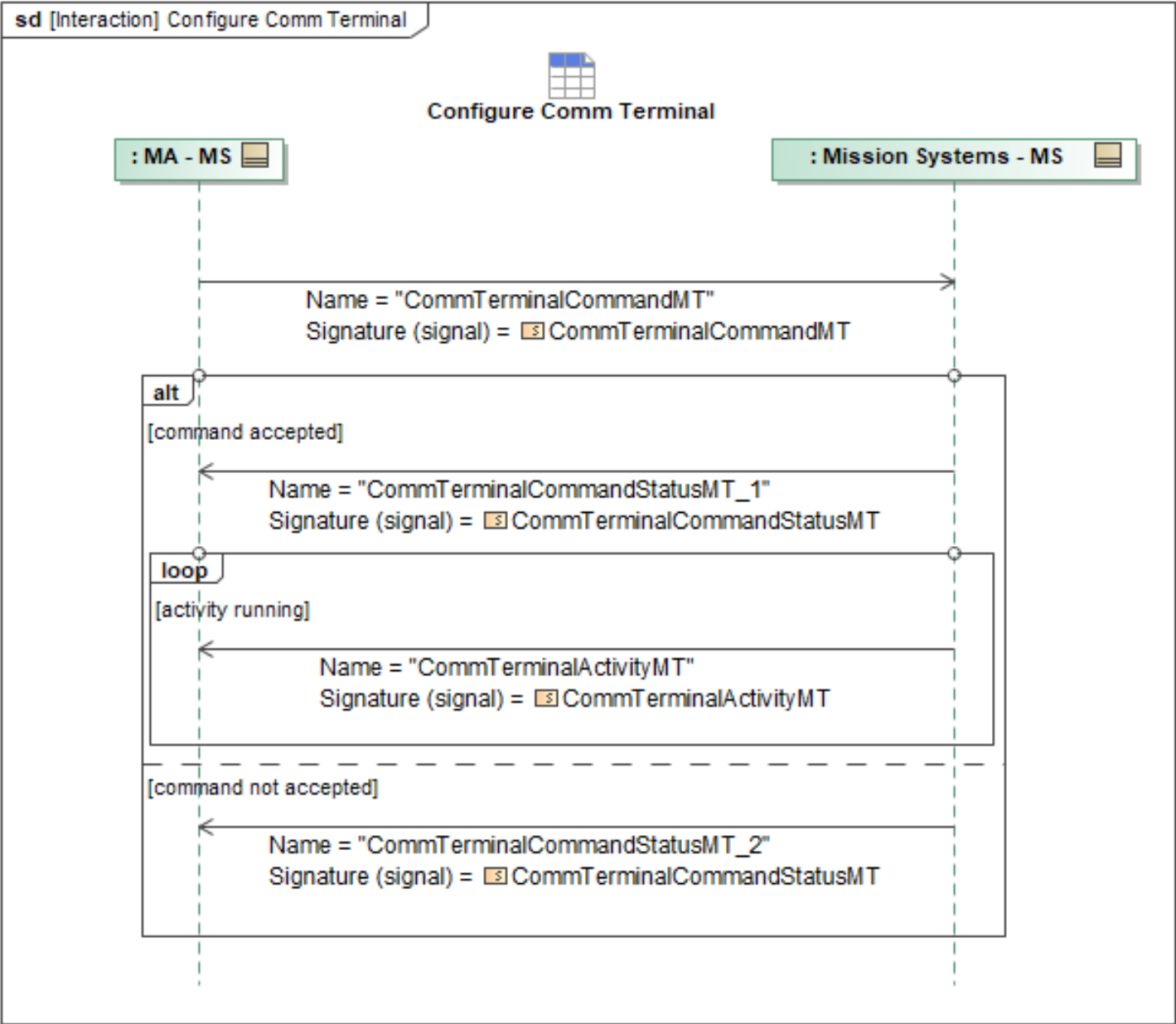


Figure 8-3: Configure Comm Terminal

Table 8-2: Configure Comm Terminal

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalActivityMT: CommTerminalActivityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalCommandMT: CommTerminalCommandMT	None
CommTerminalCommandStatusMT_1: CommTerminalCommandStatusMT	Activity: CommTerminalActivityCommandType
CommTerminalCommandStatusMT_2: CommTerminalCommandStatusMT	CommandProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.3 Configure Gateway

Mission Use Case: Additional Comms Capabilities

Typically MS would handle the configuration of a Gateway Capability using the Data Dissemination Plan, but if MA needs to make a direct modification MA can send a *MA_GatewayCommandMT* message to MS directly. Then, if the command is accepted, MS sends the *MA_GatewayCommandStatusMT_1* to MA. If a Gateway Activity is running, MS also sends the *MA_GatewayActivityMT* message to MA. If the command is NOT accepted, MS sends the *MA_GatewayCommandStatusMT_2* message to MA with *CommandProcessingStateEnum* set to "REJECTED" and the *CommandProcessingStateReason* filled out as needed.

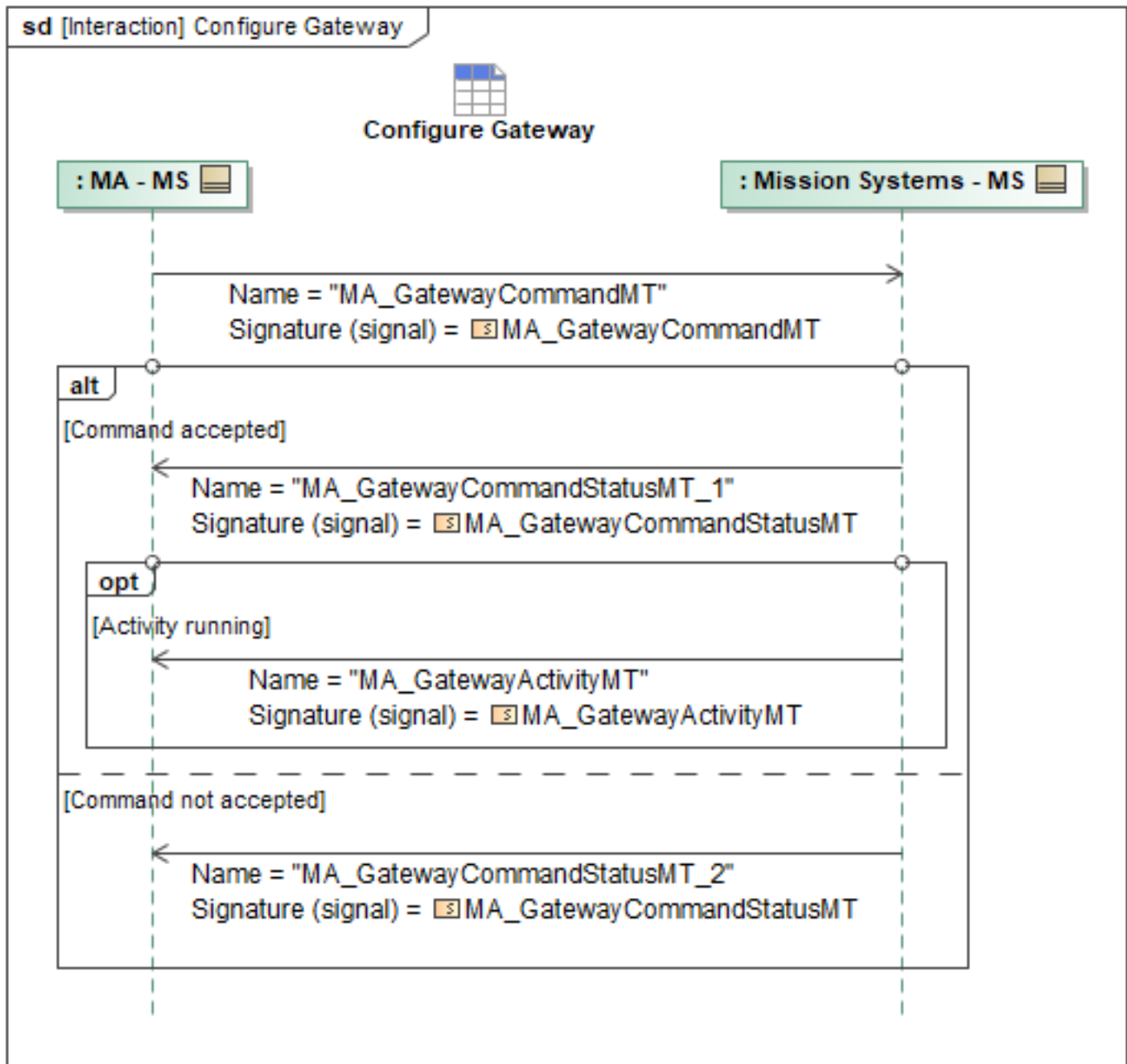


Figure 8-4: Configure Gateway

Table 8-3: Configure Gateway

Message Name (Name:Type)	Required Fields (Name:Type)
MA_GatewayActivityMT: MA_GatewayActivityMT	None
MA_GatewayCommandMT: MA_GatewayCommandMT	None
MA_GatewayCommandStatusMT_1: MA_GatewayCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_GatewayCommandStatusMT_2: MA_GatewayCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.4 Establish Comm Terminal

The Establish Comm Terminal interaction provides MA with a periodic description of the MS communications system, communications assets, and communication capabilities that may be utilized. The communications system and its assets are defined through *CommSystemMT* and *CommAssetMT*. A *CommAssetMT* is provided for each *CommAssetID* that is defined in *CommSystemMT*. *CommTerminalCapabilityMT* is provided by Mission Systems to establish the capabilities of the communication terminal along with each capability's configurable characteristics and settings. Lastly, the message sequence provides MA the current schedule for communication activities that are scheduled for execution with *CommScheduleAllocationMT*.

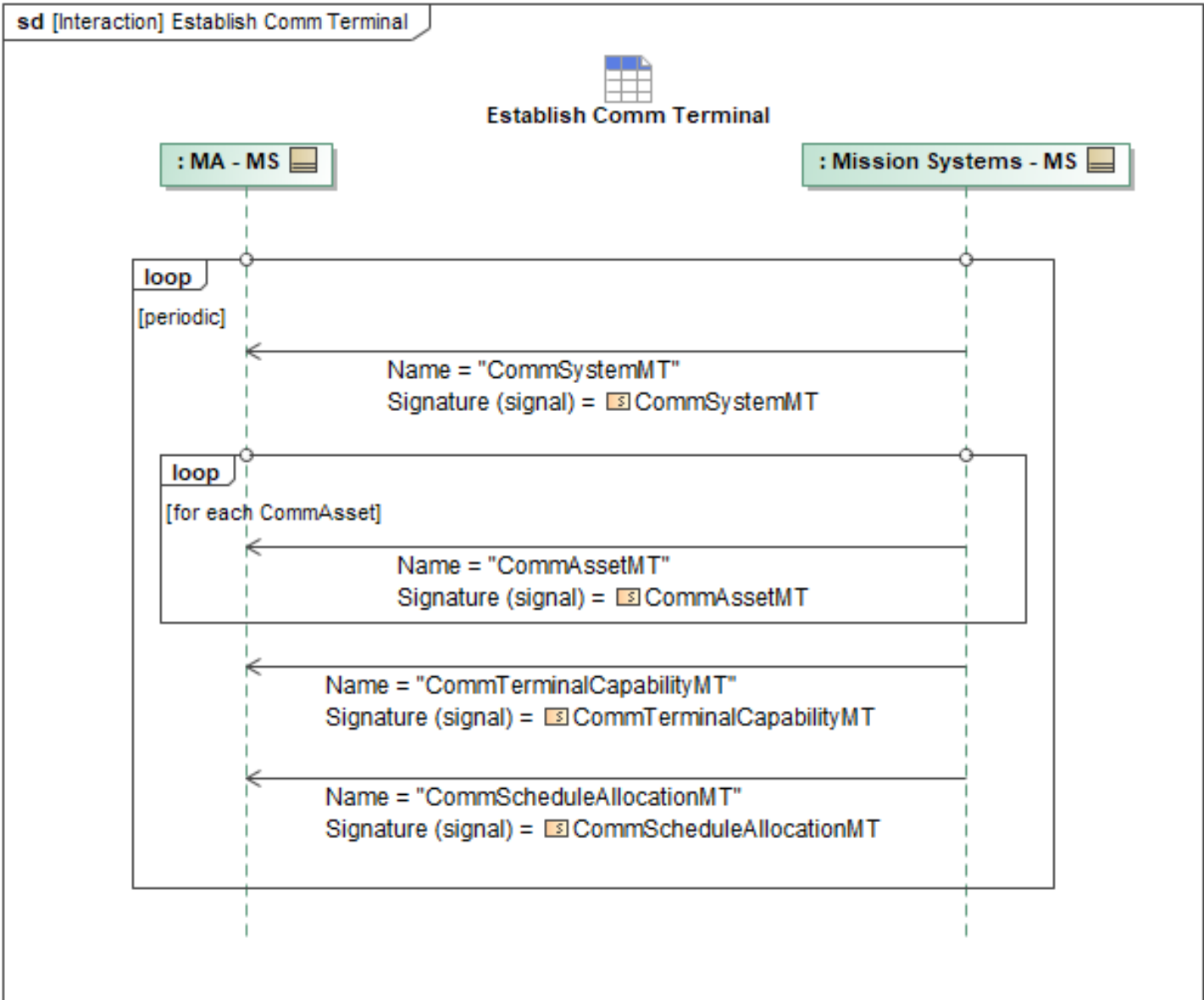


Figure 8-5: Establish Comm Terminal

Table 8-4: Establish Comm Terminal

Message Name (Name:Type)	Required Fields (Name:Type)
CommAssetMT: CommAssetMT	None
CommScheduleAllocationMT: CommScheduleAllocationMT	None
CommSystemMT: CommSystemMT	None
CommTerminalCapabilityMT: CommTerminalCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.5 Maintain Comm Terminal

MS will periodically send *CommTerminalCapabilityStatusMT* to MA to report the current states of waveforms and the capabilities those waveforms are being used for within the comm terminal and system. If a communications activity is running, MS will send *CommTerminalActivityMT*, which provides detailed information on what exactly a waveform capability is doing at any point in time through the *SubsystemID* and *Activity* information fields. MS specifically statuses the data links settings and statistics through *ConnectionStatus*, *CommonSettingsStatus*, and *LinkStatistics* information fields. MS reports specific waveform settings status (such as Link 16 waveform settings status) through the *WaveformSpecificSettingsStatus* field. This series of message exchanges is intended to be exchanged while comm terminals are active within an operational state.

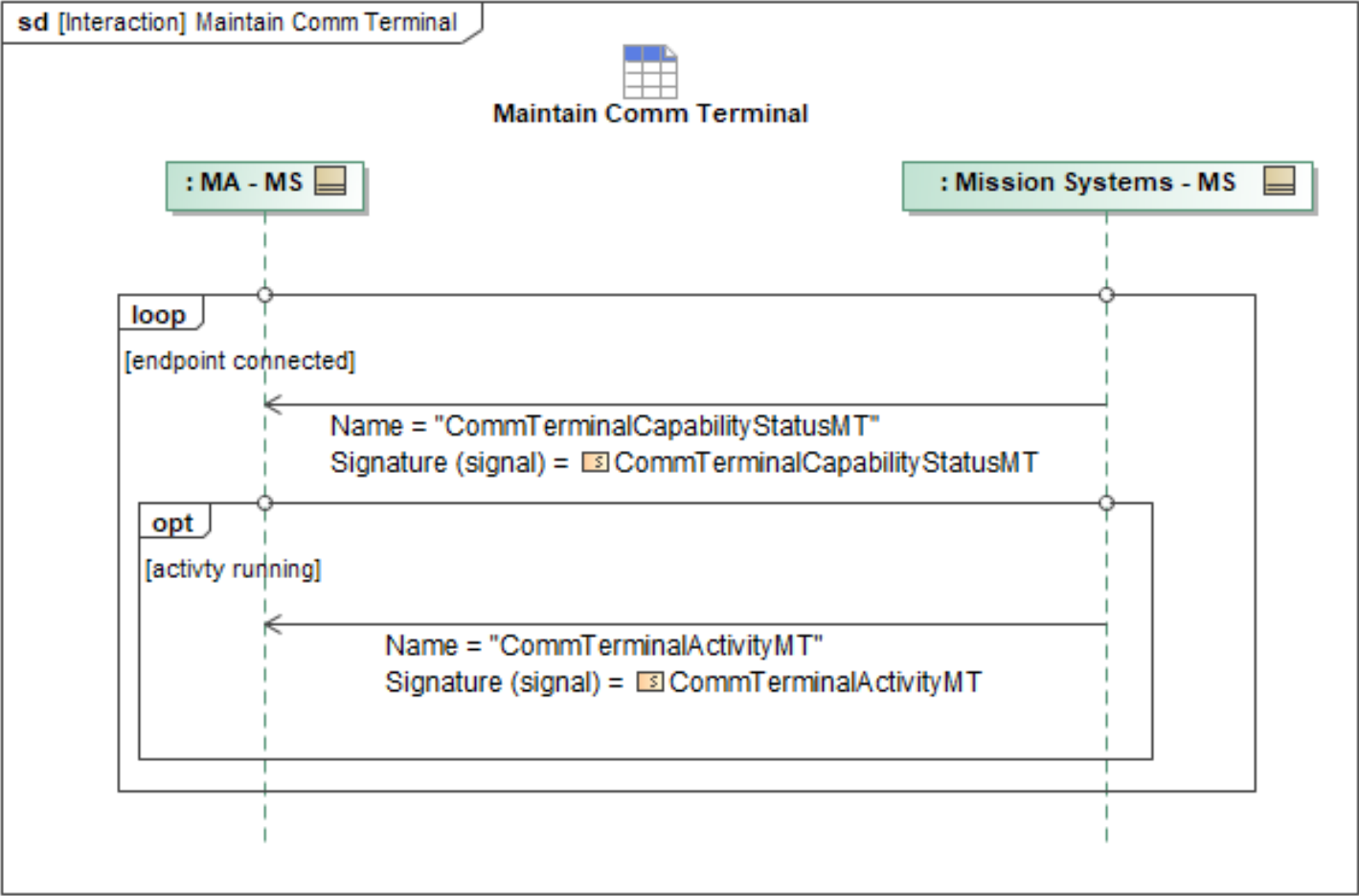


Figure 8-6: Maintain Comm Terminal

Table 8-5: Maintain Comm Terminal

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalActivityMT: CommTerminalActivityMT	CommonSettingsStatus: CommControlType LinkStatistics: CommLinkStatisticsType WaveformSpecificSettingsStatus: CommWaveformActivityPET ConnectionStatus: CommConnectionStateEnum

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalCapabilityStatusMT: CommTerminalCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.6 Override Data Dissemination Plan

Mission Use Case: Additional Comms Capabilities

In some situations a modification to the Data Dissemination Plan is required that are short term in nature and a complete update to the *MA_DataDisseminationPlanMT* message is not required. The *MA_DataDisseminationPlanOverrideRequestMT* can be used for these sorts of modifications. MA sends the *MA_DataDisseminationPlanOverrideRequestMT* message to MS. MS can then accept or reject the request(s). If the request(s) are accepted, MS sends the *MA_DataDisseminationPlanOverrideRequestStatusMT_1* message to MA with the Request(s) having the *Request.RequestProcessingState* set to "QUEUED", "PROCESSING", "CANCELED", or "COMPLETED" as needed. Depending on the original request, the *EffectivityActivationStatus* or *OperatorModificationStatus* fields will be filled out as needed. The interaction "Update Data Dissemination Plan Activation Status" is initiated which will provide an update to the active status and include the *OverrideModificationState* associated with the override request status. For each applicable Gateway Activity, MS sends the *MA_GatewayActivityMT* message(s) to MA that will include any modifications that were made. If the request is NOT accepted, MS sends the *MA_DataDisseminationPlanOverrideRequestStatusMT_2* message to MA with the *RequestProcessingState* set to "REJECTED" or "FAILED", and the *RequestProcessingStateReason* filled out as needed. The *EffectivityActivationStatus* or *OperatorModificationStatus* fields will be filled out outlining the reason for the rejections/failures.

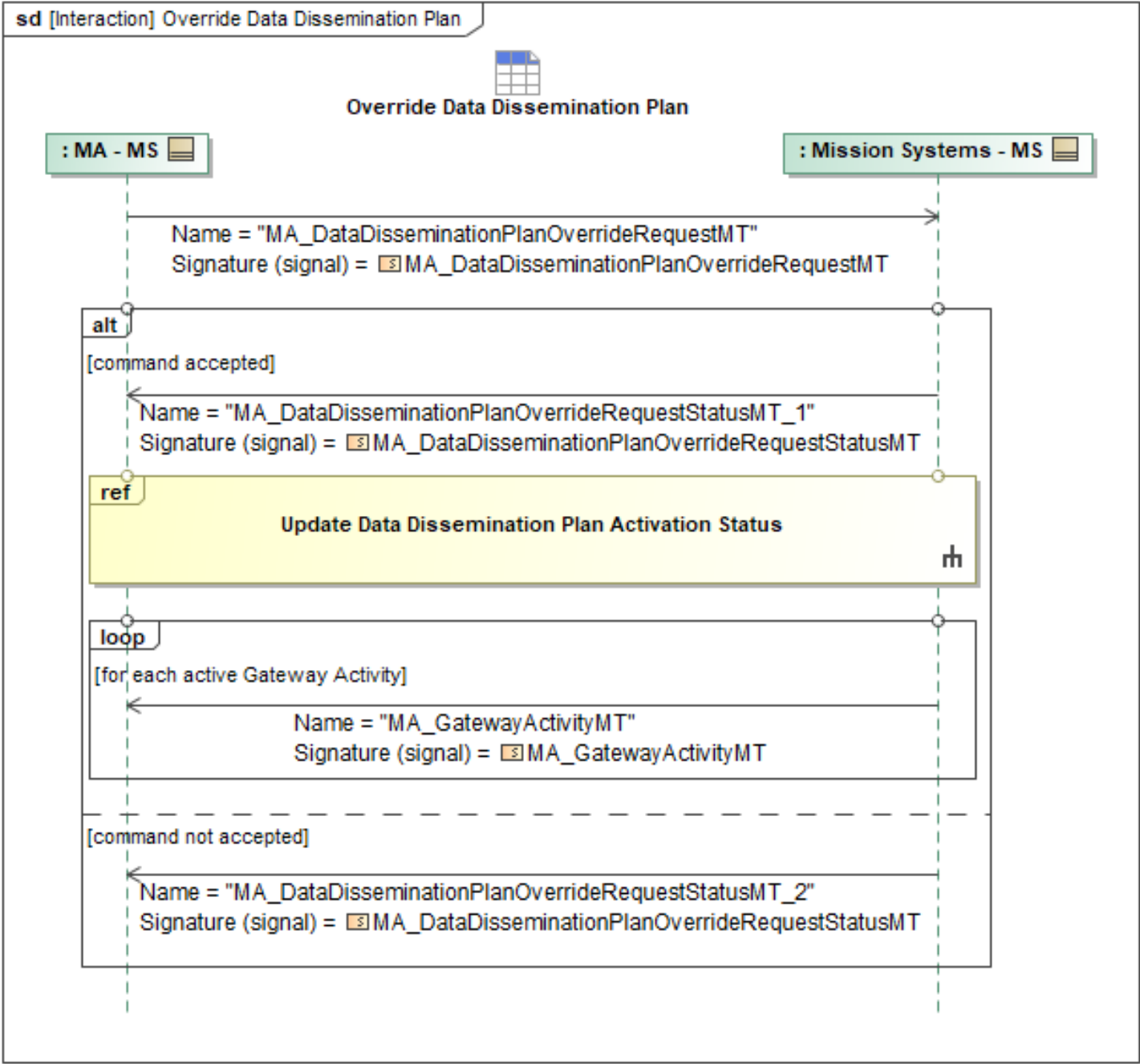


Figure 8-7: Override Data Dissemination Plan

Table 8-6: Override Data Dissemination Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanOverrideRequestMT : MA_DataDisseminationPlanOverrideRequestMT	AllocatedToSystemIDs: MA_PackageSystemType

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanOverrideRequestStatusMT_1: MA_DataDisseminationPlanOverrideRequestStatusMT	None
MA_DataDisseminationPlanOverrideRequestStatusMT_2: MA_DataDisseminationPlanOverrideRequestStatusMT	RequestProcessingStateReason: CannotComplyType
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.7 Publish Comm Team Report

MS publishes the Communication Team Report for MA's situational awareness when a new node/endpoint is discovered or leaves the team. If a team exists, the *MA_CommTeamReportMT_1* message reports a team's composition to MA by using the *SystemID* and *EndpointLabel* for each endpoint. If there are no node/endpoints to report, MS reports *MA_CommTeamReportMT_2*.

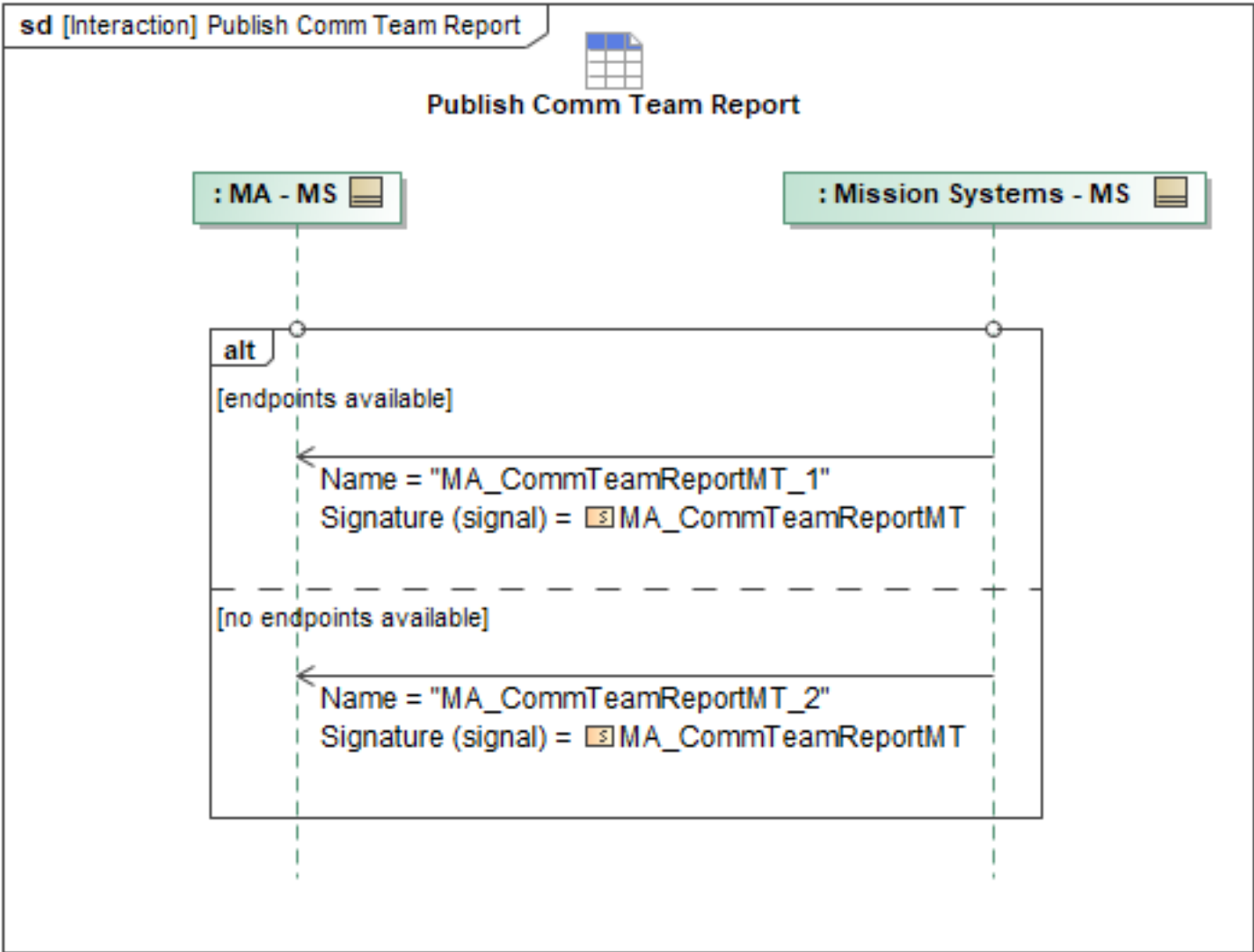


Figure 8-8: Publish Comm Team Report

Table 8-7: Publish Comm Team Report

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CommTeamReportMT_1: MA_CommTeamReportMT	EndpointLabel: VisibleString256Type
MA_CommTeamReportMT_2: MA_CommTeamReportMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.8 Publish Network Endpoint Availability

The MS information service will publish the *MA_CommAvailableEndpointsMT* message when new endpoints become available in the network, or when the network membership is updated. This message conveys to MA the available network endpoints by its *SystemID* and *EndpointLabel* in the *AvailableNetworkEndpoints* field, if endpoints are available to report.

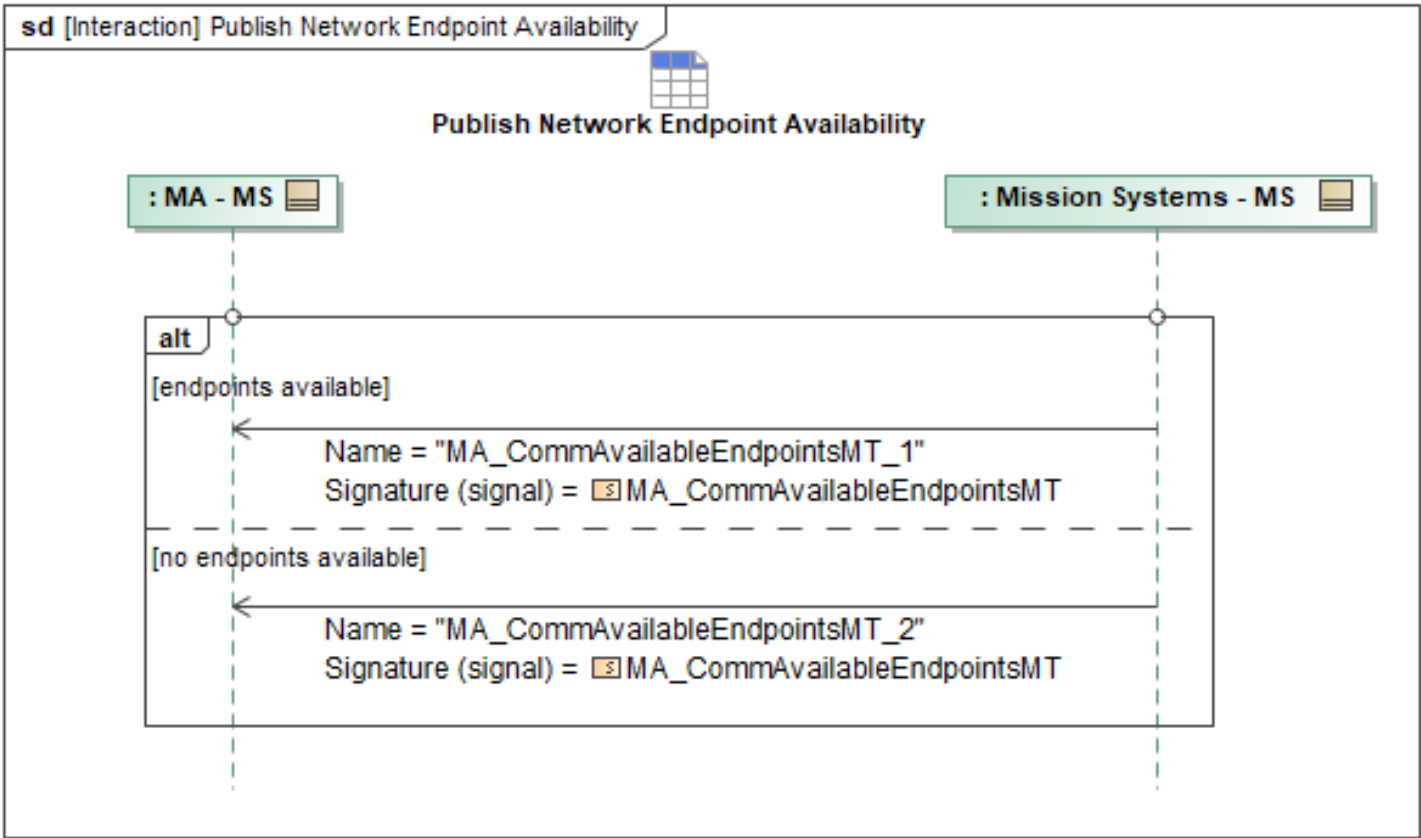


Figure 8-9: Publish Network Endpoint Availability

Table 8-8: Publish Network Endpoint Availability

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CommAvailableEndpointsMT_1: MA_CommAvailableEndpointsMT	EndpointLabel: VisibleString256Type AvailableNetworkEndpoints: MA_AvailableNetworkEndPointsType
MA_CommAvailableEndpointsMT_2: MA_CommAvailableEndpointsMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.9 Receive Communication Error Report

MS can report communication faults to MA using the *MA_CommErrorReportMT* message. This message includes a *CommErrorReportID* field to identify the communication error report and an *ErrorDetail* field to specify details, including the source of the fault for a specified *SystemID*.

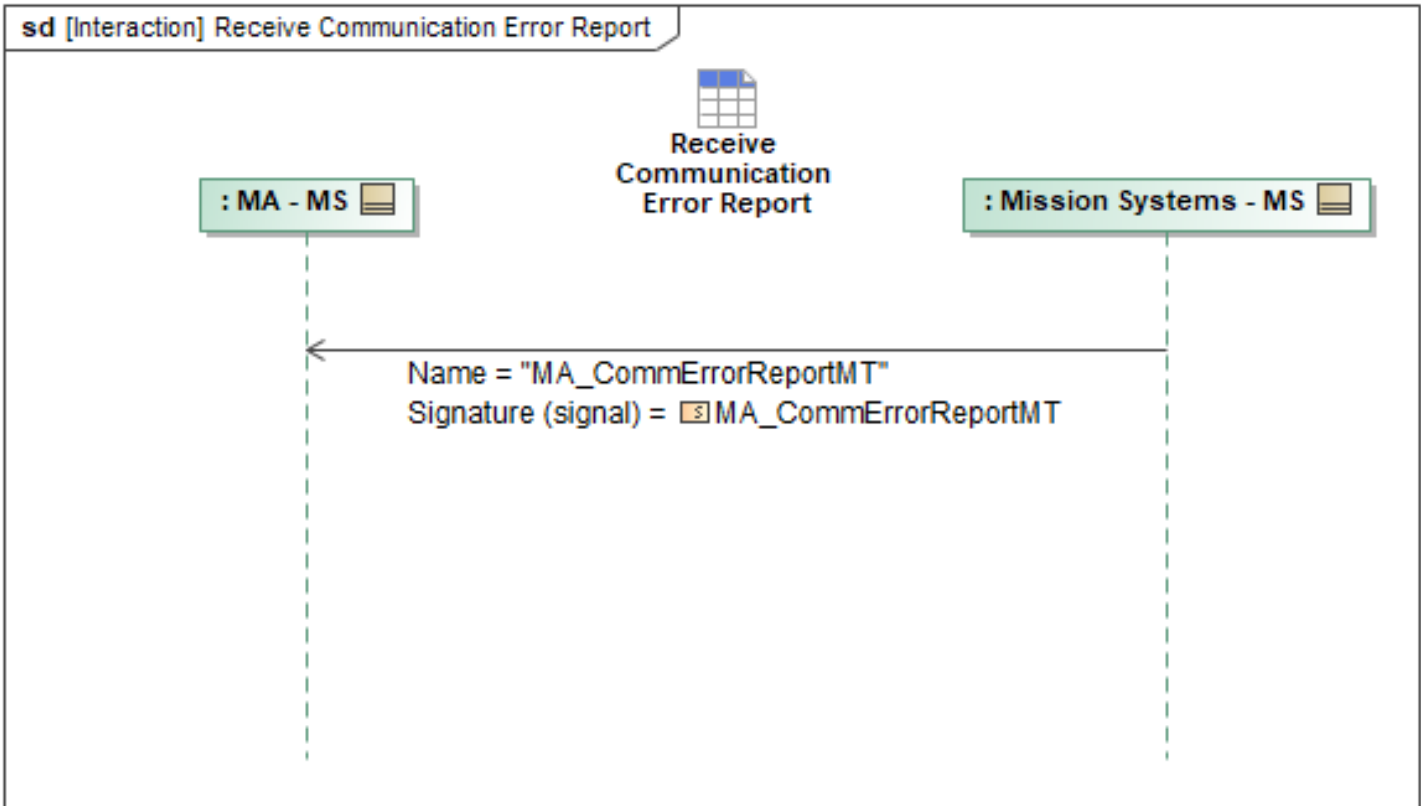


Figure 8-10: Receive Communication Error Report

Table 8-9: Receive Communication Error Report

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CommErrorReportMT: MA_CommErrorReportMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.10 Request Communication Asset Status

Mission Use Case: Additional Comms Capabilities

The Request Communication Asset Status sequence is used by MA to request the estimated position and status of a MS communication asset. MA would request information on the communication asset through *CommAssetEstimationRequestMT* with the *CommAssetID* of the asset of interest and *ResultsInNativeMessage* set to FALSE. MS would respond with the *EstimationResult* of the communication asset that was requested in the *CommAssetEstimationRequestStatusMT*.

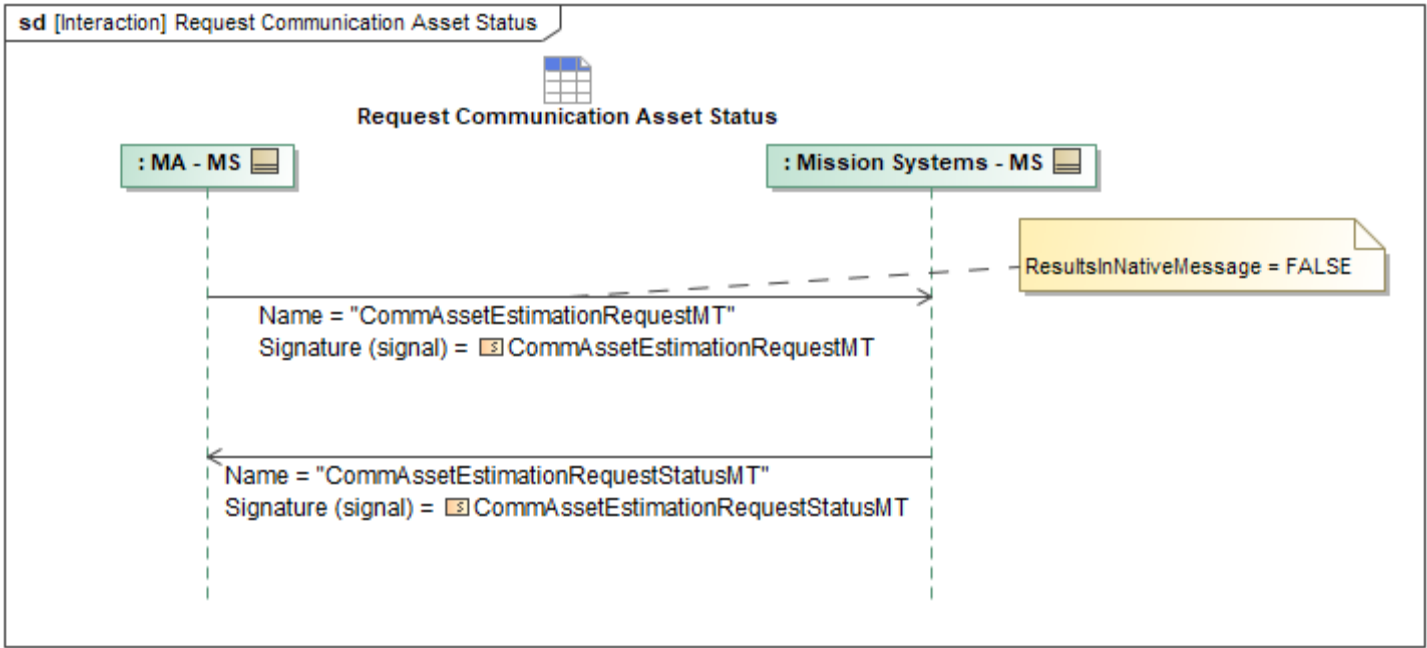


Figure 8-11: Request Communication Asset Status

Table 8-10: Request Communication Asset Status

Message Name (Name:Type)	Required Fields (Name:Type)
CommAssetEstimationRequestMT: CommAssetEstimationRequestMT	None
CommAssetEstimationRequestStatusMT: CommAssetEstimationRequestStatusMT	EstimationResult: CommAssetEstimationResultType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.11 Update Comm Terminal Plan

Mission Use Case: Additional Comms Capabilities

The *CommTerminalPlanMT* is used to configure how individual Comm Terminal Capabilities should be configured for an individual system. The plan contains a set of *Effectivities* that can be activated directly or setup with *Triggers* for different Mission Plans, sub-plans, etc. The *Effectivity* can specify the configuration of the entire Comm Terminal subsystem using a Mission Data File (MDF) or using Capability specific settings. These configurations of the Comm Terminal Capabilities allows a system to connect to other systems over various communication networks. First, MA shall post a *CommTerminalPlanMT* message that the communication service, in MS, will use when a command is issued. *CommTerminalPlanMT* will specify a set of desired states of the network communication capability and how they can be activated with the configuration setup in individual *Effectivities* and associated *Triggers*. Second, MA shall send a *CommTerminalPlanActivationCommandMT* to MS to command the activation of the previously posted *CommTerminalPlanMT* referencing the associated *CommTerminalPlanID*, and optionally populating the *EffectivityActivation* with an *EffectivityID* and *ConfigurationID*. Finally, MS replies to MA with a *CommTerminalPlanActivationCommandStatusMT_1* message if the command is accepted. While the plan activation is in-progress, until the activation

status is in a success or fail state, MS will report *CommTerminalPlanActivationCommandStatusMT_2*. If the command is not accepted, MS will report *CommTerminalPlanActivationCommandStatusMT_3*.

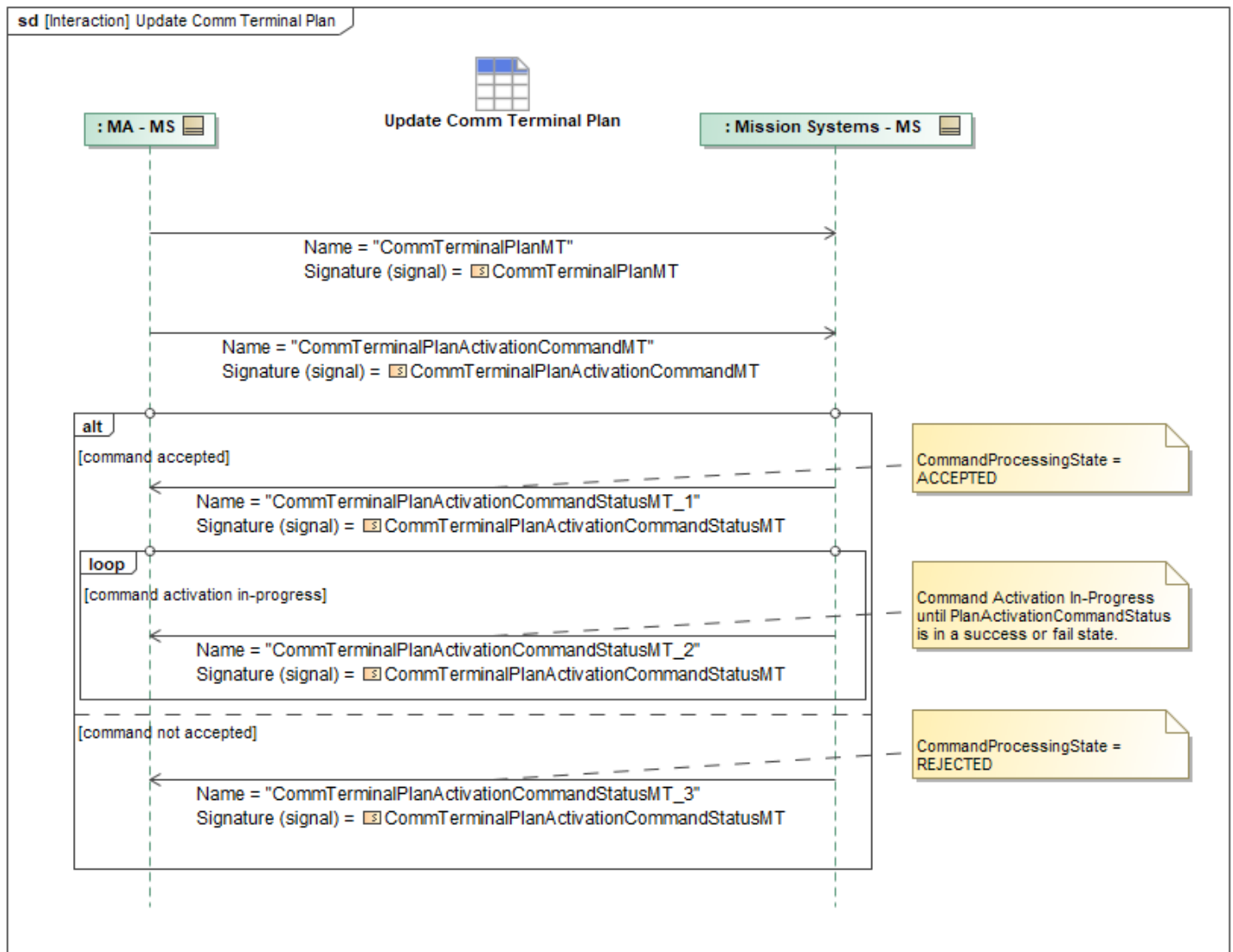


Figure 8-12: Update Comm Terminal Plan

Table 8-11: Update Comm Terminal Plan

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalPlanActivationCommandMT: CommTerminalPlanActivationCommandMT	EffectivityActivation: CommTerminalPlanEffectivityActivationType
CommTerminalPlanActivationCommandStatus MT_1: CommTerminalPlanActivationCommandStatus MT	None
CommTerminalPlanActivationCommandStatus MT_2: CommTerminalPlanActivationCommandStatus MT	None

Message Name (Name:Type)	Required Fields (Name:Type)
CommTerminalPlanActivationCommandStatus MT_3: CommTerminalPlanActivationCommandStatus MT	CommandProcessingStateReason: CannotComplyType
CommTerminalPlanMT: CommTerminalPlanMT	PlannedCommCapability: PlannedCommCapabilityType ConnectionRequest: CommConnectionCommandEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.12 Update Comms Gateway Status

Mission Use Case: Additional Comms Capabilities

The Gateway Capability is how messages egress or ingress over the air. To provide information about each of the active gateways available for use, MS sends the *GatewayCapabilityMT* message and the *GatewayCapabilityStatusMT* message to MA. Then, for each active Gateway Activity, MS send the *MA_GatewayActivityMT* message(s) to MA. These messages provide the information needed to understand the state of the over the air messaging, they will provide context for how things are currently configured, and provide information for how to configure over the air messaging. MS typically configures the gateways, but the information can be useful when creating the plans that the MS uses.

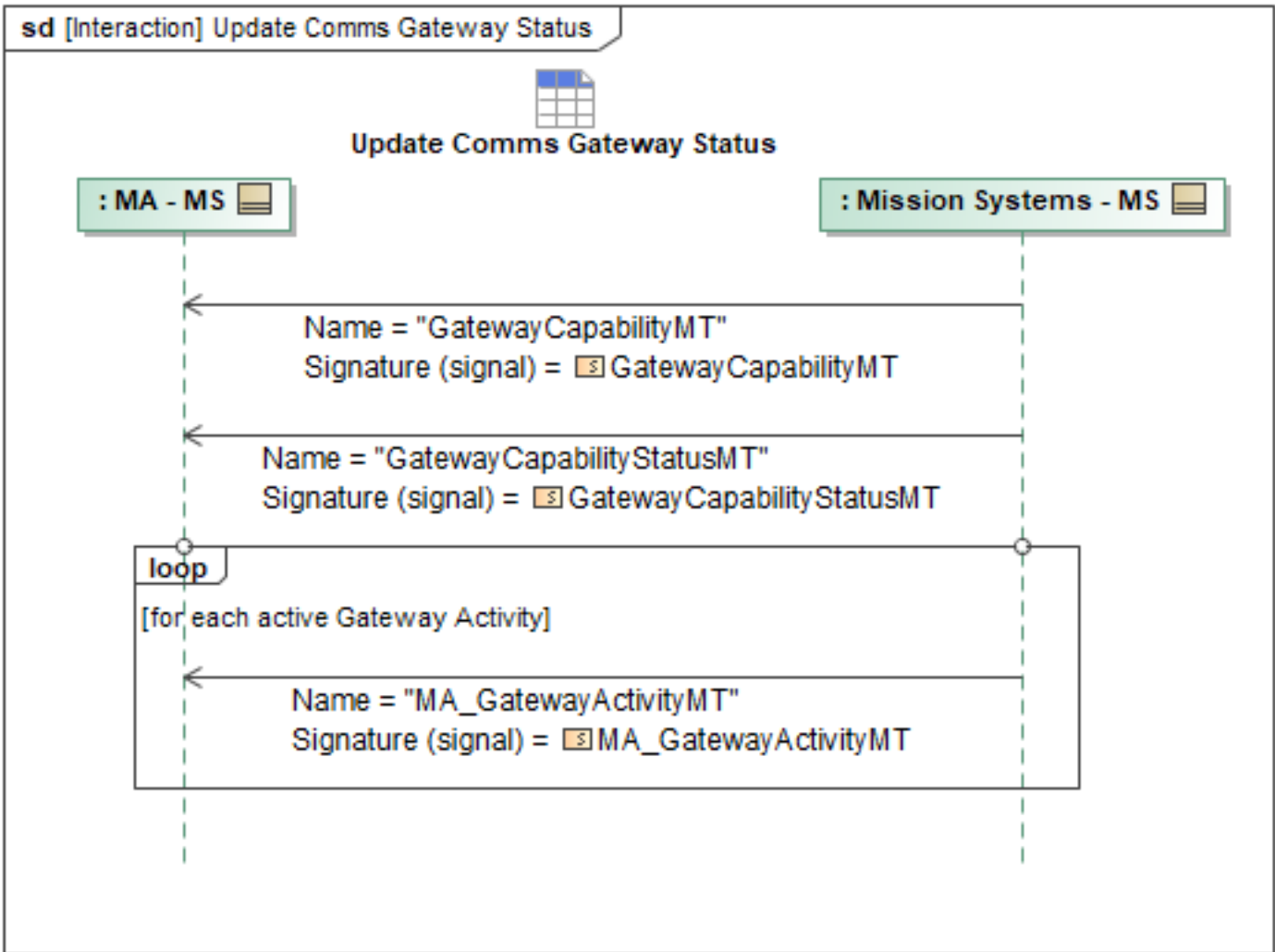


Figure 8-13: Update Comms Gateway Status

Table 8-12: Update Comms Gateway Status

Message Name (Name:Type)	Required Fields (Name:Type)
GatewayCapabilityMT: GatewayCapabilityMT	None
GatewayCapabilityStatusMT: GatewayCapabilityStatusMT	None
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.13 Update Data Dissemination Plan

Mission Use Case: Additional Comms Capabilities

The *MA_DataDisseminationPlanMT* is used to configure how messages ingress and egress over the air. The message contains a set of Effectivities that can be activated (using the

MA_DataDisseminationPlanActivationCommandMT message) or overridden (using the *MA_DataDisseminationPlanOverrideRequestMT* message) as needed. This implementation focuses using DDS as the over the air transport. The *Effectivity* field is used to match Information Exchange Requirement(s) (IERs) to DistributionGroup(s) and RateGroup(s) if the messages are periodic. The Applicability field in the *MA_DataDisseminationPlanMT* is used to set what systems or packages this messages applies to. The *AssociatedPlanID* field can be used to link together multiple *MA_DataDisseminationPlanMT* as needed.

The *IER* field is used to define the set of messages that will ingress or egress from the local system. The *IER.ContentType* field specifies the message filters. Periodic messages must not be combined into a single *IER.ContentType*, but non-periodic messages that have other matching items like Direction or Query can be combined. The *IER.ContentType.Type.TacticalDataLink* can be used to define messages that are meant to be sent over Tactical Data Links specifically. Alternatively, *IER.ContentType.Type.UCI_Message* can be used for DDS type connections when matched with a Distribution Group configured for DDS. When DDS is configured and the *IER.ContentType.Direction* is set to "EGRESS", "INGRESS", or "BOTH" they would match to "PUBLISH", "SUBSCRIBE", "PUB_AND_SUB", if the *DistributionGroup.NetworkNativeEndpoint.TopicPublishSubscribe* field is not set. If the *IER.ContentType.AllowWrappedEgressMessages* field is set to "TRUE", this means that the implementing Gateway will accept a wrapped *MA_TxDataPayloadCommandMT* message along with the non-wrapped message before using the Query filter for that IER. It is up to the implementation to not send duplicate messages if the same message is received both wrapped and unwrapped. The *IER.Priority* field will define the priority of that IER compared to other IERs and will be used as the DDS topic priority, if the *DistributionGroup.NetworkNativeEndpoint.TopicQosPriority* field is not set.

The DistributionGroup field is used to configure the DDS specific items using the *DistributionGroup.NetworkNativeEndpoint* with the *DDS_DataLinkNativeInfoType*. The settings in this field will override settings from other fields. The Domain field is used to define the DDS Domain ID the Gateway will connect to. The TopicName field is used to define an individual topic name specifically, or if left blank will default to the message type name defined in the connected IER. The *TopicPublishSubscribe* field can be used to specifically set the intended direction of the message(s) from the IER, or if left blank will default to the connected IER settings for Direction. The TopicQosReliability field can be use to specifically set "RELIABLE" or "BEST_EFFORT", but if set to "NONE" or left unset the A-GRA method for UCI_PRIMITIVE_TYPE is used (defined in the DMS section). Setting the TopicQosReliability field will also override the DeliveryProtocol. If the DirectMessagingAllowed field is set to "TRUE" in any of the active Distribution Groups, Specific Destination messages to/from SystemIDs via a System UUID topic name is enabled. Alternatively the *MA_TxDataPayloadCommandMT* wrapper can be used with message query filters applied where the entire *MA_TxDataPayloadCommandMT* message would be sent and receiving systems would have to filter for their System UUID from the *SpecificDestinationID* field.

The *RateGroup* field is used to define rate values that can be matched to multiple IERs. The *RateType* field should be set to "PERIODIC". The *TransmitFrequency* is applied to all messages inside of the connected IER(s) individually. Therefore it is recommended that any connected IER only contain a single message so that the rate is applied per message. The *TransmitFrequency* field is the target delta time between messages sent (e.g. 1 msg per second would mean a value of "PT1.0S" or 1 second).

The *Effectivity.IER_Assignment* field is used to match an *IER_ID* to *Endpoint_Assignment.DistributionGroupID*. If the *IER* is considered periodic, a *RateGroupID* can also

be set. The *Endpoint_Assignment.VectorID* can be used if the Plan is meant for specific point to point type configuration. The *Effectivity.ActivationTriggerID* field can also be used to make an Effectivity that is activated automatically given the specific Trigger.

If the Data Dissemination Plan needs update or is a new plan, MA sends the *MA_DataDisseminationPlanMT* message and the *MA_DataDisseminationPlanActivationCommandMT* message to MS. If the command is accepted by MS, MS sends the *MA_DataDisseminationPlanActivationCommandStatusMT_1* message to MA with the *CommandProcessingStateEnum* set to "ACCEPTED", the interaction "Update Data Dissemination Plan Activation Status" is initiated, and for each applicable Gateway Activity MS sends the *MA_GatewayActivityMT* message(s) to MA. If the command is NOT accepted, MS send the *MA_DataDisseminationPlanActivationCommandStatusMT_2* message to MA with the *CommandProcessingStateEnum* set to "REJECTED" and the *PlanActivationCommandStatus* is set according to the reason it was rejected.

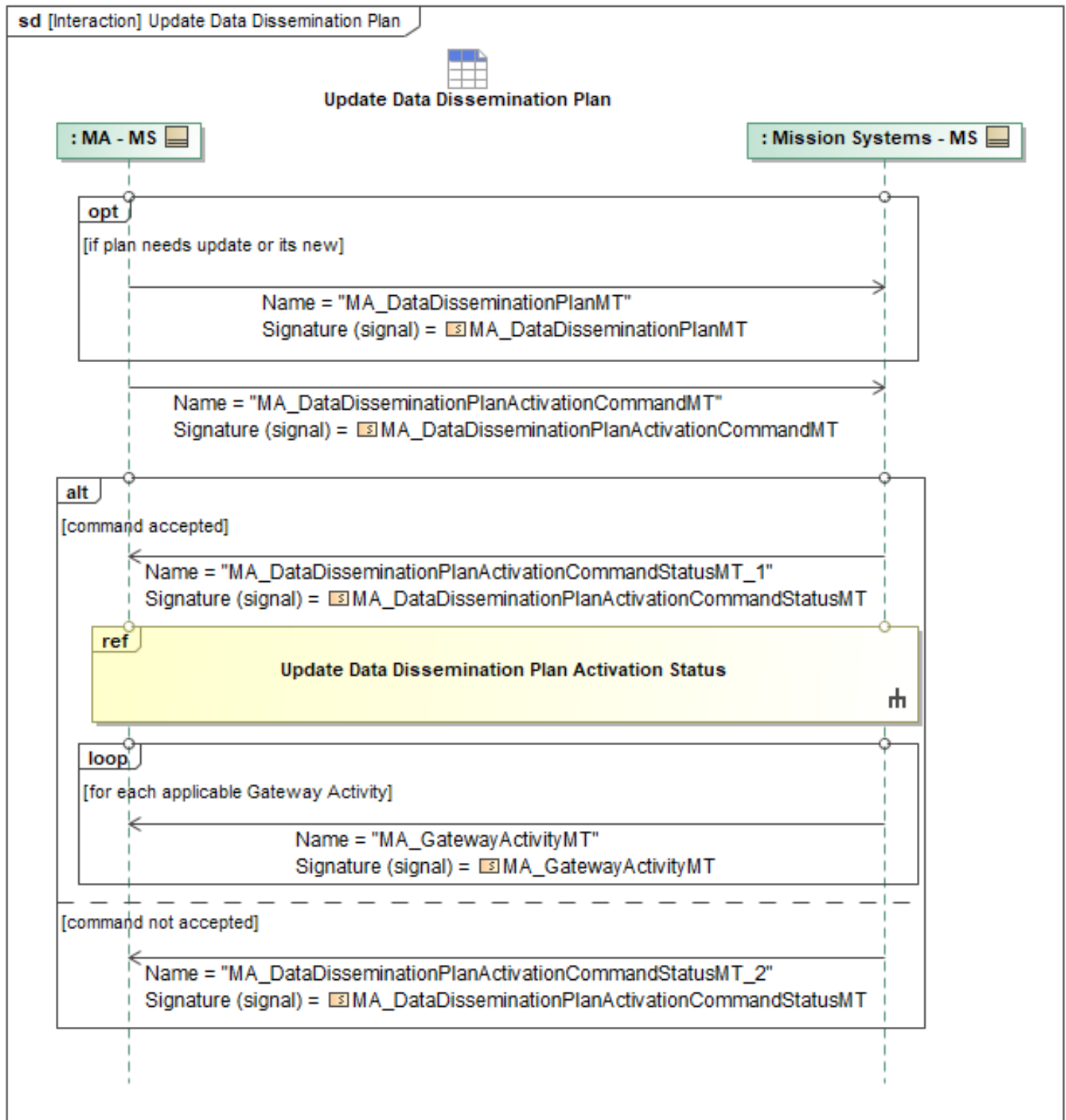


Figure 8-14: Update Data Dissemination Plan

Table 8-13: Update Data Dissemination Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanActivationCommandMT: MA_DataDisseminationPlanActivationCommandMT	Applicability: MA_PackageSystemType
MA_DataDisseminationPlanActivationCommandStatusMT_1: MA_DataDisseminationPlanActivationCommandStatusMT	None
MA_DataDisseminationPlanActivationCommandStatusMT_2: MA_DataDisseminationPlanActivationCommandStatusMT	RequestProcessingStateReason: CannotComplyType
MA_DataDisseminationPlanMT: MA_DataDisseminationPlanMT	None
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.1.14 Update Data Dissemination Plan Activation Status

Mission Use Case: Additional Comms Capabilities

The MS will send the *MA_DataDisseminationPlanActivationStatusMT* message to MA that will reflect the active state of Data Dissemination Plan. This can be sent periodically, but will always be sent upon change to the active state. The message will reflect the active Effectivities and Override Modifications of the Data Dissemination Plan.

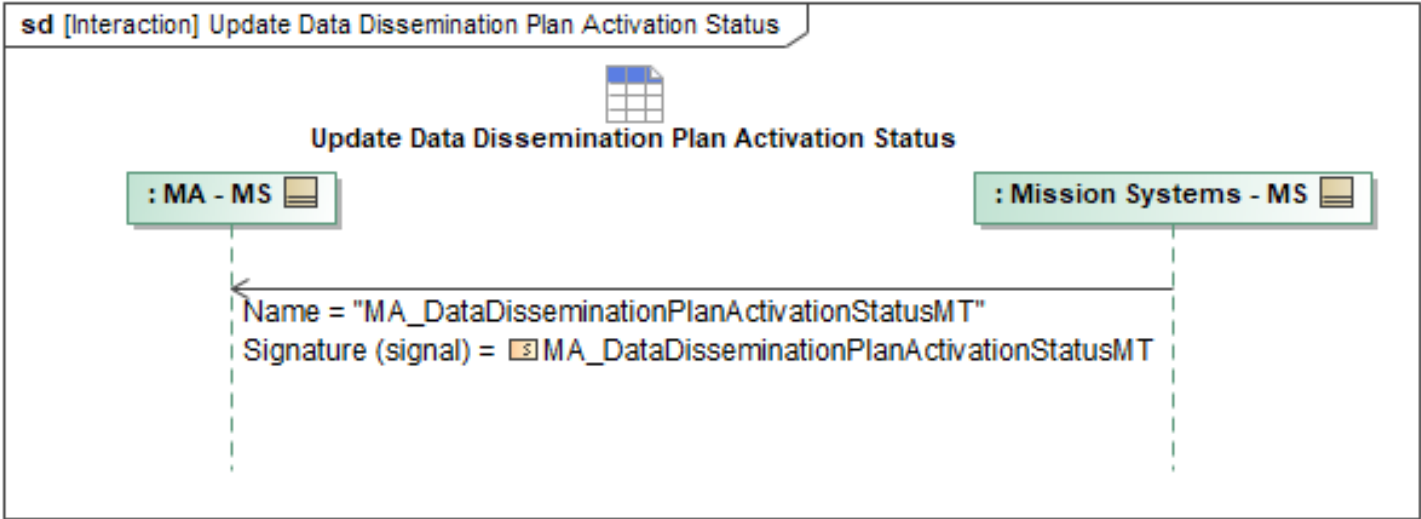


Figure 8-15: Update Data Dissemination Plan Activation Status

Table 8-14: Update Data Dissemination Plan Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanActivationStatusMT	None
: MA_DataDisseminationPlanActivationStatusMT	

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.2 Contingencies Package

8.2.2.1 Intra-Vehicle Comms Failure_MS

Mission Systems (MS) will provide regular status to Mission Autonomy (MA) via *ServiceStatusMT*. In the event MA doesn't receive a status from MS within the designated time frame, MA will request status from MS via the *ServiceStatusDataRequestMT*. MS will continue to attempt to report status via *ServiceStatusMT*. If communication between MA and MS is not restored, MA will refer to and follow the contingency plan associated with a loss of communication with MS.

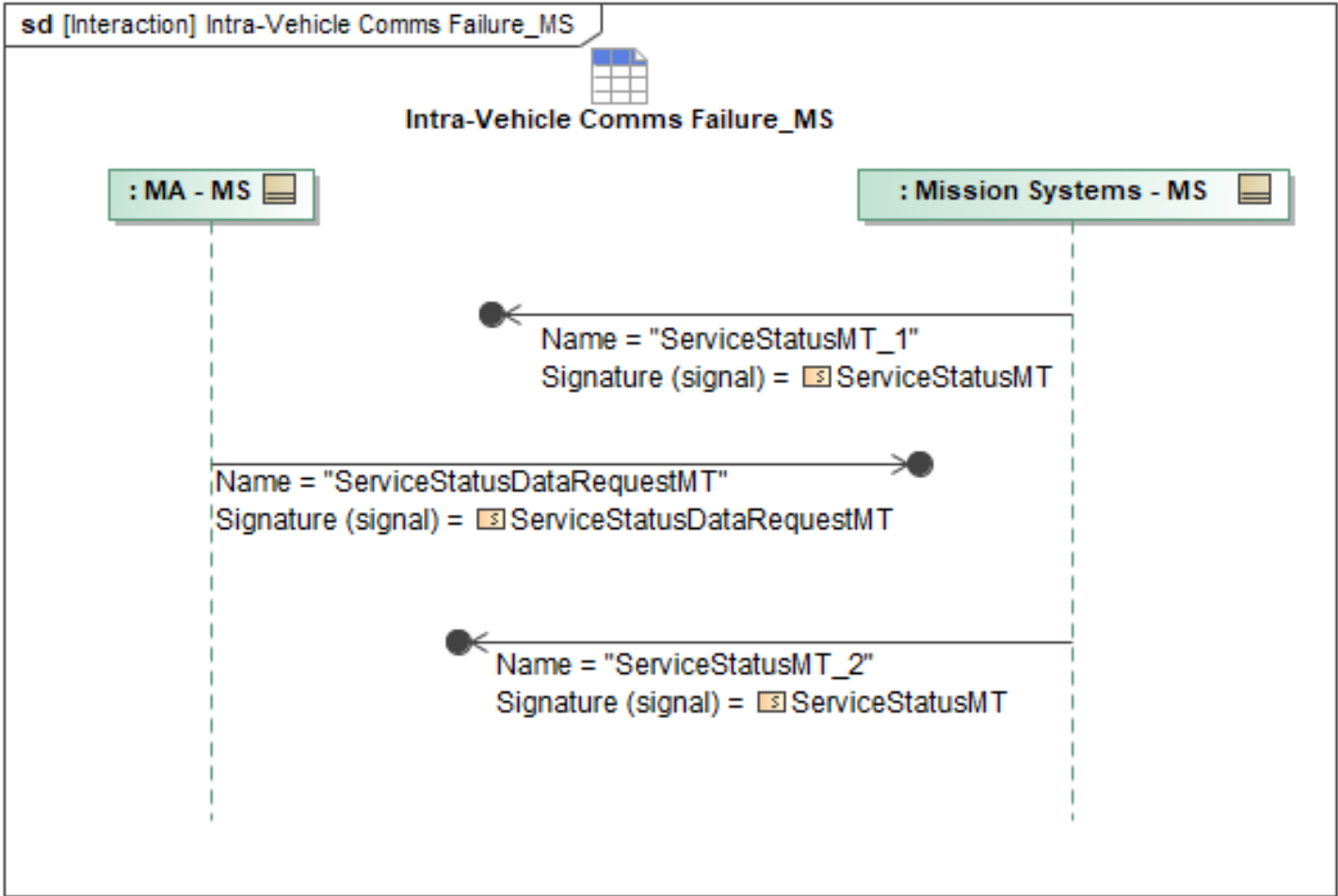


Figure 8-16: Intra-Vehicle Comms Failure_MS

Table 8-15: Intra-Vehicle Comms Failure_MS

Message Name (Name:Type)	Required Fields (Name:Type)
ServiceStatusDataRequestMT: ServiceStatusDataRequestMT	None
ServiceStatusMT_1: ServiceStatusMT	None
ServiceStatusMT_2: ServiceStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.2.2 Logging Failure

Mission Systems (MS) will report the status of the services it provides to Mission Autonomy (MA) via the *ServiceStatusMT* UCI message. In the event of a service failure, MS will report what service has failed after running a self initiated BIT. Status of the failed service will be reported via the *SubsystemBIT_StatusMT* UCI message.

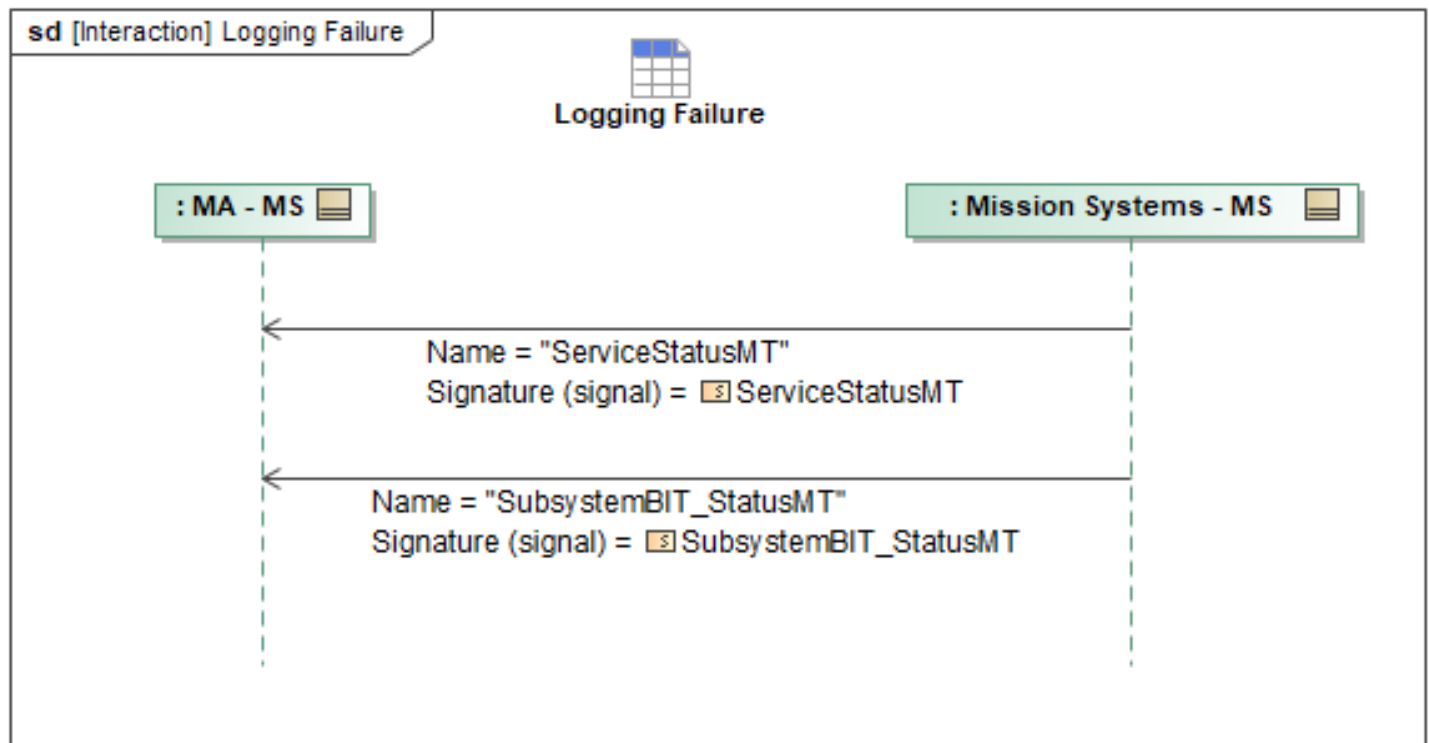


Figure 8-17: Logging Failure

Table 8-16: Logging Failure

Message Name (Name:Type)	Required Fields (Name:Type)
ServiceStatusMT: ServiceStatusMT	None
SubsystemBIT_StatusMT: SubsystemBIT_StatusMT	CompletedBIT: SubsystemCompletedBIT_Type FailReason: VisibleString256Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.2.3 MS Malformed Message Response

Upon receipt of a malformed message, MA will send *MA_SystemNotificationMT* to MS, indicating that the message could not be successfully processed. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with *INVALID_INPUT_PARAMETER*. *NotificationDetail.Description* will contain a human-readable string stating which fields are malformed. *AssociatedMessage* will reference the malformed message. The *MessageType* will be set to the same type as the malformed message. *AssociatedID* will be populated with a reference ID from the malformed message if applicable, the *UUID* of that message. If additional data is required to resolve the condition and the fault remains active, MA will issue *QueryDataRequestMT* with the *Query* field containing the ID of the specific message. MS will respond with *QueryDataRequestStatusMT*, specifically the *Message* field within the *Result* field will contain the corrected message containing the originally missing data. If the fault has been addressed, then MA will only send a new *MA_SystemNotificationMT* with the same fields as above and *NotificationState* set to CLOSED.

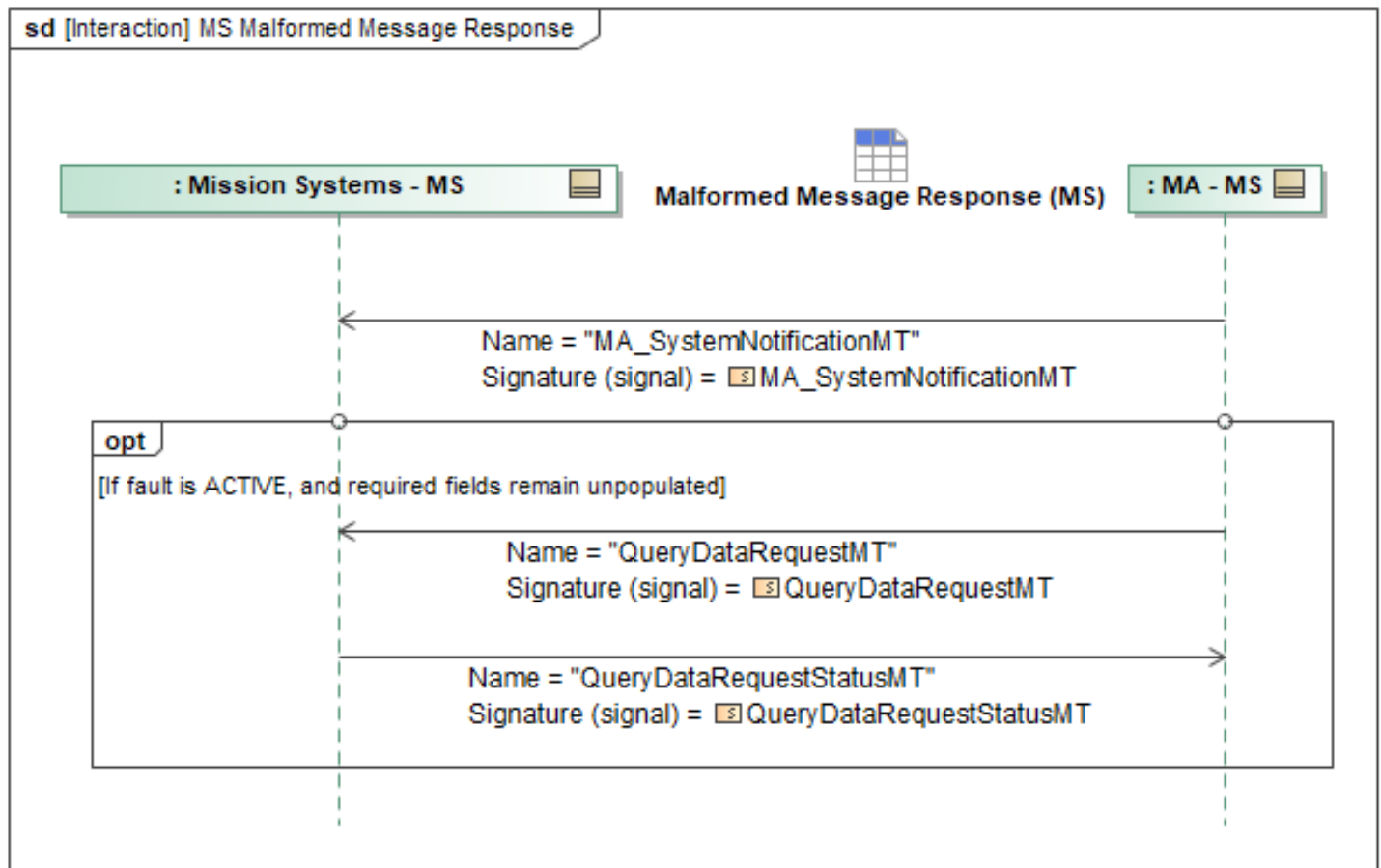


Figure 8-18: MS Malformed Message Response

Table 8-17: MS Malformed Message Response

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	Description: VisibleString1024Type AssociatedID: ID_Type
QueryDataRequestMT: QueryDataRequestMT	Query: QueryPET
QueryDataRequestStatusMT: QueryDataRequestStatusMT	Message: MessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.2.4 Payload Management Failure

MS will report status of the onboard payloads MS is responsible for via *SubsystemStatusMT*. In the event that a fault is detected, or there is a payload deployment failure, MS may optionally report the fault and necessary details to MA via the *MA_FaultMT* message, depending on whether MS supports this functionality. Faults are expected to be periodically reported by MS in the Receive Periodic Health, Status, & BIT Information sequence.

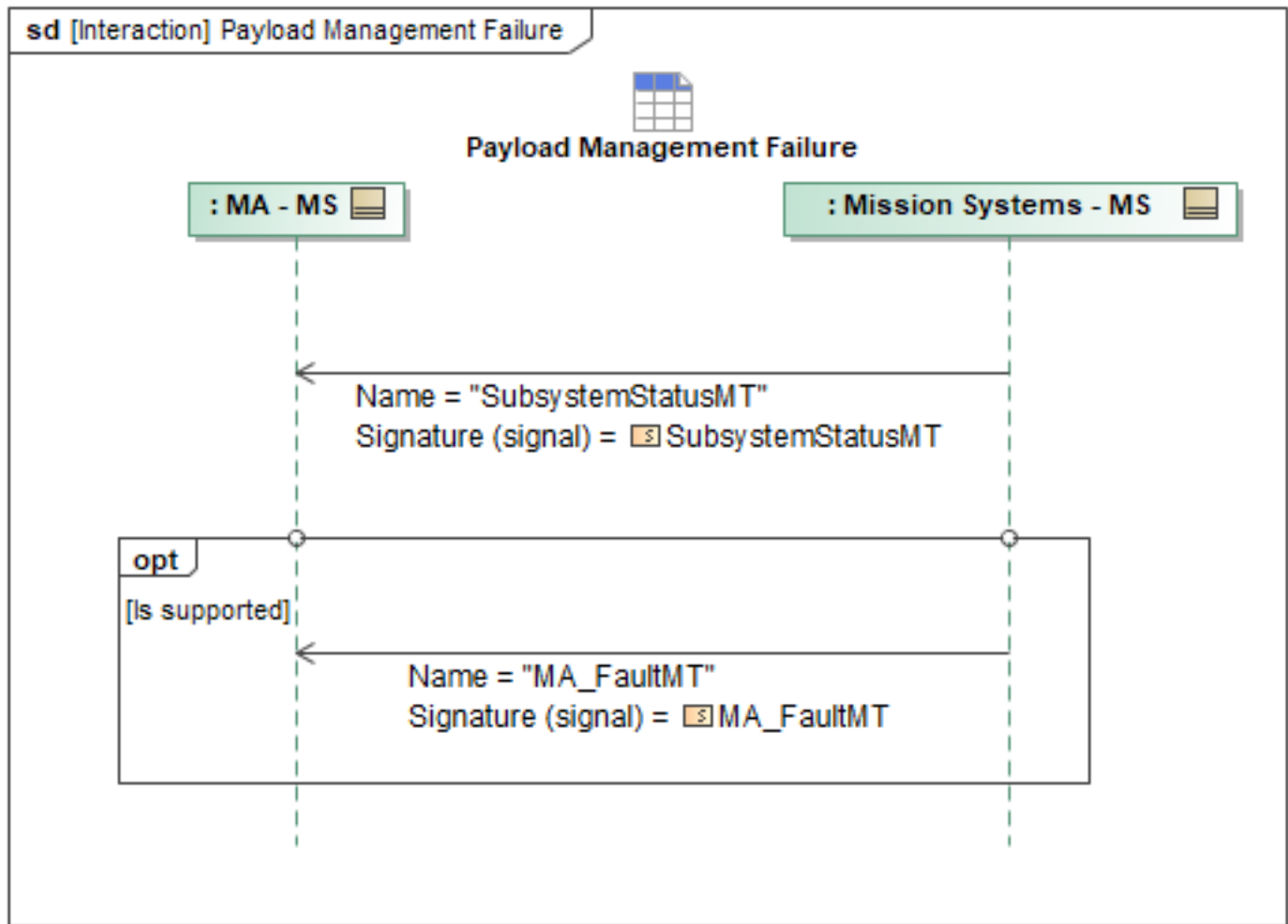


Figure 8-19: Payload Management Failure

Table 8-18: Payload Management Failure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	Severity: MA_FaultSeverityEnum FaultState: FaultStateEnum FaultDescription: VisibleString256Type
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.2.5 Sensor Failure

ServiceStatusMT is the UCI message that covers the status of all services for which MS is responsible. In the event a sensor subsystem fault is detected, MS may optionally report the fault and necessary fault detail to MA via the MA_FaultMT message, depending on whether MS supports this functionality. This reporting sequence is specific to sensor subsystem failures only. Faults are expected to be periodically reported in the Receive Periodic Health, Status, & BIT Information

sequence.

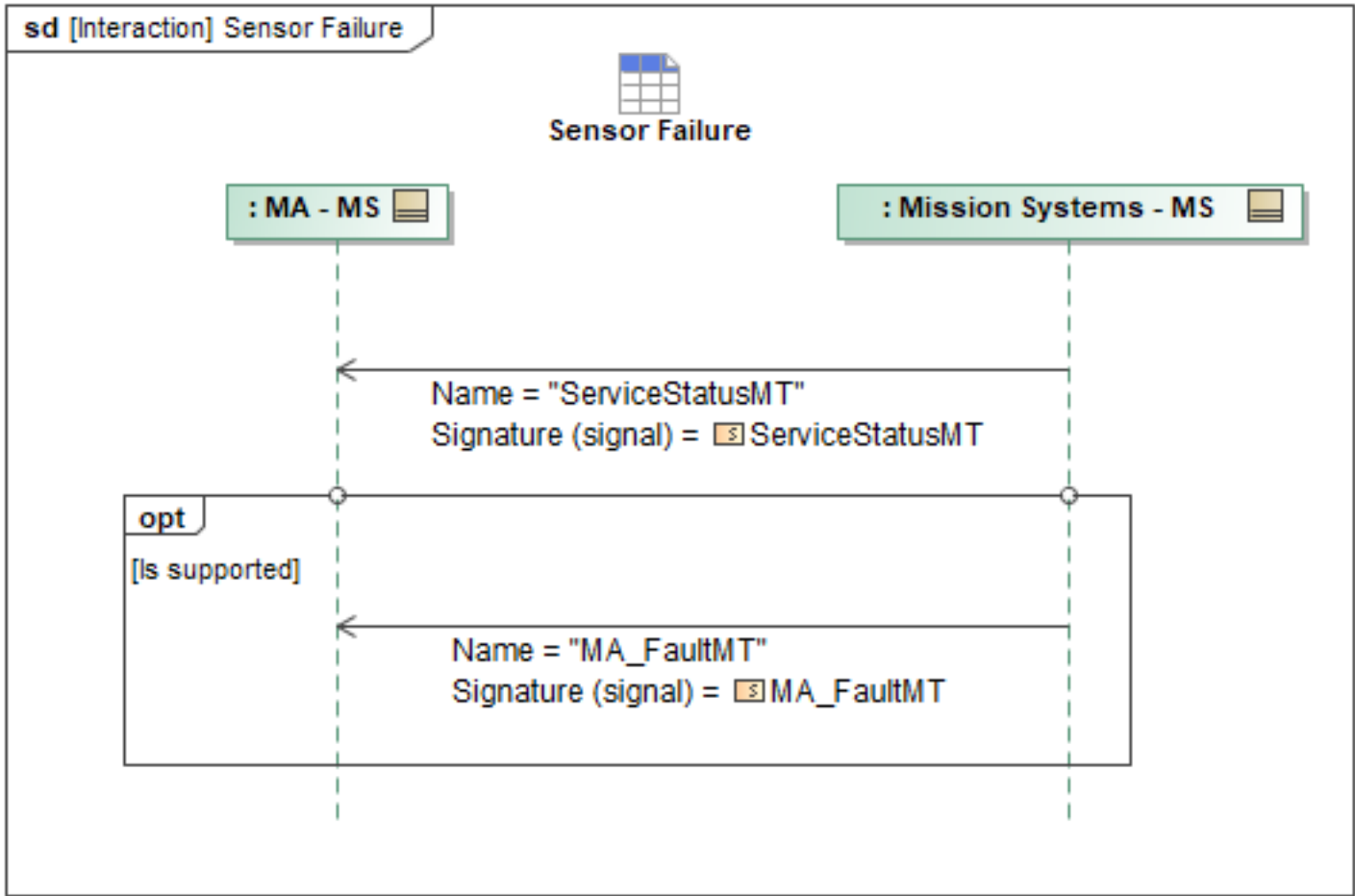


Figure 8-20: Sensor Failure

Table 8-19: Sensor Failure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	CapabilityID: CapabilityID_Type ServiceID: ServiceID_Type AmbiguityGroup: SubsystemFaultAmbiguityGroupType SubsystemID: SubsystemID_Type SupportCapabilityID: SupportCapabilityID_Type ComponentID: ComponentID_Type
ServiceStatusMT: ServiceStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.3 COP Package

8.2.3.1 MS Comms Status Updates to COP

The MS Comms Updates to COP sequence diagram specifies the interaction of the MS sending updates of the comms state to MA.

At a predefined period, the *SubsystemStatusMT* is sent from MS to MA. The *SubsystemStatusMT* must have the *SubsystemID* of the comms subsystem and *SubsystemState* set to the state of the comms subsystem and its links.

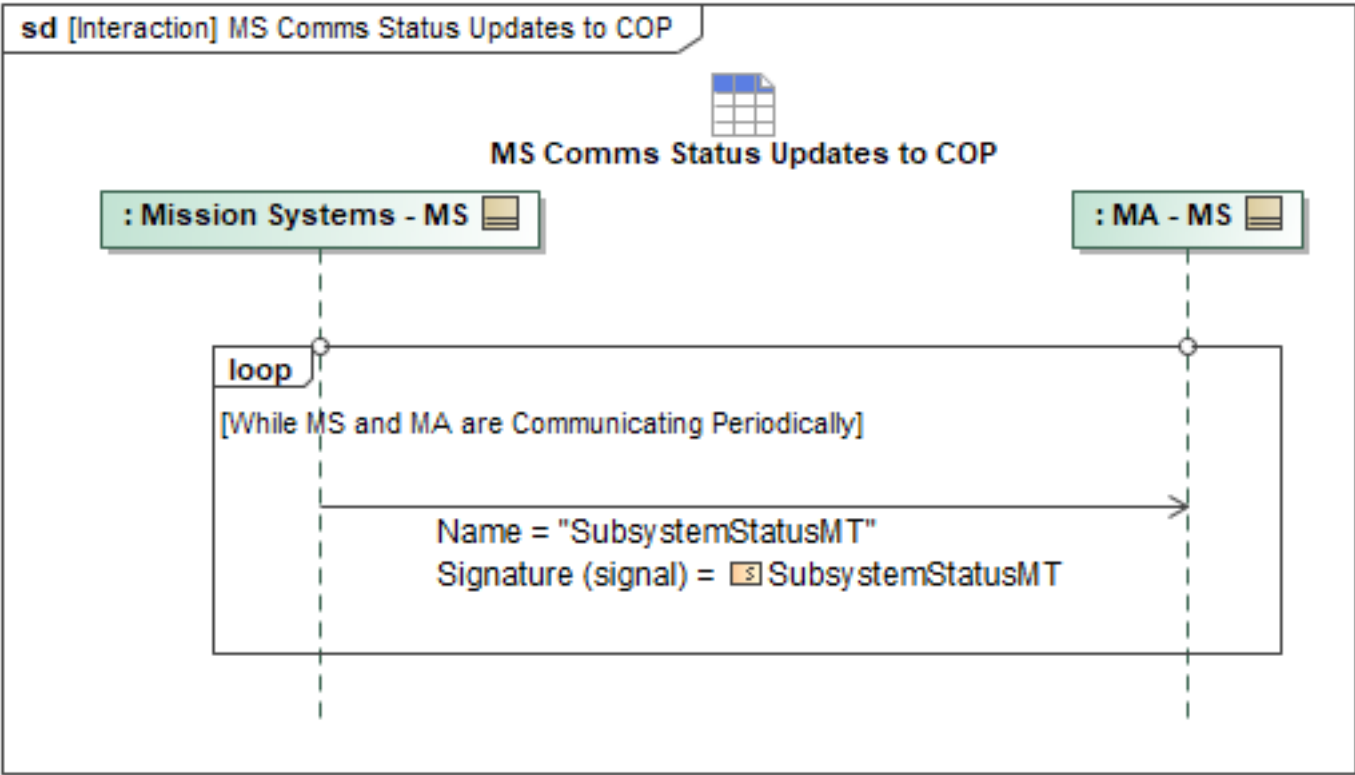


Figure 8-21: MS Comms Status Updates to COP

Table 8-20: MS Comms Status Updates to COP

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.3.2 MS Sensor AMTI Status Updates to COP

Mission Use Case: AMTI

The MS AMTI Sensor Status Updates to COP sequence diagram specifies the interaction of MS communicating the status of the sensor Airborne Moving Target Indicator (AMTI) sensor capability to MA for common operating picture (COP) reporting purposes. This sequence is only present if AMTI sensing capabilities exist in MS. MS publishes *MA_AMTI_CapabilityMT* with at least one *Capability*

specified. MS periodically publishes *AMTI_CapabilityStatusMT* and *AMTI_ActivityMT* to MA to report status and usage of the AMTI capability. COP uses *MA_AMTI_CapabilityMT* and *AMTI_CapabilityStatusMT* to know if the AMTI sensor capability is available to command. COP uses *AMTI_ActivityMT* to know if the AMTI capability can effectively be used (e.g. not fully scheduled). Requirements put upon *MA_AMTI_CapabilityMT*, *AMTI_CapabilityStatusMT*, and *AMTI_ActivityMT* are set by other sequences requiring use-case specific functionality (e.g. “Advertise Sensor AMTI Capability for Search and Track”, “Report Sensor AMTI Capability State”), for COP purposes these messages are received and aggregated into general status.

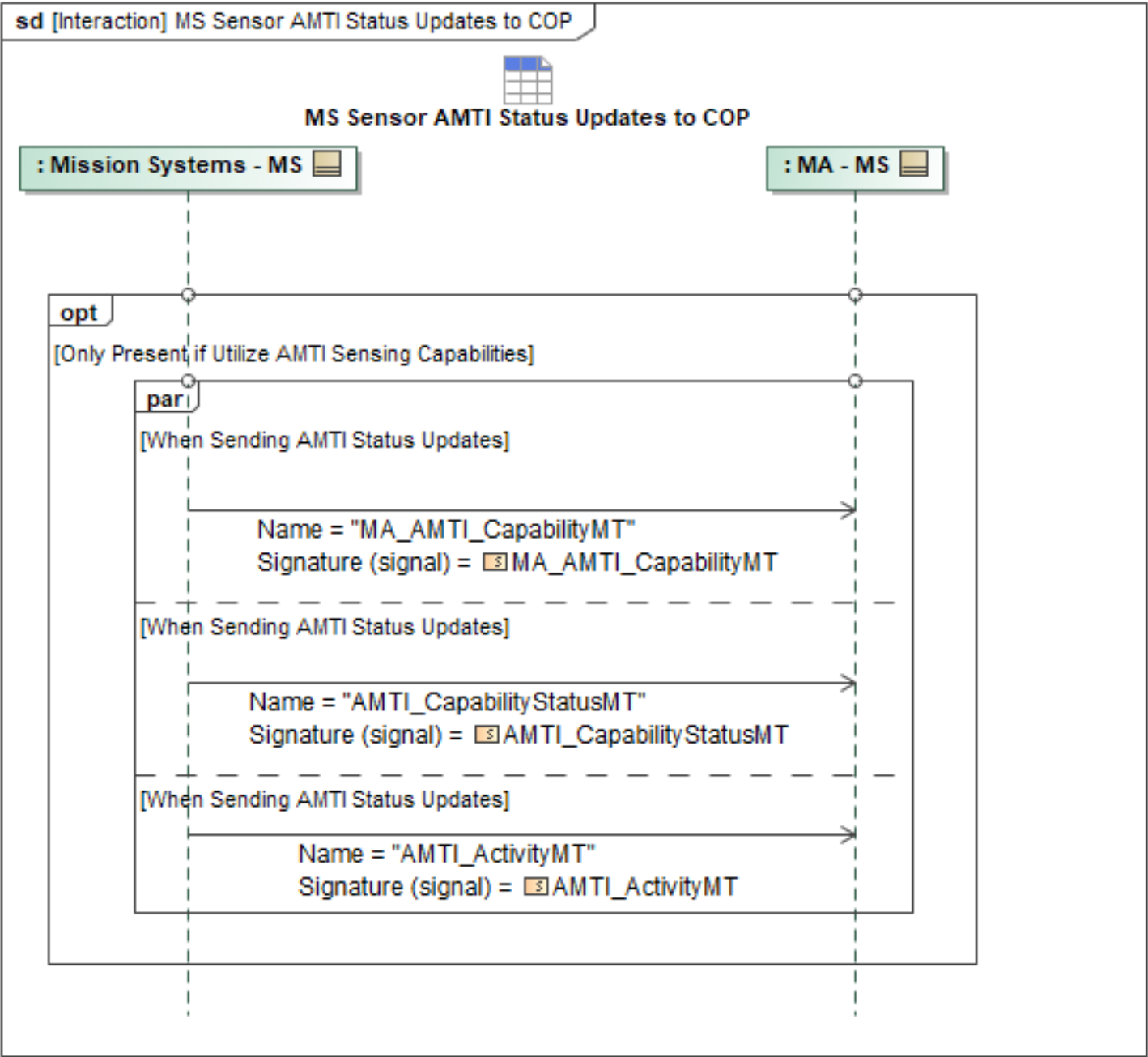


Figure 8-22: MS Sensor AMTI Status Updates to COP

Table 8-21: MS Sensor AMTI Status Updates to COP

Message Name (Name:Type)	Required Fields (Name:Type)
AMTI_ActivityMT: AMTI_ActivityMT	None
AMTI_CapabilityStatusMT: AMTI_CapabilityStatusMT	None
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.3.3 MS Sensor PO Status Updates to COP

Mission Use Case: PO

The MS PO Sensor Status Updates to COP sequence diagram specifies the interaction of MS communicating the status of the sensor Passive Optical (PO) sensor capability to MA for common operating picture (COP) reporting purposes. This sequence is only present if PO sensing capabilities exist in MS. MS publishes *PO_CapabilityMT* with at least one *Capability* specified. MS periodically publishes *PO_CapabilityStatusMT* and *PO_ActivityMT* to MA to report status and usage of the PO capability. COP uses *PO_CapabilityMT* and *PO_CapabilityStatusMT* to know if the PO sensor capability is available to command. COP uses *PO_ActivityMT* to know if the PO capability can effectively be used (e.g. not fully scheduled). Requirements put upon *PO_CapabilityMT*, *PO_CapabilityStatusMT*, and *PO_ActivityMT* are set by other sequences requiring use-case specific functionality (e.g. “Advertise Sensor PO Capability for Search and Track”, “Report Sensor PO Capability State”), for COP purposes these messages are received and aggregated into general status.

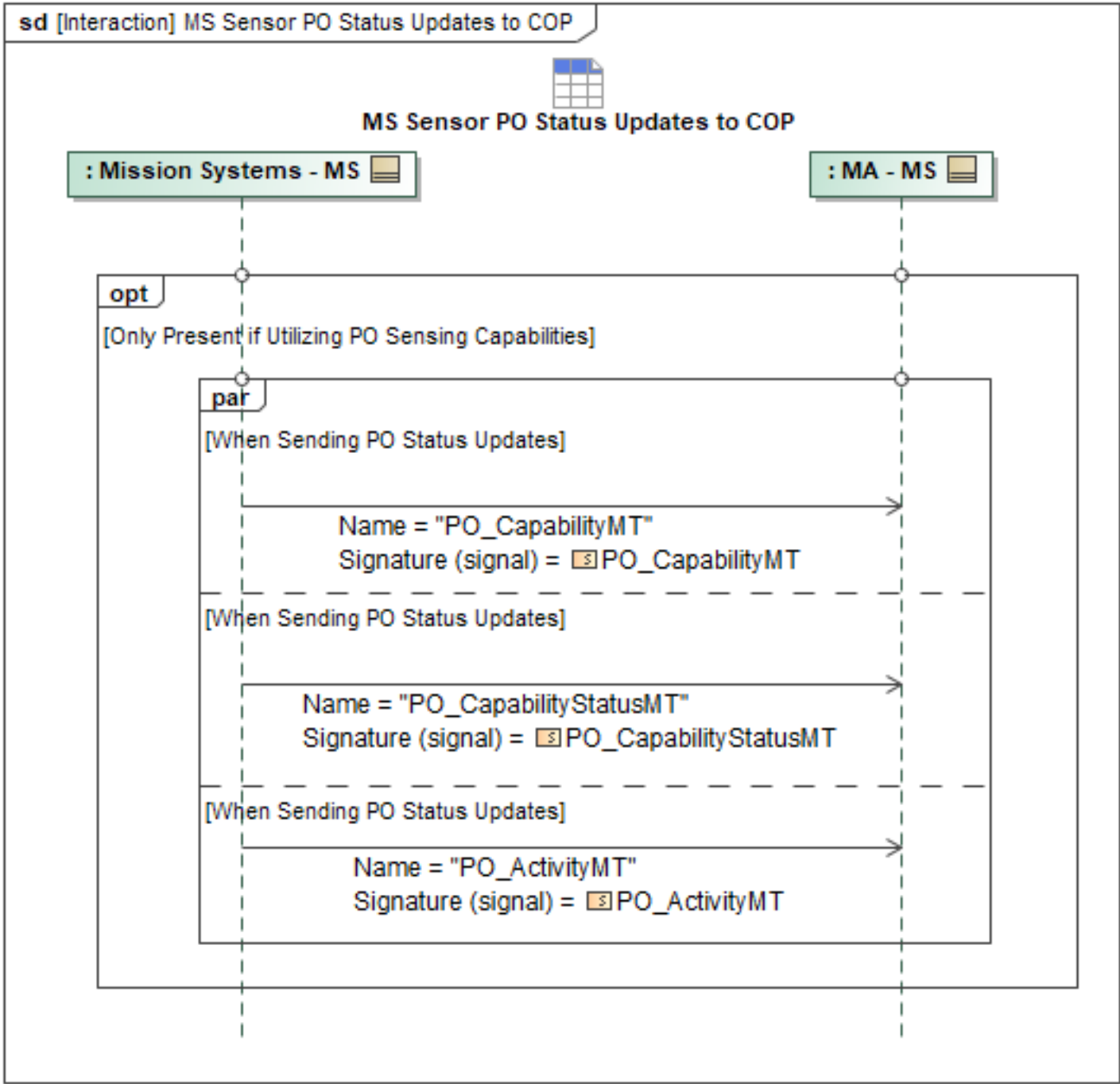


Figure 8-23: MS Sensor PO Status Updates to COP

Table 8-22: MS Sensor PO Status Updates to COP

Message Name (Name:Type)	Required Fields (Name:Type)
PO_ActivityMT: PO_ActivityMT	None
PO_CapabilityMT: PO_CapabilityMT	None
PO_CapabilityStatusMT: PO_CapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.3.4 MS Weapon Status Updates to COP

Mission Use Case: Store Management

The MS Weapon Status Updates to COP sequence diagram specifies the interaction of Mission Systems (MS) communicating the status of the weapons subsystem to Mission Autonomy (MA) for common operating picture (COP) reporting purposes.

MS publishes *StoreManagementStatusMT* messages to report the status of individual weapons and allow MA to know weapon quantities and individual statuses. This is complementary to general usage status provided by *MA_StrikeCapabilityMT* and *StrikeCapabilityStatusMT*. The *SupportCapabilityID* field specifies a unique identifier used by other messaging (not in scope) to manipulate store settings. Each *StoreStation* has a *StationIndex* to identify station using MS implementation specific schemes. The *CompatibleStore* field specifies the type of weapons that station can hold. The *InstalledStore* field is optional but required when the station currently holds a weapon. When specified, the *InstalledStore* field identifies the weapon type. If the *InstalledStore* field and sub-fields are omitted, it is interpreted that this station does not hold a weapon. Each station is associated with a *Capability* defined by *MA_StrikeCapabilityMT* by specifying the *CapabilityID* field, which is required.

MS publishes *MA_StrikeCapabilityMT* messages with *Capability* required. Each *Capability* has its own unique identifier set in the *CapabilityID*. The *AcceptedInterface* field must include at least *CAPABILITY_COMMAND*. The *StoreType* and *StoreCategory* fields are used to identify the type of weapons the *Capability* is used to status and command. MS implementation determines how the *InterruptOtherActivities* field is set. Other schema-optional fields are not required but may be specified for more information.

MS publishes *StrikeCapabilityStatusMT* messages to report status of the weapon capabilities identified by the *Capability* fields in the *MA_StrikeCapabilityMT* message. Each *CapabilityStatus* has the *CapabilityID* set to match the *CapabilityID* field of the corresponding *Capability*. The *Availability* field is used to report general usability of the weapons *Capability* by MA. The *StoreStatus* field is required with *StoreState* and *StoreStatusDetail* communicating the specific state information of the weapon(s) associated with the *Capability*. Other schema-optional fields are not required but may be specified for more information.

MS publishes *MA_StrikeActivityMT* to communicate more information about the current employment of weapons to MA. Each *Activity* specifies a unique identifier in the *ActivityID* field, corresponding to a scheduled weapon employment. The *CapabilityID* field associates the employment with a specific *Capability* defined in the *MA_StrikeCapabilityMT* message. *Interactive* is set according to MS implementation. The *ActivityState* field is used to specify the state of the weapon employment activity. The *SelectedForKeyLoad*, *WeaponArmed*, *ReadyForRelease*, *SelectedForRelease*, and *SelectedForJettison* fields provide information about the current state of the weapon while it prepares for the weapon employment activity. The *AssignedTarget* field is required when the weapon is scheduled to be launched at a target and not jettisoned. The *TimeToRelease* field is required and specifies the timing of the weapon release. Optionally, MS can provide MA with the *MunitionActive* field that pertains to the specific weapon state data and orientation during weapon flyout. Active weapon states include *TARGETED*, *REQUESTED*, *SEEKER_INACTIVE*, *SEEKER_ACTIVE*, *FUZE_INACTIVE*, *FUZE_ACTIVE*, *NOT_FULLY_AUTONOMOUS*, *FULLY_AUTONOMOUS*,

DEFEATED, TIMED_OUT, SUCCESS.

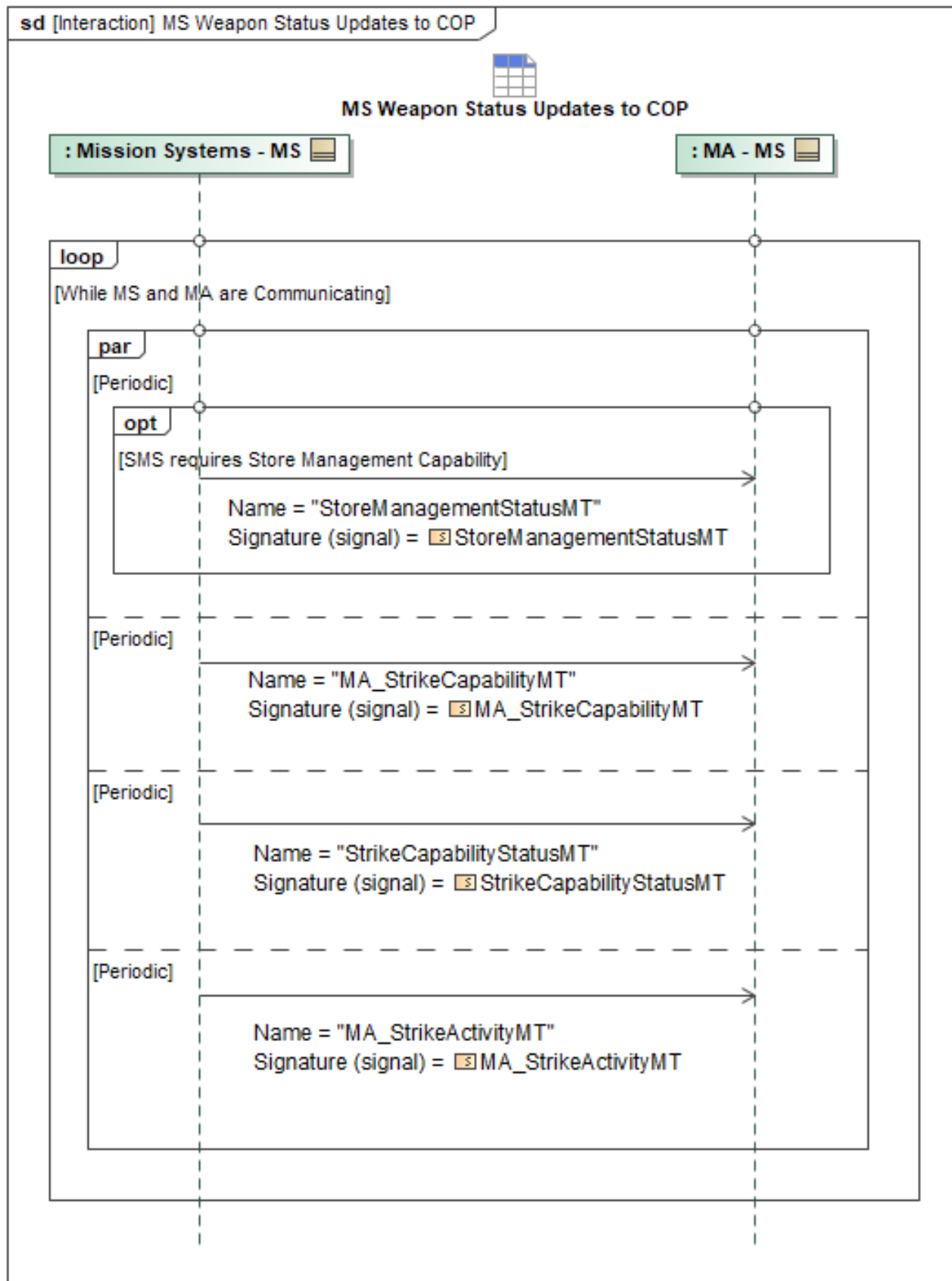


Figure 8-24: MS Weapon Status Updates to COP

Table 8-23: MS Weapon Status Updates to COP

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeActivityMT: MA_StrikeActivityMT	TimeToRelease: DurationType
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None
StoreManagementStatusMT: StoreManagementStatusMT	CapabilityID: CapabilityID_Type InstalledStore: StoreType
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	StoreStatus: StoreStatusType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.4 Data Fusion Package

8.2.4.1 Distribute Single Sensor Track Data to Perform Local Fusion

This interaction describes how MS provides tracks from onboard sensors to MA to enable local fusion. This is accomplished via the *EntityMT* message, which contains the kinematic data of the track. Additionally, the *ObservationMeasurementReportMT* message contains the sensor data that is used to generate the track or entity. Track data sources include both onboard cooperative sensors (e.g., Mode 5 IFF, ADS-B) and onboard non-cooperative sensors. It is important to note that only a subset of the total Observation Measurement Reports (OMRs) recorded are associated with a given track, and can therefore be referred to as Associated Measurement Reports (AMRs). These associated OMRs are identified in the *EntityMT* by the *MeasurementID* field. Once fusion has taken place, the resultant fused tracks can then be utilized by and integrated into the Common Operating Picture (COP). Alternately, if a platform is incapable of performing local fusion, the pre-fused tracks may still be used by COP.

Additionally, a kinematic and Combat ID (CID) fusion implementation will likely want the capability that OMRs were generated from, and which system/subsystem had the capability generating the OMR.

Within the *ObservationMeasurementReportMT* message, there are two optional paths to different sets of *SubsystemIDs* and *SystemIDs*.

1) *ObservationMeasurementReportMDT* → *ObservationMeasurement* → *MeasurementSource* → *ElementDetails* → *SourceElementIdentifier* → *SystemID* + *SubsystemID*

- These messages provide the system/subsystem that's the source of the OMR's kinematics.

2) *ObservationMeasurementReportMDT* → *ObservationMeasurement* → *MeasurementSource* → *SourceIdentity* → *CapabilityID* + *SystemID* + *SubsystemID*

- These messages provide the system/subsystem and capability for the source of identity in the OMR (it's expected this will usually be the same system/subsystem as the kinematics). A client would want to know the capability type of the provided entity ID, and may have to subscribe to additional messages to understand the type (e.g. [Capability]StatusMT messages).

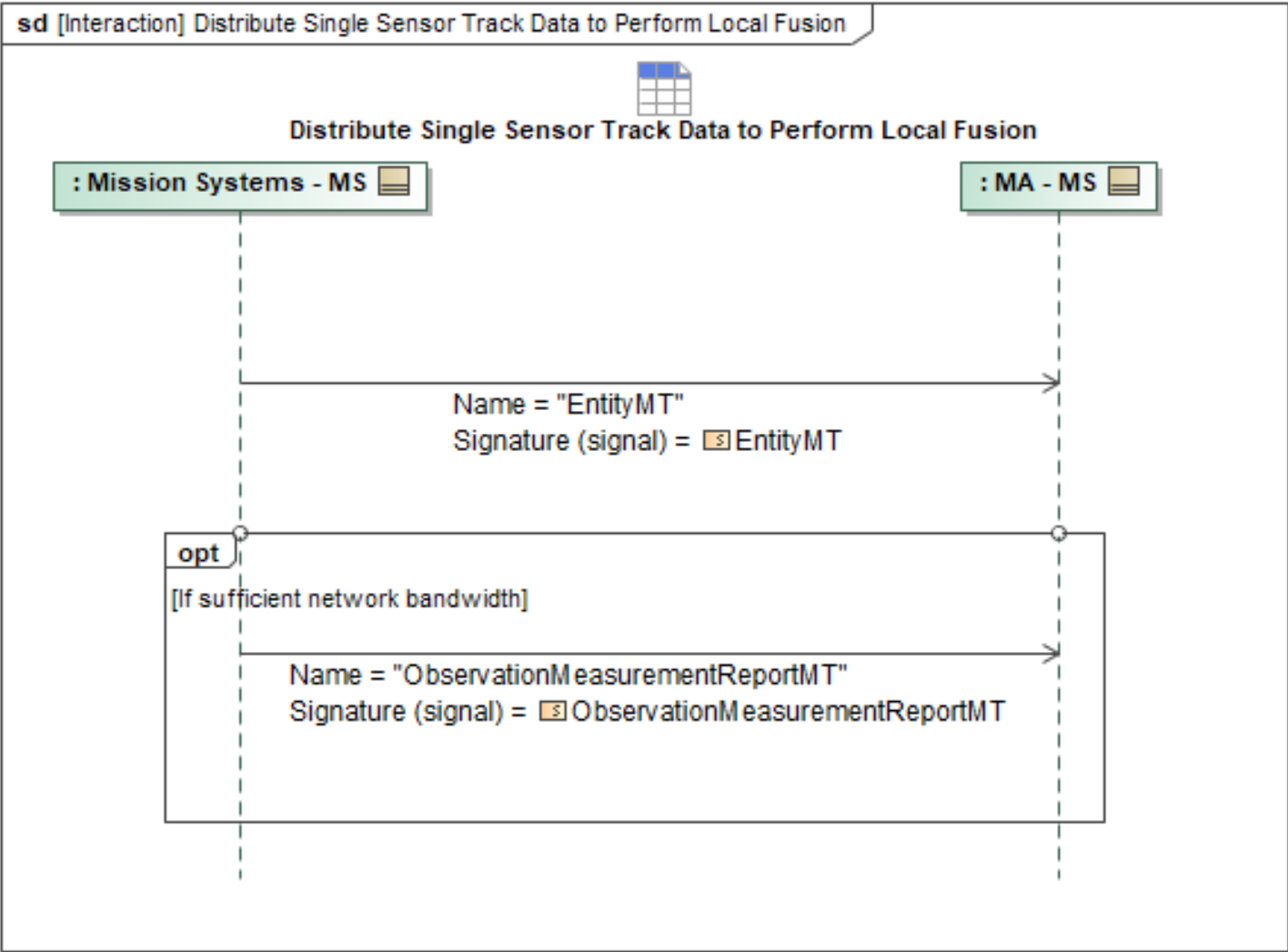


Figure 8-25: Distribute Single Sensor Track Data to Perform Local Fusion

Table 8-24: Distribute Single Sensor Track Data to Perform Local Fusion

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	None
ObservationMeasurementReportMT: ObservationMeasurementReportMT	MeasurementSource: ObservationMeasurementSourceType ElementDetails: ElementDetailsType SourceElementIdentifier: SourceKinematicsElementIdentityType SystemID: SystemID_Type SubsystemID: SubsystemID_Type SourceIdentity: MeasurementSourceIdentityType CapabilityID: CapabilityID_Type SystemID: SystemID_Type SubsystemID: SubsystemID_Type MeasurementKinematics: MeasurementKinematicsChoiceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.4.2 Receive Processed Track

The Receive Processed Track sequence diagram describes how MA receives track information from an onboard data fusion service. The basic functionality of a data fusion service is to process and merge target reports from various internal and external sources to provide a resulting track, called *Entity*. Fields that assist MA on decision making include *Source*, *EntityStatus*, *Identity*, *Kinematics*, and *EstimatedKinematics*. This information, processed from 2D unfused tracks or 3D tracks (cues) via *EntityMT*, not only provides MA a picture of the battlespace, but also includes information about an entity's kinematic characteristics to support a variety of mission objectives, such as following an Entity. This sequence shows a loop of *EntityMT* messages as long as there is more than one *Entity* to convey to MA. This message is also triggered when new tracks are determined, or there are changes to already established tracks.

Losing a track is a special optional case that happens when a sensor can no longer detect and track an entity; when this occurs, the *EntityLostMT* message will notify Mission Autonomy of this event.

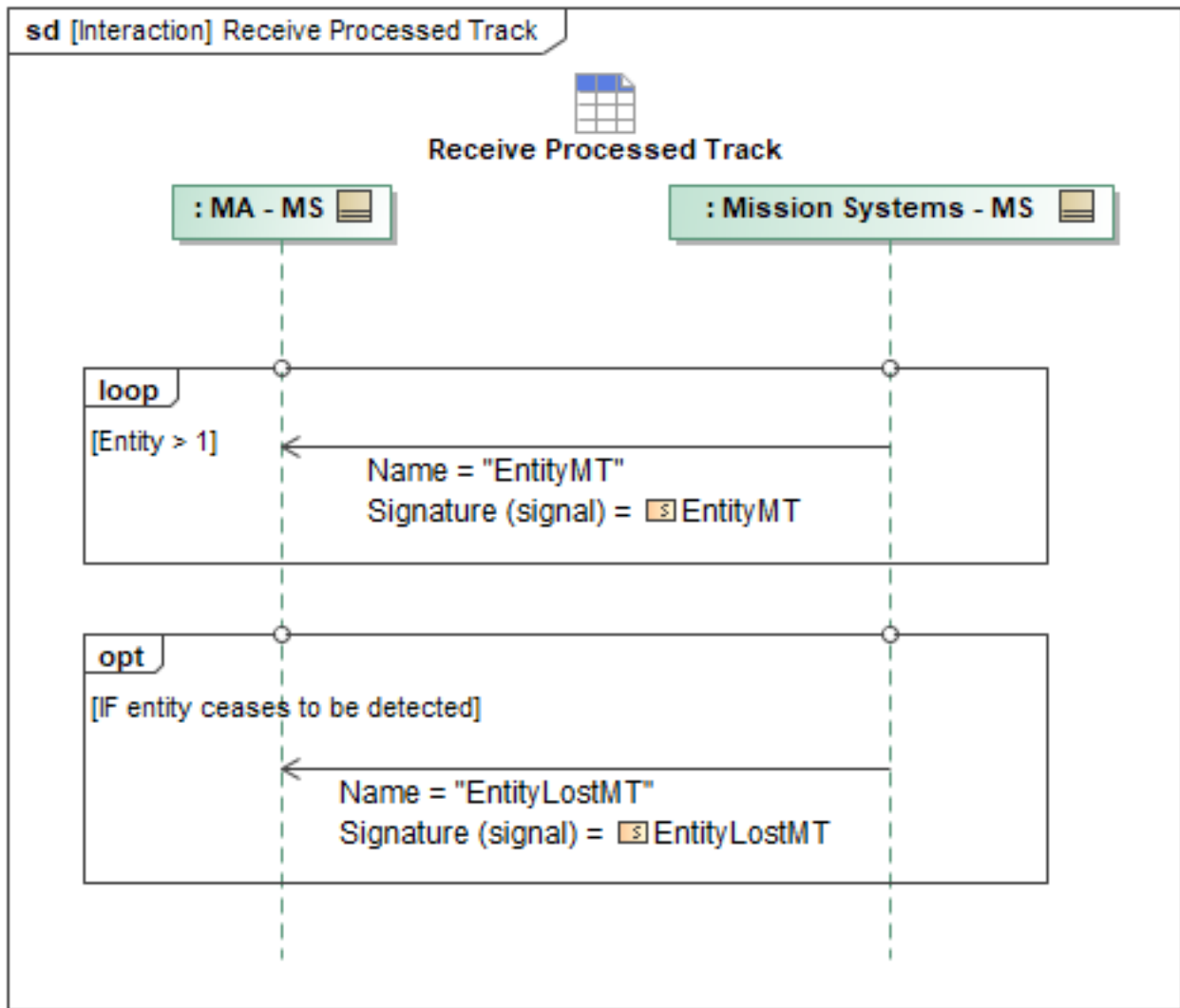


Figure 8-26: Receive Processed Track

Table 8-25: Receive Processed Track

Message Name (Name:Type)	Required Fields (Name:Type)
EntityLostMT: EntityLostMT	None
EntityMT: EntityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.5 Decentralized Messaging Service Package

The Decentralized Messaging Service (DMS) Interface is a communications Gateway Capability implementation that provides an interface for MA to ingress/egress UCI messages offbaord. This interface facilitates the passing of UCI messages to/from C2 and between ACP's over an IP network using the Data Distribution Service (DDS) standard. In the case when a non-IP or link-specific encoding must be used, such as Link 16, that encoding will be specified separately from the general offbaord message format. The core Minimum Message Set (MMS) accounted for in the DMS consists

of the messages required to transmit data payloads off-board (*MA_TxDataPayloadCommandMT*, *MA_TxDataPayloadCommandStatusMT*), a message to pass a received message from MS to MA (*MA_RxDataPayload*), a set of messages for MA to indicate desired subscriptions to MS (*MA_MessagingSubscriptionSettingCommandMT*, *MA_MessagingSubscriptionSettingCommandStatusMT*) and a message for MS to send existing subscription information to MA (*MA_MessagingSubscriptionStatusMT*). MA can populate a *MA_TxDataPayloadCommandMT* with a UCI message payload, and optionally a list of network endpoints by *SystemID* to send to MS. In order to send the message off-board, MS will use the DMS to send the message OTA to another system's DMS. The receiving system's DMS is responsible for receiving the message, formatting it as a *MA_RxDataPayloadMT* message, and forwarding the message to the appropriate interface for decoding and processing. The *MA_TxDataPayloadCommandMT* and *MA_RxDataPayloadMT* UCI wrapper messages require an *EncodedPayload* field which will have a datatype while crossing the L1 boundary between MA and MS is UTF-8 XML represented as hexBinary. The OTA wrapper message sent between the DMSs, *MA_DataPayloadWrapper* IDL, also requires an *EncodedPayload* field. The *MA_DataPayloadWrapper's* *EncodedPayload* field datatype while crossing between DMSs will be in EXI representation. The "Serialization and Data Encoding" section describes the EXI representation in detail.

With the "Additional Comms Capability" MUC, a Gateway can be configured to convert any message on the ASB to be transmitted offboard by either subscribing to it directly or by using a UCI wrapper (*MA_TxDataPayloadCommandMT* with the corresponding status message *MA_TxDataPayloadCommandStatusMT*). A Gateway can be configured, by the *MA_DataDisseminationPlanMT* or by a direct *MA_GatewayCommandMT*, to receive any message from the offboard network and either publish it out directly or use a UCI wrapper (*MA_RxDataPayloadMT*). In either case the UCI message header will contain the source information from the sender of the message for authentication and other processing. The configuration of ingress/receive is typically done through the *MA_DataDisseminationPlanMT* with associated activated *Effectivities*, *IERs*, and *DistributionGroups*, but can also be done by using the *MA_MessagingSubscriptionSettingCommandMT* and associated status messages *MA_MessagingSubscriptionSettingCommandStatusMT* and *MA_MessagingSubscriptionStatusMT*. These and the *MA_GatewayCommandMT* provides a way to directly configure the DMS, but the *MA_DataDisseminationPlanMT* allows MS to manage and update all the Gateways as needed based on the plan. The *MA_DataDisseminationPlanMT* has an optional *IER_DesiredService.CompressionType* field that can be set to "EXI" to defines the encoding/compression format for the *EncodedPayload* field in the OTA *MA_DataPayloadWrapper* IDL that is used for communication between DMSs.

In order to successfully send data between instances of DMS via the OTA network, the Data Distribution Service (DDS) standard shall be used as the middleware. DDS provides a scalable, real-time, and decentralized publish-subscribe communication model well-suited to dynamic and distributed environments. It allows systems to exchange structured data via shared topics in a flexible deployment, resilient to node failure, and provides easier integration.

All implementations of the DMS must conform to the DDS specification (see <https://www.omg.org/spec/DDS/1.4/>) and be compatible with the Real-Time Publish-Subscribe(RTPS) wire protocol, minimum version 2.3. While a specific DDS implementation is not mandated, all system components must comply with the interface and behavioral expectations described herein to ensure interoperability. This section covers discovery mechanisms, topic definitions, quality of service

requirements, serialization and data encoding, and security features.

To enable interoperability, it is necessary to standardize several aspects of the DDS configuration. First, two types of DDS topics will be used:

1. Pub/sub topics, by default, are named according to the UCI message type that will be sent via those topics. Examples include "SystemStatus" or "ActionCommand". Subscriptions to these topics can be managed via the "Subscribe to Message Topic" interaction. The "Additional Comms Capability" MUC uses the "Update Data Dissemination Plan" interaction to manage subscripsts and publications.
2. Specific destination topics shall be used to send messages according to *MA_TxDataPayloadCommandMT.SpecificDestinationID* fields. The DMS shall subscribe to a DDS topic that is equal to the UUID of its own SystemID. Since DDS uses pub/sub rather than point-to-point semantics, these topics will be used to send OTA messages to systems listed in *SpecificDestination* fields of the *MA_TxDataPayloadCommand*. These topics will use RELIABLE QoS settings. Topic names that correspond to SystemIDs will be prefixed with "id_", use all uppercase letters, and use "_" instead of "-" for separators to comply with DDS Topic naming rules. A SystemID of "3b39Bb83ee-d994-5f64-ae73-e62df8ec3c72" becomes a topic name of "id_B39B83EE_D994_5F64_AE73_E62DF8EC3C72". When using the "Additional Comms Capability" MUC, UCI wrapped messaging and Direct Messaging can be enabled through a configurations option.

DMS implementations shall perform content filtering to ensure that duplicate messages are not publish to MA when a platform receives the same message via a topic subscription and via UCI wrapped *SpecificDestination* routing. This will be accomplished by requiring DDS Readers for normal topics (those other than the *SpecificDestination* topics described in Item 2 above) to create a content filter which excludes any message that is also sent to that system via *SpecificDestination*. Since the message will be delivered via *SpecificDestination* topic it will use RELIABLE QoS rather than whatever QoS are used by the normal topic. This may be implemented via DDS content filters which can accomplish content filtering at the DataWriter to save bandwidth or implementations may implement this feature via other means within the DMS.

To facilitate use of the *MA_TxDataPayloadCommandStatusMT.TransmissionStatus* field, every DDS DomainParticipant, DataWriter, and DataReader shall set its UserQoS data to be a vector of bytes equal to its host system's SystemID.UUID.

Disclaimer: If the "Additional Comms Capability" MUC is not implemented, the DMS can function as a stand alone service without the need of the Data Dissemination Plan sequences. Any references to the *MA_DataDisseminationPlanMT* in this section can be ignored. This means the Minimum Message Set (MMS) accounted for in the DMS consists of the messages required to transmit data payloads off-board (*MA_TxDataPayloadCommandMT*, *MA_TxDataPayloadCommandStatusMT*), a message to pass a received message from MS to MA (*MA_RxDataPayloadMT*), a set of messages for MA to indicate desired subscriptions to MS (*MA_MessagingSubscriptionSettingCommandMT*, *MA_MessagingSubscriptionSettingCommandStatusMT*) and a message for MS to send existing subscription information to MA (*MA_MessagingSubscriptionStatusMT*).

8.2.5.1 Receive Message Subscription Status

If the DMS was subscribing to topics using the MA configured by the "Subscribe to Message Topic" sequence or by a configuration file, MS will send MA information about the existing manually configured subscriptions using the *MA_MessagingSubscriptionStatusMT* message. Subscriptions

may be loaded at startup using a DMS config file, therefore this interaction enables MA to receive information about subscriptions that were not directly requested by MA. This interaction also enables MA to manage any topics which were manually configured by the "Subscribe to Message Topic" sequence. This message may be sent periodically by the DMS or upon any change to existing subscriptions. When using the "Additional Comms Capability" MUC, subscription information on other topics implemented from the active *Effectivities* from a *MA_DataDisseminationPlanMT* will be provided to MA through the *MA_GatewayActivityMT* status message.

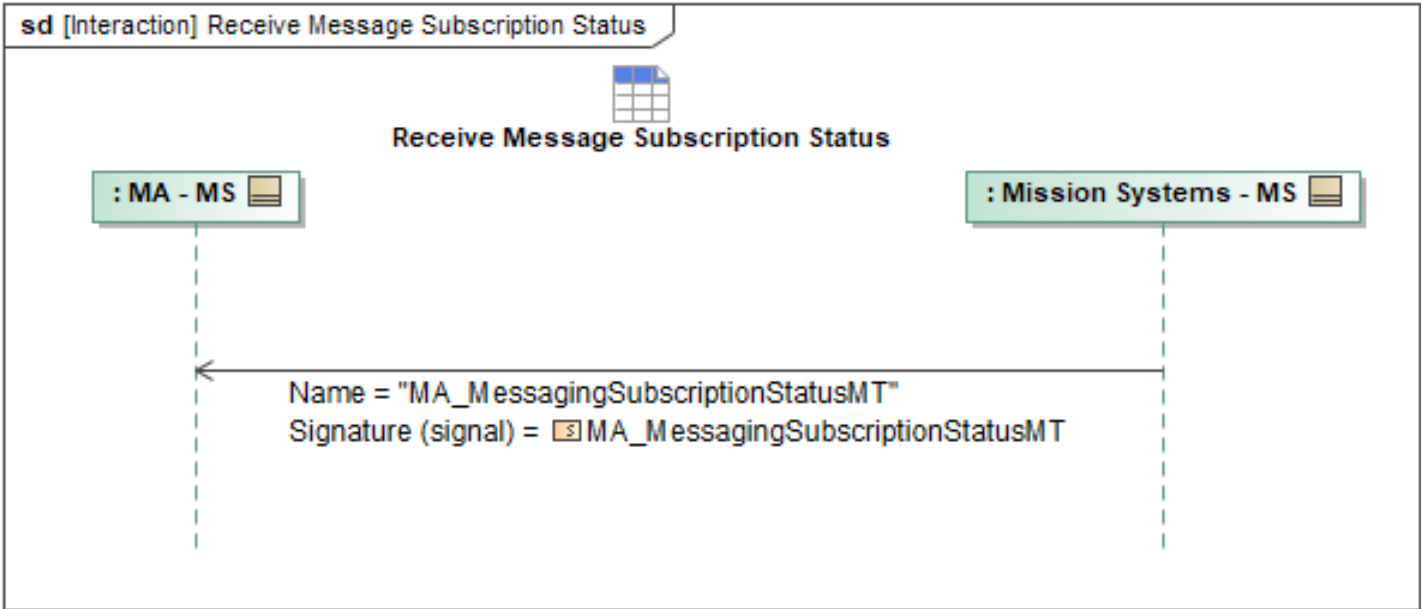


Figure 8-27: Receive Message Subscription Status

Table 8-26: Receive Message Subscription Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessagingSubscriptionStatusMT: MA_MessagingSubscriptionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.5.2 Receive UCI Wrapped Message

When MS receives an offboard message from C2 or a peer ACP it can send the message to MA using the UCI wrapper *MA_RxDataPayloadMT*. When using the "Additional Comms Capability" MUC, the active *Effectivity* specified by the activated *MA_DataDisseminationPlanMT* must be configured for *WrapIngressMessages*="TRUE" for the DMS to UCI wrap the receiving messages in a *MA_RxDataPayloadMT* before sending to MA. The *MA_RxDataPayloadMT* contains a *hexBinary* field called *EncodedPayload* that contains a UTF-8 XML UCI message represented as hexBinary. This message also has message fields *RxDataPayloadID*, *DataPayloadOriginator*, *Priority*, *MessageType*, *DestinationRouting*, *SpecificDestinationID*, *Timestamp* and *MessageExpiry*. *RxDataPayloadID* is a unique ID for this received data payload message. The *DataPayloadOriginator* field in the *MA_RxDataPayloadMT* specifies the ID of the ACP that originally sent the message. The *Priority* field indicates the priority of a received message using a number as follows: 0-10 indicates high priority,

10-20 indicates medium priority, and 20-30 indicates low priority. *MessageType* specifies the UCI type of the message in the *EncodedPayload* field. *DestinationRouting* documents whether the message was routed only to specific destinations or also to the topic corresponding to the *MessageType*. *SpecificDestinationID* is optional, but one or more *SpecificDestinationIDs* will be included if the message was send to those systems directly. The *Timestamp* field indicates the time at which the message was received and the *MessageExpiry* field indicates when the payload message expires. The fields contained within the *MA_RxDataPayloadMT* enable implementers to receive UCI messages for facilitating communication with other ACPs.

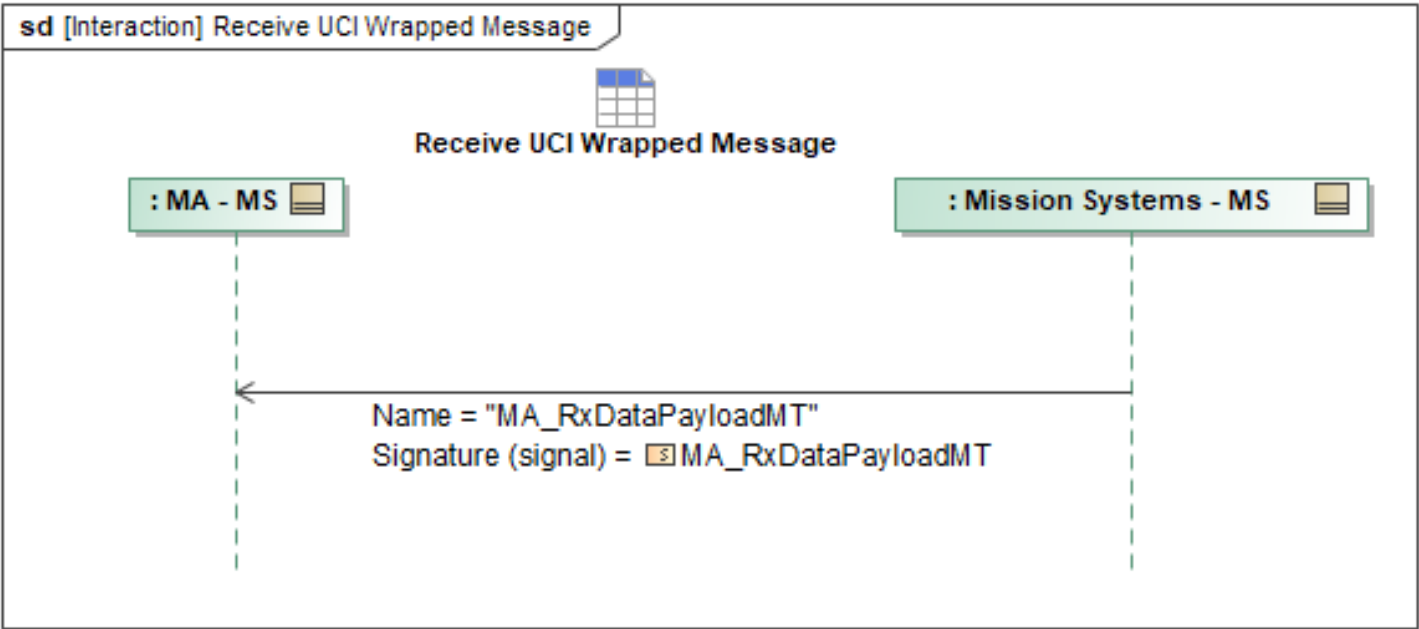


Figure 8-28: Receive UCI Wrapped Message

Table 8-27: Receive UCI Wrapped Message

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RxDataPayloadMT: MA_RxDataPayloadMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.5.3 Send UCI Wrapped Message

When the MA needs to send or publish messages to an external system, such as a peer or group of peer ACPs, it can send the UCI message to MS using a *MA_TxDataPayloadCommandMT* message. When Using the "Additional Comms Capability" MUC, the *IER.AllowWrappedEgressMessages* needs to be set to "TRUE" in the active *Effectivities* in the related *MA_DataDisseminationPlanMT* to enable UCI wrapped egress messages. The *MA_DataDisseminationPlanMT* can also be configured to subscribe to UCI messages on the local ASB directly for sending message offboard. The *MA_TxDataPayloadCommandMT* contains a hexBinary field called *EncodedPayload* that contains a UTF-8 XML UCI message represented as hexBinary. The *MA_TxDataPayloadCommandMT* message has fields *MessageType*, *Priority*, *Timestamp*, *MessageExpiry*, *SpecificDestinationID*, and *DestinationRoutingfields*. The *MessageType* indicates the UCI type of the message in the

EncodedPayload field. The *Priority* field indicates the priority of a message being sent using a number as follows: 0-10 indicates high priority, 10-20 indicates medium priority, and 20-30 indicates low priority. The *Timestamp* field indicates the time at which the message was created and the *MessageExpiry* field indicates when the payload message expires. The *SpecificDestinationID* field is either a list of the ACP endpoints the data is to be sent to or is an empty list. When using the "Additional Comms Capability" MUC, both sender and receiver must have the *DistributaionGroups.DirectedMessagingAllowed* field set to "TRUE" in the related *MA_DataDisseminationPlanMTs* for Specific Destination messaging to be sent/received. The *DestinationRouting* field specifies whether the message should be sent only to *SpecificDestinationID* destinations or to both the appropriate topic (based on *MessageType*) as well as the *SpecificDestinationIDs*. If *DestinationRouting*=SPECIFIC_DESTINATION_ONLY then at least one of *SpecificDestinationID* must be provided. Otherwise, a failure status message will be sent to MA. The fields contained within the *MA_TxDataPayloadCommandMT* enable implementers to send UCI messages for facilitating communication with other ACPs. One or more accompanying *MA_TxDataPayloadCommandStatusMT* messages are returned from MS to MA to indicate the status of the process.

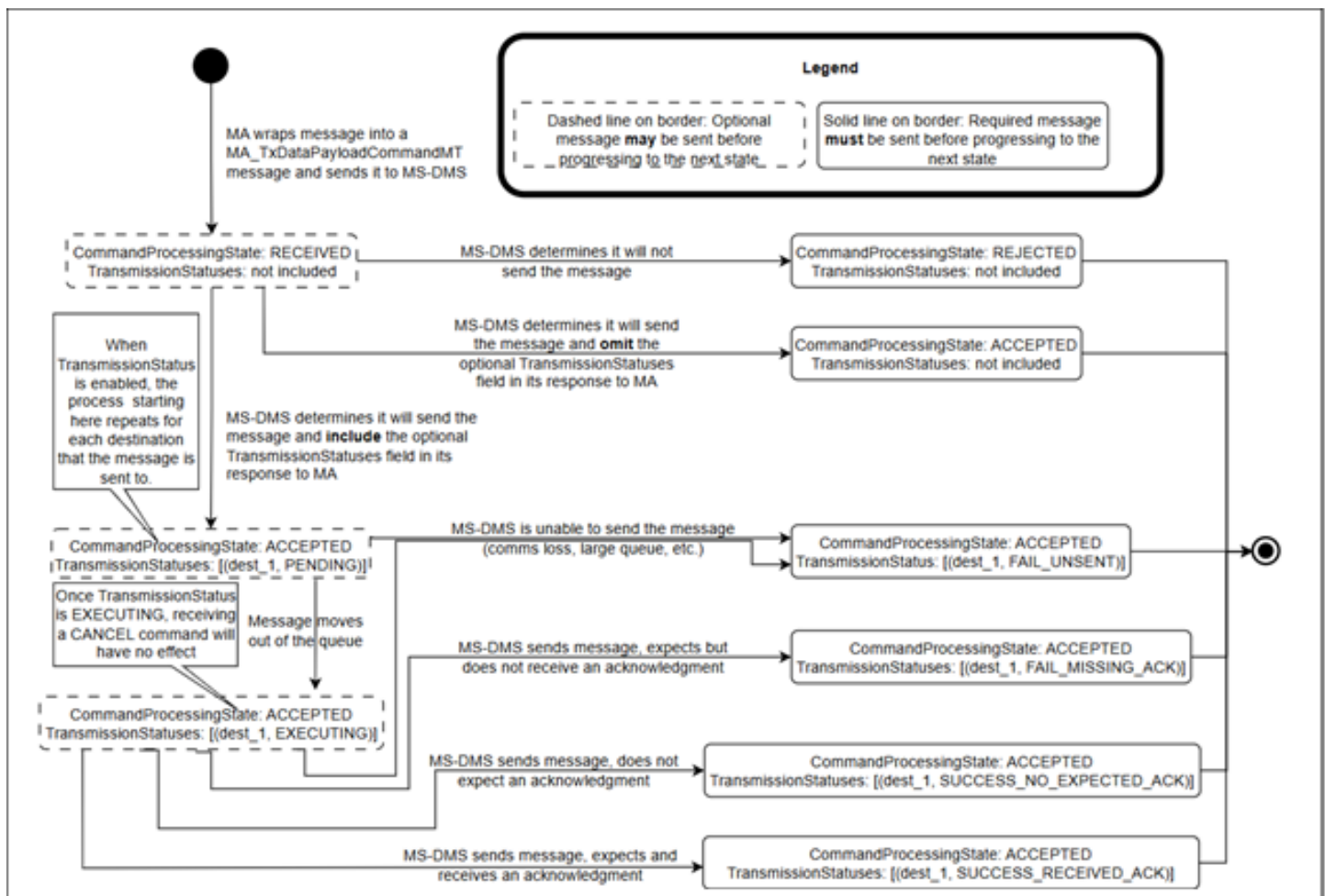


Figure 8-29: State Machine for Send UCI Wrapped Message UCI Diagram

State machine diagram for the "Send UCI Wrapped Message" interaction with transmission to one destination with *SpecificDestinationID*=dest_1.

The steps of sending a message to a single destination are detailed in the state machine diagram shown above, which is an alternative representation of the interaction below. First, MA packs a UTF-8 XML UCI message into the payload of *MA_TxDataPayloadCommandMT*. MA sends this message to MS-DMS. Next, MS-DMS may send an optional confirmation message, *MA_TxDataPayloadCommandStatusMT_1* (with *CommandProcessingState*=RECEIVED), to confirm that it received the message and will attempt to send it offboard. If the message is determined to be invalid (e.g. *DestinationRouting*= SPECIFIC_DESTINATION_ONLY and no *SpecificDestinationID* fields are specified), MS-DMS sends a failure message back to MA with *CommandProcessingState*=REJECTED. This message, represented by *MA_TxDataPayloadCommandStatusMT_2* in the interaction, indicates that the message will not be processed. This case terminates the interaction and no further processing occurs. Otherwise, the message is valid, and MS-DMS will attempt to send it offboard to all specified destinations, as shown in the table below:

Logic for determining set of all destinations to send message to:

Table 8-28: Message Send Destination Logic

		Number of <i>SpecificDestinationID</i> fields	
		None	One or more
<i>DestinationRouting</i>	SPECIFIC_DESTINATION_ONLY	Error: A failure status message with <i>CommandProcessingState</i> =REJECTED will be sent to MA.	The message will be sent to all <i>SpecificDestinationID</i> systems.
	TOPIC_AND_SPECIFIC_DESTINATION	The message will be published to the relevant topic.	The message will be published to the relevant topic and sent to all <i>SpecificDestinationID</i> systems.

Note that after a message is determined to be valid, all subsequent messages in the interaction have a value of ACCEPTED for the *CommandProcessingState* field.

MS-DMS sends one or more status messages back to MA to indicate the result of sending the message offboard. The decision of which optional messages to send may be influenced by configuration and internal logic, the details of which are considered an implementation detail that is out of the scope of this interaction. If the optional (but recommended) *DestinationTransmissionStatus* field is omitted, MS-DMS simply sends *MA_TxDataPayloadCommandStatusMT_3* to MA, indicating that it processed the message, and the interaction terminates. In this case, it cannot provide any additional information such as whether transmission was successful. Without this information, MA cannot perform potentially useful behaviors like application-level resend and smart deletion of sent messages from MD logs when storage is low. Otherwise, MS-DMS provides more detailed information regarding the process of sending the message offboard to each destination. To facilitate use of the *MA_TxDataPayloadCommandStatusMT.TransmissionStatus* field, every DDS DomainParticipant, DataWriter, and DataReader shall set its UserQoS data to be a vector of bytes equal to its host system's *SystemID.UUID*. MS-DMS obtains the set of all destinations specified by *MA_TxDataPayloadCommandMT*. This is either read directly in the *SpecificDestinationID* field or can be determined by retrieving the UserData for all the subscribers to the topic corresponding to the *MessageType* (if publishing to a topic). Then, MS-DMS attempts to send the message to each destination. Transmission to multiple destinations may happen in parallel. For each destination, one or more *MA_TxDataPayloadCommandStatusMT_4* messages are sent from MS-DMS to MA to provide information on the status or result of transmission. The *DestinationTransmissionStatus* field in each of these messages has one or more elements. Each element consists of a

MA_TransmissionStatusType, which contains the *SpecificDestinationID* indicating where the message is transmitted to and the associated *TransmissionStatus* containing the status. For transmission to a single destination “dest_1”, the process is as follows:

- First, MS-DMS may optionally send the message with *DestinationTransmissionStatus* =[(dest_1, PENDING)] to MA. This confirms that the message has been added into the queue to be sent offboard.
- Next, if a problem occurs that results in the message being removed from the queue, such as a loss of comms, MS-DMS send it with *DestinationTransmissionStatus* =[(dest_1, FAIL_UNSENT to MA)].
- Otherwise, MS-DMS may optionally send the message with *DestinationTransmissionStatus* =[(dest_1, EXECUTING)] to MA. This indicates that MS-DMS is actively attempting to send the message offboard and the sending can no longer be canceled or updated. While canceling and updating are not explicitly part of this interaction, if MS-DMS supports canceling or updating (via a *MA_TxDataPayloadCommandMT* with *CommandState*=CANCEL or UPDATE), it must happen before this point.
- Finally, MS-DMS sends the message to MA to indicate the final status of transmission. This message must have one of the following values for the entry in the *DestinationTransmissionStatus* field for the associated destination:
 - **FAIL_UNSENT:** Indicates that some problem (such as a loss of comms) prevented the message from being sent.
 - **FAIL_MISSING_ACK:** Indicates that the message was sent via a protocol that supports confirmation that the message was received (e.g., TCP), but the protocol was unable to confirm that the message was received.
 - **SUCCESS_NO_ACK_EXPECTED:** Indicates that the message was sent using a protocol that does not support confirmation that the message was received (e.g., UDP.)
 - **SUCCESS_RECEIVED_ACK:** Indicates that the message was sent via a protocol that supports confirmation that the message was received (e.g., TCP), and the protocol was able to confirm that the message was received.

This process repeats until a final transmission status has been sent for every destination that MS-DMS attempts to send the UCI message to. Note that DDS does not provide a standard way to access message receipt acknowledgments so SUCCESS_NO_ACK_EXPECTED may be the most common status.

Examples

To clarify the details of the interaction described above, the following examples show two different cases for this interaction.

Example 1. Simplest possible success case for the "Send UCI Wrapped Message" interaction.

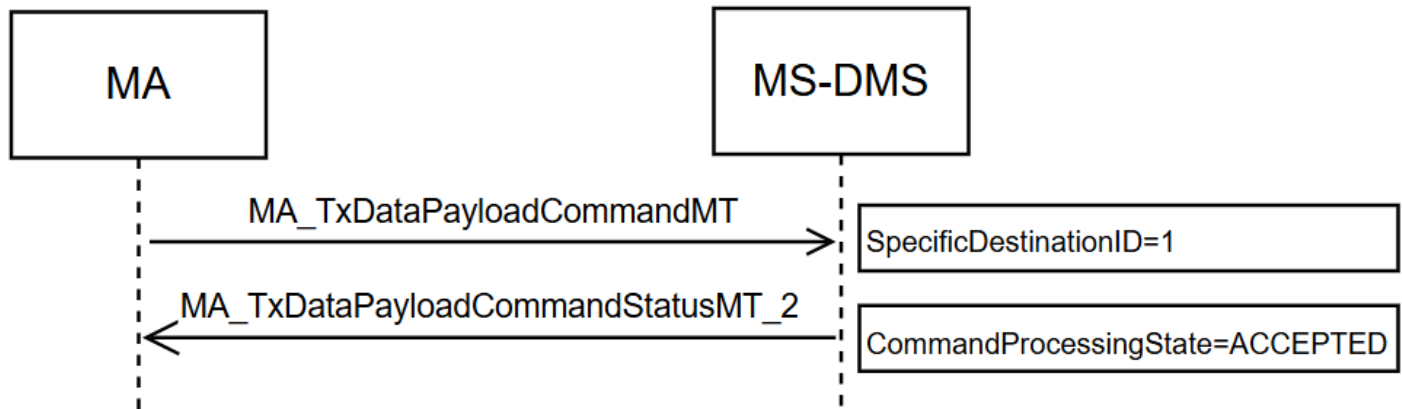


Figure 8-30: Example 1

Example 1 above shows the simplest success case for the interaction where all optional messages are omitted:

- MA sends *MA_TxDataPayloadCommandMT* to MS-DMS
- MS-DMS replies with *MA_TxDataPayloadCommandStatusMT_2* message indicating that the command was accepted.

Example 2. General case for the "Send UCI Wrapped Message" interaction.

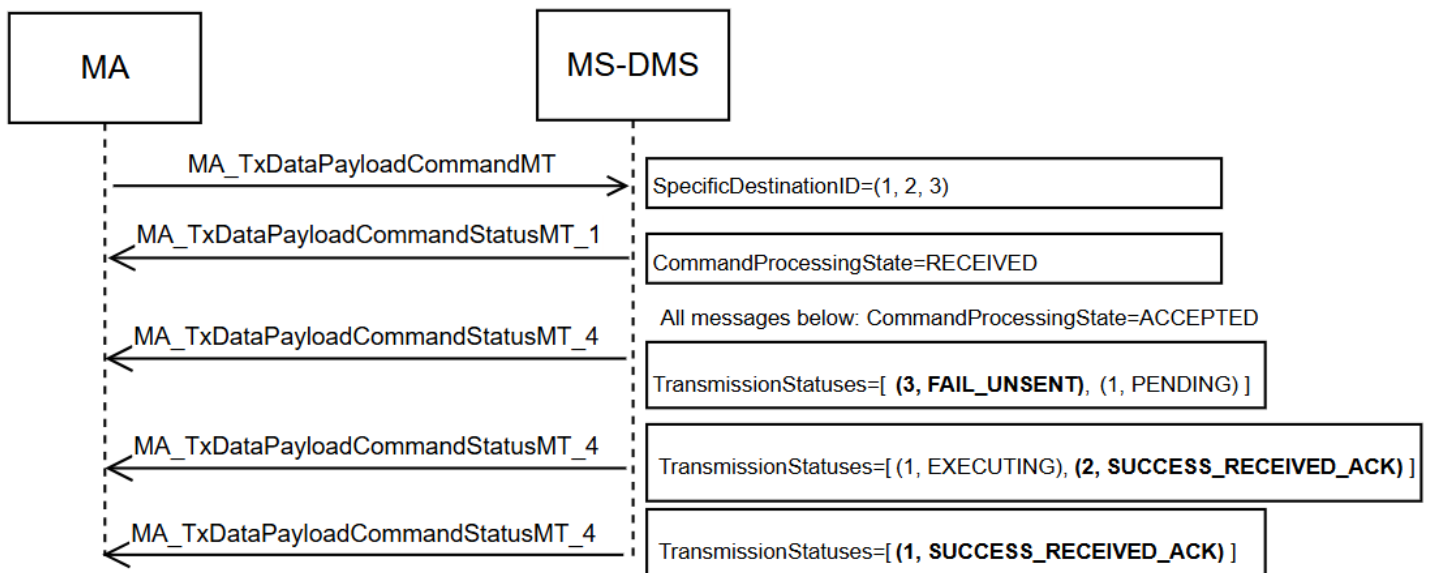


Figure 8-31: Example 2

Example 2 shows an example of the interaction when some optional messages are included:

- MA sends *MA_TxDataPayloadCommandMT* to MS-DMS indicating that the message should be sent to 3 destinations.
- MS-DMS replies with *MA_TxDataPayloadCommandStatusMT_1* message indicating that the command was received.
- MS-DMS sends back a series of *MA_TxDataPayloadCommandStatusMT_4* messages.

- Each message contains *TransmissionStatus* entries for one or more destinations. The destinations are not ordered and different destinations in the same message may have different values for *TransmissionStatus*.
- The interaction terminates when the final transmission status (shown in bold for each destination) is sent for all destinations.

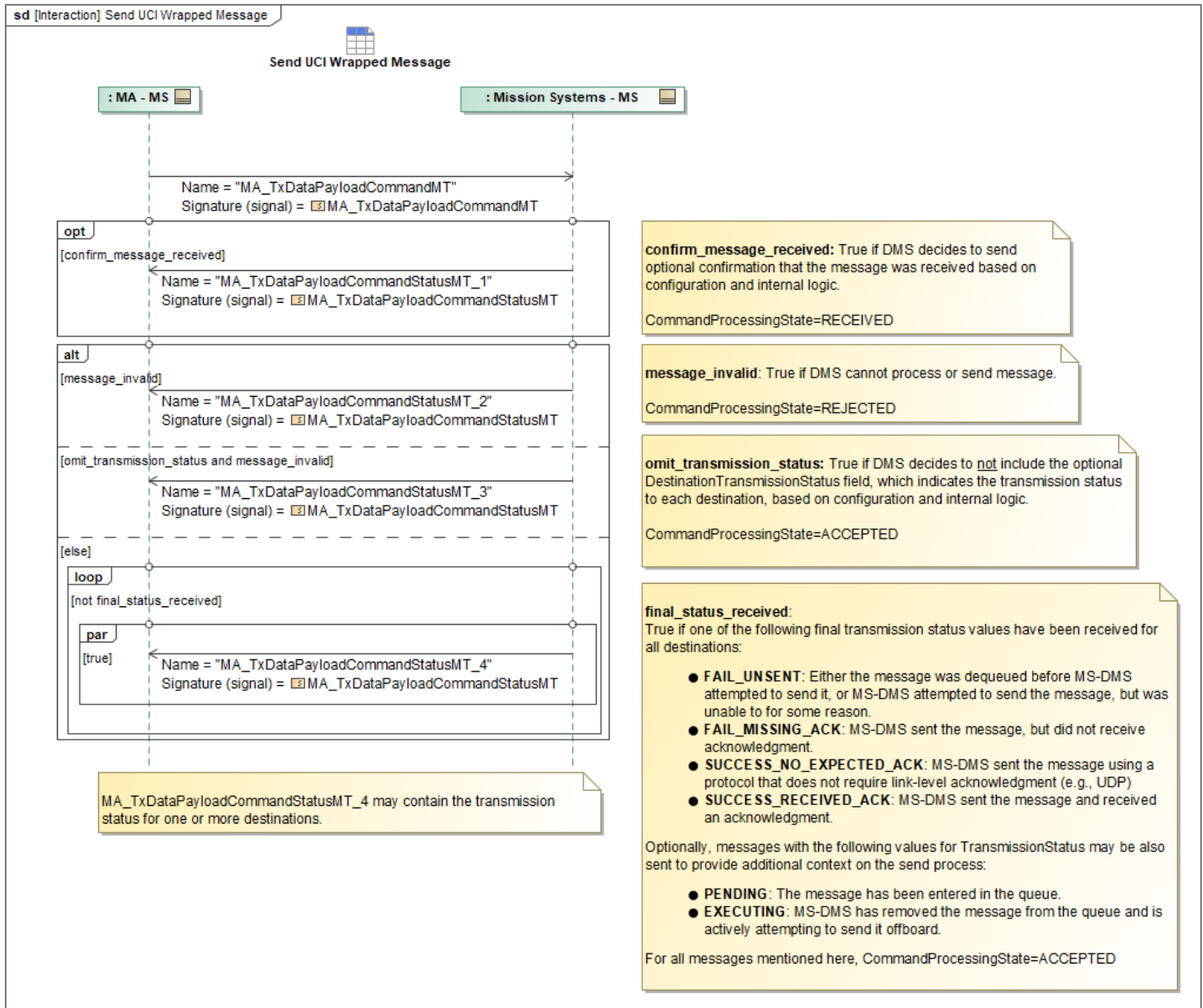


Figure 8-32: Send UCI Wrapped Message

Table 8-29: Send UCI Wrapped Message

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TxDataPayloadCommandMT: MA_TxDataPayloadCommandMT	SpecificDestinationID: SystemID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TxDataPayloadCommandStatusMT_1: MA_TxDataPayloadCommandStatusMT	None
MA_TxDataPayloadCommandStatusMT_2: MA_TxDataPayloadCommandStatusMT	None
MA_TxDataPayloadCommandStatusMT_3: MA_TxDataPayloadCommandStatusMT	None
MA_TxDataPayloadCommandStatusMT_4: MA_TxDataPayloadCommandStatusMT	DestinationTransmissionStatus: MA_TransmissionStatusType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.5.4 Subscribe to Message Topic

For inter-ACP communication utilizing the DMS, the DMS will subscribe to message topics to let other DMSs know it wants to receive those messages on that topic. When using the "Additional Comms Capability" MUC, MS would handle the configuration of the DMS Gateway Capability using the *MA_DataDisseminationPlanMT*, but if MA needs to make a direct modification of the subscriptions this sequence can be used. The active *Effectivity* in the related *MA_DataDisseminationPlanMT* specifies the set of ingress messages and associated topics the DMS should subscribe to. If subscriptions are needed outside of the active *MA_DataDisseminationPlanMT*, MA can send MS a *MA_MessagingSubscriptionSettingCommandMT_1* with the *AddTopicSubscription* field populated with the *Topic(s)* to subscribe to and the *SubscriptionID*. Optionally, MA can specify the *Priority* of the subscription as well as additional filtering of the subscription using the *SubscriptionFilterType* field. A *Topic* must be provided for a valid subscription. MA can also optionally specify the *DataPayloadOriginator* for the subscription. If there is no *DataPayloadOriginator* in the message, then MA will be subscribed to all publishers of the *Topic*. If the *DataPayloadOriginator* is populated, then the subscription will only be for data payloads of that specified *Topic* that were generated by the specified originator. To modify an existing subscription created by this sequence in the past or configuration file, MA will send MS a *MA_MessagingSubscriptionSettingCommandMT_2* with the *ChangeTopicSubscription* field populated with the *SubscriptionChangeType* and the *SubscriptionID*. To unsubscribe from a topic created by this sequence in the past or configuration file, the same *MA_MessagingSubscriptionSettingCommandMT* is passed from MA to MS, but the *ChangeTopicSubscription* field is populated with the *SubscriptionID(s)* that are to be removed and the *SubscriptionChangeType* set to *DELETE*. In response to the *MA_MessagingSubscriptionSettingCommandMT* message, MS will return a *MA_MessagingSubscriptionSettingCommandStatusMT* message to MA to report the status of the subscription settings command.

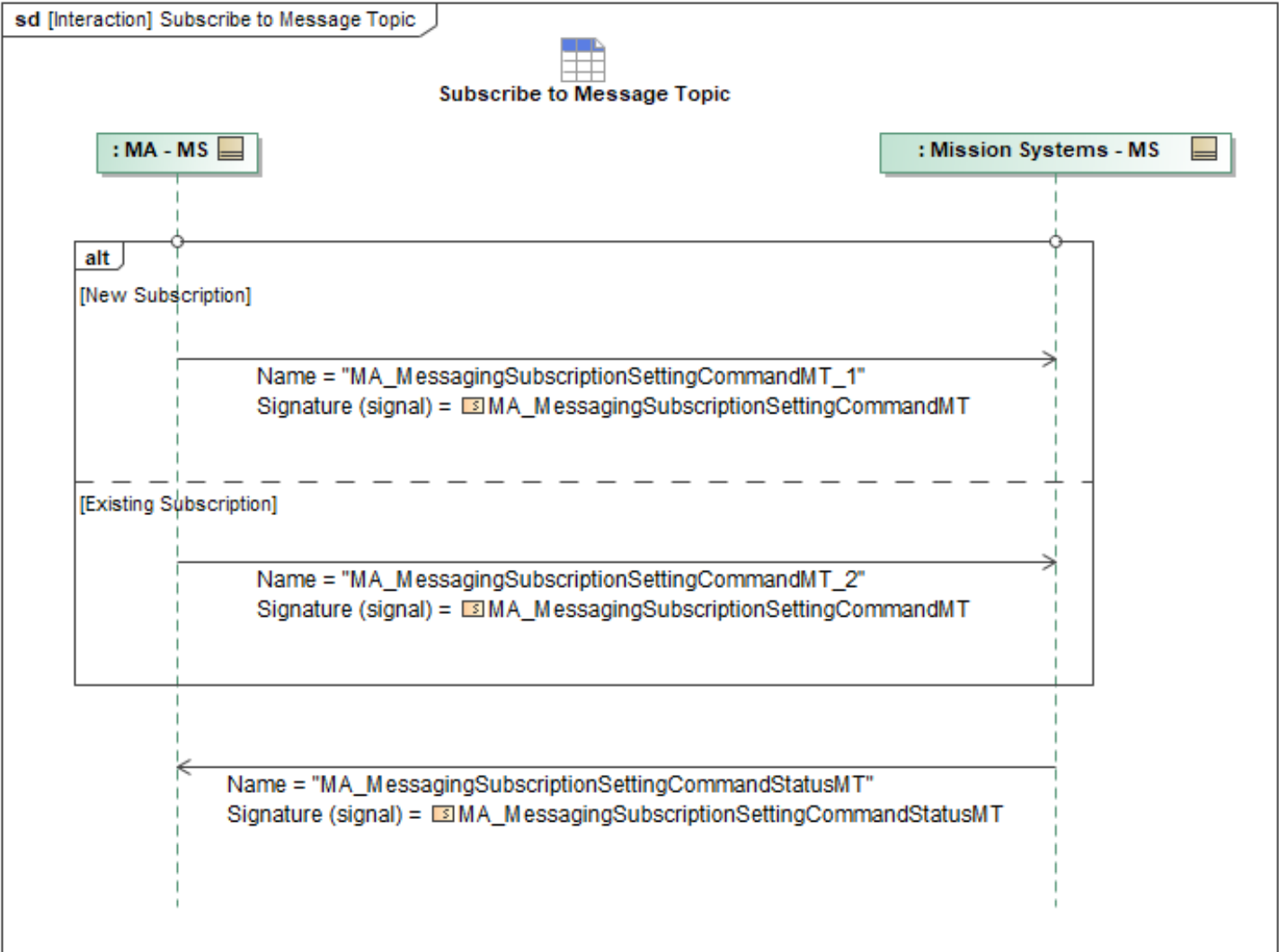


Figure 8-33: Subscribe to Message Topic

Table 8-30: Subscribe to Message Topic

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessagingSubscriptionSettingCommandMT_1: MA_MessagingSubscriptionSettingCommandMT	AddTopicSubscription: MA_TopicSubscriptionType
MA_MessagingSubscriptionSettingCommandMT_2: MA_MessagingSubscriptionSettingCommandMT	ChangeTopicSubscription: MA_ChangeTopicSubscriptionType
MA_MessagingSubscriptionSettingCommandStatusMT: MA_MessagingSubscriptionSettingCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

8.2.5.5 Quality of Service

To meet system requirements for reliability, latency, and data freshness, each topic, on the DDS network, is governed by a set of Quality of Service (QoS) policies. These policies control how data is transmitted, stored, and delivered between systems. At a minimum, all topics must declare their QoS settings explicitly.

The default DDS Reliability QoS policies include:

- **RELIABLE** delivery for critical data. This includes messages that require a response, such as acknowledgments or other types of responses, ensuring that messages are delivered and a link level acknowledgment is received, providing guaranteed delivery. Specifically, this applies to UCI primitive types: *Command-2*, *ActionRequest-2*, *DataRequest-2*, *DataRecord-1*, and the *MA_SystemNotificationMT*, *OperatorNotificationMT*, and *OperatorNotificationAckMT* messages.
- **BEST_EFFORT** delivery where data loss is acceptable. This includes messages that do not require an acknowledgment or response. Specifically, this applies to UCI primitive types: *Data-1* and *Status-1*.

When using the "Additional Comms Capability" MUC, the DDS QoS configuration can be configured or changed from the default by using the *DistributionGroup.NetworkNativeEndpoint* field in the *MA_DataDisseminationPlanMT* message and linking it to one or multiple *IER*s. The "Update Data Dissemination Plan" sequence discusses the configuration in more detail. The *DistributionGroup.NetworkNativeEndpoint* field allows for setting the following QoS specific items:

- *TopicQosReliability*: Setting this to "BEST_EFFORT" or "RELIABLE" will directly set this value for the topic(s). If set to "NONE" or not set, either the associated *DeliveryProtocol* is used or the DDS Reliability values above is used to determine the QoS Reliability Setting.
- *TopicQosKeepLast*: If "TRUE", upon reconnect the last message published on the ASB will be sent out to the subscriber. This will configure the DDS publisher with the Keep Last QoS setting. This should only be done for status and some data type messages and it should not be used for commands, requests, or requirement messages that could cause issues with the receiving system. This provides a nice way for status messages to be delivered specifically for messages that are non-periodic.
- *TopicQosPriority*: If Empty, the *IER.Priority* is used. In DDS, the higher the value, the higher the priority. Therefore the *IER.Priority* would need to be inverted or mapped.

QoS compatibility is essential for interoperability and successful communication, ensuring that a subscriber's QoS expectations are compatible with those of the publisher by ensuring that the *MA_DataDisseminationPlanMT*s are aligned and activated appropriately.

8.2.5.6 Security

System communication requires authentication, encryption, and access control to meet security requirements. The DDS Security specification (see <https://www.omg.org/spec/DDS-SECURITY/1.2/>) shall be leveraged to enforce these measures. Systems must be authenticated using X.509 certificates or equivalent credentials before joining the network. Implementers are responsible for managing key sets, certificates, and revocation policies.

All data transmitted over the network must be encrypted to protect it from eavesdropping and tampering. Implementers must use the DDS Security plugins to enforce Transport Layer Security (TLS) with mutual TLS (mTLS). This ensures data confidentiality, integrity, and authenticity during transmission.

8.2.5.7 Serialization and Data Encoding

Serialization and data encoding are critical for ensuring interoperability between different systems in the DMS. The DDS middleware supports various serialization formats to encode data structures defined in the Interface Definition Language (IDL). The Common Data Representation (CDR) format shall be used for serialization due to its interoperability, performance, and compatibility with DDS. All data structures shared between systems must be defined in the IDL and serialized according to the defined schema.

All topics transmit data using the `MA_DataPayloadWrapper` IDL type. The actual UCI message content will be EXI encoded into the `EncodedPayload` field. The IDL of the `MA_DataPayloadWrapper` is included in the Autonomy Starter Kit release with the Decentralized Messaging Service component of the A-GRI. When using the "Additional Comms Capability" MUC the `MA_DataDisseminationPlanMT` has an optional `IER_DesiredService.CompressionType` field that can be set to "EXI" to defines the encoding/compression format for the `EncodedPayload` field in the OTA `MA_DataPayloadWrapper` IDL that is used for communication between DMSs.

The table below specifies in detail the EXI specific Settings that should be used:

Table 8-31: EXI Settings

EXI Settings	Details
Schema Informed	True
EXI Strict	True
EXI Compression	False
EXI Cookie	Omitted
EXI Options Field	Include except for Link16
UUID DTR	UUIDs shall be represented as an EXI string table partition
ScaledDecimal DTR	Decimal precision will be as follows: 7 - Latitude, Longitude 6 - VnVn, VnVe, VeVe 4 - NorthAcceleration, EastAcceleration, DownAcceleration, YawRate, PitchRate, RollRate, PnPn, PnPe, PePe, PnVn, PnVe, PnVn, PeVe 2 - Yaw, Pitch, Roll, Bearing, BearingError, MagneticHeading, Azimuth, AzimuthError, Heading/Value, Kollsman 1 - Altitude, NorthSpeed, EastSpeed, DownSpeed, Confidence, TrueAirspeed, Fuel, ProbabilityOfCorrectAssociation, IndicatedBaroAltitude, PowerLevel, Percent, EstimatedPercentComplete, TargetRange
DateTimeTable DTR	The DateTimeTable DTR uses an EXI string table partition for all <code>xsd:DateTimes</code> to represent those that fully or partially match previous DateTime stamps in a message more efficiently. It starts with an EXI Boolean indicating whether this is a repeated DateTime (i.e., whether it exists in the DateTime string table partition). If the DateTime is repeated,

	the Boolean is set to true and is followed by the index of the DateTime in the table represented an EXI n-bit Unsigned Integer, where n is $\lceil \log_2 m \rceil$ and m is the number of entries in DateTime string table partition. If the DateTime only partially matches one of the entries in the DateTime string table partition, the Boolean is set to false and is followed by a special year value indicating how much of the DateTime matched, followed by the table index of the partially matching DateTime, represented as an EXI n-bit Unsigned Integer (as described above), followed by the non-matching parts of the DateTime. Table below shows the special year values used to indicate which parts of the DateTime matched. <table><tr><th>Matching Parts</th><th>Year Value</th><th>Non-matching parts</th></tr><tr><td>Year,Month,Day,Hour,Minute,Second</td><td>-1</td><td>presence,[FractionalSeconds]</td></tr><tr><td>Year,Month,Day,Hour,Minute</td><td>-2</td><td>Seconds,presence,[FractionalSeconds]</td></tr><tr><td>Year,Month,Day,Hour</td><td>-3</td><td>Minutes*64+Seconds,presence,[FractionalSeconds]</td></tr><tr><td>Year,Month,Day</td><td>-4</td><td>Time,presence,[FractionalSeconds]</td></tr><tr><td>Year,Month</td><td>-5</td><td>Day,Time,presence,[FractionalSeconds]</td></tr></table> The non-matching parts are represented according to section 7.1.87 of the EXI specification. If the DateTime does not fully or partially match any of the entries in the DateTime string table partition, the Boolean is set to false and is followed by the entire DateTime. All parts of the DateTime are represented according to section 7.1.8 of the EXI specification, except the Year. The DateTimeTable DTR represents years after 1999 as an EXI Unsigned Integer with the value $(y - 2000) * 2 + s$, where y is the Year and s is 0. The DateTimeTable DTR represents years before 2000 as an EXI Unsigned Integer with the value $(y - 2000 - 6) * 2 + s$, where y is the Year and s is 1. This representation eliminates the need for a separate sign bit and reserves the values -1 through -5 to signal partial matches.	Matching Parts	Year Value	Non-matching parts	Year,Month,Day,Hour,Minute,Second	-1	presence,[FractionalSeconds]	Year,Month,Day,Hour,Minute	-2	Seconds,presence,[FractionalSeconds]	Year,Month,Day,Hour	-3	Minutes*64+Seconds,presence,[FractionalSeconds]	Year,Month,Day	-4	Time,presence,[FractionalSeconds]	Year,Month	-5	Day,Time,presence,[FractionalSeconds]
Matching Parts	Year Value	Non-matching parts																	
Year,Month,Day,Hour,Minute,Second	-1	presence,[FractionalSeconds]																	
Year,Month,Day,Hour,Minute	-2	Seconds,presence,[FractionalSeconds]																	
Year,Month,Day,Hour	-3	Minutes*64+Seconds,presence,[FractionalSeconds]																	
Year,Month,Day	-4	Time,presence,[FractionalSeconds]																	
Year,Month	-5	Day,Time,presence,[FractionalSeconds]																	
Dictionary Pre-Population	The system shall support an EXI state file that allows for pre-population of the EXI string table with known, stable UUIDs. May only be necessary for Link16 but should be supported.																		

8.2.5.8 System Discovery

DDS enables distributed components to automatically discover each other on the network and exchange metadata about the topics they publish or subscribe to. The DDS middleware relies on the RTPS (Real-Time Publish-Subscribe) protocol, specifically the Simple Participant Discovery Protocol (SPDP) and Simple Endpoint Discovery Protocol (SEDP), to dynamically form a mesh of communicators without requiring a centralized broker. Upon startup, each node registers itself and advertises its capabilities using these discovery protocols.

Participants must be uniquely identifiable by their *SystemID*, defined via configuration. Multicast-based discovery is the default mechanism, but support for static discovery and peer list configuration is required for network-constrained environments.

8.2.5.9 Topic Publication and Subscription

Communication between system components occurs over well-defined topics. Each topic represents a stream of typed data, with any number of publishers and subscribers. Topics are uniquely named and associated with a specific data structure defined in an Interface Definition Language (IDL). All systems must agree on this schema and the semantics of each topic they interact with

As described previously, by default each message type is assigned a dedicated topic, with the topic name exactly matching the message name. This enforces a consistent mapping between data and transport, simplifies subscription management, and allows fine-grained QoS control. Publishers and

subscribers may join or leave at any time, and DDS guarantees delivery according to the configured QoS policies. When using the "Additional Comms Capability" MUC, the *MA_DataDisseminationPlanMT* does allow for configuring unique topic names, but this should be limited as it makes interoperability harder to manage. Additionally, each DMS subscribes to a topic that is equal to its host system's *SystemID.UUID* (with a "id_" prefix and "-"→"_") to facilitate *SpecificDestinationID* messaging.

All topics transmit data using the *MA_DataPayloadWrapper* IDL type. The actual UCI message content will be EXI Encoded into the *EncodedPayload* field. The "Serialization and Data Encoding" section describes the EXI representation in detail. The IDL of the *MA_DataPayloadWrapper* is included in the Autonomy Starter Kit release with the Decentralized Messaging Service component of the A-GRI.

The DMS will advertise topics on the DDS interface based on the *MA_TxDataPayloadCommandMT*. When using the "Additional Comms Capability" MUC, topics are advertized using the *IERs* and *DistributionGroups* in the active *Effectivities* in the related *MA_DataDisseminationPlanMT*. The message format on all topics will be the *MA_DataPayloadWrapper* IDL that is filled with either the contents from a *MA_TxDataPayloadCommandMT* message or, when using the "Additional Comms Capability" MUC, the directly ASB-subscribed UCI message. The DMS will publish messages on the appropriate topics to go off-board based on the message type or, when using the "Additional Comms Capability" MUC, the mapping of messages to topics in the *MA_DataDisseminationPlanMT*. For subscriptions when using the "Additional Comms Capability" MUC, the DMS will use the *IERs* and *DistributionGroups* in the active *Effectivities* in the related *MA_DataDisseminationPlanMT* to determine what topics it should subscribe to on the DDS interface. Along with the *SystemID* topic specified above. When a *MA_DataPayloadWrapper* IDL message is received, it will be processed through the filter in the *MA_DataDisseminationPlanMT* and will either be published to the local ASB directly as the UCI message (preserving the senders message header information) or will be wrapped in a *MA_RxDataPayloadMT* message. MA will then receive and process these messages as needed.

8.2.5.10 Transport Protocols

To ensure robust and flexible communication, systems must support both UDP and TCP transport protocols. TCP is recommended for scenarios requiring reliable, ordered delivery of messages. It ensures data integrity and is suitable for messages that require acknowledgments or responses. UDP is recommended for scenarios where low latency and minimal transmission overhead are critical. It is suitable for non-critical data where occasional message loss is acceptable and where speed is prioritized over reliability.

8.2.6 Mission Plan Behaviors Package

8.2.6.1 Receive Plans

The diagram contains the collection of plan updates specifically for MS, performed following the update logic described below for each individual plan represented in the diagram.

Plan Data is updated the via the specific *XYZ_PlanMT* message for the plan that is being updated. If the message being sent represents the first instance of the data/object, the *ObjectState* field is set to "NEW". When sending data to revise an existing set or object, *ObjectState* is set to "UPDATED".

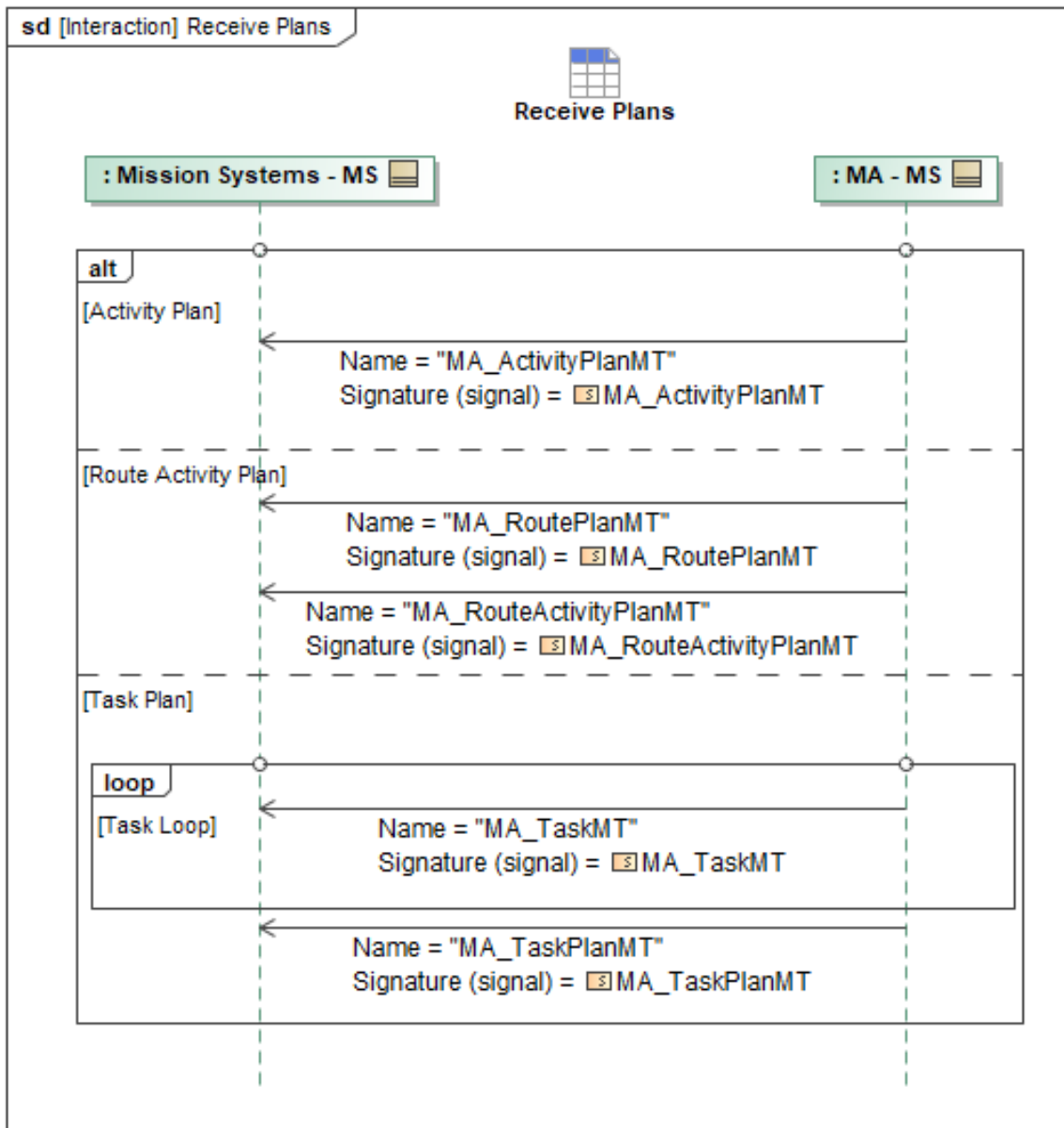


Figure 8-34: Receive Plans

Table 8-32: Receive Plans

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	ObjectState: ObjectStateEnum
MA_RouteActivityPlanMT: MA_RouteActivityPlanMT	ObjectState: ObjectStateEnum
MA_RoutePlanMT: MA_RoutePlanMT	ObjectState: ObjectStateEnum
MA_TaskMT: MA_TaskMT	ObjectState: ObjectStateEnum
MA_TaskPlanMT: MA_TaskPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

8.2.6.2 Verify Plan Patch Is Possible - MS

Depending on the type of *Plan* being patched, one of *MA_MissionPlanValidationCommandMT*, *TaskPlanValidationCommandMT*, *RouteActivityPlanValidationCommandMT*, *ActivityPlanValidationCommandMT*, or *ActionPlanValidationCommandMT* is sent from MA to MS. MS then responds with the corresponding *CommandStatusMT*, i.e., *MA_MissionPlanValidationCommandStatusMT*, *TaskPlanValidationCommandStatusMT*, *RouteActivityPlanValidationCommandStatusMT*, *ActivityPlanValidationCommandStatusMT*, or *ActionPlanValidationCommandStatusMT*. Within the **CommandMT* message, the *PlanPart* contains the plan patch that needs to be validated.

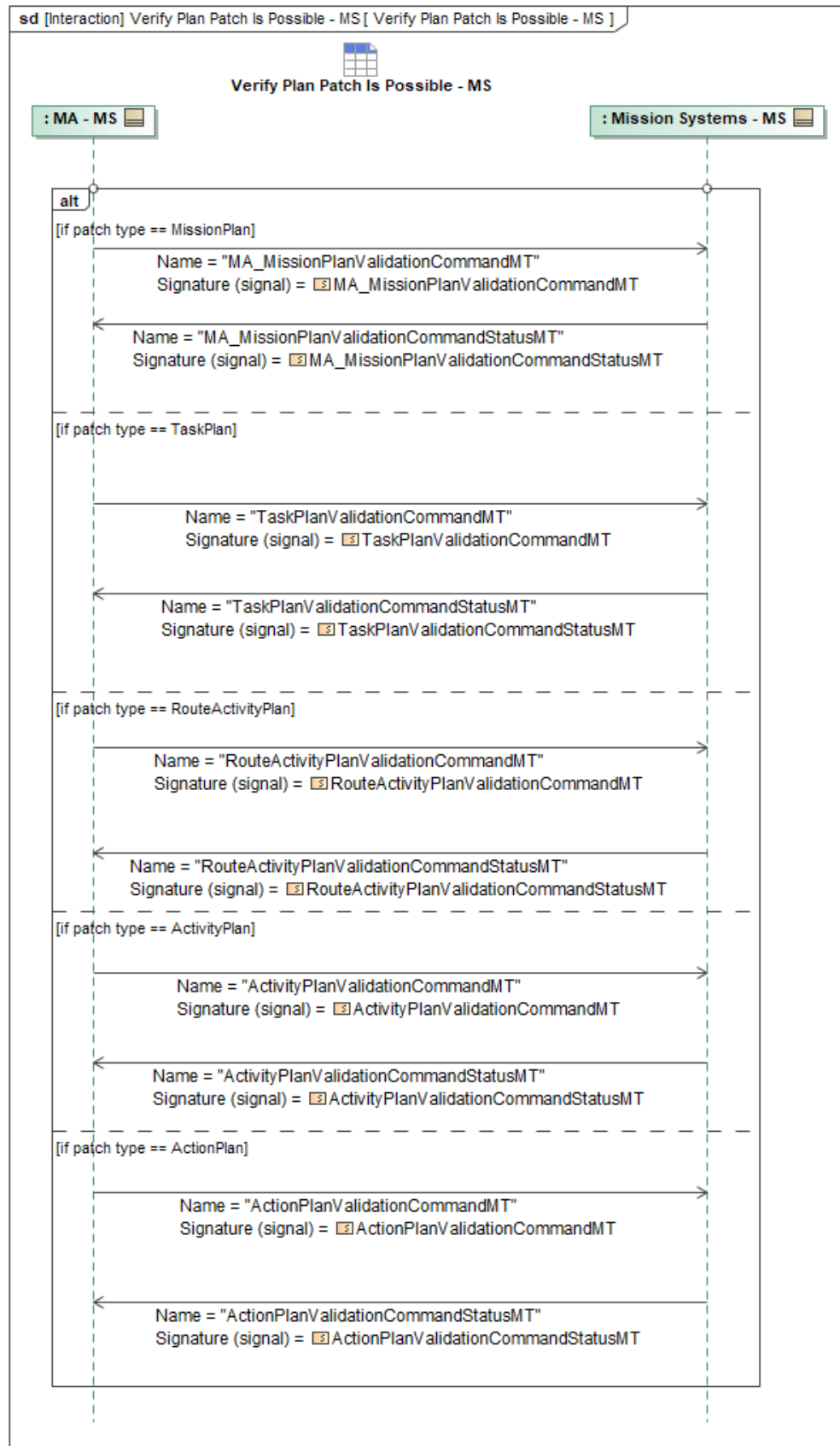


Figure 8-35: Verify Plan Patch Is Possible - MS

Table 8-33: Verify Plan Patch Is Possible - MS

Message Name (Name:Type)	Required Fields (Name:Type)
ActionPlanValidationCommandMT: ActionPlanValidationCommandMT	PlanPart: ActionTypeEnum
ActionPlanValidationCommandStatusMT: ActionPlanValidationCommandStatusMT	None
ActivityPlanValidationCommandMT: ActivityPlanValidationCommandMT	PlanPart: ActivityPlanPartsEnum
ActivityPlanValidationCommandStatusMT: ActivityPlanValidationCommandStatusMT	None
MA_MissionPlanValidationCommandMT: MA_MissionPlanValidationCommandMT	PlanPart: PlanPartsType
MA_MissionPlanValidationCommandStatusMT: MA_MissionPlanValidationCommandStatusMT	None
RouteActivityPlanValidationCommandMT: RouteActivityPlanValidationCommandMT	PlanPart: ActivityPlanPartsEnum
RouteActivityPlanValidationCommandStatusMT: RouteActivityPlanValidationCommandStatusMT	None
TaskPlanValidationCommandMT: TaskPlanValidationCommandMT	PlanPart: TaskTypeEnum
TaskPlanValidationCommandStatusMT: TaskPlanValidationCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.7 PNT Service Package

8.2.7.1 Request PNT Navigation Data

Mission Use Case: PNT

This interaction sequence enables MA to request precision navigation and time from the MS in order to perform its functions that are position- and time-dependent. First MA requests the status of the MS to verify it can provide position information using the *SubsystemStatusDataRequestMT* message. The MS will respond to the request with a *SubsystemStatusDataRequestStatusMT* message that provides a *SubsystemStatusData* field that reports the health and status of the MS. MA will then command the PNT service to adjust the frequency it posts PNT data to other services using the *SubsystemSettingsCommandMT*. The PNT service will reply, providing MA with a *MA_PositionReportDetailedMT* with detailed position information of the ACP. This interaction provides MA integrators a way to retrieve precision time and navigation information from the MS.

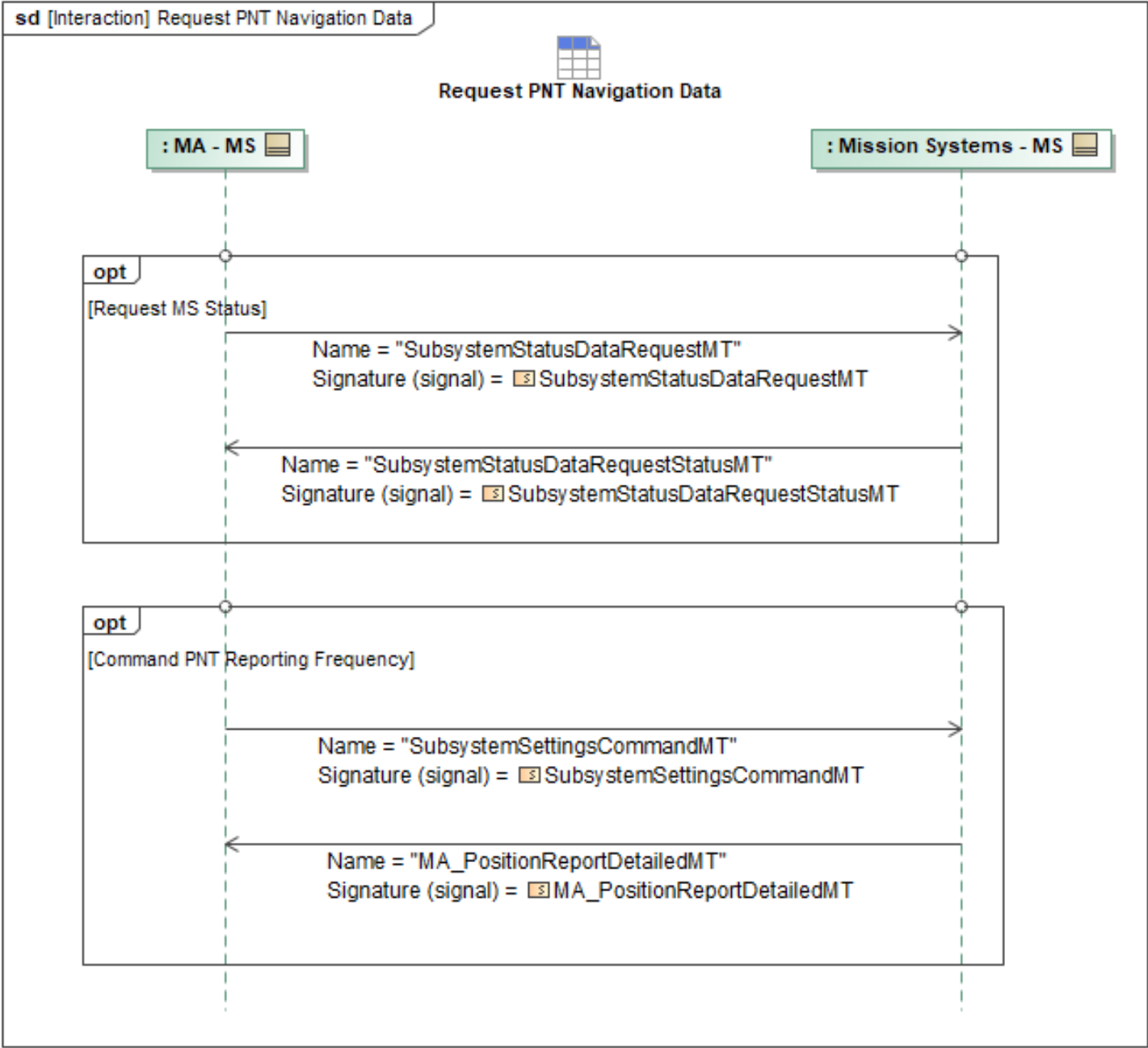


Figure 8-36: Request PNT Navigation Data

Table 8-34: Request PNT Navigation Data

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	Orientation: OrientationType MagneticHeading: AngleType WanderAngle: AngleType OrientationRate: OrientationRateType OrientationAcceleration: OrientationAccelerationType

Message Name (Name:Type)	Required Fields (Name:Type)
	Acceleration: Acceleration3D_Type AirData: MA_AirDataType
SubsystemSettingsCommandMT: SubsystemSettingsCommandMT	None
SubsystemStatusDataRequestMT: SubsystemStatusDataRequestMT	None
SubsystemStatusDataRequestStatusMT: SubsystemStatusDataRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8 Sensor/Vehicle Tasking Package

8.2.8.1 Advertise Sensor AMTI Capability

Mission Use Case: AMTI

The Advertise Sensor AMTI Capability sequence diagram below specifies the interaction of MS communicating the Airborne Moving Target Indicator (AMTI) sensor capability to MA so that it may be used by MA to detect aircraft through active radar. This sequence is only present if AMTI sensing capabilities exist in MS. MS publishes *MA_AMTI_CapabilityMT* with at least one *Capability* specified. Multiple *Capability* fields are specified to advertise different command use cases. Each capability must have a unique identifier set in *CapabilityID*. The *AcceptedInterface* field must specify at least *CAPABILITY_COMMAND*. The *CapabilityType* should be set to the types of commands that the capability supports. The *SubCapabilityType* field can be set to any of the four options, as MS is implemented. The *CapabilitiesOptions* field and sub-fields will be set by MS according to the ability of MS (e.g. *InterruptOtherActivities* and *ConcurrentOperationSupported* are descriptive of particular MS implementation). Similarly, The *SupportedFrequencies* and *ComponentFieldofRegard* fields and sub-fields are set by MS according to the ability of MS as well.

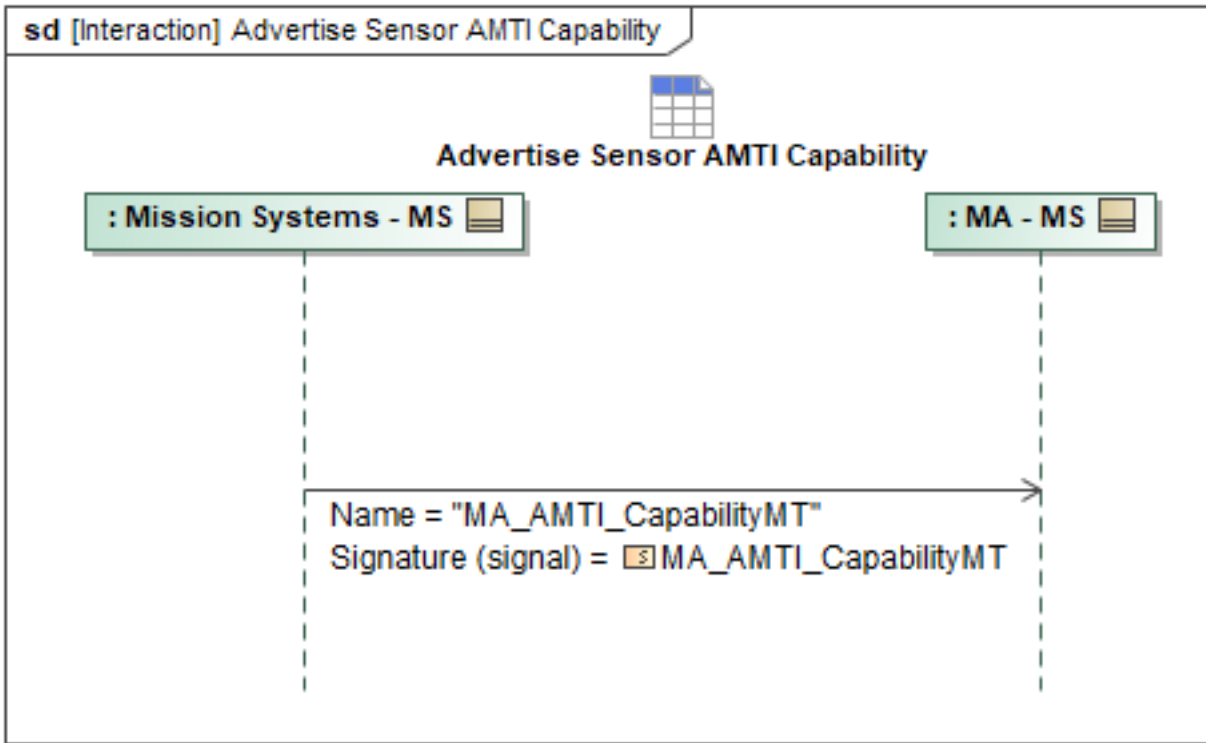


Figure 8-37: Advertise Sensor AMTI Capability

Table 8-35: Advertise Sensor AMTI Capability

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	ComponentFieldOfRegard: ComponentFieldOfRegardType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.2 Advertise Sensor ESM Capability

Mission Use Case: ESM

The Advertise Sensor ESM Capability sequence diagram below specifies the interaction of MS communicating the Electronic Support Measures (ESM) sensor capability to MA so that it may be used by MA to detect aircraft through electronic support measures. This sequence is only present if ESM sensing capabilities exist in MS. MS publishes *ESM_CapabilityMT* with at least one *Capability* specified. Multiple *Capability* fields are specified to advertise multiple command use cases. Each *Capability* must have a unique identifier set in *CapabilityID*. The *AcceptedInterface* field must specify at least *CAPABILITY_COMMAND*. The *CapabilityType* should either be *SEARCH_VOLUME* (for searching a volume of space to generate tracks) or *TRACK* (for updating a specified track). The *CapabilitiesOptions* field and sub-fields will be set by MS according to the ability of MS (e.g. *InterruptOtherActivities* and *CalibrateOnActivity* are descriptive of particular MS implementation). If the capability supports providing a Field of Regard, the *CapabilityFOR* field must specify the field of regard of the sensor ESM capability. No other schema-optional fields are required but may be specified for more information.

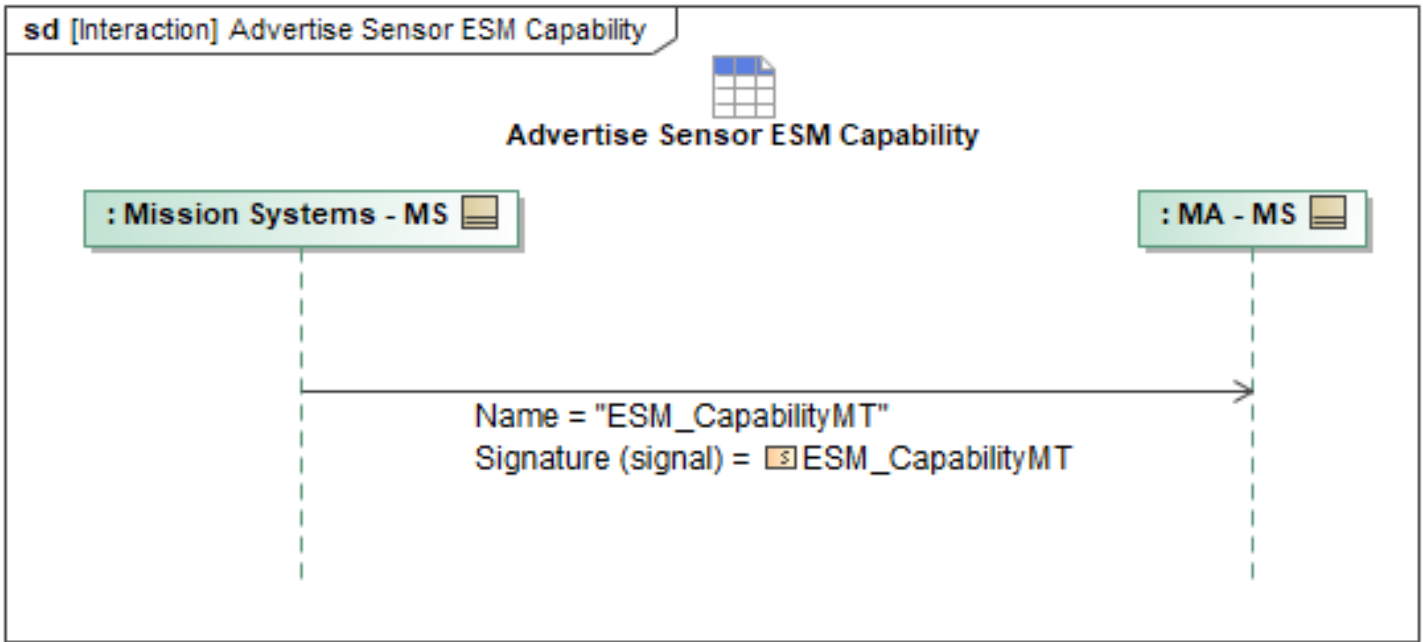


Figure 8-38: Advertise Sensor ESM Capability

Table 8-36: Advertise Sensor ESM Capability

Message Name (Name:Type)	Required Fields (Name:Type)
ESM_CapabilityMT: ESM_CapabilityMT	CapabilityFOR: FOR_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.3 Advertise Sensor PO Capability

Mission Use Case: PO

The Advertise Sensor PO Capability sequence diagram specifies the interaction of MS communicating the Passive Optical (PO) sensor capability to MA so that it may be used by MA to detect aircraft through passive optical search and track. This sequence is only present if PO sensing capabilities exist in MS. MS publishes *PO_CapabilityMT* with at least one Capability specified. Multiple *Capability* fields are specified to advertise multiple command use cases. Each *Capability* must have a unique identifier set in *CapabilityID*. The *AcceptedInterface* field must specify at least *CAPABILITY_COMMAND*. The *CapabilityType* should either be *SEARCH_VOLUME* (for searching a volume of space to generate tracks) or *TRACK* (for updating a specified track). The *CapabilitiesOptions* field and sub-fields will be set by MS according to the ability of MS (e.g. *InterruptOtherActivities* and *CalibrateOnActivity* are descriptive of particular MS implementation). The *CapabilityFOR* field is required to specify the field of regard of the sensor PO capability.

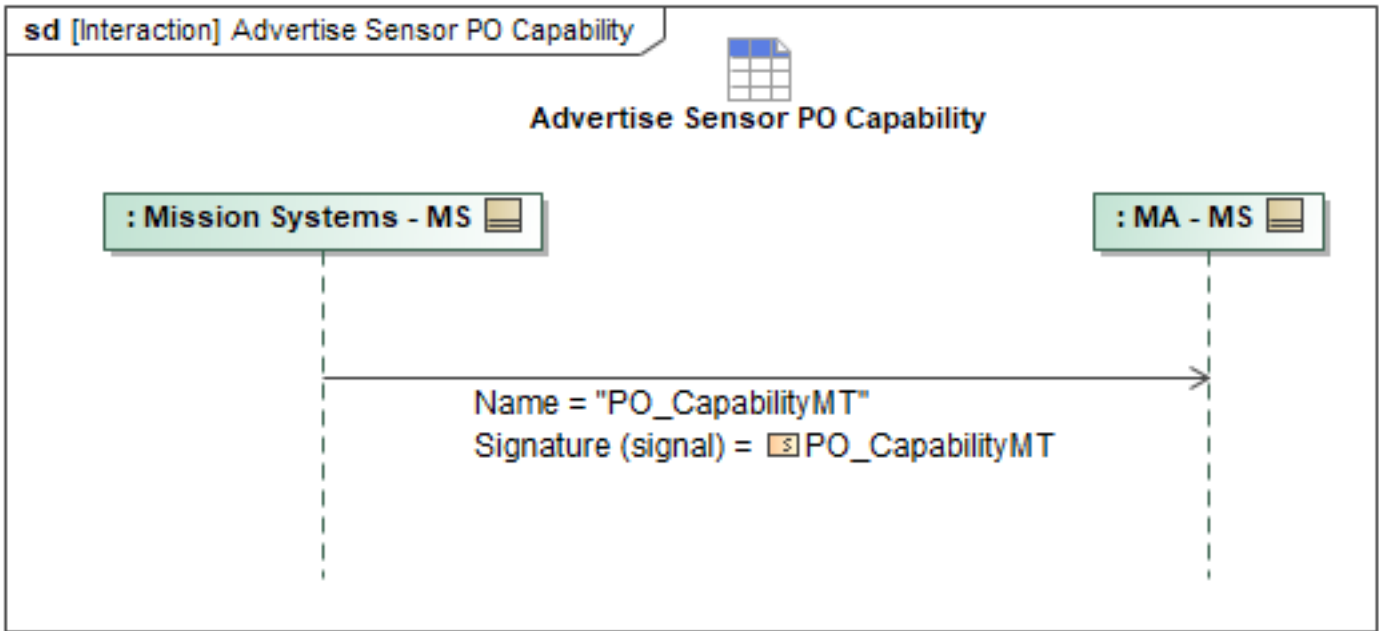


Figure 8-39: Advertise Sensor PO Capability

Table 8-37: Advertise Sensor PO Capability

Message Name (Name:Type)	Required Fields (Name:Type)
PO_CapabilityMT: PO_CapabilityMT	CapabilityFOR: FOR_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.4 Command Sensor AMTI Capability

Mission Use Case: AMTI

The Command Sensor AMTI Capability State sequence diagram specifies the interaction of MA commanding MS to schedule activity of the Airborne Moving Target Indicator (AMTI) sensor capability and then MS responding to the command with status. MA publishes *AMTI_CommandMT* with the Command choice set to *Capability* and a unique *CommandID*. The *CommandState* is set to *NEW* to schedule new AMTI capability execution, *UPDATE* to change a previous command (scheduled or not), and *CANCEL* to stop or remove a scheduled AMTI capability execution. The *CapabilityID* specified corresponds to one of the *CapabilityID* fields published by MS in *AMTI_CapabilityMT*. The *Ranking* field and sub-fields are set according to MS implementation and desired effect on MS scheduling priorities. The schema-optional field *TemporalConstraints* must be specified with select sub-fields. *StartTimeWindow* containing both *Begin* and *End* values must be specified to provide the MS scheduler a time window to start the AMTI capability execution. *EndTimeWindow* containing both *Begin* and *End* must be specified to provide the MS scheduler a time window to the end the AMTI capability execution.

The *TargetVolume* choice field is set based upon the *CapabilityType* of the *Capability* from *MA_AMTI_CapabilityMT* that the *AMTI_CommandMT* *CapabilityID* specifies. There should only be one *TargetVolume* specified. If the *CapabilityType* is *VOLUME* (for searching a volume of space to generate tracks), then *TargetVolume* must be set to *AirVolume* with either of the required

VolumeLocation choices used to specify a volume of space by sensor reference values or 3D LLA geometry. If the *CapabilityType* is TRACK (for updating a specified track), then *TargetVolume* must be set to *EntityID* with a unique identifier that both MA and MS are aware of corresponding to the track that should be updated by MS. No remaining schema-optional fields are required but may be specified for more information.

In response to the *AMTI_CommandMT* message sent from MA, MS sends back *AMTI_CommandStatusMT* first acknowledging that the command was received. The *CommandID* must match the *CommandID* set in the *AMTI_CommandMT* being responded to. *CommandProcessingState* must be set to *RECEIVED*. Following this, MS assesses the command and sends another *AMTI_CommandStatusMT* with the same *CommandID* but with *CommandProcessingState* set to *ACCEPTED* if MS has scheduled the AMTI command, *REJECTED* if MS rejected the command, or *CANCELED* if the *CommandState* in the *AMTI_CommandMT* is set to *CANCEL* and the command was approved (removing an AMTI capability execution from the schedule).

In addition, *AMTI_ActivityMT* is published periodically with the *SubsystemID* set to a unique identifier for MS or the MS subsystem responsible for AMTI command scheduling. The Activity sequence specified must be ordered temporally according to the AMTI command schedule, with earlier commands coming first. Each Activity has a unique *ActivityID* and the *CapabilityID* corresponds to one of the *Capability* fields specified in the *MA_AMTI_CapabilityMT* message published by MS. Interactive is set according to MS implementation (MS is only required to support *MA_AMTI_CapabilityMT* AcceptedInterface of *CAPABILITY_COMMAND*, which is not modifying activities, only scheduling or cancelling them). Each Activity *ActivityState* is populated appropriately as required by the schema. *EstimatedStartTime* and *EstimatedCompletionTime* are required to be populated, showing that capability commands are scheduled by MS as commanded.

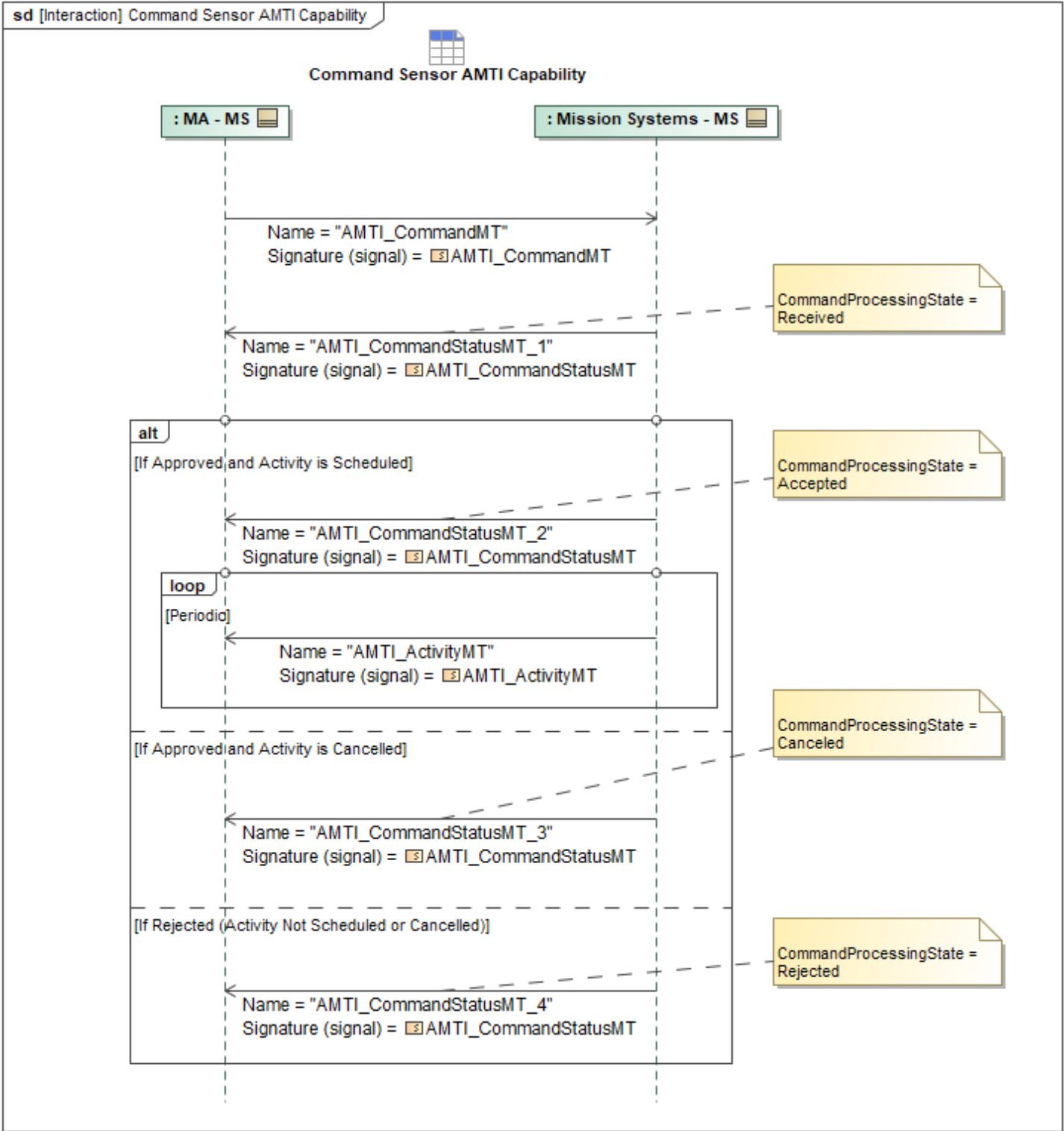


Figure 8-40: Command Sensor AMTI Capability

Table 8-38: Command Sensor AMTI Capability

Message Name (Name:Type)	Required Fields (Name:Type)
AMTI_ActivityMT: AMTI_ActivityMT	EstimatedStartTime: DateTimeType

Message Name (Name:Type)	Required Fields (Name:Type)
	EstimatedCompletionTime: DateTimeType
AMTI_CommandMT: AMTI_CommandMT	StartTimeWindow: DateTimeRangeType Begin: DateTimeType End: DateTimeType EndTimeWindow: DateTimeRangeType Capability: AMTI_CapabilityCommandType
AMTI_CommandStatusMT_1: AMTI_CommandStatusMT	None
AMTI_CommandStatusMT_2: AMTI_CommandStatusMT	None
AMTI_CommandStatusMT_3: AMTI_CommandStatusMT	None
AMTI_CommandStatusMT_4: AMTI_CommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.5 Command Sensor ESM Capability

Mission Use Case: ESM

The Command Sensor ESM Capability State sequence diagram below specifies the interaction of MA commanding MS to schedule activity of the Electronic Support Measures (ESM) sensor capability and then MS responding to the command with status. MA publishes *ESM_CommandMT* with the *Command* choice set to *Capability* and a unique *CommandID* (see “Advertise Sensor ESM Capability”). The *CommandState* is set to *NEW* to schedule new ESM capability execution, *UPDATE* to change a previous command (scheduled or not), and *CANCEL* to stop or remove a scheduled ESM capability execution. The *CapabilityID* specified corresponds to one of the *CapabilityID* fields published by MS in *ESM_CapabilityMT*. The *Ranking* field and sub-fields are set according to MS implementation and desired effect on MS scheduling priorities. The schema-optional field *TemporalConstraints* must be specified with select sub-fields. *StartTimeWindow* containing both *Begin* and *End* values must be specified to provide the MS scheduler a time window to start the ESM capability execution. *EndTimeWindow* containing both *Begin* and *End* must be specified to provide the MS scheduler a time window to the end the ESM capability execution.

The schema-optional *ESM_Location* choice field is required. The choice field is set based upon the *CapabilityType* of the *Capability* from *ESM_CapabilityMT* that the *ESM_CommandMT CapabilityID* specifies. If the *SubCapabilityType* is *ACQUISITION* (for searching a volume of space to generate tracks), then *ESM_Location* must be set to *ESM_AirVolume* with the required parameters to specify the volume. If the *CapabilityType* is *SINGLE_SUBSYSTEM_GEOLOCATION* or *MULTI_SUBSYSTEM_GEOLOCATION* (for locating a specific emitter), then *TargetLocationData* must be set to *EntityID* with a unique identifier that both MA and MS are aware of corresponding to the track that should be updated by MS. No remaining schema-optional fields are required but may be specified for more information.

In response to the *ESM_CommandMT* message sent from MA, MS sends back *ESM_CommandStatusMT* first acknowledging that the command was received. The *CommandID* must match the *CommandID* set in the *ESM_CommandMT* being responded to. *CommandProcessingState* must be set to *RECEIVED*. Following this, MS assesses the command and sends another *ESM_CommandStatusMT* with the same *CommandID* but with *CommandProcessingState* set to *ACCEPTED* if MS has scheduled the ESM command, *REJECTED* if MS rejected the command, or *CANCELED* if the *CommandState* in the *ESM_CommandMT* is set to *CANCEL* and the command was approved (removing an ESM capability execution from the schedule).

In addition, *ESM_ActivityMT* is published periodically with the *SubsystemID* set to a unique identifier for MS or the MS subsystem responsible for ESM command scheduling. The *Activity* sequence specified must be ordered temporally according to the ESM command schedule, with earlier commands coming first. Each *Activity* has a unique *ActivityID* and the *CapabilityID* corresponds to one of the *Capability* fields specified in the *ESM_CapabilityMT* message published by MS. *Interactive* is set according to MS implementation (MS is only required to support *ESM_CapabilityMT AcceptedInterface* of *CAPABILITY_COMMAND*, which is not modifying activities, only scheduling or canceling them). Each *Activity* *ActivityState* is populated appropriately as required by the schema. *EstimatedStartTime* and *EstimatedCompletionTime* are required to be populated, showing that capability commands are scheduled by MS as commanded. The rest of the schema-optional fields are not required but may be specified for more information.

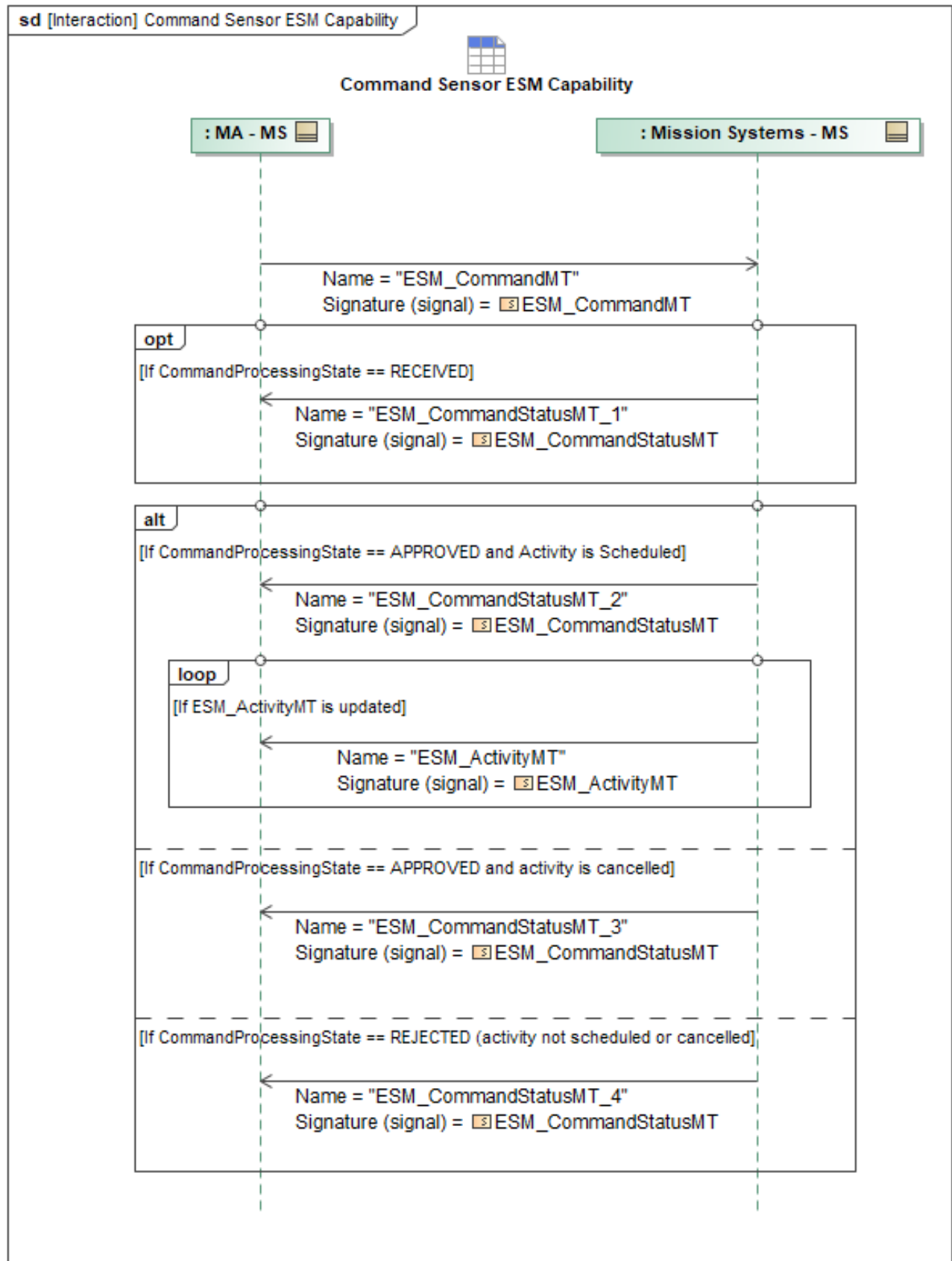


Figure 8-41: Command Sensor ESM Capability

Table 8-39: Command Sensor ESM Capability

Message Name (Name:Type)	Required Fields (Name:Type)
ESM_ActivityMT: ESM_ActivityMT	EstimatedStartTime: DateTimeType EstimatedCompletionTime: DateTimeType
ESM_CommandMT: ESM_CommandMT	ESM_Location: ESM_LocationType EndTimeWindow: DateTimeRangeType StartTimeWindow: DateTimeRangeType Capability: PO_CapabilityCommandType Begin: DateTimeType End: DateTimeType
ESM_CommandStatusMT_1: ESM_CommandStatusMT	None
ESM_CommandStatusMT_2: ESM_CommandStatusMT	None
ESM_CommandStatusMT_3: ESM_CommandStatusMT	None
ESM_CommandStatusMT_4: ESM_CommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.6 Command Sensor PO Capability

Mission Use Case: PO

The Command Sensor PO Capability State sequence diagram specifies the interaction of MA commanding MS to schedule activity of the Passive Optical (PO) sensor capability and then MS responding to the command with status. MA publishes *PO_CommandMT* with the Command choice set to *Capability* and a unique *CommandID* (see “Advertise Sensor PO Capability”). The *CommandState* is set to *NEW* to schedule new PO capability execution, *UPDATE* to change a previous command (scheduled or not), and *CANCEL* to stop or remove a scheduled PO capability execution. The *CapabilityID* specified corresponds to one of the *CapabilityID* fields published by MS in *PO_CapabilityMT*. The *Ranking* field and sub-fields are set according to MS implementation and desired effect on MS scheduling priorities. The schema-optional field *TemporalConstraints* must be specified with select sub-fields. *StartTimeWindow* containing both *Begin* and *End* values must be specified to provide the MS scheduler a time window to start the PO capability execution. *EndTimeWindow* containing both *Begin* and *End* must be specified to provide the MS scheduler a time window to the end the PO capability execution.

The schema-optional *Pointing* choice field is required and set to the *Volume* choice type. There should be only one *Volume* specified, and the *Volume* choice field is set based upon the *CapabilityType* of the Capability from *PO_CapabilityMT* that the *PO_CommandMT* *CapabilityID* specifies. If the *CapabilityType* is *SEARCH_VOLUME* (for searching a volume of space to generate tracks), then *Volume* must be set to *AirVolume* with either of the required choices used to specify a volume of space by sensor reference values or 3D LLA geometry. If the *CapabilityType* is *TRACK* (for

updating a specified track), then *Volume* must be set to *EntityID* with a unique identifier that both MA and MS are aware of corresponding to the track that should be updated by MS. No remaining schema-optional fields are required but may be specified for more information.

In response to the *PO_CommandMT* message sent from MA, MS sends back *PO_CommandStatusMT* first acknowledging that the command was received. The *CommandID* must match the *CommandID* set in the *PO_CommandMT* being responded to. *CommandProcessingState* must be set to RECEIVED. Following this, MS assesses the command and sends another *PO_CommandStatusMT* with the same *CommandID* but with *CommandProcessingState* set to ACCEPTED if MS has scheduled the PO command, REJECTED if MS rejected the command, or CANCELED if the *CommandState* in the *PO_CommandMT* is set to CANCEL and the command was approved (removing an PO capability execution from the schedule).

In addition, *PO_ActivityMT* is published periodically with the *SubsystemID* set to a unique identifier for MS or the MS subsystem responsible for PO command scheduling. The Activity sequence specified must be ordered temporally according to the PO command schedule, with earlier commands coming first. Each Activity has a unique *ActivityID* and the *CapabilityID* corresponds to one of the *Capability* fields specified in the *PO_CapabilityMT* message published by MS. *Interactive* is set according to MS implementation (MS is only required to support *PO_CapabilityMT AcceptedInterface* of *CAPABILITY_COMMAND*, which is not modifying activities, only scheduling or cancelling them). Each Activity *ActivityState* is populated appropriately as required by the schema. *EstimatedStartTime* and *EstimatedCompletionTime* are required to be populated, showing that capability commands are scheduled by MS as commanded.

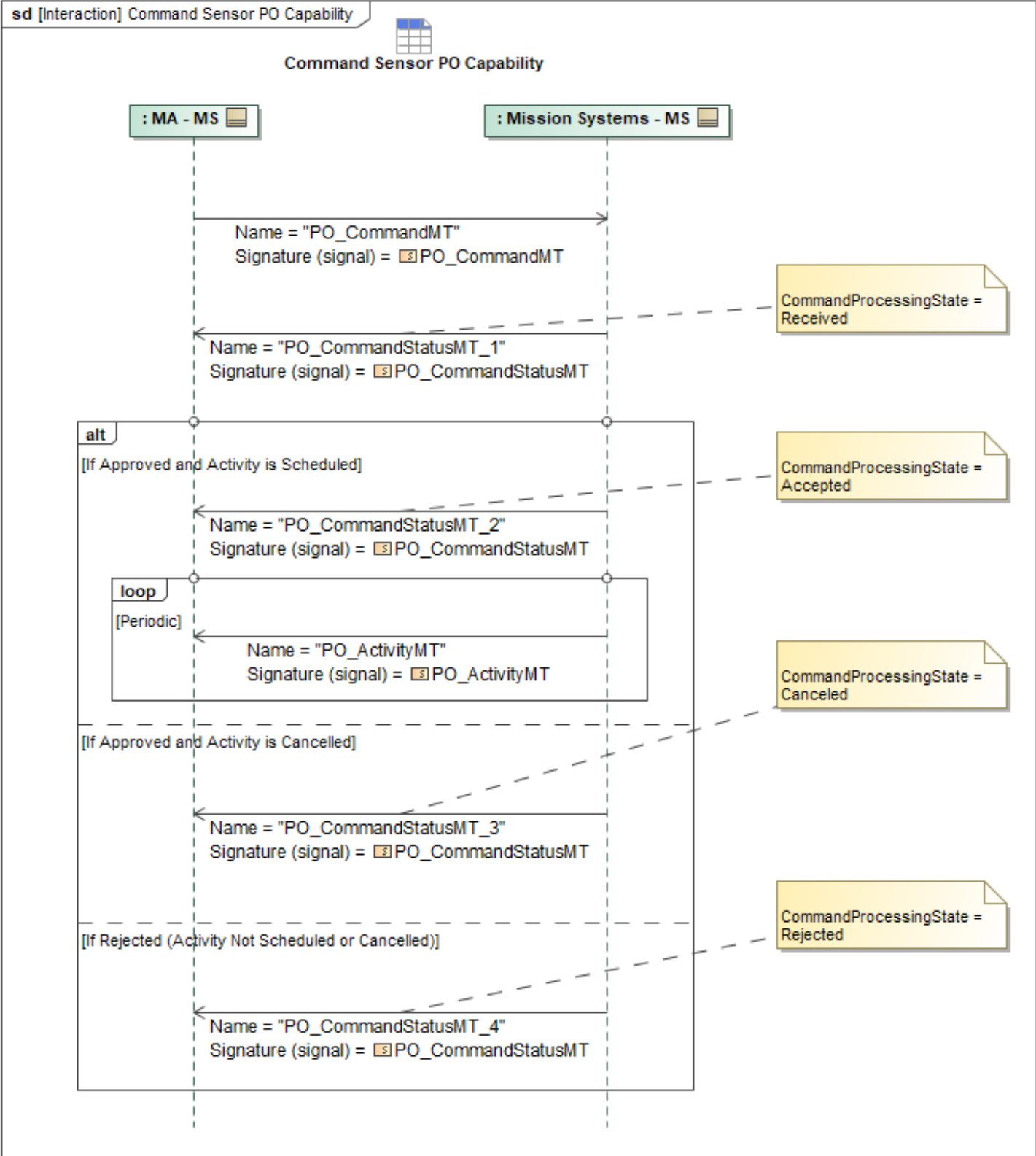


Figure 8-42: Command Sensor PO Capability

Table 8-40: Command Sensor PO Capability

Message Name (Name:Type)	Required Fields (Name:Type)
PO_ActivityMT: PO_ActivityMT	None
PO_CommandMT: PO_CommandMT	Begin: DateTimeType Pointing: PO_TargetType Capability: PO_CapabilityCommandType End: DateTimeType
PO_CommandStatusMT_1: PO_CommandStatusMT	None
PO_CommandStatusMT_2: PO_CommandStatusMT	None
PO_CommandStatusMT_3: PO_CommandStatusMT	None
PO_CommandStatusMT_4: PO_CommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.7 Publish External Weather Reports

MS can request weather data from off-board, periodically-updated weather services through comms channels (ex: XM weather from satellite). MA can initiate a request of that data by sending MS *DataUpdateRequestMT* with the *RequestType* set to *Query*, *Query.MessageType* set to *WEATHER*, and *Query.Query* included. The query will contain details for specifics of the off-board weather data being requested.

MS will respond with *DataUpdateRequestStatusMTs* to confirm that it has received the *DataUpdateRequestMT* from MA and to subsequently provide updates on the state of the request using the *RequestProcessingState* field. Finally, once MS has received the updated weather data, it will send it to MA as a *WeatherMT* message, and send a final *DataUpdateRequestStatusMT* with *RequestProcessingState* set to *COMPLETED*. If MS was unable to obtain the updated weather data, it will not send *WeatherMT*, and the final *DataUpdateRequestStatusMT* will contain *RequestProcessingState* set to *FAILED*.

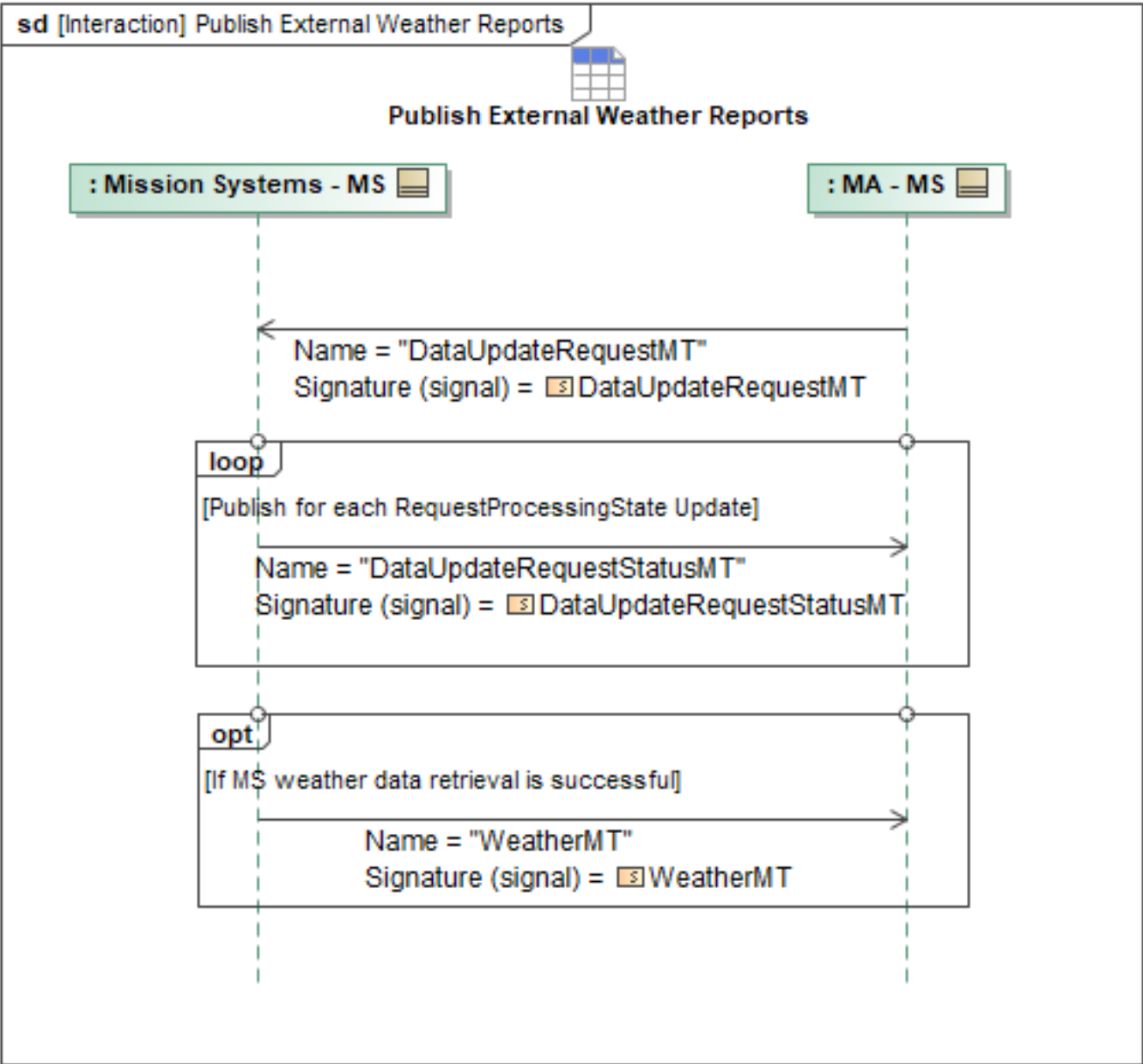


Figure 8-43: Publish External Weather Reports

Table 8-41: Publish External Weather Reports

Message Name (Name:Type)	Required Fields (Name:Type)
DataUpdateRequestMT: DataUpdateRequestMT	MessageType: MessageTypeEnum Query: QueryPET
DataUpdateRequestStatusMT: DataUpdateRequestStatusMT	None
WeatherMT: WeatherMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

8.2.8.8 Report Sensor AMTI Capability State

Mission Use Case: AMTI

The Report Sensor AMTI Capability State sequence diagram specifies the interaction of MS communicating the Airborne Moving Target Indicator (AMTI) sensor capability availability and scheduled activity to MA. This sequence is only present if AMTI sensing capabilities exist in MS (see “Advertise Sensor AMTI Capability for Search and Track”). MS periodically publishes *AMTI_CapabilityStatusMT* and *AMTI_ActivityMT* to MA. MS also publishes *AMTI_CapabilityStatusMT* asynchronously in response to changes in the AMTI capability availability to MA (e.g. sensors are in an unusable state). MS also publishes *AMTI_ActivityMT* asynchronously in response to changes in the scheduled AMTI capability commands MS will execute (e.g. a new command was accepted).

AMTI_CapabilityStatusMT is published with at least one *CapabilityStatus* specified. Each *CapabilityStatus* must have a unique identifier set in *CapabilityID* that corresponds to a *Capability* included in *MA_AMTI_CapabilityMT* previously published by MS. The *Availability* enumeration is used to specify if MA can command this capability. *Availability* of *AVAILABLE* signifies that the capability is functioning, and MA can issue commands. All other *Availability* enumerations signify that MA cannot issue commands. No schema-optional fields are required but may be specified for more information.

AMTI_ActivityMT is published with the *SubsystemID* set to a unique identifier for MS or the MS subsystem responsible for AMTI command scheduling. The *Activity* sequence specified must be ordered temporally according to the AMTI command schedule, with earlier commands coming first. Each *Activity* has a unique *ActivityID* and the *CapabilityID* corresponds to one of the *Capability* fields specified in the *MA_AMTI_CapabilityMT* message published by MS. *Interactive* is set according to MS implementation (MS is only required to support *MA_AMTI_CapabilityMT AcceptedInterface* of *CAPABILITY_COMMAND*, which is not modifying activities, only scheduling or cancelling them). Each *Activity* *ActivityState* is populated appropriately as required by the schema. *EstimatedStartTime* and *EstimatedCompletionTime* are required to be populated, showing that capability commands are scheduled by MS as commanded.

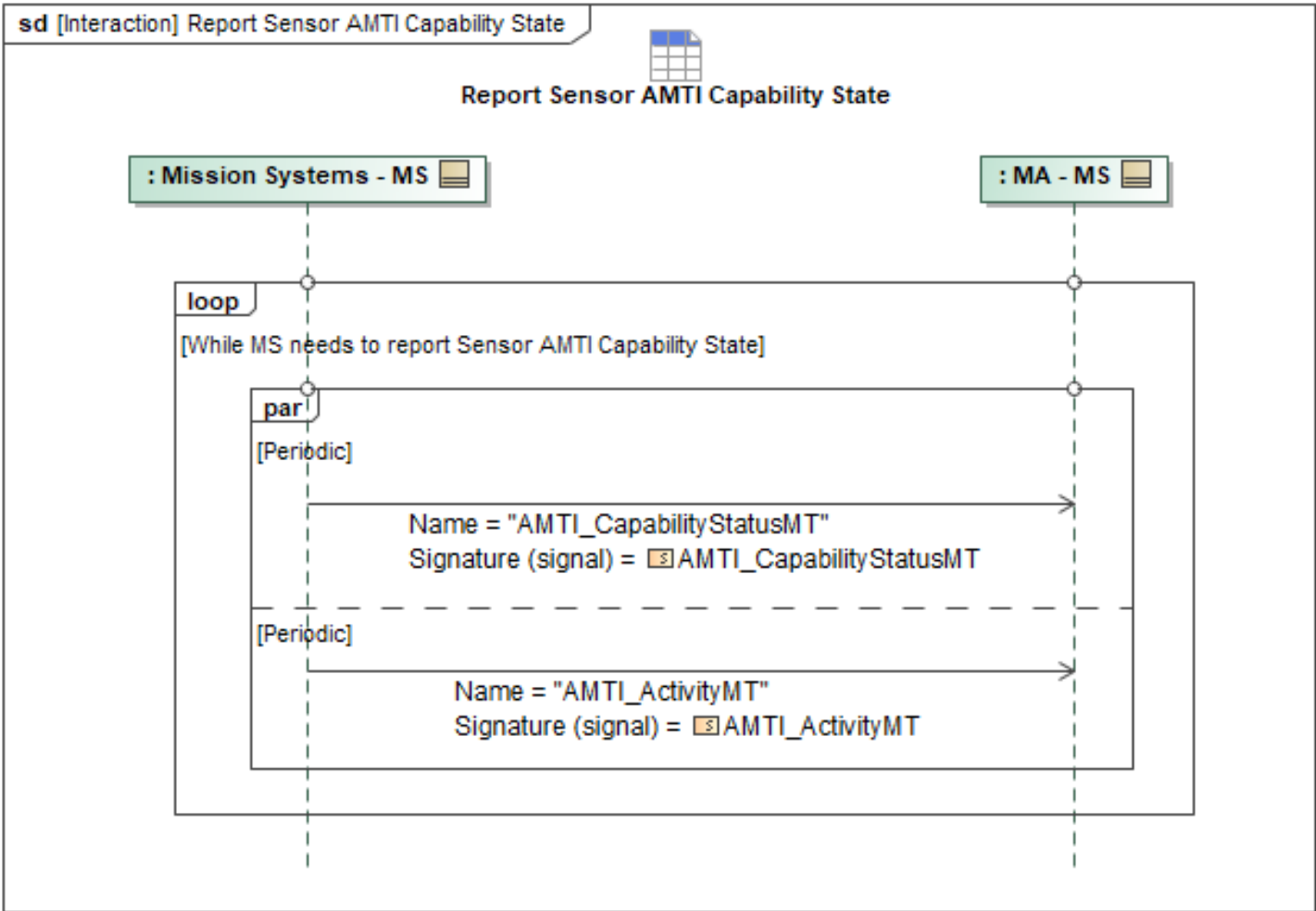


Figure 8-44: Report Sensor AMTI Capability State

Table 8-42: Report Sensor AMTI Capability State

Message Name (Name:Type)	Required Fields (Name:Type)
AMTI_ActivityMT: AMTI_ActivityMT	EstimatedCompletionTime: DateTimeType EstimatedStartTime: DateTimeType
AMTI_CapabilityStatusMT: AMTI_CapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.9 Report Sensor ESM Capability State

Mission Use Case: ESM

The Report Sensor ESM Capability State sequence diagram below specifies the interaction of MS communicating the Electronic Support Measures (ESM) sensor capability availability and scheduled activity to MA. This sequence is only present if ESM sensing capabilities exist in MS (see “Advertise Sensor ESM Capability for Search and Track”). MS publishes *ESM_ActivityMT* asynchronously in

response to changes in the scheduled ESM capability commands MS will execute (e.g. a new command was accepted).

ESM_ActivityMT is published with the *SubsystemID* set to a unique identifier for MS or the MS subsystem responsible for ESM command scheduling. The *Activity* sequence specified must be ordered temporally according to the ESM command schedule, with earlier commands coming first. Each *Activity* has a unique *ActivityID*. The *CapabilityID* corresponds to one of the *Capability* fields specified in the *ESM_CapabilityMT* message published by MS. *Interactive* is set according to MS implementation (MS is only required to support *ESM_CapabilityMT AcceptedInterface* of *CAPABILITY_COMMAND*, which is not modifying activities, only scheduling or cancelling them). Each *Activity* *ActivityState* is populated appropriately as required by the schema. *EstimatedStartTime* and *EstimatedCompletionTime* are required to be populated, showing that capability commands are scheduled by MS as commanded. The rest of the schema-optional fields are not required but may be specified for more information.

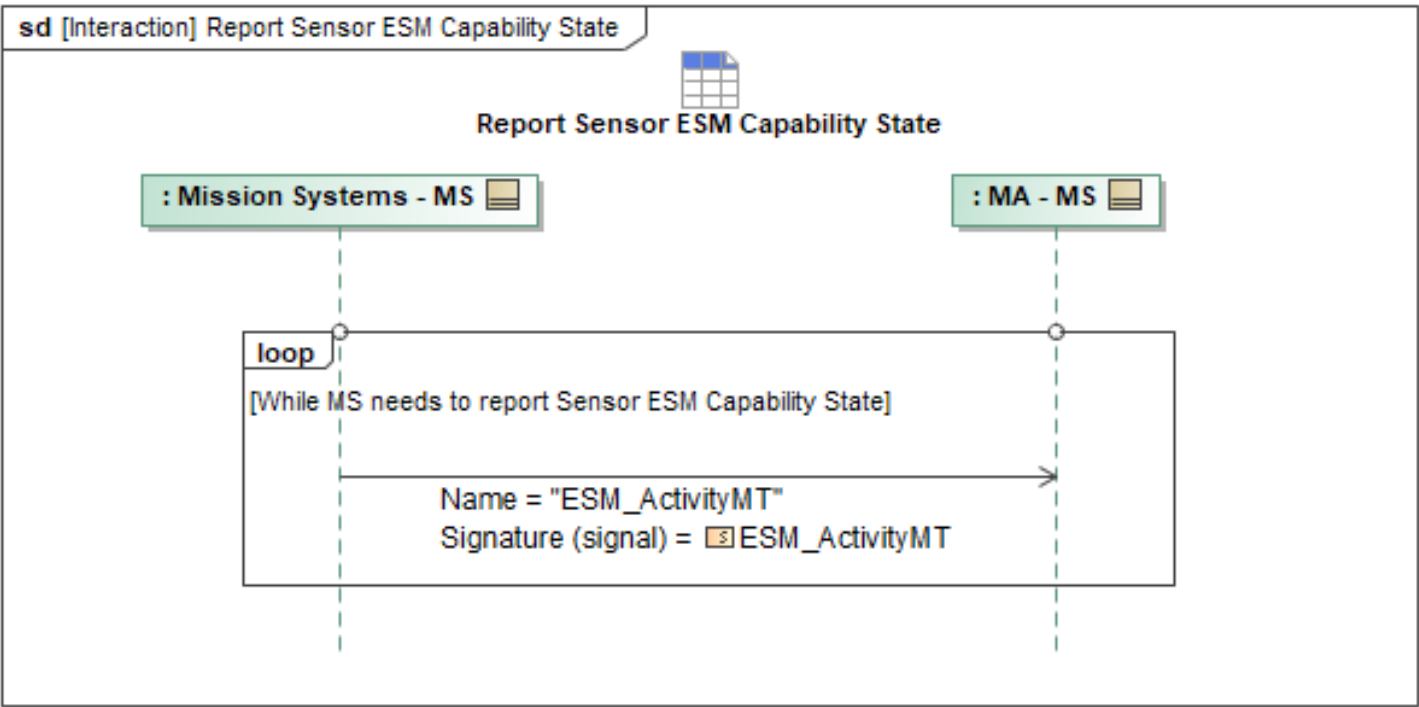


Figure 8-45: Report Sensor ESM Capability State

Table 8-43: Report Sensor ESM Capability State

Message Name (Name:Type)	Required Fields (Name:Type)
ESM_ActivityMT: ESM_ActivityMT	EstimatedStartTime: DateTimeType EstimatedCompletionTime: DateTimeType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.10 Report Sensor PO Capability State

Mission Use Case: PO

The Report Sensor PO Capability State sequence diagram specifies the interaction of MS communicating the Passive Optical (PO) sensor capability availability and scheduled activity to MA. This sequence is only present if PO sensing capabilities exist in MS (see “Advertise Sensor PO Capability for Search and Track”). MS periodically publishes *PO_CapabilityStatusMT* and *PO_ActivityMT* to MA. MS also publishes *PO_CapabilityStatusMT* asynchronously in response to changes in the PO capability availability to MA (e.g. sensors are in an unusable state). MS also publishes *PO_ActivityMT* asynchronously in response to changes in the scheduled PO capability commands MS will execute (e.g. a new command was accepted).

PO_CapabilityStatusMT is published with at least one *CapabilityStatus* specified. Each *CapabilityStatus* must have a unique identifier set in *CapabilityID* that corresponds to a *Capability* included in *PO_CapabilityMT* previously published by MS. The *Availability* enumeration is used to specify if MA can command this capability. *Availability* of *AVAILABLE* signifies that the capability is functioning, and MA can issue commands. All other *Availability* enumerations signify that MA cannot issue commands. No schema-optional fields are required but may be specified for more information.

PO_ActivityMT is published with the *SubsystemID* set to a unique identifier for MS or the MS subsystem responsible for PO command scheduling. The *Activity* sequence specified must be ordered temporally according to the PO command schedule, with earlier commands coming first. Each *Activity* has a unique *ActivityID*. The *CapabilityID* corresponds to one of the *Capability* fields specified in the *PO_CapabilityMT* message published by MS. *Interactive* is set according to MS implementation (MS is only required to support *PO_CapabilityMT AcceptedInterface* of *CAPABILITY_COMMAND*, which is not modifying activities, only scheduling or cancelling them). Each *Activity* *ActivityState* is populated appropriately as required by the schema. *EstimatedStartTime* and *EstimatedCompletionTime* are required to be populated, showing that capability commands are scheduled by MS as commanded.

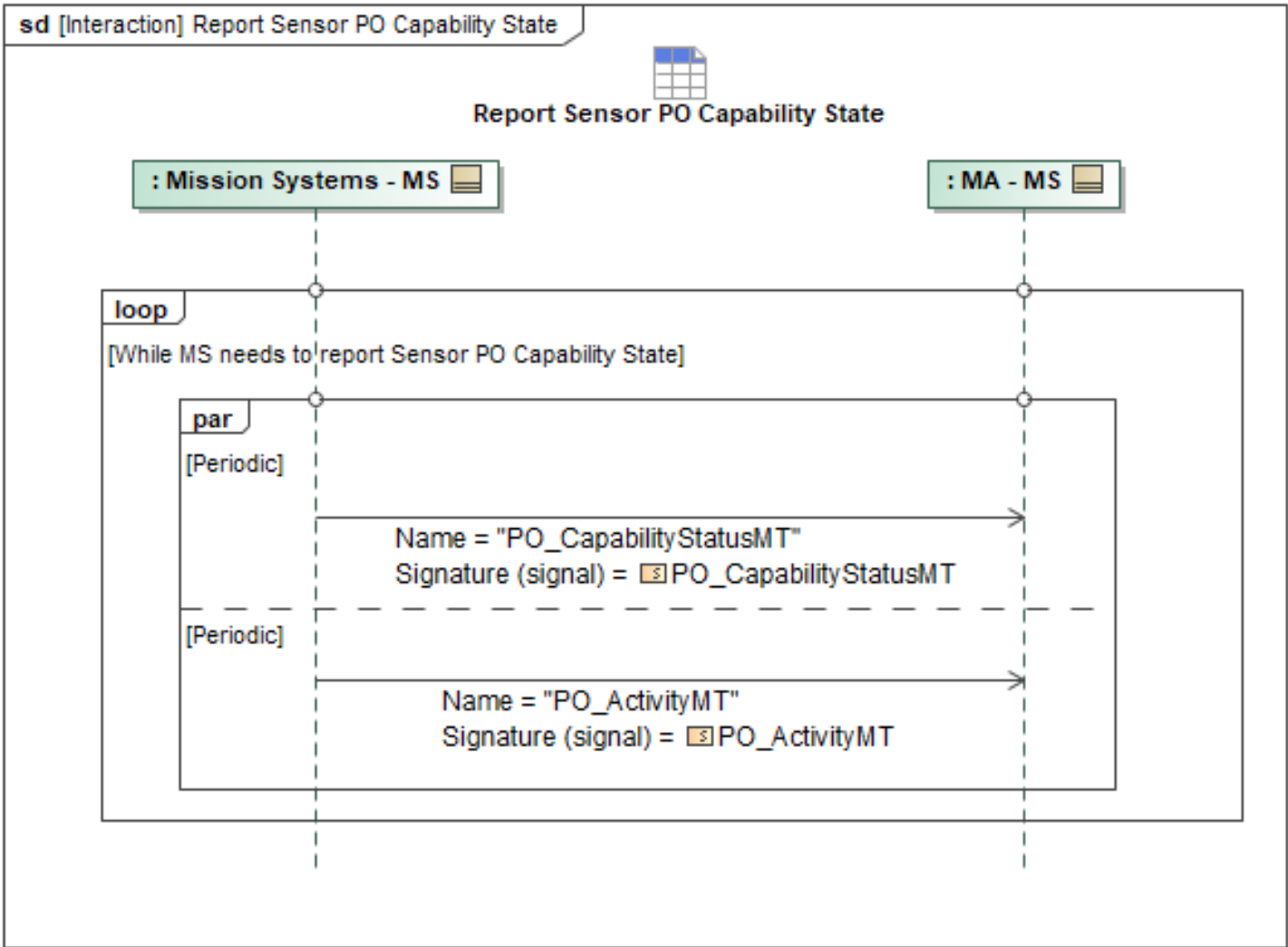


Figure 8-46: Report Sensor PO Capability State

Table 8-44: Report Sensor PO Capability State

Message Name (Name:Type)	Required Fields (Name:Type)
PO_ActivityMT: PO_ActivityMT	None
PO_CapabilityStatusMT: PO_CapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.8.11 Report Weather Radar Capability Observations

Mission Use Case: Weather Radar

If a platform has a weather radar capability, *WeatherRadarObservationMT* will be used to to send weather data collected by the radar from MS to MA.

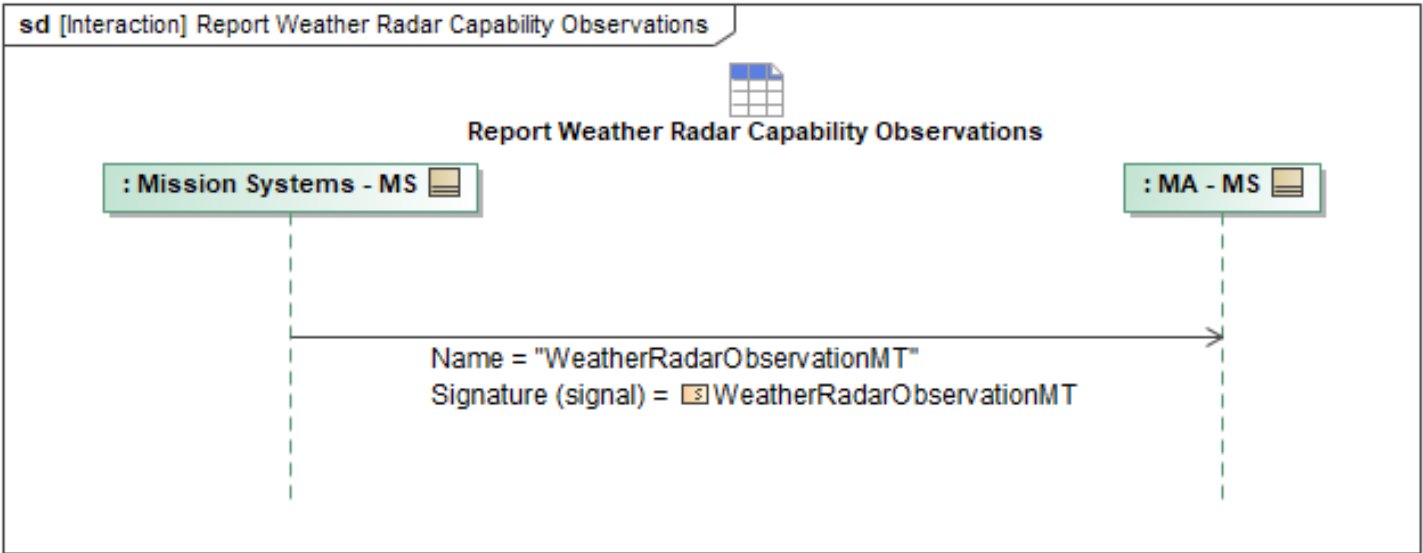


Figure 8-47: Report Weather Radar Capability Observations

Table 8-45: Report Weather Radar Capability Observations

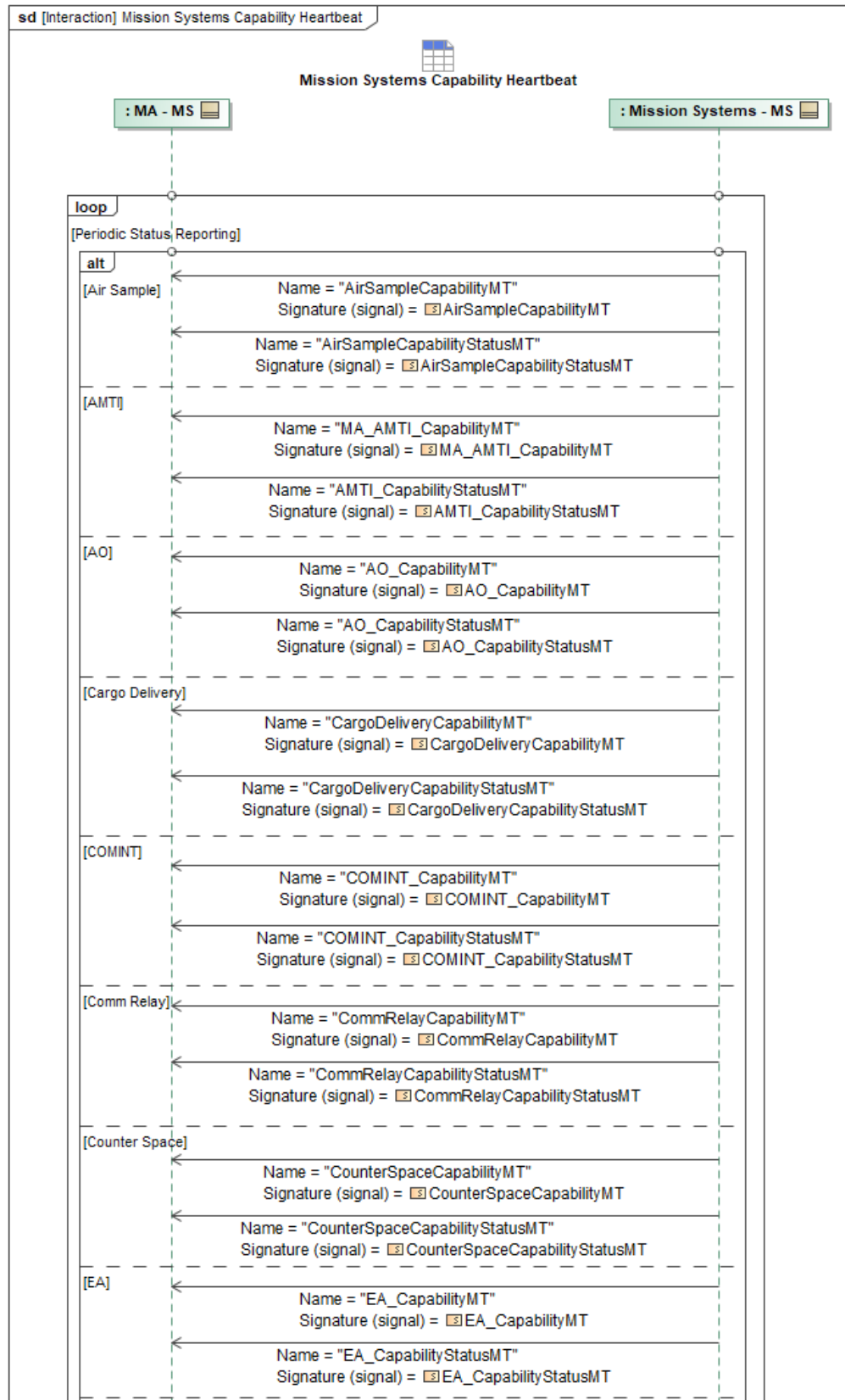
Message Name (Name:Type)	Required Fields (Name:Type)
WeatherRadarObservationMT: WeatherRadarObservationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.9 Status Service Package

8.2.9.1 Mission Systems Capability Heartbeat

MS periodically sends **CapabilityMT* and **CapabilityStatusMT* messages to MA to describe the capabilities that MS is generally able to complete and those capabilities that are currently available for MS to execute, respectively. This periodic "heartbeat" continues throughout the duration of the mission. As these messages are published periodically, MA checks the provided status messages against the Minimum Essential Capabilities List (MECL) to identify potential contingencies.



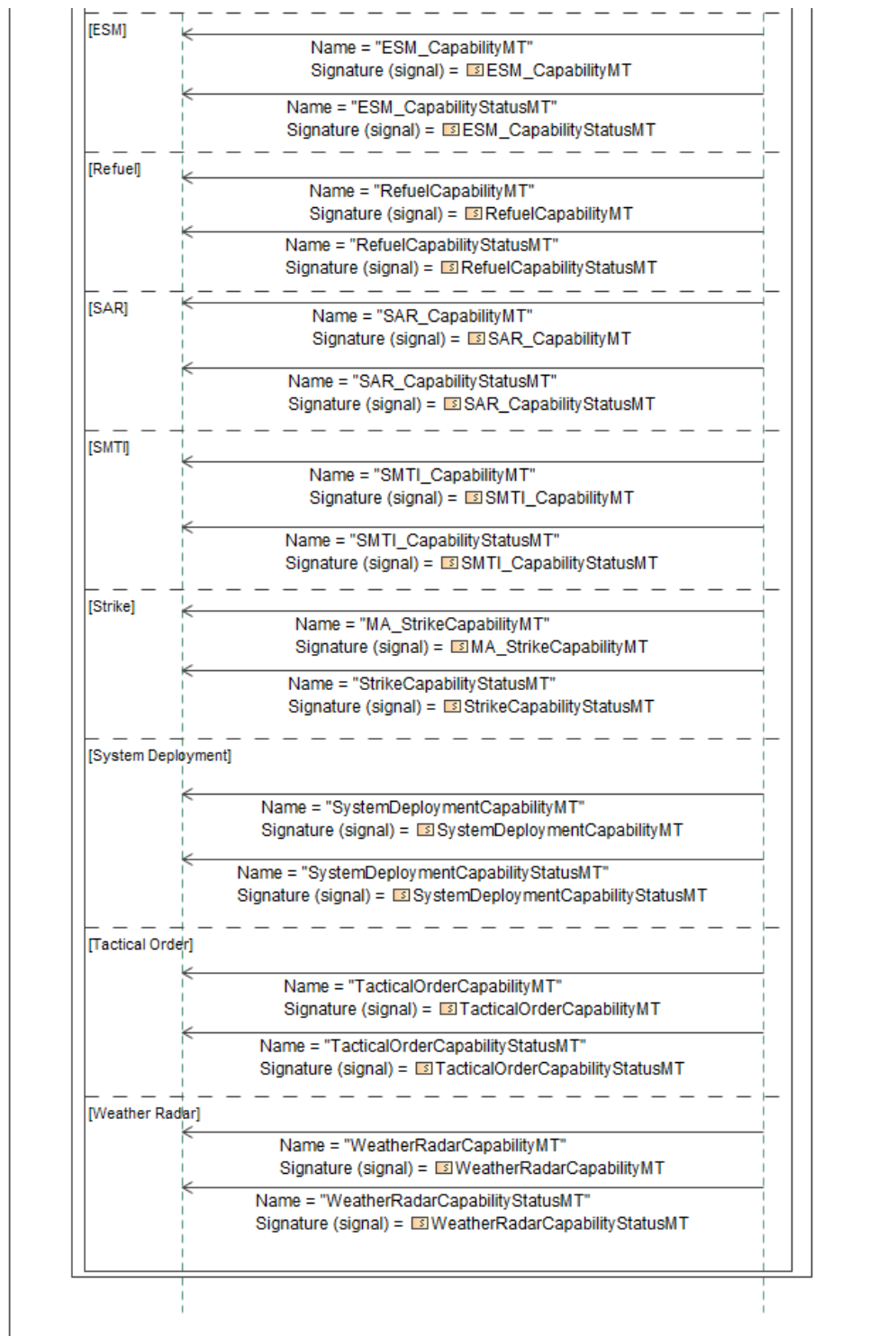


Figure 8-48: Mission Systems Capability Heartbeat

Table 8-46: Mission Systems Capability Heartbeat

Message Name (Name:Type)	Required Fields (Name:Type)
AirSampleCapabilityMT: AirSampleCapabilityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
AirSampleCapabilityStatusMT: AirSampleCapabilityStatusMT	None
AMTI_CapabilityStatusMT: AMTI_CapabilityStatusMT	None
AO_CapabilityMT: AO_CapabilityMT	None
AO_CapabilityStatusMT: AO_CapabilityStatusMT	None
CargoDeliveryCapabilityMT: CargoDeliveryCapabilityMT	None
CargoDeliveryCapabilityStatusMT: CargoDeliveryCapabilityStatusMT	None
COMINT_CapabilityMT: COMINT_CapabilityMT	None
COMINT_CapabilityStatusMT: COMINT_CapabilityStatusMT	None
CommRelayCapabilityMT: CommRelayCapabilityMT	None
CommRelayCapabilityStatusMT: CommRelayCapabilityStatusMT	None
CounterSpaceCapabilityMT: CounterSpaceCapabilityMT	None
CounterSpaceCapabilityStatusMT: CounterSpaceCapabilityStatusMT	None
EA_CapabilityMT: EA_CapabilityMT	None
EA_CapabilityStatusMT: EA_CapabilityStatusMT	None
ESM_CapabilityMT: ESM_CapabilityMT	None
ESM_CapabilityStatusMT: ESM_CapabilityStatusMT	None
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	None
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None
RefuelCapabilityMT: RefuelCapabilityMT	None
RefuelCapabilityStatusMT: RefuelCapabilityStatusMT	None
SAR_CapabilityMT: SAR_CapabilityMT	None
SAR_CapabilityStatusMT: SAR_CapabilityStatusMT	None
SMTI_CapabilityMT: SMTI_CapabilityMT	None
SMTI_CapabilityStatusMT: SMTI_CapabilityStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	None
SystemDeploymentCapabilityMT: SystemDeploymentCapabilityMT	None
SystemDeploymentCapabilityStatusMT: SystemDeploymentCapabilityStatusMT	None
TacticalOrderCapabilityMT: TacticalOrderCapabilityMT	None
TacticalOrderCapabilityStatusMT: TacticalOrderCapabilityStatusMT	None
WeatherRadarCapabilityMT: WeatherRadarCapabilityMT	None
WeatherRadarCapabilityStatusMT: WeatherRadarCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.9.2 MS Responds to Query for Capabilities

Mission Systems Capability Messages are delivered from MS to MA in the Mission Systems Capability Heartbeat. This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *QueryDataRequestStatusMT* messages with *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and "PROCESSING".

When complete, the *QueryDataRequestStatusMT* message's *RequestProcessingStateEnum* is set to "COMPLETED" and contains the requested data.

When the original *QueryDataRequestMT*'s optional field *ResultsInNative* is populated with the matching message type, the underlying data management services instead publishes the requested data in its original or raw format as a separate message and indicates *QueryDataRequestStatusMT*.

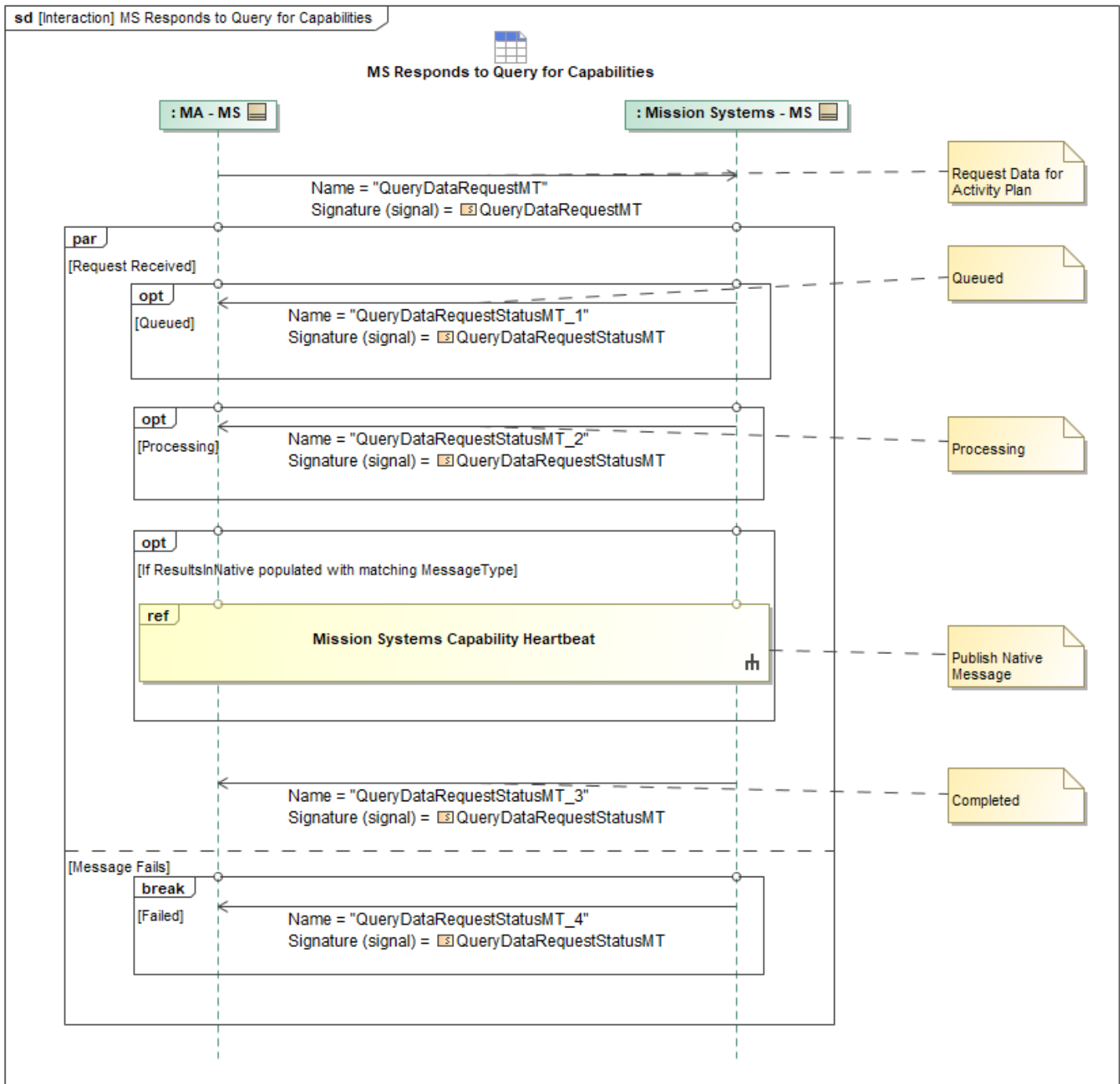


Figure 8-49: MS Responds to Query for Capabilities

Table 8-47: MS Responds to Query for Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.9.3 Publish Periodic Passive Optical Sensor Configuration

Mission Use Case: PO

The following sequence shows the interface between MS and MA to support Sensor Configurations. To enable MA to be platform agnostic, MA does not need to have a priori knowledge of the vehicle and its subsystems. This is achieved via the *PO_CapabilityMT* message that "advertises the mostly static characteristics of a mission-ready Passive Optical Capability (of a Subsystem of System) so that it can be "discovered" and used. It is assumed that the underlying sensor subsystem routinely publishes the *PO_CapabilityMT* and *PO_CapabilityStatusMT* messages, which proceed to MA through the MS L1 interface. The Passive Optical sensing capability is exemplified in this sequence but exchanges following the same pattern are expected for all MS-based Capabilities.

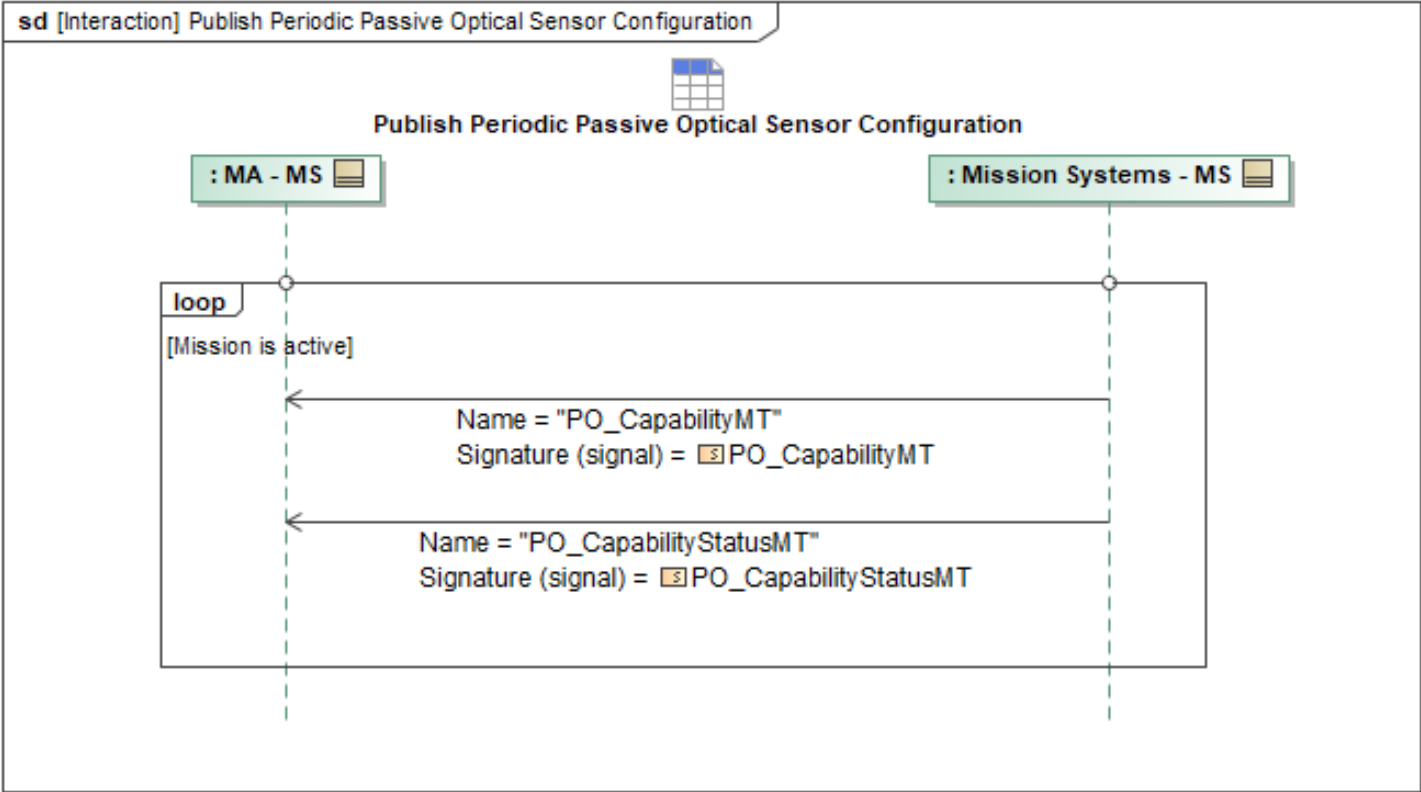


Figure 8-50: Publish Periodic Passive Optical Sensor Configuration

Table 8-48: Publish Periodic Passive Optical Sensor Configuration

Message Name (Name:Type)	Required Fields (Name:Type)
PO_CapabilityMT: PO_CapabilityMT	DomainPairing: EnvironmentPairingEnum PointingCapability: PointingTypeEnum PackageOperation: PackageOperationEnum Lens: PO_ComponentLensAssemblyType AvailableMDF: MDF_Type MessageOutput: PO_MessageOutputsEnum CapabilityFOR: FOR_Type TrackControlOptions: PO_TrackCapabilityTypeEnum PO_Component: PO_ComponentType TrackingRange: DoubleMinMaxType ATR_Filter: IdentityBasicType
PO_CapabilityStatusMT: PO_CapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.9.4 Receive Periodic Health, Status, & BIT Information

The Mission System reports subsystem status, service status, and subsystem BIT status continuously, at a configurable periodic rate, after initializing. Mission Autonomy uses these messages to verify the Mission System is operational. Subsystems contained within the Mission System will relay status information to Mission Autonomy using the *SubsystemStatusMT* message. The *SubsystemStatusMT* contains the fields *SubsystemID*, *SubsystemState* and other optional fields in order to indicate the subsystem's current state. Subsystems can also report periodic BIT information to Mission Autonomy using the *SubsystemBIT_StatusMT* message. BIT or Built In Test is a function of hardware and software to test its functionality to verify it is operating as intended and the *SubsystemBIT_StatusMT* message facilitates the reporting of this information to Mission Autonomy. The *SubsystemBIT_StatusMT* contains the fields *ActiveBIT*, *CompletedBIT* and *Fault* to allow the implementer to report the progress of an active BIT and information about completed BITs. Finally, the Mission System can report the status of services inside the subsystem using the *ServiceStatusMT* message. The *ServiceStatusMT* message contains the fields *TimeUp*, *ServiceStatus* and other optional fields that provide context on the health of the service. This periodic interaction allows Mission Autonomy to understand the current state of the Missions System subsystems and services.

The optional *ServiceConfigurationParamsMT* message informs Mission Autonomy which service specific configuration values can be manipulated. Mission Autonomy may use the *ServiceConfigurationChangeRequestMT* message to update any of the configurable parameters identified in the *ServiceConfigurationParamsMT*. If a service does not provide a *ServiceConfigurationParamsMT* it is assumed that the service will not respond to *ServiceConfigurationChangeRequestMT* messages. A configurable Service will respond to the Mission Autonomy with a *ServiceConfigurationChangeRequestStatusMT* which should indicate if the configuration change succeed (*RequestProcessingState* = "COMPLETED") or failed (*RequestProcessingState* = "FAILED").

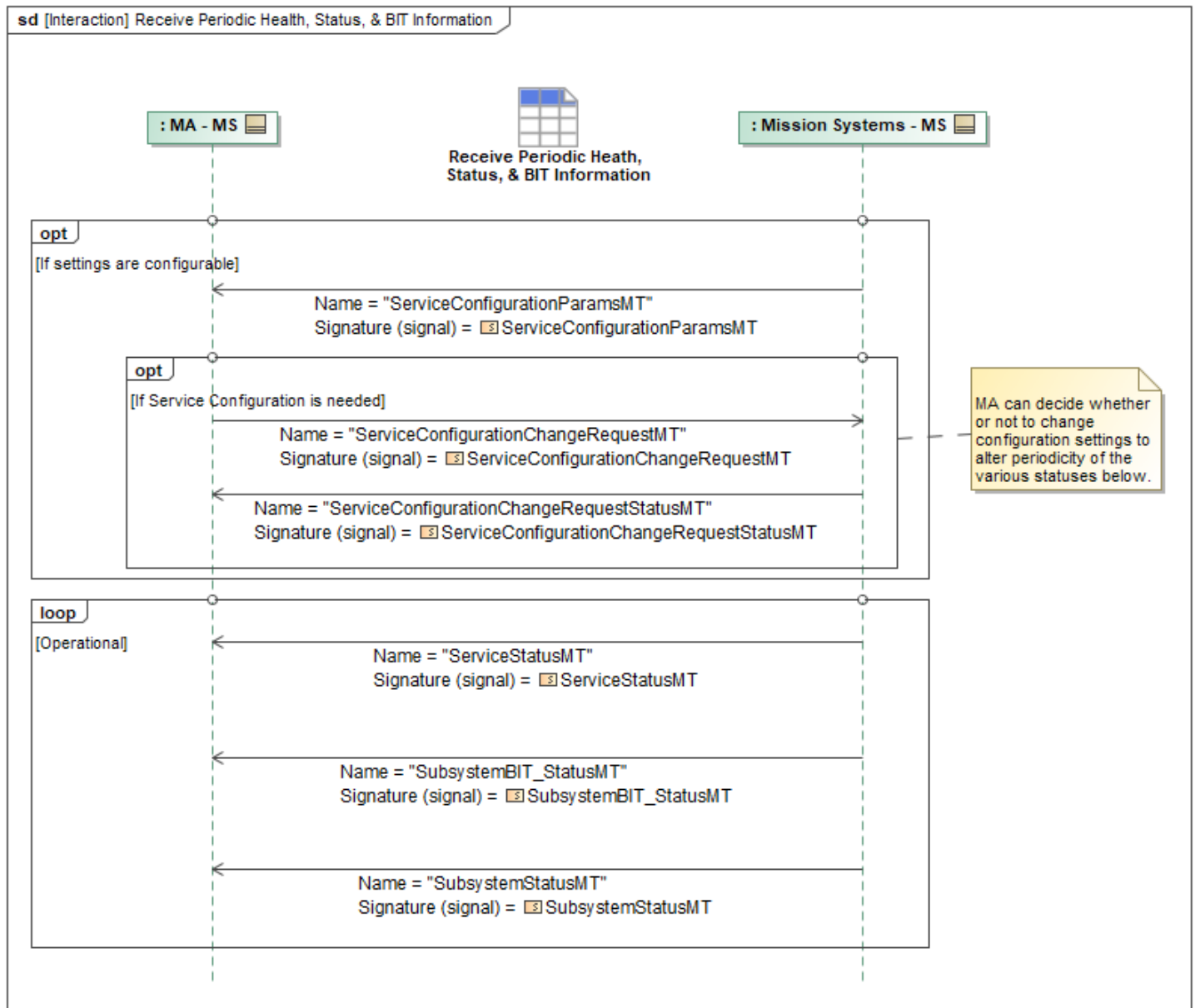


Figure 8-51: Receive Periodic Health, Status, & BIT Information

Table 8-49: Receive Periodic Health, Status, & BIT Information

Message Name (Name:Type)	Required Fields (Name:Type)
ServiceConfigurationChangeRequestMT: ServiceConfigurationChangeRequestMT	None
ServiceConfigurationChangeRequestStatusMT: ServiceConfigurationChangeRequestStatusMT	None
ServiceConfigurationParamsMT: ServiceConfigurationParamsMT	None
ServiceStatusMT: ServiceStatusMT	None
SubsystemBIT_StatusMT: SubsystemBIT_StatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.9.5 Request Mission System Health, Status & BIT Information

MA can request health and status information from MS in order to verify MS is operational. Subsystem status can be requested through the *SubsystemStatusDataRequestMT* message. The *SubsystemStatusDataRequestMT* message specifies the *SubsystemID* it wants a status update from and the subsystem inside MS will respond with a *SubsystemStatusDataRequestStatusMT*. The Subsystem contained within MS will report status information to MA using the *SubsystemStatusMT* message. The *SubsystemStatusMT* contains the fields *SubsystemID*, *SubsystemState* and other optional fields in order to indicate the subsystem's current state. Similarly, MA can also request subsystem BIT and service status information by sending a *SubsystemBIT_CommandMT* or *ServiceStatusDataRequestMT* respectively. It is required that the *SubsystemID* or *ServiceID* for the subsystem or service the information was requested from is provided in the request message. Once MS receives either a *SubsystemBIT_CommandMT* or *ServiceStatusDataRequestMT* it will respond with a *SubsystemBIT_CommandStatusMT* or *ServiceStatusDataRequestStatusMT* detailing the processing state of the command or request. Subsystems report BIT information to MA using the *SubsystemBIT_StatusMT* message. Built In Test (BIT) is a function of hardware and software to test its functionality to verify it is operating as intended and the *SubsystemBIT_StatusMT* message facilitates the reporting of this information to MA. The *SubsystemBIT_StatusMT* contains the fields *ActiveBIT*, *CompletedBIT* and *Fault* to allow the implementer to report the progress of an active BIT and information about completed BITs. Finally, MS reports the status of services inside the subsystem using the *ServiceStatusMT* message. The *ServiceStatusMT* message contains the fields *TimeUp*, *ServiceStatus* and other optional fields that provide context on the health of the service. This interaction allows MA to understand the current state of MS subsystems and services on an on-demand basis.

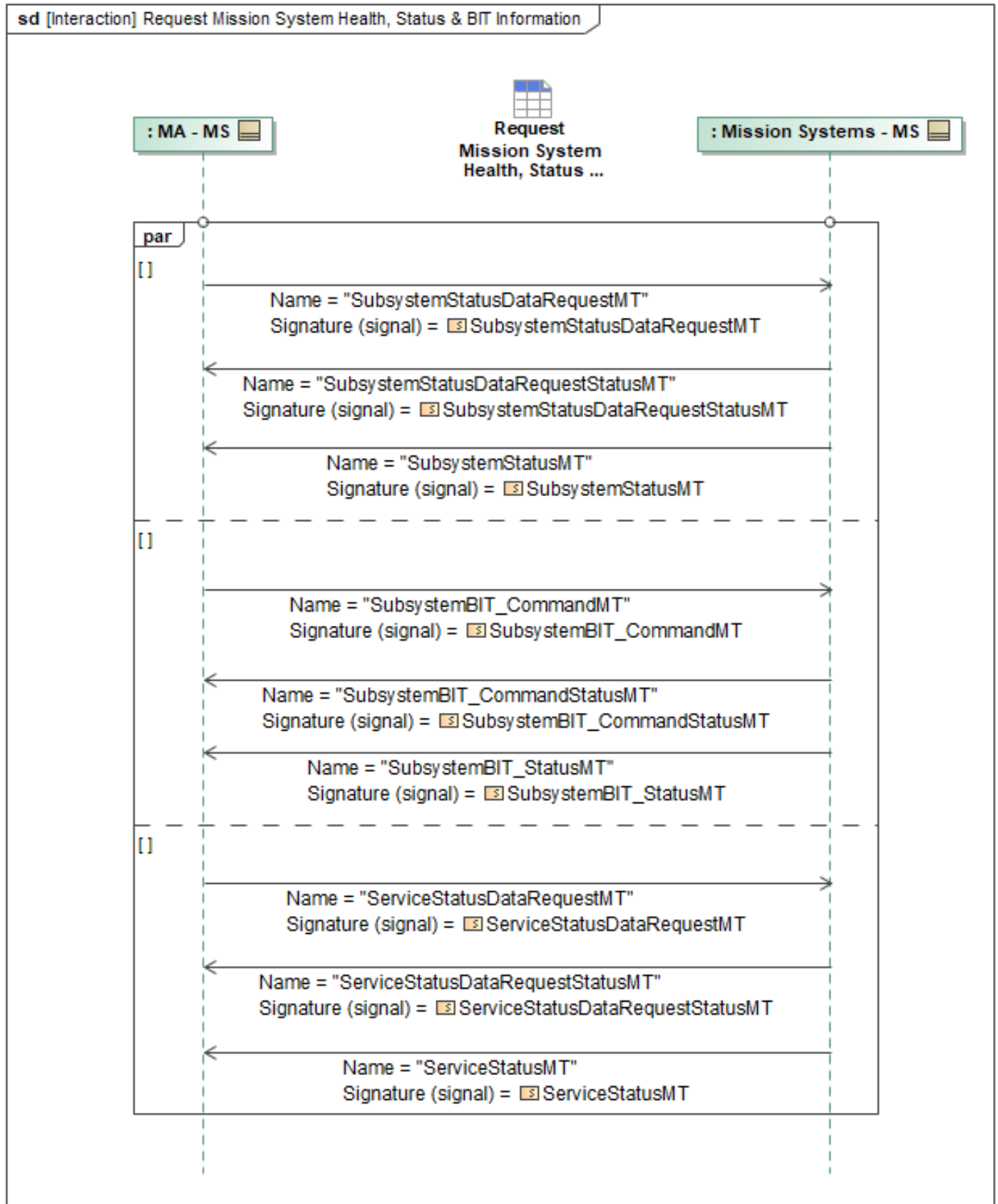


Figure 8-52: Request Mission System Health, Status & BIT Information

Table 8-50: Request Mission System Health, Status & BIT Information

Message Name (Name:Type)	Required Fields (Name:Type)
ServiceStatusDataRequestMT: ServiceStatusDataRequestMT	None
ServiceStatusDataRequestStatusMT: ServiceStatusDataRequestStatusMT	None
ServiceStatusMT: ServiceStatusMT	None
SubsystemBIT_CommandMT: SubsystemBIT_CommandMT	None
SubsystemBIT_CommandStatusMT: SubsystemBIT_CommandStatusMT	None
SubsystemBIT_StatusMT: SubsystemBIT_StatusMT	CompletedBIT: SubsystemCompletedBIT_Type Fault: SubsystemFaultType ActiveBIT: SubsystemActiveBIT_Type
SubsystemStatusDataRequestMT: SubsystemStatusDataRequestMT	None
SubsystemStatusDataRequestStatusMT: SubsystemStatusDataRequestStatusMT	None
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10 Weapon Employment Package

8.2.10.1 Abort Release Sequence

Once a task type of strike has been commanded to be executed, MA may need to abort the weapon release sequence. To abort the strike task, MA will send a *MA_TaskCommandMT* to MS with the corresponding *TaskID* and *CommandState* of *CANCEL*. MS will respond with *MA_TaskStatusMT* with *ExecutionState* set to *CANCELLED* and Mission Store Status messages from MS to verify the abort was successful.

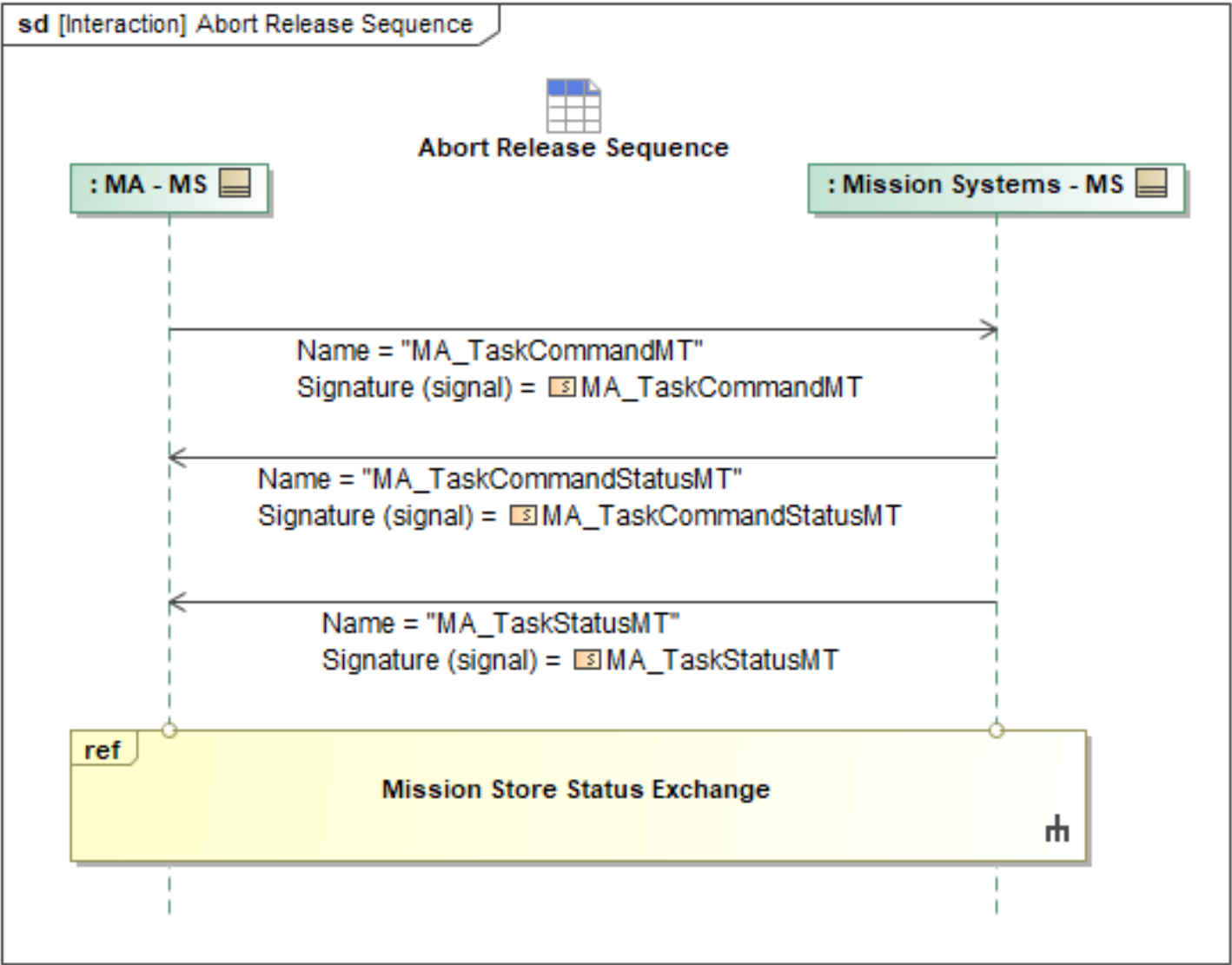


Figure 8-53: Abort Release Sequence

Table 8-51: Abort Release Sequence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskCommandMT: MA_TaskCommandMT	CANCEL: CommandState: CommandStateEnum TaskID: TaskID_Type
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskStatusMT: MA_TaskStatusMT	CapabilityID: CapabilityID_Type CANCELED:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.2 Confirm Threat ID

The following sequence is meant to standardize how MA assesses MS data to confirm identification criteria for each target in the queue or matrix. The *EntityMT* represents a track in the battle space and is ingested by MA through the MS interface. MS will classify the detected enemy as friendly, hostile, or ambiguous. The *StandardIdentityType* contains the enumeration, *StandardIdentityEnum* that can classify the target as either *PENDING*, *UNKNOWN*, *ASSUMED_FRIEND*, *FRIEND*, *NEUTRAL*, *SUSPECT*, *HOSTILE*, or *UNDEFINED*. The result of target classification is included in the corresponding *EntityMT* and can be used in combination with active engagement criteria to ensure entity ID criteria is met during or in the lead up to an engagement. MA may also require updates to an *EntityMT* to maintain positive custody during weapons employment activities.

The *ThreatType* field will also be passed to validate the ID of the track. The *ThreatType* categorizes the threat platform to validate it is a hostile entity.

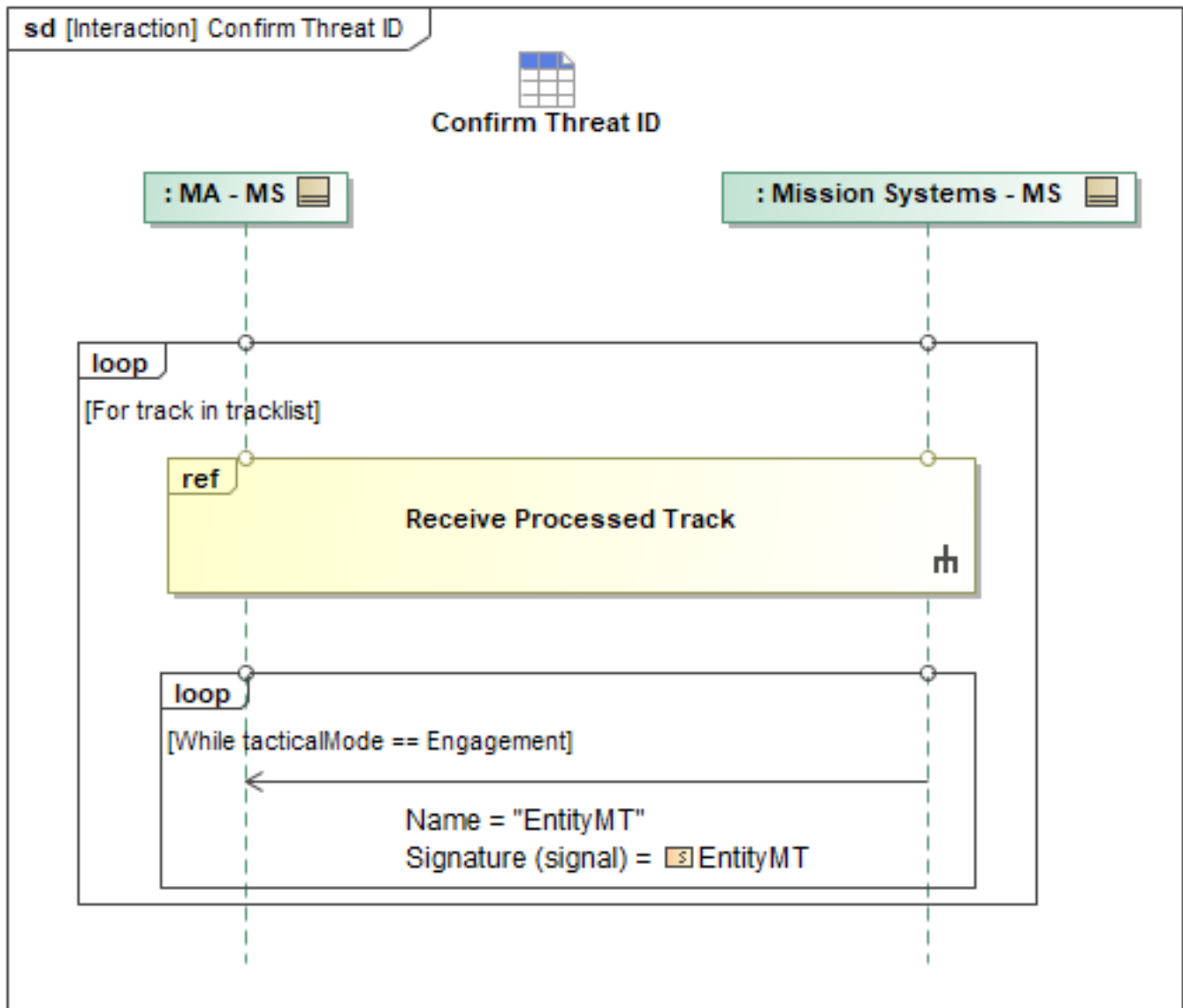


Figure 8-54: Confirm Threat ID

Table 8-52: Confirm Threat ID

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.3 Exchange DLZ

MA sends out a *DLZ_RequestMT* with required field *ResultsInNativeMessage* set to *TRUE* to MS and receives a *DLZ_RequestStatusMT* response back to provide processing state. When processing is complete MS sends a *DLZ_MT* back to MA and utilizes *DLZ_RequestID* to reference the originating request.

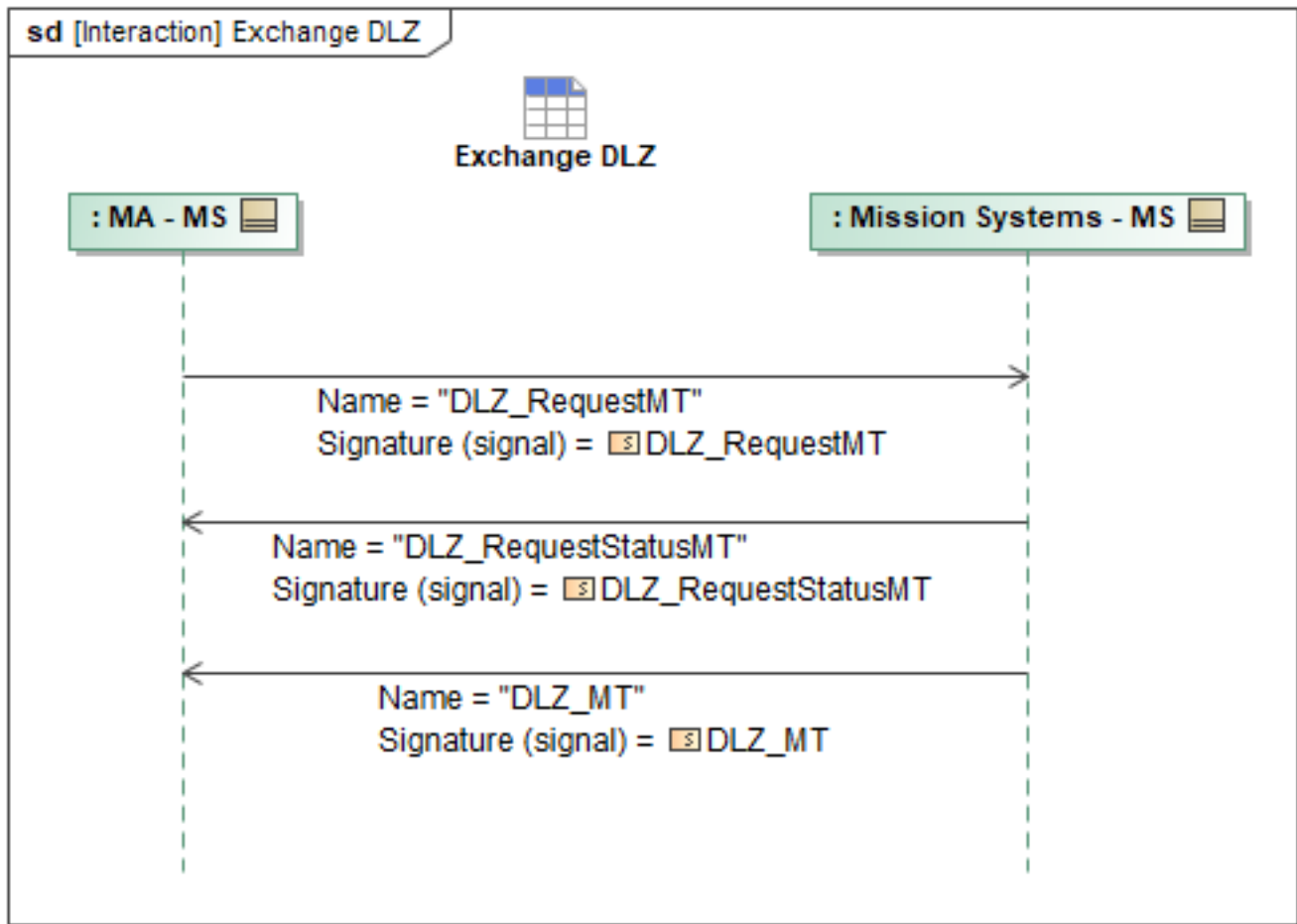


Figure 8-55: Exchange DLZ

Table 8-53: Exchange DLZ

Message Name (Name:Type)	Required Fields (Name:Type)
DLZ_MT: DLZ_MT	DLZ_RequestID: RequestID_Type
DLZ_RequestMT: DLZ_RequestMT	ResultsInNativeMessage: boolean
DLZ_RequestStatusMT: DLZ_RequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.4 Exchange WEZ

MA will receive the Weapon Engagement Zone (WEZ) through the MS interface. The WEZ represents a region around the ownship in which an adversary is vulnerable to a shot and is defined by the range of the weapon system. The WEZ can be utilized in addition to the Dynamic Launch Zone (DLZ) in the case that there is no target state data, targets have not been assigned, or target state data lacks fidelity. The WEZ is made up of a Weapon's various ranges. Those ranges are a function of the weapon type, the azimuth relative to the ownship which the weapon would need to turn to, the ownship state/position data, and a target's heading, velocity and altitude. The *MA_WEZ_RequestMT* message uses the *StoreInformation* field to identify the specific capability the WEZ is being requested

for. The schema then uses the *WEZ_FieldOfView* field to identify the region for the WEZ by indicating the desired azimuth range around the ownship for which the capability ranges are being requested for. For example, the *FRONT_WEZ* enumeration represents an array of azimuths from 0 to 180 deg off the nose of the ownship with a default 5 deg step defining the array. This array of ranges can be pre-calculated by the service prior to flight or calculated upon the request while in flight. To account for ownship and potential target state/position data, the implementer has a choice of options that are outlined by the *WEZ_ChoiceType*.

The *StaticWEZ* choice uses the ownship's position data as reported by the vehicle interface and then the service assumes that the target heading is pointed at the ownship per azimuth outlined in the *WEZ_FieldOfView*. The service also will assume the target as the same altitude and velocity as the ownship. This is the most static, least dynamic representation of the WEZ.

The *PlannedStaticWEZ* choice allows for the user to specify a planned ownship state/position using the *PlannedReleaseConditions* field. The service then also assumes the target's heading is pointed at the ownship per azimuth but assumes that its velocity and altitude is the same as what is identified in the *PlannedReleaseConditions* field.

The *PredictedWEZ* choice allows for a more dynamic WEZ and requires the user to input the target state/position data using the *PredictedTargetStateData* field. This allows the user to input a specific velocity, altitude, and target heading aspect off the ownship per azimuth. The service would then subscribe to the ownship state/position data as reported by the vehicle interface.

Lastly, the *PlannedPredictedWEZ* choice, allows the implementer to specify the ownship and predicted target state/position data using both the *PlannedReleaseConditions* and *PredictedTargetStateData* fields.

This WEZ request is then acknowledged by MS using *MA_WEZ_RequestStatusMT*. With the store information and required inputs for a capability range provided by MA, MS can now send MA the WEZ specific to that store. MS does so by providing the *MA_WEZ* message in which provides a various set of ranges per azimuth requested, which builds out the requested 2D region. The required ranges that define the WEZ are *RangeMinimum*, *RangeTurnAndRun*, *RangeProbabilityOfIntercept*, *RangeMaxGuidance*, *RangeOptimal*, and *RangeMaxAero*. The WEZ UCI message structures are very similar to that of the DLZ messages with the difference being WEZ does not require target state data

Potential WEZ Use Cases:

- Course Planning - Plan a specific course based on weapon capability
- Set up Formation - Specific ACP spacing based on weapon Capability
- Projection of Power consideration - radius overlapping the correct amount
- Swarm impacts
- CAP Geometry - CAP radius based off weapon capability
- Task Allocation - Allocate tasks based on weapon capability - Platform A is in the range of detected entity (not a target yet) - DLZ would follow on as entity becomes target set

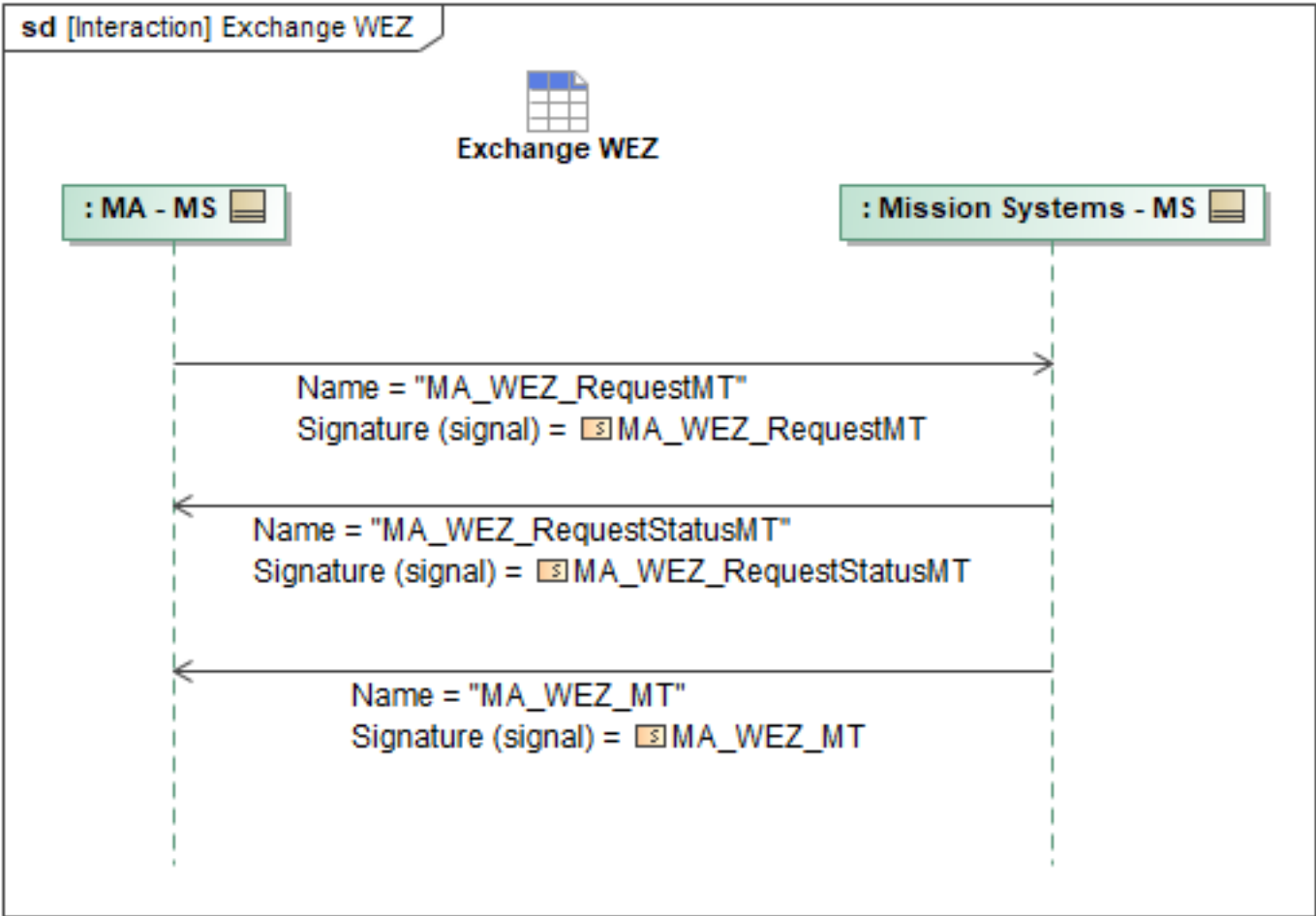


Figure 8-56: Exchange WEZ

Table 8-54: Exchange WEZ

Message Name (Name:Type)	Required Fields (Name:Type)
MA_WEZ_MT: MA_WEZ_MT	None
MA_WEZ_RequestMT: MA_WEZ_RequestMT	None
MA_WEZ_RequestStatusMT: MA_WEZ_RequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.5 Fire Command Exchange

As a prerequisite to initiating a weapons release, MA must assess and determine the threat prioritization, maintain custody of hostile threat(s), and obtain release consent authorization in accordance with the engagement criteria. MA will update *MA_Task.MessageData.TaskType.Strike* contents as required to support weapons employment. MS will respond with *MA_TaskStatus* with the selected *CapabilityID* and a *TaskID*. MA will then send a *TaskCommand* message to MS to command the execution of the previously referenced *TaskID*. MS will respond to *TaskCommand* with *TaskCommandStatus* indicating the command was accepted or rejected. MS will periodically update

MA with *MA_TaskStatus* on the progress and state changes of the task being executed. At task completion, MA will also receive *StrikeCapability* and *StrikeCapabilityStatus* to confirm the weapon (*CapabilityID*) was successfully released.

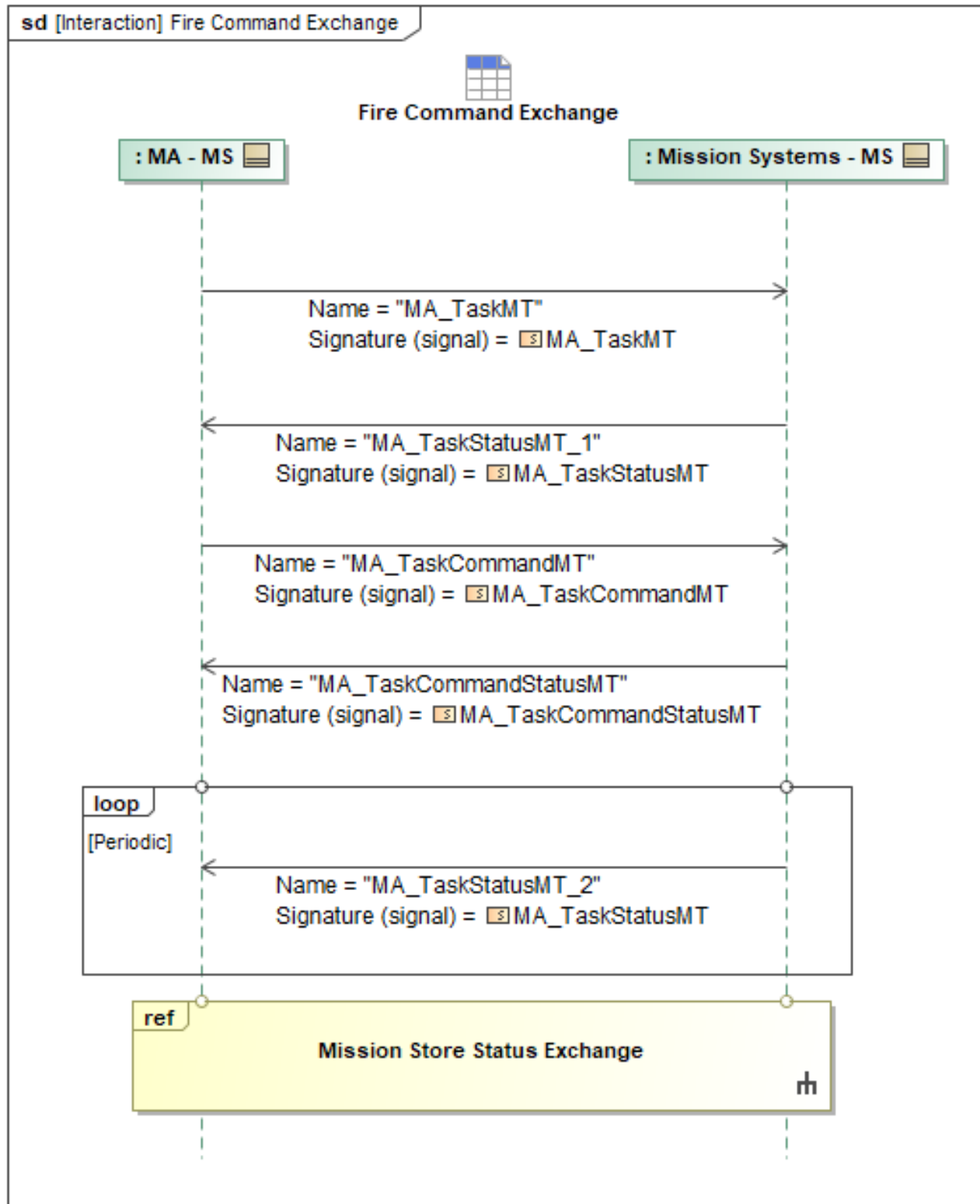


Figure 8-57: Fire Command Exchange

Table 8-55: Fire Command Exchange

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskCommandMT: MA_TaskCommandMT	None
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None
MA_TaskStatusMT_1: MA_TaskStatusMT	None
MA_TaskStatusMT_2: MA_TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.6 Jettison Release Sequence

The jettison function enables MA to send a jettison task to MS to perform an off nominal release sequence of the weapon regardless of the weapon's powered state. MA will send a task, *MA_Task.MessageData.TaskType.Jettison*, to MS to initiate the Jettison sequence. The *MA_JettisonTaskType* requires both *MA_JettisonStoreSelection* and *MA_StoreJettisonType* to be provided. *MA_JettisonStoreSelection* specifies either *JettisonAllCapableWeapons (EmptyType)* to jettison all weapons capable of being jettisoned, or the *CapabilityID(s)* of the stores to be jettisoned. *MA_StoreJettisonType* has two enumerations of Standard or Emergency. Standard jettison will inform MS that if the weapon is powered on, it is to zeroize and power down before jettison. Emergency jettison will inform MS that the weapon is to be released immediately regardless of powered state. MS will respond to *MA_Task* with *MA_TaskStatusMT* with each of the *CapabilityID(s)* for the selected stores for jettison. MA will execute the task with *MA_TaskCommandMT*. MS will respond with *MA_TaskCommandStatusMT*. MA will monitor *MA_TaskStatusMT* and Mission Store Status sequence for the completion of the task.

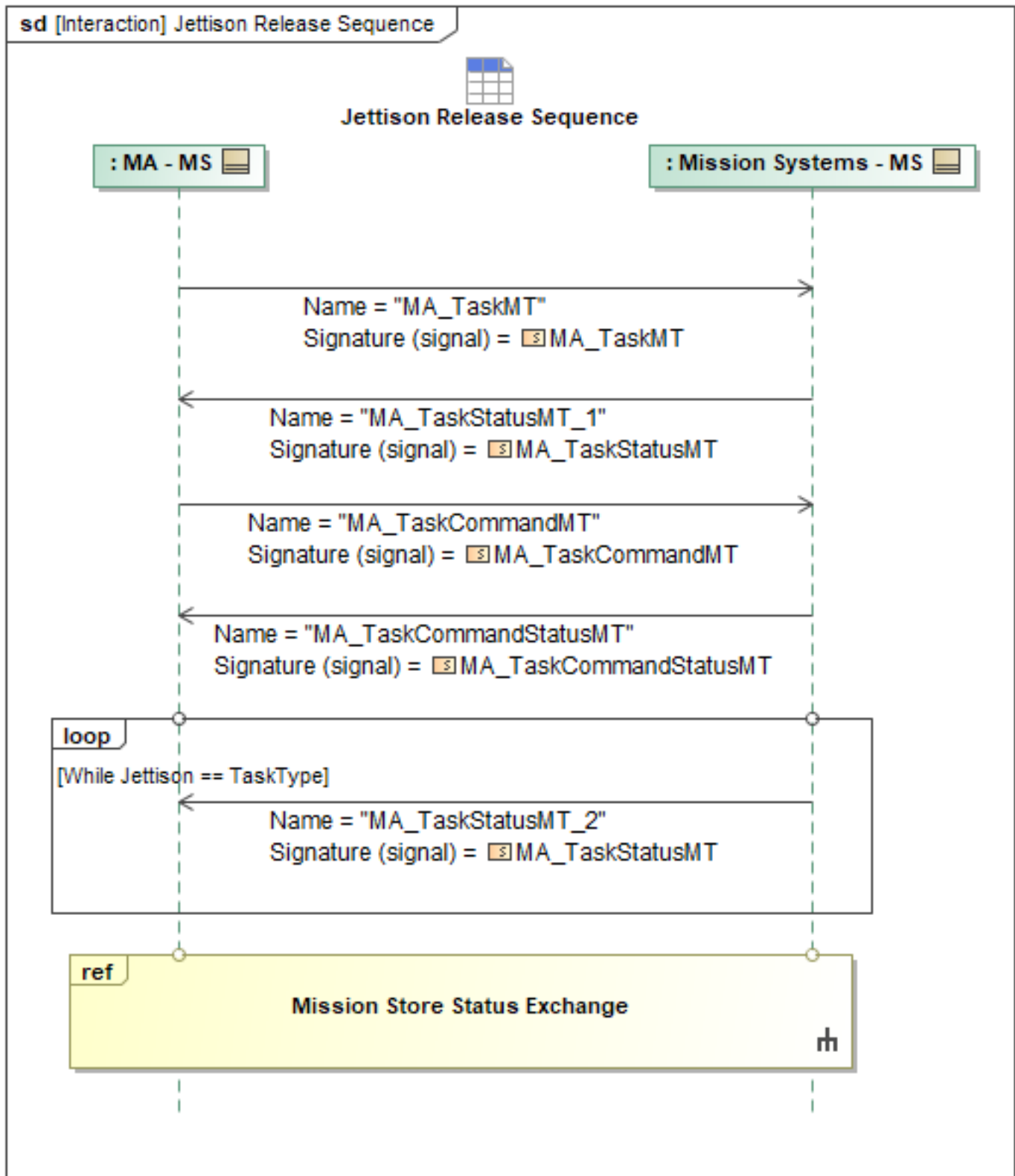


Figure 8-58: Jettison Release Sequence

Table 8-56: Jettison Release Sequence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskCommandMT: MA_TaskCommandMT	None
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Jettison: MA_JettisonTaskType
MA_TaskStatusMT_1: MA_TaskStatusMT	Traceability: RequirementStatusTraceabilityType
MA_TaskStatusMT_2: MA_TaskStatusMT	Traceability: RequirementStatusTraceabilityType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.7 Mission Store Release Initialization Sequence

This sequence is the start of the weapons employment release sequence and initiates mission store initialization. MA will send the *TaskID* with *MA_Task.MessageData.TaskType.Strike*. The *Strike* task shall include the following: *TaskType.Strike.Target (Entity or SpecificType)*, *TaskType.Strike.WeaponList.Weapon.RequiredWeapon*, *TaskType.Strike.WeaponList.Weapon.StoreInformation.StoreType* and *TaskType.Strike.WeaponList.Weapon.StoreInformation.StoreQuantity*. MS will reply with *MA_TaskStatusMT* with the assigned CapabilityID(s) via *Traceability.CapabilityID*. MS will periodically send Mission Store Status Exchange to provide details of each of the installed capabilities.

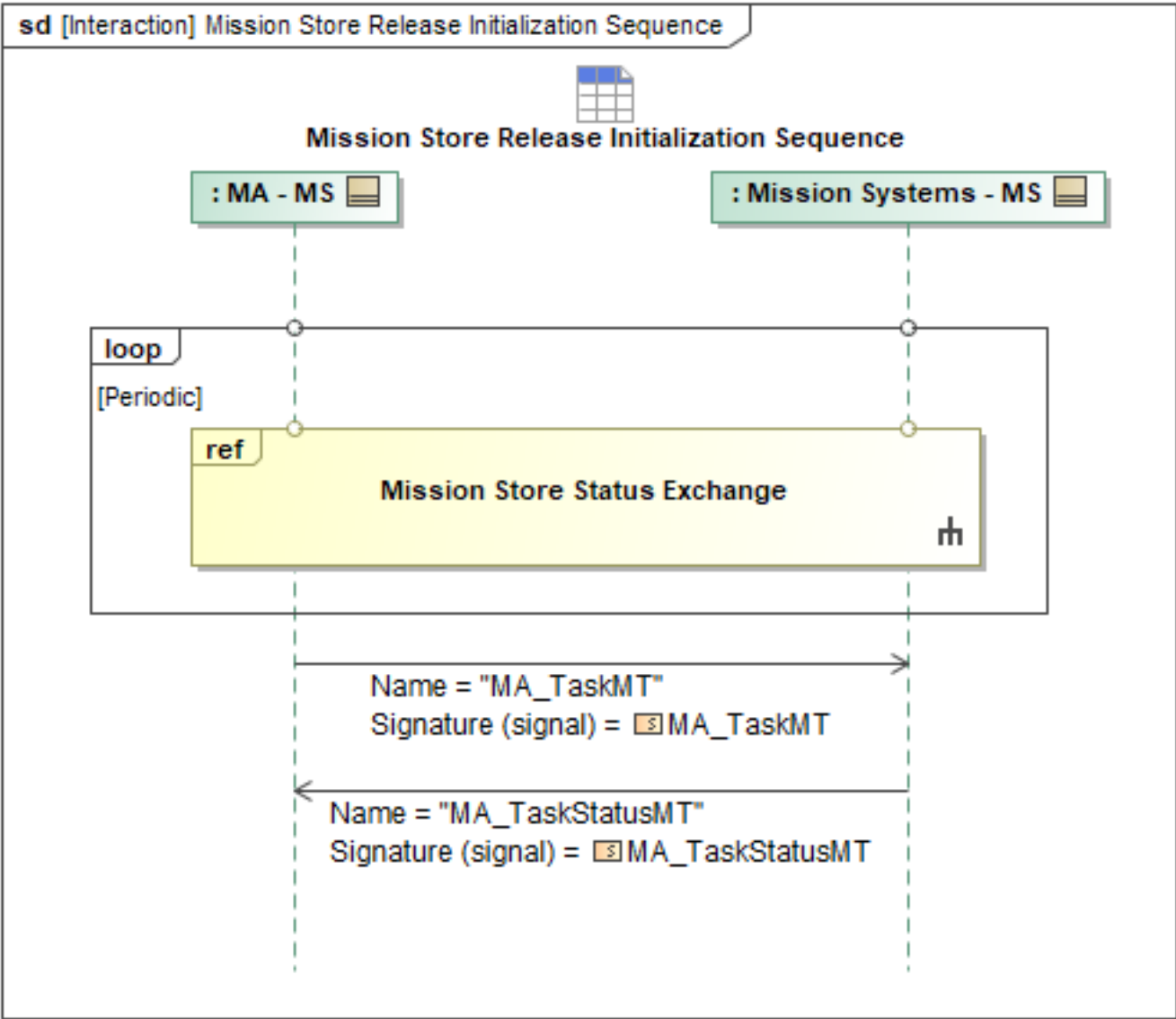


Figure 8-59: Mission Store Release Initialization Sequence

Table 8-57: Mission Store Release Initialization Sequence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	Weapon: StrikeTaskWeaponType StoreInformation: WeaponeeringStoreType Strike: StrikeTaskType
MA_TaskStatusMT: MA_TaskStatusMT	CapabilityID: CapabilityID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.8 Mission Store Status Exchange

The store management system within the MS will periodically publish *StrikeCapabilityStatus* for each

store capability ID regardless of the store presence. MA will utilize *StrikeCapabilityStatusMT* and *MA_StrikeCapabilityMT* to maintain persistent awareness of the store state, loaded mission data, and mission store parameters in accordance with the selected Mission Data File (MDF). In the event that MS has an unexpected or undesirable state reported, MS is then required to update *StrikeCapabilityStatus.MessageData.CapabilityStatus.AvailabilityInfo.AvailabilityReason*.

Upon strike tasking from MA to MS and the strike command being sent internal to MS, the Strike Activity will become true. In the case the Strike Activity becomes true, MS must pass the *MA_StrikeActivityMT_1* to MA to include the *ActivityState* of the *MA_StrikeActivity*. The *ActivityState* pertains to the state of the activity and includes enumerations for *DISABLED*, *ENABLED*, *ACTIVE_UNCONSTRAINED*, *ACTIVE_PARTIALLY_CONSTRAINED*, *ACTIVE_FULLY_CONSTRAINED*, *COMPLETED*, *FAILED*, *DELETED*. MS should publish the *MA_StrikeActivityMT* once to MA regardless of the *ActivityState*. If the *ActivityState* is not equal to *DISABLED*, *FAILED* or *DELETED* MS will publish the *MA_StrikeActivityMT_2* to MA periodically throughout the strike activity. Optionally, MS can provide MA with the *MunitionActive* field that pertains to the specific weapon state data and orientation during weapon flyout. Active weapon states include *TARGETED*, *REQUESTED*, *SEEKER_INACTIVE*, *SEEKER_ACTIVE*, *FUZE_INACTIVE*, *FUZE_ACTIVE*, *NOT_FULLY_AUTONOMOUS*, *FULLY_AUTONOMOUS*, *DEFEATED*, *TIMED_OUT*, *SUCCESS*.

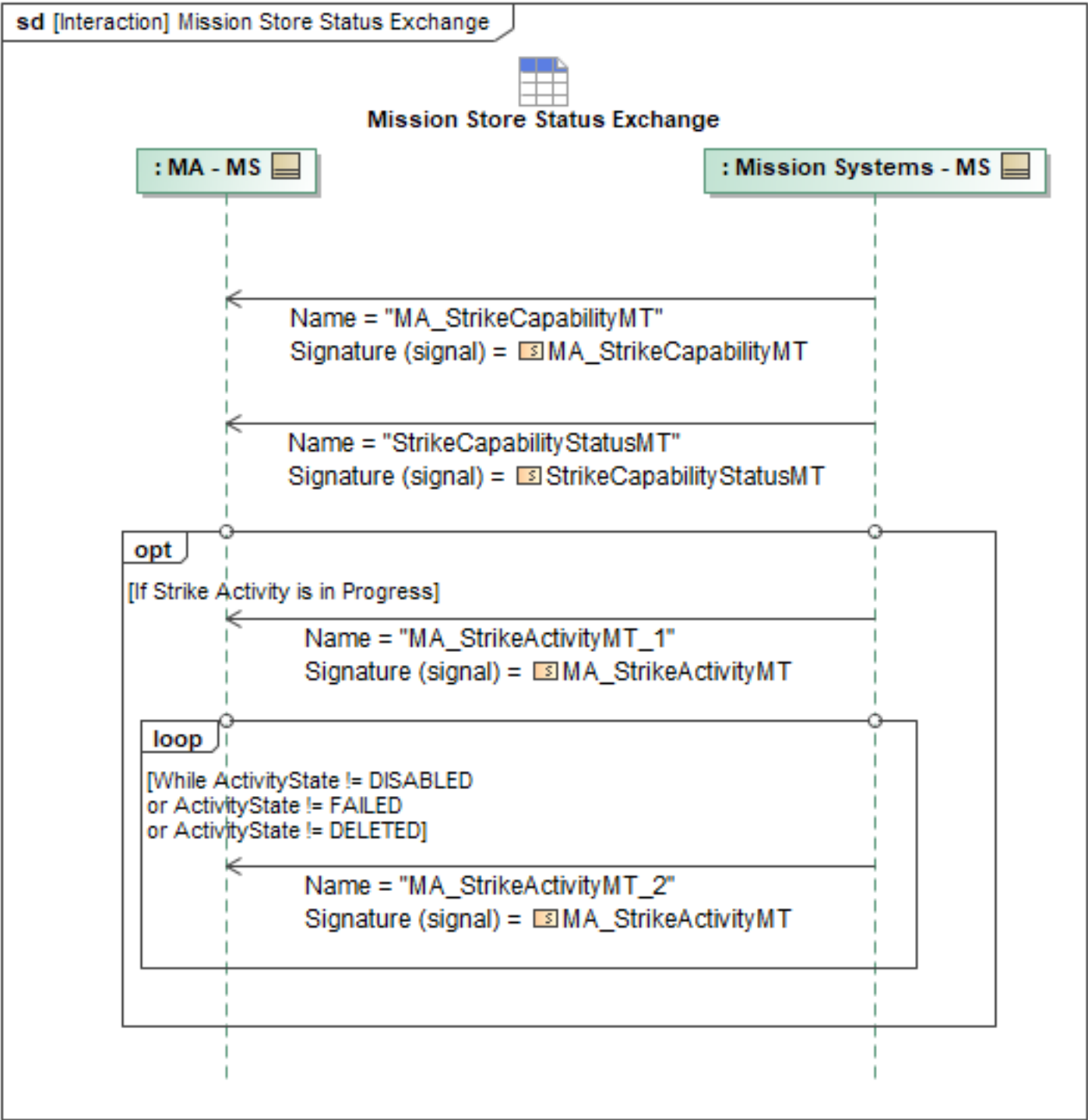


Figure 8-60: Mission Store Status Exchange

Table 8-58: Mission Store Status Exchange

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeActivityMT_1: MA_StrikeActivityMT	None
MA_StrikeActivityMT_2: MA_StrikeActivityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	StoreStatus: StoreStatusType AvailabilityReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.9 Receive Changes to Stores Primary Mission Plan Data

Prior to the communication exchange between MS and MA, MS sends a *ServiceStatus* message to MA to indicate that it is ready to receive *DataRequest* messages. After receipt of *ServiceStatus* message, there are two different sets of interactions that can occur - updating existing parameters on the current MDF or loading a new MDF.

To update existing MDF parameters, MA sends a *ServiceConfigurationParamsDataRequest* message in order to retrieve the latest configuration parameters for the mission data set. MS will reply with the *ServiceConfigurationParamsDataRequestStatus* and afterwards send the *ServiceConfigurationParams* message to MA in order to fulfill the receipt of primary mission data set updates. The *ServiceConfigurationChangeRequest* will then be submitted including all parameters to be modified along with their respective updated values. The updated parameter should be among the list of configuration parameters provided in the *ServiceConfigurationParams* message.

To load a new Mission Data File, MA sends *StrikeSettingsCommandMDT.ActivateMDF* command to MS and MS responds accordingly in the *StrikeSettingsCommandStatus* message. The updated *MA_StrikeCapabilityMT* message will be relayed from MS to MA, as well as the *StrikeCapabilityStatus* message.

This is an optional capability, and not every service may have it. The "Receive Changes to Stores Primary Mission Plan Data" sequence should be implemented by services that have this capability if MA deems that it needs updated or new MDF parameter(s).

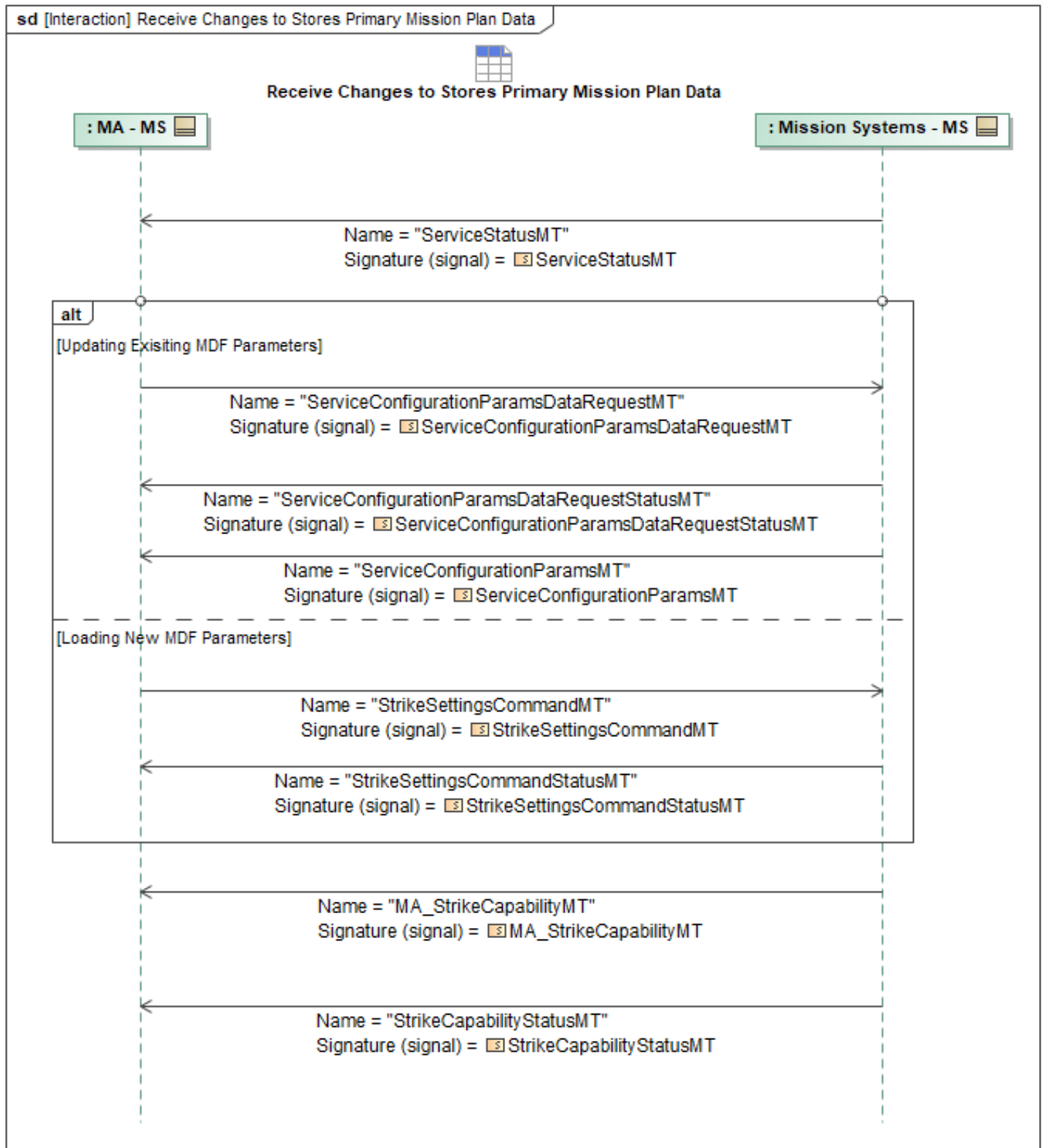


Figure 8-61: Receive Changes to Stores Primary Mission Plan Data

Table 8-59: Receive Changes to Stores Primary Mission Plan Data

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	AvailableMDF: MDF_Type
ServiceConfigurationParamsDataRequestMT: ServiceConfigurationParamsDataRequestMT	None
ServiceConfigurationParamsDataRequestStatusMT: ServiceConfigurationParamsDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
ServiceConfigurationParamsMT: ServiceConfigurationParamsMT	None
ServiceStatusMT: ServiceStatusMT	None
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	StoreStatus: StoreStatusType
StrikeSettingsCommandMT: StrikeSettingsCommandMT	ActivateMDF: MDF_CommandType
StrikeSettingsCommandStatusMT: StrikeSettingsCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.10 Receive Weapon Flyout State

Mission Use Case: Store Management

MA sends an *AssessmentRequestMT*. MS will reply with an *AssessmentRequestStatusMT_1* of *accepted* or *AssessmentRequestStatusMT_2* of *rejected*. If request is *accepted*, MS sends a periodic *MA_EntityMT* until it returns the *EngagementState* of *LOST*, *SOFT_LOST*, *END_OF_FLIGHT*.

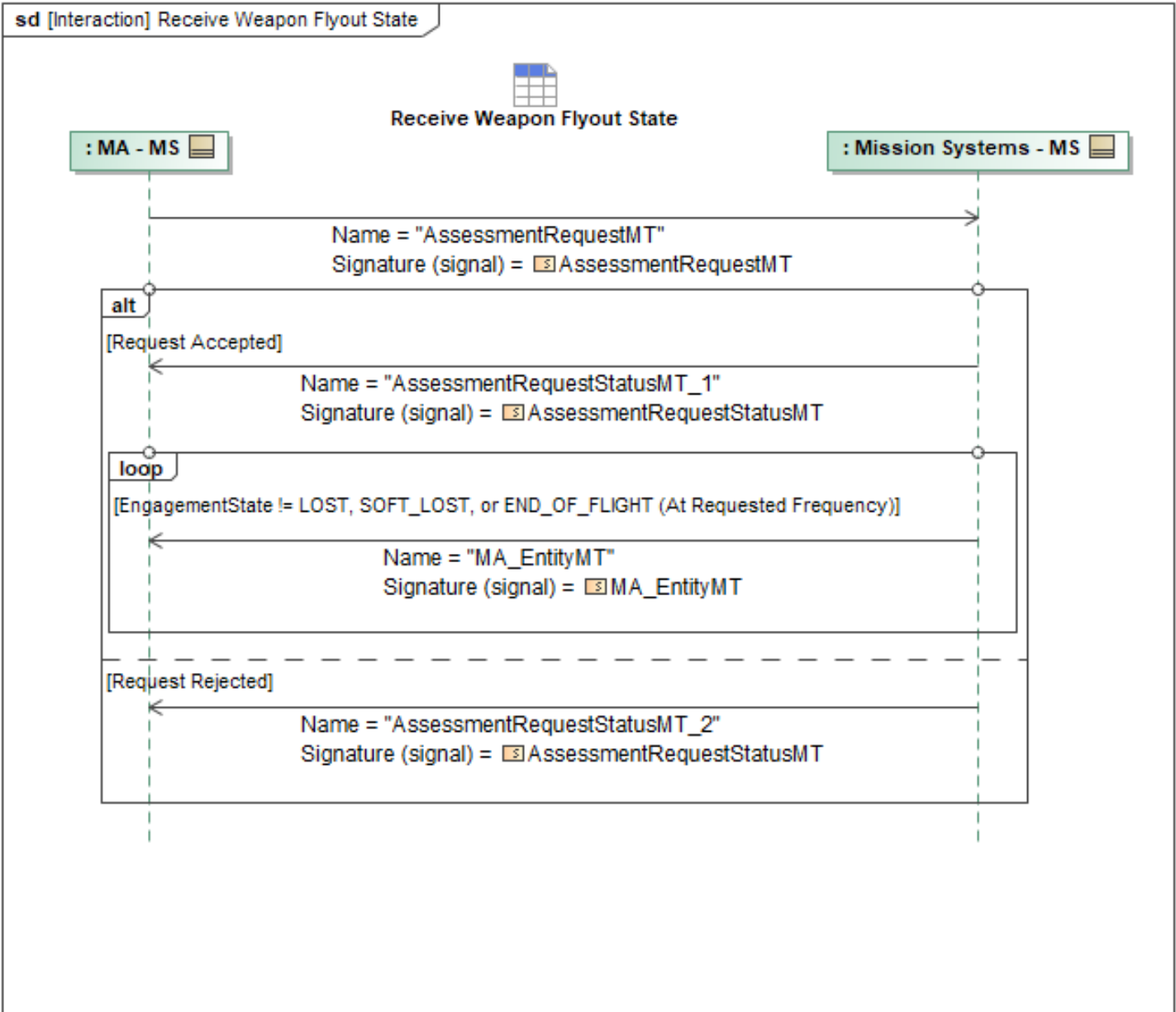


Figure 8-62: Receive Weapon Flyout State

Table 8-60: Receive Weapon Flyout State

Message Name (Name:Type)	Required Fields (Name:Type)
AssessmentRequestMT: AssessmentRequestMT	None
AssessmentRequestStatusMT_1: AssessmentRequestStatusMT	Assessment: MA_AssessmentType
AssessmentRequestStatusMT_2: AssessmentRequestStatusMT	Assessment: MA_AssessmentType
MA_EntityMT: MA_EntityMT	EngagementState: MA_EngagementStateEnum TimeToActive: DurationType

Message Name (Name:Type)	Required Fields (Name:Type)
	TimeToIntercept: DurationType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.11 Release Consent Sequence

The following sequence is meant to standardize the interface between MS and MA for Weapon Release Consent. Prior to this exchange, the MA has received information that specifies system "authorizers" for final fire consent, i.e. which system(s) can authorize the weapons release action. The MA will use that list of authorizers when enforcing the Weapons Release Authority to decide where to send the *StrikeConsentRequestMT*. The Weapons Release Authority Criteria allows (if configured) for MA to automatically respond to the *StrikeConsentRequestMT*. The 3 options, configured independently, are to automatically respond if C2 has sent a *MA_DesignationMT*, C2 has approved a *ResponseMT* that starts the Requirement against the target, or if C2 has sent an arming command. After MA executes the Fire Command exchange, MS will respond with a request for final authorization to fire. This involves the following messaging:

- MS will send a *MA_StrikeActivityMT* message to MA with the *ActivityID*. The *ActivityID* identifies the fire activity. For Activity/Task association in the MA, MS must send a *MA_StrikeActivityMT* message in response to a strike command within MS. The Fire Command sequence provides the MA with the *TaskID*; however, MA needs the *ActivityID* to be able to enforce a Weapons Release Authority Criteria from the Engagement Criteria message. Otherwise, MA will not have sufficient information to send a *StrikeConsentRequestMT* to C2.
- MS will send a *StrikeConsentRequestMT* to MA, requesting authorization to fire.
- Once the appropriate "authorizers" have been determined, MA either sends a *StrikeConsentRequestMT* to C2 to obtain final authorization to fire or automatically confirms final authorization (modeled in the C2 profile's Release Consent section).
- C2 responds with *StrikeConsentRequestStatusMT* if this was required (modeled in the C2 profile's Release Consent section).
- MA sends a *StrikeConsentRequestStatusMT* to MS.

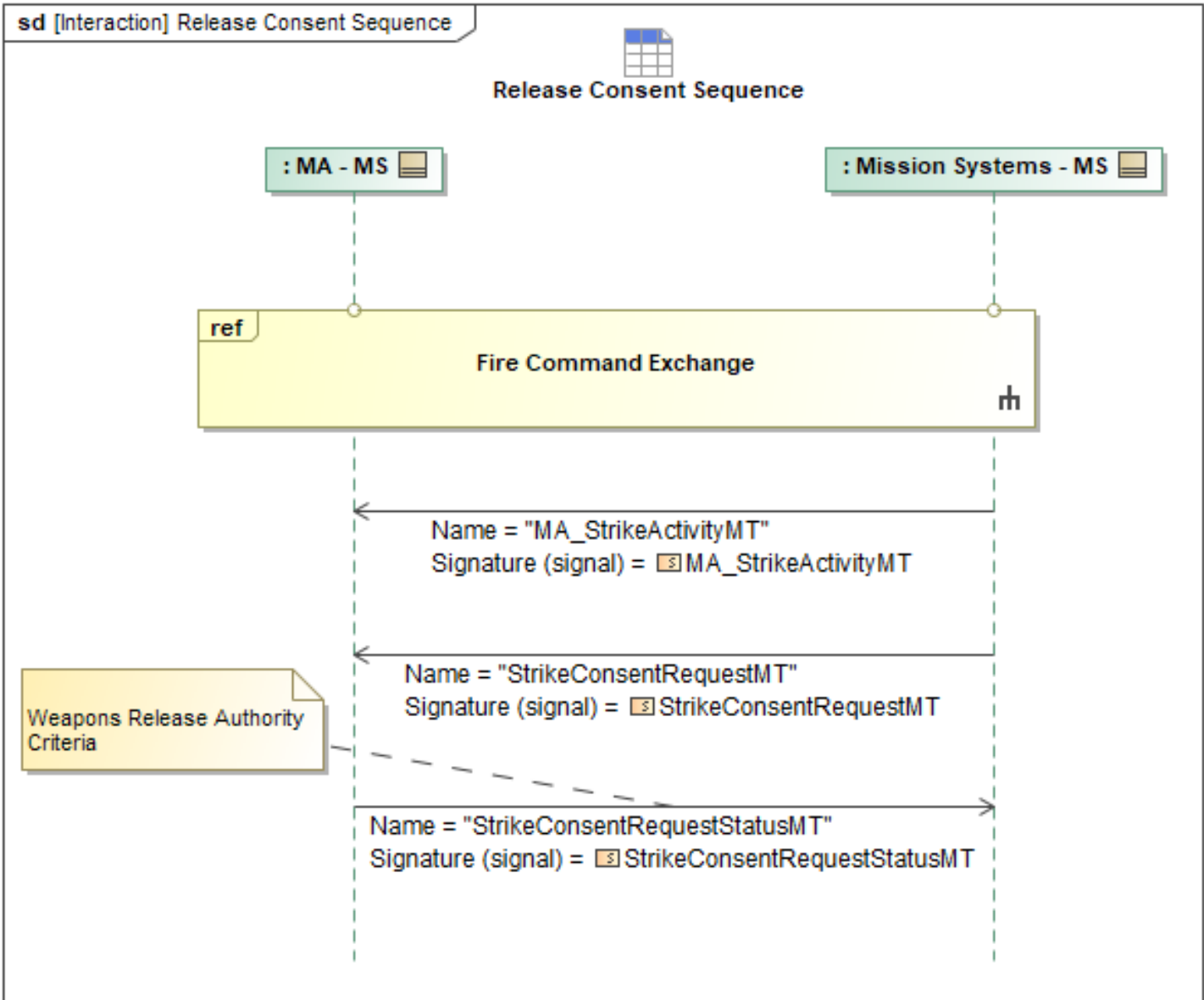


Figure 8-63: Release Consent Sequence

Table 8-61: Release Consent Sequence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeActivityMT: MA_StrikeActivityMT	None
StrikeConsentRequestMT: StrikeConsentRequestMT	None
StrikeConsentRequestStatusMT: StrikeConsentRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.12 Send Weapon Entity Data

The Send Weapon Entity Data sequence diagram describes how MA distributes fused tracks and global COP data to the Mission System to inform weapon systems of assigned Entity details. The basic functionality of a data fusion service is to process and merge target reports from various internal and external sources to provide a resulting track, called *Entity*. MA sends the updated Global COP *EntityMT* to MS to provide the latest and best information for both internal and external sources on a given target. Fields that are necessary for MS to inform weapons systems include *EntityID*, *Source*, *EntityStatus*, *Identity*, and *Kinematics*. The *Kinematics* field is used to provide current kinematic information for the given Entity, including *Position* of the target. This position information provides data to MS and the corresponding weapon system for navigation to the assigned target. The "current" nature of the data is determined by the given system, and the necessary *KinematicTypes* used are determined by the given implementation. This message is also triggered when new tracks are determined, or there are changes to already established tracks.

Losing a track is a special optional case that happens when a sensor can no longer detect and track an entity; when this occurs, the *EntityLostMT* message will notify the Mission System of this event. MS may already have this information, but in the instance that MS is not the detecting sensor, the Global COP updates from MA are necessary to inform MS of updates for itself and the weapons systems.

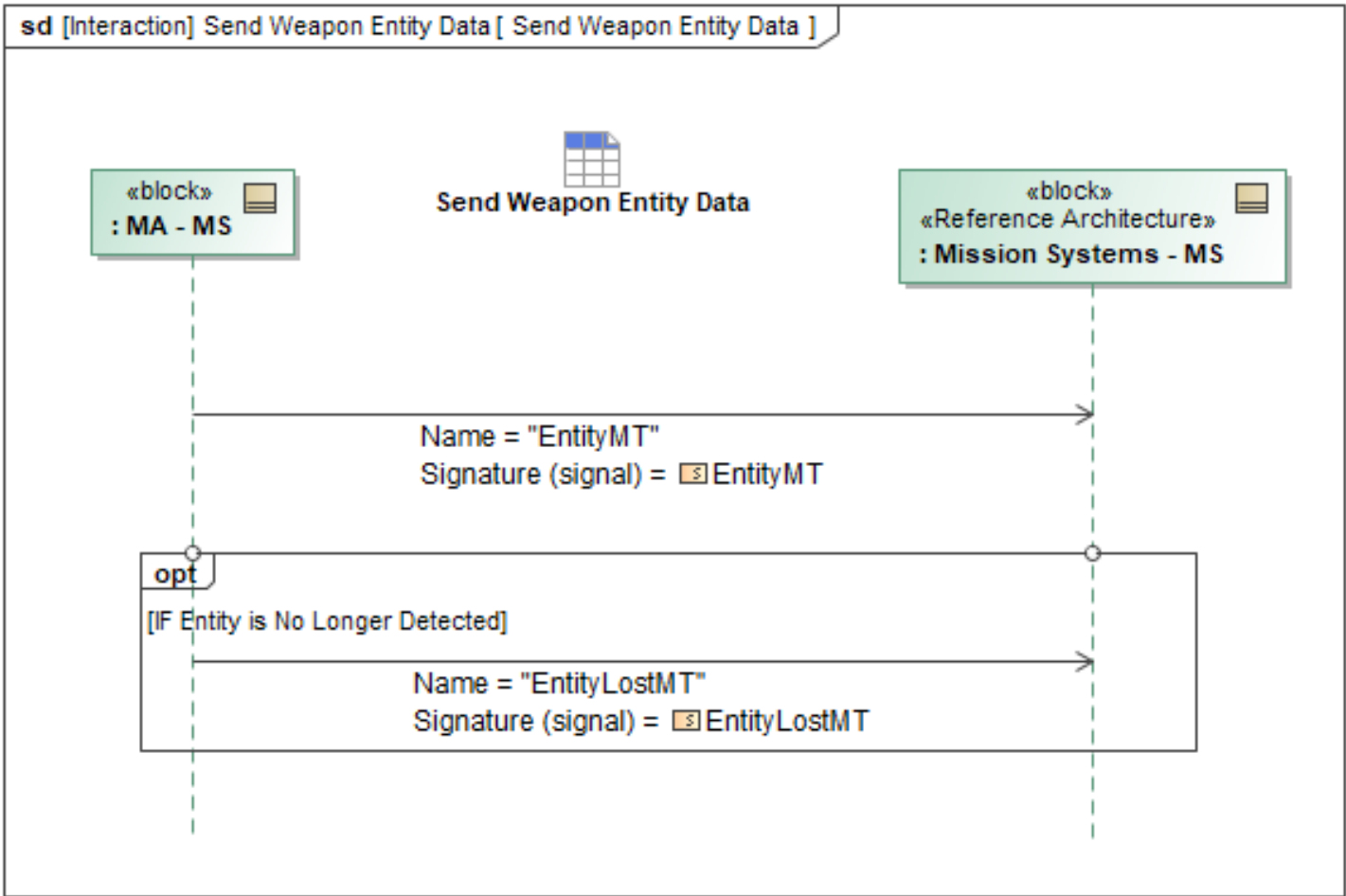


Figure 8-64: Send Weapon Entity Data

Table 8-62: Send Weapon Entity Data

Message Name (Name:Type)	Required Fields (Name:Type)
EntityLostMT: EntityLostMT	None
EntityMT: EntityMT	Kinematics: KinematicsType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

8.2.10.13 Store Inventory and Descriptions Exchange

The supporting weapon subsystems (i.e. Stores Management System, Weapons Resource Manager) within the Mission Systems (MS) are required to publish store inventory data (i.e. installed mission stores and carriages) using *MA_StrikeCapabilityMT*. Mission Autonomy (MA) will continuously monitor updates to *MA_StrikeCapabilityMT* to determine the inventory of the installed store types, station locations and installed capabilities. Mission Systems will assign a capability ID for each installed mission store. The supporting capability is related to the carriage or rack that the store is attached. MA can request the weapon subsystem to verify the reported inventory is compatible with the platform using *StoreManagementCommand* and receive a response with *StoreManagementCommandStatus*. Store compatibility infers the platform has completed weapon

system integration testing to include safe separation testing.

This message is used for inventory and mission capability. This message will be updated as required when a tactic is selected and a Mission Data File (MDF) is loaded. The table below is only focused on the required fields for MA to determine initial store inventory and store descriptions.

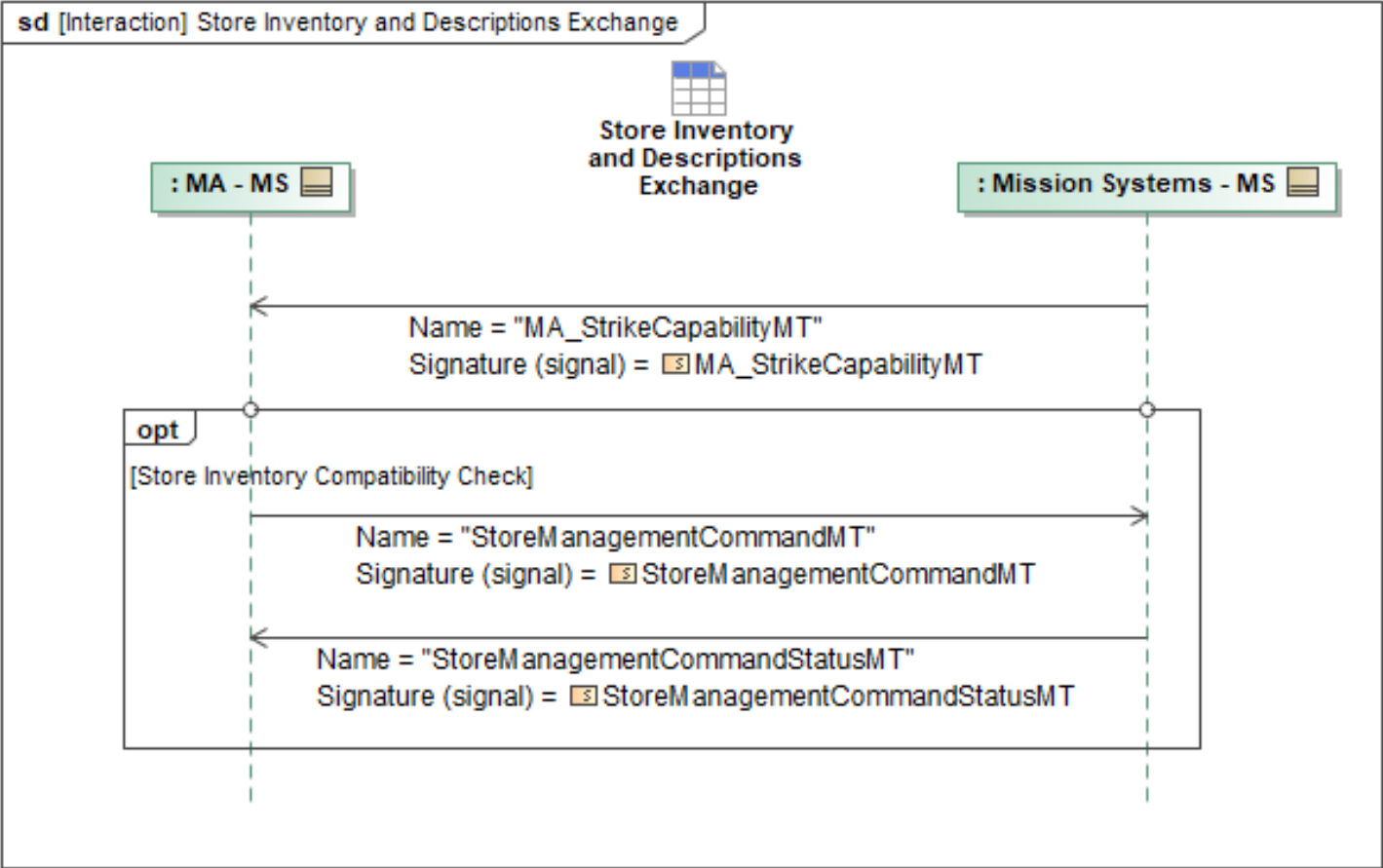


Figure 8-65: Store Inventory and Descriptions Exchange

Table 8-63: Store Inventory and Descriptions Exchange

Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	StoreTypeVariant: ForeignKeyType Mnemonic: VisibleString256Type Powerable: boolean LoadoutMnemonic: VisibleString256Type Carriage: StoreCarriageCapabilityType
StoreManagementCommandMT: StoreManagementCommandMT	None
StoreManagementCommandStatusMT: StoreManagementCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

8.3 Interface Compliance

The Interface Compliance table contains the compliance requirements for the L1 MS interface.

Table 8-64: Mission Systems Interface Compliance Requirements

ID	Requirement
MA-L1-008	For a Level 1 Mission Systems interface, MA shall only communicate with Mission Systems via the platform hosted ASB.
MA-L1-007	A Level 1 Mission Systems interface shall implement all required messaging sequences in the MS feature profile (A-GRA Location: 1 Problem Domain > 2 White Box > 3 Functional Analysis > L1 Functional Analysis > L1 Interface Behaviors > MS Interface Behavior), marked Core and all non-Core Mission Use Case sequences that apply to the platform specific implementation.
MA-L1-006	A Level 1 Mission Systems interface shall support the Mission Systems Interface Minimum Message Set defined in the MS MMS Table in the Mission Systems feature profile (A-GRA location: 1 Problem Domain > 3 Exchange Items > Minimum Message Set (MMS)) for all messages in Core Mission Use Case sequences and all non-Core Mission Use Case sequences required for platform specific implementations.
MA-L1-010	The DMS's DDS middleware shall use the MA_DataPayloadWrapper IDL (Interface Definition Language) to send messages OTA across all topics.
MA-L1-009	The DMS's DDS middleware shall use the RTPS (Real-Time Publish-Subscribe) protocol, specifically the Simple Participant Discovery Protocol (SPDP) and Simple Endpoint Discovery Protocol (SEDP) for system discovery.
MA-L1-011	The DMS's DDS middleware shall use topic names that exactly match the message name of the EncodedPayload message. The only messages sent on that topic shall have an EncodedPayload that matches the corresponding topic name.
MA-L1-012	The DMS shall use a DDS middleware to handle OTA communication with other DMS instances.
MA-L1-019	Upon receipt of a Malformed Message (a message missing expected fields (i.e., required by the behavior but not the UCI schema) or with invalid context-dependent information within the scope of that individual message) from the MS interface, MA shall respond to the error by notifying MS of the malformed message and requesting the corrected message according to the Malformed Message Response (MS) sequence.
MA-L1-025	The DMS shall use the EXI standard for the MA_DataPayloadWrapper IDL EncodedPayload field using the settings table found in the "Serialization and Data Encoding" section of the L1 MA ICD. The contents to decode/encode will come from the EncodedPayload field within the MA_TxDataPayloadCommandMT. The datatype of the MA_TxDataPayloadCommandMT/MA_RxDataPayloadMT EncodedPayload fields is UTF-8 XML represented as hexBinary.

9 PEER INTERFACE

The role of the Peer interface is to provide communication between multiple MA-enabled ACPs, enabling data to be shared to facilitate collaborative perception and behaviors among one or more teams of ACPs. The Peer interface covers sharing information relating to teaming and formation, custody, contingency, and discovery via UCI messaging. ACPs send and receive off-board OTA network messages via the DMS (Decentralized Messaging Service).

9.1 Interface Description

The role of an MA to peer system in the operation of MA-enabled ACPs is to provide the transfer of vehicle data, facilitate direct communication among multiple ACPs, and enable collaboration between all MA platforms by simplifying the sending and receiving of messages from C2 to a team through the RBAC construct. A peer is defined as an A-GRA compliant ACP operating within the same theater as another instance of MA. This document will refer to one ACP as MA and will refer to additional MA-compliant ACPs as peers. Although most diagrams depict transfer between MA and a single peer, MA can communicate with multiple peers simultaneously.

The RBAC construct does not require peers to have custody over one another but rather relies on the team leader to authorize the message received from C2 to execute team commands. MA will verify that all messages sent by C2 are authorized via use of role and certificate information provided during mission planning. Once the team leader has verified that the message comes from an authorized role, it will send any required messages to peers in order to support the C2 command. Upon creating or joining a team, peers will automatically accept any message sent to them by the team leader. Peers should only accept messages either from the team leader or directly from C2 to be verified with an authorized role. If the message sent to the team leader is unauthorized, MA will respond to the C2 source with the according rejection response and will not send any message to peers. For more information, see the C2 RBAC section.

9.2 Interface Interactions

The peer interactions with MA have been scoped as shown in the below figure. Within each interaction there is a baseline set of functionalities that must be supported by MA and peer systems for a compliant system.

All off-board messaging between ACPs occurs via their respective DMS which is defined in the Mission Systems section. The core method for messages to transfer between peer MAs utilizes a UCI message wrapper (*MA_TxDataPayloadCommandMT*) to send messages to the DMS and another UCI message wrapper (*MA_RxDataPayloadMT*) the DMS publishes when messages are received. These wrapper messages are specified in more detail in the Mission Systems section. The "Additional Comms Capability" MUC specifies a more holistic approach to managing the OTA communications with a Data Dissemination Plan. The Data Dissemination Plan specifies how often and whether ingress/egress messages will be subscribed/published directly on the ASB or if the UCI wrappers can be used. The MAs are responsible for authentication of the received messages before processing.



Figure 9-1: Peer to Peer Interactions

9.2.1 Peer Acknowledgment Configurations Package

9.2.1.1 P2P Message Acknowledgment Configuration Status

This interaction provides the current Message Acknowledgment Configuration defining what messages one MA will respond to with an acknowledgment upon receipt. MA will report the current Message Acknowledgment Configuration on change by sending a *MA_MessageAcknowledgmentConfigurationMT* message.

The *DefaultGroupMessageFreq* field defines the frequency at which the Grouped Acknowledgment message is set. If the field is not set, there is no grouping. The *DefaultMessagesToAck* field sets the list of messages that an Acknowledgment message will be sent for when received. If the field is not set, all messages listed in *MA_AckMessageEnum* will receive a response. The *ReceiverSpecificSettings* field specifies how MA will respond to individual Peers if they do not fall under the default settings. If MA does not support Message Acknowledgment Configuration, the *ConfigurationSupported* field will be set to "FALSE" and it will respond with an ungrouped acknowledgment for all possible messages listed in *MA_AckMessageEnum*.

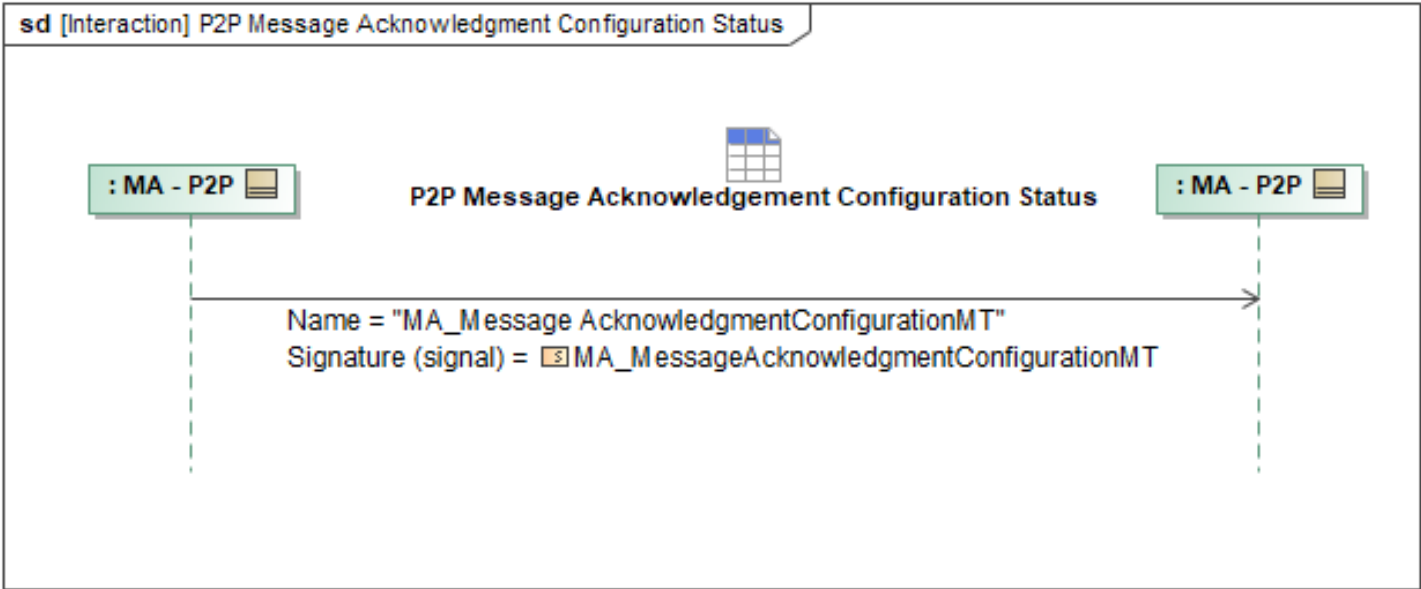


Figure 9-2: P2P Message Acknowledgment Configuration Status

Table 9-1: P2P Message Acknowledgment Configuration Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_Message AcknowledgmentConfigurationMT: MA_MessageAcknowledgmentConfigurationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.1.2 P2P Update Message Acknowledgment Configurations

This interaction specifies when one MA wants to set or update the Message Acknowledgment Configuration of another MA. MA can update default responses by filling out the

DefaultGroupMessageFreq and *DefaultMessagesToAck*. MA can also use the *RequestedReceiverSpecificSetting* field to define the messages that MA expects a Peer MA to respond with an acknowledgment for and if the messages will be grouped. MA should only request *RequestedReceiverSpecificSetting* for itself and not other systems.

MA sends the *MA_MessageAcknowledgmentConfigurationRequestMT* message to the Peer MA. The Peer MA may accept or reject the configuration request. The Peer MA accepts the request by sending a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_1* with the *RequestProcessingState* set to "COMPLETED", followed by a *MA_MessageAcknowledgmentConfigurationMT* that reflects the updated configuration.

The Peer MA may reject the configuration request for the following reasons:

- It does not support configuration. Peer MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "SYSTEM_UNAVAILABLE."
- There is limited bandwidth. Peer MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message, sets the *RequestProcessingStateReason* to "INSUFFICIENT_RESOURCES", and fills out the description field.
- The system receives an unsupported input. Peer MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "INVALID_INPUT_PARAMETER."
- MA requests *RequestedReceiverSpecificSetting* for another system. Peer MA sends a *MA_MessageAcknowledgmentConfigurationRequestStatusMT_2* message and sets the *RequestProcessingStateReason* to "INELIGIBLE_CONTROL_SOURCE."



Figure 9-3: P2P Update Message Acknowledgment Configurations

Table 9-2: P2P Update Message Acknowledgment Configurations

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MessageAcknowledgmentConfigurationMT : MA_MessageAcknowledgmentConfigurationMT	None
MA_MessageAcknowledgmentConfigurationRequestMT: MA_MessageAcknowledgmentConfigurationRequestMT	None
MA_MessageAcknowledgmentConfigurationRequestStatusMT_1: MA_MessageAcknowledgmentConfigurationRequestStatusMT	None
MA_MessageAcknowledgmentConfigurationRequestStatusMT_2: MA_MessageAcknowledgmentConfigurationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.2 Peer CAP Package

9.2.2.1 Receive CAP Execution Status

This interaction specifies the CAP Execution Status message that is sent between peers. For ACPs within a Package, Followers executing a CAP will send *MA_CAP_ExecutionStatusReportMT* periodically to other ACPs, including the Package Leader, or when their *CAP_States* have changed. Defined in the *MA_CAP_ExecutionStatusReportMT* messages is a set of package-level fields as well as a set of vehicle-level fields. At the package-level, the *PackageID* may be specified, and the *PackageRequirementID* may be defined to include the IDs the package-level CAP Task and/or action. The *AOI_Coverage* and *PlanMetrics* fields are not intended to be specified for P2P exchanges as these fields are intended to be defined when reporting status to C2 (See "Share CAP Execution Status" for more details). At the vehicle-level, *SystemID* shall be set to the ID of the ACP, *RequirementID* shall be set to the ID of the vehicle-level CAP Task, and the *CAP_Status* object shall be defined. Included in the *CAP_Status* object is the required *CAP_State* field and optional status fields. Defined in the *MA_TaskStatusMT* message are status fields of the vehicle-level CAP Tasks. The *TaskID*, *ExecutingSystemID*, and *ExecutionState* fields shall be defined. Optionally, the *TimeWindow* field may be defined to specify the remaining time left performing the CAP Task.

Within the *MA_TaskStatusMT* message, the *TimeWindow* field may be defined to specify the remaining time-on-station for performing the CAP Task.

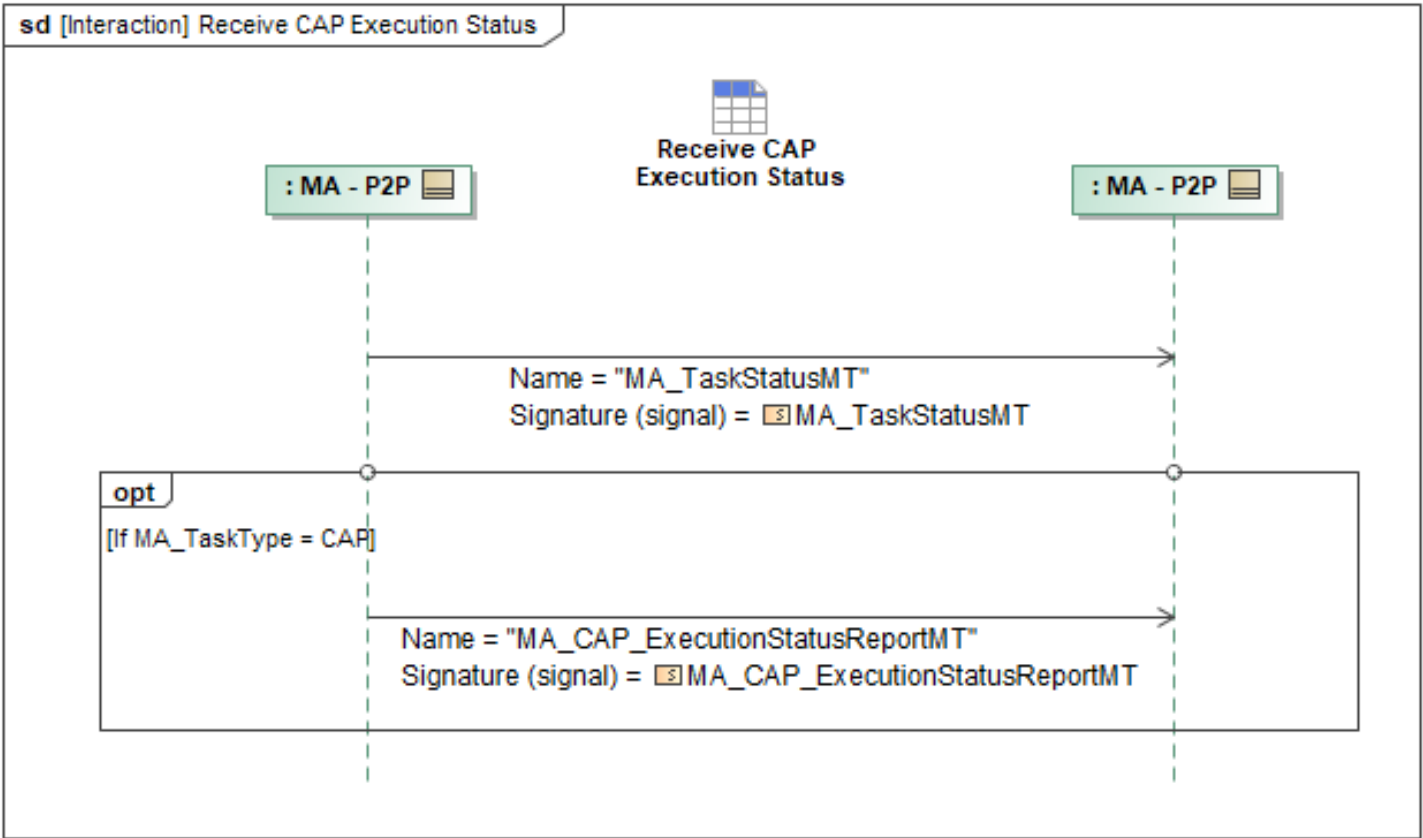


Figure 9-4: Receive CAP Execution Status

Table 9-3: Receive CAP Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CAP_ExecutionStatusReportMT: MA_CAP_ExecutionStatusReportMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.3 Peer Communications Package

9.2.3.1 Peer Overrides Data Dissemination Plan

Mission Use Case: Additional Comms Capabilities

In some situations a modification to the Data Dissemination Plan is required that are short term in nature and a complete update to the *MA_DataDisseminationPlanMT* message is not required. The *MA_DataDisseminationPlanOverrideRequestMT* can be used for these sorts of modifications. Peer-1 MA can send the *MA_DataDisseminationPlanOverrideRequestMT* message to Peer-2 MA which forwards it to Peer-2 MS. Peer-2 MS can then accept or reject the request(s). If the request(s) are accepted, Peer-2 MS sends the *MA_DataDisseminationPlanOverrideRequestStatusMT_1* message to Peer-2 MA who forwards it to Peer-1 MA with the Request(s) having the Request.RequestProcessingState set to "QUEUED", "PROCESSING", "CANCELED", or

"COMPLETED" as needed. Depending on the original request, the *EffectivityActivationStatus* or *OperatorModificationStatus* fields will be filled out as needed. The interaction "Peer Updates Data Dissemination Plan Activation Status" is initiated which will provide an update to the active status and include the *OverrideModificationState* associated with the override request status. For each applicable Gateway Activity, Peer-2 MS sends the *MA_GatewayActivityMT* message(s) to Peer-2 MA who forwards it to Peer-1 MA which will include any modifications that were made. If the request is NOT accepted, Peer-2 MS sends the *MA_DataDisseminationPlanOverrideRequestStatusMT_2* message to Peer-2 MA who forwards it to Peer-1 MA with the *RequestProcessingState* set to "REJECTED" or "FAILED", and the *RequestProcessingStateReason* filled out as needed. The *EffectivityActivationStatus* or *OperatorModificationStatus* fields will be filled out outlining the reason for the rejections/failures.

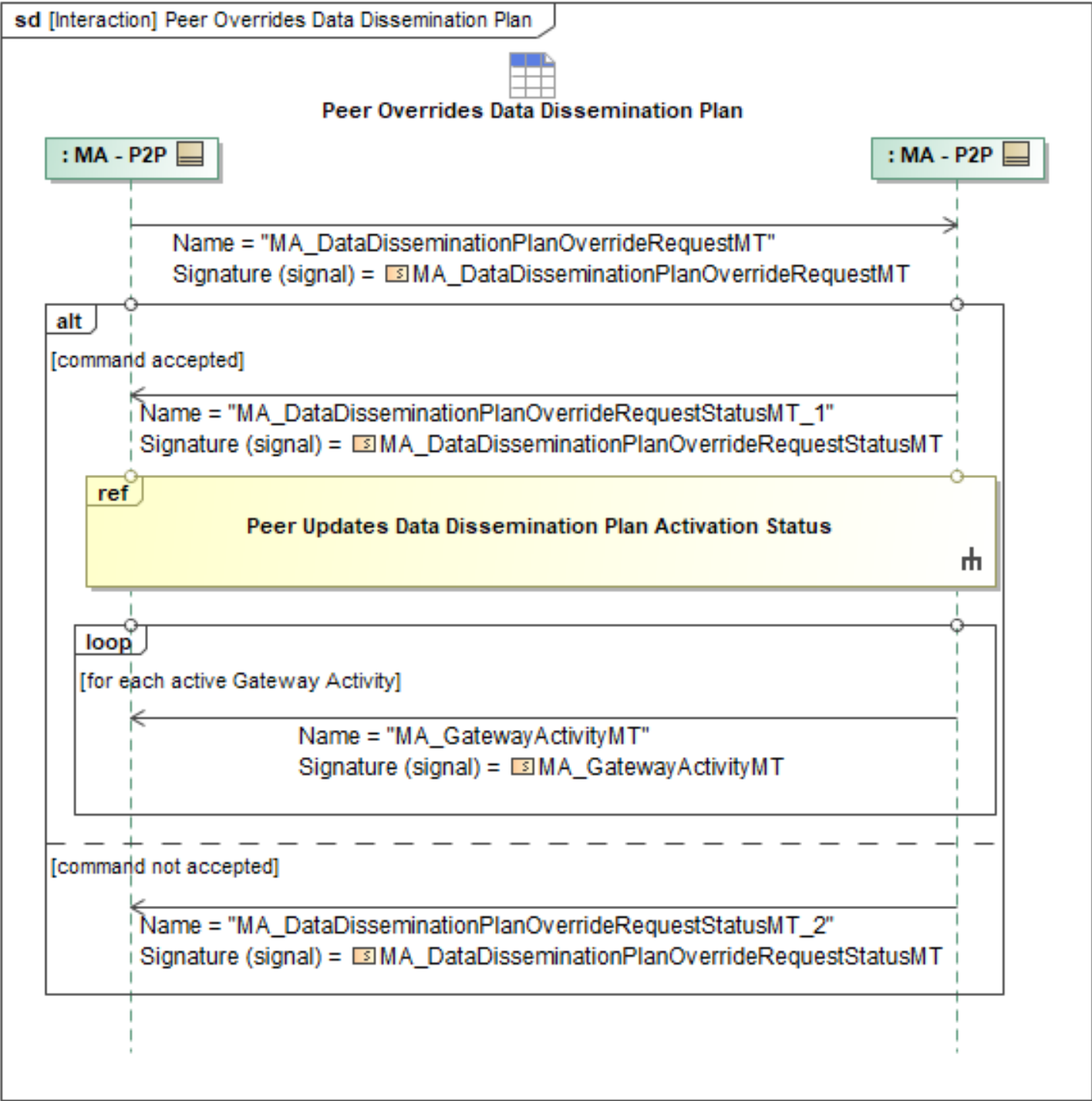


Figure 9-5: Peer Overrides Data Dissemination Plan

Table 9-4: Peer Overrides Data Dissemination Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanOverrideRequestMT	AllocatedToSystemIDs: MA_PackageSystemType
: MA_DataDisseminationPlanOverrideRequestMT	

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanOverrideRequestStatusMT_1: MA_DataDisseminationPlanOverrideRequestStatusMT	None
MA_DataDisseminationPlanOverrideRequestStatusMT_2: MA_DataDisseminationPlanOverrideRequestStatusMT	RequestProcessingStateReason: CannotComplyType
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.3.2 Peer Updates Comms Gateway Status

Mission Use Case: Additional Comms Capabilities

The Gateway Capability in the local MS is how messages egress or ingress over the air. To provide information about each of the active gateways available for use, the local MA receives and forwards the *GatewayCapabilityMT* and *GatewayCapabilityStatusMT* messages from the local MS to a peer MA. Then, for each active Gateway Activity, the local MA also forwards the *MA_GatewayActivityMT* message(s) to a peer MA. These messages provide the information needed to understand the state of the over the air messaging, they will provide context for how things are currently configured, and provide information for how to configure over the air messaging. MS typically configures the gateways, but the information can be useful when creating the plans that a MS uses.

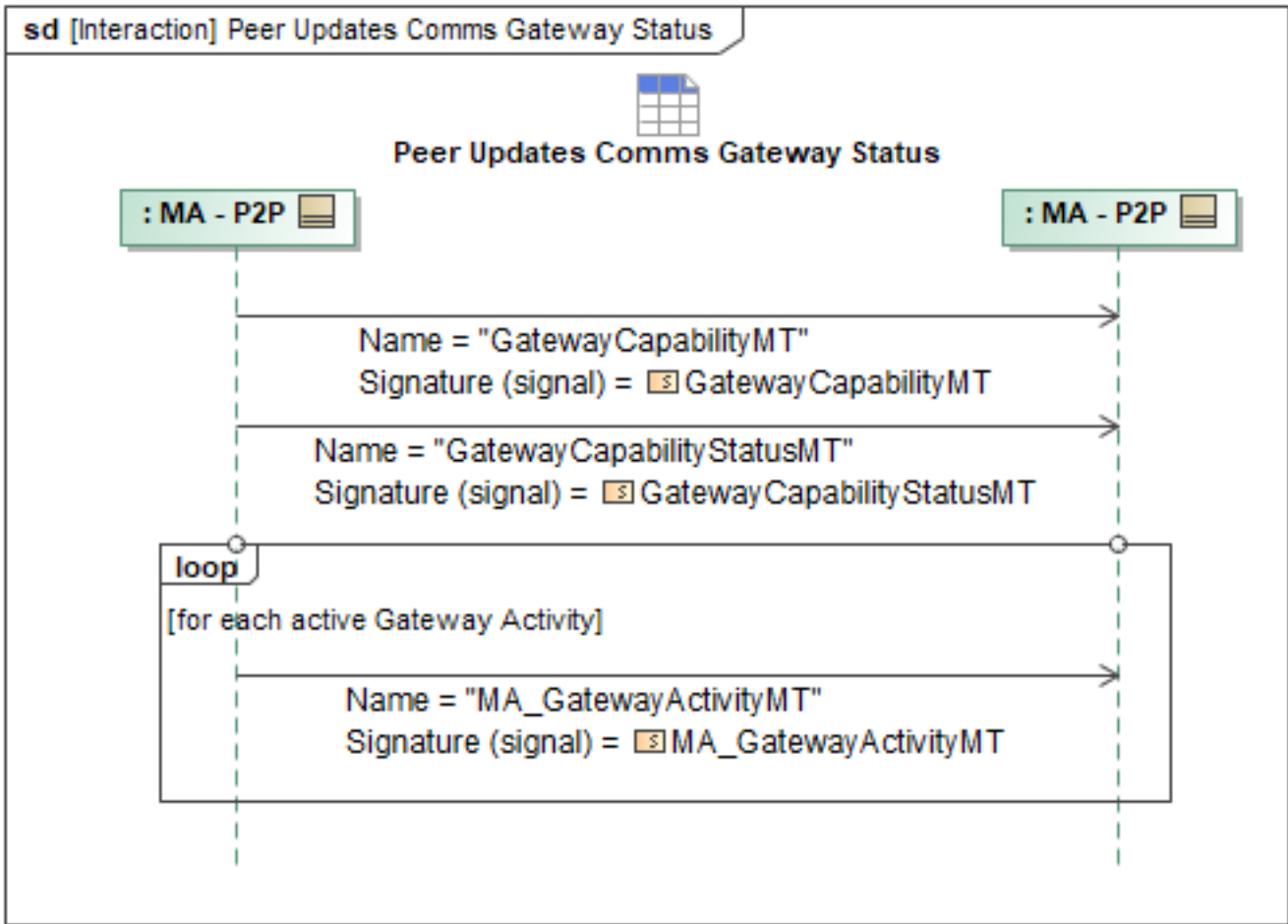


Figure 9-6: Peer Updates Comms Gateway Status

Table 9-5: Peer Updates Comms Gateway Status

Message Name (Name:Type)	Required Fields (Name:Type)
GatewayCapabilityMT: GatewayCapabilityMT	None
GatewayCapabilityStatusMT: GatewayCapabilityStatusMT	None
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.3.3 Peer Updates Data Dissemination Plan

Mission Use Case: Additional Comms Capabilities

The *MA_DataDisseminationPlanMT* is used to configure how messages ingress and egress over the air. MS manages/implements the Data Dissemination Plan and Mission Autonomy acts as a pass through if a Peer MA needs to update and/or activate the plan. The MS section "Update Data Dissemination Plan" details the contents of the *MA_DataDisseminationPlanMT* message and how it can be configured.

If the Data Dissemination Plan needs update or is a new plan, Peer-1 MA sends the *MA_DataDisseminationPlanMT* message and the *MA_DataDisseminationPlanActivationCommandMT* message to Peer-2 MA who forwards them to Peer-2 MS. If the command is accepted by Peer-2 MS, Peer-2 MS sends the *MA_DataDisseminationPlanActivationCommandStatusMT_1* message to Peer-2 MA who forwards it to Peer-1 MA with the *CommandProcessingStateEnum* set to "ACCEPTED", the interaction "Peer Updates Data Dissemination Plan Activation Status" is initiated, and for each applicable Gateway Activity Peer-2 MS sends the *MA_GatewayActivityMT* message(s) to Peer-2 MA who forwards them to Peer-1 MA. If the command is NOT accepted, Peer-2 MS send the *MA_DataDisseminationPlanActivationCommandStatusMT_2* message to Peer-2 MA who forwards it to Peer-1 MA with the *CommandProcessingStateEnum* set to "REJECTED" and the *PlanActivationCommandStatus* is set according to the reason it was rejected.

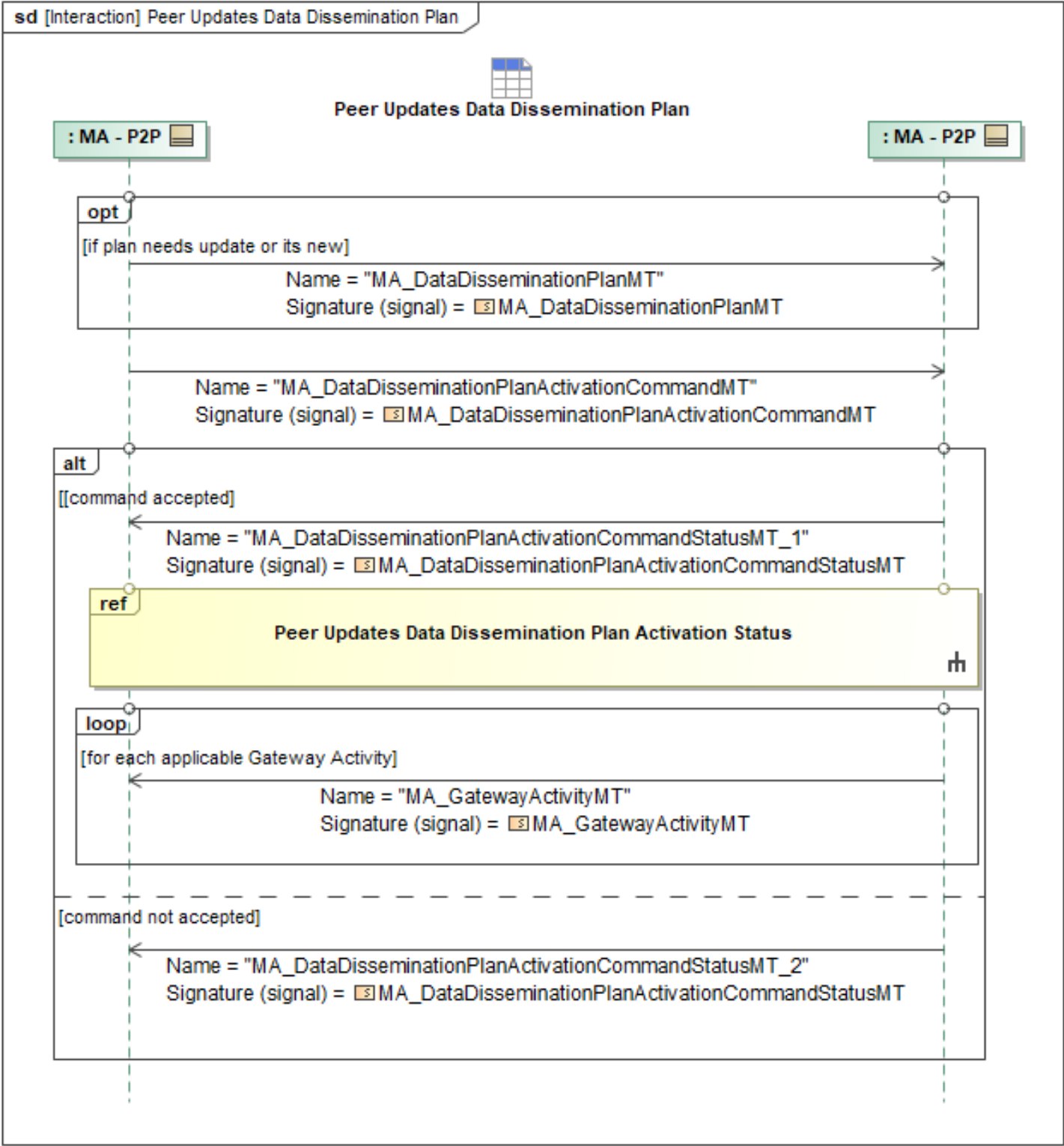


Figure 9-7: Peer Updates Data Dissemination Plan

Table 9-6: Peer Updates Data Dissemination Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanActivationCommandMT: MA_DataDisseminationPlanActivationCommandMT	Applicability: MA_PackageSystemType
MA_DataDisseminationPlanActivationCommandStatusMT_1: MA_DataDisseminationPlanActivationCommandStatusMT	None
MA_DataDisseminationPlanActivationCommandStatusMT_2: MA_DataDisseminationPlanActivationCommandStatusMT	RequestProcessingStateReason: CannotComplyType
MA_DataDisseminationPlanMT: MA_DataDisseminationPlanMT	None
MA_GatewayActivityMT: MA_GatewayActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.3.4 Peer Updates Data Dissemination Plan Activation Status

Mission Use Case: Additional Comms Capabilities

The local MS will send the *MA_DataDisseminationPlanActivationStatusMT* message to the local MA who forwards it to a peer MA which reflect the active state of local Data Dissemination Plan. This can be sent periodically, but will always be sent upon change to the active state. The message will reflect the active Effectivities and Override Modifications of the local Data Dissemination Plan.

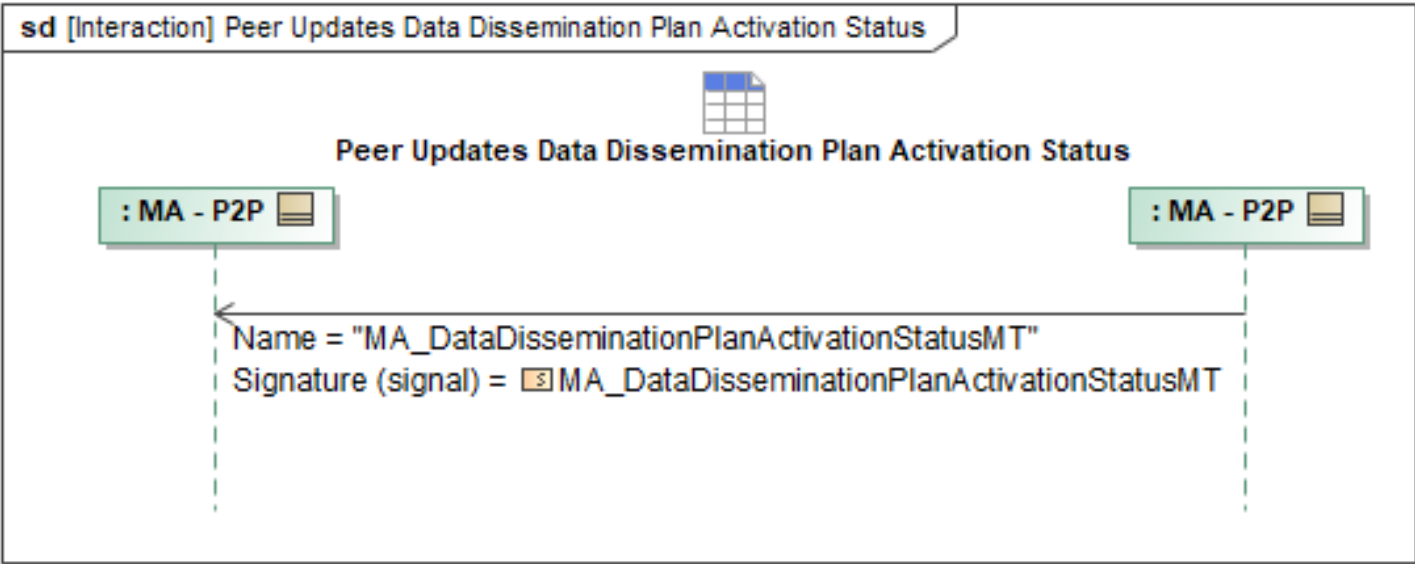


Figure 9-8: Peer Updates Data Dissemination Plan Activation Status

Table 9-7: Peer Updates Data Dissemination Plan Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DataDisseminationPlanActivationStatusMT : MA_DataDisseminationPlanActivationStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4 Peer Contingencies Package

In the event of a lost communications between members or a team, or a Team Leader and C2 the team will behave in the ways described below. It is important to note that MA assumes that loss of communications to a node, whether it be C2 or a peer, is reported by the Mission Systems Communications Manager or is determined by the absence of a heartbeat message past a configurable timeout interval. During planned communication outages, MA depends on MS for lost comms notification. Further material on these requirements can be found in section 4.6 Mission Systems Interface.

9.2.4.1 Define Autonomous Contingency Re-Plan

MA Team Leads can configure Peer MA to initiate autonomous replanning in the event of a contingency. Team Lead MA will send a *MA_PlanningFunctionSettingsCommandMT* to set the contingency trigger and indicate the desired replanning that should occur if the contingency condition is met. *MissionPlanningAutonomy.AutonomousPlanningResponseChoice* field should use *AutonomousPlanningAction* or *AlertOnly* field should be "FALSE". *MissionPlanningAutonomy.AutonomousPlanningResponseChoice.AutonomousPlanningAction* is used to configure what type of replanning Peer MA should initiate. *MissionPlanningAutonomy.AutonomousPlanningResponseChoice.AutonomousPlanningAction* should include an *AutonomousPlanningActionID* to be used to reference this planning action, as well as *AllowedType* field which will configure the specifics of the planning process.

AllowedType.PlanningProcessID should be set to the desired ID for this planning process and will be used to reference this process in future messages. *AllowedType.ForPlanningUseOnly* should be set to "FALSE" as this plan must be executed to resolve the contingency.

When Peer MA receives *MA_PlanningFunctionSettingsCommandMT*, it replies with *MA_PlanningFunctionSettingsCommandStatusMT*, with *CommandedProcessingState* set to "ACCEPTED" if the system is able to apply the autonomy settings or "REJECTED" if the system is not able to apply these settings. Should it be rejected, the *CommandProcessingStateReason* field will be populated with the reason for the rejection of the *PlanningFunctionSettingsCommand*.

Peer MA will determine whether Team Lead MA is authorized to command the initial *MA_PlanningFunctionSettingsCommandMT*. If Team Lead MA is unauthorized, Peer MA will ignore the *MA_PlanningFunctionSettingsCommandMT* command and reply with *MA_PlanningFunctionSettingsCommandStatusMT* where *CommandProcessingStateEnum* is set to "REJECTED" and *CommandProcessingStateReason* shall define the *CannotComplyEnum* as "INELIGIBLE_CONTROL_SOURCE".

In order for Peer MA to execute a generated plan, the plan must first be approved per the *ApprovalPolicy*. Mission Planning will set the criteria for what plans can be approved with the use of *MA_ApprovalPolicyMT* by an operator vs. which plans can be automatically activated by MA.

In the event Team Lead MA wishes to modify an existing approval policy, Team Lead MA sends an *MA_ApprovalPolicyMT* message, which includes a *DataRecordInstanceID*. For contingencies, the approval policy will have an *ApprovalPolicyID* set to the desired ID of the *ApprovalPolicy*. The *Plans* field should have *Plans.ApplicablePlanningActivity* set to *_PLAN, where the asterisk indicates the appropriate requirement for this planning service. Additionally, the approval policy should have *ByTriggerPolicy.TriggerSource* set to "AUTONOMOUS_TRIGGER" to indicate this policy applies to replanning from autonomous triggers and not operator-commanded replanning.

ByTriggerPolicy.Trigger can be used to specify that this policy is only for a specific type of trigger. The *MA_ApprovalPolicyMT* will also have *PlanActivation.Policy.ApprovalRequirement* set to either "AUTO_APPROVE" if Team Lead MA wishes the contingency replan to be executed immediately, or "APPROVAL_REQUIRED" which will ask Team Lead MA for approval of the proposed plan immediately before execution. *MA_ApprovalPolicyMT* with the desired settings, specifically *Plans.ByTriggerPolicy.Policy* can be set to "APPROVAL_REQUIRED" to require MA to wait for operator approval before activating an autonomous replan in response to the defined contingency trigger.

For more information on execution sequences with respect to autonomous replanning, see section Triggered Autonomous Contingency Replan.

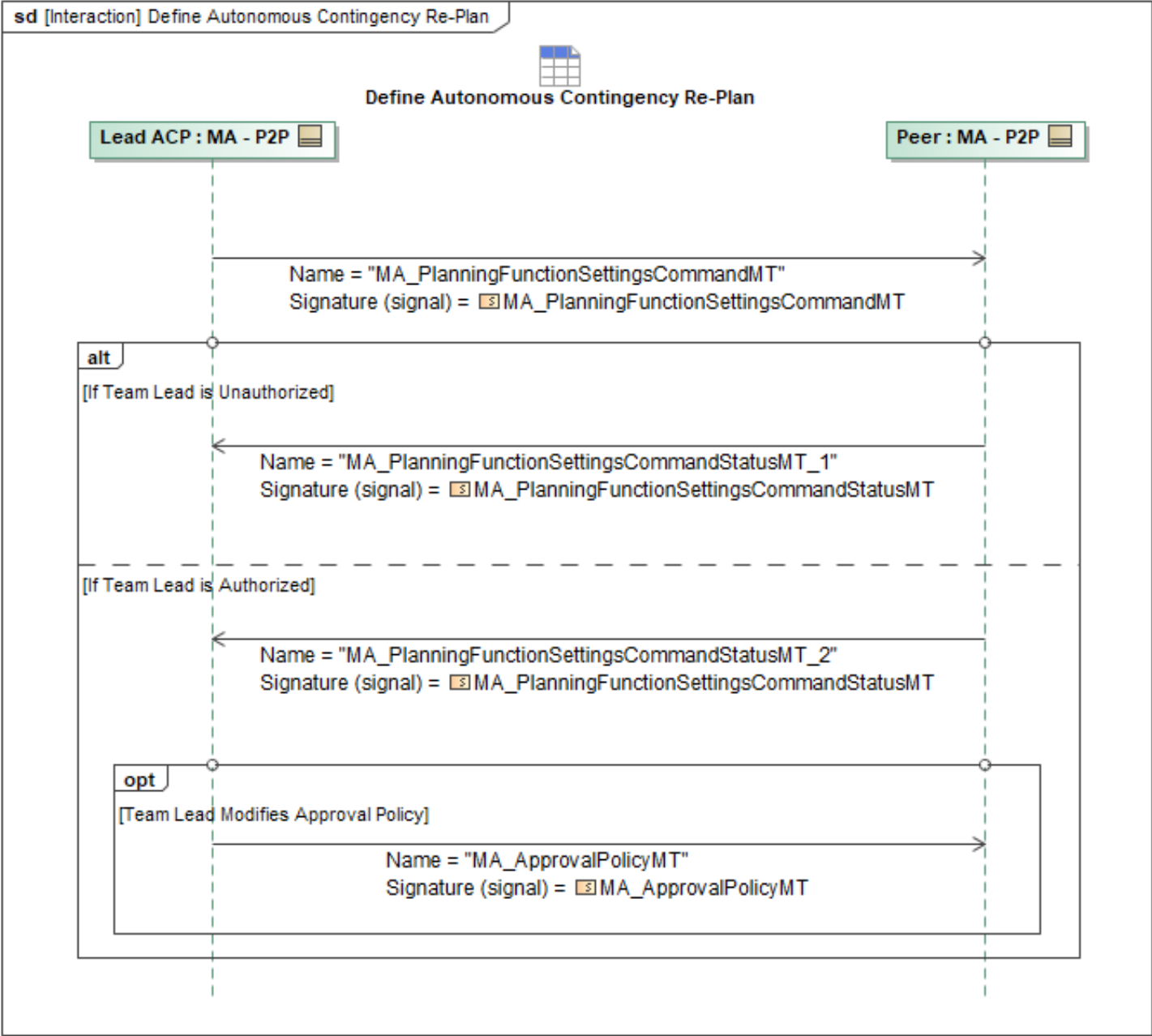


Figure 9-9: Define Autonomous Contingency Re-Plan

Table 9-8: Define Autonomous Contingency Re-Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	DataRecordInstanceID: DataRecordInstanceID_Type PlanActivation: MA_PlanActivationPolicyType Plans: MA_PlanPolicyType ByTriggerPolicy: ByTriggerPolicyType Policy: ApprovalPolicyBaseType

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanningFunctionSettingsCommandMT: MA_PlanningFunctionSettingsCommandMT	MissionPlanningAutonomy: MA_MissionPlanningAutonomySettingType
MA_PlanningFunctionSettingsCommandStatus MT_1: MA_PlanningFunctionSettingsCommandStatus MT	CommandProcessingStateReason: CannotComplyType
MA_PlanningFunctionSettingsCommandStatus MT_2: MA_PlanningFunctionSettingsCommandStatus MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.2 Define Team Lost Comms Contingency

In order to set up a lost communication contingency between a peer and its team, C2 systems send a *MA_SystemManagementRequestMT* to the Team Leader of a team. (See section *Define Team Contingency* for more details on the Team Leader's communication with C2). When the Team Leader Receives a *MA_SystemManagementRequestMT*, it will configure itself in accordance with the message, and then send each of its team members a *MA_SystemManagementRequestMT* instructing them to configure themselves in the same way. Once a team member successfully configures itself, it will send back a *MA_SystemManagementRequestStatusMT* to the Team Leader with the *RequestProcessingState* set to "COMPLETED". If a team member is not able to configure themselves, it will instead set the *RequestProcessingState* to "FAILED" and send at least one *RequestProcessingStateReason* for why it cannot apply the changes.

Once the team's contingency trigger has been configured, the Team Leader will send a *ResponsePlanMT* to each of its team members. To begin execution of the new *ResponsePlanMT*, the Team Leader will send a *MissionPlanActivationCommandMT* with *Command.MissionPlanID* set to the ID of the *ResponsePlanMT*. Once activated, response monitoring will begin and each team member will start sending periodic *MA_ResponseActivityMTs* to their Team Leader. An *ActivityStatus* of "ENABLED" means the trigger is currently being monitored. An *ActivityStatus* of "ACTIVE_*" indicates the response trigger was tripped and the response is currently executing the desired *Plan. An *ActivityStatus* of "COMPLETED" means that the desired *Plan has finished execution. (See C2 *Contingency Management* section for more details on Responses).

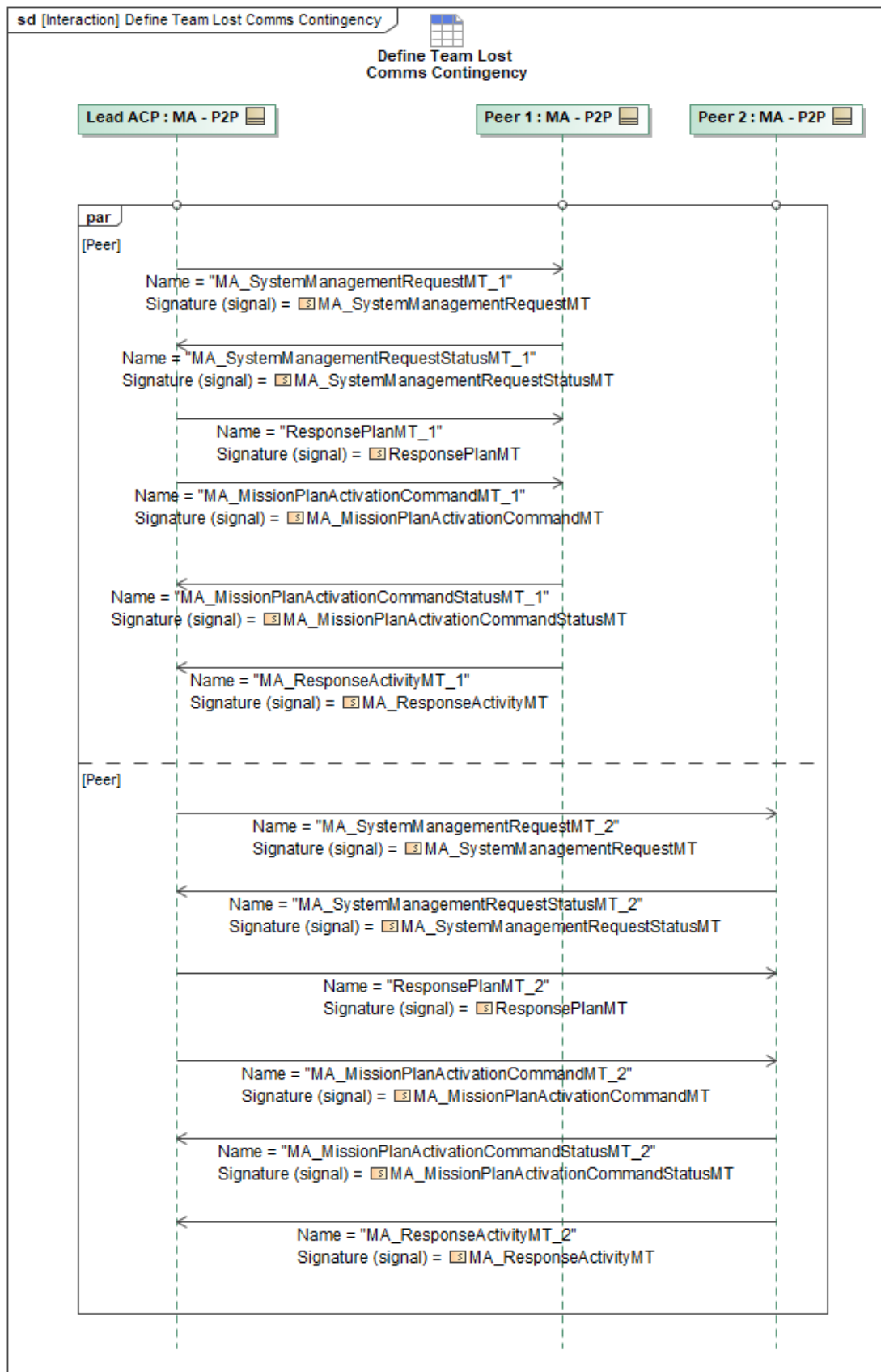


Figure 9-10: Define Team Lost Comms Contingency

Table 9-9: Define Team Lost Comms Contingency

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT_1: MA_MissionPlanActivationCommandMT	ResponsePlan: ResponsePlanActivationType
MA_MissionPlanActivationCommandMT_2: MA_MissionPlanActivationCommandMT	ResponsePlan: ResponsePlanActivationType
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	CommandStatus: ProcessingStatusEnum
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	CommandStatus: ProcessingStatusEnum
MA_ResponseActivityMT_1: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type
MA_ResponseActivityMT_2: MA_ResponseActivityMT	AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type
MA_SystemManagementRequestMT_1: MA_SystemManagementRequestMT	LostCommTimeout: DurationType
MA_SystemManagementRequestMT_2: MA_SystemManagementRequestMT	LostCommTimeout: DurationType
MA_SystemManagementRequestStatusMT_1: MA_SystemManagementRequestStatusMT	None
MA_SystemManagementRequestStatusMT_2: MA_SystemManagementRequestStatusMT	None
ResponsePlanMT_1: ResponsePlanMT	AllocatedResponse: ResponseAllocationType PlanCommandID: ResponsePlanCommandID_ChoiceType
ResponsePlanMT_2: ResponsePlanMT	AllocatedResponse: ResponseAllocationType PlanCommandID: ResponsePlanCommandID_ChoiceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.3 Multiple Peers Lost Comms with Team Leader

In the event that multiple peers are disconnected from the Team Leader, the team will split into two separate teams. The side of the split with the original Team Leader will retain its original *PackageID*, but update C2 on who the remaining members are. See section "*Team Leader Lost Comms with Multiple Team Members*" in the C2 section for more information on Team Leaders communication with C2. If it the team leader cannot connect to a C2 system, refer to section "*C2 Lost Comms with*

Team Leader."

The side of the split that doesn't have a leader will immediately begin the process of forming a new team among the peers they still have communications with (see section "Create a Full Package Amongst Peers.") This includes initialization of the new package and election of a new Team Leader. Once elected, the new Team Leader will try to connect to a C2 system and update C2 on the status of the second half of the split team. See section "*Team Leader Lost Comms with Multiple Team Members*" in the C2 section for more information. In the event that the new Team Leader can't communicate with C2, see section "*Team Leader Lost Comms with C2.*"

In the event that a team was separated into two groups, and they are able to regain communication, then the two teams will stay as two separate teams until commanded to merge. See section "*Team Leader Lost Comms with Multiple Team Members*" in the C2 section for more information on notifying C2.

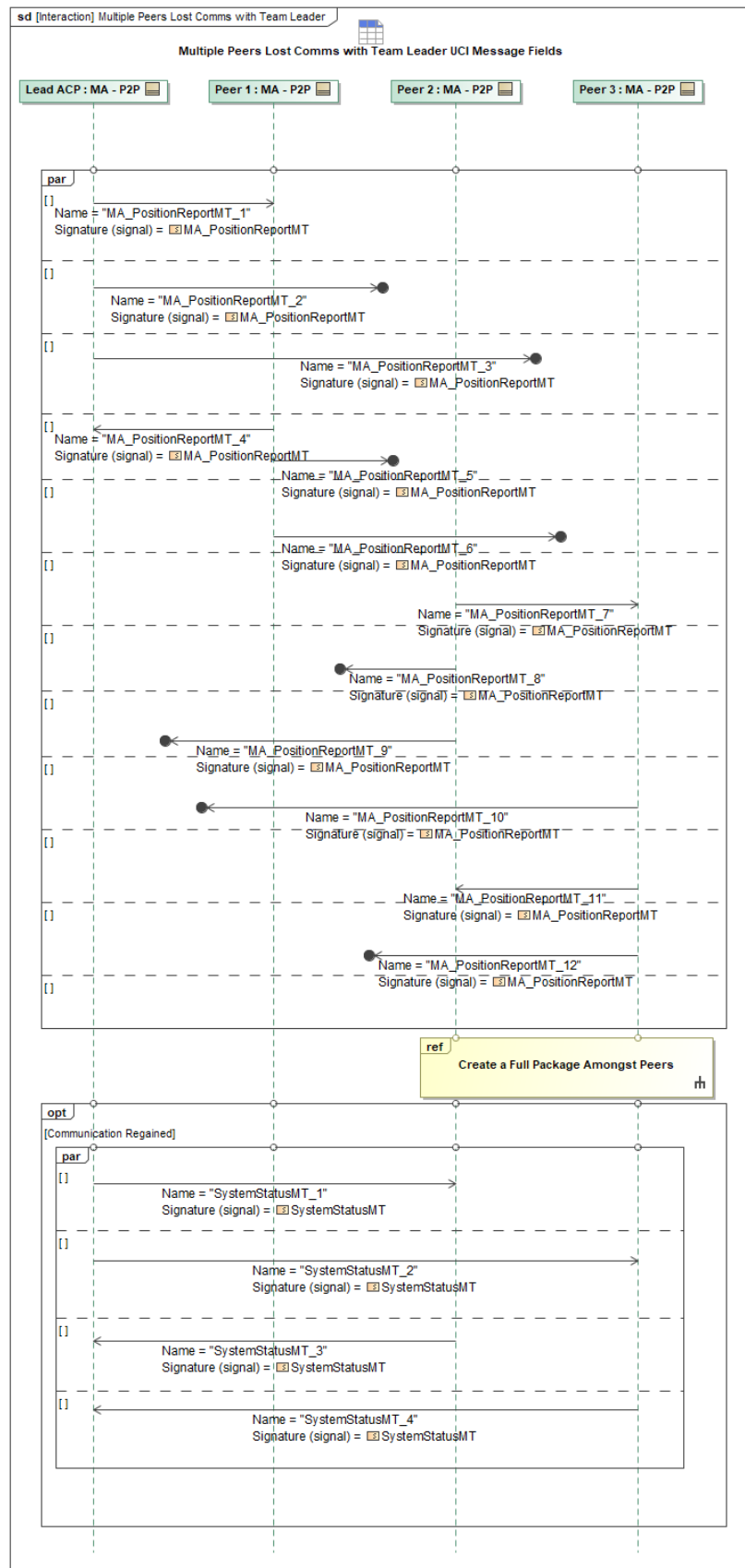


Figure 9-11: Multiple Peers Lost Comms with Team Leader

Table 9-10: Multiple Peers Lost Comms with Team Leader

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PositionReportMT_1: MA_PositionReportMT	CalibratedAirspeed: SpeedType DomainVelocity: Velocity3D_Type Mach: MachType GroundVelocity: Velocity2D_Type
MA_PositionReportMT_2: MA_PositionReportMT	GroundVelocity: Velocity2D_Type Mach: MachType CalibratedAirspeed: SpeedType DomainVelocity: Velocity3D_Type
MA_PositionReportMT_3: MA_PositionReportMT	Mach: MachType DomainVelocity: Velocity3D_Type CalibratedAirspeed: SpeedType GroundVelocity: Velocity2D_Type
MA_PositionReportMT_4: MA_PositionReportMT	Mach: MachType CalibratedAirspeed: SpeedType GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type
MA_PositionReportMT_5: MA_PositionReportMT	GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type CalibratedAirspeed: SpeedType Mach: MachType
MA_PositionReportMT_6: MA_PositionReportMT	DomainVelocity: Velocity3D_Type CalibratedAirspeed: SpeedType Mach: MachType GroundVelocity: Velocity2D_Type
MA_PositionReportMT_7: MA_PositionReportMT	CalibratedAirspeed: SpeedType Mach: MachType GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type
MA_PositionReportMT_8: MA_PositionReportMT	Mach: MachType CalibratedAirspeed: SpeedType DomainVelocity: Velocity3D_Type GroundVelocity: Velocity2D_Type
MA_PositionReportMT_9: MA_PositionReportMT	CalibratedAirspeed: SpeedType GroundVelocity: Velocity2D_Type Mach: MachType

Message Name (Name:Type)	Required Fields (Name:Type)
	DomainVelocity: Velocity3D_Type
MA_PositionReportMT_10: MA_PositionReportMT	CalibratedAirspeed: SpeedType DomainVelocity: Velocity3D_Type GroundVelocity: Velocity2D_Type Mach: MachType
MA_PositionReportMT_11: MA_PositionReportMT	DomainVelocity: Velocity3D_Type Mach: MachType GroundVelocity: Velocity2D_Type CalibratedAirspeed: SpeedType
MA_PositionReportMT_12: MA_PositionReportMT	Mach: MachType CalibratedAirspeed: SpeedType GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type
SystemStatusMT_1: SystemStatusMT	None
SystemStatusMT_2: SystemStatusMT	None
SystemStatusMT_3: SystemStatusMT	None
SystemStatusMT_4: SystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.4 P2P Incorrect Messaging Sequence Error Handling

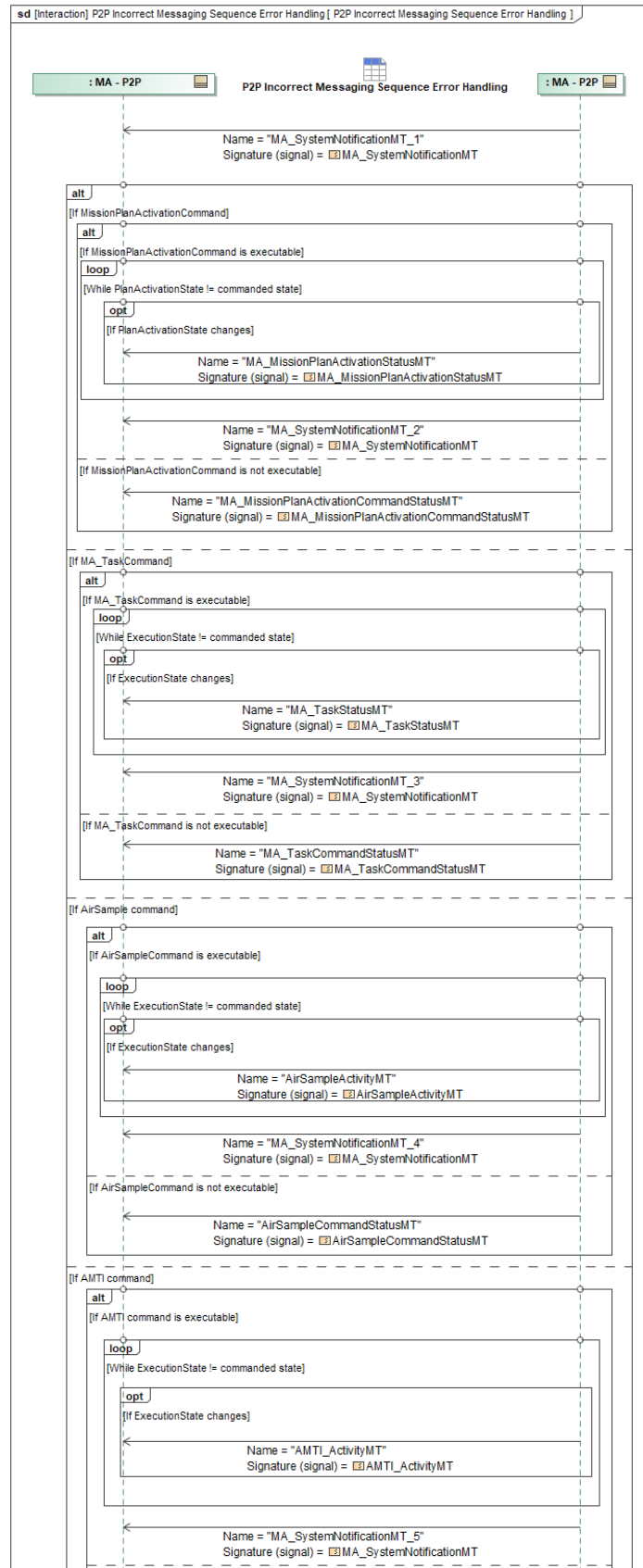
When MA receives a command, it is possible that the commanded state of an object (e.g., *MissionPlan*, *Task*, *EA_Capability*) is more than one state away from the object's current state. If this is the case, an Incorrect Messaging Sequence error is triggered. Upon identification of an Incorrect Messaging Sequence error, MA will send *MA_SystemNotificationMT* to Peer indicate that the commanded state is out of sequence and that corrective state transitions will be performed. Within the message, MA sets *NotificationState* to *ACTIVE* and sets *NotificationDetail.Reason* to *STATE_OR_SETTINGS*. *NotificationState* communicates that the error exists and *NotificationDetail.Reason* indicates that the error exists due to the object's current state. *AssociatedMessage.MessageType* identifies the command associated with the error, and *AssociatedMessage.AssociatedID* will reference the ID of the associated command.

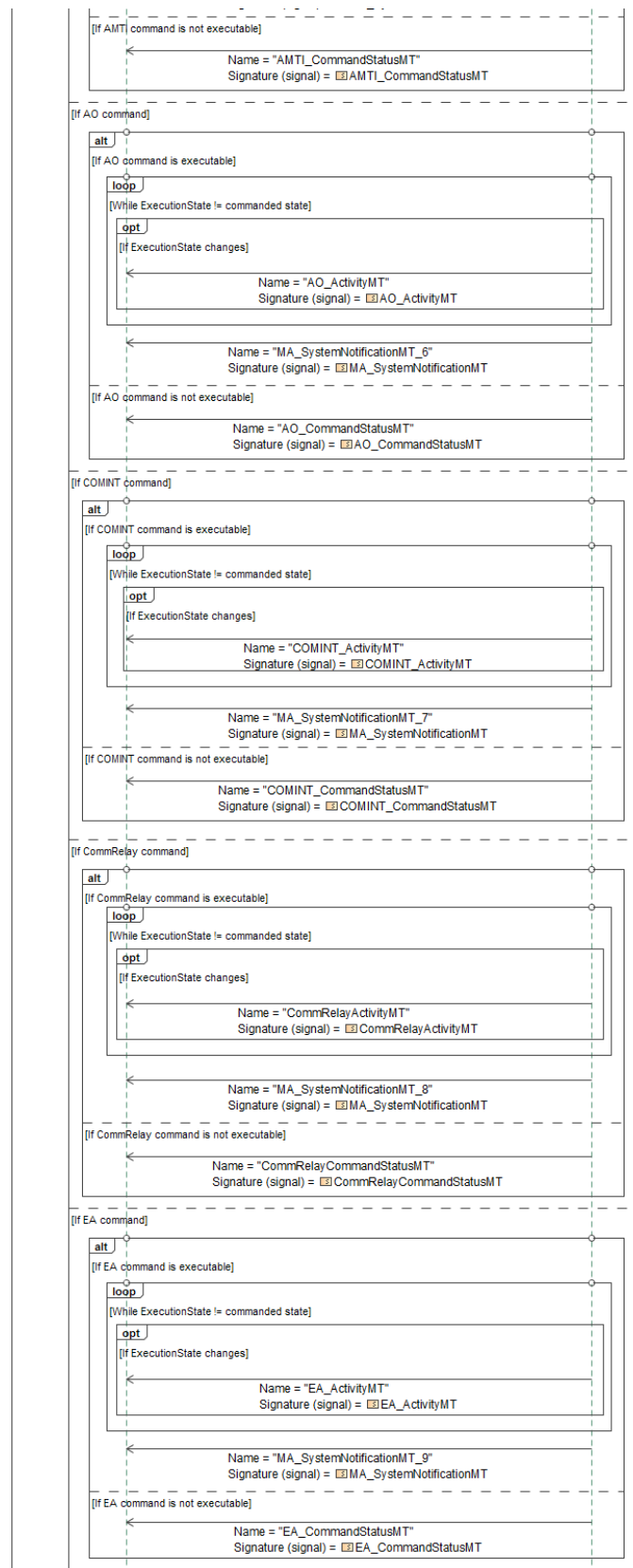
If MA determines that the command cannot be carried out for any reason, MA will publish the corresponding **CommandStatus* message with *CommandProcessingState* set to *REJECTED*. If MA determines that the command can be carried out, MA will publish the corresponding **CommandStatus* message with *CommandProcessingState* set to *ACCEPTED*, and will progress the object's status towards the commanded state.

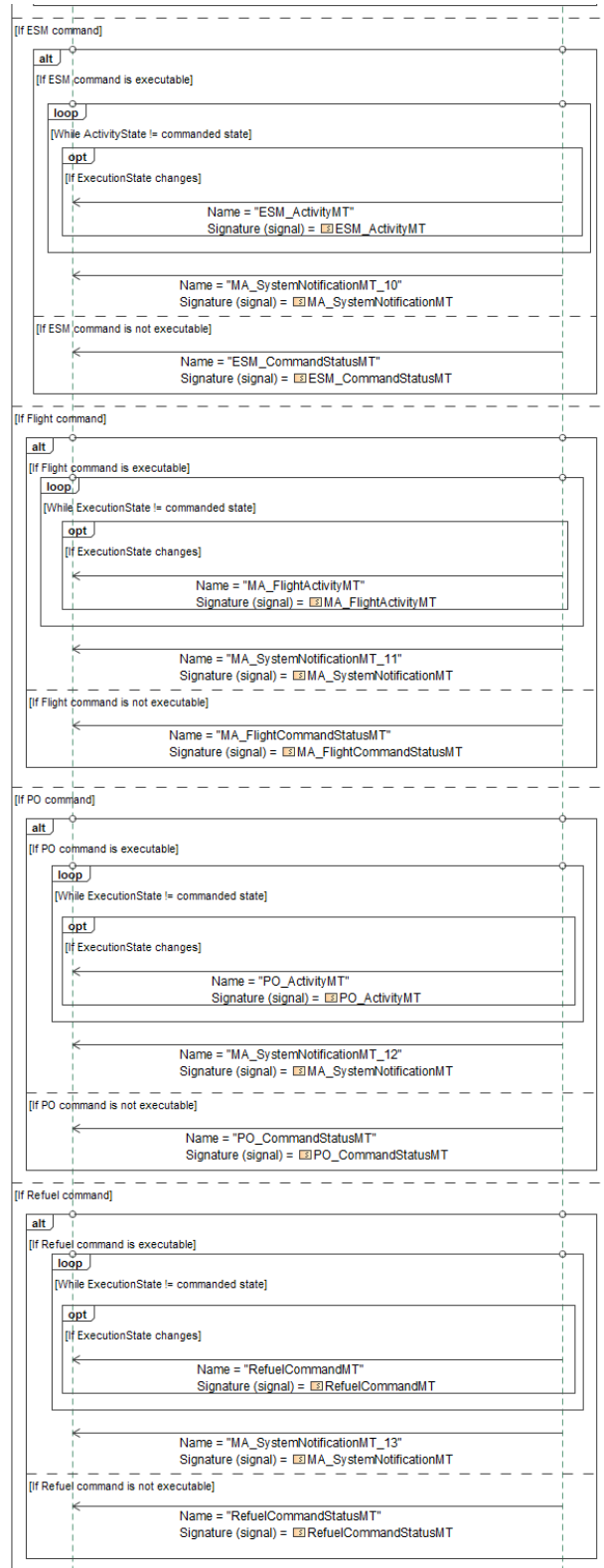
MA communicates the object's status when the object's status changes during corrective actions. Status updates are not intended to be periodic; they are sent when MA transitions the object to a different state while progressing toward the commanded state. Object status reflects the object's

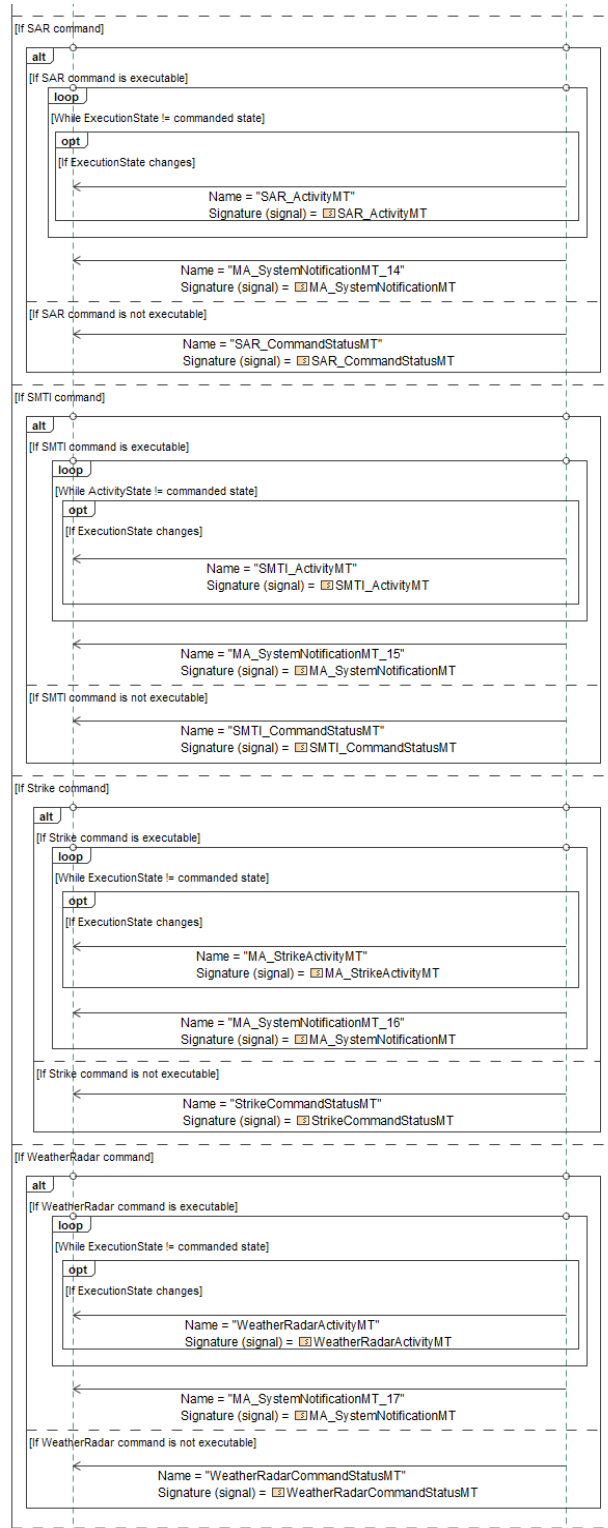
actual state at the time the status is reported and provides visibility into the intermediate transitions performed by MA during resolution of the Incorrect Messaging Sequence error. The applicable status message depends on the affected object. For mission plan activation commands, MA uses *MA_MissionPlanActivationStatus* to communicate the current *PlanActivationState* (e.g., *READY_FOR_ACTIVATION*, *ACTIVATING*, *ACTIVATED*, *ACTIVE_DEGRADED*, *DEACTIVATING*, and *DEACTIVATED*). For a TaskCommand, MA uses *MA_TaskStatusMT* to communicate the current *ExecutionState* (e.g. *AWAITING_PLAN_ACTIVATION*, *EXECUTING*, *COMPLETED*, *DROPPED*, *FAILED*, *CANCELED*). For UCI capability commands, MA uses the capability's corresponding **Activity message* (e.g., *EA_Activity*, *AMTI_Activity*, *AO_Activity*) to communicate the current *ActivityState* (e.g., *ENABLED*, *ACTIVE_PARTIALLY_CONSTRAINED*, *ACTIVE_UNCONSTRAINED*, *COMPLETED*, *FAILED*).

The commanded state represents the state requested by the originating command and serves as the target state against which MA evaluates progress during executing corrective actions. When the current object state matches the commanded state, MA considers the Incorrect Messaging Sequence Error to be resolved and will send updated *MA_SystemNotificationMT* with *NotificationState* set to *CLOSED*.









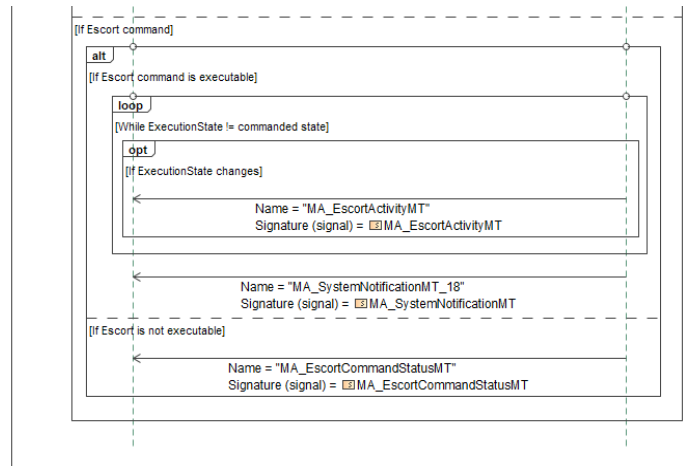


Figure 9-12: P2P Incorrect Messaging Sequence Error Handling

Table 9-11: P2P Incorrect Messaging Sequence Error Handling

Message Name (Name:Type)	Required Fields (Name:Type)
AirSampleActivityMT: AirSampleActivityMT	None
AirSampleCommandStatusMT: AirSampleCommandStatusMT	None
AMTI_ActivityMT: AMTI_ActivityMT	None
AMTI_CommandStatusMT: AMTI_CommandStatusMT	None
AO_ActivityMT: AO_ActivityMT	None
AO_CommandStatusMT: AO_CommandStatusMT	None
COMINT_ActivityMT: COMINT_ActivityMT	None
COMINT_CommandStatusMT: COMINT_CommandStatusMT	None
CommRelayActivityMT: CommRelayActivityMT	None
CommRelayCommandStatusMT: CommRelayCommandStatusMT	None
EA_ActivityMT: EA_ActivityMT	None
EA_CommandStatusMT: EA_CommandStatusMT	None
ESM_ActivityMT: ESM_ActivityMT	None
ESM_CommandStatusMT: ESM_CommandStatusMT	None
MA_EscortActivityMT: MA_EscortActivityMT	None
MA_EscortCommandStatusMT: MA_EscortCommandStatusMT	None
MA_FlightActivityMT: MA_FlightActivityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_MissionPlanActivationCommandStatusMT: MA_MissionPlanActivationCommandStatusMT	None
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	None
MA_StrikeActivityMT: MA_StrikeActivityMT	None
MA_SystemNotificationMT_1: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_2: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_3: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_4: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_5: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_6: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_7: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_8: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_9: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_10: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_11: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_12: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_13: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_14: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_15: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_16: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_17: MA_SystemNotificationMT	AssociatedID: ID_Type
MA_SystemNotificationMT_18: MA_SystemNotificationMT	AssociatedID: ID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None
PO_ActivityMT: PO_ActivityMT	None
PO_CommandStatusMT: PO_CommandStatusMT	None
RefuelCommandMT: RefuelCommandMT	None
RefuelCommandStatusMT: RefuelCommandStatusMT	None
SAR_ActivityMT: SAR_ActivityMT	None
SAR_CommandStatusMT: SAR_CommandStatusMT	None
SMTI_ActivityMT: SMTI_ActivityMT	None
SMTI_CommandStatusMT: SMTI_CommandStatusMT	None
StrikeCommandStatusMT: StrikeCommandStatusMT	None
WeatherRadarActivityMT: WeatherRadarActivityMT	None
WeatherRadarCommandStatusMT: WeatherRadarCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.5 P2P Malformed Message/Precondition Not Met Handling

Upon receipt of a malformed message, MA will send *MA_SystemNotificationMT* to P2P, indicating that the message could not be successfully processed. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with *INVALID_INPUT_PARAMETER*. *NotificationDetail.Description* will contain a human-readable string stating which fields are malformed. *AssociatedMessage* will reference the malformed message. The *MessageType* will be set to the same type as the malformed message. *AssociatedID* will be populated with a reference ID from the malformed message if applicable, the *UUID* of that message. If additional data is required to resolve the condition and the fault remains active, MA can issue *QueryDataRequestMT* with the Query field containing the ID of the specific message. P2P would then respond with *QueryDataRequestStatusMT*, specifically the Message field within the Result field will contain the corrected message containing the originally missing data. If the fault has been addressed, then MA will only send a new *MA_SystemNotificationMT* with the same fields as above and *NotificationState* set to CLOSED.

Upon receipt of a command from P2P for which the required inputs are not present on the platform/data store, MA will send *MA_SystemNotificationMT* to P2P, indicating that the command could not be successfully executed. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with *UNKNOWN_ID*. *NotificationDetail.Description* will contain a

human-readable string stating which data is not present. *AssociatedMessage* will reference the attempted command. The *MessageType* will be set to the same type as the command. *AssociatedID* will be populated with a reference ID from the command if applicable, the *UUID* of that command. If additional data is required to resolve the condition and the fault remains active, MA can issue *QueryDataRequestMT* with the Query field containing the ID of the specific command. P2P would then respond with *QueryDataRequestStatusMT*, specifically the Message field within the Result field will contain the corrected command containing the originally missing data. If the fault has been addressed, then MA will only send a new *MA_SystemNotificationMT* with the same fields as above and *NotificationState* set to CLOSED.

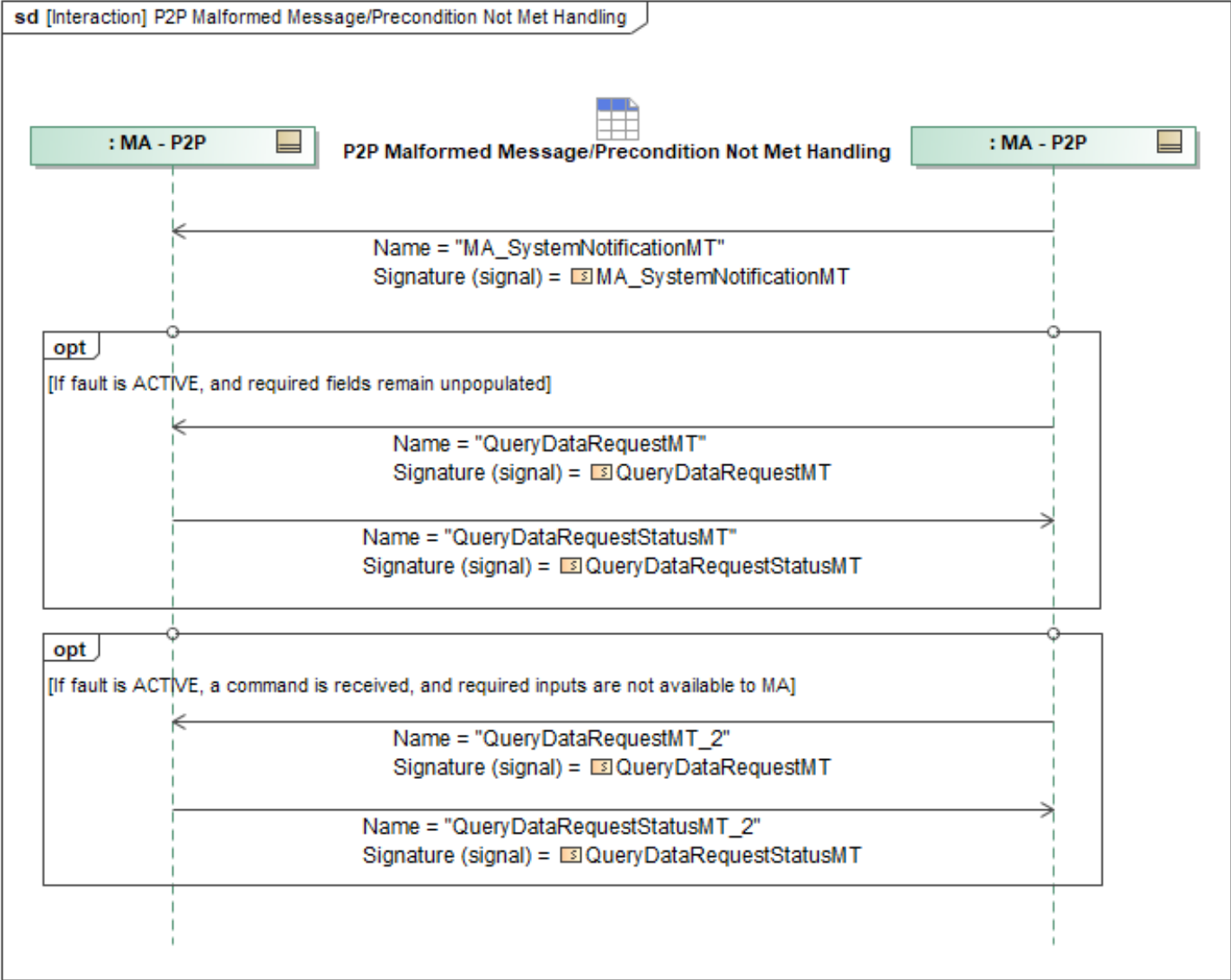


Figure 9-13: P2P Malformed Message/Precondition Not Met Handling

Table 9-12: P2P Malformed Message/Precondition Not Met Handling

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	Description: VisibleString1024Type AssociatedID: ID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	Query: QueryPET
QueryDataRequestMT_2: QueryDataRequestMT	None
QueryDataRequestStatusMT: QueryDataRequestStatusMT	Message: MessageType
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.6 P2P Path Deconfliction Proposal

During the collaborative path deconfliction process, MA sends the current proposed deconfliction solution via an *MA_DeconflictionSolutionRequestMT* to connected peers it has a path conflict with. This message includes the projected trajectory of the sending MA in the *RoutePlan* field, a unique *ConflictID*, and a *ConflictStatusEnum* indicating whether the conflict has been resolved.

After receipt of the *MA_DeconflictionSolutionRequestMT*, the peer in conflict responds with an *MA_DeconflictionSolutionRequestStatusMT*, setting the *RequestProcessingState* to one of *QUEUED*/*PROCESSING*/*COMPLETED*/*FAILED*/*CANCELED*/*REJECTED*.

The connected peers in conflict responds using *MA_DeconflictionSolutionRequestMT_1* with the *RequestProcessingState* field is set to *QUEUED* upon receiving a request and is updated once the request can be processed. If *RequestProcessingState* is set to *FAILED*, *CANCELED*, or *REJECTED*, the *MA_DeconflictionSolutionRequestStatusMT_3* message will include an accompanying *RequestProcessingStateReason*. Once the *RequestProcessingState* is set to *COMPLETED*, the *MA_DeconflictionSolutionRequestStatusMT_2* will include a *DeconflictionProposalStatus* field will be set to either *ACCEPTED* or *REJECTED*. If *DeconflictionProposalStatus* field is set to *ACCEPTED*, the message will include the *ConflictID* and *RoutePlanID* to indicate the specific conflict and route plan of the sequence originating ACP that are being responded to. If *DeconflictionProposalStatus* field is set to *REJECTED*, this will indicate to the sequence originating ACP that the conflict remains unresolved and that it may produce and send a new solution for peer's approval.

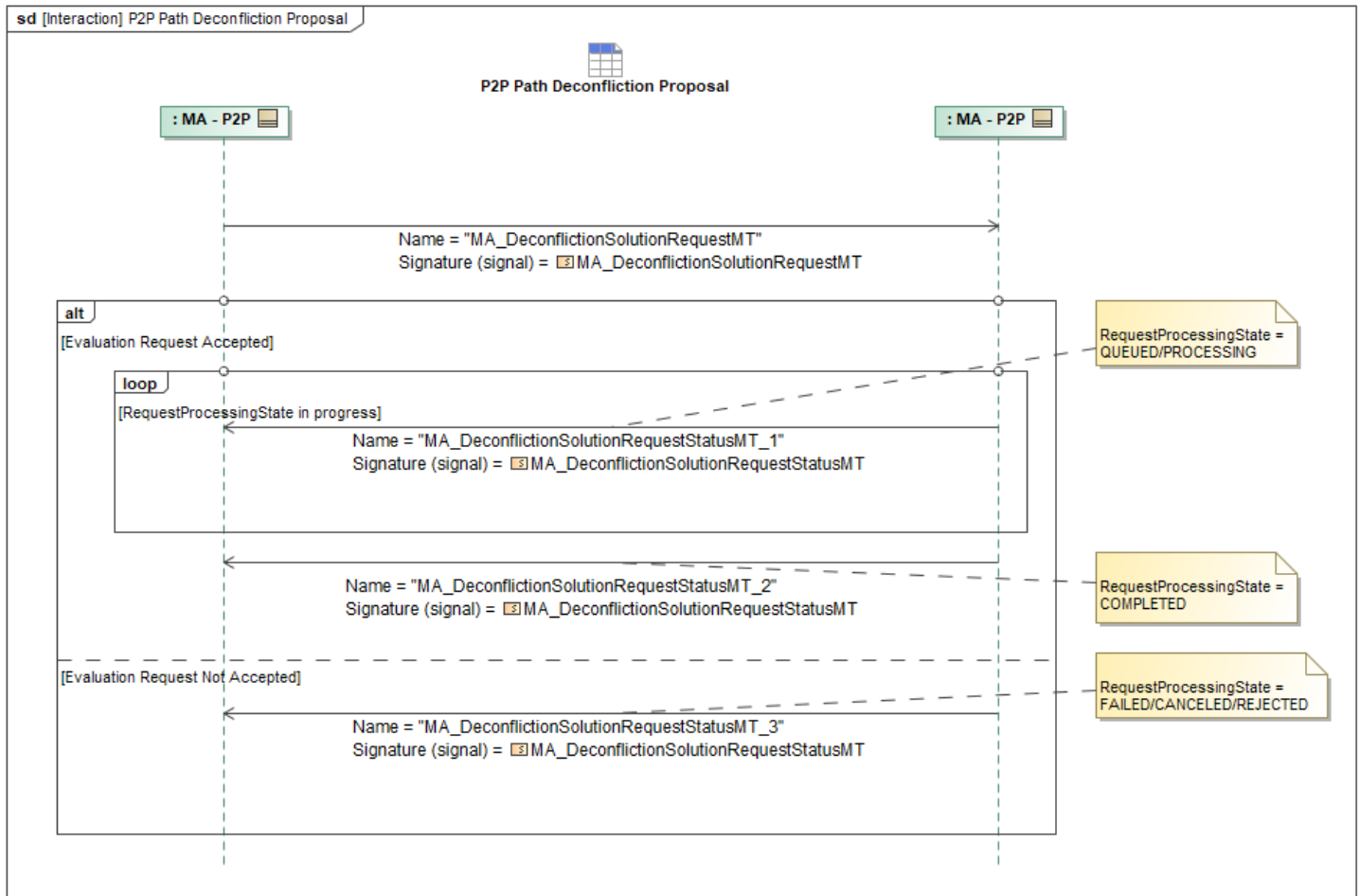


Figure 9-14: P2P Path Deconfliction Proposal

Table 9-13: P2P Path Deconfliction Proposal

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DeconflictionSolutionRequestMT: MA_DeconflictionSolutionRequestMT	None
MA_DeconflictionSolutionRequestStatusMT_1: MA_DeconflictionSolutionRequestStatusMT	None
MA_DeconflictionSolutionRequestStatusMT_2: MA_DeconflictionSolutionRequestStatusMT	None
MA_DeconflictionSolutionRequestStatusMT_3: MA_DeconflictionSolutionRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.7 Peer Activate a Contingency

Peer sends a *MA_ResponseCommandMT* to activate or deactivate a previously uploaded *MA_ResponseMT*. The *MA_ResponseCommandMT* will set the *ConstraintOverride.Allocation.RequiredAllocation.Allocation* field to indicate the target *SystemIDs* or

PackageID to activate the *MA_ResponseMT*. Optionally, the *AssociatedPlan* field can be used to link back to a plan which contains the *MA_ResponseMT*. MA responds with a *MA_ResponseCommandStatusMT* to indicate if the command was successful or not.

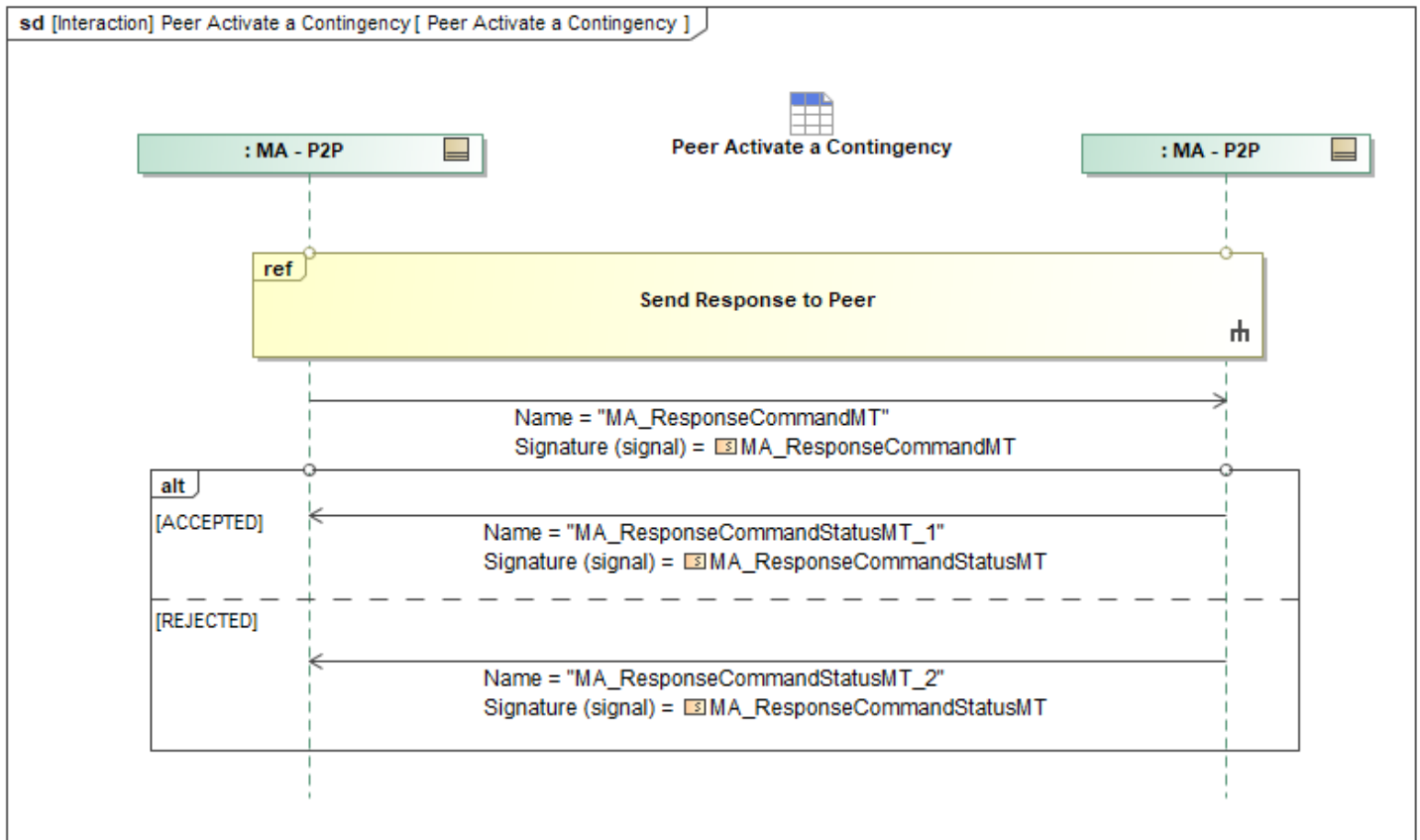


Figure 9-15: Peer Activate a Contingency

Table 9-14: Peer Activate a Contingency

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseCommandMT: MA_ResponseCommandMT	None
MA_ResponseCommandStatusMT_1: MA_ResponseCommandStatusMT	None
MA_ResponseCommandStatusMT_2: MA_ResponseCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.8 Peer Receive Contingency Status

MA will periodically publish *MA_ResponseActivityMT* to convey monitoring and triggered status for all active contingencies. An *ActivityState* of *ENABLED* indicates MA is actively monitoring the trigger condition for the *Response* while *DISABLED* means monitoring has stopped. When the trigger condition has been satisfied, the *ActivityState* will change to *COMPLETED* with the



Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseActivityMT: MA_ResponseActivityMT	None

9.2.4.9 Peer Total Comms Lost with Team

MA_PositionReportMT 1-4 denote unique message instances between each peer attempting to report their individual positions. *MA_PositionReportMT* 5-8 are unique message instances that show the message flows between each peer with comms established.

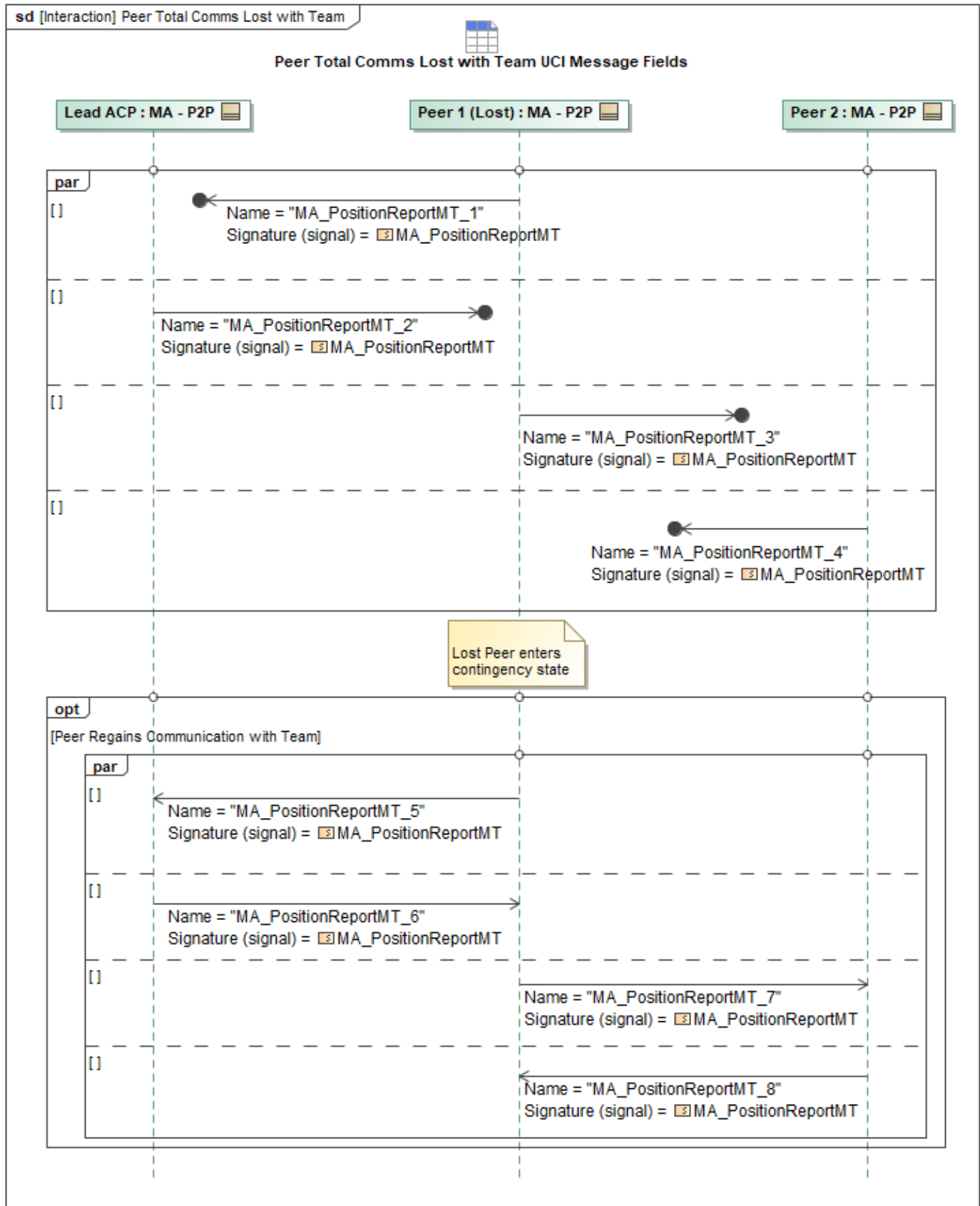


Figure 9-17: Peer Total Comms Lost with Team

Table 9-16: Peer Total Comms Lost with Team

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PositionReportMT_1: MA_PositionReportMT	DomainVelocity: Velocity3D_Type CalibratedAirspeed: SpeedType GroundVelocity: Velocity2D_Type Mach: MachType
MA_PositionReportMT_2: MA_PositionReportMT	DomainVelocity: Velocity3D_Type Mach: MachType CalibratedAirspeed: SpeedType GroundVelocity: Velocity2D_Type
MA_PositionReportMT_3: MA_PositionReportMT	GroundVelocity: Velocity2D_Type CalibratedAirspeed: SpeedType Mach: MachType DomainVelocity: Velocity3D_Type
MA_PositionReportMT_4: MA_PositionReportMT	Mach: MachType GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type CalibratedAirspeed: SpeedType
MA_PositionReportMT_5: MA_PositionReportMT	Mach: MachType GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type CalibratedAirspeed: SpeedType
MA_PositionReportMT_6: MA_PositionReportMT	Mach: MachType DomainVelocity: Velocity3D_Type GroundVelocity: Velocity2D_Type CalibratedAirspeed: SpeedType
MA_PositionReportMT_7: MA_PositionReportMT	DomainVelocity: Velocity3D_Type GroundVelocity: Velocity2D_Type Mach: MachType CalibratedAirspeed: SpeedType
MA_PositionReportMT_8: MA_PositionReportMT	CalibratedAirspeed: SpeedType DomainVelocity: Velocity3D_Type GroundVelocity: Velocity2D_Type Mach: MachType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.10 Team Leader Lost Comms with C2

In the event that the Team Leader loses comms with C2, the leader shall alert its team members via the *MissionContingencyAlertMT*. The *MissionContingencyAlertMT* contains a unique *MissionContingencyAlertID* field set to the Alert ID generated for the triggered contingency and the *ContingencyCondition.ConflictState* will be set to "CONFLICTED". Once the contingency response behavior has been activated (see *Define Team Lost Comms Contingency*), response activity monitoring shall begin between each member of the team, including the leader, whereby each team member will send *MA_ResponseActivityMTs* to each other with *ActivityState* set to "ACTIVE_*". If Communication has been regained with C2, the leader shall again alert its team members via the *MissionContingencyAlertMT*. The *MissionContingencyAlertMT* will have the same fields set as when the contingency was triggered, with the exception of the *MissionContingencyAlertID* (which will be unique to this message) and the *ContingencyCondition.ConflictState* will be updated to "CLEARED".

MA_ResponseActivityMT_1-6 denote unique message instances on specific message paths between each team member. Each message instance is the originating team member reporting its own activity to one individual peer.

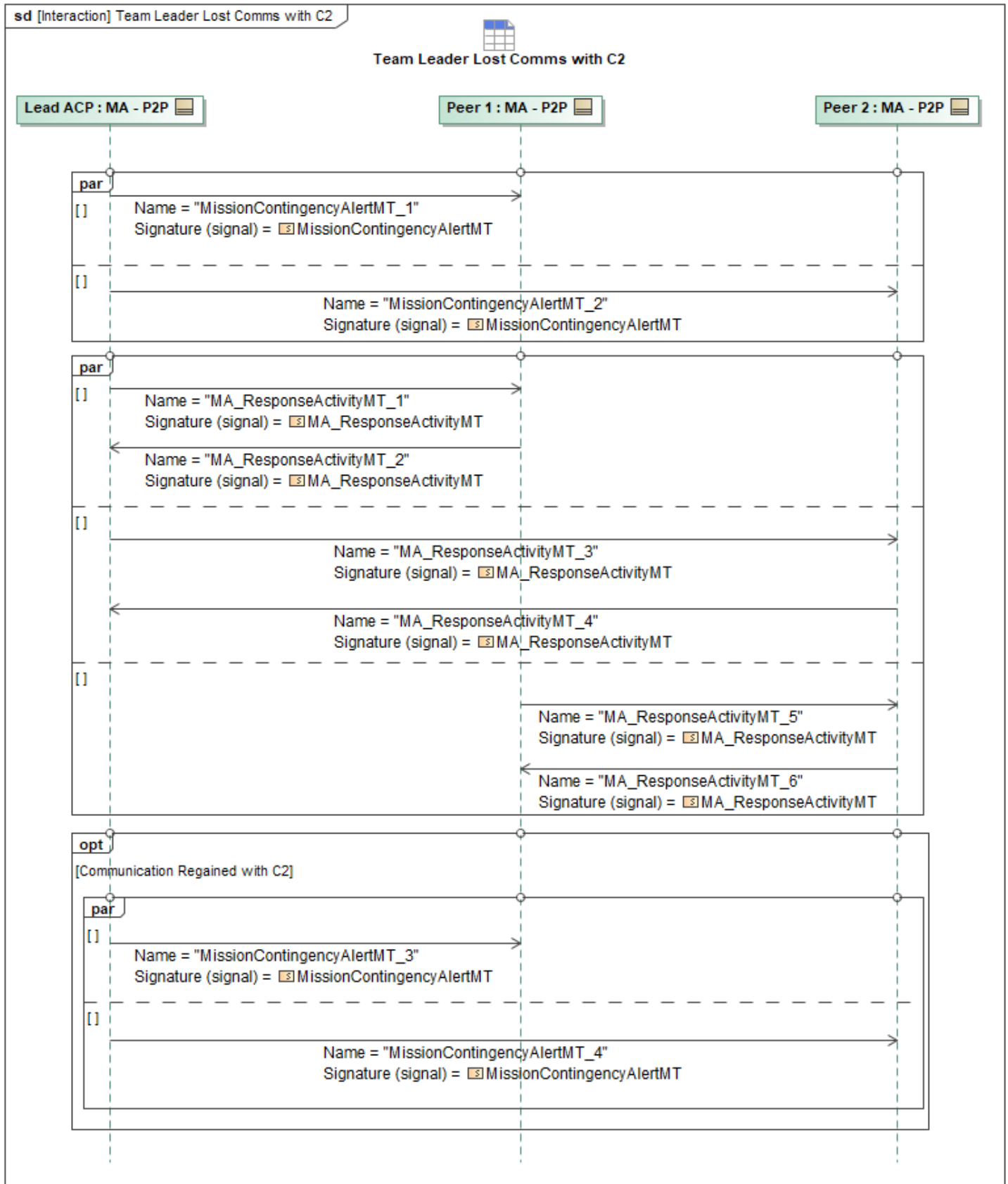


Figure 9-18: Team Leader Lost Comms with C2

Table 9-17: Team Leader Lost Comms with C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseActivityMT_1: MA_ResponseActivityMT	ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MA_ResponseActivityMT_2: MA_ResponseActivityMT	ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MA_ResponseActivityMT_3: MA_ResponseActivityMT	ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MA_ResponseActivityMT_4: MA_ResponseActivityMT	ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MA_ResponseActivityMT_5: MA_ResponseActivityMT	ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MA_ResponseActivityMT_6: MA_ResponseActivityMT	ActivatedPlans: MA_PlansReferenceType AlertID: MissionContingencyAlertID_Type AlertID: MissionContingencyAlertID_Type ActivatedPlans: MA_PlansReferenceType
MissionContingencyAlertMT_1: MissionContingencyAlertMT	AlertOnly: EmptyType
MissionContingencyAlertMT_2: MissionContingencyAlertMT	AlertOnly: EmptyType
MissionContingencyAlertMT_3: MissionContingencyAlertMT	AlertOnly: EmptyType
MissionContingencyAlertMT_4: MissionContingencyAlertMT	AlertOnly: EmptyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.4.11 Triggered Contingency Alert to Peer

When a trigger for a contingency is tripped, MA will send a *MissionContingencyAlertMT_1* to the relevant Peer systems. The *MissionContingencyAlertMT_1* will have the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detects the contingency and is generating the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that has the contingency condition, the *ContingencyCondition.ConflictState* field set to "CONFLICTED" to indicate that the conflict that caused the trigger has not been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique ID for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", and the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions. If the *MissionContingencyAlertMT_1* is not to result in any action being taken, set the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field to "TRUE" to indicate that there will be no action taken.

If the contingency has been cleared, the MA will send the relevant Peer systems a *MissionContingencyAlertMT_2* with the *MissionContingencyAlertID* field set to the ID of the alert, the *SourceSystemID* field set to the ID of the system that detected the contingency and generated the alert, the *ContingencyCondition.ConflictedSystemID* field set to the ID of the system that had the contingency condition, the *ContingencyCondition.ConflictState* field set to "CLEARED" to indicate that the conflict that caused the trigger has been resolved, the *ContingencyCondition.Conflict.ConflictID* set to the unique id for this conflict, the *ContingencyCondition.Trigger.Source* field set to "AUTONOMOUS_TRIGGER", and the *ContingencyCondition.Trigger.ByCaseTrigger* field set to the initial by case trigger for autonomous mission planning functions. If no action was taken as a result of the initial *MissionContingencyAlertMT_1*, set the *ContingencyCondition.AutonomousActionStatus.AlertOnly* field to "TRUE" to indicate that no was action taken.

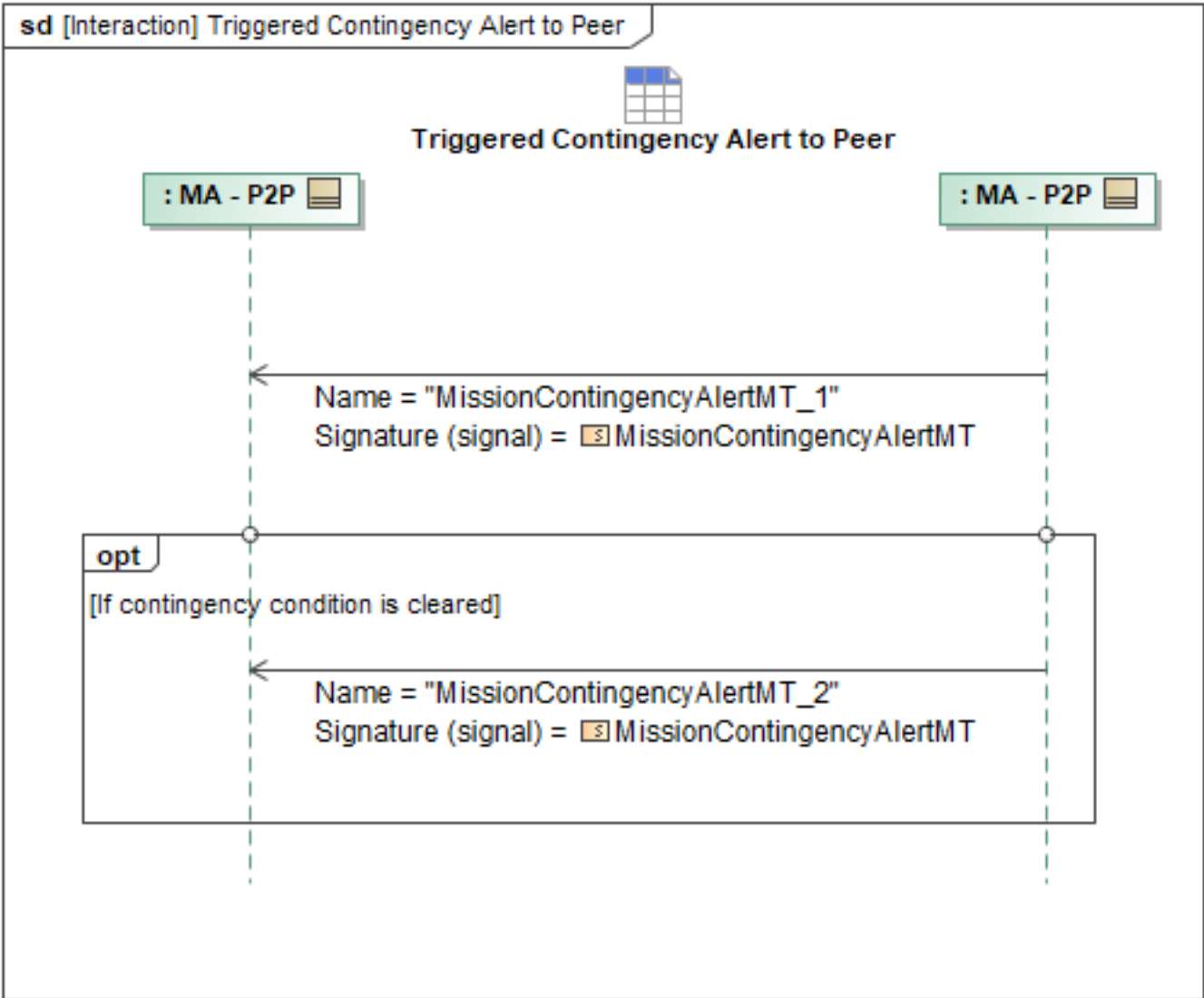


Figure 9-19: Triggered Contingency Alert to Peer

Table 9-18: Triggered Contingency Alert to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MissionContingencyAlertMT_1: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType
MissionContingencyAlertMT_2: MissionContingencyAlertMT	ByCaseTrigger: PlanningByCaseTriggerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.5 Peer COP and Coordination Package

9.2.5.1 Configure COP and Request COP Configuration Settings Status (P2P)

This interaction describes the process of the Lead ACP configuring the COP using the P2P interface.

If the Lead ACP configures or updates the COP configuration settings, the *MA_COP_ConfigurationSettingsCommandMT* will be sent to Mission Autonomy (MA) at the Recipient ACPs, with the *COP_Participants*, *COP_LeadPrioritized*, and *RequestAuthority* field set to *REQUEST*, to update their COP configuration settings accordingly. If both C2 and the Lead ACP send *MA_COP_ConfigurationSettingsCommandMT* to Recipient ACPs, the C2 command will be prioritized over the Lead ACP command. To update a Configuration item, set *ConfigurationID* to the UUID of the item to be updated. To add a Configuration item, the *ConfigurationID* is optional. To delete a Configuration item, set the *ConfigurationID* to the UUID of the item to be removed and omit the remaining fields in the Configuration. Once received, MA on the recipient ACP can optionally respond with *MA_COP_ConfigurationSettingsCommandStatusMT_2* with *CommandProcessingState* set to *RECEIVED*. If the command is completed, the ACP will send *MA_COP_ConfigurationSettingsCommandStatusMT_3* with *CommandProcessingState* set to *ACCEPTED*. If unable to execute the command, the ACP will send *MA_COP_ConfigurationSettingsCommandStatusMT_4* with *CommandProcessingState* set to *REJECTED*. MA will determine whether the COP Leader is authorized to command the initial *MA_COP_ConfigurationSettingsCommandMT*. If unauthorized, MA will ignore the *MA_COP_ConfigurationSettingsCommandMT* command and reply with *MA_COP_ConfigurationSettingsCommandStatusMT_1* where *CommandProcessingStateEnum* is set to *REJECTED* and *CommandProcessingStateReason* shall define the *CannotComplyEnum* as *INELIGIBLE_CONTROL_SOURCE*.

In addition to this, the Lead ACP may send *MA_COP_ConfigurationSettingsRequestMT*, requesting a status update on the COP Configuration Settings from all ACPs. While processing the request, the ACP may optionally send a *MA_COP_ConfigurationSettingsRequestStatusMT_2* with *RequestProcessingState* set to *PROCESSING*. Once the request is completed, the ACP will send *MA_COP_ConfigurationSettingsRequestStatusMT_3* with the *RequestProcessingState* set to *COMPLETED* and the current settings in the *ConfigurationSettings*. If unable to complete the command, the ACP will send *MA_COP_ConfigurationSettingsRequestStatusMT_4* with the *RequestProcessingState* set to *FAILED*. MA will determine whether the COP Leader is authorized to command the initial *MA_COP_ConfigurationSettingsRequestMT*. If the COP leader is unauthorized, MA will ignore the *MA_COP_ConfigurationSettingsRequestMT* command and reply with *MA_COP_ConfigurationSettingsRequestStatusMT_1* where *RequestProcessingStateEnum* is set to *REJECTED* and *RequestProcessingStateReason* shall define the *CannotComplyEnum* as *INELIGIBLE_CONTROL_SOURCE*.



Figure 9-20: Configure COP and Request COP Configuration Settings Status (P2P)

Table 9-19: Configure COP and Request COP Configuration Settings Status (P2P)

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsCommandMT: MA_COP_ConfigurationSettingsCommandMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_1: MA_COP_ConfigurationSettingsCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_COP_ConfigurationSettingsCommandStatusMT_2: MA_COP_ConfigurationSettingsCommandStatusMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_3: MA_COP_ConfigurationSettingsCommandStatusMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_4: MA_COP_ConfigurationSettingsCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_COP_ConfigurationSettingsRequestMT: MA_COP_ConfigurationSettingsRequestMT	None
MA_COP_ConfigurationSettingsRequestStatusMT_1: MA_COP_ConfigurationSettingsRequestStatusMT	RequestProcessingStateReason: CannotComplyType
MA_COP_ConfigurationSettingsRequestStatusMT_2: MA_COP_ConfigurationSettingsRequestStatusMT	None
MA_COP_ConfigurationSettingsRequestStatusMT_3: MA_COP_ConfigurationSettingsRequestStatusMT	None
MA_COP_ConfigurationSettingsRequestStatusMT_4: MA_COP_ConfigurationSettingsRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.5.2 Coordinate Simultaneous Arrival Time

To start coordinating arrival times between team members, the Lead ACP sends *MA_OpPointMT_1* with an OpPoint of reference and the *Category* set to *RENDEZVOUS*. Upon receipt of the initial *MA_OpPointMT_1*, all Follower ACPs compute their earliest and latest arrival times that can be achieved to the referenced OpPoint. All Follower ACPs will then send *MA_OpPointMT_2* to the Lead ACP with *Category* set to *RENDEZVOUS* and the *AssociatedTime* fields for both the earliest and

latest arrival times populated. The *AssociatedTime* fields will have the *Function* set to *ARRIVAL* and *Value* field set to the earliest/latest arrival time respectively. The Lead ACP will then compute the coordinated arrival time that all ACPs can support and sends out *MA_OpPointMT_3* to all Follower ACPs with the final coordinated arrival time in the *AssociatedTime* field and *Category* set to *RENDEZVOUS*. All ACPs will execute simultaneous arrival based on the coordinated time of arrival that the Lead ACP has computed.

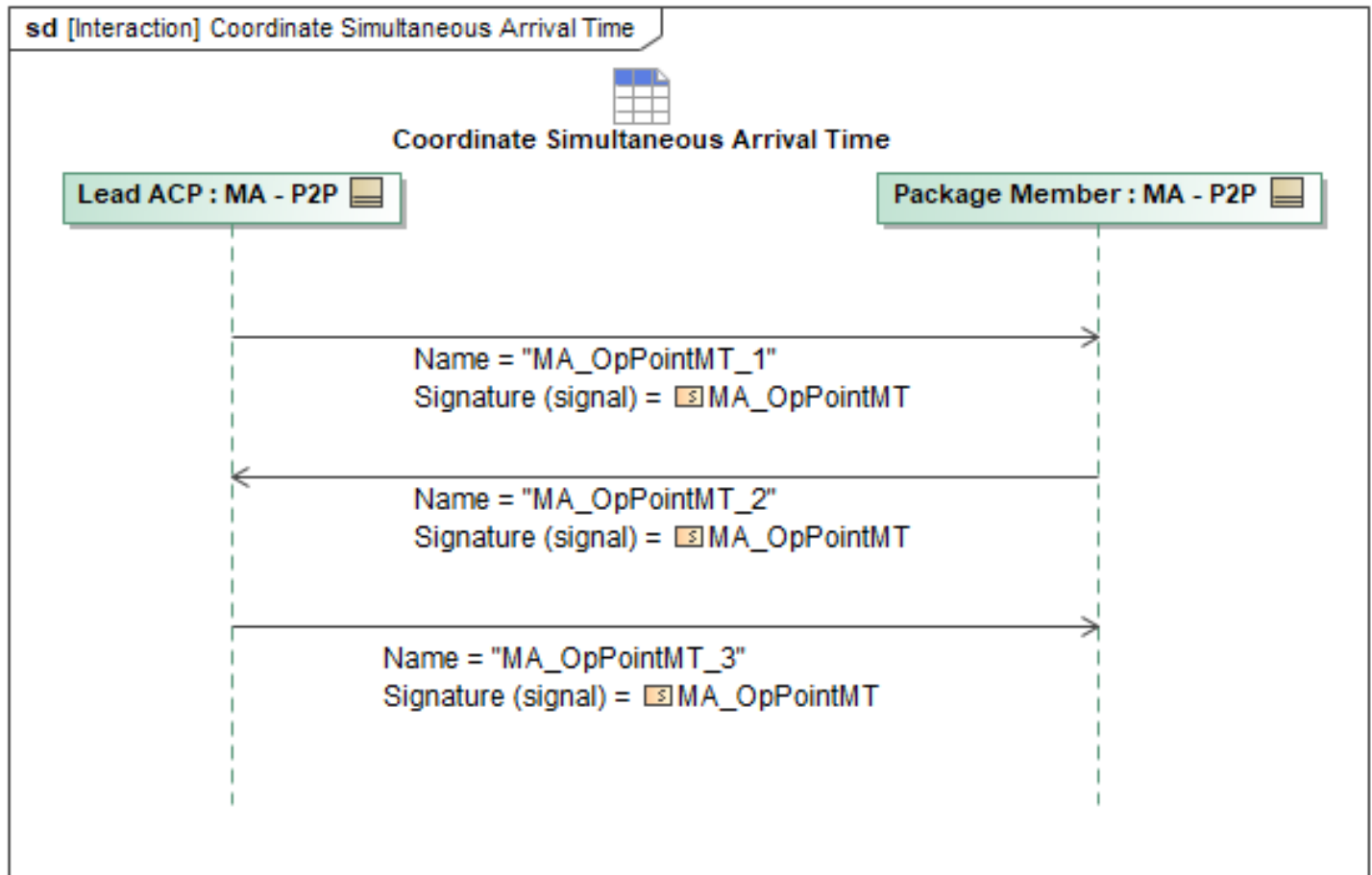


Figure 9-21: Coordinate Simultaneous Arrival Time

Table 9-20: Coordinate Simultaneous Arrival Time

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPointMT_1: MA_OpPointMT	None
MA_OpPointMT_2: MA_OpPointMT	AssociatedTime: TimeFunctionType
MA_OpPointMT_3: MA_OpPointMT	AssociatedTime: TimeFunctionType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.5.3 COP Follower Configuration Update

The COP Follower Configuration Update interaction supports the use case in which an ACP determines the need to update its own COP Configuration settings. This could be triggered by a

change in capabilities or environment, but is not constrained to a specific trigger. After the trigger occurs, the COP Follower makes an update locally to the COP Configuration Settings. The COP Follower then must notify the COP Leader of their configuration settings update. The ACP sends *MA_COP_ConfigurationSettingsMT* to the COP Leader with the *Configuration* updates populated. Once received, the COP Leader reviews the updates and has the option to respond with additional changes or adjustments to the ACP COP Configuration Settings. If the COP Leader wants to make changes to the COP Follower Configuration Settings, they send *MA_COP_ConfigurationSettingsCommandMT* to the COP Follower with the new *Configuration* settings. To update a Configuration item, set *ConfigurationID* to the UUID of the item to be updated. Once received, MA on the COP Follower can optionally respond with *MA_COP_ConfigurationSettingsCommandStatusMT_2* with *CommandProcessingState* set to *RECEIVED*. If the command is completed, the COP Follower will send *MA_COP_ConfigurationSettingsCommandStatusMT_3* with *CommandProcessingState* set to *ACCEPTED*. If unable to execute the command, the ACP will send *MA_COP_ConfigurationSettingsCommandStatusMT_4* with *CommandProcessingState* set to *REJECTED*. MA will determine whether the COP Leader is authorized to command the initial *MA_COP_ConfigurationSettingsCommandMT*. If unauthorized, MA will ignore the *MA_COP_ConfigurationSettingsCommandMT* command and reply with *MA_COP_ConfigurationSettingsCommandStatusMT_1* where *CommandProcessingStateEnum* is set to *REJECTED* and *CommandProcessingStateReason* shall define the *CannotComplyEnum* as *INELIGIBLE_CONTROL_SOURCE*.

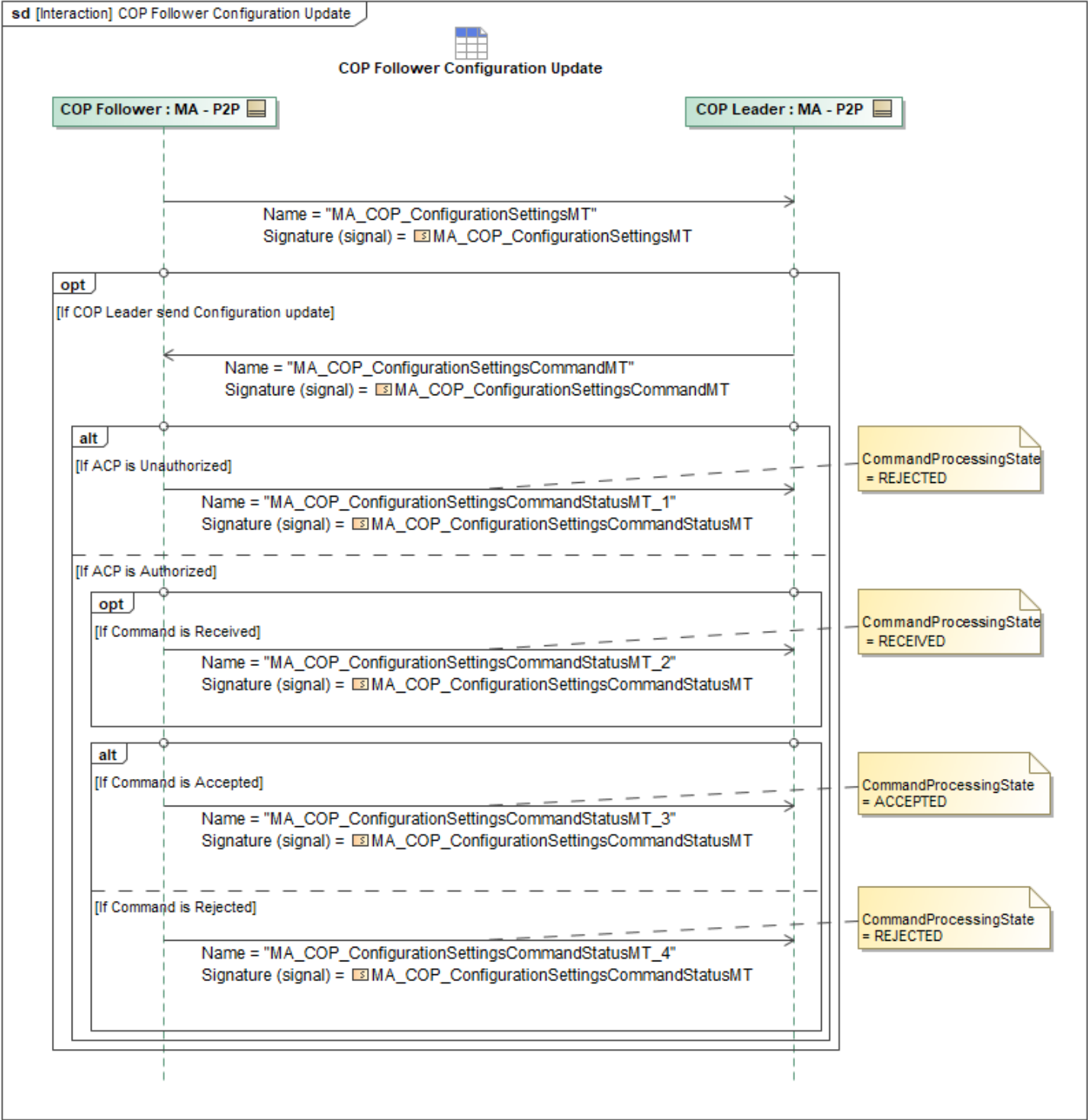


Figure 9-22: COP Follower Configuration Update

Table 9-21: COP Follower Configuration Update

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsCommandMT: MA_COP_ConfigurationSettingsCommandMT	ConfigurationID: MA_COP_ConfigurationID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_COP_ConfigurationSettingsCommandStatusMT_1: MA_COP_ConfigurationSettingsCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_COP_ConfigurationSettingsCommandStatusMT_2: MA_COP_ConfigurationSettingsCommandStatusMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_3: MA_COP_ConfigurationSettingsCommandStatusMT	None
MA_COP_ConfigurationSettingsCommandStatusMT_4: MA_COP_ConfigurationSettingsCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_COP_ConfigurationSettingsMT: MA_COP_ConfigurationSettingsMT	ConfigurationID: MA_COP_ConfigurationID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.5.4 Distribute Sensor Track Data to Facilitate Package Fusion and Update COP

For context, it is important to clarify what is meant by "un-fused" and "fused" tracks. Un-fused tracks, that is, tracks that have not yet been fused together, are the result of an onboard sensor tracker associating a series of observation measurement reports (OMRs) and linking them to an entity. This can be performed across all sensors that a given platform is equipped with. Once these sensor trackers have produced a set of entities, they may then be run through the sensor fusion engine, which will ingest all available entities and associate, fuse, and/or deconflict them. The output is what is called a fused entity (or track)

The interaction shown here describes the sharing of track data (either fused or un-fused) between ACPs in a package by sending *EntityMT* across P2P interface to facilitate package level fusion. If the network has sufficient bandwidth, *ObservationMeasurementReportMT* data may also be sent to better inform the *EntityMT* data. A kinematic and CID fusion implementation will likely want the capability that OMRs were generated from, and from which system/subsystem had the capability generating the OMR

Within the *ObservationMeasurementReportMT* message, there are two optional paths to different sets of *SubsystemIDs* and *SystemIDs*

1. *ObservationMeasurementReportMDT* → *ObservationMeasurement* → *MeasurementSource* → *ElementDetails* → *SourceElementIdentifier* → *SystemID* + *SubsystemID*

These messages provide the system/subsystem that's the source of the OMR's kinematics

2. *ObservationMeasurementReportMDT* → *ObservationMeasurement* → *MeasurementSource* → *SourceIdentity* → *CapabilityID* + *SystemID* + *SubsystemID*

These messages provide the system/subsystem and capability for the source of identity in the OMR (it's expected this will usually be the same system/subsystem as the kinematics) A client would want to know the capability type of the provided entity ID, and may have to subscribe to additional messages to understand the type (e.g. [*Capability*]*StatusMT* messages)

The *EntityMT* message contains the *MeasurementID* of all of the OMRs that have contributed to the formation of the entity. Whether the entity or track being shared between ACPs is un-fused or fused can be determined by the *SourceTypeIdentifier* field, and is represented as *EntityMT_1* and *EntityMT_2*, respectively. Within the *SourceTypeIdentifier* is a *Fusion* field which contains all of the fusion contributors, that is, the un-fused entities that have contributed to a fused track, meaning that an un-fused entity will not contain any fusion contributors in the *SourceTypeIdentifier* data. Once un-fused entities (and optionally OMRs) have been received, the fusion engine may produce fused entities that can then be ingested and utilized by the Common Operating Picture (COP).

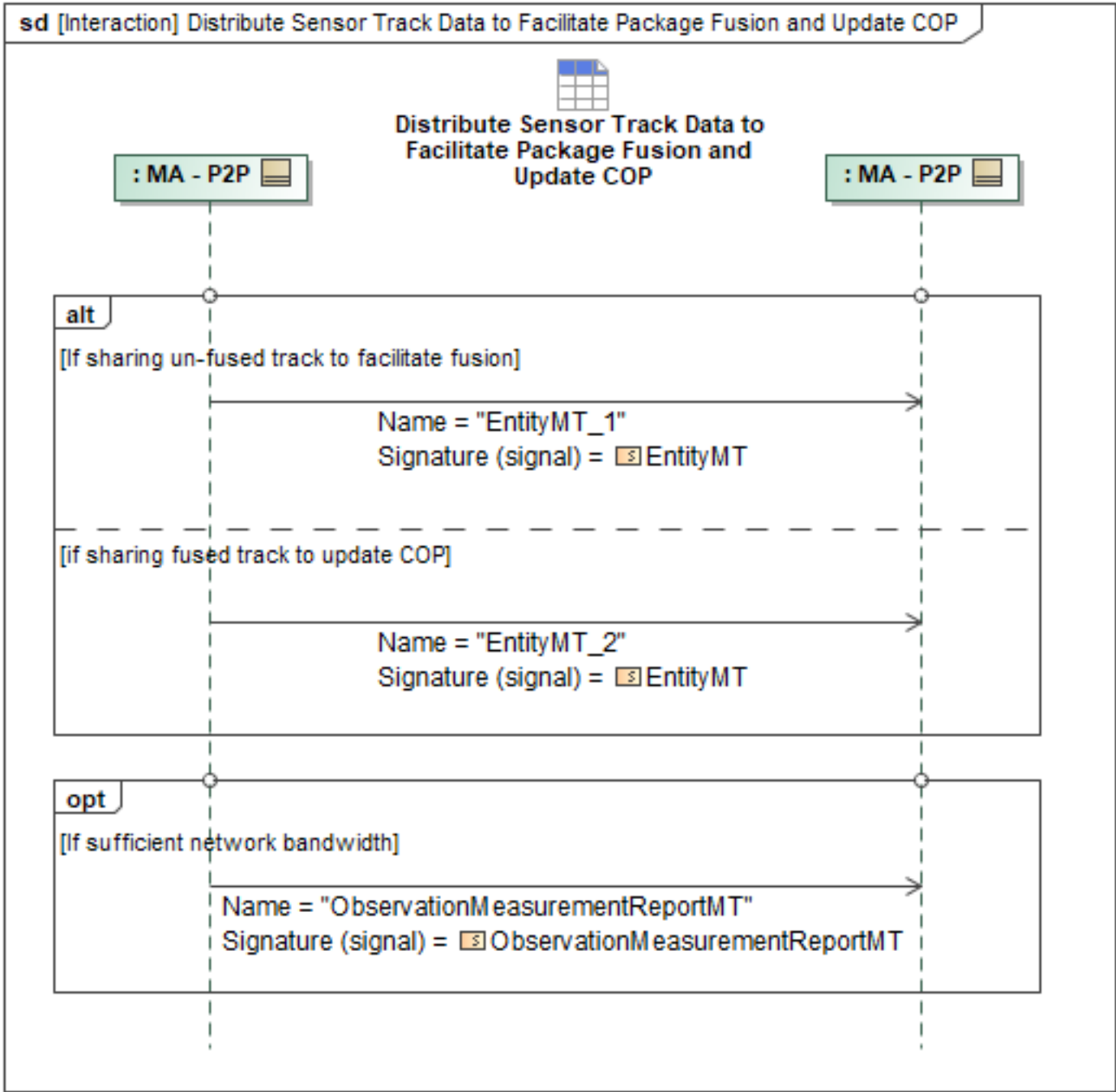


Figure 9-23: Distribute Sensor Track Data to Facilitate Package Fusion and Update COP

Table 9-22: Distribute Sensor Track Data to Facilitate Package Fusion and Update COP

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT_1: EntityMT	MeasurementID: MeasurementID_Type
EntityMT_2: EntityMT	MeasurementID: MeasurementID_Type SourceTypeIdentifier: EntitySourceIdentifierType
ObservationMeasurementReportMT: ObservationMeasurementReportMT	MeasurementSource: ObservationMeasurementSourceType

Message Name (Name:Type)	Required Fields (Name:Type)
	ElementDetails: ElementDetailsType SourceElementIdentifier: SourceKinematicsElementIdentityType SystemID: SystemID_Type SubsystemID: SubsystemID_Type SourceIdentity: MeasurementSourceIdentityType CapabilityID: CapabilityID_Type SystemID: SystemID_Type SubsystemID: SubsystemID_Type MeasurementKinematics: MeasurementKinematicsChoiceType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.5.5 Synchronize Global COP to Peer

The Synchronize Global COP to Peer interaction supports the use case in which the COP Leader has generated a new or updated Global COP and needs to distribute the data to the COP Followers. The output of the Global COP includes updated Entity data and updated Team States. This information needs to be synchronized to each peer in the executing package each time the Global COP is updated. The COP Leader sends *EntityMT* and *PackageStatusMT* to each of the COP Followers. *EntityMT* includes the *MeasurementID* and the *SourceTypeIdentifier* fields to provide context for the updated measurements and originating sources, respectively. *PackageStatusMT* communicates package status, as well as Team States of each member in the package using *PackagePartnerStatus*.

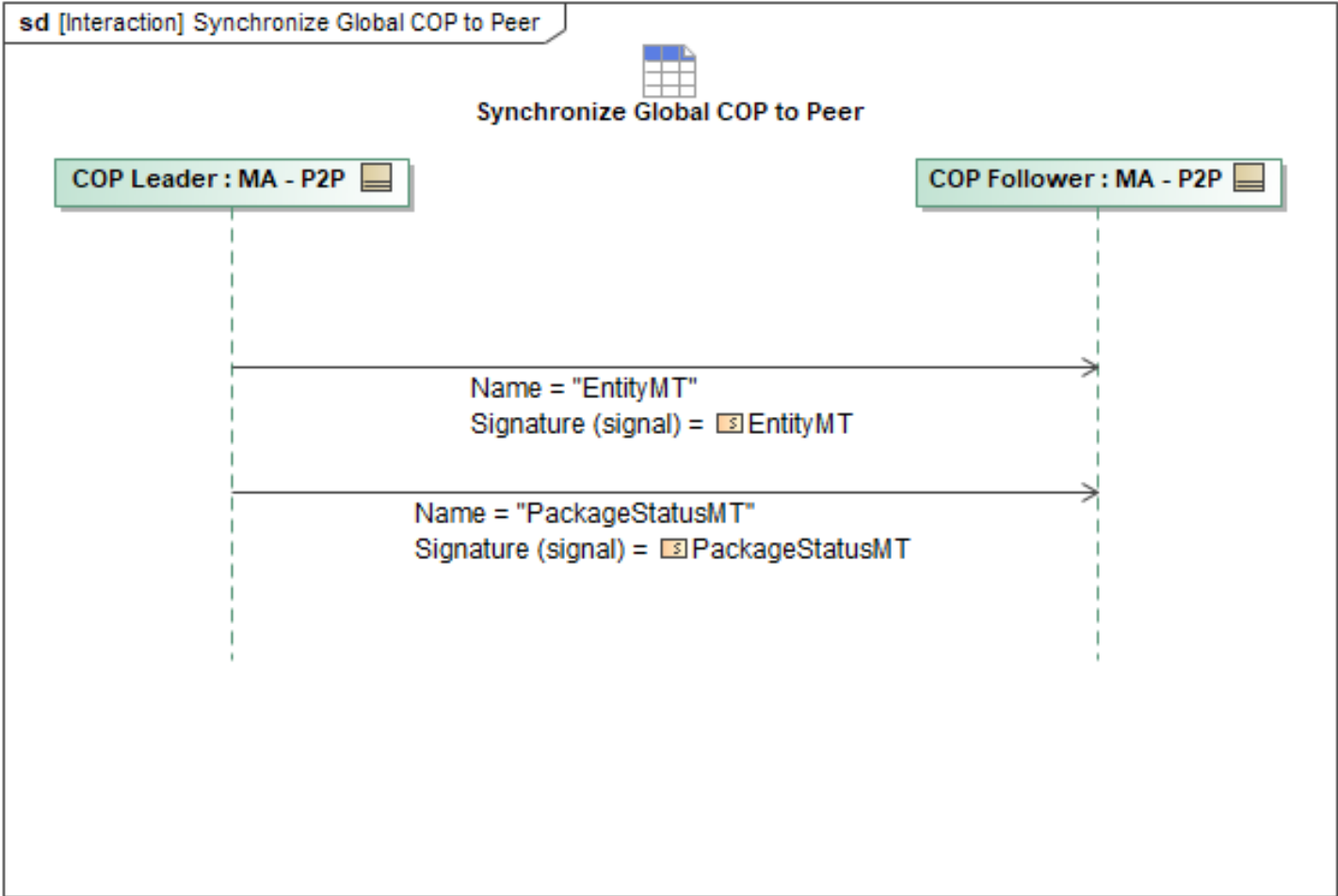


Figure 9-24: Synchronize Global COP to Peer

Table 9-23: Synchronize Global COP to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	MeasurementID: MeasurementID_Type SourceTypeIdentifier: EntitySourceIdentifierType
PackageStatusMT: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6 Peer COP Vehicle State Package

9.2.6.1 P2P Send Platform Override Status

The Peer-to-Peer (P2P) Send Platform Override Status sequence specifies the interaction of Mission Autonomy (MA) sharing the active override(s) with peers. The transmission of this message allows Autonomy Capable Platforms (ACPs) to understand the operating state of peers in the airspace. At predefined frequencies, the *MA_PlatformOverrideStatusMT* is sent. The *MA_PlatformOverrideStatusMT* must contain the active global override settings; the shared value will be a speed, altitude, formation, or Identification Friend or Foe (IFF) setting conveyed via the

PlatformOverrideStatus field with the value of each active override setting. In addition, the *ActiveOverrides* field must be set.

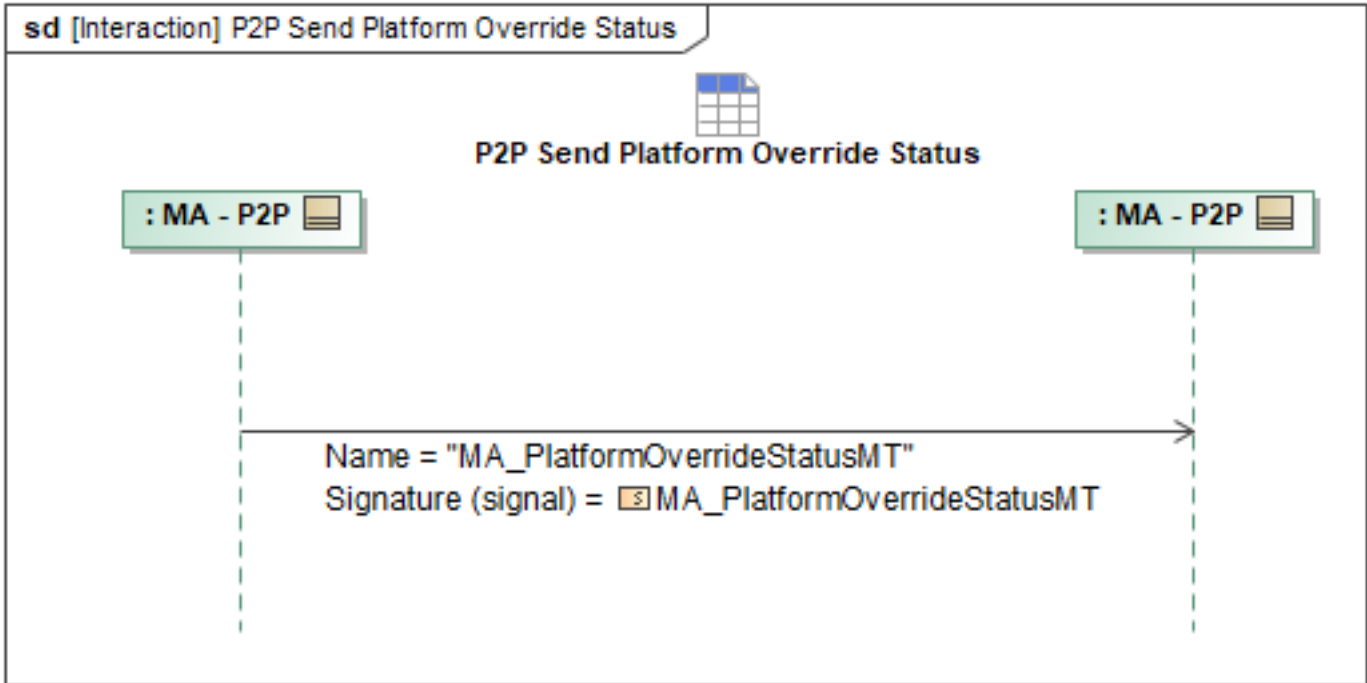


Figure 9-25: P2P Send Platform Override Status

Table 9-24: P2P Send Platform Override Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlatformOverrideStatusMT: MA_PlatformOverrideStatusMT	PlatformOverrideStatus: MA_PlatformDefaultAndOverrideDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.2 P2P State Data

Vehicle State Data is reported periodically between peers. Vehicle state data is included in the messages *MA_FlightActivityMT*, *ComponentStatusMT*, *MA_PositionReportMT*, and *MA_NavigationReportMT*. *ComponentStatusMT* and *MA_TaskStatusMT* can be included to provide further tasking details when required between peers in instances such as formation tasks. *MA_PositionReportMT* includes *InertialState*, defining *Position*, *DomainVelocity*, *GroundVelocity*, *CalibratedAirspeed*, and *Mach*. *DomainVelocity* represents the True Airspeed of the vehicle. The transmission of these messages among peers will give each peer a full picture of the package they are all part of. Vehicle State Data is reported periodically or optionally requested by MA. This may be a timed interval, like for telemetry, or during an action that requires the peers to have more information, like for waypoint following.

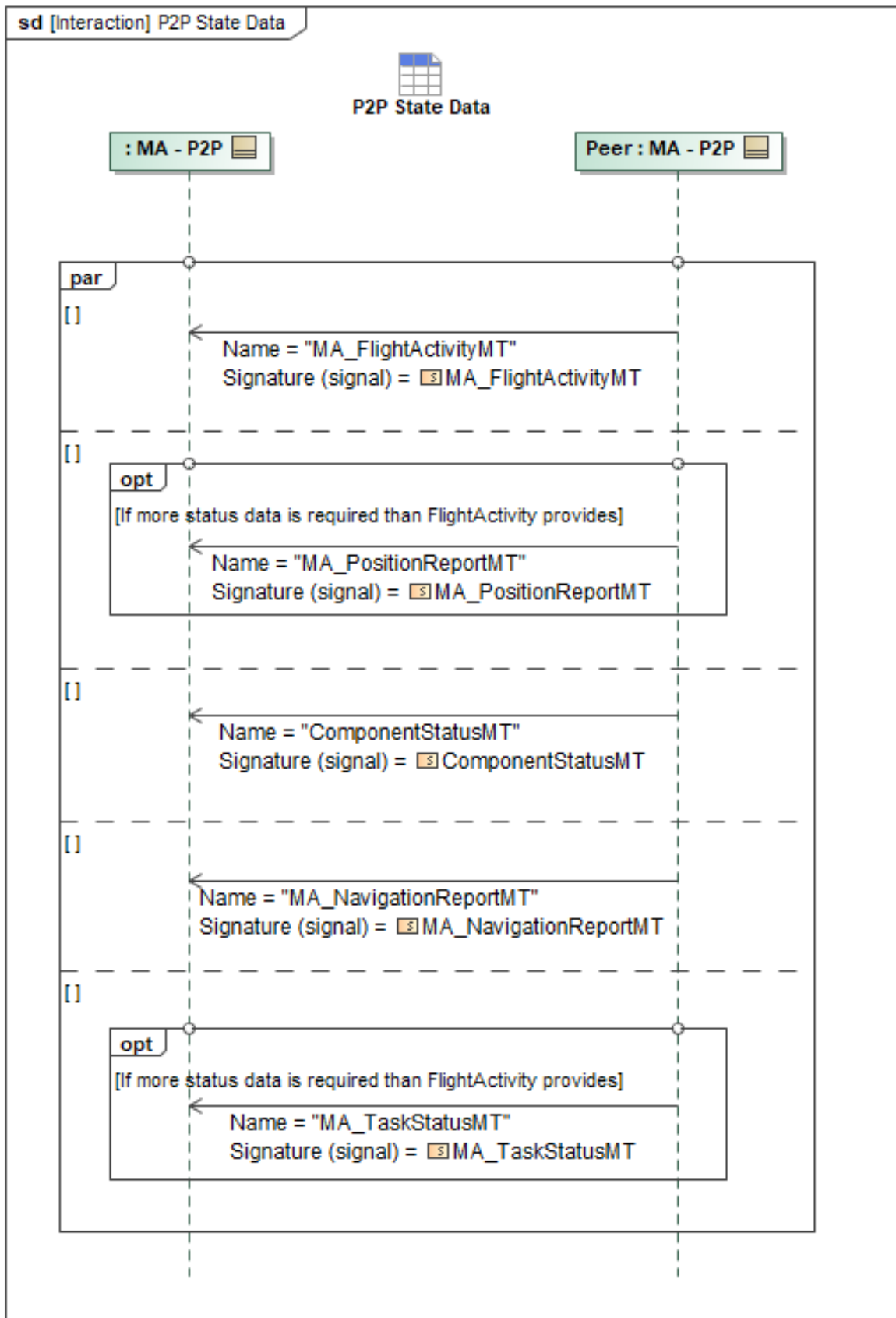


Figure 9-26: P2P State Data

Table 9-25: P2P State Data

Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_NavigationReportMT: MA_NavigationReportMT	None
MA_PositionReportMT: MA_PositionReportMT	CalibratedAirspeed: SpeedType Mach: MachType GroundVelocity: Velocity2D_Type DomainVelocity: Velocity3D_Type
MA_TaskStatusMT: MA_TaskStatusMT	Metrics: MetricsType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.3 Peer Heartbeat

Peers are responsible for issuing periodic heartbeat messages to each other via the *SystemStatusMT* message. For the purposes of vehicle identification, the field *Identity.SpecificVehicle.VehicleUniqueIdentifier.TailNumber* shall be populated. The *TailNumber* and other system identification information is sourced from pre-mission planning data. No response is required for heartbeat messages issued.

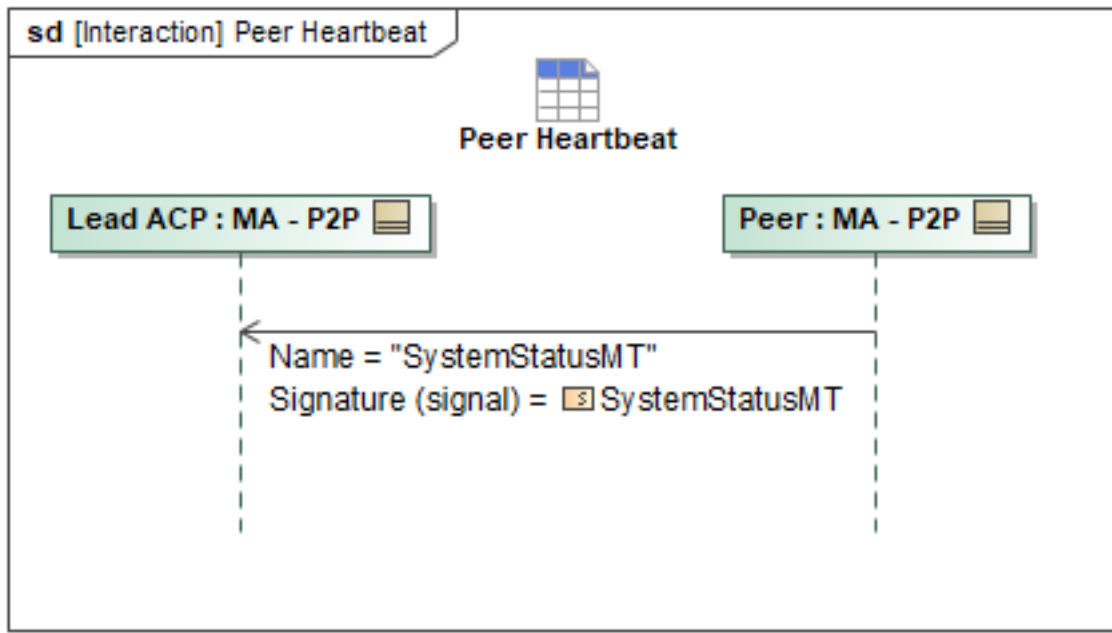


Figure 9-27: Peer Heartbeat

Table 9-26: Peer Heartbeat

Message Name (Name:Type)	Required Fields (Name:Type)
SystemStatusMT: SystemStatusMT	TailNumber: TailNumberType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.4 Peer Publish Control Status

Flight Autonomy (FA), through the Vehicle Interface (VI), will periodically publish the *ControlStatusMT* message to Mission Autonomy (MA), indicating the current controlling system and service. MA must then relay the *PRIMARY_CONTROLLER* and *SECONDARY_CONTROLLER* designations to Peers for monitoring. This is depicted through the Peer Publish Controls Status interaction, where the status is indicated by specifying through *ControlStatusMT/MessageData/ControlType/CapabilityControl/PrimaryController* and *ControlStatusMT/MessageData/ControlType/CapabilityControl/SecondaryController*.

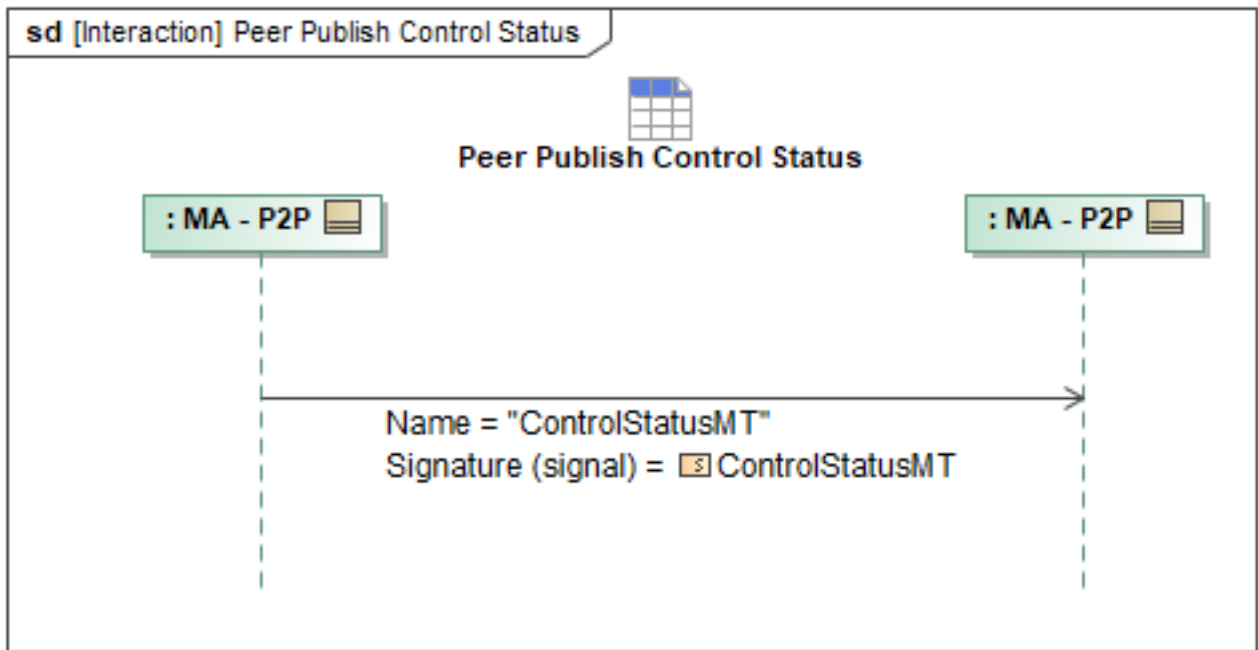


Figure 9-28: Peer Publish Control Status

Table 9-27: Peer Publish Control Status

Message Name (Name:Type)	Required Fields (Name:Type)
ControlStatusMT: ControlStatusMT	PrimaryController: PrimaryControllerType SecondaryController: SecondaryControllerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.5 Send Abort to Peer

When there is an abort for an ACP, MA may forward that information to the Lead MA or to any or all it's Peers. This interaction between MA and one of it's Peer (could be the Lead MA or just a normal Peer) forwards the *MissionPlanActivationStatusMT* and *RoutePlanExecutionStatusMT* from MA to the Peer which will be used to communicate:

- RoutePlanID* of the route plan that has been aborted
- MissionPlanActivationStatus:SubPlanActivationState:ActivationState* is *DEACTIVATED*
- PlanID* of the route plan that has been aborted and a *PlanExecutionState* of *Canceled*

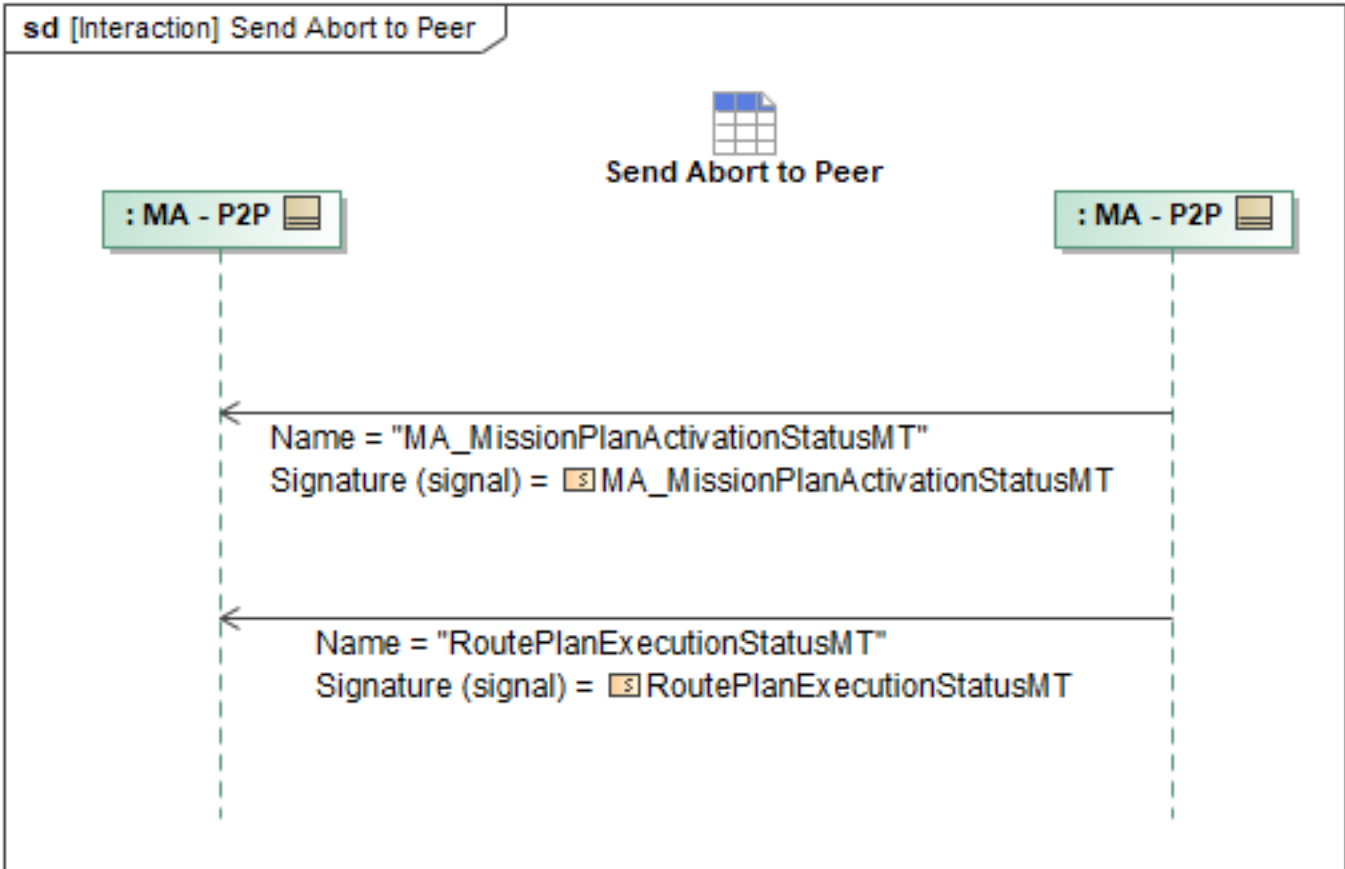


Figure 9-29: Send Abort to Peer

Table 9-28: Send Abort to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	DEACTIVATED: RoutePlanID: RoutePlanID_Type ActivationState: PlanActivationStateEnum
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	PlanExecutionState: PlanExecutionStateEnum PlanID: RoutePlanID_Type CANCELED:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.6 Send AMTI Capability State to Peer

Mission Use Case: AMTI

The “Send AMTI Capability State Data to Peer” sequence diagram specifies the interaction of the Mission Autonomy (MA) sharing Airborne Moving Target Indicator (AMTI) Capability State Data with peers. The transmission of these messages allows ACPs to understand the operational AMTI capability of peers in the airspace. In the context of a centralized Common Operating Picture (COP)

distribution setup, the AMTI capability state messages would be aggregated by the COP lead for distribution outside of the package.

If the platform has Airborne Moving Target Indicator (AMTI) sensing capabilities, the following messages are sent periodically: *MA_AMTI_CapabilityMT*, *AMTI_CapabilityStatusMT*, and *AMTI_ActivityMT*.

The *MA_AMTI_CapabilityMT* is published with at least one *Capability* specified.

The *AMTI_CapabilityStatusMT* and *AMTI_ActivityMT* report status and usage of the AMTI capability. COP uses *MA_AMTI_CapabilityMT* and *AMTI_CapabilityStatusMT* to know if the AMTI sensor capability is available to command.

COP uses *AMTI_ActivityMT* to know if the AMTI capability can effectively be used (e.g. not fully scheduled).

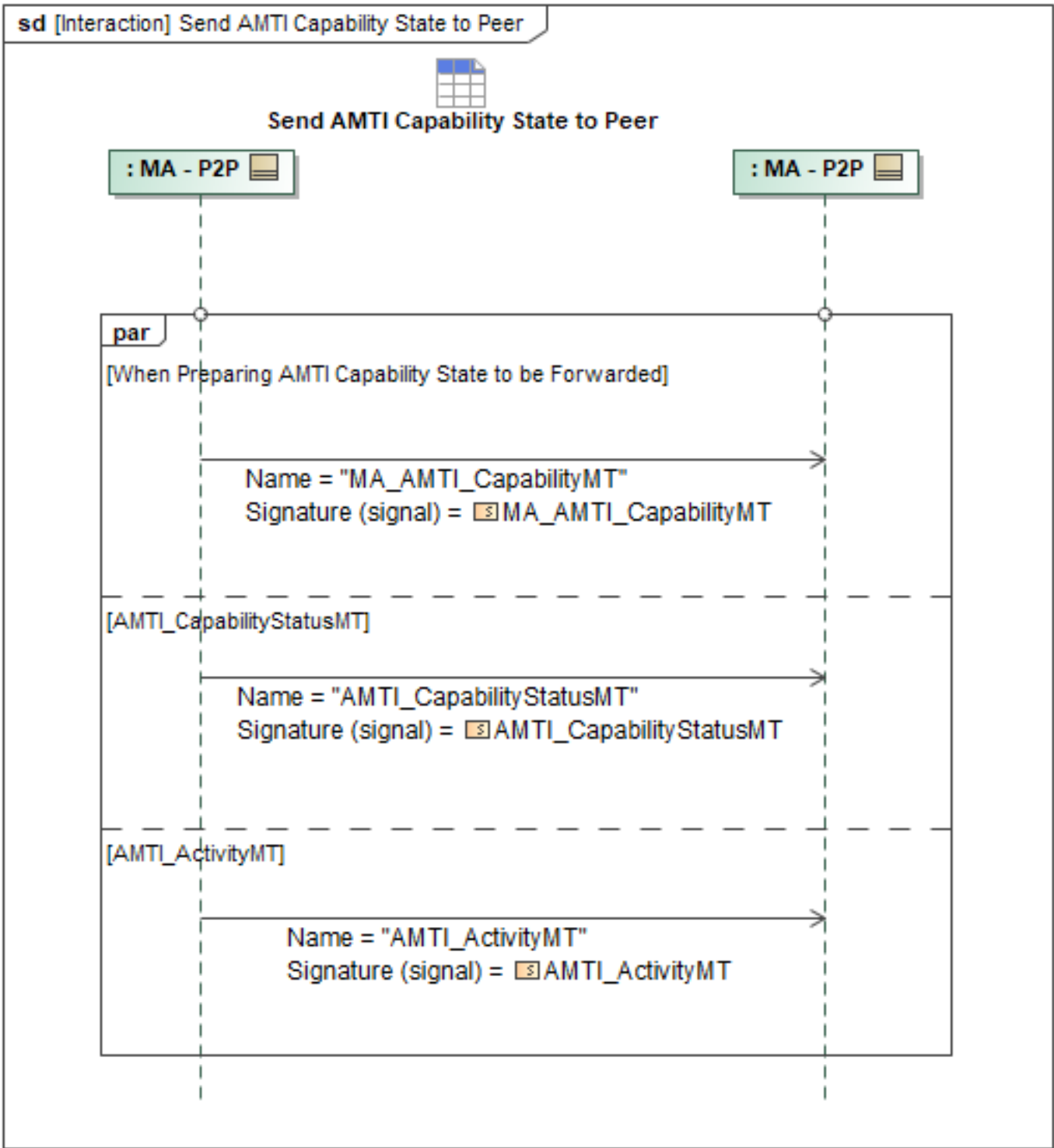


Figure 9-30: Send AMTI Capability State to Peer

Table 9-29: Send AMTI Capability State to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
AMTI_ActivityMT: AMTI_ActivityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
AMTI_CapabilityStatusMT: AMTI_CapabilityStatusMT	None
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.7 Send PO Capability State to Peer

Mission Use Case: PO

The “Send Passive Optical (PO) Capability State Data to Peer” sequence diagram specifies the interaction of the Mission Autonomy (MA) sharing PO Capability State Data with peers. The transmission of these messages allows ACPs to understand the operational PO capability of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the PO capability state messages would be aggregated by the COP lead for distribution outside of the package.

If the platform has Passive Optical (PO) sensor capability, the following messages are sent periodically: *PO_CapabilityMT*, *PO_CapabilityStatusMT*, *PO_ActivityMT*.

The *PO_CapabilityMT* is published with with at least one *Capability* specified.

The *PO_CapabilityStatusMT* and *PO_ActivityMT* report status and usage of the PO capability. COP uses *PO_CapabilityMT* and *PO_CapabilityStatusMT* to know if the PO sensor capability is available to command. COP uses *PO_ActivityMT* to know if the PO capability can effectively be used (e.g. not fully scheduled).

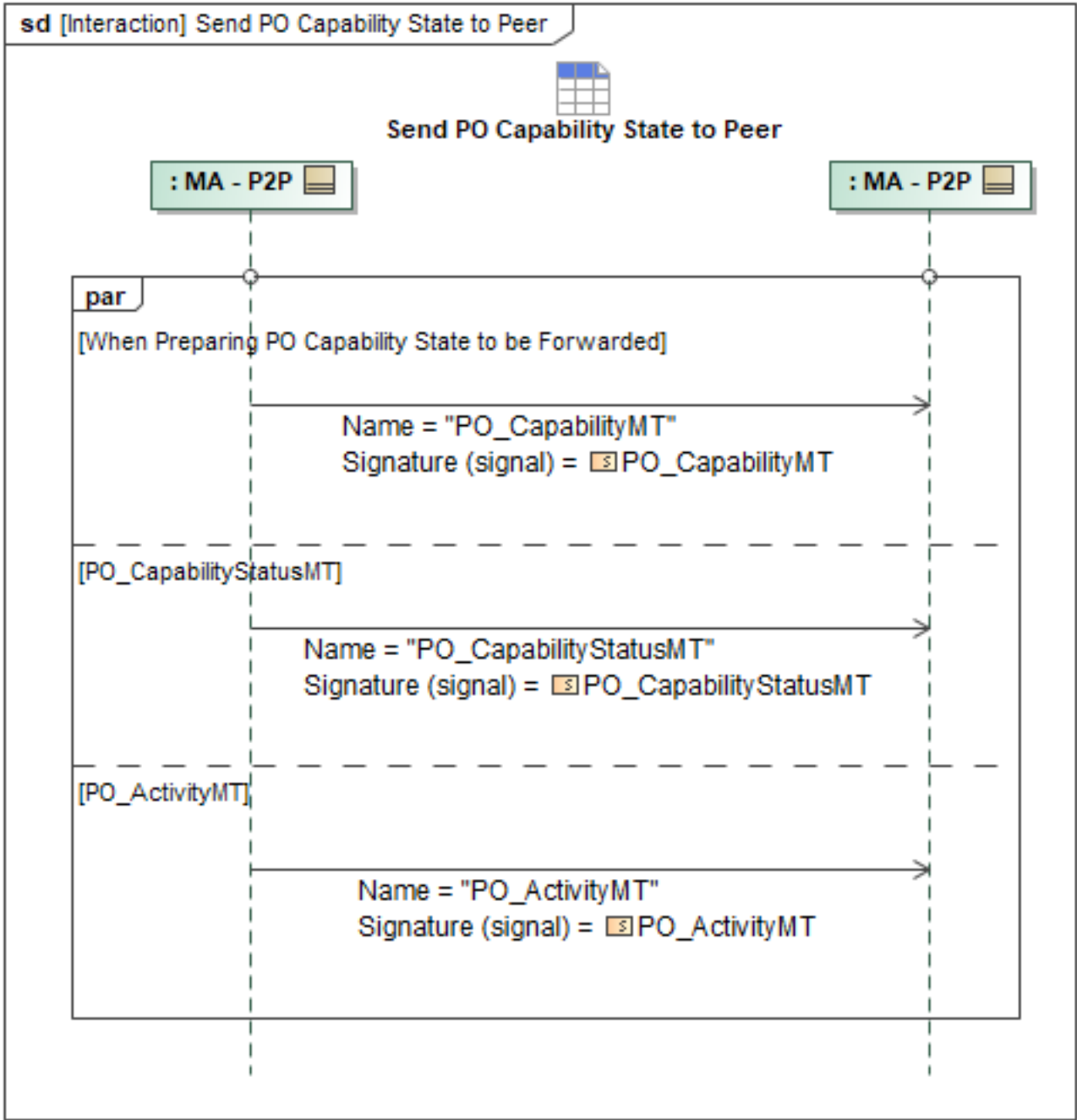


Figure 9-31: Send PO Capability State to Peer

Table 9-30: Send PO Capability State to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
PO_ActivityMT: PO_ActivityMT	None
PO_CapabilityMT: PO_CapabilityMT	None
PO_CapabilityStatusMT: PO_CapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.8 Send Strike Capability State to Peer

Mission Use Case: Store Management

The “Send Strike Capability State Data to Peer” sequence diagram specifies the interaction of the Mission Autonomy (MA) sharing Strike Capability State Data with peers. The transmission of these messages allows ACPs to understand the operational strike capability of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the strike capability state messages would be aggregated by the COP lead for distribution outside of the package.

If the platform has a Strike Capability, the following messages are sent periodically:

MA_StrikeCapabilityMT, *StrikeCapabilityStatusMT*, *StoreManagementStatusMT*, *MA_StrikeActivityMT*.

The *MA_StrikeCapabilityMT* message is published with at least one *Capability* required. Each *Capability* has its own unique identifier set in the *CapabilityID*. The *AcceptedInterface* field must include at least *CAPABILITY_COMMAND*. The *StoreType* and *StoreCategory* fields are used to identity the type of weapons the *Capability* is used to status and command. MS implementation determines how the *InterruptOtherActivities* field is set. Other schema-optional fields are not required but may be specified for more information.

The *StrikeCapabilityStatusMT* messages report status of the weapon capabilities identified by the *Capability* fields in the *MA_StrikeCapabilityMT* message. Each *CapabilityStatus* has the *CapabilityID* set to match the *CapabilityID* field of the corresponding *Capability*. The *Availability* field is used to report general usability of the weapons *Capability* by MA. The *StoreStatus* field is required with *StoreState* and *StoreStatusDetail* communicating the specific state information of the weapon(s) associated with the *Capability*. Other schema-optional fields are not required but may be specified for more information.

The *StoreManagementStatusMT* messages report the status of individual stores and allow MA to know weapon quantities and individual statuses. This is complementary to general usage status provided by *MA_StrikeCapabilityMT* and *StrikeCapabilityStatusMT*. The *SupportCapabilityID* field specifies a unique identifier used by other messaging to manipulate store settings. Each *StoreStation* has a *StationIndex* to identify station using MS implementation specific schemes. The *CompatibleStore* field specifies the type of weapons that station can hold. The *InstalledStore* field is optional but required when the station currently holds a weapon. When specified, the *InstalledStore* field identifies the weapon type. If the *InstalledStore* field and sub-fields are omitted, it is interpreted that this station does not hold a weapon. Each station is associated with a *Capability* defined by *MA_StrikeCapabilityMT* by specifying the *CapabilityID* field, which is required.

The *MA_StrikeActivityMT* communicates more information about the current employment of weapons to MA. Each *Activity* specifies a unique identifier in the *ActivityID* field, corresponding to a scheduled weapon employment. The *CapabilityID* field associates the employment with a specific *Capability* defined in the *MA_StrikeCapabilityMT* message. Interactive is set according to MS implementation. The *ActivityState* field is used to specify the state of the weapon employment activity. The *SelectedForKeyLoad*, *WeaponArmed*, *ReadyForRelease*, *SelectedForRelease*, and *SelectedForJettison* fields provide information about the current state of the weapon while it prepares for the weapon employment activity. The *AssignedTarget* field is required when the weapon is

scheduled to be launched at a target and not jettisoned. The *TimeToRelease* field is required and specifies the timing of the weapon release.

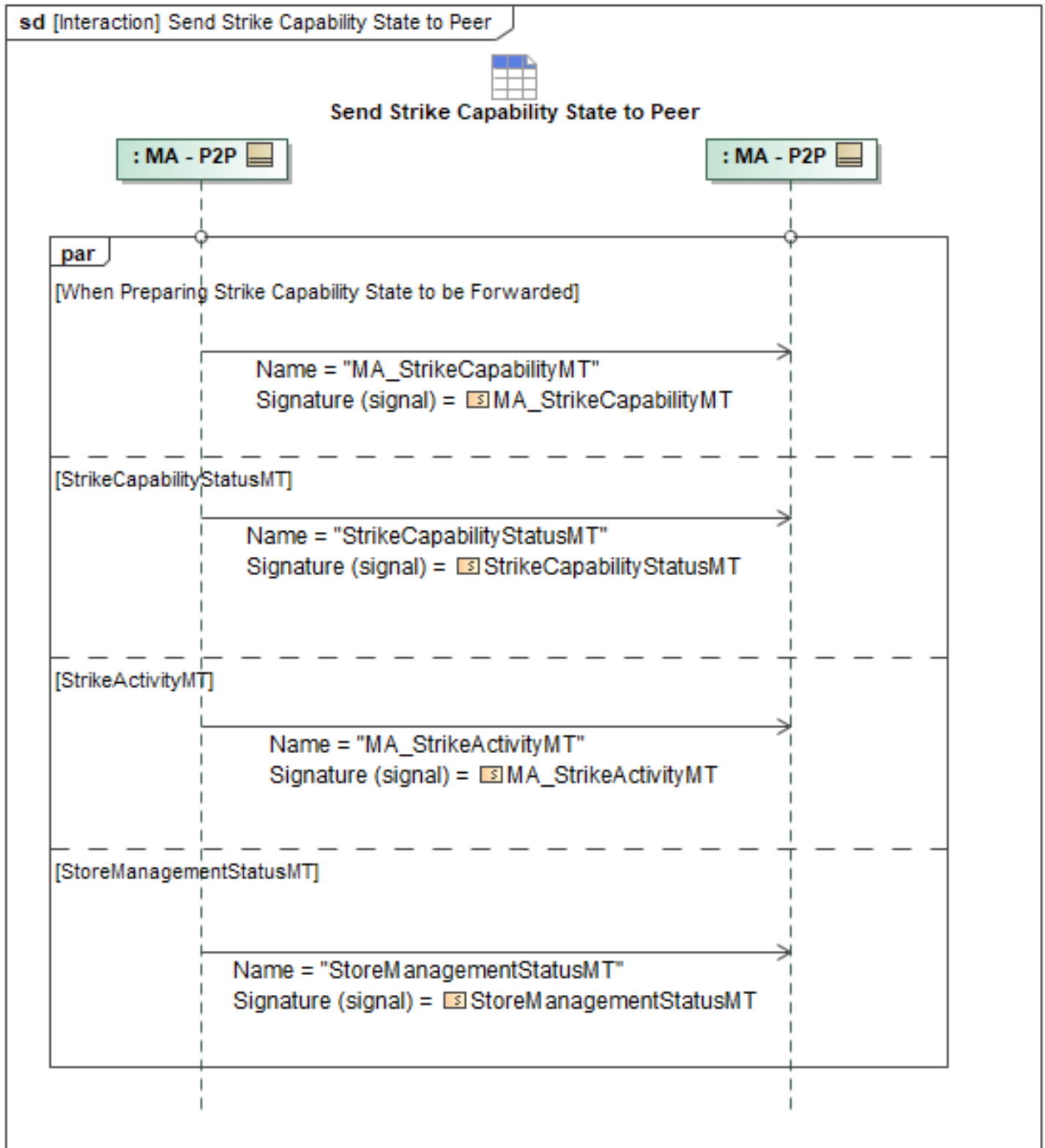


Figure 9-32: Send Strike Capability State to Peer

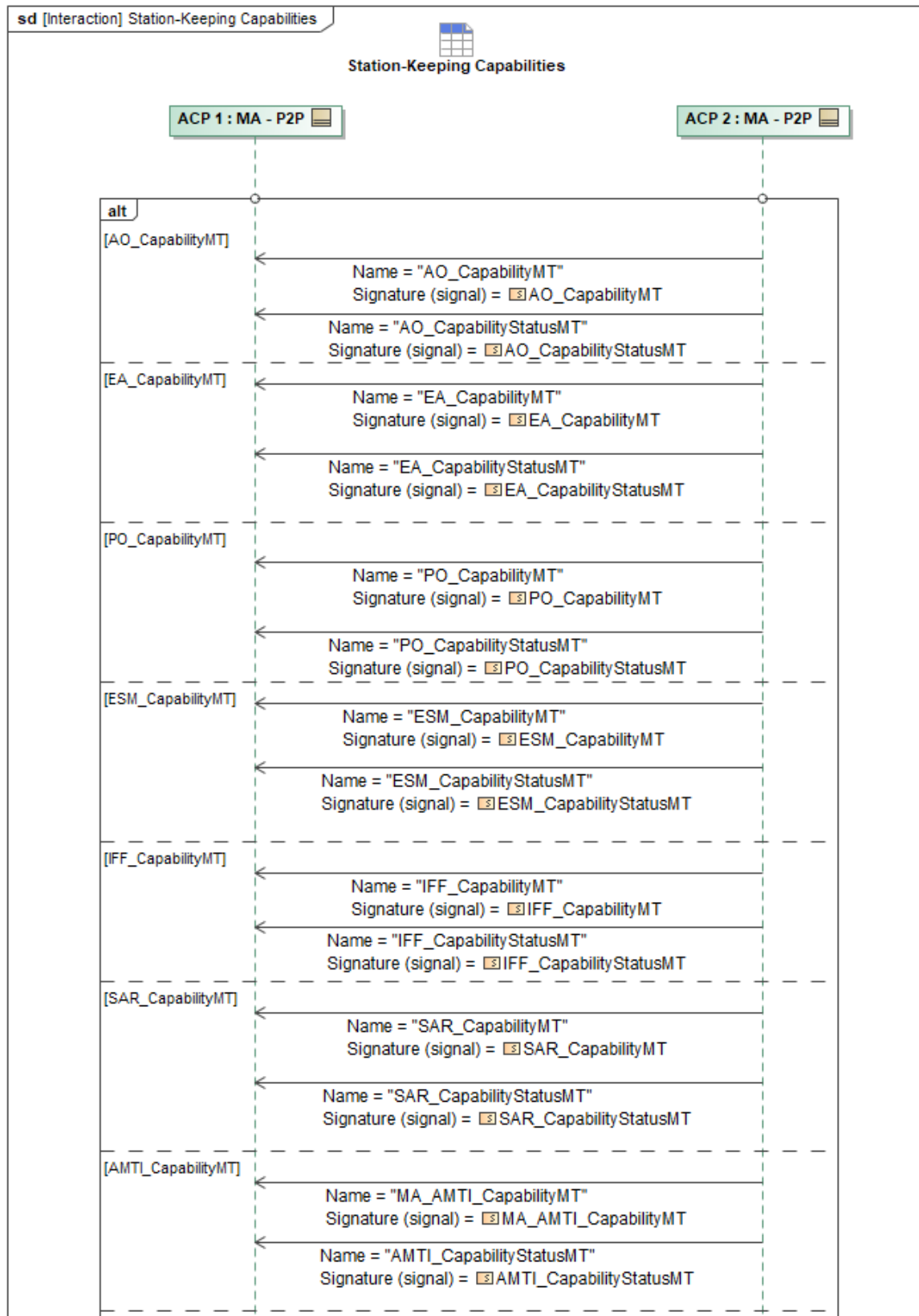
Table 9-31: Send Strike Capability State to Peer

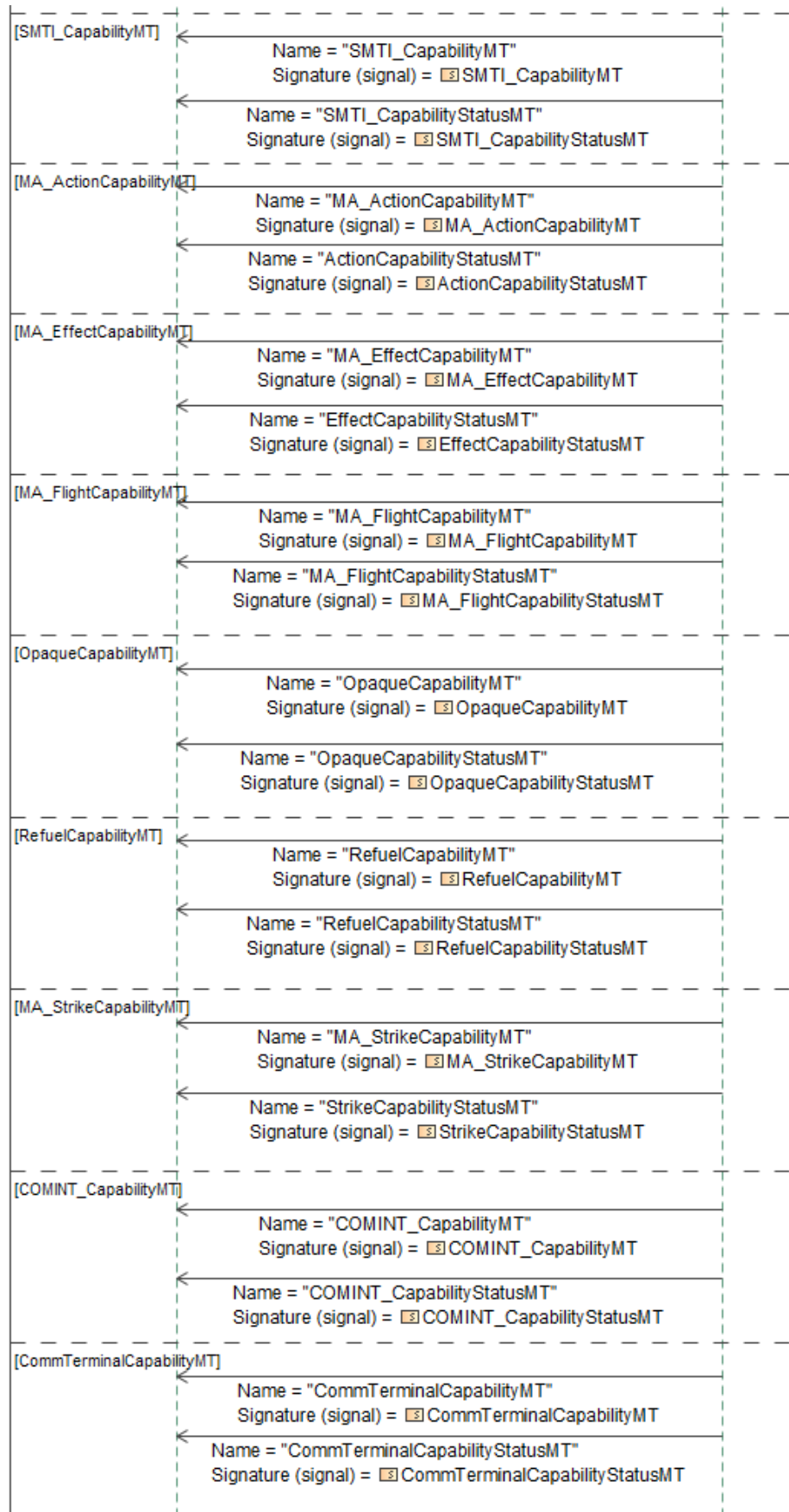
Message Name (Name:Type)	Required Fields (Name:Type)
MA_StrikeActivityMT: MA_StrikeActivityMT	ActivityState: ActivityStateEnum
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	Capability: MA_StrikeCapabilityType
StoreManagementStatusMT: StoreManagementStatusMT	CapabilityID: CapabilityID_Type
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	StoreStatus: StoreStatusType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.6.9 Station-Keeping Capabilities

The known set of station-keeping capabilities is presented in the following sequence diagram. A platform may advertise multiple of these capabilities (*[Capability]ActivityMT* and *[Capability]StatusMTs*) based on platform configuration, and each of which is sent periodically as a separate instance of this sequence.





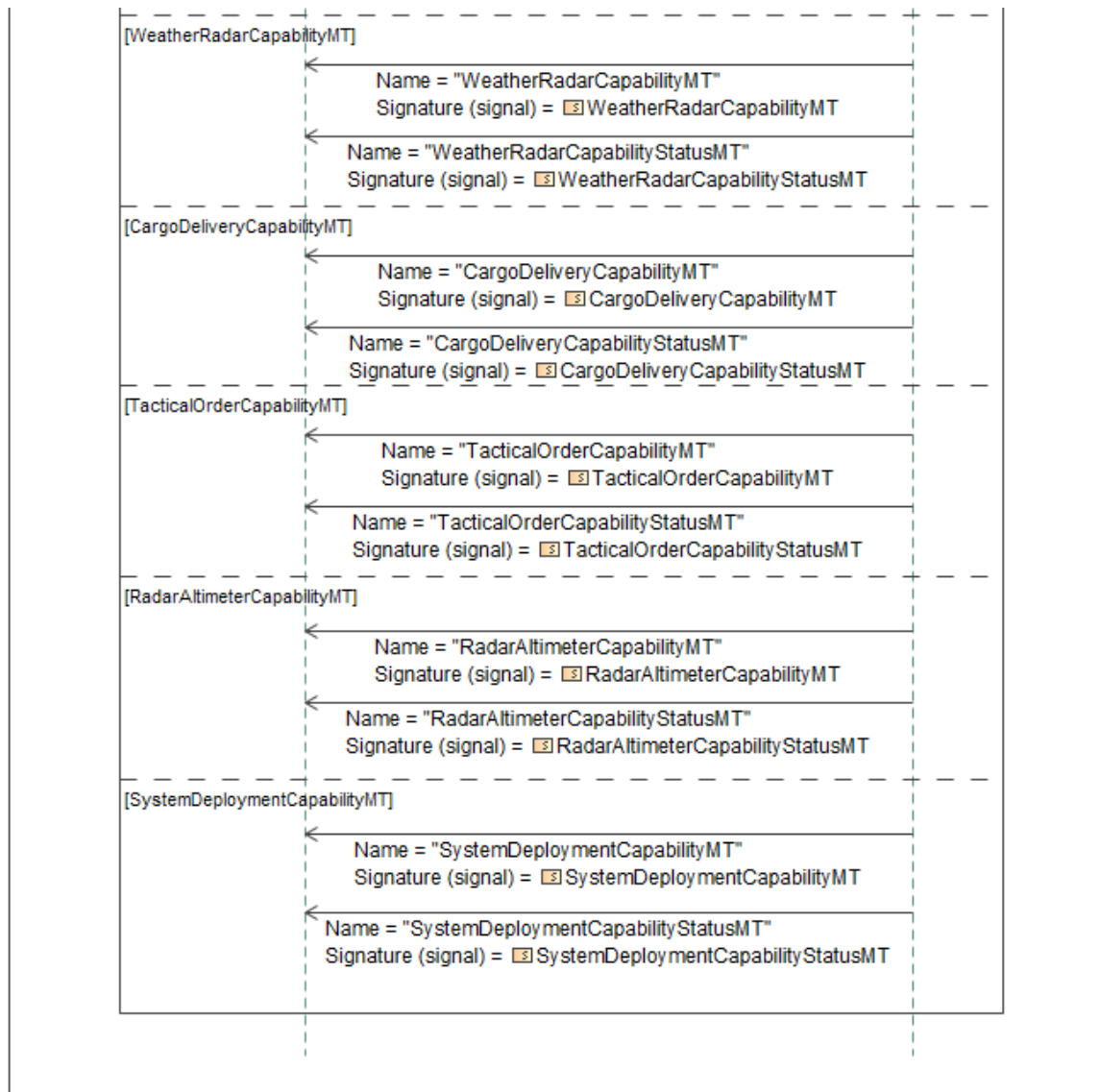


Figure 9-33: Station-Keeping Capabilities

Table 9-32: Station-Keeping Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCapabilityStatusMT: ActionCapabilityStatusMT	None
AMTI_CapabilityStatusMT: AMTI_CapabilityStatusMT	None
AO_CapabilityMT: AO_CapabilityMT	None
AO_CapabilityStatusMT: AO_CapabilityStatusMT	None
CargoDeliveryCapabilityMT: CargoDeliveryCapabilityMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
CargoDeliveryCapabilityStatusMT: CargoDeliveryCapabilityStatusMT	None
COMINT_CapabilityMT: COMINT_CapabilityMT	None
COMINT_CapabilityStatusMT: COMINT_CapabilityStatusMT	None
CommTerminalCapabilityMT: CommTerminalCapabilityMT	None
CommTerminalCapabilityStatusMT: CommTerminalCapabilityStatusMT	None
EA_CapabilityMT: EA_CapabilityMT	None
EA_CapabilityStatusMT: EA_CapabilityStatusMT	None
EffectCapabilityStatusMT: EffectCapabilityStatusMT	None
ESM_CapabilityMT: ESM_CapabilityMT	None
ESM_CapabilityStatusMT: ESM_CapabilityStatusMT	None
IFF_CapabilityMT: IFF_CapabilityMT	None
IFF_CapabilityStatusMT: IFF_CapabilityStatusMT	None
MA_ActionCapabilityMT: MA_ActionCapabilityMT	None
MA_AMTI_CapabilityMT: MA_AMTI_CapabilityMT	None
MA_EffectCapabilityMT: MA_EffectCapabilityMT	None
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None
MA_StrikeCapabilityMT: MA_StrikeCapabilityMT	None
OpaqueCapabilityMT: OpaqueCapabilityMT	None
OpaqueCapabilityStatusMT: OpaqueCapabilityStatusMT	None
PO_CapabilityMT: PO_CapabilityMT	None
PO_CapabilityStatusMT: PO_CapabilityStatusMT	None
RadarAltimeterCapabilityMT: RadarAltimeterCapabilityMT	None
RadarAltimeterCapabilityStatusMT: RadarAltimeterCapabilityStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
RefuelCapabilityMT: RefuelCapabilityMT	None
RefuelCapabilityStatusMT: RefuelCapabilityStatusMT	None
SAR_CapabilityMT: SAR_CapabilityMT	None
SAR_CapabilityStatusMT: SAR_CapabilityStatusMT	None
SMTI_CapabilityMT: SMTI_CapabilityMT	None
SMTI_CapabilityStatusMT: SMTI_CapabilityStatusMT	None
StrikeCapabilityStatusMT: StrikeCapabilityStatusMT	None
SystemDeploymentCapabilityMT: SystemDeploymentCapabilityMT	None
SystemDeploymentCapabilityStatusMT: SystemDeploymentCapabilityStatusMT	None
TacticalOrderCapabilityMT: TacticalOrderCapabilityMT	None
TacticalOrderCapabilityStatusMT: TacticalOrderCapabilityStatusMT	None
WeatherRadarCapabilityMT: WeatherRadarCapabilityMT	None
WeatherRadarCapabilityStatusMT: WeatherRadarCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.7 Peer Engagement Criteria Behaviors Package

A Package Lead manages the activation of the engagement criteria for its Followers based on the activation commanded by C2. The Package Lead will also manage delegation of Targets that were designated by a Target Authority. The Package Lead can act as a Target Authority to the package if and only if a Target Authority has already designated that Target to the package or a specific follower in the package. Once the Target Authority has Designated a Target, the Package Lead can:

- If the Target was assigned to the Package ID:
 - The Lead can use the Centralized Computation of Target Assignments sequence to determine the allocation of Targets to ACPs and can then send a *MA_DesignationMT* or directly command a Targeting Act to the appropriate Followers.
 - The Lead can use the Distributed Computation of Target Assignments sequence to allow the ACPs to determine which Target they should go after. The followers will send a *MA_DesignationRequestMT* to the Leader. The leader will then send a *MA_DesignationMT* or directly command a Targeting Act to the appropriate Follower.

- If the Target was assigned to a specific System ID that is a Follower in the Package, the Lead can send a *MA_DesignationMT* or directly command a Targeting Act to that Follower.

Any Follower can send a *MA_DesignationRequestMT* to the Leader, if the Target was not already approved, the leader would send a *MA_DesignationRequestMT* to a Target Authority for approval. Each Package member can still receive Target assignments directly from a Targeting Authority.

9.2.7.1 Activate Peer Engagement Criteria

The Activate Peer Engagement Criteria sequence diagram specifies the interaction of the lead Autonomous Collaborative Platform (ACP) commanding its follower ACPs to activate one of its stored engagement criteria messages. First, the lead ACP sends a *MA_EngagementCriteriaCommandMT* message where the *AppliesToID* field is defined with the System ID of the follower ACP. The *MA_EngagementCriteriaCommandMT* message shall specify the *EngagementCriteriaID* field containing the ID of the engagement criteria message that the lead ACP wishes to be active. Any new *MA_EngagementCriteriaCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_EngagementCriteriaCommandMT* is sent with the *CommandedState* set to *CANCEL*, this will cancel the active command and engagement criteria activation schema. In response to the command, the follower ACPs shall send *MA_EngagementCriteriaCommandStatusMT_1* to the lead ACP, with the *AppliesToID* field set to their System ID and the *CommandProcessingState* field set to *RECEIVED* to indicate receipt of the command. Once the follower ACPs have finished querying engagement criteria from onboard MP, the follower ACPs shall send *MA_EngagementCriteriaCommandStatusMT_2* to the lead ACP with the *AppliesToID* field set to their System ID and the *CommandProcessingState* set to *ACCEPTED*. The follower ACPs will send *MA_EngagementCriteriaStatusMT* to the lead ACP containing the *EngagementCriteriaID*, the *AppliesToID* field set to their System ID, and other configured data as needed.

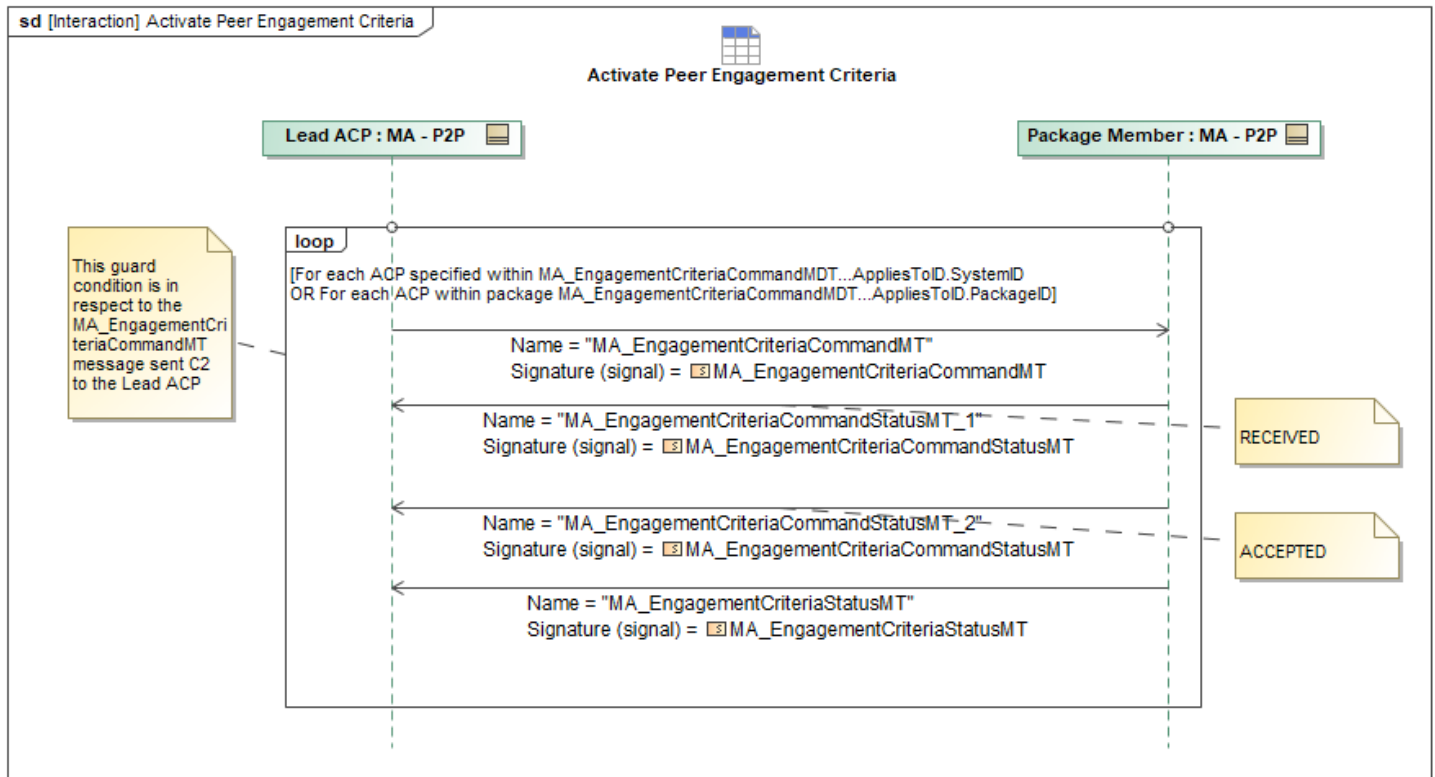


Figure 9-34: Activate Peer Engagement Criteria

Table 9-33: Activate Peer Engagement Criteria

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EngagementCriteriaCommandMT: MA_EngagementCriteriaCommandMT	None
MA_EngagementCriteriaCommandStatusMT_1: MA_EngagementCriteriaCommandStatusMT	None
MA_EngagementCriteriaCommandStatusMT_2: MA_EngagementCriteriaCommandStatusMT	None
MA_EngagementCriteriaStatusMT: MA_EngagementCriteriaStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.8 Peer Generic Package

9.2.8.1 Peer Sends Message Acknowledgment

Peer Acknowledgment Configurations sequences defines which messages MA would expect a Peer MA to respond with an acknowledgment of receipt. If a Peer MA receives a configured message, the Peer MA will send a *MA_SystemNotificationMT* with the *NotificationState* set to *CONFIRMED*, *Source* set to its own *SystemID*, *Severity* set to *INFORMATIONAL*, *SystemSubjectIDs* set to its own *SystemID*, *SystemPerspective* set to *SOURCE*, and *AssociatedMessage* set to the *MessageType* of the received message and the *AssociatedID* of the applicable ID in the received message. The

MA_SystemNotificationMT can also be sent at limited periodic rates for acknowledgments based on the Message Acknowledgment Configuration where it would group together all received *AssociatedMessages* over the periodic time that was configured. This is done by adding multiple entries to the *AssociatedMessage* field for each message. The *MA_SystemNotificationMT* would only be sent at that periodic rate if there was a received message during that time.

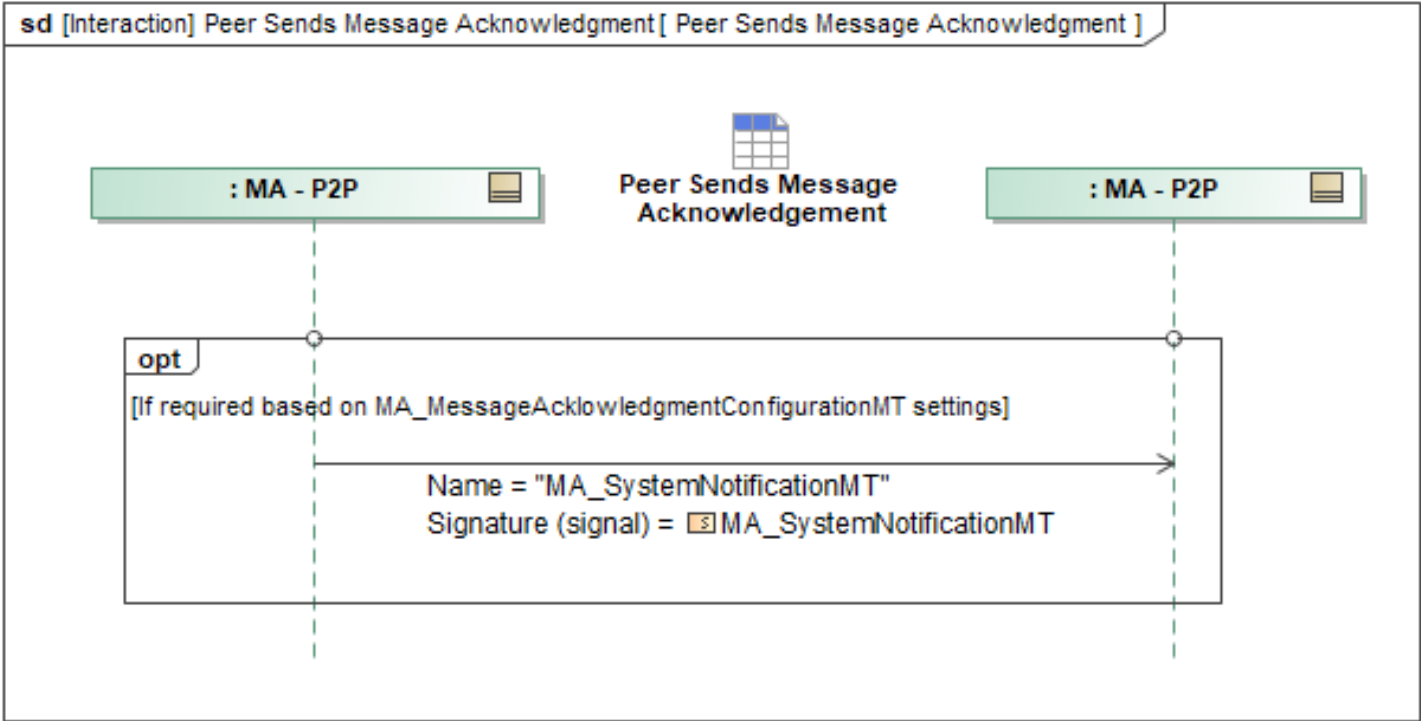


Figure 9-35: Peer Sends Message Acknowledgment

Table 9-34: Peer Sends Message Acknowledgment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.9 Peer Geozones Package

9.2.9.1 Peer Command Geozone

This diagram contains the collection of data messages used when Peer commands the state of any Op* message (e.g., *MA_OpZoneMT*, *MA_OpVolumeMT*, etc.) currently stored on another Peer ACP, independent of mission plan. When the state of a given Op* message is "ACTIVE", the constraints imposed by said message must be considered by MA.

Upon receipt of an *MA_OpCommandMT*, the receiving Peer MA shall store the command locally. The *MA_OpCommandMT* includes a reference to an existing Op* message (*OpID*), an indication of whether the command is to activate or de-activate (*CommandedState*), and any relevant conditions that must be met in order for the command to execute (*ConstraintsOverride*). Additionally, the *Rank*

field allows the receiving Peer MA to resolve conflicting command messages through comparative ranking. The receiving Peer MA acknowledges a received command by sending *MA_OpCommandStatusMT_1* with *CommandProcessingState* of "RECEIVED". The receiving Peer MA then evaluates the conditions and ranking of the received *MA_OpCommandMT* and sends an *MA_OpCommandStatusMT_2* message with *CommandProcessingState* of "ACCEPTED" if successful. Otherwise, *MA_OpCommandStatusMT_3* is sent with the *CommandProcessingState* of "REJECTED", and the accompanying *CommandProcessingStateReason* populated with the reason for the rejection (e.g., "RANKING").

MA continuously monitors the conditions specified in *ConstraintsOverride* for all stored *MA_OpCommandMT* messages. If there are no conditions specified (in the *ConstraintsOverride* field), or all conditions are met, MA changes the state of the referenced Op* message and sends an *MA_OpStatusMT* notifying the sending Peer MA of the updated status (*CurrentState*) of the Op* message and the command that triggered the change (*CommandID*).

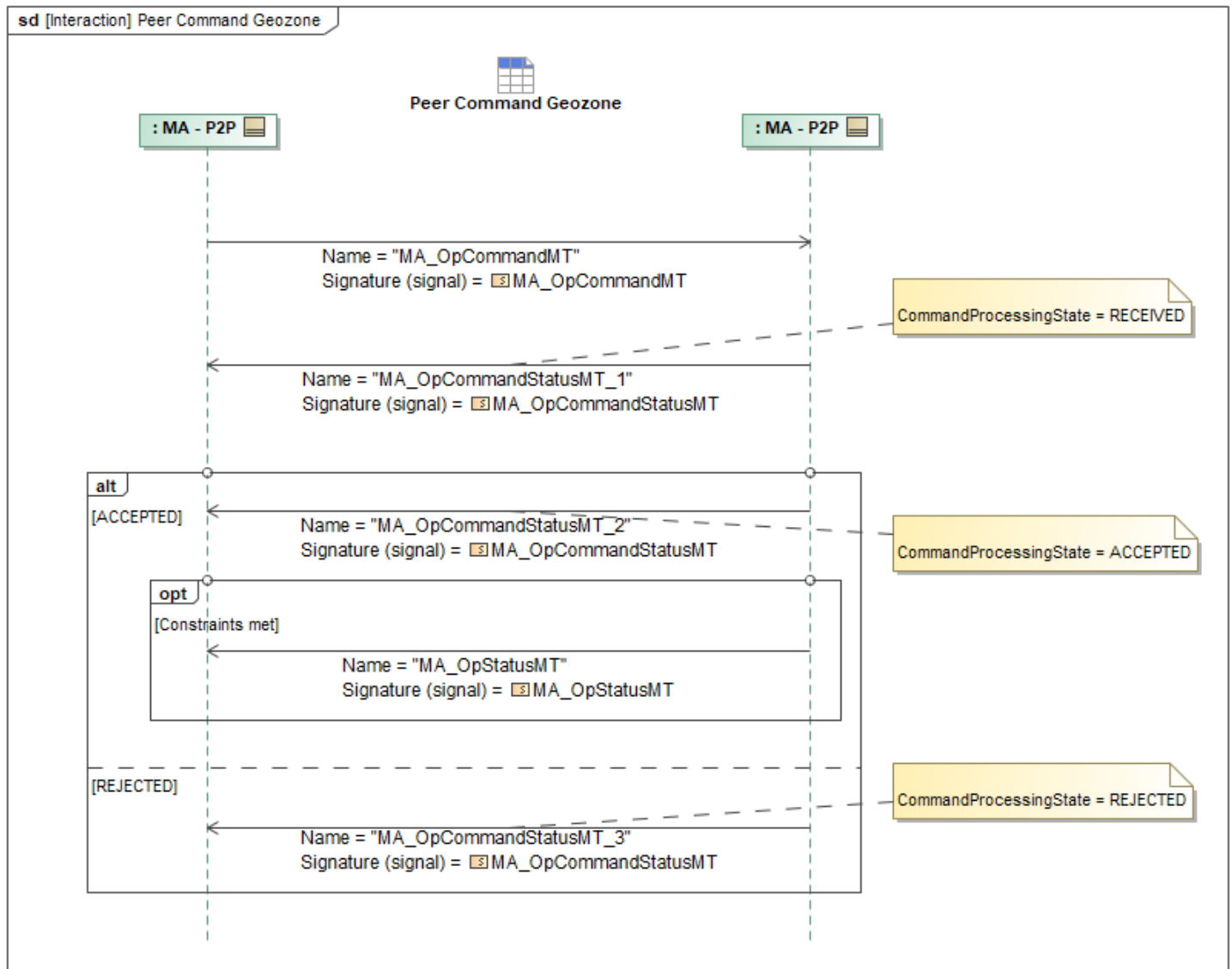


Figure 9-36: Peer Command Geozone

Table 9-35: Peer Command Geozone

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpCommandMT: MA_OpCommandMT	OpID: OpID_ChoiceType
MA_OpCommandStatusMT_1: MA_OpCommandStatusMT	None
MA_OpCommandStatusMT_2: MA_OpCommandStatusMT	None
MA_OpCommandStatusMT_3: MA_OpCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_OpStatusMT: MA_OpStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10 Peer Mission Plan Management Package

9.2.10.1 P2P Contingency Alert

This interaction describes the process of a package member alerting the package lead that a contingency has occurred. The package member sends a *MissionContingencyAlertMT* message to the lead indicating what has happened. This may trigger additional actions by the package lead which could include retasking or a replan.

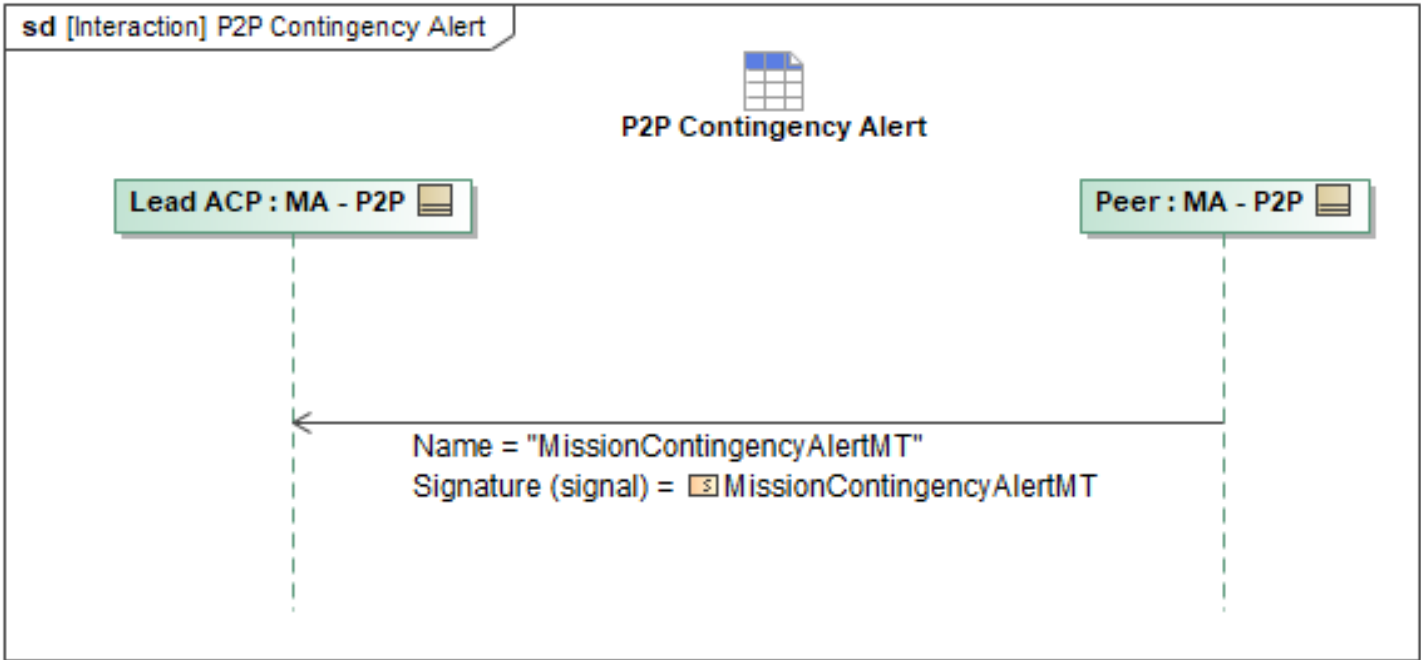


Figure 9-37: P2P Contingency Alert

Table 9-36: P2P Contingency Alert

Message Name (Name:Type)	Required Fields (Name:Type)
MissionContingencyAlertMT: MissionContingencyAlertMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.2 P2P Detect Mission Plan Route Discontinuity Response

Upon identification of a Mission Plan Route Discontinuity Error, MA will send *MA_SystemNotificationMT* to C2 and P2P, to indicate that a Mission Plan Route Discontinuity condition exists. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with INFEASIBLE_ROUTE. This indicates that the ACP can no longer continue along the nominal Mission Plan path from its current or projected position. *AssociatedMessage.MessageType* will have the ROUTE_PLAN enumeration to indicate to P2P that the route is no longer feasible. *AssociatedMessage.AssociatedID* may reference the affected *PathSegmentID* that precedes the discontinuity, or alternatively, the *FlightCommandID* can be used if a *PathSegmentID* does not exist or cannot be referenced.

At the time of the discontinuity, MA also sends *MA_PositionReportDetailed* so that P2P has access to the current and projected position data relevant to the error condition. The *Kinematics* field communicates the ACP's current position and velocity associated with the discontinuity. Resolution actions may include MA executing a plan modification or receiving a resolution from C2 or P2P. Upon resolution of the error, MA will send *MA_SystemNotificationMT* to C2 and P2P with *NotificationState* set to CLOSED and the same other fields mentioned in the initial *MA_SystemNotificationMT*, including *NotificationDetail.Reason* and AssociatedMessage details, to provide a consistent update on the resolved condition.

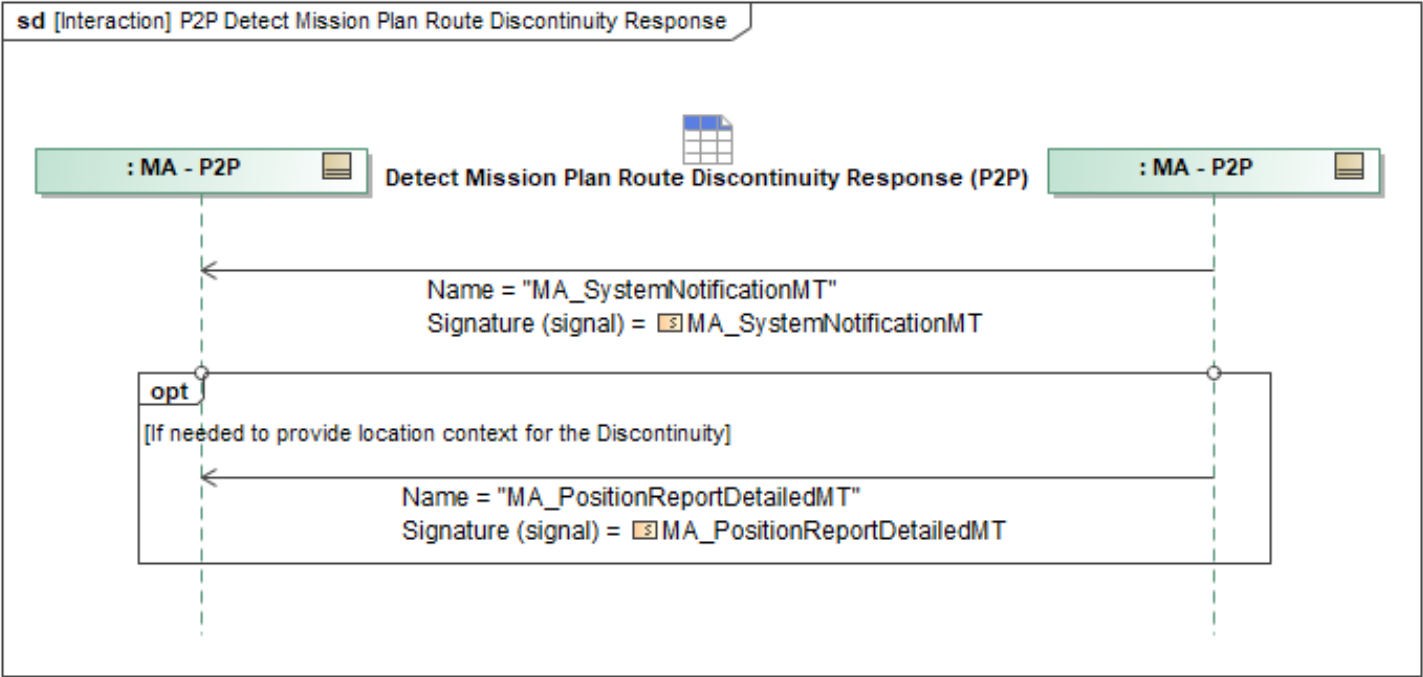


Figure 9-38: P2P Detect Mission Plan Route Discontinuity Response

Table 9-37: P2P Detect Mission Plan Route Discontinuity Response

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	None
MA_SystemNotificationMT: MA_SystemNotificationMT	Description: VisibleString1024Type AssociatedID: ID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.3 P2P Request Plan Patch

In the event that Package Lead determines that a Plan Patch is necessary, a Package Lead can request that a Follower MA perform a Plan Patch. If the desired Plan Patch is applicable to a Route Plan, Package Lead sends a Follower MA a *RouteModificationRequestMT*.

The *RouteModificationRequestMT* contains a Path field and a *PathSegment* field that communicate one of three possible outcomes: the addition of a path, removal of a path, or update a path, through the *Add*, *RemoveID*, or *Update* fields, respectively. Contained in the *PathSegment* field, the *PathID*

field identifies the Path which the segment changes apply to. If the desired Plan Patch is applicable to any other type of Plan, Package Lead sends a Follower MA an *MA_PlanModificationRequestMT*. The *MA_PlanModificationRequest* contains a *PlanModificationDetails* field which allows for a number of potential plan modification methods. For Plan Patching, the *SubPlansModification* field will be used to either add a *SubPlan* (*SubPlanAdditions* field) or retract a *SubPlan* (*SubPlanRetractions* field). Follower MA then responds with the corresponding **StatusMT* message (*RouteModificationRequestStatusMT* or *MA_PlanModificationRequestStatusMT*), which contains the *RequestID* and the *RequestProcessingState* which indicates the status of the request.

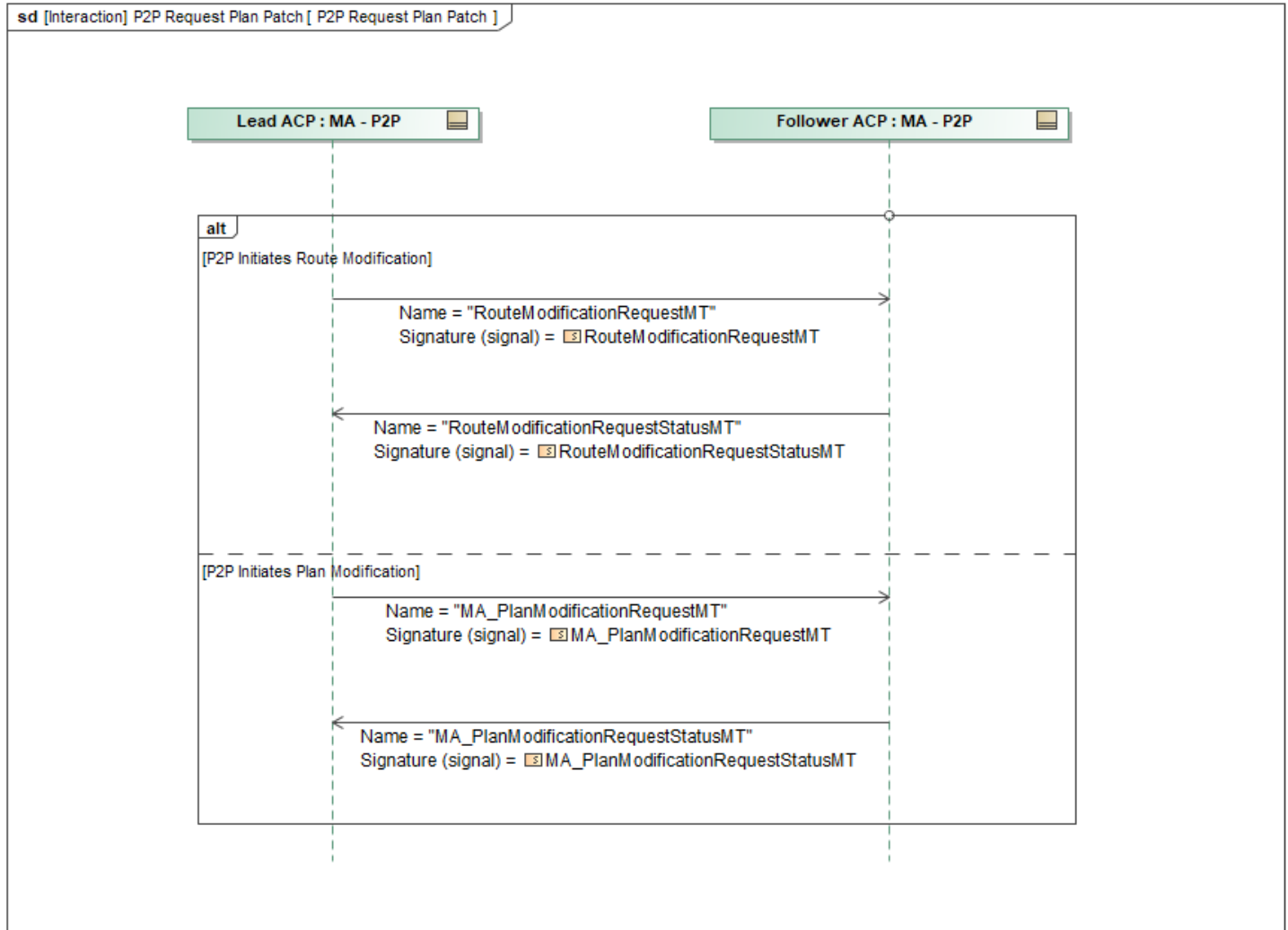


Figure 9-39: P2P Request Plan Patch

Table 9-38: P2P Request Plan Patch

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanModificationRequestMT: MA_PlanModificationRequestMT	None
MA_PlanModificationRequestStatusMT: MA_PlanModificationRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
RouteModificationRequestMT: RouteModificationRequestMT	None
RouteModificationRequestStatusMT: RouteModificationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.4 Peer Receive Plan Modification Notification

When MA updates a Plan, there may be Peer elements that need to be informed for situational awareness or synchronization reasons. This sequence describes the messaging that MA can use to broadcast the fact that MA has changed a Plan. If MA has modified a *MissionPlan* or a *SubPlan*, MA will send a *MA_PlanModificationRequestMT* to Peer, requesting that Peer updates the Plan in question. Likewise, if MA has only modified a *RoutePlan*, MA will send a *RouteModificationRequestMT* to the same effect. Contained in the *MessageData* fields of these messages are fields that describe the Plan modifications which are populated as required to mirror the change that MA has already made to the Plan. Upon receipt of request, Peer will respond to MA with a *MA_PlanModificationRequestStatusMT* or *RouteModificationStatusMT* describing the status of the Plan update.

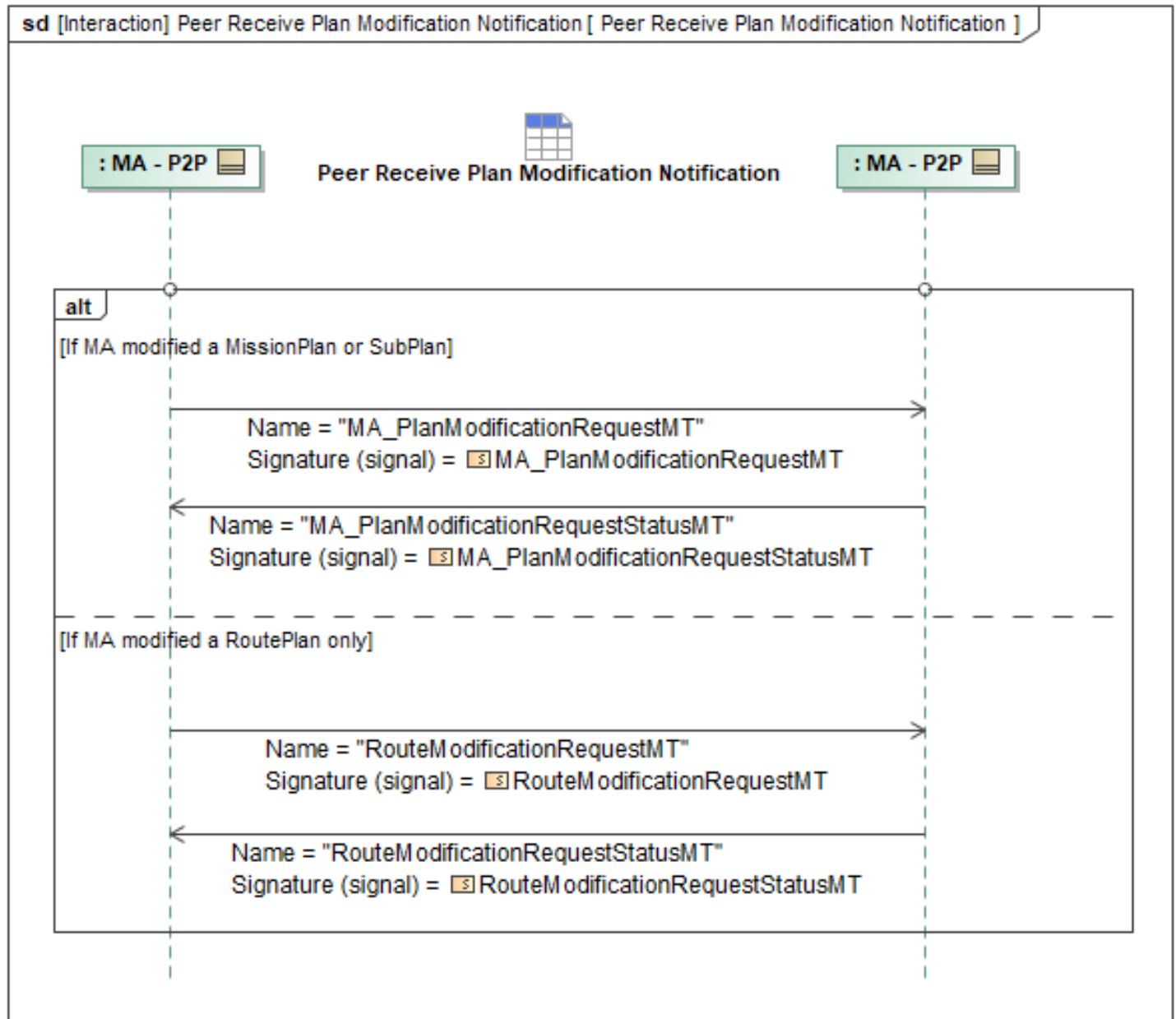


Figure 9-40: Peer Receive Plan Modification Notification

Table 9-39: Peer Receive Plan Modification Notification

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PlanModificationRequestMT: MA_PlanModificationRequestMT	None
MA_PlanModificationRequestStatusMT: MA_PlanModificationRequestStatusMT	None
RouteModificationRequestMT: RouteModificationRequestMT	None
RouteModificationRequestStatusMT: RouteModificationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.5 Query Deconfliction Data

This interaction describes the process of an ACP requesting deconfliction data from another ACP. Asset Deconfliction data, used to prevent the ACP from colliding with another platform, is defined by *OpLineMT*, *OpZoneMT*, and *OpRoutingMT*.

OpLineMT contains data used to convey a Corridor (a line with width along the designated Route) describing the passageway designated for Mission planning deconfliction for each ACP. Using the *Category* field set to *CORRIDOR*. The *LeftWidth* and *RightWidth* fields are required to describe the overall width of the corridor. The defined data is to be used when deconflicting entities routes or generating routes to ensure proper separation. These parameters are applied at the vehicle level.

OpZoneMT, contains data used to convey Altitude. Data in the *Zone* field captures the shape of the region, along with the Max and Min altitudes. Describing the region designated for mission planning deconfliction for each ACP. The defined data is to be used when deconflicting entities routes or generating routes to ensure proper Altitude block compliance. These parameters are applied at the vehicle level.

OpRoutingMT contains data used to convey a Bubble: The *SpecificBlueSeparation* and *SpecificRedSeparation* define parameters to use when deconflicting entity routes or generating routes to ensure compliance of the designated keep in and keep out zones. These parameters are applied at the vehicle type level.

Data is stored or cached in data manager services which may have a database backend. The ACP can query this data via *QueryDataRequestMT* by including three instances of *MessageType* with their enumerations set to *OP_LINE*, *OP_ROUTING*, and *OP_ZONE* respectively. This will be responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *RequestProcessingState* field such as “*QUEUED*” and “*PROCESSING*” to indicate additional in-process states. When complete, the *QueryDataRequestStatusMT* message’s *RequestProcessingState* field is set to “*COMPLETED*” and contains the requested data. When the original *QueryDataRequestMT*’s optional field *ResultsInNative* is populated, the underlying data management services publishes the requested data in its original or raw format as a separate message from the *QueryDataRequestStatusMT* message at completion.

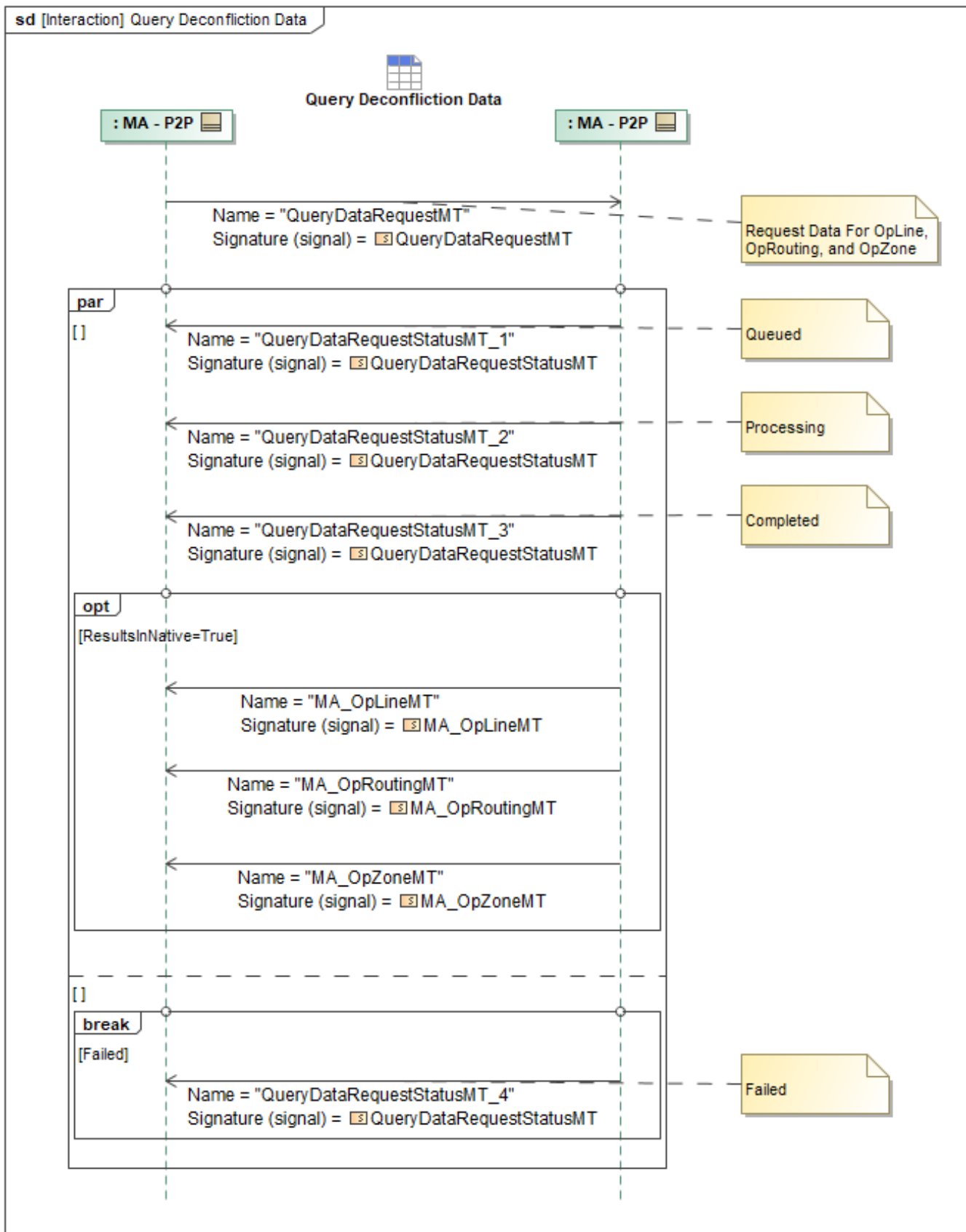


Figure 9-41: Query Deconfliction Data

Table 9-40: Query Deconfliction Data

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpLineMT: MA_OpLineMT	RightWidth: DistanceType LeftWidth: DistanceType
MA_OpRoutingMT: MA_OpRoutingMT	SpecificBlueSeparation: SpecificBlueSeparationType SpecificRedSeparation: SpecificRedSeparationType
MA_OpZoneMT: MA_OpZoneMT	None
QueryDataRequestMT: QueryDataRequestMT	MessageType: MessageTypeEnum
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.6 Synchronize Contingency Plan

This diagram contains the messages sent between Peers to synchronize a contingency plan. The *MA_ApprovalPolicyMT* and *ResponsePlanMT* will be sent with an *ObjectState* enumerations set to UPDATED.

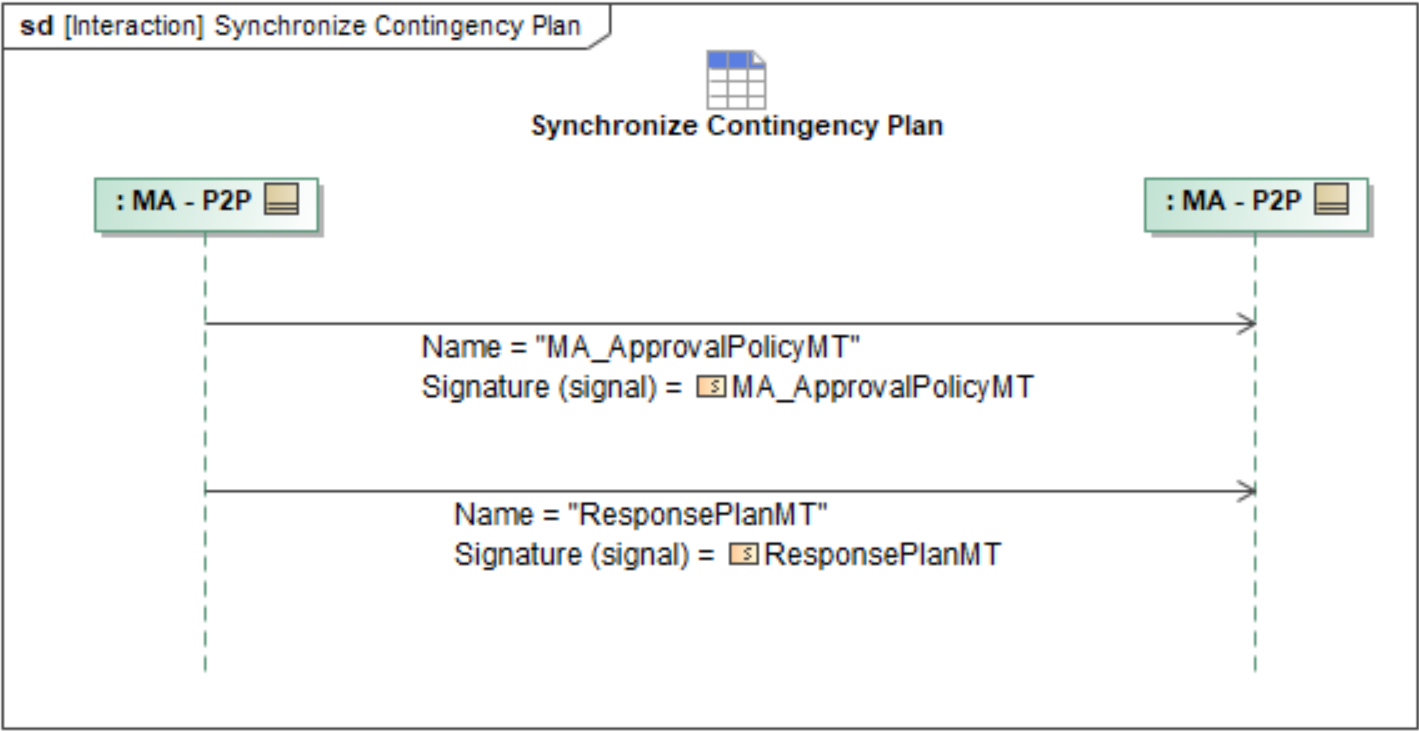


Figure 9-42: Synchronize Contingency Plan

Table 9-41: Synchronize Contingency Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	DataRecordInstanceID: DataRecordInstanceID_Type
ResponsePlanMT: ResponsePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.7 Update Peer Airfield Data

Mission Plan Data such as Airfield Reports are updated via UCI *AirfieldReportMT* message with the Optional field *ObjectState*.

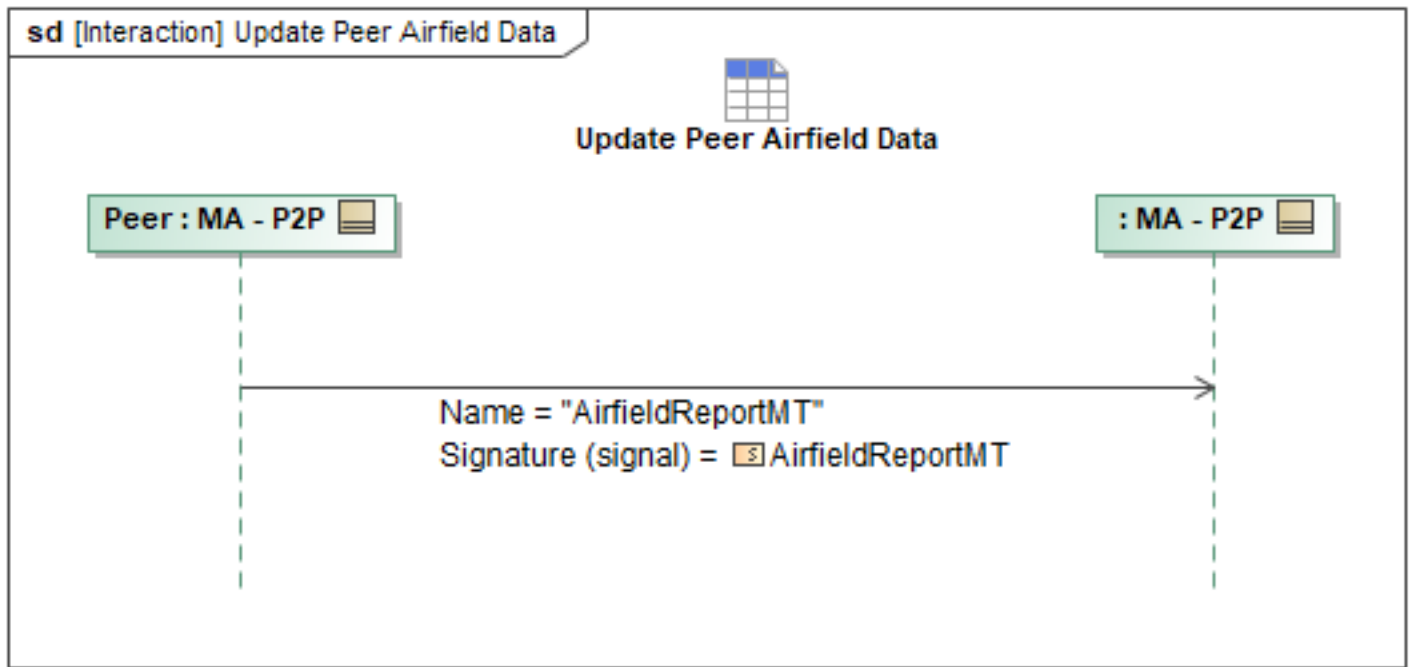


Figure 9-43: Update Peer Airfield Data

Table 9-42: Update Peer Airfield Data

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.8 P2P Lead Updates Members

9.2.10.8.1 P2P Lead Updates Member with Constraint Priority List

This interaction specifies how a lead ACP can update various data stored in the follower ACPs. For each follower ACP, the lead ACP shares *MA_ConstraintPriorityListMT* with the package member. Upon receiving this message, follower ACPs update onboard MP with new CPL.

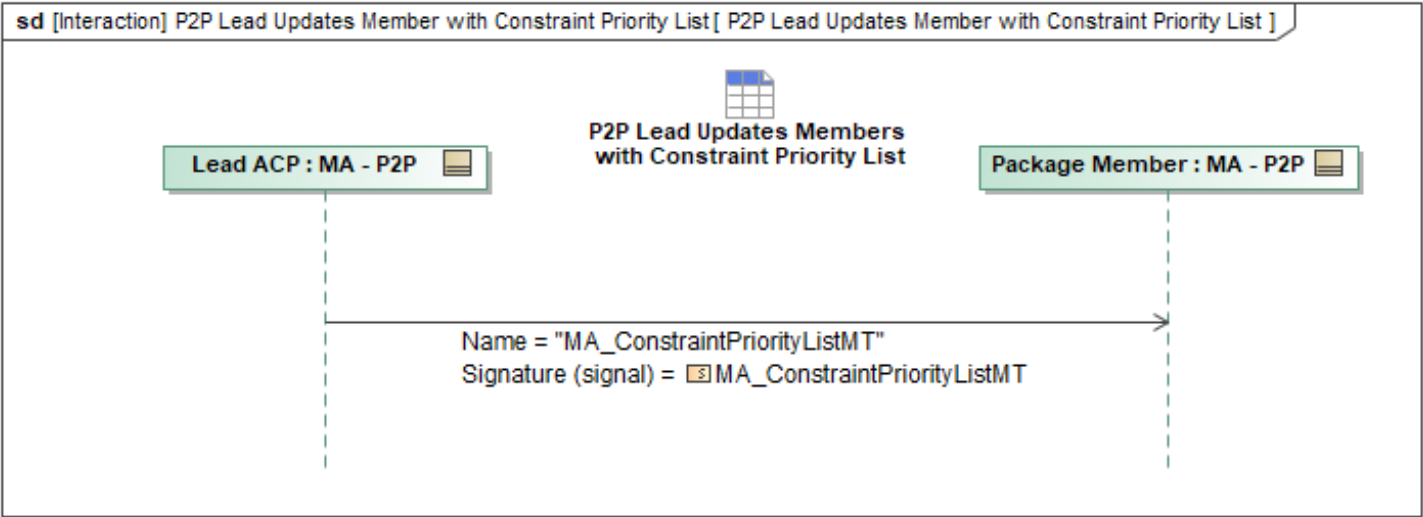


Figure 9-44: P2P Lead Updates Member with Constraint Priority List

Table 9-43: P2P Lead Updates Member with Constraint Priority List

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListMT:	ObjectState: ObjectStateEnum
MA_ConstraintPriorityListMT	ListItem: MA_ConstraintPriorityListItemType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.8.2 P2P Lead Updates Member with Constraint Priority List Activation Data

This interaction specifies how a lead ACP can update various data stored in the Follower ACPs. For each follower ACP, the lead ACP shares *MA_ConstraintPriorityListActivationDataMT* with the package member. Upon receiving this message, follower ACPs update onboard MP with new CPL activation data.

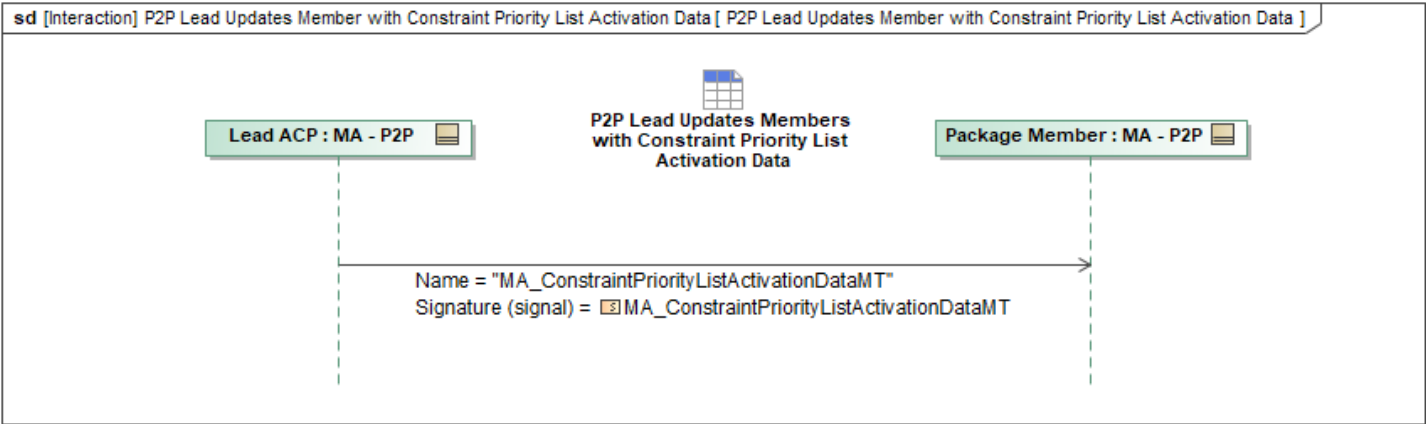


Figure 9-45: P2P Lead Updates Member with Constraint Priority List Activation Data

Table 9-44: P2P Lead Updates Member with Constraint Priority List Activation Data

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationDataMT: MA_ConstraintPriorityListActivationDataMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.8.3 P2P Lead Updates Member with Engagement Criteria

This interaction specifies how a lead ACP can update various data stored in the Follower ACPs. For each follower ACP, the lead ACP shares *MA_EngagementCriteriaMT* with the package member. Upon receiving this message, follower ACPs update onboard MP with new engagement criteria data.

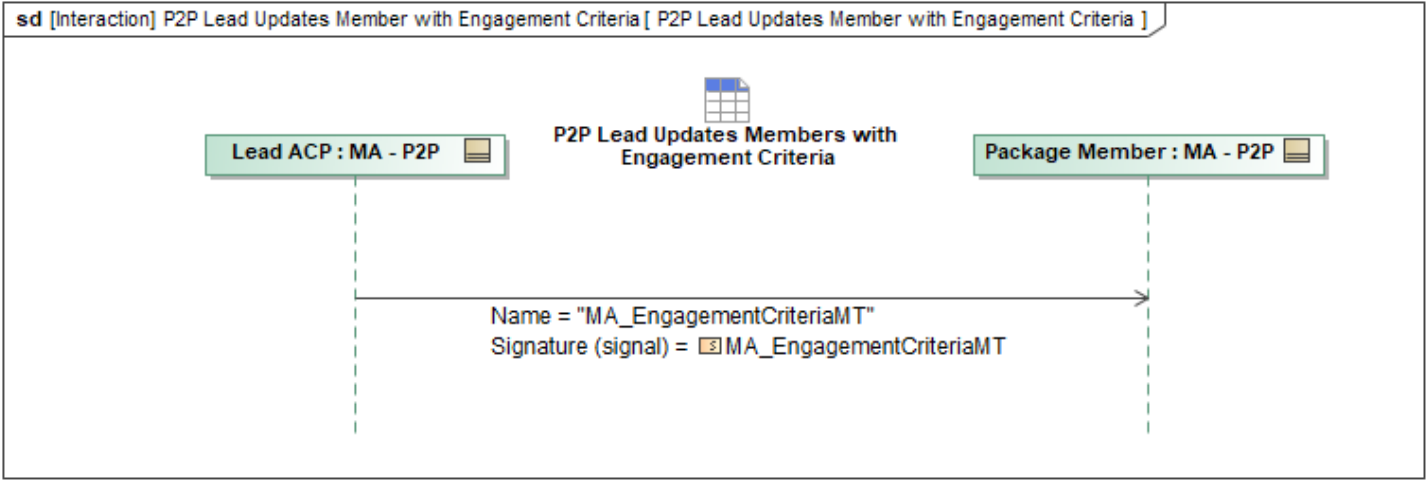


Figure 9-46: P2P Lead Updates Member with Engagement Criteria

Table 9-45: P2P Lead Updates Member with Engagement Criteria

Message Name (Name:Type)	Required Fields (Name:Type)
MA_EngagementCriteriaMT: MA_EngagementCriteriaMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema

Page 684

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.8.4 P2P Lead Updates Member with Execution Priority List

This interaction specifies how a lead ACP can update various data stored in the Follower ACPs. For each follower ACP, the lead ACP shares *MA_ExecutionPriorityListMT* with the package member. Upon receiving this message, follower ACPs update onboard MP with new EPL.

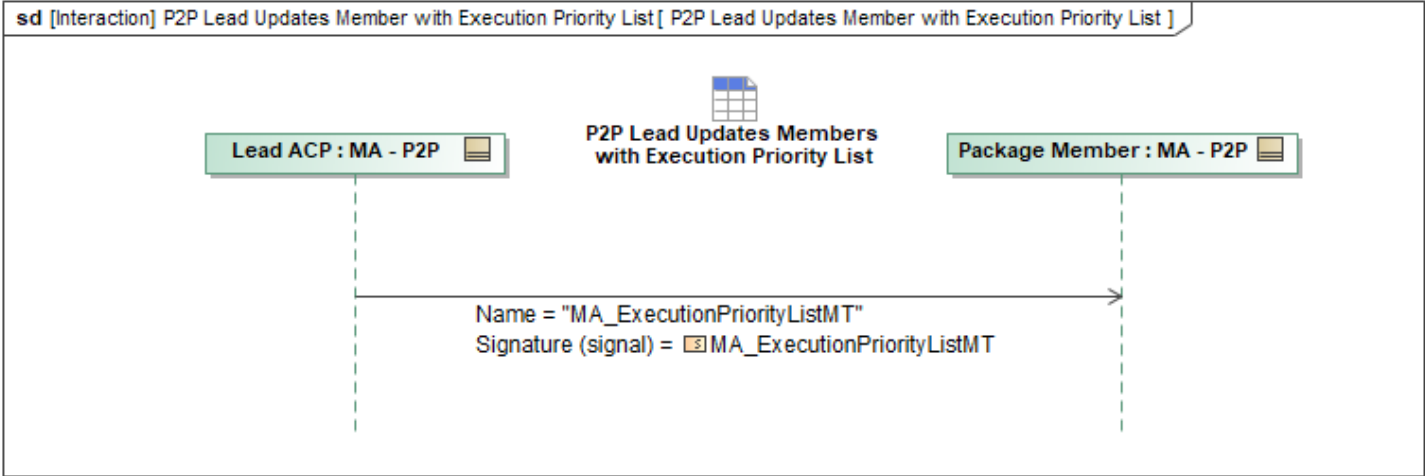


Figure 9-47: P2P Lead Updates Member with Execution Priority List

Table 9-46: P2P Lead Updates Member with Execution Priority List

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListMT:	ListItem: MA_ExecutionPriorityListItemType
MA_ExecutionPriorityListMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.8.5 P2P Lead Updates Member with Execution Priority List Activation Data

This interaction specifies how a lead ACP can update various data stored in the Follower ACPs. For each follower ACP, the lead ACP shares *MA_ExecutionPriorityListActivationDataMT* with the package member. Upon receiving this message, follower ACPs update onboard MP with new EPL activation data.

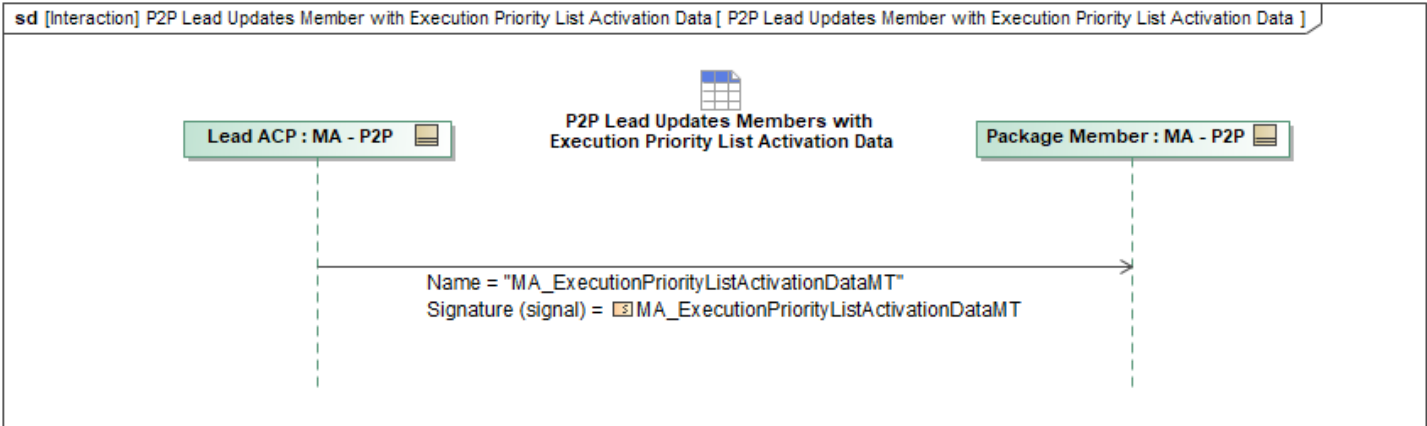


Figure 9-48: P2P Lead Updates Member with Execution Priority List Activation Data

Table 9-47: P2P Lead Updates Member with Execution Priority List Activation Data

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationDataMT: MA_ExecutionPriorityListActivationDataMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.9 P2P Member Updates Lead

9.2.10.9.1 P2P Member to Lead Update with Constraint Priority List Activation Status

This interaction allows the package members to update the lead regarding mission/task related data. The package member shares *MA_ConstraintPriorityListActivationStatusMT* with the lead ACP.

Members notify lead of new active CPL after entering a new geozone by sending the lead *MA_ConstraintPriorityListActivationStatusMT* containing the *ConstraintPriorityListActivationDataID* and *ActiveConstraintPriorityListID*. The *AppliesToID* shall be set to the *System ID* of the follower ACP.

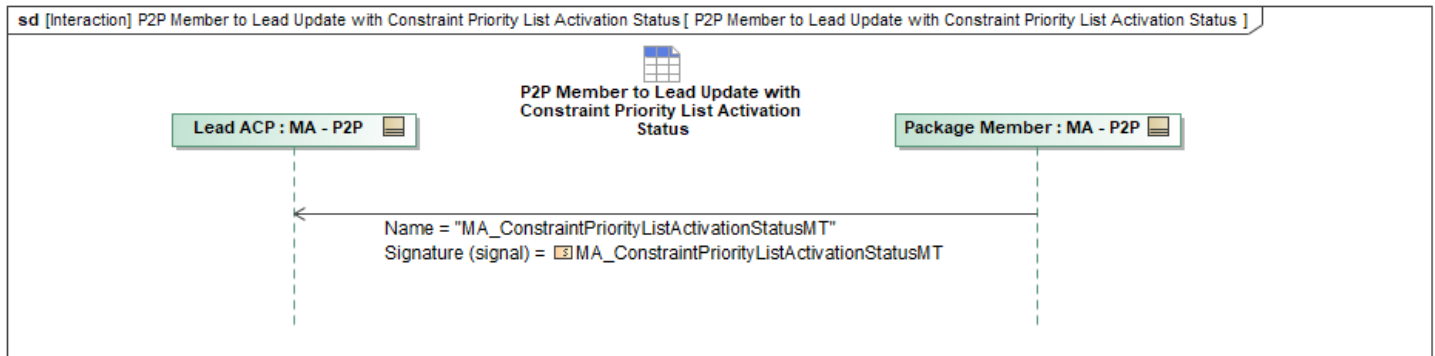


Figure 9-49: P2P Member to Lead Update with Constraint Priority List Activation Status

Table 9-48: P2P Member to Lead Update with Constraint Priority List Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationStatusMT: MA_ConstraintPriorityListActivationStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.9.2 P2P Member to Lead Update with Execution Priority List Activation Status

This interaction allows the package members to update the lead regarding mission/task related data. The package member shares *MA_ExecutionPriorityListActivationStatusMT* with the lead ACP.

Followers notify lead of a new active EPL after entering a geozone by sending the lead *MA_ExecutionPriorityListActivationStatusMT* containing the *ExecutionPriorityListActivationDataID* and *ActiveExecutionPriorityListID*. The *AppliesToID* shall be set to the *System ID* of the follower ACP.

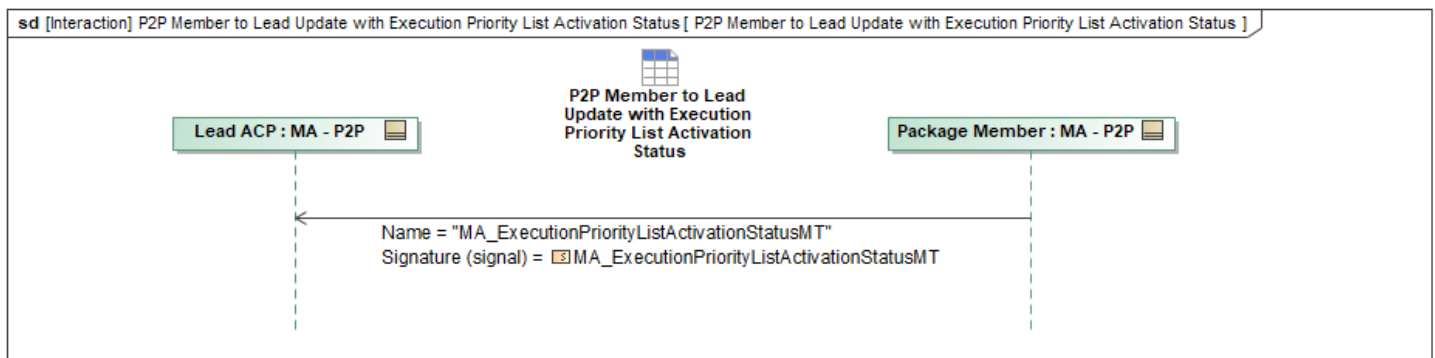


Figure 9-50: P2P Member to Lead Update with Execution Priority List Activation Status

Table 9-49: P2P Member to Lead Update with Execution Priority List Activation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationStatusMT: MA_ExecutionPriorityListActivationStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.9.3 P2P Member to Lead Update with System Notification

This interaction allows the package members to update the lead regarding mission/task related data. The package member shares *MA_SystemNotificationMT* with the lead ACP.

To update the lead of a member's synchronization completion status, the package members send *MA_SystemNotificationMT* message with the *SystemSubjectID* field containing the *System ID* of the sending package member. The *AppliesTo.SystemID* should be the *SystemID* of the package leader. The *AssociatedMessage.MessageType* should be set to *MISSION_PLAN* and the *AssociatedMessage.AssociatedID* should be set to the UUID of the Mission Plan that completed synchronization.

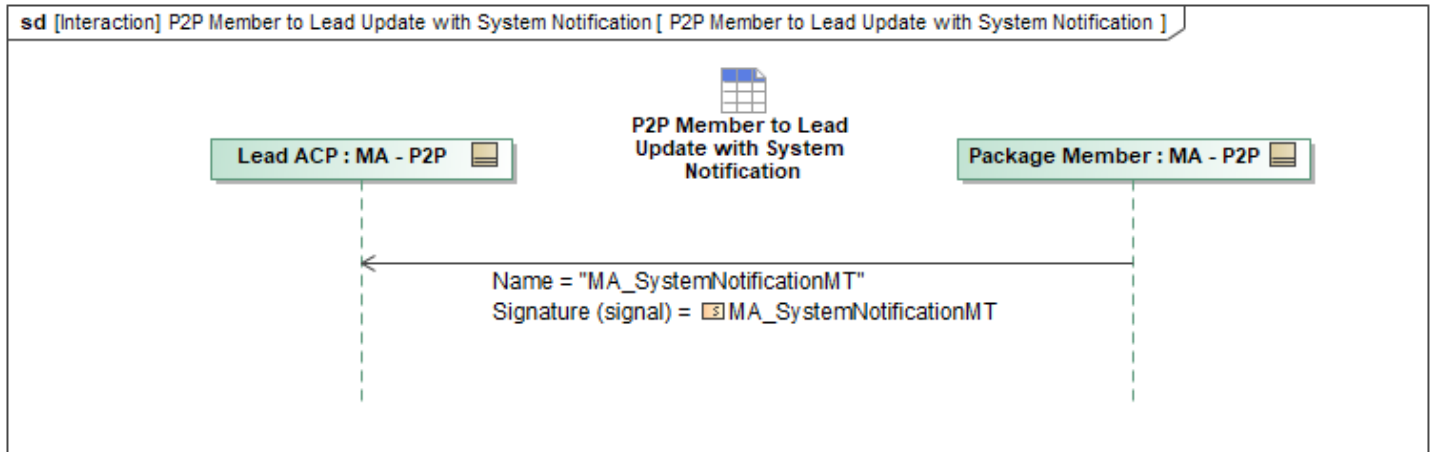


Figure 9-51: P2P Member to Lead Update with System Notification

Table 9-50: P2P Member to Lead Update with System Notification

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	AppliesTo: SubjectType AssociatedMessage: AssociatedMessageType AssociatedID: ID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10 P2P Query Data

9.2.10.10.1 P2P Query Activity Plan

MA_ActivityPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to

COMPLETED, and this message contains the requested *MA_ActivityPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_ActivityPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_ActivityPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_ActivityPlanMT*.

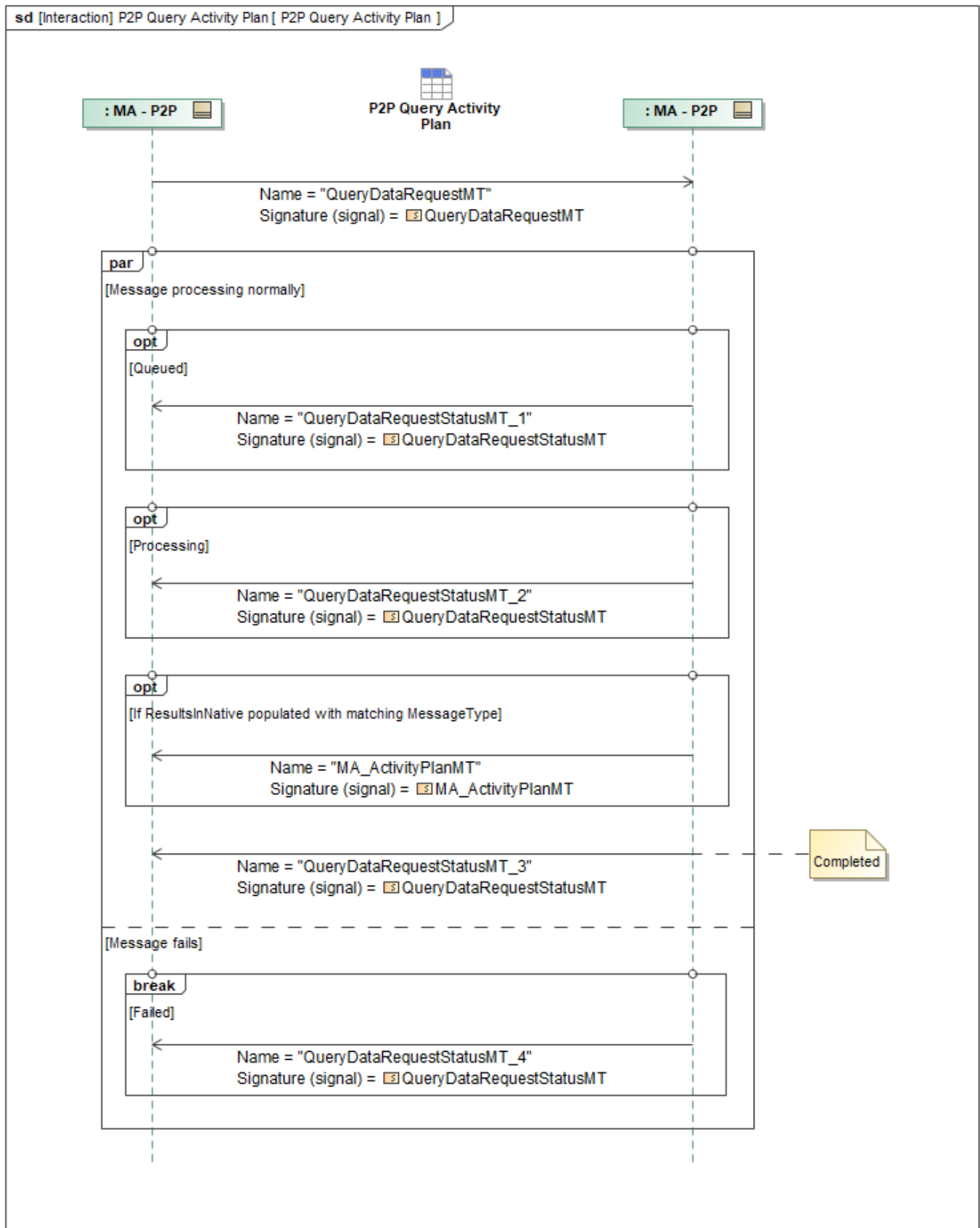


Figure 9-52: P2P Query Activity Plan

Table 9-51: P2P Query Activity Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.2 P2P Query Airfield Report

AirfieldReportMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *AirfieldReportMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *AirfieldReportMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *AirfieldReportMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *AirfieldReportMT*.

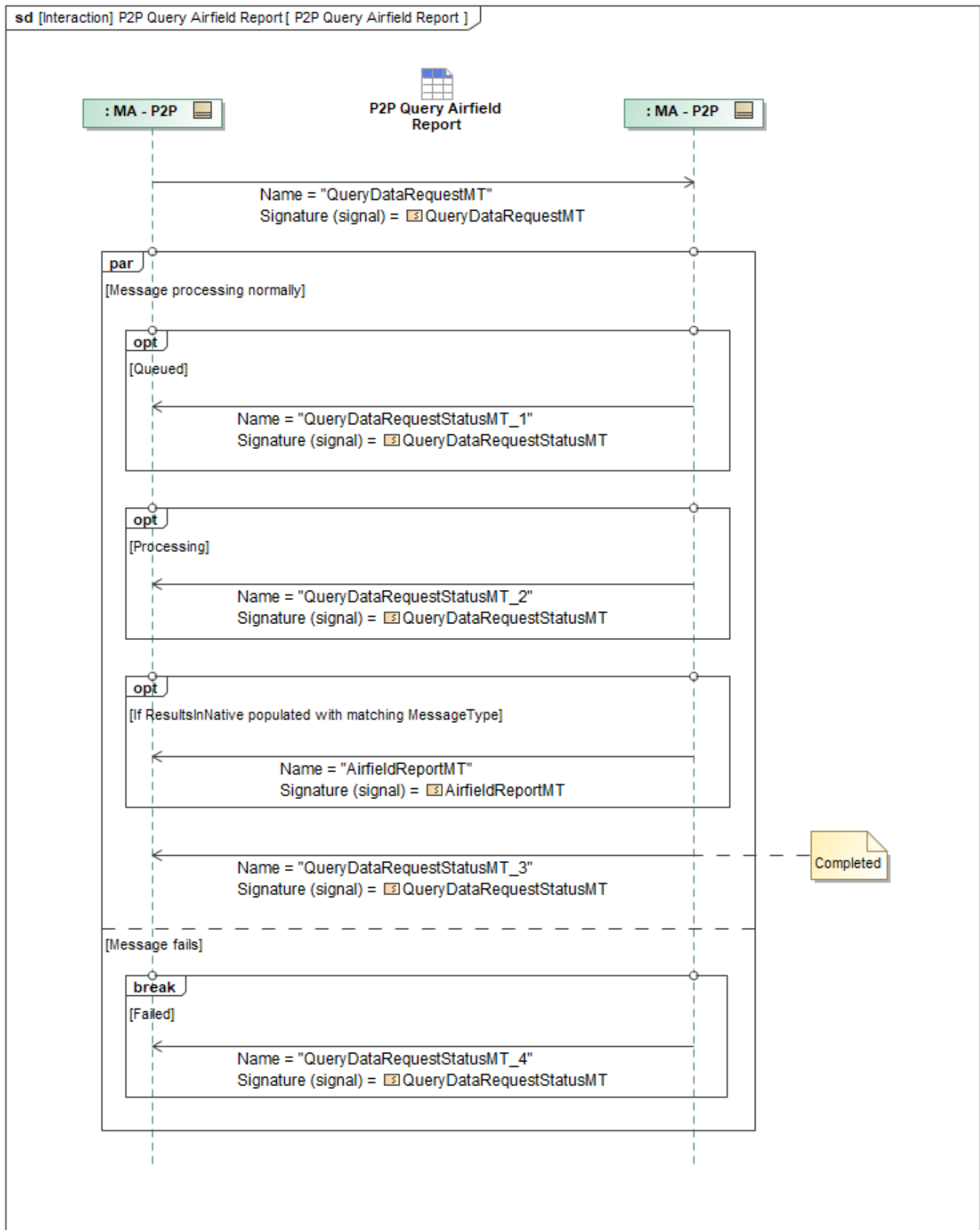


Figure 9-53: P2P Query Airfield Report

Table 9-52: P2P Query Airfield Report

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.3 P2P Query Mission Plan

MA_MissionPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_MissionPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_MissionPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_MissionPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_MissionPlanMT*.

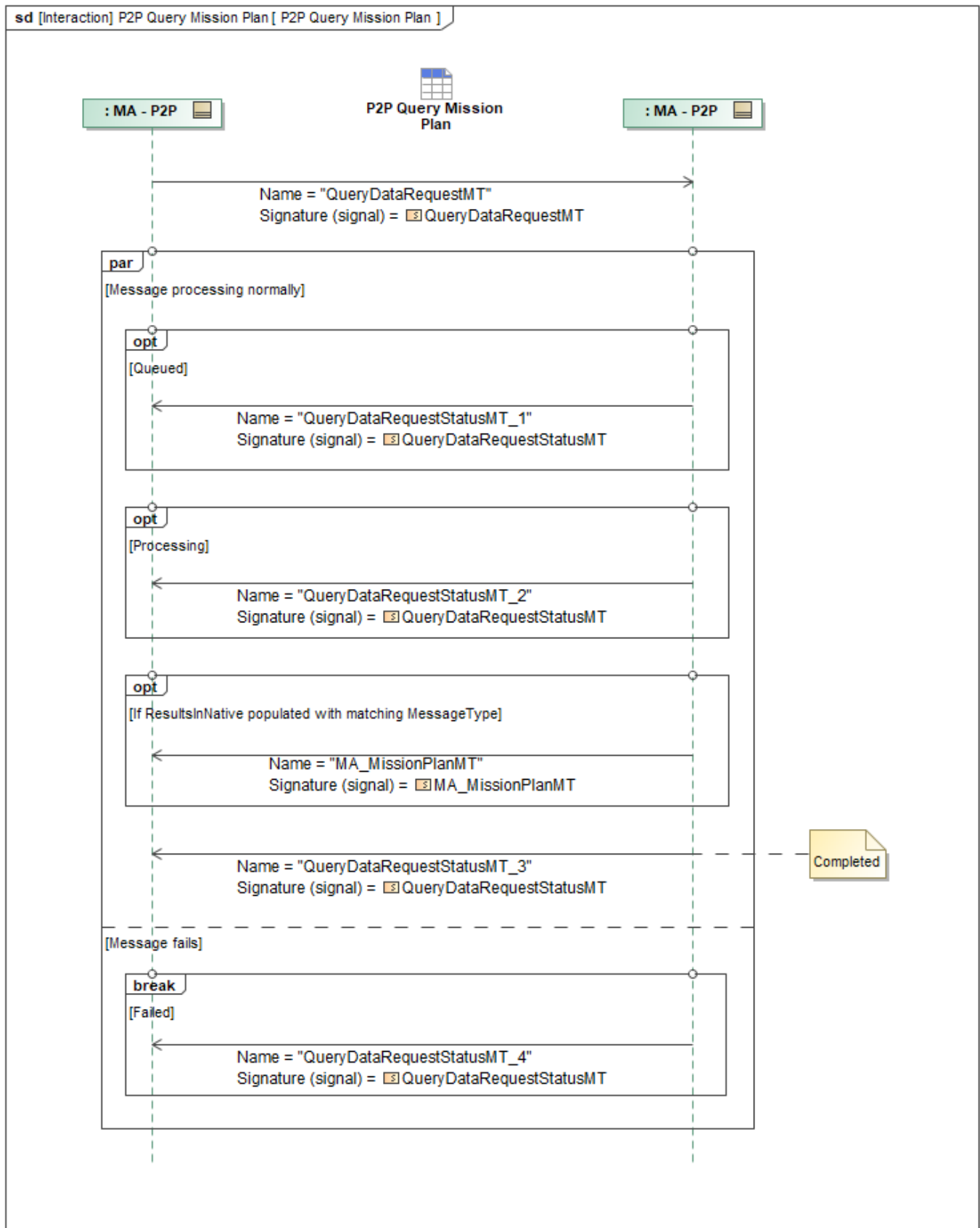


Figure 9-54: P2P Query Mission Plan

Table 9-53: P2P Query Mission Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.4 P2P Query OpZone

MA_OpZoneMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_OpZoneMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_OpZoneMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_OpZoneMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_OpZoneMT*.

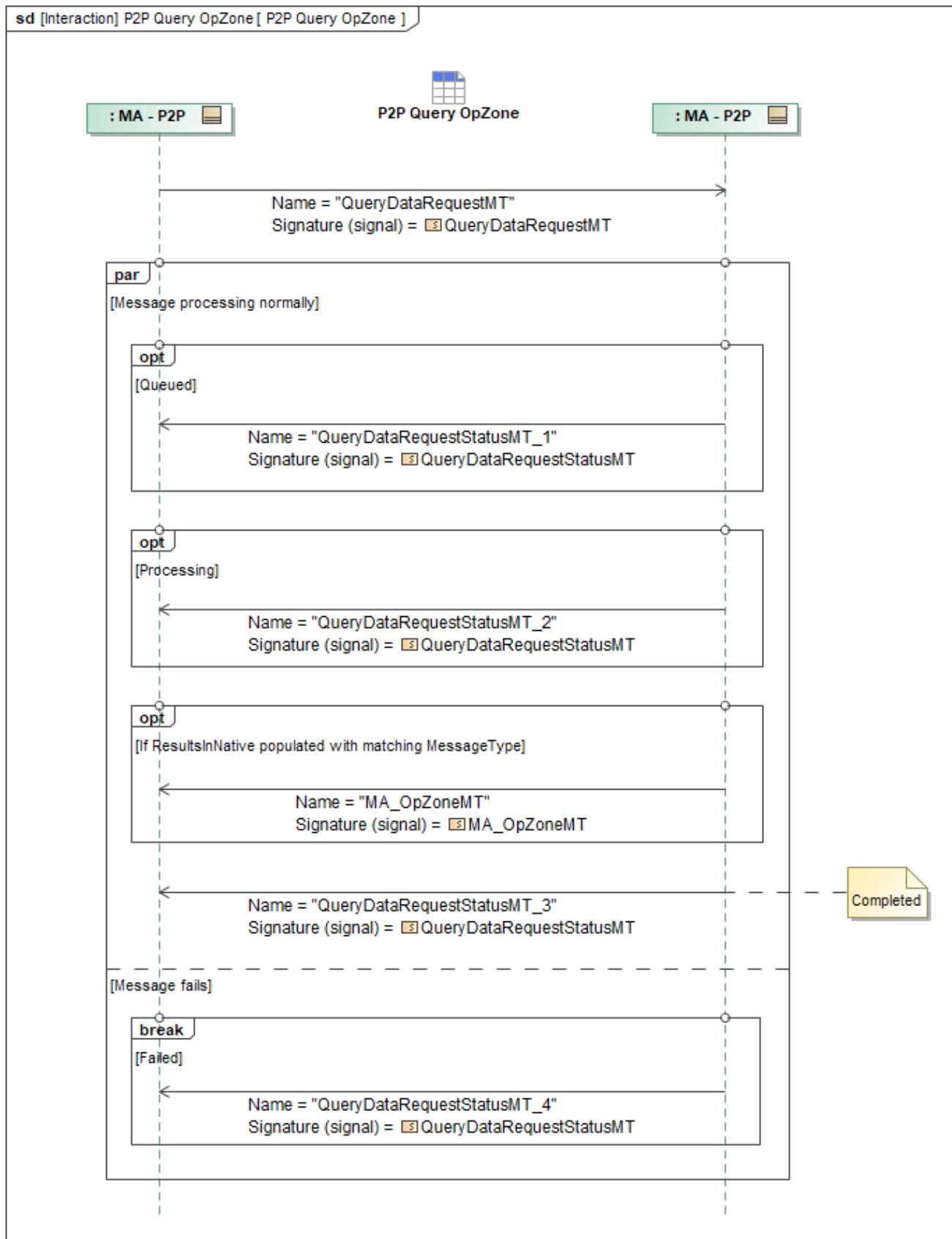


Figure 9-55: P2P Query OpZone

Table 9-54: P2P Query OpZone

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpZoneMT: MA_OpZoneMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.5 P2P Query Response Plan

ResponsePlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *ResponsePlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *ResponsePlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *ResponsePlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *ResponsePlanMT*.

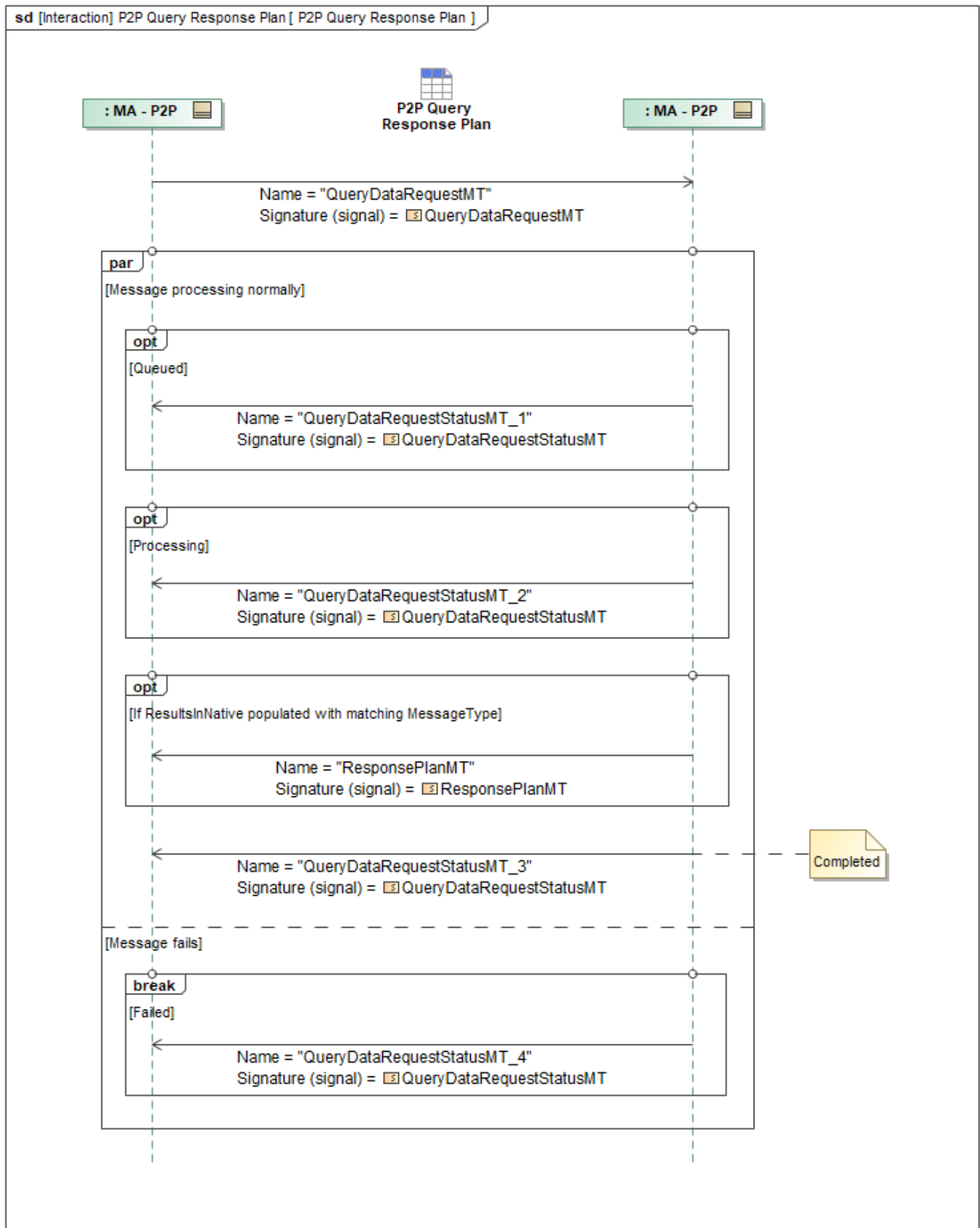


Figure 9-56: P2P Query Response Plan

Table 9-55: P2P Query Response Plan

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
ResponsePlanMT: ResponsePlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.6 P2P Query Route Activity Plan

MA_RouteActivityPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_RouteActivityPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_RouteActivityPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_RouteActivityPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_RouteActivityPlanMT*.

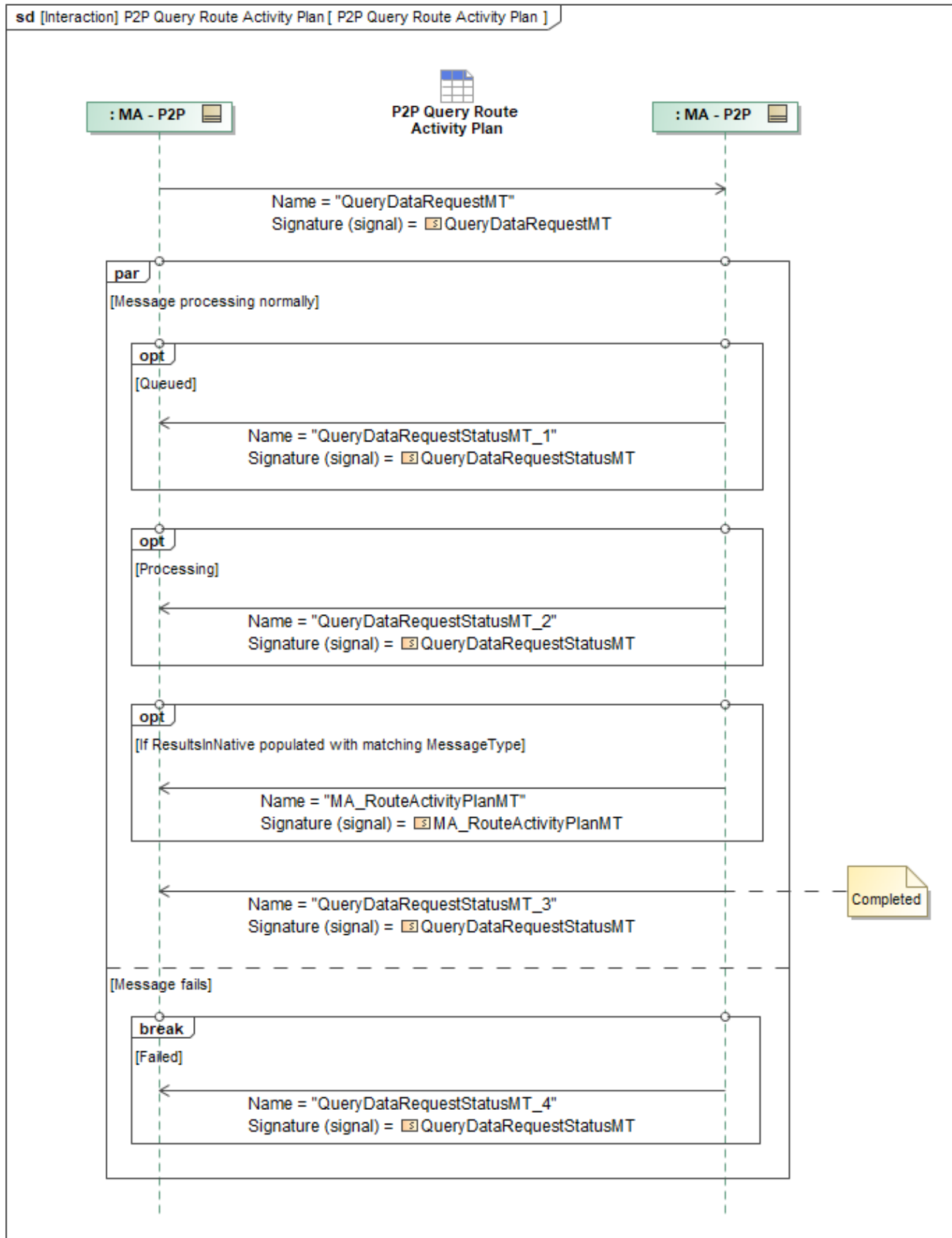


Figure 9-57: P2P Query Route Activity Plan

Table 9-56: P2P Query Route Activity Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteActivityPlanMT: MA_RouteActivityPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.7 P2P Query Route Plan

MA_RoutePlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_RoutePlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_RoutePlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_RoutePlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_RoutePlanMT*.

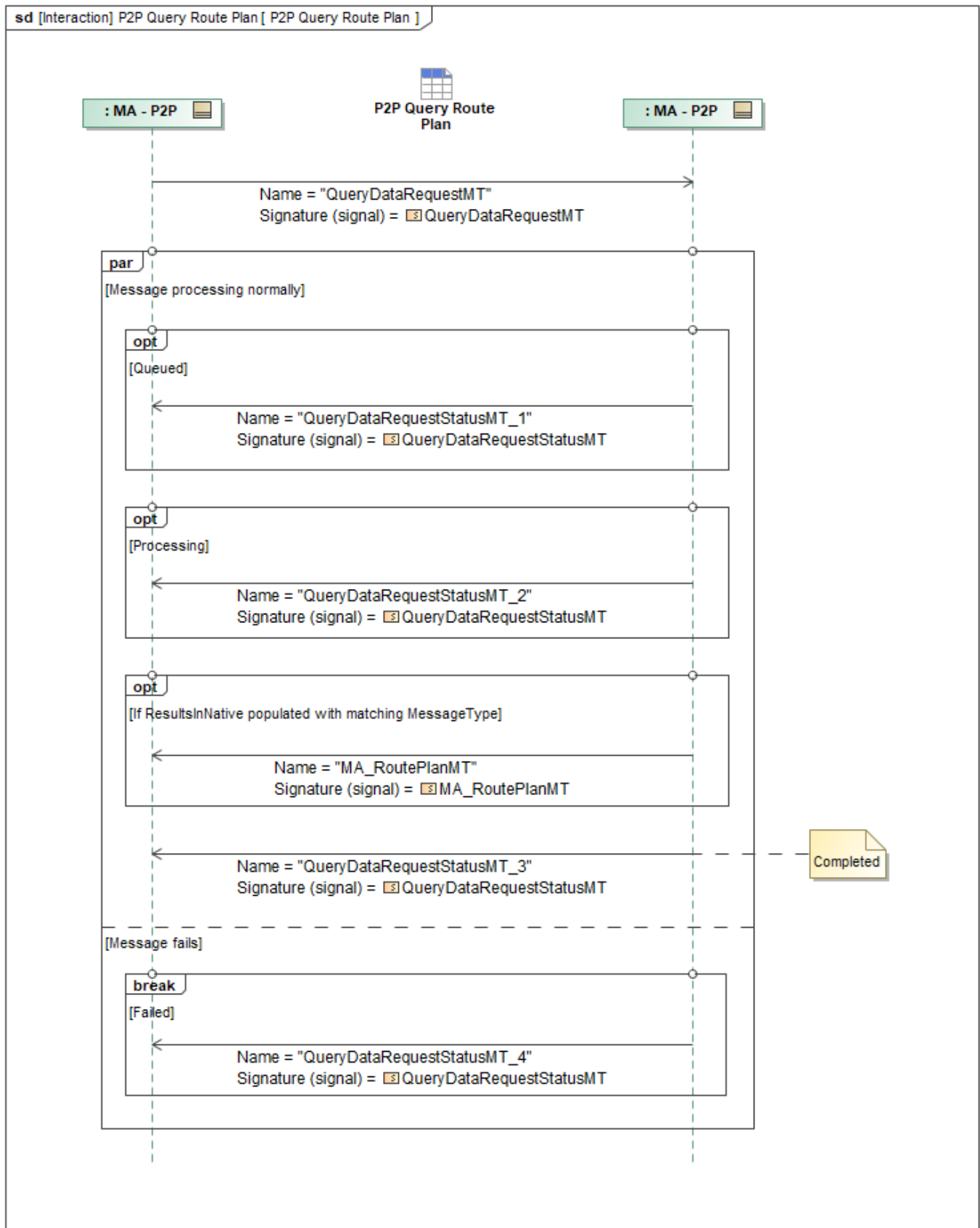


Figure 9-58: P2P Query Route Plan

Table 9-57: P2P Query Route Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.8 P2P Query Task Plan

MA_TaskPlanMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *MA_TaskPlanMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *MA_TaskPlanMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *MA_TaskPlanMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *MA_TaskPlanMT*.

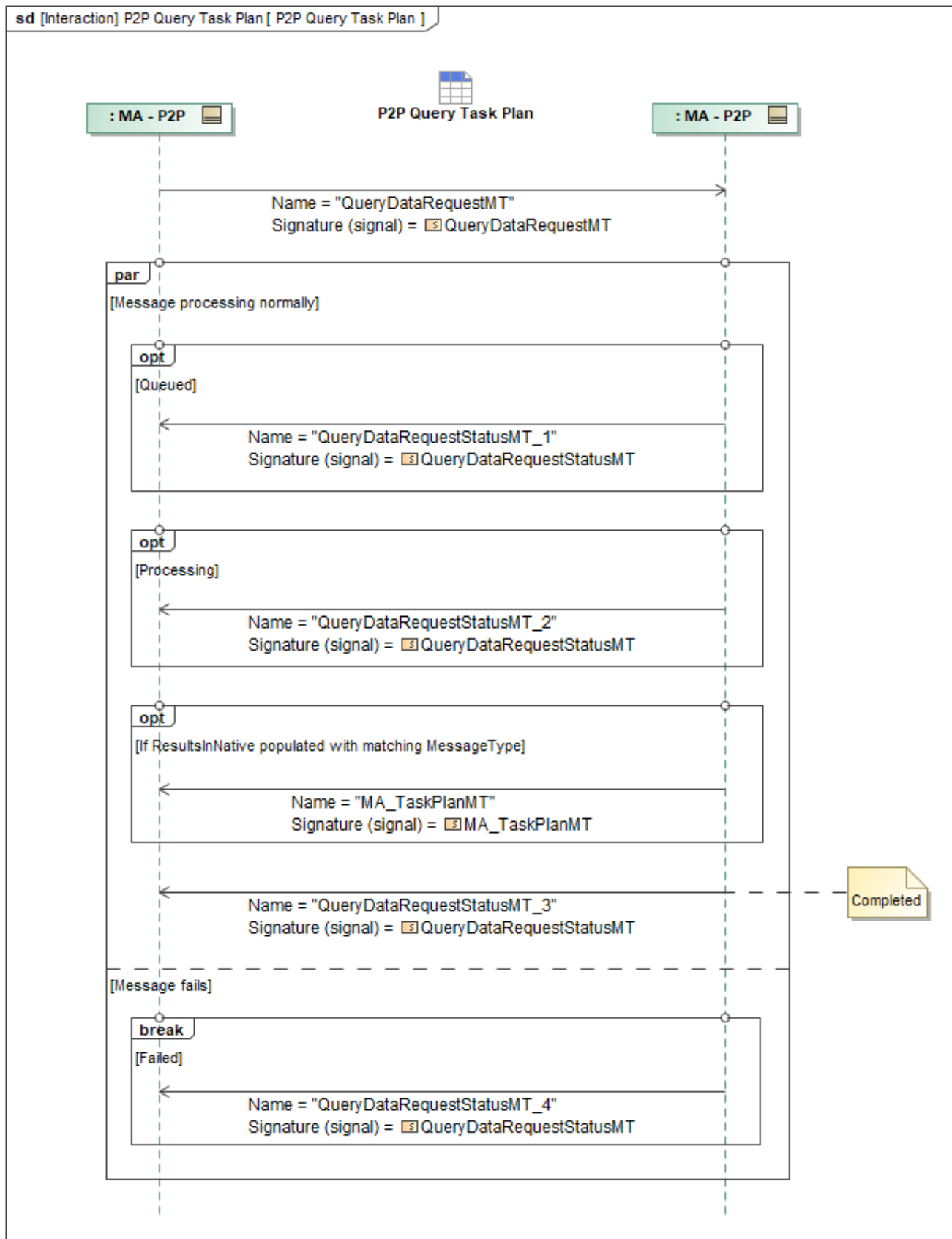


Figure 9-59: P2P Query Task Plan

Table 9-58: P2P Query Task Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskPlanMT: MA_TaskPlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.10.9 P2P Query Weather

WeatherMT is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may respond with intermediate status messages, beginning with *QueryDataRequestStatusMT_1*, which has *RequestProcessingStateEnum* set to *QUEUED* to indicate that the request has been queued for processing. The service also may send *QueryDataRequestStatusMT_2* with *RequestProcessingStateEnum* set to *PROCESSING*, indicating that the request is actively being processed.

When processing is complete and the requested data is returned within the status message itself, the service sends *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED*, and this message contains the requested *WeatherMT* payload. When the original *QueryDataRequestMT* message's optional field *ResultsInNative* is populated with the matching *WeatherMT* message type, the underlying data management services instead publish the requested data in its original or raw format as a separate *WeatherMT* message and still send *QueryDataRequestStatusMT_3* with *RequestProcessingStateEnum* set to *COMPLETED* to indicate that requested data has been published.

If the request fails, the service sends *QueryDataRequestStatusMT_4* with *RequestProcessingStateEnum* set to *FAILED*, indicating that processing has terminated without successfully providing the requested *WeatherMT*.

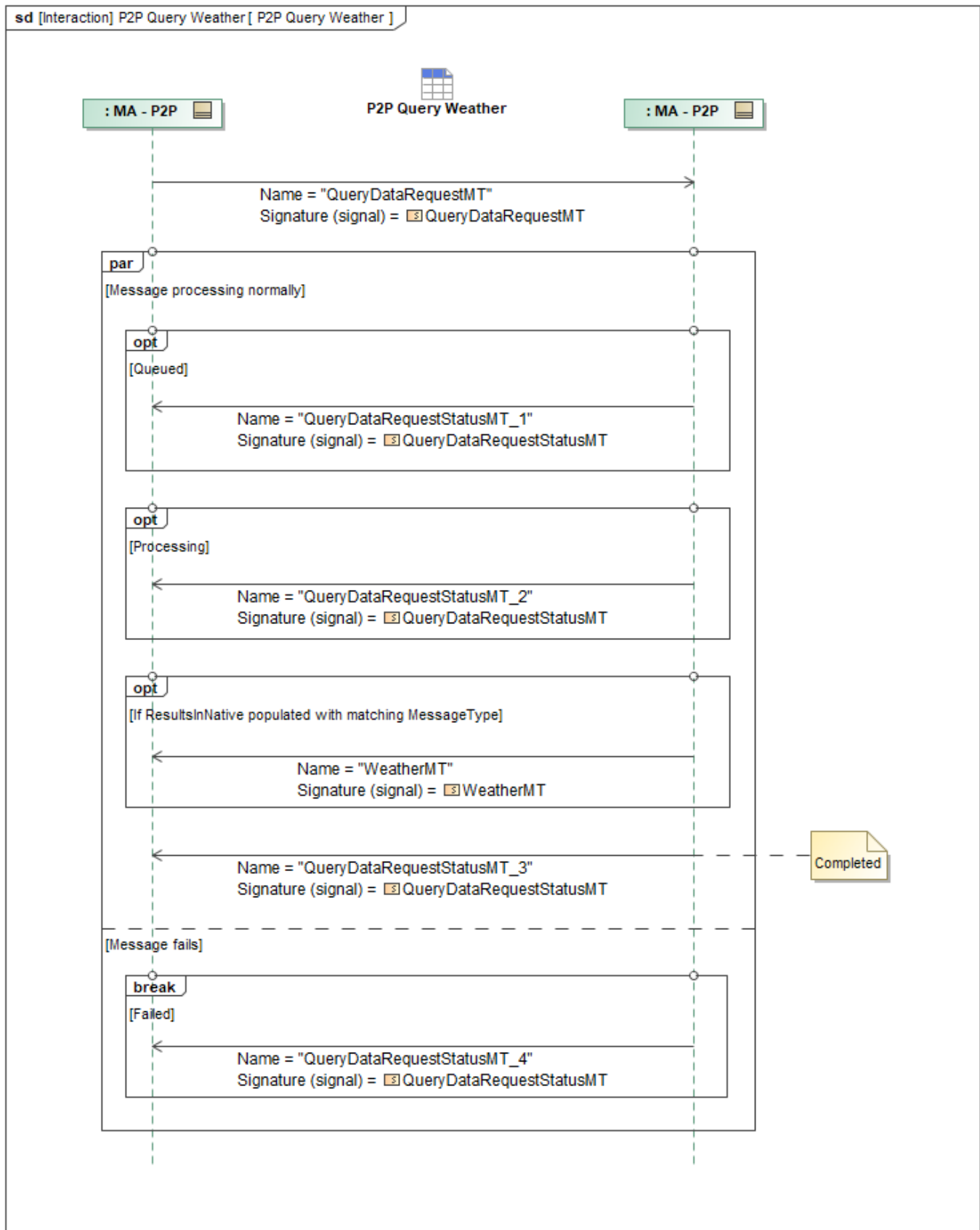


Figure 9-60: P2P Query Weather

Table 9-59: P2P Query Weather

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
WeatherMT: WeatherMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11 P2P Send Data

9.2.10.11.1 Send Action Plan to Peer

This diagram contains the message, *MA_ActionPlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

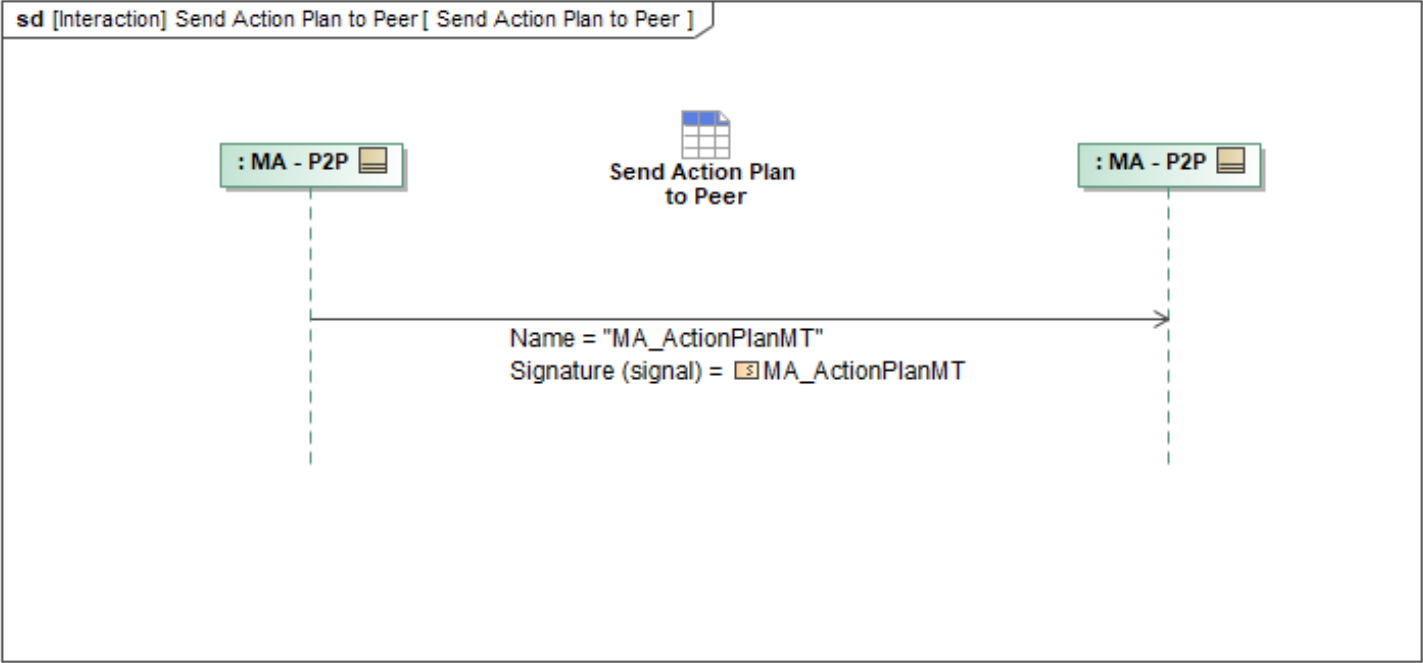


Figure 9-61: Send Action Plan to Peer

Table 9-60: Send Action Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionPlanMT: MA_ActionPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.2 Send Action to Peer

This diagram contains the message, *MA_ActionMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

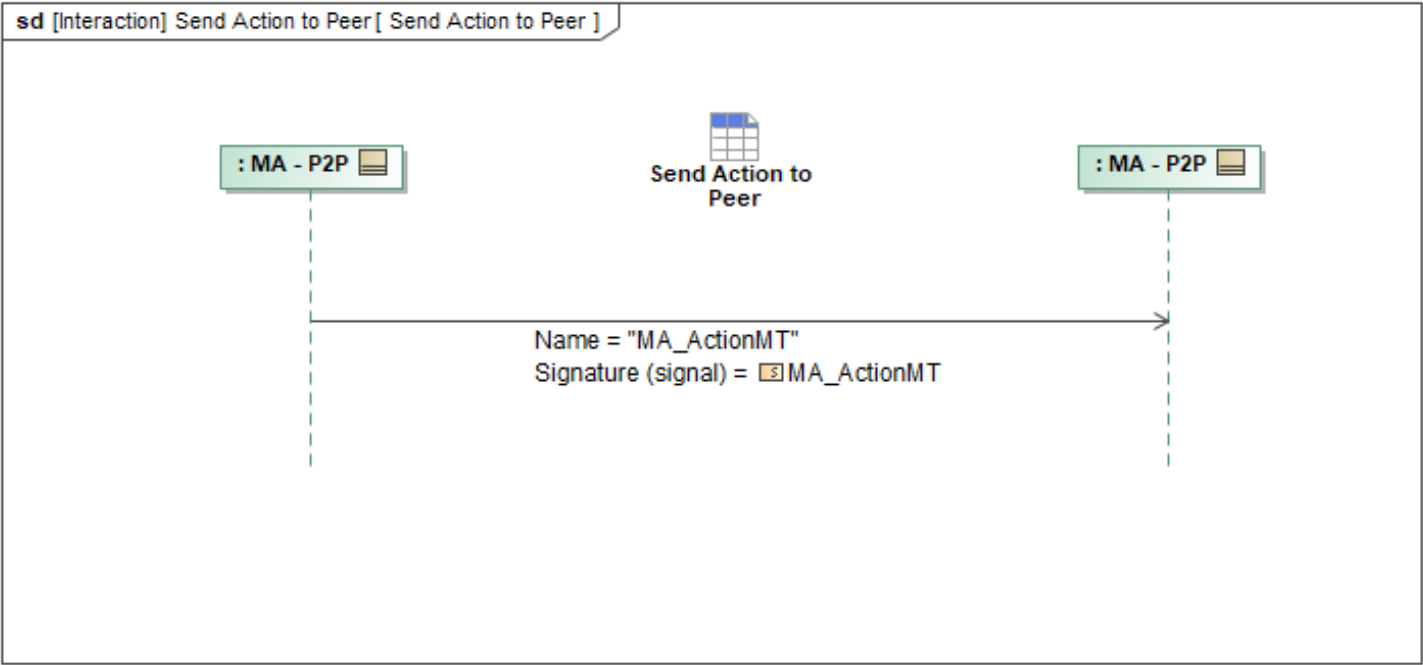


Figure 9-62: Send Action to Peer

Table 9-61: Send Action to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionMT: MA_ActionMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.3 Send Activity Plan to Peer

This diagram contains the message, *MA_ActivityPlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

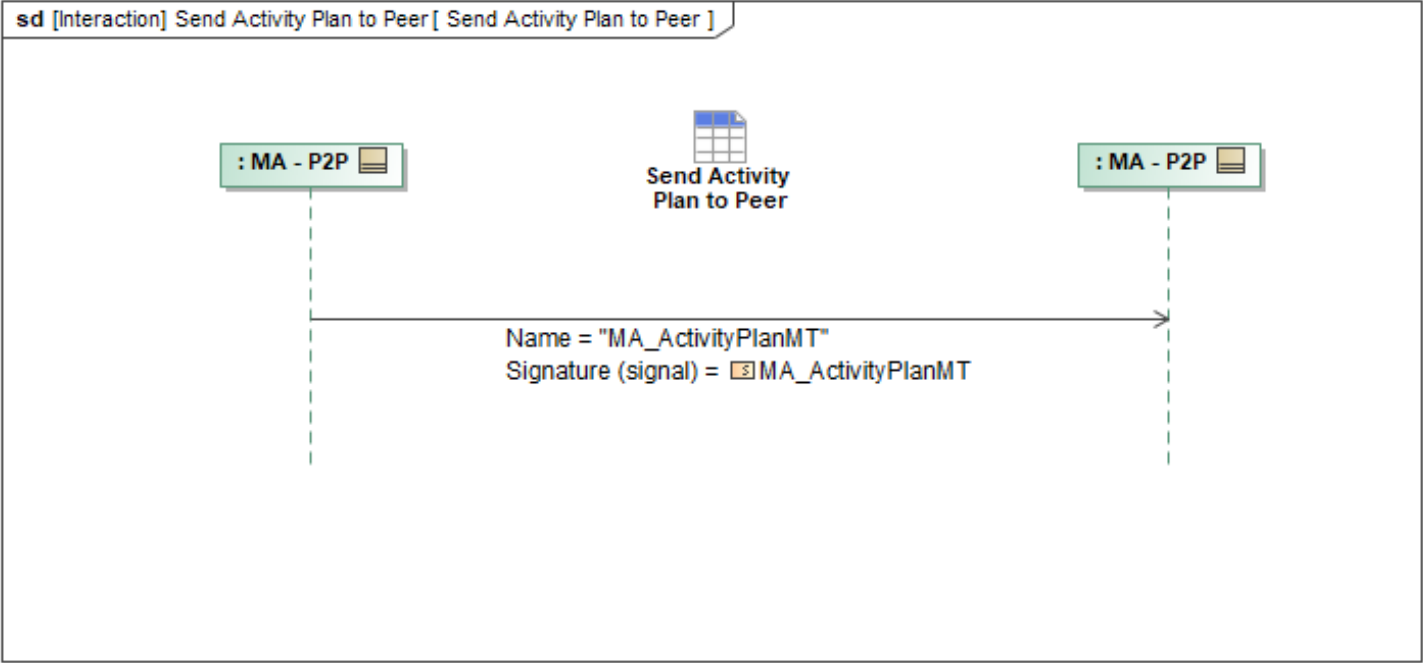


Figure 9-63: Send Activity Plan to Peer

Table 9-62: Send Activity Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActivityPlanMT: MA_ActivityPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.4 Send Approval Authority to Peer

This diagram contains the message, *MA_ApprovalAuthorityMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

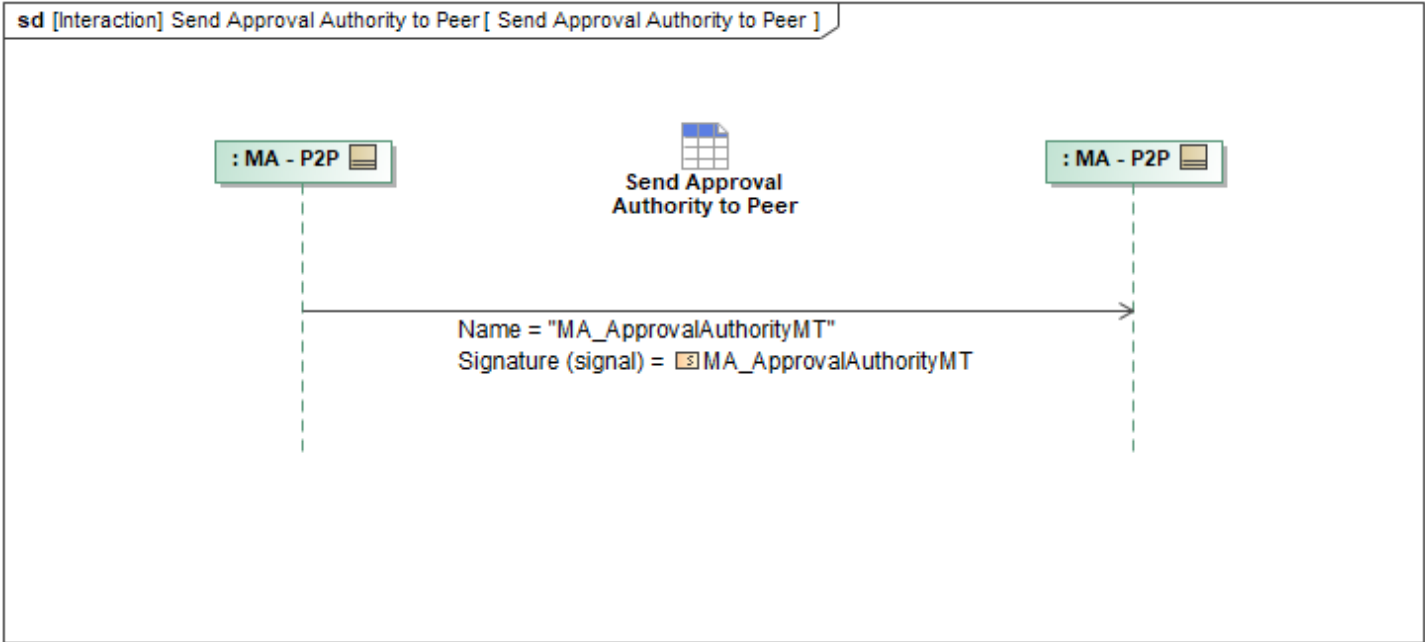


Figure 9-64: Send Approval Authority to Peer

Table 9-63: Send Approval Authority to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalAuthorityMT: MA_ApprovalAuthorityMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.5 Send Approval Policy to Peer

This diagram contains the message, *MA_ApprovalPolicyMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.



Figure 9-65: Send Approval Policy to Peer

Table 9-64: Send Approval Policy to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalPolicyMT: MA_ApprovalPolicyMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.6 Send Capability Set to Peer

This diagram contains the message, *MA_CapabilitySetMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

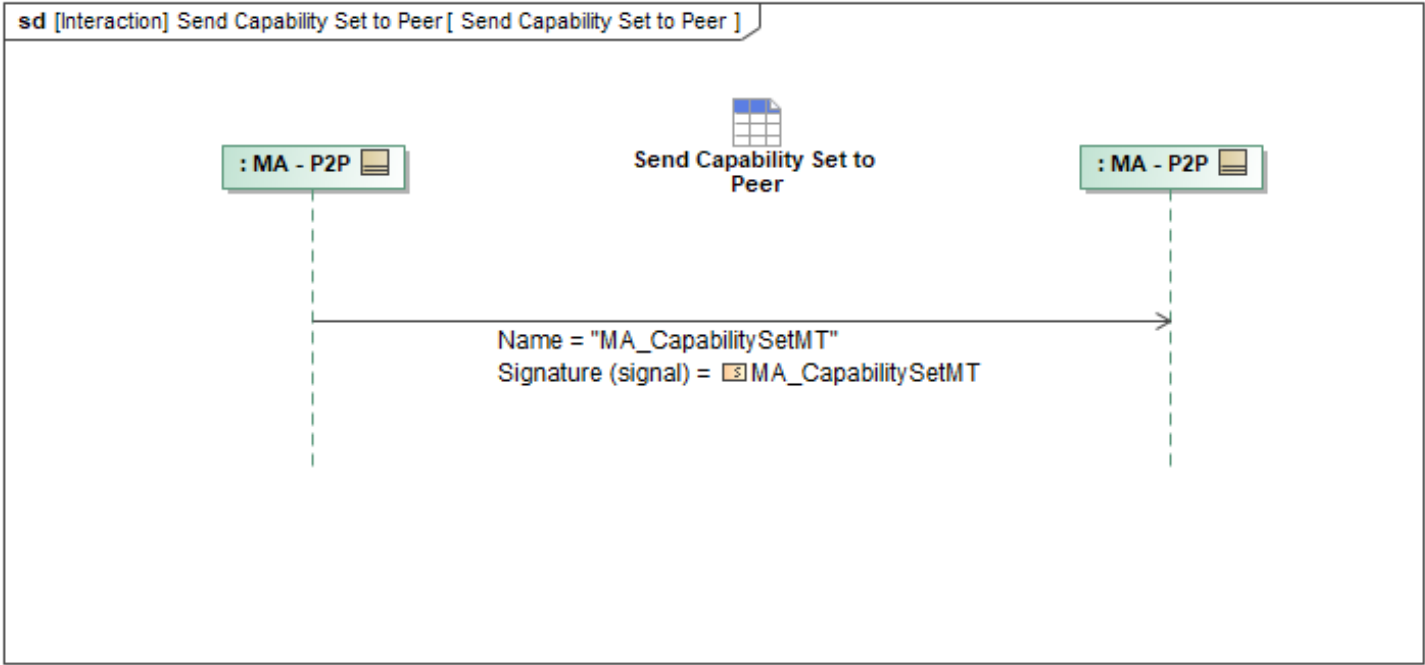


Figure 9-66: Send Capability Set to Peer

Table 9-65: Send Capability Set to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_CapabilitySetMT: MA_CapabilitySetMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.7 Send Component Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *ComponentStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *ComponentStatusMT* is sent. The *ComponentStatusMT* must have the *ComponentID* set to the *SystemID* and the *ComponentState* set to the Vehicle Autonomy's state.

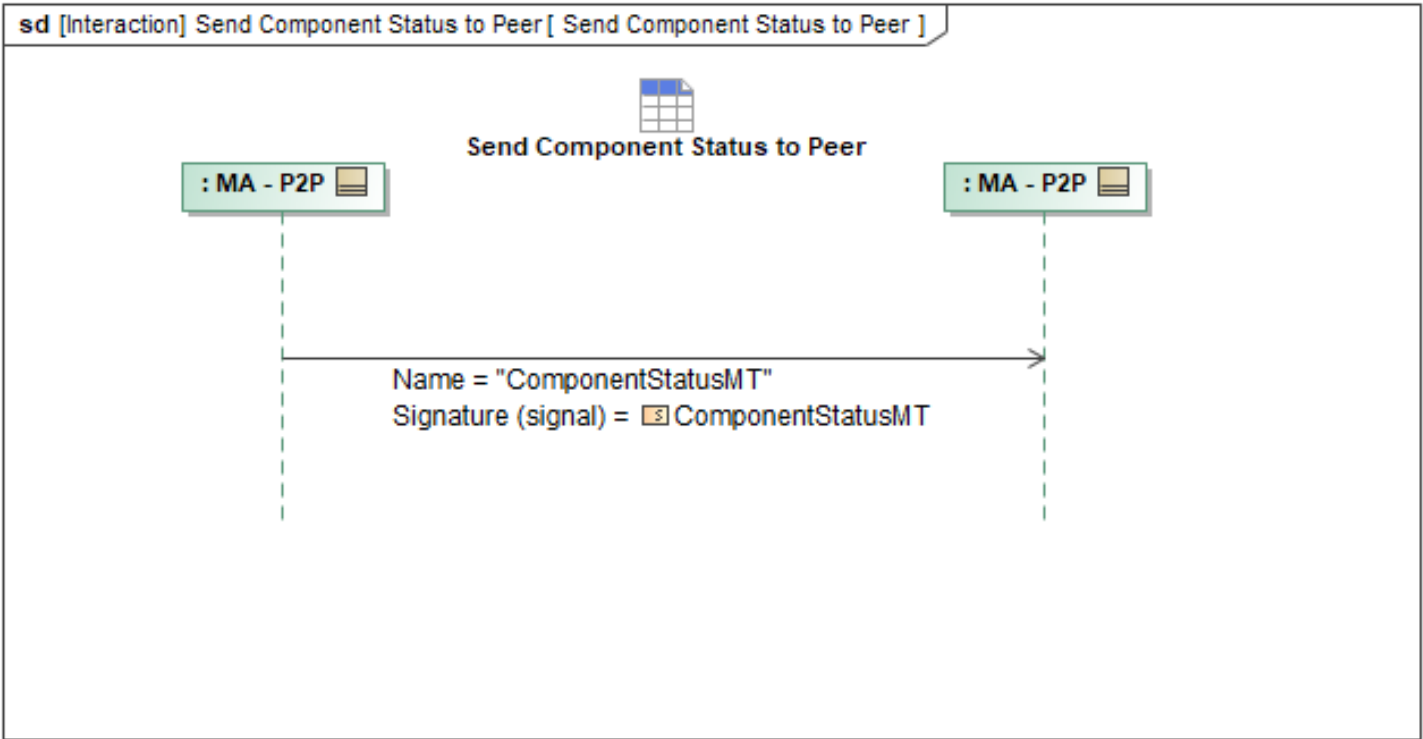


Figure 9-67: Send Component Status to Peer

Table 9-66: Send Component Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.8 Send Entity to Peer

This diagram contains the message, *EntityMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

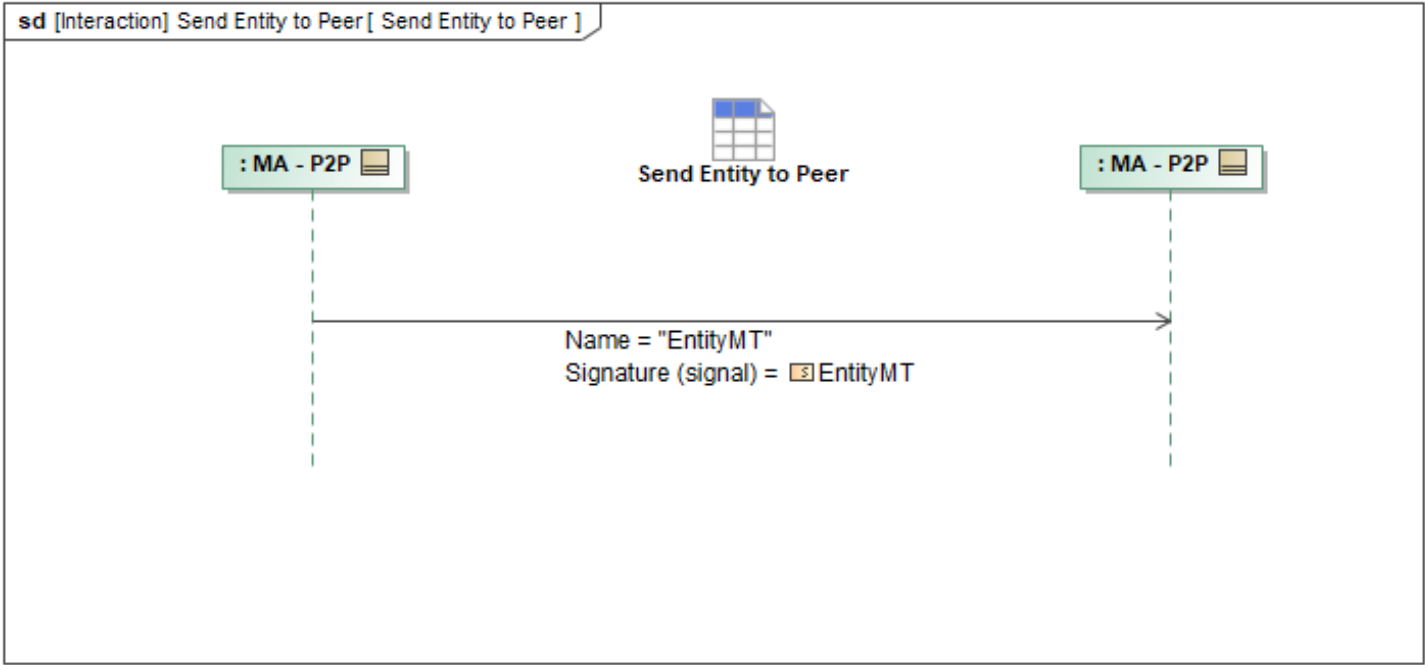


Figure 9-68: Send Entity to Peer

Table 9-67: Send Entity to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
EntityMT: EntityMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.9 Send Flight Activity to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_FlightActivityMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *MA_FlightActivityMT* message is sent. The *MA_FlightActivityMT* must contain the *CommandID* of the last flight command (if any) in the *CapabilityCommandID* and any commanded flight values in the *VehicleCommandState*. If the vehicle is flying waypoints, they must be listed in the *ActualEndPoint*.

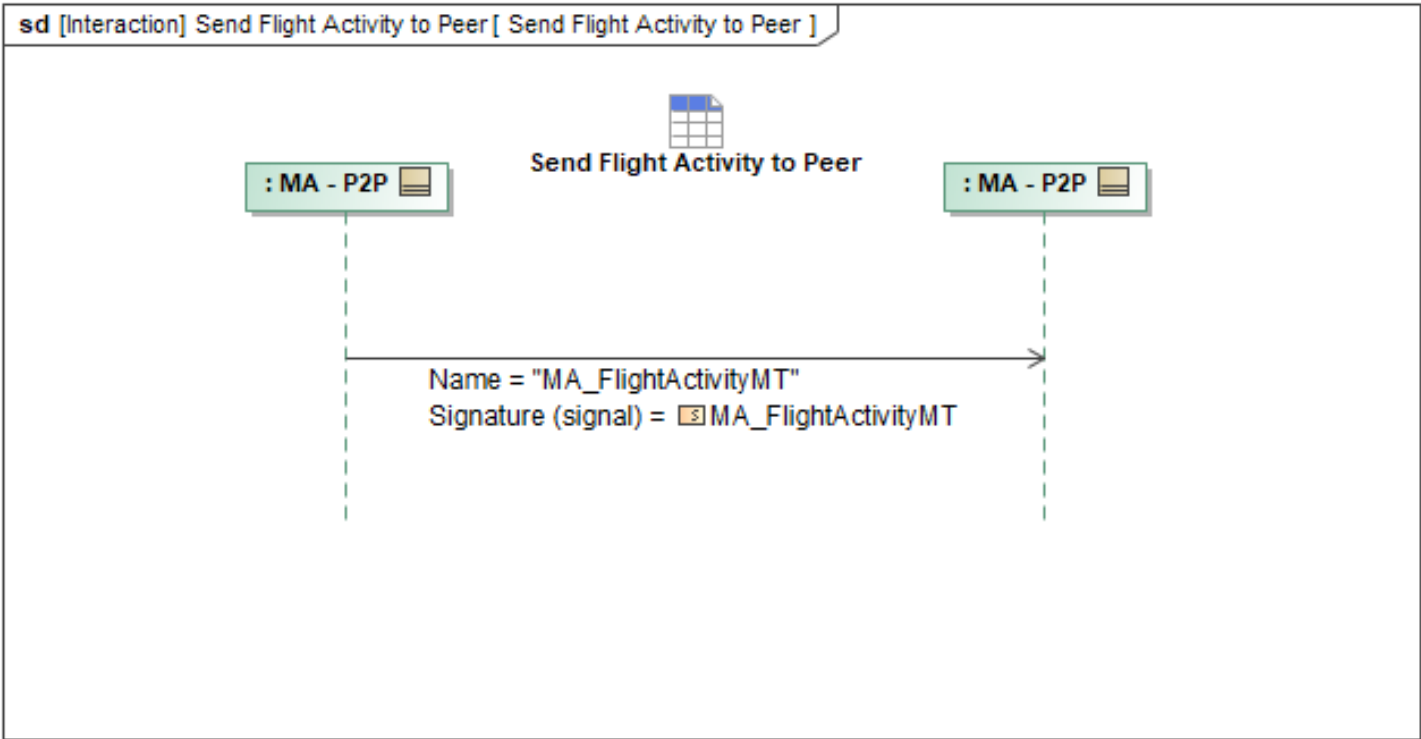


Figure 9-69: Send Flight Activity to Peer

Table 9-68: Send Flight Activity to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.10 Send Flight Capability Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_FlightCapabilityStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *MA_FlightCapabilityStatusMT* message is sent.

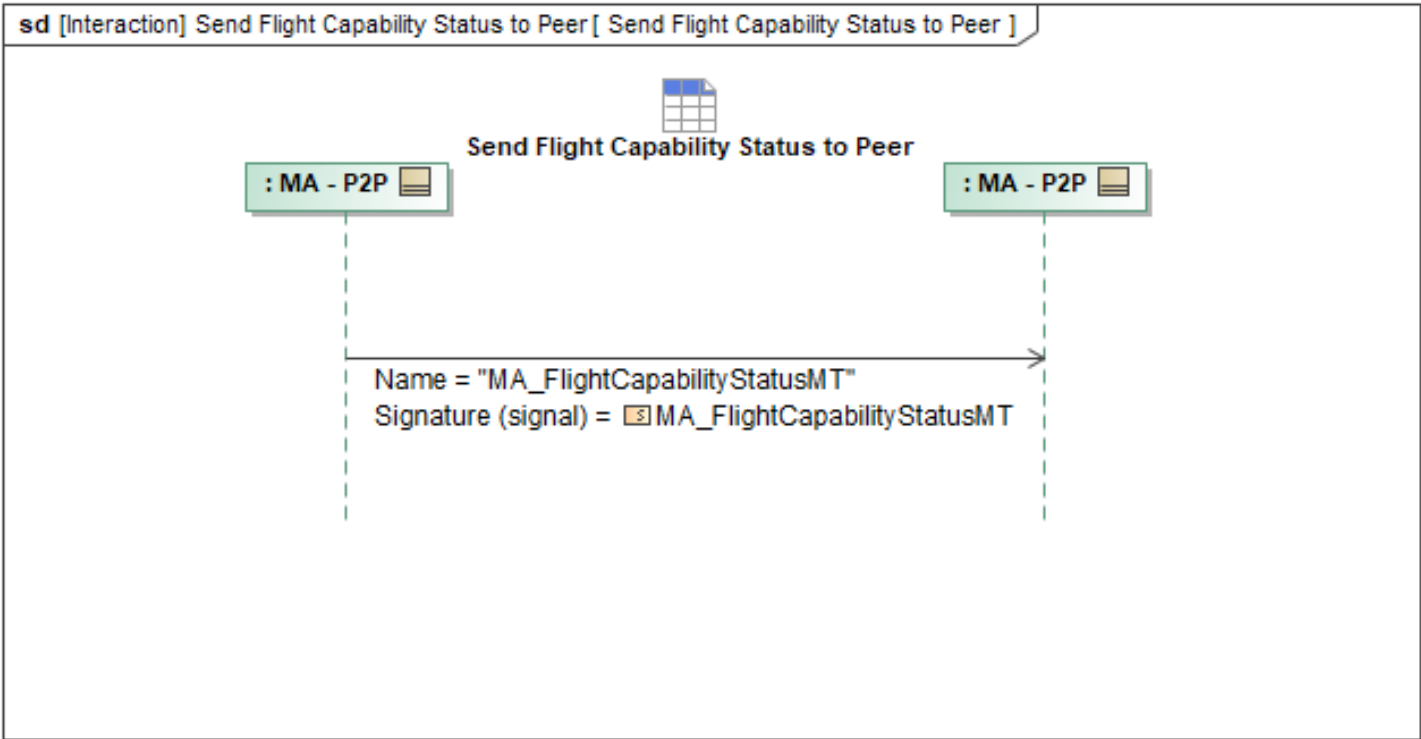


Figure 9-70: Send Flight Capability Status to Peer

Table 9-69: Send Flight Capability Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	Availability: CapabilityAvailabilityEnum CapabilityID: CapabilityID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.11 Send Flight Capability to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_FlightCapabilityMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *MA_FlightCapabilityMT* message is sent. The *MA_FlightCapabilityMT* must contain the applicable performance limits. The *MA_FlightCapabilityStatusMT* must contain the *CapabilityID* of the flight capability and the *Availability* set the availability of Flight.

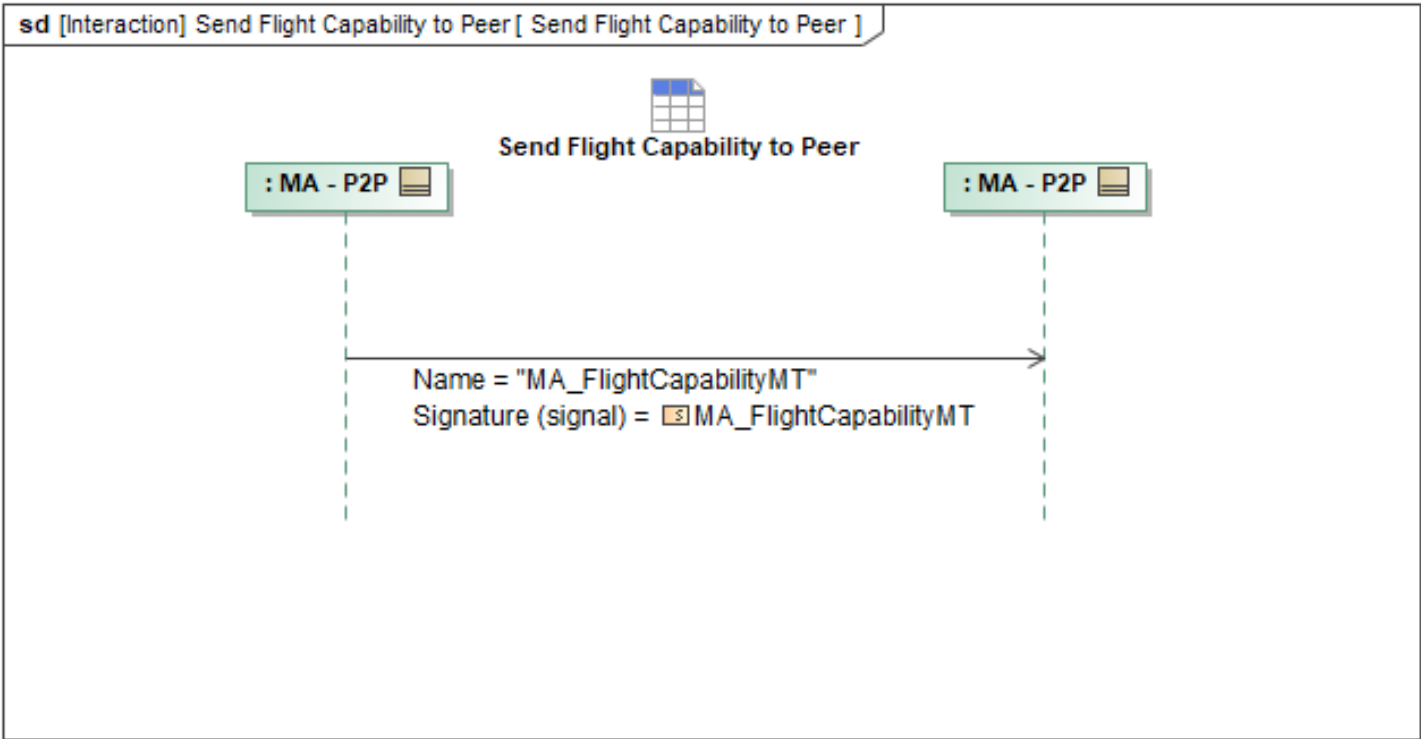


Figure 9-71: Send Flight Capability to Peer

Table 9-70: Send Flight Capability to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.12 Send Mission Plan Execution Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_MissionPlanExecutionStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *MA_MissionPlanExecutionStatusMT* message is sent. The *MissionPlanExecutionStatusMT* must have the *SystemID* of the vehicle and a *PlanExecutionStatus* for all active MissionPlans.

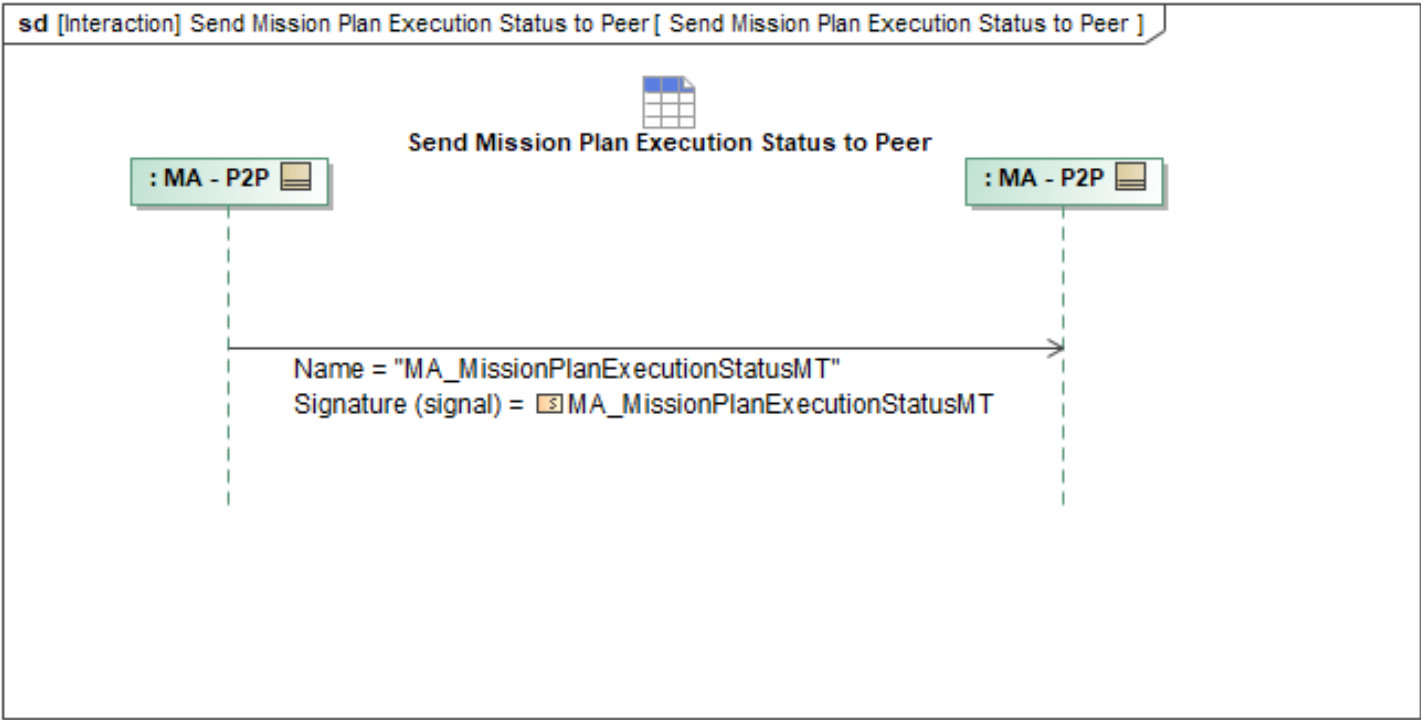


Figure 9-72: Send Mission Plan Execution Status to Peer

Table 9-71: Send Mission Plan Execution Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.13 Send Mission Plan to Peer

This diagram contains the message, *MA_MissionPlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

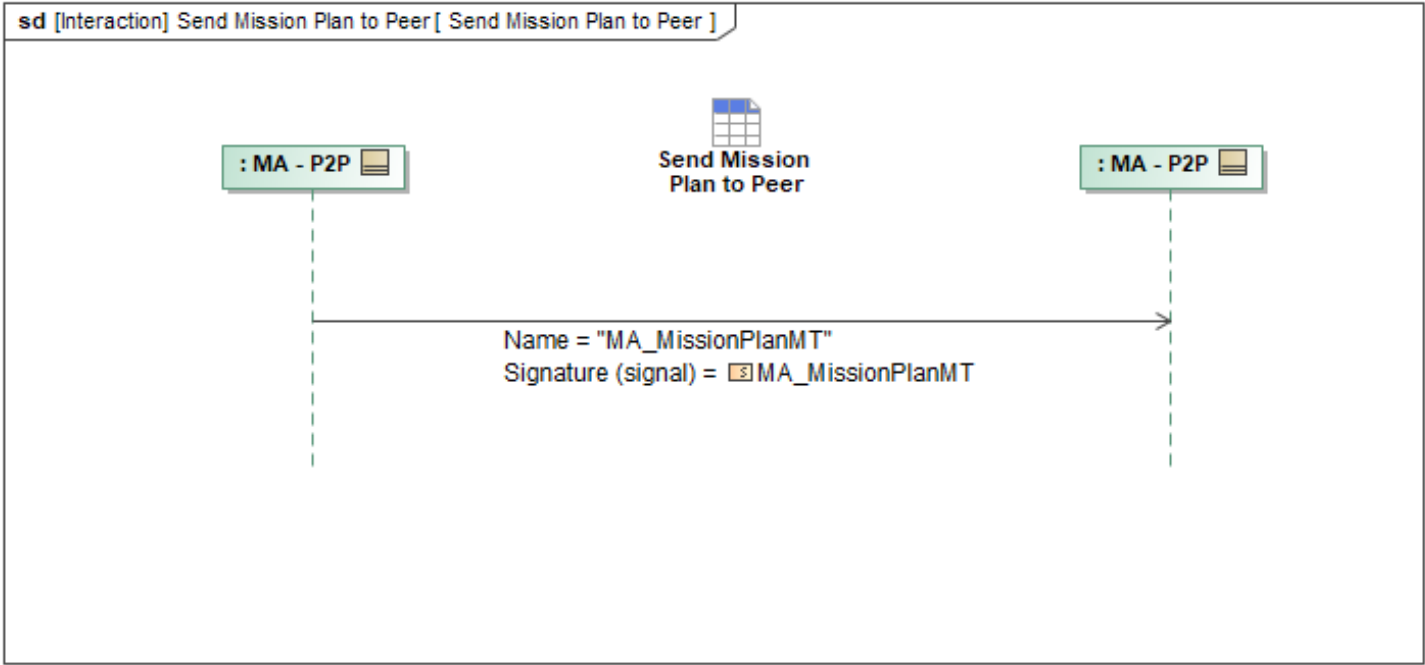


Figure 9-73: Send Mission Plan to Peer

Table 9-72: Send Mission Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanMT: MA_MissionPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.14 Send Navigation Report to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_NavigationReportMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *MA_NavigationReportMT* message is sent.

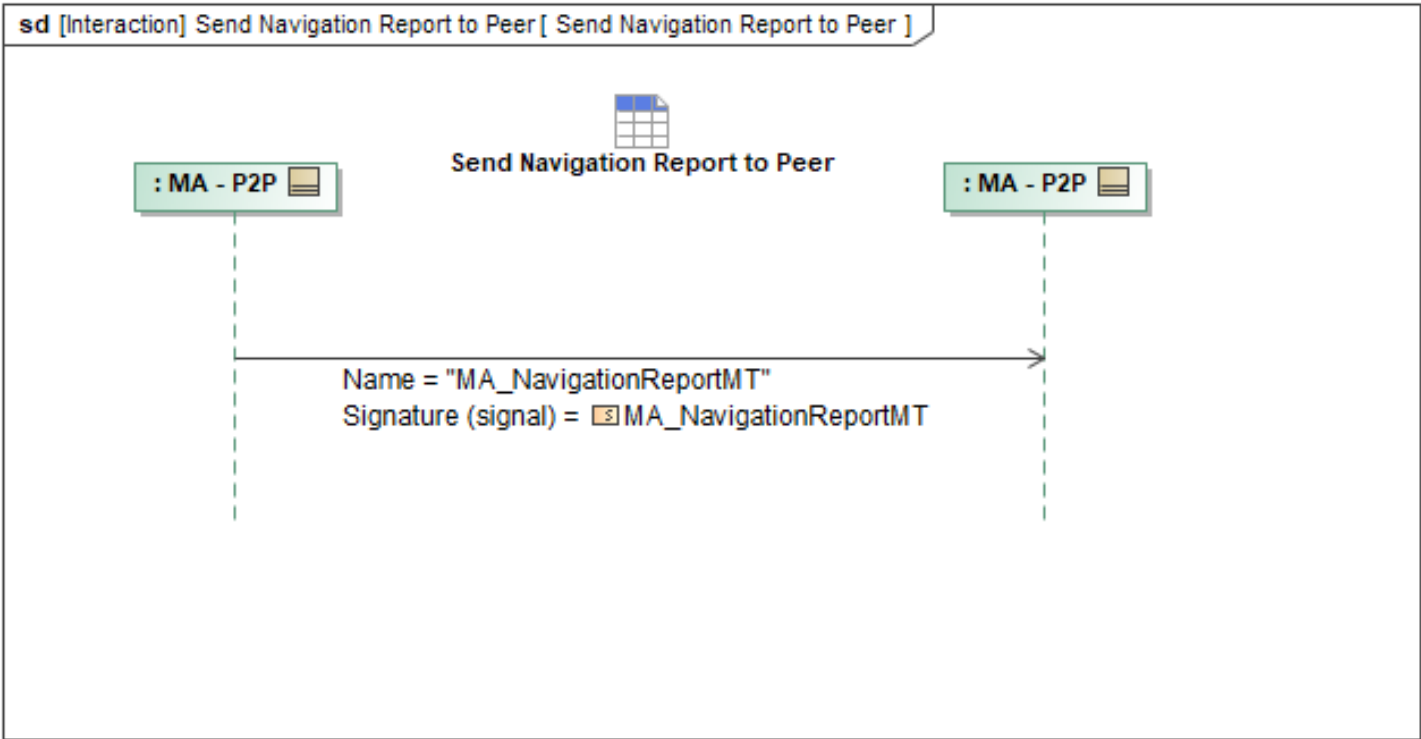


Figure 9-74: Send Navigation Report to Peer

Table 9-73: Send Navigation Report to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_NavigationReportMT: MA_NavigationReportMT	Fuel: MassType Duration: DurationRangeType Percent: PercentType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.15 Send Operator Role to Peer

This diagram contains the message, *MA_OperatorRoleMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

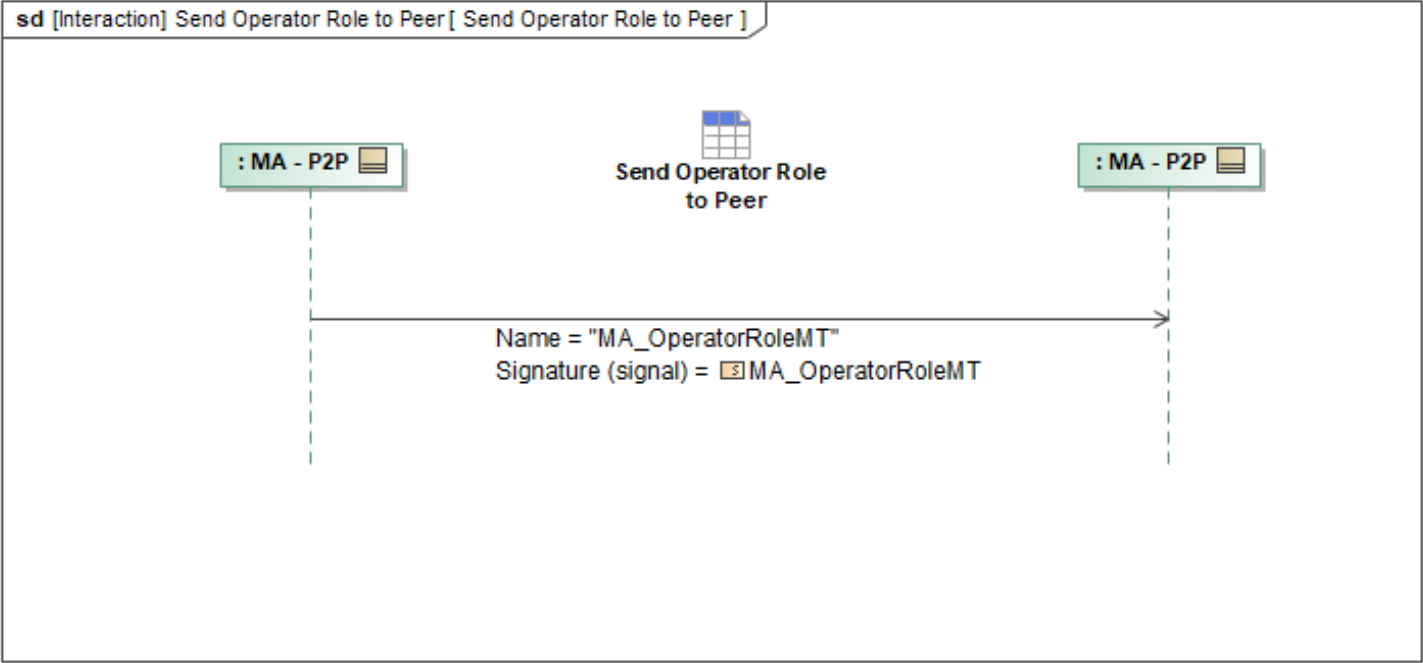


Figure 9-75: Send Operator Role to Peer

Table 9-74: Send Operator Role to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OperatorRoleMT: MA_OperatorRoleMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.16 Send Operator to Peer

This diagram contains the message, *OperatorMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

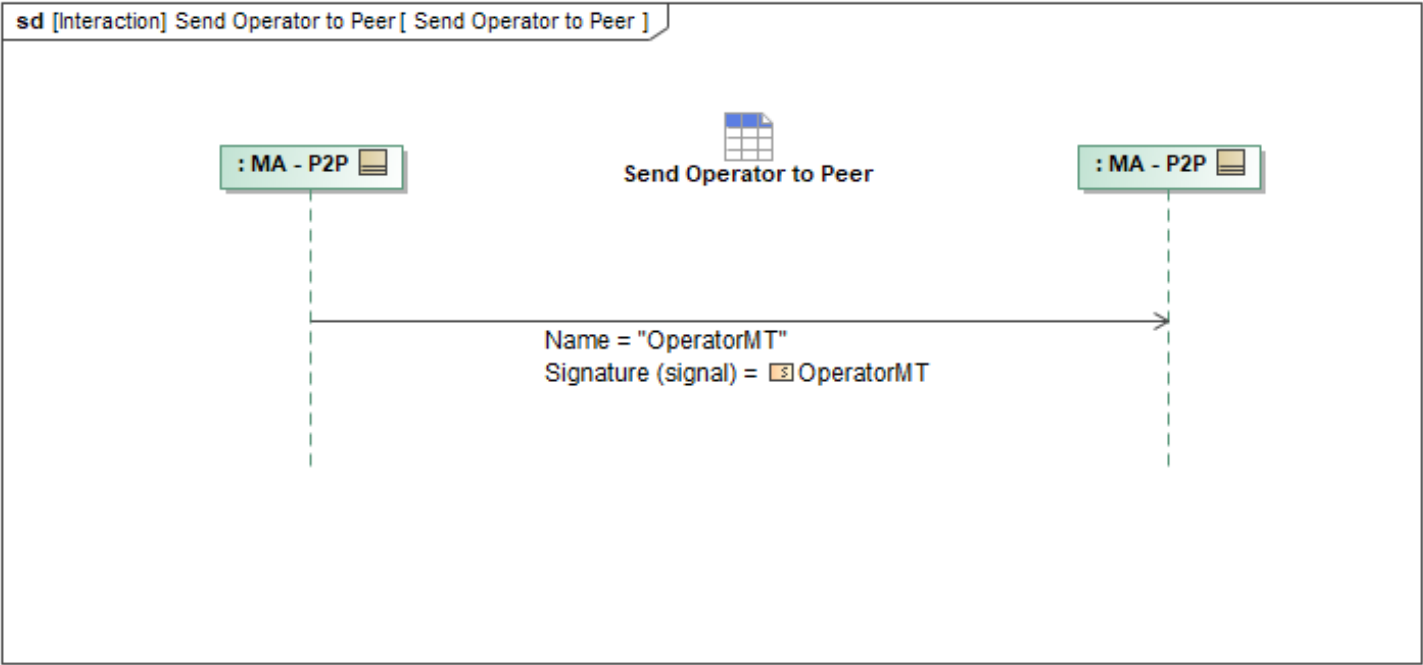


Figure 9-76: Send Operator to Peer

Table 9-75: Send Operator to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
OperatorMT: OperatorMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.17 Send OpLine to Peer

This diagram contains the message, *MA_OpLineMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

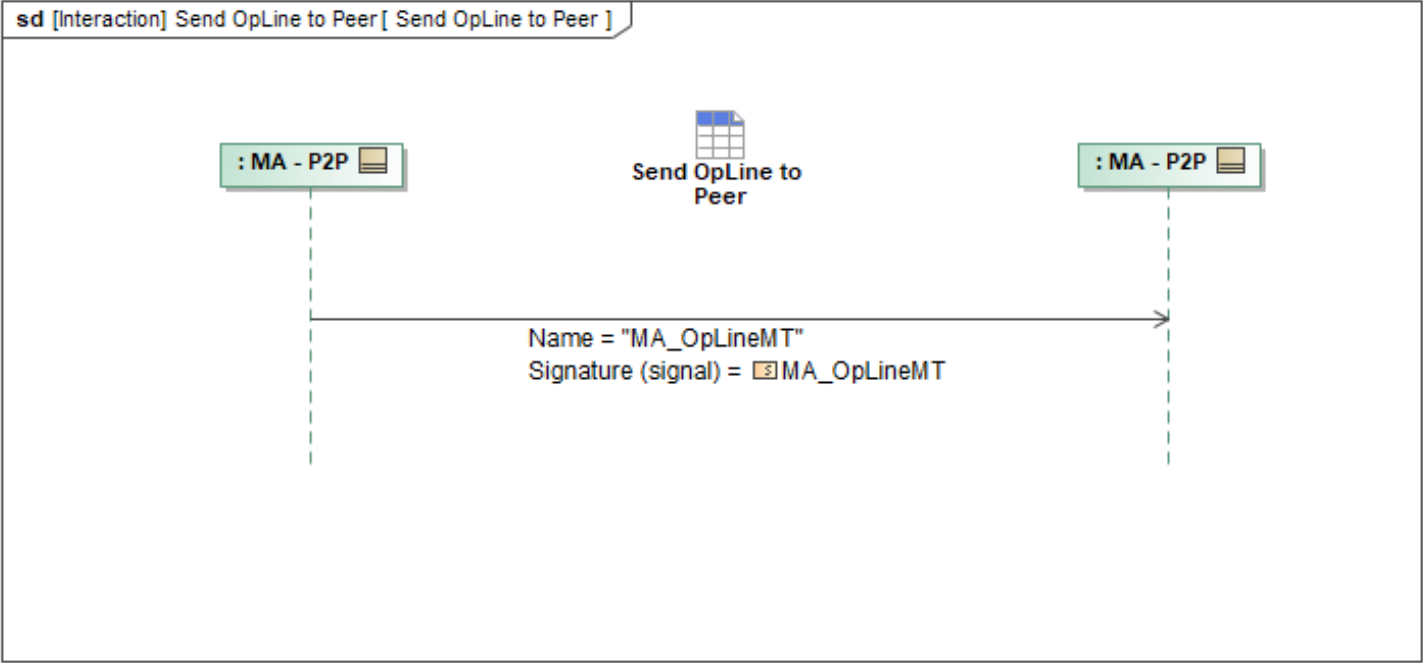


Figure 9-77: Send OpLine to Peer

Table 9-76: Send OpLine to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpLineMT: MA_OpLineMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.18 Send OpPlan to Peer

This diagram contains the message, *MA_OpPlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

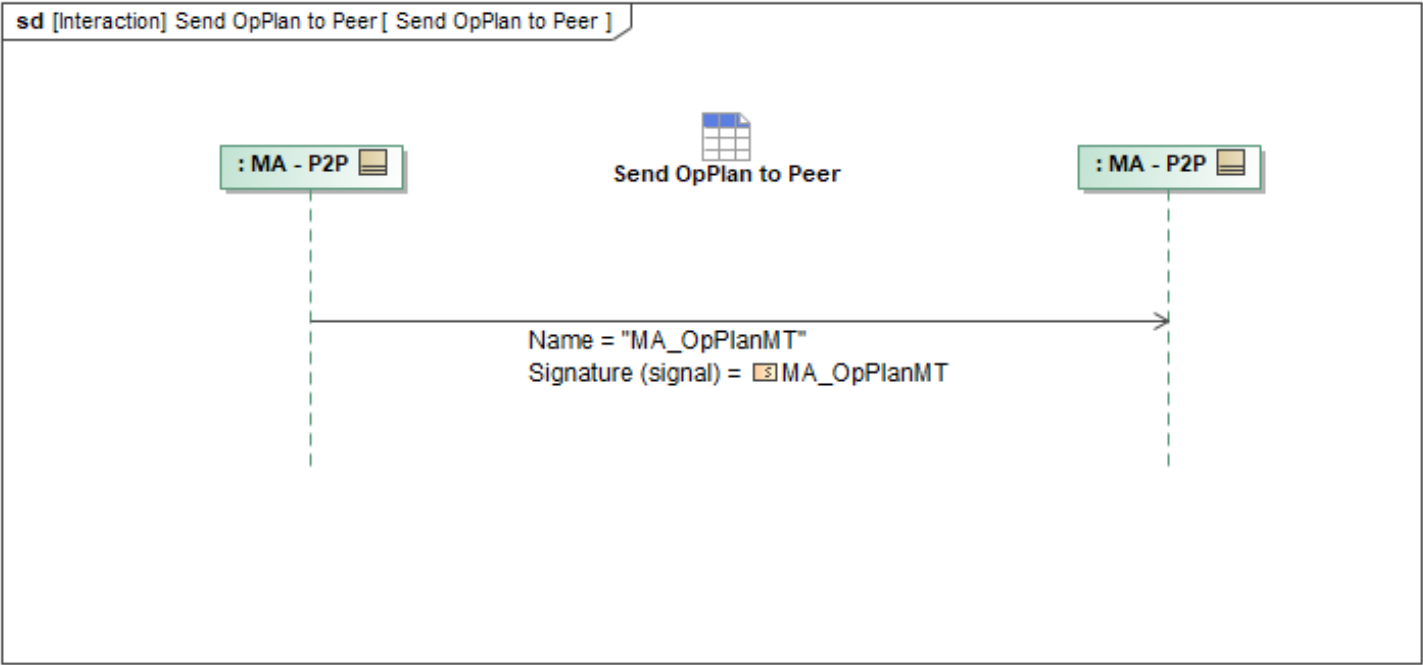


Figure 9-78: Send OpPlan to Peer

Table 9-77: Send OpPlan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpPlanMT: MA_OpPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.19 Send OpRouting to Peer

This diagram contains the message, *MA_OpRoutingMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

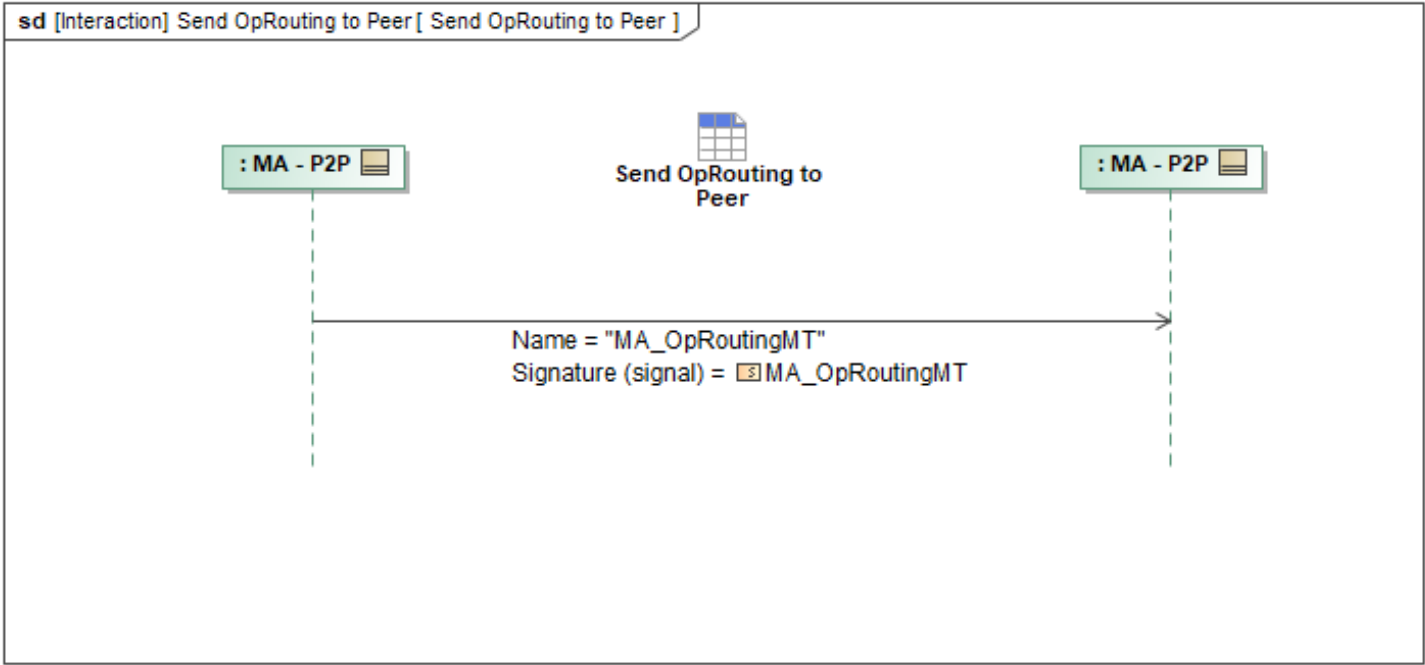


Figure 9-79: Send OpRouting to Peer

Table 9-78: Send OpRouting to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpRoutingMT: MA_OpRoutingMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.20 Send OpVolume to Peer

This diagram contains the message, *MA_OpVolumeMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

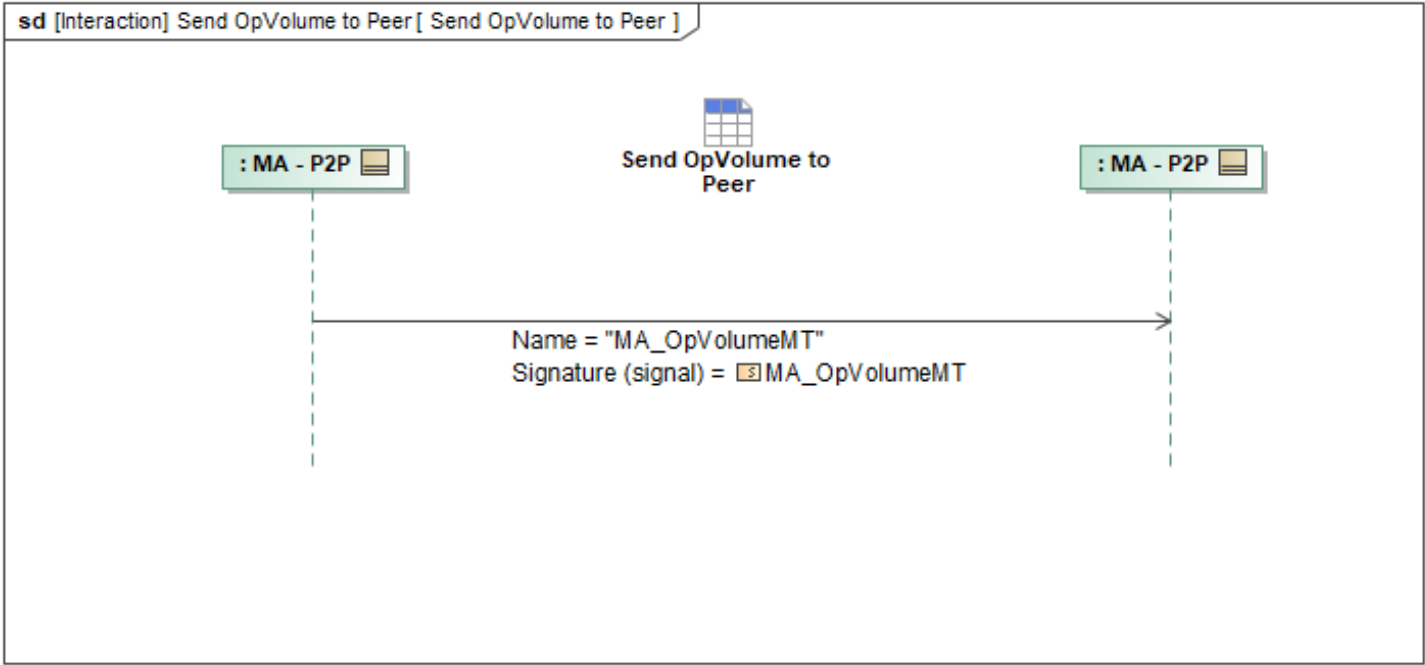


Figure 9-80: Send OpVolume to Peer

Table 9-79: Send OpVolume to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpVolumeMT: MA_OpVolumeMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.21 Send OpZone to Peer

This diagram contains the message, *MA_OpZoneMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

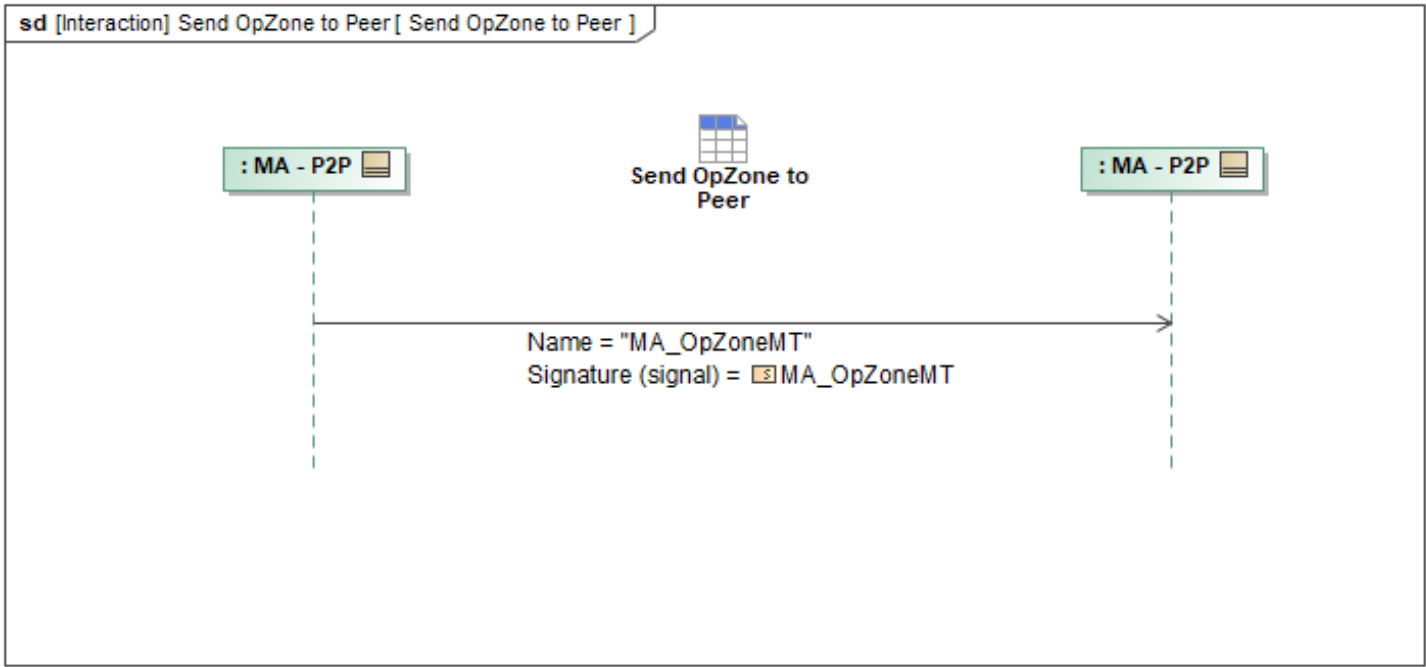


Figure 9-81: Send OpZone to Peer

Table 9-80: Send OpZone to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_OpZoneMT: MA_OpZoneMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.22 Send Position Report to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_PositionReportMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the *MA_PositionReportMT* message is sent. The *MA_PositionReportMT* must have the *PositionSource* set to the *SystemID* of the vehicle. *Position* is set including the timestamp of the data. In addition, *Velocity* and *Orientation* must be set.

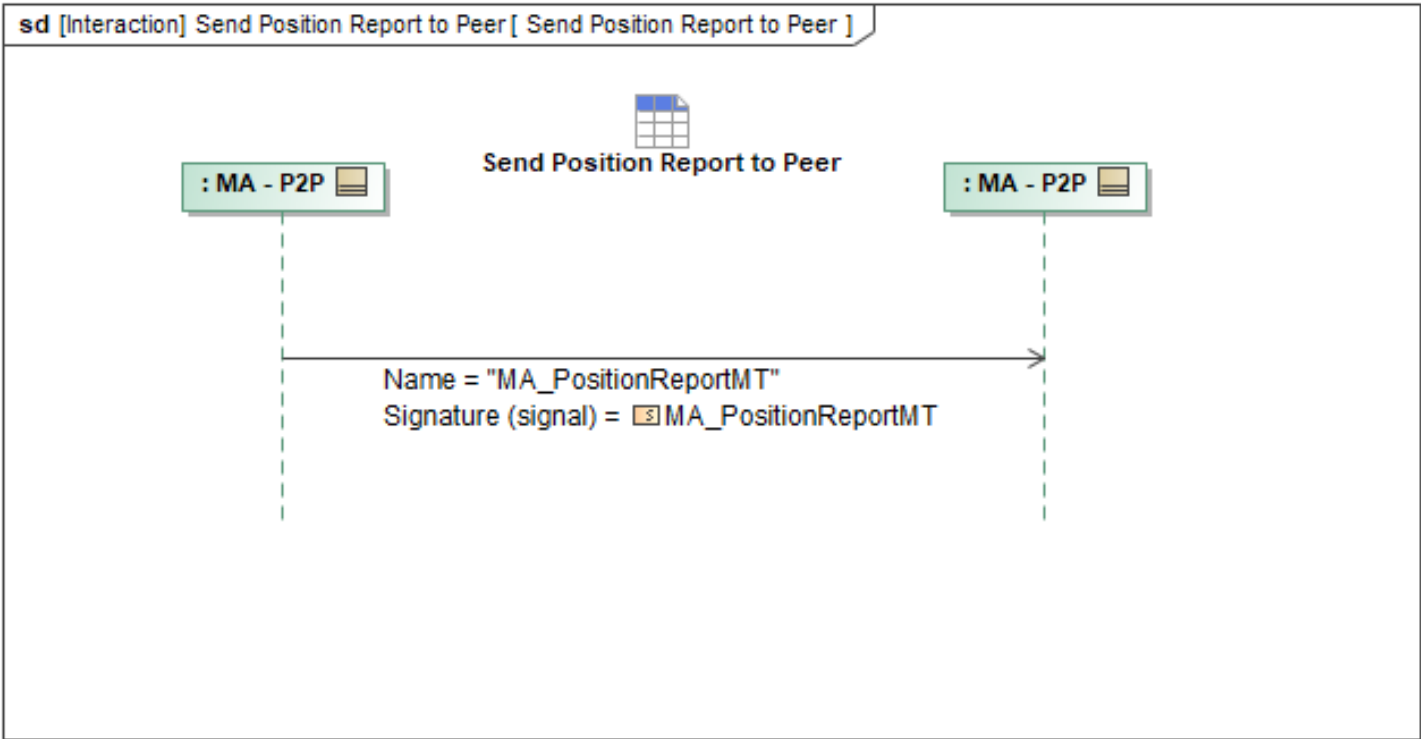


Figure 9-82: Send Position Report to Peer

Table 9-81: Send Position Report to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PositionReportMT: MA_PositionReportMT	Orientation: OrientationType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.23 Send Response Plan to Peer

This diagram contains the message, *ResponsePlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

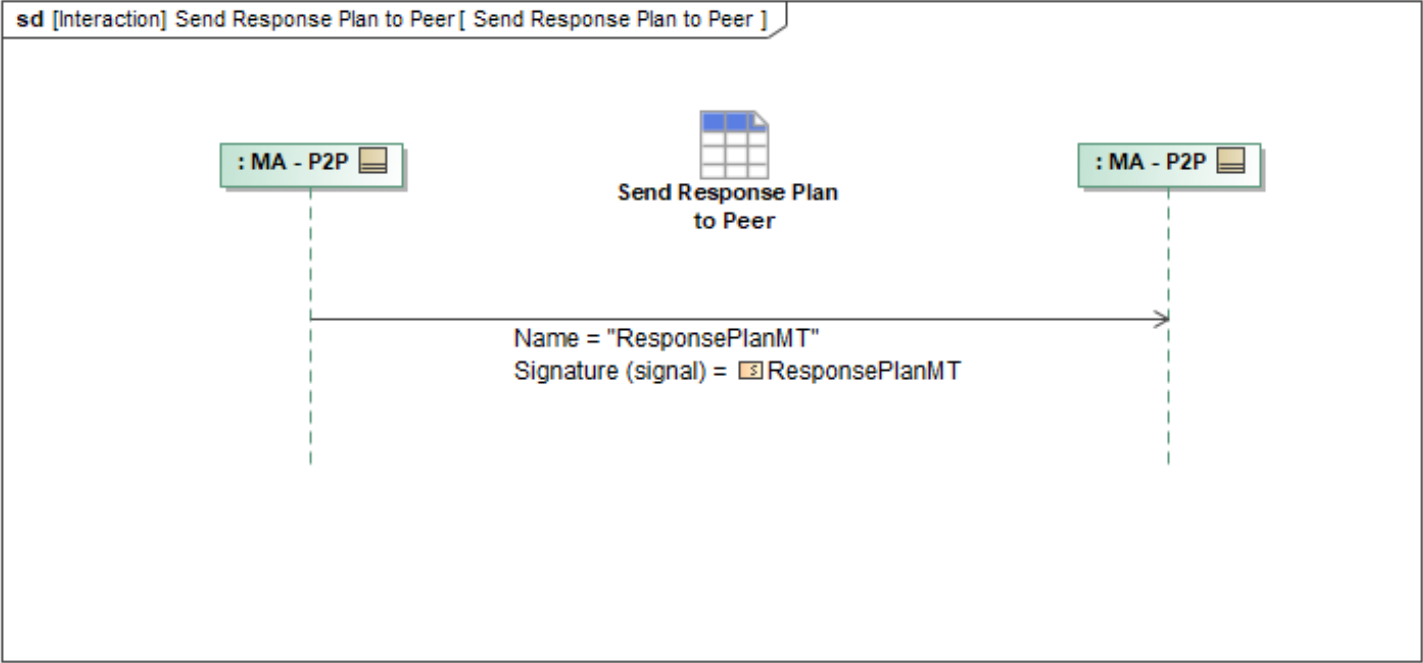


Figure 9-83: Send Response Plan to Peer

Table 9-82: Send Response Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
ResponsePlanMT: ResponsePlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.24 Send Response to Peer

This diagram contains the message, *MA_ResponseMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

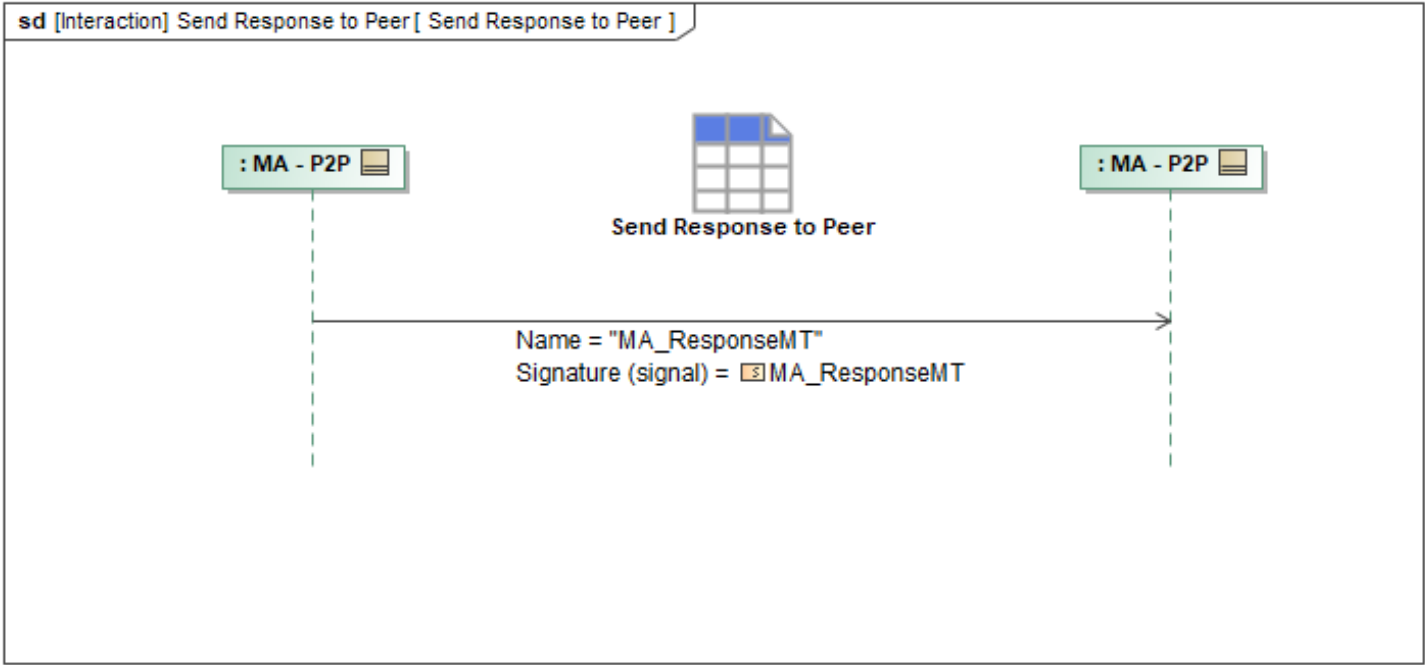


Figure 9-84: Send Response to Peer

Table 9-83: Send Response to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseMT: MA_ResponseMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.25 Send Route Activity Plan to Peer

This diagram contains the message, *MA_RouteActivityPlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

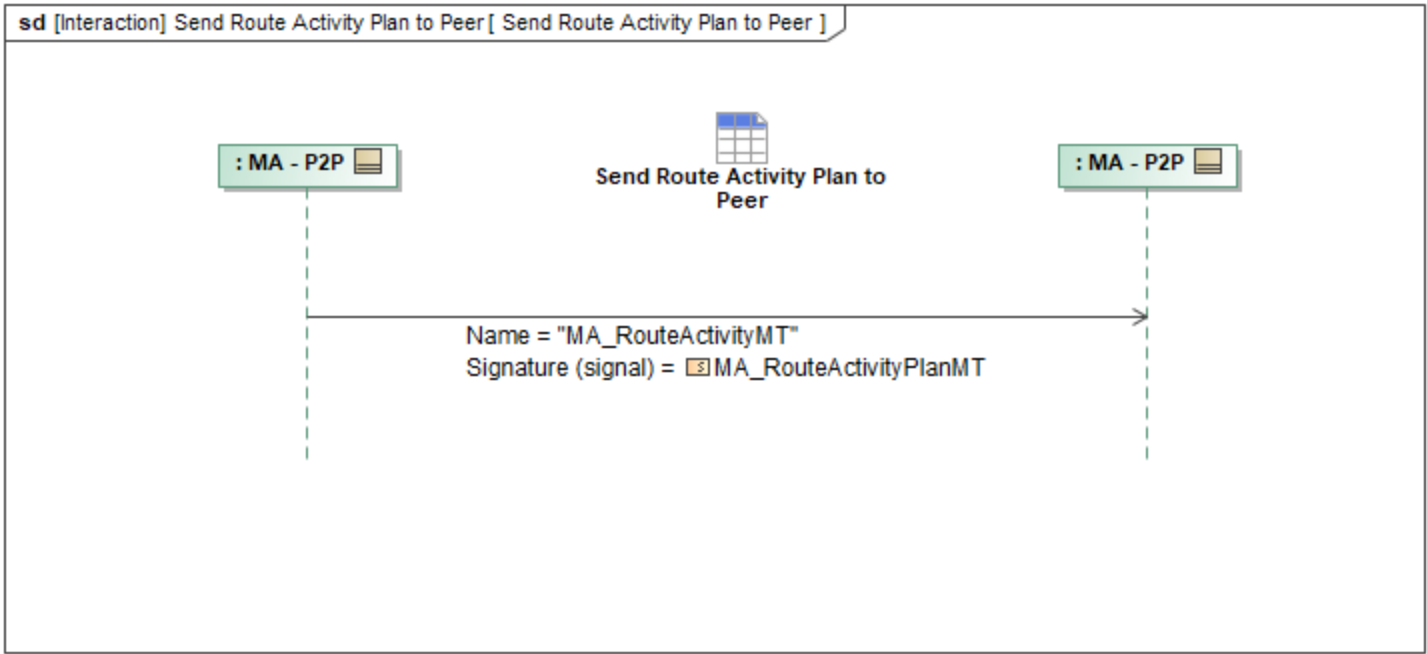


Figure 9-85: Send Route Activity Plan to Peer

Table 9-84: Send Route Activity Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteActivityMT: MA_RouteActivityPlanMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.26 Send Route Deconfliction Alert to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_RouteDeconflictionAlertMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

The *MA_RouteDeconflictionAlertMT* is intended to alert other ACPs of a detected route conflict. This message contains a *ConflictPoint* field to represent inputs from the original request and data fields to represent specific *ConflictID* and *RoutePlanID* for potential deconfliction solutions.

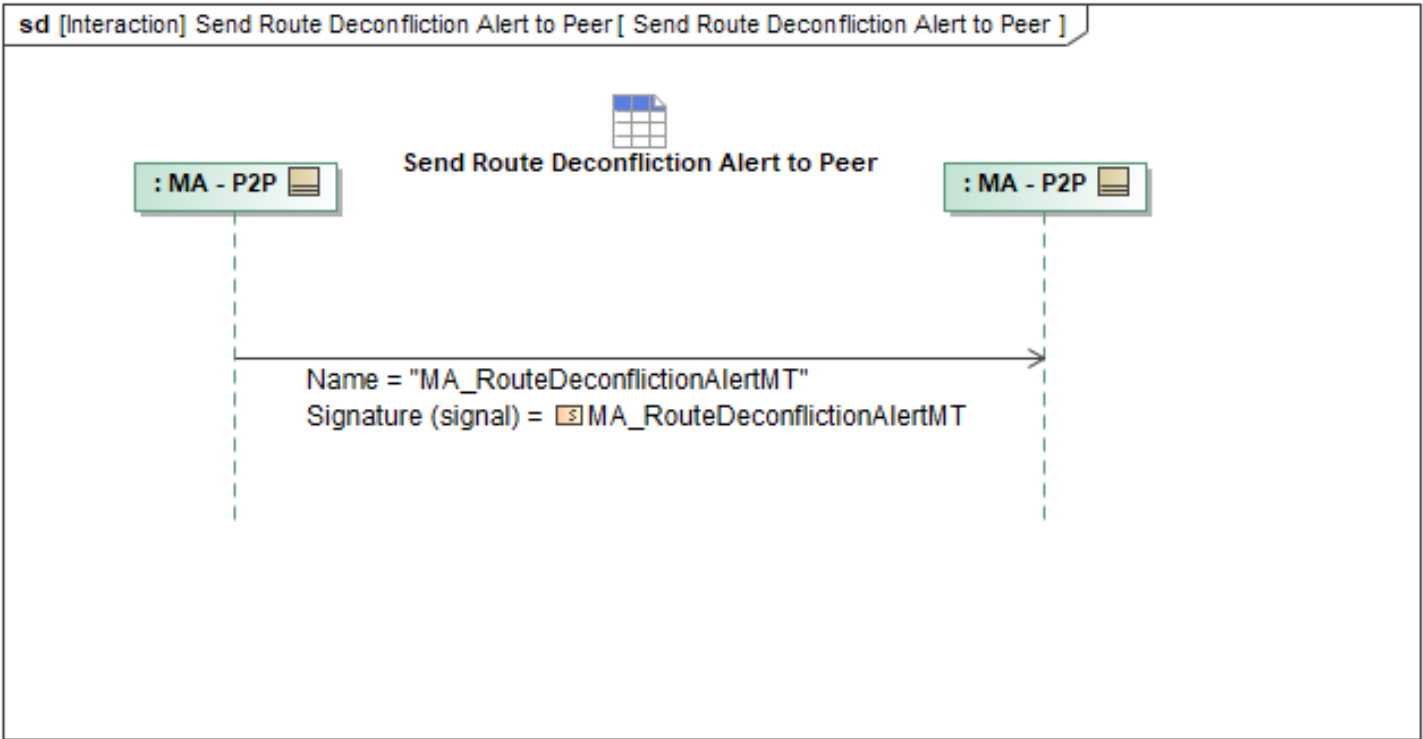


Figure 9-86: Send Route Deconfliction Alert to Peer

Table 9-85: Send Route Deconfliction Alert to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RouteDeconflictionAlertMT: MA_RouteDeconflictionAlertMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.27 Send Route Plan Execution Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *RoutePlanExecutionStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the message *RoutePlanExecutionStatusMT* is sent. The *RoutePlanExecutionStatusMT* must have the *SystemID* of the vehicle and the *CurrentSegment* contain the *RoutePlanID*, *PathID*, and *SegmentID* of the current segment

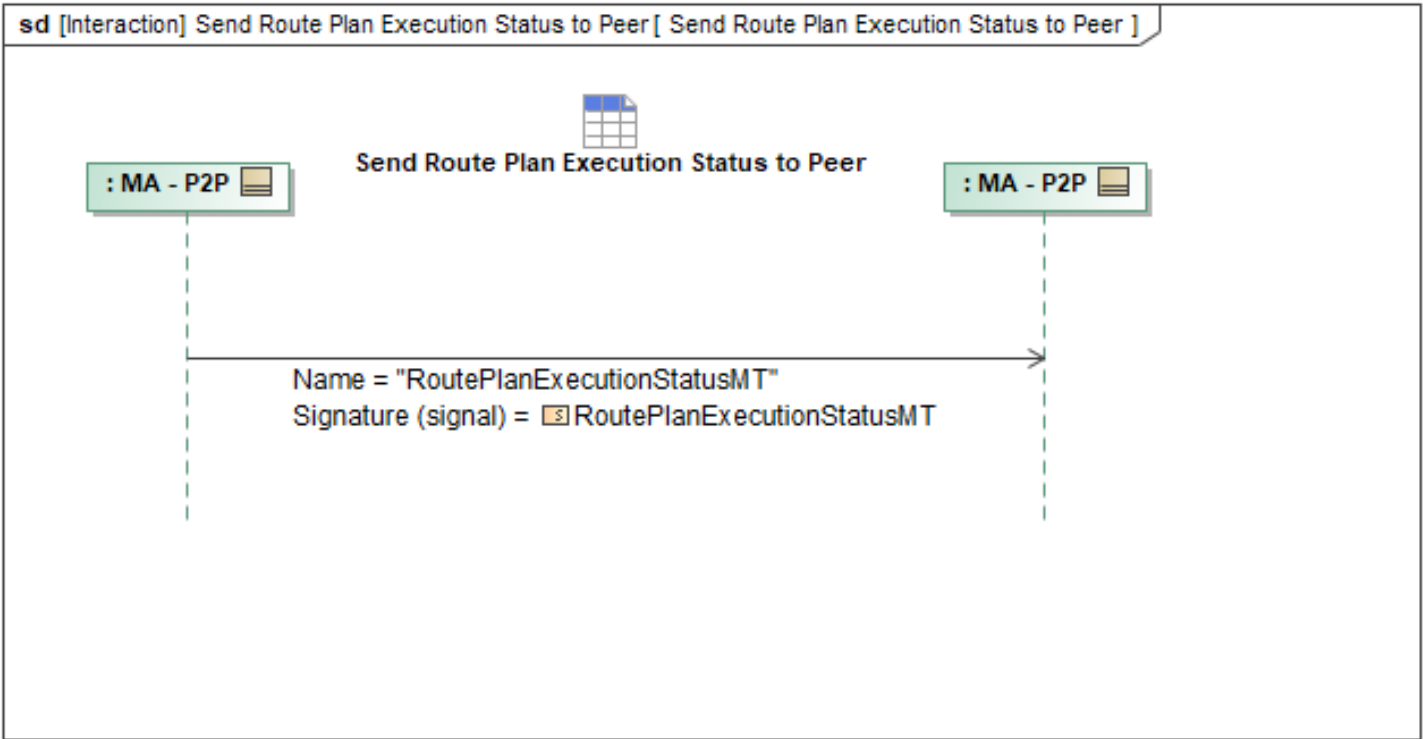


Figure 9-87: Send Route Plan Execution Status to Peer

Table 9-86: Send Route Plan Execution Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.28 Send Route Plan to Peer

This diagram contains the message, *MA_RoutePlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

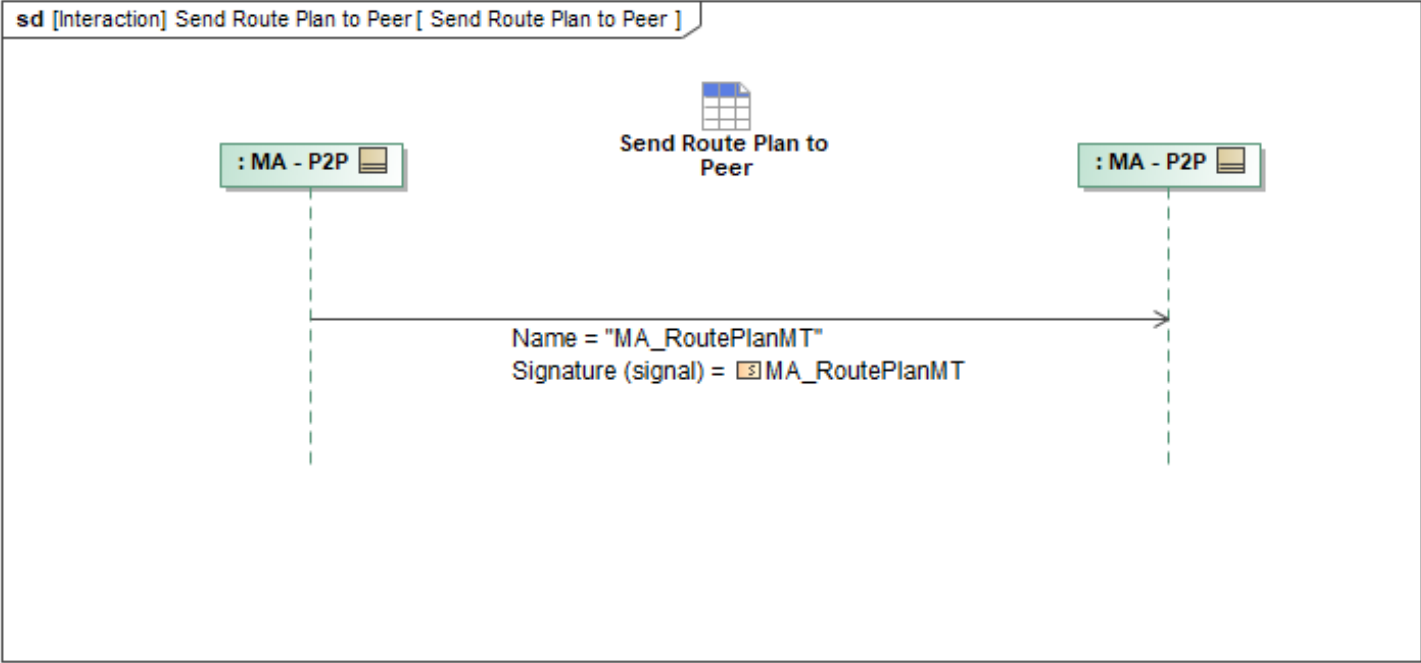


Figure 9-88: Send Route Plan to Peer

Table 9-87: Send Route Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	MA_RoutePlanMT:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.29 Send Subsystem Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *SubsystemStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

The *SubsystemStatusMT* must have the *SubsystemID* of the comms subsystem and *SubsystemState* set to the state of the comms subsystem and its links.

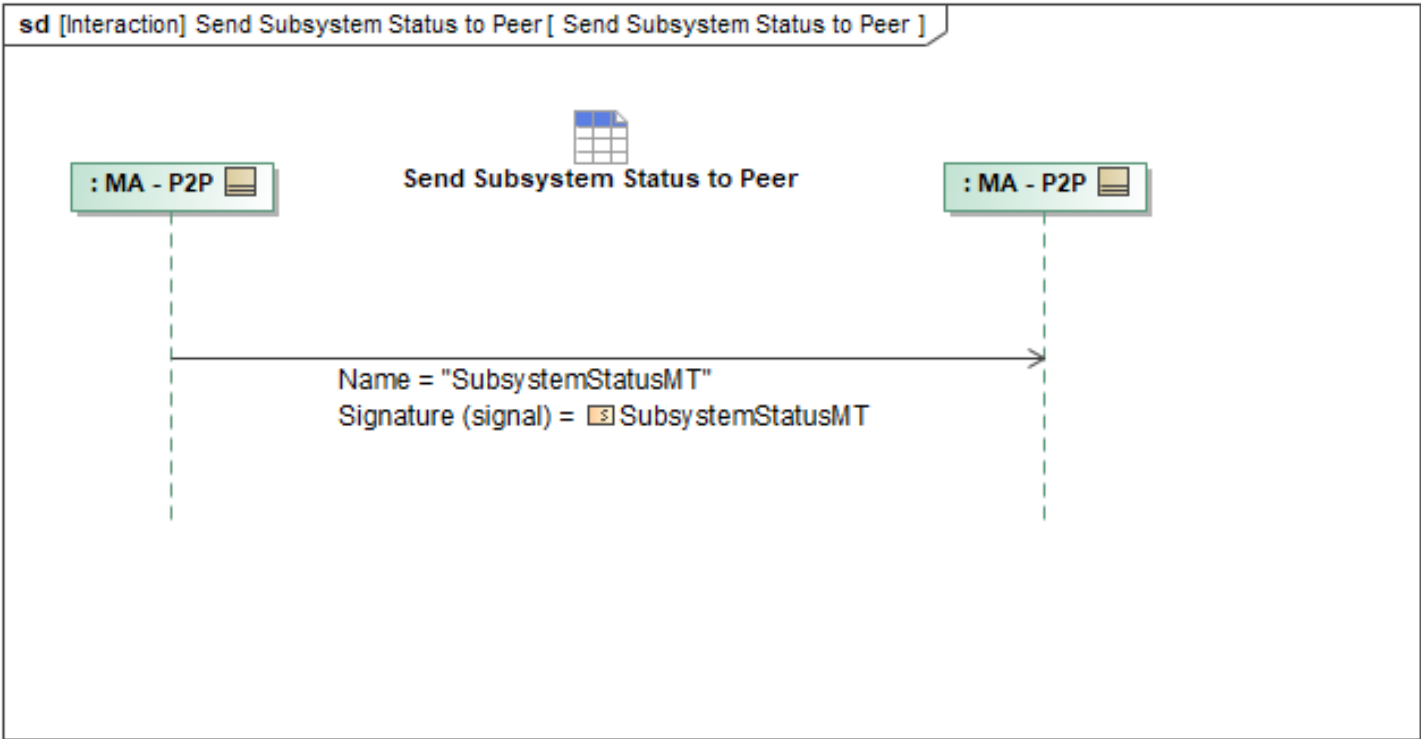


Figure 9-89: Send Subsystem Status to Peer

Table 9-88: Send Subsystem Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.30 Send System Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *SystemStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the message *SystemStatusMT* is sent. The *SystemStatusMT* must have the *SystemID* of the vehicle and the *SystemState* with the overall state of the system. The *Source* must be set to the data source and the *MissionCommunicationsState* must be set to the current state of the comms. For the purposes of vehicle identification, the field *Identity.SpecificVehicle.VehicleUniqueIdentifier.TailNumber* shall be populated. The *TailNumber* and other system identification information is sourced from pre-mission planning data. In addition, the message should list the IDs of the *Subsystems*, *Capabilities*, and *Services* in the system.

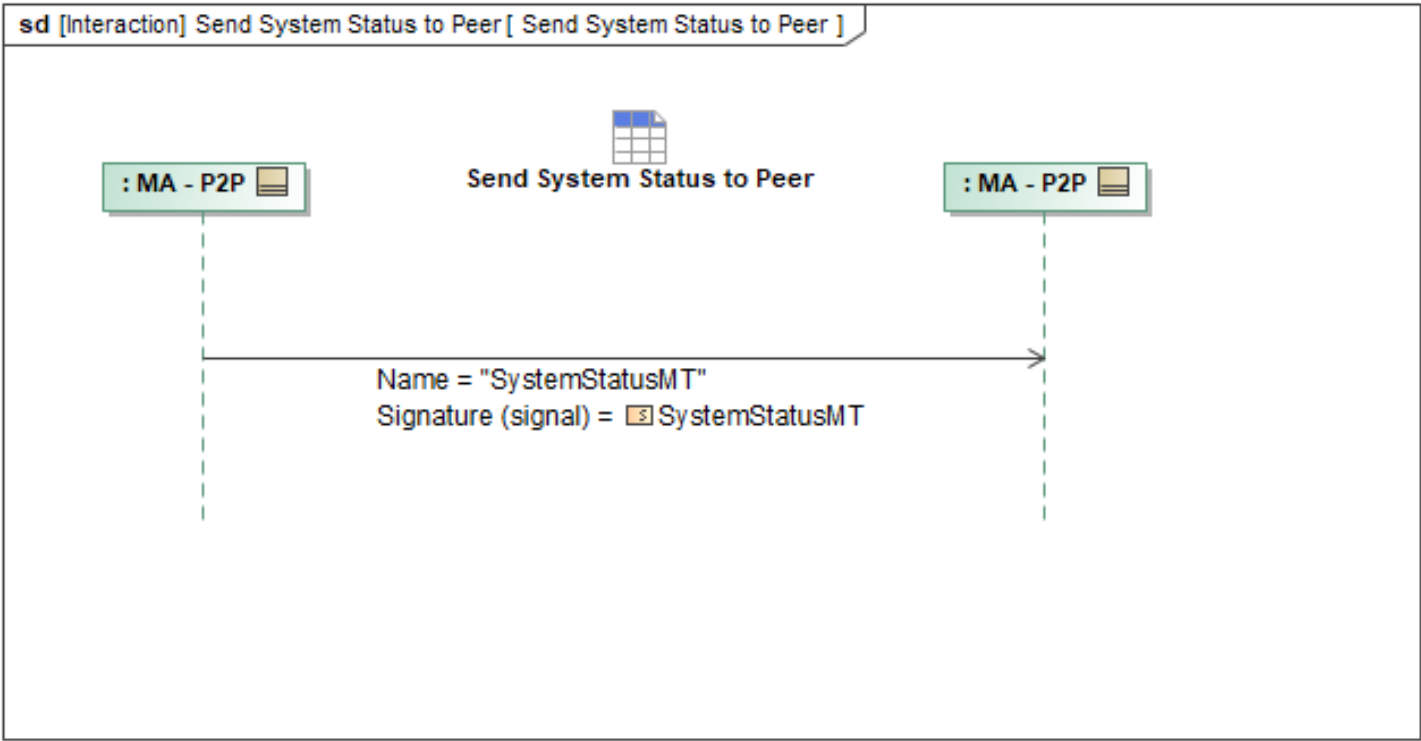


Figure 9-90: Send System Status to Peer

Table 9-89: Send System Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
SystemStatusMT: SystemStatusMT	TailNumber: TailNumberType SubsystemID: SubsystemID_Type CapabilityID: CapabilityID_Type ServiceID: ServiceID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.31 Send Task Plan to Peer

This diagram contains the message, *MA_TaskPlanMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

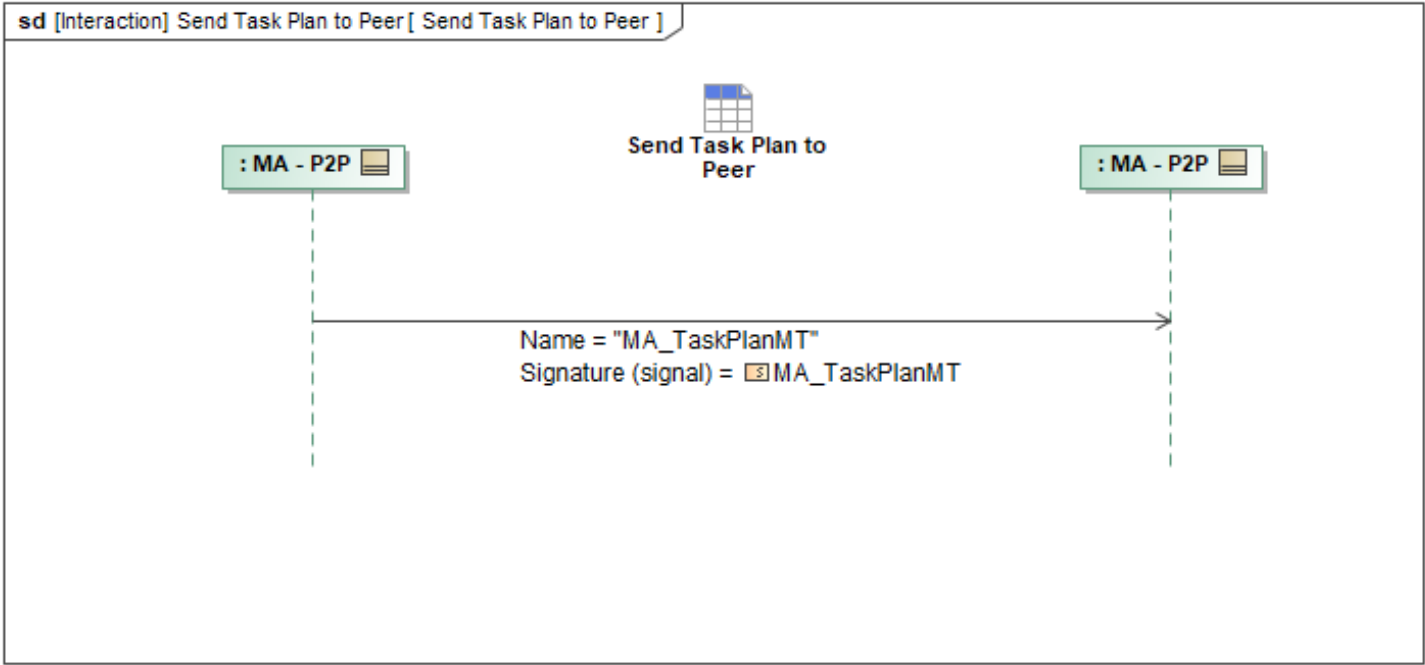


Figure 9-91: Send Task Plan to Peer

Table 9-90: Send Task Plan to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskPlanMT: MA_TaskPlanMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.32 Send Task Status to Peer

This interaction specifies how Mission Autonomy (MA) shares *MA_TaskStatusMT* through the MA-P2P interface. The transmission of this message allows ACPs to understand the operating state of peers in the airspace. In the context of a centralized Common Operating Picture (COP) distribution setup, the vehicle state messages would be aggregated by the COP lead for distribution outside of the package.

At predefined frequencies, the message *TaskStatusMT* is sent. The *MA_TaskStatusMT* must have the *ExecutingSystemID* set to the *SystemID* of the system. The *ExecutionState* will be *EXECUTING* if MA is executing the task, *COMPLETED* when the target is acquired, and *FAILED* if the execution fails. If the *ExecutionState* is *FAILED*, the *ExecutionStateReason* is required. The *TaskID* contains the ID of the task being executed.

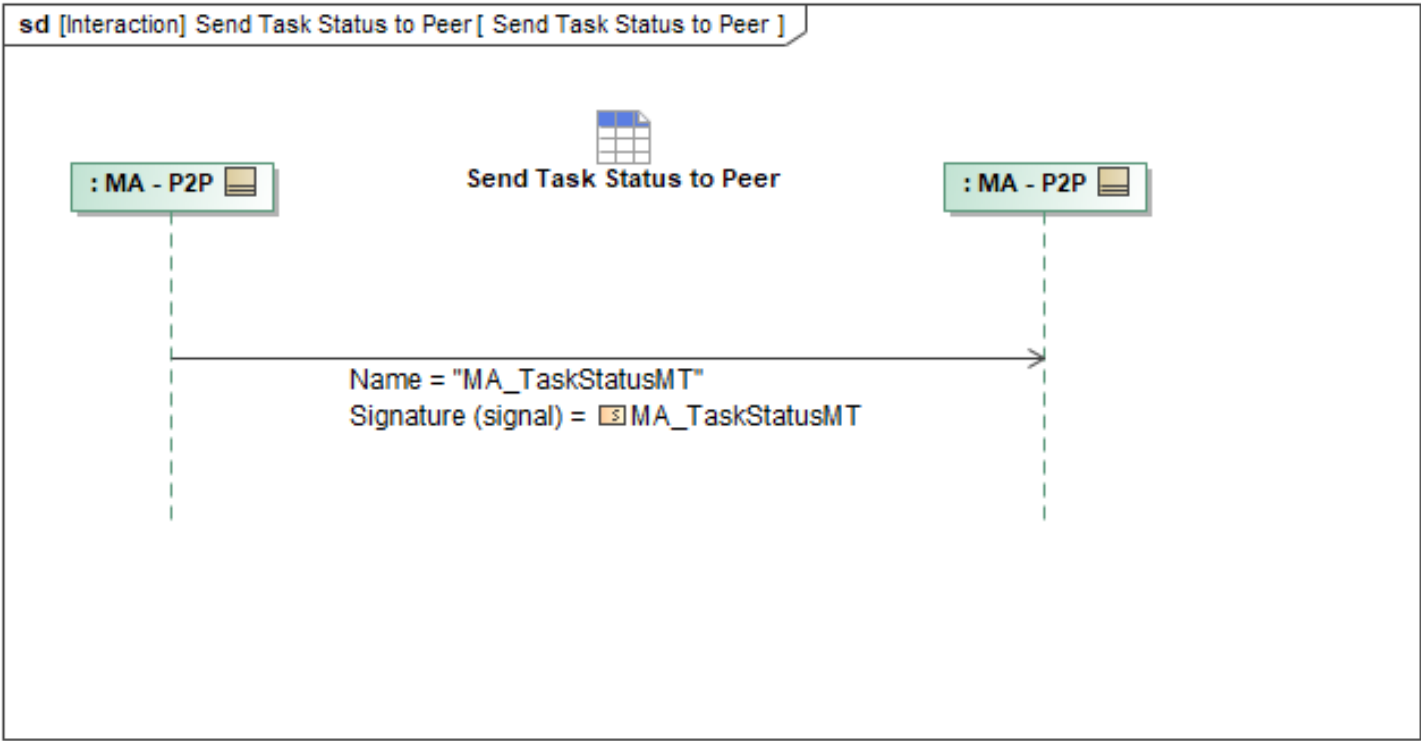


Figure 9-92: Send Task Status to Peer

Table 9-91: Send Task Status to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskStatusMT: MA_TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.10.11.33 Send Task to Peer

This diagram contains the message, *MA_TaskMT*, exchanged between Peers. This interaction is not limited to any specific package roles. Nominally, the initiating peer may be the Leader where the leader is relaying plan data received from C2 to a Follower.

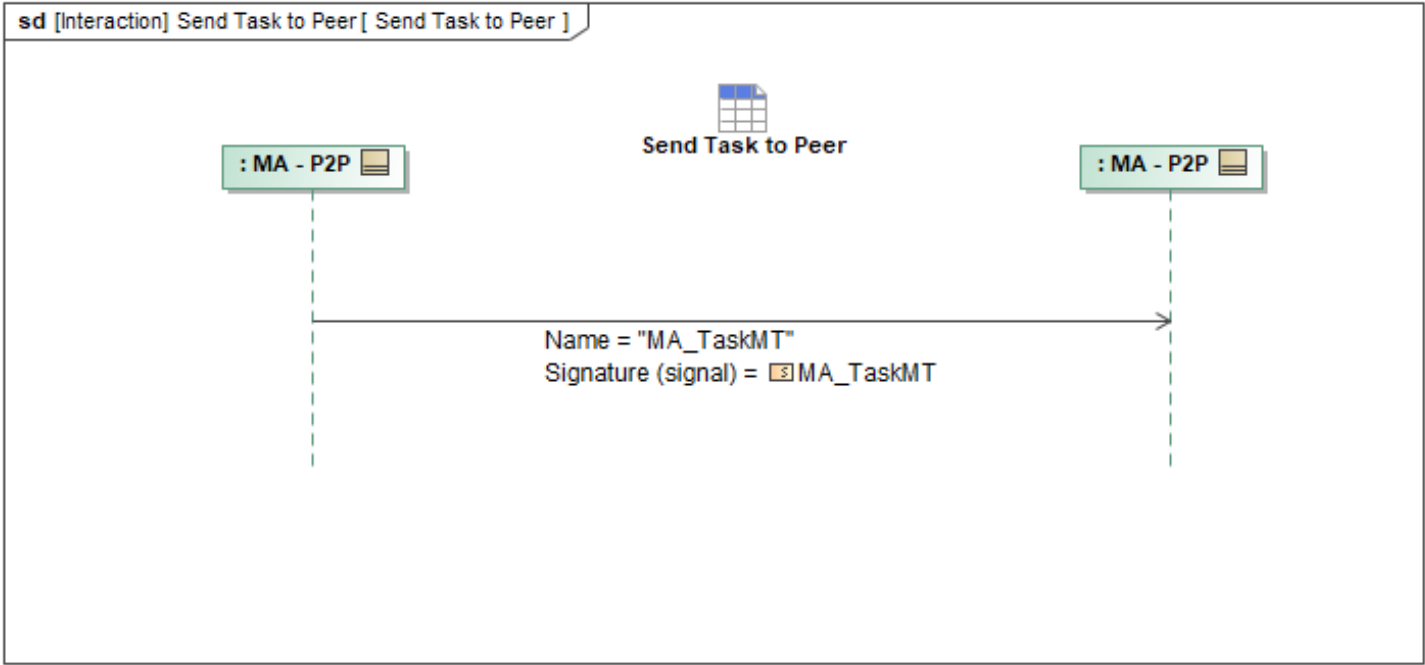


Figure 9-93: Send Task to Peer

Table 9-92: Send Task to Peer

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskMT: MA_TaskMT	ObjectState: ObjectStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11 Peer Package Management and Leader Election Package

This section specifies the set of interactions for Peer Package Management and Leader Election. Leader Election includes multiple possible election methods (as specified by separate interactions) that can be implemented at the discretion of the implementer. All the specified election methods in this section are considered optional and which method an implementer decides to use can verify based on tradeoffs, including communication complexity, support for static vs. dynamic fitness scores and off-nominal considerations.

The ICD includes the following methods for Leader Election:

- 1. Bully Leader Election
- 2. Maximum Consensus Leader Election
- 3. Raft Leader Election
- 4. Static Fitness Score Leader Election
- 5. Off Nominal Leader Election

The implementer can decide whether to utilize a Static or Dynamic Leadership Fitness Score for electing a leader. Note: the Raft Leader Election algorithm does not require a leadership fitness score (whether static or dynamic).

Static Leadership Fitness Scores are intended to be uploaded as part of pre-Mission Planning or specified by C2. These scores are not subject to change during the Mission (unless C2 decides to upload a different Static Leadership Fitness Score for one or multiple ACPs).

Dynamic Leadership Fitness Scores are intended to change throughout the mission based on changing conditions (e.g. Comms Health). A Dynamic Leadership Fitness Score is associated with a determination method, which may be selected based on the enumerated list specified within that message field. This enumerated list also includes a "user-defined" value where an implementation may choose to determine a Dynamic Leadership Fitness Score based on a method at the discretion of the implementation. Utilizing a Dynamic Leadership Fitness is optional (for any of the election methods) for the implementation, as well as the specified method for determining how the fitness score is dynamically computed.

Table 9-93: Leader Election Method Static/Dynamic Support

Method	Supports Static Leadership Fitness Score	Supports Dynamic Leadership Fitness Score
Bully Leader Election	Yes	Yes
Maximum Consensus Leader Election	Yes	Yes
Raft Leader Election	N/A	N/A
Static Fitness Score Leader Election	Yes	No
Off Nominal Leader Election	Yes	Yes

9.2.11.1 Bully Leader Election

Mission Use Case: Bully Leader Election

This leader election method is optional and implementers are expected to implement at least one of the leader election methods.

This outlines the peer interactions for the Bully Leader Election process, including the nominal case as well as some off-nominal cases. The diagram is explained, followed by a description of the general algorithm from a single ACP's perspective.

Diagram Description

Assumptions:

- The package has been initialized (See "Initializing a Package Amongst Peers" interaction).
- Leadership fitness scores are $ACP\ 1 < ACP\ 2 < ACP\ 3$. Since ACP 3 has the highest fitness score, it is the intended leader.
- ACP 1 is the initiator of the election. Note that any ACP can be the initiator, and multiple initiators would not be problematic. Once an ACP determines the package has been initialized, it will initiate the election if no other ACP has done so yet.

ACP 1 initiates the election by sending *MA_LeaderUpdateRequestMT_1* to ACP 2 and *MA_LeaderUpdateRequestMT_2* to ACP 3. The *Request* field has a blank *ProposedLeaderID* and the *Status* field is set to *UNSPECIFIED*. ACP 1 populates its own static fitness score in the *StaticLeadershipFitnessScore* field. ACP 1 may also populate its own dynamic fitness score in the *DynamicLeadershipFitnessScore* field (if applicable, based on the selected *PackageLeadershipFitnessScoreDeterminationMethod*).

Next, a consensus block is triggered in which ACPs reach consensus on their fitness scores. ACP 3 sends *MA_LeaderUpdateRequestMT_3 -4* to share its fitness score with the other two ACPs by populating the *StaticLeadershipFitnessScore* field and *DynamicLeadershipFitnessScore* (if applicable). Similarly, ACP 2 sends *MA_LeaderUpdateRequestMT_5 -6* with the *StaticLeadershipFitnessScore* and *DynamicLeadershipFitnessScore* (if applicable) fields populated with its score(s). These messages have a blank *ProposedLeaderID* in the *Request* field, and the *Status* field set to *UNSPECIFIED*.

Once ACP 1 and ACP 2 have received all fitness scores, they each begin an election timer. In the nominal case, ACP 3 declares its leadership after receiving all fitness scores (before an election timeout is reached), and will proceed to declare leadership via "Peer Declares Package Leadership to Peers."

If ACP 3 hasn't declared leadership before one of the election timers reaches timeout, ACP 1 or 2 (whichever reaches timeout first) should send ACP 3 *MA_LeaderUpdateRequestMT* with the *Status* field set to *ASSIGN* and ACP 3's System ID populated in *ProposedLeaderID* of the *Request* field. These messages are shown as *MA_LeaderUpdateRequestMT_7* and *MA_LeaderUpdateRequestMT_8* in the diagram. After ACP 3 receives this leader assignment message, it will declare leadership via "Peer Declares Package Leadership to Peers."

Other off-nominal cases of leader election are illustrated in the referenced sub-diagram.

Algorithm Description (from an ACP's perspective)

First, I undergo the process of reaching consensus on leadership fitness scores and rank ACPs from highest to lowest scores. Ties are broken by highest tail number, which is found via the heartbeat system status messages in the Peer Heartbeat and Send Vehicle State to Peer interactions. If I receive *MA_LeaderUpdateRequestMT* assigning me as leader before I receive all fitness scores, another ACP has determined I have highest fitness score, so I will declare myself leader. If I determine that I have the highest fitness score among all peer ACPs, I will declare myself leader.

If I determine I do not have the highest fitness score, I wait until the election times out and should send *MA_LeaderUpdateRequestMT* assigning leadership to the highest scoring ACP. I start a new election timer, and if the highest-scoring ACP does not declare itself leader before the timeout, I should send *MA_LeaderUpdateRequestMT* to the next highest-scoring ACP with *Status* set to *NOMINATE*. I wait to receive a response of *MA_LeaderUpdateRequestStatusMT* with *RequestProcessingState* of *PROCESSING* from the ACP, indicating the nomination was received and pending (See "Off Nominal Leader Election" sequence). If this election also times out, I should repeat the process of sending *MA_LeaderUpdateRequestMT* to the next highest-scoring ACP in the ranked list of fitness scores until receiving *MA_LeaderUpdateRequestStatusMT* with *RequestProcessingState* of *PROCESSING* from an ACP.

During this process:

- If a leader is declared, terminate the current election as it is no longer needed.
- If I determine myself to be the next highest-scoring ACP after an election reaches timeout, send *MA_LeaderUpdateRequestMT* to everyone nominating myself as leader. After a majority of ACPs accept, declare my leadership and terminate the election. If rejected, wait for a leader to be declared.
- If an ACP nominates itself as leader, accept their leadership request if they have a higher fitness score than myself. Otherwise, reject the request.

- If I receive a message from another ACP nominating me for leadership, respond with *MA_LeaderUpdateRequestStatusMT* with *RequestProcessingState* of *PROCESSING*. Continue following the election algorithm.

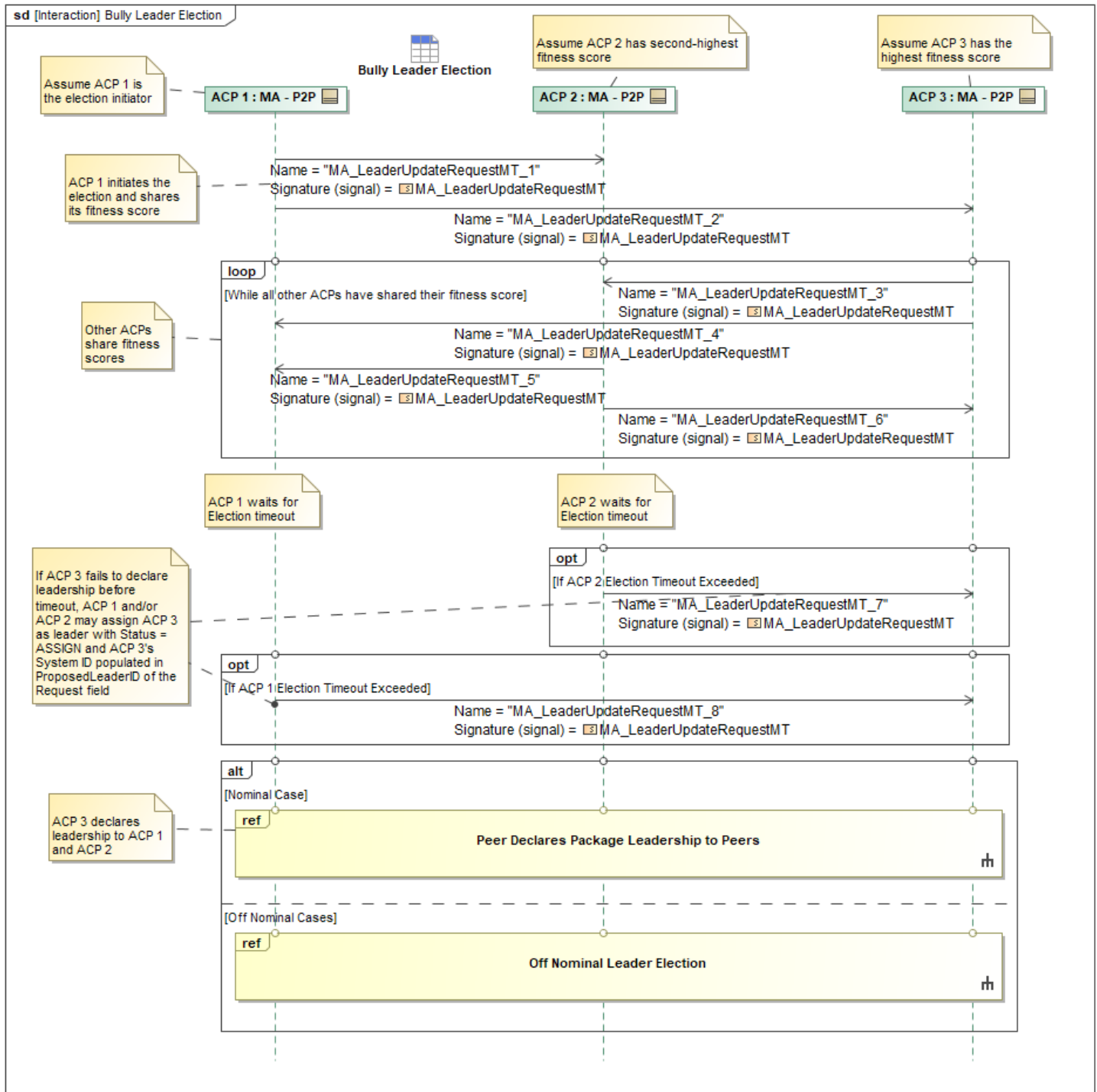


Figure 9-94: Bully Leader Election

Table 9-94: Bully Leader Election

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeaderUpdateRequestMT_1: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_2: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_3: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_4: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_5: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_6: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_7: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_8: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.2 Create a Full Package Amongst Peers

This is the peer interaction which results in a fully-formed package. First, the ACPs follow the "Initializing a Package Amongst Peers" interaction, which is triggered by an Initiator ACP sending *MA_PackageManagementCommandMT* to all intended package member ACPs. After confirming status of all ACPs in the initialization process, the leader must be determined and declared. If the *PackageLeaderID* field of the *MA_PackageManagementCommandMT* message was populated, the identified Lead ACP declares its leadership to all other package members via "Peer Declares Package Leadership to Peers". Otherwise, the ACPs perform leader election according to the method specified in the *PackageLeaderElectionMethod* field of *MA_PackageManagementCommandMT*. Election methods are currently defined for "Static Fitness Score Leader Election," "Maximum Consensus Leader Election," "Raft Leader Election," and "Dynamic Fitness Score Leader Election." These leader election methods are optional, and implementors are expected to implement at least one of the leader election methods. If the election method is unspecified, the implementor will specify a default election method for leader election.

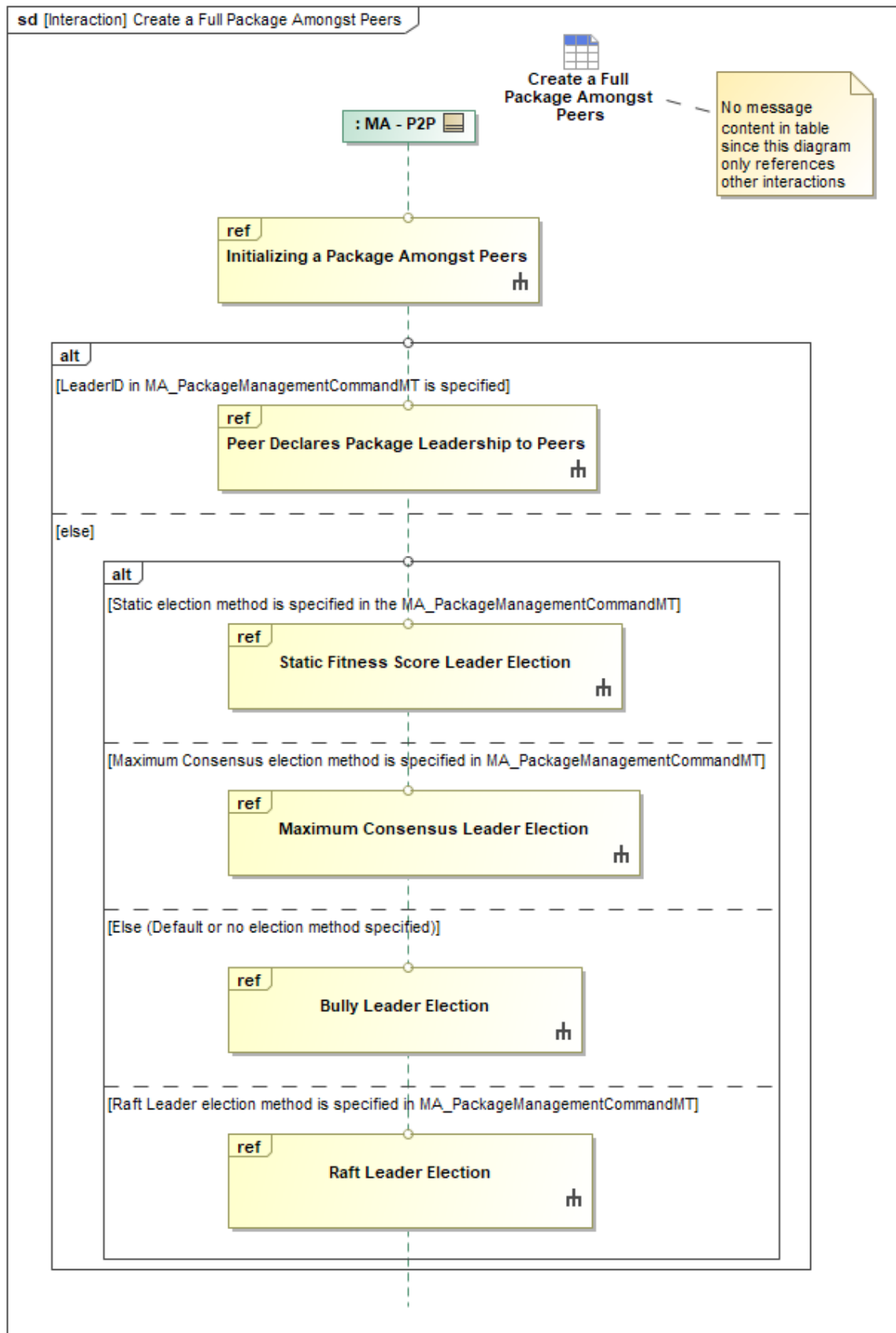


Figure 9-95: Create a Full Package Amongst Peers

Table 9-95: Create a Full Package Amongst Peers

Message Name (Name:Type)	Required Fields (Name:Type)
--------------------------	-----------------------------

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.3 Current Package Lead Assigns New Package Lead

This is the peer interaction where the Current Leader ACP assigns a New Leader ACP. This interaction occurs in the use case "C2 Specifies Team Lead on Existing Team" after the Current Leader receives a command from C2 to update the package leader. First, the Current Leader sends *MA_LeaderUpdateRequestMT* to the New Leader with the *Status* field set to *ASSIGN*, and the *ProposedLeaderID* within the *Request* field populated with the New Leader's ID. In response, the New Leader sends *MA_LeaderUpdateRequestStatusMT* with *RequestProcessingState* of *COMPLETED*. The New Leader then declares its leadership status to all followers via "Peer Declares Package Leadership to Peers."

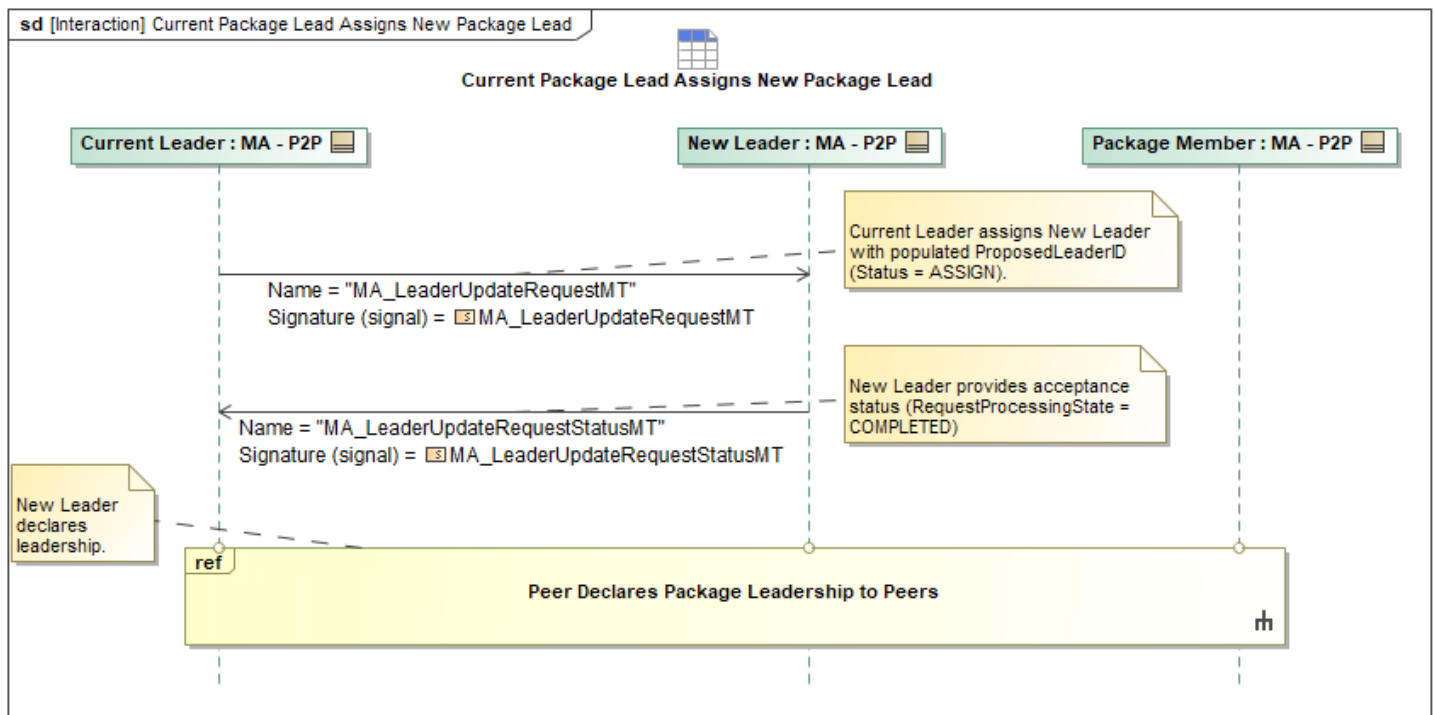


Figure 9-96: Current Package Lead Assigns New Package Lead

Table 9-96: Current Package Lead Assigns New Package Lead

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeaderUpdateRequestMT: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestStatusMT: MA_LeaderUpdateRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.4 Initializing a Package Amongst Peers

This is the peer interaction which results in an "initialized" package, where the status of all intended package members is confirmed before leader election occurs. Note that the package will not be "fully formed" yet, since a leader has not been determined. Package initialization takes place after ACPs have received a command to form a package. In this diagram, ACP 1 is the first ACP to receive the package formation command, so it acts as the initiator. Note that if multiple ACPs attempt to initiate, it doesn't pose a problem for the interaction.

If the leader election method requires fitness scores, ACP 1 sends *MA_PackageManagementCommandMT_1* to ACP 2 and *MA_PackageManagementCommandMT_2* to ACP 3, as the intended package members specified in the original package formation command. If the leader election method does not require fitness scores, ACP 1 sends *MA_PackageManagementCommandMT_3* to ACP 2 and *MA_PackageManagementCommandMT_4* to ACP 3, as the intended package members specified in the original package formation. This message contains all information in the original command, and the *ChangeType* field is set to *FORM_NEW_PACKAGE*.

ACPs 2 and 3 respond with *MA_PackageManagementCommandStatusMT_1* and *MA_PackageManagementCommandStatusMT_2*, respectively.

If the leader election method requires static fitness scores, after ACP 1 receives *MA_PackageManagementCommandStatusMT* messages with *CommandProcessingState* of *ACCEPTED* from both ACPs, it sends *MA_PackagePartnerStatusReportMT_1* to ACP 2 and *MA_PackagePartnerStatusReportMT_2* to ACP 3, to inform them of the presence and status of all peers.

If the leader election method requires dynamic fitness scores, after ACP 1 receives *MA_PackageManagementCommandStatusMT* messages with *CommandProcessingState* of *ACCEPTED* from both ACPs, it sends *MA_PackagePartnerStatusReportMT_3* to ACP 2 and *MA_PackagePartnerStatusReportMT_4* to ACP 3, to inform them of the presence and status of all peers.

If the leader election method does not require fitness scores, after ACP 1 receives *MA_PackageManagementCommandStatusMT* messages with *CommandProcessingState* of *ACCEPTED* from both ACPs, it sends *MA_PackagePartnerStatusReportMT_5* to ACP 2 and *MA_PackagePartnerStatusReportMT_6* to ACP 3, to inform them of the presence and status of all peers.

Since the leader has not yet been declared, via election or otherwise, the *PackageLeaderID* field will be unpopulated.

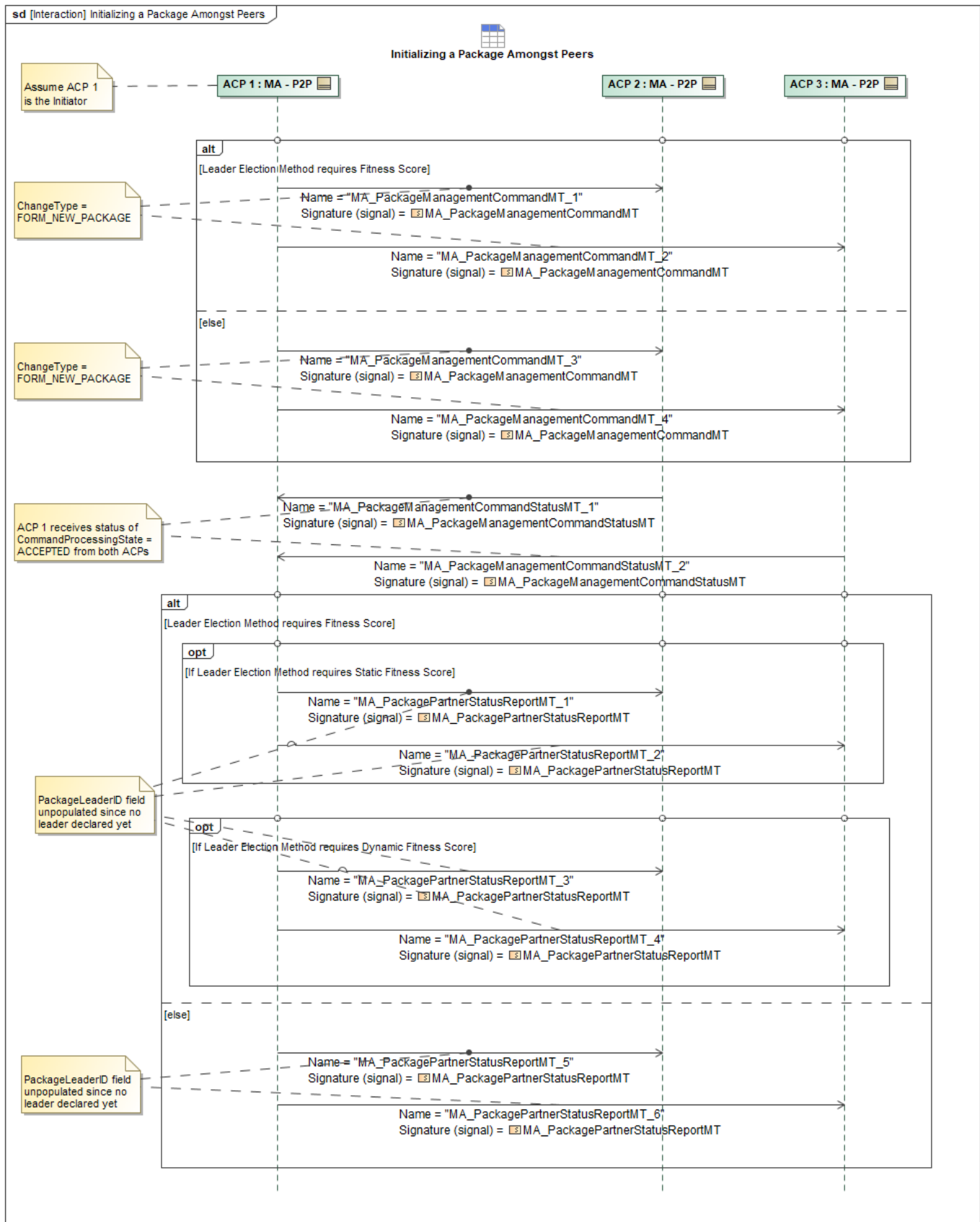


Figure 9-97: Initializing a Package Amongst Peers

Table 9-97: Initializing a Package Amongst Peers

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackageManagementCommandMT_1: MA_PackageManagementCommandMT	PackagePartnerMetrics: MA_PartnerMetricsType
MA_PackageManagementCommandMT_2: MA_PackageManagementCommandMT	PackagePartnerMetrics: MA_PartnerMetricsType
MA_PackageManagementCommandMT_3: MA_PackageManagementCommandMT	None
MA_PackageManagementCommandMT_4: MA_PackageManagementCommandMT	None
MA_PackageManagementCommandStatusMT_1: MA_PackageManagementCommandStatusMT	None
MA_PackageManagementCommandStatusMT_2: MA_PackageManagementCommandStatusMT	None
MA_PackagePartnerStatusReportMT_1: MA_PackagePartnerStatusReportMT	StaticLeadershipFitnessScore: FloatMinMaxType
MA_PackagePartnerStatusReportMT_2: MA_PackagePartnerStatusReportMT	StaticLeadershipFitnessScore: FloatMinMaxType
MA_PackagePartnerStatusReportMT_3: MA_PackagePartnerStatusReportMT	DynamicLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_PackagePartnerStatusReportMT_4: MA_PackagePartnerStatusReportMT	DynamicLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_PackagePartnerStatusReportMT_5: MA_PackagePartnerStatusReportMT	None
MA_PackagePartnerStatusReportMT_6: MA_PackagePartnerStatusReportMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.5 Leader Commanded Package Addition

This peer interaction describes the process of an ACP joining an existing package via command from the Leader of the new package. In this scenario, the Lead ACP will only execute the package addition command if it was sent by an authorized role. The Leader sends

MA_PackageManagementCommandMT to the new asset, with the *ChangeType* field set to *JOIN_PACKAGE*, and the *SystemID* field populated with the new asset's ID. The new asset responds by sending the requesting Leader *MA_PackageManagementCommandStatusMT* with *CommandProcessingState* of *ACCEPTED* to indicate the ACP has joined the package. To all followers (including the newly joined ACP), the Leader sends *PackageStatusMT*, indicating the updated membership of the package. The newly-joined ACP (and all followers) will send *MA_PackagePartnerStatusReportMT* periodically to report status to the leader.

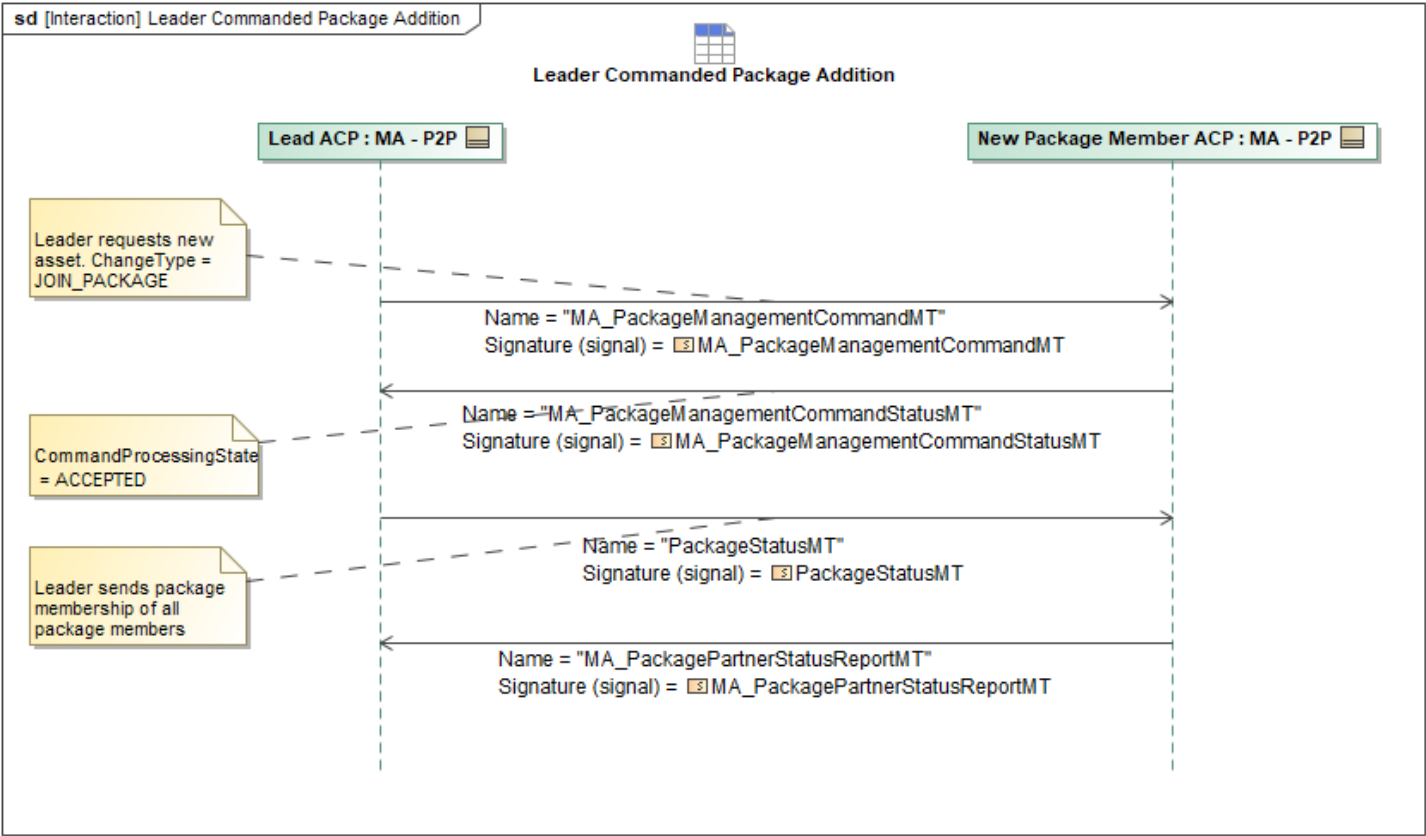


Figure 9-98: Leader Commanded Package Addition

Table 9-98: Leader Commanded Package Addition

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackageManagementCommandMT: MA_PackageManagementCommandMT	None
MA_PackageManagementCommandStatusMT: MA_PackageManagementCommandStatusMT	None
MA_PackagePartnerStatusReportMT: MA_PackagePartnerStatusReportMT	None
PackageStatusMT: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.6 Maximum Consensus Leader Election

Mission Use Case: Max Consensus Leader Election

This leader election method is optional and implementers are expected to implement at least one of the leader election methods. This election method supports both a *StaticLeadershipFitnessScore* or a *DynamicLeadershipFitnessScore* based on the selected *PackageLeadershipFitnessScoreDeterminationMethod*. Either score can be used as the “primary fitness score” (further detailed in this section). If the *DynamicLeadershipFitnessScore* is used as the primary fitness score, the *StaticLeadershipFitnessScore* may be used as the “secondary fitness

score” rather than the *SystemID*

This interaction describes how ACPs would use the maximum consensus algorithm to reach a team consensus on which ACP has the maximum leadership fitness score, if the Maximum Consensus Leader Election method is chosen as the method to determine consensus on the ACP leader. Once the algorithm begins, each ACP's fitness score remains constant for the duration of the election. For the Maximum Consensus Method to Leader Election, each ACP initially sends out its primary leadership fitness score to its neighbors. Every ACP will do this by setting its maximum leadership fitness score equal to its primary leadership fitness score in *MA_LeaderUpdateRequestMT* with the *RequestState* set to *NEW*, the *Status* set to *UNSPECIFIED*, the *StaticLeadershipFitnessScore* and *DynamicLeadershipFitnessScore* (if applicable). Providing its *AssociatedSystemID* as the secondary leadership fitness score in the *StaticLeadershipFitnessScore* is optional. Every ACP updates its tracked maximum leadership fitness score, if it is smaller than the largest maximum leadership fitness scores it has received from its in-neighbors. Optionally, each ACP keeps track of the *AssociatedSystemID* that corresponds to that new maximum leadership score. Any ties of primary fitness scores are broken via the *AssociatedSystemID*. Every ACP sends *only* its updated, tracked maximum leadership fitness score to its out-neighbors in *MA_LeaderUpdateRequestMT*. Every ACP sends its updated maximum leadership fitness score *only* when it has changed. Optionally, every ACP includes the max fitness score's corresponding *AssociatedSystemID* in the *StaticLeadershipFitnessScore*. This will be repeated for a pre-defined amount of time. The ACP with the primary fitness score equal to the tracked maximum leadership fitness score (and optionally the *AssociatedSystemID* if provided) nominates itself the leader by sending a nomination *MA_LeaderUpdateRequestMT* with the primary *LeadershipFitnessScore* and the secondary fitness score in the *AssociatedSystemID* field to all other ACPs. There may be multiple ACPs nominating themselves leaders when the algorithm terminates before it has converged, there are message dropouts, or there is a tie on the primary leadership fitness score. All other ACPs send a reply message accepting the nomination for the leader with *MA_LeaderUpdateRequestStatusMT*. When there are multiple ACPs declaring themselves leaders, ACPs will only accept the one with the largest primary leadership fitness score and largest secondary leadership fitness score (unique *AssociatedSystemID* in *StaticLeadershipFitnessScore*) in the case there is a tie on the primary leadership fitness score. Other nomination requests will be rejected by sending *MA_LeaderUpdateRequestStatusMT*. The ACP that received acceptance from the majority of ACPs will send a notification to all other ACPs of its leader nomination. Once the new leader ACP sends a notification of its leader nomination, any ACPs that had accepted the wrong ACP leader nomination previously will be overridden.

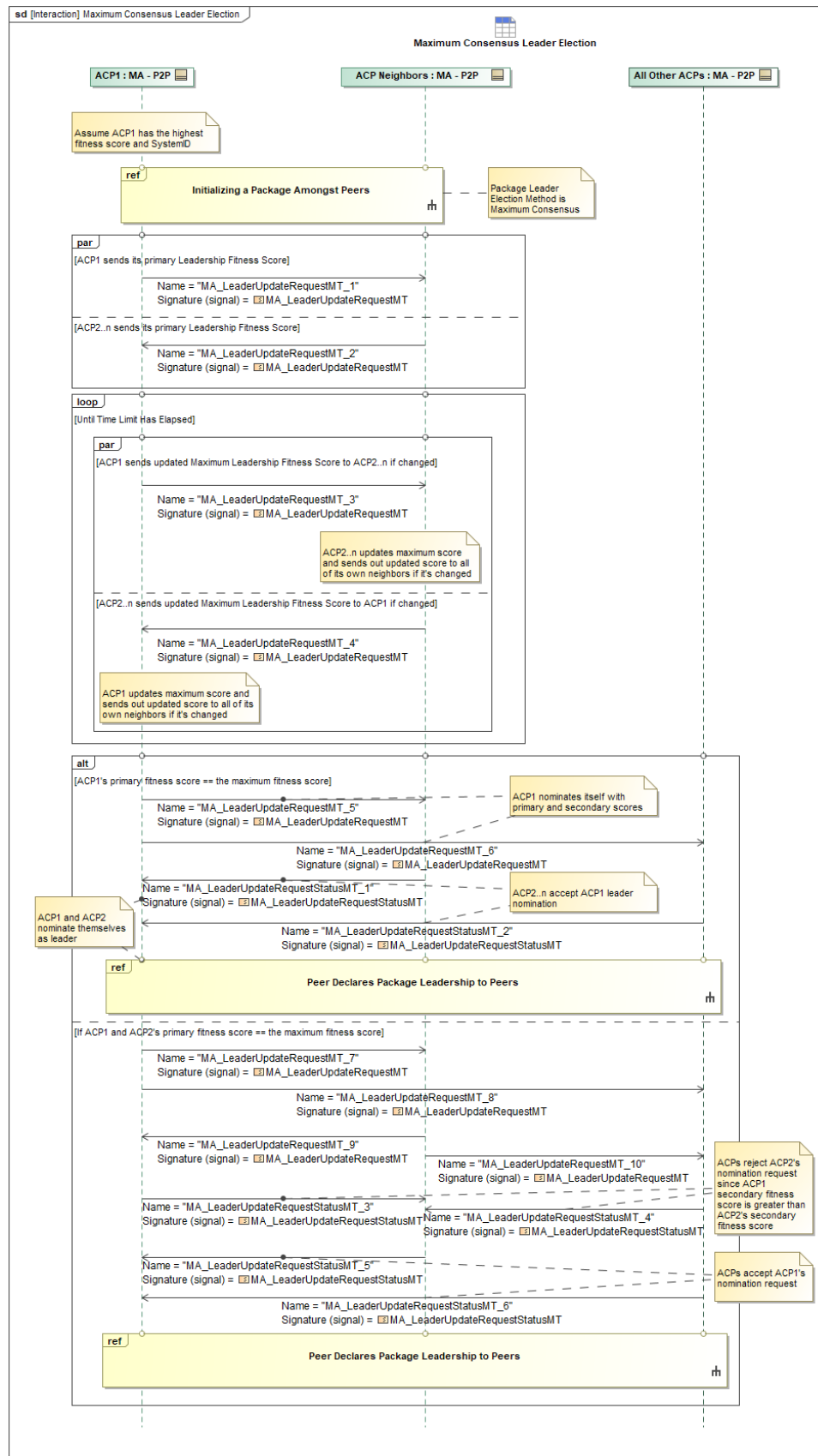


Figure 9-99: Maximum Consensus Leader Election

Table 9-99: Maximum Consensus Leader Election

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeaderUpdateRequestMT_1: MA_LeaderUpdateRequestMT	None
MA_LeaderUpdateRequestMT_2: MA_LeaderUpdateRequestMT	None
MA_LeaderUpdateRequestMT_3: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_4: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_5: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_6: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_7: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_8: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_9: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestMT_10: MA_LeaderUpdateRequestMT	StaticLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType
MA_LeaderUpdateRequestStatusMT_1: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_2: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_3: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_4: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_5: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_6: MA_LeaderUpdateRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.7 Notify Peers of Package Formation Command from Unauthorized C2

This is the peer interaction component for the off-nominal use case where an unauthorized C2 operator attempts to create or initialize a package including the ACP. In response to receiving *MA_PackageManagementCommandMT* from an unauthorized C2 operator, the Commanded Peer ACP will send *MA_PackageManagementCommandStatusMT* to all other Peers (members of the commanded package) with *CommandProcessingState* of *REJECTED* and

CommandProcessingStateReason of *INELIGIBLE_CONTROL_SOURCE*. This provides notification to the Peers that package formation attempt has failed.

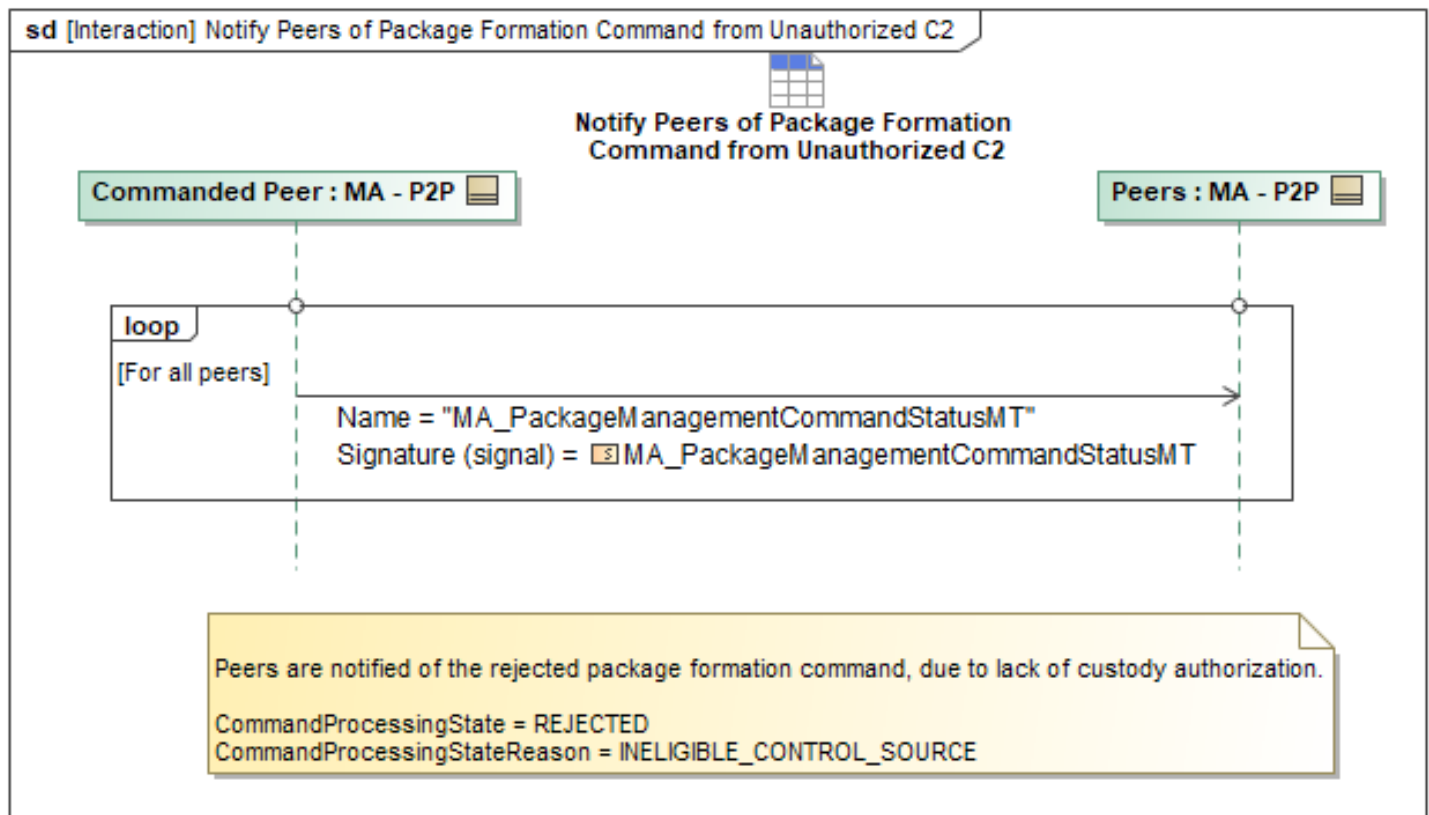


Figure 9-100: Notify Peers of Package Formation Command from Unauthorized C2

Table 9-100: Notify Peers of Package Formation Command from Unauthorized C2

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackageManagementCommandStatusMT: MA_PackageManagementCommandStatusMT	CommandProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.8 Off Nominal Leader Election

Mission Use Case: Bully Leader Election, Static Fitness Score Leader Election, Max Consensus Leader Election

The First and Second Off-Nominal Case are applicable if the Bully Leader Election Method, Maximum Consensus Leader Election, or the Static Fitness Score Leader Election Method is implemented.

This is the peer interaction describing two off-nominal cases for leader election, which may occur at the point when the ACP with highest fitness score would be declaring itself leader. Refer to the Algorithm Description section within the documentation of the "Bully Leader Election" sequence for a generalized description in the case of additional failures (e.g. if ACP 2 also fails during the first off-nominal case).

First Off-Nominal Case: Best Leader Candidate Fails During Election

If ACP 3, which has the highest fitness score, fails before declaring its leadership, the ACPs wait for the election timeout. Assume that ACP 1 reaches its timeout first. At that point, ACP 1 should send *MA_LeaderUpdateRequestMT_1* with *ProposedLeaderID* within the *Request* field set to ACP 2's System ID (since ACP 2 has the second-highest fitness score) and the *Status* field set to *NOMINATE*. ACP 2 responds with *MA_LeaderUpdateRequestStatusMT_1* with *RequestProcessingState* of *PROCESSING*. Once ACP 2 reaches its election timeout, it sends the ACP peers *MA_LeaderUpdateRequest_2-3* with the *Status* field set to *NOMINATE* and *ProposedLeaderID* within the *Request* field set to its own System ID. Note that ACP 3 will not receive this message since it is undergoing a communications failure. All connected ACPs (in this case ACP 1), respond with *MA_LeaderUpdateRequestStatusMT_2*, with *RequestProcessingState* of *COMPLETED*. Since a majority of connected ACPs accepted the request, ACP 2 declares leadership via "Peer Declares Package Leadership to Peers."

Second Off-Nominal Case: Peer Communications Failure

In the case where ACP 2 undergoes a communications failure and doesn't receive ACP 3's leader declaration, it may try to incorrectly assume leadership. ACP 3 performs the "Peer Declares Package Leadership to Peers" process, sending *PackageMT_1*, and *PackageStatusMT_1* messages which are received by ACP 1. The *PackageMT_2*, and *PackageStatusMT_2* messages are not received by ACP 2 due to its communication failure. ACP 2 waits for the election timeout to occur, and then nominates itself as leader by sending *MA_LeaderUpdateRequestMT_4-5* with the *Status* field set to *NOMINATE* and *ProposedLeaderID* within the *Request* field set to its own System ID. ACPs 1 and 3 respond by rejecting this command, sending *MA_LeaderUpdateRequestStatusMT_3-4* with *RequestProcessingState* of *REJECTED*. Regardless of whether ACP 2 receives these rejection messages, it will not assume leadership.

sd [Interaction] Off Nominal Leader Election

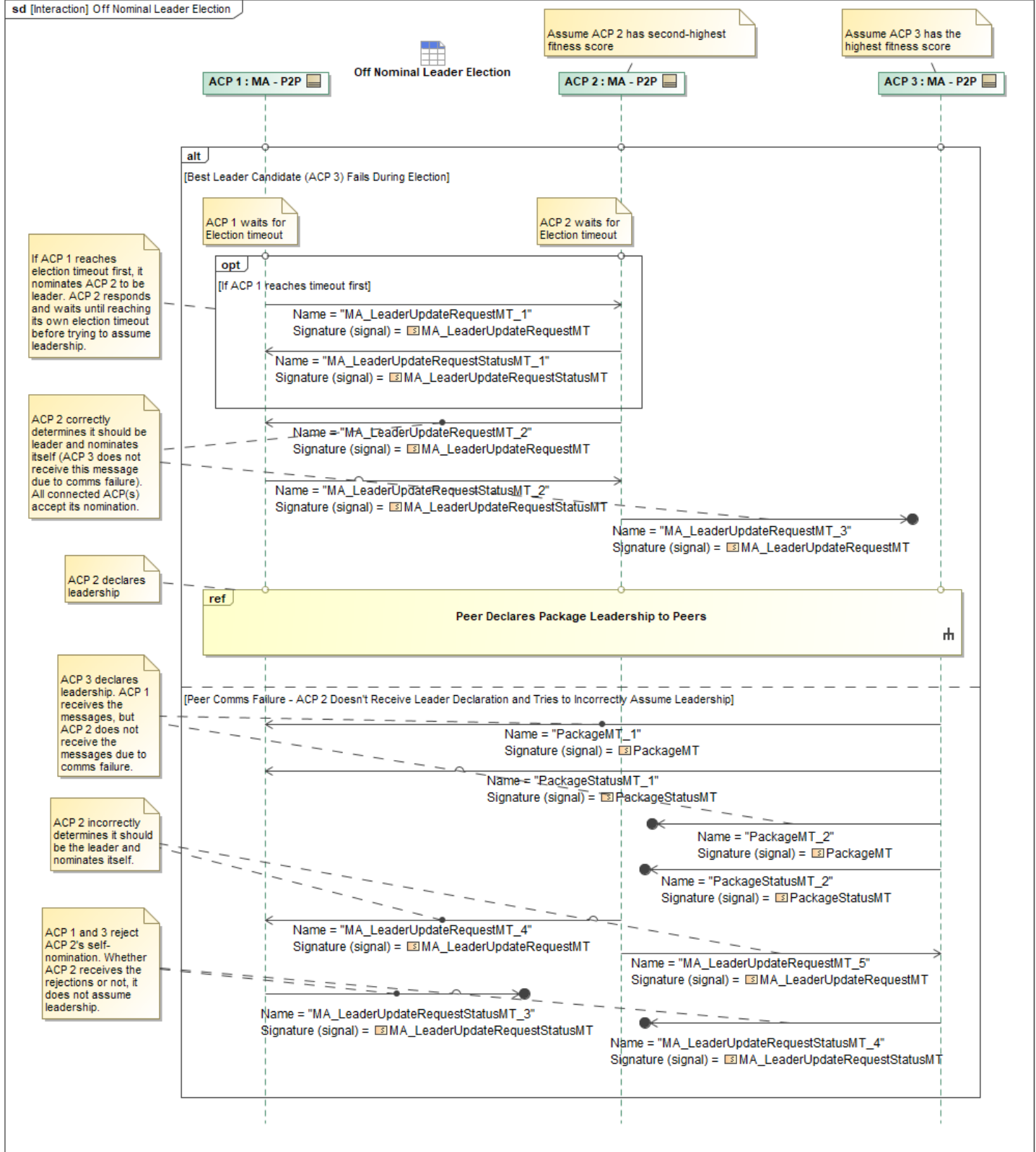


Figure 9-101: Off Nominal Leader Election

Table 9-101: Off Nominal Leader Election

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeaderUpdateRequestMT_1: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_2: MA_LeaderUpdateRequestMT	None
MA_LeaderUpdateRequestMT_3: MA_LeaderUpdateRequestMT	None
MA_LeaderUpdateRequestMT_4: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_5: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestStatusMT_1: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_2: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_3: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_4: MA_LeaderUpdateRequestStatusMT	None
PackageMT_1: PackageMT	LeadSystemID: SystemID_Type
PackageMT_2: PackageMT	LeadSystemID: SystemID_Type
PackageStatusMT_1: PackageStatusMT	None
PackageStatusMT_2: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.9 Peer Declares Package Leadership to Peers

This is the peer interaction where a newly elected or assigned leader declares its leadership to the followers in the package. Note that this is a sub-process within Leader Election as a whole, and this interaction is referenced by other sequences after a new leader is determined. For each follower within the package, the new leader sends *PackageMT*, with the *LeadSystemID* field containing its own System ID. The leader sends *PackageStatusMT*, containing the status of all followers. Each follower reports status to the new leader by sending *MA_PackagePartnerStatusReportMT*.

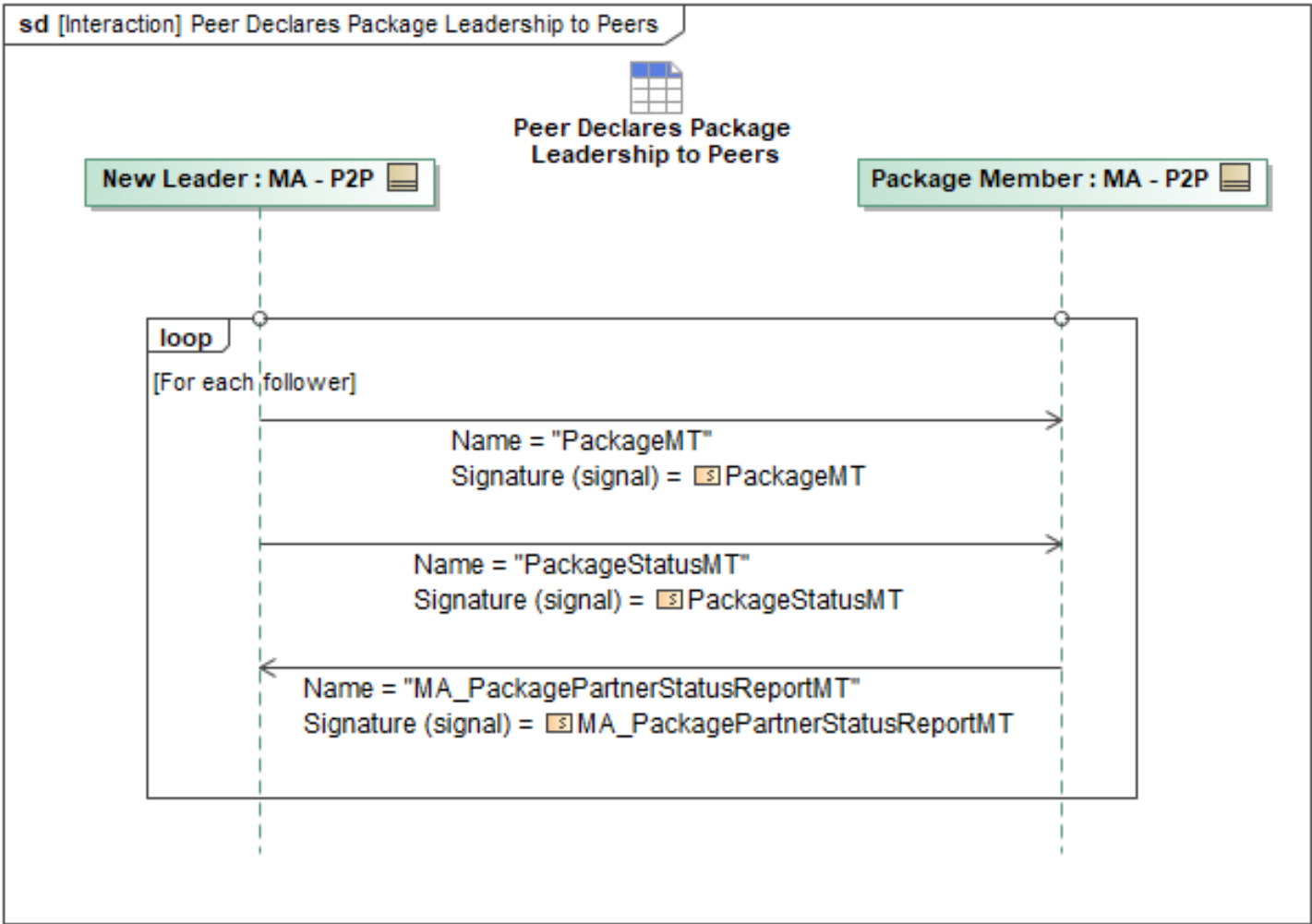


Figure 9-102: Peer Declares Package Leadership to Peers

Table 9-102: Peer Declares Package Leadership to Peers

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackagePartnerStatusReportMT: MA_PackagePartnerStatusReportMT	None
PackageMT: PackageMT	LeadSystemID: SystemID_Type
PackageStatusMT: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.10 Peer Joining a Package

This peer interaction describes the process of an ACP joining an existing package. The Leader, or an ACP that is a current member in a package, sends *MA_PackageManagementCommandMT* to the new asset, with the *ChangeType* field set to *JOIN_PACKAGE*, and the *SystemID* field populated with the new asset's ID. The new asset responds by sending the requesting Leader or requesting ACP *MA_PackageManagementCommandStatusMT* with *CommandProcessingState* of *ACCEPTED* to

indicate the ACP has joined the package. To all followers (including the newly joined ACP), the Leader sends *PackageStatusMT*, indicating the updated membership of the package. The newly-joined ACP (and all followers) will send *MA_PackagePartnerStatusReportMT* periodically to report status to the Leader.

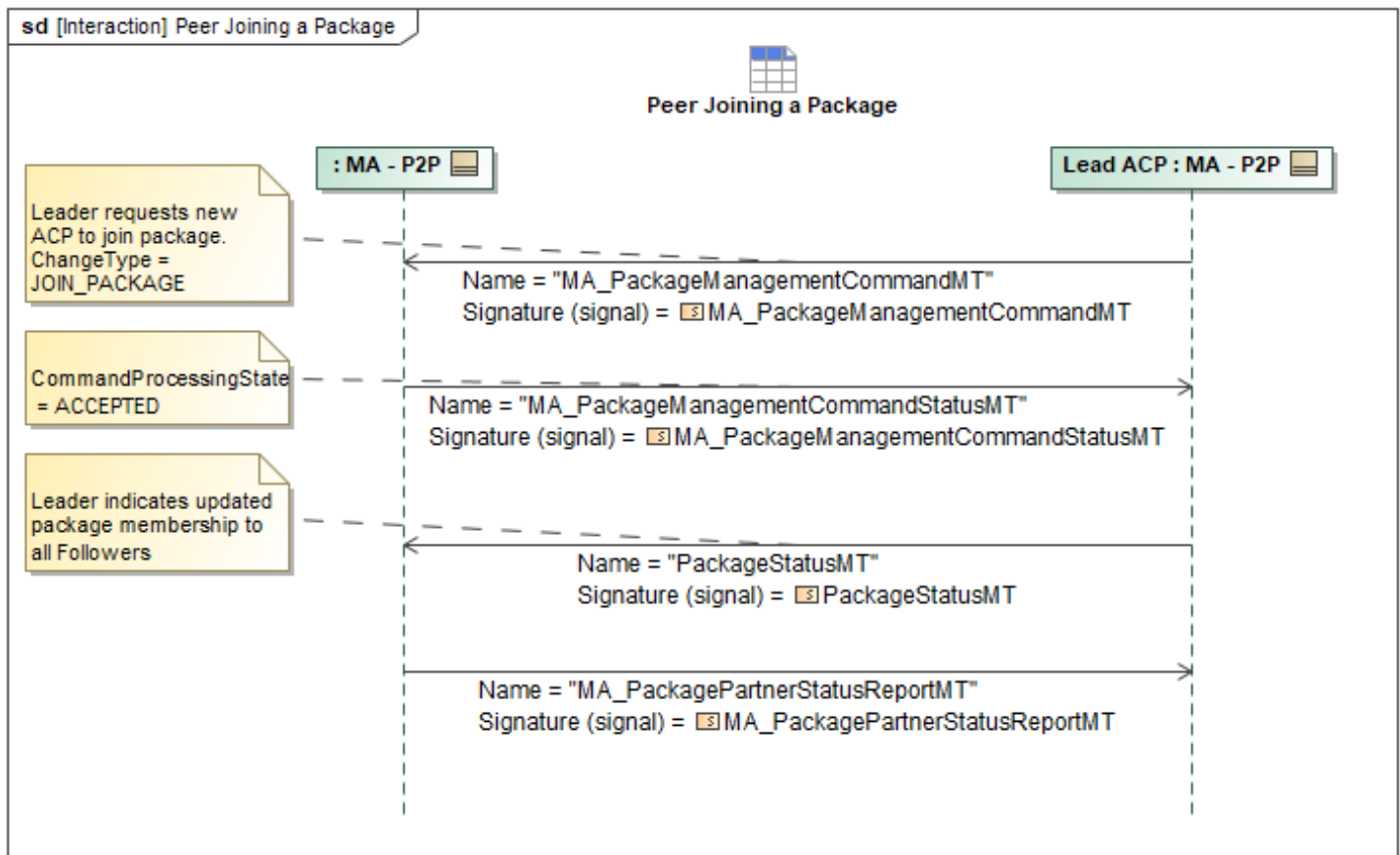


Figure 9-103: Peer Joining a Package

Table 9-103: Peer Joining a Package

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackageManagementCommandMT: MA_PackageManagementCommandMT	None
MA_PackageManagementCommandStatusMT: MA_PackageManagementCommandStatusMT	None
MA_PackagePartnerStatusReportMT: MA_PackagePartnerStatusReportMT	None
PackageStatusMT: PackageStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.11 Raft Leader Election

Mission Use Case: Raft Leader Election

This leader election method is optional and implementors are expected to implement at least one of the leader election methods.

This describes the peer interaction for the Raft Leader Election process. If the Raft Leader Election method is chosen as the method to the package leader, this method will allow for dynamic leader election that is robust, well-tested, and easily comprehensible. As part of the Raft Consensus Algorithm, the Raft Leader Election uses a heartbeat mechanism and a randomized heartbeat timeout to maintain leaders (randomized heartbeat timeout ensures that split votes are rare and quickly resolved). There can be at most one Leader per term (Raft divides time into terms of arbitrary length, and terms are numbered with consecutive integers). Each ACP will maintain the current term value. If a peer has a later term, then the ACP will update the term and revert to the Follower state.

Each Follower expects to receive regular heartbeats from the Leader. An ACP remains in the Follower state as long as it receives a valid heartbeat (*MA_RaftAppendEntriesRequestMT*) from a leader or *VoteRequest* (*MA_RaftVoteRequestMT*) from a candidate. If a Follower receives no communication over a period of time called the election timeout, then it assumes there is no viable leader and begins an election to choose a new leader.

On package formation, since all ACPs are initialized as a Follower, the ACPs will timeout and enter the Candidate state (randomized heartbeat timeout). In the Candidate state, the ACP will send *MA_RaftVoteRequestMT* to all of the other peers with an incremented term and a vote for itself as leader (CandidateID). The ACPs, upon receipt of *MA_RaftVoteRequestMT*, will compare the term sent from the Candidate. If the term is greater than the term of the ACP, the ACP will respond with a positive vote (*MA_RaftVoteRequestStatus*) for the Candidate to become leader (note: log replication is currently not implemented as part of the Raft Leader Election method).

While waiting for votes, a candidate may receive a *MA_RaftAppendEntriesRequestMT* from another ACP claiming to be leader. If the proclaimed leader's term (included in its *MA_RaftAppendEntriesRequestMT*) is at least as large as the candidate's current term, then the candidate recognizes the leader as legitimate and returns to follower state. If the term in the *MA_RaftAppendEntriesRequestMT* is smaller than the candidate's current term, then the candidate rejects the *MA_RaftAppendEntriesRequestMT* and continues the leader election in candidate state. When the Candidate ACP has received the majority of votes from Peers in the package, then the Candidate will enter the Leader state and begin sending periodic heartbeats to maintain package leadership (*MA_RaftAppendEntriesRequestMT*). If the Candidate ACP did not receive the majority of votes from Peers in the package such as the case of a split vote between multiple Candidate ACPs, the Candidate ACP(s) will start a new election by sending *MA_RaftVoteRequestMT* (the election timeout is randomized). This process repeats until a Leader has been elected.

The following section is optional and is left open for interpretation by implementers:

In instances where the Leader is synchronizing log entries with the package, the Leader will send *MA_RaftAppendEntriesRequestMT* with *MA_RaftLogEntry* populated and the Followers will respond with *MA_RaftAppendEntriesRequestStatusMT* to confirm successful log commit. Within *MA_RaftLogEntry*, the *Command* field will be populated with key/value pairs that will be open for implementers to determine.

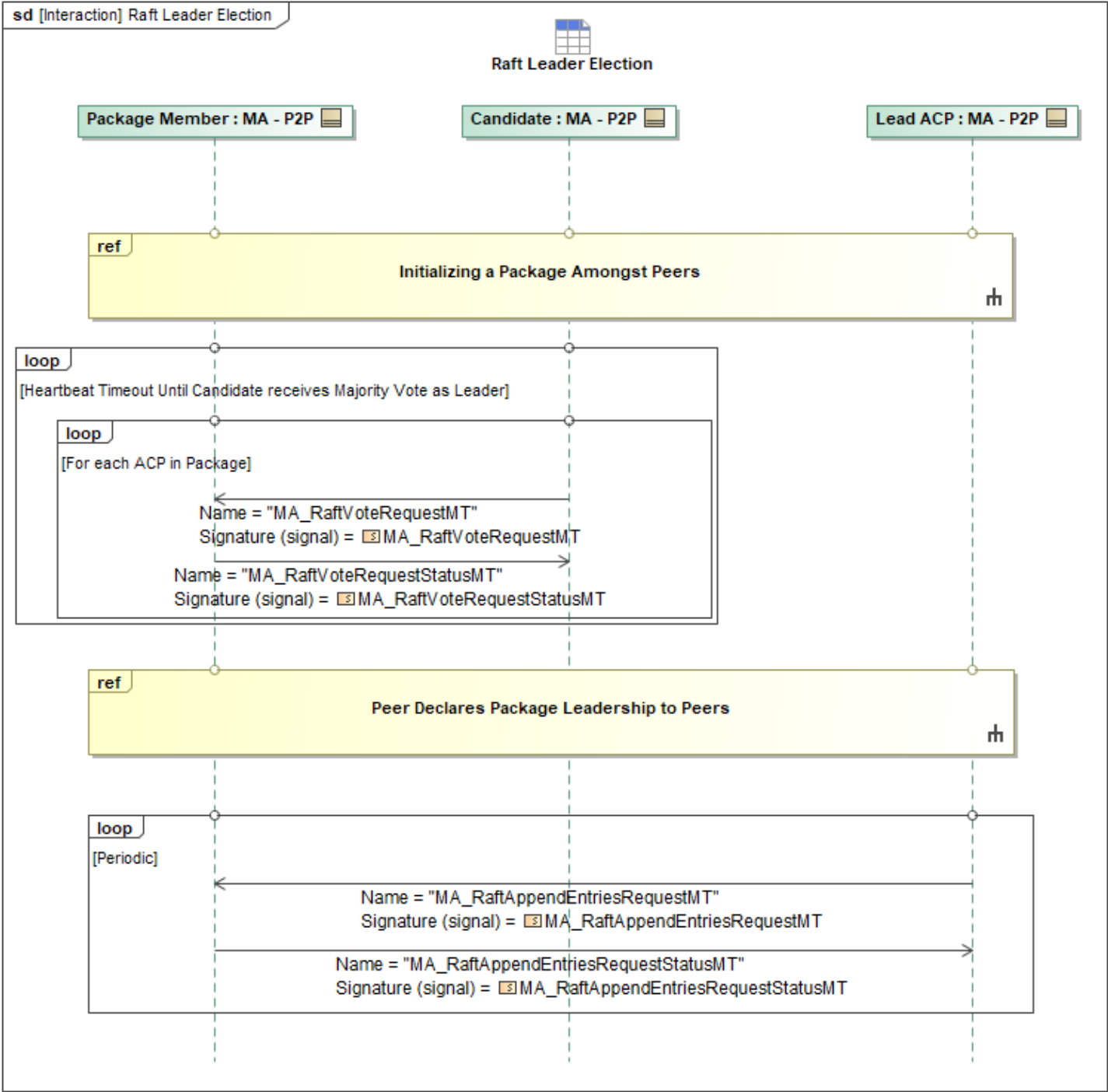


Figure 9-104: Raft Leader Election

Table 9-104: Raft Leader Election

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RaftAppendEntriesRequestMT: MA_RaftAppendEntriesRequestMT	None
MA_RaftAppendEntriesRequestStatusMT: MA_RaftAppendEntriesRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RaftVoteRequestMT: MA_RaftVoteRequestMT	None
MA_RaftVoteRequestStatusMT: MA_RaftVoteRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.12 Share DLFS with Peers

The Share DLFS with Peers sequence diagram specifies the interaction between two ACP peers which is a required interaction ONLY when *MA_LeadershipFitnessScoreDeterminationMethod* equals *DYNAMIC* or *USER_DEFINED*, and allows an ACP to share its Dynamic Leadership Fitness Score with other ACP peers (which may trigger a new leader election depending on implementation). One ACP peer (MA) sends a *MA_PackagePartnerStatusReportMT* to other ACP peers (MA) with the *DynamicLeadershipFitnessScore* attribute set to reflect their current (most recently calculated) dynamic leadership fitness score.

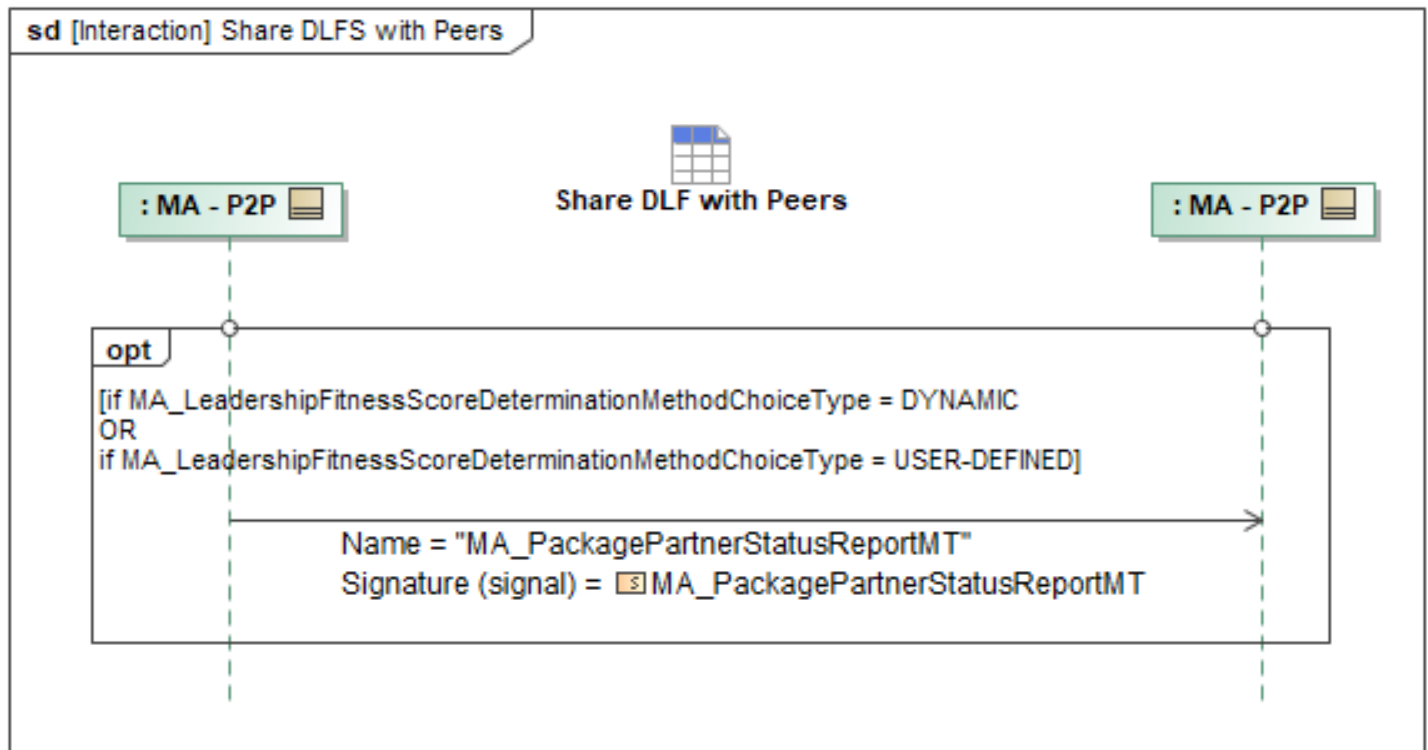


Figure 9-105: Share DLFS with Peers

Table 9-105: Share DLFS with Peers

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PackagePartnerStatusReportMT: MA_PackagePartnerStatusReportMT	DynamicLeadershipFitnessScore: MA_PartnerLeadershipFitnessScoreType

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.11.13 Static Fitness Score Leader Election

Mission Use Case: Static Fitness Score Leader Election

This leader election method is optional and implementers are expected to implement at least one of the leader election methods.

This describes the peer interaction for the Static Fitness Score Leader Election process. This election method can be used when fitness scores are static (unchanging), based on the selected *PackageLeadershipFitnessScoreDeterminationMethod* or do not need to be shared at the time of election. This section describes using this method as the first election after package initialization, when leader communications are lost, and in other off-nominal cases.

For this interaction, assume ACP 3 has the highest fitness score. Ties are broken by highest tail number, which is found via the heartbeat system status messages in the Peer Heartbeat and Send Vehicle State to Peer interactions.

First Election after Package Initialization:

Once ACP peers determine the package has completed initialization via "Initializing a Package Amongst Peers," their election timers begin. In the nominal case, ACP 3 recognizes it has the highest *StaticLeadershipFitnessScore* and declares its leadership via "Peer Declares Package Leadership to Peers." If ACP 3 fails to declare leadership before ACP 1 or 2's election timeout is reached, ACP 1 or 2 should assign leadership to ACP 3 by sending *MA_LeaderUpdateRequestMT_1* or *MA_LeaderUpdateRequestMT_2* with the *Status* field set to *ASSIGN* and ACP 3's System ID populated in *ProposedLeaderID* of the *Request* field.

Otherwise (e.g. Lost Leader Scenario):

In the case where Package Initialization did not occur directly before the election (for instance, if the leader lost communications), the package should gain concurrence that a leader election will be held. This allows for the handling of various off-nominal communication scenarios.

Once ACP peers determine an election is underway (either by initiating the election or receiving *MA_LeaderUpdateRequestMT* as part of the election), their election timers begin. In a typical case, ACP 3 will recognize an election is required and that it has the highest fitness score, and will nominate itself for leadership by sending *MA_LeaderUpdateRequestMT_5* and *MA_LeaderUpdateRequestMT_6* to ACP 2 and ACP 1, respectively. This message will have the *Status* field set to *NOMINATE* and ACP 3's System ID populated in *ProposedLeaderID* of the *Request* field.

If ACP 3 fails to nominate itself for leadership before ACP 1 or 2's election timeout is reached, ACP 1 or 2 should nominate ACP 3 by sending *MA_LeaderUpdateRequestMT_3* or *MA_LeaderUpdateRequestMT_4* with the *Status* field set to *NOMINATE* and ACP 3's System ID populated in *ProposedLeaderID* of the *Request* field. In recognition of the nomination, ACP 3 sends *MA_LeaderUpdateRequestMT_5* and *MA_LeaderUpdateRequestMT_6* to ACP 2 and ACP 1, respectively, so all ACPs are aware of its nomination

ACPs 2 and 1 respond with *MA_LeaderUpdateRequestStatusMT_1* and *MA_LeaderUpdateRequestStatusMT_2*, respectively, with *RequestProcessingState* of *COMPLETED* to indicate acceptance of ACP 3's leader nomination. ACP 3 then declares leadership via "Peer

Declares Package Leadership to Peers."

Off-Nominal:

Other off-nominal cases of leader election are illustrated in the referenced sub-diagram. These are handled in the same way as the off-nominal cases within Bully Leader Election method.

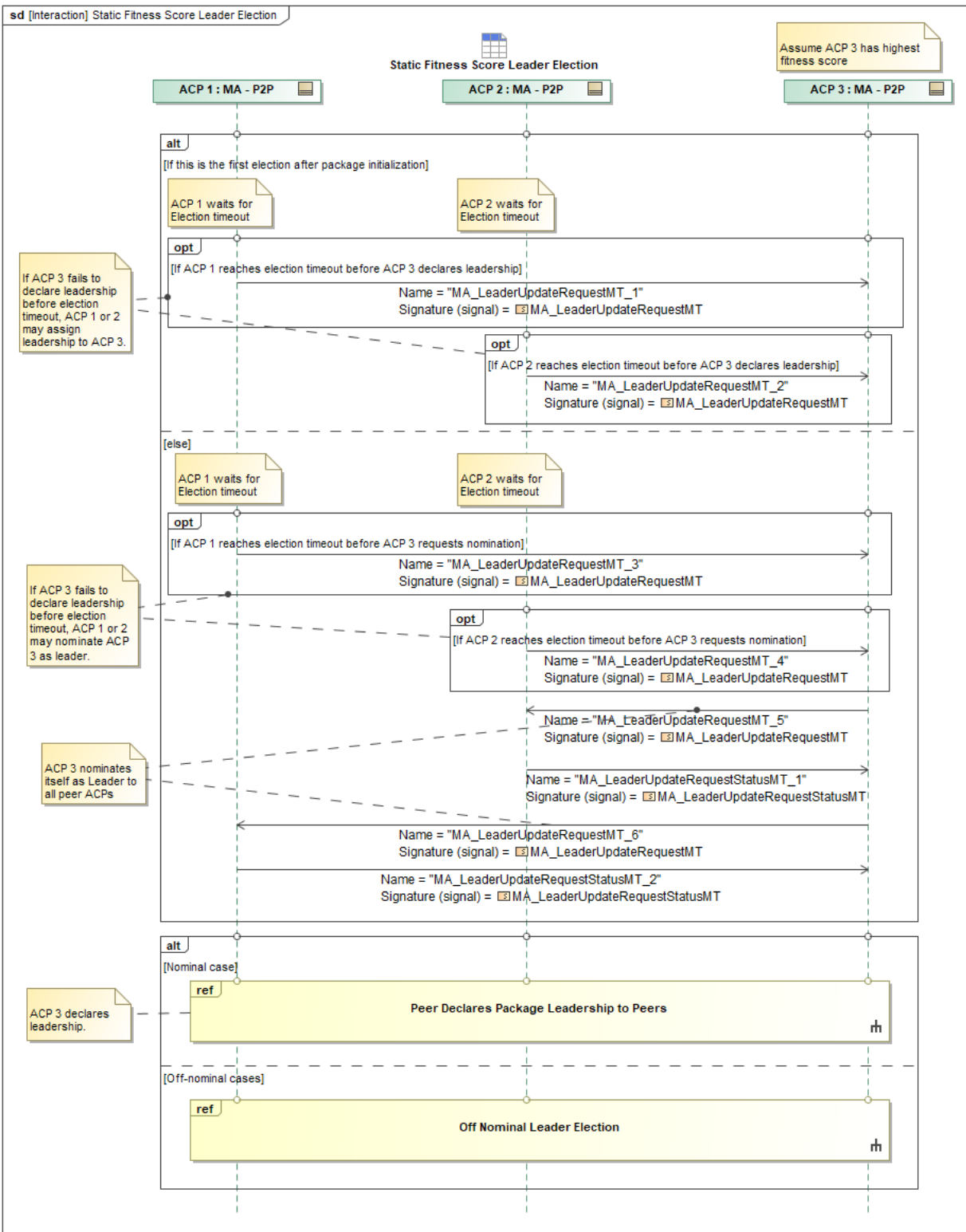


Figure 9-106: Static Fitness Score Leader Election

Table 9-106: Static Fitness Score Leader Election

Message Name (Name:Type)	Required Fields (Name:Type)
MA_LeaderUpdateRequestMT_1: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_2: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_3: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_4: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_5: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestMT_6: MA_LeaderUpdateRequestMT	ProposedLeaderID: SystemID_Type
MA_LeaderUpdateRequestStatusMT_1: MA_LeaderUpdateRequestStatusMT	None
MA_LeaderUpdateRequestStatusMT_2: MA_LeaderUpdateRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.12 Peer Task Prioritization and Mission Context Package

9.2.12.1 Peer Acceptable Level of Risk Behaviors

9.2.12.1.1 Peer Provides Route Risk Assessment Results

Mission Use Case: ALR

The optional Peer Provides Route Risk Assessment Results sequence diagram specifies the interaction where an ACP provides a Peer ACP with risk assessment results for a Route Plan. This may occur if 1) A Peer ACP requests a Route Plan Threat Assessment or 2) Updated threat(s) trigger an ACP to assess the current Route Plan and deliver the results to a Peer ACP. This interaction assumes that the Route Plan to be assessed has been previously uploaded to the ACP.

In the first case, the Requestor MA will request a risk assessment for a Route Plan, sending *MA_AssessmentRequestMT* with a specified *AnalysisRouteID* and the *Parameters* of the *RouteThreatAssessment* field within the *Category* field populated. The Assessor MA may optionally report status on the request using *MA_AssessmentRequestStatusMT_1* to convey the processing state and/or estimated completion information. The Assessor MA will then deliver the risk assessment results with the *MA_AssessmentRequestStatusMT_2* message, populating the *Data* of the *RouteThreatAssessment* field within the *Assessment* field.

In the second case, MA internally triggers a risk assessment due to updated threats, rather than receiving a request to perform the assessment. Upon determining that the current Route Plan exceeds the specified Acceptable Level of Risk (ALR) setting, the ACP may attempt to generate a new Route Plan through the threat environment which complies with the ALR setting. (If the current Route Plan does not exceed the ALR setting, no risk assessment will be required, and this interaction

would not apply). After a new Route Plan is generated, the ACP sends *AssessmentMT* to the Peer ACP to inform of the risk assessment results, populating the *Data* of the *RouteThreatAssessment* field within the *Assessment* field of this message. The message will have a populated *AnalysisRouteID* to identify the Route Plan for which the assessment applies. If a Route Plan which complies with the ALR setting is not generated or able to be achieved, the *MA_AssessmentMT* message will convey this by populating the *Achievability* field within *AchievabilityAssessment* as *NOT_ACHIEVABLE*. After, if the ACP receiving the assessment is the Lead ACP, it may forward the risk assessment results to C2 as shown in the “Provide Risk Assessment Results” interaction in the C2 profile.

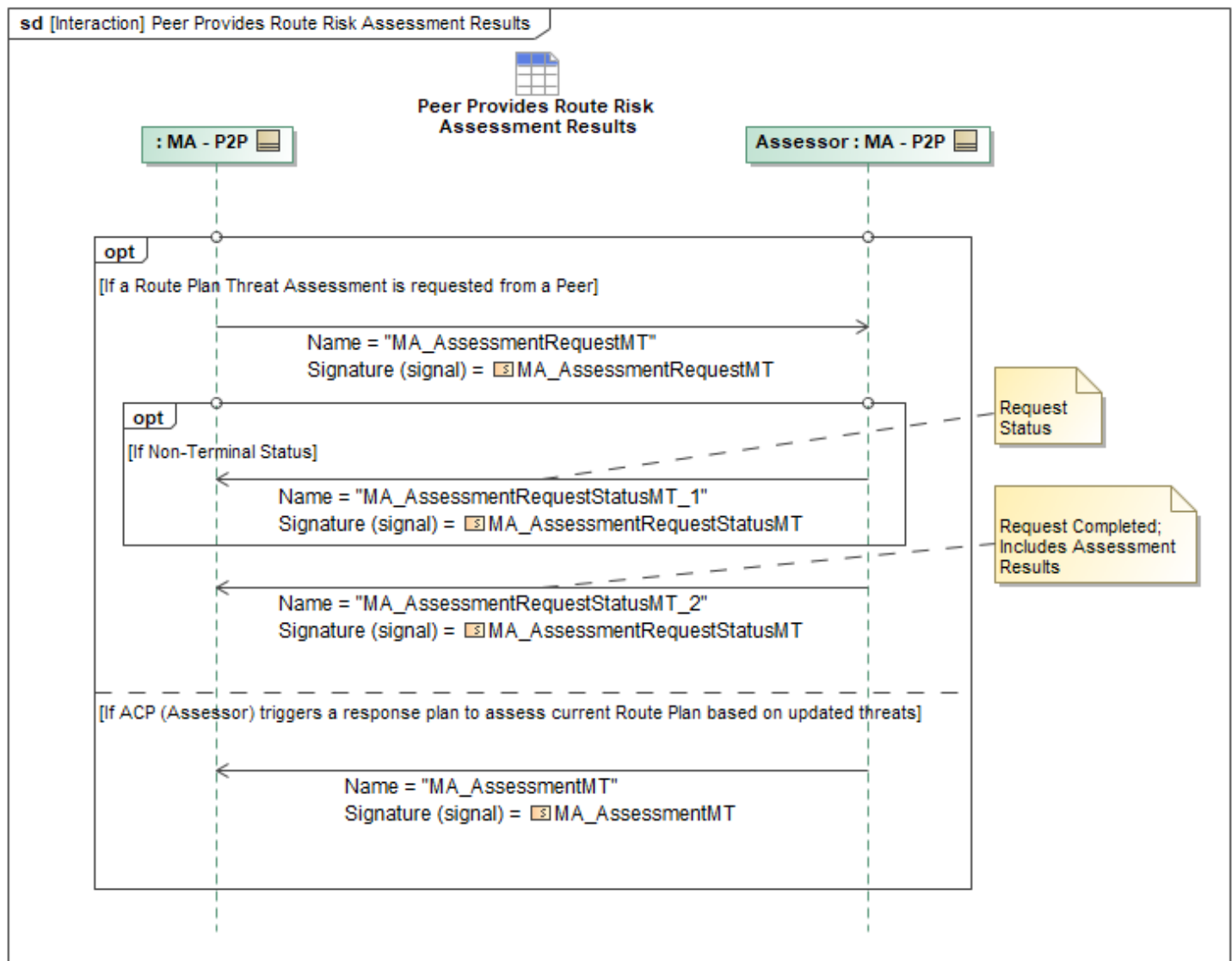


Figure 9-107: Peer Provides Route Risk Assessment Results

Table 9-107: Peer Provides Route Risk Assessment Results

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentMT: MA_AssessmentMT	AnalysisRouteID: AnalysisRouteID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentRequestMT: MA_AssessmentRequestMT	AnalysisRouteID: AnalysisRouteID_Type
MA_AssessmentRequestStatusMT_1: MA_AssessmentRequestStatusMT	None
MA_AssessmentRequestStatusMT_2: MA_AssessmentRequestStatusMT	Assessment: AssessmentType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.12.1.2 Update Peer Acceptable Level of Risk

Mission Use Case: ALR

The Update Peer Acceptable Level of Risk (ALR) sequence diagram below specifies the interaction of the lead ACP commanding its follower ACPs to update its ALR settings. First, the lead ACP sends a *MA_AcceptableLevelOfRiskCommandMT* message where the *AppliesToID* field is defined with the *SystemID* of the follower ACP. The *MA_AcceptableLevelOfRiskCommandMT* message defines an *AcceptableRiskSetting*, which allows for the ALR to be set as *HIGH*, *MEDIUM*, *LOW*, or *NONE* at the package or vehicle level. This ALR setting applies to all tasks and actions for the vehicle or package. It also defines a *SurvivabilityRiskLevelID* which allows for a *RiskSetting* to be populated to describe what *HIGH*, *MEDIUM*, and *LOW* mean with respect to the *MaximumContinuousExposure* for a given Vulnerability Type. Any new *MA_AcceptableLevelOfRiskCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_AcceptableLevelOfRiskCommandMT* is sent with the *CommandState* set to *CANCEL*, this will cancel the active command.

In response to the command, the follower ACPs shall send *MA_AcceptableLevelOfRiskCommandStatusMT_1* to the lead ACP, with the *AppliesToID* field set to their System ID and the *CommandProcessingState* field set to *RECEIVED* to indicate receipt of the command. The Follower ACPs proceed to update their ownship ALR setting and send *MA_AcceptableLevelOfRiskCommandStatusMT_2* to the lead ACP with the *AppliesToID* field set to their System ID and the *CommandProcessingState* set to *ACCEPTED*, to indicate they have completed the update to the new ALR setting. The Follower ACPs will then send *MA_AcceptableLevelOfRiskStatusMT* to the Lead ACP to provide status on their currently activated ALR setting and Survivability Risk Level, using the *AcceptableRiskSetting* and *SurvivabilityRiskLevelID* fields, and populating their System ID in the *AppliesToID* field.



Figure 9-108: Update Peer Acceptable Level of Risk

Table 9-108: Update Peer Acceptable Level of Risk

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AcceptableLevelOfRiskCommandMT: MA_AcceptableLevelOfRiskCommandMT	None
MA_AcceptableLevelOfRiskCommandStatusMT _1: MA_AcceptableLevelOfRiskCommandStatusMT	None
MA_AcceptableLevelOfRiskCommandStatusMT _2: MA_AcceptableLevelOfRiskCommandStatusMT	None
MA_AcceptableLevelOfRiskStatusMT: MA_AcceptableLevelOfRiskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.12.1.3 Update Peer Survivability Risk Definition

Mission Use Case: ALR

The optional Update Peer Survivability Risk Definition sequence diagram specifies the interaction where the Lead ACP sends Follower ACPs updated Survivability Risk Definition data. Upon receiving a new Survivability Risk Definition from C2 (shown in "Update Survivability Risk Definition" interaction

in C2 profile), the Lead ACP will send *SurvivabilityRiskLevelMT* to all applicable follower ACPs with the *ObjectState* field set to *UPDATED* and the *RiskSetting* field populated. Unless one or a subset of Follower System IDs are specified in the *AppliesToSystemID* field, the Lead ACP will send the update to all package members (i.e. if *AppliesToSystemID* is unpopulated, it is sent to entire package). Note that this does not initiate activation of the Survivability Risk Definition (See “Update Peer Acceptable Level of Risk” for interaction which commands activation).

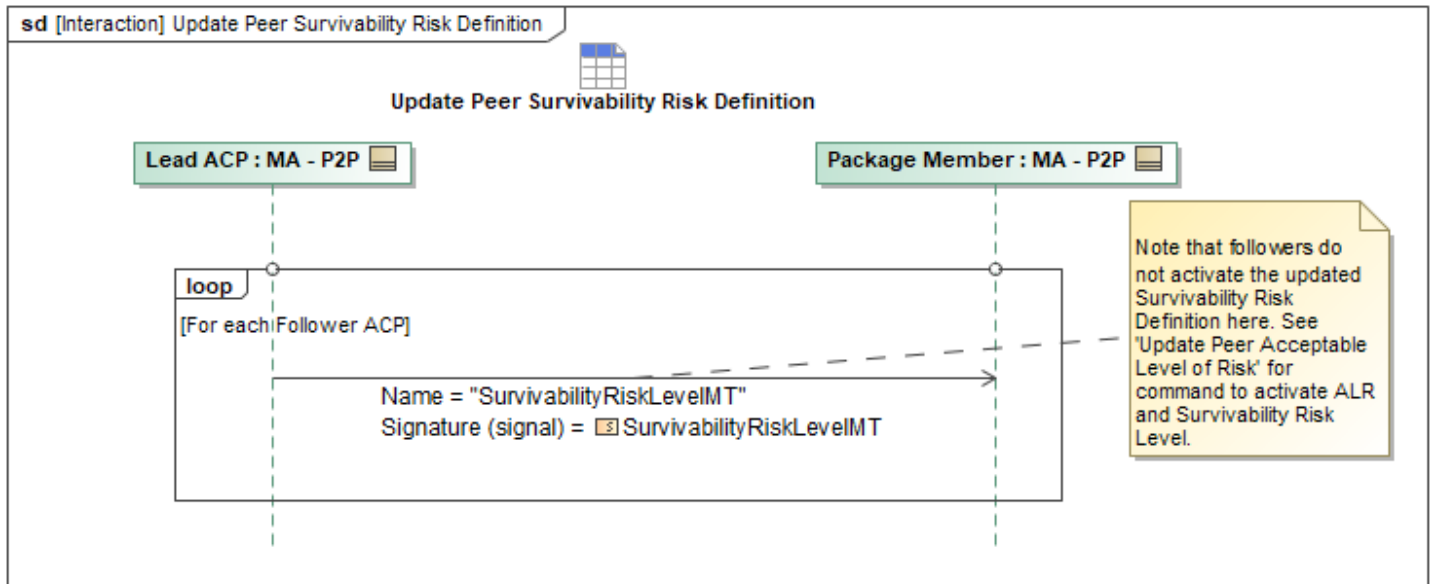


Figure 9-109: Update Peer Survivability Risk Definition

Table 9-109: Update Peer Survivability Risk Definition

Message Name (Name:Type)	Required Fields (Name:Type)
SurvivabilityRiskLevelMT: SurvivabilityRiskLevelMT	ObjectState: ObjectStateEnum AssociatedID: ID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.12.2 Peer Constraint Priority List Behaviors

9.2.12.2.1 Activate Peer Constraint Priority List

The Activate Peer Constraint Priority List (CPL) sequence diagram below specifies the interaction of the lead ACP commanding its follower ACPs to activate one of its stored CPL activation schema. First, the lead ACP sends a *MA_ConstraintPriorityListCommandMT* message where the *AppliesToID* field is defined with the *SystemID* of the follower ACP. The *MA_ConstraintPriorityListCommandMT* message shall specify the *ConstraintPriorityListActivationDataID* field containing the ID of the Constraint Priority List Activation Data message (or schema) that the lead ACP wishes to be active. Any new *MA_ConstraintPriorityListCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_ConstraintPriorityListCommandMT* is sent with the *CommandedState* set to *CANCEL*, this will cancel the active command and CPL activation schema.

In response to the command, the follower ACPs shall send *MA_ConstraintPriorityListCommandStatusMT_1* to the lead ACP, with the *AppliesToID* field set to

their System ID and the *CommandProcessingState* field set to *RECEIVED* to indicate receipt of the command. The follower ACPs shall then use the *ConstraintPriorityListActivationDataID* to query their onboard MP to retrieve the *MA_ConstraintPriorityListActivationDataMT* (see "Query Constraint Priority List Activation Data"). Then the follower ACPs will use the *MA_ConstraintPriorityListActivationDataMT* to query for all the CPL messages (see "Query Constraint Priority List"). Once the follower ACPs have finished querying, the follower ACPs shall send *MA_ConstraintPriorityListCommandStatusMT_2* to the lead ACP with the *AppliesToID* field set to their System ID and the *CommandProcessingState* set to *ACCEPTED*.

The follower ACPs shall use their location and the *MA_ConstraintPriorityListActivationDataMT* to determine which CPL ID should be activated (default or geozone based). Once doing so, the follower ACPs will send *MA_ConstraintPriorityListActivationStatusMT* to the lead ACP containing the *ConstraintPriorityListActivationDataID*, *ActiveConstraintPriorityListID*, and the *AppliesToID* field set to their System ID.

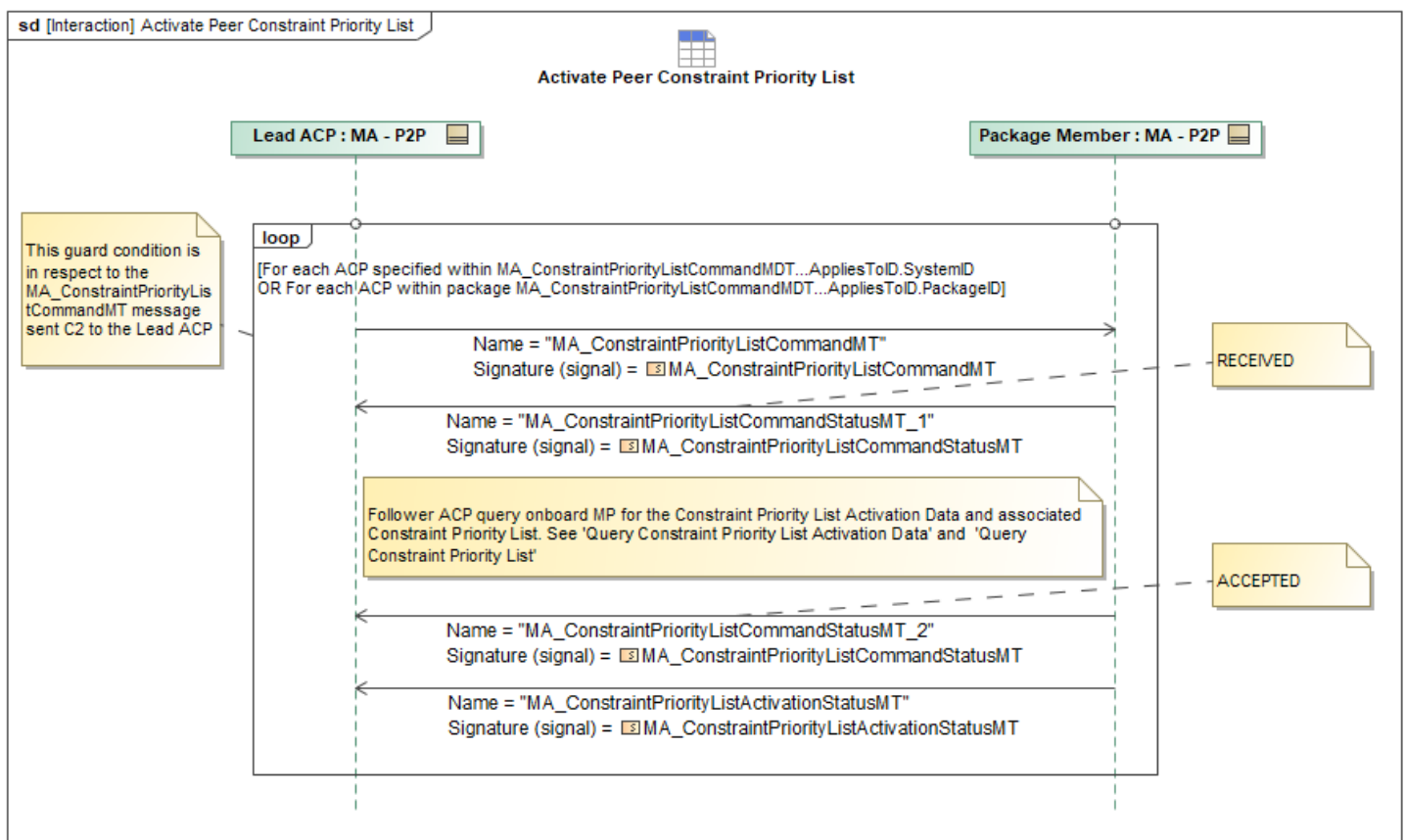


Figure 9-110: Activate Peer Constraint Priority List

Table 9-110: Activate Peer Constraint Priority List

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListActivationStatusMT: MA_ConstraintPriorityListActivationStatusMT	None
MA_ConstraintPriorityListCommandMT: MA_ConstraintPriorityListCommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ConstraintPriorityListCommandStatusMT_1: MA_ConstraintPriorityListCommandStatusMT	None
MA_ConstraintPriorityListCommandStatusMT_2: MA_ConstraintPriorityListCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.12.3 Peer Execution Priority List Behaviors

9.2.12.3.1 Activate Peer Execution Priority List

The Activate Peer Execution Priority List sequence diagram below specifies the interaction of the lead ACP commanding its follower ACPs to activate one of its stored EPL activation schema. First, the lead ACP sends an *MA_ExecutionPriorityListCommandMT* message where the *AppliesToID* field is defined with the *SystemID* of the follower ACP. The *MA_ExecutionPriorityListCommandMT* message shall specify the *ExecutionPriorityListActivationDataID* field containing the ID of the Execution Priority List Activation Data message (or schema) that the lead ACP wishes to be active. Any new *MA_ExecutionPriorityListCommandMT* will override any previously sent message, even if the *CommandID* has changed. When a *MA_ExecutionPriorityListCommandMT* is sent with the *CommandedState* set to *CANCEL*, this will cancel the active command and EPL activation schema

In response to the command, the follower ACPs shall send

MA_ExecutionPriorityListCommandStatusMT_1 to the lead ACP, with the *AppliesToID* field set to their System ID and the *CommandProcessingState* field set to *RECEIVED* to indicate receipt of the command. The follower ACPs shall then use the *ExecutionPriorityListActivationDataID* to query their onboard MP to retrieve the *MA_ExecutionPriorityListActivationDataMT* (see "Query Execution Priority List Activation Data"). Then the follower ACPs will use the *MA_ExecutionPriorityListActivationDataMT* to query for all the EPL messages (see "Query Execution Priority List"). Once the follower ACPs have finished querying, the follower ACPs shall send *MA_ExecutionPriorityListCommandStatusMT_2* to the lead ACP with the *AppliesToID* field set to their System ID and the *CommandProcessingState* set to *ACCEPTED*.

The follower ACPs shall use their location and the *MA_ExecutionPriorityListActivationDataMT* to determine which EPL ID should be activated (default or geozone based). Once doing so, the follower ACPs will send *MA_ExecutionPriorityListActivationStatusMT* to the lead ACP containing the *ExecutionPriorityListActivationDataID*, *ActiveExecutionPriorityListID*, and the *AppliesToID* field set to their System ID.

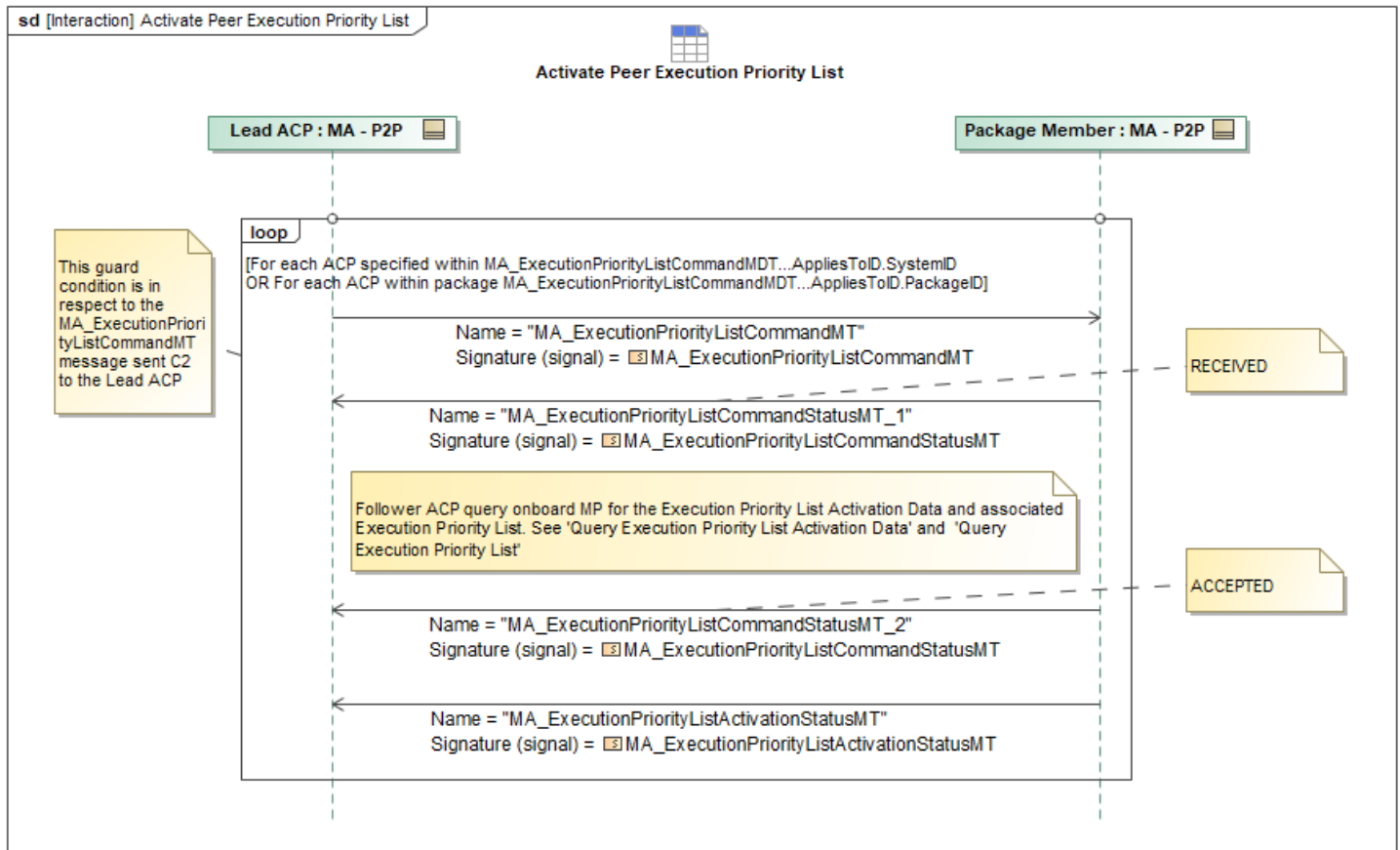


Figure 9-111: Activate Peer Execution Priority List

Table 9-111: Activate Peer Execution Priority List

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ExecutionPriorityListActivationStatusMT: MA_ExecutionPriorityListActivationStatusMT	None
MA_ExecutionPriorityListCommandMT: MA_ExecutionPriorityListCommandMT	None
MA_ExecutionPriorityListCommandStatusMT_1 : MA_ExecutionPriorityListCommandStatusMT	None
MA_ExecutionPriorityListCommandStatusMT_2 : MA_ExecutionPriorityListCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.12.4 Peer Ignore Zone Constraints Behaviors

9.2.12.4.1 Activate Peer Ignore Zone Constraints

The Activate Peer Ignore Zone Constraints sequence diagram below specifies the interaction where the lead ACP enables a zone constraint override for one or more tasks for its followers. Note that the intent of this sequence only covers OpZone and OpVolume constraints. First, the lead ACP sends a *MA_IgnoreConstraintCommandMT* message where the *AppliesToID* field is defined with the

SystemID of the follower ACP. This message contains either a list of zone constraints to ignore (*ListOfConstraintsToIgnore*), or an indicator that all constraints should be ignored (*IgnoreAllConstraints*). It also contains the ID(s) of the task(s) the command applies to (*ReferenceType*). Multiple commands can be active on the MA and each should have a unique *CommandID*. For new commands, the *CommandState* shall be set to *NEW*. For updates to active commands, the *CommandState* shall be set to *UPDATE* and the *CommandID* shall match that of the active message being updated by the leader MA. To cancel an active command, the *CommandState* shall be set to *CANCEL* with *CommandID* set to that of the message being canceled.

In response to the command, the follower ACPs shall send *MA_IgnoreConstraintCommandStatusMT_1* to the lead ACP, with the *AppliesToID* field set to their System ID and the *CommandProcessingState* field set to *RECEIVED* to indicate receipt of the command. The Follower ACPs proceed to execute the command by updating their ignored constraint and send *MA_IgnoreConstraintCommandStatusMT_2* to the lead ACP with the *AppliesToID* field set to their System ID and the *CommandProcessingState* set to *ACCEPTED*, to indicate they have completed the command.

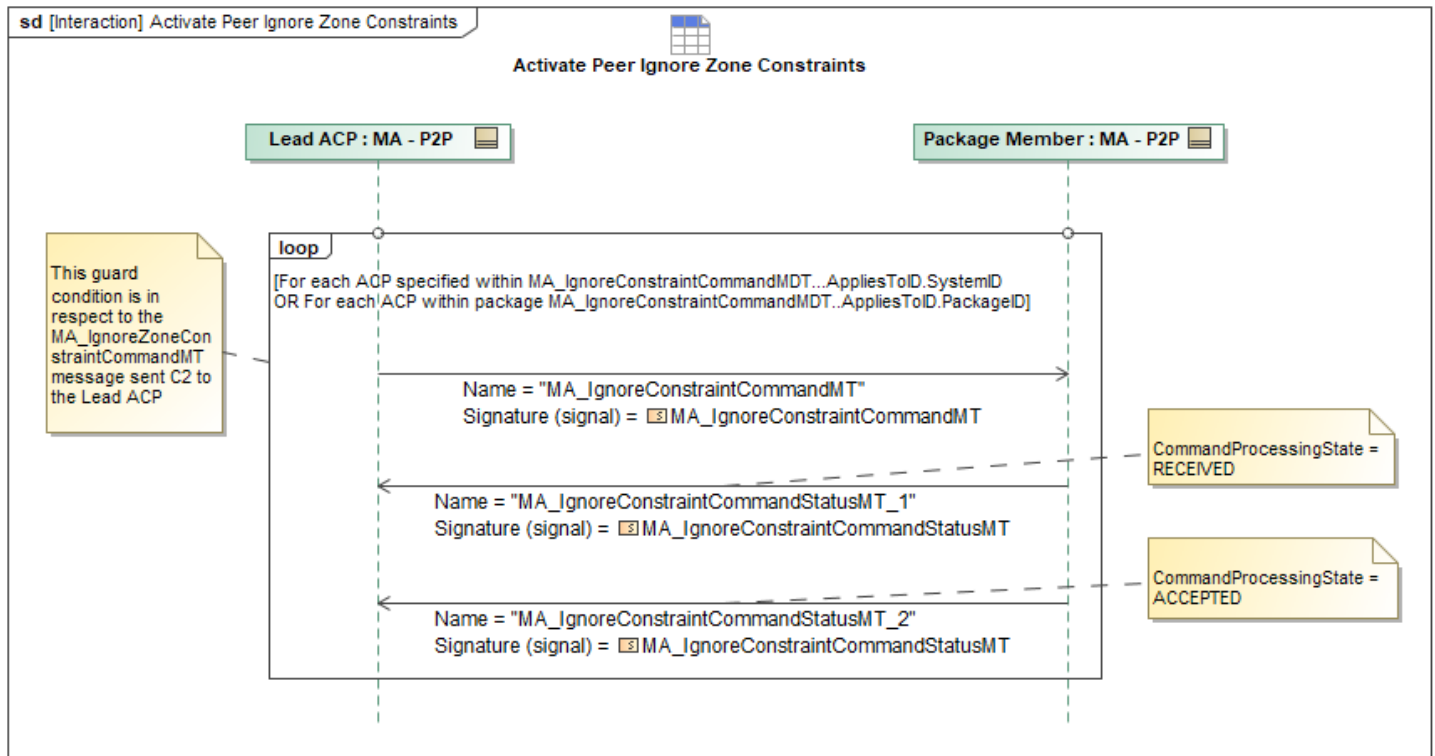


Figure 9-112: Activate Peer Ignore Zone Constraints

Table 9-112: Activate Peer Ignore Zone Constraints

Message Name (Name:Type)	Required Fields (Name:Type)
MA_IgnoreConstraintCommandMT: MA_IgnoreConstraintCommandMT	ListOfConstraintsToIgnore: MA_RegionChoiceType IgnoreAllConstraints: boolean
MA_IgnoreConstraintCommandStatusMT_1: MA_IgnoreConstraintCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_IgnoreConstraintCommandStatusMT_2: MA_IgnoreConstraintCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13 Peer Tasking Package

9.2.13.1 Allocate an Altitude Stacked Marshall

To allocate an Altitude Stacked Marshall, the ACP designated as the Package lead sends an *MA_FlightCommandMT_1* to all allocated Package partners, using Capability for *MA_FlightCommandType* with the *Capability.AltitudeStackedMarshall* field populated with the relevant kinematic parameters for a new task or with *MA_FlightCommandMT_2* using Activity to modify an existing flight activity. All Package members then respond with *MA_FlightCommandStatusMT_1* to indicate if the command was accepted or otherwise. If the command is rejected, the *MA_FlightCommandStatusMT_2* message will specify the *CannotComplyDetails* field to provide further details. The peer may optionally specify a suggested flight task with adjusted parameters using the *MA_TaskMT* message. The peer ACP will forward *MA_TaskMT* to the Package lead. The Package lead can restart the sequence with the suggestion or choose an entirely different flight command.

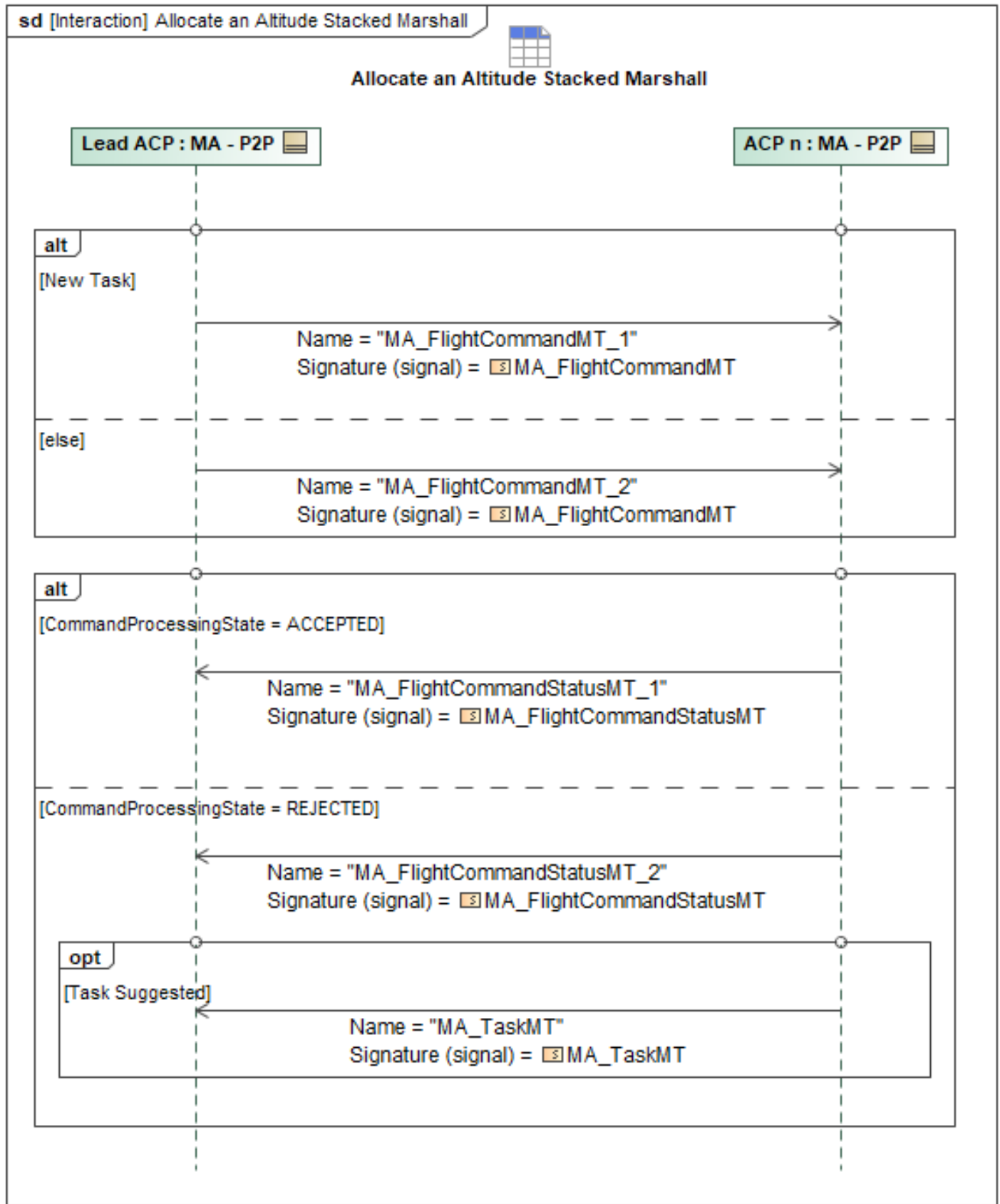


Figure 9-113: Allocate an Altitude Stacked Marshall

Table 9-113: Allocate an Altitude Stacked Marshall

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCommandMT_1: MA_FlightCommandMT	Capability: MA_FlightCapabilityCommandType AltitudeStackedMarshall: MA_AltitudeStackedMarshallType
MA_FlightCommandMT_2: MA_FlightCommandMT	Activity: MA_FlightActivityCommandType
MA_FlightCommandStatusMT_1: MA_FlightCommandStatusMT	None
MA_FlightCommandStatusMT_2: MA_FlightCommandStatusMT	CannotComplyDetails: MA_CannotComplyDetailsType
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.2 Allocating an Escort Task

To allocate an Escort Task, the ACP designated as the Package lead sends an *MA_EscortCommandMT* to all allocated Package partners to invoke a previously advertised *MA_EscortCapabilityMT*. This command is not necessarily sent to all members, only to those that are being engaged in the Escort command. All commanded Package members then respond with *MA_EscortCommandStatusMT_1* to indicate if the command was accepted or *MA_EscortCommandStatusMT_2* with the *CommandProcessingStateReason* field populated if rejected.

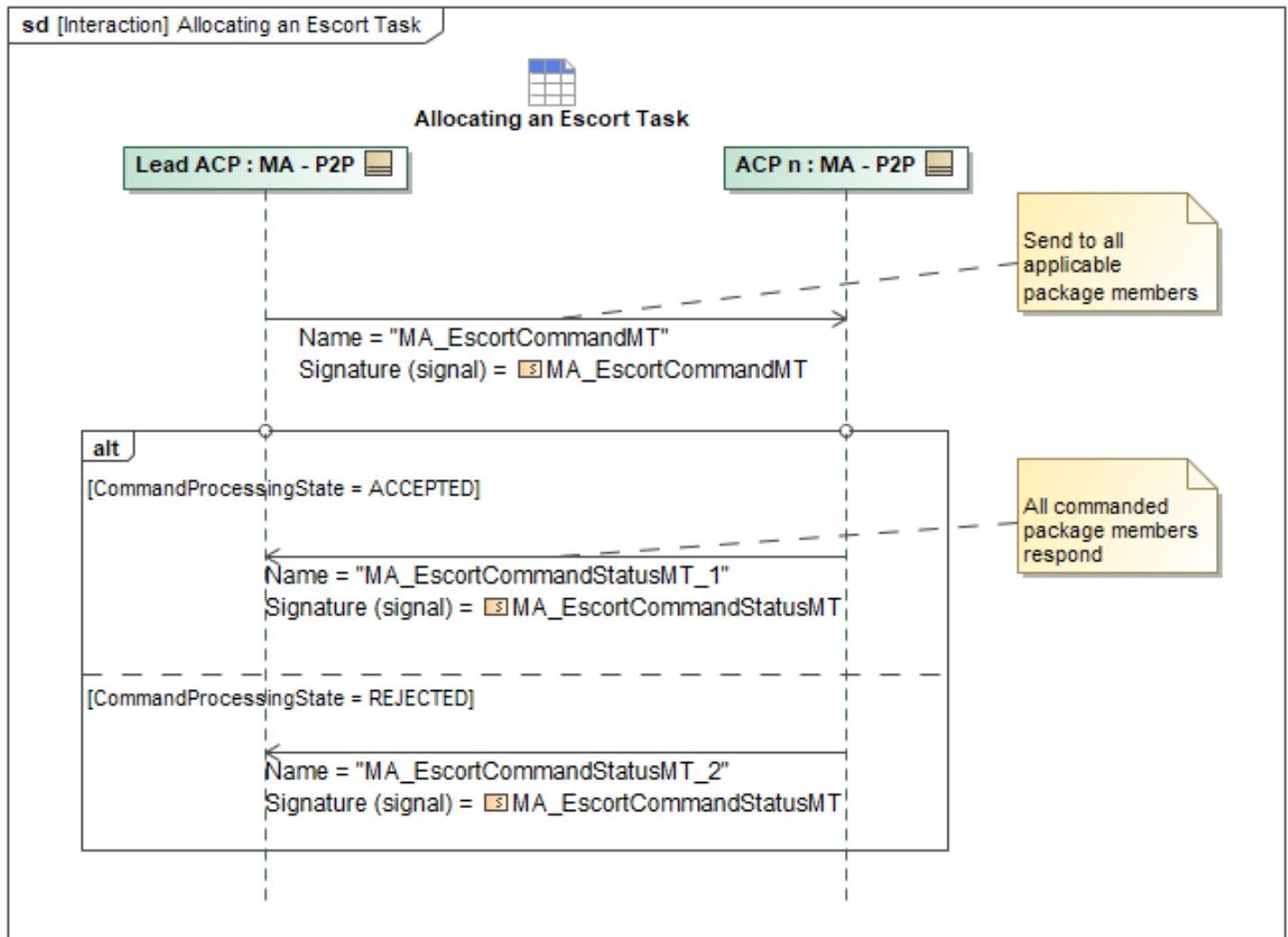


Figure 9-114: Allocating an Escort Task

Table 9-114: Allocating an Escort Task

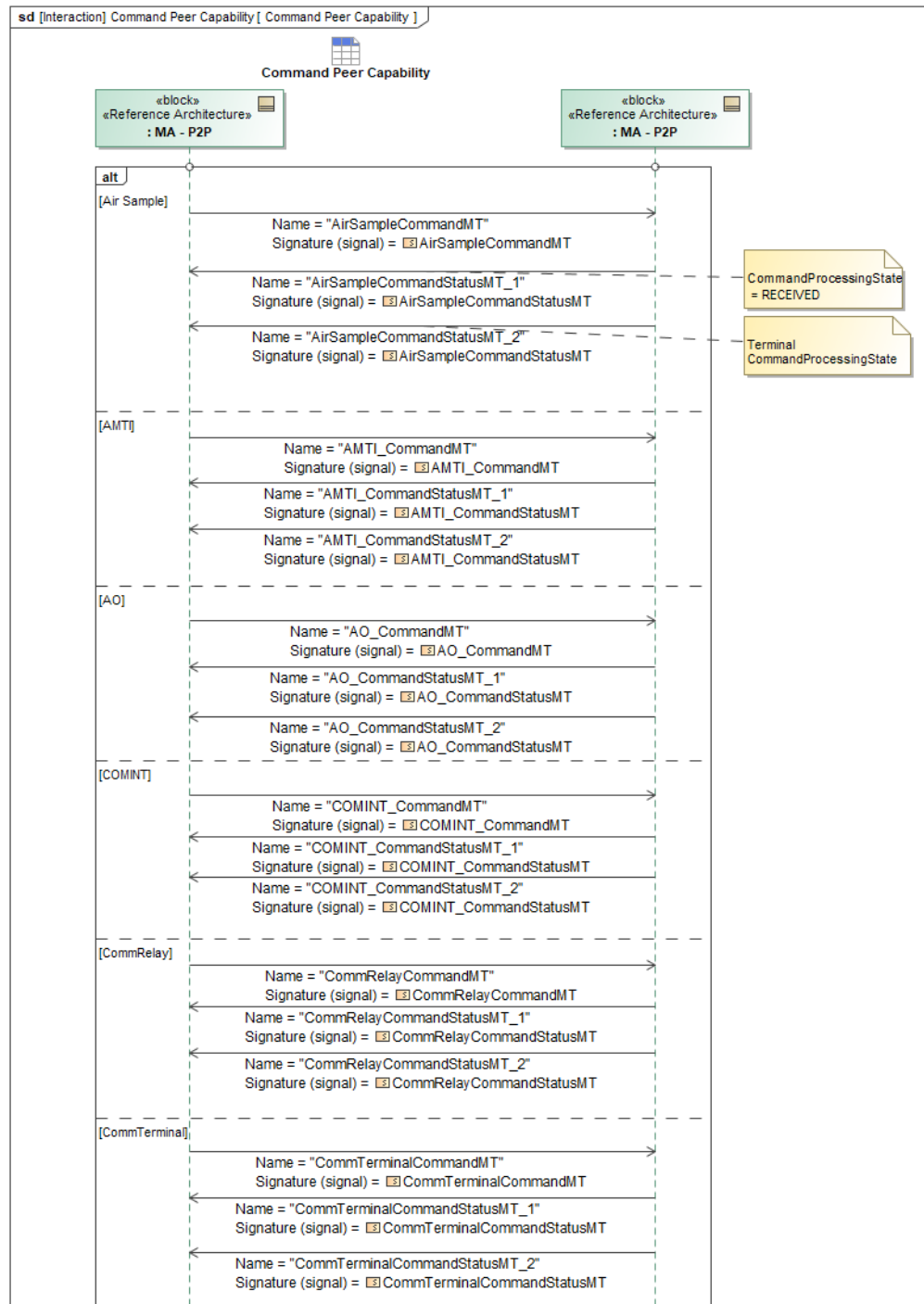
Message Name (Name:Type)	Required Fields (Name:Type)
MA_EscortCommandMT: MA_EscortCommandMT	None
MA_EscortCommandStatusMT_1: MA_EscortCommandStatusMT	None
MA_EscortCommandStatusMT_2: MA_EscortCommandStatusMT	CommandProcessingStateReason: CannotComplyType

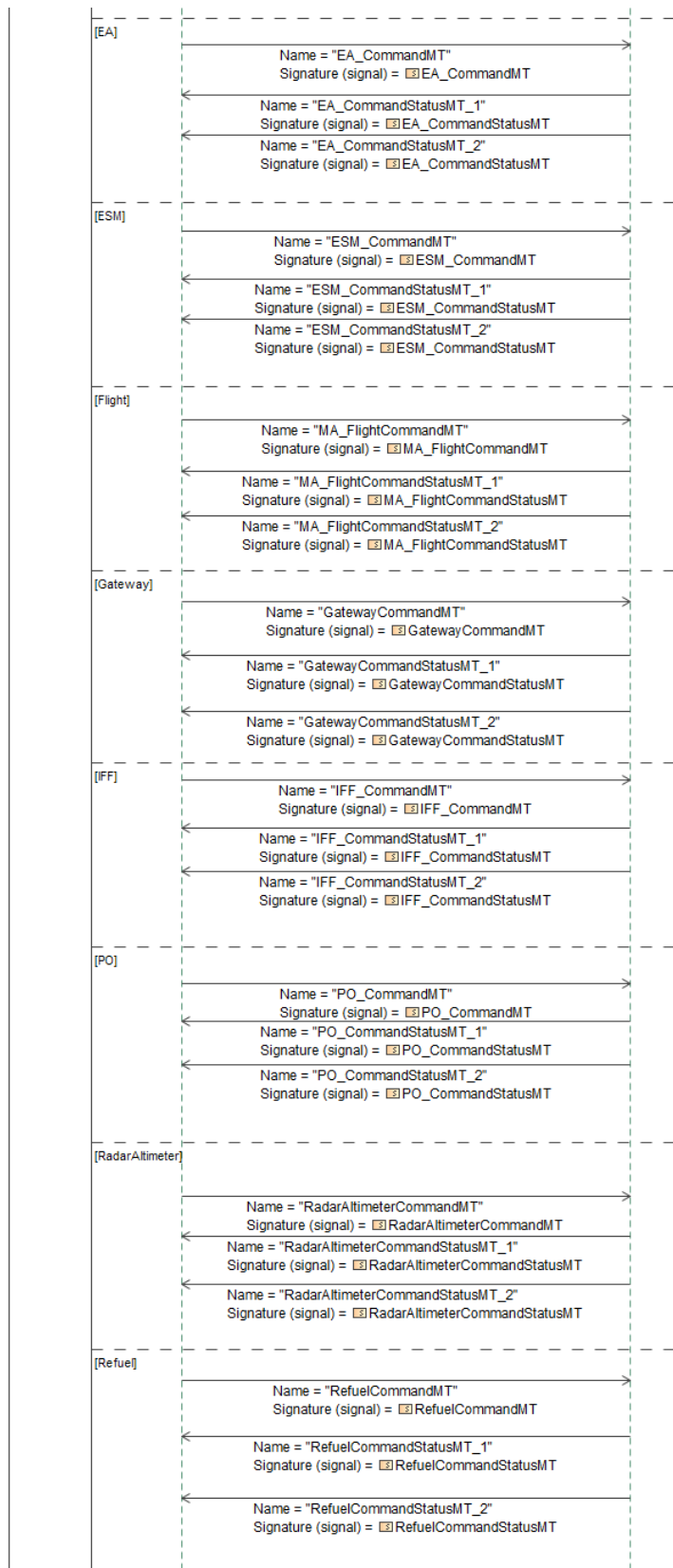
Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.3 Command Peer Capability

An ACP can send a *[Capability]Command* to a Peer, commanding the Peer to execute the appropriate *Capability*. In response, the recipient of the command checks its *ApprovalPolicy* to verify that the commanding ACP has authority to command this specific *Capability*. The Peer will send a

[Capability]CommandStatus ("_CommandStatusMT_1) indicating that the command has been received. Once the command reaches a terminal state (ACCEPTED, REJECTED, CANCELED), the Peer will send a [Capability]CommandStatus ("_CommandStatusMT_2) indicating that the command has reached that terminal state. This interaction assumes that the commanding ACP is aware that the commanded ACP is able to execute the commanded Capability.





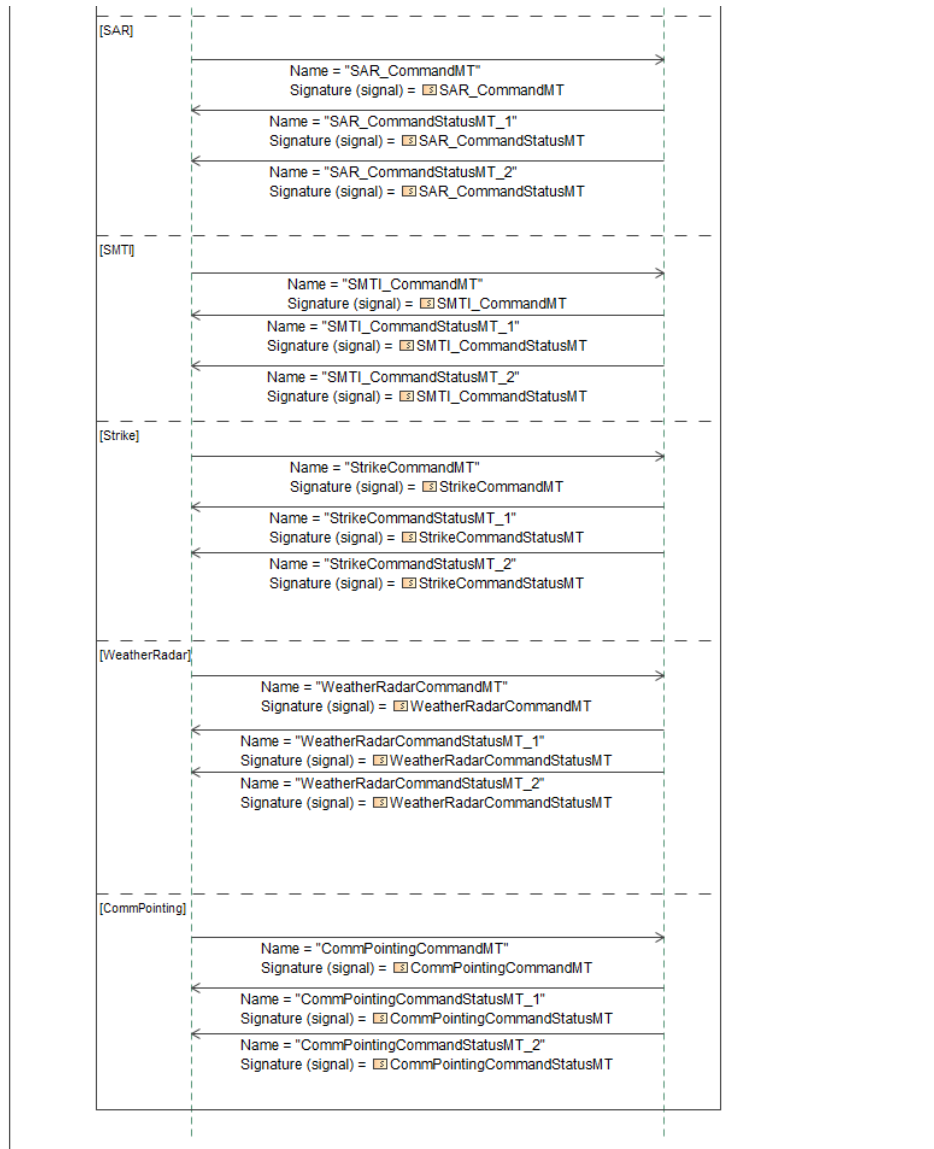


Figure 9-115: Command Peer Capability

Table 9-115: Command Peer Capability

Message Name (Name:Type)	Required Fields (Name:Type)
AirSampleCommandMT: AirSampleCommandMT	Capability: AirSampleCapabilityCommandType
AirSampleCommandStatusMT_1: AirSampleCommandStatusMT	None
AirSampleCommandStatusMT_2: AirSampleCommandStatusMT	None
AMTI_CommandMT: AMTI_CommandMT	Capability: AMTI_CapabilityCommandType
AMTI_CommandStatusMT_1: AMTI_CommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
AMTI_CommandStatusMT_2: AMTI_CommandStatusMT	None
AO_CommandMT: AO_CommandMT	Capability: AO_CapabilityCommandType
AO_CommandStatusMT_1: AO_CommandStatusMT	None
AO_CommandStatusMT_2: AO_CommandStatusMT	None
COMINT_CommandMT: COMINT_CommandMT	Capability: COMINT_CapabilityCommandType
COMINT_CommandStatusMT_1: COMINT_CommandStatusMT	None
COMINT_CommandStatusMT_2: COMINT_CommandStatusMT	None
CommPointingCommandMT: CommPointingCommandMT	None
CommPointingCommandStatusMT_1: CommPointingCommandStatusMT	None
CommPointingCommandStatusMT_2: CommPointingCommandStatusMT	None
CommRelayCommandMT: CommRelayCommandMT	Capability: CommRelayCapabilityCommandType
CommRelayCommandStatusMT_1: CommRelayCommandStatusMT	None
CommRelayCommandStatusMT_2: CommRelayCommandStatusMT	None
CommTerminalCommandMT: CommTerminalCommandMT	Capability: CommTerminalCapabilityCommandType
CommTerminalCommandStatusMT_1: CommTerminalCommandStatusMT	None
CommTerminalCommandStatusMT_2: CommTerminalCommandStatusMT	None
EA_CommandMT: EA_CommandMT	Capability: EA_CapabilityCommandType
EA_CommandStatusMT_1: EA_CommandStatusMT	None
EA_CommandStatusMT_2: EA_CommandStatusMT	None
ESM_CommandMT: ESM_CommandMT	Capability: ESM_CapabilityCommandType
ESM_CommandStatusMT_1: ESM_CommandStatusMT	None
ESM_CommandStatusMT_2: ESM_CommandStatusMT	None
GatewayCommandMT: GatewayCommandMT	Capability: GatewayCapabilityCommandType

Message Name (Name:Type)	Required Fields (Name:Type)
GatewayCommandStatusMT_1: GatewayCommandStatusMT	None
GatewayCommandStatusMT_2: GatewayCommandStatusMT	None
IFF_CommandMT: IFF_CommandMT	Capability: IFF_CapabilityCommandType
IFF_CommandStatusMT_1: IFF_CommandStatusMT	None
IFF_CommandStatusMT_2: IFF_CommandStatusMT	None
MA_FlightCommandMT: MA_FlightCommandMT	Capability: MA_FlightCapabilityCommandType
MA_FlightCommandStatusMT_1: MA_FlightCommandStatusMT	None
MA_FlightCommandStatusMT_2: MA_FlightCommandStatusMT	None
PO_CommandMT: PO_CommandMT	Capability: PO_CapabilityCommandType
PO_CommandStatusMT_1: PO_CommandStatusMT	None
PO_CommandStatusMT_2: PO_CommandStatusMT	None
RadarAltimeterCommandMT: RadarAltimeterCommandMT	Capability: RadarAltimeterCapabilityCommandType
RadarAltimeterCommandStatusMT_1: RadarAltimeterCommandStatusMT	None
RadarAltimeterCommandStatusMT_2: RadarAltimeterCommandStatusMT	None
RefuelCommandMT: RefuelCommandMT	Capability: RefuelCapabilityExtendCommandType
RefuelCommandStatusMT_1: RefuelCommandStatusMT	None
RefuelCommandStatusMT_2: RefuelCommandStatusMT	None
SAR_CommandMT: SAR_CommandMT	Capability: SAR_CapabilityCommandType
SAR_CommandStatusMT_1: SAR_CommandStatusMT	None
SAR_CommandStatusMT_2: SAR_CommandStatusMT	None
SMTI_CommandMT: SMTI_CommandMT	Capability: SMTI_CapabilityCommandType
SMTI_CommandStatusMT_1: SMTI_CommandStatusMT	None
SMTI_CommandStatusMT_2: SMTI_CommandStatusMT	None
StrikeCommandMT: StrikeCommandMT	Capability: StrikeCapabilityCommandType

Message Name (Name:Type)	Required Fields (Name:Type)
StrikeCommandStatusMT_1: StrikeCommandStatusMT	None
StrikeCommandStatusMT_2: StrikeCommandStatusMT	None
WeatherRadarCommandMT: WeatherRadarCommandMT	Capability: WeatherRadarCapabilityCommandType
WeatherRadarCommandStatusMT_1: WeatherRadarCommandStatusMT	None
WeatherRadarCommandStatusMT_2: WeatherRadarCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.4 Command Peer via Action

This interaction allows a lead to command a peer via an action. It also allows a lead to propose an action to a peer and for the peer to accept or reject the proposed action. If the peer rejects the proposed action, it can propose an alternative.

The lead sends *MA_ActionMT_1* to the peer. If the lead wants the peer to evaluate but not execute the action, the *ActionGuidance.TacticalPlanningAndExecution* field is set to *BE_PREPARED_TO*.

The lead then sends an *MA_ActionCommandMT* to prompt the peer to evaluate (and possibly execute) the *MA_ActionMT_1*.

While the command is being processed, the peer may optionally send *MA_ActionCommandStatusMT_1* with the *CommandProcessingState* set to *RECEIVED*.

If the *Action* can be executed, the peer will respond with an *MA_ActionCommandStatusMT_3* with the *CommandProcessingState* set to *ACCEPTED*. The peer then sends an *MA_ActionStatusMT* whenever the status of the action changes.

If the peer cannot execute the *Action*, it will respond with *MA_ActionCommandStatusMT_2* with the *CommandProcessingState* set to *REJECTED* and set the *CommandProcessingStateReason.AssociatedID* in *MA_ActionCommandStatusMT_2* to the *ActionID* of *MA_ActionMT_1*. If the peer is able to generate an action that it can accept, it will respond with *MA_ActionMT_2*. The *Metadata.Traceability.ActionID* field in *MA_ActionMT_2* must be set to the *ActionID* of *MA_ActionMT_1*.

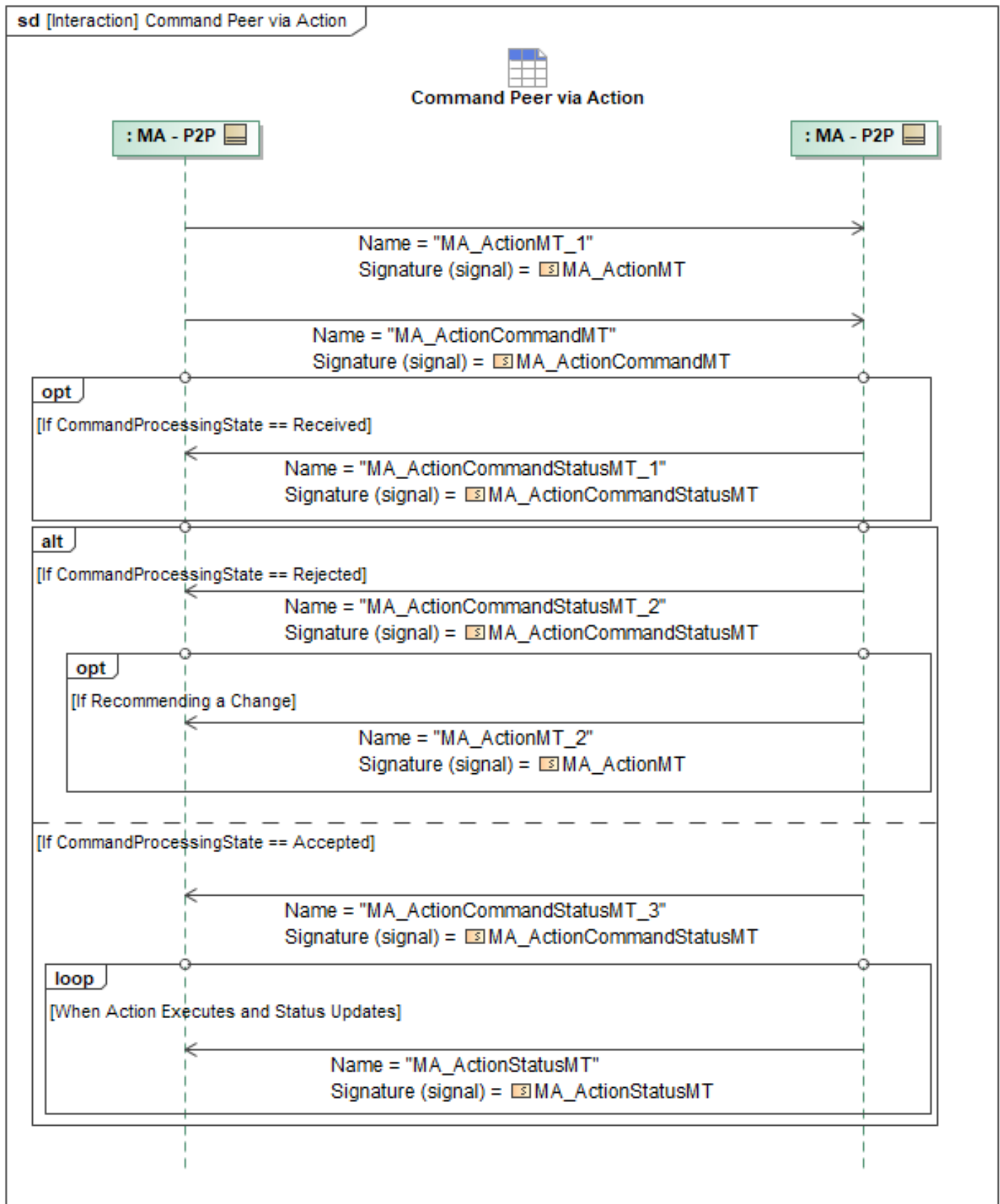


Figure 9-116: Command Peer via Action

Table 9-116: Command Peer via Action

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionCommandMT: MA_ActionCommandMT	None
MA_ActionCommandStatusMT_1: MA_ActionCommandStatusMT	None
MA_ActionCommandStatusMT_2: MA_ActionCommandStatusMT	CommandProcessingStateReason: CannotComplyType AssociatedID: ID_Type
MA_ActionCommandStatusMT_3: MA_ActionCommandStatusMT	None
MA_ActionMT_1: MA_ActionMT	None
MA_ActionMT_2: MA_ActionMT	Metadata: RequirementMetadataType Traceability: TraceabilityType
MA_ActionStatusMT: MA_ActionStatusMT	SystemID: SystemID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.5 Command Peer(s) via Task

MA_TaskMT contains different tasks that a system can perform against a specific entity or geographical location. The MA Lead System sends the *MA_TaskMT* to the peer MA immediately followed by a *MA_TaskCommandMT* to activate it. Within the *MA_TaskCommandMT* the *ConstraintOverride.Allocation.RequiredAllocation* is used to specify the ACP(s) by their *SystemID*. *MA_TaskCommandStatusMT* is then sent back to indicate if the command was accepted. Each individual ACP will then report *MA_TaskStatusMT* individually to the MA Lead system.

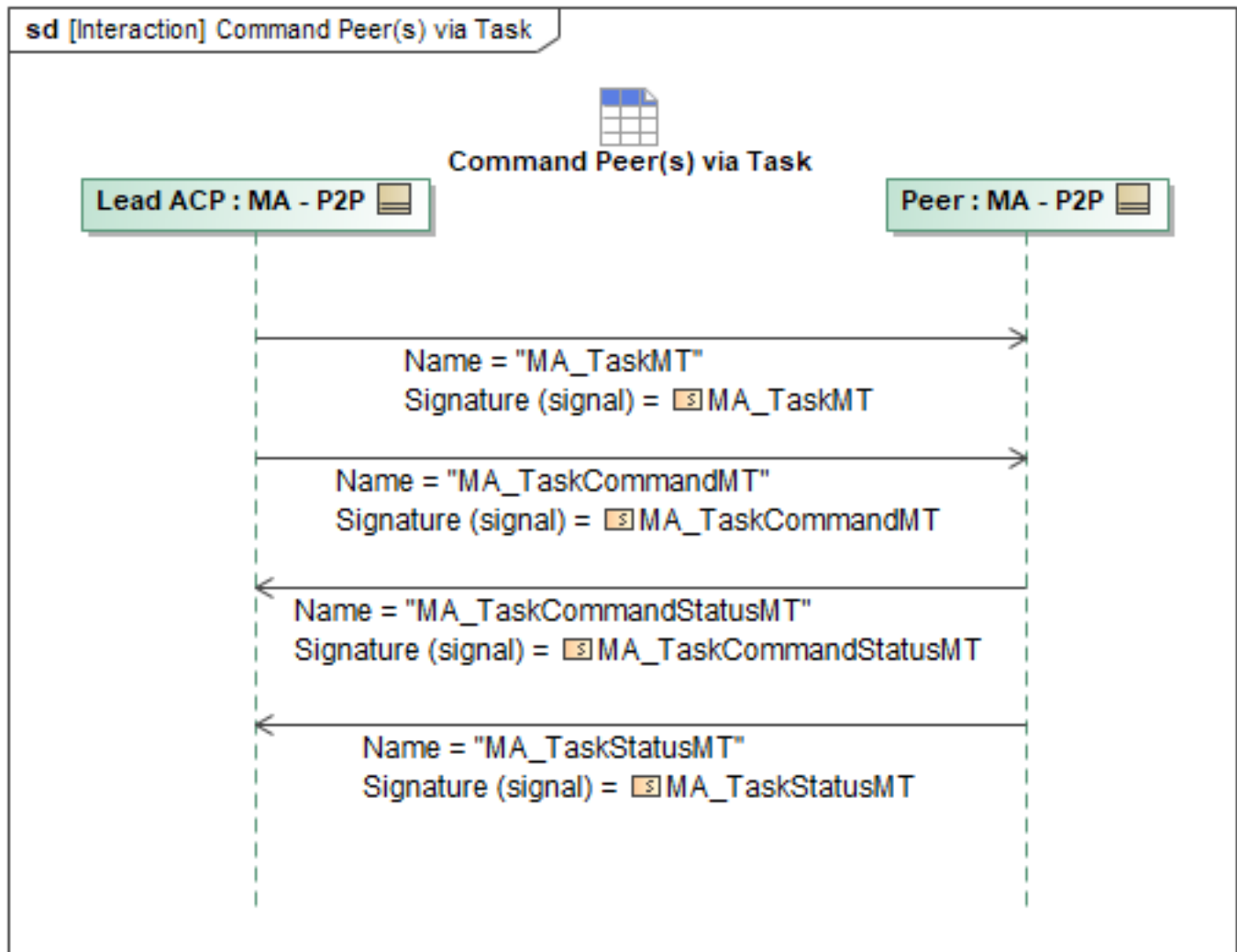


Figure 9-117: Command Peer(s) via Task

Table 9-117: Command Peer(s) via Task

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskCommandMT: MA_TaskCommandMT	RequiredAllocation: MA_RequirementAllocationConstraintType SystemID: SystemID_Type Allocation: MA_AllocationChoiceType ConstraintOverride: MA_RequirementConstraintsType
MA_TaskCommandStatusMT: MA_TaskCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.6 Control Mode Authorization

Peer systems seeking to control MA should be made aware of the capabilities that MA has been granted by Flight Autonomy. The VI communicates how MA can control the vehicle with *MA_FlightCapabilityMT*. The *MA_FlightCapabilityMT* message requires the *FlightCapabilityPerformanceProfile* field be populated with the appropriate profile subtypes of the capabilities advertised and any parameters needed by MA for commanding each type present. It then sends *MA_FlightCapabilityStatusMT* message to communicate the availability of control modes to MA control (see Control Authorization in the Vehicle Interface for more details). Both of these messages should be passed to Peer nodes when connecting to MA and *MA_FlightCapabilityStatusMT* should be forwarded along to Peer nodes whenever there is a change in flight capabilities.

Once the required capabilities are available, Peers with custody can issue commands to the MA via the Direct HSA/CSA Command use case or the Direct Waypoint Command use case. The *MA_FlightCommandMT* message within these use cases carries control mode-specific data, as described in later sections.

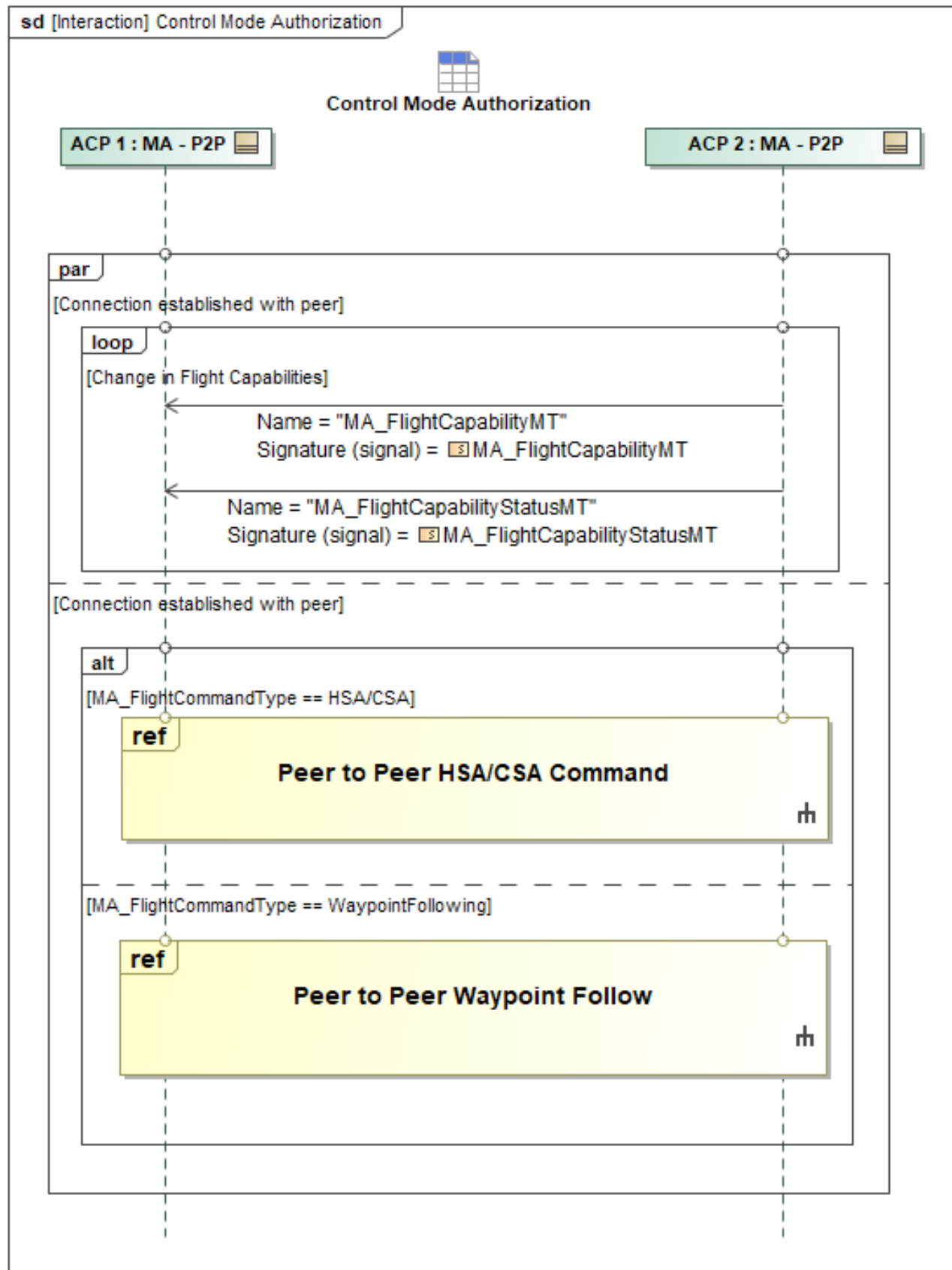


Figure 9-118: Control Mode Authorization

Table 9-118: Control Mode Authorization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	FlightCapabilityPerformanceProfile: MA_FlightCapabilityPerformanceProfileType
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.7 Package Plan Approval Status

The Package Plan Approval Status sequence diagram specifies the interaction of two ACPs where the ACP has received approval for the plans indicated, and the ACP is synchronizing across the package that the plans have now been designated as approved. While the Approval Status specifically statuses the response of the Approval Request, the plan approval status messages are a status of the approval state of the plan itself, and they are utilized by the system to confirm individual plans approval or rejection status. As each executable plan is approved, the ACP sends the plan approval status to another ACP for awareness and tracking purposes via the *TaskPlanApprovalStatusMT*, *MissionPlanApprovalStatusMT*, *RoutePlanApprovalStatusMT*, and *ActivityPlanApprovalStatusMT* messages. For each of the messages sent, the *PlanID* and *PlanApprovalState* is populated with the applicable plan UUID and approval state, respectively.

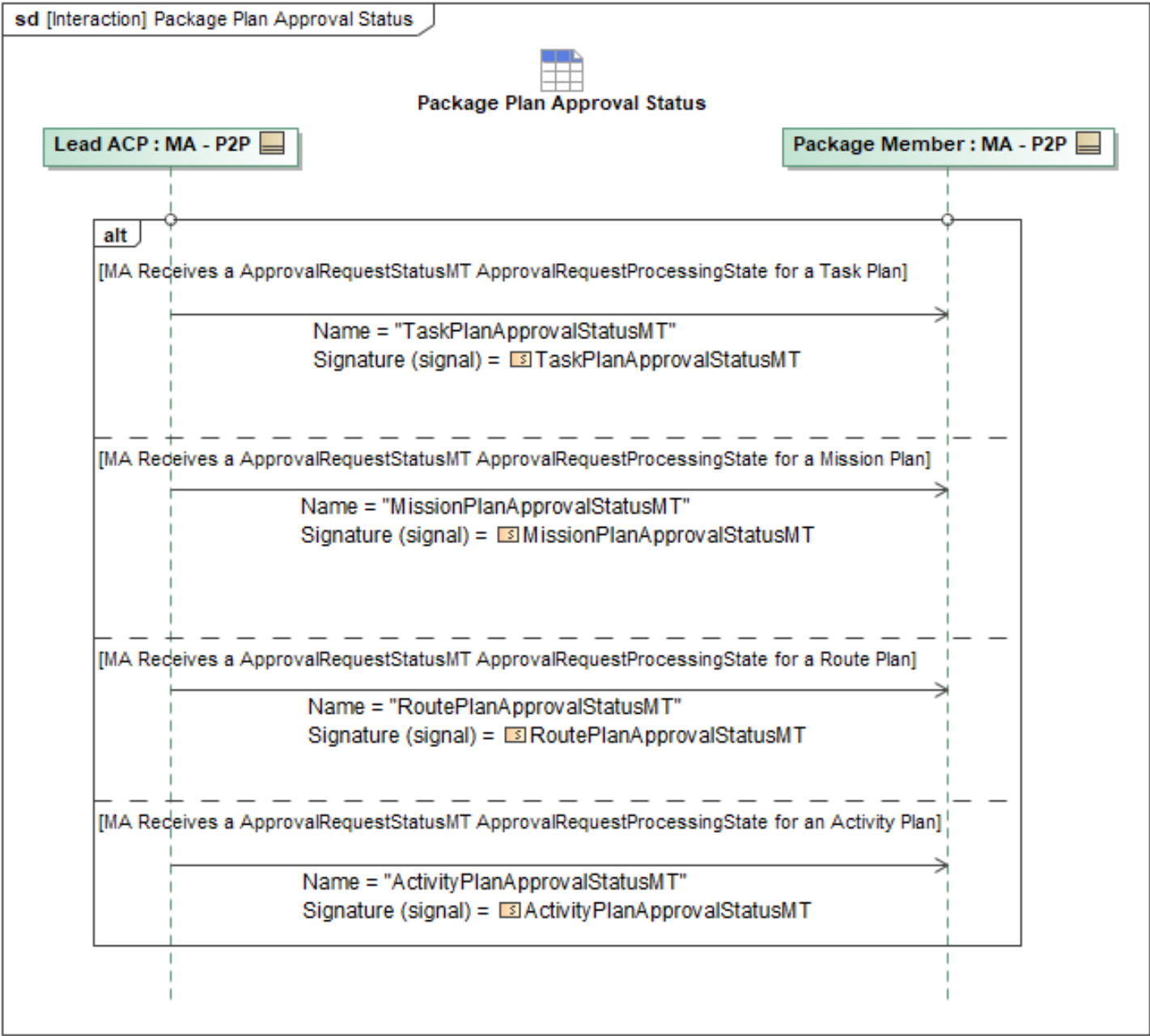


Figure 9-119: Package Plan Approval Status

Table 9-119: Package Plan Approval Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityPlanApprovalStatusMT: ActivityPlanApprovalStatusMT	None
MissionPlanApprovalStatusMT: MissionPlanApprovalStatusMT	None
RoutePlanApprovalStatusMT: RoutePlanApprovalStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
TaskPlanApprovalStatusMT: TaskPlanApprovalStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.8 Peer Activate or Deactivate Mission Plan

A Package Leader can activate or deactivate a mission plan by sending a *MA_MissionPlanActivationCommandMT* message to a Follower. The *MA_MissionPlanActivationCommandMT*'s **CommandState** field shall be *NEW*, the **Command** field shall specify which mission plan to activate or deactivate, and **CommandID** field shall be a unique identifier for the command. To activate or deactivate a mission plan and all its sub-plans, send *Command.ActivationDetails.ByMissionPlan*. **To activate an entire mission plan, set *Command.ActivationDetails.ByMissionPlan.ActivationCommand* to *ACTIVATE*. To deactivate an entire mission plan, set *Command.ActivationDetails.ByMissionPlan.ActivationCommand* to *DEACTIVATE*.** To activate or deactivate only a subset of subplans belonging to a mission plan, send *Command.ActivationDetails.BySubPlan* detailing which subplans should be activated or deactivated. **To activate a subplan, set *Command.ActivationDetails.BySubPlan.*Plan.CommandType* to *ACTIVATE*. To deactivate a subplan, set *Command.ActivationDetails.BySubPlan.*Plan.CommandType* to *DEACTIVATE*.** To activate or deactivate only a specific *ExecutionPlanSet* belonging to a mission plan, send *Command.ActivationDetails.ByExecutionPlanSet*. **To activate a specific *ExecutionPlanSet*, set *Command.ActivationDetails.ByExecutionPlanSet.CommandType* to *ACTIVATE*. To deactivate a specific *ExecutionPlanSet*, set *Command.ActivationDetails.ByExecutionPlanSet.CommandType* to *DEACTIVATE*.**

The Follower will send at least one *MA_MissionPlanActivationCommandStatusMT* message, the final of which (*MissionPlanActivationCommandStatusMT_2*) must indicate a terminal status, i.e. ***ActivationStatus.CommandProcessingState* field set to *ACCEPTED* or *REJECTED* and *ActivationStatus.CommandStatus* field set to *COMPLETED* or *FAILED*.** The Follower may optionally send *MA_MissionPlanActivationCommandStatusMT* messages (*MA_MissionPlanActivationCommandStatusMT_1*) prior to the terminal status to indicate in-progress processing of the *MA_MissionPlanActivationCommandMT*.

If a mission plan or subplan was activated or deactivated successfully, the Follower will send the Leader a *MA_MissionPlanActivationStatusMT_2* detailing the current state of the mission plan. If plan execution status has changed, the Follower will send the Leader a *MA_MissionPlanExecutionStatusMT* with *Source* set to *ACTUAL* and *PlanExecutionStatus* detailing the change in execution status. For a mission plan operating with an *ExecutionSequence*, *PlanExecutionStatus.ExecutionPlanSetStatus* can be included to provide statuses on the *ExecutionPlanSets* within the mission plan.

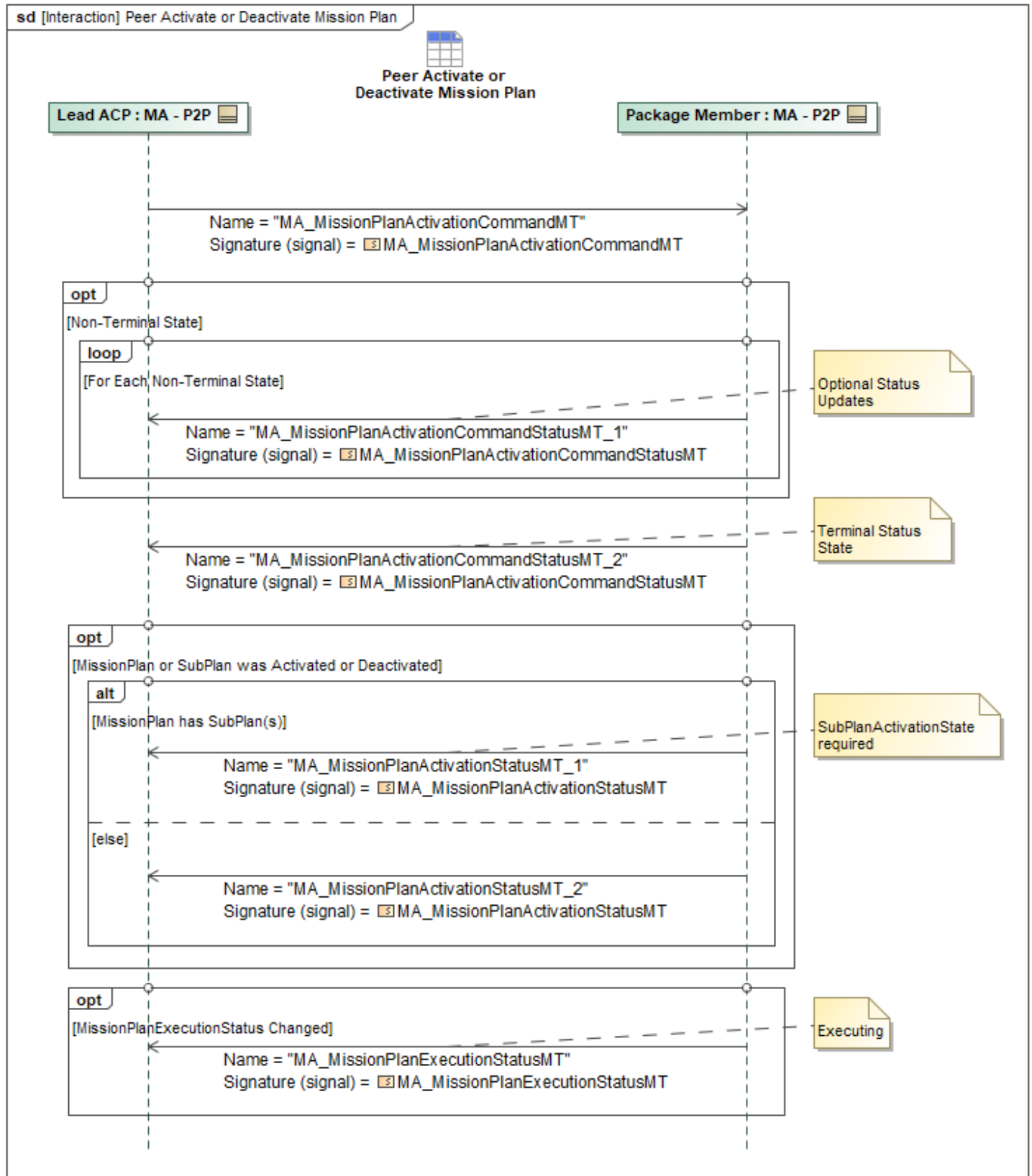


Figure 9-120: Peer Activate or Deactivate Mission Plan

Table 9-120: Peer Activate or Deactivate Mission Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	None
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	None
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	None
MA_MissionPlanActivationStatusMT_1: MA_MissionPlanActivationStatusMT	SubPlanActivationState: MA_PlanActivationStateType SubPlanActivationState: MA_PlanActivationStateType
MA_MissionPlanActivationStatusMT_2: MA_MissionPlanActivationStatusMT	None
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	PlanExecutionStatus: MissionPlanExecutionStateType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.9 Peer Responds to Approval Request

In the event that MA requires external approval from a Peer, such as when another package member is the Approval Authority for an *MA_ApprovalPolicyMT* which contains a *Policy* field that is set to *APPROVAL_REQUIRED*, the Approval Authority Peer can respond to the approval request sent by the Package Member. A Package Member sends *MA_ApprovalRequestMT* to the Approval Authority ACP when it identifies, per the Approval Policy, that it must request approval for something. **The Approval Authority sends *MA_ApprovalRequestStatusMT_1* with *RequestProcessingState* set to *QUEUED* in order to acknowledge the request. Optionally, the Approving Authority can send non-terminal status updates using *MA_ApprovalRequestStatusMT_2* by setting *RequestProcessingState* as *PROCESSING*.** The Approval Authority will make an approval determination and send *MA_ApprovalRequestStatusMT_3* to the Package Member, populating the *ApprovalRequestProcessingState* field with the approval state.

This interaction is applicable for engagement-related approval requests when a Target Authority has already authorized a Targeting Act against a particular Target, whether or not the Leader ACP is specified as a Target Authority itself. The Leader ACP is able to approve a Follower's *MA_ApprovalRequestMT* for performing the Targeting Act in these cases of having Target Authority pre-approval, as well as cases where the Leader ACP is listed as a Target Authority.

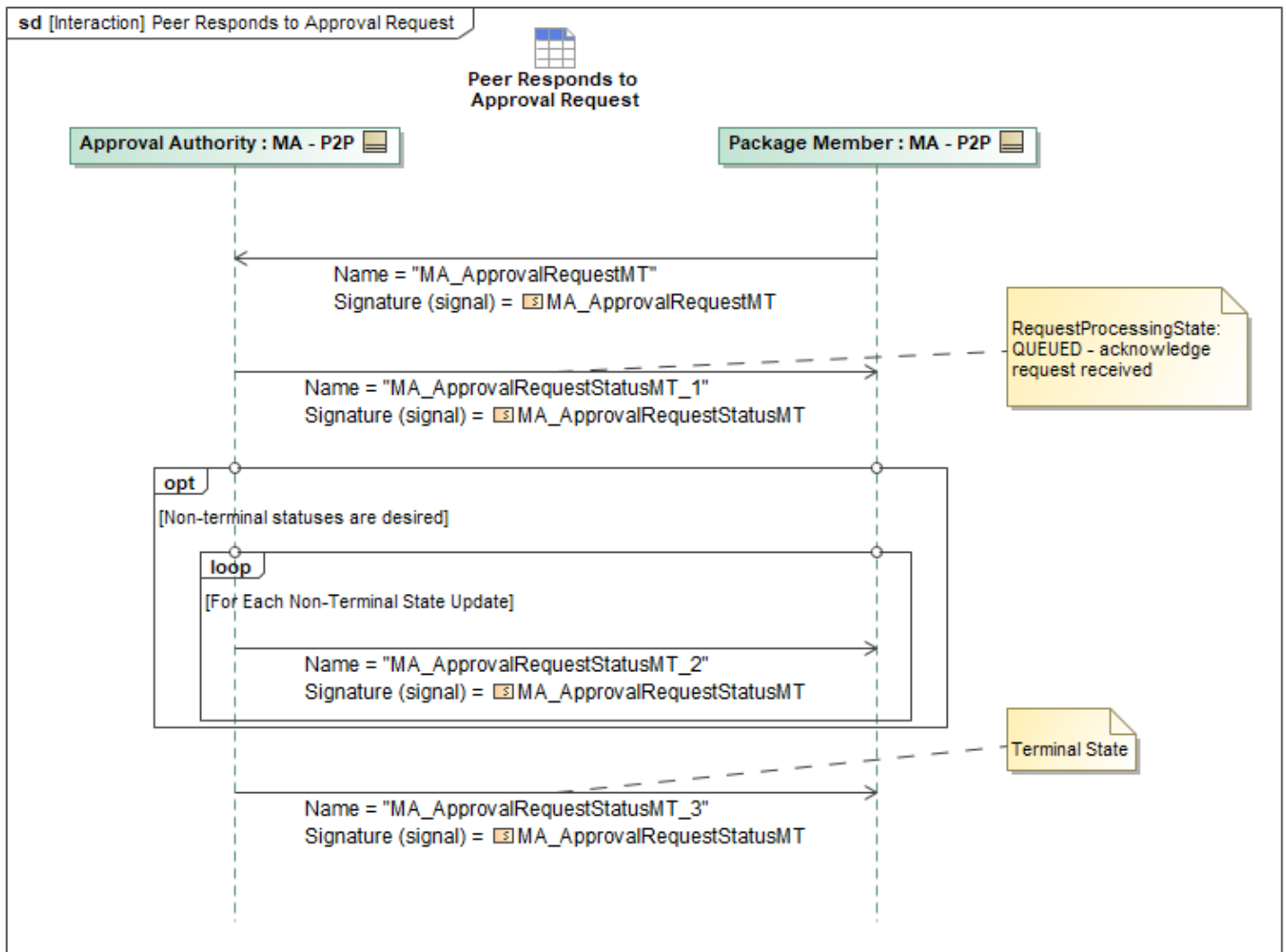


Figure 9-121: Peer Responds to Approval Request

Table 9-121: Peer Responds to Approval Request

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ApprovalRequestMT: MA_ApprovalRequestMT	None
MA_ApprovalRequestStatusMT_1: MA_ApprovalRequestStatusMT	None
MA_ApprovalRequestStatusMT_2: MA_ApprovalRequestStatusMT	None
MA_ApprovalRequestStatusMT_3: MA_ApprovalRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.10 Peer to Peer HSA/CSA Command

Peer systems with custody can send heading, course, speed, and altitude (HSA/CSA) commands to a peer's MA via the *MA_FlightCommandMT* UCI message. The ACP1 system sends an *MA_FlightCommandMT* UCI message to command the system and invokes a previously advertised capability. The *MA_FlightCommandMT* specifies the *Command.Capability.FlightControlMode.HSA_CSA* field in order to specify the flight command mode. The peer MA responds with a *MA_FlightCommandStatusMT* message, either approving or rejecting the command. If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* message will specify the *CannotComplyDetails* field to provide further details. ACP2 may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task with the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the suggestion or choose an entirely different flight command. The break may also occur if the *Activity State* is *COMPLETED* or *FAILED*.

If the command is *APPROVED*, the ACP will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until either control is taken away from MA, or MA relinquishes control.

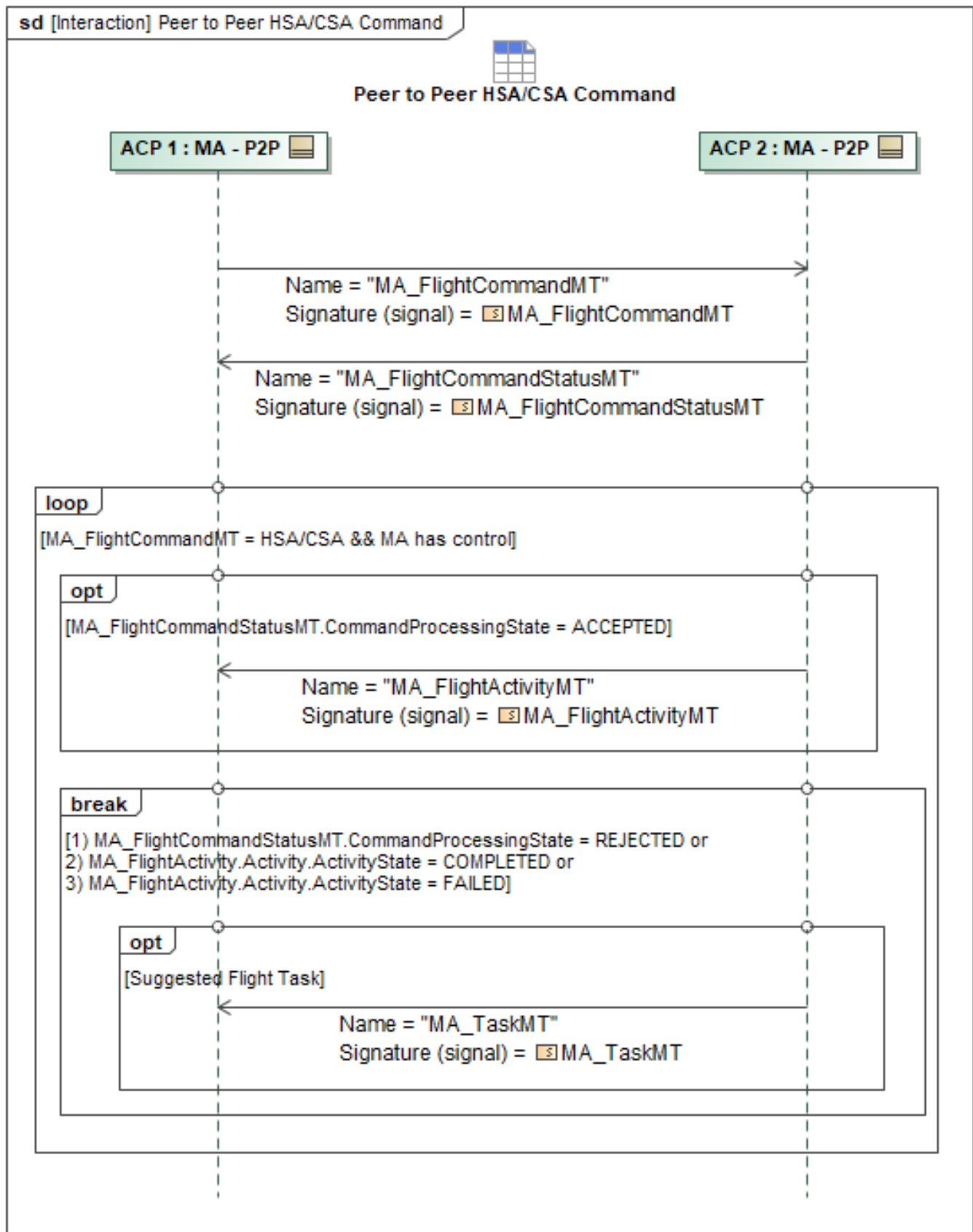


Figure 9-122: Peer to Peer HSA/CSA Command

Table 9-122: Peer to Peer HSA/CSA Command

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	HSA_CSA: MA_HSA_CSA_Type Capability: MA_FlightCapabilityCommandType FlightControlMode: MA_FlightControlModesChoiceType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.11 Peer to Peer Waypoint Follow

Waypoint following, alternatively referred to as route following, is a control mode where Mission Autonomy provides the peer a series of waypoints describing a desired flight path. Waypoint behavior may include simple commands such as fly-by or fly-through; complex commands including payload action and loiter/hold; or bounded by constraints such as vehicle attitude or timing. Route paths will be transferred between Mission Autonomy and its peers whenever the C2 system sends Mission Autonomy a new route, or an update to an existing route.

Communication of new waypoints begins with the transmission of *MA_FlightCommandMT* from Mission Autonomy to the peer receiving waypoint following information. A well-conditioned *Command.Capability.FlightControlMode.WaypointFollowing* field will simply include a *Command.Capability.FlightControlMode.WaypointFollowing.Route* field. Detailed information on the contents of the *Route* and *Path* fields are available within the OMS UCI 2.3 schema. After this, the peer will send back *MA_FlightCommandStatusMT* in response to the command. If the command is *APPROVED*, *MA_FlightActivityMT* will report data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* message will include the *CannotComplyDetails* field to provide further details. MA can then restart the sequence or use *MA_TaskMT* to encapsulate a suggested flight task with defined parameters. The break may also occur if the *Activity State* is *COMPLETED* or *FAILED*.

The cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until either control is taken away from MA, or MA relinquishes control.

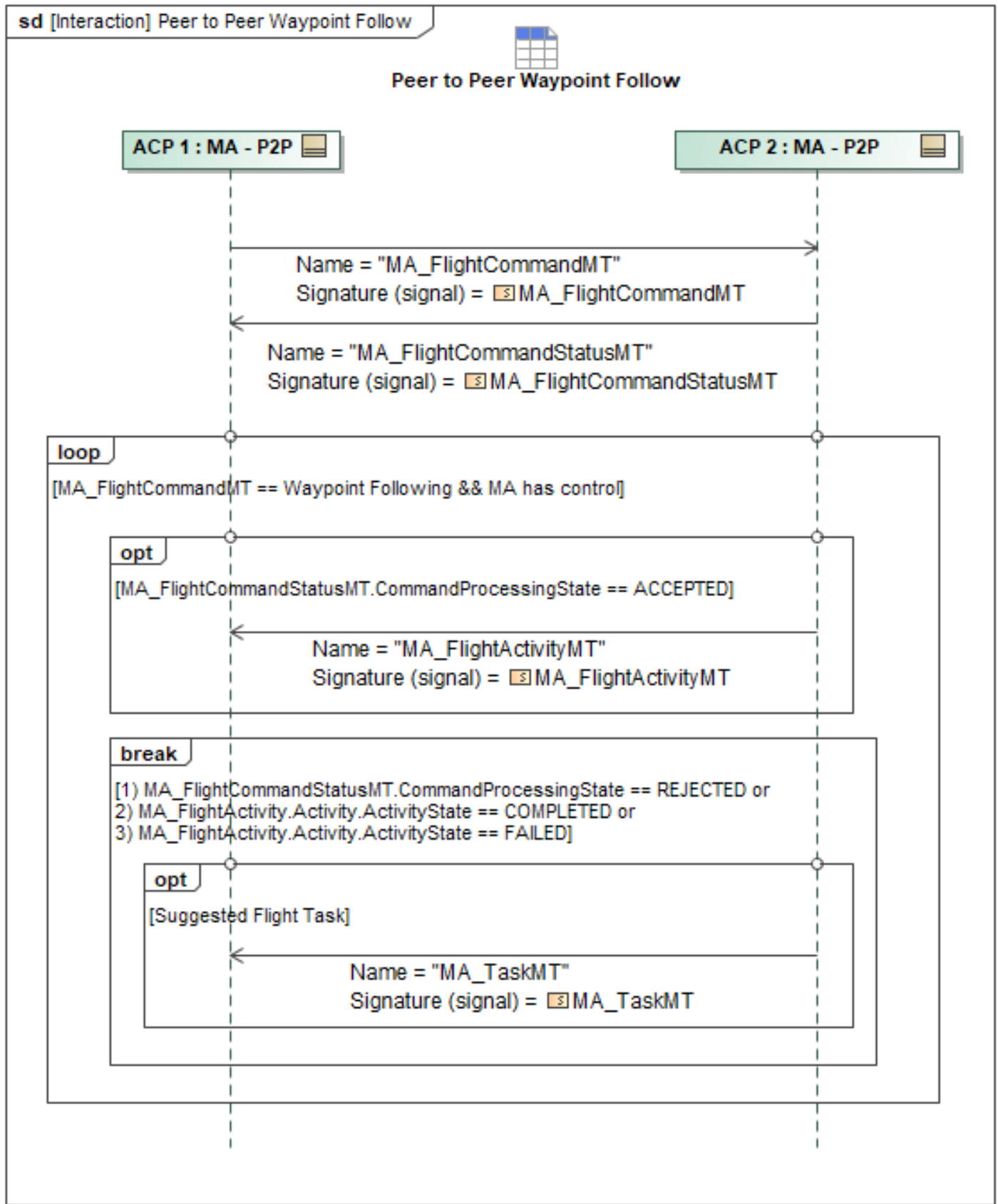


Figure 9-123: Peer to Peer Waypoint Follow

Table 9-123: Peer to Peer Waypoint Follow

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	WaypointFollowing: MA_WaypointFollowingType Capability: MA_FlightCapabilityCommandType FlightControlMode: MA_FlightControlModesChoiceType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.12 Peer Validate Action Plan

This interaction allows a lead to request a peer verify an action plan. This allows a lead to propose an action plan to a peer and for the peer to accept or reject the proposed action. If the peer rejects the proposed action, it can propose an alternative.

The lead sends *ActionPlanValidationCommandMT* to the peer with the *ActionPlanID* set to the plan of the action to evaluate.

While the command is being processed, the peer may optionally send *ActionPlanValidationCommandStatusMT_1* with the *CommandProcessingState* set to *RECEIVED*.

If the *ActionPlan* cannot execute the validation, the peer will respond with an *ActionPlanValidationCommandStatusMT_2* with the *CommandProcessingState* set to *REJECTED*.

If the *ActionPlan* is valid, the peer will respond with an *ActionPlanValidationCommandStatusMT_3* with the *CommandProcessingState* set to *ACCEPTED* and a *MA_ActionPlanValidationMT_1* with the *ValidationState* set to *VALID*.

If the *ActionPlan* is invalid, the peer will respond with *ActionPlanValidationCommandStatusMT_4* with the *CommandProcessingState* set to *ACCEPTED* and a *MA_ActionPlanValidationMT_2* with the *ValidationState* set to *INVALID*. If the peer is able to generate an *ActionPlan* that it can accept, it will respond with *MA_ActionMT* and *MA_ActionPlanMT* and set the *ModifiedPlanID* to the *ActionPlanID* of *MA_ActionPlanMT*.

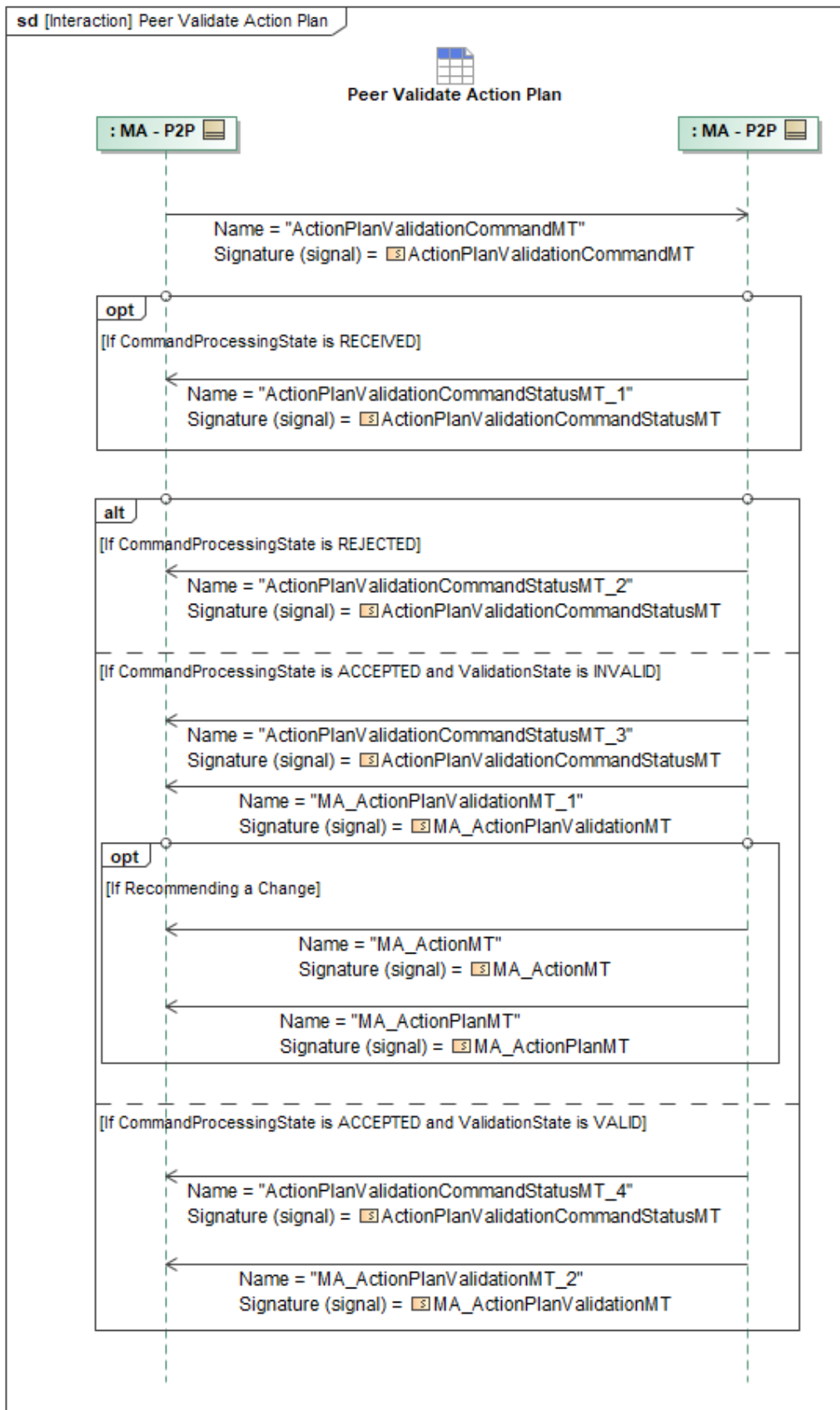


Figure 9-124: Peer Validate Action Plan

Table 9-124: Peer Validate Action Plan

Message Name (Name:Type)	Required Fields (Name:Type)
ActionPlanValidationCommandMT: ActionPlanValidationCommandMT	None
ActionPlanValidationCommandStatusMT_1: ActionPlanValidationCommandStatusMT	None
ActionPlanValidationCommandStatusMT_2: ActionPlanValidationCommandStatusMT	None
ActionPlanValidationCommandStatusMT_3: ActionPlanValidationCommandStatusMT	None
ActionPlanValidationCommandStatusMT_4: ActionPlanValidationCommandStatusMT	None
MA_ActionMT: MA_ActionMT	None
MA_ActionPlanMT: MA_ActionPlanMT	None
MA_ActionPlanValidationMT_1: MA_ActionPlanValidationMT	None
MA_ActionPlanValidationMT_2: MA_ActionPlanValidationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.13 Provide and Receive Formation Status

Once the Team Leader and Peers are within a Formation, a transfer of *MA_TaskStatusMT* messages between each other will take place to maintain required communication. These messages are sent periodically to each other, based on the chosen implementation.

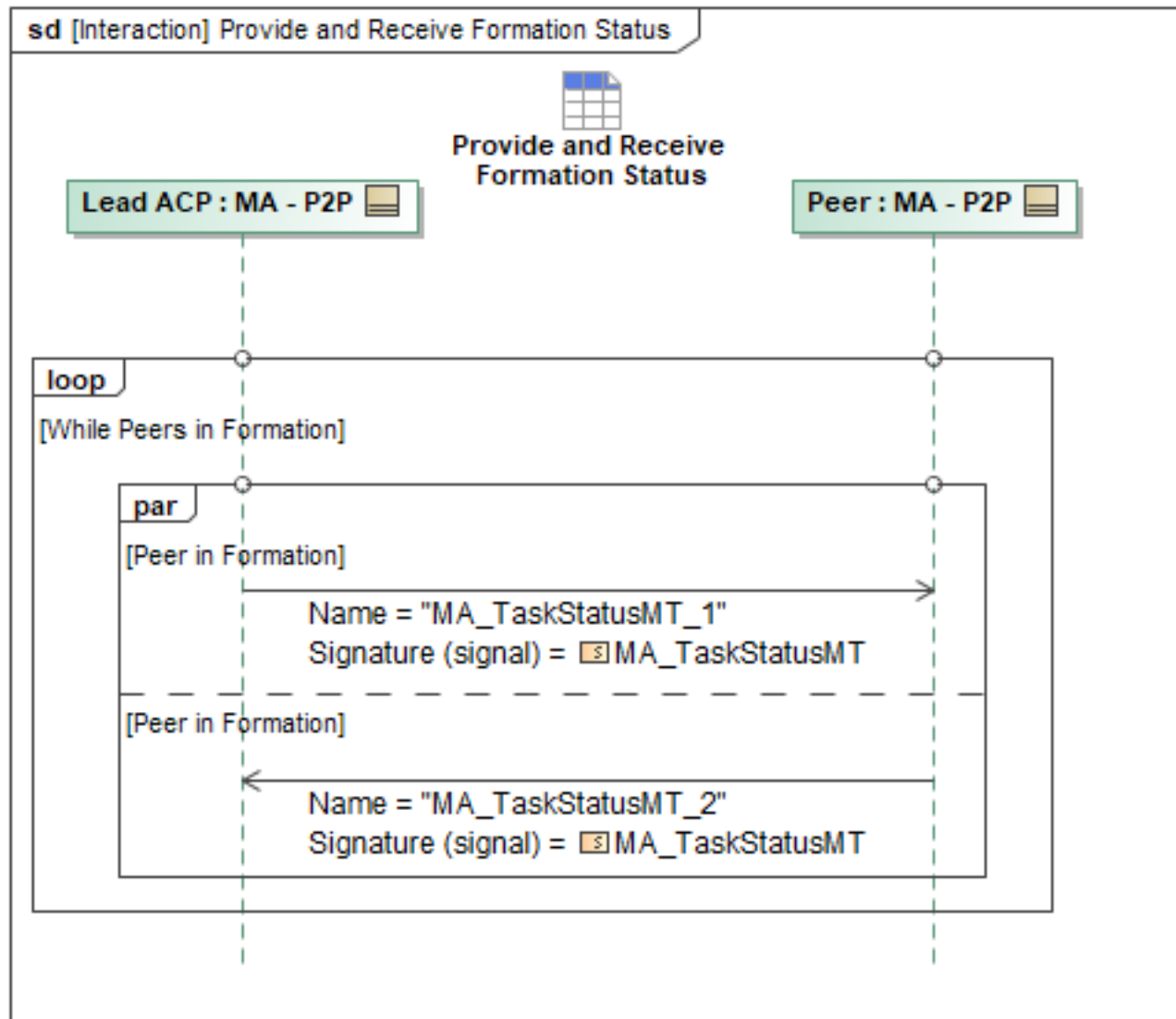


Figure 9-125: Provide and Receive Formation Status

Table 9-125: Provide and Receive Formation Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskStatusMT_1: MA_TaskStatusMT	None
MA_TaskStatusMT_2: MA_TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.14 Report Task Status to Leader

This interaction specifies the Task Status Notification between the Leader and the Follower in a Peer-to-Peer (P2P) interaction. The Follower MA periodically reports *MA_TaskStatusMT* and *TaskPlanExecutionStatusMT* to the Leader MA. Within *MA_TaskStatusMT* and *TaskPlanExecutionStatusMT* is message data describing the condition of the task that was carried out by the followers and the status of the task plan that is associated with the previous mentioned

task. Defined within the *MA_TaskStatusMT* is the *TaskID* which indicates the unique ID of the task whose execution status is being reported to help with identification. Defined within the *TaskPlanExecutionStatusMT* are *SystemID*, *Source*, and *PlanExecutionStatus*. *SystemID* is the unique ID of the System whose Plan is being statused to help with plan identification. *Source* indicates the original location of the data being utilized while *PlanExecutionStatus* indicates the execution state of a *TaskPlan* of the System.

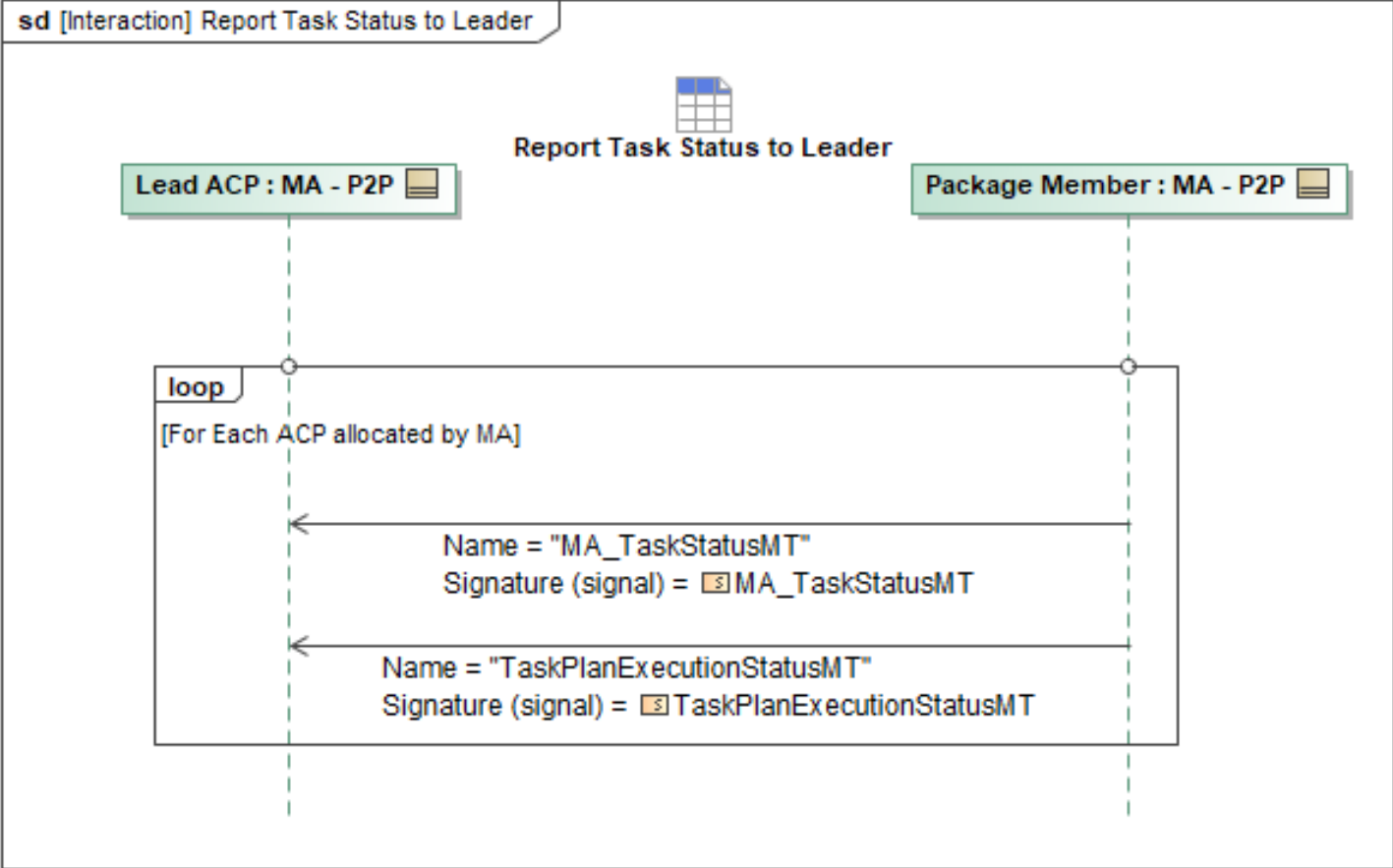


Figure 9-126: Report Task Status to Leader

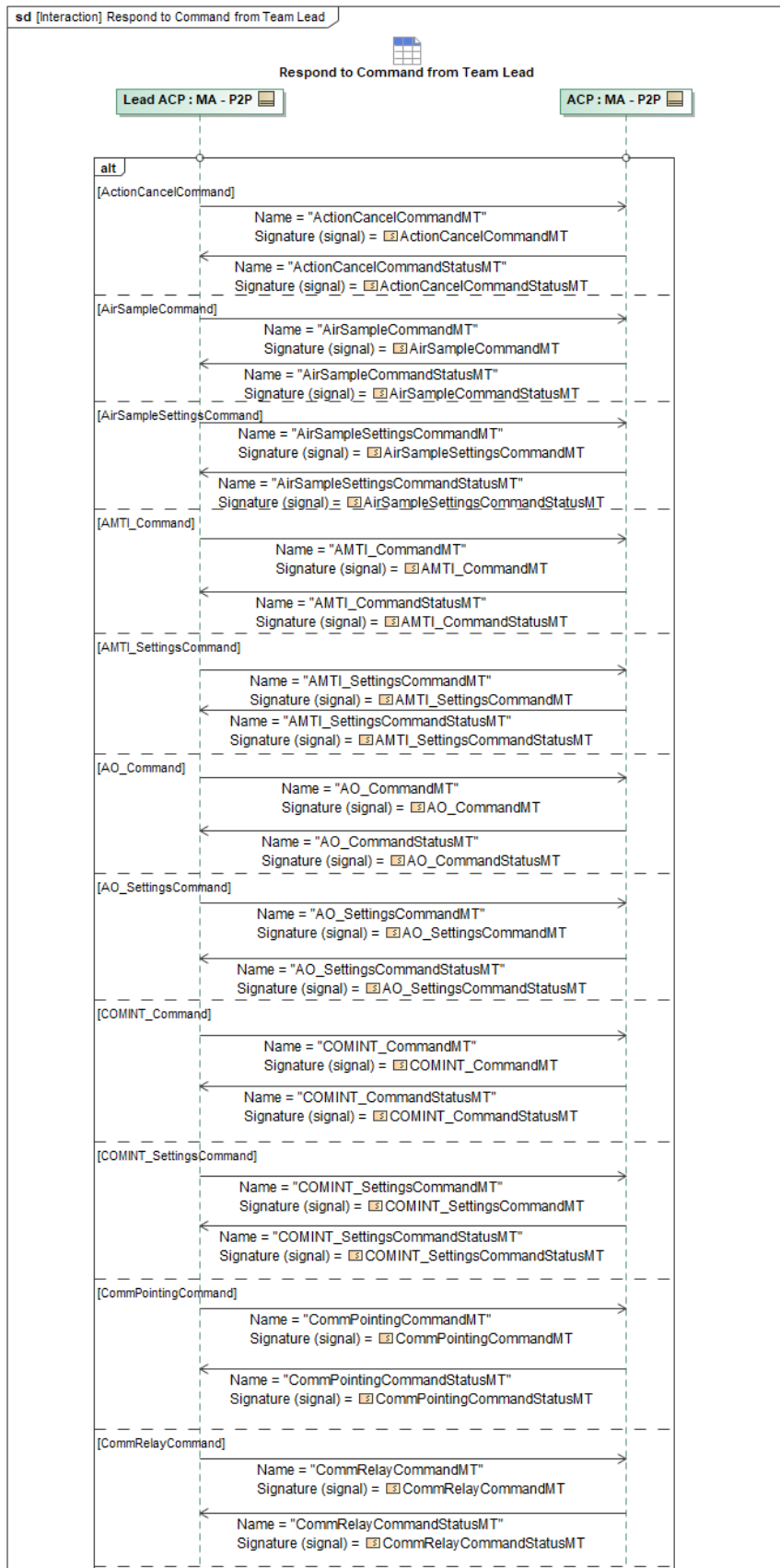
Table 9-126: Report Task Status to Leader

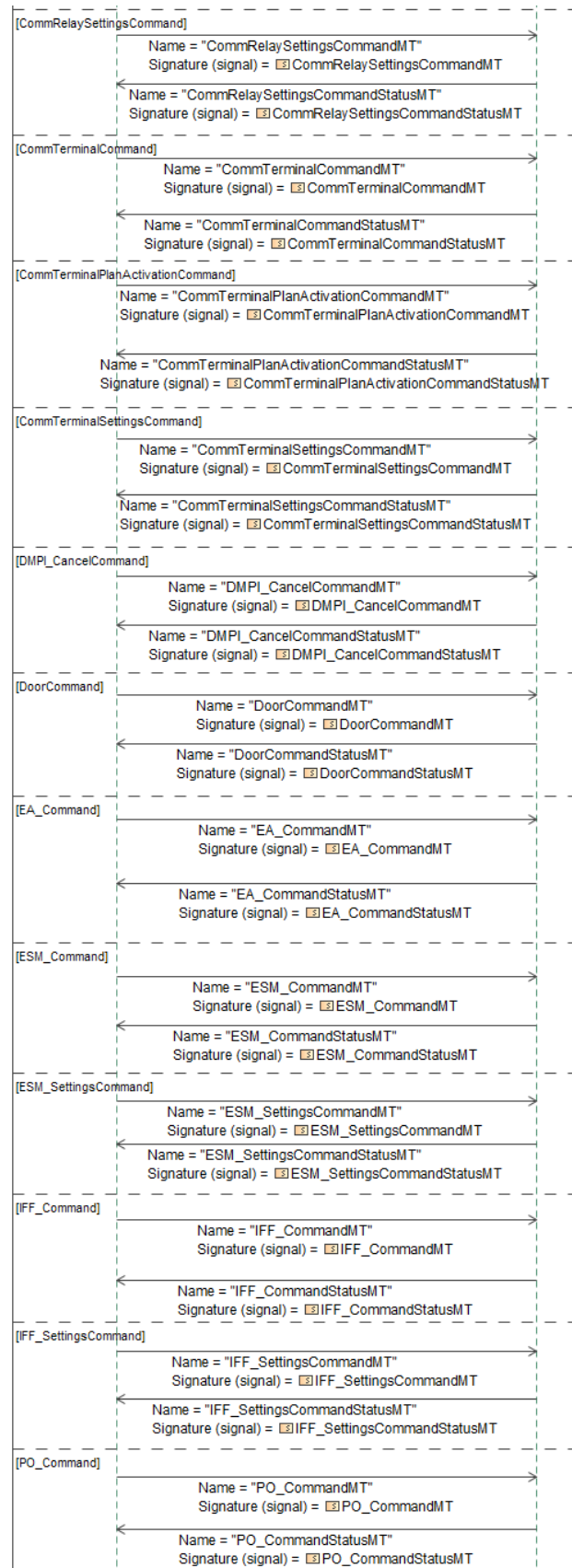
Message Name (Name:Type)	Required Fields (Name:Type)
MA_TaskStatusMT: MA_TaskStatusMT	None
TaskPlanExecutionStatusMT: TaskPlanExecutionStatusMT	PlanExecutionStatus: TaskPlanExecutionStateType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.13.15 Respond to Command from Team Lead

The lead platform that is responsible for distributing command messages to the appropriate platform(s) in the formation. The receiving platform is to then respond back with the respective status message of the original command message sent.







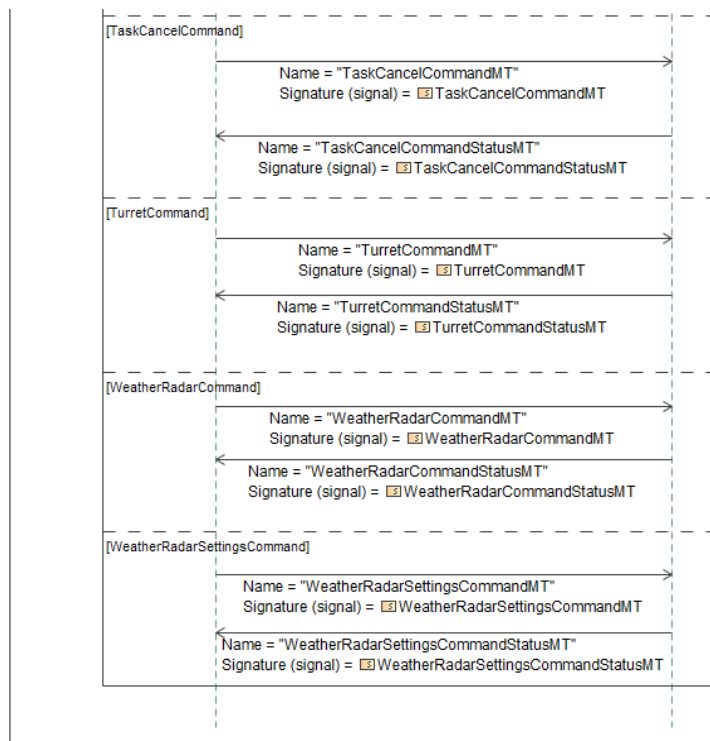


Figure 9-127: Respond to Command from Team Lead

Table 9-127: Respond to Command from Team Lead

Message Name (Name:Type)	Required Fields (Name:Type)
ActionCancelCommandMT: ActionCancelCommandMT	None
ActionCancelCommandStatusMT: ActionCancelCommandStatusMT	None
AirSampleCommandMT: AirSampleCommandMT	None
AirSampleCommandStatusMT: AirSampleCommandStatusMT	None
AirSampleSettingsCommandMT: AirSampleSettingsCommandMT	None
AirSampleSettingsCommandStatusMT: AirSampleSettingsCommandStatusMT	None
AMTI_CommandMT: AMTI_CommandMT	None
AMTI_CommandStatusMT: AMTI_CommandStatusMT	None
AMTI_SettingsCommandMT: AMTI_SettingsCommandMT	None
AMTI_SettingsCommandStatusMT: AMTI_SettingsCommandStatusMT	None
AO_CommandMT: AO_CommandMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
AO_CommandStatusMT: AO_CommandStatusMT	None
AO_SettingsCommandMT: AO_SettingsCommandMT	None
AO_SettingsCommandStatusMT: AO_SettingsCommandStatusMT	None
COMINT_CommandMT: COMINT_CommandMT	None
COMINT_CommandStatusMT: COMINT_CommandStatusMT	None
COMINT_SettingsCommandMT: COMINT_SettingsCommandMT	None
COMINT_SettingsCommandStatusMT: COMINT_SettingsCommandStatusMT	None
CommPointingCommandMT: CommPointingCommandMT	None
CommPointingCommandStatusMT: CommPointingCommandStatusMT	None
CommRelayCommandMT: CommRelayCommandMT	None
CommRelayCommandStatusMT: CommRelayCommandStatusMT	None
CommRelaySettingsCommandMT: CommRelaySettingsCommandMT	None
CommRelaySettingsCommandStatusMT: CommRelaySettingsCommandStatusMT	None
CommTerminalCommandMT: CommTerminalCommandMT	None
CommTerminalCommandStatusMT: CommTerminalCommandStatusMT	None
CommTerminalPlanActivationCommandMT: CommTerminalPlanActivationCommandMT	None
CommTerminalPlanActivationCommandStatus MT: CommTerminalPlanActivationCommandStatus MT	None
CommTerminalSettingsCommandMT: CommTerminalSettingsCommandMT	None
CommTerminalSettingsCommandStatusMT: CommTerminalSettingsCommandStatusMT	None
DMPI_CancelCommandMT: DMPI_CancelCommandMT	None
DMPI_CancelCommandStatusMT: DMPI_CancelCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
DoorCommandMT: DoorCommandMT	None
DoorCommandStatusMT: DoorCommandStatusMT	None
EA_CommandMT: EA_CommandMT	None
EA_CommandStatusMT: EA_CommandStatusMT	None
ESM_CommandMT: ESM_CommandMT	None
ESM_CommandStatusMT: ESM_CommandStatusMT	None
ESM_SettingsCommandMT: ESM_SettingsCommandMT	None
ESM_SettingsCommandStatusMT: ESM_SettingsCommandStatusMT	None
IFF_CommandMT: IFF_CommandMT	None
IFF_CommandStatusMT: IFF_CommandStatusMT	None
IFF_SettingsCommandMT: IFF_SettingsCommandMT	None
IFF_SettingsCommandStatusMT: IFF_SettingsCommandStatusMT	None
PO_CommandMT: PO_CommandMT	None
PO_CommandStatusMT: PO_CommandStatusMT	None
PO_SettingsCommandMT: PO_SettingsCommandMT	None
PO_SettingsCommandStatusMT: PO_SettingsCommandStatusMT	None
RadarAltimeterCommandMT: RadarAltimeterCommandMT	None
RadarAltimeterCommandStatusMT: RadarAltimeterCommandStatusMT	None
RadarAltimeterSettingsCommandMT: RadarAltimeterSettingsCommandMT	None
RadarAltimeterSettingsCommandStatusMT: RadarAltimeterSettingsCommandStatusMT	None
RefuelCommandMT: RefuelCommandMT	None
RefuelCommandStatusMT: RefuelCommandStatusMT	None
RefuelSettingsCommandMT: RefuelSettingsCommandMT	None
RefuelSettingsCommandStatusMT: RefuelSettingsCommandStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
SAR_CommandMT: SAR_CommandMT	None
SAR_CommandStatusMT: SAR_CommandStatusMT	None
SAR_SettingsCommandMT: SAR_SettingsCommandMT	None
SAR_SettingsCommandStatusMT: SAR_SettingsCommandStatusMT	None
SMTI_CommandMT: SMTI_CommandMT	None
SMTI_CommandStatusMT: SMTI_CommandStatusMT	None
SMTI_SettingsCommandMT: SMTI_SettingsCommandMT	None
SMTI_SettingsCommandStatusMT: SMTI_SettingsCommandStatusMT	None
StoreManagementCommandMT: StoreManagementCommandMT	None
StoreManagementCommandStatusMT: StoreManagementCommandStatusMT	None
StrikeCommandMT: StrikeCommandMT	None
StrikeCommandStatusMT: StrikeCommandStatusMT	None
TaskCancelCommandMT: TaskCancelCommandMT	None
TaskCancelCommandStatusMT: TaskCancelCommandStatusMT	None
TurretCommandMT: TurretCommandMT	None
TurretCommandStatusMT: TurretCommandStatusMT	None
WeatherRadarCommandMT: WeatherRadarCommandMT	None
WeatherRadarCommandStatusMT: WeatherRadarCommandStatusMT	None
WeatherRadarSettingsCommandMT: WeatherRadarSettingsCommandMT	None
WeatherRadarSettingsCommandStatusMT: WeatherRadarSettingsCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.14 Peer Threat Assessment Package

9.2.14.1 P2P Threat Assessment

Mission Use Case: ALR

The optional Threat Assessment sequence diagram specifies the interactions where an ACP Peer provides ACP Lead with vehicle threat assessment results. ACP Lead may request an ACP Peer to perform vehicle threat assessment(s) for specific Entity ID's or all known red Entity ID's by sending *MA_AssessmentRequestMT_1* with respective *VehicleThreatAssessmentRequestType* field. ACP Lead can request the assessment to be performed once or periodically at a defined frequency by setting the *RequestFrequencyType* to PERIODIC and adding a value to *RequestFrequencyPeriod*. ACP Lead may also request what capability MA peer will use to perform the vehicle threat assessment (i.e. onboard sensors), however if ACP Lead wants MA peer to use historical tracks provided by track management function, then the *ThreatCapability* enumeration will be *OTHER_UNSPECIFIED_DETECTION*. The ACP will respond once or periodically with a *MA_AssessmentRequestStatusMT_1* to acknowledge the request and provide the assessment. If the request specified the *ResultsInAssessmentMessage* as FALSE, the assessment will be sent in a *MA_AssessmentMT* message. If periodic assessments were requested, MA Lead will send *MA_AssessmentRequestMT_1* with *RequestFrequencyType* set to STOP. MA will respond with the corresponding *MA_AssessmentRequestStatusMT_1* to indicate request complete.

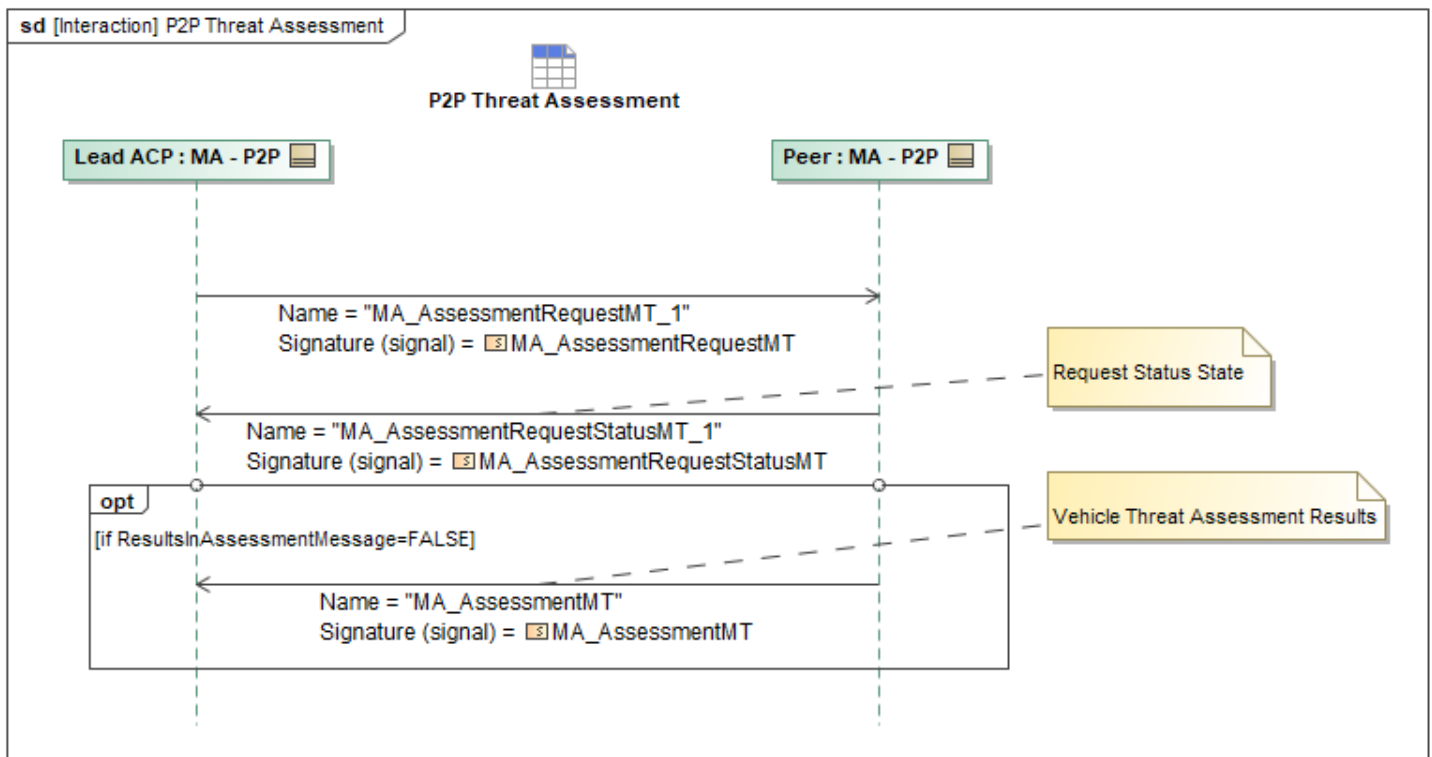


Figure 9-128: P2P Threat Assessment

Table 9-128: P2P Threat Assessment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_AssessmentMT: MA_AssessmentMT	VehicleThreatAssessment: VehicleThreatAssessmentType

Message Name (Name:Type)	Required Fields (Name:Type)
	AssessmentRequestID: RequestID_Type ObjectState: ObjectStateEnum
MA_AssessmentRequestMT_1: MA_AssessmentRequestMT	VehicleThreatAssessment: VehicleThreatAssessmentRequestType
MA_AssessmentRequestStatusMT_1: MA_AssessmentRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15 Peer Weapon Employment Package

9.2.15.1 Centralized Computation of Target Assignment

This interaction describes the ACP lead computing the targeting assignments for ACP followers. First, the ACP lead sends out *AccessAssessmentRequestMT* to request all ACP followers to evaluate and score their capabilities for executing each target. Then, ACP followers send *AccessAssessmentRequestStatusMT* to the ACP lead to report their scores. Once the scores have been received by ACP Lead from all ACP followers to assist in computing targeting assignments, a Lead ACP must consider other variables that go into the assignment of Target-ACP pairings such as Shot Queue, Threat Prioritization, and targeting and intercept maneuver parameters set in a Mission Plan or by C2.

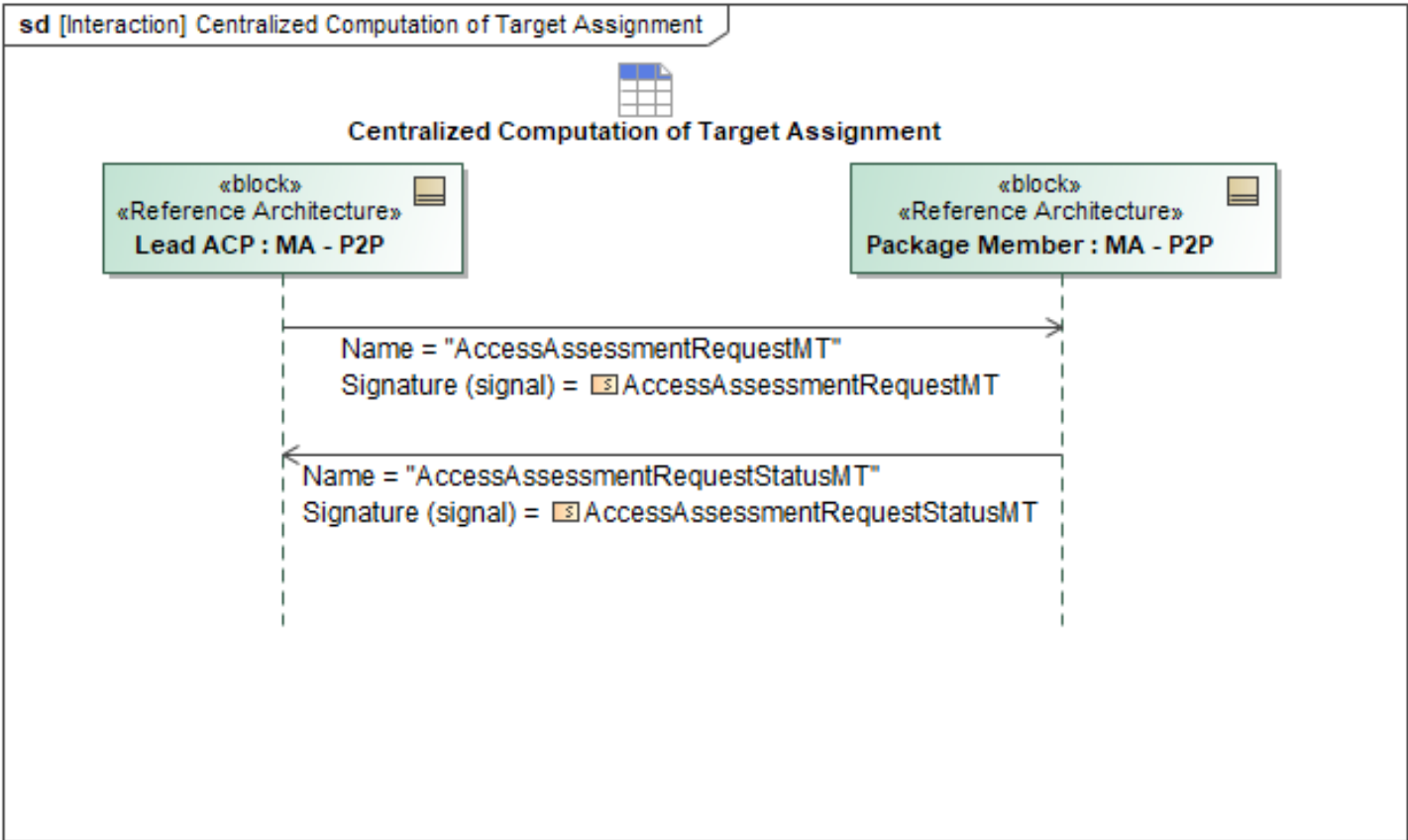


Figure 9-129: Centralized Computation of Target Assignment

Table 9-129: Centralized Computation of Target Assignment

Message Name (Name:Type)	Required Fields (Name:Type)
AccessAssessmentRequestMT: AccessAssessmentRequestMT	EffortToAssess: AccessEffortEnum CapabilitiesToAssess: CapabilityTaxonomyUniversalType
AccessAssessmentRequestStatusMT: AccessAssessmentRequestStatusMT	CompletionStatus: CompletionStatusType ProbabilityOfAccessAttempt: PercentType Result: AccessAssessmentResultType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.2 Distributed Computation of Target Assignment

This interaction describes the ACPs computing the target assignments by themselves. When all ACPs receive notice to start computing the assignments for target execution, every ACP evaluates and scores its capability for executing each target. Then, every ACP sends *AccessAssessmentRequestStatusMT* to all other ACPs to report its scores. When an ACP receives the scores from all other ACPs, it uses the scores to compute the target assignments for all other ACPs. This process leaves the ACPs with computed scores for target pairings. The assignment of targets, along with desired targetting or intercept parameters for these pairings is completed using the "Peer Assign Target Pairings" interaction.

Note:

- In the distributed case, it is critical that every ACP compute the same target assignment for all ACPs.
- This includes ensuring any arbitrary choices (e.g. choosing between targets that have the same score for an ACP) are made the same on all ACPs.

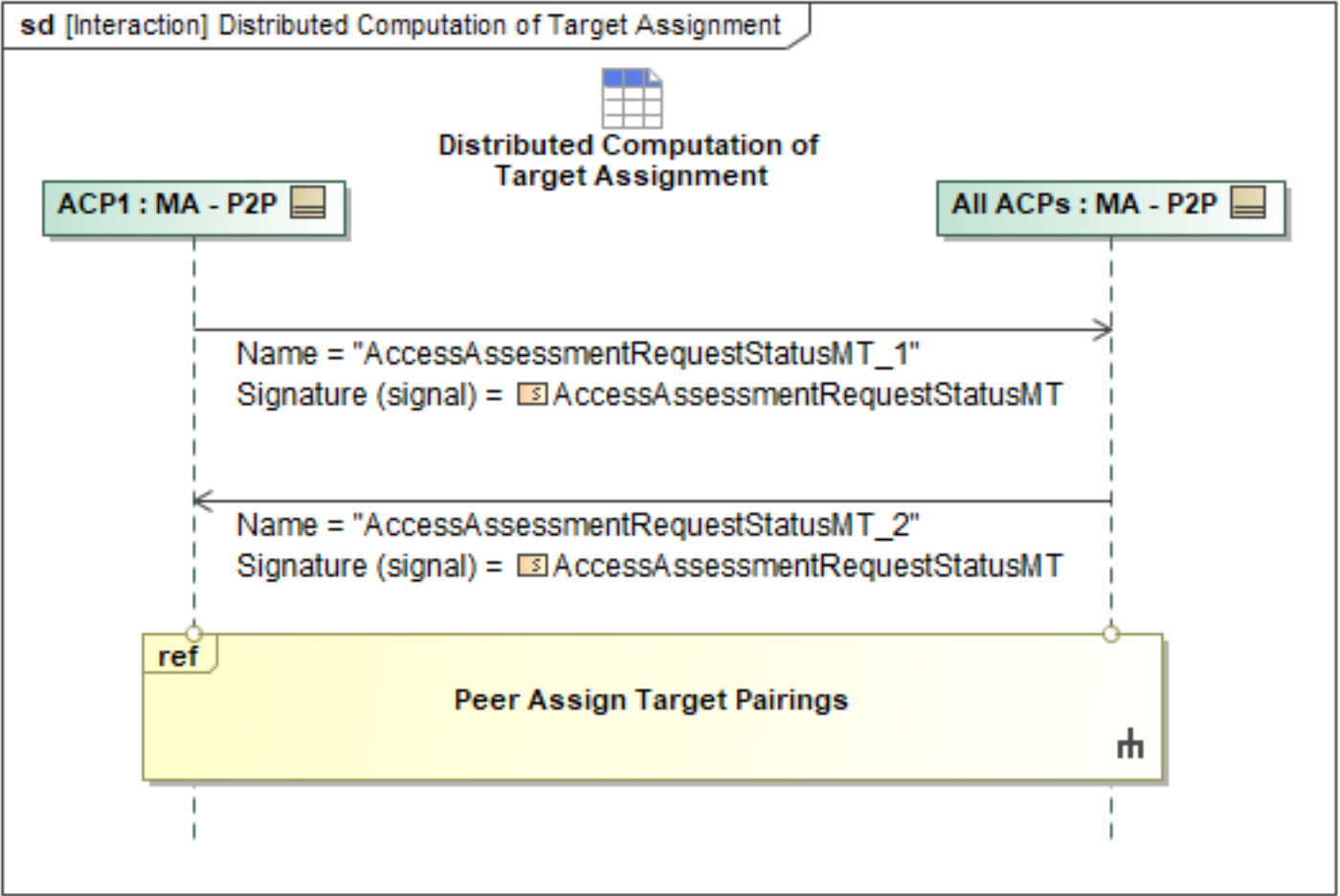


Figure 9-130: Distributed Computation of Target Assignment

Table 9-130: Distributed Computation of Target Assignment

Message Name (Name:Type)	Required Fields (Name:Type)
AccessAssessmentRequestStatusMT_1: AccessAssessmentRequestStatusMT	ProbabilityOfAccessAttempt: PercentType Result: AccessAssessmentResultType CompletionStatus: CompletionStatusType
AccessAssessmentRequestStatusMT_2: AccessAssessmentRequestStatusMT	CompletionStatus: CompletionStatusType ProbabilityOfAccessAttempt: PercentType Result: AccessAssessmentResultType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.3 DLZ Compound Convergence

DLZ Compound Convergence is a fusion of multiple DLZ from team members within a flight group. The fusion of DLZ enables P2P (Team Lead) to generate a composite view of flight group's weapon engagement capabilities to assess to allocate tasking for the best shot. Once ACPs are producing a DLZ in support of strike task, MA will send *DLZ_MT* messages periodically to P2P (Team Lead).

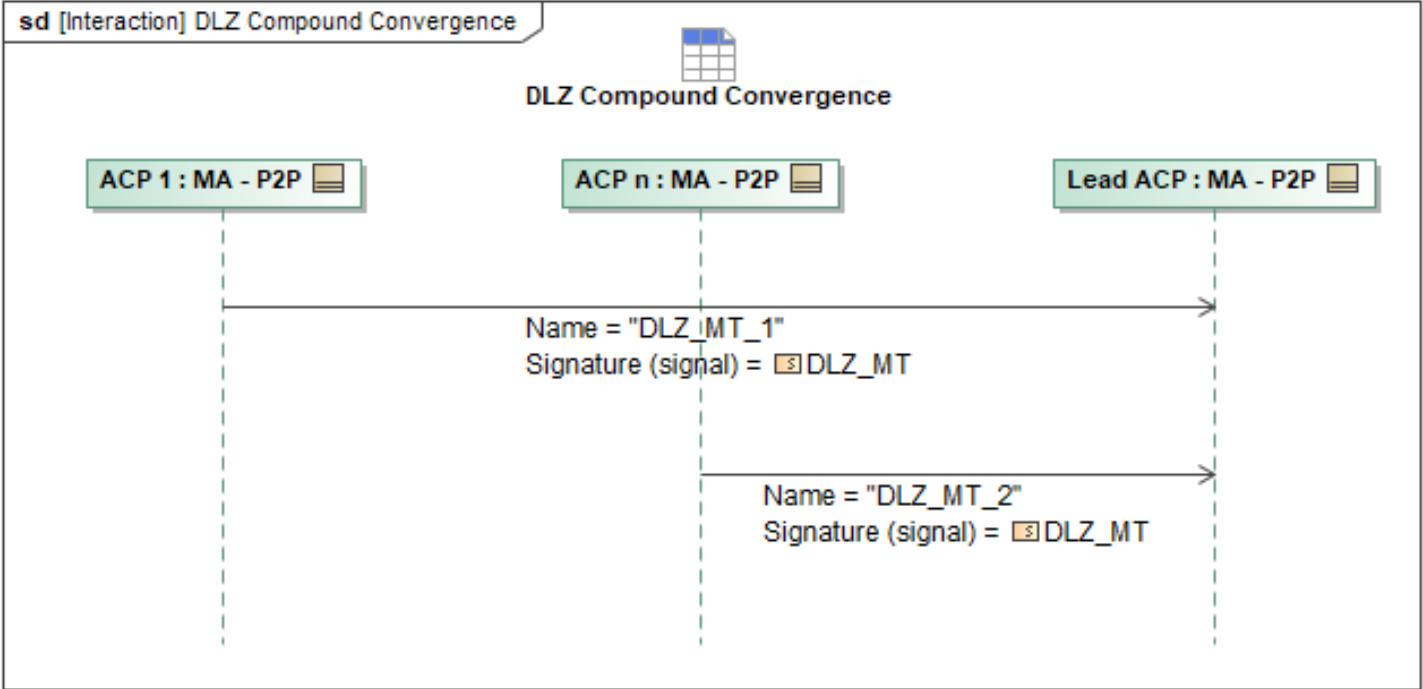


Figure 9-131: DLZ Compound Convergence

Table 9-131: DLZ Compound Convergence

Message Name (Name:Type)	Required Fields (Name:Type)
DLZ_MT_1: DLZ_MT	None
DLZ_MT_2: DLZ_MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.4 Peer Assign Target Pairings

The Assign Target Pairings sequence on the Peer Interface describes how a team leader would assign target pairings to peer ACPs, along with a specific task and accompanying constraints and parameters to engage the paired entity. C2 will likely command an ACP package, via an *ActionCommandMT* message to engage on a group of hostile entities. Lead MA is responsible for fusing various data to compute the best shot allocations and intercept pairings for all of its Peers within the package. This may include the Lead ACP utilizing either a Centralized Computation of Target Assignments, or a Distributed Computation of Target Assignments, which are Peer interactions. Along with these target assignment computation, the Lead ACP will also account for a Shot Queue and Threat Prioritization and Threat Assessment. Once this process has completed, the Lead MA would send a *RelationshipDesignationMT* message. The *RelationshipDesignationMT*

message contains information about the entity which the Peer follower is being tasked to engage. The *Destination* field should indicate the target, likely an *EntityID*, but could be any of the other types contained within the *GeoLocatedObjectType* field. The *Source* field, also a *GeoLocatedObjectType*, indicates the ACP being assigned this target pairing relationship.

The sequential *MA_TaskMT* message is where the leader provides the type of *Task* to be carried out on the given relationship. The *TaskType* will likely be that of a *StrikeTask* for *targeting*, however there are other options given the initial higher level *MA_ActionTypeEnum* that was received by the team leader. Intercept maneuver and shot doctrine parameters would be shared within this *MA_TaskMT* message as applicable. The *MA_TaskMT* message contains the same fields present within an *MA_ActionMT* to denote parameters for *EngagementParameters* within *TaskConstraints*. This sequence does not actually command the Peer to begin executing the engagement but rather provides all of the information the Peer ACP will need to execute the engagement including the ACP-target pairing(s), intercept maneuver parameters, and shot doctrine parameters. The Peer ACP's response is a *MA_SystemNotificationMT* message which acts as an acknowledgment that the data was received.

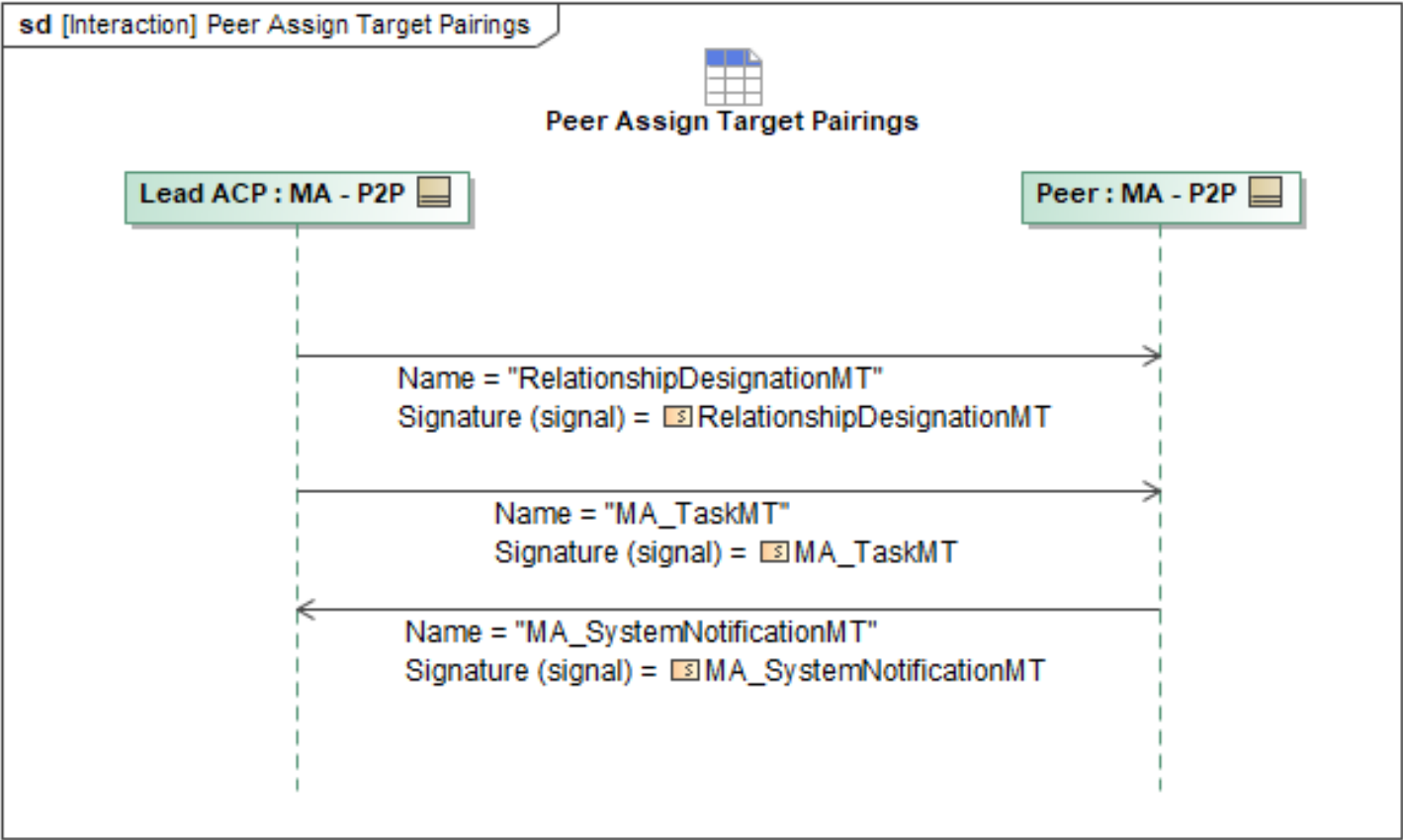


Figure 9-132: Peer Assign Target Pairings

Table 9-132: Peer Assign Target Pairings

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType
MA_TaskMT: MA_TaskMT	TaskType: TaskType

Message Name (Name:Type)	Required Fields (Name:Type)
	TaskID: TaskID_Type TaskConstraints: RequirementConstraintsType EngagementParameters: MA_EngagementParametersType
RelationshipDesignationMT: RelationshipDesignationMT	RelationshipDesignationID: RelationshipDesignationID_Type Pairing: PairingRelationshipEnum Source: GeoLocatedObjectType Destination: GeoLocatedObjectType Relationship: RelationshipType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.5 Peer Report Tactic Status

The Report Tactic Status sequence on the Peer Interface describes a status reporting interaction between a Lead ACP and one of it's followers. The follower will send *MA_ActionStatusMT* periodically to report the status of its commanded tactic as a part of a package. Any time there is an update to the on-going engagement, the follower should update the Lead MA with a new *MA_ActionStatusMT* message.

The *Metric* field within *MA_ActionStatusMT* contains information on *EnduranceUsage*, *Expendables*, *Weapons*, and more specifically a *StoreType*, *StoreCategory*, and *StoreTypeVariant* can be specified. The *ExecutingSystemOrPackage* field will be set to a *SystemID* for peer reporting. This *MA_ActionStatusMT* message will trace to a specific Activity using the *ActivityID* field within *Traceability*. For more information on the tactic being employed, the Peer ACP will send *MA_StrikeActivityMT* periodically to the Lead to inform the lead about the specific Activity it is executing. Information contained in this message will inform the lead on the Peer's *ActivityState*, *EstimatedStateTime* or *EstimatedCompletionTime*, if a weapon has been *SelectedForKeyload* or not, if a weapon is armed via *WeaponArmed*, if a weapon is *SelectedForRelease* or *ReadyForRelease*, and more information on an *AssignedTarget*.

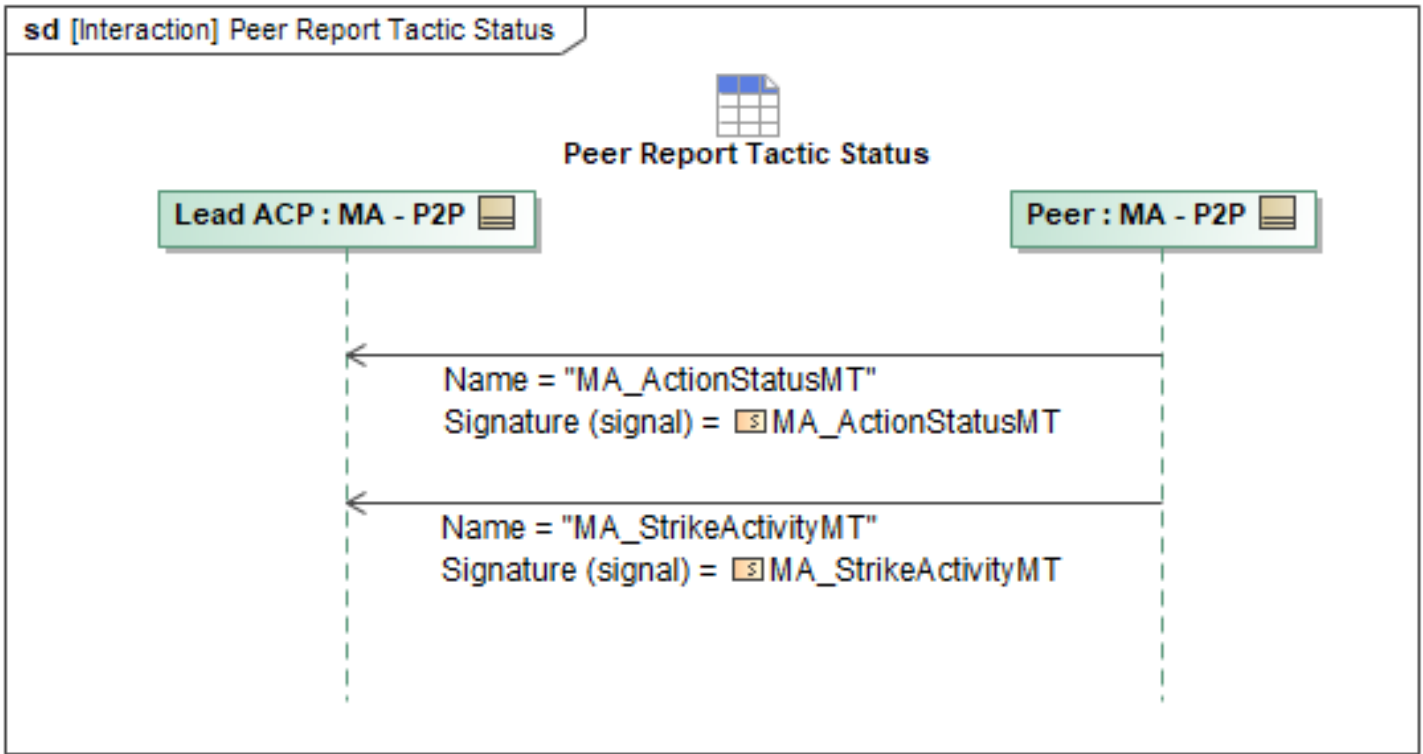


Figure 9-133: Peer Report Tactic Status

Table 9-133: Peer Report Tactic Status

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionStatusMT: MA_ActionStatusMT	SystemID: SystemID_Type ActionID: ActionID_Type ExecutingSystemOrPackage: MA_PackageSystemType
MA_StrikeActivityMT: MA_StrikeActivityMT	Activity: StrikeActivityType CapabilityID: CapabilityID_Type ActivityState: ActivityStateEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.6 Peer Send Tactic Parameters

The Peer Send Tactic Parameters sequence diagram specifies the interaction that enables a lead MA to provide tactic parameter updates to its followers within a package. The Lead MA will send a *MA_ActionMT* message to all follower ACPs.

Specific tactic parameters are contained within the *MA_ActionMT* message that enable sharing of data to inform specific tactics stored as an *Action*. For more information about these tactic parameters and what fields to utilize for detailed shot doctrine and intercept maneuver parameters, see the C2 behavior "Set Tactic Shot Doctrine and Intercept Maneuver Parameters".

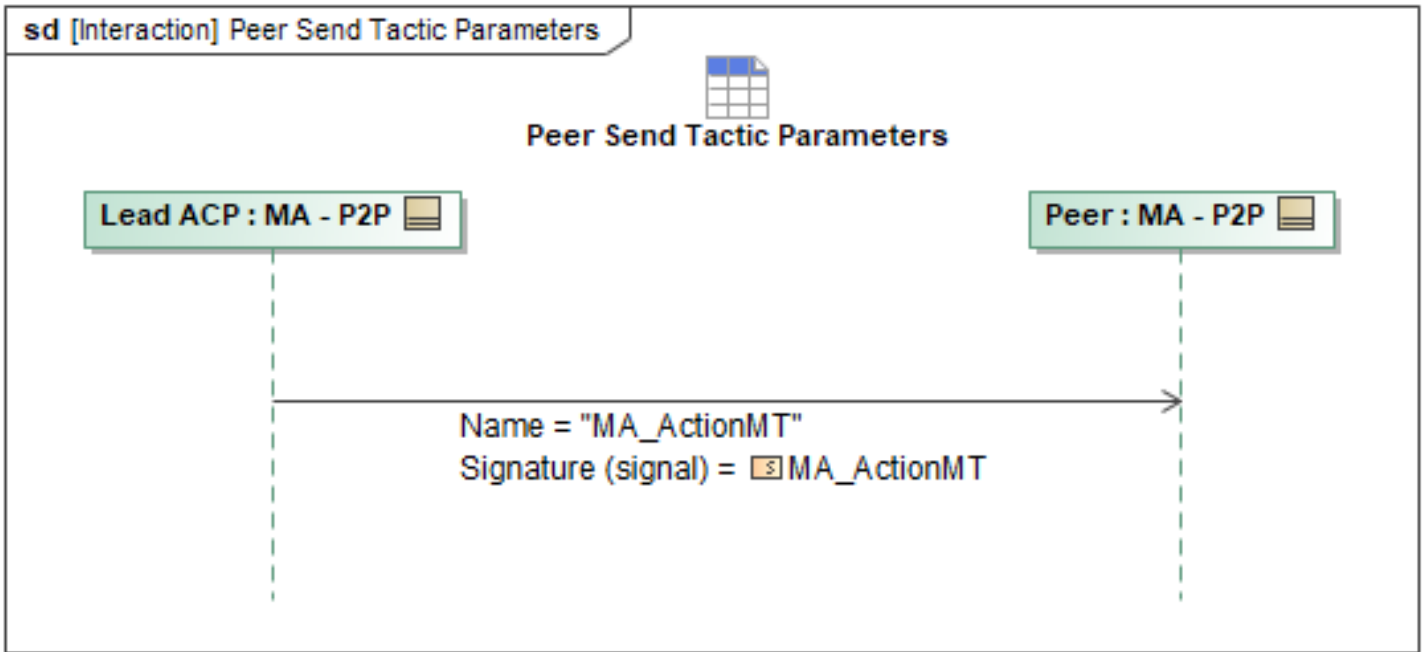


Figure 9-134: Peer Send Tactic Parameters

Table 9-134: Peer Send Tactic Parameters

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ActionMT: MA_ActionMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.7 Peer Target Designation Request

The Peer Target Designation Request sequence diagram specifies the interaction that happens after a Follower MA identifies one or more targets to be Designated. The Target Designation Criteria or the Distributed Computation of Target Assignment sequence can be used to identify Targets for engagement. The MA would then send a *MA_DesignationRequestMT* to the Package Lead. The Package Lead could then use the C2 Target Designation Request sequence to gain authorization from a C2 or respond directly if authorization has already been approved or is not needed. For an engagement, if an authorized C2 has already approved or commanded an engagement against that target to the Package Lead, the Lead may respond directly without another request. The *MA_DesignationRequestMT* contains one or more *DesignationData* fields that contain the target designation information. The *DesignationID* field is a specific identifier for that specific target. The Target field can be set to many items, but for an engagement, the Target Designation Criteria is only applicable to *EntityMTs*. The *TypeOfDesignation* field would likely be set to *DESIGNATED_TARGET* but could also be set to *DESIGNATED_NO_TARGET* if the service determines the target should no longer be a target. The *AppliesToID* field will specify the *SystemID* or *PackageID* which shall be assigned to the target. The use of *PackageID* allows a Lead ACP to delegate the target assignment, whereas *SystemID* allows for direct assignment to a specific ACP. If sending the message to C2, the package Lead would set the *AppliesToID* to the ID of the Package and will only send a single *MA_DesignationRequestMT* to C2 when receiving the same Target Designation Request from

multiple package followers. The C2 Target Designation Request section describes the Package Lead and C2 messaging sequence. The optional *DesignationObjective* field allows for specification of a Requirement that the ACP should use against the target. For an engagement, these Requirement types would match what is in the *ApprovalPolicyMT* that is linked to the Target Authority. If C2 sends a *MA_DesignationMT* with an *AppliesToID* set to the Package, the Package Lead can assign targeting as needed using the Peer Target Designation sequence. For an engagement, the Centralized Computation of Target Assignment sequence or the Distributed Computation of Target Assignment sequence can be used depending on the Package configured to assign the targets. The Package Lead will respond to the follower with a *DesignationRequestStatusMT* populating the *RequestID* field with the same ID as was in the request. As mentioned before for an engagement, the Distributed Computation of Target Assignment sequence could use *MA_DesignationRequestMT(s)* messages to request the lead perform the final allocation of targets to followers, and the lead would then use the Peer Target Designation sequence. The *RequestProcessingState* field will be filled out as needed with the main options being *COMPLETED* (if C2 or Lead sent the *MA_DesignationMT* messages) or *REJECTED* (if C2 or Lead did not send the *MA_DesignationMT* messages). The *DesignationDataStatus* field will be populated with the relevant target data from the request message, including the associated *DesignationID* and the *RequestProcessingState* with similar options as stated above. There is an optional *RequestProcessingStateReason* field that can be filled out if desired.

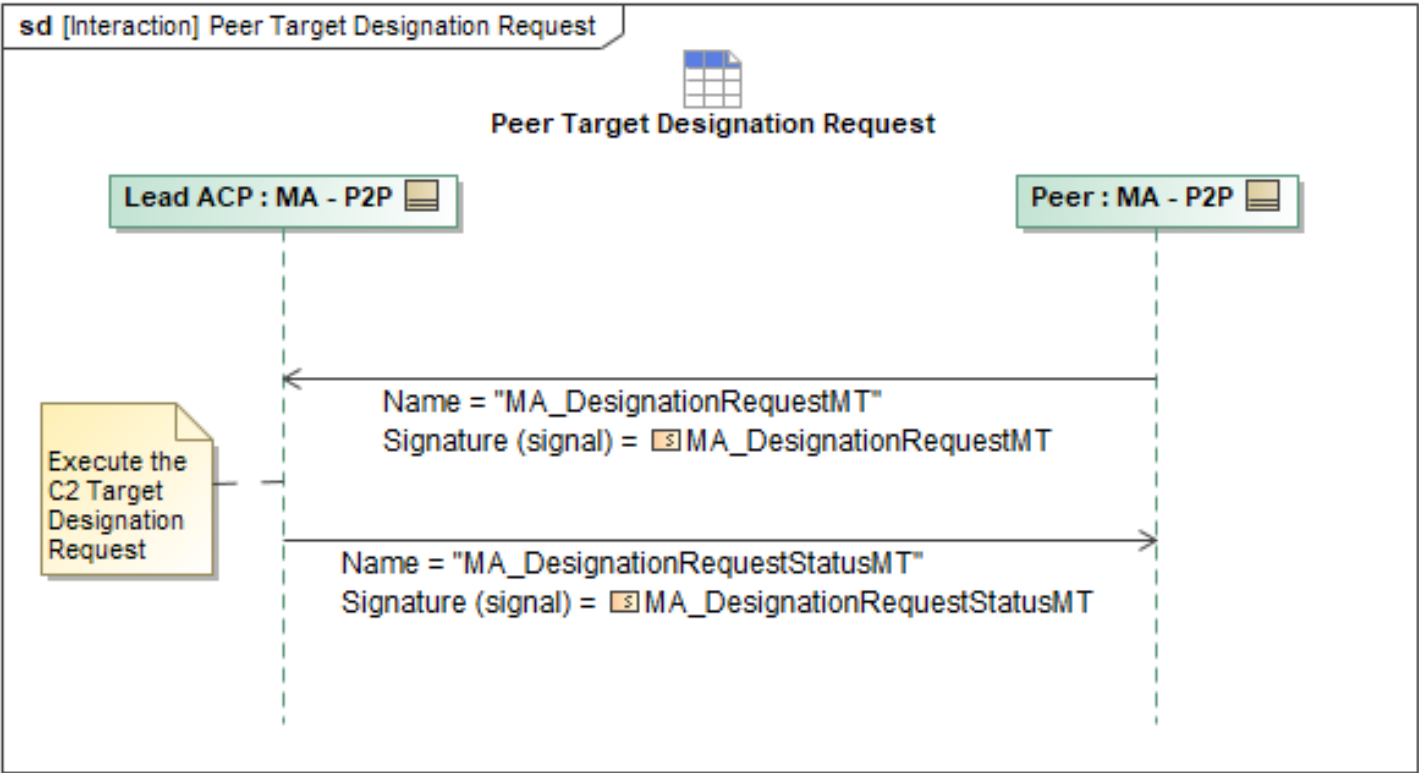


Figure 9-135: Peer Target Designation Request

Table 9-135: Peer Target Designation Request

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationRequestMT: MA_DesignationRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_DesignationRequestStatusMT: MA_DesignationRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.8 Query Entity History

The following sequence is meant to standardize the interface between MA (Flight Lead) and Peer to query request for pulling entity track management historical or entity propagation data from the peer node.

The query for entity historical and propagation from MA to Peer node will be using the existing UCI message : *QueryDataRequest*.

- *ResultsInNative* extension needs *MessageType* & *ReceiverAddress* for querying *Entity* & *EntityPropagation* historical data from C2 to MA. This would not serve the purpose since it doesn't specify a *EntityId(s)* that need to be queried.
- *QueryMessage* extension needs *MessageType* & *QueryPET* (Polymorphic Extension Type) for querying *Entity* & *EntityPropagation* historical data. *QueryPET* a polymorphic extension type that is the base for *QueryType*. When creating a *QueryDataRequest* message, *QueryType* will replace the *QueryPET*.

To create a query, *QueryDataRequest* uses *QueryDataRequest.MessageData.QueryMessage*. An example *QueryDataRequest* (starting from the query element) shows querying for *EntityID* from the historical *Entity* messages stored by MA service (Entity Management Service). The query can be combined for querying both *Entity* & *EntityPropagation* messages (historical) in the *QueryType* for *EntityID*.

<!-- Example 1 --->

<QueryMessage>

<MessageType>ENTITY</MessageType>

<Query xsi:type="uci:QueryType">

<Equals>

<Step><Element><Name>MessageData</Name></Element></Step>

<Step><Element><Name>EntityID</Name></Element></Step>

<Value><UUID>xxxxxxx</UUID></Value>

</Equals>

</Query>

</QueryMessage>

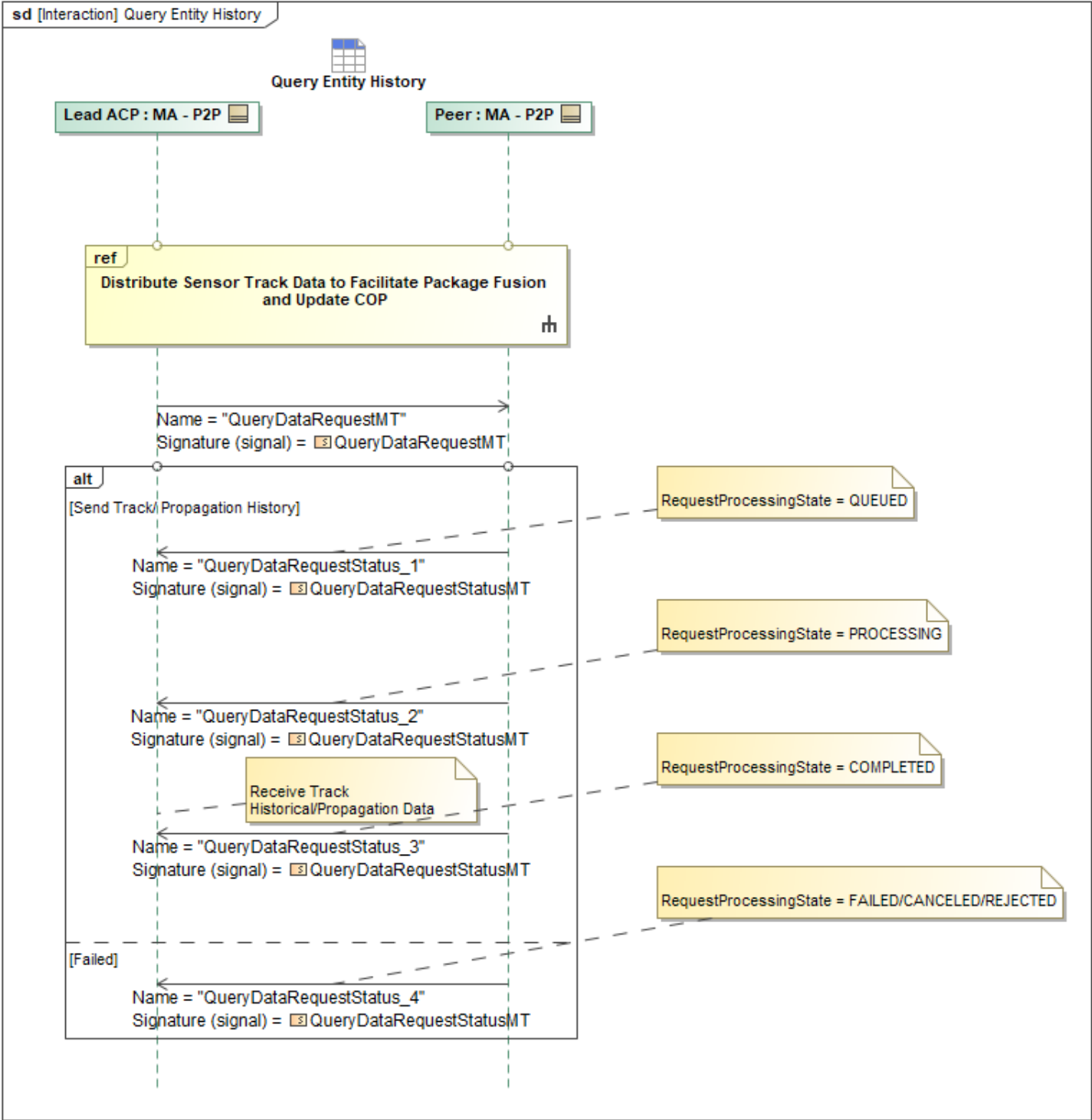


Figure 9-136: Query Entity History

Table 9-136: Query Entity History

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatus_1: QueryDataRequestStatusMT	None
QueryDataRequestStatus_2: QueryDataRequestStatusMT	None
QueryDataRequestStatus_3: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType
QueryDataRequestStatus_4: QueryDataRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.9 Query Entity Track History

The Track Management functionality for multi-ship/single-ship weapon employment is to maintain target tracks historical data along with entity predictions data communicated from Peer to MA when requested using a data query request message. MA queries the P2P nodes to get the historical and prediction track data. The track data may be used by other functions such as sending external track reports, stimulating alerts/notifications if rule sets are triggered, and updating the track display, history, and attribute data. MA will be getting Entity data from the COP, so the Track Management Data Fusion engine will take the COP data and its sensor data to build track history. Track propagation service takes the track state updates and predicts the extrapolated position (Kinematics) of an entity in the battle space and maintains these as a database within MA.

The query for entity and propagation data from MA to P2P will be using the existing UCI message : *QueryDataRequest*.

- *ResultsInNative* field needs *MessageType* & *ReceiverAddress* for querying *EntityPropagation* historical data from MA to P2P. This would not serve the purpose since it doesn't specify *EntityId(s)* that need to be queried.
- *QueryMessage* field needs *MessageType* & *Query PET* (Polymorphic Extension Type) for querying *Entity* and *EntityPropagation* historical data from MA to P2P.

To create a query, *QueryDataRequest UCI message* uses *QueryMessage* field. For example, *QueryDataRequestMT* (starting from the query element) shows querying for *entityID* from the historical *Entity* messages stored by MA's Entity Management Service. The query can be combined for querying both *Entity* & *EntityPropagation* messages (historical) in the *QueryType* for *entityid*.

<!-- Example 1 -->

<QueryMessage>

<MessageType>ENTITY</MessageType>

<Query xsi:type="uci:QueryType">

<Equals>

<Step><Element><Name>MessageData</Name></Element></Step>

<Step><Element><Name>EntityID</Name></Element></Step>

<Value><UUID> xxxxxxxxxx</UUID></Value>

</Equals>

</Query>

</QueryMessage>

MA receives *QueryDataRequestStatusMT_1* response from the P2P node with intermediate response of *RequestProcessingState* field set to *QUEUED*, followed by *QueryDataRequestStatusMT_2* response with *RequestProcessingState* set to *PROCESSING*. Upon successful query for the entity track management or propagation data, MA would receive *QueryDataRequestStatusMT_3* message with the entity database along with the *RequestProcessingState* set to *COMPLETED*. A failure response can result in a *QueryDataRequestStatus_4* with *RequestProcessingState* set to *FAILED* or *REJECTED* or *CANCELED* from P2P node to MA. For any non-compliance related issues, *QueryDataRequestStatusMT_4* response is sent with *RequestProcessingStateReason* set to appropriate non-compliance code.

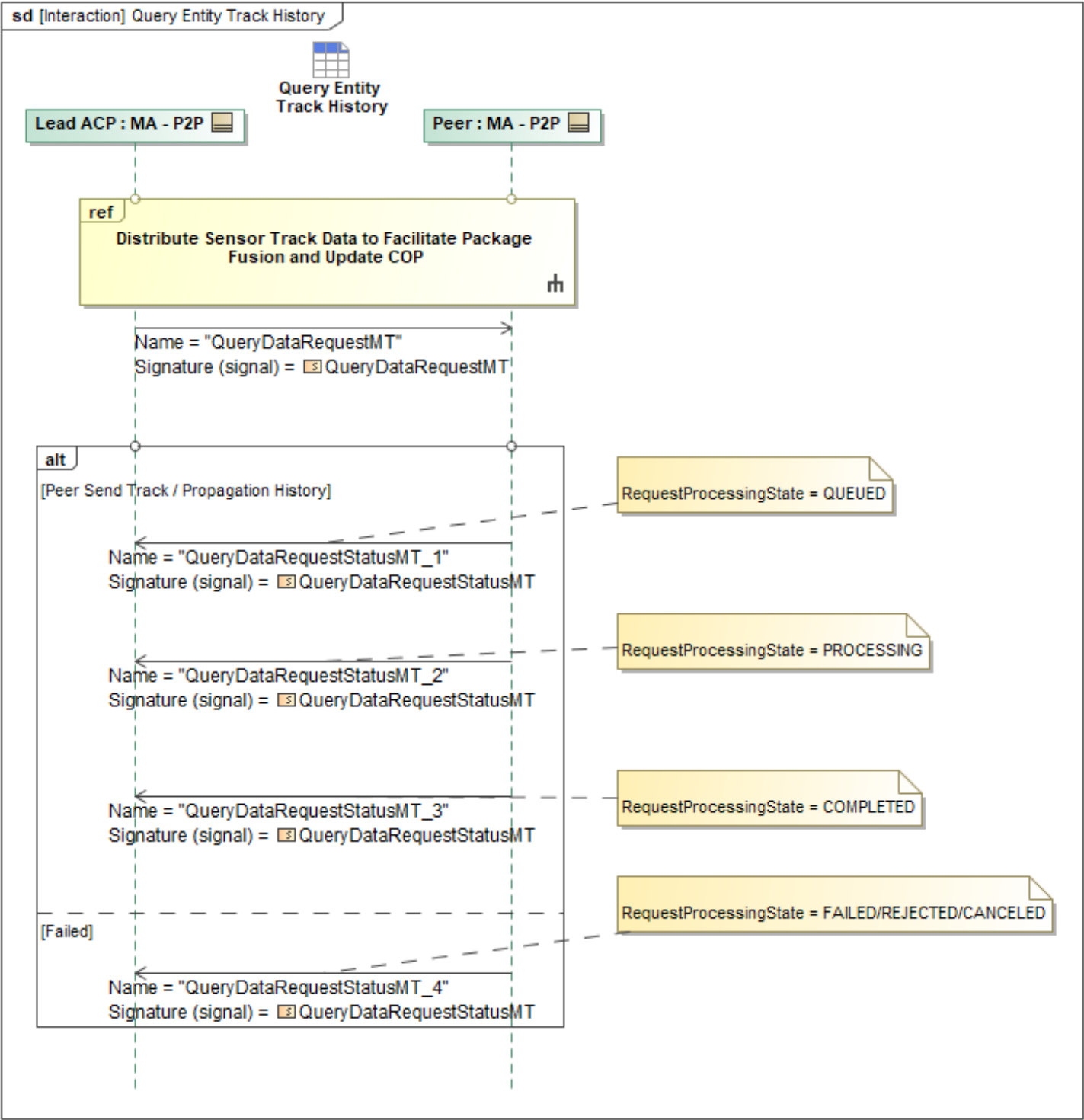


Figure 9-137: Query Entity Track History

Table 9-137: Query Entity Track History

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	Query: QueryMessageType

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.10 Query Shot Queue Prioritization

Shot Queue Prioritization List is a list of mission targets ranked by assessing multiple factors to determine which target entities should be engaged first. The shot queue will utilize *MA_PrioritizationListMT* with a *ListType* of *MISSION_TARGET_LIST* enumeration. The list requires subjects (*EntityID*) to be assigned a priority, where the lowest assigned value is the highest priority level. The message also includes an optional field, *ProbabilityOfKill*, that may be included with each assessed subject entity. P2P (Multi-Ship Lead) may query (*QueryDataRequest*) an ACP (Multi-Ship Peer) to request *MA_PrioritizationListMT* in its native form. MA will respond with *QueryDataRequestStatusMT* to provide state changes (Queued "_1", Processing "_2", and Completed "_3") in the processing of the request as depicted in the sequence diagram. MA will send *MA_PrioritizationListMT* result in native form to the requesting *SystemID*.

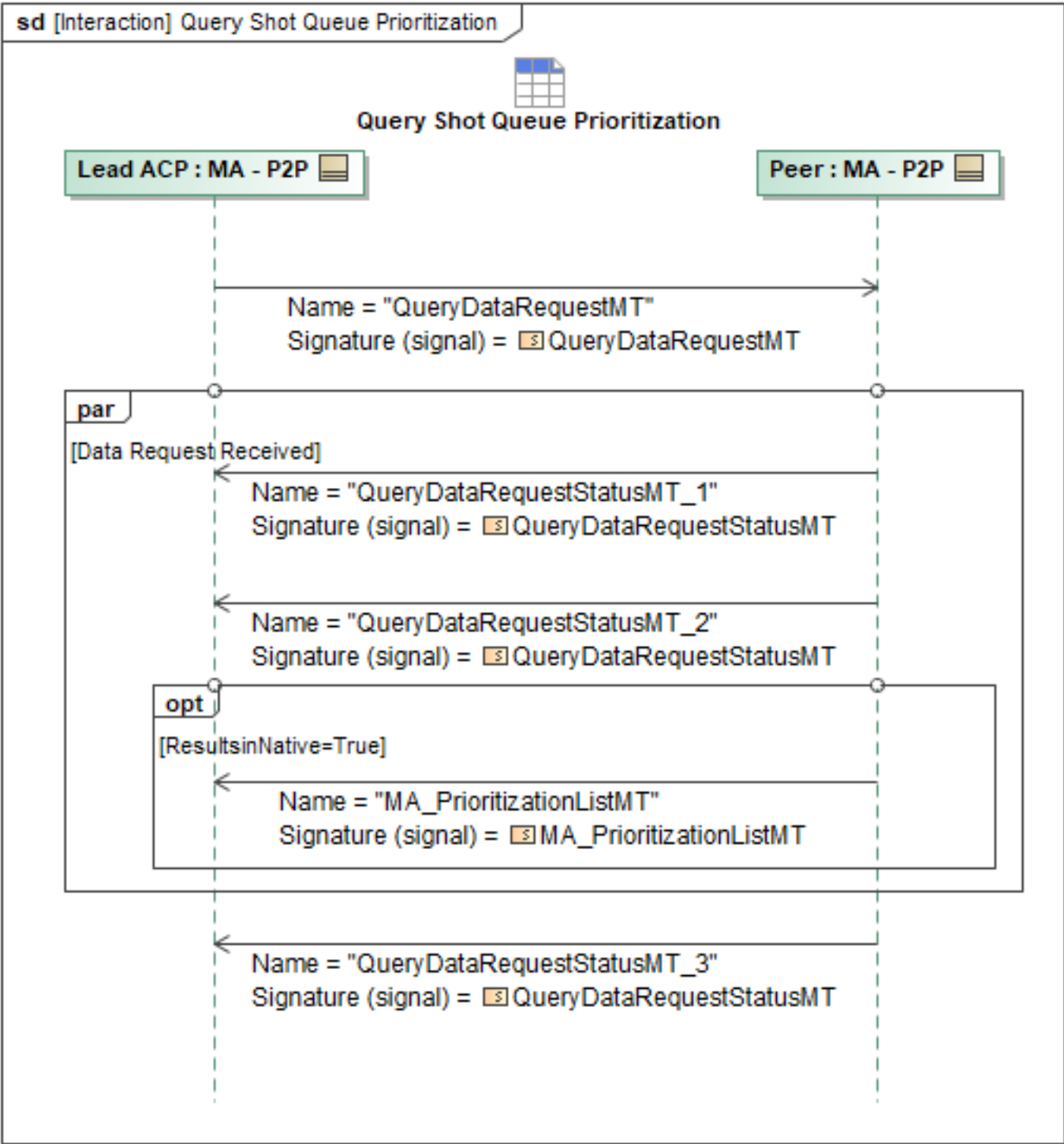


Figure 9-138: Query Shot Queue Prioritization

Table 9-138: Query Shot Queue Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	MISSION_TARGET_LIST: ObjectState: ObjectStateEnum

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	ResultsInNative: QueryTopicPairType
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.11 Send Shot Queue Prioritization

Shot Queue Prioritization List is a list of mission targets ranked by assessing multiple factors to determine which target entities should be engaged first. The shot queue will utilize *MA_PrioritizationListMT* with a *ListType* of *MISSION_TARGET_LIST* enumeration. The list requires subjects (*EntityID*) to be assigned a priority, where the lowest assigned value is the highest priority level. The message also includes an optional field, *ProbabilityOfKill*, that may be included with each assessed subject entity. MA (Multi-Ship Peer) may send an updated shot queue prioritization list to P2P (Multi-ship Lead) based on the active engagement criteria. Once *MA_PrioritizationListMT* is received, P2P (Multi-ship Lead) will respond to the originating System with an *MA_SystemNotificationMT*.

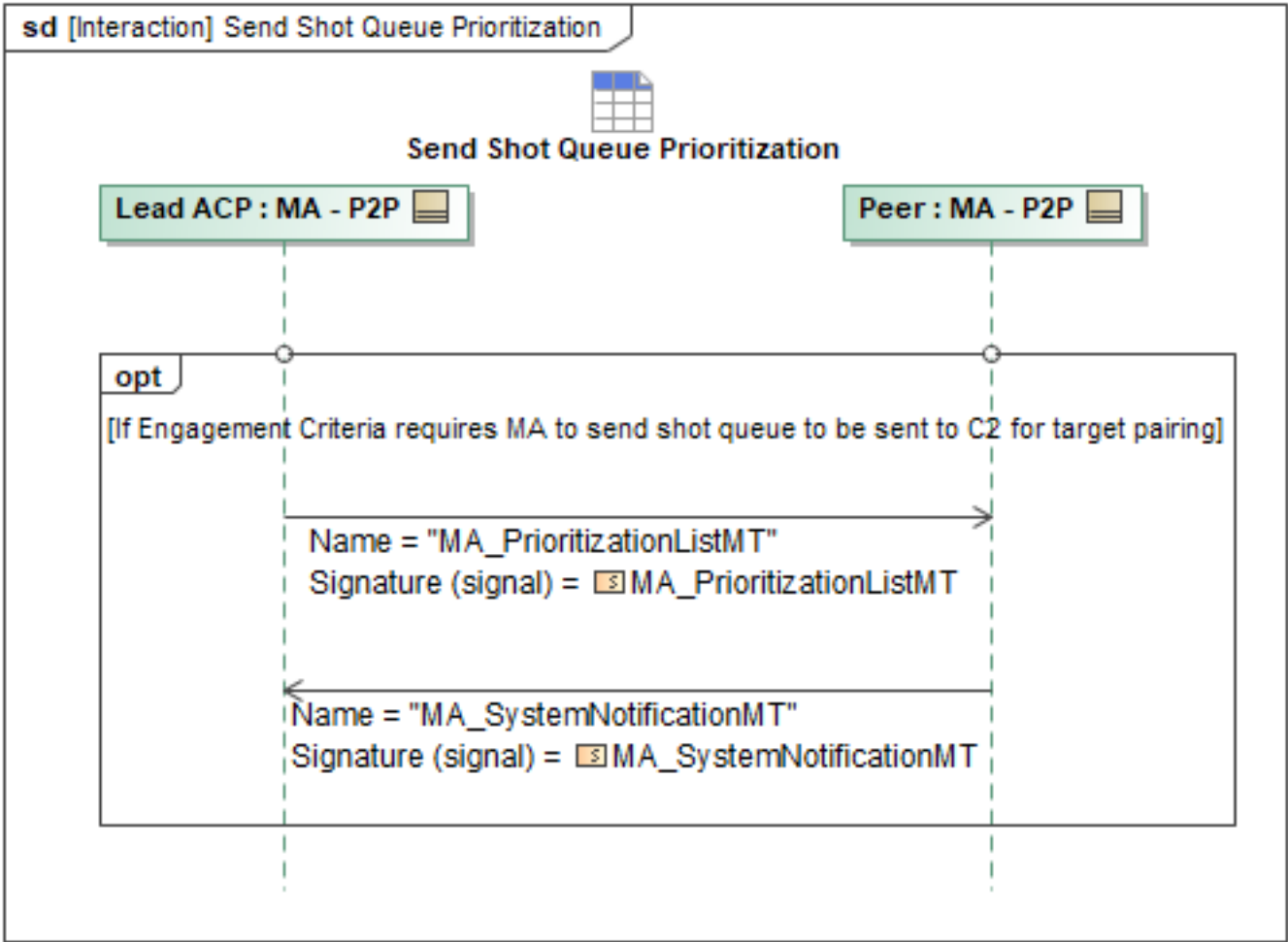


Figure 9-139: Send Shot Queue Prioritization

Table 9-139: Send Shot Queue Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	MISSION_TARGET_LIST: ObjectState: ObjectStateEnum
MA_SystemNotificationMT: MA_SystemNotificationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.12 Threat Prioritization

Mission Use Case: ALR

The Threat Prioritization List ranks entities by assessing the movements and behaviors to determine threat levels to a respective position, whether it is the ACP ownership or another platform. Threat Prioritization will utilize *MA_PrioritizationListMT* with a ListType of THREAT_TARGET_PRIORITIZATION_LIST enumeration. The list requires subjects (*EntityID*) to be assigned a priority, where the lowest assigned value is the highest threat level. Once an updated

MA_PrioritizationListMT is received, P2P will respond to the originating System with an *MA_SystemNotificationMT*.

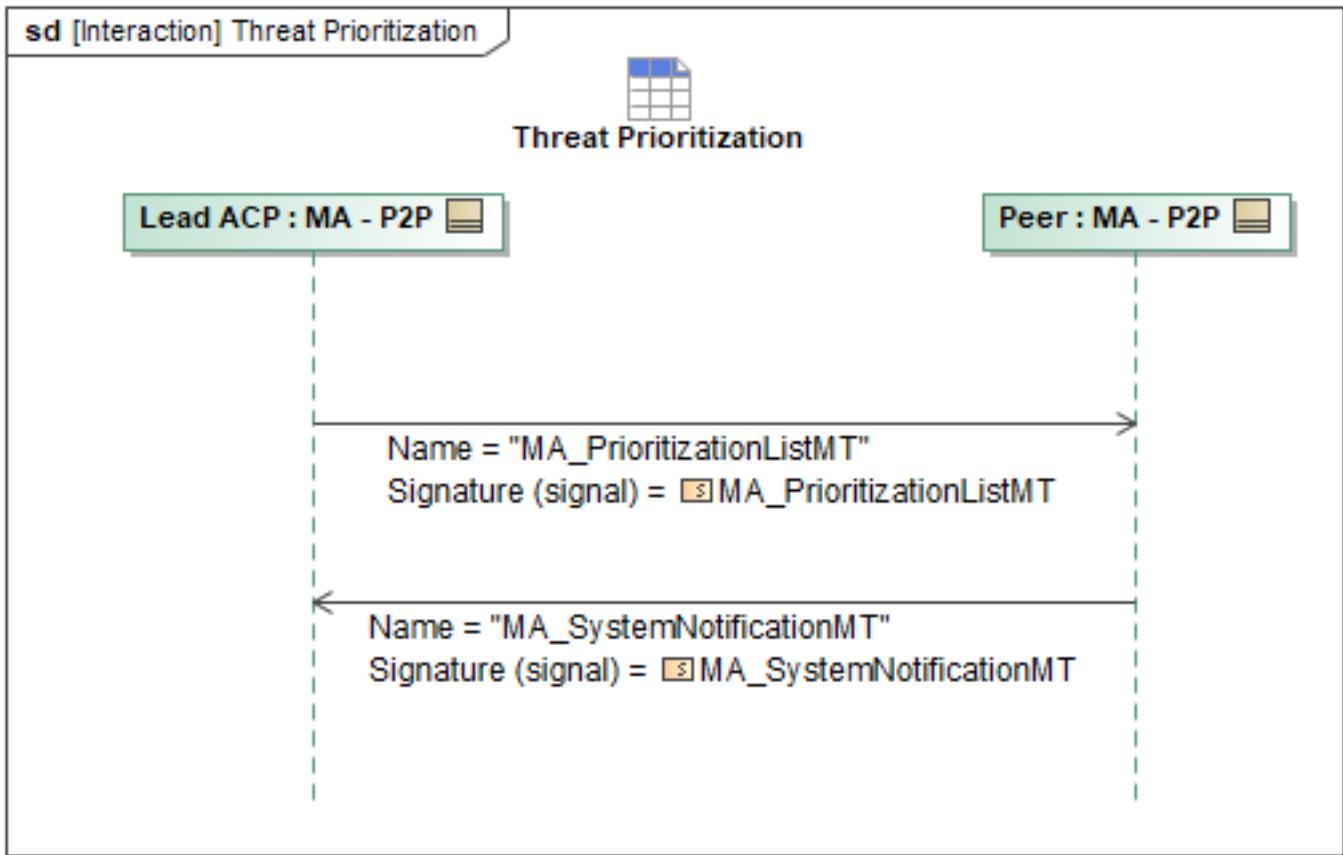


Figure 9-140: Threat Prioritization

Table 9-140: Threat Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	EntityID: EntityID_Type ObjectState: ObjectStateEnum THREAT_TARGET_PRIORITIZATION_LIST: THREAT_TARGET_PRIORITIZATION_LIST: ObjectState: ObjectStateEnum
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.13 Update Shot Queue Prioritization

Shot Queue Prioritization List is a list of mission targets ranked by assessing multiple factors to determine which target entities should be engaged first. The shot queue will utilize *MA_PrioritizationListMT* with a *ListType* of MISSION_TARGET_LIST enumeration. The list requires

subjects (*EntityID*) to be assigned a priority, where the lowest assigned value is the highest priority level. The message also includes an optional field, *ProbabilityOfKill*, that may be included with each assessed subject entity. P2P (Multi-ship Lead) may send an updated shot queue prioritization list an ACP (Multi-Ship Peer). Once an updated *MA_PrioritizationListMT* is received, MA will respond to the originating System with an *MA_SystemNotificationMT*.

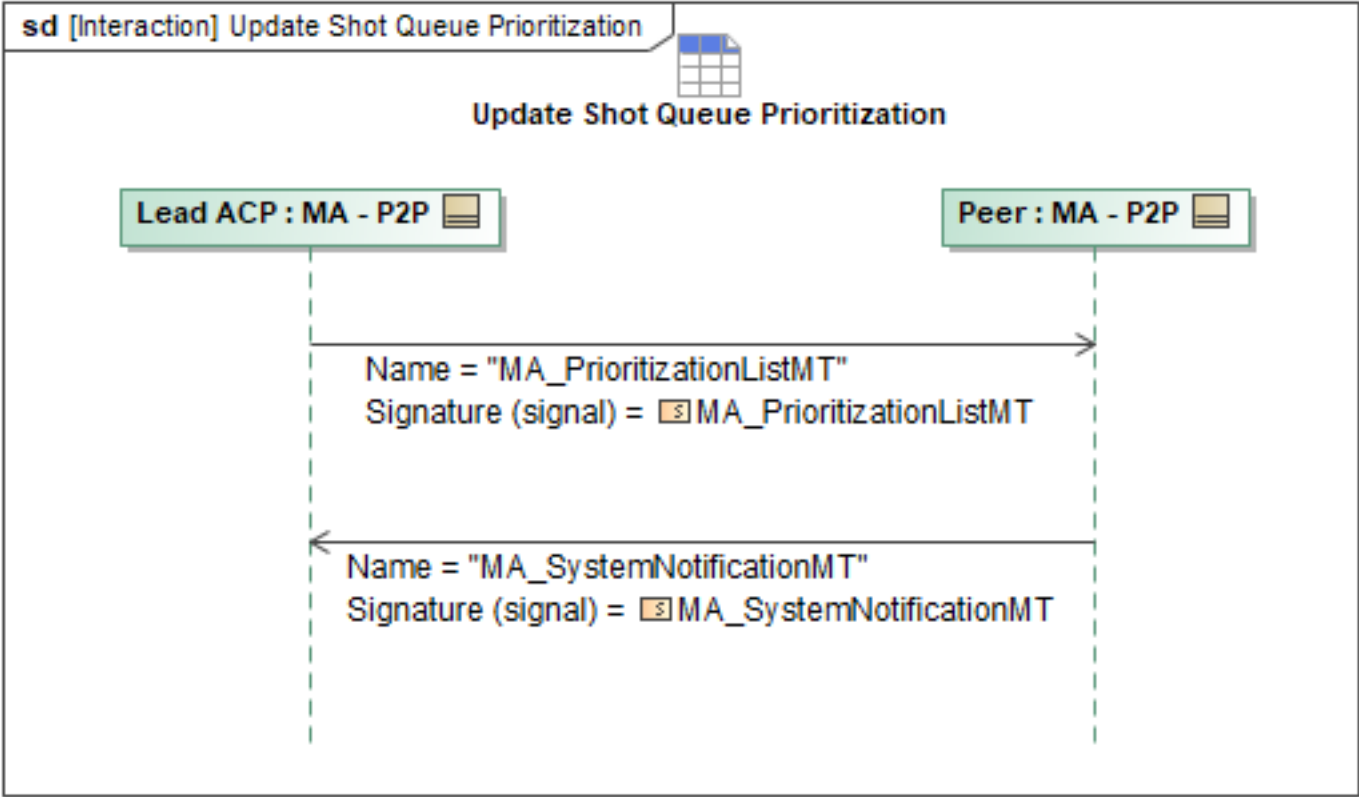


Figure 9-141: Update Shot Queue Prioritization

Table 9-141: Update Shot Queue Prioritization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_PrioritizationListMT: MA_PrioritizationListMT	MISSION_TARGET_LIST: ObjectState: ObjectStateEnum
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.2.15.14 WEZ Compound Convergence

WEZ Compound Convergence is a fusion of multiple WEZ from team members within a flight group. The fusion of WEZ enables P2P (Team Lead/QB) to generate a composite view of flight group’s weapon engagement capabilities to assess tasking for the best shot. Once ACPs are producing a WEZ in support of strike task, MA will send *MA_WEZ_MT* periodically to P2P (Team Lead /QB).

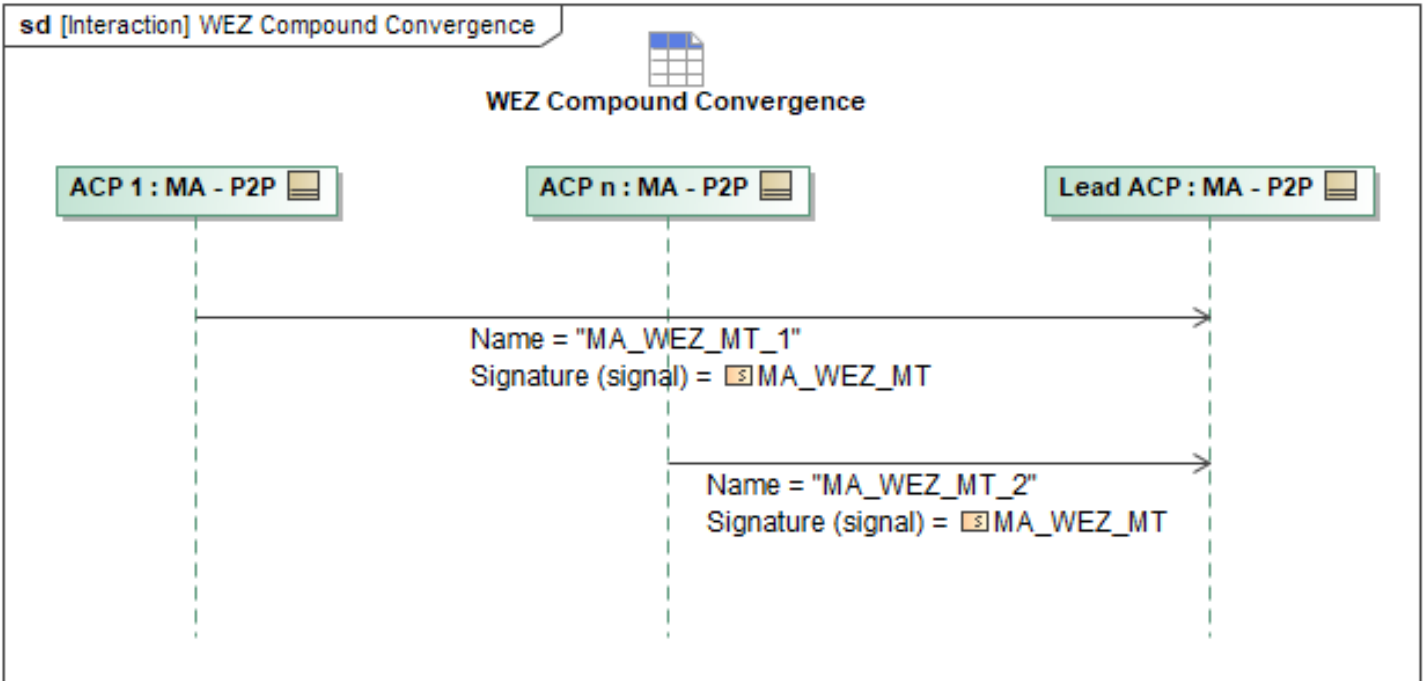


Figure 9-142: WEZ Compound Convergence

Table 9-142: WEZ Compound Convergence

Message Name (Name:Type)	Required Fields (Name:Type)
MA_WEZ_MT_1: MA_WEZ_MT	None
MA_WEZ_MT_2: MA_WEZ_MT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

9.3 Interface Compliance

The Interface Compliance table contains the L1 compliance requirements for the Peer interface.

Table 9-143: Peer Interface Compliance Requirements

ID	Requirement
MA-L1-003	A Level 1 Peer interface shall support the Peer Interface Minimum Message Set defined in the Peer MMS table in the Peer feature profile (A-GRA location: 1 Problem Domain > 3 Exchange Items > Minimum Message Set (MMS)) for all messages in Core Mission Use Case sequences and all non-Core Mission Use Case sequences required for platform specific implementations.
MA-L1-004	A Level 1 Peer interface shall implement all required messaging sequences in the Peer feature profile (A-GRA location: 1 Problem Domain > 2 White Box > 3 Functional Analysis > L1 Functional Analysis > L1 Interface Behaviors > Peer Interface Behavior), marked Core and all non-Core Mission Use Case sequences that apply to the platform specific implementation.
MA-L1-018	Upon receipt of a Malformed Message (a message missing expected fields (i.e., required by the behavior but not the UCI schema) or with invalid context-dependent information within the scope of that individual message) from the P2P interface, MA shall respond to the error by notifying P2P of the malformed message and requesting the corrected message according to the Malformed Message Response (P2P) sequence.
MA-L1-024	A Level 1 Peer interface shall follow state transitions for CommandProcessingState and RequestProcessingState in accordance with OAC-SPC-001 Rev C, Section 5.1.1.1 and 5.1.2.1. Representations of non-terminal statuses are optional and should align with the definitions and usage prescribed by the UCI Schema Style and Design Specification, ensuring consistent message state handling.

10 VEHICLE INTERFACE

The VI defines how MA commands vehicle motion to the platform. From MA's perspective, the VI is a system that includes guidance, navigation, high- and low-level flight control algorithms, and any flight safety-related functionality, some of which may be beyond the scope of a traditional VMS. MA focuses purely on mission execution by providing commands to the VI, but MA does not handle safety-critical functions; the VI must accept or reject commands as necessary to ensure safe operation.

The interface between MA and VI should act as an outgrowth of the VI. While MA is not responsible for safety-critical tasks, it is responsible for execution of mission-critical tasks, and the interface between MA and the VI is the primary gateway for control commands, vehicle state data, and other important parameters.

10.1 Interface Description

The Vehicle Interface (VI) defines the interface and architectural components required for interactions between Mission Autonomy (MA) and the vehicle, including any vendor-specific VI with vehicle control, command validation, and other capabilities. Focus is given to defining the boundary between MA and the VI and establishing the messaging patterns between the two.

The VI defines a set of messages and interactions for UCI communication between the MA and the VI OMS Isolator. This isolator is the primary conduit for data: all UCI communications from the OMS Mission Package to the safety-critical enclave are routed through it, and the vehicle will also pass data to MA without a specific request via the same component. The VI OMS Isolator is responsible for publishing and subscribing to the topics and messages detailed in the VI ICD section.

The separation between non-safety-critical systems (like the OMS Mission Package) and the safety-critical enclave is called the "Airworthiness Boundary". Systems assumed to be within this enclave could include the Vehicle Management System (VMS), Flight Autonomy (FA), or any other onboard system deemed safety-critical. MA makes several assumptions about functionality within this architecture; however, the VI ICD outlines interactions that require functionality resident within the VI OMS Isolator, the safety-critical enclave, or both. The definition of the external-facing boundary of the VI OMS Isolator is a platform-specific consideration potentially subject to certification. A nominal OMS Mission Package configuration containing MA and the VI OMS Isolator is depicted in the figure below.

It is assumed that the VI OMS Isolator will contain functions like a message validator and a platform adapter to interface with non-UCI compliant systems. However, the message validation strategy could be federated between the VI OMS Isolator and Flight Autonomy (FA). While this does not preclude MA from doing its own message validation, this validation is not considered safety-critical. A nominal breakdown of the validation functions is shown in the figure below.

The MA-FA interface is contained within the MA-VI interface, where specific safety-critical functionality within FA is considered read-only for MA. For example, FA is responsible for adhering to geofences while MA is responsible for adhering to dynamically managed geozones. The vehicle's FA handles a set of safety-critical, vehicle-specific, and last-second contingencies that can be reported to MA. Flight Autonomy notifies MA of contingencies and remedies taken that affect mission outcomes, such as fuel range or endurance, reduced speed/range, sensor failures, etc. Additional functionality, such as JPALS, is considered safety-critical, is located within FA, and does not interact directly with MA.

To ensure A-GRA compliance, vehicle manufacturers, VI providers, and MA software implementers should refer to this section to ensure that their systems support the required messages and interactions.

10.2 Interface Interactions

The VI interactions with MA have been scoped as shown in the below figure. Within interaction, there is a baseline set of functionalities that must be supported by VI and MA for a compliant system.

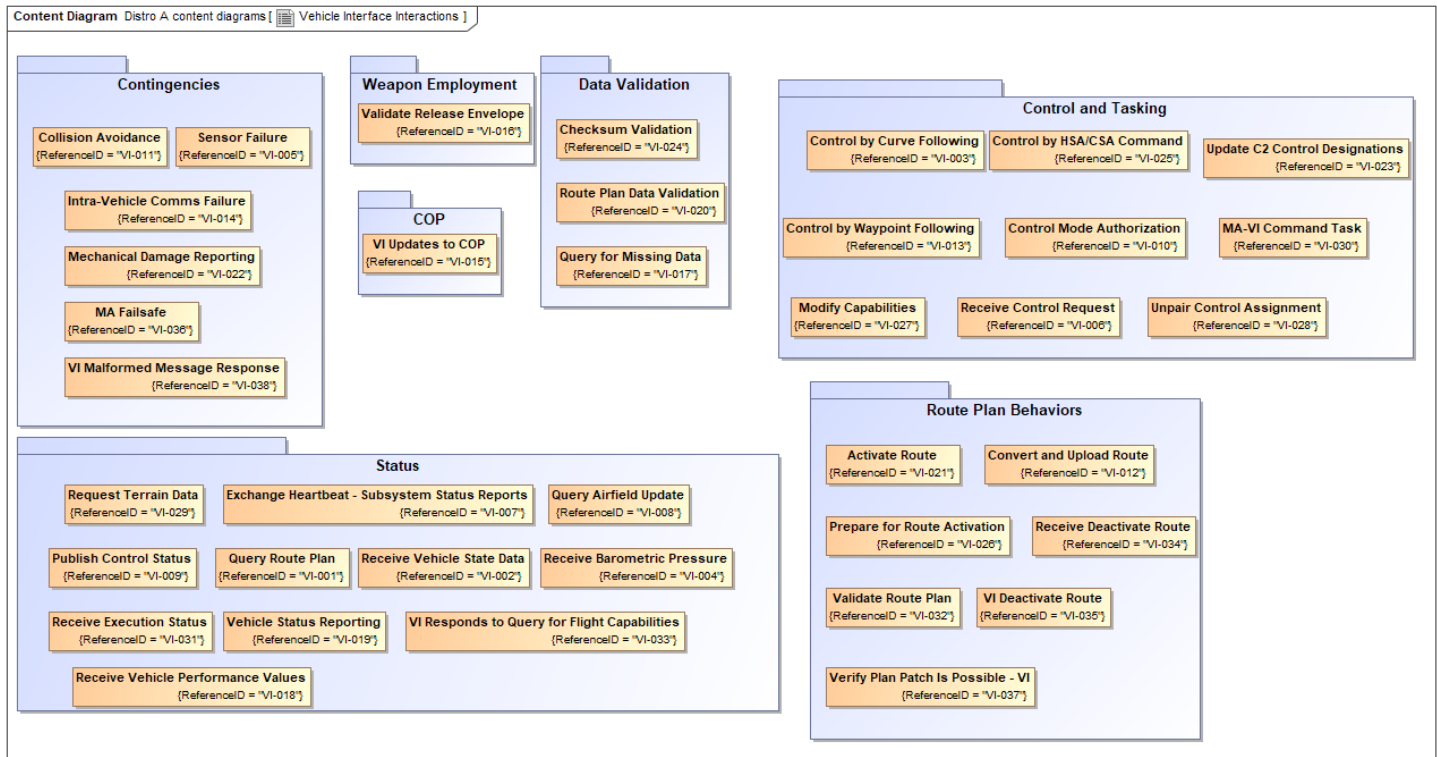


Figure 10-1: Vehicle Interface Interactions

10.2.1 Contingencies Package

10.2.1.1 Collision Avoidance

If VI is required to take safety critical action, VI will perform the necessary actions and inform MA that new actions are not being accepted due to evasive maneuvers being performed through the *MA_FlightCapabilityStatusMT* message in the *CapabilityStatus.AvailabilityInfo.Availability* field. VI will also provide the reason for denying new commands in the *MA_FlightCapabilityStatusMT.AvailabilityInfo.AvailabilityReason.Reason* field filled with *CONSTRAINT_COLLISION_AVOIDANCE*. After evasive maneuvers are performed, VI will inform MA of actions performed and resume accepting necessary tasking/messages from MA through the *MA_FlightCapabilityMT*.

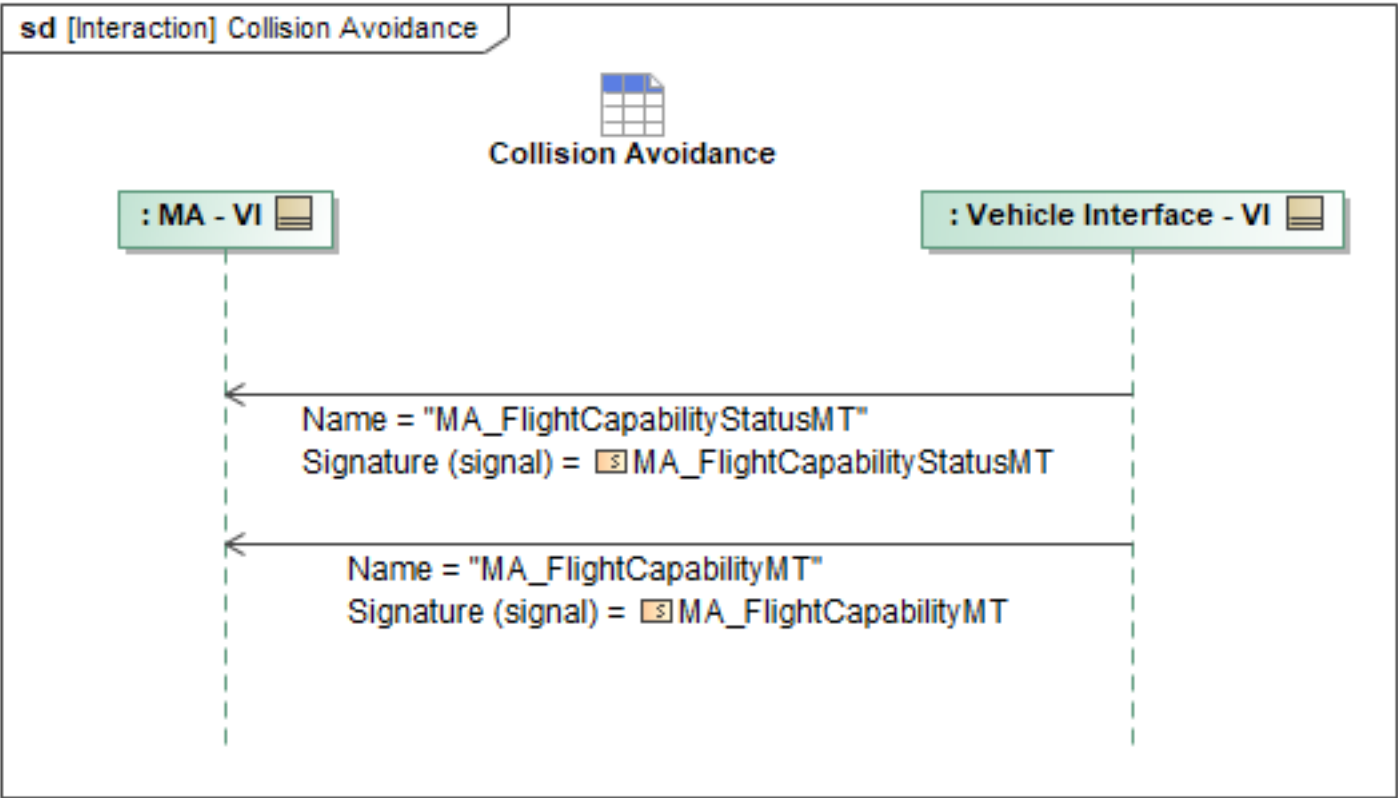


Figure 10-2: Collision Avoidance

Table 10-1: Collision Avoidance

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	AvailabilityReason: MA_CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.1.2 Intra-Vehicle Comms Failure

Vehicle Interface (VI) will provide regular status to Mission Autonomy (MA) via the *SubsystemStatusMT_1* UCI message. If MA doesn't receive a status from VI within the designated time frame, MA will request status from VI via the *SubsystemStatusDataRequestMT* UCI message. VI will continue to attempt to report status via the *SubsystemStatusMT_2* UCI message. If communication between MA and VI is not repaired within a specified time window, MA will refer to and follow the on-board contingency plan associated with a loss of communication with VI.

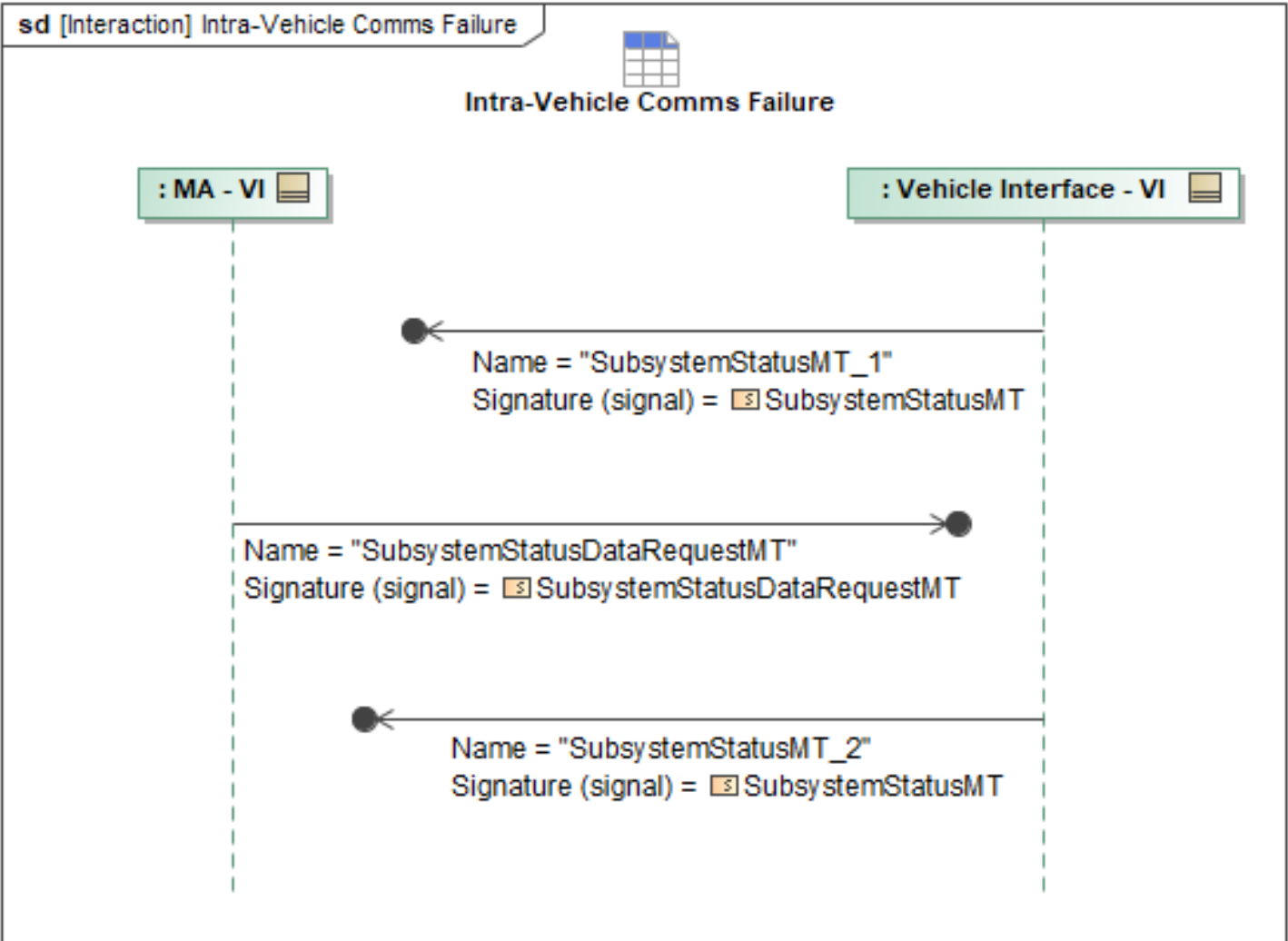


Figure 10-3: Intra-Vehicle Comms Failure

Table 10-2: Intra-Vehicle Comms Failure

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusDataRequestMT: SubsystemStatusDataRequestMT	None
SubsystemStatusMT_1: SubsystemStatusMT	None
SubsystemStatusMT_2: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.1.3 MA Failsafe

This behavior enables MA to pick a "failsafe plan", a plan for how FA should attempt to recover the ACP in case of MA failure.

MA will send an *MA_ResponseMT* to FA (VI) to describe the failsafe plan. If FA already has an MA failsafe plan in place, this *MA_ResponseMT* can be used to update the plan by using the same

ResponseID as the current plan and by incrementing the *ResponseID.Version*. The *Option.Trigger* set to *System* and the *Option.Trigger.System.SystemID* populated with the UUID for Mission Autonomy will be used to convey that this response is the failsafe plan. *ResponseType* will be set to *PLAN_ACTIVATION*, and as such the *Option.Response* will be set to *ActivatePlan*, with *Option.Response.ActivatePlan.ActivationDetails* set to *BySubPlan* and populated with *RoutePlans*. The *RoutePlans* must at minimum include one of the recovery route plans planned and stored on FA pre-mission as the terminal point of the failsafe plan. Additional transit route plans can also be included in the *RoutePlans* to get the ACP from its current position to the beginning of the pre-planned recovery route plan if MA has created said transit route plans as a *MA_RoutePlanMT* and sent them to FA prior to sending this *MA_ResponseMT*.

To acknowledge receipt of the *MA_ResponseMT* FA (VI) will respond by sending MA the *MA_SystemNotificationMT* message, referencing the *SystemSubjectID*, *SystemPerspective*, and *AssociatedMessage* fields.

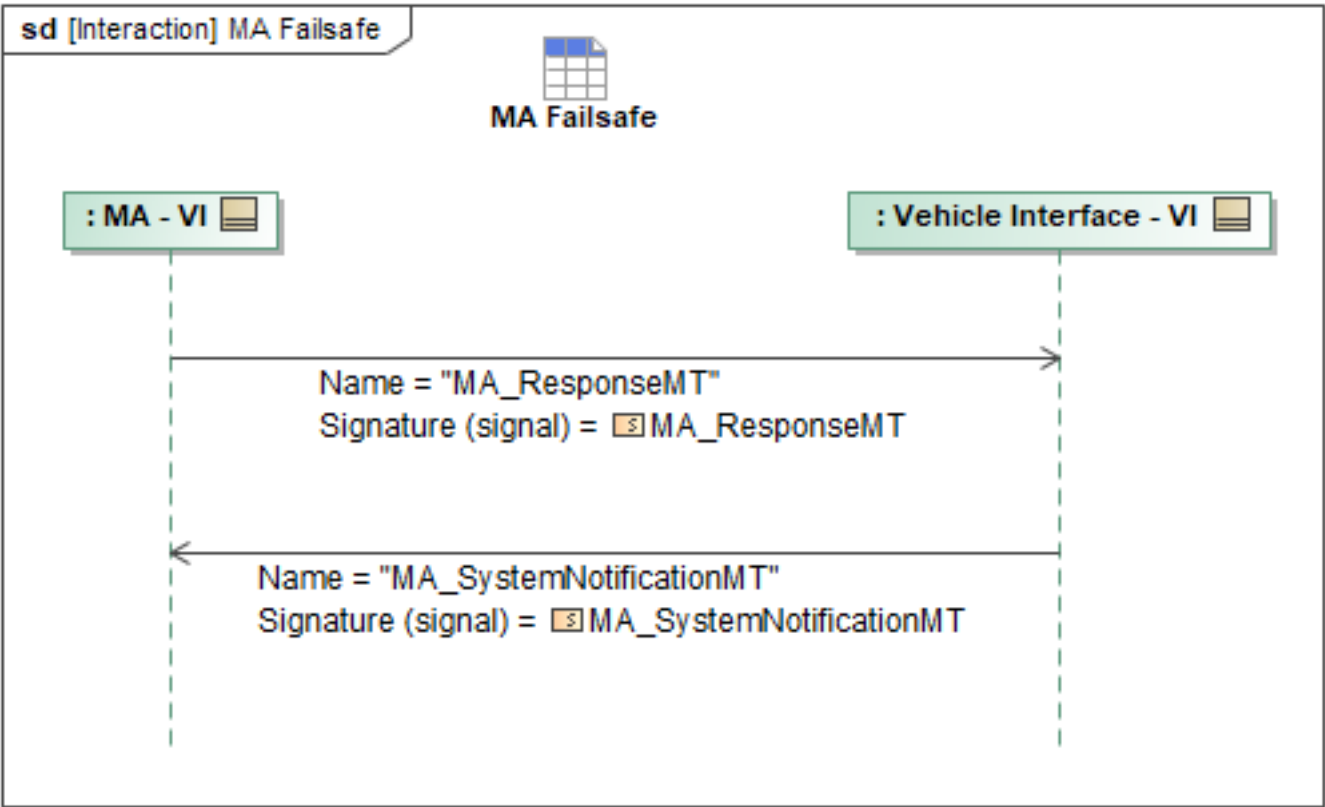


Figure 10-4: MA Failsafe

Table 10-3: MA Failsafe

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseMT: MA_ResponseMT	System: SystemFilterType SystemID: SystemID_Type ActivatePlan: MA_MissionPlanActivationCommandType BySubPlan: MA_MissionPlanSubplanActivationType RoutePlan: RoutePlanActivationType

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.1.4 Mechanical Damage Reporting

Vehicle Interface (VI) will perform periodic BITs. If any mechanical damage is identified, VI will report fault data to Mission Autonomy via the MA_FaultMT including data about what the fault was, affected services, severity, and what the subsystems capability is currently. Examples of mechanical damage could include actuator limitations, airframe damage detected by inputs required to fly, etc.

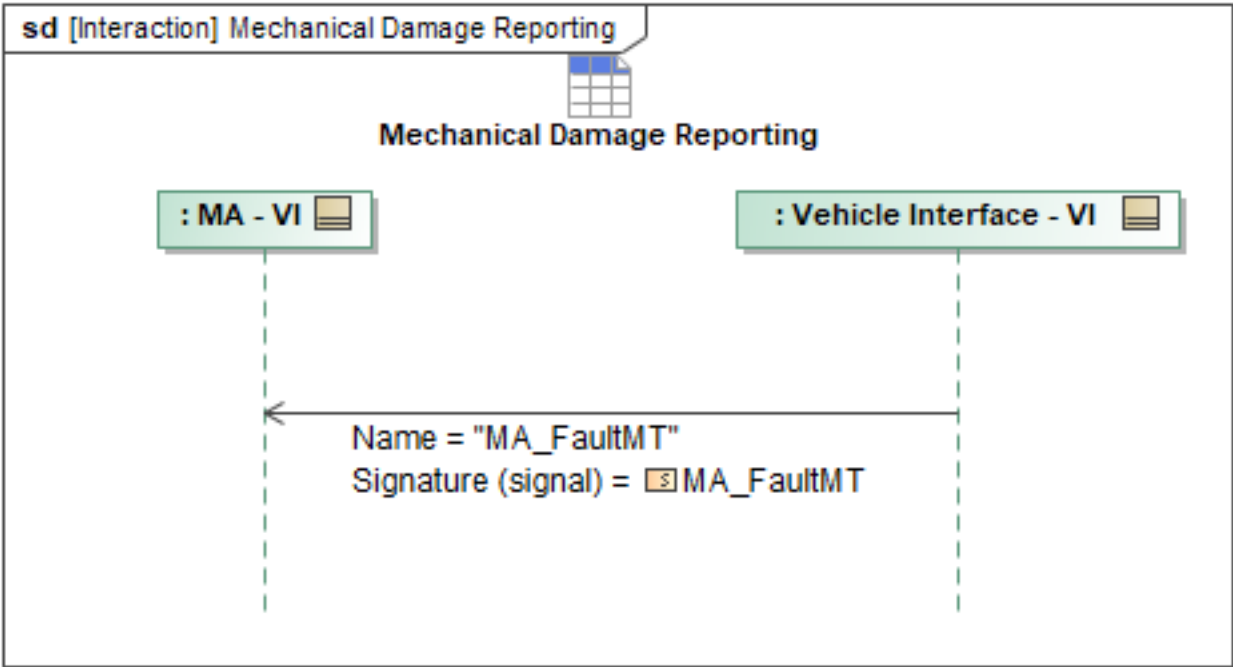


Figure 10-5: Mechanical Damage Reporting

Table 10-4: Mechanical Damage Reporting

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	FaultDescription: VisibleString256Type Severity: MA_FaultSeverityEnum FaultCodeCount: int FaultData: FaultDataType AmbiguityGroup: SubsystemFaultAmbiguityGroupType ServiceID: ServiceID_Type DetectionTime: DateTimeType FaultState: FaultStateEnum SubsystemID: SubsystemID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.1.5 Sensor Failure

Vehicle Interface (VI) will report subsystem status via the *SubsystemStatusMT* UCI message. If a fault is detected, VI will report the fault and necessary fault detail to Mission Autonomy (MA) via the *MA_FaultMT*.

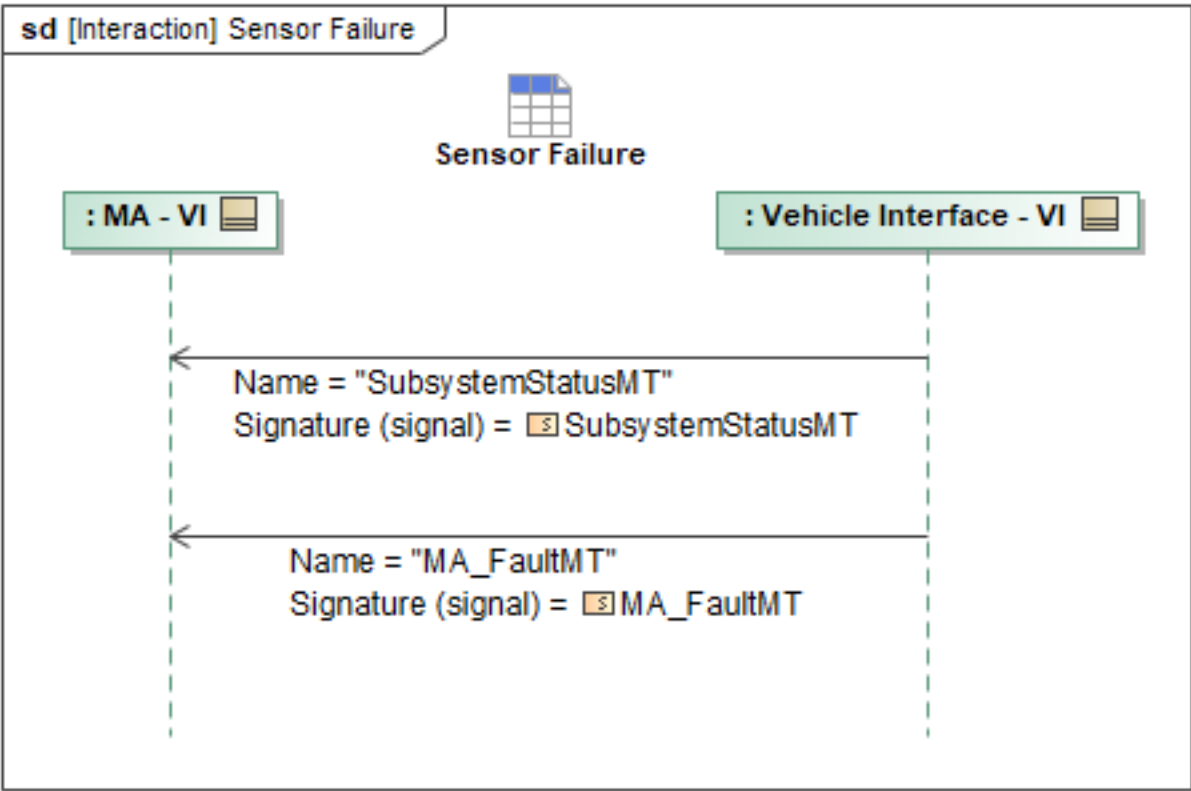


Figure 10-6: Sensor Failure

Table 10-5: Sensor Failure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	FaultCodeCount: int FaultState: FaultStateEnum AmbiguityGroup: SubsystemFaultAmbiguityGroupType Severity: MA_FaultSeverityEnum ServiceID: ServiceID_Type DetectionTime: DateTimeType FaultData: FaultDataType SubsystemID: SubsystemID_Type FaultDescription: VisibleString256Type
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.1.6 VI Malformed Message Response

Upon receipt of a malformed message, MA will send *MA_SystemNotificationMT* to VI, indicating that the message could not be successfully processed. Within the message, MA sets *NotificationState* to ACTIVE and populates *NotificationDetail.Reason* with *INVALID_INPUT_PARAMETER*. *NotificationDetail.Description* will contain a human-readable string stating which fields are malformed. *AssociatedMessage* will reference the malformed message. The *MessageType* will be set to the same type as the malformed message. *AssociatedID* will be populated with a reference ID from the malformed message if applicable, the *UUID* of that message. If additional data is required to resolve the condition and the fault remains active, MA will issue *QueryDataRequestMT* with the *Query* field containing the ID of the specific message. VI will respond with *QueryDataRequestStatusMT*, specifically the *Message* field within the *Result* field will contain the corrected message containing the originally missing data. If the fault has been addressed, then MA will only send a new *MA_SystemNotificationMT* with the same fields as above and *NotificationState* set to CLOSED.

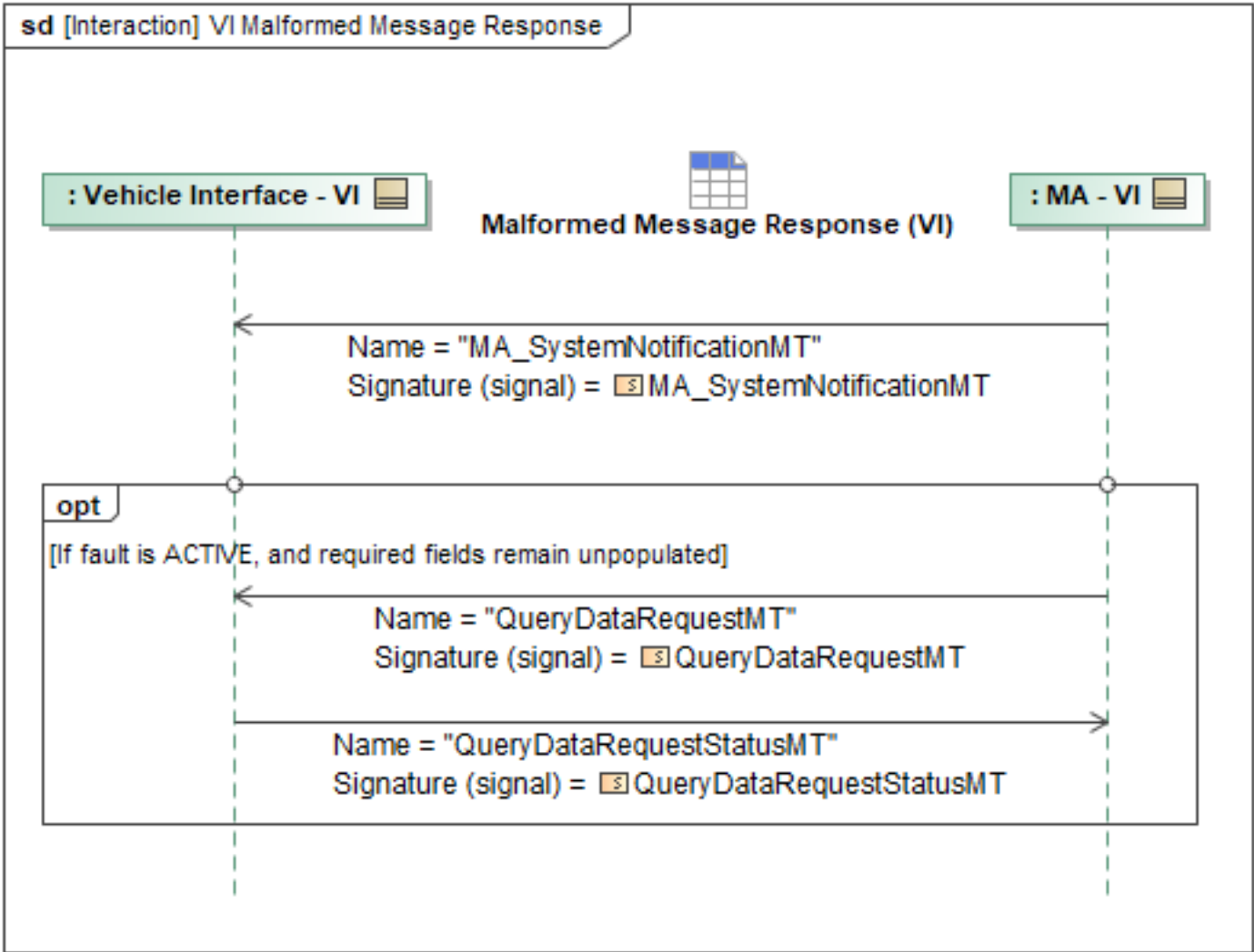


Figure 10-7: VI Malformed Message Response

Table 10-6: VI Malformed Message Response

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	Description: VisibleString1024Type AssociatedID: ID_Type
QueryDataRequestMT: QueryDataRequestMT	Query: QueryPET
QueryDataRequestStatusMT: QueryDataRequestStatusMT	Message: MessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2 Control and Tasking Package

10.2.2.1 Control by Curve Following

Curve Following control mode enables MA to command more dynamic maneuvers than are possible

through either *Waypoint Following* or *HSA/CSA* control. The command paths are specified as curves relative to the *ownship's* local NED coordinates. Each commanded path is composed of 5th-order *Bézier curves*, described by six control points per segment. For *Bézier curves*, the curve passes through the first and last control points, but the curve does not generally pass through the four interior control points. The first control point of a segment matches the last control point of the previous segment to produce a piecewise continuous curve (i.e. C0).

The *MA_FlightCommandMT* message contains the data required to execute the *Curve Following* control mode via the *MA_CurveControlType* field. For a given curve following command, *MA_CurveControlType* may specify a maximum of ten curve segments. Each curve segment defines the six control points using the *MA_NURBS_PointType*, which defines the *CenterReference*, the list of six *ControlPoints*, and a list of *KnotVector* values.

- *CenterReference* indicates the reference from which the control points are defined, representing the *ownship*.
- *ControlPoints* define six control points relative to *CenterReference*, with a *Weight* field set to 1.0 to represent *Bézier curves*.
- *KnotVector* is defined as [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1] to represent a 5th degree *Bézier curve* using six control points. This clamped knot vector ensures the curve begins at the first control point and ends at the last, with no internal knots, resulting in a single-span *Bézier curve*.

MA_CurveControlType may optionally include the *AppendCurve* field to specify the commanded curve is to be appended to the currently active curve following command. This optional field is therefore only valid when the vehicle is currently executing a curve following command. The appended curve following command will use the *ownship CenterReference* of the preceding curve following command. If the *AppendCurve* field is not specified, the vehicle will issue a new curve following command, terminating any active commands. *MA_CurveControlType* may also optionally specify *EndOfCurveBehavior* to specify the vehicle's behavior at the completion of the curve following command. These behaviors include:

- Continue at the previous Course-Speed-Altitude (CSA) state.
- Begin a circular loiter pattern centered around the curve's endpoint.

In addition, *MA_CurveTraversingType* allows for specification of how long an ACP has to traverse the curve or what the minimum and maximum speed it should use to traverse the curve.

If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* can optionally provide the *CannotComplyDetails* field to include further details on why the command was rejected. The *ValidationResult* field provides the reason or multiple reasons for rejection: *INVALID_CURVE*, *INVALID_WAYPOINT*, *PERFORMANCE_LIMIT_EXCEEDED*, *VIOLATION_ENDURANCE*, *VIOLATION_GEOFENCE*, *VIOLATION_AIR_TRAFFIC*, *VIOLATION_TERRAIN*, and *CAPABILITY_NOT_SUPPORTED*. For each rejection reason enumeration, additional context can be optionally provided by FA with the *MA_ValidationResultReasonType*. Each rejection reason enumeration has an analogous field that provides more information on the rejection. The *MA_ValidationResultReasonType* fields include details such as the specific expected invalid curve section of a commanded curve, the expected invalid segment of a commanded path, the performance constraints that are expected to be violated by a flight command, the expected insufficient endurance details including the remaining and required endurance, the specific geofence IDs, entity IDs, or terrain points that FA expects the flight command

to violate, as well as a plain-text description for any additional context required. Specifically for curve following rejections, the *MA_CurveSectionType* details the *StartCurveParameterValue*, *EndCurveParameterValue* and the *SegmentNumber* to communicate the specific section of the curve expected to be invalid.

VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command. In the case of a *FAILED ActivityState*, if a best effort *MA_TaskMT* is provided, VI may also optionally reference the UUID of the suggested task in the *ActivityReason.AssociatedID* field of the *FAILED MA_FlightActivityMT*.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* message, which includes data on the commanded state of the *ownership*, the execution state of the command described initially in *MA_CurveControlType*, and the status of the curve following, including segment number and percentage complete. The command cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until the sequence is terminated. The sequence terminates if a command is *REJECTED*, if the activity state is *COMPLETED* or *FAILED*, if control is taken away from MA, or if MA relinquishes control to the VI.

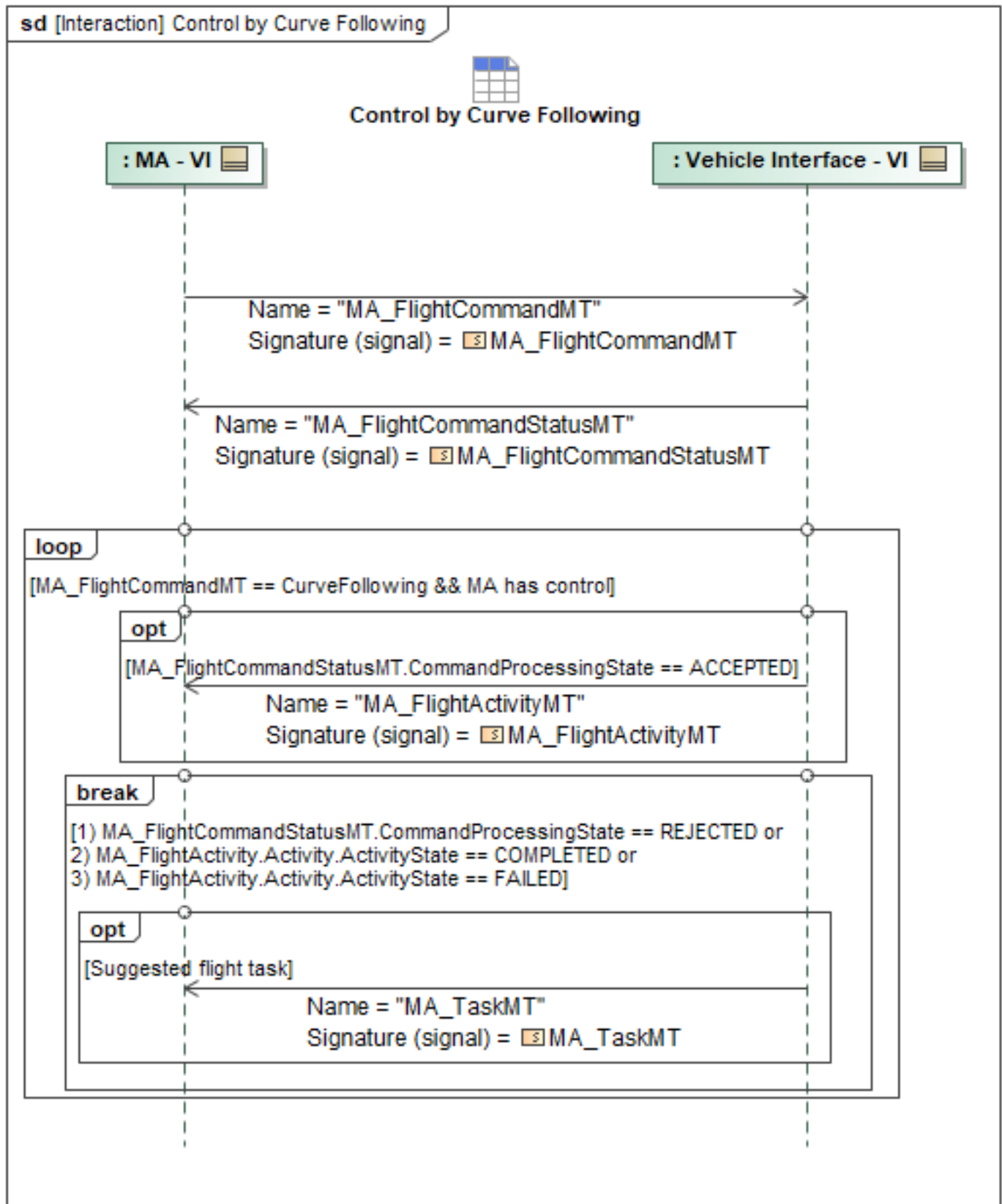


Figure 10-8: Control by Curve Following

Table 10-7: Control by Curve Following

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	CurveFollowing: MA_CurveControlType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Flight: MA_FlightTaskType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.2 Control by HSA/CSA Command

Heading-Speed-Altitude/Course-Speed-Altitude (HSA/CSA) control mode commands the vehicle to achieve supplied heading or course, speed, and altitude values. Heading or course can be specified as the desired direction vector. The vehicle tracks the supplied command data subject to its flight envelope and environmental conditions. No rate is specified or required for this control mode. The *HSA/CSA* control mode is persistent and does not have a defined completion state; the vehicle will continue using the current *HSA/CSA* command until it is terminated or interrupted by another command. Vehicle motion (turn rate, acceleration, and altitude change behavior) to achieve a given *HSA* command is left unspecified by Mission Autonomy

Partial HSA/CSA commands are allowed to update the most recent commanded heading or course, speed, and altitude values. For instance, only an altitude can be specified if it is the only change desired. Furthermore, the speed can be specified in terms of Mach number or some other reference, or can be left up to the VI to be optimized according to desired supported enumerations

HSA/CSA Altitude is specified using the standard UCI *AltitudeReferenceType*, along with an altitude value.

HSA/CSA direction is specified using an extended UCI type to specify a *MA_DirectionReferenceType* (true and magnetic north), along with a desired heading or course value.

HSA Speed can be specified using a value, along with the standard UCI *SpeedReferenceEnum* to indicate if the value is based on the ground speed or true air speed, or alternatively *MachType* which indicates speed as a unitless Mach number. Alternatively, the commanded airspeed can be passed in via *SpeedOptimizationEnum*, which supports max cruise or max endurance.

The Vehicle will perform validity checking on the *MA_FlightCommandMT* and accept or reject the command using the *MA_FlightCommandStatusMT* message based on the results of validity checks. If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* can optionally provide the *CannotComplyDetails* field to include further details on why the command was rejected. The *ValidationResult* field provides the reason or multiple reasons for rejection: *INVALID_CURVE*, *INVALID_WAYPOINT*, *PERFORMANCE_LIMIT_EXCEEDED*, *VIOLATION_ENDURANCE*, *VIOLATION_GEOFENCE*, *VIOLATION_AIR_TRAFFIC*, *VIOLATION_TERRAIN*, and *CAPABILITY_NOT_SUPPORTED*. For each rejection reason enumeration, additional context can be optionally provided by FA with the *MA_ValidationResultReasonType*. Each rejection reason enumeration has an analogous field that

provides more information on the rejection. The *MA_ValidationResultReasonType* fields include details such as the specific expected invalid curve section of a commanded curve, the expected invalid segment of a commanded path, the performance constraints that are expected to be violated by a flight command, the expected insufficient endurance details including the remaining and required endurance, the specific geofence IDs, entity IDs, or terrain points that FA expects the flight command to violate, as well as a plain-text description for any additional context required.

VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command. In the case of a *FAILED ActivityState*, if a best effort *MA_TaskMT* is provided, VI may also optionally reference the UUID of the suggested task in the *ActivityReason.AssociatedID* field of the *FAILED MA_FlightActivityMT*.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until the sequence is terminated. The sequence terminates if a command is *REJECTED*, if the activity state is *COMPLETED* or *FAILED*, if control is taken away from MA, or if MA relinquishes control to the VI.

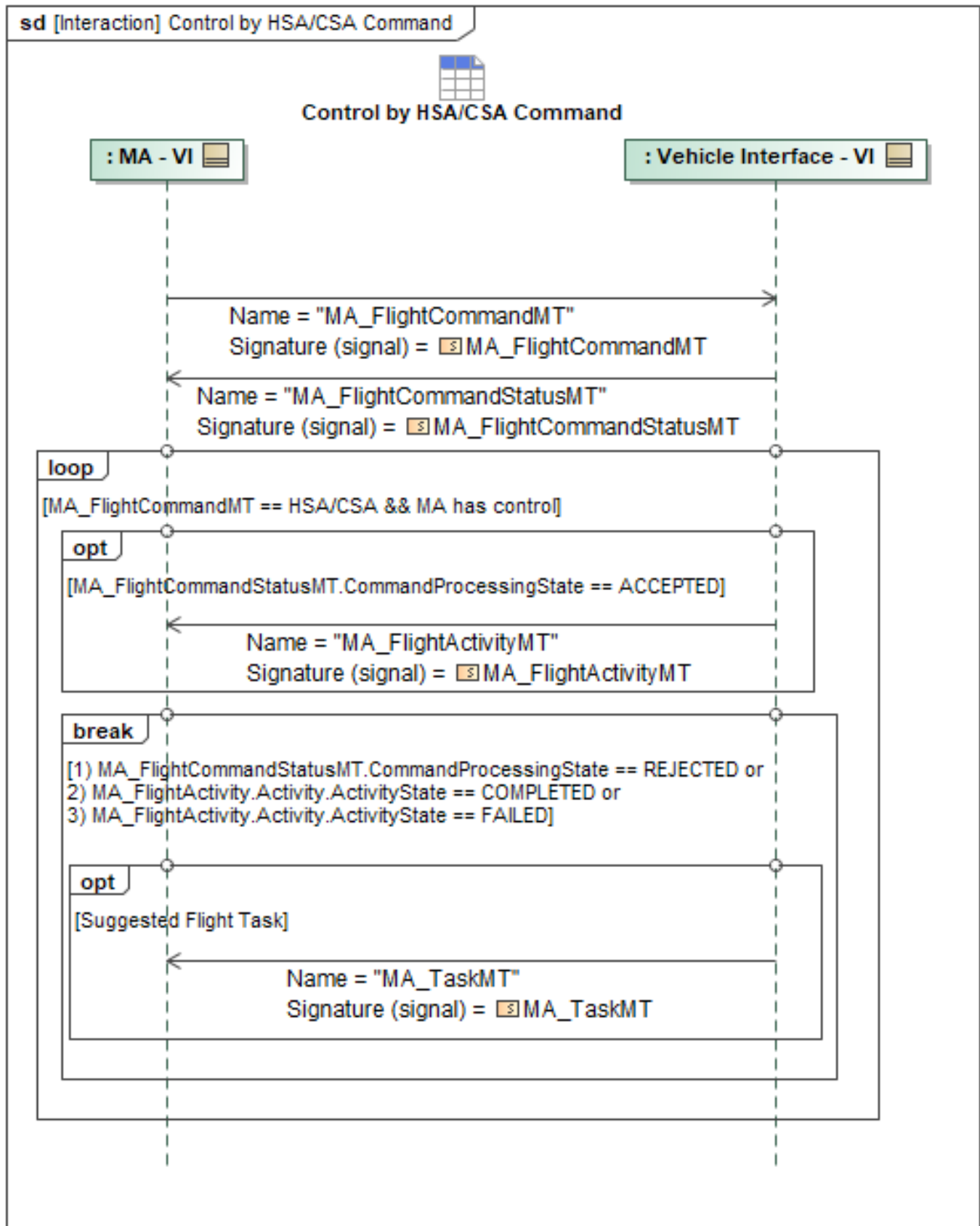


Figure 10-9: Control by HSA/CSA Command

Table 10-8: Control by HSA/CSA Command

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	HSA_CSA: MA_HSA_CSA_Type
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Flight: MA_FlightTaskType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.3 Control by Waypoint Following

To ensure safe operation, the Flight Autonomy (FA) has final authority over the vehicle. Before Mission Autonomy (MA) can issue commands, the FA must publish its set of available control modes and signal when it is ready to accept them.

This is a two-step process: First, the FA broadcasts the *FA_FlightCapabilityMT* message. This message describes the types of control available (e.g., waypoint following, curve following) and includes the specific performance parameters (i.e., *FlightCapabilityPerformanceProfile*) MA needs to use them. Second, the FA sends the *FA_FlightCapabilityStatusMT* message. This acts as a ready signal, formally telling MA when it can begin issuing commands.

Finally, once MA receives this ready signal, it can take control by sending commands to the FA.

Waypoint following, alternatively referred to as route following, is a control mode where MA provides the VI a series of waypoints describing a desired flight path. MA may define and send multiple routes to the VI, but a specific route is activated for execution via a command. The vehicle validates the route(s) based on flight envelope and environmental conditions and provides updates regarding route completion. MA may also terminate the route following activity by issuing a new command that supersedes it. Waypoint behavior may include simple commands such as fly-by or fly-through; complex commands including payload action and loiter/hold; or bounding constraints such as vehicle attitude or timing.

For taxi routes, the *PathType* will be designated as *TAXI*, and with *PathID* referencing a validated taxi path. The vehicle validates the taxi route(s) based on the vehicle performance conditions (e.g. taxi speed, turn rate, etc.) and provides updates regarding route completion. This allows MA to send commands to FA to taxi based upon conditions safely achievable by the vehicle along a route. It is understood that the associated taxi commands and constraints are only relevant when the aircraft is on the ground.

For loiter types, *Circle*, *Racetrack*, *Figure-eight*, or *ATC hold* may be selected, along with duration, direction, and other parameters to define the pattern. A *Circle* defines the radius, center point, and relative offset if applicable. For the *Figure-eight*, two circles define the loops with the appropriate relative offsets. For the *Racetrack*, two circle types define the semi-circle ends of the *Racetrack* connected by legs. For the *ATC hold*, two circles also define the semi-circle ends of the *Racetrack*, but with the appropriate relative offsets to accommodate the holding fix. To command a specific loiter pattern outside of *Circles*, *Racetrack*, *Figure-Eight*, or *ATC Hold*, use the *Point InboundDirection*,

LegLength, *TurnGeometry*, and *TurnDirection* fields within the *MA_FixOrbitType* portion of the *MA_FlightCommand* message. This allows MA to command specific holds based upon a relative point with defined leg lengths, turns, directions, and geometries to provide a more refined pattern when needed. If no inbound course is specified with the *ATC hold*, MA will use an inbound course equal to the arrival course. If no turn direction is specified, MA will use right turns by default. If no leg length is specified, the inbound leg length will be based on timing: 60 seconds, when at or below 14,000 ft MSL; or 90 seconds, when above.

Communication of new waypoints begins with the transmission from MA to the VI of *MA_FlightCommandMT* to begin receiving a *Waypoint Following* command. A well-conditioned *MA_WaypointFollowingType* message will simply include a *RouteType* message for the VI to execute. Detailed information on the contents of *RouteType* and *RoutePathType* are available within the OMS UCI 2.3 schema. The Vehicle will perform validity checking on the *MA_FlightCommandMT* and accept or reject the command using the *MA_FlightCommandStatusMT* message based on the results of validity checks.

If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* can optionally provide the *CannotComplyDetails* field to include further details on why the command was rejected. The *ValidationResult* field provides the reason or multiple reasons for rejection: *INVALID_CURVE*, *INVALID_WAYPOINT*, *PERFORMANCE_LIMIT_EXCEEDED*, *VIOLATION_ENDURANCE*, *VIOLATION_GEOFENCE*, *VIOLATION_AIR_TRAFFIC*, *VIOLATION_TERRAIN*, and *CAPABILITY_NOT_SUPPORTED*. For each rejection reason enumeration, additional context can be optionally provided by FA with the *MA_ValidationResultReasonType*. Each rejection reason enumeration has an analogous field that provides more information on the rejection. The *MA_ValidationResultReasonType* fields include details such as the specific expected invalid curve section of a commanded curve, the expected invalid segment of a commanded path, the performance constraints that are expected to be violated by a flight command, the expected insufficient endurance details including the remaining and required endurance, the specific geofence IDs, entity IDs, or terrain points that FA expects the flight command to violate, as well as a plain-text description for any additional context required. Specifically for waypoint following rejections, the *RouteValidationInvalidPathType* details the *PathID* and *PathSegmentID* to communicate the specific segment of the path expected to be invalid.

VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The command cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until the sequence is terminated. The sequence terminates if a command is *REJECTED*, if the activity state is *COMPLETED* or *FAILED*, if control is taken away from MA, or if MA relinquishes control to the VI.

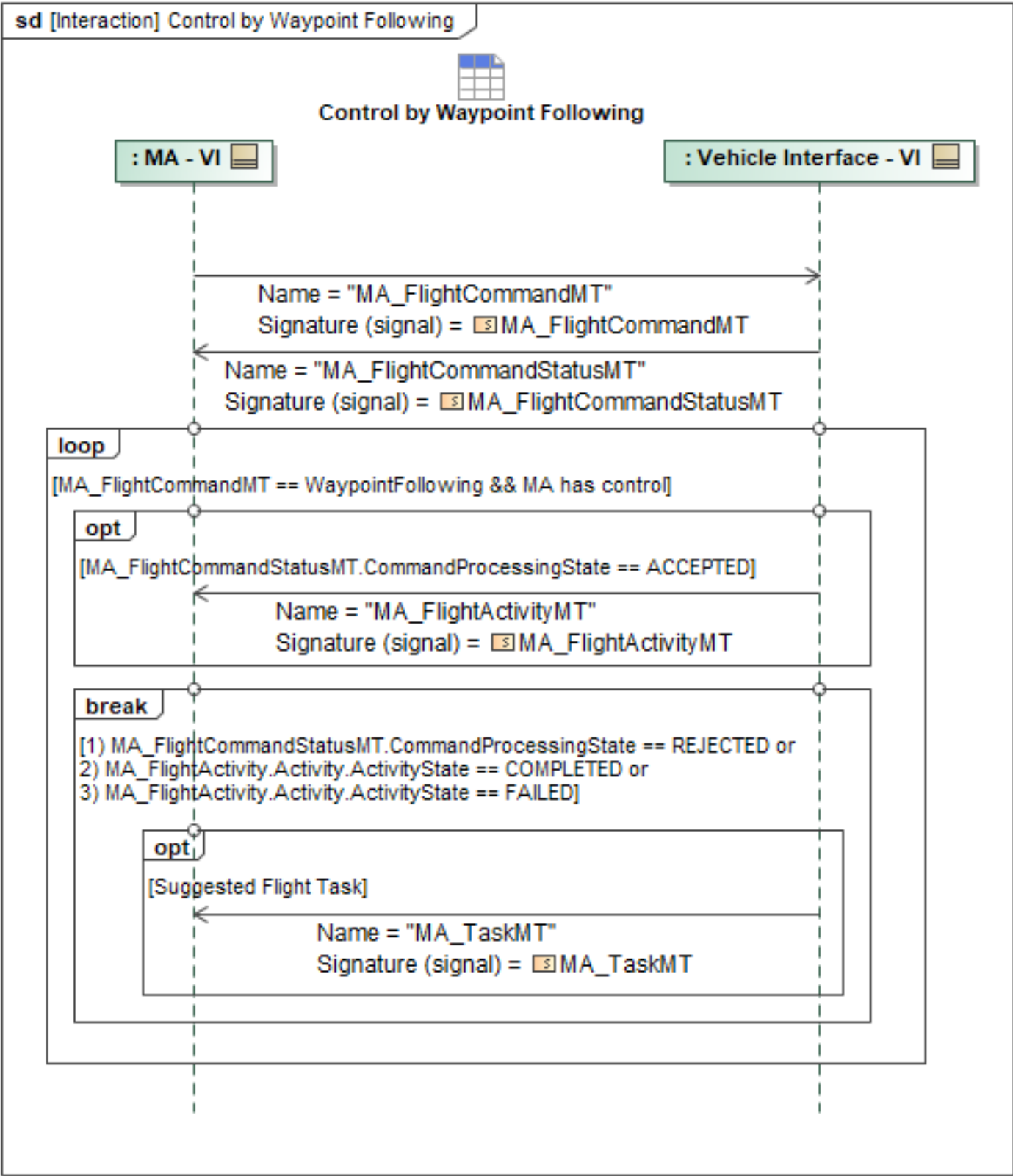


Figure 10-10: Control by Waypoint Following

Table 10-9: Control by Waypoint Following

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	WaypointFollowing: MA_WaypointFollowingType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Flight: MA_FlightTaskType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.4 Control Mode Authorization

The VI is the ultimate arbitrator of control authority over the ownship. Therefore, for MA to command the VI, the VI must first offer control once it is available. The *MA_FlightCapabilityMT* message published by the VI outlines which control modes are available to MA. The *MA_FlightCapabilityMT* message requires the *FlightCapabilityPerformanceProfile* field be populated with the appropriate subtypes of the capabilities advertised and any parameters needed by MA for commanding each type present. Additionally, *MA_FlightCapabilityStatusMT* signals availability of control from the VI to MA. Once ready, MA can issue commands to the VI via the Control by HSA/CSA Command use case, Control by Waypoint Following use case, or Control by Curve Following use case.

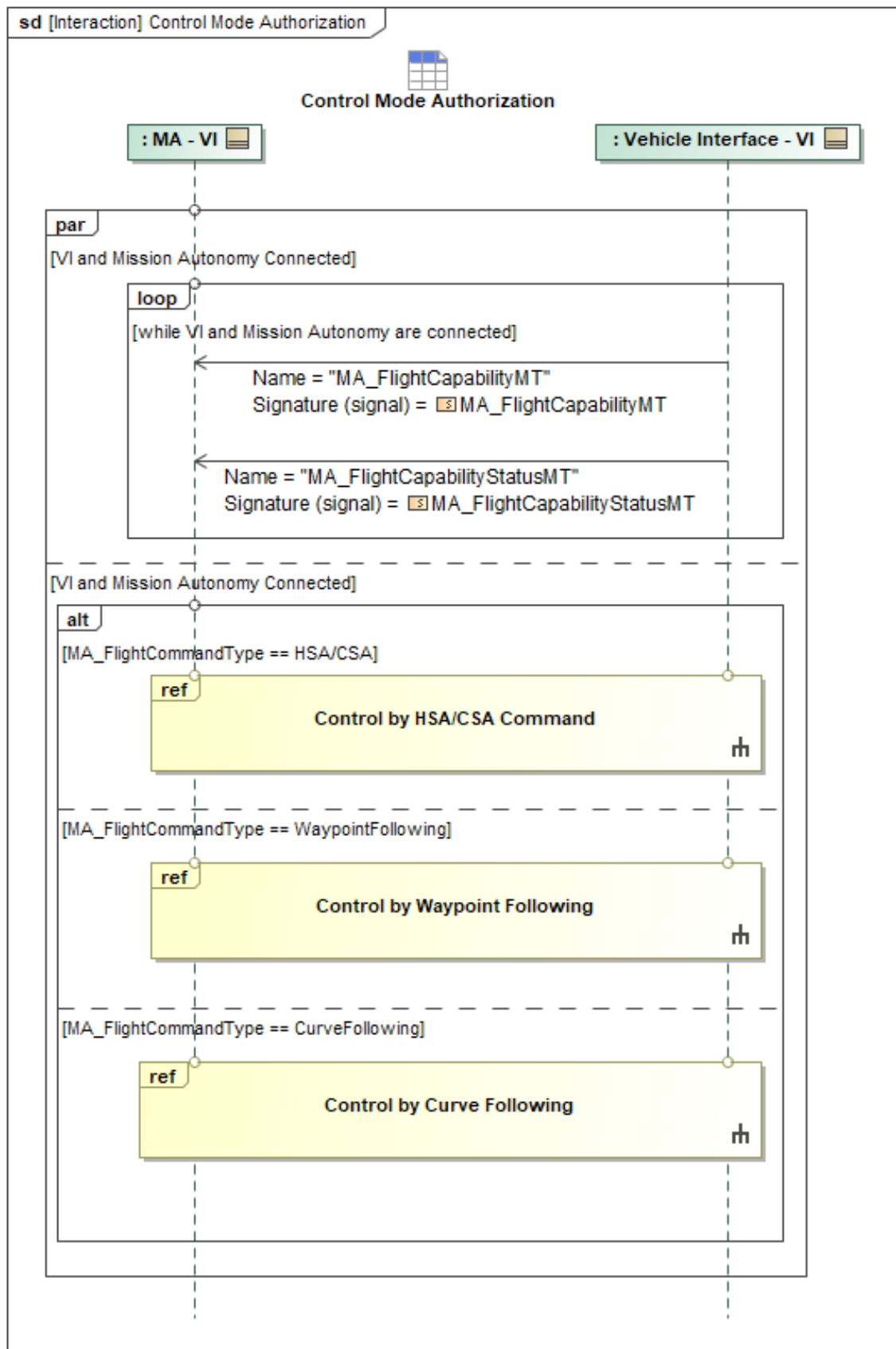


Figure 10-11: Control Mode Authorization

Table 10-10: Control Mode Authorization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.5 MA-VI Command Task

MA can command VI using the UCI *MA_Task* construct. When MA has the intent to command VI via *MA_Task*, MA first needs to make sure that VI has the *MA_Task* of *TaskID* that MA intends to command. In the event that MA has not already sent VI the *MA_TaskMT*, MA sends *MA_TaskMT* to VI, and VI responds with *MA_SystemNotificationMT*. Once VI has the *MA_Task*, MA sends an *MA_TaskCommandMT* to command VI to execute the *MA_Task*. VI responds with *MA_TaskCommandStatus* VI responds with *MA_TaskCommandStatus* messages and additionally, since VI will also be executing an *MA_Task*, VI will publish a *TaskStatus* message. If VI rejects the command, VI indicates this by sending MA an *MA_TaskCommandStatusMT* with *CommandProcessingState* = *REJECTED*.

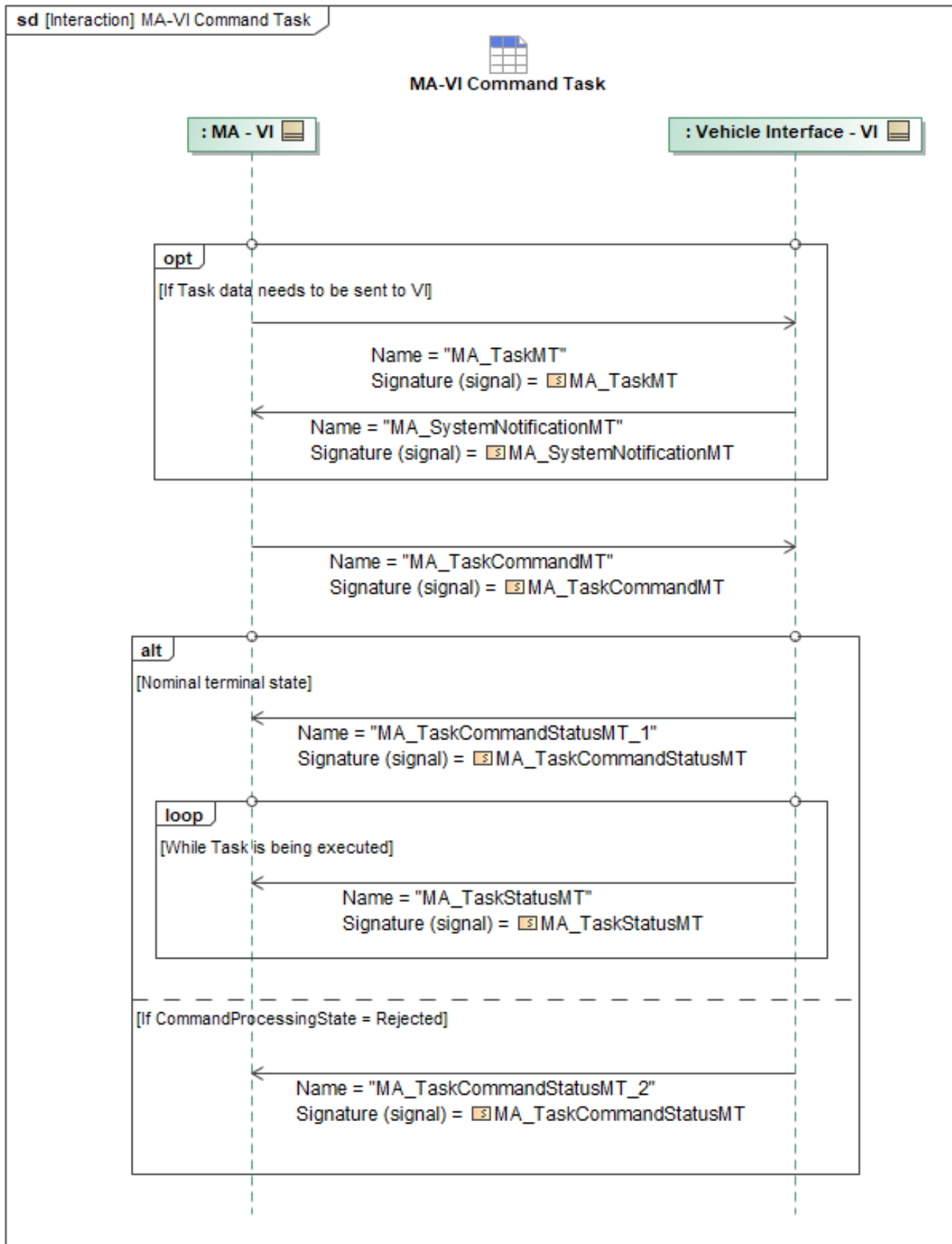


Figure 10-12: MA-VI Command Task

Table 10-11: MA-VI Command Task

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedID: ID_Type AssociatedMessage: AssociatedMessageType NotificationDetail: CannotComplyType
MA_TaskCommandMT: MA_TaskCommandMT	None
MA_TaskCommandStatusMT_1: MA_TaskCommandStatusMT	None
MA_TaskCommandStatusMT_2: MA_TaskCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_TaskMT: MA_TaskMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.6 Modify Capabilities

This sequence is intended for when FA has a condition that requires modification of MA capability control but does not constitute a complete removal of secondary control for MA. FA, through the VI, will send a command to reduce MA's ACP control functionalities. When FA must reduce MA control, the *MA_FlightCapabilityStatusMT* message, containing the *CapabilityStatus.AvailabilityInfo.Availability* field will be populated with "TEMPORARILY_UNAVAILABLE" to provide the status that capabilities are unavailable for MA at the given time. FA should also provide reasoning for reducing MA's control using the *MA_FlightCapabilityStatusMT* message containing the *CapabilityStatus.AvailabilityInfo.Availability.AvailabilityReason.Reason* field. The *MA_FlightCapabilityMT* will also be sent to capture any capability changes outside of *MA_FlightCapabilityStatusMT*. It should be noted, a different interaction will be used in the event of a full unpairing scenario, where FA must remove *SECONDARY_CONTROLLER* from MA. In the condition that Capabilities have a state change to available (i.e. "TEMPORARILY_UNAVAILABLE" to "AVAILABLE"), the reasoning for this capability change that would be conveyed within *CapabilityStatus.AvailabilityInfo.Availability.AvailabilityReason.Reason* is considered optional. It's expected that capabilities upon entry into a negative state (*UNAVAILABLE*, *TEMPORARILY_UNAVAILABLE*, etc.) will have listed a reason and thus coming out of the state should not require a repeat of that reason.

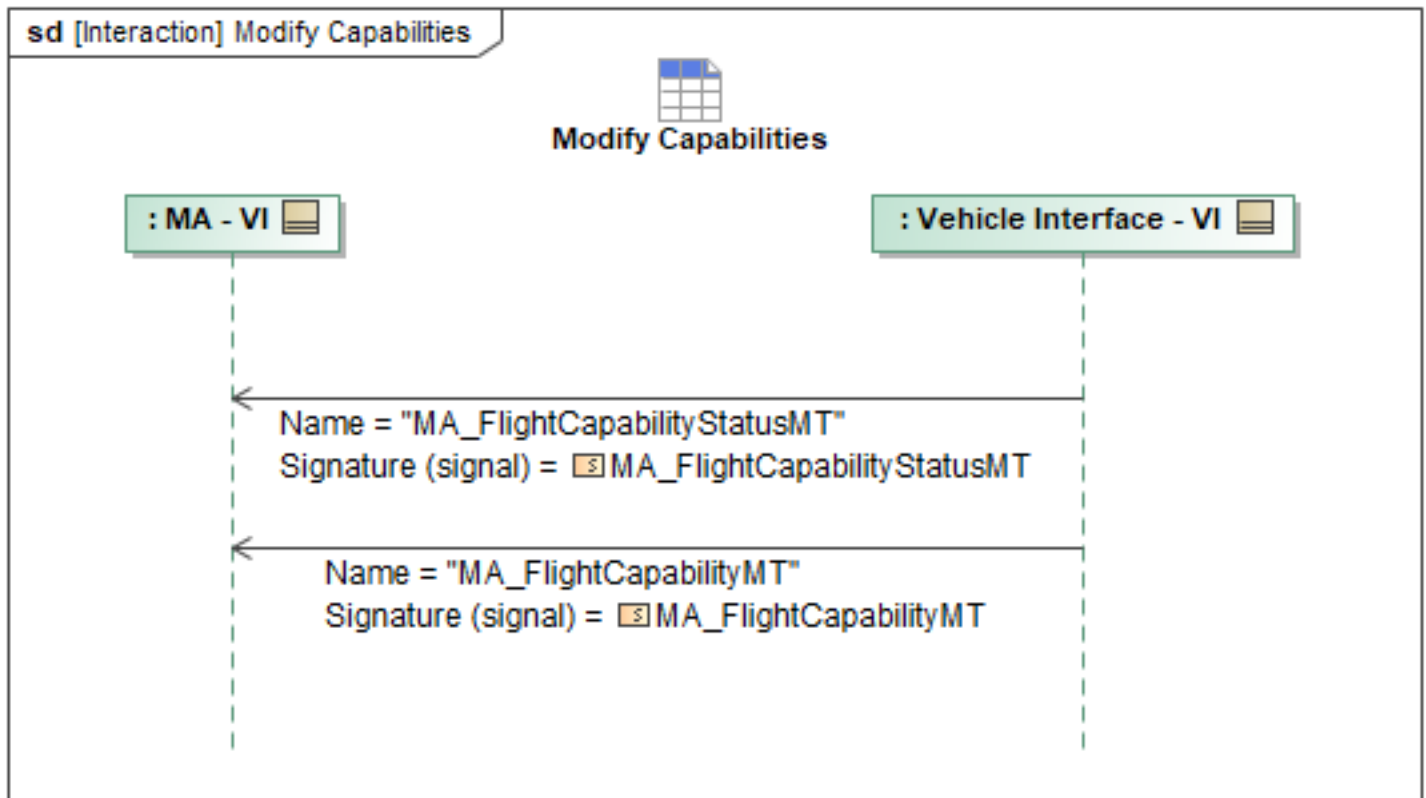


Figure 10-13: Modify Capabilities

Table 10-12: Modify Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.7 Receive Control Request

MA can attain capability control over one or more capabilities advertised by VI. Advertised capabilities can be discovered by subscribing to *MA_FlightCapabilityMT* messages in the exchange described by the "Control Mode Authorization" behavior and corresponding sequence diagram.

MA can initiate the request to attain capability control by sending *ControlRequestMT* to VI. MA will specify *ControlType* of "CAPABILITY_SECONDARY". MA will specify that the sibling Controller be set with the *SystemID* and *ServiceID* corresponding to the particular MA service that would be requesting to send subsequent messages to VI. MA will specify the *Controllee* with the *SystemID* corresponding to the VI and the *CapabilityID* matching the *desired* capability advertised by VI in a previous *MA_FlightCapabilityMT* message. The *RequestType* should be specified as "ACQUIRE".

VI can optionally provide a *ControlRequestStatusMT* to MA indicating the initial request was received and is being processed by specifying the *ApprovalRequestProcessingState* as "PENDING".

If VI accepts the request and is ready to start listening to the MA service that sent the original request, VI can reply with a *ControlRequestStatusMT* with the *ApprovalRequestProcessingState* set to "APPROVED". Before sending *MA_FlightCommandMT* or *MA_Task* messages, the requesting MA service should consume the next published *ControlStatusMT* published by VI to determine the granted *CapabilityIDs*. See the referenced "Publish Control Status" behavior for details.

If VI rejects the request and will subsequently not forward managed capability control to the requesting MA service, VI will reply with a *ControlRequestStatusMT* message with the *ApprovalRequestProcessingState* set to "REJECTED". Furthermore, the *ApprovalRequestProcessingStateReason* will be set to an enumeration value appropriate for the situation. While MA has secondary control the "Publish Control Status" message exchange, in particular the *ControlStatusMDT ControlType/CapabilityControl/SecondaryController/ServiceID* indicates which MA service has secondary flight capability control. If Flight Autonomy (FA) via the Vehicle Interface (VI) needs to revoke Mission Autonomy (MA) secondary control it can publish *ControlRequestStatusMT* (indicated by *ControlRequestStatusMT_4*) and *ControlStatusMT*. VI will inform secondary controllers, per capability, with their remaining capabilities, if any, with *ControlStatusMT*. The *ControlRequestStatusMT* message's *ApprovalRequestProcessingState* should indicate "CANCELED". The *ApprovalRequestProcessingStateReason* will be set to an enumeration value appropriate for the situation. This is a similar pattern to "Control by..." VI behaviors where VI can end the exchange when it determines MA secondary control must be revoked or modified. The *ControlStatusMT* message will indicate *ControlType/CapabilityControl/PrimaryController/SystemID* and *ControlType/CapabilityControl/PrimaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* corresponding to an appropriate OMS service on the VI. It is an aperiodic version of the same information sent in the *ControlStatusMT* message sent in "Publish Control Status". For each *CapabilityID*, VI will indicate *ControlType/CapabilityControl/SecondaryController/SystemID* and *ControlType/CapabilityControl/SecondaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* for any MA service that still retains control. If no MA services have secondary controller status for a given capability, none will be reported. The *AcceptedInterface* will be populated with the appropriate command interfaces for the given capability.

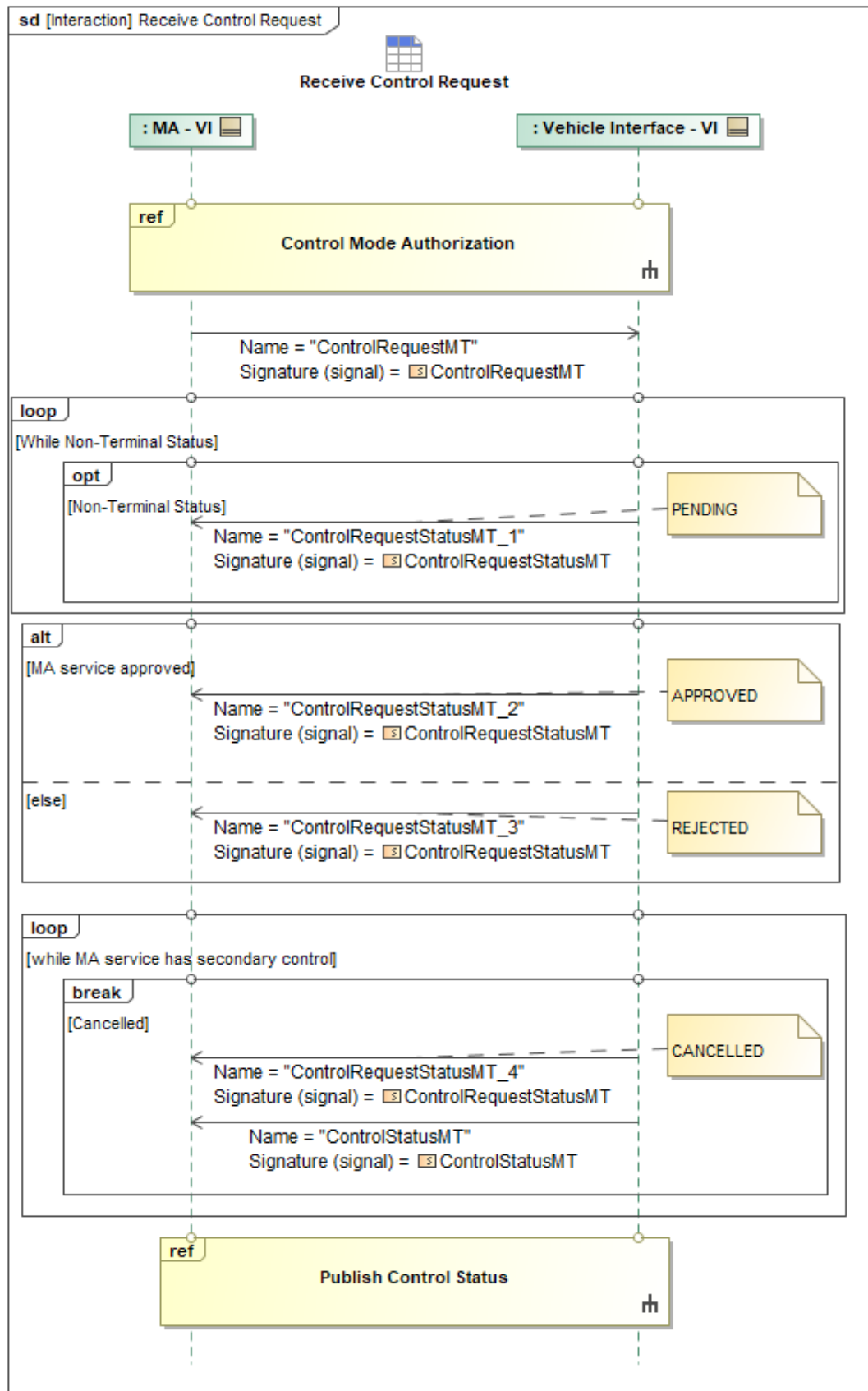


Figure 10-14: Receive Control Request

Table 10-13: Receive Control Request

Message Name (Name:Type)	Required Fields (Name:Type)
ControlRequestMT: ControlRequestMT	ServiceID: ServiceID_Type CapabilityID: CapabilityID_Type
ControlRequestStatusMT_1: ControlRequestStatusMT	None
ControlRequestStatusMT_2: ControlRequestStatusMT	None
ControlRequestStatusMT_3: ControlRequestStatusMT	ApprovalRequestProcessingStateReason: CannotComplyType
ControlRequestStatusMT_4: ControlRequestStatusMT	None
ControlStatusMT: ControlStatusMT	SecondaryController: SecondaryControllerType CapabilityControl: ControlStatusCapabilityControlType PrimaryController: PrimaryControllerType AcceptedInterface: CapabilityControlInterfacesEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.8 Unpair Control Assignment

If FA via the VI needs to revoke MA secondary control it can publish *ControlRequestStatusMT* and *ControlStatusMT*. Using *ControlStatusMT*, VI will inform secondary controllers of their remaining capabilities, if any. This message will be sent on a per capability basis for each secondary controller. The *ControlRequestStatusMT* message's *ApprovalRequestProcessingState* should indicate "CANCELED". The *ApprovalRequestProcessingStateReason* will be set to an enumeration value appropriate for the situation. The *ControlStatusMT* message will also indicate *ControlType/CapabilityControl/PrimaryController/SystemID* and *ControlType/CapabilityControl/PrimaryController/ServiceID* with a *SystemID* and *ServiceID* corresponding to an appropriate OMS service on the VI. For each *CapabilityID*, VI will indicate *ControlType/CapabilityControl/SecondaryController/SystemID* and *ControlType/CapabilityControl/SecondaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* for any MA service that still retains control. If no MA services have secondary controller status for a given capability, none will be reported. The *AcceptedInterface* will be populated with the appropriate command interfaces for the given capability.

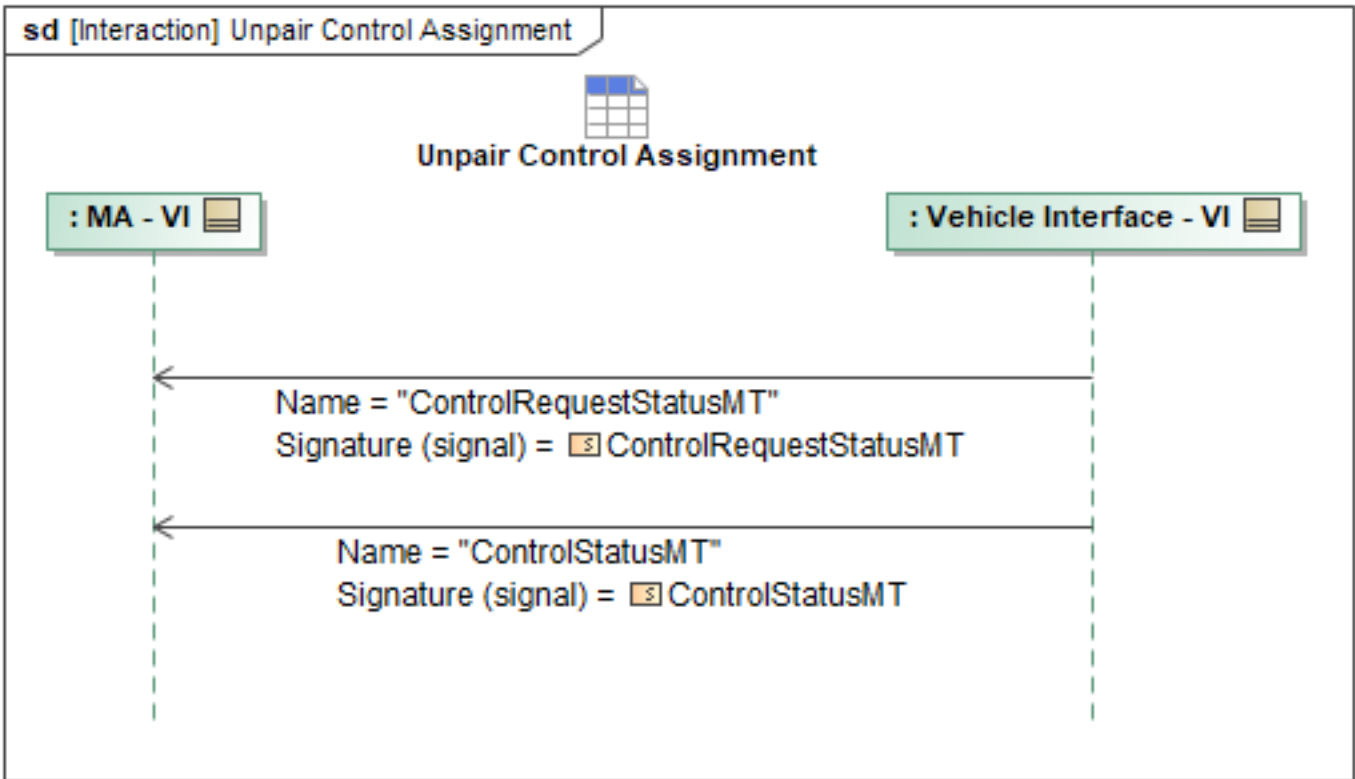


Figure 10-15: Unpair Control Assignment

Table 10-14: Unpair Control Assignment

Message Name (Name:Type)	Required Fields (Name:Type)
ControlRequestStatusMT: ControlRequestStatusMT	ApprovalRequestProcessingStateReason: CannotComplyType
ControlStatusMT: ControlStatusMT	SecondaryController: SecondaryControllerType CapabilityControl: ControlStatusCapabilityControlType PrimaryController: PrimaryControllerType AcceptedInterface: CapabilityControlInterfacesEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.2.9 Update C2 Control Designations

C2 may command the aircraft control paradigm including revoking or authorizing MA to command the flight of the aircraft. C2 will revoke or authorize MA to command specific flight capabilities of the aircraft as outlined by the "Update MA-FA Control Designations" sequence in the C2 interface. Upon receiving the *MA_FlightCapabilityMT* from C2, MA will forward it to FA to communicate the authorized flight capabilities. The *MA_FlightCapabilityMT* will have the *CapabilityType.MA_FlightCapabilityEnum* field populated with the allowable control modes that MA can command. This enumeration list will be redacted to revoke specific flight capabilities or amended to authorize additional flight capabilities as desired by C2.

MA will also forward the C2 provided *ControlRequestMT* with the *Controller* populated with the MA System ID and the *Controllee* populated with the FA system ID as MA will be controlling FA through flight commands. The *ControlRequestMT* will also include the *ControlType* for MA as *CAPABILITY_SECONDARY*, as the *CAPABILITY_PRIMARY* control type is reserved for FA as the safety critical system on the aircraft. Lastly, the *CapabilityID* will be set as the *CapabilityID* of the *MA_FlightCapabilityMT* that proceeded the *ControlRequestMT*.

FA will respond through the VI with the *ControlRequestStatusMT* to acknowledge the commanded control paradigm. Then FA will broadcast its update flight capabilities using the *MA_FlightCapabilityMT* that will have the *CapabilityType.MA_FlightCapabilityEnum* populated to align with the C2 commanded flight capabilities. FA will then send the *MA_FlightCapabilityStatusMT* to communicate the status of the broadcasted flight capabilities.

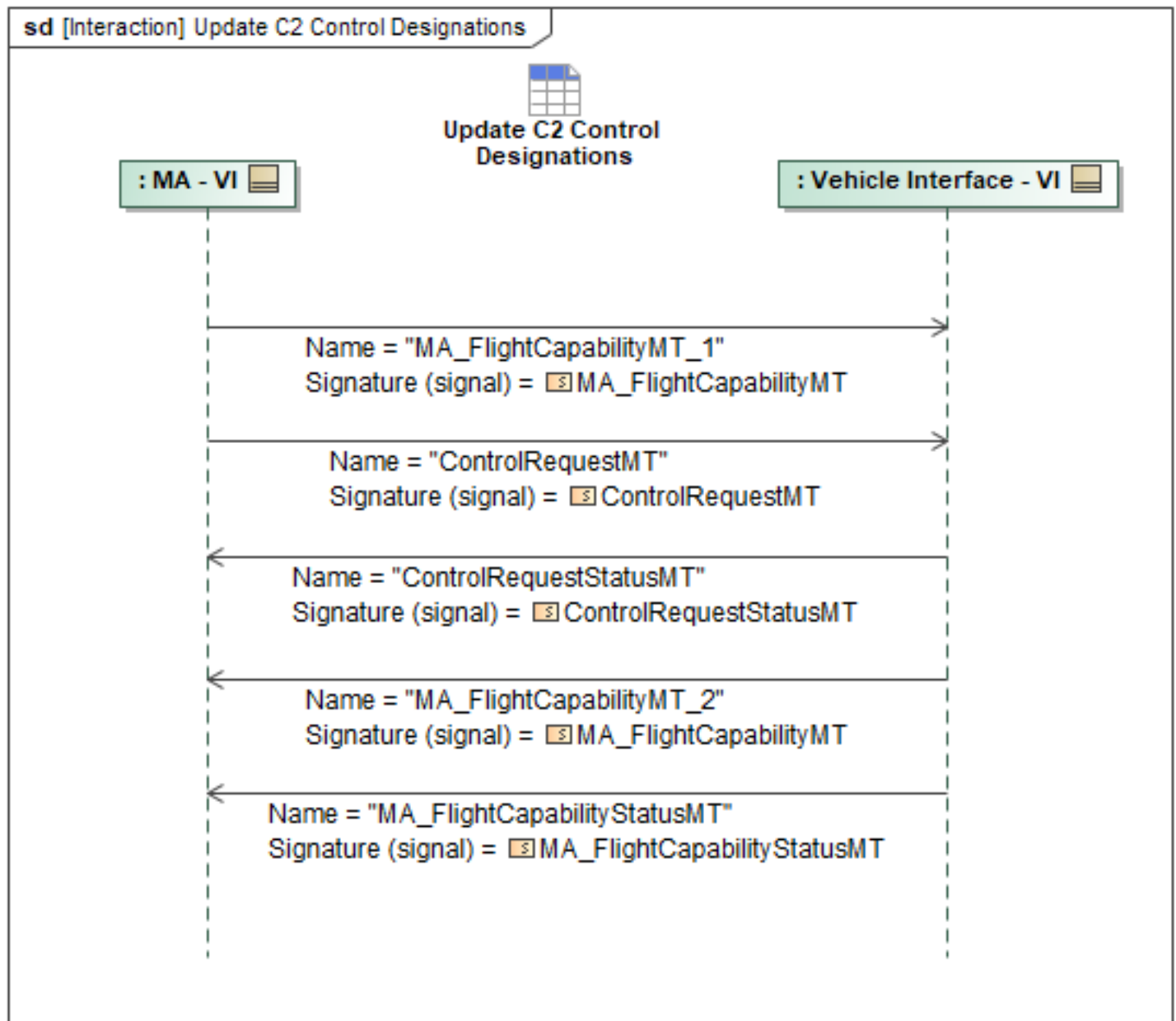


Figure 10-16: Update C2 Control Designations

Table 10-15: Update C2 Control Designations

Message Name (Name:Type)	Required Fields (Name:Type)
ControlRequestMT: ControlRequestMT	CapabilityID: CapabilityID_Type
ControlRequestStatusMT: ControlRequestStatusMT	None
MA_FlightCapabilityMT_1: MA_FlightCapabilityMT	None
MA_FlightCapabilityMT_2: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.3 COP Package

10.2.3.1 VI Updates to COP

The VI Updates to COP sequence diagram specifies the interaction of VI sending updates of the vehicle state to MA.

At a predefined periodic rate, the following messages are sent: *MA_PositionReportDetailedMT*, *MA_NavigationReportMT*, *ComponentStatusMT*, *MA_FlightCapabilityMT*, and *MA_FlightCapabilityStatusMT*. *MA_FlightActivityMT* will be sent at a predefined periodic rate while a flight activity is active.

The *MA_PositionReportDetailedMT* must include *AirData*. The *MA_NavigationReportMT* must have the fuel mass set in *Fuel*, the duration set in *Duration*, the fuel percent set in *Percent*, and the *ContingencyLevel* set to the contingency level. The *ComponentStatusMT* must have the *ComponentID* set to the *System ID* and the *ComponentState* set to the *Vehicle Autonomy's state*. The *MA_FlightCapabilityMT* must contain the applicable performance limits. The *MA_FlightCapabilityStatusMT* must contain the *CapabilityID* of the flight capability and the *Availability* set the availability of Vehicle Autonomy to accept flight commands. The *MA_FlightActivityMT* must contain the last flight command (if any).

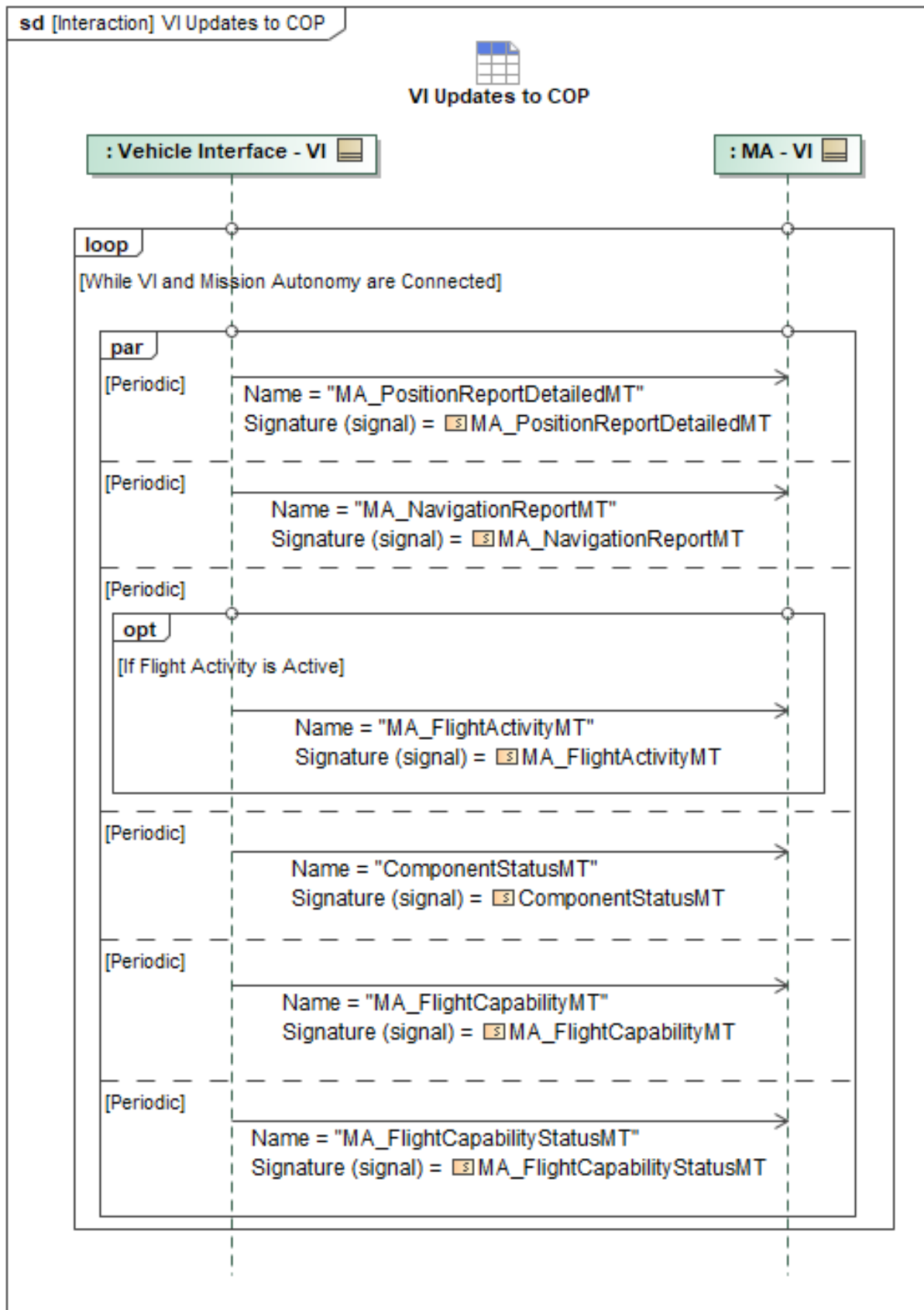


Figure 10-17: VI Updates to COP

Table 10-16: VI Updates to COP

Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None
MA_NavigationReportMT: MA_NavigationReportMT	Fuel: MassType Percent: PercentType Duration: DurationType
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	AirData: MA_AirDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.4 Data Validation Package

10.2.4.1 Checksum Validation

MA can check that the data stored on MA and FA is consistent using a checksum approach. MA can compare the *SHA_2_Hash* in the *ProductMetadataMT* for the associated object (airfield, route plan, etc.) to identify if there are discrepancies between the files. Because MA and FA may store files in different native formats, this checksum should compare the *SHA_2_Hash* on the UCI version of the file (after conversion from native format for MA, before conversion to native file format for FA).

If MA is missing the data records to be able to validate data stored on MA and FA via checksum, it can send a *QueryDataRequestMT* to FA with a query including the associated ID of the applicable data record. Once requested, FA will send the *ProductLocationMT* and *ProductMetadataMT* containing the details of the relevant data record, including the *SHA_2_Hash* field in *ProductMetadataMT*. If FA is unable to complete the query, *QueryDataRequestStatusMT* will be returned with *RequestProcessingState* set to *FAILED*, and including *RequestProcessingStateReason* with the reason for the failure.

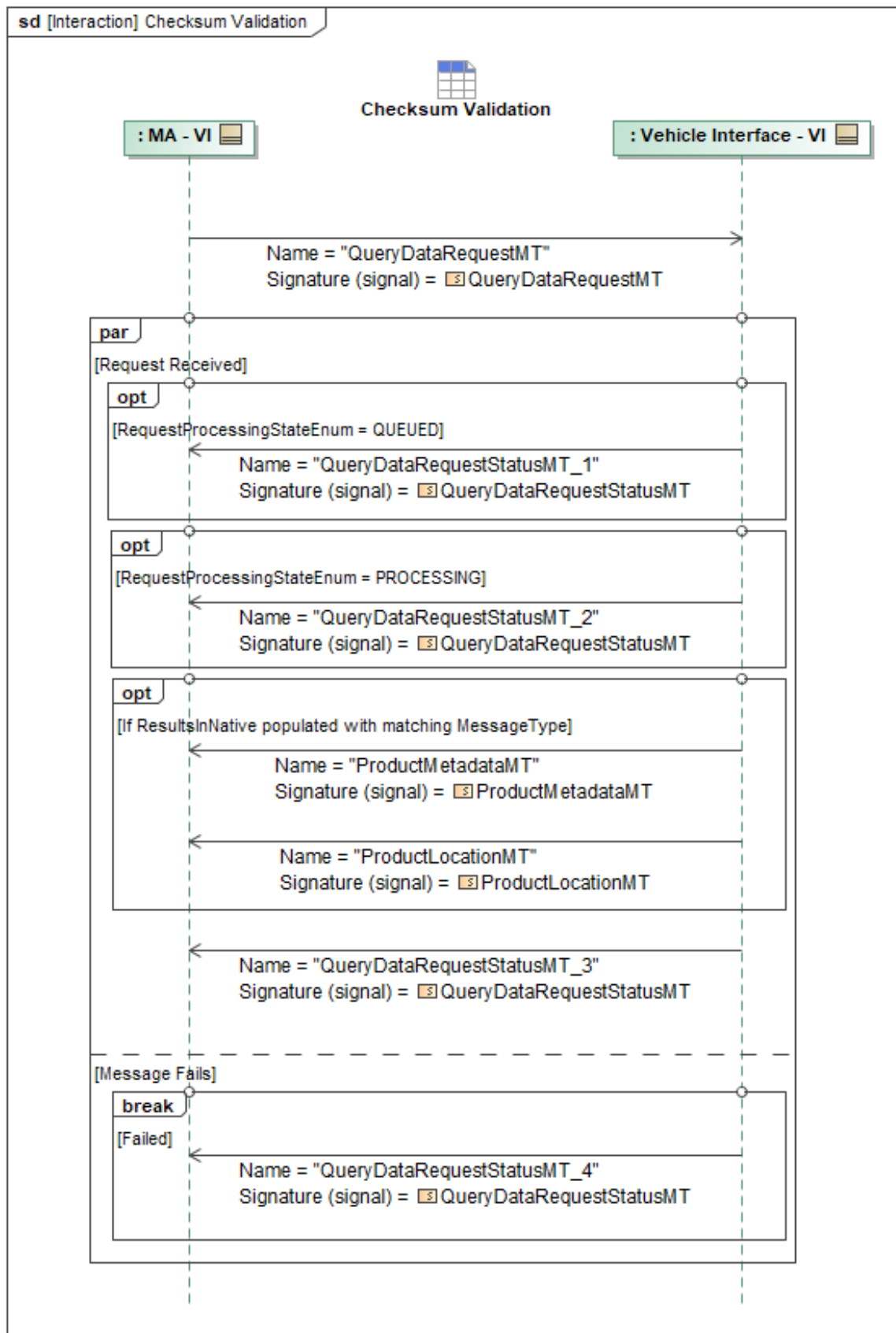


Figure 10-18: Checksum Validation

Table 10-17: Checksum Validation

Message Name (Name:Type)	Required Fields (Name:Type)
ProductLocationMT: ProductLocationMT	None
ProductMetadataMT: ProductMetadataMT	SHA_2_Hash: SHA_2_256_HashType
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.4.2 Query for Missing Data

MA can query FA for a list of all object IDs of a given data type (ex: Route Plans and Airfields for route plan data validation) stored on FA by sending *QueryDataRequestMT* to FA (via VI), with *QueryIdentifiersOnly* to indicate only IDs should be returned, and *QueryMessage.Query* specifying the types of data for which MA is querying. FA will respond with *QueryDataRequestStatusMT* with the list of requested IDs in *Results*.

Once the list of IDs is returned, MA evaluates if there are any it was not aware of or any that FA is missing. For any data that MA is not aware of (e.g. due to lost awareness after a reboot), it will then call 'Query Airfield Update' and/or 'Query Route Plan' behaviors to request the missing data, along with the associated metadata required for checksum validation, from FA.

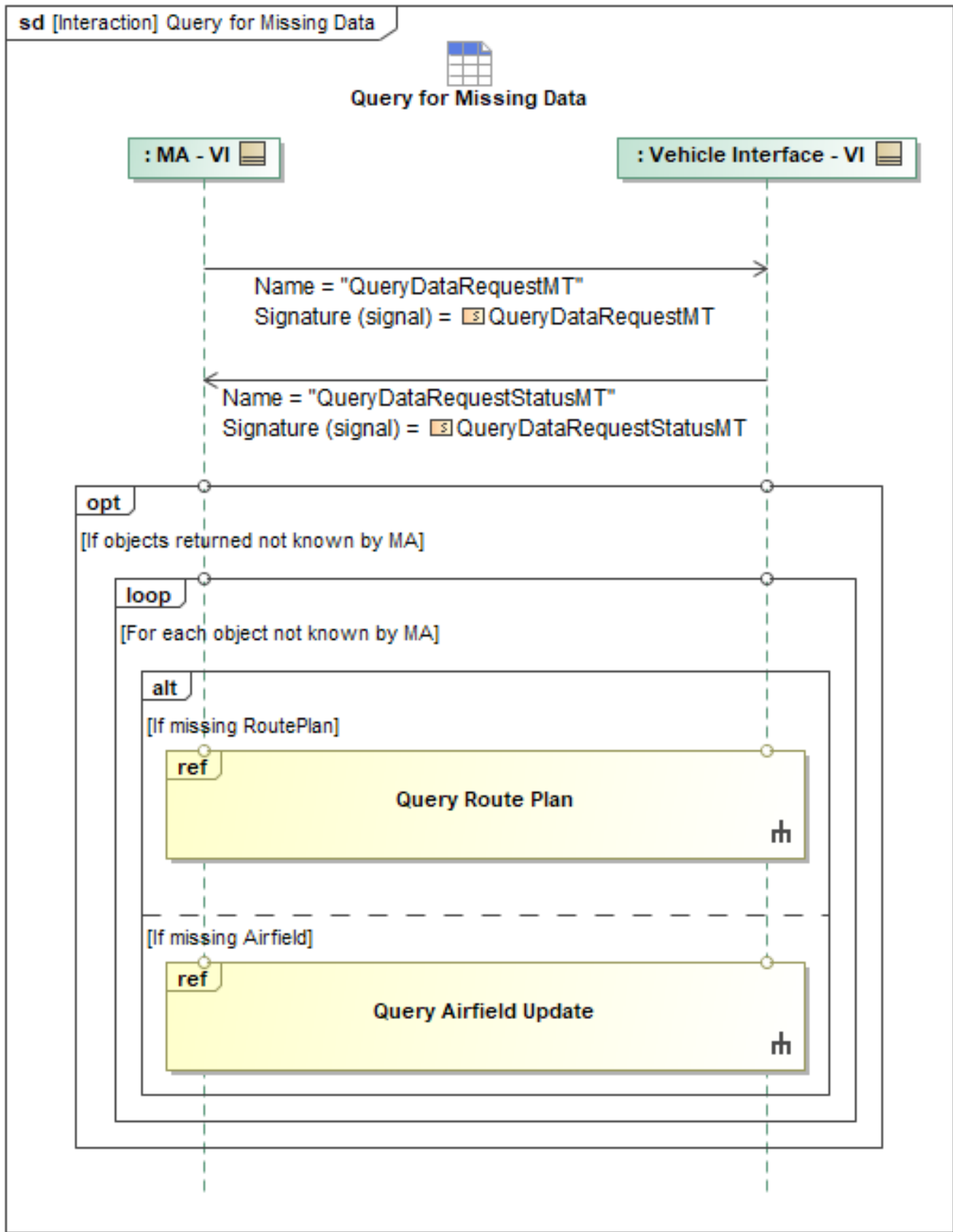


Figure 10-19: Query for Missing Data

Table 10-18: Query for Missing Data

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	Query: QueryPET
QueryDataRequestStatusMT: QueryDataRequestStatusMT	Result: QueryResultType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.4.3 Route Plan Data Validation

When MA needs to validate that it has the same route plan-related data as FA, there are three steps that can be used as needed for this data validation. Route plan-related data stored on FA and thus covered here include *MA_RoutePlanMTs* and Airfields (contained in *AirfieldReportMTs*).

First, MA can use the 'Query for Missing Data' behavior to query for a list of all object IDs of a given data type (RoutePlans or Airfields) stored on FA. This step must be performed on startup and if MA ever reboots. It can subsequently be re-run if there is ever a concern that MA has lost awareness of data stored on FA, or as needed. One example of wanting to double check for missing data is if MA determines this platform needs to land safely as soon as possible, and it can query to make sure there are no airfields stored on FA that it does not have awareness of.

Once the list of IDs is returned, MA checks the list for inconsistencies. For any data that MA is not aware of, it will then query FA for the missing data, as well as the file location and metadata (including Hash) of the missing object(s).

Second, MA can check that the data stored on MA and FA is consistent using a checksum approach. MA can compare the *SHA_2_Hash* in the *ProductMetadataMT* for the associated object (airfield, route plan, etc.) to identify if there are discrepancies between the files. Because MA and FA may store files in different native formats, this checksum should compare the *SHA_2_Hash* on the UCI version of the file before conversion to/from native file format. If MA is missing the file metadata of the data it wants to validate, or has reason to request a fresh hash, it can query by ID for the *ProductMetadataMT* and *ProductLocationMT* of the file in question using the 'Checksum Validation' behavior.

Finally, if a checksum is performed and finds that there is a discrepancy between data saved on MA and FA, it will be corrected by having the 'authority system' send the correct data to the other system. FA is the authority for airfields and for route plans associated with takeoff, departure, approach, and landing, while MA is the authority for other, non-safety critical route plans. If there is inconsistency with data for which MA is the authority, it will send the correct data to FA. As MA is the one performing the checksum, if there is inconsistency with data for which FA is the authority, MA will query for the correct versions from FA.

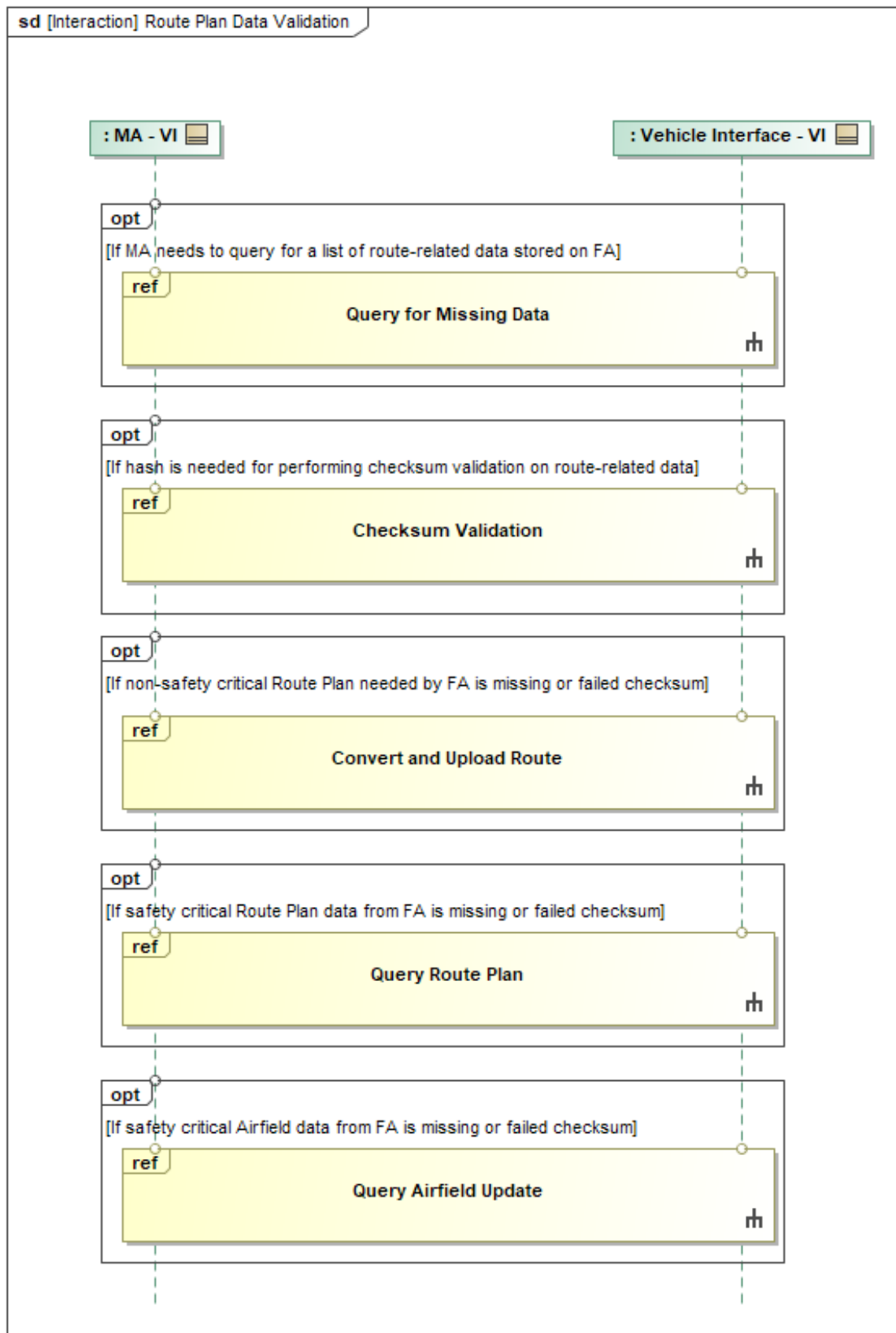


Figure 10-20: Route Plan Data Validation

10.2.5 Route Plan Behaviors Package

10.2.5.1 Activate Route

Activate Route is used to signal that FA should begin to follow (i.e. execute) a stored *MA_RoutePlan* that has been uploaded (if not already stored on FA pre-mission) and prepared for activation by previous interactions.

The precondition for this interaction is that the *MA_RoutePlan* is already in the *READY_FOR_ACTIVATION* state. The activation state of the route plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatus* that corresponds to the *MA_RoutePlan* in question.

Mission Autonomy (MA) can coordinate activation of routes with the VI isolator. This can be used for takeoff, landing, or any other phase of flight. In order to activate takeoff, MA will send *MA_MissionPlanActivationCommand* with the *BySubPlan* field populated, referring to the takeoff *MA_RouteActivityPlan* stored on FA. Before routes can be activated, the *MA_RoutePlanMT* must be taken through the plan activation state machine provided by the semantics of the OMS/UCI schema. The diagrams: "Convert and Upload Route" and "Prepare for Route Activation" show the interactions required to achieve this between MA and FA.

MA coordinates the activation of the route plan by sending an *MA_MissionPlanActivationCommandMT* with *CommandType* set to *ACTIVATE*. Once VI activates the route and begins guiding, it replies with *MA_MissionPlanActivationCommandStatusMT* with *ActivationCommandByState/PlanActivationCommandState* set to *ACTIVATE* and *ActivationStatus/CommandStatus* set to *COMPLETED*.

Each *MA_MissionPlanActivationCommandMT* should use choice type *Command/ActivationDetails/BySubPlan*. Furthermore, the *SubPlan* should be a *RoutePlanActivationType*. *MA_MissionPlanActivationCommandStatusMT* should specify *ActivationCommandByState/Plans* with *RoutePlanID*.

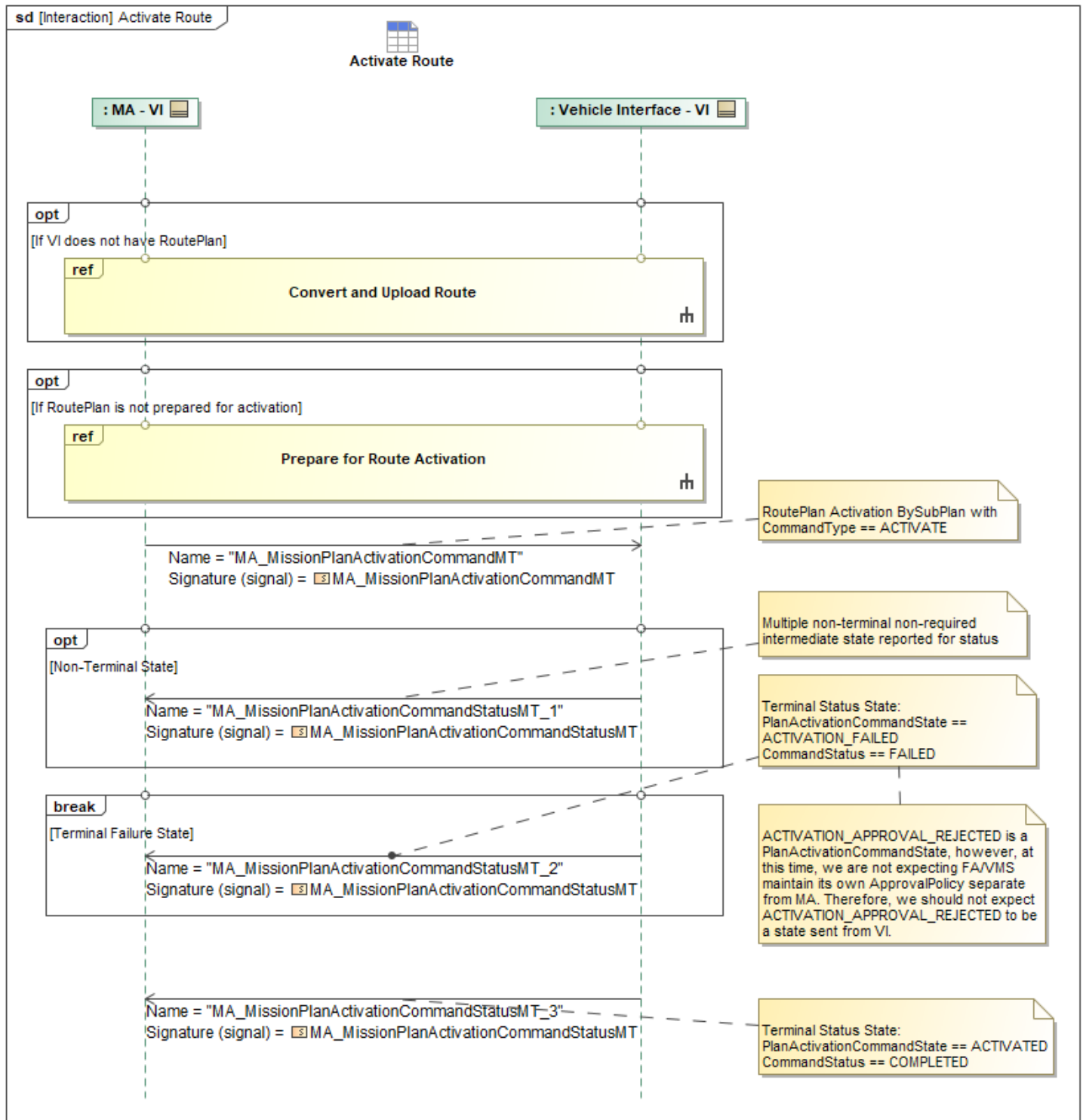


Figure 10-21: Activate Route

Table 10-19: Activate Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	RoutePlan: MA_RoutePlanActivationType

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type ActivationCommandByState: MA_PlanActivationStatusType
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type ActivationCommandByState: MA_PlanActivationStatusType
MA_MissionPlanActivationCommandStatusMT_3: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type ActivationCommandByState: MA_PlanActivationStatusType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.5.2 Convert and Upload Route

Mission Autonomy (MA) can coordinate conversion, upload, and preparation for route activation with the VI isolator. Note, MA may not send new *MA_RoutePlanMTs* to FA related to takeoff, departure, approach, or landing, as those are considered safety critical. For those phases of flight, MA must activate one of the pre-planned, safety-validated takeoff, departure, approach, and/or landing Route Plans stored pre-flight on FA.

The *PREPARE_FOR_UPLOAD* exchange serves two purposes: data conversion and upload preparation. *MA_RoutePlanMTs* typically require translation to native vehicle management system (VMS) formats. Conversion can occur at this time if required. Upload is critical to ensuring routes are properly loaded to FA/VMS via VI. At any given time, many *MA_RoutePlans* may be transmitted on MA message busses that are not of concern to FA. Since the VMS isolator may be running in a real time and/or compute constrained environment it is important to signal that routes are being prepared for it to consume. In this way, the VI isolator does not need to consume each and every *MA_RoutePlanMT* published. VI isolator only needs to receive *MA_RoutePlanMTs* during the upload state and only for the UUIDs associated with the corresponding *MA_MissionPlanActivationCommandMT*.

PREPARE_FOR_UPLOAD is accomplished by sending an *MA_MissionPlanActivationCommandMT* with *Command/ActivationDetails/BySubPlan/RoutePlan*. The corresponding *CommandType* is set to *PREPARE_FOR_UPLOAD*. In previous UCI major versions *PREPARE_FOR_UPLOAD* was *CONVERT*. When conversion is complete VI will reply with a *MA_MissionPlanActivationCommandStatusMT* with *ActivationCommandByState/PlanActivationCommandState PREPARE_FOR_UPLOAD* and *ActivationStatus/CommandStatus* set to *COMPLETED*.

In response to FA receiving *MA_RoutePlanMT*, a *MA_SystemNotificationMT* is sent back to MA with the *SubsystemID* and *ServiceID* set to the corresponding message receiver. This could be the VI isolator or wrapper. The *AssociatedID* is set to the *RoutePlanID* corresponding to the original message. *NotificationState* should be set to *CONFIRMED* if the message was received properly. When FA returns a queried *MA_RoutePlanMT*, FA also sends *ProductLocationMT* and *ProductMetadataMT* (including the *SHA_2_Hash* field) to MA, both containing details of FA's stored UCI version (as opposed to the version in FA's native file format) of the *AirfieldReportMT*. These will enable the performance of MA-FA route plan data validation behaviors. A single *MA_MissionPlanActivationCommandMT* can be used to coordinate multiple route or route/airfield

updates. Loop over each artifact as required to pass all data to the VI.

Once the VI isolator is prepared for upload, *MA_RoutePlanMTs* are published by MA. MA signals VI that the routes have been published by sending a *MA_MissionPlanActivationCommandMT* with *CommandType* set to *UPLOAD*. The intent is for the converted *MA_RoutePlanMT* to be saved within the VMS's memory for future reference. Once this is complete VI replies to MA with *MA_MissionPlanActivationCommandStatusMT* with *ActivationCommandByState/PlanActivationCommandState* set to *UPLOAD* and *ActivationStatus/CommandStatus* set to *COMPLETED*. This gives the VI isolator the opportunity to stop listening.

Each *MA_MissionPlanActivationCommandMT* uses choice type *Command/ActivationDetails/BySubPlan*. Furthermore, the *SubPlan* should be a *RoutePlanActivationType*. *MA_MissionPlanActivationCommandStatusMT* should specify *ActivationCommandByState/Plans* with *RoutePlanID*.

If the VI is not able to upload or process data as commanded, the VI can publish *MA_MissionPlanActivationCommandStatusMT* with *CommandStatus* set to *FAILED*.

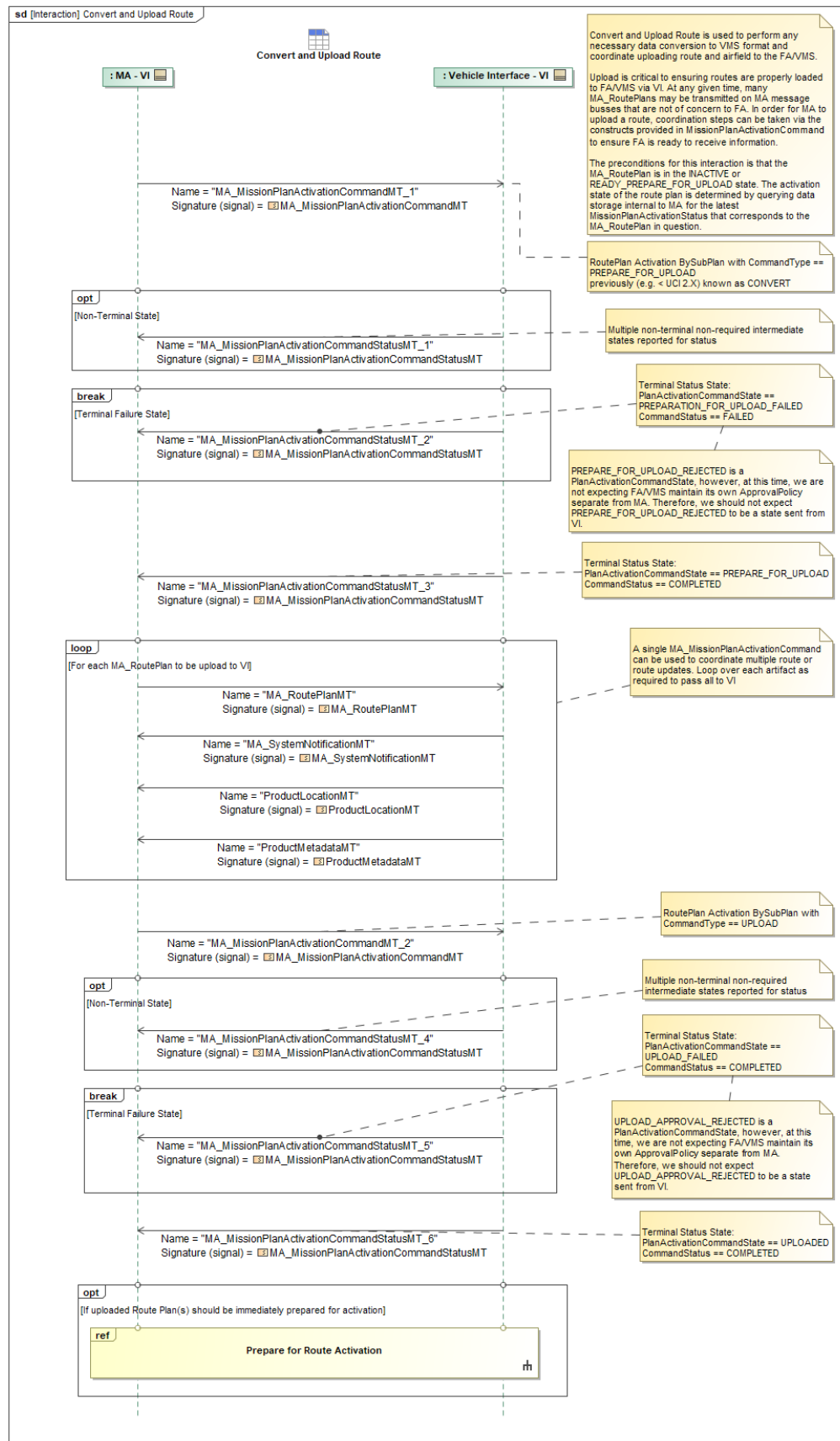


Figure 10-22: Convert and Upload Route

Table 10-20: Convert and Upload Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT_1: MA_MissionPlanActivationCommandMT	RoutePlan: MA_RoutePlanActivationType
MA_MissionPlanActivationCommandMT_2: MA_MissionPlanActivationCommandMT	RoutePlan: MA_RoutePlanActivationType
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_3: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_4: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_5: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_6: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_RoutePlanMT: MA_RoutePlanMT	None
MA_SystemNotificationMT: MA_SystemNotificationMT	None
ProductLocationMT: ProductLocationMT	None
ProductMetadataMT: ProductMetadataMT	SHA_2_Hash: SHA_2_256_HashType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.5.3 Prepare for Route Activation

Prepare for Route Activation is used to ensure that an *MA_RoutePlan* saved within FA/VMS is ready to activate for execution without delay upon a subsequent Activate Route exchange. This includes any final checks, preparations, memory allocations, etc.

A precondition for this interaction is that the *MA_RoutePlan* is already in the *UPLOAD* state. The activation state of the plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatusMT* that corresponds to the *MA_RoutePlan* in question.

MA send VI a *MA_MissionPlanActivationCommandMT* containing the *RoutePlan* along with a *CommandType* of *PREPARE_FOR_ACTIVATION*.

If VI fails to prepare the *MA_RoutePlan* for activation, for any reason, it is possible for VI to report a

PlanActivationCommandState of *PREPARATION_FOR_ACTIVATION_FAILED* with corresponding *CommandStatus* set to *FAILED* along with the applicable *RoutePlanID*.

If VI succeeds in preparing the *MA_RoutePlan* for subsequent activation this sequence can terminate with a *PlanActivationCommandState* of *READY_FOR_ACTIVATION* and a corresponding *CommandStatus* set to *COMPLETED* along with the applicable *RoutePlanID*.

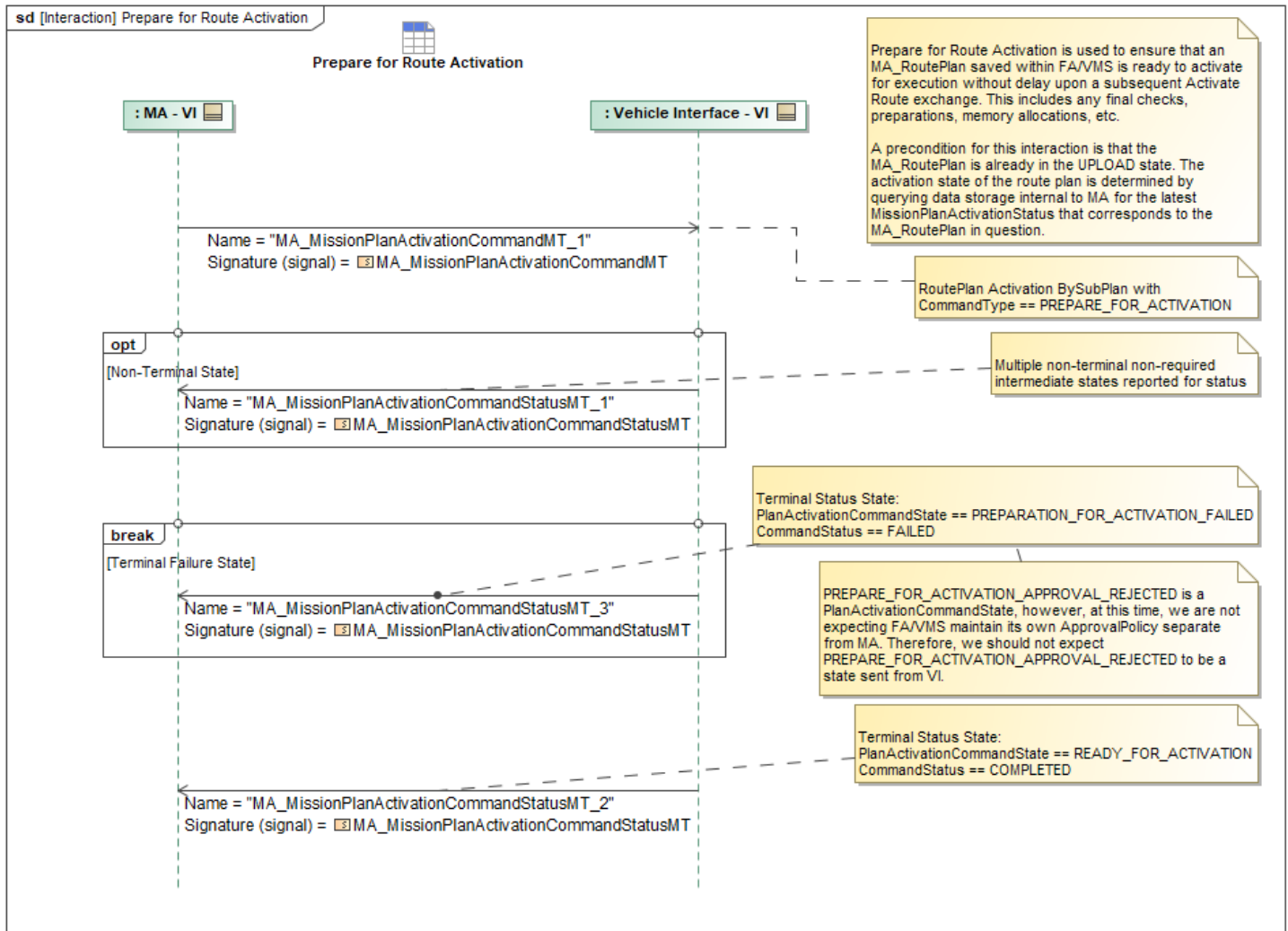


Figure 10-23: Prepare for Route Activation

Table 10-21: Prepare for Route Activation

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT_1: MA_MissionPlanActivationCommandMT	RoutePlan: MA_RoutePlanActivationType
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandStatusMT_3: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.5.4 Receive Deactivate Route

This interaction will forward the command to deactivate a route from MA to VI and return the status.

The precondition for this interaction is:

MA_MissionPlanMT and *MA_RoutePlanMT* is loaded and either ready for activation or that the *MA_RoutePlanMT* is already in the *ACTIVATED* state.

The activation state of the route plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatusMT* that corresponds to the *MA_RoutePlanMT* in question.

This interaction will use *MA_MissionPlanActivationCommandMT* to indicate the *MA_RoutePlanMT* to *DEACTIVATE*. It will set the *BySubPlan:RoutePlanID* to the UUID of the route to deactivate.

If the route plan was not yet *EXECUTING* will get back a *MA_MissionPlanActivationCommandStatusMT* with an *ActivationStatus* of *COMPLETED*.

Else the route plan is *EXECUTING* will get back a *MA_MissionPlanActivationCommandStatusMT* with an *ActivationStatus* of *FAILED*.

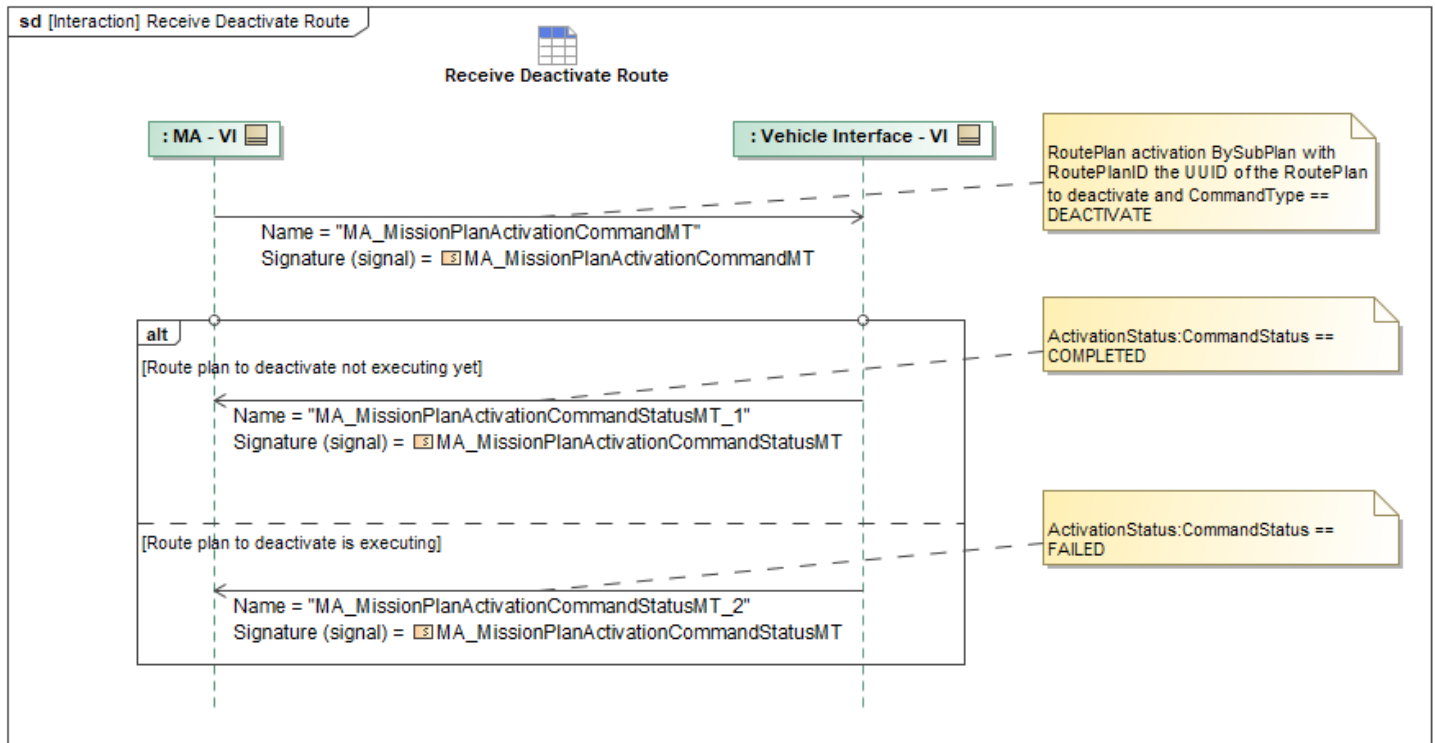


Figure 10-24: Receive Deactivate Route

Table 10-22: Receive Deactivate Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	CommandType: PlanActivationCommandEnum BySubPlan: MissionPlanSubplanActivationType DEACTIVATE:
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	CommandStatus: ProcessingStatusEnum COMPLETED:
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	FAILED: CommandStatus: ProcessingStatusEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.5.5 Validate Route Plan

This sequence allows MA to have VI validate a RoutePlan. First the *RoutePlanValidationCommandMT* is sent from MA to VI. VI acknowledges the command by replying with *RoutePlanValidationCommandStatusMT*, and then follows up with the validation results in *RoutePlanValidationMT*.

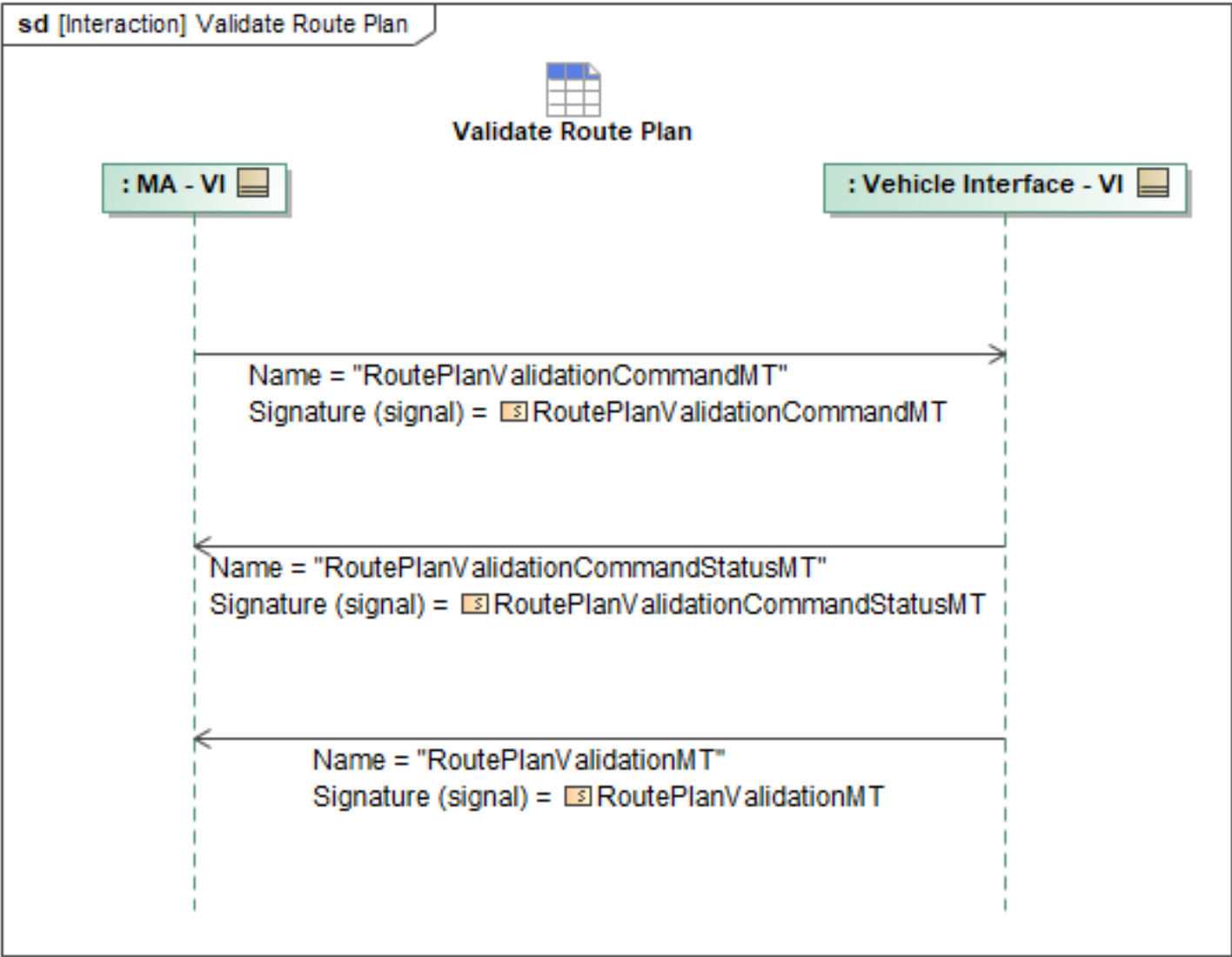


Figure 10-25: Validate Route Plan

Table 10-23: Validate Route Plan

Message Name (Name:Type)	Required Fields (Name:Type)
RoutePlanValidationCommandMT: RoutePlanValidationCommandMT	WeatherAreaData: WeatherAreaDataType
RoutePlanValidationCommandStatusMT: RoutePlanValidationCommandStatusMT	None
RoutePlanValidationMT: RoutePlanValidationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.5.6 Verify Plan Patch Is Possible - VI

Depending on the type of *Plan* being patched, one of *RoutePlanValidationCommandMT*,

TaskPlanValidationCommandMT, is sent from MA to MS. MS then responds with the corresponding *CommandStatusMT*, i.e., *RoutePlanValidationCommandStatusMT*, *TaskPlanValidationCommandStatusMT*. The messages selected are based on what plan is being patched, i.e., *RoutePlan* or *TaskPlan*, respectively. Within the **CommandMT* message, the *PlanPart* contains the plan patch that needs to be validated.

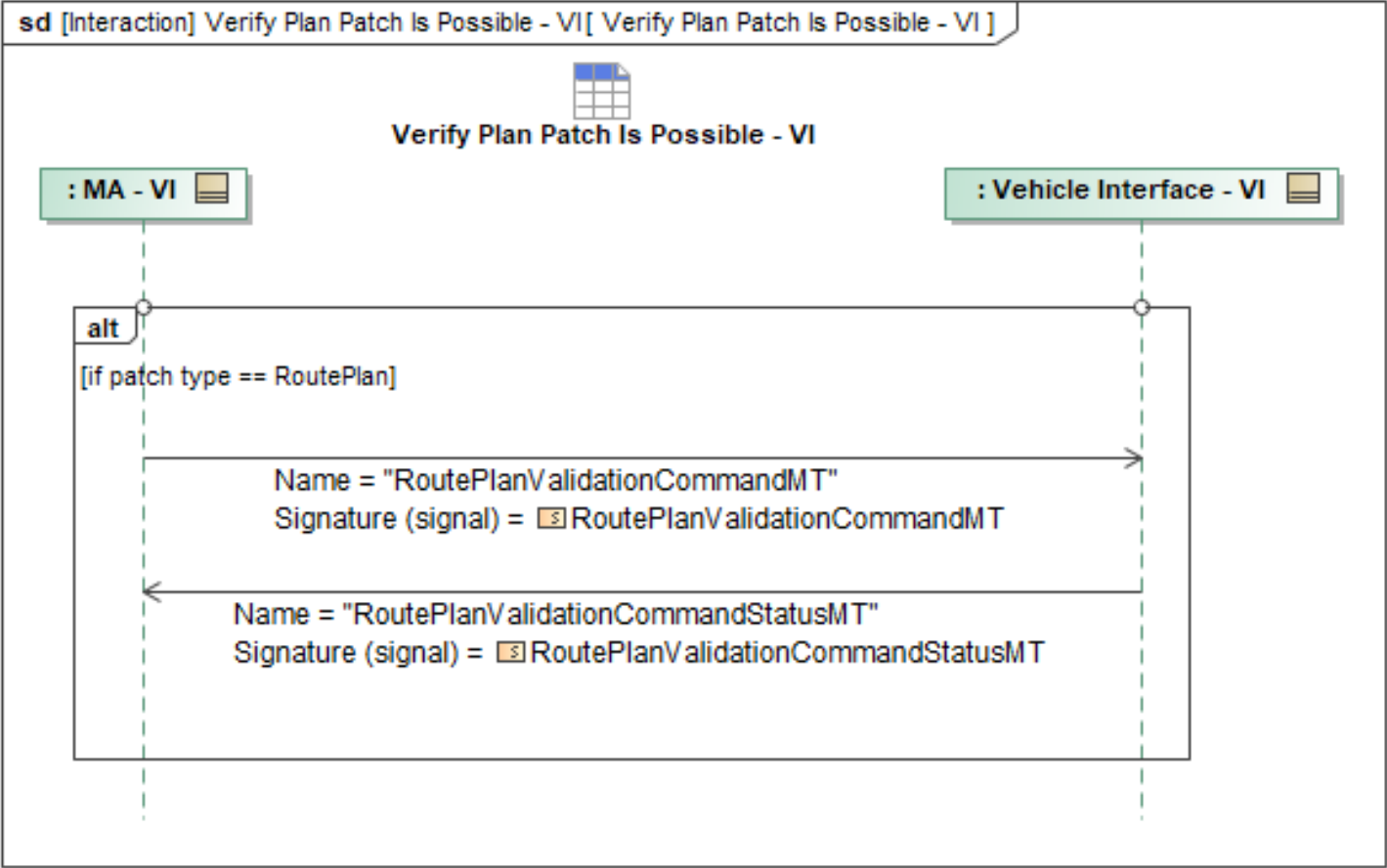


Figure 10-26: Verify Plan Patch Is Possible - VI

Table 10-24: Verify Plan Patch Is Possible - VI

Message Name (Name:Type)	Required Fields (Name:Type)
RoutePlanValidationCommandMT: RoutePlanValidationCommandMT	PlanPart: PathTypeEnum
RoutePlanValidationCommandStatusMT: RoutePlanValidationCommandStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.5.7 VI Deactivate Route

VI Deactivate Route Activation is used to signal that FA determined it should abort a route either before or after it starts executing based on previous interactions (Activate Route)

The precondition for this interaction is:

EITHER *MissionPlan* and *RoutePlan* are loaded and ready for activation

OR that the *MA_RoutePlan* is already in the *ACTIVATED* state. The activation state of the route plan is determined by querying data storage internal to MA for the latest *MA_MissionPlanActivationStatus* that corresponds to the *MA_RoutePlan* in question.

At some point FA/VI will abort a route. This can be either before or after the *RoutePlan* starts executing.

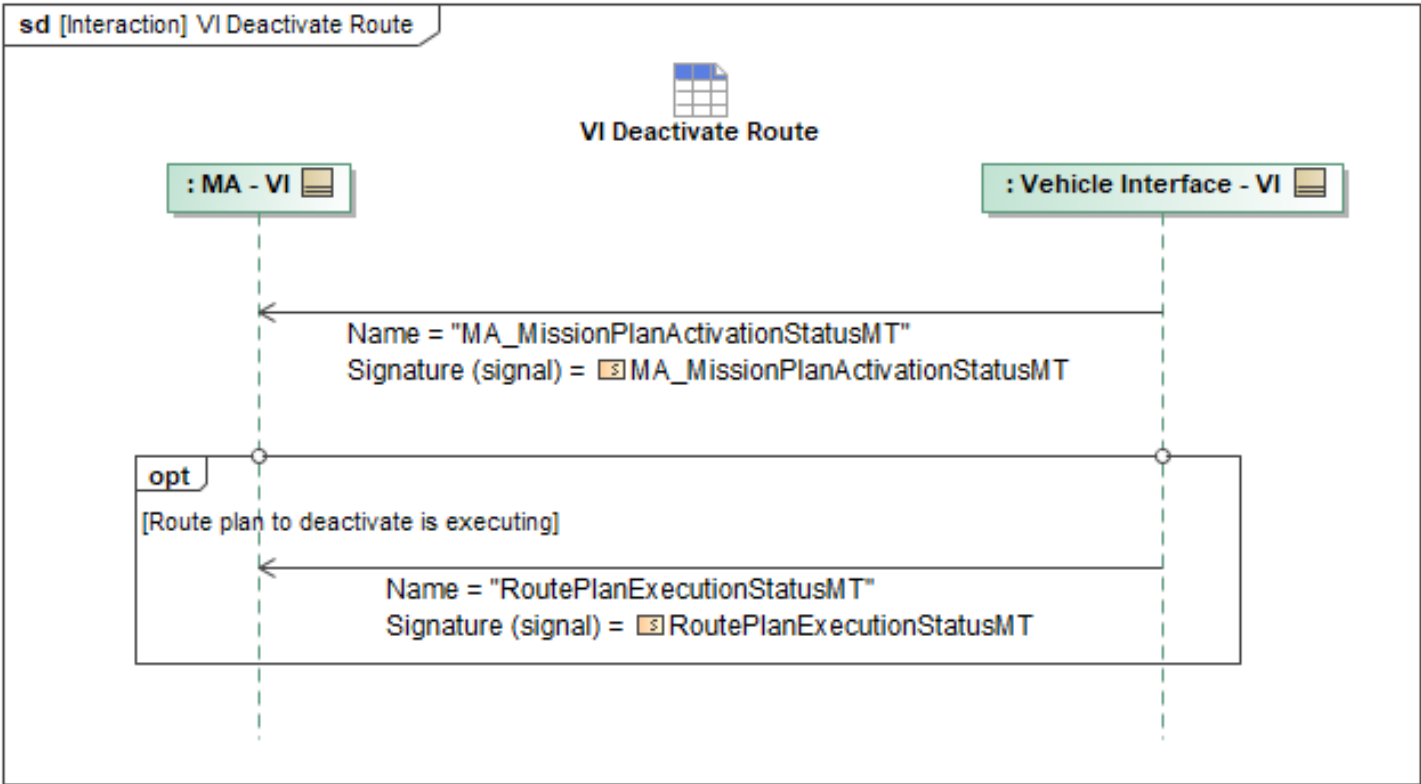


Figure 10-27: VI Deactivate Route

Table 10-25: VI Deactivate Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	RoutePlanID: RoutePlanID_Type DEACTIVATED: SubPlanActivationState: PlanActivationStateType
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	CANCELED: PlanExecutionStatus: RoutePlanExecutionStateType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6 Status Package

10.2.6.1 Exchange Heartbeat - Subsystem Status Reports

The OMS UCI construct allows MA to broadcast MA health by using periodic UCI messages, all of which use a common *timestamp* in the message header. It is expected that the vehicle will employ an OMS Isolator to interact with the OMS Mission Package and MA. At a minimum, the OMS Isolator will publish *ServiceStatus* (for the Isolator software itself) and *MA_FlightCapabilityStatus* to expose availability status of its flight capabilities. MA will subscribe to all of these UCI messages to confirm VI connectivity and health/status. MA will also publish *ServiceStatus* to confirm its presence and connectivity to the VI.

The *ServiceStatus* rate should be set in a configuration file for the Vehicle Interface with a default value of 1 second. While it is expected that all messages published and subscribed to by the OMS Isolator can and will be used to monitor overall health of the VI, the minimum message set for heartbeat/status exchange are *ServiceStatus* and *MA_FlightCapabilityStatus* from VI to MA, and *ServiceStatus* from MA to VI. The component heartbeat message from Flight Autonomy to Mission Autonomy is achieved with the *SubsystemStatus* UCI message. The heartbeat message from the OMS Isolator is achieved with the *ServiceStatus* UCI message. The *ServiceStatus* is a periodic message with a configured time interval. MA can send *ServiceStatusDataRequestStatus* to provide the status of the OMS Isolator.

SubsystemStatus is used to report vehicle subsystem status, and doubles as a heartbeat message from Flight Autonomy (FA) to the MA. This is a periodic message with a configurable specific time interval. The *SubsystemStatusDataRequest* is originated from the MA to the FA to query the status of the FA. A response for the *SubsystemStatusDataRequest* is send out with *SubsystemStatusDataRequestStatus*. The *SubsystemStatus* schema identifies individual components of the FA in the schema to provide multiple components in the *SpecificStatus* of the UCI schema. All component aggregations are reported to Heath & Status Aggregator. This status gives MA the overall health of the FA and serves as the Heartbeat message. Note that Component identification and description are left to the interface implementers to define. *MA_Fault* contains a *Severity* field that supports the following enumerations: *NOMINAL*, *ADVISORY*, *CAUTION*, *WARNING*, and *FAILED*. This message should be used in conjunction with *SubsystemStatus* to report on both the component state and fault status.

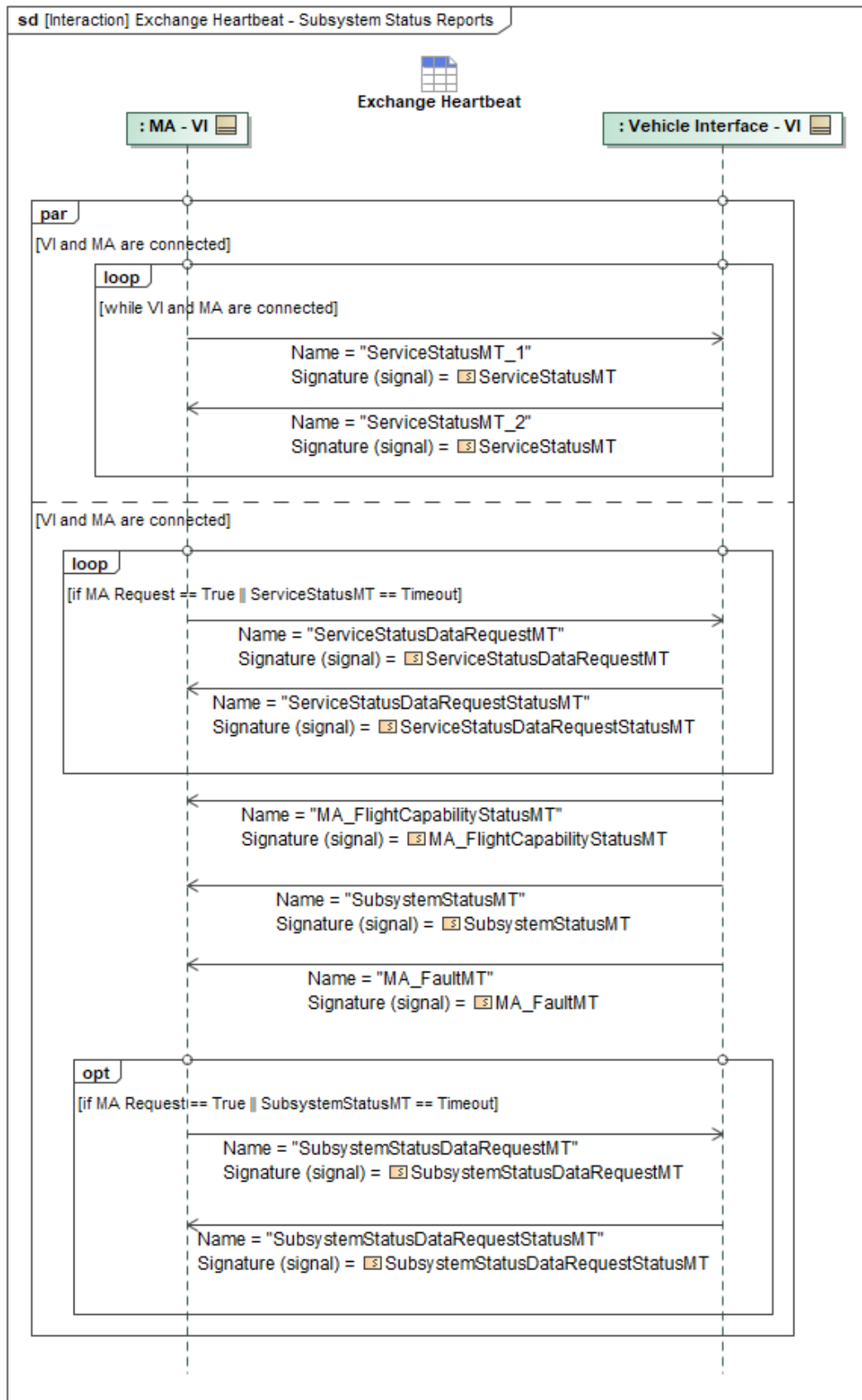


Figure 10-28: Exchange Heartbeat - Subsystem Status Reports

Table 10-26: Exchange Heartbeat - Subsystem Status Reports

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	Severity: FaultSeverityEnum
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None
ServiceStatusDataRequestMT: ServiceStatusDataRequestMT	None
ServiceStatusDataRequestStatusMT: ServiceStatusDataRequestStatusMT	None
ServiceStatusMT_1: ServiceStatusMT	None
ServiceStatusMT_2: ServiceStatusMT	None
SubsystemStatusDataRequestMT: SubsystemStatusDataRequestMT	None
SubsystemStatusDataRequestStatusMT: SubsystemStatusDataRequestStatusMT	None
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.2 Publish Control Status

Flight Autonomy (FA), via Vehicle Interface (VI), publishes the *ControlStatusMT* message to Mission Autonomy (MA) to indicate the current controlling system and service. This status is indicated per *ControlType/CapabilityControl/CapabilityID* where *CapabilityIDs* can be identified by referencing the *MA_FlightCapabilityMT* messages from the "Control Mode Authorization" diagram. FA is responsible for safe operation of the vehicle and will always publish itself as the *PrimaryController* for all flight-related capabilities. This is indicated by specifying *ControlType/CapabilityControl/PrimaryController/SystemID* and *ControlType/CapabilityControl/PrimaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* corresponding to an appropriate OMS service on the VI. If VI is accepting inputs for a given capability from MA it will indicate a *ControlType/CapabilityControl/SecondaryController/SystemID* and *ControlType/CapabilityControl/SecondaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* for a particular MA service and *CapabilityID*. Alternatively if VI is not accepting inputs from MA it will not populate any *ControlType/CapabilityControl/SecondaryController* entries.

VI will also indicate which level of control is appropriate for a given combination of *CapabilityID*. This is accomplished by enumerating the types of commands accepted by VI for that *CapabilityID* in the *ControlType/CapabilityControl/AcceptedInterface* list.

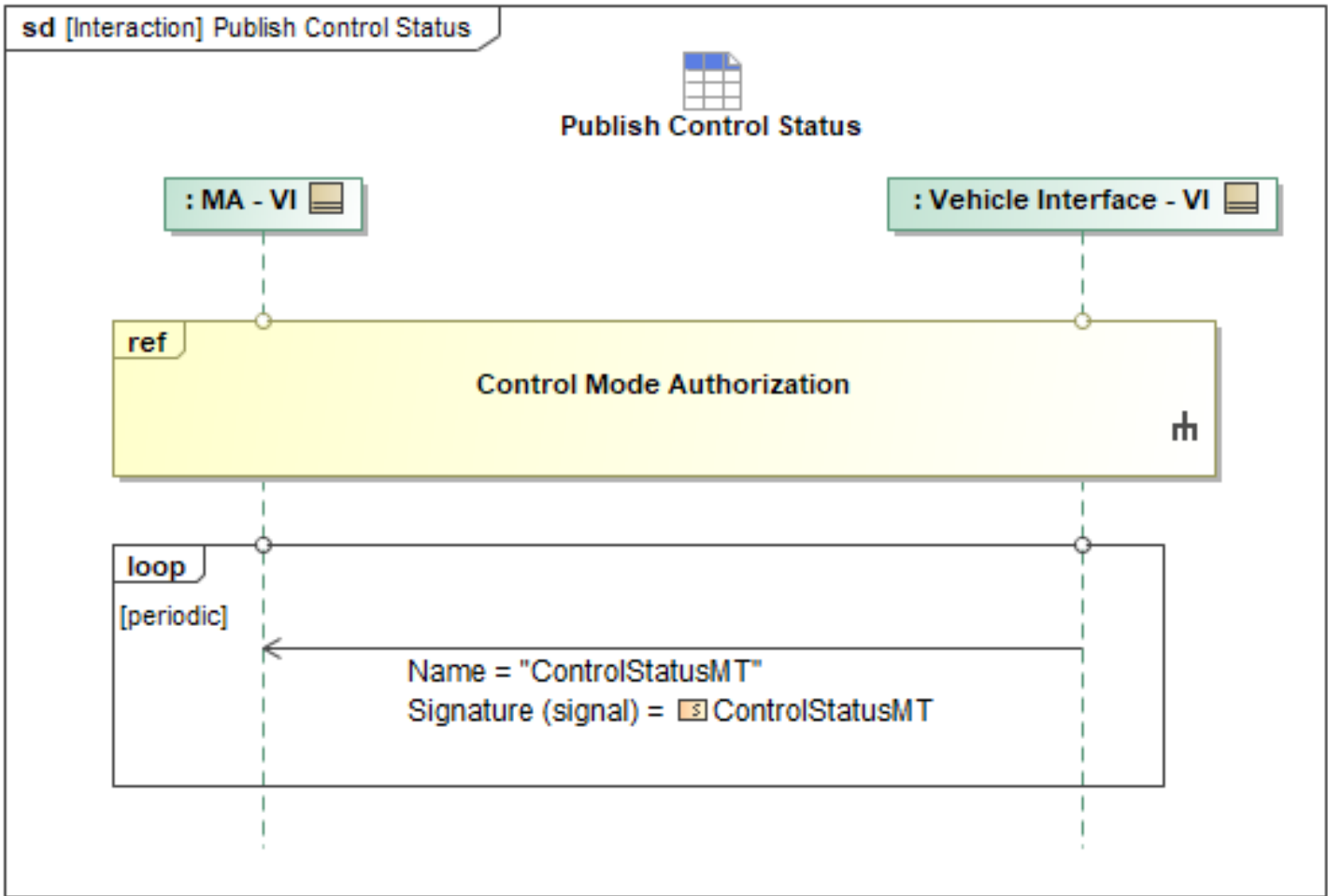


Figure 10-29: Publish Control Status

Table 10-27: Publish Control Status

Message Name (Name:Type)	Required Fields (Name:Type)
ControlStatusMT: ControlStatusMT	SecondaryController: SecondaryControllerType CapabilityControl: ControlStatusCapabilityControlType PrimaryController: PrimaryControllerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.3 Query Airfield Update

Mission Autonomy (MA) can query Flight Autonomy (FA) for pre-loaded and pre-validated Airfield and Take off and Landing (TO/L) route information stored within FA. This data includes alternate airfields for emergency landings as well as optional alternate TO/L routes for a given airfield.

In terms of sequence, once FA has received MA's query via *QueryDataRequest*, it will respond with *QueryDataRequestStatus* indicating *QUEUED* and *PROCESSING* states. If the query is successful and *ResultsInNative* is populated with matching *MessageType*, FA will respond with *AirfieldReportMT* message, that shall contain *AirfieldRunwayType* data structures that describe the geometry of takeoff

and landing runways. To respond to the query for TO/L routes, FA shall respond with *MA_RoutePlanMT* data structures, indicating TO/L routes.

If describing a takeoff airfield, runway, and route combination, the corresponding reference fields in the two messages—*AirfieldReportMT* and *MA_RoutePlanMT*—must be populated appropriately. For example, *MA_RoutePlanMT* must have a take-off path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT* Take-off coordinates and limits must also be specified. For takeoff, the 'Path' element within *MA_RouteType* will capture the takeoff route data, with its 'PathType' subelement designating the particular path as a takeoff route path, and 'PathSegment' elements representing the takeoff route data by use of waypoints. Wherever in the takeoff route that specific climb rate requirements apply (as denoted in the mission SIDS), the *Climb* subfield for the respective *PathSegment* in the takeoff procedure can specify the required Climb Rate. For aborted Take-off taxi routes, a separate 'Path' element, with a *PathType* enum of *AbortedTakeoffRoute* can be defined. The *PathSegment* element within this path can define the taxi route to exit the runway after a takeoff abort. This additional *Path* can be linked to the takeoff route through conditional chaining.

Note: To achieve conditional chaining, the *ConditionalPathSegment* subelement within the main takeoff *Path* element can be used. The *PathID* field within this *ConditionalPathSegment* subelement can point to the ID of the Aborted Takeoff Route taxi path. The *Condition* element in the *ConditionalPathSegment* can define when to take the abort path.

If describing a landing airfield, runway, and route combination, the corresponding fields must be populated appropriately. For example, the *MA_RoutePlanMT* must have a landing path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT* landing coordinates and limit must also be specified. Similar to take-off procedure's implementation, the *Path* element in *MA_RouteType* will be used to define the landing approach route. If applicable, the *Climb* subfield in the *PathSegment* element will be used to define any required autonomous descent rate for the landing approach route. Lastly, similar to the aborted take-off taxi route for the take-off case, a combination of additional defined *Path* for missed approach hold procedure, along with conditional chaining to tie it to the main approach route can be used to define the missed approach procedure.

If the *QueryDataRequestMT* fails, FA will respond with a *QueryDataRequestStatusMT* with the *RequestProcessingState* indicating *FAILED*.

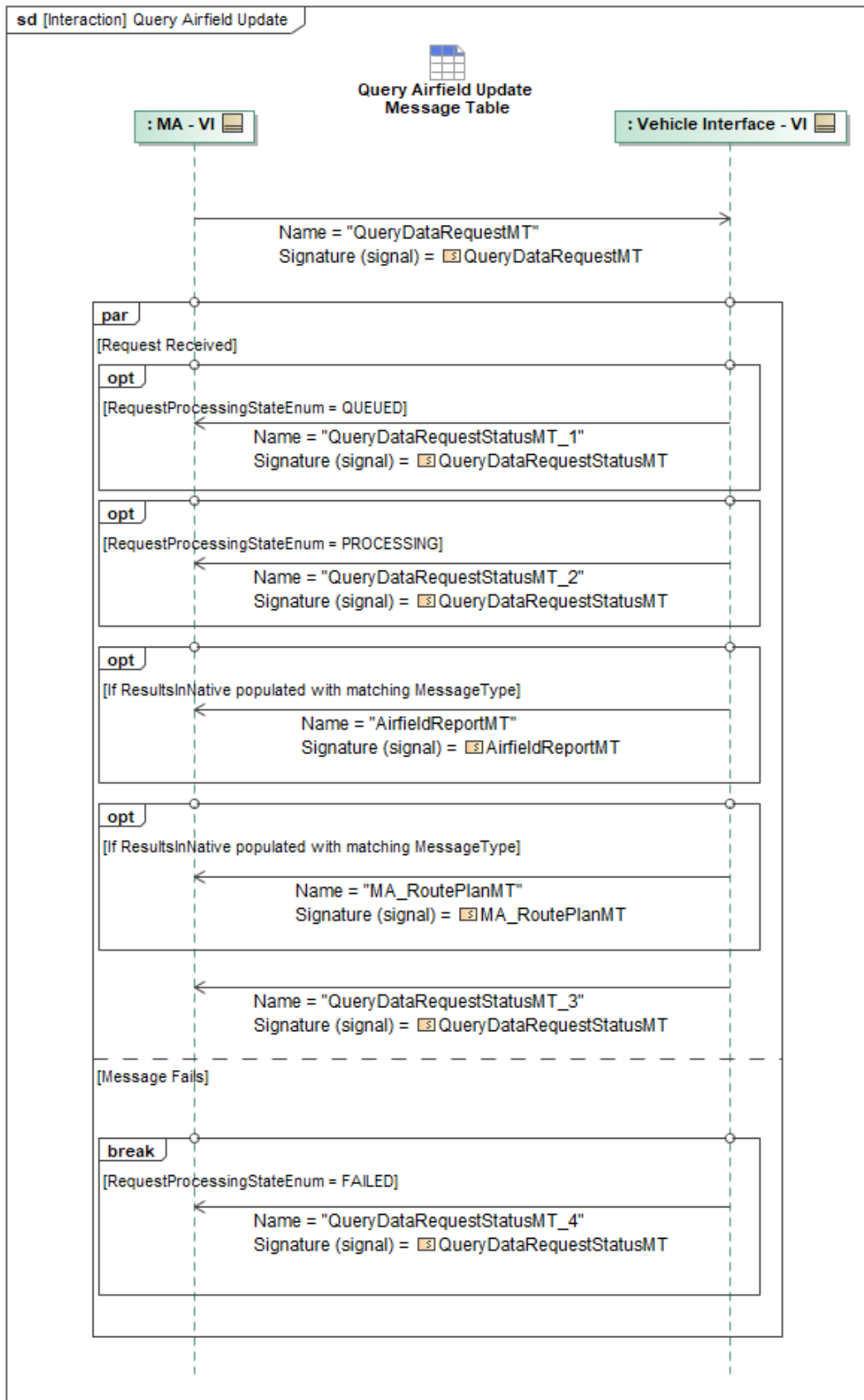


Figure 10-30: Query Airfield Update

Table 10-28: Query Airfield Update

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	TakeoffCoordinates: RunwayCoordinatesType LandingCoordinates: RunwayCoordinatesType Runway: AirfieldRunwayType Limit: Point3D_Type
MA_RoutePlanMT: MA_RoutePlanMT	AirfieldID: AirfieldID_Type RunwayID: RunwayID_Type
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.4 Query Route Plan

Mission Autonomy (MA) can use *QueryDataRequestMT* to query for route plan information stored in Flight Autonomy (FA) via the Vehicle Interface (VI) including preload options for takeoff, landing, and alternate airfields for emergency. Once FA has received the query, it will respond with *QueryDataRequestStatusMT* messages providing MA with status updates for the query until it reaches a terminal state, along with the queried *MA_RoutePlanMT* (assuming the query does not fail). When FA returns the queried *MA_RoutePlanMT*, FA also sends *ProductLocationMT* and *ProductMetadataMT* (including the *SHA_2_Hash* field) to MA, both containing details of FA's stored UCI version (as opposed to the version converted to FA's native file format) of the *MA_RoutePlanMT*. These will enable the performance of MA-FA route plan data validation behaviors.

The *MA_RoutePlanMT* data structures can indicate takeoff and landing paths. Critically, the *AirfieldReportMT* contains *AirfieldRunwayType* data structures that describe the geometry of the associated runways for takeoff and landing.

If describing a takeoff airfield, runway, and route combination, the corresponding fields must be populated appropriately. For example, the *MA_RoutePlanMT* must have a takeoff path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT* *TakeoffCoordinates/Limit* must also be specified. In this case, it is important that the *Route/Path/PathType* be set appropriately to indicate *TAKEOFF*.

If describing a landing airfield, runway, and route combination, the corresponding fields must be populated appropriately. For example, the *MA_RoutePlanMT* must have a landing path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT*

LandingCoordinates/Limit must also be specified. In this case, it is important that the *Route/Path/PathType* be set appropriately to indicate *LANDING*.

Ideally, takeoff and landing information can be specified simultaneously. Furthermore, multiple approach departure, and missed approach routes can be specified for an airport and per runway(s). This can be used during normal takeoff, landing, emergency diverts, and contingency management scenarios when MA is not in control of the vehicle. When specifying a takeoff or landing path within the *MA_RoutePlanMT* the *AirfieldID* and *RunwayID* must be specified and match associated fields in the corresponding *AirfieldReportMT*

This sequence should occur early in the mission before taxi to ensure MA is aware of all available route plans and airfields saved on FA to safely takeoff and land. The sequence can occur subsequently as required if MA needs to confirm the routes stored on FA.

This airfield information and the route plans associated with takeoff, departure, approach, and landing are loaded on FA pre-mission. They are considered Safety Critical and are thus "read only". New airfields (*AirfieldReportMTs*), as well as route plans pertaining to safety critical operations takeoff, departure, approach, and are not to be sent to FA.

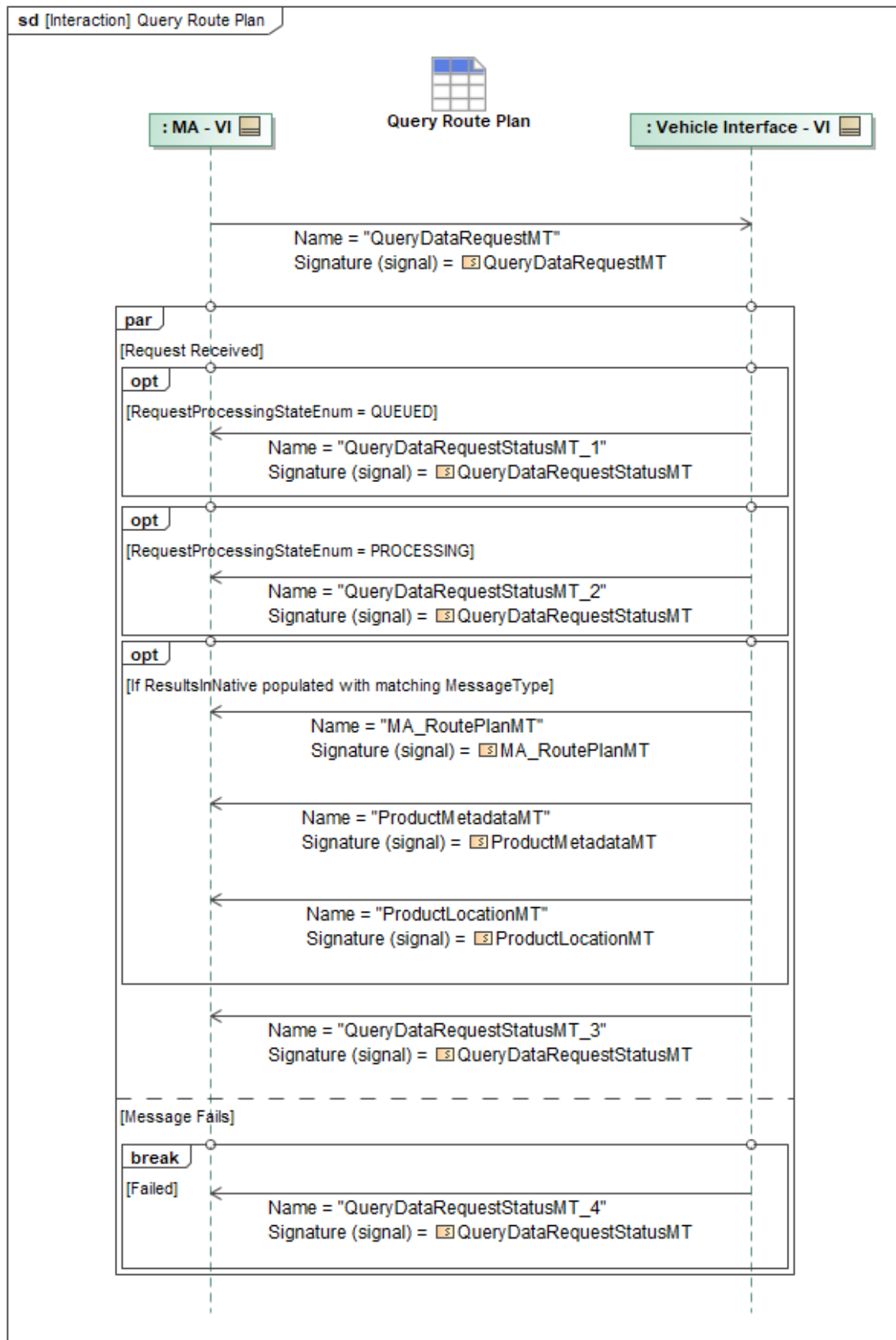


Figure 10-31: Query Route Plan

Table 10-29: Query Route Plan

Message Name (Name:Type)	Required Fields (Name:Type)
MA_RoutePlanMT: MA_RoutePlanMT	None
ProductLocationMT: ProductLocationMT	None
ProductMetadataMT: ProductMetadataMT	SHA_2_Hash: SHA_2_256_HashType
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.5 Receive Barometric Pressure

Barometric pressure reference is used for determining the vehicle's altitude above mean sea level. This value is the QNH pressure and this use case allows the value to be changed. MA provides the VI the QNH setting using the *MA_SystemManagementRequestMT* command. The *RequestType* will be *VehicleSettings*, with *VehicleSettings.QNH_Setting* set to the barometric pressure in kilopascals (kPa). The VI accepts or rejects the command using the *MA_SystemManagementRequestStatusMT* message.

The *MA_SystemManagementRequestStatusMT* will have the *RequestProcessingState* field set to "COMPLETED". If the *MA_SystemManagementRequestMT* was not able to be applied, then the *RequestProcessingState* will be set to "FAILED" and the *RequestProcessingStateReason* set to the reason the *MA_SystemManagementRequestMT* couldn't be applied.

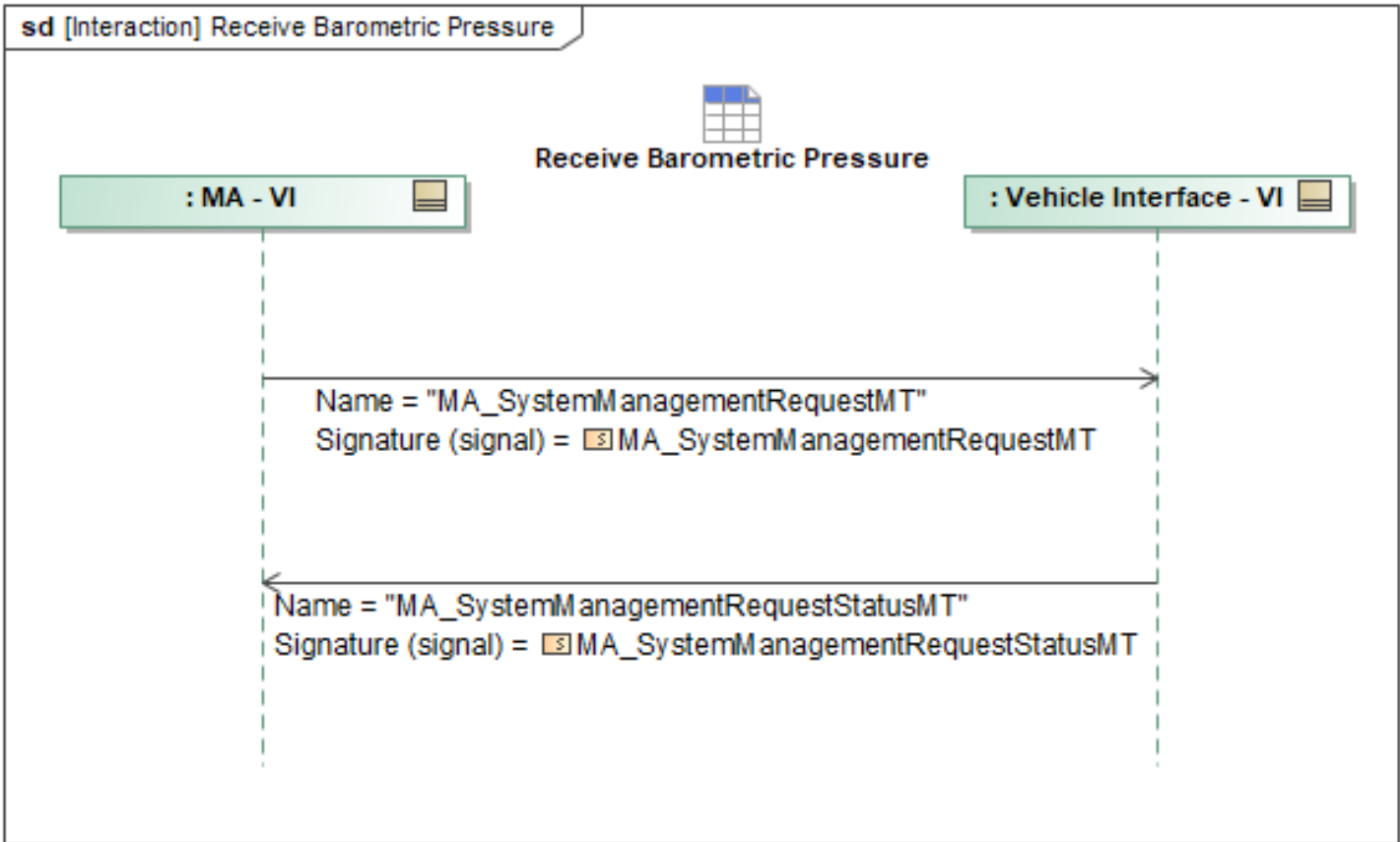


Figure 10-32: Receive Barometric Pressure

Table 10-30: Receive Barometric Pressure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemManagementRequestMT: MA_SystemManagementRequestMT	None
MA_SystemManagementRequestStatusMT: MA_SystemManagementRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.6 Receive Execution Status

VI will periodically provide updates to MA on the status of actively executing or queued actions, tasks, plans, routes, and missions.

MA_ActionStatusMT and TaskStatusMT convey the status of the ongoing Action or Task. Both ActionStatusMT and TaskStatusMT contain the required ExecutionState field denoting the executing state.

If VI is executing a route, it will send RoutePlanExecutionStatusMT and RouteActivityPlanExecutionStatusMT containing the required field PlanExecutionStatus.PlanExecutionState and PlanExecutionStatus.RouteActivityPlanExecutionStateType, respectively, denoting the executing

state.

If VI is executing a mission, it will send *MissionPlanExecutionStatusMT* and *MissionPlanActivationStatusMT* containing the required field *PlanExecutionStatus.MissionPlanExecutionStateType* and *PlanActivationState.PlanActivationStateEnum*, respectively, denoting the executing state.

If VI is executing an activity plan or task plan, it will send *ActivityPlanExecutionStatusMT* and *TaskPlanExecutionStatusMT* containing the required field *PlanExecutionStatus.ActivityPlanExecutionStateType* and *PlanExecutionStatus.TaskPlanExecutionStateType*, respectively, denoting the executing state.

If VI is executing a response, it will send *ResponsePlanExecutionStatusMT* containing the required field *PlanExecutionStatus.ResponsePlanExecutionStateType* denoting the executing state of the response plan.

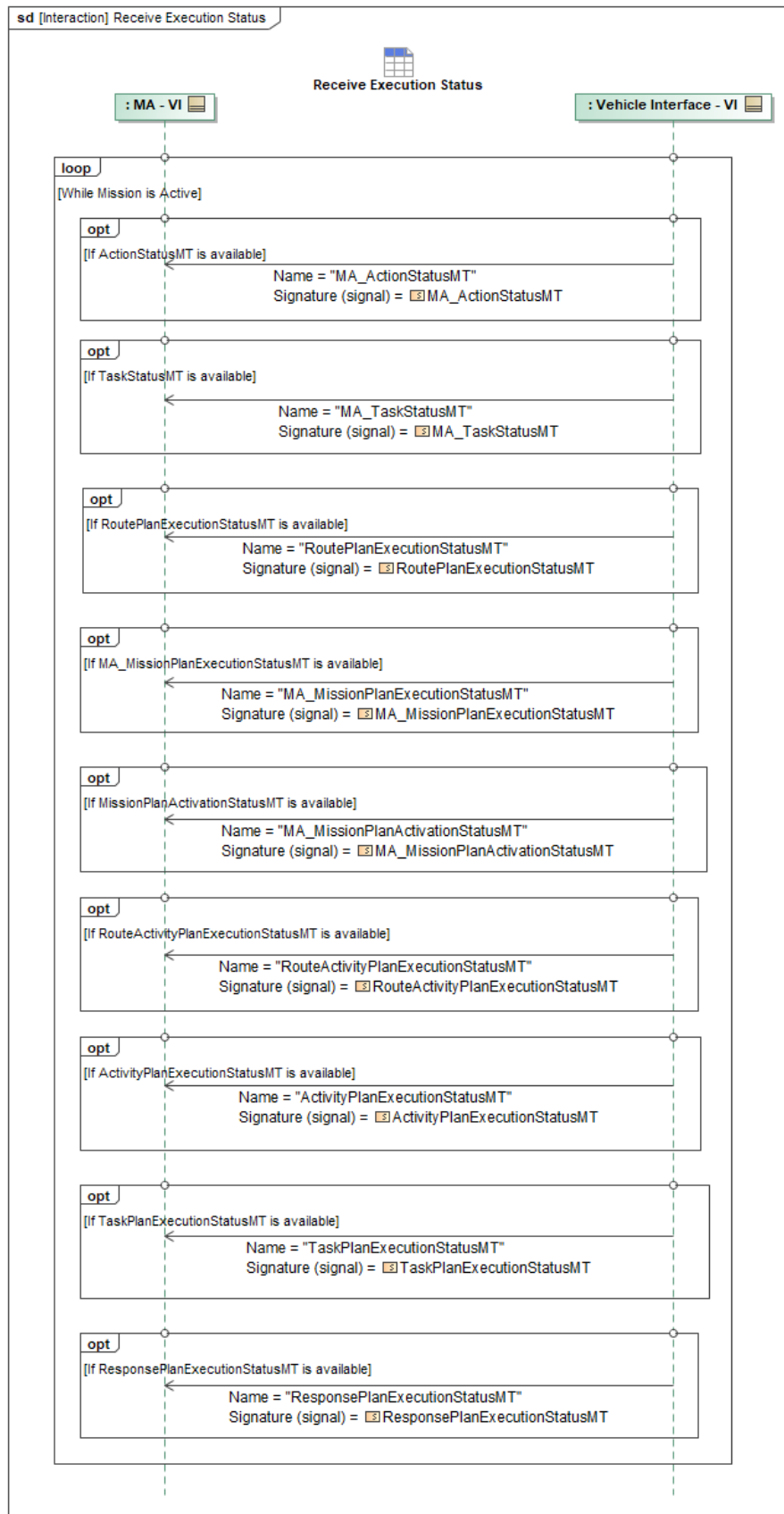


Figure 10-33: Receive Execution Status

Table 10-31: Receive Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityPlanExecutionStatusMT: ActivityPlanExecutionStatusMT	None
MA_ActionStatusMT: MA_ActionStatusMT	None
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationStatusMT	None
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	None
MA_TaskStatusMT: MA_TaskStatusMT	None
ResponsePlanExecutionStatusMT: ResponsePlanExecutionStatusMT	None
RouteActivityPlanExecutionStatusMT: RouteActivityPlanExecutionStatusMT	None
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	None
TaskPlanExecutionStatusMT: TaskPlanExecutionStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.7 Receive Vehicle Performance Values

The MA subscribes to *MA_FlightCapability* and *MA_FlightCapabilityStatus* as published across the Vehicle Interface (VI) to discover the currently available control modes. Also included in the *MA_FlightCapability* message are command limits that are applicable to the current command mode. These limits can be used by MA to inform trajectory shaping and commands to ensure MA commands to the vehicle are executable. As the vehicle should be continuously publishing *MA_FlightCapability*, these limits can be updated on a regular basis such that MA can monitor any changes to the performance profile as a function of vehicle configuration or flight condition. These limits do not have to specifically be limits associated with the vehicle AOL (aircraft operational limits), but should be provided as guard rails for MA commands, and based on the control mode limitations, could correlate to AOLs. The profile limits will be found under *MA_FlightCapability* as *FlightCapabilityPerformance*. The *FlightCapabilityPerformanceProfile* will be filled by parameters relevant to the selected control mode (currently HSA/CSA, Waypoint Following, or Curve Following) using the fields *HSA_CSA_PerformanceProfile*, *WaypointFollowingPerformanceProfile*, or *CurveFollowingPerformanceProfile*. The data package's elements may be optionally filled according to the needs of each control mode, and will be defined with *MA_FlightControlModesPerformanceProfile*. Maximum and minimum airspeed limits are referenced to *TRUE_AIRSPEED* in *SpeedReferenceEnum* and will be defined as a set from 0 to n to allow for a progression of velocities with an altitude pair to aid in the definition of a flight envelope. *MinAltitude* and *MaxAltitude* will also be defined to aid in checking trajectory validity. These values will be updated to reflect the current altitude limits at the current point in a vehicle's flight envelope. The *MinAccelerationLimits* and the *MaxAccelerationLimits* are defined using the same data structure. The acceleration limits are indexed from 0 to n to create a flight envelope boundary. The limits in all three

axes can be reported in both inertial and body axes. The body axis limits are scheduled as a function of *MachValue* in order to define a boundary that resembles a load's flight envelope. The *MaxOrientationLimits* also known as the attitude of the vehicle can also be limited in all three axes and reported at the current position in the flight envelope. The *MaxOrientationRateLimits* are the reported orientation rate limits paired with airspeed to define a boundary to the flight envelope. The rate limits are defined by roll rate, pitch rate, and yaw rate in order to account for both symmetric and asymmetric aircraft loading. There are also limiting values that are not part of the basic aircraft operational limits but are defined by the control system. These limits are reported at the current part of the flight envelope. The limits include maximum turn rate, maximum climb rate, and maximum descent rate.

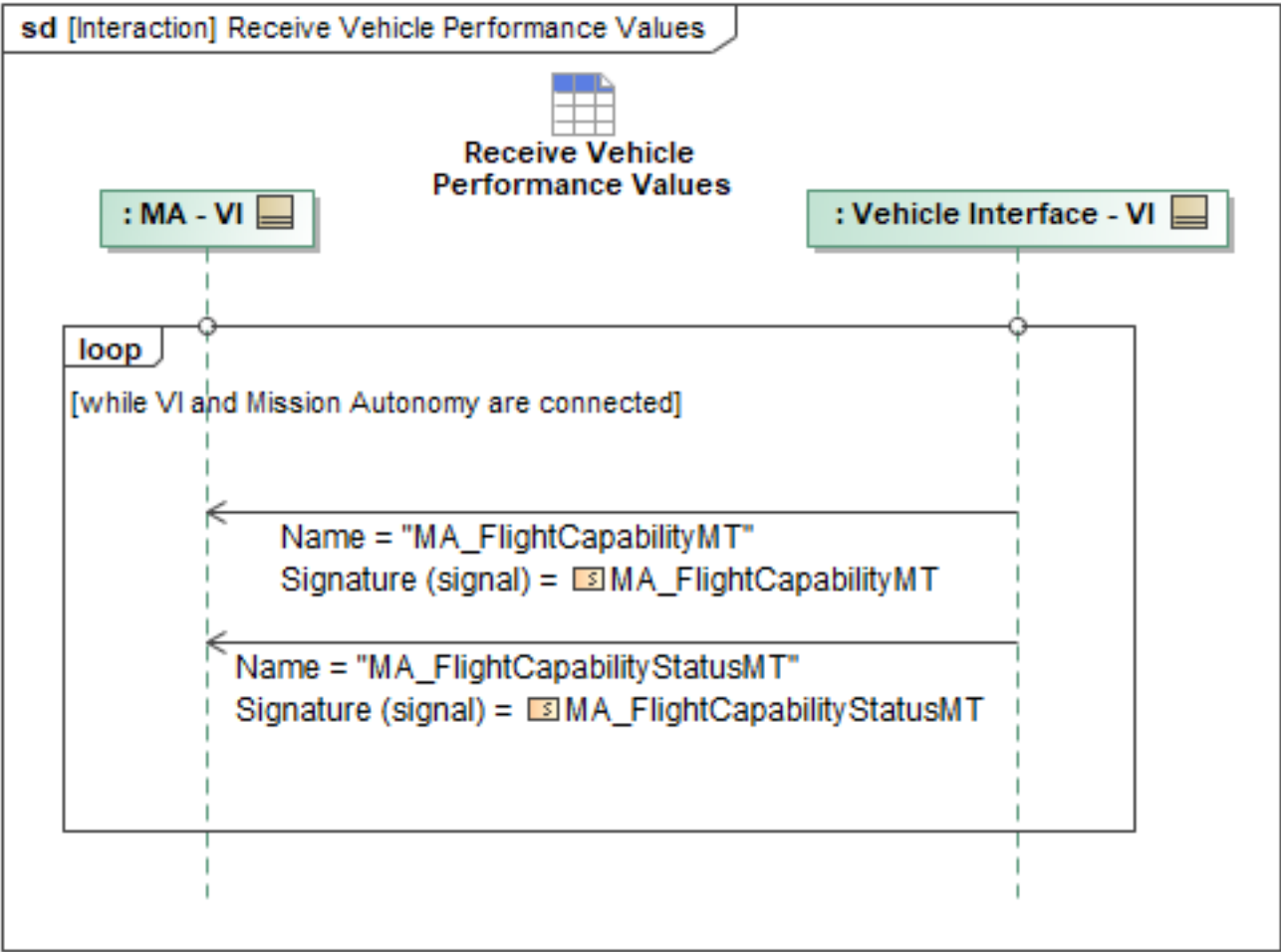


Figure 10-34: Receive Vehicle Performance Values

Table 10-32: Receive Vehicle Performance Values

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

10.2.6.8 Receive Vehicle State Data

MA treats the VI as a time, space, and position information (TSPI) truth source. A selection of parameters is provided in this ICD to ensure MA receives appropriate state information from the VI to operate in a safe and efficient manner. Within the *Kinematics* field in the *MA_PositionReportDetailedMT* message, *Position*, *Velocity*, *AirData*, *Acceleration*, *Orientation*, *WanderAngle*, *MagneticHeading*, *OrientationRate*, and *OrientationAcceleration* must be provided. The *AirData* field defines *IndicatedBaroAltitude*, *Kollsman*, *TrueAirspeed*, *CalibratedAirspeed*, and *Mach*. These messages should be published from the vehicle to MA at the highest supportable rates. In addition to providing the *MA_PositionReportDetailedMT* message, "Winds" are conveyed to MA via the *WeatherObservationMT* message with the *WindDataType* field. Within the *WindDataType* field, the *Source* enumeration will be set to *Other* to indicate the measurements are from the vehicle platform and only contain wind data. *ObservationPoint* will also be provided to indicate time and location of the wind data measurements.

FlightActivity has been extended to include Vehicle Command States. These are the commands FA is using to fly the aircraft, and can be used by MA to better estimate the vehicle's energy state. Finally, the base schema messages of *ComponentStatusMT* and *MA_NavigationReportMT* are used to report component status information, such as engine data, and the overall fuel state of the vehicle, respectively.

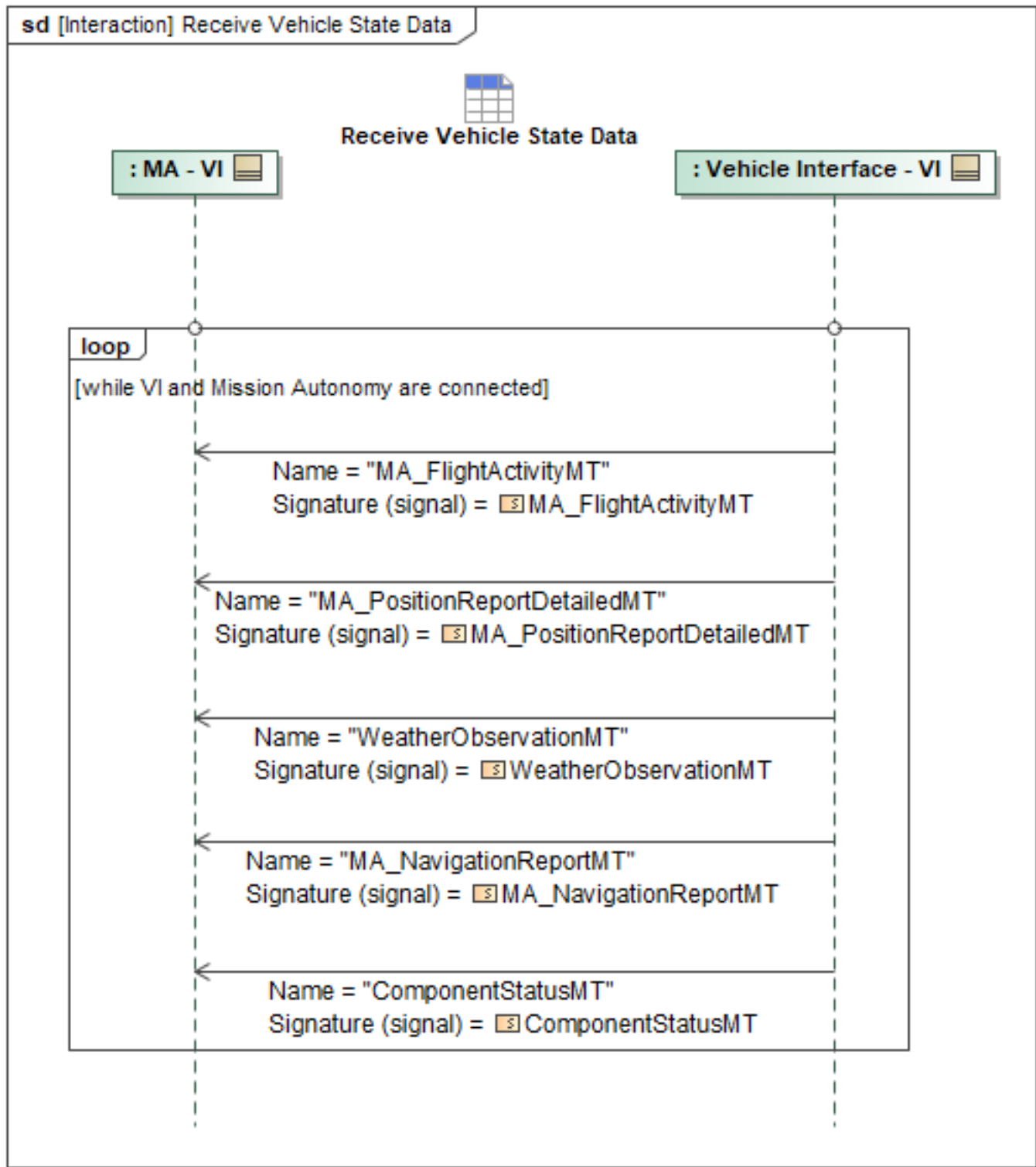


Figure 10-35: Receive Vehicle State Data

Table 10-33: Receive Vehicle State Data

Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_NavigationReportMT: MA_NavigationReportMT	None
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	Orientation: OrientationType MagneticHeading: AngleType WanderAngle: AngleType OrientationRate: OrientationRateType OrientationAcceleration: OrientationAccelerationType Acceleration: Acceleration3D_Type AirData: MA_AirDataType
WeatherObservationMT: WeatherObservationMT	WindChoice: WindDataChoiceType WindData: WindDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.9 Request Terrain Data

Mission Use Case: MA Terrain Data

MA must have a representation of terrain data to accomplish the validation of flight commands and routes prior to sending to FA. This increases the likelihood that MA will produce flight commands that will be accepted by FA's safety critical validator. Digital Terrain Elevation Data (DTED) is a standard of digital datasets which consists of a matrix of terrain elevation values. DTED is a standard National Geospatial-Intelligence Agency (NGA) product that provides medium resolution, quantitative data in a digital format for military system applications that require terrain elevation. Although DTED is the most widely used there could be other elevation sources like the Global Terrain Elevation MAP (GDEM). DTED is specifically referenced here for contextual reasons only, not as a requirement for A-GRA compliance.

The DTED format for level 0, 1 and 2 is described in U.S. Military Specification Digital Terrain Elevation Data (DTED) MIL-PRF-89020B, and amongst other parameters describes the resolution for each level:

- Level 0 has a post spacing of approximately 900 meters.
- Level 1 has a post spacing of approximately 90 meters.
- Level 2 has a post spacing of approximately 30 meters.

For FA safety critical systems such as a Ground Collision Avoidance System (GCAS), level 2 or higher DTED is required for a higher fidelity representation of the terrain. Because FA will have higher fidelity terrain representation's, MA can query the higher fidelity terrain data for specific areas to perform specific missions that are tactically relevant. To do this, MA will send the *ElevationRequestMT* to FA that will specify multiple *RequestPoint* fields building a specific zone of elevation data. If FA has the specific *RequestPoint(s)* requested by MA, FA will respond with *ElevationRequestStatusMT*, specifying the *ElevationReturned* field, which will include *RequestPoint*

fields characterized by level 2 or higher DTED or similar elevation data. If FA does not have the specific *RequestPoint(s)* specified by MA, FA will respond with a *RequestProcessingState* of *FAILED*, *CANCELED*, or *REJECTED* and MA will do an internal terrain request for lower fidelity, non-safety critical terrain data. In this case, FA is still the safety critical system responsible for ground collision avoidance and geofence violation avoidance, which may include a defined floor geofence.

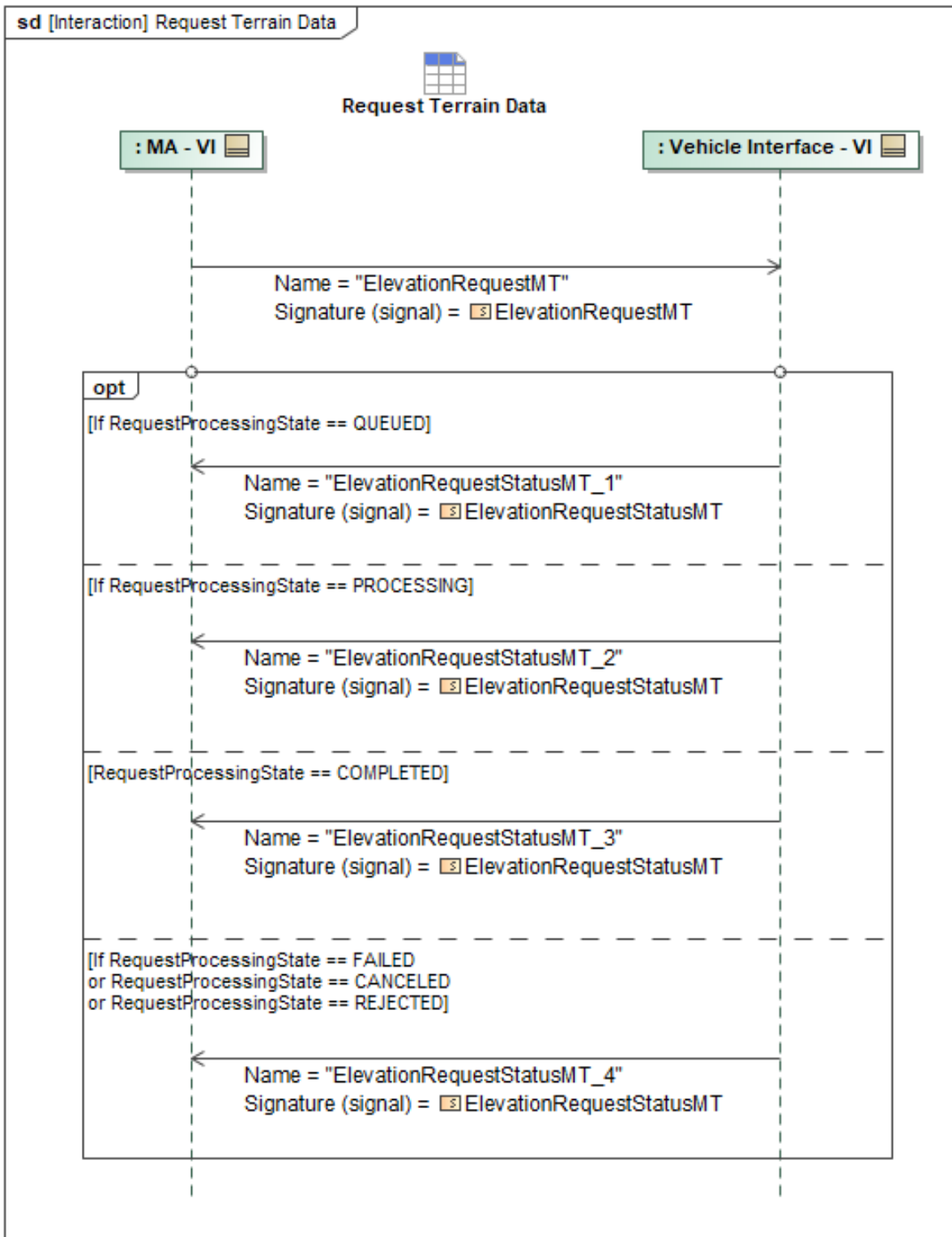


Figure 10-36: Request Terrain Data

Table 10-34: Request Terrain Data

Message Name (Name:Type)	Required Fields (Name:Type)
ElevationRequestMT: ElevationRequestMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
ElevationRequestStatusMT_1: ElevationRequestStatusMT	QUEUED:
ElevationRequestStatusMT_2: ElevationRequestStatusMT	PROCESSING:
ElevationRequestStatusMT_3: ElevationRequestStatusMT	COMPLETED:
ElevationRequestStatusMT_4: ElevationRequestStatusMT	FAILED: REJECTED: CANCELED:

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.10 Vehicle Status Reporting

SubsystemStatusMT shall be used to report status of specific FA components, as well as doubles as a heartbeat message between MA and FA. MA and FA exchange *SubsystemStatusMT_1* and *SubsystemStatusMT_2* asynchronously, not in response to one another. All component aggregations are reported to MA. The *SubsystemStatusMT* schema can identify individual components of FA generically using the *SubsystemComponent.SpecificStatus* field.

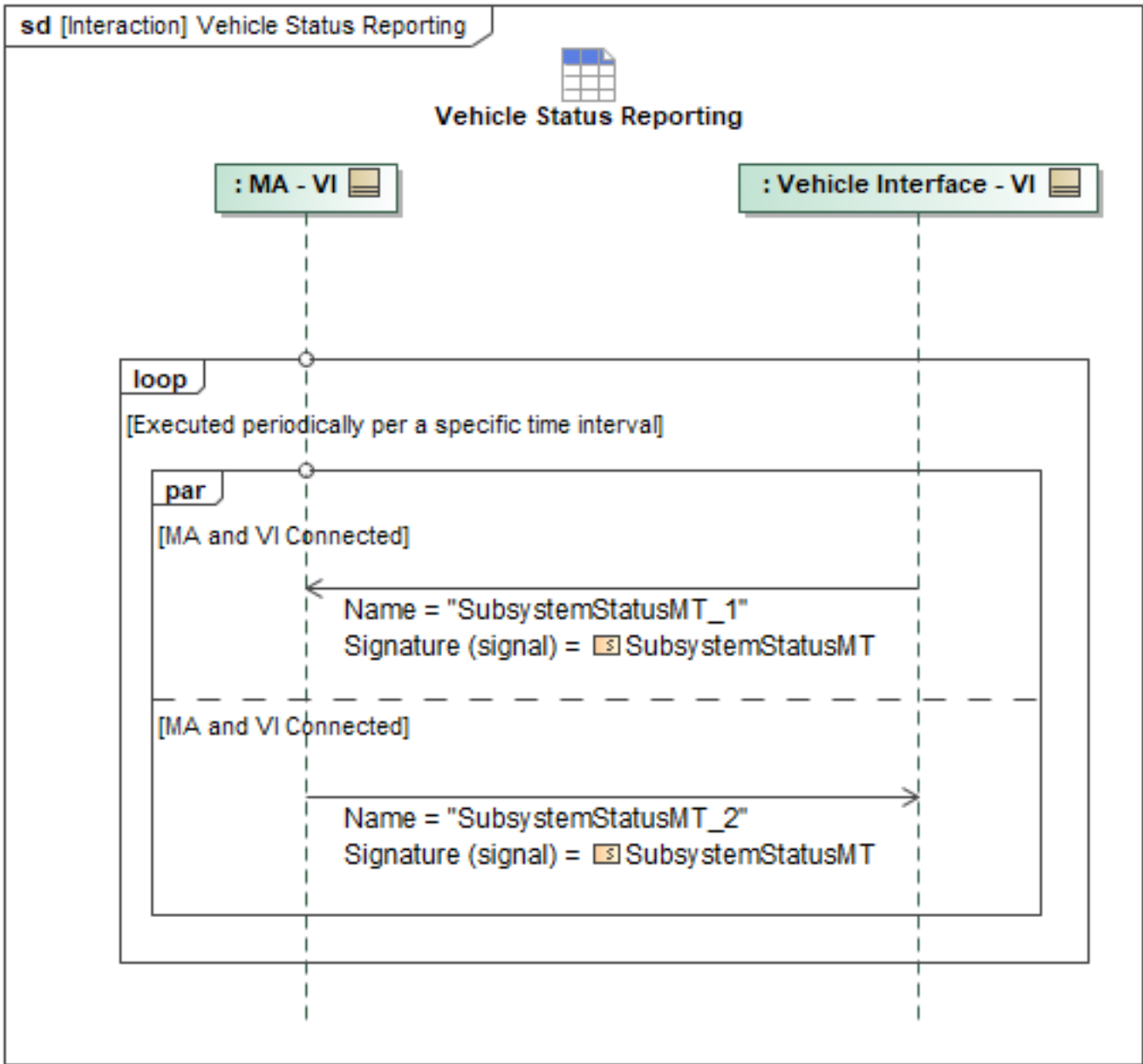


Figure 10-37: Vehicle Status Reporting

Table 10-35: Vehicle Status Reporting

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusMT_1: SubsystemStatusMT	None
SubsystemStatusMT_2: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.6.11 VI Responds to Query for Flight Capabilities

MA_FlightCapabilityMT and *MA_FlightCapabilityStatusMT* are delivered from VI to MA in the **Receive** Page 906

Vehicle Performance Values reference. This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *QueryDataRequestStatusMT* messages with *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and "PROCESSING".

When complete, the *QueryDataRequestStatusMT* message's *RequestProcessingStateEnum* is set to "COMPLETED" and contains the requested data.

When the original *QueryDataRequestMT*'s optional field *ResultsInNative* is populated with the matching message type, the underlying data management services instead publishes the requested data in its original or raw format as a separate message and indicates *QueryDataRequestStatusMT*.

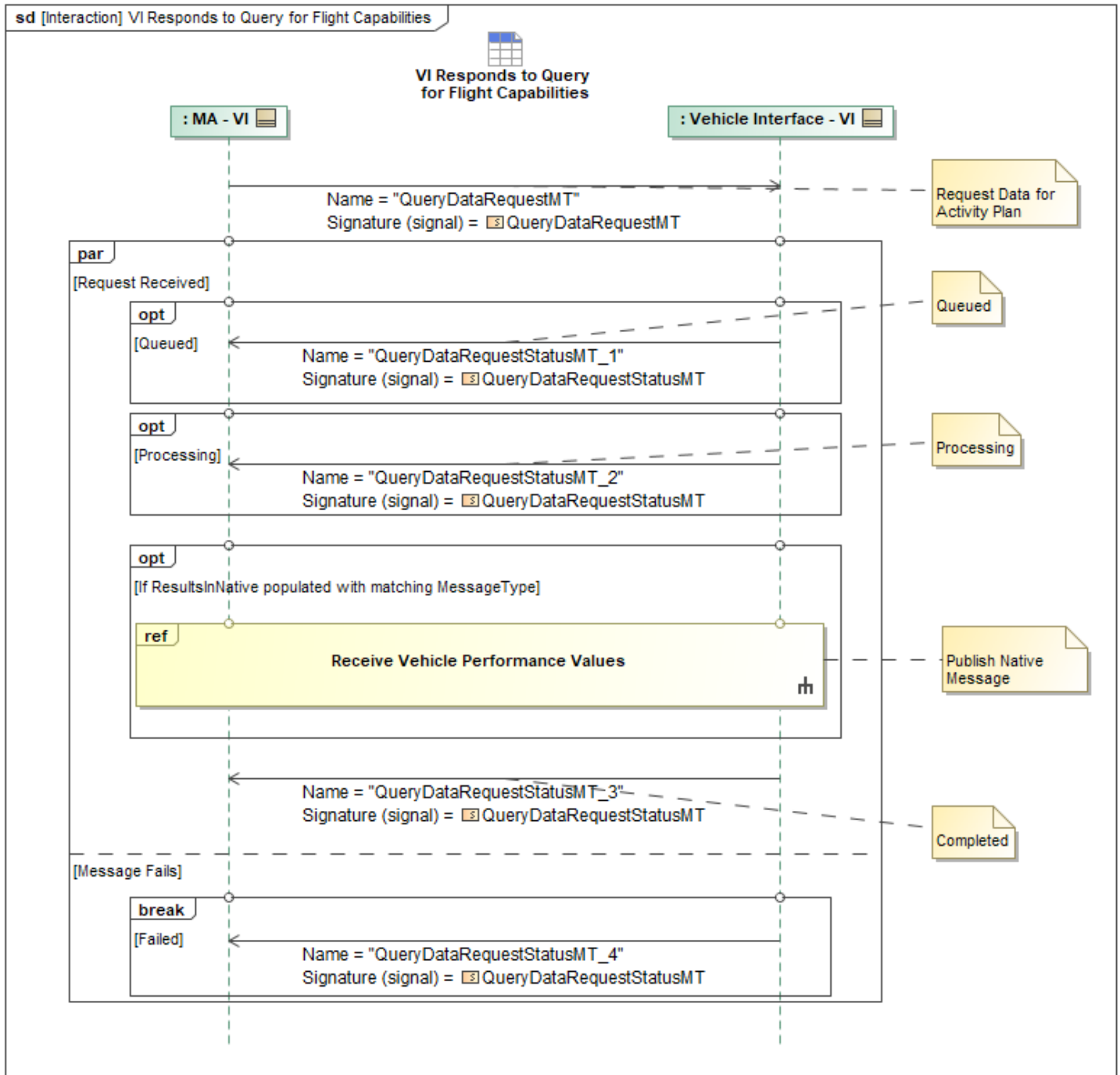


Figure 10-38: VI Responds to Query for Flight Capabilities

Table 10-36: VI Responds to Query for Flight Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

10.2.7 Weapon Employment Package

10.2.7.1 Validate Release Envelope

Prior to this interaction, MA has sent MS a *MA_Task* of type *StrikeTask* for the intended target and calculated its preferred maneuver for striking a target. MA has also validated the target's ID and that the strike complies with any enabled engagement criteria. To begin, MA sends its maneuver through a *MA_FlightCommandMT* to VI. This command is associated with the existing Strike Task through its identifier (*TaskID*), set in the *MA_FlightCommandMT* message's *Capability.Traceability.Requirement* field. Upon receiving this message, VI will determine via vehicle and store position if the criteria to release the weapons store is met. If the release envelope criteria are met, VI will return the *MA_FlightCommandStatusMT* to MA with status *ACCEPTED*, indicating acceptance of the *MA_FlightCommandMT*, and begin the maneuver. If the criteria are not met for release, VI will send a *MA_FlightCommandStatusMT* with status of *REJECTED* to MA. As maneuvering continues, MA will use position information in *MA_PositionReportDetailedMT* to determine if the ownship has maneuvered to the expected position for release.

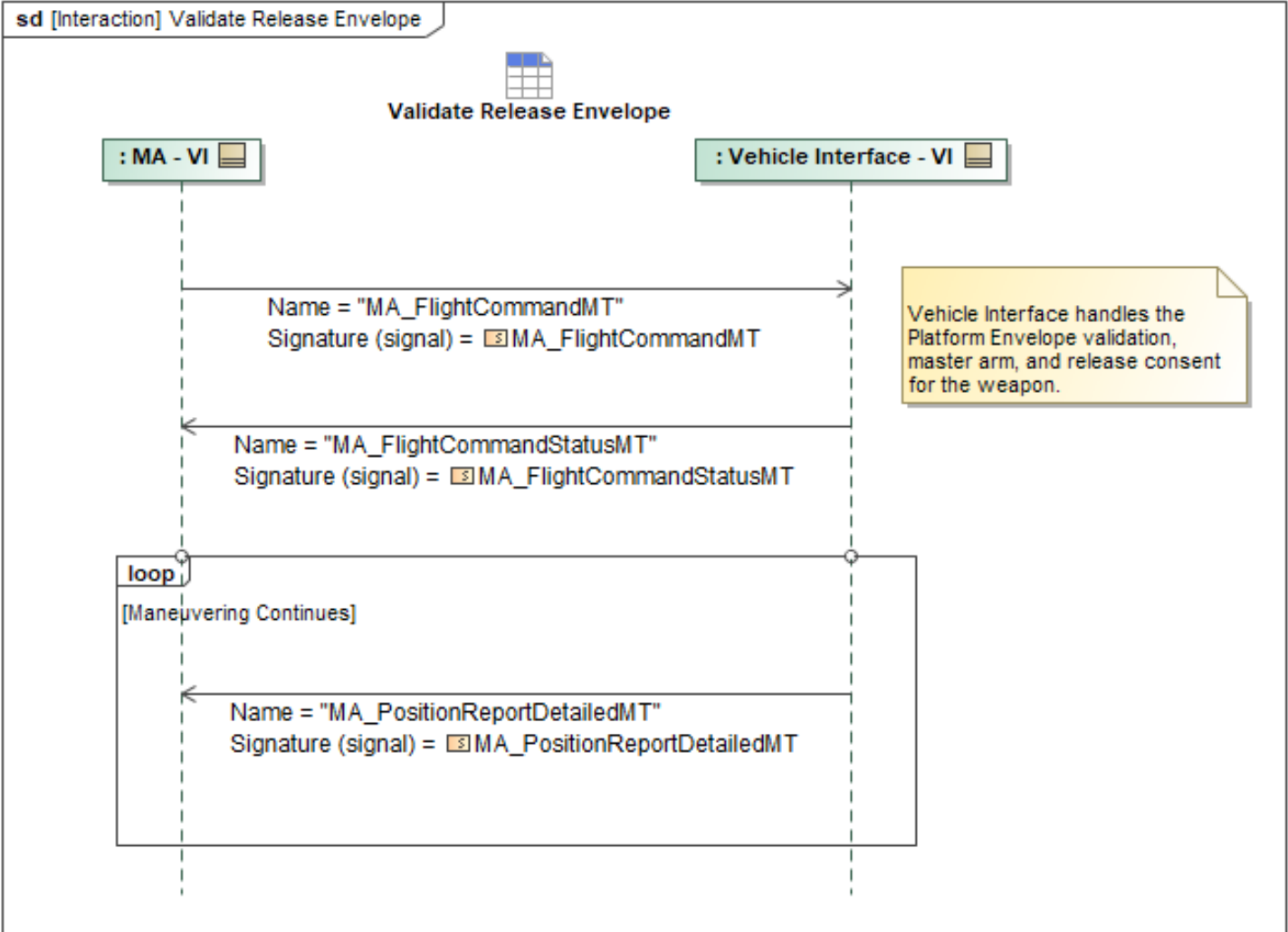


Figure 10-39: Validate Release Envelope

Table 10-37: Validate Release Envelope

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCommandMT: MA_FlightCommandMT	Traceability: TraceabilityType Requirement: RequirementInstanceID_ChoiceType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	Orientation: OrientationType MagneticHeading: AngleType WanderAngle: AngleType OrientationRate: OrientationRateType OrientationAcceleration: OrientationAccelerationType Acceleration: Acceleration3D_Type AirData: MA_AirDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy

Page 910

Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

This diagram captures all the interactions expected between VI and MA. Each interaction contains a sequence diagram describing the interaction.

10.3 Interface Compliance

The Interface Compliance table contains the compliance requirements for the L1 C2 interface.

Table 10-38: Vehicle Interface Compliance Requirements

ID	Requirement
MA-L1-013	A Level 1 Vehicle Interface shall support the Vehicle Interface Minimum Message Set defined in the VI MMS table in the VI feature profile (A-GRA location: 1 Problem Domain > 3 Exchange Items > Minimum Message Set (MMS)) for all messages in Core Mission Use Case sequences and all non-Core Mission Use Case sequences required for platform specific implementations.
MA-L1-014	A Level 1 VI interface shall implement all required messaging sequences in the VI feature profile (1 Problem Domain > 2 White Box > 3 Functional Analysis > L1 Functional Analysis > L1 Interface Behaviors > VI Interface Behavior), marked Core and all non-Core Mission Use Case sequences that apply to the platform specific implementation.
MA-L1-020	Upon receipt of a Malformed Message (a message missing expected fields (i.e., required by the behavior but not the UCI schema) or with invalid context-dependent information within the scope of that individual message) from the VI interface, MA shall respond to the error by notifying VI of the malformed message and requesting the corrected message according to the Malformed Message Response (VI) sequence.

11 COMPONENT INTERFACE COMPLIANCE

Mission Autonomy's L2 boundary is the component-level internal interfaces within Mission Autonomy. Mission Autonomy's L2 interfaces are not yet defined but compliance is expected to be based on interactions within MA analogous to L1 interactions across MA's six logical external interfaces. The A-GRA will eventually include L2 interface definitions and compliance requirements.

Mission Autonomy L2 components will communicate over the OMS ASB and utilize an OMS CAL. The Component Interface Compliance table outlines the requirements for the Mission Autonomy message bus and CAL selection.

Table 11-1: Component Interface Compliance

ID	Requirement
MA-L2-001	The CAL implementation shall be developed per RQMT STD-000190 in the OMS standard.
MA-L2-002	The CAL selected for MA shall support schema extensions capabilities.
MA-L2-003	The MA message bus shall conform to all OMS ASB requirements per RQMT STD-000144, RQMT STD-000146, and RQMT STD-009706 in the OMS standard.